IoT-rendszerek és szoftverkomponensek – az eszköztől a felhőig, programozóknak

Tartalomjegyzék

I. RÉSZ – Indulás: az IoT-rendszer anatómiája és a labor **5**

1 Olvasói útmutató **6**
1.1 A könyv használata: előismeretek, mintarendszer, olvasási utak 6

2 Az IoT-rendszer anatómiája **20**
2.1 Mi az IoT? Rétegek, szereplők, adatút 20
2.2 Nem funkcionális követelmények: energia, költség, élettartam, megbízhatóság, biztonság 26
2.3 A TerraSense mintarendszer architektúrája 36

3 Laborkörnyezet **44**
3.1 A labor felállítása: Linux host, Docker, Zephyr, Mosquitto, Java 44
3.2 Opcionális valós hardver: fejlesztőkártyák és szenzorok 55

II. RÉSZ – Az eszköz: mikrovezérlő, RTOS, szenzorok, energia **65**

4 Mikrovezérlők és perifériák **66**
4.1 Mikrovezérlő vs. Linuxos processzor: memória, OS nélküli világ 66
4.2 Perifériák: GPIO, UART, I2C, SPI, ADC, időzítők 76
4.3 Megszakítások, időzítés és konkurens hozzáférés 83

5 RTOS és Zephyr **92**
5.1 Miért RTOS? A Zephyr felépítése és az első alkalmazás 92
5.2 Szálak és ütemezés 100
5.3 Szinkronizáció és üzenetátadás: mutex, szemafor, üzenetsor, work queue 108
5.4 Devicetree és Kconfig: a hardver és a konfiguráció leírása 116

6 Szenzorok és aktuátorok **127**
6.1 Szenzorolvasás a Zephyr sensor API-val 127
6.2 Mintavétel, kalibráció és szűrés 142
6.3 Aktuátorok: biztonságos vezérlés és visszajelzés 153

7 Energiagazdálkodás **166**
7.1 Energiamodell és az elemélettartam kiszámítása 166
7.2 Alvómódok és energiatakarékos szoftvertervezés 175

8 Robusztus eszközszoftver **185**
8.1 Eszközszoftver-architektúra: rétegek és állapotgépek 185
8.2 Hibatűrés: watchdog, újraindulás, hibanaplózás 203
8.3 Eszközszoftver tesztelése a hoston 212

III. RÉSZ – Kommunikáció: rádiók, hálózatok, protokollok, adatformátumok **222**

9 Rádiós technológiák és hálózati topológiák **223**
9.1 Rádiós technológiák összevetése: hatótáv, energia, sávszélesség, költség 223

9.2 Topológiák: csillag, mesh, gateway-alapú hálózatok . . . 232
9.3 Bluetooth Low Energy: hirdetés, GATT, kapcsolatok . . . 242
9.4 LoRaWAN: nagy hatótáv kis energiával . . . 254
9.5 IP az eszközön: IPv6, 6LoWPAN, Thread és a Zephyr hálózati stack . . . 263

10 Üzenetkezelési protokollok 271
10.1 MQTT: publish/subscribe, broker, topicok . . . 271
10.2 MQTT QoS, munkamenetek, retain, Last Will és MQTT 5 . . . 280
10.3 MQTT a gyakorlatban: Zephyr-kliens és Mosquitto . . . 287
10.4 CoAP: REST UDP fölött, Observe és blokkátvitel . . . 302
10.5 LwM2M: eszközmenedzsment objektummodellel . . . 308
10.6 Ipari protokollok: Modbus és OPC UA . . . 316

11 Adatformátumok, sémák és idő 324
11.1 Adatformátumok: JSON, CBOR, Protocol Buffers, SenML . . . 324
11.2 Sémák és verziózás: kompatibilis változtatások . . . 335
11.3 Idő az IoT-ban: időbélyeg, időszinkron, sorrend . . . 343

IV. RÉSZ – Edge és gateway 352

12 A gateway 353
12.1 A gateway szerepe: híd, puffer, biztonsági határ . . . 353
12.2 A ts-gw szolgáltatás C++-ban: protokollfordítás MQTT-re . . . 360
12.3 Store-and-forward: offline működés és visszatöltés . . . 388
12.4 A gateway üzemeltetése: systemd, konténerek, naplózás . . . 407

13 Edge-feldolgozás 431
13.1 Helyi feldolgozás: szűrés, aggregálás, eseménydetektálás . . . 431
13.2 Gépi tanulás az eszközön és az edge-en: TinyML áttekintés . . . 451

V. RÉSZ – Backend és enterprise integráció 466

14 Backend-architektúra és a JEE-híd 467
14.1 Az IoT-backend felépítése: ingest, tárolás, API, alkalmazások . . . 467
14.2 JEE-ből Spring Bootba: fogalmi híd C/C++ fejlesztőknek . . . 473
14.3 A ts-backend váza: REST API és egészségellenőrzés . . . 483

15 Ingest és adattárolás 496
15.1 Ingest: MQTT-ből a backendbe, idempotensen . . . 496
15.2 Idősoros adatbázisok és adatmodellezés . . . 517
15.3 Megőrzés, ritkítás és tárolási költség . . . 533

16 Streamfeldolgozás és integrációs minták 546
16.1 Eseményvezérelt feldolgozás: Kafka és a kézbesítési szemantika . . . 546
16.2 Enterprise integrációs minták IoT-ban . . . 560

17 Parancsok, digitális iker és megjelenítés 589
17.1 Parancsok és konfiguráció lefelé: kézbesítés, nyugtázás, időkorlát . . . 589
17.2 Eszközárnyék és digitális iker: kívánt és jelentett állapot . . . 613
17.3 Dashboardok Grafanával . . . 633
17.4 Riasztás: szabályok, értesítés, riasztási fáradtság . . . 645

VI. RÉSZ – Biztonság, eszközmenedzsment és üzemeltetés 659

18 IoT-biztonság alapjai **660**
18.1 Fenyegetésmodellezés: támadási felület és STRIDE 660
18.2 Tipikus IoT-sebezhetőségek és a szabályozás 668

19 Kriptográfia, identitás és biztonságos kommunikáció **681**
19.1 Kriptográfiai alapok fejlesztőknek 681
19.2 Eszközidentitás: X.509-tanúsítványok, PKI és kulcstárolás 689
19.3 Biztonságos kapcsolat: TLS 1.3, mTLS és DTLS az MQTT/CoAP alatt 700

20 Eszközmenedzsment: provisioning, OTA, flotta **720**
20.1 Biztonságos boot és firmware-aláírás 720
20.2 Gyártás és üzembe helyezés (provisioning, onboarding) 730
20.3 OTA-frissítés: A/B képek, rollback, delta, fokozatos kiadás 739
20.4 Flottakezelés: nyilvántartás, konfiguráció, állapot 749

21 Megfigyelhetőség, tesztelés és szállítás **762**
21.1 Megfigyelhetőség végponttól végpontig: metrikák, naplók, nyomkövetés 762
21.2 Tesztelési szintek: unit, integráció, HIL, terheléses teszt 784
21.3 CI/CD a firmware-től a backendig 798

VII. RÉSZ – Platformok, záróprojekt és összefoglalás **815**

22 IoT-platformok **816**
22.1 Felhős és nyílt forrású IoT-platformok: mikor építs, mikor vásárolj 816

23 Záróprojekt és összefoglalás **827**
23.1 Záróprojekt: a teljes TerraSense rendszer összeállítása és átvétele 827
23.2 Összefoglalás: döntési térkép, ellenőrzőlista, továbbtanulás 853

I. RÉSZ – Indulás: az IoT-rendszer anatómiája és a labor

1 Olvasói útmutató

1.1 A könyv használata: előismeretek, mintarendszer, olvasási utak

1.1.1 Motiváció

Egy IoT-rendszer szoftveres vertikuma sok programozó számára ismeretlen terület: a mikrovezérlőn futó, néhány KiB memóriában gazdálkodó firmware-től a felhőben futó, skálázódó backendig minden réteget más nyelv, más futásidő és más megbízhatósági modell jellemez. Ha valaki erős C/C++ és Linux-tudással rendelkezik, de még nem épített IoT-rendszert, akkor a legnagyobb kihívás nem a nyelvi szintaxis, hanem a rétegek közötti szemléletváltás: hogyan lesz egy RTOS-szálból kiolvasott szenzorértékből megbízhatóan tárolt idősoros adat, miközben az eszköz akár évekig elemről fut, és a hálózat időnként elérhetetlen.

Ez az alfejezet rögzíti, milyen előismeretekre épít a könyv, bemutatja a végigvitt mintarendszert, és ajánlott olvasási utakat ad, hogy mindenki a saját céljának megfelelő mélységben haladhasson.

1.1.2 Elvárt előismeretek: mit nem tanítunk újra?

A könyv feltételezi, hogy az olvasó programozóként dolgozik, és a következő területeken **nem igényel alapozó magyarázatot**:

- **C:** mutatók és cím szerinti átadás, struktúrák és unionok, bitműveletek és bitmezők, makrók és előfeldolgozó, `malloc`/`free` szemlélet, `void *` és típuskonverzió.
- **C++:** osztályok és öröklődés, konstruktor/destruktor és a RAII-elv, sablonok alapjai, az STL leggyakoribb elemei (`vector`, `map`, `string`, `thread`, `mutex`), kivételkezelés alapjai.
- **Linux:** shell-használat, fájlrendszer és jogosultságok, folyamatok és szignálok, TCP/UDP socketek, `Makefile` vagy CMake alapú build, naplózás és hibakeresés `strace`-szel vagy `gdb`-vel.
- **Java/JEE:** csak **fogalmi szintű** ismeret szükséges: mi a servlet, az EJB, a JPA és a JMS. Nem kell tudni Java-szintaxist vagy Jakarta EE konfigurációt, mert a backend-rész ebből a fogalmi hídból indul, és C++-os analógiákkal vezet át Spring Bootba.

Ha ezek közül valamelyik hiányzik, érdemes előbb pótolni, mert a könyv a C/C++/Linux fogalmakra rövid analógiákkal hivatkozik, és nem magyarázza el őket újra. A következő önellenőrző lista segít felmérni az erősségeket:

Terület	Erős, ha például…	Gyenge, ha…
C bitműveletek	Gond nélkül ír `reg |= (1 << 3)` kifejezést	Nem tudja egy bitmaszk hatását fejben kiértékelni
C++ RAII	`lock_guard`-ot vagy saját RAII-osztályt használ	Destruktorban nem érti a felszabadítás sorrendjét
Linux socketek	Írt már TCP/UDP klienst vagy szervert	Csak elméletből ismeri a `connect`/`bind` hívásokat

Terület	Erős, ha például…	Gyenge, ha…
JEE fogalmak	Tudja, mi a JMS-üzenet és a JPA-entitás	A Java-nyelvet és a Jakarta EE-t is most látná először

Ez a lista nem vizsga, hanem iránytű: ha valamelyik sorban bizonytalan, akkor a hozzá tartozó fejezetben érdemes lassabban haladni.

1.1.3 Ami új lesz a könyvben

A fenti alapokra építve a könyv **nulláról tanítja** a következőket:

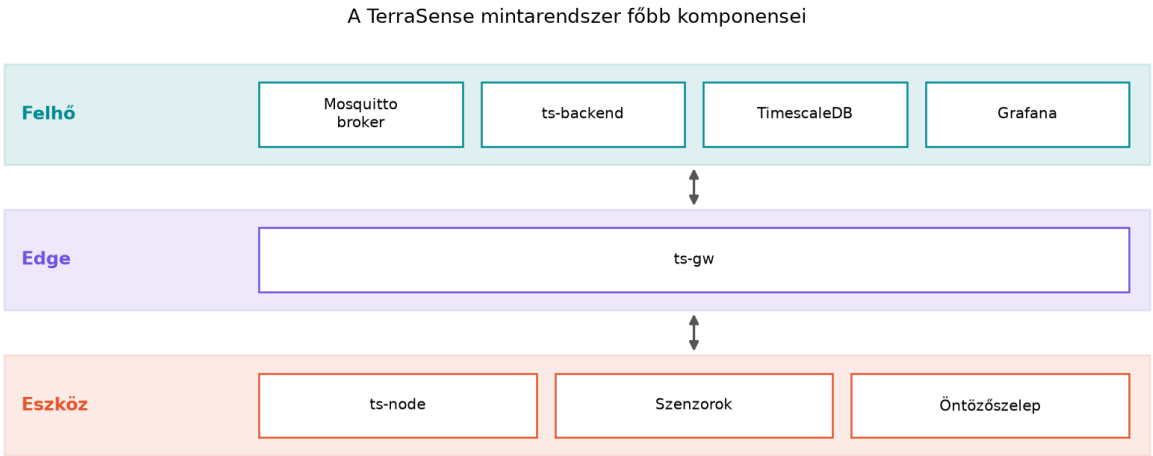
- mikrovezérlők és perifériák (GPIO, I2C, SPI, ADC, megszakítások);
- valós idejű operációs rendszerek (RTOS), elsősorban a Zephyr;
- devicetree és Kconfig alapú hardver- és konfigurációkezelés;
- rádiós technológiák (BLE, LoRaWAN, IP-alapú 802.15.4 hálózatok);
- IoT-protokollok (MQTT, CoAP, LwM2M, Modbus, OPC UA);
- adatformátumok és sémaverziózás (JSON, CBOR, Protocol Buffers, SenML);
- idősoros adatbázisok és streamfeldolgozás;
- eszközbiztonság, provisioning, OTA-frissítés és flottakezelés;
- megfigyelhetőség, tesztelési szintek és CI/CD a firmware-től a backendig.

Ezeket a témákat mindig a TerraSense mintarendszer konkrét problémájához kötjük, így nem „lexikonként”, hanem működő rendszer építése közben sajátíthatók el.

1.1.4 A TerraSense mintarendszer

Definíció. A TerraSense egy olyan, a könyvben végigvitt mintarendszer, amely egy üvegház klímáját és talaját figyeli, és a mért adatokból riasztásokat, vezérléseket és dashboardokat állít elő. Minden labor ennek egy darabját építi meg vagy bővíti.

A rendszer fő komponensei:



1.1. ábra: A TerraSense mintarendszer főbb komponensei - A könyvben végigvitt rendszer rétegei és főbb elemei.

A rétegek és a hozzájuk tartozó komponensek:

- **Eszközréteg:** a `ts-node` nevű szenzorcsomópont Zephyr RTOS alatt, C nyelven fut. Méri a hőmérsékletet (°C), a relatív páratartalmat (%RH) és a talajnedvességet (%), valamint egy öntözőszelepet vezérel. Elsődlegesen a Zephyr `native_sim` célján fut, így valós hardver nélkül is fejleszthető.
- **Edge-réteg:** a `ts-gw` egy Linux-gateway, C++17/20 és CMake használatával. Bemenete a szimulációban UDP, valós hardveren BLE GATT; kimenete MQTT. Feladata a protokollfordítás, a pufferelés és a biztonsági határ kijelölése.
- **Felhőréteg:** az Eclipse Mosquitto broker MQTT-üzenetközvetítő, a `ts-backend` Java 21 és Spring Boot alapú backend, a PostgreSQL + TimescaleDB idősoros tárolás, valamint a Grafana dashboard. A backend fogadja a telemetriát, REST API-t és eszközárnyékot biztosít, parancsokat küld lefelé, és riasztásokat generál.

Az azonosítók és üzenetek konvencióit a könyv rögzíti: a site például `gh1` (üvegház), a node például `n001`; a telemetria-üzenetben `site`, `node_id`, `boot_id`, a bootoláson belül monoton `seq` sorszám és `ts` eseményidő (UTC, epoch ms) szerepel. A topicok a `terrasense/<site>/<node>/...` sémát követik. Ezeket a konvenciókat az A TerraSense mintarendszer architektúrája alfejezet véglegesíti, és a későbbi laborok feltételezik.

1.1.5 Terminológiai jegyzék

A könyv az alábbi jelöléseket használja következetesen; a hasonló nevű fogalmak nem helyettesíthetők egymással.

Fogalom	Jelölés a könyvben	Mit jelent	Nem tévesztendő össze
helyszín	`site`	a telepítési hely, például `gh1`	a csomóponttal (`node`)
csomópont	`node_id`	az eszköz azonosítója, például `n001`	az MQTT kliens-azonosítóval
MQTT kliens-azonosító	`client_id`	az MQTT munkamenet azonosítója	a `node_id`-val
bootolási munkamenet	`boot_id`	egy bootolás egyedi azonosítója	a `seq` számlálóval
eseménysorszám	`seq`	bootoláson belüli számláló	a globális sorrenddel
parancsazonosító	`cmd.id`	a parancs korrelációs azonosítója	az aktuátorral (`actuator`)
aktuátor	`actuator`	a vezérelt eszköz, például `valve1`	a parancs azonosítójával

1.1.6 A könyv íve és a Továbbvitt eredmények elve

A könyv hét fő részből áll, amelyek a teljes szoftveres vertikumot lefedik:

1. **Indulás:** IoT-anatómia, követelmények, a labor felállítása.
2. **Az eszköz:** mikrovezérlő, perifériák, Zephyr RTOS, szenzorok, energia, robusztus firmware.

3. **Kommunikáció:** rádiók, topológiák, MQTT/CoAP/LwM2M/ipari protokollok, adatformátumok, időkezelés.
4. **Edge és gateway:** a `ts-gw` C++-ban, store-and-forward, üzemeltetés, edge-feldolgozás.
5. **Backend és enterprise integráció:** Spring Boot, ingest, idősoros tárolás, Kafka, parancsok, dashboard, riasztás.
6. **Biztonság, eszközmenedzsment és üzemeltetés:** fenyegetésmodell, kriptográfia, TLS/mTLS, provisioning, OTA, flotta, tesztelés, CI/CD.
7. **Platformok és záróprojekt:** build vs. buy döntés, a teljes rendszer összeállítása és átvétele.

Továbbvitt eredmény. Minden alfejezet laborja a `~/iot-labor/` könyvtárban, azon belül `iNNN/` alkönyvtárban dolgozik. A közös futtatókörnyezet a `~/iot-labor/stack/compose.yaml` fájlban gyűlik össze. Egy alfejezet kimenete (kód, konfiguráció, konvenció, döntés) **továbbvitt eredménynek** számít, ha egy későbbi alfejezet arra épít. Ilyen például a TerraSense komponenslistája és azonosítórendszere, amelyre minden labor hivatkozik: ezt az A TerraSense mintarendszer architektúrája alfejezet rögzíti, a labor futtatókörnyezetét pedig a A labor felállítása: Linux host, Docker, Zephyr, Mosquitto, Java alfejezet állítja elő. A továbbvitt eredményeket a szöveg mindig kiemelten, „Továbbvitt eredmény." kezdetű bekezdésben mondja ki, és a „Rövid összefoglaló" című szakasz is felsorolja, így a későbbi alfejezetek egyértelműen az előzményre hivatkozhatnak.

1.1.7 Labor-repó szerkezete

A laborok a következő szerkezetet követik:

```
~/iot-labor/
  i004/
      ...            # a labor futtatókörnyezet felállításának fájljai
  i009/
      ...            # az első Zephyr alkalmazás
  i028/
      ...            # MQTT-kliens labor
  stack/
      compose.yaml   # közös Docker Compose fájl
```

Minden labor elején egy „Munkakönyvtár:" feliratú blokk adja meg a pontos elérési utat, és a `mkdir -p ~/iot-labor/iNNN && cd ~/iot-labor/iNNN` parancsot. A korábbi labor fájljait explicit `cp`-vel vagy relatív hivatkozással kell felhasználni, így a függőségek nyomon követhetők maradnak.

1.1.8 Verziózott referenciakód és laborállapotok

A könyv laborjai nem különálló példák, hanem egyetlen, fokozatosan bővülő rendszer állapotai. Hogy ez önálló tanulásnál is követhető legyen, a laborokhoz a következő állapotkezelési szerződés tartozik:

- **Referenciakiadás.** A könyvhöz tartozó referenciakód verziókezelt tárolóban él, kiadási taggal (például `v1.0`) és a verziómátrixban rögzített eszközverziókkal. A tag nem azonos a könyv szövegének kiadásával: a szöveg a tagre és a hozzá tartozó commitra hivatkozik.
- **Kiinduló állapot.** Minden `iNNN` labor elején a „Munkakönyvtár:" blokk megadja, mely korábbi labor mely fájljait kell `cp`-vel átemelni, vagy mely relatív útvonalon érhetők el. Az átemelés nélkül a labor nem reprodukálható.
- **Kész állapot.** A labor végén a `~/iot-labor/iNNN/` könyvtár tartalmazza a labor kimeneteit; a „Rövid összefoglaló" néven nevezi a továbbvitt fájlokat, konvenciókat és döntéseket.

- **Állapotjegyzék.** A `~/iot-labor/REFERENCE.md` fájl laboronként rögzíti a kiinduló és kész állapotot, a létrehozott vagy módosított fájlokat és a következő laborba átadott elemeket. Ez a jegyzék a laborok közötti migráció ellenőrzőpontja.
- **Migráció.** Ha egy későbbi labor fájlnevet, topicot vagy sémát változtat, a változtatást a labor szövege kimondja, és a `REFERENCE.md`-ben külön bejegyzés rögzíti az érintett korábbi laborokat.

Ez a szerződés akkor teljesül, ha a könyv utasításaiból a 3.1–23.1 laborút tiszta Ubuntu-környezetben, a referenciakód tagjének megfelelő állapotból végigjárható. A jelen kézirat a szerződést rögzíti; a referenciatároló és a laboronkénti állapotjegyzék feltöltése a kiadás előtti feladat.

1.1.9 Verziómátrix és referenciakörnyezet

A laborok reprodukálhatósága érdekében a könyv az alábbi, kiadáshoz rögzített verziómátrixot használja. Ez nem azt jelenti, hogy minden lépés csak ezekkel a verziókkal fut; ha a tanuló eltér, a szövegben jelölt kompatibilitási pontokat ellenőrizni kell. A mátrix a könyv példáinak és parancsainak közös hivatkozási alapja.

Komponens	Rögzített verzió	Megjegyzés
Ubuntu	24.04.1 LTS, x86_64	a labor hostja
Linux kernel	6.8	az Ubuntu 24.04 alapkernelje
GCC / G++	13.2	a host C/C++ fordítója
CMake / Ninja	3.28.3 / 1.11.1	a gateway és a Zephyr build eszközei
Python	3.12.3	rendszer-Python; a `west` külön venvben fut
west	1.2.0	Zephyr meta-eszköz
Zephyr	3.7.0 LTS	a firmware és a `native_sim` alapja
Zephyr SDK	0.16.8	`linux-x86_64`, SHA-256 ellenőrzéssel
Docker Engine / Compose	27.3.1 / v2.29.7	konténeres szolgáltatások
Eclipse Mosquitto	2.0.18	`eclipse-mosquitto:2.0.18` image
PostgreSQL / TimescaleDB	16.4 / 2.17.2	`timescale/timescaledb:2.17.2-pg16` image
Grafana	11.2.0	dashboard
OpenJDK / Maven	21.0.4 / 3.9.9	backend
Spring Boot	3.3.4	backend keretrendszer
Eclipse Paho C / C++	1.3.13 / 1.3.2	MQTT kliensek
OpenSSL	3.0.13	Ubuntu 24.04 rendszercsomag

A mátrixban szereplő verziók a könyv példáinak referenciaállapotát adják, és ez a könyv egyetlen verzióforrása: a laborokban szereplő konkrét verziószámok ehhez igazodnak. Ahol egy fejezet ettől eltérő verziót vagy API-t használ, ott a szöveg külön kimondja az eltérést és annak következményét; a legújabb verzió vak átvétele nem része a módszernek. A verziómátrixot a kiadás előtt a referenciakörnyezetben lefuttatott füsttesztekkel kell igazolni; a mátrix önmagában nem bizonyíték a működésre. A mátrix a kiadás dátumával együtt kerül a szerkesztői mellékletbe, és a füsttesztek jegyzőkönyve ugyanott kap helyet.

1.1.9.1 Kiadási ellenőrzési politika

A kiadás nem tekinthető késznek, ha egy kötelező labor működésének feltétele nyitott. Az „ellenőrizendő” jelölés önmagában nem elfogadható lezárás: a kéziratban előforduló összes ilyen helyet a kiadás előtt egyenként felül kell vizsgálni, és négy mezővel le kell zárni:

1. **Állítás és hatókör:** melyik API-névre, szabványazonosítóra, verzióra vagy jogi kötelezettségre vonatkozik, és mely fejezetekre.
2. **Forrás és reprodukció:** a konkrét verzióhoz kötött dokumentum, illetve a reprodukáló parancs vagy teszt.
3. **Felelős és határidő:** aki a pontot lezárja, és a kiadás előtti dátum.
4. **Eredmény:** igazolt, módosított vagy elvetett; a bizonyíték helye.

Ami a kiadásig nem zárható le, az pontos hatókörrel és ismert bizonytalanságként nevezhető meg, de nem lehet a kötelező labor működésének feltétele. A szabványazonosítókat és a jogi kötelezettségeket a hivatkozott verzióval együtt kell ellenőrizni; az „aktuális” vagy „legújabb” hivatkozás önmagában nem elfogadható. A kiadás előtti ellenőrzőlistát a kézirat szerkesztői melléklete tartalmazza.

1.1.10 Olvasási utak

A könyv többféleképpen olvasható, attól függően, hogy az olvasó melyik rétegre koncentrál.

Olvasási út	Kinek ajánlott	Fő fejezetek	Elhagyható	Kötelező labor és előfeltétel
Teljes út	Mindenki, aki a teljes vertikumot akarja érteni	I–VII. rész, sorrendben	—	A labor felállítása: Linux host, Docker, Zephyr, Mosquitto, Java, majd a laborok sorrendben; a korábbi labor kész állapota
Eszközoldali út	Beágyazott C/C++ fejlesztő, aki a firmware-re és az edge-re koncentrál	I., II., III. (MQTT-ig), IV. rész; a VI. részből a biztonsági alapok	V. rész (backend), kivéve az ingest fogalmát	A labor felállítása: Linux host, Docker, Zephyr, Mosquitto, Java, Miért RTOS? A Zephyr felépítése és az első alkalmazás, MQTT a gyakorlatban: Zephyr-kliens és Mosquitto; a Zephyr-laborokhoz aktivált venv

Olvasási út	Kinek ajánlott	Fő fejezetek	Elhagyható	Kötelező labor és előfeltétel
Backend-oldali út	Java/backend fejlesztő, aki a felhőoldalt építi	I. rész; III. részből az MQTT és az adatformátumok; V. rész; VI. részből a biztonság és tesztelés	II. rész (mikrovezérlő) és IV. rész (gateway) mély technikai részei	A labor felállítása: Linux host, Docker, Zephyr, Mosquitto, Java (broker és visszajátszó), A ts-backend váza: REST API és egészségellenőrzés, Ingest: MQTT-ből a backendbe, idempotensen, Parancsok és konfiguráció lefelé: kézbesítés, nyugtázás, időkorlát; firmware és gateway laborja nélkül

Az eszközoldali úton haladóknak is meg kell érteniük legalább a MQTT-topicok és a telemetriaséma konvencióját, mert a firmware ezeket állítja elő. A backend-oldali úton haladóknak pedig ismerniük kell a `native_sim` korlátait, hogy a kapott adatok értelmezhetők legyenek.

A backend-oldali út szimulált bemenete. A backend-oldali út nem igényli a firmware megírását. A szükséges MQTT-forgalmat a `~/iot-labor/i004/telemetry_replay.sh` szimulátor állítja elő, amely a `telemetry.jsonl` fájl sorait publikálja a 4.1 laborban létrehozott brokerre. A backend ezt a valós MQTT-forgalmat dolgozza fel; a firmware és a gateway ebben az útban elhagyható. A script és a mintafájl:

```
#!/usr/bin/env bash
# ~/iot-labor/i004/telemetry_replay.sh
set -euo pipefail
BROKER_CONTAINER="${BROKER_CONTAINER:-lab-broker}"
TOPIC="${1:-terrasense/gh1/n001/telemetry}"
FILE="${2:-$HOME/iot-labor/i004/telemetry.jsonl}"
while IFS= read -r line; do
  docker exec "$BROKER_CONTAINER" mosquitto_pub -t "$TOPIC" -m "$line" -q 1
  sleep 1
done < "$FILE"
```

```
{"site":"gh1","node_id":"n001","boot_id":"b7c1e2a4","seq":1,"ts":1717000000000,"t":23.4}
{"site":"gh1","node_id":"n001","boot_id":"b7c1e2a4","seq":2,"ts":1717000010000,"t":23.6}
{"site":"gh1","node_id":"n001","boot_id":"b7c1e2a4","seq":3,"ts":1717000020000,"t":23.5}
```

```
chmod +x ~/iot-labor/i004/telemetry_replay.sh
~/iot-labor/i004/telemetry_replay.sh
```

A visszajátszás ütemezése (egy üzenet másodpercenként) illusztratív, nem mért érték. A szimulátor a brokert használja, ezért a 4.1 labor brokerének futnia kell; a backend-oldali út első lépése ennek ellenőrzése.

1.1.11 Fejezet- és munkakönyvtár-térkép

A PDF fejezetszámai és a forrás `iNNN` azonosítói eltérnek, ezért az alábbi táblázat a gyors megfeleltetést adja. A tartományon belül a PDF-alalfejezet és az `iNNN` sorszáma együtt nő; a munkakönyvtár a fejezet első alfejezetének könyvtára, a további alfejezetek a saját `iNNN/` könyvtárukban dolgoznak. Például a 8.1 alfejezet az `i018` fájl, a 8.2 az `i019`, a 8.3 az `i020`.

PDF fejezet	Cím	Alfejezetek (PDF → iNNN)	Munkakönyvtárak
1	Olvasói útmutató	1.1 → A könyv használata: előismeretek, mintarendszer, olvasási utak	`~/iot-labor/`
2	Az IoT-rendszer anatómiája	2.1–2.3 → Mi az IoT? Rétegek, szereplők, adatút–A TerraSense mintarendszer architektúrája	`~/iot-labor/i001/`–`i003/`
3	Laborkörnyezet	3.1–3.2 → A labor felállítása: Linux host, Docker, Zephyr, Mosquitto, Java–Opcionális valós hardver: fejlesztőkártyák és szenzorok	`~/iot-labor/i004/`–`i005/`
4	Mikrovezérlők és perifériák	4.1–4.3 → Mikrovezérlő vs. Linuxos processzor: memória, OS nélküli világ–Megszakítások, időzítés és konkurens hozzáférés	`~/iot-labor/i006/`–`i008/`
5	RTOS és Zephyr	5.1–5.4 → Miért RTOS? A Zephyr felépítése és az első alkalmazás–Devicetree és Kconfig: a hardver és a konfiguráció leírása	`~/iot-labor/i009/`–`i012/`

PDF fejezet	Cím	Alfejezetek (PDF → iNNN)	Munkakönyvtárak
6	Szenzorok és aktuátorok	6.1–6.3 → Szenzorolvasás a Zephyr sensor API-val–Aktuátorok: biztonságos vezérlés és visszajelzés	`~/iot-labor/i013/`–`i015/`
7	Energiagazdálkodás	7.1–7.2 → Energiamodell és az elemélettartam kiszámítása–Alvómódok és energiatakarékos szoftvertervezés	`~/iot-labor/i016/`–`i017/`
8	Robusztus eszközszoftver	8.1–8.3 → Eszközszoftver-architektúra: rétegek és állapotgépek–Eszközszoftver tesztelése a hoston	`~/iot-labor/i018/`–`i020/`
9	Rádiós technológiák és hálózati topológiák	9.1–9.5 → Rádiós technológiák összevetése: hatótáv, energia, sávszélesség, költség–IP az eszközön: IPv6, 6LoWPAN, Thread és a Zephyr hálózati stack	`~/iot-labor/i021/`–`i025/`
10	Üzenetkezelési protokollok	10.1–10.6 → MQTT: publish/subscribe, broker, topicok–Ipari protokollok: Modbus és OPC UA	`~/iot-labor/i026/`–`i031/`
11	Adatformátumok, sémák és idő	11.1–11.3 → Adatformátumok: JSON, CBOR, Protocol Buffers, SenML–Idő az IoT-ban: időbélyeg, időszinkron, sorrend	`~/iot-labor/i032/`–`i034/`
12	A gateway	12.1–12.4 → A gateway szerepe: híd, puffer, biztonsági határ–A gateway üzemeltetése: systemd, konténerek, naplózás	`~/iot-labor/i035/`–`i038/`

PDF fejezet	Cím	Alfejezetek (PDF → iNNN)	Munkakönyvtárak
13	Edge-feldolgozás	13.1–13.2 → Helyi feldolgozás: szűrés, aggregálás, esemény-detektálás–Gépi tanulás az eszközön és az edge-en: TinyML áttekintés	`~/iot-labor/i039/–i040/`
14	Backend-architektúra és a JEE-híd	14.1–14.3 → Az IoT-backend felépítése: ingest, tárolás, API, alkalmazások–A ts-backend váza: REST API és egészségellenőrzés	`~/iot-labor/i041/–i043/`
15	Ingest és adattárolás	15.1–15.3 → Ingest: MQTT-ből a backendbe, idempotensen–Megőrzés, ritkítás és tárolási költség	`~/iot-labor/i044/–i046/`
16	Streamfeldolgozás és integrációs minták	16.1–16.2 → Eseményvezérelt feldolgozás: Kafka és a kézbesítési szemantika–Enterprise integrációs minták IoT-ban	`~/iot-labor/i047/–i048/`
17	Parancsok, digitális iker és megjelenítés	17.1–17.4 → Parancsok és konfiguráció lefelé: kézbesítés, nyugtázás, időkorlát–Riasztás: szabályok, értesítés, riasztási fáradtság	`~/iot-labor/i049/–i052/`
18	IoT-biztonság alapjai	18.1–18.2 → Fenyegetésmodellezés: támadási felület és STRIDE–Tipikus IoT-sebezhetőségek és a szabályozás	`~/iot-labor/i053/–i054/`
19	Kriptográfia, identitás és biztonságos kommunikáció	19.1–19.3 → Kriptográfiai alapok fejlesztőknek–Biztonságos kapcsolat: TLS 1.3, mTLS és DTLS az MQTT/CoAP alatt	`~/iot-labor/i055/–i057/`

PDF fejezet	Cím	Alfejezetek (PDF → iNNN)	Munkakönyvtárak
20	Eszközmenedzsment: provisioning, OTA, flotta	20.1–20.4 → Biztonságos boot és firmware-aláírás–Flottakezelés: nyilvántartás, konfiguráció, állapot	`~/iot-labor/i058/`–`i061/`
21	Megfigyelhetőség, tesztelés és szállítás	21.1–21.3 → Megfigyelhetőség végponttól végpontig: metrikák, naplók, nyomkövetés–CI/CD a firmware-től a backendig	`~/iot-labor/i062/`–`i064/`
22	IoT-platformok	22.1 → Felhős és nyílt forrású IoT-platformok: mikor építs, mikor vásárolj	`~/iot-labor/i065/`
23	Záróprojekt és összefoglalás	23.1–23.2 → Záróprojekt: a teljes TerraSense rendszer összeállítása és átvétele–Összefoglalás: döntési térkép, ellenőrzőlista, továbbtanulás	`~/iot-labor/i066/`–`i067/`

A fejezeteken belüli pontos bemenetet és a továbbadott fájlokat a labor „Munkakönyvtár:” blokkja és a „Rövid összefoglaló” nevezi meg; ez a táblázat csak a fejezet- és könyvtárszintű tájékozódást szolgálja.

1.1.12 Tanulási kimenetek és ellenőrzési küszöbök

A hosszú, sok technológiát érintő útvonalhoz részenként mérhető kimeneteket és küszöböket rögzítünk. A küszöböt a tanuló a megoldások elolvasása nélkül is ellenőrizni tudja; a kötelező labor kimenetének reprodukálhatónak kell lennie.

Rész	Tanulási kimenet	Ellenőrzési küszöb
I. Indulás	Ismeri az IoT rétegmodelljét, az NFR-ek szerepét és a labor eszközeit	Önállóan fel tudja sorolni a TerraSense komponenseit, és a 4.1 labor füsttesztjei sikeresek
II. Eszköz	Zephyr-alkalmazást ír szenzorral, állapotgéppel és watchdoggal	A `native_sim` build lefut, a küszöb- és aktuátorteszt a várt állapotváltást adja, a watchdog-teszt újraindulást okoz

Rész	Tanulási kimenet	Ellenőrzési küszöb
III. Kommunikáció	MQTT-klienst kezel reconnecttel, duplikációkezeléssel, retained és Last Will viselkedéssel	Újracsatlakozás után a telemetria folytatódik, a duplikált üzenet egyszer számít, a `status` retained üzenete a feliratkozó későbbi indulásakor is megjelenik
IV. Edge és gateway	C++ gatewayet épít store-and-forward pufferrel	Hálózatkiesés alatt az adat pufferelődik, visszatöltéskor hiány és duplikáció nélkül a backendbe kerül
V. Backend és adat	Idempotens ingestet, idősoros tárolást, REST API-t és parancsutat készít	A duplikált üzenet egyszer tárolódik, a REST lekérdezés az elvárt adatot adja, a parancs nyugtázása és időkorlátja működik
VI. Biztonság és OTA	Fenyegetésmodellt készít, mTLS-t és OTA rollbacket valósít meg	A hitelesítés nélküli kapcsolat elutasításra kerül, a megszakított OTA után az előző firmware indul
VII. Platform és záróprojekt	Build vs. buy döntést hoz, a teljes rendszert átvételi teszttel igazolja	Az átvételi ellenőrzőlista minden tétele reprodukálható bizonyítékkal lezárható

1.1.13 Feladatok és megoldások használata

Minden alfejezet végén négy fokozatos feladat található: (a) fogalmi vagy döntési kérdés, (b) számolási vagy tervezési feladat, (c) implementációs feladat a TerraSense rendszerben, (d) hibakeresési vagy kiterjesztési feladat. A feladatok megoldása a „Megoldások” című részben teljes, indoklással ellátott választ ad.

Feladatkártya. A feladat címe a tényleges tevékenységet nevezze meg, ne a sablonhelyet. Minden feladatnál öt mezőt adunk meg: (1) típus a tényleges tevékenység szerint (fogalmi/döntési, számítási/tervezési, implementációs, hibakeresési/kiterjesztési), (2) előfeltétel, (3) késztermék, (4) elfogadási feltétel, (5) becsült nehézség. Ha egy feladatnak nincs egyetlen helyes megoldása, a „nyitott tervezési feladat” jelölést kapja. Az alábbi táblázat ezt a sémát mutatja be a jelen alfejezet feladatain.

Feladat	Típus	Előfeltétel	Késztermék	Elfogadás	Nehézség
1	fogalmi/döntési	a fejezet elolvasása	rövid indoklás	a válasz a C-tudást nevezi meg indoklással	könnyű
2	tervezési	olvasási utak ismerete	útvonalválasztás indoklással	a választás a firmware-szerephez illeszkedik	könnyű

Feladat	Típus	Előfeltétel	Késztermék	Elfogadás	Nehézség
3	implementációs	`~/iot-labor/` írható	`stack/` könyvtár	`ls -la ~/iot-labor/` mutatja a `stack` könyvtárat	könnyű
4	hibakeresési/kiterjesztési	a továbbvitt eredmények elve	magyarázat és helyes `cp`-lépés	a válasz kimondja, hogy nincs automatikus feloldás	közepes

Kódblokkok státusza. Minden kódblokk előtt vagy a bevezető mondatban jelöljük a státuszát:

- **Teljesen futtatható:** a blokk önmagában vagy a megadott fájlba illesztve lefordul és lefut.
- **Meglévő fájl módosítása:** a blokk egy korábbi labor fájljának megadott helyére illesztendő; a környező fájl a hivatkozott laborban teljes.
- **Fogalmi pszeudokód:** nem fordítható le, csak a működést szemlélteti; a laborhoz szükséges teljes kód külön blokkban szerepel.

A „Megoldások” a labor működéséhez szükséges kódrészletet teljesen adja meg; a nem kötelező kiegészítések jelölt vázak lehetnek. A „teljes releváns kódrészlet” ígérete tehát a kötelező laborútra vonatkozik, nem minden szemléltető blokkra.

Várt kimenet. Minden „Várt kimenet” blokk kétféle lehet: **futtatott** (a kiadás előtt ténylegesen lefuttatott, determinisztikus példa kimenete) vagy **illusztratív** (a helyes működés várható tartalmát mutatja, de nem futási eredmény). A blokk címkéje vagy a közvetlenül előtte álló mondat mondja meg, melyik. Determinisztikus programnál a kiadás előtt a tényleges kimenetből kell generálni a példát; nem determinisztikus vagy hardverfüggő kimenetnél az illusztratív jelölés kötelező. A könyvgenerátor a kódot nem futtatja, ezért a jelen kéziratban szereplő kimenetek illusztratívak, kivéve, ahol a szöveg futtatott eredményt jelöl.

A laborokban megadott mért értékeket (késleltetés, fogyasztás, hatótáv) a könyv csak akkor közöl, ha azok forrásból származnak vagy „illusztratív” jelölést kapnak. A `native_sim` időzítése és energiafogyasztása nem hardverhű, ezért minden ilyen mérésnél a szöveg ezt jelzi.

1.1.14 Feladatok

1. Fogalmi kérdés: Melyik előismeret hiánya okozna legnagyobb nehézséget a Mikrovezérlő vs. Linuxos processzor alfejezet megértésekor? Indokolja röviden.
2. Tervezési feladat: Firmware-fejlesztőként dolgozik, és a TerraSense `ts-node` szoftverét kell elkészítenie. A fenti olvasási utak közül melyiket választaná, és mely fejezeteket tartaná feltétlenül szükségesnek? Válaszát indokolja.
3. Implementációs feladat: Hozza létre a TerraSense labor-repót a leírt szerkezettel: `~/iot-labor/` és benne a `stack/` könyvtár. Ellenőrizze, hogy a `mkdir -p` parancs után a `ls -la ~/iot-labor/` a `stack` könyvtárat mutatja.

4. Hibakeresési vagy kiterjesztési feladat: Egy kolléga azt állítja, hogy a `~/iot-labor/i004` könyvtárban létrehozott fájlokat a következő laborok automatikusan megtalálják. Mi ezzel a probléma? Mit kell tennie a helyes használathoz a Továbbvitt eredmények elve szerint?

1.1.15 Megoldások

1. A legnagyobb nehézséget a **C pointerek és bitműveletek hiánya** okozná, mert a mikrovezérlős fejezetek a regisztereket és a memóriát közvetlenül, pointerekkel és bitmaszkokkal érik el. RTOS-ismeret hiánya nem akadály, mert azt a könyv nulláról tanítja; a C++ hiánya a gateway-résznél, a Linux hiánya a laborok futtatásánál okozna gondot, de a legalapvetőbb az eszközrétegben a C-tudás.
2. Az eszközoldali út a megfelelő választás, mert a firmware-fejlesztőnek a II. rész (mikrovezérlő, RTOS, szenzorok) és a III. rész MQTT-részei a legfontosabbak. A IV. rész a gateway miatt hasznos, de a backend (V. rész) elhagyható, kivéve az ingest fogalmát, hogy tudja, milyen formában érkezik az adat a felhőbe. A VI. részből a biztonsági alapok szükségesek, mert a firmware-nek is biztonságosnak kell lennie.
3. A parancsok:

```
mkdir -p ~/iot-labor/stack
cd ~/iot-labor
ls -la
```

A várt kimenetben a `stack` könyvtár látható.

4. A kolléga téved: a későbbi laborok **nem automatikusan** találják meg a korábbi fájlokat. A könyv elve szerint minden labor elején explicit `cp`-vel vagy relatív hivatkozással kell a korábbi labor fájlját felhasználni. Ha ez elmarad, a labor nem reprodukálható, és a továbbvitt eredmény függősége sérül. A helyes gyakorlat: a „Munkakönyvtár:” blokkban megadott `cp` parancsot futtatni, vagy a fájlra relatív elérési úttal hivatkozni a saját `iNNN/` könyvtárból.

1.1.16 Rövid összefoglaló

Ez az alfejezet rögzítette a könyv előismeret-feltételeit: erős C, C++ és Linux-tudás, valamint JEE-ből csak fogalmi ismeret. Bemutatta a TerraSense mintarendszert, amelynek komponensei: `ts-node` (Zephyr, C), `ts-gw` (Linux-gateway, C++), Mosquitto broker, `ts-backend` (Spring Boot), TimescaleDB és Grafana. A rendszer azonosító- és topic-konvencióit az A TerraSense mintarendszer architektúrája alfejezet véglegesíti. A laborok a `~/iot-labor/iNNN/` könyvtárban, a közös futtatókörnyezet a `~/iot-labor/stack/compose.yaml` fájlban található; a futtatókörnyezet felállítását a A labor felállítása: Linux host, Docker, Zephyr, Mosquitto, Java alfejezet írja le. A továbbvitt eredmények elve biztosítja, hogy minden labor a korábbiak kimenetére épül. Ajánlott olvasási utak: teljes, eszközoldali és backend-oldali.

1.1.17 Források és ellenőrzési hiányok

Ez az alfejezet bevezető jellegű, nincs szükség külső forrásra. A technológiai verziókat (Zephyr, Spring Boot, Mosquitto, Java) a fenti verziómátrix rögzíti; a konkrét API-neveket a későbbi alfejezetek ehhez a mátrixhoz kötve adják meg. Ahol ez nem történik meg, az a kiadás előtti, blokkoló szerkesztői feladat a fenti ellenőrzési politika szerint. A szabályozási határidők és a hardverjellemzők a megfelelő szakaszoknál megnevezett forrásokhoz kötve szerepelnek.

2 Az IoT-rendszer anatómiája

2.1 Mi az IoT? Rétegek, szereplők, adatút

2.1.1 Motiváció

Egy üvegházban több száz növény sorsa múlhat azon, hogy egy hőmérséklet- vagy talajnedvesség-érték időben eljut-e a gazdához, és hogy a rendszer automatikusan kinyitja-e a szelepet, mielőtt kiszárad a termés. Ha egyetlen mikrovezérlőre bízzuk az egészet, amely lokálisan mér és kapcsol, az már önmagában is hasznos – de ha a gazda több hektárról, több üvegházból akar egyetlen felületen áttekintést kapni, és távolról beavatkozni, akkor a rendszernek hálózatra, üzenetközvetítőre, adatbázisra és megjelenítő rétegre is szüksége van. Az IoT-rendszer éppen ezt a teljes láncot jelenti a fizikai jel digitalizálásától a felhasználói döntésig vagy az automatikus beavatkozásig.

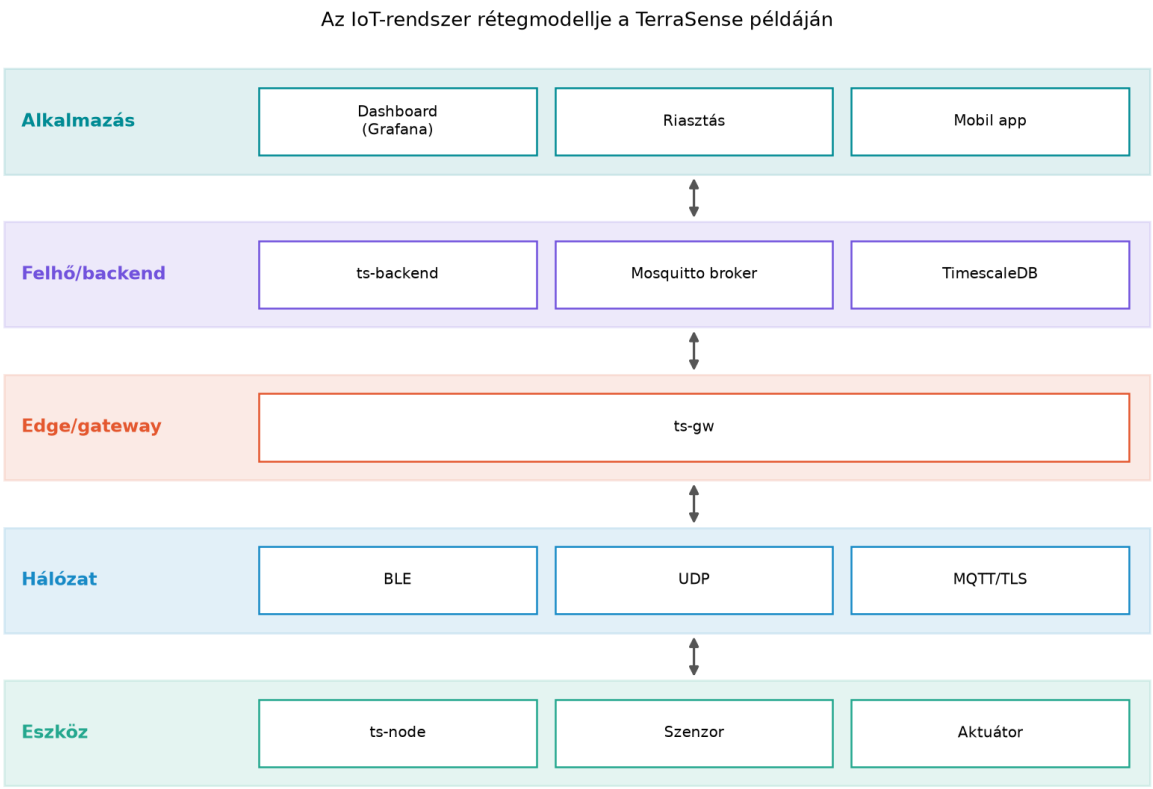
2.1.2 Fogalmak és működés

Definíció. Az **IoT-rendszer** olyan elosztott rendszer, amely a fizikai világ jeleit szenzorokkal digitalizálja, az adatokat valamilyen hálózaton továbbítja, feldolgozza, és az eredményt felhasználói felületen megjeleníti, vagy aktuátorokon keresztül visszahat a fizikai világra.

A definícióban kulcsfontosságú a *visszahatás*: a tiszta adatgyűjtés még nem feltétlenül IoT, ahhoz a vezérlési irány is hozzátartozik, még ha az csak egy relé bekapcsolása is.

Határok és rokon fogalmak.
- A **beágyazott rendszer** (embedded system) zárt, önállóan működő számítógép, amelynek nincs feltétlenül hálózati kapcsolata. Egy IoT-eszköz mindig beágyazott rendszer, de a beágyazott rendszer nem mindig IoT. - Az **IIoT** (Industrial Internet of Things) az ipari környezetben alkalmazott IoT, ahol a megbízhatóság, a determinisztikus időzítés és a meglévő ipari protokollok (például Modbus, OPC UA) integrációja hangsúlyosabb. - A **CPS** (cyber-physical system, kiberfizikai rendszer) olyan rendszer, ahol a számítási folyamatok és a fizikai folyamatok szorosan összefonódnak, gyakran valós idejű visszacsatolással. Az IoT és a CPS átfedik egymást; az IoT inkább a hálózati és felhőszolgáltatási oldalt hangsúlyozza, a CPS a fizikai folyamatokkal való szoros kölcsönhatást.

Rétegmodell. Az IoT-rendszereket gyakorlati okokból rétegekre bontjuk, mert a különböző funkciók más-más hardveren, operációs rendszeren és programozási modellen futnak. A TerraSense mintarendszerben a következő öt réteget különböztetjük meg:

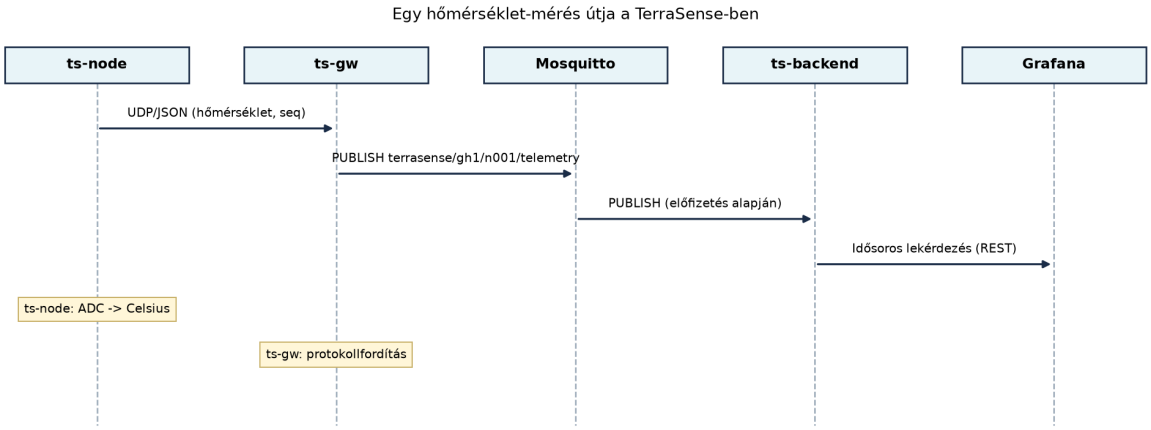


2.1. ábra: Az IoT-rendszer rétegmodellje a TerraSense példáján - A rétegek alulról felfelé mutatják az adatút irányát; a nyilak a köztük lévő függőséget jelzik.

A rétegek felelőssége:

- **Eszközréteg:** a fizikai világ közvetlen érzékelése és a beavatkozás. Itt fut a Zephyr RTOS a mikrovezérlőn (ts-node), csatlakoznak a szenzorok (hőmérséklet, páratartalom, talajnedvesség) és az aktuátor (öntözőszelep). Az eszköz digitalizálja a jeleket, előszűri azokat, időbélyeggel látja el, és elküldi vagy fogadja a parancsokat.
- **Hálózati réteg:** az adatcsere fizikai és logikai csatornája. TerraSense-ben ez lehet BLE a valós hardveren, UDP a szimulációban, illetve MQTT/TLS a gateway és a felhő között. A hálózati réteg határozza meg a hatótávot, az energiafelhasználást és a megbízhatóságot.
- **Edge/gateway réteg:** az eszközök és a felhő közötti híd. A ts-gw C++ szolgáltatás feladata a protokollfordítás (például UDP-ből MQTT-be), a pufferezés és a biztonsági határ kialakítása. Itt történhet helyi aggregálás vagy eseménydetektálás is, csökkentve a felhőbe küldött adatmennyiséget.
- **Felhő/backend réteg:** a központi adatfeldolgozás és tárolás. A ts-backend Spring Boot alkalmazás fogadja az MQTT-üzeneteket, idősoros adatbázisba (TimescaleDB) írja azokat, REST API-t biztosít a lekérdezéshez, és kezeli az eszközök digitális árnyékát. A Mosquitto broker az üzenetközvetítő.
- **Alkalmazási réteg:** a felhasználóval való kapcsolat. A Grafana dashboard vizualizálja az idősorokat, a riasztási szabályok értesítéseket küldenek, és innen indíthatók a felhasználói parancsok is (például szelep nyitása).

Adatút és vezérlési út. A rendszerben kétirányú kommunikáció zajlik. A *telemetria* (felfelé irányuló adat) a szenzortól a felhasználóig halad; a *parancs* (lefelé irányuló vezérlés) a felhasználótól vagy a backend logikától indul, és az aktuátorig jut.



2.2. ábra: Egy hőmérséklet-mérés útja a TerraSense-ben - Telemetria felfelé: az eszköz küld, a backend fogad. Az időbélyeg az eszközön keletkezik.

A telemetria útja során az adat többször átalakul: az ADC-nyersértékből fizikai mennyiség (például °C) lesz, majd strukturált üzenet (JSON, CBOR vagy más), később adatbázisrekord, végül vizuális grafikonpont. A vezérlési út fordított irányú: a felhasználó vagy egy szabály elindít egy parancsot, amely a backendtől MQTT-üzenetként a gatewayen át az eszközhöz jut, ahol az aktuátort vezérli, majd visszanyugtázást küld.

Szereplők. Egy IoT-rendszer életciklusában több szervezeti és emberi szereplő érintett:

- **Gyártó (OEM):** megtervezi és legyártja az eszközt, beleértve a firmware-t és a biztonsági elemeket.
- **Üzemeltető:** telepíti, konfigurálja és felügyeli a rendszert, kezeli a frissítéseket, a hibákat és a kapacitást.
- **Felhasználó:** a végfelhasználó, aki a dashboardot nézi, riasztásokat kap, és manuálisan beavatkozik (például szelepet nyit).
- **Platformszolgáltató:** a felhő-infrastruktúrát, a broker szolgáltatást vagy a teljes IoT-platformot nyújtja (például AWS IoT, Azure IoT). A TerraSense-ben ezt a közös laborstack (Mosquitto, TimescaleDB) szimulálja.

2.1.3 Döntési szempontok

Az IoT-architektúra tervezésekor a legfontosabb kérdés, hogy **melyik rétegbe helyezzük a logikát**. A döntés kihat az energiafogyasztásra, a késleltetésre, a sávszélességre és az adatvédelemre.

Szempont	Eszközön	Edge/gateway	Felhőben
Energiafelhasználás – helyi számítás	magasabb lehet, ha sok mintát dolgoz fel	közepes	nincs helyi számítás
Energiafelhasználás – rádiós adatküldés	alacsony, ha kevés adatot küld	közepes, ha továbbít	magas, ha minden nyers adat felmegy
Energiafelhasználás – teljes rendszer	a helyi számítás és a rádiózás együtt; akkor előnyös, ha kiváltja a rádiózást	a két végpont közötti átvitelt csökkenti	az eszköz energiájában nem jelenik meg, de a teljes rendszerébe beletartozik

Szempont	Eszközön	Edge/gateway	Felhőben
Késleltetés a döntésig	minimális	néhány ms–s	100 ms–s vagy több
Sávszélesség-igény	alacsony, ha előszűr	csökkentheti	magas, ha minden nyers adat megy
Rugalmasság, frissíthetőség	nehézkes (OTA szükséges)	egyszerűbb	egyszerű
Adatvédelem	helyben marad az adat	részben helyben	központosított, kockázatos

Az energiafogyasztásról szóló döntéshez a teljes ciklust kell összehasonlítani, nem külön a CPU-t és külön a rádiót. Legyen n a ciklusonként keletkező minta száma, E_{minta} egy minta helyi feldolgozásának energiája, E_{radio} egy rádiós adás energiája. Ha minden mintát azonnal továbbítunk, a ciklus energiaigénye $n \cdot E_{\text{radio}}$; ha helyben előszűrünk, és csak az eredményt küldjük, $n \cdot E_{\text{minta}} + E_{\text{radio}}$. A helyi feldolgozás akkor takarít meg energiát, ha

$$n \cdot E_{\text{minta}} + E_{\text{radio}} < n \cdot E_{\text{radio}},$$

azaz $E_{\text{minta}} < E_{\text{radio}} \cdot \frac{n-1}{n}$. Például $n = 60$, $E_{\text{minta}} = 0{,}5$ mJ és $E_{\text{radio}} = 50$ mJ esetén a nyers továbbítás 3000 mJ, a helyi előszűrés 80 mJ egy ciklusban; a számok illusztratívak, nem mérési eredmények. Ellenben $n = 1$ esetén a helyi feldolgozás csak hozzáad a költséghez, és egy számításigényes algoritmusnál E_{minta} nagyobb lehet, mint E_{radio}, ezért a helyi feldolgozás önmagában nem energiatakarékos. A felhőben végzett számítás az eszköz szempontjából nem növeli a fogyasztást, de a teljes rendszer energiaigényébe a szerveroldali fogyasztás is beletartozik; ezt a döntésnél külön kell mérlegelni.

A TerraSense-ben a szenzoradat előfeldolgozása (például átlagolás, küszöbérték-figyelés) az eszközön vagy a gatewayen történik, míg a hosszú távú tárolás és a komplex elemzés a felhőben. A döntést mindig a konkrét követelményekhez kell kötni: ha a szelepet azonnal kell zárni fagy esetén, a szabályt az eszközre vagy a gatewayre kell tenni, nem a felhőbe.

2.1.4 Kidolgozott példa

Tegyük fel, hogy a TerraSense `n001` csomópontja 10 másodpercenként méri a hőmérsékletet, és a `ts-gw` gateway továbbítja a felhőbe. Kövessük végig egy mérés útját lépésről lépésre:

1. **Érzékelés:** a hőmérséklet-szenzor analóg feszültséget ad, amelyet a mikrovezérlő ADC-je digitális értékké alakít. A Zephyr szenzor-API ezt már kalibrált fizikai mennyiséggé (°C) konvertálja.
2. **Helyi feldolgozás:** az eszközszoftver hozzáadja a `site` (`gh1`), a `node_id` (`n001`) és a `boot_id` azonosítót, a bootoláson belül monoton növekvő `seq` sorszámot és az eseményidőt (`ts`, UTC epoch ms). Az üzenet tartalma például: `{"site":"gh1","node_id":"n001","boot_id":"b7c1`
3. **Átvitel a gatewayhez:** a szimulációban az eszköz UDP-csomagban küldi az adatot a `ts-gw`-nek. A gateway fogadja, ellenőrzi a sorszámot, és átalakítja MQTT-üzenetté.
4. **Mqtt publish:** a `ts-gw` publikálja a `terrasense/gh1/n001/telemetry` topicra a Mosquitto brokerhez. A broker az előfizetőknek továbbítja az üzenetet.
5. **Backend ingest:** a `ts-backend` feliratkozott a brokerre, és fogadja az üzenetet. Ellenőrzi az idempotenciát és a sémát, majd beírja a TimescaleDB-be.
6. **Megjelenítés:** a Grafana egy REST API-n keresztül lekérdezi az utolsó 5 perc adatait, és vonaldiagramon ábrázolja a hőmérséklet változását.

Ez a példa mutatja, hogy az adat minden rétegben más-más reprezentációt kap: analóg jel → digitális szám → fizikai mennyiség → JSON-üzenet → adatbázisrekord → vizuális pont.

Továbbvitt eredmény. Az itt bevezetett rétegmodell (eszköz–hálózat–edge/gateway–felhő–alkalmazás) és az adatút/vezérlési út kettőssége az egész könyv alapja. A Nem funkcionális követelmények: energia, költség, élettartam, megbízhatóság, biztonság alfejezet ezekre a rétegekre építve tárgyalja a nem funkcionális elvárásokat, a A TerraSense mintarendszer architektúrája alfejezet konkrét komponensekké bontja a rétegeket, a A gateway szerepe: híd, puffer, biztonsági határ alfejezet a gateway felelősségét mélyíti el, a Ingest: MQTT-ből a backendbe, idempotensen alfejezet pedig a felhőbe érkező adatok fogadását részletezi.

2.1.5 Ellenőrzés és hibakeresés

Mivel ez a fejezet elméleti, az ellenőrzés leginkább a modell megértésére irányul. Tegyük fel a következő kérdéseket, és próbáljuk megválaszolni őket a rétegmodell alapján:

- **Hol keletkezik az időbélyeg?** Helyesen az eszközön, mert a hálózati késleltetés torzíthatja az időt. Ha a backend adná az időt, a valós eseményidő elveszne.
- **Mi történik, ha a gateway és a felhő közötti kapcsolat megszakad?** A gatewaynek pufferelnie kell az üzeneteket (store-and-forward), különben adatvesztés lép fel. Ezt később terjedelmesen tárgyaljuk.
- **Hogyan lehet nyomon követni egy üzenet útját?** A `(site, node_id, boot_id, seq)` négyes kanonikus eseményazonosságot ad: a `site` és a `node_id` megmondja, honnan jött, a `boot_id` elválasztja az újraindításokat, a `seq` pedig a bootoláson belüli sorrendet és a duplikátumokat kezeli.

Gyakorlati ellenőrzésként a későbbi laborokban minden réteghez lesz konkrét teszt: az eszköz logjai, az MQTT-forgatókönyv, a backend API válaszai.

2.1.6 Tipikus hibák és félreértések

1. **Az IoT-t a sima beágyazott rendszerrel azonosítani.** Ha egy mikrokontroller csak lokálisan mér és kapcsol, de nincs hálózati kapcsolata vagy távoli elérhetősége, az nem IoT. Az IoT-hez a hálózati réteg és a távoli adatfeldolgozás is hozzátartozik.
2. **Minden logikát a felhőbe tenni.** Ez növeli a késleltetést és az energiafogyasztást az eszközön, mert minden nyers adatot továbbítani kell. A kritikus, gyors beavatkozást igénylő szabályokat az eszközre vagy az edge-re kell helyezni.
3. **Az időbélyeget a backend hozzáadni.** Az eseményidőnek az érzékelés pillanatát kell tükröznie, nem a fogadás pillanatát. A helyi idő UTC nélkül szintén hibás.

2.1.7 Feladatok

1. **(Fogalmi/döntési)** Sorolja fel a TerraSense rétegmodell öt rétegét, és mindegyikhez nevezzen meg egy konkrét komponenst és egy felelősséget.
2. **(Számolási/tervezési)** A ts-node 10 másodpercenként küld egy 50 bájtos telemetria-üzenetet. Mennyi nyers adat keletkezik naponta? Ha a telemetriát egy évig megőrizzük, mekkora tárhelyre van szükség (a TimescaleDB tömörítését figyelmen kívül hagyva)?
3. **(Implementációs előkészítő)** Tervezze meg a TerraSense-hez tartozó MQTT topic-struktúrát. Gondoljon a telemetriára, az állapotra, a parancsokra és a jelenlétre. Ez a feladat előkészíti a későbbi fejezetekben használt topic-konvenciót.

4. **(Hibakeresési/kiterjesztési)** A Grafana dashboard hirtelen nem frissül, de a ts-node továbbra is mér és küld. Végigmenve a rétegeken, sorolja fel, hol lehet a hiba, és milyen ellenőrzést végezne el az egyes rétegekben.

2.1.8 Megoldások

1. A rétegek: eszköz (ts-node, szenzorok, aktuátorok; érzékelés és beavatkozás), hálózat (BLE, UDP, MQTT/TLS; adatcsere), edge/gateway (ts-gw; protokollfordítás, pufferezés), felhő/backend (ts-backend, Mosquitto, TimescaleDB; feldolgozás, tárolás), alkalmazás (Grafana, riasztás; megjelenítés, felhasználói interakció).
2. Napi üzenetszám: $24 \cdot 3600/10 = 8640$. Napi adatmennyiség: $8640 \cdot 50$ B $= 432\,000$ B $=$ 432 kB. Éves adat: $432 \text{ kB} \cdot 365 = 157\,680$ kB $\approx 157{,}7$ MB. Tárhelyigény tehát körülbelül 158 MB (tömörítés és indexelés nélkül).
3. Elfogadható topic-struktúra (a könyv későbbi konvencióját megalapozva):
   - `terrasense/<site>/<node>/telemetry` – telemetria
   - `terrasense/<site>/<node>/state` – állapot
   - `terrasense/<site>/<node>/cmd` – parancsok
   - `terrasense/<site>/<node>/cmd/result` – parancsválaszok
   - `terrasense/<site>/<node>/status` – jelenlét (retained, Last Will)
     A `<site>` például `gh1`, a `<node>` például `n001`.
4. Hiba lehetőségek rétegenként:
   - **Eszköz:** a szenzor nem olvas, a küldési logika meghibásodik, a `seq` nem növekszik. Ellenőrzés: eszköz logja, UDP-csomag a gatewayen.
   - **Hálózat:** a BLE vagy UDP kapcsolat megszakad, a gateway nem éri el a brokert. Ellenőrzés: `ping`, MQTT-kapcsolat állapota.
   - **Gateway:** a ts-gw nem fut, vagy a protokollfordítás hibás. Ellenőrzés: gateway log, `systemctl status`.
   - **Broker:** a Mosquitto leállt, vagy a topic nem létezik. Ellenőrzés: `mosquitto_sub` feliratkozás.
   - **Backend:** az ingest folyamat nem fut, vagy a séma nem egyezik. Ellenőrzés: backend log, REST API hívás.
   - **Alkalmazás:** a Grafana adatforrása rossz, vagy a lekérdezés hibás. Ellenőrzés: Grafana query inspector.

2.1.9 Rövid összefoglaló

Ebben az alfejezetben definiáltuk az IoT-rendszert, és megkülönböztettük a beágyazott rendszerektől, az IIoT-tól és a CPS-től. Bevezettük a TerraSense öt rétegét: eszköz, hálózat, edge/gateway,

felhő/backend és alkalmazás. Végigkövettük egy hőmérséklet-mérés útját a szenzortól a dashboardig, láttatva az adat átalakulásait. Bemutattuk a szereplőket (gyártó, üzemeltető, felhasználó, platformszolgáltató). A **továbbvitt eredmény** a rétegmodell és az adatút/vezérlési út kettőssége, amelyre a Nem funkcionális követelmények, a TerraSense architektúra, a Gateway szerepe és a Backend ingest alfejezetek építenek. A döntési szempontoknál kiemeltük, hogy a logika elhelyezése hatással van az energiafelhasználásra, a késleltetésre és a sávszélességre.

2.1.10 Források és ellenőrzési hiányok

A fejezet kiindulópontjai:
- NIST Cybersecurity for IoT Program: https://www.nist.gov/itl/applied-cybersecurity/nist-cybersecurity-iot-program
- ETSI EN 303 645: https://www.etsi.org/deliver/etsi_en/303600_303699/303645/
- AWS IoT dokumentáció: https://docs.aws.amazon.com/iot/

Ezek a hivatkozások kiindulópontként szolgálnak; a fejezet konkrét API-nevet, szolgáltatásverziót vagy jogszabályi hivatkozást nem állít. A rétegmodell és az adatút általános elvek. A források kiadásának és a jogszabályi hivatkozásoknak a rögzítése a kiadás előtti, blokkoló szerkesztői feladat (hatókör: e fejezet forrásjegyzéke).

2.2 Nem funkcionális követelmények: energia, költség, élettartam, megbízhatóság, biztonság

2.2.1 Motiváció

A TerraSense üvegházában először az tűnik a legfontosabbnak, hogy a szenzorok jól mérjék a hőmérsékletet, a páratartalmat és a talajnedvességet. Az első prototípusban a fejlesztő egy Wi-Fi-képes mikrovezérlőt választ, percenként küld JSON-adatot a felhőbe, és minden eszközre ugyanazt a gyári jelszót állítja be. A demó működik, de amikor a termelő 100 csomópontot telepítene, kiderül, hogy az elemeket hetente cserélni kell, a hálózat nem bírja a sok kis üzenetet, és egy nyilvános porton keresztül illetéktelenek is elérik a rendszert.

Ez a fejezet azt tanítja meg, hogyan kerüljük el ezt a helyzetet. Az IoT-rendszerekben a nem funkcionális követelmények (Non-Functional Requirements, NFR) gyakran erősebben korlátozzák a tervezést, mint a funkcionális elvárások. Egy hőmérséklet-mérés önmagában egyszerű, de ha évekig kell elemről működnie, olcsón, nagy darabszámban, megbízhatóan és biztonságosan, akkor minden döntés kompromisszum.

2.2.2 Fogalmak és működés

Definíció. A nem funkcionális követelmény az IoT-rendszer olyan minőségi jellemzője vagy korlátja, amely nem egy konkrét funkciót ír le, hanem azt, hogy a rendszer milyen feltételek mellett nyújtja azt. Ide tartozik többek között az energia, a költség, az élettartam, a késleltetés, az adatgyakoriság, a sávszélesség, a megbízhatóság, a rendelkezésre állás, az offline működés, a biztonság és az adatvédelem.

A funkcionális követelmény például az, hogy a ts-node mérje a hőmérsékletet Celsius-fokban. Az NFR pedig az, hogy a mintavétel 10 másodpercenként történjen, a telemetriát 10 percenként, 200 bájtnál kisebb üzenetben, legalább 99,9%-os kézbesítéssel, TLS-titkosított csatornán keresztül küldje, miközben az eszköz két AA elemről legalább 12 hónapig üzemel.

2.2.2.1 Energia és élettartam

Az IoT-eszközök jelentős része akkumulátorról vagy elemről működik, mert a telepítési helyen nincs kiépített tápegység. Az energiafelhasználást az határozza meg, hogy az eszköz milyen gyakran ébred fel, mennyi ideig aktív a rádió, mekkora a processzor terhelése, és milyen mély alvómódokat használ. Az élettartamot az energiatároló kapacitása és az átlagos áramfelvétel hányadosa adja.

A cél az, hogy az eszköz évekig működjön beavatkozás nélkül, mert a szervizelés (elemcsere, karbantartás) költsége nagy telephelyen jelentős. Az energiatervezés részletes számítását az Energiamodell és az elemélettartam kiszámítása alfejezetben végezzük el.

A szelep energiamodellje. A TerraSense öntözőszelepe bistabil (impulzusvezérelt) szelep: 3,0 V-on 0,5 A-t vesz fel, egy kapcsolási impulzus 50 ms. Egy impulzus energiája $3{,}0 \cdot 0{,}5 \cdot 0{,}05 = 0{,}075$ J; a tápátalakító 80%-os hatásfokával ez $0{,}075/0{,}8 = 0{,}094$ J. A követelmény napi legfeljebb egy nyitás-zárás ciklus, azaz két impulzus, így a szelep napi energiaigénye $2 \cdot 0{,}094 = 0{,}188$ J, egy évben $365 \cdot 0{,}188 = 68{,}4$ J. Két AA lítium elem névleges energiája mintegy 32,4 kJ, ennek a szelep 0,21%-át használja el. A folyamatosan gerjesztett szelep ugyanekkora árammal 1,5 W-ot vesz fel; évi folyamatos működése 47,3 MJ, ami messze meghaladja az elemek kapacitását, és már napi 1 s gerjesztés is 548 J/év, a 12 hónapos cél 1,7%-a. A számok illusztratívak, a tényleges szelep adatlapját és a tápátalakító hatásfokát méréssel kell igazolni. A modellt az Energiamodell és az elemélettartam kiszámítása alfejezetnek is át kell vennie.

2.2.2.2 Költség és darabszám

A költségnél nem elég a mikrovezérlő árát nézni. A teljes bekerülési költség (Total Cost of Ownership, TCO) magában foglalja az alkatrészeket, a nyomtatott áramkört, a tokozást, a gyártást, a tesztelést, a logisztikát, a telepítést, az üzemeltetést és a karbantartást. Nagy darabszám esetén az egységköltség csökken, de a hibás eszközök cseréjének és a távoli frissítéseknek a költsége nő. Ezért a tervezéskor a darabszámot is rögzíteni kell: teljesen más megoldást választunk 10 prototípushoz, mint 100 000 telepített eszközhöz.

2.2.2.3 Késleltetés, adatgyakoriság, sávszélesség

A késleltetés (latency) az az idő, ami a fizikai esemény (például a hőmérséklet küszöb átlépése) és a rendszer válasza (riasztás, szelepnyitás) között eltelik. Az adatgyakoriság azt mondja meg, milyen sűrűn küld az eszköz adatot. A sávszélesség az időegység alatt átvitt adatmennyiség, amelyet az üzenet mérete és a gyakoriság együtt határoz meg.

Három mennyiség szorosan összefügg: ha ritkábban küldünk, kevesebb energiát fogyasztunk és kisebb a sávszélesség-igény, de nő a lehetséges késleltetés. Ha gyors riasztás kell, vagy gyakrabban kell küldeni, vagy az eszköznek helyben kell észlelnie a kritikus állapotot, és csak akkor kell azonnal kommunikálnia.

2.2.2.4 Megbízhatóság, rendelkezésre állás, offline működés

A megbízhatóság (reliability) azt fejezi ki, hogy a rendszer a kívánt ideig hiba nélkül működik. A rendelkezésre állás (availability) pedig azt, hogy a rendszer milyen arányban elérhető a felhasználó számára. Az IoT-ban a hálózati kapcsolat gyakran megszakad, ezért az eszköznek és a gatewaynek offline módban is működnie kell: az adatokat pufferelni kell, és a kapcsolat helyreállása után

vissza kell tölteni. Ezt a mintát részletesen a Store-and-forward: offline működés és visszatöltés alfejezet tárgyalja.

A 99,9%-os üzenetérkezés nem az MQTT QoS 1 szintje, hanem rendszerszintű, egyedi esemény-azonosítókon mért arány. Definíció szerint 30 napos gördülő ablakban a nevező a gateway által a backend felé kiadott egyedi események száma a `(site, node_id, boot_id, seq)` kulcs szerint; a számláló azoké, amelyek a gateway átvétele után legfeljebb 5 percen belül, hálózatkiesés esetén a kapcsolat helyreállása után legfeljebb 5 percen belül tartósan bekerültek a TimescaleDB-be. A duplikált kézbesítés egyszer számít. A QoS 1 csak a broker és a kliens közötti átvitelt garantálja, a végpontok közötti tárolást nem, ezért a méréshez a gateway és a backend egyeztetése szükséges.

Az elvárt hibaforgatókönyvek: broker-újraindítás, backend-újraindítás, hálózati szakadás, eszköz-újraindítás és betelt háttértár. A gateway a visszatöltésig pufferel, a backend idempotens ingestje a kanonikus kulcson deduplikál. A napi egyeztetés a gateway által kiadott eseményszámlálót veti össze a TimescaleDB-ben tárolt egyedi kulcsok számával; a különbség riasztást ad. 30 napnál rövidebb üzem nem ad értékelhető arányt, ezért a 99,9% csak 30 nap után értékelhető.

2.2.2.5 Biztonság és adatvédelem

A biztonság a klasszikus CIA-hármast jelenti: bizalmasság (confidentiality), sértetlenség (integrity) és rendelkezésre állás (availability). Az IoT-eszközök különösen kitettek, mert fizikailag hozzáférhetők, korlátos erőforrásúak, és gyakran hosszú élettartamúak. A biztonsági követelményeket a tervezés elején kell meghatározni, nem utólag hozzáadni.

Az adatvédelem a személyes adatok kezelésére vonatkozó jogi és etikai követelményeket foglalja magában. A GDPR (General Data Protection Regulation) az EU-ban szabályozza a személyes adatok gyűjtését, tárolását, felhasználását és törlését. Egy üvegház hőmérséklet-adata önmagában nem személyes adat, de ha az adatbázisban összekapcsoljuk a telepítő nevével, telefonszámával vagy a termelő cégével, akkor már személyes adatnak minősülhet. Ezért a tervezéskor adatleltárt kell készíteni, és minimalizálni kell a gyűjtött adatokat.

A szabályozási környezet gyorsan változik. Az ETSI EN 303 645 szabvány alapvető biztonsági követelményeket fogalmaz meg fogyasztói IoT-eszközökre: például nem lehet gyári alapértelmezett jelszó, biztosítani kell a szoftverfrissítést és a sebezhetőségek bejelentését. Az EU Cyber Resilience Act (CRA) a digitális termékekre ír elő horizontális biztonsági követelményeket; a konkrét határidőket és alkalmazási köröket mindig ellenőrizni kell a hatályos jogszabályban.

Két biztonsági profil. A laborok helyi, elkülönített oktatási profilban indulnak: a broker és a backend csak a loopback interfészen érhető el, és a példák a lépések megértéséhez HTTP-t és névtelen MQTT-t használhatnak. Ez a profil kizárólag a helyi gépen, elkülönített laborhálózaton elfogadható, és nem része a kiadásra szánt rendszernek. A záróprofil és az átvételi teszt kötelező követelménye a TLS 1.3, az mTLS eszközönkénti tanúsítvánnyal, valamint a REST API hitelesítése és jogosultsághoz kötése: a hitelesítés nélküli REST-hívás 401 vagy 403 választ kap, a tanúsítvány nélküli MQTT-kapcsolatot a broker elutasítja. A két profil közötti átmenet a biztonsági fejezetekben lépésenként történik, és a záróprojekt csak a záróprofilban fogadható el.

2.2.3 Döntési szempontok

Az NFR-ek között gyakoriak az ütközések. A döntést mindig a konkrét alkalmazás kontextusában kell meghozni, és a kompromisszumot explicit módon rögzíteni kell a követelménytáblázatban.

Ütközés	Jellemző hiba	Feloldási irány
Energia vs. adatgyakoriság	Túl gyakori küldés, gyors elemlemerülés	Csak a szükséges adatot küldeni, helyi előfeldolgozás, adaptív gyakoriság
Költség vs. biztonság	Olcsó hardver, gyenge vagy hiányzó titkosítás	Hardveres kriptográfiai gyorsító, minimális biztonsági szint előírása
Megbízhatóság vs. költség	Redundancia mellőzése, egyetlen meghibásodási pont	Csak a kritikus útvonalon redundancia, olcsó tartalék (pufferelés)
Késleltetés vs. energia	Gyors válaszhoz folyamatos rádiókapcsolat	Helyi eseménydetektálás, csak rendkívüli eseménykor azonnali küldés
Darabszám vs. egyedi konfiguráció	Minden eszközt kézzel konfigurálnak	Automatikus provisioning, tanúsítvány-alapú identitás

A TerraSense esetében a legfontosabb ütközés az energia és a késleltetés között van. A termelőnek gyorsan kell tudnia, ha az üvegházban fagypont közelébe esik a hőmérséklet, de a folyamatos adatküldés lemerítené az elemeket. A feloldás az, hogy a ts-node helyben, 10 másodpercenként mintát vesz, a normál telemetriát 10 percenként küldi, és csak akkor küld azonnali üzenetet, ha a mért érték átlép egy kritikus küszöböt. A három periódus tehát külön követelmény: a **mintavételi periódus** 10 s, a **normál küldési periódus** 10 perc, a **kritikus esemény késleltetése** pedig a küszöbátlépés észlelésétől számítandó. A helyi vezérlési hurok (például a szelep működtetése) 1 s periódussal fut, és nem függ a felhő elérhetőségétől. Normál üzemben tehát ritka a rádiós kommunikáció, rendkívüli eseménykor viszont azonnali.

A 30 másodperces riasztási követelmény a **fizikai esemény** (a küszöb átlépésének) pillanatától a dashboardon való megjelenésig értendő, nem a kész minta továbbításától. A legrosszabb esetben az esemény a mintavételi periódus végén következik be, ezért a 10 s mintavételi periódus is a késleltetés része. A tervezett késleltetési büdzsé:

Szakasz	Tervezett felső korlát
Mintavételi periódus (legrosszabb fázis)	10 s
Helyi küszöbészlelés és döntés	1 s
MQTT publish a gatewayen és a brokeren át	2 s
Backend feldolgozás és tárolás	1 s
Dashboard frissítés	1 s
Összesen	legfeljebb 15 s, a 30 s-os követelményen belül

A felső korlátok tervezési célok, nem mért értékek; a tényleges késleltetést a kiadás előtti integrációs teszt méri. A teszt a mintavételi periódus legrosszabb fázisában idézi elő a küszöbátlépést, és a teljes láncot méri, beleértve az energiamérleget is.

2.2.4 Kidolgozott példa

Állítsuk össze a TerraSense nem funkcionális követelménytáblázatát. A sablon oszlopai:

- **Kategória:** az NFR területe.
- **Követelmény:** rövid, egyértelmű megfogalmazás.

- **Mérhető célérték:** szám, határérték vagy tesztelhető feltétel.
- **Ellenőrzés módja:** hogyan győződünk meg a teljesülésről.
- **Prioritás:** MoSCoW-besorolás (Must = kötelező, Should = fontos, Could = opcionális, Won't = most nem).

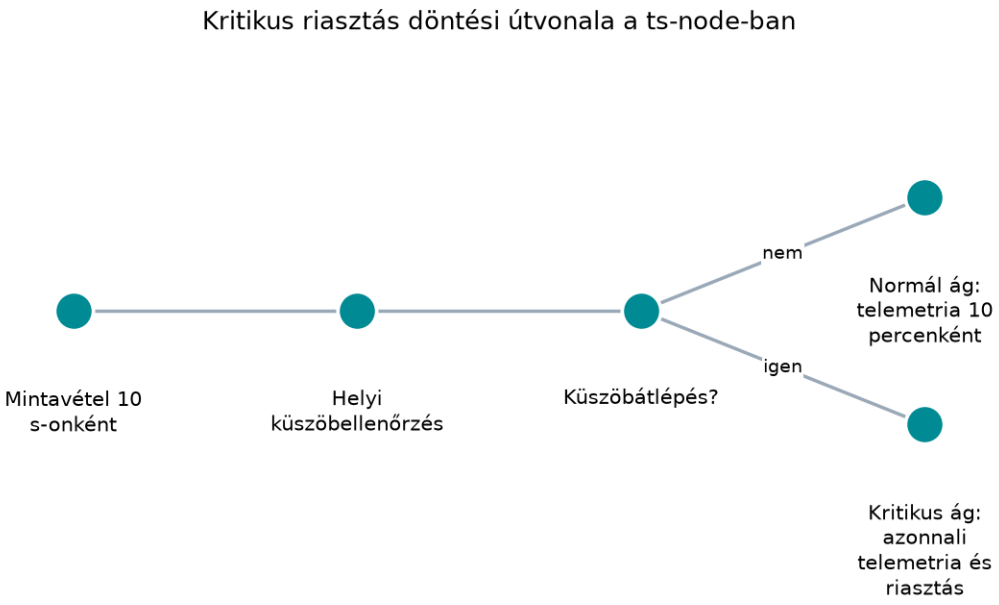
A táblázatot ne dokumentációként kezeljük, hanem élő tervezési eszközként: ha egy későbbi alfejezetben döntés változik, a táblázatot is frissíteni kell.

Kategória	Követelmény	Mérhető célérték	Ellenőrzés módja	Prioritás
Energia, élettartam	A ts-node valós hardveren 2 db AA lítium elemről legalább 12 hónapig üzemel	10 s mintavétel, 10 percenkénti telemetria, napi legfeljebb 1 nyitás-zárás ciklus (2 impulzus), átlagos áramfelvétel alapján	Energiabüdzsé-számítás a 2.2 szelepmodelljével (Energiamodell és az elemélettartam kiszámítása) és laboratóriumi teszt	Must
Költség	A ts-node anyagköltsége tömeggyártás esetén 60 EUR alatt marad	BOM-költség 1000 darabnál	Beszerzési árajánlat, gyártási elemzés	Should
Adatgyakoriság	Mintavételi periódus 10 s, normál telemetriai küldési periódus 10 perc	Konfigurálható időzítők	Konfiguráció-ellenőrzés, naplóelemzés	Should
Sávszélesség	Egy telemetriai üzenet CBOR-kódolással legfeljebb 200 B	Üzenet-séma, kódolási teszt	Unit teszt, protokoll-validálás	Should
Késleltetés	Kritikus riasztás a fizikai eseménytől (a küszöb átlépésétől) a dashboardon való megjelenésig legfeljebb 30 s	10 s mintavétel + helyi küszöbészlelés + azonnali MQTT publish	Integrációs teszt a mintavételi periódus legrosszabb fázisában, szimulált hálózattal	Must

Kategória	Követelmény	Mérhető célérték	Ellenőrzés módja	Prioritás
Megbízhatóság	A gateway-ből a backendbe legalább 99,9% üzenetérkezés 30 napos gördülő ablakban; a késés a gateway átvétele után (kiesésnél a helyreállás után) legfeljebb 5 perc	Egyedi `(site, node_id, boot_id, seq)` kulcsok, MQTT QoS 1, idempotens ingest, újraküldés	Eseményazonosítós egyeztetés broker- és backend-újraindítás után, hosszú távú megfigyelés	Must
Offline működés	A ts-gw legalább 7 nap telemetriát pufferel hálózatkiesés esetén	Helyi tároló kapacitása, visszatöltési logika	Szimulált hálózatkiesés, visszatöltési teszt	Must
Biztonság	A záróprofilban minden MQTT- és REST-kapcsolat TLS 1.3/mTLS, eszközönként egyedi tanúsítvánnyal; a REST-hitelesítés hiánya 401/403	Nincs jelszó-alapú azonosítás, tanúsítvány-validálás, jogosultság-ellenőrzés	Biztonsági teszt: tanúsítvány nélküli MQTT és hitelesítés nélküli REST elutasítása	Must
Adatvédelem	Csak az üzemeltetéshez szükséges adatokat gyűjtjük, és biztosított a törlés	Adatleltár, hozzáférés-kezelés, törlési eljárás	GDPR-megfelelőségi audit	Should
Szabályozás	Az ETSI EN 303 645 alapelvei és a CRA vonatkozó követelményei teljesülnek	Nincs alapértelmezett jelszó, frissítési mechanizmus, sebezhetőség-kezelés	Megfelelőségi ellenőrzés	Must

A táblázatban több célérték tudatosan konzervatív, mert a `native_sim` környezet nem helyettesíti a valós hardveres mérést. A valós hardverre vonatkozó energia- és késleltetés-adatokat a későbbi alfejezetekben vagy valós méréssel, vagy forrással jelölt becsléssel igazoljuk.

A kritikus riasztás folyamatát a következő ábra szemlélteti:



2.3. ábra: Kritikus riasztás döntési útvonala a ts-node-ban - A mintavétel 10 s-onként történik; a küszöbellenőrzés után csak az egyik ág fut le. Normál esetben a telemetria 10 percenként megy ki, kritikus esetben azonnal.

A folyamat lépései: a ts-node 10 másodpercenként felébred, kiolvassa a szenzorokat, és a mért értékeket összehasonlítja a helyben tárolt küszöbértékekkel. Ha minden érték a normál tartományban van, az eszköz visszaalszik, és csak a 10 perces küldési periódus lejártakor küld telemetriát. Ha bármelyik érték átlépi a kritikus határt, a csomópont azonnal felépíti az MQTT-kapcsolatot, és elküldi a riasztási telemetriát. A két ág közül tehát mindig csak az egyik fut le; a normál ág ritka rádiós forgalmat, a kritikus ág azonnali értesítést ad. Ezzel a normál üzemben alacsony az energiafogyasztás, rendkívüli eseménykor viszont rövid a késleltetés. A döntési pontot és a két ágat a fenti ábra külön mutatja.

Továbbvitt eredmény. A TerraSense követelménytáblázatát a `~/iot-labor/i002/terrasense-requirements.md` fájlban rögzítettük. Ezt a táblázatot használja az A TerraSense mintarendszer architektúrája alfejezet az architekturális döntésekhez, az Energiamodell és az elemélettartam kiszámítása alfejezet az energiabüdzsé számításához, a Rádiós technológiák összevetése: hatótáv, energia, sávszélesség, költség alfejezet a rádióválasztáshoz, és a Fenyegetésmodellezés: támadási felület és STRIDE alfejezet a biztonsági követelmények lebontásához.

2.2.5 Ellenőrzés és hibakeresés

A nem funkcionális követelmények teljesülését csak akkor tudjuk igazolni, ha minden sornál előre meghatároztuk az ellenőrzés módját. Az ellenőrzési terv a következőket tartalmazza:

Követelmény	Ellenőrzési eszköz, módszer	Mikor
Élettartam	Energiabecslés a szelepmodell figyelembevételével, majd valós hardveres árammérés	Tervezéskor és prototípus-tesztkor
Költség	BOM-árajánlat, gyártási költségkalkuláció	Beszerzés előtt
Késleltetés	Integrációs teszt a mintavételi periódus legrosszabb fázisában, időbélyeg-napló	Minden kiadás előtt
Megbízhatóság	Egyedi eseményazonosítós egyeztetés broker- és backend-újraindítás után, üzenetszámláló, nyugták	Folyamatosan
Offline	Hálózatkiesés szimulálása, visszatöltés ellenőrzése	Integrációs tesztben
Biztonság	Fenyegetésmodell, TLS-validálás, tanúsítvány nélküli MQTT és hitelesítés nélküli REST elutasítása	Tervezéskor és auditkor

A hibakeresés során gyakori, hogy egy követelmény nem mérhető, vagy a teszt nem reprodukálható. Ilyenkor először a mérőszámot kell pontosítani, nem a tesztet erőltetni.

Tünet	Lehetséges ok	Javítás
Az eszköz eleme a tervezettnél hamarabb lemerül	A rádió túl sokat ad, az alvómód nem lép be	Energiaprofil mérése, alvási időzítők ellenőrzése
A riasztás késve érkezik	A küszöbellenőrzés nem helyben fut, vagy a mintavételi periódus túl hosszú	Helyi küszöbellenőrzés és 10 s mintavétel bevezetése
A biztonsági teszt elutasítja a kapcsolatot	A tanúsítvány lejárt, vagy a CA nem megbízható	Tanúsítvány-életciklus és CA-lánc ellenőrzése
Offline tesztben adatvesztés	A puffer mérete kicsi, vagy a visszatöltés sorrendje hibás	Pufferméret növelése, sorrend és idempotencia javítása

2.2.6 Tipikus hibák és félreértések

1. **A fejlesztő a funkcionális prototípus után foglalkozik csak az NFR-ekkel.** A TerraSense-wi-fi prototípus működik, de a gyártás előtt kiderül, hogy az energia- vagy biztonsági követelmény nem teljesíthető a választott hardverrel. Az NFR-eket a követelménytáblázatban már a tervezés elején rögzíteni kell, mert ezek határozzák meg a hardver- és protokollválasztást.
2. **Az energia- és a késleltetés-követelményt egymástól függetlenül kezelik.** Ha gyors riasztást akarunk, gyakori mintavételt és adatküldést választunk, és az elem gyorsan lemerül. A megoldás a helyi feldolgozás: az eszköz sűrűn mintavételez, de normál üzemben ritkán kommunikál, rendkívüli eseménykor viszont azonnal.

3. **A biztonságot utólag „hozzáteszik”.** Ha a tanúsítvány-kezelést, az mTLS-t és az OTA-frissítést nem tervezik be az induláskor, később drága átalakításra van szükség, vagy a termék sebezhető marad.

2.2.7 Feladatok

2.2.7.1 1. Fogalmi vagy döntési kérdés

Soroljon fel három, az IoT-ban gyakori nem funkcionális követelményt, és magyarázza el, miért fontosabbak ezek egy telephelyi szenzorhálózatban, mint egy asztali webalkalmazásban.

2.2.7.2 2. Számolási vagy tervezési feladat

Egy TerraSense ts-node normál üzemben 10 percenként küld egy 180 B méretű telemetriai üzenetet. Hány bájt adatot küld naponta? Hány KiB ez 1000-es alapú átváltással? Ha egy hónap 30 nap, mennyi a havi adatmennyiség node-onként?

2.2.7.3 3. Implementációs feladat a TerraSense-ben

A TerraSense követelménytáblázatához adjon egy új sort: „Firmware-frissítés időtartama”. A mérhető célérték legyen az, hogy egy ts-node OTA-frissítése sikeresen befejeződik 5 perc alatt, és meghibásodás esetén az előző firmware visszaáll. Határozza meg az ellenőrzés módját és a prioritást. Mentse el a táblázatot a `~/iot-labor/i002/terrasense-requirements.md` fájlba.

2.2.7.4 4. Hibakeresési vagy kiterjesztési feladat

A TerraSense üvegházában a termelő azt tapasztalja, hogy a riasztások gyakran 10 percet késnek. A rendszer jelenleg 10 percenként vesz mintát, és ugyanilyen ütemben küld minden adatot a felhőbe, ahol a backend ellenőrzi a küszöbértékeket. Javasoljon egy olyan módosítást, amely a fizikai eseménytől számítva 30 másodpercen belüli riasztást tesz lehetővé, és mutassa meg, hogyan tartható az energia-büdzsé. Indokolja, hogyan befolyásolja ez a ts-node, a ts-gw és a ts-backend közötti munkamegosztást.

2.2.8 Megoldások

2.2.8.1 1. Fogalmi vagy döntési kérdés

Három tipikus NFR:

- **Energia/élettartam:** egy webalkalmazás szerver folyamatosan hálózatról üzemel, egy szenzorcsomópont viszont elemről, ezért az energia minden döntést korlátoz.
- **Offline működés:** a webes kliens-szerver kapcsolat megszakadása legfeljebb kényelmetlenség, a telephelyen a hálózatkiesés adatvesztéshez vezet, ezért pufferelni kell.
- **Biztonság:** az IoT-eszközök fizikailag hozzáférhetők, és gyakran hosszú élettartamúak, ezért a tanúsítvány- és frissítéskezelés már a tervezéskor kritikus.

2.2.8.2 2. Számolási vagy tervezési feladat

Napi küldések száma: $24 \cdot 60/10 = 144$.
Napi adatmennyiség: $144 \cdot 180 = 25\,920$ B.
Ez kB-ban (1000-es osztóval): $25\,920/1000 = 25{,}92$ kB/nap.
Havi adatmennyiség 30 nappal: $30 \cdot 25\,920 = 777\,600$ B $= 777{,}6$ kB, azaz körülbelül 0,78 MB.

Ez az érték csak a telemetriai payload; a protokoll-fejlécek (MQTT, TLS, IP) további bájtokat adnak hozzá, ezért a tényleges forgalom nagyobb.

2.2.8.3 3. Implementációs feladat

Az új sor a táblázatban:

Kategória	Követelmény	Mérhető célérték	Ellenőrzés módja	Prioritás
Eszközmenedzsment	A ts-node OTA-frissítése sikeresen befejeződik 5 perc alatt, hiba esetén az előző firmware visszaáll	Frissítési idő < 5 perc, rollback automatikus	OTA-teszt szimulált frissítéssel és megszakított átvitellel	Should

A frissítési mechanizmust a későbbi OTA-frissítés: A/B képek, rollback, delta, fokozatos kiadás alfejezetben tervezzük meg.

2.2.8.4 4. Hibakeresési vagy kiterjesztési feladat

A megoldás az, hogy a küszöbellenőrzést a ts-node-ba helyezzük, és szétválasztjuk a periódusokat: az eszköz 10 másodpercenként vesz mintát, a normál telemetriát 10 percenként küldi, kritikus küszöbátlépéskor pedig azonnal üzenetet küld. A 30 másodperces követelmény így a fizikai eseménytől számítható: a legrosszabb esetben 10 s a mintavételi periódus, ehhez legfeljebb néhány másodperc helyi feldolgozás és hálózati út járul. A sűrűbb mintavétel növeli a helyi ébredések energiaigényét, ezért a megoldás csak akkor tartható, ha a szenzorolvasás és a helyi döntés alacsony energiájú, a rádiós küldés pedig továbbra is ritka; az energiamérleget a 7. fejezet számításával kell igazolni. A ts-gw változatlanul továbbítja az MQTT-üzeneteket, a ts-backend pedig a riasztási üzenetre azonnal értesítést generál. Ez a munkamegosztás kihasználja az eszköz helyi számítási kapacitását, és csökkenti a felhő felé irányuló forgalmat.

2.2.9 Rövid összefoglaló

A nem funkcionális követelmények (energia, élettartam, költség, darabszám, késleltetés, adatgyakoriság, sávszélesség, megbízhatóság, offline működés, biztonság, adatvédelem, szabályozás) határozzák meg, hogyan működik egy IoT-rendszer a valós fizikai világban. A tervezés során ezeket mérhető célértékekkel, ellenőrzési móddal és prioritással rögzítjük. A TerraSense követelménytáblázata a `~/iot-labor/i002/terrasense-requirements.md` fájlban található; fő döntései: 10 s mintavételi periódus, 10 perces küldési periódus, helyi küszöbészlelés kritikus riasztáshoz, legalább 12 hónapos elem-élettartam a bistabil szelep energiaigényével együtt, 200

B-nál kisebb CBOR-üzenet, 30 napos gördülő ablakban mért 99,9%-os üzenetérkezés, 7 nap gateway-puffer, a záróprofilban TLS 1.3/mTLS és ETSI EN 303 645/CRA-megfelelés. Ezt a táblázatot használja az A TerraSense mintarendszer architektúrája, az Energiamodell és az elemélettartam kiszámítása, a Rádiós technológiák összevetése és a Fenyegetésmodellezés.

2.2.10 Források és ellenőrzési hiányok

- NIST Cybersecurity for IoT Program – ebben a fejezetben csak a CIA-hármas és az általános biztonsági keretrendszer hivatkozása; konkrét NIST-ajánlást a fejezet nem állít. A hivatkozott kiadás rögzítése a kiadás előtti, blokkoló szerkesztői feladat (hatókör: a 2.2 biztonsági bekezdése).
- ETSI EN 303 645 – a fejezet a gyári jelszó tilalmát, a frissítési kötelezettséget és a sebezhetőség-bejelentést idézi; a hivatkozott kiadás és a megfelelőségi kritériumok rögzítése a kiadás előtti, blokkoló szerkesztői feladat (hatókör: a 2.2 biztonsági bekezdése és a követelménytáblázat „Szabályozás” sora).
- EU Cyber Resilience Act – a fejezet nem állít hatálybalépési dátumot, alkalmazási kört vagy határidőt; ezek rögzítése a hivatalos jogszabályszöveg alapján a kiadás előtti, blokkoló szerkesztői feladat (hatókör: a 2.2 biztonsági bekezdése és a követelménytáblázat).
- GDPR (EU) 2016/679 – az adatvédelmi követelményeknél a személyes adat fogalmának konkrét értelmezéséhez jogi szakvélemény szükséges; a fejezet konkrét megfelelőséget nem állít.
- A TerraSense költségcélértéke illusztratív, a tényleges beszerzési árakat a tervezéskor piaci árajánlattal kell ellenőrizni; ez nem kiadást blokkoló tétel, mert a fejezet csak tervezési célt rögzít.

2.3 A TerraSense mintarendszer architektúrája

A TerraSense mintarendszer azért épül fel fokozatosan a könyv során, hogy ne „big bang” módon kelljen egyszerre megérteni az eszközt, a gatewayt és a felhőt. Ebben az alfejezetben rögzítjük a komponensek nevét, felelősségét, a köztük lévő interfészeket és a laborok térképét. Ez a térkép stabil hivatkozási alap lesz: amikor egy későbbi alfejezetben a `ts-node`, a `ts-gw` vagy a `ts-backend` neve előkerül, mindig erre az architektúrára hivatkozunk.

2.3.1 Motiváció: egy üvegház, három világ határán

Tegyük fel, hogy egy üvegházban (`site=gh1`) több szenzorcsomópontot helyezünk el. Az egyik csomópont (`node_id=n001`) hőmérsékletet, páratartalmat és talajnedvességet mér, valamint egy öntözőszelepet vezérel. A csomópont akkumulátorról működik, ezért nem akarunk rá teljes Linuxot tenni, és nem akarjuk közvetlenül az internetre kötni. Ugyanakkor az üzemeltetőnek egy böngészős felületen látnia kell az adatokat, és parancsot kell tudnia küldeni a szelep nyitására.

Ebből a hétköznapi helyzetből három, egymástól markánsan eltérő szoftverkörnyezet rajzolódik ki:

1. **Eszköz:** erőforráskorlátos mikrovezérlő, valós idejű közelség, energiatakarékosság.
2. **Edge/gateway:** Linuxos gép, amely a helyi rádiós világot és az internetes protokollvilágot köti össze.
3. **Felhő/backend:** nagy tárolókapacitás, REST API, adatbázis, riasztás, többfelhasználós hozzáférés.

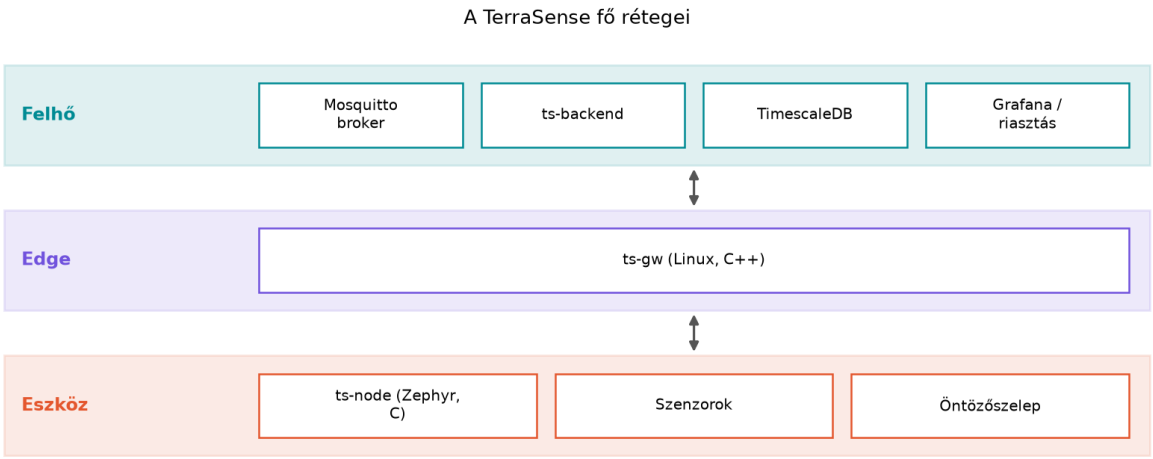
A TerraSense pontosan ezt a hármas határt teszi taníthatóvá. A cél nem egy „okos kütyü” bemutatása, hanem egy olyan referenciarendszer, amelyben minden réteg kódja, konfigurációja és hibakezelése a saját fejezetében tárgyalható, miközben a rétegek együtt is működnek.

2.3.2 Rendszerdefiníció és rétegek

Definíció. A **TerraSense** egy üvegház-monitorozó és öntözésvezérlő IoT-mintarendszer, amelyben a `ts-node` nevű szenzorcsomópontok érzékelnek és beavatkoznak, a `ts-gw` nevű gateway protokollfordítást és pufferezést végez, a `Mosquitto` broker üzenetközvetítőként köti össze az eszközoldalt a backenddel, a `ts-backend` pedig tárolja, feldolgozza és REST API-n keresztül kiszolgálja az adatokat.

Az architektúra a klasszikus **eszköz–edge–felhő** rétegmodellt követi:

- **Eszközréteg:** `ts-node`, szenzorok, szelep.
- **Edge-réteg:** `ts-gw`, opcionálisan helyi szűrés, offline puffer.
- **Felhőréteg:** broker, backend, adatbázis, dashboard, riasztás.



2.4. ábra: A TerraSense fő rétegei - Szimulációban a BLE helyett UDP, a perifériák helyett emulált szenzorok lépnek be; a vastag határvonal az MQTT-alapú gerincet jelzi.

Az ábra fentről lefelé mutatja az adat útját a felhasználó felől az eszköz felé, illetve lentről felfelé a telemetria útját. A nyilak iránya szándékosan mindkét irányú kommunikációt jelez, mert egy IoT-rendszerben a megfigyelés mellett a vezérlés is elsőrendű követelmény.

2.3.3 A komponensek felelőssége és interfészei

Az alábbi táblázat a rendszer komponenseit, a könyv során használt technológiáját és a szomszédos komponensek felé mutatott interfészeit foglalja össze.

Komponens	Technológia	Fő felelősség	Bemeneti interfész	Kimeneti interfész
`ts-node`	Zephyr RTOS, C	mérés, helyi vezérlés, telemetria	emulált szenzor; parancs MQTT/UDP felől	telemetria; szelep állapot
`ts-gw`	Linux, C++17/20, CMake	BLE/UDP → MQTT protokollfordítás, offline puffer	BLE GATT vagy UDP	MQTT
`Mosquitto`	Eclipse Mosquitto, MQTT 5/3.1.1	topic-alapú üzenetközvetítés	MQTT publish/subscribe	MQTT publish/subscribe
`ts-backend`	Java 21, Spring Boot, Maven	ingest, REST API, eszközárnyék, parancsok	MQTT subscribe; REST	SQL; REST; MQTT parancs
`TimescaleDB`	PostgreSQL 16 + TimescaleDB	idősoros tárolás, lekérdezés, megőrzés	SQL	SQL
`Grafana`	Grafana	dashboard, riasztás	TimescaleDB	böngésző, értesítés

A `ts-node` az eszközréteg mintakomponense. Nem „általános Zephyr demó”, hanem a TerraSense egy konkrét csomópontja: három fizikai mennyiséget mér, egy digitális kimenetet vezérel, és a rendszer közös topic-konvencióját használja. A szenzorokat szimulációban emuláljuk, hogy a laborok hardver nélkül is futtathatók legyenek.

A `ts-gw` az edge-réteg. Feladata kettős: egyrészt a helyi rádiós protokollt (valós hardveren BLE GATT, szimulációban UDP) MQTT-vé fordítja, másrészt akkor is megőrzi az adatokat, ha a felhő felé átmenetileg nincs kapcsolat. Ezzel a gateway nem egyszerű vezeték, hanem a rendelkezésre állás és a hibatűrés egyik fontos komponense.

A `Mosquitto` a rendszer üzenetgerince. A broker nem végez üzleti logikát, hanem topicok alapján irányítja az üzeneteket. A TerraSense-ben a broker minden komponens számára az MQTT-csatlakozási pont; még akkor is ezt használjuk, ha a gyártói rendszerekben néha közvetlen REST vagy AMQP is szerepel.

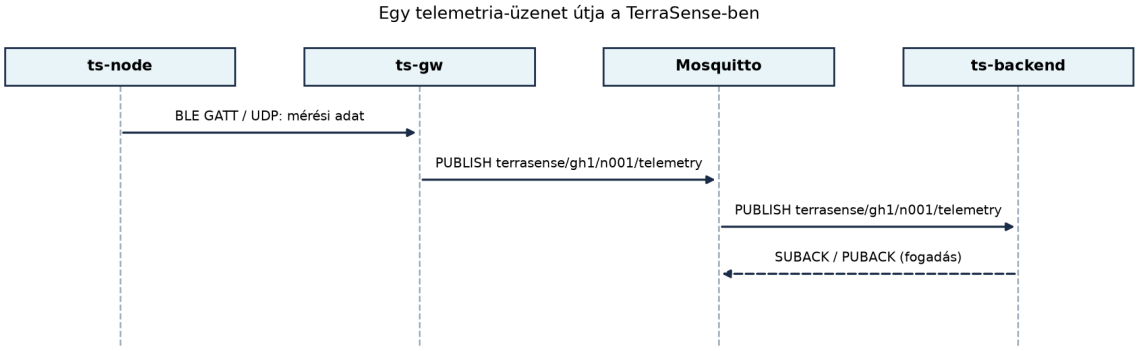
A `ts-backend` a felhőoldal alkalmazási belépési pontja. Ő iratkozik fel a telemetria-topicokra, validálja a bejövő adatokat, idempotensen tárol, és REST API-t kínál a megjelenítő rétegnek. A backend tartja nyilván az eszközárnyékot is, vagyis azt a digitális képet, amely a kívánt és a jelentett állapotot összeveti.

A `TimescaleDB` relációs adatbázisra épülő idősoros tároló. Azért választjuk, mert a TerraSense-adatok természetes kulcsa az eszköz és az idő, és a későbbi lekérdezések túlnyomórészt időablakokra, aggregátumokra és megőrzési szabályokra épülnek.

A `Grafana` a megjelenítő és riasztási réteg. Közvetlenül a TimescaleDB-t olvassa, így a dashboard nem a backend REST API-ján keresztül megy, ami a laborokban is jól mutatja a rétegek közötti munkamegosztást.

2.3.4 Kommunikációs minta: MQTT gerinc, REST a peremen

A TerraSense központi kommunikációs mintája a **publish/subscribe** MQTT-modell. Az eszközoldal üzenetei topicokra érkeznek, a backend feliratkozik ezekre, a parancsok pedig a másik irányban haladnak. A REST API csak ott jelenik meg, ahol a felhasználói vagy üzemeltetői kliens kérdez le adatot vagy küld parancsot a backendnek.



2.5. ábra: Egy telemetria-üzenet útja a TerraSense-ben - A ts-backend feliratkozott a telemetria-topicra. A diagram nem mutatja a TLS-t és a nyugtázást, mert azok a későbbi biztonsági és QoS-alfejezetekben kerülnek elő.

Az ábra szándékosan leegyszerűsített. A `ts-gw` a helyi kapcsolat felől érkező adatcsomagot MQTT-üzenetté alakítja, és a brokernek küldi. A broker az aktuális feliratkozások alapján továbbítja a `ts-backend` felé. A visszafelé haladó PUBACK itt csak a broker és a backend közötti nyugtázást jelzi; az MQTT QoS részleteit a MQTT QoS, munkamenetek, retain, Last Will és MQTT 5 alfejezet tárgyalja.

A vezérlési irány hasonló, de fordított: a backend parancsot publikál a `cmd` topicra, a `ts-gw` továbbítja a csomópontnak, a `ts-node` pedig az eredményt a `cmd/result` topicra teszi. A parancsok és a nyugtázás pontos szerződését a Parancsok és konfiguráció lefelé: kézbesítés, nyugtázás, időkorlát alfejezetben dolgozzuk ki.

A parancs azonosítója és a cél aktuátor két külön fogalom. A parancs `id` mezője a korrelációt és a deduplikációt szolgálja: minden parancs külön `id`-t kap, még akkor is, ha ugyanarra az aktuátorra irányul. A `type` a műveletet (például `open` vagy `close`), a `params` a paramétereket, az `expiry` a lejáratot, az `actuator` pedig a csomóponton belüli cél eszközt (például `valve1`) jelöli. A `node_id` a csomópontot címzi, az `actuator` ezen belül választ. Ugyanarra a szelepre két külön parancs két külön `id`-t kap, de az `actuator` mezőjük azonos. A Parancsok és konfiguráció lefelé: kézbesítés, nyugtázás, időkorlát alfejezet szerződése az `id`, `type`, `params` és `expiry` mezőket használja; az `actuator` mezőt a több aktuátoros kiterjesztésnél kell a szerződéshez hozzáadni, és soha nem helyettesítheti az `id`-t.

2.3.5 Azonosítók és topic-konvenció előzetesen

A TerraSense minden csomópontja két hierarchikus azonosítót kap:

- **`site`**: egy fizikai helyszín, például üvegház, `gh1`.
- **`node`**: az adott helyszínen belüli csomópont, például `n001`.

A teljes eszközazonosító a laborokban `gh1/n001`, de az üzenetekben az alábbi mezők mindig szerepelnek:

- `site`: a helyszín azonosítója, `"gh1"` formátumban;
- `node_id`: a csomópont azonosítója, `"n001"` formátumban;
- `boot_id`: a bootolási munkamenet egyedi azonosítója, például egy rövid véletlen hexaérték;
- `seq`: 32 bites, előjel nélküli számláló, amely minden bootoláskor 1-től indul, és a bootoláson belül szigorúan monoton nő;
- `ts`: az esemény időpontja UTC szerint, epoch ezredmásodpercben.

A `(site, node_id, boot_id, seq)` négyes a **kanonikus eseményazonosság**: két külön helyszín `n001` eszköze a `site` miatt nem ütközik, két újraindítás után keletkező `seq=1` eseményt pedig a `boot_id` különböztet meg. A `seq` önmagában nem ad globális sorrendet az újraindítások között, mert bootolásonként újraindul; a bootoláson belüli sorrendet a `seq`, a bootolások közötti sorrendet a `boot_id` és a `ts` együtt adja. Ha a `seq` a 32 bites tartomány végére ér, az eszköz új bootolási munkamenetet kezd (új `boot_id`-dal), nem csavarja vissza a számlálót. A `ts` az eseményidő, nem a fogadás ideje, ezért a késleltetett vagy visszatöltött üzenet is megtartja az eredeti időpontját.

A MQTT-topicok alapja a következő séma:

```
terrasense/<site>/<node>/telemetry
terrasense/<site>/<node>/state
terrasense/<site>/<node>/cmd
terrasense/<site>/<node>/cmd/result
terrasense/<site>/<node>/status
```

A `telemetry` a rendszeres mérési adatokat hordozza. A `state` a csomópont jelentett állapotát, például a szelep aktuális helyzetét. A `cmd` a backend felől érkező parancs, a `cmd/result` pedig a végrehajtás eredménye. A `status` retained üzenet és Last Will segítségével jelzi a csomópont jelenlétét.

Továbbvitt eredmény. Ez a komponensnév- és topic-konvenció alapozza meg a MQTT: publish/subscribe, broker, topicok, a A ts-gw szolgáltatás C++-ban: protokollfordítás MQTT-re, a Az IoT-backend felépítése: ingest, tárolás, API, alkalmazások és a Záróprojekt: a teljes TerraSense rendszer összeállítása és átvétele alfejezeteket. A későbbi alfejezetek ezt a sémát használják; ha módosításra lesz szükség, azt új alfejezetben indokoljuk.

2.3.6 Mi lesz szimulált, és mi opcionálisan valós?

A könyv elsődleges laborcélja Linux host. Ennek előnye, hogy a teljes rendszer felépíthető hardver nélkül, Docker-konténerekkel és a Zephyr `native_sim` futtatókörnyezetével.

Szimulált komponensek:

- a `ts-node` szenzorai: emulált hőmérséklet-, páratartalom- és talajnedvesség-forrás;
- a `ts-node` és a `ts-gw` közötti rádiókapcsolat: UDP-csomagok;
- a szelep fizikai visszajelzései: a `state` topicra írt emulált állapot.

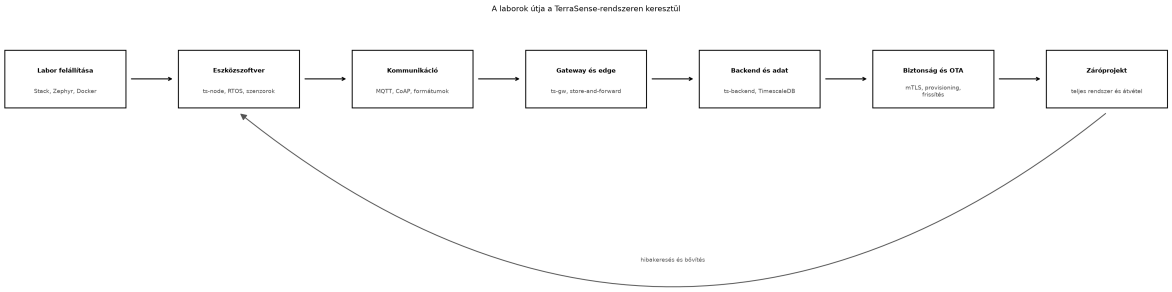
Opcionális valós hardveres kiterjesztés:

- fejlesztőkártya Zephyr-támogatással;
- I2C-szenzor a hőmérséklet és páratartalom mérésére;
- BLE-kapcsolat a `ts-node` és a `ts-gw` között;
- egy tényleges relé vagy szelepillesztő áramkör.

A `native_sim` időzítése és energiafelhasználása nem hardverhű. Ezért minden mérési vagy fogyasztási állítást a későbbi alfejezetekben kifejezetten illusztratívnak vagy forrásból származónak jelölünk.

2.3.7 A laborok térképe

A laborok a rendszer térképén haladnak végig, és minden rész egy-egy réteget vagy metszetet épít meg.



2.6. ábra: A laborok útja a TerraSense-rendszeren keresztül - A lépések a rétegek sorrendjét követik, de a visszacsatolások miatt a rendszer nem egyszerű lineáris vízesés.

A térkép nem azt jelenti, hogy minden labor pontosan egy komponenst érint. Például a MQTT a gyakorlatban: Zephyr-kliens és Mosquitto labor egyszerre használja a `ts-node` kliensét és a Mosquitto brokert. A záróprojektben pedig minden korábbi komponenst egyetlen futtatható rendszerbe integrálunk.

2.3.8 Fejlesztési és tesztelési felfogás

A TerraSense laborjaiban a komponenseket úgy kezeljük, mintha külön csapatok szállítanák őket, de közös szerződések mentén dolgoznának. A szerződés három pillére:

1. **topicok és üzenetirányok** – ki hova publikál, ki mire iratkozik fel;
2. **adatformátum** – a telemetria JSON-sémája, a mezők nevei és típusai;
3. **azonosítók és időkezelés** – `site`, `node_id`, `boot_id`, `seq`, `ts`, a kanonikus eseményazonosság `(site, node_id, boot_id, seq)` szerint.

Ezt a három pillért minden labor a `~/iot-labor/iNNN/` munkakönyvtárban úgy valósítja meg, hogy a későbbi alfejezet visszakereshető fájlnevekkel és döntésekkel dolgozhasson.

A rendszer nem „előre tökéletes”. A könyv szándékosan épít be valós fejlesztési helyzeteket: sémaverziót kell bevezetni, QoS-t kell választani, idempotenst kell készíteni az ingest elé, és a gatewaynek offline puffert kell biztosítania. Az architektúra célja nem az, hogy minden kérdésre azonnal választ adjon, hanem hogy a döntések következményeit egy működő rendszerben lehessen tesztelni.

2.3.9 Ellenőrzési pontok ebben a szakaszban

Az architektúra megértését a következőkkel ellenőrizhetjük:

1. A fenti rétegábrát eltakarva fel tudjuk sorolni a hat fő komponenst és mindegyikhez egy technológiát.
2. A topic-sémából kiindulva meg tudjuk mondani, hogy egy `gh1/n001` csomópont parancseredménye melyik topicra kerül.
3. A szekvenciadiagram alapján el tudjuk választani a telemetria útját a parancs útjától.
4. A labor-térképen be tudjuk jelölni, melyik későbbi alfejezet melyik komponenshez tartozik.

Tipikus hiba, hogy valaki az MQTT-topicot REST-végpontnak nézi, vagy a REST API-t közvetlenül az eszköz felé akarja kitenni. A TerraSense-ben a REST a backend és a felhasználói kliensek közötti interfész, az eszköz pedig MQTT-topicokon keresztül kommunikál. Az is gyakori félreértés, hogy a `seq` az eszköz újraindulása után is folytonos lesz. Valójában a `seq` minden bootoláskor 1-től indul, ezért önmagában nem folytonos; a bootolási munkameneteket a `boot_id` választja el, a tárolási idempotencia kulcsa pedig a `(site, node_id, boot_id, seq)` négyes.

2.3.10 Feladatok

1. **Fogalmi kérdés.** Miért nem MQTT-n keresztül kérdezi le a Grafana a múltbeli adatokat? Indokolja a komponensek felelősségével.
2. **Tervezési feladat.** Rajzolja fel a topic-fát a `gh1` üvegházban működő `n001` és `n002` csomópontokra. Milyen új topic kellene akkor, ha nemcsak szelepet, hanem ventilátort is vezérelnénk?
3. **Implementációs feladat.** Hozza létre a `~/iot-labor/i003/` munkakönyvtárat, és készítsen benne egy `terrasense_topics.md` fájlt, amely a fenti topic-konvenciót rögzíti.
4. **Hibakeresési/kiterjesztési feladat.** Módosítsa a konvenciót úgy, hogy egy csomópont telemetriájában a `site` is szerepeljen az üzenetben, ne csak a topicban. Milyen előnyt ad ez a broker nélküli hibanaplóban?

2.3.11 Megoldások

1. A Grafana számára a múltbeli adatok visszakeresésére az idősoros adatbázis való, nem a broker. Az MQTT-topic pillanatnyi üzeneteket közvetít; a broker alapból nem felelős a régmúlt adatok tartós lekérdezéséért. A TimescaleDB SQL-lel lekérdezhető, aggregálható és szűrhető; a Grafana ezt használja. Az MQTT a valós idejű adatáramláshoz jó, az SQL a tárolt múlt feldolgozásához.
2. A topic-fa:

```
terrasense/gh1/n001/telemetry
terrasense/gh1/n001/state
terrasense/gh1/n001/cmd
terrasense/gh1/n001/cmd/result
terrasense/gh1/n001/status

terrasense/gh1/n002/telemetry
terrasense/gh1/n002/state
terrasense/gh1/n002/cmd
terrasense/gh1/n002/cmd/result
terrasense/gh1/n002/status
```

Ha ventilátort is vezérelnénk, a `cmd` topic maradhat, de az üzenetben szereplő `actuator` mező egyértelműsítené a célpontot; a parancs azonosítója (`id`) továbbra is a korrelációt szolgálja, nem a cél kijelölését. Alternatíva lehet a `terrasense/<site>/<node>/cmd/<actuator>` altopic, de ezt a parancsok alfejezetében akkor vezetnénk be, ha valóban szét akarjuk választani a különböző aktuátorokat. A tananyag induló konvenciója szerint a `cmd` marad közös, az üzenetben hordozott `actuator` mező dönt a célról, az `id` pedig a parancspéldányt azonosítja.

3. A munkakönyvtár és a fájl létrehozása:

```
mkdir -p ~/iot-labor/i003 && cd ~/iot-labor/i003
```

A `terrasense_topics.md` tartalma:

```
# TerraSense topic-konvenció

terrasense/<site>/<node>/telemetry
terrasense/<site>/<node>/state
terrasense/<site>/<node>/cmd
terrasense/<site>/<node>/cmd/result
terrasense/<site>/<node>/status
```

4. Ha az üzenetben is szerepel a `site`, akkor a `ts-gw` vagy a backend naplója akkor is beazonosítja az adatot, ha a topic nem elérhető, vagy ha egy hibakeresési eszköz közvetlenül az üzenet-törzset rögzíti. Ez segít a hibás gateway-konfiguráció vagy a topic-átírás okozta problémák felderítésében, mert az üzenet és a topic keresztellenőrizhető.

2.3.12 Rövid összefoglaló

A TerraSense három fő rétegre tagolódik: a `ts-node` (Zephyr, C) az eszköz, a `ts-gw` (Linux, C++) az edge, a `Mosquitto`, `ts-backend` (Spring Boot), `TimescaleDB` és `Grafana` alkotja a felhőt. A rendszer MQTT-gerincre épül; az eszközoldal topicokra publikál, a backend feliratkozik, a parancsok pedig a `cmd` topicon haladnak lefelé. A topicok alapja `terrasense/<site>/<node>/...`, a telemetria, az állapot, a parancs, a parancs-eredmény és a jelenlét számára. A laborok Linux hoston futnak, `native_sim` és Docker segítségével; valós hardver opcionális. A továbbvitt eredmények a komponensnevek, az azonosítók és a topic-konvenció: ezeket használják a kommunikációs, gateway-, backend- és záróprojekt-alfejezetek. Az architektúra a későbbi döntések stabil kerete.

2.3.13 Források és ellenőrzési hiányok

Ez az alfejezet a könyv saját TerraSense-koncepcióját vezeti be, ezért nem épül külső webes forrásokra. A komponensek konkrét verzióit a Laborkörnyezet és a könyv verziómátrixa rögzíti; ez az alfejezet nem állít ettől eltérő verziót. A Zephyr `native_sim` hardverhűségének korlátját az Energiamodell és az elemélettartam kiszámítása alfejezet is jelzi. A biztonsági protokollok, a Mosquitto-verzió és a Spring Boot-verzió a verziómátrixban rögzítettek; itt csak architektúra-szintű elnevezéseket rögzítettünk.

3 Laborkörnyezet

3.1 A labor felállítása: Linux host, Docker, Zephyr, Mosquitto, Java

3.1.1 Motiváció

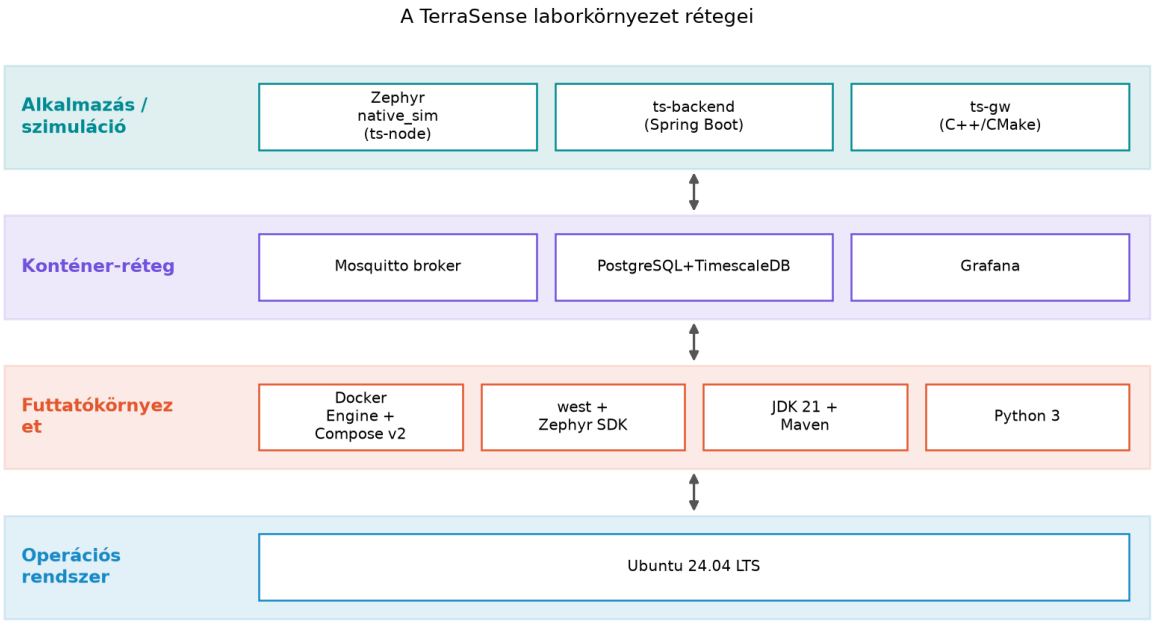
A TerraSense rendszer minden rétegét – eszköz, gateway, broker, backend, adatbázis, megjelenítés – olyan laborban fogjuk fejleszteni, amelyben nincs szükség valódi szenzorra vagy mikrovezérlőre. Ez két dolgot követel: a labor legyen reprodukálható, és az egyes eszközök helyettesíthetők legyenek konténerben vagy natív szimulációban futó megfelelőikkel. Ebben az alfejezetben felállítjuk a közös laborkörnyezetet Ubuntu 24.04-en, és minden komponenst egy-egy füsttesztel ellenőrzünk. Aki ezt egyszer végigcsinálja, a későbbi fejezetek laborjai azonnal futtathatók lesznek.

A fejezet „Várt kimenet” blokkjai illusztratívak: a tényleges kimenet a telepített verzióktól és a host állapotától függ. A futtatott ellenőrzéseket a labor végén a füsttesztek adják.

3.1.2 A laborkörnyezet felépítése

A labor négy fő pillére:

1. **Docker Engine + Compose v2** – a broker és a backend szolgáltatások konténeres futtatásához.
2. **Zephyr RTOS, west és Zephyr SDK** – a `ts-node` firmware-jének fejlesztéséhez és a `native_sim` boardon való szimulációjához.
3. **Mosquitto MQTT broker** – konténerben futó központi üzenetközvetítő.
4. **Java 21 + Maven + Python 3** – a backend fejlesztéséhez, illetve segédszkriptekhez.



3.1. ábra: A TerraSense laborkörnyezet rétegei - A host felett Dockerben futó szolgáltatások és a natívan futtatott Zephyr-szimuláció elkülönül.

A host gép egyetlen erős munkaállomás is lehet: a Zephyr `native_sim` a host CPU-ján, a konténerek pedig a Docker hálózatán futnak, így a teljes rendszer egyetlen számítógépen is tesztelhető.

3.1.3 Döntési szempontok

Miért Docker a brokerhez és a későbbi szolgáltatásokhoz? A Mosquitto, a TimescaleDB és a Grafana telepítése és verziókezelése natívan törékeny lenne. A Docker Compose deklaratív módon írja le a szolgáltatásokat, a verziók pedig a `compose.yaml`-ben rögzíthetők. A konténerek ráadásul izolálják a labor által használt portokat és köteteket. A takarítás nem egyetlen parancs: a `docker compose stop` leállítja a konténereket, de megtartja a köteteket; a `docker compose down` eltávolítja a konténereket és a hálózatot, a named volume-okat viszont megtartja; a `docker compose down -v` a named volume-okat is törli, a bind mountolt hostfájlokat viszont nem. A labor alapértelmezett leállítása a `stop`; a `down -v` csak biztonsági mentés után, teljes újrakezdéshez használható.

Parancs	Konténer	Hálózat	Named volume	Bind mount
`docker compose stop`	leállítja	megtartja	megtartja	megtartja
`docker compose down`	eltávolítja	eltávolítja	megtartja	megtartja
`docker compose down -v`	eltávolítja	eltávolítja	törli	megtartja (a hostfájlokat nem érinti)

Miért `native_sim` a Zephyr célnak? A Zephyr hivatalosan támogatja a natív szimulációt, amely a firmware-t a host processzeként futtatja. Ez nem hardverhű időzítést és energiafogyasztást ad, de a szálkezelés, a protokollkód és a logika tesztelésére kiváló. A valós hardveres kiegészítés opcionális, és az Opcionális valós hardver alfejezetben jelenik meg.

Miért Java 21 és Maven? A Spring Boot 3.x-hez Java 17 vagy újabb kell; a 21 hosszú távú támogatást élvez. A Maven pedig a Spring Boot projektek leggyakoribb build-eszköze, és a C++-os CMake-hez hasonló, deklaratív életciklust ad.

Miért Python 3? A laborban kisebb segédszkripteket (például adatszimuláció, üzenetgenerálás) Pythonban írunk, mert gyorsan fejleszthető és a JSON/SenML kezelése is egyszerű.

Globális elnevezési konvenciók

- A labor gyökere: `~/iot-labor/`.
- Minden alfejezet munkakönyvtára: `~/iot-labor/iNNN/`.
- A közös Docker Compose fájl: `~/iot-labor/stack/compose.yaml`.
- Az új laborobjektumok (fájl, konténer, topic) neve `ts-` vagy `lab-` előtagú.

Továbbvitt eredmény. Az itt létrehozott `~/iot-labor/` könyvtárszerkezetet és a `stack/compose.yaml` vázat használja az Miért RTOS? A Zephyr felépítése és az első alkalmazás, az MQTT a gyakorlatban: Zephyr-kliens és Mosquitto, a A ts-gw szolgáltatás C++-ban: protokollfordítás MQTT-re, a A ts-backend váza: REST API és egészségellenőrzés és a Záróprojekt: a teljes TerraSense rendszer összeállítása és átvétele alfejezet.

3.1.4 Labor: a környezet telepítése és füsttesztje

Munkakönyvtár:

```
mkdir -p ~/iot-labor/i004 && cd ~/iot-labor/i004
```

3.1.4.1 1. Alapcsomagok és git ellenőrzése

Ubuntu 24.04-en a `git` általában már telepítve van, de a laborkörnyezet miatt frissítjük a csomaglistát, és telepítjük az alapvető eszközöket.

```
sudo apt-get update
sudo apt-get install -y git curl wget ca-certificates gnupg xz-utils \
  build-essential gcc g++ make cmake ninja-build gperf \
  gcc-multilib g++-multilib libc6-dev-i386 file \
  device-tree-compiler python3 python3-pip python3-venv python3-dev
git --version
cmake --version
ninja --version
python3 --version
```

Várt kimenet: például `git version 2.43.0` és `Python 3.12.3` (a pontos verzió a telepített csomagoktól függ).

A `gcc-multilib`, a `g++-multilib` és a `libc6-dev-i386` a 32 bites `native_sim` buildhez szükséges; a `gnupg` a Docker repository kulcsának fogadásához, az `xz-utils` pedig a Zephyr SDK `.tar.xz` kicsomagolásához kell. A Zephyr Python-függőségeit nem a rendszer-Pythonba, hanem a 4. szakaszban létrehozott virtuális környezetbe telepítjük a `zephyr/scripts/requirements.txt` fájlból.

3.1.4.2 2. Docker Engine és Compose v2 telepítése

A labor támogatott hostja Ubuntu 24.04 x86_64 (amd64). A hivatalos Docker repository `noble stable` csatornáját használjuk; a `deb` sor `arch=amd64` értéke ezt a támogatott architektúrát rögzíti. ARM64 hoston a repository sora és a Zephyr SDK URL-je is eltér, ezt a kézirat nem támogatja. Először ellenőrizze a host architektúráját; a várt kimenet `amd64`:

```
dpkg --print-architecture
```

A kulcs és a repository hozzáadása után telepítjük a `docker-ce` csomagokat:

```
sudo install -m 0755 -d /etc/apt/keyrings
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | \
  sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg
echo \
  "deb [arch=amd64 signed-by=/etc/apt/keyrings/docker.gpg] \
  https://download.docker.com/linux/ubuntu noble stable" | \
  sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
sudo apt-get update
sudo apt-get install -y docker-ce docker-ce-cli containerd.io \
  docker-buildx-plugin docker-compose-plugin
```

A repository a verziómátrixban rögzített főverziót szolgálja; a telepítés után a `docker --version` és a `docker compose version` kimenetét a verziómátrixszal kell összevetni, és eltérés esetén a mátrixot kell frissíteni.

Ezután a felhasználót a `docker` csoportba tehetjük, hogy `sudo` nélkül is futtathassunk konténereket. A csoporttagság kényelmi, de nagy jogosultságú hozzáférés: a szokásos root daemonhoz hozzáférő `docker` csoport gyakorlatilag root szintű jogosultságot ad, ezért csak megbízható laborhoston használható. Biztonságosabb alternatíva a rootless Docker; a `sudo docker` nem biztonságosabb, mert a parancsot root joggal futtatja, és a konténer felhasználóját önmagában nem változtatja meg.

```
sudo usermod -aG docker $USER
newgrp docker
```

Ellenőrzés:

```
docker --version
docker compose version
```

Várt kimenet: a Docker Engine és a Compose v2 verziószáma, például `Docker version 27.x` és `Docker Compose version v2.x`.

> **Ellenőrzési igény.** Az Ubuntu 24.04 kódneve `noble`; ha a Docker repository URL-je megváltozik, akkor a fenti `deb` sor kiigazítása szükséges. A pontos Docker-verziók a telepítés időpontjától függenek.

3.1.4.3 3. A labor-könyvtárszerkezet és a közös Compose-váz létrehozása

Létrehozzuk a labor gyökérkönyvtárát, a közös `stack` mappát és a `scripts` segédkönyvtárat:

```
mkdir -p ~/iot-labor/stack ~/iot-labor/scripts
```

Most létrehozzuk a `~/iot-labor/stack/compose.yaml` vázat, amely egyelőre csak a Mosquitto broker szolgáltatást tartalmazza. Ez a fájl a későbbi alfejezetekben bővül a backend és az adatbázis szolgáltatásokkal.

```yaml
# ~/iot-labor/stack/compose.yaml
# Közös Docker Compose váz a TerraSense laborhoz.
# A későbbi alfejezetek további szolgáltatásokat adnak hozzá.

services:
  lab-broker:
    image: eclipse-mosquitto:2.0.18
    container_name: lab-broker
    ports:
      - "127.0.0.1:1883:1883"
    volumes:
      - ./mosquitto/mosquitto.conf:/mosquitto/config/mosquitto.conf:ro
    healthcheck:
      test: ["CMD", "mosquitto_sub", "-t", "$$SYS/#", "-C", "1", "-W", "1"]
      interval: 10s
      timeout: 3s
      retries: 5
```

A compose váz egy `lab-broker` nevű szolgáltatást definiál, amely az `eclipse-mosquitto:2.0.18` image-et használja, a 1883-as MQTT portot csak a `127.0.0.1` loopback interfészen publikálja, és a konfigurációs fájlt csak olvasható kötetként csatolja. A healthcheck a `$SYS/#` rendszertopicra iratkozik fel, és egyetlen üzenetet vár rövid időn belül. A konténer neve a `lab-` előtagot viseli.

A `127.0.0.1` kötés miatt a broker más gépről nem érhető el; ez a helyi labor alapértelmezése. Hálózati eléréshez külön, tudatos konfiguráció és hitelesítés szükséges, nem elegendő a portpublikálás megnyitása.

A rögzített címke mellett a reprodukálható kiadáshoz az image digestjét is meg kell adni. A digest a letöltött image-ből olvasható ki:

```
docker image inspect --format '{{index .RepoDigests 0}}' eclipse-mosquitto:2.0.18
```

A kapott `eclipse-mosquitto@sha256:...` értéket a compose `image` mezőjébe írva két külön időpontban végzett telepítés ugyanazt az image-et használja. A digest rögzítése a kiadás előtti lépés; a jelen kézirat nem tartalmaz digestet, mert a szerkesztés során nem történt image-letöltés.

A Mosquitto konfigurációs fájlhoz létrehozzuk a minimális beállítást. Az MQTT 5-ös laborokhoz majd itt lesz szükség a listener és a híd beállításaira.

```
mkdir -p ~/iot-labor/stack/mosquitto
cat > ~/iot-labor/stack/mosquitto/mosquitto.conf <<'EOF'
listener 1883
allow_anonymous true
EOF
```

> **Biztonsági megjegyzés.** Az `allow_anonymous true` csak a laborban, zárt hoston elfogadható; a Biztonságos kapcsolat: TLS 1.3, mTLS és DTLS az MQTT/CoAP alatt alfejezetben hitelesítést és TLS-t kapcsolunk be.

Indítsuk el a brokert, és ellenőrizzük a konténer állapotát:

```
cd ~/iot-labor/stack
docker compose up -d lab-broker
docker compose ps
```

Várt kimenet: a `lab-broker` konténer fut, a `STATUS` oszlopban `Up` (egészséges) állapottal, ha a healthcheck sikeres.

3.1.4.4 4. Zephyr: west, SDK és az első `native_sim` build

A Zephyr fejlesztéshez a `west` eszközre és a Zephyr SDK-ra van szükség. A `west` egy Python-alapú meta-eszköz, amely a Zephyr forrását, moduljait és a buildelési folyamatot kezeli.

Az Ubuntu 24.04 rendszer-Pythonja PEP 668 miatt nem engedi a `pip install --user` telepítést. A `west` ezért külön virtuális környezetbe kerül; ezt minden új terminálban aktiválni kell:

```
python3 -m venv ~/zephyr-venv
source ~/zephyr-venv/bin/activate
python -m pip install --upgrade pip
python -m pip install "west==1.2.0"
west --version
```

A west verzióját a `west==1.2.0` rögzíti; a `requirements.txt` a Zephyr saját Python-függőségeit sorolja fel, nem a west verzióját. A virtuális környezet aktiválását a későbbi Zephyr-laborok minden terminálablakában meg kell ismételni; a `source ~/zephyr-venv/bin/activate` sor a laborok „Munkakönyvtár:" blokkjában is szerepel, ahol Zephyr-parancs következik.

Ezután inicializáljuk a Zephyr workspace-t a `~/zephyrproject` könyvtárban. A `west init --mr` nélkül a manifest repository alapértelmezett ágát használná, ami fejlesztési főág is lehet; itt a `--mr v3.7.0` kifejezetten a 3.7.0 LTS kiadást rögzíti. A rögzített kiadást a manifest és a git revízió alapján ellenőrizzük.

```
west init -m https://github.com/zephyrproject-rtos/zephyr --mr v3.7.0 ~/zephyrproject
cd ~/zephyrproject
west update
python -m pip install -r zephyr/scripts/requirements.txt
west zephyr-export
west manifest --path
git -C zephyr rev-parse --short HEAD
git -C zephyr describe --tags --exact-match
```

A `west update` letölti a Zephyr fő forrását és a manifestben hivatkozott modulokat, ami több percet is igénybe vehet. A `git describe --tags --exact-match` a `v3.7.0` címkét adja, ha a rögzítés sikeres; a `west zephyr-export` a Zephyr CMake-csomagleírást a virtuális környezetbe exportálja, így a workspace-en kívüli saját alkalmazás is megtalálja a Zephyr build rendszerét. Az exportot ugyanabban az aktivált venvben kell futtatni, mint a buildet.

A Zephyr SDK a hivatalos GitHub releases oldalról tölthető le. A verziómátrixban rögzített 0.16.8 verziót használjuk, a letöltést pedig SHA-256 ellenőrzi:

```
mkdir -p ~/Downloads
cd ~/Downloads
wget -O zephyr-sdk-0.16.8_linux-x86_64.tar.xz \
 ↪  https://github.com/zephyrproject-rtos/sdk-ng/releases/download/v0.16.8/zephyr-sdk-0.16.8_linux-x86_64.tar.xz
printf '%s  %s\n' \
  'cb4e4012751e4526aaf1ec1e8ab9b4ded5681e2e01711b64f7a1b519ff7dbc6a' \
  'zephyr-sdk-0.16.8_linux-x86_64.tar.xz' | sha256sum -c -
tar xf zephyr-sdk-0.16.8_linux-x86_64.tar.xz -C "$HOME"
cd "$HOME/zephyr-sdk-0.16.8"
./setup.sh -t x86_64-zephyr-elf -h -c
test -d "$HOME/zephyr-sdk-0.16.8/x86_64-zephyr-elf"
```

A `sha256sum -c -` sornak `zephyr-sdk-0.16.8_linux-x86_64.tar.xz: OK` kimenetet kell adnia; ha nem, a letöltés sérült, és nem szabad telepíteni. A `tar -C "$HOME"` azért szükséges, mert a `setup.sh` és a `west` az SDK-t a `~/zephyr-sdk-0.16.8` könyvtárban keresi; a `~/Downloads`

csak a letöltés helye. A `setup.sh -t x86_64-zephyr-elf -h -c` a `native_sim`/x86_64 célhoz szükséges eszközláncot, a host-eszközöket és a CMake-csomagleírást telepíti. A `test -d` ellenőrzi, hogy az x86_64-es eszközlánc könyvtára létrejött. Ha az SDK máshová kerül, a `ZEPHYR_SDK_INSTALL_DIR` környezeti változóval kell megadni.

3.1.4.5 5. Hello world `native_sim` boardon

A füstteszthez a Zephyr beépített `hello_world` mintáját használjuk. A `-b native_sim` kapcsoló a natív szimulációs boardot választja, a `-d` pedig a build kimeneti könyvtárát adja meg.

```
cd ~/zephyrproject/zephyr
west build -b native_sim samples/hello_world -d ~/iot-labor/i004/build-hello
```

A build sikerét a következő kimenet jelzi:

```
...
[ 100%] Linking C executable zephyr.exe
...
```

Futtassuk a szimulációt:

```
~/iot-labor/i004/build-hello/zephyr/zephyr.exe
```

Várt kimenet:

```
*** Booting Zephyr OS build v3.7.0 ***
Hello World! native_sim
```

A `native_sim` a programot a host processzeként futtatja, ezért a kilépéshez `Ctrl+C` szükséges. A pontos verziószám a build kimenetben jelenik meg.

3.1.4.6 6. Mosquitto `mosquitto_pub`/`mosquitto_sub` próba

A brokert a compose már elindította. A füstteszthez a konténerbe lépve futtasson egy feliratkozót és egy publikálót. Nyisson egy második terminált, és futtassa a feliratkozót:

```
docker exec -it lab-broker mosquitto_sub -t 'terrasense/gh1/n001/telemetry'
```

Az első terminálban publikáljon egy tesztüzenetet:

```
docker exec -it lab-broker mosquitto_pub -t 'terrasense/gh1/n001/telemetry' \
  -m '{"node_id":"n001","ts":1717000000000,"temp":23.4}' -q 1
```

A feliratkozó terminálban megjelenik az üzenet:

```
{"node_id":"n001","ts":1717000000000,"temp":23.4}
```

A próba után a feliratkozót `Ctrl+C`-vel állíthatja le. A broker konténert később sem kell leállítani, mert a későbbi laborok is használják.

3.1.4.7 7. Java 21 és Maven telepítése

A Spring Boot 3.x-hez OpenJDK 21-et és Mavent telepítünk:

```
sudo apt-get install -y openjdk-21-jdk maven
java -version
mvn -version
```

Várt kimenet: `openjdk version "21.x"` és `Apache Maven 3.9.x`. A pontos verzió a Ubuntu csomagtárolótól függ.

3.1.4.8 8. Python 3 és a segédkönyvtár ellenőrzése

A Python 3 már telepítve van; a `pip`-et a labor segédszkriptjeihez fogja használni. Hozzon létre egy `lab_hello.py` fájlt az `i004` munkakönyvtárban, és futtassa:

```
# ~/iot-labor/i004/lab_hello.py
import json

telemetry = {"node_id": "n001", "seq": 1, "temp": 23.4}
print(json.dumps(telemetry))
```

```
python3 ~/iot-labor/i004/lab_hello.py
```

Várt kimenet:

```
{"node_id": "n001", "seq": 1, "temp": 23.4}
```

Ezzel minden laborpillér telepítve és füsttesztelve van.

3.1.5 Ellenőrzés és hibakeresés

A labor végén érdemes az összes komponenst egyetlen ellenőrző listával átnézni:

Komponens	Ellenőrző parancs	Várt eredmény
Docker	`docker --version && docker compose version`	verziószámok
Zephyr	`west --version`	west verziószám
Hello build	`~/iot-labor/i004/build-hello/zephyr/zephyr.exe`	Hello World!
Mosquitto	`docker compose -f ~/iot-labor/stack/compose.yaml ps`	`broker` fut
Java	`java -version`	OpenJDK 21
Maven	`mvn -version`	Maven 3.9.x
Python	`python3 --version`	Python 3.x

A tipikus hibatüneteket és javításukat az alábbi táblázat foglalja össze:

Tünet	Valószínű ok	Javítás
`docker` parancs: `permission denied`	A felhasználó nincs a `docker` csoportban	`sudo usermod -aG docker $USER`, majd új bejelentkezés
`west build` hibát ad: `unable to find toolchain`	A Zephyr SDK nincs telepítve vagy nem található	Futtassa a `setup.sh`-t, vagy ellenőrizze a `ZEPHYR_SDK_INSTALL_DIR` környezeti változót
`zephyr.exe` futás közben `No such file or directory`	Hiányzik a build kimeneti fájl vagy rossz a könyvtár	Ellenőrizze a `-d` kapcsolóval megadott build könyvtárat
Mosquitto nem indul: `Address already in use`	A 1883-as port már foglalt	`sudo ss -ltnp | grep 1883`, állítsa le a foglaló folyamatot, vagy módosítsa a compose portját
`docker compose up` hiba: `no configuration file provided`	Nem a `~/iot-labor/stack` könyvtárban futtatja	`cd ~/iot-labor/stack` vagy `-f` kapcsolóval adja meg a fájlt
`mosquitto_sub` nem kap üzenetet	A broker nem fut, vagy a topicok eltérnek	Ellenőrizze a konténer állapotát, és hogy a feliratkozás és a publikálás ugyanazt a topicot használja

3.1.6 Tipikus hibák és félreértések

1. **A `native_sim` nem valós idejű hardver.** Az időzítés és az energiafogyasztás a szimulációban nem reprezentálja a valós mikrovezérlőt. Például egy 1 másodperces alvás a `native_sim`-ben a host CPU-idejét használja, de nem ugyanazt a fogyasztást adja, mint egy valódi nRF52840. Minden mérésnél ezt a különbséget ki kell mondani.
2. **A Docker-portpublikálás nem azonos a konténer belső hálózatával.** A `127.0.0.1:1883` a hostról éri el a brokert; a konténeren belül viszont a `broker` szolgáltatásnév a `compose` hálózatán érhető el. A későbbi backend-laborokban a konténerből a brokerhez a `broker` hosztnéven kell csatlakozni, nem a `localhost`-on.
3. **A Compose v2-ben a `version` mező elavult.** A `compose.yaml` elején ne adjunk meg `version:`-t, mert a Compose v2 figyelmen kívül hagyja, és figyelmeztetést adhat. A vázban ezért nem szerepel.
4. **A `west` workspace és a build könyvtár összekeverése.** A `west build` parancsot a Zephyr forráskönyvtárból (`~/zephyrproject/zephyr`) kell futtatni, vagy a `west`-nek tudnia kell a workspace-t. Ha a build könyvtárat a labor mappába tesszük, akkor a `-d` kapcsolóval kell megadni, de a parancs futási helye a Zephyr forrás legyen.

3.1.7 Feladatok

3.1.7.1 (a) Fogalmi/döntési kérdés

Magyarázza el, miért érdemes a Mosquitto brokert Docker-konténerben futtatni a laborban, és miért nem a hostra telepített csomaggal dolgozunk. Térjen ki a verziókezelésre és a takarításra is.

3.1.7.2 (b) Tervezési feladat

Tervezze meg a `~/iot-labor/stack` könyvtár további struktúráját a későbbi szolgáltatásokhoz: PostgreSQL + TimescaleDB, Grafana, ts-backend. Hol tárolná az adatbázis-állományokat, a konfigurációs fájlokat és a logokat, hogy a laborok között ne vesszenek el, de takarításkor könnyen törölhetők legyenek?

3.1.7.3 (c) Implementációs feladat

Egészítse ki a `~/iot-labor/stack/compose.yaml` vázat egy `mqtt` nevű szolgáltatással, amely ugyanazt a Mosquitto image-et használja, de a nevét `lab-mqtt`-re változtatja, és a 1883-as portot a loopback interfészen `1884`-re képezi le, hogy a meglévő broker mellett fusson. Indítsa el, és tesztelje `mosquitto_pub`/`mosquitto_sub` parancsokkal a `terrasense/test` topicot.

3.1.7.4 (d) Hibakeresési feladat

Egy hallgató a labor futtatása közben a következő üzenetet kapja: `docker: permission denied while trying to connect to the Docker daemon socket`. Mi a hiba oka, és hogyan javítható? Adjon két lehetséges megoldást is, és mondja meg, melyik a biztonságosabb.

3.1.8 Megoldások

3.1.8.1 (a)

A Docker-konténer használata mellett szól, hogy a Mosquitto verzióját a `compose.yaml`-ben rögzíthetjük (például `eclipse-mosquitto:2.0.18`), így a labor minden résztvevője ugyanazt a brokerszoftvert futtatja, függetlenül a host disztribúciójától. A takarítás nem egyetlen parancs: a `docker compose stop` leállít, de megtartja a köteteket; a `docker compose down` eltávolítja a konténereket és a hálózatot, a named volume-okat viszont megtartja; a `docker compose down -v` a named volume-okat is törli, a bind mountolt hostfájlokat nem. A labor alapértelmezett leállítása a `stop`; a `down -v` csak biztonsági mentés után, teljes újrakezdéshez használható. Ha a brokert natívan telepítenénk, a verziók és a függőségek eltérhetnek, a teljes eltávolítás pedig bonyolultabb lenne.

3.1.8.2 (b)

A `stack` könyvtárban érdemes alkönyvtárakat létrehozni szolgáltatásonként: `postgres/` (init script-ek, adatkötet), `grafana/` (dashboards, provisioning), `ts-backend/` (Java jar vagy Dockerfile). Az adatbázis-állományokat named volume-ban tároljuk, mert azt a Compose jól kezeli: a `docker compose down` megtartja, és csak a `down -v` törli. A konfigurációs fájlokat a `stack/` forráskönyvtárban tartjuk, hogy gitben verziózhatók legyenek; a logokat host könyvtárba (például `~/iot-labor/stack/logs/`) irányítjuk bind mounttal, mert a `down -v` a bind mountolt fájlokat nem törli. A logok így megmaradnak a konténerek újraépítésekor is, és külön kell törölni őket.

3.1.8.3 (c)

A `compose.yaml` második szolgáltatásaként:

```
  mqtt:
    image: eclipse-mosquitto:2.0.18
    container_name: lab-mqtt
    ports:
      - "127.0.0.1:1884:1883"
    volumes:
      - ./mosquitto/mosquitto.conf:/mosquitto/config/mosquitto.conf:ro
```

Indítás:

```
cd ~/iot-labor/stack
docker compose up -d mqtt
```

Először a feliratkozót indítsa el az egyik terminálban:

```
docker exec -it lab-mqtt mosquitto_sub -t 'terrasense/test'
```

Ezután a másik terminálban publikáljon:

```
docker exec -it lab-mqtt mosquitto_pub -t 'terrasense/test' -m 'hello' -q 1
```

A `hello` üzenet a feliratkozó terminálban jelenik meg. Nem retained üzenetről van szó, ezért a feliratkozás előtt publikált üzenetet a broker nem tárolja; a sorrend ezért számít. Retained üzenet demonstrálásához `-r` kapcsolóval publikáljon, majd indítson új feliratkozót:

```
docker exec -it lab-mqtt mosquitto_pub -t 'terrasense/test/retained' -m 'hello retained' -q 1 -r
docker exec -it lab-mqtt mosquitto_sub -t 'terrasense/test/retained' -C 1
```

A teszt után a szolgáltatás leállítható: `docker compose stop mqtt`. A `stop` megtartja a köteteket; a `down` eltávolítja a konténert, de a named volume-okat szintén megtartja.

3.1.8.4 (d)

A hiba oka, hogy a felhasználó nem éri el a Docker daemon socketet (`/var/run/docker.sock`). Az első megoldás: `sudo usermod -aG docker $USER`, majd új bejelentkezés; ez kényelmes, de a `docker` csoport a daemonon keresztül gyakorlatilag root szintű jogosultságot ad. A második megoldás: a parancsokat `sudo docker ...` formában futtatni; ez sem biztonságosabb, mert a klienst root joggal indítja, és a konténer felhasználóját önmagában nem változtatja meg. A két lehetőség közül egyik sem ad nagyobb biztonságot; a valóban biztonságosabb út a rootless Docker használata. A laborban a csoporttagság elfogadható kényelmi megoldás megbízható hoston, de a jogosultsági kockázatot dokumentálni kell.

3.1.9 Rövid összefoglaló

Ebben az alfejezetben telepítettük a TerraSense labor teljes futtatókörnyezetét Ubuntu 24.04-en: Docker Engine-t Compose v2-vel, a Zephyr workspace-t `west`-tel és a Zephyr SDK-val, valamint a Java 21 + Maven és Python 3 eszközöket. Mindegyik komponenst füsttesztte ellenőriztük. Létrehoztuk a `~/iot-labor/` könyvtárszerkezetet, benne a `stack/compose.yaml` közös vázat, amely a `lab-broker` Mosquitto szolgáltatást definiálja, és a `stack/mosquitto/mosquitto.conf` minimalista konfigurációt. A `~/iot-labor/i004/build-hello/zephyr/zephyr.exe` hello world build a Zephyr `native_sim` célt igazolja. Ezt a struktúrát és compose-vázat a későbbi alfejezetek bővítik a ts-backend, a TimescaleDB és a Grafana szolgáltatásaival.

3.1.10 Források és ellenőrzési hiányok

- Zephyr dokumentáció: https://docs.zephyrproject.org/latest/ – a `west init` a `--mr v3.7.0` kapcsolóval a verziómátrixban rögzített 3.7.0 LTS kiadást használja; a hivatkozás általános, konkrét branch-ajánlást a fejezet nem tesz.
- Zephyr `native_sim` board: https://docs.zephyrproject.org/latest/boards/native/native_sim/doc/index.html – a `west build -b native_sim` parancs a Zephyr 3.7.0 részeként használja a boardot; a támogatás változása a verziómátrix frissítését igényli.
- Docker dokumentáció: https://docs.docker.com/ – a telepítés a `noble stable` csatornát és az `amd64` architektúrát használja; a `noble` kódnév és az architektúra a 3.1.4.2-ben rögzített. A repository URL változása a kiadás előtti, blokkoló szerkesztői feladat.
- Mosquitto dokumentáció: https://mosquitto.org/documentation/ – a compose az `eclipse-mosquitto:2.0.18` image-et és a healthchecket rögzíti; az image-digest rögzítése a kiadás előtti lépés.
- Java 21 és Maven: a verziómátrix OpenJDK 21.0.4 és Maven 3.9.9 verziót rögzít; az Ubuntu csomagtároló ettől eltérhet, ezért a telepítés után a verziómátrixhoz kell igazítani.
- Zephyr SDK letöltési URL és verzió: a 3.1.4.4 a 0.16.8 verziót, a hivatalos URL-t és a SHA-256 ellenőrzőösszeget rögzíti; a letöltés ellenőrzőösszeg nélkül nem telepíthető.

3.2 Opcionális valós hardver: fejlesztőkártyák és szenzorok

A `native_sim` környezetben a TerraSense szenzorcsomópontja hardver nélkül, meghatározott időzítésekkel és emulált szenzoradatokkal fut. Ez kiválóan alkalmas az alkalmazáslogika fejlesztésére, de nem ad választ arra, hogyan viselkedik a kód egy valódi I2C buszon, egy valódi rádióval, vagy hogyan fogyaszt egy valódi akkumulátoros kártya. Akkor van szükség valós hardverre, ha a cél az eszközszoftver és a periféria közötti integráció, a rádiós kommunikáció vagy az energiamérés tesztelése. Ebben az alfejezetben kiválasztunk egy Zephyr által támogatott fejlesztőkártyát és egy I2C szenzort, majd felépítjük azt a minimális laborkiterjesztést, amelyre a későbbi szenzorolvasási, energiamérési és OTA-frissítési laborok építenek.

3.2.1 Fogalmak és működés

Definíció. A **fejlesztőkártya** (development board, devboard) olyan nyomtatott áramköri lap, amely egy mikrovezérlőt vagy SoC-t, valamint a programozáshoz, tápláláshoz, hibakereséshez és perifériák csatlakoztatásához szükséges kiegészítő áramköröket (USB–UART híd, debugger, feszültségátalakító, LED-ek, csatlakozók) tartalmaz. A Zephyr kontextusában a **board-támogatás**

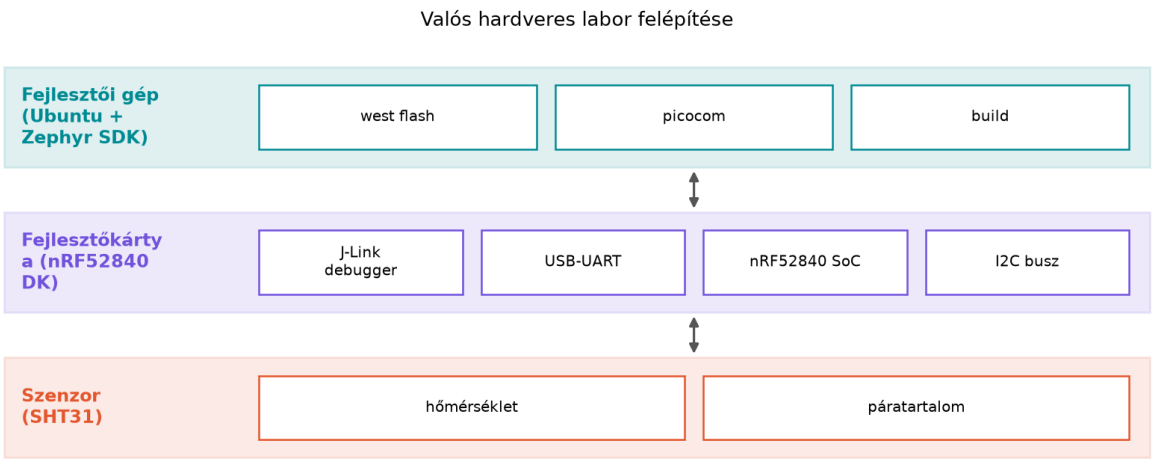
azt jelenti, hogy a Zephyr rendelkezik az adott kártyához tartozó devicetree-leírással, konfigurációs alapértelmezésekkel és build-célokkal, így az alkalmazáskód nagyrészt board-független maradhat.

A labor valós hardveres kiterjesztéséhez két komponenst választunk: egy **nRF52840 DK** fejlesztőkártyát és egy **SHT31** típusú I2C hőmérséklet/páratartalom-szenzort. Az nRF52840 egy Nordic Semiconductor gyártotta multimódusú SoC, amely Bluetooth 5-öt, 802.15.4-et (Thread/Zigbee alapsávot) és ARM Cortex-M4 magot tartalmaz. A DK (Development Kit) változat integrált J-Link debuggerrel, virtuális soros porttal és Arduino-kompatibilis lábkiosztással érkezik, ezért a Zephyr egyik leggyakrabban használt célkártyája. A board-név a Zephyr verziójához kötött: a könyvben rögzített Zephyr 3.7.0 LTS-ben a kanonikus board-cél `nrf52840dk/nrf52840` (hardvermodell v2), a régi `nrf52840dk_nrf52840` név elavult, de a 3.7.0 még elfogadja. A build előtt mindig ellenőrizzük a `west boards` kimenetében, hogy a telepített kiadás melyik nevet ismeri.

Az SHT31 az SHT3x szenzorcsalád tagja; az I2C buszon a 7 bites címe 0x44, de a 0x45-ös változat is létezik (az ADDR láb bekötése dönti el). A szenzor a hőmérsékletet és a relatív páratartalmat digitális formában, 16 bites felbontással adja vissza. A Zephyr `sht3xd` drivere közvetlenül támogatja, így a szenzor API-n keresztül olvashatjuk az értékeket anélkül, hogy a nyers I2C regiszterleírást ismernünk kellene.

A valós hardver és a `native_sim` közötti eltérések négy csoportba sorolhatók:

1. **Időzítés.** A szimulációban a `k_sleep()` és az időzítők virtuális időben futnak, nem feltétlenül felelnek meg a valós hardver órajelének és késleltetéseinek. Egy I2C-tranzakció időtartama a szimulátorban becsült, míg a valós hardveren a buszsebesség (tipikusan 100 vagy 400 kHz) és a slave válaszideje határozza meg.
2. **Rádió.** A `native_sim` nem modellezi a rádiós fizikai réteget. A Bluetooth Low Energy, a 802.15.4 vagy a LoRaWAN stackek egyes részei futtathatók szimulált környezetben, de a csomagvesztés, az interferencia és a hatótáv csak valós környezetben vizsgálható.
3. **Energiafogyasztás.** A szimulátor nem méri a mikrovezérlő és a perifériák tényleges áramfelvételét. Az alvómódok és a fogyasztáscsökkentés hatékonysága csak hardveren, árammérővel vagy a kártya energiaprofilozó funkcióival ellenőrizhető.
4. **Konkurens hardver-hozzáférés és hibák.** A valós hardveren megjelenhetnek olyan jelenségek, mint az I2C busz húzása (stuck bus), a szenzor nem válaszol (NACK), vagy a megszakítások prioritási versenye, amelyek a szimulátorban nem reprodukálhatók pontosan.



3.2. ábra: Valós hardveres labor felépítése - A fejlesztői gép USB-n keresztül flasheli a fejlesztőkártyát, és soros konzolon olvassa a kimenetet. A szenzor I2C-n csatlakozik.

A diagram felülről lefelé mutatja a rétegeket. A fejlesztői gép a `west flash` paranccsal tölti fel a firmware-t a kártyára a J-Link debuggeren keresztül, és `picocom`-mal csatlakozik a virtuális soros porthoz. A kártya az I2C buszon kommunikál a szenzorral, amely a mért értékeket digitálisan adja vissza. Ez a kapcsolat a TerraSense `ts-node` valós hardveres megfelelője, amely később a telemetriai adatokat küldi tovább.

3.2.2 Döntési szempontok

A fejlesztőkártya kiválasztásakor a következő szempontokat mérlegeljük:

Szempont	nRF52840 DK	ESP32-DevKitC	STM32 Nucleo (pl. L4)
Zephyr board-támogatás	kiváló, rendszeresen tesztelt	jó, de a WiFi stack Zephyr alatt korlátozott	kiváló, főleg a hivatalos támogatás
Beépített rádió	BLE 5, 802.15.4	WiFi, Bluetooth	nincs (külső modul kell)
Debugger	integrált J-Link	nincs (USB-UART csak)	integrált ST-Link
Energiamérési lehetőség	kiépített mérőpontok	korlátozott	fajtától függ
Ár (irányadó, 2024)	közepes	alacsony	alacsony–közepes
Szenzorcsatlakozás	I2C, SPI, GPIO, Qwiic/Stemma	I2C, SPI, GPIO	I2C, SPI, GPIO, Arduino fejléc

A TerraSense tananyaghoz az **nRF52840 DK** az ajánlott választás, mert a későbbi BLE- és Thread-laborok ezen valós hardveren is futtathatók, a debugger támogatja a gyors fejlesztést, és az energiamérésekhez szükséges mérőpontok is elérhetők. Az ESP32 azoknak előnyös, akik WiFi-alapú eszközt szeretnének, de a Zephyr alatti WiFi-támogatás és a fogyasztásmérés kevésbé

kényelmes. Az STM32 Nucleo olcsó belépő, viszont nincs beépített rádiója, ezért a kommunikációs laborokhoz kiegészítő modul kell.

A szenzorválasztásban az SHT31 mellett a BME280 is gyakori. A BME280 a hőmérséklet és páratartalom mellett légnyomást is mér, de az I2C címe (0x76 vagy 0x77) és a driver beállítása kissé eltér. A TerraSense üvegház-monitorozásához a hőmérséklet és a páratartalom elegendő, ezért az SHT31 egyszerűbb, és a Zephyr `sht3xd` drivere stabil. A végleges típus kiválasztását a beszerzési lehetőség és a foglalat befolyásolja; a labor kódjában a devicetree overlayben kell megadni a ténylegesen használt modellt.

3.2.3 Kidolgozott példa és labor

Munkakönyvtár: hozzuk létre a `~/iot-labor/i005` könyvtárat, és abban indítsuk el a projektet. Az előfeltétel a Labor felállítása alfejezetben telepített Zephyr SDK és `west`.

```
mkdir -p ~/iot-labor/i005 && cd ~/iot-labor/i005
```

Hozzuk létre a projektvázat a `ts-node-hardware` mappában. Ez nem a végleges `ts-node` alkalmazás, hanem egy minimális szenzorolvasó minta, amelyet a későbbi laborok bővítenek. A projekt a következő fájlokból áll:

`CMakeLists.txt`:

```
cmake_minimum_required(VERSION 3.20)
find_package(Zephyr REQUIRED HINTS $ENV{ZEPHYR_BASE})
project(ts_node_hw)
target_sources(app PRIVATE src/main.c)
```

`prj.conf`:

```
CONFIG_I2C=y
CONFIG_SENSOR=y
CONFIG_SHT3XD=y
CONFIG_PRINTK=y
```

`boards/nrf52840dk_nrf52840.overlay` (a `nrf52840dk/nrf52840` board-célhoz tartozó overlay; a fájlnévben a board-cél `/` karaktere `_`-ra képeződik):

```
/ {
    aliases {
        sensor0 = &sht31;
    };
};

&i2c0 {
    status = "okay";
    clock-frequency = <100000>;

    sht31: sht31@44 {
        compatible = "sensirion,sht3xd";
        reg = <0x44>;
    };
};
```

Megjegyzés: az `&i2c0` címke a kártya devicetree-jéből származik; ha az `i2c1` busz van kivezetve a szenzor-csatlakozóra, akkor azt kell használni. Ezt mindig ellenőrizzük a Zephyr board dokumentációjában vagy a kártya kapcsolási rajzán.

Bekötés: nRF52840 DK (PCA10056) és SHT31 modul. A laborban használt kártya a hivatalos nRF52840 DK (PCA10056), amelyet a `nrf52840dk/nrf52840` board-cél ír le. Az alábbi táblázat a DK Arduino-kompatibilis fejlécének I2C-kivezetéseit és egy tipikus SHT31 breakout (például Adafruit SHT31-D) jeleit köti össze. A DK digitális lábai 3,3 V-os logikai szintűek, ezért a szenzort is 3,3 V-ról tápláljuk, és a felhúzók is 3,3 V-ra mutassanak.

SHT31 modul	nRF52840 DK	Megjegyzés
VIN	3V3	3,3 V; a DK 3V3 kimenete a szenzor kis áramfelvételéhez elegendő
GND	GND	közös föld a tápláláshoz és a jelvezetékekhez
SDA	P0.26 (Arduino SDA)	az `i2c0` alapértelmezett SDA-lába a DK board-leírásában
SCL	P0.27 (Arduino SCL)	az `i2c0` alapértelmezett SCL-lába a DK board-leírásában
ADDR	GND	7 bites cím 0x44; 3V3-ra kötve 0x45
RST	szabadon hagyva	aktív alacsony reset; a modul adatlapján ellenőrizzük a belső felhúzót

A DK board devicetree-je az `i2c0` csomóponthoz `pinctrl-0 = <&i2c0_default>` bejegyzést rendel, amely P0.26-ot SDA-nak, P0.27-et SCL-nek állítja be. Ezért a fenti bekötéshez az overlayben nem kell új pinctrl-csoportot felvenni: a board leírása és a felülírt `&i2c0` csomópont együtt adja a végleges pinmuxot. Ha más lábpárra kötünk, a `&pinctrl` csomópontban új csoportot kell definiálni, és az `i2c0` `pinctrl-0` tulajdonságában arra hivatkozni; a pontos `NRF_PSEL` szintaxist a Zephyr 3.7.0 board-dokumentációja szerint ellenőrizzük.

Az I2C busz open-drain felépítésű, ezért az SDA és SCL vonalnak felhúzó ellenállásra van szüksége. A legtöbb SHT31 breakout tartalmaz 4,7–10 kΩ-os felhúzót; ezt a modul kapcsolási rajzán ellenőrizzük. Ha a modulon nincs felhúzó, a DK 3,3 V-os kimenete és az SDA/SCL vonal közé 4,7 kΩ-os ellenállást kell kötni. A felhúzókat soha ne 5 V-ra kössük, mert a DK GPIO-bemenetei nem 5 V-tűrők. A kártyarevízió és a szenzormodul típusa a lábkiosztást és a felhúzókat befolyásolhatja, ezért a bekötés előtt a kártya és a modul kapcsolási rajzát is ellenőrizzük.

`src/main.c`:

```
#include <zephyr/kernel.h>
#include <zephyr/device.h>
#include <zephyr/drivers/sensor.h>

static const struct device *sensor = DEVICE_DT_GET(DT_ALIAS(sensor0));

/*
 * A sensor_value elojeles fixpontos ertek: val1 az egesz resz, val2 a
 * mikroresz. A 64 bites kozbulso ertek miatt a negativ ertekek is helyesen
 * jelennek meg, es nem hasznalunk lebegopontos printf-formatumot.
 */
static void print_sensor_value(const char *label,
                               const struct sensor_value *value,
```

```
                                const char *unit)
{
    int64_t micro = (int64_t)value->val1 * 1000000 + value->val2;
    int sign = (micro < 0) ? -1 : 1;
    uint64_t abs_micro = (uint64_t)(sign * micro);

    printk("%s: %s%u.%06u %s\n", label, sign < 0 ? "-" : "",
           (uint32_t)(abs_micro / 1000000),
           (uint32_t)(abs_micro % 1000000), unit);
}

void main(void)
{
    if (!device_is_ready(sensor)) {
        printk("Szenzor nem elerheto\n");
        return;
    }
    printk("SHT31 szenzor inicializalva\n");

    while (1) {
        struct sensor_value temp, hum;
        int rc;

        if (sensor_sample_fetch(sensor) != 0) {
            printk("Mintavetel hiba\n");
            k_sleep(K_SECONDS(5));
            continue;
        }

        rc = sensor_channel_get(sensor, SENSOR_CHAN_AMBIENT_TEMP, &temp);
        if (rc != 0) {
            printk("Homerseklet csatorna hiba: %d\n", rc);
            k_sleep(K_SECONDS(5));
            continue;
        }

        rc = sensor_channel_get(sensor, SENSOR_CHAN_HUMIDITY, &hum);
        if (rc != 0) {
            printk("Paratartalom csatorna hiba: %d\n", rc);
            k_sleep(K_SECONDS(5));
            continue;
        }

        print_sensor_value("Homerseklet", &temp, "C");
        print_sensor_value("Paratartalom", &hum, "%RH");
        k_sleep(K_SECONDS(5));
    }
}
```

A kód a Zephyr `sensor` API-t használja. A `DEVICE_DT_GET(DT_ALIAS(sensor0))` makró a devicetree-ben definiált `sensor0` alias alapján kéri le az eszközleírót. A `device_is_ready()` ellenőrzi, hogy a driver betöltődött és a hardver elérhető. A ciklus öt másodpercenként kéri le a mintákat: a `sensor_sample_fetch()` elindítja az I2C-átvitelt, a `sensor_channel_get()` pedig a már beolvasott értékeket másolja a `sensor_value` struktúrába. A két `sensor_channel_get()` visszatérési értékét is ellenőrizzük: hibás vagy nem támogatott csatorna esetén nem formázunk értéket, hanem hibaüzenetet írunk, és kihagyjuk az iterációt. A `print_sensor_value()` a `val1`/`val2` párt előjeles fixpontos alakként jeleníti meg: a `val1` az egész rész, a `val2` a mikrorész, és negatív értéknél a két mező előjele azonos (az egész rész nulla lehet). A 64 bites közbülső értéknek köszönhetően a -0,5 °C és a -12,25 °C is helyesen, `-0.500000`, illetve `-12.250000` alakban jelenik meg. A `printk` alapértelmezett Zephyr-konfigurációban nem támogatja a `%f` formátumot, ezért szándékosan nem használunk lebegőpontos kiírást.

Build parancs (a `west` munkaterület gyökeréből vagy a projektkönyvtárból):

```
west build -b nrf52840dk/nrf52840 -d build/hw
```

Ha a board-cél nem található, először listázzuk:

```
west boards | grep nrf52840
```

Flashelés (a kártya USB-n csatlakoztatva, J-Link driverrel):

```
west flash -d build/hw
```

Soros konzol a kimenet megtekintéséhez (a `/dev/ttyACM0` a virtuális soros port, eltérhet):

```
picocom -b 115200 /dev/ttyACM0
```

Kilépés a picocom-ból: `Ctrl-A` majd `Ctrl-X`.

Várt kimenet (illusztratív, nem tényleges mérés):

```
SHT31 szenzor inicializalva
Homerseklet: 23.456789 C
Paratartalom: 47.123456 %RH
Homerseklet: 23.512345 C
Paratartalom: 46.987654 %RH
```

A konkrét értékek a környezettől és a szenzor kalibrációjától függnek. A `native_sim` esetén ez a labor nem futtatható, mert az I2C és a szenzor valós hardvert igényel; a szimulált szenzorokhoz a Szenzorolvasás a Zephyr sensor API-val alfejezet ad útmutatást.

3.2.4 Ellenőrzés és hibakeresés

A labor sikerességét a konzolon megjelenő ismétlődő mérési sorok jelzik. Ha a kimenet nem jelenik meg, a következő táblázat segít:

Tünet	Lehetséges ok	Javítás
`Szenzor nem elerheto`	Az I2C cím hibás, vagy a szenzor nincs bekötve	Ellenőrizzük a bekötést, a feszültségszinteket, és hogy az overlayben a `reg` értéke 0x44 (vagy 0x45)
Nincs semmilyen kimenet a konzolon	A soros port nem a `/dev/ttyACM0`, vagy a baudrate eltér	Ellenőrizzük `dmesg | tail` és a port nevét; próbáljunk más baudrate-et (pl. 9600, 115200)
`west flash` hibát ad	A J-Link driver hiányzik, vagy a kártya nincs csatlakoztatva	Telepítsük a J-Link Software-t (ellenőrizendő verzió), és csatlakoztassuk a kártyát

Tünet	Lehetséges ok	Javítás
A build sikeres, de a szenzor nem válaszol	Az `i2c0` helyett `i2c1` busz kell, vagy nincs engedélyezve a `CONFIG_I2C`	Nézzük meg a végleges devicetree-t a `build/zephyr/zephyr.dts` fájlban, és a generált `devicetree_generated.h` fejlécet: ott látszik a szenzor busza és `reg` értéke. A `CONFIG_I2C`-t a `west build -t menuconfig` alatt ellenőrizzük, majd szükség esetén módosítsuk az overlayt

Hasznos ellenőrzési lépés az I2C busz szkennelése egy külön kis programmal, amely mind a 127 címen megpróbál egy írási tranzakciót. A Zephyrben ez néhány sorból megvalósítható, de a laborhoz elegendő a fenti hibakeresési út.

3.2.5 Tipikus hibák és félreértések

1. **A szimulációban működő kód változatlanul átvihető a valós hardverre.** A `native_sim` nem modellezi az I2C busz valós időzítését és hibáit, ezért a szimulált úton sikeres kód hardveren NACK hibát vagy váratlan késleltetést okozhat. Mindig számítsunk a hardveres perifériák inicializálási és időzítési különbségeire.
2. **A `native_sim` energiaadatai valósnak tekinthetők.** A szimulátor nem méri a fogyasztást, ezért az energiamodellezéshez valós hardveren, megfelelő mérőeszközzel végzett mérés szükséges. A szimulátorban kapott értékek legfeljebb a szoftveres állapotváltások relatív gyakoriságát jelzik.
3. **A fejlesztőkártya tápfeszültsége mindig 3,3 V.** Sok kártya 5 V-os USB-ről táplálkozik, de a GPIO és az I2C lábak 3,3 V-os logikai szintűek lehetnek. Az 5 V-os szenzor csatlakoztatása károsíthatja a mikrovezérlőt, ezért mindig ellenőrizzük a feszültségszinteket.
4. **Az ESD-védelem nem fontos a laborban.** A szenzorok és a kártyák érzékenyek az elektrosztatikus kisülésre. A földelt csuklópánt és a vezető munkafelület használata megelőzi a statikus feltöltődésből eredő meghibásodásokat.

3.2.6 Feladatok

1. **Fogalmi/döntési kérdés.** Miért nem alkalmas a `native_sim` a rádiós kommunikáció és az energiafogyasztás hiteles tesztelésére? Fogalmazza meg két-két mondatban mindkét korlátot.
2. **Számolási/tervezési feladat.** Az SHT31 I2C buszon 400 kHz sebességgel kommunikál. Egy mérési tranzakció 6 bájtot ír, majd 6 bájtot olvas. Becsülje meg, mennyi időt vesz igénybe egyetlen mérés az adatátvitel szempontjából, ha a busz overheadjét (start/stop bitek, ACK) az adatbitek 20%-ára becsüljük. (Feltételezzük, hogy minden bájt 8 bit + 1 ACK bitet jelent, és a vezérlőjelek elhanyagolhatók.) Adja meg a képletet és a végeredményt mikroszekundumban.

3. **Implementációs feladat.** Módosítsa a labor devicetree overlayjét úgy, hogy a szenzor a 0x45 címen legyen elérhető, és az alias neve `sensor0` maradjon. Írja le a szükséges `dts` kódrészletet, és magyarázza el, miért változik a cím.

4. **Hibakeresési/kiterjesztési feladat.** A konzolon a `Mintavetel hiba` üzenet jelenik meg, és a szenzor nem ad értéket. Soroljon fel legalább három lehetséges okot, és mindegyikhez egy ellenőrzési lépést.

3.2.7 Megoldások

1. Fogalmi kérdés. A `native_sim` nem modellezi a rádió fizikai rétegét, ezért nem tudja szimulálni a csomagvesztést, az interferenciát és a hatótávtényezőket. Az energiafogyasztást sem tudja mérni, mert a szimulátor a virtuális időben fut, és nem rendelkezik a valós hardver áramfelvételének analóg modelljével. Az energiaprofil csak hardveren, árammérővel vagy a kártya beépített mérőpontjain keresztül határozható meg.

2. Számolási feladat. A tranzakció 12 bájtnyi adatot mozgat. Egy bájt 8 bit, mindegyikhez 1 ACK bit tartozik, így bájtonként 9 bit. Az overhead 20%, tehát az effektív bitek száma:

$$N_{\text{eff}} = 12 \times 9 \cdot 1{,}20 = 129{,}6\,\text{bit}$$

Az átviteli idő 400 kHz buszsebességnél:

$$t = \frac{N_{\text{eff}}}{f_{\text{I2C}}} = \frac{129{,}6}{400 \cdot 10^3} \approx 0{,}324\,\text{ms} = 324\,\mu\text{s}$$

Tehát egyetlen mérés adatátvitele körülbelül 324 mikroszekundumot vesz igénybe, a szenzor belső mérési ideje (amely tipikusan 10–20 ms) ehhez hozzáadódik. (Az érték becslés, a busz overheadje a konkrét protokolltól függ.)

3. Implementációs feladat. Az SHT31 I2C címe az ADDR láb bekötésétől függ. Ha az ADDR lábat alacsony szintre kötjük, a cím 0x44, ha magasra, akkor 0x45. A devicetree overlayben csak a `reg` értékét kell módosítani:

```
sht31: sht31@45 {
    compatible = "sensirion,sht3xd";
    reg = <0x45>;
};
```

Az alias nem változik, mert az továbbra is `sensor0`-ként hivatkozik a szenzorra. A cím megváltoztatása lehetővé teszi két azonos típusú szenzor egy buszra kötését is.

4. Hibakeresési feladat. Lehetséges okok és ellenőrzésük:

- **A szenzor nincs megfelelően táplálva.** Ellenőrizzük a tápfeszültséget (3,3 V) és a VDD/GND bekötést multiméterrel.
- **Az I2C busz címhibás.** Próbáljuk ki a másik címet (0x45), vagy olvassuk le az ADDR láb bekötését.

- **A devicetree overlayben nem a megfelelő I2C buszra hivatkozunk.** A build után nyissuk meg a `build/zephyr/zephyr.dts` fájlt: ez a végleges, összefűzött devicetree, benne az `sht31` csomópont szülője mutatja a buszt, a `reg` tulajdonság pedig a címet. A generált `build/zephyr/include/generated/zephyr/devicetree_generated.h` fejléc ugyanezt gépi formában tartalmazza (a pontos útvonal a Zephyr-verziótól függhet). A `west build -t menuconfig` nem a devicetree-t, hanem a Kconfig-beállításokat szerkeszti; azt csak a `CONFIG_I2C` és a `CONFIG_SHT3XD` ellenőrzésére használjuk.
- **A `CONFIG_I2C` vagy a `CONFIG_SHT3XD` hiányzik a `prj.conf`-ból.** Győződjünk meg róla, hogy a konfigurációban szerepelnek, és a build naplója nem jelzi a sensor driver kihagyását.
- **Az I2C busz húzva maradt (stuck bus).** Ha a szenzor korábban félbehagyott tranzakcióban maradt, a busz SDA vonala alacsonyan maradhat. Kapcsoljuk ki-be a kártyát, vagy programozottan küldjünk 9 órajelciklust a busz felszabadításához.

3.2.8 Rövid összefoglaló

Ez az alfejezet a valós hardveres laborkiterjesztés alapjait mutatta be. Kiválasztottuk az nRF52840 DK fejlesztőkártyát és az SHT31 I2C szenzort, megadtuk a szimuláció és a valós hardver közötti fő eltéréseket (időzítés, rádió, energia, hardverhibák), valamint a bekötési táblázatot és a 3,3 V-os feszültségszinteket. A laborban létrehoztuk a `~/iot-labor/i005/ts-node-hardware` projektet, amely a devicetree overlay (`boards/nrf52840dk_nrf52840.overlay`), a `prj.conf` és a `src/main.c` fájlokból áll; a build a Zephyr 3.7.0 LTS `nrf52840dk/nrf52840` board-céljára történik. A program öt másodpercenként kiolvassa a hőmérsékletet és páratartalmat, ellenőrzi a csatornák hibáit, és a soros konzolon előjeles fixpontos alakban jeleníti meg az értékeket. A projekt a későbbi Szenzorolvasás a Zephyr sensor API-val alfejezetben a valós szenzorok olvasásának, az Energiagazdálkodás fejezetben az energiaméréseknek, az OTA-frissítés alfejezetben pedig a távoli frissítéseknek az alapjául szolgál. A board-cél, a J-Link driver verziója és a szenzor pontos típusa a beszerzéskor ellenőrizendő.

3.2.9 Források és ellenőrzési hiányok

- A könyvben rögzített Zephyr 3.7.0 LTS board-célja `nrf52840dk/nrf52840`; a régi `nrf52840dk_nrf52840` név elavult, de a 3.7.0 még elfogadja. Az overlay fájlneve továbbra is `nrf52840dk_nrf52840.overlay`, mert a fájlnévben a board-cél `/` karaktere `_`-ra képeződik. A labor előtt ellenőrizzük `west boards` paranccsal.
- A nRF52840 DK (PCA10056) Arduino-fejlécének I2C-lábai (P0.26/P0.27) és a board `pinctrl` beállítása a Zephyr 3.7.0 board-dokumentációjában ellenőrizendő; a kártyarevízió és a szenzormodul eltérhet.
- Az SHT31 szenzor I2C címe az ADDR láb bekötésétől függ (0x44 vagy 0x45); az adatlapi megerősítés szükséges.
- A SHT31 breakout felhúzóinak megléte és értéke a modul kapcsolási rajzán ellenőrizendő; szükség esetén 4,7 kΩ-os külső felhúzó kell a 3,3 V-ra.
- A J-Link driver telepítési lépései és a Linux disztribúciós csomag neve a Nordic Semiconductor aktuális útmutatójában ellenőrizendő.
- A szenzor belső mérési ideje 10–20 ms között van, ez az adatlap alapján pontosítandó.
- Az nRF52840 DK energiamérési mérőpontjainak elrendezése a kártya dokumentációjában található.

II. RÉSZ – Az eszköz: mikrovezérlő, RTOS, szenzorok, energia

4 Mikrovezérlők és perifériák

4.1 Mikrovezérlő vs. Linuxos processzor: memória, OS nélküli világ

4.1.1 Motiváció

A TerraSense-rendszer eszközoldalán, a **ts-node** szenzorcsomóponton egy mikrovezérlő (MCU) fut, nem pedig a megszokott Linuxos processzor. Ha a ts-node-ot Zephyr RTOS-szel és `native_sim` céllal fejlesztjük, akkor is pontosan tudnunk kell, milyen erőforrás-korlátokkal kell számolnunk a valós hardveren. Az MCU-n nincs virtuális memória, nincs MMU, és gyakran nincs operációs rendszer sem abban az értelemben, ahogyan a Linuxot ismerjük. A memória néhány száz KiB, a flash pedig legfeljebb néhány MiB nagyságrendű. Ha ezt figyelmen kívül hagyjuk, a firmware nem fér el a flash-ben, vagy futás közben kifogy a RAM-ból. Ezért az alfejezet célja, hogy a Linuxos programozói szemléletet átfordítsuk a beágyazott, erőforrás-szűkös világba.

Továbbvitt eredmény. Az itt bemutatott memória-korlát-tudatosság alapozza meg a Zephyr-szálak veremméret-választását a Szálak és ütemezés alfejezetben, a robusztus, determinisztikus eszközszoftver-architektúra tervezését a Eszközszoftver-architektúra: rétegek és állapotgépek alfejezetben, valamint az OTA-frissítés flash-méretkorlátjainak kezelését a OTA-frissítés: A/B képek, rollback, delta, fokozatos kiadás alfejezetben. A laborban mért szekcióméretek és a map-fájl elemzése ezekben a későbbi fejezetekben válik napi rutinná.

4.1.2 Fogalmak és működés

Definíció (mikrovezérlő, MCU). A mikrovezérlő egyetlen integrált áramkör, amely processzormagot, flash programmemóriát, SRAM-ot, perifériákat (GPIO, UART, I2C, SPI, ADC, időzítők) és gyakran rádiós interfészt is tartalmaz. Célja, hogy kis fogyasztással, külső komponensek nélkül, közvetlenül a hardvert vezérelje.

Definíció (mikroprocesszor). A mikroprocesszor ezzel szemben általában csak a CPU-magot és gyorsítótárat tartalmazza, a memóriát (DDR SDRAM, flash) és a perifériákat külön áramkörök biztosítják. A mikroprocesszoros rendszerek – mint a Linuxot futtató SoC-k – rendszerint MMU-val rendelkeznek, és teljes értékű operációs rendszert futtatnak.

Definíció (MMU, Memory Management Unit). Az MMU a CPU azon egysége, amely a virtuális címeket fizikai címekre fordítja, memórialapokat kezel, és a fordítási táblákban rögzített jogosultságok révén hozzáférés-védelmet is adhat a folyamatok között. A legtöbb kisméretű MCU-n nincs MMU; nincs virtuális memória, minden kód és adat ugyanabban a fizikai címtérben osztozik.

Definíció (MPU, Memory Protection Unit). Az MPU nem végez címfordítást: a fizikai memóriát legfeljebb néhány régióra osztja, és régióként szabályozza az olvasási, írási és végrehajtási jogosultságot. Számos Cortex-M mag tartalmaz MPU-t, és a Zephyr userspace támogatása (`CONFIG_USERSPACE`) ilyen hardveres régióvédelemre épül a felhasználói és a kernel mód elkülönítéséhez. Az MPU hiánya tehát nem a memóriavédelem fogalmának hiányát jelenti, hanem azt, hogy az adott magon nincs hardveresen kikényszerített régióvédelem.

Definíció (memory-mapped I/O). A memory-mapped I/O olyan technika, amelyben a perifériák vezérlő- és állapotregiszterei meghatározott fizikai memóriacímeken érhetők el. A program úgy ír vagy olvas egy regisztert, mintha memóriát írna vagy olvasna, például egy C pointerrel: `*(volatile uint32_t *)0x40000000 = 1;`.

Definíció (reset vektor). A reset vektor a memória egy rögzített címe (gyakran a flash elején), ahonnan a processzor bekapcsolás vagy reset után a kezdeti végrehajtási állapotot betölti. A jelentése architektúrafüggő: a tárgyalt ARM Cortex-M magon a flash elején álló vektortábla első bejegyzése a kezdeti veremmutató (MSP), a második a resetkezelő rutin címe; a mag ezt a két értéket tölti be, nem feltétlenül egy ugrásutasítást hajt végre. Más architektúrákon a reset vektor gyakran valóban egy ugrás a firmware belépési pontjára.

Definíció (BSS szekció). A BSS (Block Started by Symbol) a RAM azon része, amelyet a globális és statikus, explicit kezdőérték nélküli vagy nullával inicializált változók foglalnak el. A C nyelvi garancia szerint ezek a változók induláskor nullák; a C futtatókörnyezet a boot során a `.bss` területet nullázza ki.

Az MCU és a Linuxos SoC erőforrás-különbségeit az alábbi táblázat foglalja össze:

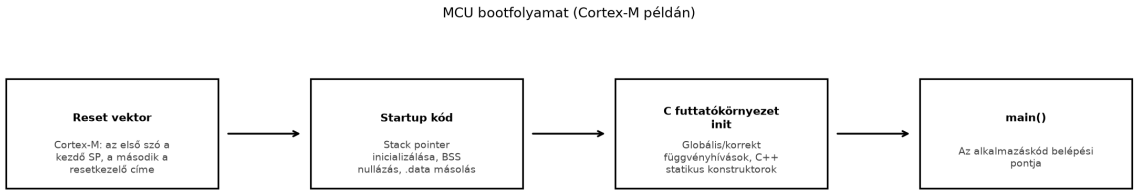
Jellemző	MCU (pl. ARM Cortex-M4)	Linuxos SoC (pl. Cortex-A72)
Programmemória	Flash, 64 KiB – 2 MiB	eMMC/SD, több GiB
Adatmemória (RAM)	SRAM, 16 – 512 KiB	DDR SDRAM, 256 MiB – 16 GiB
MMU / virtuális memória	nincs; a legtöbb Cortex-M-en külön MPU adhat régióvédelmet	van
Operációs rendszer	bare-metal vagy RTOS (Zephyr, FreeRTOS)	teljes Linux kernel
Folyamatok / címtér	egyetlen címtér, szálak	több folyamat, elszigetelt címtér
Periféria-hozzáférés	közvetlen, regiszter-szintű	kernel driveren keresztül
Heap használat	kerülendő vagy szigorúan korlátozott	általános, dinamikus allokáció
Verem méret	fordítási időben rögzített, kicsi (néhány KiB)	dinamikusan nő, nagy (MiB)

A memória-térkép egy tipikus, flash + SRAM architektúrájú MCU-n így néz ki (a címek illusztratívak, konkrét MCU-nál konkrét értékek):

Címtartomány	Tartalom	Méret
0x0000 0000 – 0x0001 FFFF	Flash (programkód, konstansok, vektortábla)	128 KiB
0x1FFF 0000 – 0x1FFF 03FF	Periféria regiszterek (GPIO, UART, …)	1 KiB
0x2000 0000 – 0x2000 7FFF	SRAM (adatok, verem, heap)	32 KiB

A periféria-regiszterek a memória-térkép egy elkülönített tartományában vannak, nem a normál RAM-ban. A C program ezeket `volatile` pointerekkel éri el, mert a fordító nem feltételezheti, hogy az érték olvasás nélkül is ugyanaz marad, és a sorrend fontos.

A bootfolyamat egy MCU-n – leegyszerűsítve – a következő lépésekből áll:



4.1. ábra: MCU bootfolyamat (Cortex-M példán) - A reset vektortól a main függvényig tartó lépések. Cortex-M-en a vektortábla első szava a kezdő MSP, a második a resetkezelő címe. A konkrét sorrend és a lépések implementációja MCU- és toolchain-függő.

A lépésekhez tartozó leírás:

1. **Reset vektor:** Cortex-M-en a mag a vektortábla első szavából a kezdeti stack pointert (MSP), a másodikból a resetkezelő címét tölti be, majd a resetkezelőn kezd végrehajtani. Más architektúrákon a reset vektor gyakran egy ugrásutasítás a startup rutinra. A pontos mechanizmus architektúra- és toolchain-függő.
2. **Startup kód:** Beprogramozza a stack pointert, nullázza a BSS szekciót, és a `.data` szekciót a flash-ből a RAM-ba másolja (ezek a nem null kezdőértékű globális változók).
3. **C futtatókörnyezet inicializálása:** Meghívja a C szabvány által előírt inicializáló függvényeket (pl. `__libc_init_array`), majd meghívja a `main()` függvényt.
4. **main():** Az alkalmazás belépési pontja. Innentől a firmware fut.

A fenti folyamat lényegesen egyszerűbb, mint egy Linux rendszer bootja: nincs kernel betöltés, nincs initramfs, nincs systemd. A teljes folyamat mikroszekundumok alatt lezajlik, ami kritikus az energiahatékonyság szempontjából.

4.1.3 Döntési szempontok

Amikor egy beágyazott eszközhöz platformot választunk, a következő fő szempontokat mérlegeljük:

Szempont	MCU (bare-metal / RTOS)	Linuxos SoC
Energiafogyasztás	alacsony (µA – mA, alvó üzemmódok)	magas (100 mA – több A)
BOM-költség	alacsony (1–10 USD)	magas (20–100+ USD)
Determinizmus	jó (nincs virtuális memória, megszakítások pontosak)	gyengébb (ütemező, laphibák, driverek)
Fejlesztői ismeret	C, hardverközeli programozás	teljes Linux stack, magas szintű nyelvek
Hálózati stack	korlátozott (IPv6/6LoWPAN, MQTT kliens)	teljes TCP/IP, bármilyen protokoll
Szoftverfrissítés	OTA, A/B partíciók, korlátozott méret	teljes csomagkezelő, nagyobb tárhely
Bonyolultság	alacsony, egyetlen kép	magas, kernel + userspace

A TerraSense **ts-node** esetében az MCU a kézenfekvő választás: a feladat néhány szenzor olvasása, egy szelep vezérlése és adatküldés rádiós interfészen keresztül. Ehhez nem kell teljes

Linux, viszont az energiafogyasztásnak alacsonynak kell lennie, mert az eszköz akár elemről is üzemelhet.

A statikus és dinamikus allokáció választása szintén fontos döntés. MCU-n a dinamikus allokáció (`malloc`/`free`) kerülendő vagy erősen korlátozott:

- **Fragmentáció:** A heap memóriatöredezettsége miatt előfordulhat, hogy van elegendő szabad memória, de nincs egybefüggő blokk.
- **Determinizmus:** A `malloc` futási ideje nem állandó, ami valós idejű rendszerekben gondot okozhat.
- **Memória-felügyelet:** Statikus allokáció esetén a linker időben kiszámítja a teljes memóriaigényt, és fordítási időben detektálható a túlcsordulás.
- **Biztonság:** A heap túlcsordulás, double-free és use-after-free hibák kiküszöbölhetők, ha nem használunk dinamikus memóriát.

Ezért a Zephyr és a legtöbb beágyazott firmware-keretrendszer előnyben részesíti a statikus allokációt: a szálak veremét, az üzenetsorokat, a szemaforokat gyakran fordítási időben definiáljuk. A Zephyr is lehetővé teszi a dinamikus allokációt, de az alapértelmezett konfigurációban sok komponens statikus.

4.1.4 Kidolgozott példa vagy labor

A következő laborban egy egyszerű C programot fordítunk le, és megnézzük, hogyan oszlik el a memóriafelhasználás a különböző szekciók között. A cél nem egy Zephyr alkalmazás, hanem annak megértése, hogyan olvassuk a map-fájlt és a `size` kimenetét. Ugyanez a folyamat alkalmazandó később a Zephyr build kimenetére is.

Munkakönyvtár: `~/iot-labor/i006`

```
mkdir -p ~/iot-labor/i006 && cd ~/iot-labor/i006
```

Hozzunk létre egy egyszerű C programot, amely globális, statikus és konstans változókat tartalmaz, hogy lássuk a különböző szekciókat:

```
/* memdemo.c - szekciók bemutatása */
#include <stdio.h>
#include <stdint.h>

/* Nem null kezdőértékű globális változó -> .data szekció */
uint32_t global_counter = 42;

/* Kezdőérték nélküli globális változó -> .bss szekció (induláskor nullázva) */
uint32_t global_buffer[64];

/* Konstans, csak olvasható -> .rodata szekció */
const char hello[] = "Hello, MCU!";

/* Statikus változó nulla kezdőértékkel -> .bss, a fordítási egységen belül */
static uint8_t local_static = 0;

int main(void) {
    /* Helyi változók -> verem (stack) */
    uint32_t local_var = 0;
    printf("%s\n", hello);
    global_counter++;
    global_buffer[0] = global_counter;
    local_var = global_buffer[0];
```

```
    return 0;
}
```

Fordítsuk le a programot, és kérjünk map-fájlt a linker-től:

```
gcc -g -Wl,-Map=memdemo.map -o memdemo memdemo.c
```

Most nézzük meg a `size` parancs kimenetét:

```
size memdemo
```

Várható kimenet (Ubuntu 24.04-on, gcc 13-mal mért reprezentatív érték; más rendszeren és toolchainnel eltér):

```
   text    data     bss     dec     hex filename
   1425     604     296    2325     915 memdemo
```

Ez a klasszikus Berkeley `size` formátum: a `text` szekció a programkód és a konstansok mérete (flash-ben), a `data` a nem null kezdőértékű, inicializált adatok RAM-mérete, a `bss` pedig a nulla kezdőértékű globális és statikus változók RAM-mérete. A `dec` a három összege decimálisan. Fontos, hogy a hoston a `data` értéke nem csak a `global_counter` 4 bájtját tartalmazza: a dinamikus linker és a C futtatókörnyezet néhány száz bájtnyi inicializált adata is ide számít. A 296 bájtos `bss` pedig a `global_buffer` 256 bájtján kívül a `local_static` 1 bájtját, valamint a C futtatókörnyezet és az igazítás miatti kitöltést tartalmazza. MCU-n, illetve Zephyr alatt ugyanezek a szekciók jóval kisebbek lehetnek, mert nincs dinamikus linker és más a C könyvtár.

Fontos megjegyezni, hogy a `data` szekció RAM-ban foglal helyet, de a kezdőértékeit a flash-ben tárolják. Így a flash-foglaltság nagyjából `text` + a `data` inicializáló része, a host folyamat RAM-foglaltsága pedig `data` + `bss` + verem + heap; Zephyrben ehhez a linker által létrehozott további RAM-szekciók is hozzáadódnak (lásd az ellenőrzési szakaszt).

A map-fájlban a `size`-nél részletesebb képet kapunk. Keressük meg benne a szekciók listáját:

```
Linker script and memory map

.text           0x0000000000001060      0x136
                0x0000000000001060                _start
                0x0000000000001149                main
...
.rodata         0x0000000000002000       0x14
                0x0000000000002008                hello
...
.data           0x0000000000004000       0x14
                0x0000000000004010                global_counter
...
.bss            0x0000000000004020      0x128
...
                0x0000000000004040                global_buffer
...
```

A map-fájl megmutatja az egyes szimbólumok címét és méretét, valamint a szekciók összméretét. Ebből pontosan megállapíthatjuk, melyik változó mennyi memóriát foglal. A fenti kivonatban a teljes `.bss` 0x128 = 296 bájt, amelyen belül a `global_buffer` 256 bájtos; a többi bájt a `local_static`, a C futtatókörnyezet és az igazítás miatti kitöltés. A szimbólum méretét tehát mindig a teljes szekciómérettől külön kell értelmezni.

A fenti példa x86-os Linuxon fut, ahol a címek virtuális címek. Egy MCU-n a map-fájl ugyanígy néz ki, de a címek fizikai címek, és a szekciók a flash és SRAM tartományokban helyezkednek el. A Zephyr build során a `build/zephyr/zephyr.map` fájl ugyanígy elemezhető.

4.1.5 Ellenőrzés és hibakeresés

A memóriahasználat ellenőrzésére a következő módszerek állnak rendelkezésre:

Mit ellenőrzünk	Hogyan	Mire figyeljünk
Flash-foglaltság	`size` kimenet `text` + `data` inicializáló része, map-fájl flash szekciók összege	Ne lépje túl a flash méretét; hagyjon helyet az OTA partíciónak
RAM-foglaltság	`size` kimenet `data` + `bss`, valamint a map és a RAM-riport további RAM-szekciói; a verem és a heap külön	Mindig hagyjon tartalékot a veremnek, és ne számolja be kétszer a már linkelt veremterületet
Verem túlcsordulás	futás közbeni védőértékek (stack canary), vagy statikus elemzés	A növekedési irány ABI- és linkerfüggő; a szokásos Cortex-M és x86-64 elrendezésben a verem lefelé (a magasabb címek felől), a heap felfelé nő. A határokat a linker script és a map mutatja
Heap fragmentáció	ha használunk heapet, figyeljük a maximális blokkméretet	Töredezettség miatt a `malloc` sikertelen lehet, még ha van szabad memória

Tipikus hibatünetek és megoldások:

Tünet	Ok	Javítás
A firmware nem fér el a flash-ben	Túl nagy kód vagy konstans táblázatok	Csökkentsük a kódot, optimalizáljunk `-Os` kapcsolóval, tömörítsük az adatokat
Futás közben váratlan lefagyás vagy újraindulás	Verem túlcsordulás	Növeljük a verem méretét, vagy csökkentsük a lokális változókat, rekurziót
`malloc` sikertelen, pedig van szabad RAM	Heap fragmentáció	Kerüljük a dinamikus allokációt, vagy használjunk fix méretű memóriapoolt
A globális változó kezdőértéke nem érvényesül	A `.data` másolása nem történt meg a startupban	Ellenőrizzük a startup kódot, a linker szkriptet
A periféria-regiszter nem reagál az írásra	Nem `volatile` pointer, vagy rossz cím	Használjunk `volatile`-t, ellenőrizzük a memória-térképet

A verem és a heap növekedési iránya nem a C nyelv szabálya, hanem az adott ABI és linker script döntése. A bemutatott x86-64 host ELF-en és sok Cortex-M elrendezésben a verem a magasabb címek felől növekszik lefelé: a stack pointer függvényhíváskor csökken. A heap ezzel szemben jellemzően az alacsonyabb címek felől felfelé nő. A két terület a szabad RAM-ban találkozik, ezért a verem túlcsordulása a heap vagy a statikus adatok felé írhat.

Zephyrben a `K_THREAD_DEFINE` és a `K_THREAD_STACK_DEFINE` szálanként külön, statikusan lefoglalt veremterületet hoz létre; a kernel rendszerheapje (ha a `CONFIG_HEAP_MEM_POOL_SIZE` engedélyezi) ettől külön terület. A pontos címeket és a szekciók sorrendjét a linker script, illetve a `build/zephyr/zephyr.map` adja, ezért az irányt és a határokat mindig a konkrét buildhez kell ellenőrizni.



4.2. ábra: Tipikus RAM-elrendezés veremmel és heap-pel - Illusztratív, nem konkrét linker scriptből származó címek. A verem jellemzően a magasabb címek felől lefelé, a heap az alacsonyabb címek felől felfelé nő; Zephyrben a szálak vermei külön, statikusan lefoglalt területek. A pontos határokat a linker script és a map-fájl adja.

A map-fájl és a `size` kimenet a build folyamat részeként automatikusan generálódik. Érdemes ezeket a CI folyamatba is beépíteni, hogy minden commit után lássuk a méretek változását.

4.1.6 Tipikus hibák és félreértések

1. **A heap használata megszokásból.** Linuxon természetes a `malloc`/`free` használata, de MCU-n ez fragmentációhoz és nem determinisztikus futáshoz vezet. A Zephyrben inkább statikus allokációt használjunk, vagy specifikus memóriapoolokat.
2. **A `size` kimenet félreértelmezése.** Sokan azt hiszik, hogy a `text` a RAM-ot foglalja, pedig az a flash-ben van. A `size` csak a szabványos szekciókat mutatja; a RAM-foglaltság

ennél több: a `data` + `bss` mellett a linkelt további RAM-szekciók (például a Zephyr `.noinit` szekciója), valamint a futásidejű verem és heap is helyet foglal. Hoston a folyamat vermét és heapjét a kernel és az ABI jelöli ki, ezért azok nem jelennek meg a `size` kimenetében. Zephyrben a `K_THREAD_DEFINE`-nal statikusan létrehozott szálvermek tipikusan a `.noinit` szekcióba kerülnek: RAM-ban elfoglalják a területet, de a `size` `bss`-értékében nem feltétlenül jelennek meg. Ezért ugyanazt a veremterületet ne számítsuk be egyszer a `bss`-ben, még egyszer pedig külön. A pontos képet a `build/zephyr/zephyr.map` és a `west build -t ram_report` (illetve `rom_report`) adja, amely a linker map alapján szekciónként mutatja a foglalást.

3. **A `volatile` elhagyása.** A periféria-regisztereket `volatile` pointerrel kell elérni, különben a fordító optimalizálhatja a sorozatos olvasásokat/írásokat, és a regiszter nem úgy viselkedik, ahogy várjuk.
4. **A memória-térkép összekeverése.** Nem minden MCU rendelkezik ugyanolyan memória-térképpel; a flash és SRAM címek, méretek és a perifériák helye gyártónként és típusonként eltérő. Mindig az adott MCU referencia kézikönyvét kell használni.

4.1.7 Feladatok

4.1.7.1 a) Fogalmi/döntési kérdés

Magyarázza el, miért nincs szükség MMU-ra egy egyszerű szenzorcsomóponton, és milyen következményekkel jár ez a programozási modellre nézve?

4.1.7.2 b) Számolási vagy tervezési feladat

Egy MCU 512 KiB flash-sel és 64 KiB SRAM-mal rendelkezik. A firmware linkelése után a `size` a következőt mutatja:

```
   text    data     bss     dec     hex
  31820    2048    1024   34892    884c
```

A linker szkript 8 KiB stack-et és 4 KiB heapet foglal le. Elfér-e a firmware, ha a következő verzióban a kód 10%-kal nő? Mi a teljes RAM-foglaltság?

4.1.7.3 c) Implementációs feladat

Vegyünk egy egyszerű C programot, amelyben van egy nagy globális tömb (pl. `uint8_t buffer[4096]`) és egy nagy lokális tömb a `main()`-ben (pl. `uint8_t local[8192]`). Fordítsuk le, és vizsgáljuk meg a `size` kimenetet, valamint a map-fájlt. Hol jelennek meg ezek a tömbök? Módosítsuk a programot úgy, hogy a lokális tömböt statikussá tesszük, és hasonlítsuk össze a méreteket.

4.1.7.4 d) Hibakeresési vagy kiterjesztési feladat

Tegyük fel, hogy a ts-node firmware-je futás közben véletlenszerűen újraindul. A hibakeresés során azt tapasztaljuk, hogy a `bss` szekció mérete nem változik, de a verem mutató értéke időnként a heap területére mutat. Milyen hibára utal ez, és hogyan javítaná?

4.1.8 Megoldások

4.1.8.1 a)

Egy szenzorcsomóponton nincs szükség több folyamat futtatására vagy folyamatszintű memóriavédelemre; a firmware egyetlen címtérben fut. Az MMU hiánya azt jelenti, hogy nincs virtuális memória, tehát minden kód és adat fizikai címeken érhető el. A címfordítás hiánya azonban önmagában nem jelenti a hardveres védelem teljes hiányát: a Cortex-M MPU-ja régiószintű olvasási, írási és végrehajtási jogosultságot kényszeríthet ki, és a Zephyr userspace (`CONFIG_USERSPACE`) ezt használja a felhasználói szálak elkülönítésére. MPU nélküli magon viszont valóban nincs hardveres régióvédelem, és laphiba (page fault) sincs, mert nincs virtuális memória. A determinizmus javul, mert nincs laphiba-kezelés vagy memória-lapcsere. Azt, hogy egy adott MCU-n és konfigurációban van-e MPU-alapú védelem, mindig a mag és a Zephyr board-leírása alapján kell megítélni.

4.1.8.2 b)

A firmware jelenlegi méretei: - Flash: `text` + a `data` inicializáló része `text` + `data` = 31820 B + 2048 B = 33868 B. - RAM: `data` + `bss` + stack + heap = 2048 B + 1024 B + 8192 B + 4096 B = 15360 B.

A flash méret 512 KiB = 524288 B, a RAM 64 KiB = 65536 B. Ha a kód 10%-kal nő, akkor `text` 31820 B-ről $31820 \times 1.1 = 34982$ B-re nő (a `data` és `bss` nem feltétlenül nő, de feltételezhetjük, hogy a konstansok is nőnek). Az új flash-igény 34982 + 2048 = 37030 B, ami még mindig jóval a 524288 B alatt van, tehát elfér. A RAM-foglaltság változatlan marad, ha a verem és heap mérete nem változik, tehát 15360 B, ami szintén elfér. A válasz: elfér.

4.1.8.3 c)

A program:

```c
/* sectest.c */
#include <stdint.h>

uint8_t global_buffer[4096];      /* .bss */
uint8_t initialized[16] = {1};    /* .data */

int main(void) {
    uint8_t local_buffer[8192];   /* veremben (stack) */
    for (int i = 0; i < 8192; i++) {
        local_buffer[i] = (uint8_t)i;
    }
    return local_buffer[0];
}
```

Fordítás: `gcc -g -Wl,-Map=sectest.map -o sectest sectest.c`

`size sectest` eredménye (illusztratív):

```
   text     data      bss      dec      hex filename
   1820      640     4160     6620     19dc sectest
```

A `global_buffer` a `.bss`-ben jelenik meg (4096 bájt), a `initialized` a `.data`-ban (16 bájt), a `local_buffer` pedig nem jelenik meg a statikus méretekben, mert a veremben jön létre futás közben. A map-fájlban a `global_buffer` a `.bss` szekcióban látható 4096-os mérettel, a `initialized` a `.data`-ban 16-tal. A `local_buffer` nincs a map-fájlban, mert csak futás közben létezik.

Ha a `local_buffer`-t statikussá tesszük (`static uint8_t local_buffer[8192]`), akkor az a `.bss`-be kerül, és a `size` kimenet `bss` mérete 4096 + 8192 = 12288 B-re nő. Ez jól mutatja a statikus allokáció memóriafoglalását.

4.1.8.4 d)

A leírás alapján a verem mutató a heap területére mutat, ami arra utal, hogy a verem túlcsordult, és belelóg a heap területébe. Ez tipikusan akkor fordul elő, ha a verem mérete túl kicsi a tényleges használathoz képest: túl sok lokális változó, mély rekurzió, vagy nagy lokális tömbök. A javítás: növeljük a verem méretét, vagy csökkentsük a lokális változók méretét (pl. statikusra vagy globálisra áthelyezéssel), illetve kerüljük a rekurziót. Zephyr alatt a `CONFIG_MAIN_STACK_SIZE` vagy a szálak veremméretét lehet növelni.

4.1.9 Rövid összefoglaló

Az MCU és a Linuxos SoC közötti fő különbség az MMU és a virtuális memória hiánya, ami közvetlen fizikai címtérhez, statikus allokációhoz és szigorú memória-korlátokhoz vezet (a legtöbb Cortex-M-en az MPU adhat régiószintű védelmet). A firmware a flash-ben tárolódik, az adatok az SRAM-ban, a perifériák pedig memória-címeken érhetők el. A bootfolyamat a reset vektortól a mainig tart, és inicializálja a `.data` és `.bss` szekciókat. A memóriafelhasználást a `size` parancs, a linker map-fájl és Zephyr alatt a `ram_report`/`rom_report` segítségével ellenőrizhetjük. A statikus allokáció előnyben részesítése és a verem méretének tudatos megválasztása elengedhetetlen a robusztus, determinisztikus firmware-hez. **Továbbvitt eredmények:** a `~/iot-labor/i006/` munkakönyvtárban létrehozott `memdemo.c` és `sectest.c` programok, valamint az elemzési módszer (map-fájl, `size` kimenet) a későbbi alfejezetekben (Szálak és ütemezés, Eszközszoftver-architektúra: rétegek és állapotgépek, OTA-frissítés: A/B képek, rollback, delta, fokozatos kiadás) használt memória-korlát-tudatosság alapja.

4.1.10 Források és ellenőrzési hiányok

- Zephyr dokumentáció: https://docs.zephyrproject.org/latest/ (a linker szkriptekre és a build kimenetre vonatkozó részek)
- ARM Cortex-M Technical Reference Manual (a memória-térkép és a reset vektor leírásához)
- GNU Binutils dokumentáció: `size`, `ld` map-fájl formátum

Ellenőrzési igény: a Zephyr aktuális LTS verziójának pontos API-nevei és a `native_sim` board viselkedése a későbbi laborok során ellenőrizendő. A fenti példákban szereplő `size` kimeneti értékek illusztratívak, a konkrét értékek a fordítótól és a rendszertől függnek.

4.2 Perifériák: GPIO, UART, I2C, SPI, ADC, időzítők

4.2.1 Motiváció

A TerraSense `ts-node` egységének egyetlen feladata, hogy néhány szenzort olvasson, egy szelepet kapcsoljon, és az eredményt eljuttassa a gatewayhez. Aki eddig Linuxon socketekkel, fájlokkal és könyvtári függvényekkel dolgozott, ott a periféria szinte mindig „kész”; egy mikrovezérlőn viszont a GPIO, az I2C vagy az ADC közvetlenül, hardverszinten jön szembe. Ebben az alfejezetben a leggyakoribb perifériákat és buszokat tekintjük át a szenzorillesztés szempontjából. A cél, hogy egy adott érzékelő adatlapját látva el tudd dönteni, melyik buszra érdemes kötni, és egy I2C-tranzakciót bájtszinten is értelmezni tudj.

4.2.2 GPIO: digitális bemenet és kimenet

Definíció. A GPIO (General-Purpose Input/Output) a mikrovezérlő olyan lába, amelyet szoftveresen ki- vagy bemenetnek konfigurálhatunk, és amely általában más perifériaként is használható. Kimenetként logikai magas vagy alacsony szintet ad, bemenetként pedig a láb feszültségszintjét olvassa vissza.

Egy GPIO-kimenet leggyakrabban **push-pull**: a láb aktívan húzza magasra vagy alacsonyra. Ezzel szemben **open-drain** kimenet esetén a láb csak alacsonyra húz, magas szintet egy külső felhúzó ellenállás állít be. Az open-drain azért hasznos, mert több eszköz is rákötethető ugyanarra a vezetékre, és a nyugalmi szintet bármelyikük le tudja húzni. Hátránya, hogy a felhúzó ellenállás nélkül a vezeték lebeg, ezért a logikai szint nem definiált.

Bemenetnél hasonló a helyzet: ha nincs külső vagy belső felhúzó (pull-up) vagy lehúzó (pull-down) ellenállás, akkor egy nyitott kapcsolóval kötött bemenet véletlenszerűen olvashat 0-t vagy 1-et. A Zephyr GPIO API-ja ezt a `pull-up`, `pull-down` vagy `high-impedance` beállítással kezeli; a pontos API-nevek a Devicetree és Kconfig: a hardver és a konfiguráció leírása alfejezetben szereplő devicetree-hez igazodnak.

A TerraSense öntözőszelepét nem közvetlenül egy GPIO-láb táplálja, hanem egy relé- vagy félvezetős kapcsolómodul vezérlőbemenete. Így a mikrovezérlő csak a vezérlőjelet adja, a nagyobb áramú áramkört a modul választja le.

4.2.3 UART: aszinkron soros, pont-pont

Definíció. Az UART (Universal Asynchronous Receiver/Transmitter) pont-pont soros kommunikáció, amelyen az adatbiteket közös órajel nélkül viszik át; a vevő a saját órajelével veszi mintát, ezért a küldő és a vevő sebességét előre, pontosan kell ismerni. A keret általában egy startbitből, 7–9 adatbitből, opcionális paritásbitből és egy stopbitből áll.

A UART előnye az egyszerűség: két adatvezeték (TX, RX) és közös föld elég. Hátránya, hogy nincs protokollszintű címzés, nyugtázás vagy flow control, hacsak külön nem építjük be. Tipikus sebességei a beágyazott rendszerekben néhány ezer bit/s-tól néhány millió bit/s-ig terjednek; a legfontosabb gyakorlati buktató, hogy a `baud rate` a bitsebességet jelenti, nem a bájtsebességet.

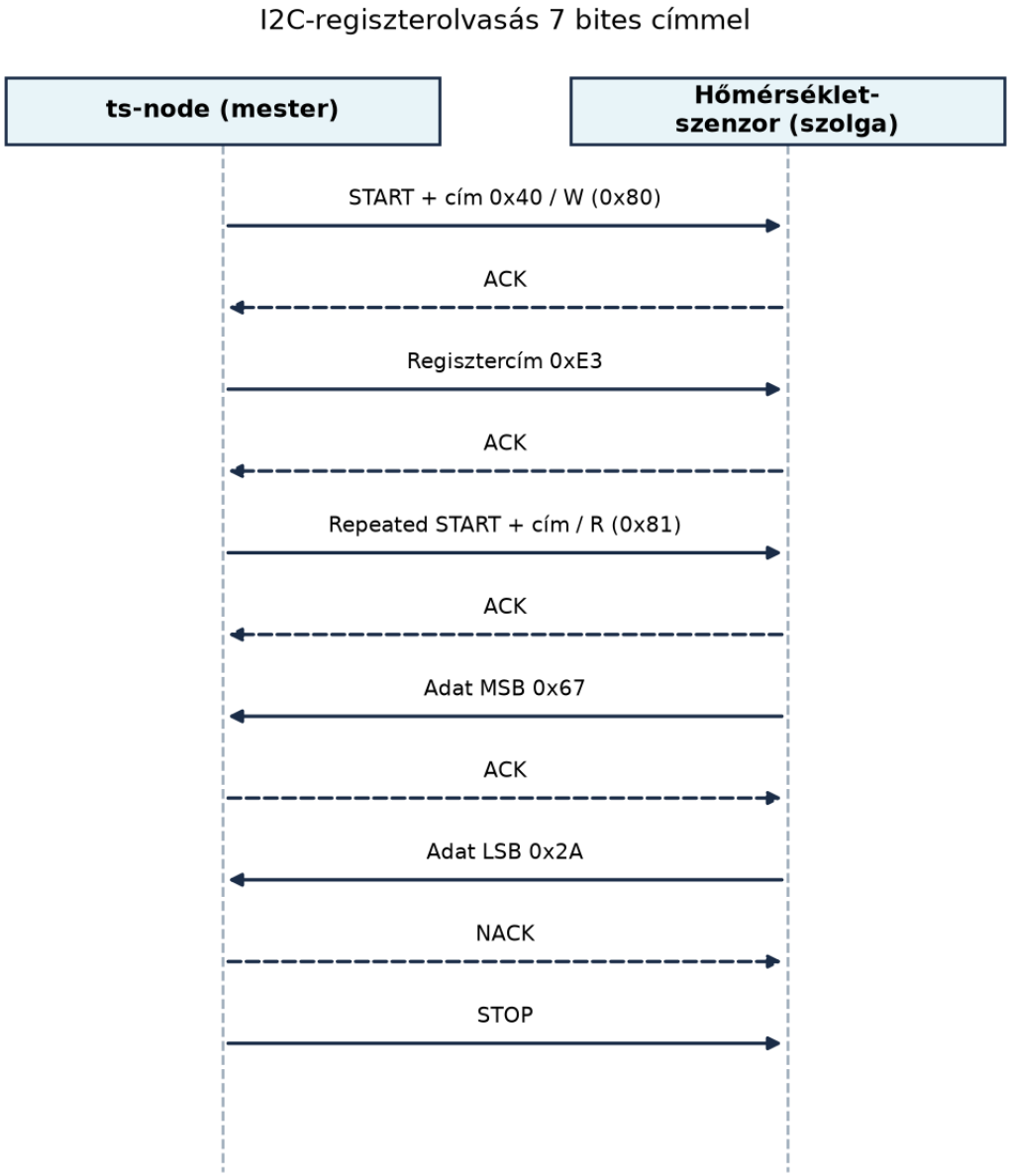
A TerraSense konzolja UART-on keresztül beszél a fejlesztői gazdagéppel. Ugyanezt a perifériát gyakran használják GPS-vevők, AT-parancsos modemek vagy egyszerű távolságmérő modulok illesztésére.

4.2.4 I2C: kevés láb, címezhető busz

Definíció. Az I2C (Inter-Integrated Circuit) kétvezetékes, órajelt vezérelt, mester-szolga architektúrájú busz. Az SDA (Serial Data) vezeték viszi a cím- és adatbiteket, az SCL (Serial Clock) a mester által kiadott órajelet. A szolga válasza a nyugtázó bit (ACK/NACK). A busz open-drain felépítésű, ezért mindkét vezetékhez külső felhúzó ellenállás kell.

A gyakorlatban az I2C a szenzorok legnépszerűbb busza, mert sok eszköz köthető ugyanarra a két vezetékre, mindegyiknek saját 7 bites (ritkábban 10 bites) címe van. A 7 bites címet az eszközgyártók gyakran bináris, hexadecimális vagy 8 bites formában is megadják, ami tipikus félreértések forrása: a buszra a mester által küldött első bájt a 7 bites cím balra tolva, a legalsó bit pedig írás/olvasás jelzés. Egy 7 bites `0x40` címhez tehát íráskor `0x80`, olvasáskor `0x81` tartozik.

A legtöbb I2C-szenzor regisztermodellt használ: a mester először megcímzi a szolgát, elküldi a regisztercímet, majd írja vagy olvassa a regiszter tartalmát. Ezt mutatja a következő példa is.



4.3. ábra: I2C-regiszterolvasás 7 bites címmel - A szaggatott nyilak az I2C-nyugtákat jelölik; a mester az utolsó bájtot NACK-kel nyugtázza, majd STOP-pal zárja a tranzakciót.

Az I2C hátránya a viszonylag alacsony sebesség: a szokásos mód 100 kbit/s, a gyors mód 400 kbit/s, a gyors+ mód pedig 1 Mbit/s körüli. A busz hossza és a résztvevők kapacitása korlátozza a sebességet, ezért I2C-t elsődlegesen kis távolságú, kis adatsebességű érzékelőkhöz használunk.

4.2.5 SPI: nagyobb sebesség, külön kiválasztóvonallal

Definíció. Az SPI (Serial Peripheral Interface) szinkron, mester-szolga, gyakran teljes duplex busz. Négy vezetéket használ: `SCLK` órajel, `MOSI` mestertől a szolgához, `MISO` szolgától a mesterhez, és `CS` (Chip Select) kiválasztójel. Az SPI-hoz nincs szabványos eszköz- vagy regisztercím, a mester a `CS` vonal alacsonyra húzásával választja ki, melyik szolgával beszél.

Az I2C-vel szemben az SPI gyorsabb, gyakran több tíz Mbit/s-ot is elérhet, és az adatátvitel gyakorlatilag folyamatos lehet. Hátránya, hogy minden szolgának külön `CS` vonal kell, tehát a lábak száma gyorsan nő. Az SPI-nek több órajel- és mintavételi üzemmódja van (CPOL, CPHA), ezeket a mester és a szolga között pontosan egyeztetni kell, különben az adatbiteket rossz órajelélre veszik. Az SPI jellemzően kijelzőkhöz, SD-kártyákhoz, nagysebességű ADC- vagy DAC-áramkörökhöz való.

4.2.6 ADC: analóg jelből digitális szám

Definíció. Az ADC (Analog-to-Digital Converter) olyan periféria, amely egy bemeneti feszültséget, például 0 és egy referenciafeszültség (`V_ref`) között, felbontásától függően diszkrét egész számmá alakít. Egy N bites ADC nullától `2^N - 1`-ig terjedő kódot ad; a kódolás közelítőleg lineáris a bemeneti feszültség függvényében.

Az átváltás:

$$D = \text{round}\left(\frac{V_{\text{in}}}{V_{\text{ref}}} \cdot (2^N - 1)\right)$$

Ahol `D` a digitális kimeneti kód, `V_in` a mért feszültség, `V_ref` a referenciafeszültség, `N` pedig a felbontás bitekben. A 0 V-hoz rendszerint 0, a referenciafeszültséghez pedig `2^N - 1` tartozik. A mérés pontosságát a referencia stabilitása, a zaj, a forrás impedanciája és a mintavételi idő is befolyásolja, de az alapösszefüggés ehhez a képlethez köthető.

A TerraSense talajnedvesség-érzékelője analóg feszültséget ad, ezért ADC-re kötjük. A nyers kódot később millivoltra, majd kalibrációs görbével százalékra kell váltani; ezt az Mintavétel, kalibráció és szűrés alfejezetben végezzük el.

4.2.7 Időzítők és PWM

Definíció. Az időzítő (timer) egy hardveres számláló, amely az órajelből vagy külső jelből számlál, és túlcsorduláskor vagy egy beállított összehasonlítási értéknél megszakítást, kimenetváltást vagy más eseményt generál. A PWM (Pulse Width Modulation) olyan négyszögjel, amelynek periódusideje rögzített, de a magas szint hossza, azaz a kitöltési tényező változtatható.

$$\text{duty} = \frac{t_{\text{high}}}{t_{\text{period}}} \cdot 100\%$$

A PWM-eszköz a simított átlagfeszültségével hasznos: ha egy ellenálláson és kondenzátoron áteresztjük, a kimenet körülbelül `V_cc · duty / 100`, tehát egyszerű digitális-analóg átalakításra is alkalmas. Ugyanakkor ez a linearitás feszültségre igaz; motornál, szervónál vagy LED-nél a munkapont, a mechanika és az érzékelés nem feltétlenül lineáris. A TerraSense-ben a szelep alapvezérlése bináris GPIO lehet; ha arányos nyitású szelepet használunk, akkor PWM- vagy analóg vezérlőjelre van szükség. A biztonságos vezérlést az Aktuátorok: biztonságos vezérlés és visszajelzés alfejezet tárgyalja.

4.2.8 Döntési szempontok

Szempont	UART	I2C	SPI
Adatvezetékek	TX, RX	SDA, SCL	SCLK, MOSI, MISO, CS
Órajel	nincs, aszinkron	közös SCL	közös SCLK
Topológia	pont-pont	több szolga, cím	több szolga, CS
Duplex	full duplex	half duplex	full duplex
Jellemző sebesség	kb. 9600–4 000 000 bit/s	100/400 kbit/s, ritkán 1 Mbit/s	több Mbit/s–több tíz Mbit/s
Címzés	nincs	7/10 bit a protokollban	nincs, hardver CS
Nyugtázás	nincs	ACK/NACK	nincs
Leggyakoribb hiba	rossz baud rate	hiányzó felhúzó, rossz cím	rossz CS vagy SPI-mód

A TerraSense `ts-node` számára a választás a következőképpen alakul:

- Hőmérséklet és páratartalom-szenzor: I2C, mert kevés lábat foglal, és alacsony adatsebesség is elég.
- Talajnedvesség: ADC, mert az érzékelő analóg kimenetű.
- Öntözőszelep vezérlés: GPIO, mert alaphelyzetben ki/be kapcsolás.
- Fejlesztői konzol: UART, mert pont-pont, egyszerű és jól támogatott.

A döntés során a sebesség önmagában ritkán dönt; a lábak száma, a szenzor gyártói illesztése és a busz topológiai korlátai legalább ilyen fontosak.

4.2.9 Kidolgozott példa: I2C-regiszterolvasás és ADC-konverzió

Legyen egy hőmérséklet/páratartalom szenzor 7 bites I2C-címe `0x40`, a hőmérséklet regisztere pedig `0xE3`. A mester két bájtot olvas, az eredmény `0x672A`. A buszon a következő történik:

1. START feltétel.
2. A mester elküldi a `0x80` bájtot: ez a `0x40` 7 bites cím balra tolva, alsó bit 0, tehát írás. A szolga ACK-t ad.
3. A mester elküldi a `0xE3` regisztercímet. A szolga ACK-t ad.
4. Ismételt START (Repeated START).
5. A mester elküldi a `0x81` bájtot: ugyanaz a cím olvasási bittel. A szolga ACK-t ad.
6. A szolga küldi a felső adatbájtot: `0x67`. A mester ACK-t ad.
7. A szolga küldi az alsó adatbájtot: `0x2A`. A mester NACK-t ad, mert nem kér több bájtot.
8. STOP feltétel.

A mester által kiírt bájtsorozat tehát `80 E3 81`; az olvasott adat pedig `67 2A`. A `0x672A` hexadecimális érték tízes számrendszerben `26410`. Azt, hogy ebből hány °C vagy %RH lesz, a gyártói képlet vagy kalibrációs függvény adja meg; ez már az Szenzorolvasás a Zephyr sensor API-val alfejezet témája.

ADC példaként vegyünk 12 bites felbontást és `3,3 V` referenciafeszültséget. Ha a bemeneti feszültség `2,5 V`, akkor:

$$D = \mathrm{round}\left(\frac{2{,}5}{3{,}3} \cdot (2^{12} - 1)\right) = \mathrm{round}(3102{,}27) = 3102$$

Visszafelé számolva:

$$V_{\text{in}} \approx \frac{3102}{4095} \cdot 3{,}3 \approx 2{,}4998 \text{ V}$$

A kódban ugyanez tizedesponttal, `uint32_t` közbülső típussal, hogy a szorzás ne csorduljon túl:

```
uint32_t adc_to_mv(uint16_t raw)
{
    const uint32_t vref_mv = 3300;
    const uint32_t full_scale = 4095; /* 12 bit */
    return (uint32_t)raw * vref_mv / full_scale;
}
```

Továbbvitt eredmény. Az itt bemutatott I2C-címzési és regiszterolvasási bájtsorrendet, valamint az ACK/NACK és a Repeated START jelentését használja az Szenzorolvasás a Zephyr sensor API-val alfejezet, amikor a TerraSense emulált szenzorát olvassa. A `nyers kód → millivolt → fizikai mennyiség` láncot az Mintavétel, kalibráció és szűrés alfejezet viszi tovább a talajnedvesség-konverzióban.

4.2.10 Ellenőrzés és hibakeresés

Jelenség	Legvalószínűbb ok	Javítás
I2C-n minden cím NACK	hiányzó vagy rossz felhúzó, SDA/SCL fel van cserélve, rossz cím	ellenőrizni a felhúzókat, a vezetékeket és a 7 bites cimet
UART kimenet értelmezhetetlen karakterek	rossz baud rate vagy rossz keretformátum	mindkét oldalon ugyanazt a baud rate-et, adatbit- és paritásbeállítást használni
SPI olvasás mindig `0xFF` vagy `0x00`	CS nincs lehúzva, rossz SPI-mód, MISO/MOSI felcserélve	CS-vezetéket, órajel-polaritást és mintavételi élet ellenőrizni
ADC érték zajos vagy telített	rossz referencia, lebegő bemenet, rossz skálázás	referenciafeszültséget, bemeneti osztót és földelést ellenőrizni
GPIO bemenet véletlenszerű értéket ad	lebegő bemenet, hiányzó felhúzó/lehúzó	belső vagy külső ellenállást bekapcsolni

4.2.11 Tipikus hibák és félreértések

Az I2C-címmel kapcsolatban a leggyakoribb hiba a 7 bites és 8 bites alak összekeverése. A `0x40` 7 bites címből a buszon `0x80` írási bájt lesz; ha valaki `0x40`-et ír a buszra, az egy másik, `0x20` 7 bites címhez tartozó írási műveletnek felel meg. A másik félreértés az ACK/NACK szerepe: az ACK nem a fogadott adat helyességét igazolja, hanem azt, hogy a címzett szolga a buszon volt, és a bájtot átvette. A master által küldött NACK az I2C-olvasás végén gyakran normális, azt jelzi, hogy nem kér több adatbájtot.

PWM esetén sokan feltételezik, hogy a 50% kitöltési tényező automatikusan fél teljesítményt jelent, mint egy feszültségosztónál. Motorok, LED-ek és fűtőszálak esetében a teljesítmény, a fény vagy a hőhatás nem feltétlenül arányos a kitöltési tényezővel, ezért a gyártói karakterisztikát vagy mérést kell használni.

4.2.12 Feladatok

1. **Döntési feladat.** Adott három eszköz: egy hőmérséklet/nyomás szenzor I2C-interfésszel, egy GPS-vevő UART-interfésszel és egy nagysebességű analóg mintavételező ADC SPI-interfésszel. Válaszd ki mindegyikhez a megfelelő buszt, és indokold a választást.

2. **Számolási feladat.** Egy 10 bites ADC referenciafeszültsége `2,048 V`. Mekkora digitális kódot ad `1,2 V` bemeneti feszültségnél? Visszaszámolva mekkora feszültség tartozik a kiszámolt kódhoz?

3. **Implementációs feladat.** Írj C függvényt, amely egy 12 bites ADC nyers értékét millivoltra váltja `3,3 V` referenciafeszültség mellett, és számítsd ki, hány mV-ot jelent a `1860` nyers érték.

4. **Hibakeresési feladat.** Egy fejlesztő I2C-n szeretné olvasni egy szenzor regiszterét. A szenzor 7 bites címe `0x48`, a regisztercíme `0x75`. A buszra írt első bájt `0x91`, a második `0x75`. Mi a hiba? A szolga fog válaszolni?

4.2.13 Megoldások

1. A hőmérséklet/nyomás szenzorhoz I2C illik, mert a gyártó már I2C-interfészen adja ki, kevés lábat foglal, és a kis adatsebesség bőven elegendő. A GPS-vevő UART-ja pont-pont, aszinkron adatfolyamot ad, amelyhez a UART természetes; nincs szükség több szolga címzésére. A nagysebességű ADC-hez SPI való, mert az I2C 100/400 kbit/s korlátja szűk lehet, az SPI pedig nagyobb mintavételi átvitelt biztosít; a külön CS is rendelkezésre áll.

2. A képletbe behelyettesítve:

$$D = \text{round}\left(\frac{1{,}2}{2{,}048} \cdot (2^{10} - 1)\right) = \text{round}(599{,}41) = 599$$

Visszafelé:

$$V_{\text{in}} \approx \frac{599}{1023} \cdot 2{,}048 \approx 1{,}199 \text{ V}$$

A hiba a kerekítésből adódóan kisebb, mint egy LSB.

3. A függvény:

```c
uint32_t adc_to_mv(uint16_t raw)
{
    const uint32_t vref_mv = 3300;
    const uint32_t full_scale = 4095; /* 12 bit */
    return (uint32_t)raw * vref_mv / full_scale;
}
```

A `1860` nyers érték:

$$\frac{1860}{4095} \cdot 3300 \approx 1498{,}9 \text{ mV}$$

4. A buszra írt első bájt `0x91` hiba. A 7 bites cím `0x48`-ból írási bájt: `0x48 << 1` plusz a legalacsonyabb `0` bit, tehát `0x90`. Az olvasási bájt `0x91` lenne, de itt az első fázis regisztercímet ír, nem olvas. Emiatt a szolga nem fogja értelmezni a tranzakciót, és NACK-kel vagy hibás működéssel válaszol. A helyes írási bájt `0x90`, majd a regisztercím `0x75`.

4.2.14 Rövid összefoglaló

A leggyakoribb beágyazott perifériák: GPIO bináris ki- és bemenetre, UART pont-pont aszinkron soros átvitelre, I2C kevés lábú, címezhető szenzorbuszra, SPI nagy sebességű, CS-választásos átvitelre, ADC analóg feszültség digitális kóddá alakítására, időzítő és PWM pedig időzítésre és modulált vezérlőjelre. Az I2C-regiszterolvasás lényege: 7 bites cím balra tolva írási/olvasási bittel, regisztercím, Repeated START, majd adatbájtok ACK/NACK-kel. Az ADC két iránya: feszültségből kód $D \approx V_{\text{in}}/V_{\text{ref}} \cdot (2^N - 1)$, kódból feszültség $V_{\text{in}} \approx D \cdot V_{\text{ref}}/(2^N - 1)$; ideális esetben a 0 kódhoz 0 V, a $2^N - 1$ kódhoz V_{ref} tartozik. **Továbbvitt eredmény:** a bemutatott I2C-bájtsorrendet az Szenzorolvasás a Zephyr sensor API-val, az ADC `nyers kód → millivolt → fizikai mennyiség` konverziót pedig az Mintavétel, kalibráció és szűrés alfejezet használja a TerraSense-ben.

4.2.15 Források és ellenőrzési hiányok

Az alfejezet az I2C, SPI, UART és ADC alapfogalmait általános, stabil protokollismeretként mutatja be, ezért nem tartalmaz webes hivatkozást. Ellenőrizendő a Zephyr aktuális GPIO/I2C/SPI/ADC API-ja és devicetree-kötése a Devicetree és Kconfig: a hardver és a konfiguráció leírása alfejezet megírásakor, valamint a TerraSense-szenzorok konkrét címe, regiszterkészlete és kalibrációs egyenlete. Az I2C-példában használt `0x40` cím és `0xE3` regiszter didaktikus, nem állítja egyetlen konkrét gyártó adatlapjának való megfelelést.

4.3 Megszakítások, időzítés és konkurens hozzáférés

4.3.1 Motiváció

A TerraSense `ts-node` szenzorcsomópontjában a hőmérséklet- és páratartalom-szenzor általában nem folyamatosan ad értéket, hanem egy mérés végén egy hardveres jelzéssel, úgynevezett megszakítással értesíti a mikrovezérlőt. Ha a program egy ciklusban állandóan kérdezgetné a szenzort, hogy elkészült-e a mérés, akkor feleslegesen égetné az energiát, és a válaszidő is rosszabb lenne. A megszakítás az a mechanizmus, amellyel a hardver maga szakítja félbe a normál programfutást, amikor egy fontos esemény történt. Ebben az alfejezetben a megszakítások, a megszakításkezelő rutinokra vonatkozó szabályok, valamint a megszakítás és a főprogram közötti biztonságos adatmegosztás alapjait tanuljuk meg, a `ts-node` adatgyűjtési ciklusára építve.

4.3.2 Megszakítás fogalma, prioritás és késleltetés

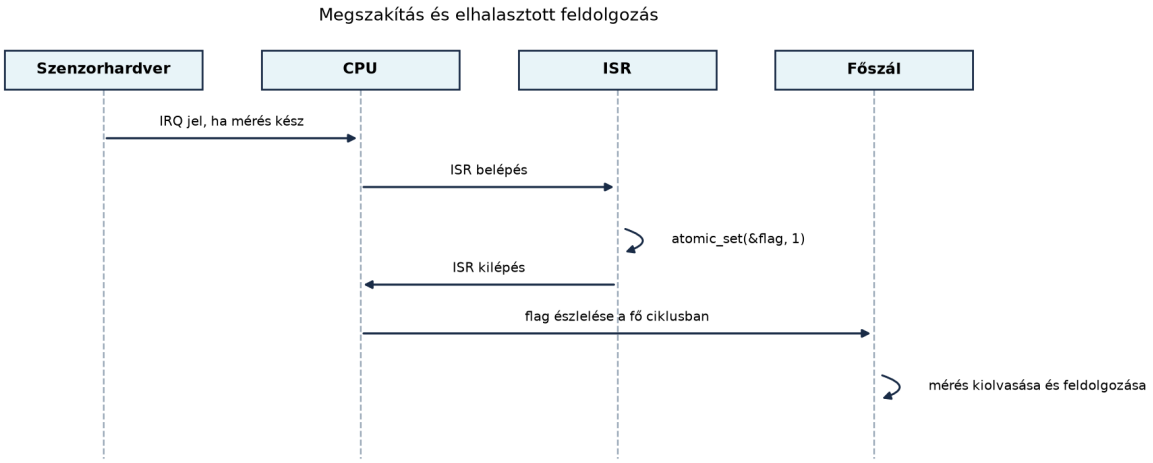
A mikrovezérlőben futó program normál végrehajtását hardveres események szakíthatják meg: egy GPIO-vonal szintet vált, egy időzítő lejár, egy ADC-átalakítás befejeződik, vagy egy soros port bájtot vesz. Az ilyen eseményre a CPU félbeszakítja az aktuális utasításfolyamot, elmenti a

visszatéréshez szükséges állapotot, és egy meghatározott címen, az úgynevezett megszakításvektor-táblában található rutinra ugrik. Ezt a rutint megszakításkezelőnek (Interrupt Service Routine, ISR) nevezzük.

Definíció. A **megszakítás** (interrupt) olyan aszinkron hardveres vagy szoftveres jel, amely az aktuálisan futó program végrehajtását félbeszakítja, és egy rögzített kezelőrutinra adja a vezérlést. A megszakításkezelő (ISR) az a rövid, nem blokkoló függvény, amely a megszakítás okát lekezeli, és a további munkát általában a főszálra halasztja.

A megszakításoknak általában **prioritásuk** van. Ha egyszerre több megszakítás van függőben, a megszakításvezérlő a magasabb prioritásút szolgálja ki előbb. Sok mikrovezérlőnél egy magasabb prioritású megszakítás megszakíthatja egy alacsonyabb prioritású ISR futását is; ezt nevezzük beágyazott megszakításkezelésnek. A `ts-node` esetében például a rádió vételi megszakítása kaphat magasabb prioritást, mint a szenzor kész jelzése, hogy a kommunikációs időzítés ne csússzon meg.

Az esemény bekövetkezésétől az ISR első utasításának végrehajtásáig eltelt időt **megszakítási késleltetésnek** (interrupt latency) nevezzük. Ez hardveres tényezőkből (a CPU-nak be kell fejeznie az aktuális utasítást, el kell mentenie a regisztereket) és szoftveres tényezőkből áll (például a kernel ideiglenesen letilthatja a megszakításokat). A késleltetés nagysága és ingadozása (jitter) kritikus lehet valós idejű vezérlésnél, ezért az ISR-t úgy tervezzük, hogy a legrövidebb idő alatt végezzen.



4.4. ábra: Megszakítás és elhalasztott feldolgozás - A hardveres eseménytől a főszáli feldolgozásig. Az ISR csak egy atomi jelzést állít be, a tényleges munkát a főszál vagy a work queue végzi.

Az ábrán látható, hogy a hardveres esemény kiváltja az ISR-t, az ISR azonban csak egy atomi jelzést állít be, majd azonnal visszatér. A tényleges munkát a főszál végzi el, amikor legközelebb ellenőrzi a jelzést. Ez a minta a későbbi Szinkronizáció és üzenetátadás: mutex, szemafor, üzenetsor, work queue alfejezetben részletesen is visszaköszön.

4.3.3 ISR-szabályok

A megszakításkezelő kontextusában nem ugyanazok az erőforrások állnak rendelkezésre, mint egy normál szálban. A legfontosabb szabályok:

1. **Az ISR legyen rövid.** Mivel futása közben az azonos vagy alacsonyabb prioritású megszakítások késnek, minden felesleges munka rontja a rendszer válaszidejét.
2. **Az ISR ne blokkoljon.** Nem hívhat olyan függvényt, amely várakozhat, például `sleep`, `mutex_lock`, `sem_take` vagy olyan soros port kiírást, amely kiüríti a puffert.
3. **Az ISR ne végezzen dinamikus memóriafoglalást.** A `malloc` és társai tipikusan nem megszakításbiztosak, és a foglalás várakozhat, ezért használatuk az ISR-ben hibás működéshez vezethet.
4. **Az ISR és a főszál közötti megosztott adathozférést szinkronizálni kell.** A közönséges változó módosítása nem atomi, ezért versenyhelyzet alakulhat ki.

Az első három szabály együtt azt jelenti, hogy az ISR tipikusan csak egy esemény bekövetkeztét jelzi, például beállít egy flaget, növel egy számlálót, vagy üzenetet helyez egy ISR-biztos várakozási sorba. A hosszabb feldolgozást a főszálra vagy egy dedikált munkaszálra kell halasztani.

4.3.4 Adatmegosztás ISR és szál között

A megszakításkezelő és a főprogram közös memórián osztozik, ezért bármely változót, amelyet az egyik oldal ír, a másik pedig olvas, versenyhelyzet fenyeget. Tipikus példa a `ts-node`-ban egy mérési számláló növelése:

```
volatile unsigned int sample_count = 0;

void sensor_ready_isr(void)
{
    sample_count++;   /* veszélyes! */
}
```

A `sample_count++` több gépi utasításból állhat: a CPU betölti az értéket egy regiszterbe, növeli, majd visszaírja. Ha a főszál ugyanekkor olvassa a változót, előfordulhat, hogy a növelés két lépése közötti köztes állapotot látja, vagy a megszakítás a főszál visszaírása elé lép, és így egy növelés elveszik.

Fontos félreértés: a `volatile` kulcsszó önmagában nem oldja meg a versenyhelyzetet. A `volatile` csak azt mondja a fordítónak, hogy a változót minden használatkor ténylegesen a memóriából olvassa be, és ne optimalizálja el a hozzáférést. Nem teszi a hozzáférést atomossá, és nem ad szinkronizációs garanciát a konkurens végrehajtásra.

A megoldás két fő eszköze:

- **Atomi műveletek.** A C11 szabvány `atomic_uint` típusa és a hozzá tartozó `atomic_fetch_add`, `atomic_load`, `atomic_store` függvények garantáltan oszthatatlanul hajtódnak végre. Ez pontosan arra való, hogy egy ISR-ben növeljünk egy számlálót, és a főszál biztonságosan olvashassa. A Zephyr saját atomi API-ja (`sys/atomic.h`) ettől különálló: `atomic_t` típust és `atomic_inc()`, `atomic_get()`, `atomic_set()`, `atomic_cas()` függvényeket használ; a két API-t egy kódon belül ne keverjük.

```
#include <stdatomic.h>

static atomic_uint sample_count = ATOMIC_VAR_INIT(0);

void sensor_ready_isr(void)
{
    atomic_fetch_add(&sample_count, 1);
}

void main_loop(void)
```

```
{
    unsigned int current;
    current = atomic_load(&sample_count);
    /* feldolgozás */
}
```

- **Kritikus szakasz.** Ha egy rövid kódrészletben biztosítani kell, hogy az adott szakaszt megszakítás ne szakíthassa félbe, akkor a megszakításokat lokálisan letiltjuk a szakasz elején, majd visszaállítjuk a korábbi engedélyezési állapotot a végén. Ez a módszer csak nagyon rövid, néhány utasításnyi szakaszra használható, mert a megszakítások tiltása növeli a késleltetést, és hosszabb letiltás esetén elveszhetnek hardveres események.

4.3.5 Elhalasztott munka: work queue előzetese

Miután az ISR beállította a jelzést, a főszálnak valahogyan meg kell tudnia, hogy munka vár rá. A legegyszerűbb megoldás a pollozás, amikor a fő ciklus minden iterációjában ellenőrzi az atomi flaget. Ez egyszerű, de nem skálázható jól, ha több megszakításforrás is van, és a főszál hosszabb munkát végez.

A Zephyr kernelben a **work queue** (munkasor) egy kernel-szálat foglal magában, amelyhez munkaelemeket (work item) lehet beküldeni. Az ISR beküldhet egy ilyen elemet, és amikor a work queue szál fut, végrehajtja a munkát szálkontextusban, ahol már lehet blokkolni, dinamikus memóriát foglalni, vagy lassabb perifériákkal beszélni. Ez a mechanizmus a későbbi Szinkronizáció és üzenetátadás: mutex, szemafor, üzenetsor, work queue alfejezetben kap részletes tárgyalást; itt csak az elvet mutatjuk be.

```
#include <zephyr/kernel.h>

static void process_sensor(struct k_work *work)
{
    ARG_UNUSED(work);
    /* Művelet szálkontextusban: szenzor kiolvasása, szűrés, telemetria frissítése. */
}

/* A munkaelem definiálása és inicializálása a kezelőfüggvénnyel. */
K_WORK_DEFINE(sensor_work, process_sensor);

void sensor_ready_isr(void)
{
    k_work_submit(&sensor_work);
}
```

Ebben a példában az ISR nem is foglalkozik a szenzoradat feldolgozásával, csak beküldi a `sensor_work` elemet a rendszer work queue-jába. A munkát a work queue szál hajtja végre, így az ISR rövid és nem blokkoló marad. A munkaelemet mindenképpen inicializálni kell (`K_WORK_DEFINE` vagy `K_WORK_INITIALIZER`); egy nullázott `struct k_work` használata definiálatlan viselkedés. Fontos a **beküldések összeolvadása** is: ha a munkaelem már várakozik vagy éppen fut, a `k_work_submit()` ismételt hívása nem tesz be újabb példányt a sorba, így az ISR-ből érkező több eseményből csak egy feldolgozás lesz. Ha minden eseményt fel kell dolgozni, a munkaelem mellé tegyünk egy atomi számlálót, amelyet az ISR növel, és a worker a kiolvasás-törléssel kapja meg a feldolgozandók számát.

4.3.6 Összevetés Linux-szignálokkal és szálakkal

A Linux alatt programozó olvasó számára a megszakítás első közelítésben hasonlít a **szignálkezelőre** (signal handler). Mindkettő aszinkron, félbeszakítja a normál futást, és mindkettőre szigorú korlátozások vonatkoznak: a szignálkezelőből csak úgynevezett async-signal-safe függvények hívhatók, és nem szabad a futást blokkolni. A megszakítás azonban hardveres szinten dolgozik, és nem szálkontextusban fut. A veremhasználat architektúra- és konfigurációfüggő: Cortex-M magokon a handler módban a CPU az MSP-t (Main Stack Pointer) használja, és a Zephyr külön megszakítási vermet állít be számára, így az ISR jellemzően nem a megszakított szál vermére épül; más architektúrákon vagy konfigurációkban viszont előfordulhat, hogy a megszakítás az aktuális veremre fut. Emiatt nem szabad az ISR-ben nagy helyi tömböket használni vagy a vermet mélyen megterhelni, és a szigorú „rövid, nem blokkoló” szabály minden esetben érvényes. A szignál kézbesítése és a kezelő lefutása között a kernel szálütemezése zajlik, míg a mikrovezérlő megszakítása közvetlenül a CPU hardveres prioritási mechanizmusán keresztül történik, és tipikusan jóval kisebb a késleltetése. A Zephyr-szál később olyan lesz, mint egy pthread, de az ISR nem szál, ezért a szálak közötti szinkronizációs primitívek nagy része nem használható benne.

4.3.7 Kidolgozott példa

Az alábbi Zephyr-kódrészlet a `ts-node` szenzor kész jelzését kezeli. A `sensor_drdy_cb` függvény megszakítás-kontextusban fut, ezért csak egy atomi flaget állít be. A főszál a `sample_req` flaget egyetlen atomi kiolvasás-törléssel (`atomic_clear()`) ellenőrzi, és ha volt esemény, elvégzi a tényleges szenzorolvasást. A példa célja a minta bemutatása, a konkrét GPIO- és szenzor-API a Szenzorolvasás a Zephyr sensor API-val alfejezetben jelenik meg.

```
#include <zephyr/kernel.h>
#include <zephyr/drivers/gpio.h>
#include <zephyr/sys/atomic.h>

/* Atomi flag: 0 = nincs minta, 1 = minta kész. */
static atomic_t sample_req;

/* Megszakítás-kontextusban futó GPIO-callback. */
static void sensor_drdy_cb(const struct device *dev,
                           struct gpio_callback *cb,
                           uint32_t pins)
{
    ARG_UNUSED(dev);
    ARG_UNUSED(cb);
    ARG_UNUSED(pins);

    /* Csak az eseményt jelezzük, munkát nem végzünk itt. */
    atomic_set(&sample_req, 1);
}

void main(void)
{
    atomic_set(&sample_req, 0);

    /* A GPIO-pin és a callback konfigurációja itt történne.
     * A valós Zephyr-kódban a gpio_pin_configure() és a
     * gpio_init_callback() hívások után a gpio_add_callback()
     * aktiválja ezt a megszakítást. */

    while (1) {
        /* Az atomic_clear egyetlen atomi műveletben olvassa ki és törli
         * a jelzőt; a visszaadott előző érték 1, ha érkezett esemény. */
```

```
        if (atomic_clear(&sample_req) != 0) {
            /* Itt történik a tényleges szenzorolvasás és feldolgozás. */
            read_and_process_sensor();
        }

        /* Egyéb háttérfeladatok, például telemetria küldése. */
        k_sleep(K_MSEC(10));
    }
}
```

Az `atomic_clear()` egyetlen atomi műveletben olvassa ki és törli a jelzőt, ezért nem veszhet el esemény a kiolvasás és a törlés között. Ez fontos: az `atomic_get()` és az utána végzett `atomic_set(&sample_req, 0)` két külön atomi művelet, amelyek közé beékelődhet egy megszakítás, így a jelző elveszhetne. Az ISR-esemény és a főszál közötti jelzésre tehát ne „olvasd ki, majd töröld” mintát használjunk, hanem atomi cserét (`atomic_clear()`), `atomic_cas()`-ciklust, vagy inkább számlálót, szemafort, illetve üzenetsort. A callback emellett nem végez szenzorolvasást, nem foglal memóriát, és nem blokkol. A `read_and_process_sensor()` függvény már szálkontextusban fut, így ott már lehetőség van lassabb műveletekre is.

4.3.8 Ellenőrzés és hibakeresés

A megszakításkezelő helyességét Zephyr `native_sim` környezetben úgy ellenőrizhetjük, hogy a callback-ben növelünk egy atomi számlálót, és a főszálból naplózzuk az értékét. Ha a callback valóban lefutott, a számláló növekszik, anélkül hogy a fő ciklus aktívan pollozná a hardvert. Tipikus hibatünetek és javításuk:

Tünet	Lehetséges ok	Javítás
A főszál soha nem látja a flaget	A változó nem atomi, vagy `volatile` hiányzik, a fordító optimalizál	Használjunk `atomic_t`-t vagy C11 atomi típust, ne hagyatkozzunk `volatile`-ra
A rendszer időnként lefagy, vagy elvesznek megszakítások	Az ISR blokkoló hívást végez	Távolítsunk el minden várakozó függvényt az ISR-ből, halasszuk a munkát work queue-ba
A számláló értéke néha kisebb a vártnál	A `counter++` nem atomi, versenyhelyzet	Használjunk `atomic_inc()`-et vagy kritikus szakaszt
A megszakítás váratlanul késik	Alacsony prioritás, vagy hosszú kritikus szakasz a főszálban	Emeljük a prioritást, és minimalizáljuk a megszakítás-tiltásos szakaszokat

A `native_sim` szimulátorban a megszakítások időzítése nem azonos a valódi hardverével, ezért a mért késleltetést csak irányszerűnek tekintsük, és valós hardveren is ellenőrizzük, ha a követelmény szigorú.

4.3.9 Tipikus hibák és félreértések

1. **A `volatile` mint szinkronizációs eszköz.** Sokan azt gondolják, hogy a `volatile` elég az ISR és a főszál közötti változókhoz. A `volatile` azonban csak a memóriából történő olvasást kényszeríti ki, nem teszi atomossá a hozzáférést. Egy `volatile unsigned int count = 0; count++;` versenyhelyzetet okozhat.

2. **Blokkoló hívás az ISR-ben.** A `printf` vagy `k_sleep` egy megszakításkezelőben várakozhat, ami megszakításkontextusban nem megengedett. A helyes gyakorlat az esemény jelzése és a munka work queue-ba szervezése.

3. **Túl hosszú kritikus szakasz.** A megszakítások globális tiltása egyszerűnek tűnik, de ha a kritikus szakasz több tíz vagy száz utasításból áll, a rendszer elveszíthet gyors megszakításokat. A kritikus szakasz csak néhány utasításra való.

4.3.10 Feladatok

1. **Fogalmi kérdés.** Magyarázd el, miért nem elegendő a `volatile` kulcsszó a megszakításkezelő és a főszál közötti adatmegosztáshoz. Milyen esetben lenne elég?

2. **Tervezési feladat.** Egy mikrovezérlő órajele 80 MHz. A megszakítás belépéséhez a hardver 12 ciklust igényel, a kontextusmentés 8 ciklust, a kilépés és visszaállítás pedig 10 ciklust vesz igénybe. Mekkora a minimális megszakítási késleltetés és a teljes ISR-átkapcsolási idő? Hány mikroszekundumot vehet igénybe az ISR törzse, ha a teljes kiszolgálási idő nem haladhatja meg a 10 µs-ot?

3. **Implementációs feladat.** A fenti kidolgozott példában a `sample_req` flag egy bitje csak egyetlen eseményt tárol. Módosítsd a kódot úgy, hogy az ISR egy atomi számlálót növeljen (`atomic_inc`), és a főszál a számláló értékét biztonságosan kiolvasva, majd nullázva gyűjtse be az összes elmaradt mintát. A számláló nullázásához használj atomi cserét.

4. **Hibakeresési feladat.** Az alábbi kód egy hibás megszakításkezelőt mutat. Melyik szabályokat sérti? Javítsd ki a kódot a work queue használatával.

```
void bad_isr(void)
{
    uint8_t *buffer = malloc(64);
    if (buffer == NULL) {
        return;
    }
    read_sensor_blocking(buffer, 64);
    printf("sensor data: %s\n", buffer);
    free(buffer);
}
```

4.3.11 Megoldások

1. A `volatile` csak azt biztosítja, hogy a fordító minden hozzáférést memóriaműveletként végez, és nem tartja a változót regiszterben. Nem ad atomosságot: egy többszavas változó írása vagy olvasása közben a megszakítás félig friss állapotot láthat. A `volatile` önmagában akkor lehet elég, ha a megosztott adat egyetlen bájt vagy gépi szó, és a hardver garantálja az ilyen méretű hozzáférések atomosságát, de hordozható kódban ezt nem érdemes feltételezni. A biztos megoldás az atomi típus vagy a kritikus szakasz.

2. A minimális megszakítási késleltetés a belépés és a kontextusmentés ideje: $12+8 = 20$ ciklus. A teljes átkapcsolási idő a belépés, mentés, kilépés és visszaállítás összege: $12+8+10 = 30$ ciklus. Egy ciklus ideje $\frac{1}{80\ \text{MHz}} = 12{,}5$ ns. A késleltetés tehát $20 \cdot 12{,}5$ ns $= 250$ ns, a teljes átkapcsolási idő $30 \cdot 12{,}5$ ns $= 375$ ns. Ha a teljes kiszolgálási idő legfeljebb 10 µs, akkor az ISR törzse legfeljebb $10\ \mu\text{s} - 0{,}375\ \mu\text{s} = 9{,}625\ \mu\text{s}$ lehet, ami 770 ciklusnak felel meg.

3. A módosított kód a következő:

```c
#include <zephyr/kernel.h>
#include <zephyr/sys/atomic.h>

static atomic_t sample_count;

static void sensor_drdy_cb(const struct device *dev,
                           struct gpio_callback *cb,
                           uint32_t pins)
{
    ARG_UNUSED(dev);
    ARG_UNUSED(cb);
    ARG_UNUSED(pins);

    atomic_inc(&sample_count);
}

void main(void)
{
    atomic_set(&sample_count, 0);

    while (1) {
        atomic_val_t pending = atomic_set(&sample_count, 0);
        /* pending az atomi csere előtti érték, azaz a minták száma. */
        for (atomic_val_t i = 0; i < pending; i++) {
            read_and_process_sensor();
        }

        k_sleep(K_MSEC(10));
    }
}
```

Itt az `atomic_set` visszaadja a korábbi értéket, és nullázza a számlálót egyetlen atomi lépésben, így nem veszik el a csere közben érkező megszakításból származó növelés.

4. A `bad_isr` három szabályt sért: dinamikus memóriafoglalást (`malloc`) végez megszakításkontextusban, blokkoló szenzorolvasást hív, és `printf`-et használ, amely szintén blokkolhat. A javított verzió egy work queue munkaelemet ütemez, és az összes érdemi munkát szálkontextusba helyezi. A `read_sensor_blocking()` itt a beolvasott bájtok számát adja vissza, hiba esetén negatív értéket; a nyers szenzoradat nem feltétlenül nulterminált szöveg, ezért `%s` helyett hexadecimális dumpot írunk ki:

```c
#include <zephyr/kernel.h>

static void sensor_worker(struct k_work *work)
{
    ARG_UNUSED(work);

    uint8_t buffer[64];
    int rc = read_sensor_blocking(buffer, sizeof(buffer));

    if (rc < 0) {
        printk("szenzorolvasasi hiba: %d\n", rc);
        return;
    }

    printk("sensor data (%d byte):", rc);
    for (int i = 0; i < rc; i++) {
        printk(" %02x", (unsigned int)buffer[i]);
    }
    printk("\n");
}
```

```c
static struct k_work sensor_work = K_WORK_INITIALIZER(sensor_worker);

void good_isr(void)
{
    /* A munkaelemet a work queue szál hajtja végre szálkontextusban. */
    k_work_submit(&sensor_work);
}
```

A munkaelem inicializálása (`K_WORK_DEFINE` vagy `K_WORK_INITIALIZER`) kötelező; egy nullázott `struct k_work` nem használható. A `k_work_submit()` megszakításból is biztonságosan hívható, és ha a munka még nem futott le, az ismételt beküldések egyetlen végrehajtásba olvadnak össze; ezért ha minden eseményt fel kell dolgozni, a munkaelem mellett egy atomi számlálót is vezessünk, és a worker a számláló kiolvasás-törlésével dolgozza fel az összeset.

4.3.12 Rövid összefoglaló

A megszakítás aszinkron hardveres esemény, amely félbeszakítja a futó programot, és egy rövid, nem blokkoló ISR-re adja a vezérlést. Az ISR-nek minimalizálnia kell a futási időt, nem végezhet dinamikus memóriafoglalást és nem blokkolhat. Az ISR és a szál közötti adatmegosztáshoz atomi változókat vagy rövid kritikus szakaszt kell használni; a `volatile` önmagában nem elegendő a versenyhelyzet elkerülésére. A hosszabb feldolgozást work queue-ba kell elhalasztani, amely szálkontextusban hajtja végre a munkát. **Továbbvitt eredmény.** Az itt bemutatott ISR-szabályok és az elhalasztott munka mintája közvetlenül épül tovább a Szinkronizáció és üzenetátadás: mutex, szemafor, üzenetsor, work queue alfejezetben, ahol a work queue részletesen előkerül, valamint a Szenzorolvasás a Zephyr sensor API-val alfejezetben, ahol a `ts-node` valós szenzor callback-jeit implementáljuk.

4.3.13 Források és ellenőrzési hiányok

A megszakítások és a megszakításkezelés alapelvei a mikrokontroller-architektúrák stabil fogalmai. A Zephyr atomi API (`sys/atomic.h`), a work queue API (`k_work_submit`) és a GPIO-callback konfiguráció pontos függvényneveit és paramétereit a felhasznált Zephyr-verzió dokumentációjában ellenőrizni kell, mert ezek verziófüggőek lehetnek. A példakód a minta bemutatását szolgálja, a Zephyr build- és konfigurációs lépéseket a későbbi alfejezetekben dolgozzuk ki.

5 RTOS és Zephyr

5.1 Miért RTOS? A Zephyr felépítése és az első alkalmazás

5.1.1 Motiváció

A TerraSense üvegházában a talajból mért adatok és a szelepvezérlés már korántsem fér el egyetlen jól megszerkesztett `while (1)` ciklusban. A csomópontnak egyszerre kell mintát vennie, rádiós kapcsolatot tartania, parancsokat fogadnia, és még a saját hibáját is figyelnie. Ha mindezt egyetlen végtelen ciklusba írjuk, akkor a leglassabb feladat – például egy hosszú rádiós várakozás – az összes többi feladatot is feltartja. Az ilyen rendszereknél nem elég gyorsnak lenni, hanem **időben determinisztikusan** kell reagálni. Ezt a problémát oldja meg egy valós idejű operációs rendszer (Real-Time Operating System, RTOS), amely az eszköz erőforrásait szálakra osztja, és futás közben ütemezi azokat. A TerraSense-ben a Zephyr RTOS-t használjuk, mert a `native_sim` céllal valódi hardver nélkül is teljes rendszert fejleszthetünk és tesztelhetünk.

5.1.2 Fogalmak és működés

5.1.2.1 Super loop vagy szálak?

Definíció (super loop). A super loop olyan programstruktúra, amely egyetlen végtelen ciklusban, sorban hívja meg az összes feladatot. Nincs ütemező, nincs párhuzamos végrehajtás, a feladatok sorrendje rögzített.

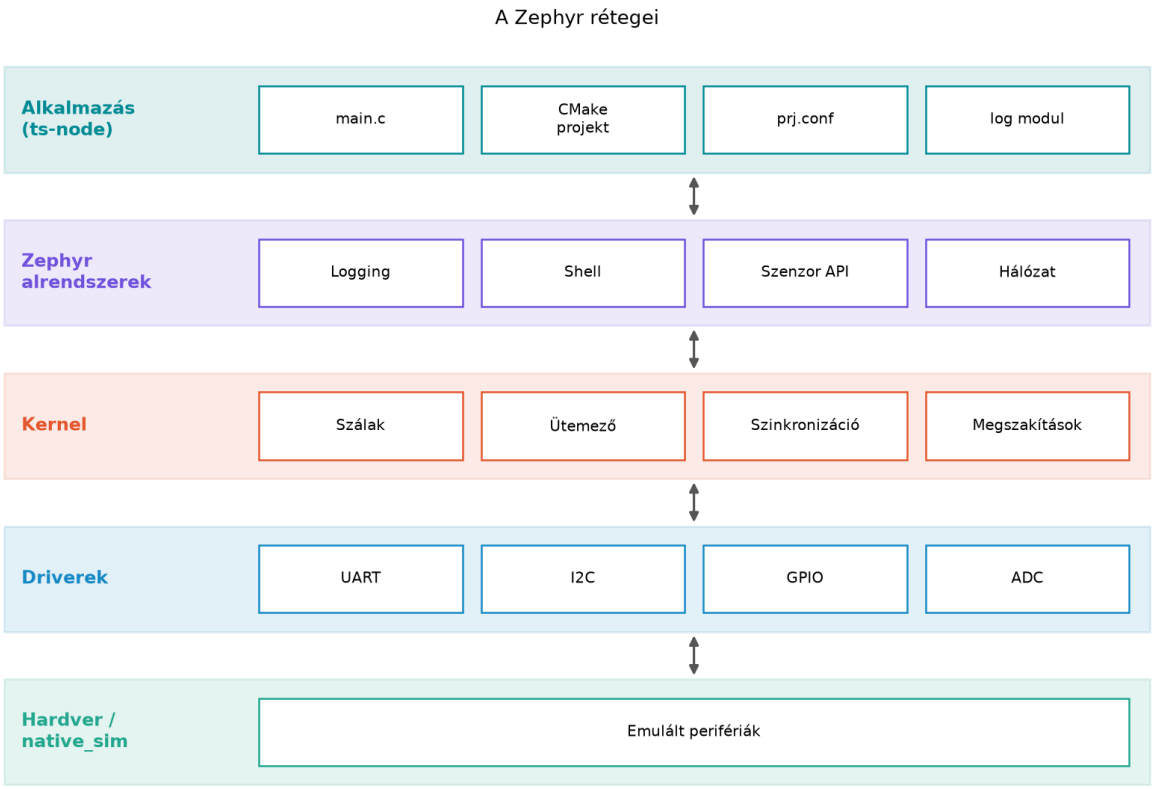
Definíció (RTOS). Az RTOS olyan operációs rendszer, amely több, egymástól független végrehajtási szálat kezel, és a szálakat prioritásuk vagy időzítéseik szerint ütemezi. Az RTOS célja, hogy a kritikus eseményekre a válaszidő felső korlátja ismert és betartható legyen.

Definíció (valós idejűség). A valós idejűség (real-time) nem azt jelenti, hogy a rendszer gyors, hanem azt, hogy a feladatok válaszideje **determinisztikus**: meg tudjuk mondani, hogy egy adott eseményre legfeljebb mennyi időn belül reagál a rendszer. Ha ez a korlát nem teljesül, a rendszer meghibásodik.

A super loop egyszerű, csökkenti a bonyolultságot, és kis erőforrásokon is elfut. Hátránya, hogy a legrosszabb esetben a válaszidő a leghosszabb feladat idejének és a többi feladat idejének összege. Az RTOS ezt úgy oldja fel, hogy különböző feladatokat különböző szálakba szervez, és az ütemező dönti el, melyik futhat. Egy magasabb prioritású szál azonnal megszakíthatja az alacsonyabb prioritású szálat, így a kritikus válaszidő már nem függ a nem kritikus feladatok hosszától.

5.1.2.2 A Zephyr felépítése

A Zephyr RTOS moduláris, Linux-szerűen felépülő rendszer. A kernel biztosítja a szálakat, az ütemezőt, a megszakításkezelést és a szinkronizációs objektumokat. A fölötte lévő alrendszerek – például a naplózás, a shell, a szenzor API, a hálózati stack, a Bluetooth stack – mind ezt a magot használják. A driverek a hardverhez vagy a `native_sim` perifériáihoz biztosítanak egységes hozzáférést.



5.1. ábra: A Zephyr rétegei - Az alkalmazás a Zephyr API-t hívja meg; a kernel szálakat ütemez, a driverek a hardvert vagy a native_sim emulációt kezelik.

A rétegek között felfelé mutató függőség van: az alkalmazás a Zephyr szolgáltatásait hívja, a kernel a drivereket vezérli, a driverek pedig a hardverhez vagy a szimulátorhoz férnek hozzá. A `native_sim` board az alsó réteget emulálja, így ugyanannak a kódnak a viselkedését valós hardver nélkül is vizsgálhatjuk.

Definíció (west). A `west` a Zephyr hivatalos meta-eszköze. Egyetlen felületen kezeli a forrás- és modulkezelést (`west init`, `west update`), a buildet (`west build`), a feltöltést (`west flash`) és a debugolást. A `west` a Zephyr workspace-ben dolgozik, és a `ZEPHYR_BASE` környezeti változón keresztül találja meg a Zephyr forráskódját.

5.1.2.3 Projektstruktúra

Egy minimális Zephyr-alkalmazás a következő fájlokból áll:

- `CMakeLists.txt` – a build belépési pontja. Megkeresi a Zephyr build rendszerét és definiálja az alkalmazás fordítási egységeit.
- `prj.conf` – a Kconfig beállítások gyűjteménye. Itt kapcsoljuk be a Zephyr alrendszereit, így a naplózást (`CONFIG_LOG`) és a shellt (`CONFIG_SHELL`).
- `src/main.c` – az alkalmazáskód belépési pontja. A `main` függvény egy szálként indul.
- `boards/<board>.overlay` – devicetree overlay, amellyel az adott board hardverleírását egészítjük ki. Az `i009` laborban ez még üres, a későbbi szenzorok és perifériák itt jelennek meg.

A `prj.conf` használatával a Zephyr konfigurációját az alkalmazás szintjén hangoljuk. A fordítási rendszer automatikusan beolvassa a projekt gyökerében lévő `prj.conf`-ot, ezért a fájlnak pontosan ezen a néven és helyen kell lennie.

5.1.2.4 Naplózás és shell

A Zephyr naplózó alrendszere futási időben gyűjti az üzeneteket. A modul regisztrálásához a `LOG_MODULE_REGISTER` makrót használjuk, amely automatikusan hozzárendeli a modul nevét és a beállított naplózási szintet. A `LOG_INF`, `LOG_DBG`, `LOG_WRN` és `LOG_ERR` makrók a printf-szerű formázást támogatják.

A beépített shell parancssori interfész, amely futás közben kérdezhető le. A shell engedélyezése a `prj.conf`-ban történik, és a shell parancsokkal például a kernel állapotát kérdezhetjük le (`kernel uptime`), vagy a naplózást vizsgálhatjuk. A shell különösen hasznos a `native_sim` fejlesztés során, mert a szimulált eszközt interaktívan tudjuk faggatni.

5.1.3 Döntési szempontok

A választás super loop és RTOS között az eszköz bonyolultságától és a kritikus időzítési követelményektől függ.

Szempont	Super loop	Zephyr RTOS
Erőforrásigény	minimális	nagyobb, de konfigurálható
Válaszidő	a ciklus legrosszabb ideje	prioritásos ütemezés, determinisztikus
Párhuzamosság	nincs	szálak, szemaforok, üzenetsorok
Bonyolultság	egyszerű	meredekebb tanulási görbe
Tipikus alkalmazás	egy feladatú, akkumulátoros minimál eszköz	hálózati protokollok, BLE, OTA, eszközmenedzsment

A TerraSense `ts-node`-nak a későbbiekben BLE-kapcsolatra, parancsfeldolgozásra és időzített szenzormintavételre lesz szüksége. Ez indokolja az RTOS használatát. A FreeRTOS alternatívát jelent, kisebb mérettel és egyszerűbb API-val, de a Zephyr beépített devicetree-támogatást, széles alrendszer-kínálatot (szenzor, hálózat, Bluetooth) és native_sim-en való futtatást kínál. A TerraSense-ben ezért a Zephyrt választjuk.

5.1.4 Kidolgozott példa és labor: `ts-node` projektváz

Munkakönyvtár: `~/iot-labor/`

```
mkdir -p ~/iot-labor/i009/ts-node/src ~/iot-labor/i009/ts-node/boards
cd ~/iot-labor/i009/ts-node
```

A labor célja egy futó Zephyr-alkalmazás, amely másodpercenként naplózza a „szenzormintát”, és shellt biztosít. A minták ebben a vázban még illusztratív értékek; a valódi szenzorolvasás a Szenzorolvasás a Zephyr sensor API-val alfejezetben kerül be.

Hozd létre a következő három fájlt.

`~/iot-labor/i009/ts-node/CMakeLists.txt`

```
cmake_minimum_required(VERSION 3.20.0)

find_package(Zephyr REQUIRED HINTS $ENV{ZEPHYR_BASE})
project(ts-node)

target_sources(app PRIVATE src/main.c)
```

A `find_package(Zephyr REQUIRED)` hívás csatolja a Zephyr build rendszerét. A `project(ts-node)` megadja a projekt nevét. A `target_sources(app PRIVATE src/main.c)` a fordítandó forrásfájlt regisztrálja; a `main.c` mellé később további forrásfájlokat is felvehetünk.

`~/iot-labor/i009/ts-node/prj.conf`

```
# TerraSense node alap konfiguráció
CONFIG_LOG=y
CONFIG_LOG_DEFAULT_LEVEL=3
CONFIG_SHELL=y
CONFIG_SHELL_BACKEND_SERIAL=y
CONFIG_PRINTK=y
```

A `CONFIG_LOG_DEFAULT_LEVEL=3` az `INFO` szintet jelenti. A `CONFIG_SHELL=y` és a `CONFIG_SHELL_BACKEND_SERIAL=y` a shellt a soros konzolon indítja; `native_sim` esetén ez a stdin/stdout-ra kötődik. A pontos backend nevek verziófüggőek lehetnek, ezért ellenőrizni kell a használt Zephyr kiadás dokumentációját.

`~/iot-labor/i009/ts-node/src/main.c`

```
#include <zephyr/kernel.h>
#include <zephyr/logging/log.h>

LOG_MODULE_REGISTER(ts_node, LOG_LEVEL_INF);

int main(void)
{
    uint32_t counter = 0;

    LOG_INF("TerraSense node starting");

    while (1) {
        k_sleep(K_SECONDS(1));
        LOG_INF("sample #%u: T=25.3 C, RH=61.2 %%", counter + 1);
        counter++;
    }

    return 0;
}
```

A `LOG_MODULE_REGISTER(ts_node, LOG_LEVEL_INF);` regisztrálja a `ts_node` naplózó modult `INFO` szinttel. A `k_sleep(K_SECONDS(1))` hívás a szálat egy másodpercre blokkolja, így a ciklus ütemezése a szimulált időben éppen egy másodperc. Mivel a várakozás a ciklus elején van, az első minta az indítás után 1 másodperccel jelenik meg, összhangban a lentebb közölt kimenettel. A `%%` arra szolgál, hogy a log formázó a százalékjelet százalékjelként értelmezze.

Hozd létre a devicetree overlay fájlt is, üresen, hogy a projektstruktúra teljes legyen.

`~/iot-labor/i009/ts-node/boards/native_sim.overlay`

```
/* TerraSense node: ebben a laborban nincs további devicetree overlay. */
```

5.1.4.1 Build és futtatás

A buildet a Zephyr workspace-ből indítjuk. A workspace pontos helye a A labor felállítása alfejezetben beállított `~/zephyrproject`; ha más, cseréld ki.

```
cd ~/zephyrproject
west build -d ~/iot-labor/i009/build -b native_sim ~/iot-labor/i009/ts-node
```

A `-d` kapcsoló adja meg a build könyvtárat, a `-b native_sim` a cél boardot, a forráskönyvtár pedig a projekt gyökere. A sikeres build után a futtatható állomány a `~/iot-labor/i009/build/zephyr/zephyr.exe`.

Futtasd a programot:

```
~/iot-labor/i009/build/zephyr/zephyr.exe
```

A várt kimenet (a verzió és az időbélyeg formátuma a buildtől függően változhat):

```
*** Booting Zephyr OS build v3.7.0 ***
[00:00:00.000,000] <inf> ts_node: TerraSense node starting
[00:00:01.000,000] <inf> ts_node: sample #1: T=25.3 C, RH=61.2 %
[00:00:02.000,000] <inf> ts_node: sample #2: T=25.3 C, RH=61.2 %
[00:00:03.000,000] <inf> ts_node: sample #3: T=25.3 C, RH=61.2 %
```

A program másodpercenként újabb sort ír ki. A futás közben a shell prompt is megjelenik, például `uart:~$`. Nyomj Entert, írd be a `help` parancsot, és Entert. A shell kilistázza az elérhető parancsokat, például a `kernel`, `log` és `device` parancsokat. A shell használata során a naplózó üzenetek továbbra is érkeznek, ezért a prompt és a logok keveredhetnek; ez a fejlesztői konzolon elfogadható. A program leállításához nyomj Ctrl+C-t.

Továbbvitt eredmény. A létrejött `~/iot-labor/i009/ts-node/` projektvázat, a `CMakeLists.txt` és `prj.conf` szerkezetet, valamint a `west build -b native_sim` folyamatot használja a Szálak és ütemezés, a Devicetree és Kconfig, a Szenzorolvasás a Zephyr sensor API-val és az Eszközszoftver-architektúra alfejezet. A későbbi laborok ebből a projektből indulnak ki, és a fájlokat további forrásfájlokkal, konfigurációs bejegyzésekkel és devicetree overlay-ekkel bővítik.

5.1.5 Ellenőrzés és hibakeresés

Tünet	Valószínű ok	Javítás
`west: command not found`	A west nincs a PATH-ban	Aktiváld a Zephyr környezetet, például `source ~/zephyrproject/.venv/bin/activate`
A build nem találja a `prj.conf`-ot	A forráskönyvtár helytelen	Ellenőrizd, hogy a `prj.conf` a projekt gyökerében van, és a `west build` megkapja ezt a könyvtárat
A log üzenetek nem jelennek meg	`CONFIG_LOG=n` vagy a log szint túl magas	`CONFIG_LOG=y` és `CONFIG_LOG_DEFAULT_LEVEL=3` a `prj.conf`-ban
A shell prompt nem jelenik meg	`CONFIG_SHELL` nincs bekapcsolva	`CONFIG_SHELL=y` és `CONFIG_SHELL_BACKEND_SERIAL=y`
A log időbélyegek gyorsabban nőnek, mint a falidő	`native_sim` szimulált időt használ, a `k_sleep` a szimulált kernel tickre vár	Ez várható; a szimuláció üteme függ a build beállításaitól

5.1.6 Tipikus hibák és félreértések

- **Valós idejűség = gyorsaság.** Sokan összekeverik a determinisztikus válaszidőt a gyors működéssel. Egy RTOS sem garantálja, hogy a rendszer gyors lesz, csak azt, hogy a kritikus határidők betarthatók.
- **Super loop-ban blokkoló hívás.** Ha egy `while (1)` ciklusban a rádió vagy a szenzor hosszan vár, a többi feladat késik. Az RTOS szálakat és prioritásokat ad, de a szálak közötti szinkronizációt továbbra is helyesen kell használni.
- **A `LOG_MODULE_REGISTER` elhagyása.** A napló makrók csak regisztrált modullal működnek; a makró hiánya fordítási hibát vagy ismeretlen modulnevet eredményez.
- **Alvás megszakításban.** A `k_sleep` csak szál kontextusban hívható. Ha egy megszakításban próbáljuk használni, a rendszer hibát jelez vagy lefagy.

5.1.7 Feladatok

5.1.7.1 (a) Fogalmi kérdés

Magyarázd el, mi a különbség a valós idejűség és a gyorsaság között. Adj egy TerraSense példát, ahol a határidő betartása fontosabb, mint az átlagos válaszidő.

5.1.7.2 (b) Tervezési feladat

Egy super loop ciklusban három feladat fut sorban: szenzorolvasás 20 ms, rádiókezelés 80 ms és parancsellenőrzés 10 ms. Mennyi a legrosszabb válaszidő egy parancs érkezésétől a feldolgozásáig? Hogyan változik ez, ha a parancsellenőrzést külön, magasabb prioritású RTOS-szálra bízzuk?

5.1.7.3 (c) Implementációs feladat

Módosítsd a `ts-node` vázat úgy, hogy a `prj.conf`-ban a naplózási szintet debugra (`CONFIG_LOG_DEFAULT_LEVEL=4`) állítod, és a `main.c`-ben indításkor egy `LOG_DBG` üzenetben kiírod a `site=gh1, node=n001` azonosítókat. Ez előkészíti a későbbi konfigurációs konvenciót, amelyet a Devicetree és Kconfig alfejezet fog bemutatni.

5.1.7.4 (d) Hibakeresési feladat

Kapcsold ki a naplózást a `prj.conf`-ban (`CONFIG_LOG=n`), fordítsd újra, és indítsd el a programot. Mi történik a log üzenetekkel és a shell prompttal? Magyarázd meg az eredményt.

5.1.8 Megoldások

5.1.8.1 (a)

A valós idejűség azt jelenti, hogy egy adott eseményre adott válasz felső korlátja ismert és betartható, függetlenül a rendszer terhelésétől. A gyorsaság ezzel szemben csak az átlagos válaszidőt vagy áteresztőképességet jellemzi. TerraSense példában az öntözőszelep vészleállító parancsának 100 ms-on belül le kell futnia; ha a rendszer néha 20 ms alatt, néha 500 ms alatt válaszol, az átlag nem számít, a határidő túllépése katasztrófát okozhat.

5.1.8.2 (b)

A super loop-ban a parancs csak akkor kerül feldolgozásra, ha a ciklus a parancsellenőrzéshez ér. A legrosszabb esetben a parancs a szenzorolvasás kezdetén érkezik, és meg kell várnia a szenzorolvasást (20 ms) és a rádiókezelést (80 ms), mielőtt a parancsellenőrzés (10 ms) elkezdődik. A legrosszabb válaszidő tehát $20 + 80 + 10 = 110$ ms. Ha a parancsellenőrzést külön szálra bízzuk, és az magasabb prioritású, mint a többi szál, akkor a parancs érkezésekor az ütemező felfüggeszti a szenzor- vagy rádiószálat, és a parancsszál azonnal futni kezd. A válaszidő ekkor már csak a parancs feldolgozásához szükséges 10 ms plusz a kontextusváltás ideje lesz (a kontextusváltás néhány mikroszekundum, amit itt elhanyagolhatunk).

5.1.8.3 (c)

Módosított `prj.conf`:

```
# TerraSense node alap konfiguráció
CONFIG_LOG=y
CONFIG_LOG_DEFAULT_LEVEL=4
CONFIG_SHELL=y
CONFIG_SHELL_BACKEND_SERIAL=y
CONFIG_PRINTK=y
```

Módosított `main.c`:

```c
#include <zephyr/kernel.h>
#include <zephyr/logging/log.h>

LOG_MODULE_REGISTER(ts_node, LOG_LEVEL_DBG);

int main(void)
{
    uint32_t counter = 0;

    LOG_INF("TerraSense node starting");
    LOG_DBG("site=gh1, node=n001");

    while (1) {
        k_sleep(K_SECONDS(1));
        LOG_INF("sample #%u: T=25.3 C, RH=61.2 %%", counter + 1);
        counter++;
    }

    return 0;
}
```

A `LOG_LEVEL_DBG` makró biztosítja, hogy a `LOG_DBG` üzenetek is megjelenjenek, ha a modul saját szintje ezt megengedi. A `prj.conf`-ban beállított default szint szintén debug kell legyen, mert a modul szintje felülbírálhatja a globális szintet.

5.1.8.4 (d)

Ha a `prj.conf`-ban `CONFIG_LOG=n`, a naplózó makrók üres utasításokká válnak, így a `LOG_INF` és a `LOG_DBG` üzenetek nem jelennek meg. A program fut, a shell továbbra is működik, mert a shell nem a napló alrendszertől függ, hanem a saját backendjétől. A shell parancsok viszont továbbra is elérhetők, és a `kernel uptime` például fut. Ez a viselkedés gyakori produkciós optimalizáláskor: a naplózás kikapcsolásával csökkenthető a kód mérete és a futási overhead.

5.1.9 Rövid összefoglaló

Ebben az alfejezetben felépítettük a TerraSense `ts-node` első Zephyr-alkalmazását. A `~/iot-labor/i009/ts-node/` projekt a `CMakeLists.txt`, a `prj.conf`, a `src/main.c` és a `boards/native_sim.overlay` fájlokból áll. A program másodpercenként naplózza az illusztratív szenzormintát, és shellt biztosít. A build a `west build -d ~/iot-labor/i009/build -b native_sim ~/iot-labor/i009/ts-node` paranccsal történik, a futtatható állomány a `~/iot-labor/i009/build/zephyr/zephyr.exe`. A projektváz a későbbi alfejezetek kiindulópontja: a szálak, a szinkronizáció, a devicetree, a szenzorok és az eszközszoftver-architektúra mind erre a vázra épül. A naplózás modul neve `ts_node`, az azonosító konvenció `site=gh1, node=n001`.

5.1.10 Források és ellenőrzési hiányok

- Zephyr dokumentáció: https://docs.zephyrproject.org/latest/ – ellenőrizendő a `west build -d` opció pontos szintaxisa és a shell backend nevek (`CONFIG_SHELL_BACKEND_SERIAL`) az aktuális Zephyr kiadásban.
- Zephyr `native_sim` board: https://docs.zephyrproject.org/latest/boards/native/native_sim/doc/index.html – ellenőrizendő a konzol és a shell viselkedése, valamint a `native_sim` szimulált idő kezelése.

- Zephyr logging alrendszer: https://docs.zephyrproject.org/latest/services/logging/index.html – ellenőrizendő a `LOG_MODULE_REGISTER` makró pontos paraméterezése és a log szint számértékei.
- Zephyr shell dokumentáció: https://docs.zephyrproject.org/latest/services/shell/index.html – ellenőrizendő a shell backend konfigurációs nevek aktuális formája.

5.2 Szálak és ütemezés

5.2.1 Motiváció

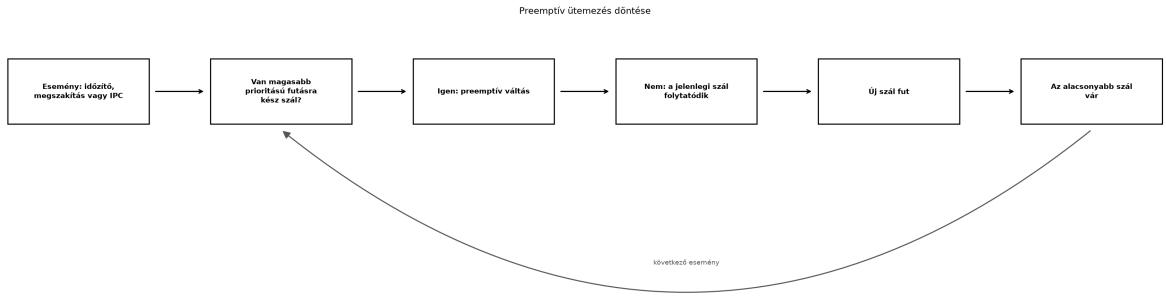
A Az RTOS és Zephyr alapok alfejezetben már lefordítottuk és elindítottuk az első Zephyr-alkalmazást. Egy valós TerraSense-csomópontban azonban több tevékenység fut egyidejűleg: a szenzort olvasni kell, az adatot elő kell készíteni a küldésre, a szelep állapotát figyelni kell, és közben a watchdogot is kezelni kell. A `main()` függvényben egyetlen „szuperhurok” (super loop) mindezt nehezen teszi időben helyessé. A Zephyr-szálak és az ütemező ezt a problémát úgy oldják meg, ahogyan egy Linuxos `pthread` tenné – azzal a különbséggel, hogy minden szál ugyanabban a címtérben fut, a verem mérete pedig fordítási időben rögzített.

Ebben az alfejezetben létrehozzuk a `ts-node` első két szálát: egy mérőszálat és egy küldőszálat. A mérőszál rendszeresen mintát vesz, a küldőszál pedig elkészíti az üzenetet. Ez a felépítés lesz az alapja a későbbi szinkronizációnak (Szinkronizáció és üzenetátadás) és a robusztus eszközszoftver-architektúrának (Eszközszoftver-architektúra).

5.2.2 Fogalmak és működés

Definíció. A Zephyrben a **szál** (thread) egy önálló végrehajtási kontextus, amelynek saját veremterülete, prioritása és belépési pontja van. A szálak ugyanabban a címtérben futnak, ezért a globális változókat közvetlenül elérik, de a hozzáférés szinkronizálást igényel.

Definíció. A **prioritás** (priority) azt határozza meg, hogy az ütemező melyik futásra kész szálat válassza ki. A Zephyrben **minél kisebb a szám, annál magasabb a prioritás**. A nemnegatív prioritású szálak **preemptívek**, a negatív prioritásúak **kooperatívak**. A preemptív szál megszakíthat egy alacsonyabb prioritású preemptív szálat, ha futásra kész állapotba kerül. A kooperatív szál csak akkor adja át a processzort, ha saját maga blokkol (például `k_sleep()`-pel) vagy kilép; más szál nem veheti el tőle a vezérlést.



5.2. ábra: Preemptív ütemezés döntése - Az ütemező minden eseménynél a legmagasabb prioritású futásra kész szálat választja.

Az ábra azt mutatja, hogy a Zephyr ütemezője minden esemény után újra értékeli a futásra kész szálakat. Ha egy magasabb prioritású szál felébred (például lejár az időzítője), akkor az alacsonyabb prioritású szál futása azonnal megszakad. Ezt nevezzük preemptív ütemezésnek.

A Zephyrben a szálak létrehozása általában kétféleképpen történhet: - **Statikus definíció** a `K_THREAD_DEFINE` makróval: a szálat a rendszer a boot során hozza létre, a verem mérete és a prioritás fordítási időben ismert. - **Dinamikus létrehozás** a `k_thread_create()` függvénnyel: a verem és a szálstruktúra memóriáját a hívó biztosítja.

A beágyazott eszközökön a statikus definíció az előnyösebb, mert a memóriahasználat már a linkeléskor ismert, és nincs dinamikus foglalási hiba.

Veremméret és ellenőrzés. A szál veremtája tartalmazza a futási kontextust (regiszterek, visszatérési címek) és a függvényhívások lokális változóit. A verem mérete fordítási időben rögzített, ezért a helyes méret megválasztása kritikus. A Zephyr két eszközt kínál a veremproblémák felderítésére: - **Stack sentinel** (`CONFIG_STACK_SENTINEL`): a verem végére egy ismert bájtot helyez, és ha azt a szál felülírja, a rendszer hibát jelez. - **Thread analyzer** (`CONFIG_THREAD_ANALYZER`): a `k_thread_stack_space_get(thread, &unused)` függvénnyel lekérdezhető, mennyi szabad veremterület maradt a megadott szálban; a második argumentum egy `size_t` kimeneti változó, a visszatérési érték pedig státuszkód. A méréshez `CONFIG_THREAD_STACK_INFO` és `CONFIG_INIT_STACKS` szükséges.

A verem méretét becsülni lehet az alábbi összefüggéssel:

$$S_{\text{total}} = S_{\text{context}} + S_{\text{max call depth}} + S_{\text{isr}} + S_{\text{margin}}$$

Ahol S_{context} a szálváltáskor mentett kontextus mérete (architektúrafüggő, jellemzően néhány száz bájt), $S_{\text{max call depth}}$ a legmélyebb hívási lánc lokális változóinak együttes mérete, S_{isr} a megszakítás-kezelők által a szál vermén használt ideiglenes terület, S_{margin} pedig biztonsági tartalék (például 30–50%). A pontos érték méréssel határozható meg; a becslés után mindig ellenőrizni kell a maradékot.

Időzítés. A szálak időzítésére két alapvető mechanizmus van: - `k_sleep()`: a hívó szálat adott időre blokkolja. Az alvó szál nem fogyaszt CPU-időt, és futásra kész állapotba kerül, amikor az idő lejár. - `k_timer`: aszinkron időzítő, amely egy callback függvényt hív meg a beállított időközönként. A callback megszakítás- vagy rendszerkontextusban fut, ezért nem blokkolhat, és csak rövid műveleteket végezhet.

A gyakorlatban a **jitter** az az időbeli bizonytalanság, amellyel egy periodikus szál ténylegesen futni kezd. A Zephyr szál `k_sleep()`-pel való periodikus futtatásánál a jittert a megszakítások késleltetése, az ütemező futási ideje és a magasabb prioritású szálak terhelése okozza. A `k_timer` callback jittere általában kisebb, de a callbackben végzett munka korlátozott. A `native_sim` környezet nem valós hardver, ezért az ott mért jitter csak becslésre alkalmas.

Prioritásinverzió. A prioritásinverzió akkor lép fel, ha egy magas prioritású szál egy olyan erőforrásra vár, amelyet egy alacsony prioritású szál tart foglalva, miközben egy közepes prioritású szál folyamatosan fut. Ezáltal a magas prioritású szál határozatlan ideig blokkolva marad, mintha a prioritása alacsonyabb lenne. A Zephyrben a **mutex** támogatja a **prioritásöröklést** (priority inheritance): amikor egy magas prioritású szál vár egy mutexre, az azt tartó szál ideiglenesen megkapja a magas prioritást, hogy minél hamarabb elengedje az erőforrást. A prioritásöröklés bekapcsolásához a megfelelő Kconfig opcióra van szükség (ellenőrzési igény: a Zephyr verziójában a mutex prioritásöröklés opció pontos neve változhat). A mutexek részletes tárgyalása a Szinkronizáció és üzenetátadás alfejezetben következik.

5.2.3 Döntési szempontok

A szálak és az ütemezés tervezésekor a következő fő kérdéseket kell eldönteni:

Szempont	Preemptív szál	Kooperatív szál
Vezérlés átadása	Automatikus, magasabb prioritású szál közbeavatkozhat	Csak explicit blokkolással (`k_sleep`, `k_yield`)
Alkalmas	Időérzékeny események, rövid futás	Hosszú, nem megszakítható szakaszok, egyszerű állapotgépek
Előny	Jó válaszidő, rugalmas ütemezés	Kiszámítható, nincs meglepetésszerű megszakítás
Hátrány	Több megosztott állapotot kell szinkronizálni	Rosszul megírt szál blokkolhatja a rendszert

A Zephyrben a prioritásokat a `K_PRIO_PREEMPT(n)` és `K_PRIO_COOP(n)` makrókkal szokás megadni. A `n` érték konkrét tartománya a konfigurációtól függ. A tipikus `ts-node`-ban a mérőszál preemptív, mert a mérési pontosság érdekében időben futnia kell; a küldőszál szintén preemptív, de alacsonyabb prioritással, mert a hálózati küldés kevésbé időérzékeny. A nagyon rövid, egyszerű feladatokhoz kooperatív szál is használható, de a valós idejű rendszerben a preemptív szál a gyakoribb választás.

5.2.4 Kidolgozott példa: mérő- és küldőszál a `ts-node`-ban

Munkakönyvtár: `~/iot-labor/i010/`. Előfeltétel: az Első Zephyr-alkalmazás laborban létrehozott `ts-node` projekt. Másoljuk át azt a saját könyvtárunkba:

```
mkdir -p ~/iot-labor/i010 && cd ~/iot-labor/i010
cp -r ../i009/ts-node .
```

Lépjünk be a projekt könyvtárába, és hozzuk létre a `src/main.c` fájlt az alábbi tartalommal:

```
#include <zephyr/kernel.h>
#include <zephyr/sys/printk.h>

struct measurement {
    int32_t temp_c;
    uint32_t humidity_rh;
    uint32_t soil_moisture;
};

static struct measurement g_data;
static bool g_new_data;

/* A g_data és a g_new_data együttes védelme. */
K_MUTEX_DEFINE(g_data_lock);

void measure_thread(void *unused1, void *unused2, void *unused3)
{
    int32_t t = 20;
    size_t unused = 0;
    ARG_UNUSED(unused1);
    ARG_UNUSED(unused2);
    ARG_UNUSED(unused3);
```

```
    while (1) {
        t += 1;

        k_mutex_lock(&g_data_lock, K_FOREVER);
        g_data.temp_c = t;
        g_data.humidity_rh = 50;
        g_data.soil_moisture = 30;
        g_new_data = true;
        k_mutex_unlock(&g_data_lock);

        int rc = k_thread_stack_space_get(k_current_get(), &unused);

        if (rc == 0) {
            printk("[measure] temp=%d C, stack free=%u\n",
                   t, (unsigned int)unused);
        } else {
            printk("[measure] temp=%d C, stack free ismeretlen (rc=%d)\n",
                   t, rc);
        }

        k_sleep(K_SECONDS(1));
    }
}

void send_thread(void *unused1, void *unused2, void *unused3)
{
    ARG_UNUSED(unused1);
    ARG_UNUSED(unused2);
    ARG_UNUSED(unused3);

    while (1) {
        bool has_new_data = false;
        int32_t temp_c = 0;

        k_mutex_lock(&g_data_lock, K_FOREVER);
        if (g_new_data) {
            has_new_data = true;
            temp_c = g_data.temp_c;
            g_new_data = false;
        }
        k_mutex_unlock(&g_data_lock);

        if (has_new_data) {
            printk("[send] sending temp=%d C\n", temp_c);
        }

        k_sleep(K_MSEC(200));
    }
}

K_THREAD_DEFINE(measure_tid, 1024, measure_thread, NULL, NULL, NULL,
                K_PRIO_PREEMPT(5), 0, 0);
K_THREAD_DEFINE(send_tid, 1024, send_thread, NULL, NULL, NULL,
                K_PRIO_PREEMPT(10), 0, 0);

void main(void)
{
    printk("ts-node szal-labor indul\n");
}
```

A `prj.conf` tartalma:

```
CONFIG_PRINTK=y
```

```
CONFIG_STACK_SENTINEL=y
CONFIG_THREAD_ANALYZER=y
CONFIG_THREAD_STACK_INFO=y
CONFIG_INIT_STACKS=y
```

Magyarázat. A `K_THREAD_DEFINE` makró létrehozza a `measure_tid` és `send_tid` szálakat. Mindkét szál 1024 bájt vermet kap. A mérőszál prioritása `K_PRIO_PREEMPT(5)`, tehát magasabb, mint a küldőszálé (`K_PRIO_PREEMPT(10)`), mert a 5 kisebb, mint a 10. A mérőszál egymásodpercenként frissíti a globális `g_data` struktúrát és beállítja a `g_new_data` jelzőt. A küldőszál 200 milliszekundumonként ellenőrzi a jelzőt, és ha új adat van, kiírja a hőmérsékletet. A `k_thread_stack_space_get()` két argumentumot vár: az első a vizsgált szál (`k_current_get()`), a második egy `size_t *` kimeneti pointer, amelybe a szabad bájtok száma kerül; a visszatérési érték státuszkód (0 = siker, negatív = hiba), nem a szabad méret. A lekérdezéshez a `CONFIG_THREAD_STACK_INFO`, az értelmes „szabad” méréshez pedig a `CONFIG_INIT_STACKS` szükséges, ezért mindkettő szerepel a `prj.conf`-ban. A mérőszál a státuszkódot ellenőrzi, és csak sikeres lekérdezés esetén írja ki a szabad veremméretet.

A `g_data` és a `g_new_data` írása és olvasása ebben a példában `g_data_lock` mutexszel védett. A mérőszál a struktúra frissítését és a jelző beállítását egyazon kritikus szakaszban végzi, a küldőszál pedig ugyanabban a kritikus szakaszban olvassa ki az adatot és törli a jelzőt, így nem fordulhat elő, hogy egy részben frissített méréshez tartozó jelzőt lásson, vagy hogy a jelző törlése elveszítsen egy közben beállított frissítést. A mutex csak szálkontextusban használható, megszakításkezelőben nem; a részletes működést és a további primitíveket (szemafor, üzenetsor) a Szinkronizáció és üzenetátadás alfejezet tárgyalja. A `volatile` itt nem szükséges: a mutex adja a szükséges memóriagátat, a `volatile` pedig önmagában nem tenne atomossá egy többmezős frissítést.

A build és a futtatás:

```
cd ~/iot-labor/i010/ts-node
west build -b native_sim
./build/zephyr/zephyr.exe
```

Várt kimenet (a futás első néhány másodpercéből, illusztratív):

```
ts-node szal-labor indul
[measure] temp=21 C, stack free=768
[send] sending temp=21 C
[measure] temp=22 C, stack free=768
[send] sending temp=22 C
[measure] temp=23 C, stack free=768
```

A pontos `stack free` érték a Zephyr verziójától és a konfigurációtól függ; a fenti érték csak illusztráció. Azt ellenőrizhetjük, hogy a mérőszál körülbelül egymásodpercenként fut, a küldőszál pedig a beérkező új adatot továbbítja. A prioritás miatt a mérőszál ébredésekor azonnal preemptálja a küldőszálat, ha az éppen fut.

Kalibrálás a veremre. Ha a `stack free` mindig kényelmesen nagy (például 500 bájtnál több), a 1024 bájt elegendő. Csökkenthetjük a verem méretét, hogy memóriát takarítsunk meg, de közben figyeljük a `stack free` értéket. Ha a rendszer `stack sentinel` hibát jelez, a verem túl kicsi volt. Ezt a hibakeresés szakaszban ismertetjük.

5.2.5 Ellenőrzés és hibakeresés

A működés ellenőrzéséhez figyeljük a konzol kimenetét. A mérőszálnak pontosan egymásodpercenként kell futnia, a küldőszálnak pedig minden új mérési értéket egyszer kell továbbítania. A szálak futási sorrendjét a prioritások megváltoztatásával vizsgálhatjuk: ha a küldőszál prioritását magasabbra állítjuk, az gyakrabban fut, de ez nem befolyásolja a mérési adatokat.

A veremhasználatot a `k_thread_stack_space_get()` második, kimeneti argumentumában kapott értékkel ellenőrizzük; a visszatérési érték csak státuszkód. A stack sentinel aktiválásakor a veremtúlcsordulás rendszerhibát vált ki, és a Zephyr hibaüzenetet ír a konzolra. Ha ilyet látunk, növeljük a veremméreteket a `K_THREAD_DEFINE`-ban.

Hibatünet	Lehetséges ok	Javítás
A szál nem fut el, vagy hamar leáll	Túl kicsi verem, túlcsordulás, a rendszer leáll	Növeljük a stack méretét, ellenőrizzük a `k_thread_stack_space_get()` kimeneti értékét
`FATAL ERROR: Stack overflow`	A szál vermét túlcsordult a hívás	Növeljük a stack méretét, keressünk nagy lokális tömböket vagy mély rekurziót
A mérőszál nem egymásodpercenként fut	Más magasabb prioritású szál folyamatosan fut	Ellenőrizzük a prioritásokat, adjunk `k_sleep`-et a hosszú szálaknak
A küldőszál többször küldi ugyanazt az adatot	A `g_new_data` jelzőt és a `g_data` struktúrát nem ugyanaz a mutex védi, vagy a küldőszál nem törli a jelzőt	Használjunk közös mutexet az adat és a jelző védelmére, és a jelző törlése ugyanabban a kritikus szakaszban történjen
A `stack free` érték csökken	A hívási mélység nő, vagy hibásan nagy lokális változók	Vizsgáljuk a hívási láncot, szüntessük meg a nagy lokális tömböket

5.2.6 Tipikus hibák és félreértések

1. **A prioritásszámot fordítva értelmezik.** A Zephyrben a kisebb szám magasabb prioritást jelent, ellentétben néhány más rendszerrel. Ha a mérőszál prioritását 10-re, a küldőét 5-re állítjuk, a küldőszál lesz a magasabb prioritású, és a mérőszál késhet.

2. **A kooperatív szál nem blokkol.** Ha egy kooperatív szál végtelen ciklusban fut, és soha nem hív `k_sleep`-et vagy `k_yield`-et, az egész rendszer processzoridejét lefoglalja, és más szálak (még magasabb prioritású preemptívek is) nem kapnak futási időt. A Zephyrben a kooperatív szálak csak akkor adják át a vezérlést, ha ezt expliciten kérik.

3. **A stack méretét alulbecsülik.** A `printk` és más rendszerhívások is használják a hívó szál vermét, nem csak a saját lokális változók. A 512 bájt sokszor nem elég, még akkor sem, ha a saját kód csak néhány változót használ.

4. **A `k_timer` callbackben blokkoló hívást használnak.** A callback nem hívhat `k_sleep`-et vagy más blokkoló függvényt, mert az ütemező kontextusában fut. Ez a hiba gyakori, amikor valaki a megszakítás-kezelőben szeretne hosszú munkát végezni. A megoldás egy üzenetsor vagy munkasor használata, amit a Szinkronizáció és üzenetátadás alfejezet tárgyal.

5.2.7 Feladatok

5.2.7.1 (a) Fogalmi kérdés

Milyen két fő kategóriába sorolja a Zephyr a szálakat? Miben különbözik a vezérlés átadása a két esetben?

5.2.7.2 (b) Tervezési feladat

A `ts-node` mérőszála 1 másodpercenként meghív egy szenzorolvasó függvényt. A szenzorolvasó hívási lánca a következő lokális változókat használja: `uint8_t buf[16]`, `int32_t raw`, `double scaled`. A Zephyr szál kontextusának mérete `native_sim`-en megközelítőleg 400 bájt. Becsülje meg a szükséges verem méretét, ha 40% biztonsági tartalékot szeretnénk, és válasszon a `K_THREAD_DEFINE`-ban alkalmazható értéket!

5.2.7.3 (c) Implementációs feladat

Egészítse ki a labor kódját egy harmadik szállal, amely a szelep állapotát kezeli. A szál 5 másodpercenként kapcsolja a szelepet: ha a talajnedvesség 30% alá esik, nyit, különben zár. A szelep állapotát globális `bool` változóban tárolja, és a küldőszál minden küldéskor jelenítse meg a szelep állapotát is. Figyeljen a prioritásra: a szabályozószál legyen alacsonyabb prioritású, mint a mérőszál, de magasabb, mint a küldőszál.

5.2.7.4 (d) Hibakeresési feladat

Csökkentse a mérőszál verem méretét 512 bájtra, és fordítsa be a stack sentinelt. Futtassa az alkalmazást, és figyelje meg a hibajelenséget. Ezután állítsa vissza a verem méretét, és magyarázza el, miért nem elegendő a 512 bájt. Melyik hibaüzenet utal arra, hogy a verem túlcsordult?

5.2.8 Megoldások

5.2.8.1 (a) Megoldás

A Zephyr **preemptív** és **kooperatív** szálakat különböztet meg. A preemptív szálakat az ütemező megszakíthatja egy magasabb prioritású szál érkezésekor; a kooperatív szál csak akkor veszíti el a processzort, ha blokkol (például `k_sleep`-et hív), kilép, vagy explicit `k_yield`-et ad.

5.2.8.2 (b) Megoldás

A lokális változók mérete: a `buf[16]` 16 bájt, az `int32_t` 4 bájt, a `double` 8 bájt; összesen 28 bájt. Ehhez hozzáadjuk a függvényhívás visszatérési címeit és a hívási konvenció miatti esetleges igazítást, de becslésként 60–100 bájttal számolhatunk. A szál kontextus 400 bájt. Az összes becsült használat 500 bájt. A 40%-os tartalék: $S_{\text{total}} = 500 \cdot 1{,}4 = 700$ bájt. Érdemes 1024 bájtot választani, mert a Zephyrben a verem mérete általában 2 hatványa, és a rendszerfüggvények is fogyaszthatnak stacket. Tehát `K_THREAD_DEFINE(... 1024, ...)`.

5.2.8.3 (c) Megoldás

A harmadik szál kódja:

```
static bool g_valve_open;

void valve_thread(void *unused1, void *unused2, void *unused3)
{
    ARG_UNUSED(unused1);
    ARG_UNUSED(unused2);
    ARG_UNUSED(unused3);

    while (1) {
        bool open;

        k_mutex_lock(&g_data_lock, K_FOREVER);
        open = (g_data.soil_moisture < 30);
        g_valve_open = open;
        k_mutex_unlock(&g_data_lock);

        printk("[valve] state=%s\n", open ? "open" : "closed");
        k_sleep(K_SECONDS(5));
    }
}

K_THREAD_DEFINE(valve_tid, 1024, valve_thread, NULL, NULL, NULL,
                K_PRIO_PREEMPT(7), 0, 0);
```

A szelep állapotát ugyanazzal a `g_data_lock` mutexszel védjük, mert a küldőszál is olvassa. A küldőszál kritikus szakasza a szelepállapot figyelembevételével így néz ki:

```
bool has_new_data = false;
bool valve_open = false;
int32_t temp_c = 0;

k_mutex_lock(&g_data_lock, K_FOREVER);
if (g_new_data) {
    has_new_data = true;
    temp_c = g_data.temp_c;
    valve_open = g_valve_open;
    g_new_data = false;
}
k_mutex_unlock(&g_data_lock);

if (has_new_data) {
    printk("[send] temp=%d C, valve=%s\n",
           temp_c, valve_open ? "open" : "closed");
}
```

Így a `g_data.soil_moisture` olvasása és a `g_valve_open` írása ugyanabban a kritikus szakaszban történik, a küldőszál pedig ugyanezen mutex alatt olvassa mindkettőt, ezért nem állhat össze különböző mérésekből származó részleges minta. A mutexet csak szálkontextusból szabad hívni; megszakításkezelőből nem.

5.2.8.4 (d) Megoldás

A stack sentinel hiba esetén a Zephyr jellemzően `***** Stack Check Fail! *****` vagy hasonló üzenetet ír a konzolra, és leállítja a rendszert. A 512 bájt túl kicsi, mert a `printk` hívás, a `k_sleep` rendszerhívás és a szálváltás kontextusa együttesen több mint 512 bájtot használ. A hiba elkerülése érdekében növeljük a stack méretet 1024 bájtra, vagy csökkentsük a hívási mélységet.

5.2.9 Rövid összefoglaló

A Zephyr szálakat és preemptív ütemezést használ a párhuzamos tevékenységek kezelésére. A prioritás kisebb száma magasabb prioritást jelent; a preemptív szálak megszakíthatják egymást, a kooperatívak csak blokkoláskor adnak vezérlést. A verem méretét becsülni és a `k_thread_stack_space_get()`-tel ellenőrizni kell; stack sentinel és thread analyzer segít a túlcsordulás felderítésében. A `k_sleep` blokkoló időzítés, a `k_timer` aszinkron callback. A megosztott mérési adatot és a jelzőt a példában `g_data_lock` mutex védi; a prioritásinverzió ellen a mutex prioritásöröklése véd. **Továbbvitt eredmény.** A `~/iot-labor/i010/ts-node/src/main.c`-ben létrehozott `measure_thread` és `send_thread` szálak, valamint a `g_data` struktúra és a `g_new_data` jelző. Ezt a szálszerkezetet használja a Szinkronizáció és üzenetátadás alfejezet, amely a mutex működését, a prioritásöröklést és az üzenetsor-alapú adatcserét részletezi, valamint a Eszközszoftver-architektúra alfejezet, amikor a mérő-, küldő- és szabályozószálat állapotgépes rétegekbe szervezi.

5.2.10 Források és ellenőrzési hiányok

- Zephyr dokumentáció, kernel API: szálak, ütemezés, veremkezelés. URL: https://docs.zephyr-project.org/latest/ (nem ellenőriztem közvetlenül).
- A `K_PRIO_PREEMPT` és `K_PRIO_COOP` makrók, valamint a `k_thread_stack_space_get()` `(const struct k_thread *, size_t *)` szignatúrája a rögzített Zephyr 3.7.0 profilhoz igazodik; a `native_sim` verem kontextus mérete architektúrafüggő.
- A mutex prioritásöröklés pontos Kconfig opciója a Zephyr verziójában ellenőrizendő.

5.3 Szinkronizáció és üzenetátadás: mutex, szemafor, üzenetsor, work queue

A TerraSense eszközén a szenzorok olvasása és a hálózati küldés ritmusa eltér egymástól: a mérőszál másodpercenként mintát vesz, a küldőszál viszont csak akkor tudja továbbítani az MQTT-üzenetet, ha a rádió és a hálózati verem éppen elérhető. Ha a két tevékenység ugyanabban a memóriában dolgozik, akkor adatverseny, elveszett frissítés vagy részben kiírt rekord keletkezhet. A Zephyr – a pthread világához hasonlóan, de beágyazott korlátokkal – szinkronizációs és üzenetátadási primitíveket ad ehhez: mutexet, szemaforet, üzenetsort és work queue-t. Ebben az alfejezetben ezeket a primitíveket a TerraSense mérési rekordjának, a `ts_sample` struktúrának a továbbításán keresztül mutatjuk be.

5.3.1 Motiváció

A `ts-node` szoftverében két szál fut együtt. Az egyik a szenzorokat olvassa, a másik az MQTT-üzeneteket állítja össze és adja fel. Ha közvetlenül egy globális `struct ts_sample` változóban cserélnének adatot, akkor a küldőszál éppen akkor is olvashatná a rekordot, amikor a mérőszál félig írta felül; ráadásul a mérőszál többször is felülírhatná a mintát, mielőtt a küldőszál elküldené. A megoldás: a mérőszál minden mintát egy üzenetsorba tesz, a küldőszál pedig onnan veszi ki. Így a két szál ritmusa szétcsatolódik, és a minta nem vész el addig, amíg a sor el nem telik.

5.3.2 Fogalmak és működés

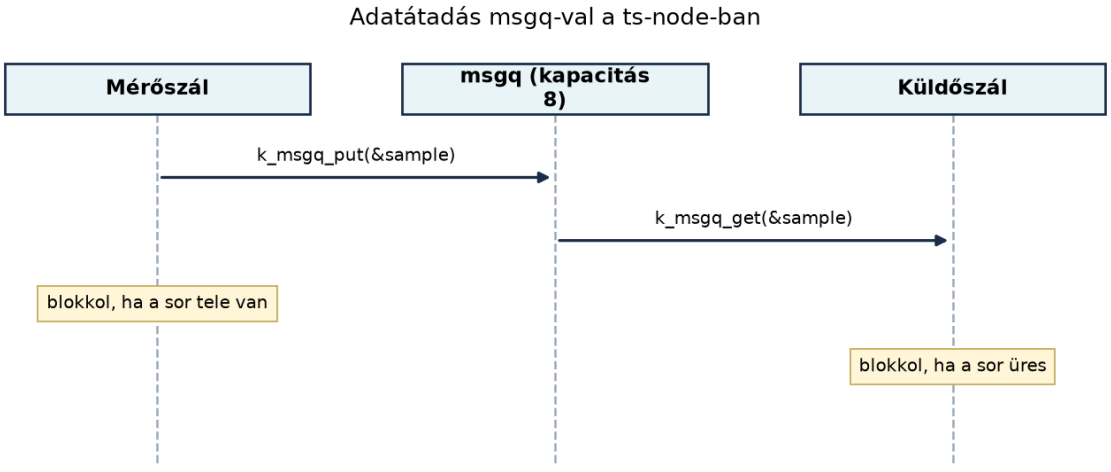
Definíció. A **mutex** (mutual exclusion) olyan szinkronizációs objektum, amely legfeljebb egy szálnak adhatja meg a belépést egy kritikus szakaszba. A Zephyr `k_mutex` típusa hasonlít a `pthread_mutex_t`-hez, de nincs külön címtér, a verem mérete fordítási időben rögzített, és a mutex tulajdonosa nem blokkolhatja magát végtelen ideig. A Zephyr mutex **prioritásöröklést** alkalmaz: ha egy alacsony prioritású szál tartja a mutexet, és egy magas prioritású szál vár rá, akkor az alacsony prioritású szál átmenetileg megkapja a magasabb prioritást, így elkerülhető a prioritásinverzió.

Definíció. A **szemafor** (semaphore) számláló alapú szinkronizációs objektum. A `k_sem_give()` növeli a számlálót, a `k_sem_take()` pedig csökkenti, ha a számláló nagyobb nullánál; különben blokkol. A mutexszel ellentétben a szemafornak nincs tulajdonosa, ezért alkalmas jelzésre (például „új minta érkezett”), nem pedig erőforrás-védelemre. Bináris szemafor 0/1 értékkel egyszerű eseményjelzésre használható. A `k_sem_give()` hívható ISR-ből, a `k_sem_take()` viszont nem, mert blokkolhat.

Definíció. Az **üzenetsor** (message queue, `k_msgq`) fix méretű elemeket tároló FIFO puffer. A Zephyr üzenetsor a teljes üzenetet **másolja**, nem csak mutatót ad át, ezért a termelő a saját veremben lévő rekordot a `k_msgq_put()` után azonnal felülírhatja. A fogyasztó a `k_msgq_get()` hívással másolatot kap. A `k_msgq` kapacitása fordítási időben rögzített, a memóriája statikusan foglalt.

Definíció. A **work queue** olyan szál, amely elhalasztott munkákat hajt végre. A `k_work` objektum egy függvényt és annak paraméterét írja le. A `k_work_submit()` a rendszer work queue-jába teszi a munkát; a work queue saját szálon, normál kontextusban futtatja a függvényt. Mivel az ISR-ekben nem szabad blokkolni, a Zephyrben az ISR-ből elhalasztott munka tipikus megoldása a work queue: az ISR csak `k_work_submit()`-ot hív, a tényleges feldolgozás később, szálkontextusban fut.

A következő diagram a mérőszál és a küldőszál közötti üzenetsoros adatátadást mutatja:



5.3. ábra: Adatátadás msgq-val a ts-node-ban - A mérőszál másolja a ts_sample rekordot az üzenetsorba; a küldőszál onnan veszi ki.

Az ábra nélkül is érthető: a mérőszál az üzenetsorba helyezi a mintát, a küldőszál pedig onnan veszi ki. Ha a sor tele van, a termelő blokkol (vagy időtúllépést kap); ha üres, a fogyasztó

blokkol.

5.3.3 Döntési szempontok

A négy primitív közötti választás a következő táblázat szerint történhet:

Helyzet	Ajánlott primitív	Indoklás
Közös globális állapot (pl. szenzorkonfiguráció) védelme több szál között	mutex	tulajdonos, prioritásöröklés, kritikus szakasz
Esemény jelzése (pl. „új minta érkezett”)	szemafor (bináris)	ISR-ből adható, nincs tulajdonos
Erőforrás-készlet kezelése (pl. 4 db puffercella)	szemafor (számláló)	a számláló a szabad erőforrások számát jelzi
Mérési rekordok átadása termelő és fogyasztó szál között	üzenetsor (`k_msgq`)	FIFO, másolás, fix méret, ISR-ből `put` lehetséges
Változó méretű adatok átadása (pl. soros keretek)	FIFO (`k_fifo`)	mutató átadása, dinamikus elemek
ISR-ből elhalasztott, nem sürgős feldolgozás	work queue	normál szálkontextus, blokkolható

A `k_msgq` előnye a `k_fifo`-val szemben, hogy a teljes üzenetet másolja, így a termelőnek nem kell a puffert a fogyasztó felszabadításáig életben tartania. Hátránya, hogy a kapacitás és az elem mérete rögzített, és nagy üzeneteknél a másolás időt és memóriát igényel.

5.3.4 Kidolgozott példa és labor

A labor a TerraSense mérési rekordjának (`ts_sample`) üzenetsoros továbbítását valósítja meg `native_sim`-en. A mérőszál másodpercenként előállít egy mintát, a küldőszál kiveszi a sorból és kiírja a konzolra.

Munkakönyvtár:

```
mkdir -p ~/iot-labor/i011 && cd ~/iot-labor/i011
```

Hozzuk létre a projektfájlokat.

`CMakeLists.txt`:

```
cmake_minimum_required(VERSION 3.20)
find_package(Zephyr REQUIRED HINTS $ENV{ZEPHYR_BASE})
project(ts_sync)

target_sources(app PRIVATE src/main.c)
```

`prj.conf`:

```
CONFIG_LOG=y
CONFIG_PRINTK=y
# A printk "%f" konverziójához a cbprintf lebegőpontos támogatása kell;
# enélkül a float argumentum nem tizedes értékként jelenik meg.
CONFIG_CBPRINTF_FP_SUPPORT=y
# A "%lld" (64 bites) konverzióhoz a teljes szélességű egész támogatás kell.
CONFIG_CBPRINTF_FULL_INTEGRAL=y
CONFIG_SHELL=n
```

A `printk` a Zephyr 3.7-ben a cbprintf formázót használja; a `%f` és a `%lld` konverzió nem része a minimális alapkészletnek, ezért a fenti két Kconfig-kapcsoló nélkül a labor kimenete nem a várt tizedes, illetve 64 bites értékeket mutatja. Aki nem akar lebegőpontos formázást a célon, az a `printk`-ban fixpontos, egész számú megjelenítést használjon (pl. `milli_c = (int)(temperature_c * 1000.0f)` és `%d.%03d`), és a `ts_sample`ben továbbra is `float` maradhat.

`src/main.c`:

```c
#include <zephyr/kernel.h>
#include <zephyr/sys/printk.h>
#include <stdint.h>
#include <string.h>

/* Továbbvitt eredmény: a ts_sample struktúra az eszközszoftver-architektúra
 * (i018) és a telemetria-séma (i032) fejezetben is használatos.
 */
struct ts_sample {
    uint32_t seq;
    int64_t uptime_ms;         /* a ts-node saját, monoton órája */
    int64_t event_ts_epoch_ms; /* UTC epoch ms; 0 = az óra még nincs szinkronizálva */
    float temperature_c;
    float humidity_rh;
    float soil_moisture_pct;
};

/* 8 elemű msgq, elem mérete a ts_sample struktúra */
K_MSGQ_DEFINE(sample_msgq, sizeof(struct ts_sample), 8, 4);

/* A k_uptime_get() monoton, de nem UTC: csak a boot óta eltelt időt adja.
 * A telemetriaszerződés viszont epoch ms-t vár, ezért a kettőt külön visszük.
 * Az offset 0 azt jelenti, hogy a ts-node órája még nincs szinkronizálva
 * (a laborban nincs RTC/SNTP). A 64 bites offsetet mutex védi, mert 32 bites
 * célon a kiírás és az olvasás nem atomikus.
 */
static K_MUTEX_DEFINE(clock_lock);
static int64_t epoch_offset_ms;

static int64_t ts_epoch_ms(int64_t uptime_ms)
{
    int64_t result;

    k_mutex_lock(&clock_lock, K_FOREVER);
    result = (epoch_offset_ms == 0) ? 0 : uptime_ms + epoch_offset_ms;
    k_mutex_unlock(&clock_lock);

    return result;
}

/* Szinkronizáció után (pl. SNTP válaszból) egyszer beállítjuk az eltolást. */
static void ts_clock_set_epoch(int64_t utc_epoch_ms, int64_t uptime_ms)
{
    k_mutex_lock(&clock_lock, K_FOREVER);
```

```
    epoch_offset_ms = utc_epoch_ms - uptime_ms;
    k_mutex_unlock(&clock_lock);
}

static void producer_thread(void *p1, void *p2, void *p3)
{
    struct ts_sample sample;
    uint32_t seq = 0;

    while (1) {
        /* Emulált szenzorolvasás */
        sample.seq = ++seq;
        sample.uptime_ms = k_uptime_get();
        sample.event_ts_epoch_ms = ts_epoch_ms(sample.uptime_ms);
        sample.temperature_c = 20.0f + (seq * 0.1f);
        sample.humidity_rh = 60.0f - (seq * 0.05f);
        sample.soil_moisture_pct = 30.0f + (seq * 0.2f);

        printk("producer: seq=%u, uptime=%lld ms, epoch=%lld ms\n",
               sample.seq, (long long)sample.uptime_ms,
               (long long)sample.event_ts_epoch_ms);
        k_msgq_put(&sample_msgq, &sample, K_FOREVER);

        k_sleep(K_MSEC(1000));
    }
}

static void consumer_thread(void *p1, void *p2, void *p3)
{
    struct ts_sample sample;

    while (1) {
        k_msgq_get(&sample_msgq, &sample, K_FOREVER);
        printk("consumer: seq=%u, temp=%.1f C, hum=%.1f %%, soil=%.1f %%\n",
               sample.seq, sample.temperature_c, sample.humidity_rh,
               sample.soil_moisture_pct);
    }
}

K_THREAD_DEFINE(producer_id, 1024, producer_thread, NULL, NULL, NULL, 5, 0, 0);
K_THREAD_DEFINE(consumer_id, 1024, consumer_thread, NULL, NULL, NULL, 6, 0, 0);

void main(void)
{
    printk("ts-sync labor indult\n");
}
```

A `K_MSGQ_DEFINE` makró statikusan létrehozza az üzenetsort. A két szál prioritása eltér: a fogyasztóé magasabb (6), így ha van feldolgozandó minta, azonnal lefut.

A két időmező szerepe. A közös telemetriaszerződés a `ts` mezőben UTC szerinti epoch ezredmásodpercet vár, mert a backend, a TimescaleDB és a Grafana ezt rendezi és jeleníti meg. A `k_uptime_get()` ezzel szemben a boot óta eltelt idő, nem epoch: értéke minden újraindítás után nulláról indul, és önmagában nem hasonlítható össze más csomópontok vagy a szerver idejével. Ezért a rekord két mezőt visz: az `uptime_ms` a helyi, monoton sorrendezéshez és a késleltetés méréséhez kell, az `event_ts_epoch_ms` pedig a szerződés szerinti eseményidő. Amíg a `ts-node` órája nincs szinkronizálva (a laborban nincs RTC és SNTP, ezért `event_ts_epoch_ms == 0`), a `ts-gw` a vétel pillanatában kapott idejét használhatja becslésként, és ezt a telemetriában külön jelzi (`ts_estimated: true`); a becslés nem mérési idő, és a backend csak sorrendezésre, nem fizikai következtetésre használhatja. A `ts_ms` korábbi, egyetlen mezős változata ezt a különbséget mosta el, ezért váltottuk ketté.

A build és a futtatás:

```
west build -b native_sim .
./build/zephyr/zephyr.exe
```

Várható kimenet (részlet):

```
ts-sync labor indult
producer: seq=1, uptime=1023 ms, epoch=0 ms
consumer: seq=1, temp=20.1 C, hum=59.9 %, soil=30.2 %
producer: seq=2, uptime=2025 ms, epoch=0 ms
consumer: seq=2, temp=20.2 C, hum=59.9 %, soil=30.4 %
```

A mérőszál másodpercenként termel, a küldőszál azonnal fogyaszt, mert a sor üres. Az `epoch=0` azt jelzi, hogy a laborban a `ts-node` órája nincs szinkronizálva; a kimenet értékei illusztratívak, a `native_sim` ütemezésétől függően néhány ms-mal eltérhetnek.

Továbbvitt eredmény. A `ts_sample` struktúra és a `sample_msgq` üzenetsor a Eszközszoftver-architektúra: rétegek és állapotgépek alfejezetben a rétegek közötti adatátadás alapja lesz, a Adatformátumok: JSON, CBOR, Protocol Buffers, SenML alfejezet pedig ebből a struktúrából készíti el a telemetria-séma első változatát.

5.3.5 Ellenőrzés és hibakeresés

A labor működését a konzolnapló igazolja: minden `producer:` sorhoz tartozik egy `consumer:` sor, a `seq` értékek szigorúan növekvők. Ha a küldőszál lassabb, a sor telik, de a `K_FOREVER` miatt a mérőszál blokkol, így egyetlen minta sem vész el.

Tünet	Lehetséges ok	Javítás
A consumer nem kap üzenetet	A `K_MSGQ_DEFINE` mérete nem egyezik a `sizeof(struct ts_sample)` értékével	Használjunk `sizeof` operátort, ne kézzel írt konstansot
`k_msgq_put` fordítási hibát ad	A struktúra nem másolható (pl. rugalmas tömb tag)	Kerüljük a mutató tagokat, használjunk fix méretet
A producer blokkol	A sor tele van, mert a consumer nem fut	Ellenőrizzük a prioritásokat, adjunk timeoutot a `put`-nak
A consumer blokkol	A sor üres, mert a producer nem termel	Ellenőrizzük, hogy a producer szál elindult-e

5.3.6 Tipikus hibák és félreértések

1. **ISR-ben blokkoló hívás.** A `k_msgq_put()` ugyan hívható ISR-ből, de csak `K_NO_WAIT` timeouttal. Ha egy ISR `K_FOREVER`-t használ, és a sor tele van, a rendszer lefagy. Az ISR-ből mindig `K_NO_WAIT`-et használjunk, és a tényleges feldolgozást work queue-ba tegyük.
2. **Mutató átadása `k_msgq`-ban.** A `k_msgq` másol, de ha a struktúra mutató tagot tartalmaz, akkor a másolt mutató a termelő veremére mutat. A fogyasztó később érvénytelen memóriát olvashat. Az üzenetsorba kerülő struktúrák ne tartalmazzanak mutatót, vagy a mutató által címzett adatot is explicit módon kezelni kell.
3. **Holtpont.** Ha két szál két mutexet fordított sorrendben vesz fel, mindkettő várhat a másikra. Ezt a sorrend rögzítésével kerüljük el, és soha ne tartsunk mutexet blokkoló hívás (például `k_msgq_get`) alatt.

5.3.7 Feladatok

5.3.7.1 a) Fogalmi kérdés

Mikor használjunk mutexet és mikor szemaforet? Indokolja a választ a TerraSense példáján: a szenzorkonfiguráció írása és az „új minta érkezett” jelzés.

5.3.7.2 b) Számolási feladat

Tegyük fel, hogy a `struct ts_sample` mérete a célon mérve 40 bájt (a pontos értéket mindig `sizeof`-szal kell megmérni, mert függ az ABI-tól és a paddingtól). Hány bájt statikus memóriát foglal a `K_MSGQ_DEFINE(sample_msgq, sizeof(struct ts_sample), 8, 4)` által létrehozott teljes üzenetsor? Bontsa fel a foglalást pufferre, vezérlőobjektumra és igazításra, és mondja meg, hogyan lehet a build után a linker `.map` fájljából visszaellenőrizni a tényleges méreteket.

5.3.7.3 c) Implementációs feladat

Bővítse a `ts_sample` struktúrát egy `rssi` mezővel (`int8_t`), és módosítsa a mérőszálat, hogy az `rssi` értéke `-40` és `-90` között változzon. A küldőszál írja ki az `rssi` értékét is.

5.3.7.4 d) Hibakeresési feladat

A küldőszál legyen lassabb, mint a mérőszál: a fogyasztó ciklusába tegyen `k_sleep(K_MSEC(2000))` hívást. Mit tapasztal? Hogyan lehet a mérőszálat úgy módosítani, hogy ne blokkoljon végtelenül, hanem 100 ms után hagyja el a `k_msgq_put` hívást, és naplózza a túlcsordulást?

5.3.8 Megoldások

5.3.8.1 a) Fogalmi kérdés

A szenzorkonfiguráció írása kritikus szakasz: egyszerre csak egy szál módosíthatja az értékeket, hogy ne keletkezzen félkész állapot. Itt mutexet használunk, mert tulajdonost és prioritásöröklést ad. Az „új minta érkezett” jelzés egy egyszerű esemény, amelyet a mérőszál `k_sem_give()`-vel jelezhet a küldőszálnak; itt nincs szükség tulajdonosra, elég a bináris szemafor.

5.3.8.2 b) Számolási feladat

A `K_MSGQ_DEFINE` három dolgot foglal statikusan. Az első a **puffer**: `msgq_sz * msgq_max_cnt`, azaz a feltételezett 40 bájttal `8 * 40 = 320` bájt. A második a **vezérlőobjektum**, egy `struct k_msgq` (várakozási sor, méret- és számlálómezők, puffer-mutatók), amelynek mérete `sizeof(struct k_msgq)`, jellemzően néhány tíz bájt. A harmadik a **padding és igazítás**: a negyedik makróargumentum (itt 4) a puffer igazítása, ehhez jön a linkerszekció igazítása, ezért a puffer és a vezérlőobjektum között, illetve a szekció végén is keletkezhet kihasználatlan hely. A teljes statikus RAM-költség tehát nem a 320 bájt, hanem a három rész összege. A build után a `.map` fájlban a `sample_msgq` (vezérlőobjektum) és a `sample_msgq_buffer` (puffer) szimbólum méretéből ellenőrizhető vissza; a `sizeof(struct ts_sample)` és `sizeof(struct k_msgq)` értékét pedig egy rövid `printk`-kel vagy a fordító `-Wpadded` figyelmeztetésével lehet megmérni. A kódban mindig `sizeof(struct ts_sample)`-et adjunk meg, ne kézzel írt számot:

így a struktúra bővítésekor a puffer mérete automatikusan követi a változást, csak a memóriakeret újraszámolása marad feladat.

5.3.8.3 c) Implementációs feladat

A struktúra:

```
struct ts_sample {
    uint32_t seq;
    int64_t uptime_ms;
    int64_t event_ts_epoch_ms;
    float temperature_c;
    float humidity_rh;
    float soil_moisture_pct;
    int8_t rssi;
};
```

A mérőszálban:

```
sample.rssi = -40 - (seq % 50);
```

A struktúra mérete ezzel 40 bájtról 48 bájtra nő (a 41 bájtot 8 bájtos igazításra kerekítve), ezért a `K_MSGQ_DEFINE` puffere `8 * 48 = 384` bájt lesz; a `sizeof`-ot itt is érdemes újra megmérni.

A küldőszálban:

```
printk("consumer: seq=%u, temp=%.1f C, hum=%.1f %%, soil=%.1f %%, rssi=%d\n",
       sample.seq, sample.temperature_c, sample.humidity_rh,
       sample.soil_moisture_pct, sample.rssi);
```

5.3.8.4 d) Hibakeresési feladat

Ha a fogyasztó lassabb, a sor előbb-utóbb megtelik. A mérőszál a `K_FOREVER` timeout miatt addig blokkol, amíg a fogyasztó ki nem vesz egy elemet. A túlcsordulás elkerülésére a `k_msgq_put` hívást `K_MSEC(100)` timeouttal hívjuk:

```
int ret = k_msgq_put(&sample_msgq, &sample, K_MSEC(100));
if (ret == -EAGAIN) {
    printk("producer: msgq tele, minta eldobva\n");
}
```

Ekkor a mérőszál nem blokkol végtelenül, hanem 100 ms után feladja a küldést, és naplózza a túlcsordulást. A minta elvesztése itt tudatos döntés; a későbbi Store-and-forward: offline működés és visszatöltés alfejezetben megvizsgáljuk, hogyan lehet ezt elkerülni.

5.3.9 Rövid összefoglaló

A Zephyr szinkronizációs és kommunikációs primitívjei közül a mutex a kritikus szakaszok védelmére, a szemafor az eseményjelzésre, az üzenetsor (`k_msgq`) a fix méretű rekordok másolásos átadására, a work queue pedig az ISR-ből elhalasztott munkára való. A laborban definiáltuk a `ts_sample` struktúrát (`seq`, `uptime_ms`, `event_ts_epoch_ms`, hőmérséklet, páratartalom, talajnedvesség), és egy `sample_msgq` üzenetsorral átadtuk a mérést a mérőszálból a küldőszálba. A `ts_sample` struktúra és az üzenetsoros átadás a továbbvitt eredmény: az Eszközszoftver-architektúra: rétegek és állapotgépek alfejezet ezt használja a rétegek közötti kommunikációhoz, az Adatformátumok: JSON, CBOR, Protocol Buffers, SenML alfejezet

pedig ebből készíti a telemetria-sémát. A laborfájlok: `~/iot-labor/i011/CMakeLists.txt`, `~/iot-labor/i011/prj.conf`, `~/iot-labor/i011/src/main.c`.

5.3.10 Források és ellenőrzési hiányok

A Zephyr kernel API-k (`k_msgq`, `k_sem`, `k_mutex`, `k_work`) nevei és a `native_sim` build menete a Zephyr hivatalos dokumentációjából származnak. A könyv rögzített verziója Zephyr 3.7.0 LTS; a `CONFIG_CBPRINTF_FP_SUPPORT` és a `CONFIG_CBPRINTF_FULL_INTEGRAL` nevek ebben a kiadásban a `lib/os/Kconfig.cbprintf` fájlban találhatók, a build után pedig a `build/zephyr/.config` ellenőrzi, hogy ténylegesen bekerültek-e. A `ts_sample` struktúra mérete illusztratív, függ a célplatform ABI-jától és a paddingtől.

5.4 Devicetree és Kconfig: a hardver és a konfiguráció leírása

5.4.1 Motiváció

A TerraSense `ts-node` eszközszoftverének egyszerre kell futnia a laborban használt `native_sim` szimulátoron és – opcionálisan – valós fejlesztőkártyán. Ha a szenzor I2C-címét, a busz sorszámát vagy az engedélyezett alrendszereket közvetlenül a C kódba írnánk, a két környezethez két forrásfát kellene fenntartani, és minden hardvermódosításnál a programlogikát is meg kellene változtatni. A Zephyr ezt a problémát két, egymástól élesen elkülönített réteggel oldja meg: a **devicetree** írja le a hardver *létezését* és kapcsolatait, a **Kconfig** pedig a szoftver *viselkedését* és az engedélyezett funkciókat. A kettő együtt teszi lehetővé, hogy ugyanaz a C kód újraforduljon a szimulátorra és valós kártyára, csak a build cél és a megfelelő overlay változzon.

5.4.2 Fogalmak és működés

5.4.2.1 Devicetree: a hardver fa

A devicetree (eszközfa) a Zephyrben a hardver leírására szolgáló hierarchikus adatszerkezet. A Linux kernelből ismert fogalom, de itt nincs futás közbeni értelmezés: a devicetree-t a build rendszer dolgozza fel, és C makrókat generál belőle.

Definíció. A *devicetree node* a fa egy csomópontja, amely egy hardverelemet (például egy I2C-vezérlőt vagy egy szenzort) ír le. A node neve és elérési útja azonosítja, például `/soc/i2c@40003000/sensor@48`. A node-okhoz *property*-k (tulajdonságok) tartoznak, amelyek kulcs-érték párok. A legfontosabb property-k:

- `compatible`: a node-ot egy adott driverthez kötő sztring; a Zephyrben gyakran gyártói előtaggal, például `"zephyr,sim-eeprom"` vagy `"vnd,tmp116"`.
- `reg`: a hardveres címzés, például egy I2C-eszköz 7 bites címe vagy egy memóriatartomány.
- `status`: `"okay"` vagy `"disabled"`, tehát aktív-e az adott node.
- `label`: választható, ember által olvasható azonosító, bár újabb Zephyr-verziókban egyre kevésbé használjuk.
- `alias`: a gyökér node alatt található `aliases` részen keresztül rövid, stabil nevet adhatunk egy node-nak, például `i2c0 = &i2c0;`.

Definíció. Az *alias* (álnév) a devicetree egy rövid, alkalmazás-szintű hivatkozása egy node-ra. A C kódban a `DT_ALIAS(i2c0)` makróval érhető el, így nem kell a teljes hardveres elérési utat ismerni.

A board-specifikus devicetree fájlok a Zephyr fa `boards/` könyvtárában, illetve a SOC-gyártó által definiált `.dtsi` fájlokban élnek. Az alkalmazás által hozott módosítások az **overlay**-ben kapnak helyet.

Definíció. A *devicetree overlay* egy olyan `.overlay` kiterjesztésű fájl, amely a board alap devicetree-jére rétegződik: a benne szereplő node-ok hozzáadódnak, meglévő property-k felülíródnak, a `status` például `"okay"`-ra állítható. A Zephyr a `boards/<board>.overlay` fájlt automatikusan betölti, ha a `boards/` mappában van és a neve megegyezik a cél boarddal.

Példa egy `native_sim.overlay`-re, amely egy emulált, 4096 bájtos EEPROM-ot vesz fel:

```
/ {
    aliases {
        eeprom0 = &sim_eeprom0;
    };

    sim_eeprom0: eeprom@0 {
        compatible = "zephyr,sim-eeprom";
        size = <4096>;
        status = "okay";
    };
};
```

A `sim_eeprom0` node egy **label**, amelyre a C kódban a `DT_NODELABEL(sim_eeprom0)` makróval hivatkozhatunk. A `compatible` a Zephyr EEPROM-szimulátorához köti a node-ot. A `size` property az emulált memória mérete bájtban.

A devicetree feldolgozása során a build rendszer legenerálja a `build/zephyr/include/generated/devicetree_g` fejlécet. Az ebben lévő `DT_` előtagú makrók fordítási időben elérhetők, így a C kódban például:

- `DT_NODELABEL(sim_eeprom0)` – a node azonosítója,
- `DT_PROP(DT_NODELABEL(sim_eeprom0), size)` – a `size` property értéke,
- `DT_ALIAS(eeprom0)` – az alias mögötti node azonosítója.

A `DT_PROP` makró fordítási időben konstanssá fejti ki az értéket, ezért nincs futás közbeni devicetree-értelmezés, és a kód mérete sem nő.

5.4.2.2 Kconfig: a szoftverkonfiguráció

A Kconfig a Zephyr konfigurációs rendszere. Segítségével nem a hardver *létezését*, hanem a szoftver *viselkedését* hangoljuk: mely alrendszerek épüljenek be, mekkora legyen egy pufferméret, hány szál legyen, mely driverek forduljanak le.

Definíció. A *Kconfig szimbólum* egy beállítható, logikai vagy érték típusú opció. A konfigurációs rendszer a szimbólumokat `CONFIG_<NÉV>` formában generálja le. Az alkalmazás `prj.conf` fájlja ezeket a szimbólumokat állítja be.

Például az EEPROM-szimulátor engedélyezése:

```
CONFIG_EEPROM=y
CONFIG_EEPROM_SIMULATOR=y
```

A `prj.conf` a projekt gyökérkönyvtárában lévő konfigurációs bemenet. A build során a Zephyr képzi a végleges `.config` fájlt (a `build/zephyr/.config` elérési úton), amely minden tényleges értéket tartalmaz, és a `autoconf.h` fejlécet, amelyből a C kód szintén `CONFIG_` makrókat kap. A konfiguráció interaktívan a `menuconfig` felülettel is szerkeszthető, de a `prj.conf` szöveges, verziózható bemenetként alkalmasabb a laborokhoz.

A Kconfig lehetőségek függőségeket fejezhetnek ki. Például `CONFIG_EEPROM_SIMULATOR` csak akkor válik elérhetővé, ha `CONFIG_EEPROM=y`. A függőségeket a Kconfig fájlokban található `depends on` vagy `select` kulcsszavak írják le. A `depends on` azt jelenti, hogy az opció csak akkor látható és állítható, ha a feltétel igaz; a `select` automatikusan beállít egy másik opciót. A gyakorlatban a `prj.conf`-ban explicit módon mindkét szükséges szimbólumot felvesszük, hogy a konfiguráció olvasható és előrelátható legyen.

A devicetree és a Kconfig közötti munkamegosztás tehát: a devicetree azt mondja meg, *hogy milyen hardver van*, a Kconfig azt, *hogy milyen szoftverfunkciót építsünk be*. Egy I2C-szenzornál a devicetree-ben adjuk meg a `compatible`-et és a buszcímet, a Kconfigban pedig engedélyezzük az I2C alrendszert és az adott szenzor drivert.

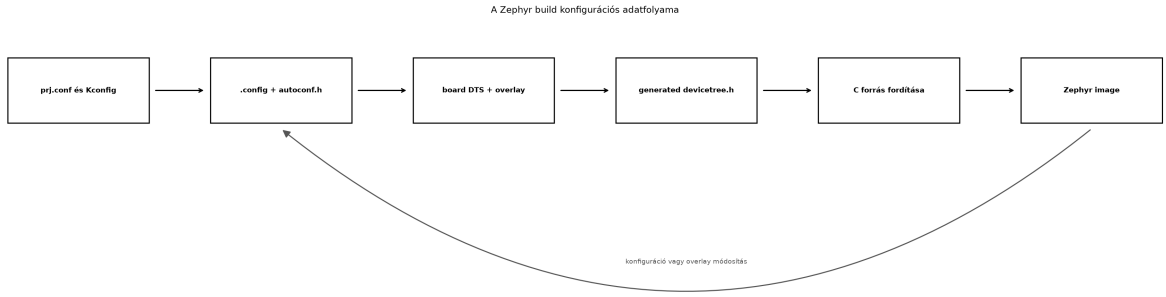
5.4.2.3 Emulált periféria native_sim-en

A `native_sim` board a Zephyr saját szimulációs célja. Itt a program Linux-folyamatként fut, de ugyanazt a Zephyr kernel API-t használja, mint egy valós beágyazott cél. A hardvert a devicetree overlay-ben emulált perifériákkal modellezzük. A Zephyr rendelkezik például az EEPROM-szimulátorral (`zephyr,sim-eeprom`), amely egy memóriában tárolt eszközt biztosít, és az EEPROM API-n keresztül írható-olvasható.

Ellenőrzési igény. A `zephyr,sim-eeprom` pontos `compatible` sztringjét és a hozzá tartozó Kconfig szimbólumot az adott Zephyr kiadás binding dokumentációjában és `drivers/eeprom` forrásában ellenőrizni kell. Hasonlóképpen, az *emulált szenzor* kifejezés alatt a jelen alfejezetben nem egy konkrét érzékelő-drivert, hanem olyan emulált perifériát értünk, amely a `native_sim`-en valós I2C-hardver nélkül is működik. A későbbi Szenzorolvasás a Zephyr sensor API-val alfejezet ezt a mintát viszi tovább, és ott egy saját, minimális szenzor-emulátort építünk.

5.4.2.4 Ugyanaz a kód szimulációra és valós kártyára

A hordozhatóság kulcsa, hogy az alkalmazáskód soha ne tartalmazzon hardverfüggő konstansokat. A C program a devicetree-ből nyeri a hardverazonosítókat, a Kconfigból pedig az engedélyezett funkciókat. Amikor a cél hardvert váltjuk, csak a board nevet és a board-specifikus overlay-t cseréljük; a `src/main.c` és a `prj.conf` érdemben változatlan maradhat.



5.4. ábra: A Zephyr build konfigurációs adatfolyama - A build elején a devicetree és a Kconfig bemenetekből generált fejlécek készülnek, amelyekből a C kód DT_ és CONFIG_ makrókat használ.

A folyamat: a Kconfig bemenetekből `.config` és `autoconf.h`, a board DTS-ből és overlay-ből pedig `devicetree_generated.h` keletkezik. A C forrás ezeket a generált fejléceket használja, így a build végén egyetlen, a célplatformra optimalizált image jön létre. Ha bármelyik bemenetet módosítjuk, a build újrafuttatásával a generált állományok frissülnek.

5.4.3 Döntési szempontok

A devicetree és a Kconfig közötti választás alapkérdése, hogy a beállítás *hardverjellemzőt* vagy *szoftverfunkciót* jelent-e.

Kérdés	Devicetree	Kconfig	Példa
Van-e az adott hardver?	igen	nem	I2C-szenzor jelen van-e
Mi a busz címe?	igen (`reg`)	nem	szenzor 0x48-on
Legyen-e a driver beépítve?	`status`	igen (`CONFIG_*=y`)	`CONFIG_EEPROM=y`
Mekkora legyen egy puffer?	eszközbindingban lehet (`DT_PROP`)	igen, globális esetén	`CONFIG_SHELL_CMD_BUFF_SIZE`
Hány szál induljon?	nem	igen	alkalmazás specifikus
Melyik UART a konzol?	igen (`chosen`)	részben	`zephyr,console`

Továbbvitt eredmény. A TerraSense `ts-node` szoftverében a hardverleírást mindig devicetree overlay-ben, a szoftverfunkciókat pedig `prj.conf`-ban tartjuk. Ezt a konvenciót használja az Szenzorolvasás a Zephyr sensor API-val alfejezet, és ez teszi lehetővé, hogy később az LoRaWAN vagy az OTA-frissítés hardverfüggő részeit is ugyanazon minta szerint válasszuk szét.

Az overlay-ek használatának további előnye a verziózhatóság: a `boards/native_sim.overlay` a szimulátorhoz, egy `boards/<valódi_board>.overlay` pedig a laborban opcionálisan csatlakoztatott kártyához tartozik. A két fájl egyszerre létezhet a projektben, és a Zephyr automatikusan a cél boardhoz tartozó nevet tölti be.

5.4.4 Kidolgozott példa: labor a `native_sim`-en

Munkakönyvtár: `~/iot-labor/i012`

Előfeltétel: a Laborkörnyezet és az Első alkalmazás laborok után működő Zephyr workspace és `west` parancs.

```
mkdir -p ~/iot-labor/i012 && cd ~/iot-labor/i012
```

Hozzuk létre a következő fájlokat.

`CMakeLists.txt`:

```
cmake_minimum_required(VERSION 3.20)
find_package(Zephyr REQUIRED HINTS $ENV{ZEPHYR_BASE})
project(ts_node_i012)

target_sources(app PRIVATE src/main.c)
```

`prj.conf`:

```
# I2C alrendszer engedélyezése (valós kártyán szükséges, native_sim-en az emulált
# periféria mögötti infrastruktúra is ezt használhatja)
CONFIG_I2C=y
# EEPROM API és a szimulátor driver beépítése
CONFIG_EEPROM=y
CONFIG_EEPROM_SIMULATOR=y
```

`boards/native_sim.overlay`:

```
/ {
    aliases {
        eeprom0 = &sim_eeprom0;
    };

    sim_eeprom0: eeprom@0 {
        compatible = "zephyr,sim-eeprom";
        size = <4096>;
        status = "okay";
    };
};
```

`src/main.c`:

```
#include <zephyr/kernel.h>
#include <zephyr/devicetree.h>
#include <zephyr/device.h>
#include <zephyr/drivers/eeprom.h>

#define SIM_EEPROM_NODE DT_NODELABEL(sim_eeprom0)
#define SIM_EEPROM_SIZE DT_PROP(SIM_EEPROM_NODE, size)

int main(void)
{
    printk("TerraSense i012: devicetree es Kconfig labor\n");
    printk("Emulalt EEPROM node: %s\n", DT_NODE_FULL_NAME(SIM_EEPROM_NODE));
    printk("Meret: %u B\n", (unsigned int)SIM_EEPROM_SIZE);

    const struct device *eeprom = DEVICE_DT_GET(SIM_EEPROM_NODE);
    if (!device_is_ready(eeprom)) {
        printk("EEPROM nem elerheto\n");
        return 0;
    }
```

```
    printk("EEPROM device: %s kesz\n", eeprom->name);

    uint8_t buf[4] = {0x11, 0x22, 0x33, 0x44};
    int ret = eeprom_write(eeprom, 0, buf, sizeof(buf));
    if (ret < 0) {
        printk("EEPROM iras hiba: %d\n", ret);
    } else {
        printk("EEPROM iras ok\n");
    }
    return 0;
}
```

A `DT_NODE_FULL_NAME` a devicetree-beli teljes nevet adja vissza, a `DT_PROP` a `size` property értékét, a `DEVICE_DT_GET` pedig a node-hoz tartozó eszközstruktúra pointerét. A kód tehát semmilyen hardvercímet nem tartalmaz.

Build és futtatás:

```
west build -b native_sim -d build
./build/zephyr/zephyr.exe
```

Várt kimenet (a pontos szövegezés eltérhet):

```
TerraSense i012: devicetree es Kconfig labor
Emulalt EEPROM node: /eeprom@0
Meret: 4096 B
EEPROM device: sim_eeprom0 kesz
EEPROM iras ok
```

Továbbvitt eredmény. A `~/iot-labor/i012/boards/native_sim.overlay` és a `~/iot-labor/i012/prj.conf` fájlokat használja és bővíti tovább az Szenzorolvasás a Zephyr sensor API-val alfejezet. Ott egy emulált hőmérséklet-szenzor node-ját vesszük fel ugyanebbe az overlay-be, és a `prj.conf`-ban engedélyezzük a `CONFIG_SENSOR=y` opciót.

5.4.5 Ellenőrzés és hibakeresés

A labor működését két szinten ellenőrizzük. Egyrészt a program kimenete igazolja, hogy a device elérhető és működik. Másrészt a build könyvtárban megnézhetjük a generált konfigurációt:

```
grep CONFIG_EEPROM build/zephyr/.config
```

Várhatóan mindkét opció `=y`. A devicetree feldolgozását a generált fejlécben is ellenőrizhetjük:

```
grep -n "sim_eeprom0" build/zephyr/include/generated/devicetree_generated.h | head
```

Ha a `sim_eeprom0` node nem jelenik meg, az overlay nem töltődött be.

Hibatünet	Lehetséges ok	Javítás
`DT_NODELABEL(sim_eeprom0)` nem definiált	Az overlay fájl neve nem egyezik a boarddal, vagy rossz könyvtárban van	Helyezd a fájlt `boards/native_sim.overlay` néven, és ellenőrizd a build cél boardját
`device_is_ready()` hamis	A `CONFIG_EEPROM_SIMULATOR` nincs engedélyezve	Add hozzá a `prj.conf`-hoz, és futtasd újra a buildet

Hibatünet	Lehetséges ok	Javítás
A `.config`-ban `# CONFIG_EEPROM is not set`	A `prj.conf` nem került beolvasásra	Ellenőrizd, hogy a `prj.conf` a projekt gyökerében van, és a `west build` a projekt könyvtárából indul
A build nem találja a boardot	A Zephyr workspace nincs aktiválva	Futtasd a `west update` és a `source/west zephyr-export` parancsokat, ahogyan az első alkalmazás laborban

A `menuconfig` interaktív felületét a `west build -t menuconfig` paranccsal nyithatjuk meg. Ez hasznos a Kconfig függőségek vizsgálatára, de a laborokban a `prj.conf` verziózott módosítását részesítjük előnyben.

5.4.6 Tipikus hibák és félreértések

1. **Hardveradatok beégetése a C kódba.** Ha valaki a szenzor I2C-címét `#define TEMP_SENSOR_ADDR 0x48` formában adja meg, a kód azonnal függeni fog egy adott kapcsolástól. A Zephyrben a helyes megoldás a devicetree `reg` property, mert innen a cím a `DT_PROP` makróval nyerhető ki, és overlay-ben felülírható anélkül, hogy a programlogikához nyúlnánk.
2. **A devicetree és a Kconfig összekeverése.** Gyakori hiba, hogy valaki a devicetree-ben próbál globális szoftverfunkciót beállítani, vagy Kconfigban hardvercímet. A munkamegosztás azonban tervezési elv, nem nyelvi tilalom: a devicetree elsődlegesen a hardverleírást hordozza, de egy eszköz bindingja definiálhat az adott eszközhöz tartozó, hardverközeli paramétert is, például pufferméretet vagy mintavételi periódust (`sample-interval-ms`), amelyet a driver `DT_PROP`-pal olvas ki. A globális, eszközfüggetlen pufferméreteket viszont továbbra is Kconfigban érdemes tartani, mert azok a teljes rendszer erőforrás-keretéhez tartoznak. A Kconfig nem tudja megmondani, hogy egy adott szenzor melyik buszra van kötve; a két réteg szerepét a fenti táblázat szerint válasszuk szét.
3. **Az `status = "disabled"` elfelejtése.** Ha egy board DTS-ében alapból letiltott perifériát akarunk használni, az overlay-ben nem elég a `compatible`-et megadni, a `status`-t is `"okay"`-ra kell állítani. Ellenkező esetben a device nem jön létre, hiába engedélyeztünk minden Kconfig opciót.
4. **Túlzott támaszkodás a `menuconfig`-ra.** A `menuconfig` jó felfedezőeszköz, de a benne mentett változtatások gyakran a build könyvtár `.config`-jába kerülnek, és elvesznek takarítás után. A reprodukálható konfigurációt mindig a `prj.conf`-ban kell rögzíteni.

5.4.7 Feladatok

5.4.7.1 1. Fogalmi kérdés

Magyarázd el egy konkrét példán, hogy miért kerül a szenzor I2C-címe a devicetree-be, és miért nem Kconfigba! Milyen változás esetén kell módosítani a Kconfigot?

5.4.7.2 2. Tervezési feladat

Egy valós kártyán a `ts-node` egy `tmp116` hőmérséklet-szenzort használ I2C1-en, `0x48` címen. Tervezd meg a devicetree overlay-részletet: milyen node-ot, `compatible`-et, `reg`-et és `status`-t adnál meg? Írd le a `boards/<board>.overlay` tartalmát pszeudokódban.

5.4.7.3 3. Implementációs feladat a TerraSense-ben

A labor EEPROM-os overlay-jét egészítsd ki egy `lab,emul-temp` node-dal, amely később az emulált hőmérséklet-szenzort fogja reprezentálni. A node legyen `temp_sensor@0` néven a gyökér alatt, `compatible = "lab,emul-temp";` és tartalmazzon egy `sample-interval-ms = <5000>;` property-t. Készítsd el a node-hoz tartozó saját YAML-bindingot is a projekt `dts/bindings/sensor/` könyvtárában. A C kódban a `DT_NODELABEL(temp_sensor)` makróval írd ki a property értékét.

5.4.7.4 4. Hibakeresési vagy kiterjesztési feladat

A labor buildje sikeres, de a program kimenetében az EEPROM nem elérhető. A `.config`-ban a `CONFIG_EEPROM` értéke `y`, de a `CONFIG_EEPROM_SIMULATOR` nincs beállítva. Miért nem engedélyeződött automatikusan, és hogyan javítanád? Fogalmazd meg a `prj.conf` helyes bejegyzéseit.

5.4.8 Megoldások

5.4.8.1 1. Fogalmi kérdés

A szenzor I2C-címe hardvertulajdonság: ugyanaz a szoftver másik kártyán lehet, hogy más címen éri el ugyanazt a szenzort, vagy másik szenzort használnak. Ezért a cím a devicetree `reg` property-jébe kerül, és overlay-ben felülírható. A Kconfig viszont szoftverfunkciókat szabályoz, például hogy a szenzor drivert egyáltalán beépítsük-e (`CONFIG_TMP116=y`). Ha csak a címet változtatjuk, a Kconfighoz nem kell nyúlni; ha egy új szenzorfajtát kell támogatni, akkor annak Kconfig opcióját is engedélyezni kell.

5.4.8.2 2. Tervezési feladat

A devicetree overlay-részlet például:

```
/ {
    aliases {
        temp0 = &temp_sensor;
    };
};

&i2c1 {
    status = "okay";

    temp_sensor: tmp116@48 {
        compatible = "ti,tmp116";
        reg = <0x48>;
    };
};
```

A `&i2c1` hivatkozás a board DTS-ében már létező I2C-vezérlőt módosítja, nem hoz létre új `i2c1` csomópontot a gyökér alatt. Ez azért fontos, mert a buszhoz tartozó címcellák, órajel-beállítás és pinctrl a board leírásából öröklődnek; egy új gyökér-szintű csomópont ezek nélkül nem lenne működő vezérlő. A `status = "okay"` engedélyezi a vezérlőt (ha a board DTS-ben alapból tiltott), a `temp_sensor` pedig gyermekcsomópontként örökli a buszt. A `pinctrl-0`/`pinctrl-names` és a `clock-frequency` értéke board-specifikus: ha a board DTS nem állítja be, az overlay-ben a board bindingja szerinti, létező pinctrl-csomópontra hivatkozva kell megadni; ezeket a neveket nem találjuk ki, hanem a board `.dts`/`.dtsi` fájljából vesszük át. A valódi `compatible`-et a gyártói binding határozza meg; a `ti,tmp116` a Zephyr TMP116 driverének bindingja, de a kiadásváltás miatt érdemes a `dts/bindings/sensor/ti,tmp116.yaml` fájlban ellenőrizni. A `reg = <0x48>` az I2C-cím; a `status` megadása a gyermekcsomóponton nem szükséges, mert a szülő `okay` állapotát örökli.

5.4.8.3 3. Implementációs feladat a TerraSense-ben

Az overlay kiegészítése:

```
/ {
    aliases {
        eeprom0 = &sim_eeprom0;
        temp0 = &temp_sensor;
    };

    sim_eeprom0: eeprom@0 {
        compatible = "zephyr,sim-eeprom";
        size = <4096>;
        status = "okay";
    };

    temp_sensor: temp_sensor@0 {
        compatible = "lab,emul-temp";
        reg = <0>;
        sample-interval-ms = <5000>;
        status = "okay";
    };
};
```

A saját `compatible` csak akkor használható, ha van hozzá **binding**: egy YAML-fájl, amely leírja a node kötelező és választható property-jeit, azok típusát és szemantikáját. Különben a build a `sample-interval-ms`-t nem ismeri fel, a `DT_PROP` makró nem generálódik, vagy a devicetree-ellenőrzés ismeretlen property-re panaszkodik. A bindingot a projekt `dts/bindings/sensor/` könyvtárába kell tenni; a Zephyr build az alkalmazás forrásfájának `dts/bindings/` fáját is beolvassa a binding-keresésbe.

`dts/bindings/sensor/lab,emul-temp.yaml`:

```
description: |
  TerraSense laboratóriumi, emulált hőmérséklet-szenzor. Ez a binding
  egyelőre csak a devicetree-node alakját és a property típusát írja le;
  a hozzá tartozó eszközdrivert az i013 alfejezet készíti el.

compatible: "lab,emul-temp"

include: base.yaml

properties:
  sample-interval-ms:
    type: int
```

```
  required: true
  description: |
    A mintavétel periódusa ezredmásodpercben. Illusztratív érték: 5000.
```

A `base.yaml` biztosítja a `reg` és a `status` szabványos értelmezését; ezért a node-on `reg = <0>;` szerepel, összhangban a `temp_sensor@0` unit-addresszel (a `@` utáni résznek egyeznie kell a `reg` értékével). A `sample-interval-ms` nevében a kötőjel a devicetree konvenciója; a C makróban ebből `sample_interval_ms` lesz.

A binding önmagában **nem driver**: csak a node elfogadását és a `DT_` makrók generálását teszi lehetővé. A `main.c`-ben a kiíráshoz:

```
#define TEMP_SENSOR_NODE DT_NODELABEL(temp_sensor)
#define TEMP_INTERVAL DT_PROP(TEMP_SENSOR_NODE, sample_interval_ms)

printk("Mintaveteli ido: %u ms\n", (unsigned int)TEMP_INTERVAL);
```

Ez a program a bindinggal együtt lefordul és kiírja az `5000` értéket akkor is, ha még nincs szenzordriver; a `DT_PROP` ugyanis a generált fejlécből dolgozik. A tényleges eszköz (a `struct device`, a `sensor_driver_api` és a `DEVICE_DT_INST_DEFINE`) csak a driverrel jön létre, ezért a `DEVICE_DT_GET(TEMP_SENSOR_NODE)` ebben a lépésben még nem használható.

Továbbvitt eredmény. A `temp_sensor` node-ot, a `sample-interval-ms` property-t és a `dts/bindings/sensor/lab,emul-temp.yaml` bindingot az Szenzorolvasás a Zephyr sensor API-val alfejezet veszi át: ott készül el a hozzá tartozó eszközdriver és a `CONFIG_LAB_EMUL_TEMP` Kconfig opció, valamint az intervallum alapján ütemezett mintavétel.

5.4.8.4 4. Hibakeresési vagy kiterjesztési feladat

A `CONFIG_EEPROM_SIMULATOR` azért nem állt be, mert a Kconfigban a szimulátor opció `depends on EEPROM` függőséggel él, de nincs `select` kapcsolat, amely automatikusan bekapcsolná. A `CONFIG_EEPROM=y` tehát csak az EEPROM alrendszert építi be, a szimulátor külön opció. A `prj.conf`-ba mindkét sort explicit fel kell venni:

```
CONFIG_EEPROM=y
CONFIG_EEPROM_SIMULATOR=y
```

Ezután a `west build` újrafuttatása a szimulátor drivert is beépíti, és a `device_is_ready()` igaz lesz.

5.4.9 Rövid összefoglaló

A fejezetben megismertük a Zephyr két fő konfigurációs rétegét. A devicetree a hardver fa leírása: node-ok, property-k, `compatible`, alias; az overlay-ek segítségével boardspecifikus módosításokat rétegzünk az alap leírásra. A Kconfig a szoftverfunkciókat és azok függőségeit kezeli, a `prj.conf` a verziózott bemenet, a tényleges értékeket a `.config` és az `autoconf.h` tartalmazzák. A labor `~/iot-labor/i012/` könyvtárában egy emulált EEPROM-ot vettünk fel `native_sim`-en, és C kódból `DT_` makrókkal, valamint a device API-val elértük. A feladatokban saját `lab,emul-temp` node-ot és a hozzá tartozó YAML-bindingot is készítettünk, megtanulva, hogy a binding a node és a property-k típusát írja le, a drivert viszont külön kell megírni. A létrehozott `boards/native_sim.overlay` és `prj.conf` a továbbvitt eredmény: az Szenzorolvasás a Zephyr sensor API-val alfejezet ebből indul ki, amikor emulált hőmérséklet-szenzort ad a TerraSense `ts-node` projekthez.

5.4.10 Források és ellenőrzési hiányok

- Zephyr Project dokumentáció: Devicetree és Kconfig hivatalos leírások.
- `native_sim` board dokumentáció a Zephyr projektben.
- A `zephyr,sim-eeprom compatible` string és a `CONFIG_EEPROM_SIMULATOR` Kconfig szimbólum pontos neve a telepített Zephyr verzióban ellenőrizendő.
- Az emulált szenzor driver elérhetősége és neve `native_sim`-en kiadásonként változhat; a jelen alfejezet EEPROM-szimulátorral demonstrálta a DT+Kconfig együttműködést, a tényleges szenzor-emulátor az Szenzorolvasás a Zephyr sensor API-val laborban készül el.

6 Szenzorok és aktuátorok

6.1 Szenzorolvasás a Zephyr sensor API-val

6.1.1 Motiváció

A TerraSense üvegház-monitorozó csomópontjának első és legfontosabb feladata, hogy a környezeti adatokat megbízhatóan, a fizikai valóságnak megfelelően kiolvassa. A hőmérséklet és a relatív páratartalom olyan analóg jelek, amelyeket a mikrovezérlő digitális világába át kell ültetni. A Zephyr RTOS ezt a feladatot a **sensor API** mögé szervezi: nem kell ismernünk az adott szenzor I2C- vagy SPI-regisztereit, a driverréteg már elvégzi a nyers bájtok és a fizikai mennyiségek közötti átalakítást. Ebben az alfejezetben bekötjük a `ts-node` emulált hőmérséklet- és páratartalom-szenzorát, és egy újrahasznosítható mérési függvényt készítünk, amelyet a későbbi szűrés, architektúra- és hálózati rétegek is használnak.

Az előző alfejezetben (Szinkronizáció és üzenetátadás) definiáltuk a `ts_sample` rekordot és az `sample_msgq` üzenetsort. Most ezt a struktúrát töltjük meg éles, szenzorból származó adatokkal. A cél, hogy a mérőszál ne keveredjen a kommunikációval: a mérési logika egyetlen függvényben legyen, amely bármelyik későbbi rétegből hívható.

6.1.2 Fogalmak és működés

A Zephyr szenzormodell két műveletet választ szét: az **adatgyűjtést** és az **adatlekérdezést**. Ez azért fontos, mert sok szenzor nem folyamatosan mér, hanem csak egy trigger hatására indít mintavételt (például egy I2C-parancs után néhány milliszekundumig tart az analóg-digitális átalakítás).

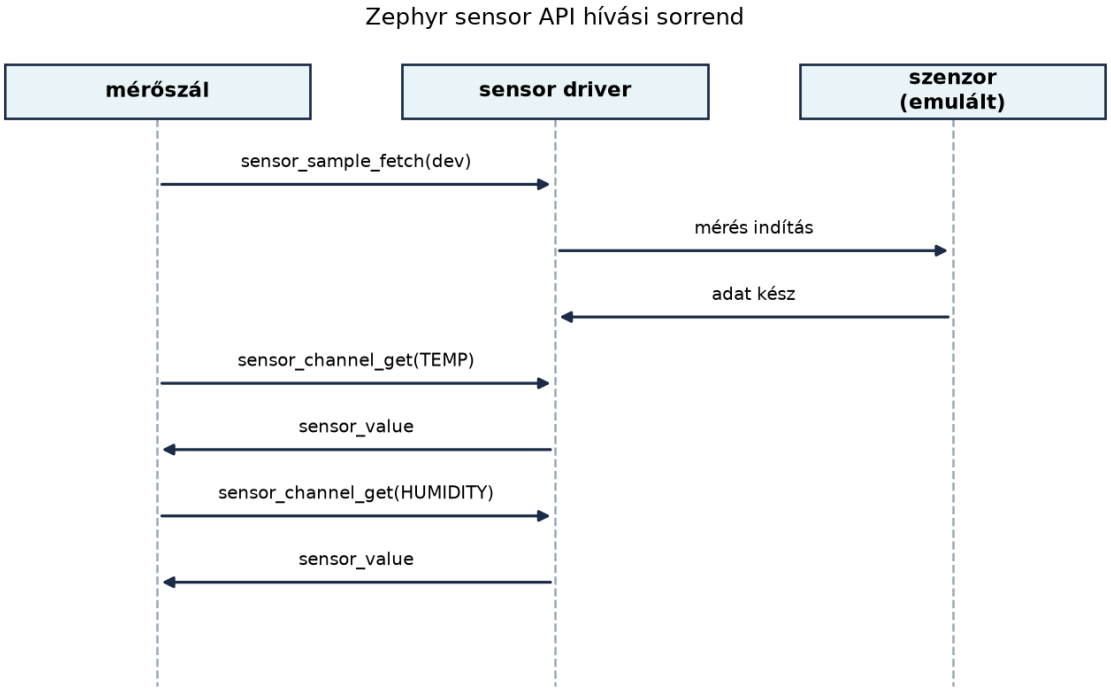
Definíció (sensor API fetch/get).
A `sensor_sample_fetch()` hívás utasítja a driver-t, hogy indítson el egy mérési ciklust, és várja meg, amíg az adat rendelkezésre áll. A `sensor_channel_get()` ezt követően kérdezi le a kiválasztott csatorna (például hőmérséklet, páratartalom) értékét. A `fetch` tehát az adatgyűjtés, a `get` a már begyűjtött adat kiolvasása.

A csatornák az `enum sensor_channel` értékeivel azonosíthatók: `SENSOR_CHAN_AMBIENT_TEMP` a környezeti hőmérséklet, `SENSOR_CHAN_HUMIDITY` a relatív páratartalom, `SENSOR_CHAN_PRESS` a légnyomás, és így tovább.

A mért értéket a driver a `sensor_value` struktúrában adja vissza:

```
struct sensor_value {
    int32_t val1;
    int32_t val2;
};
```

A `val1` az egész rész, a `val2` a mikro-egységben kifejezett törtrész (általában 10 , de a driver ettől eltérő felbontást is használhat). A javasolt átváltás a `sensor_value_to_double()` függvénnyel történik.



6.1. ábra: Zephyr sensor API hívási sorrend - A mérőszál fetch-csel indítja a mérést, majd csatornánként get-tel kéri ki az értékeket.

A Zephyr eszközmodellben a szenzor is egy `device` objektum. A devicetree-ben definiált csomópontból a `DEVICE_DT_GET()` makróval, a node labeljére hivatkozva kérhető le a mutató:

```
const struct device *dev = DEVICE_DT_GET(DT_NODELABEL(temp_sensor));
```

A `DEVICE_DT_GET_ANY()` akkor hasznos, ha a compatible-ből indulunk, és nem ismerjük a node labeljét; a laborban a label stabil, ezért a `DEVICE_DT_GET(DT_NODELABEL(...))` a pontosabb, mert fordítási időben ellenőrzi, hogy a node létezik és engedélyezett.

Ezután a `device_is_ready(dev)` ellenőrzi, hogy a driver inicializálódott-e, és a hardver elérhető-e. Ha a mutató `NULL` vagy a `device_is_ready()` hamis, akkor nem szabad hívni a sensor API-t.

A hiba- és állapotkezelés a Zephyr negatív errno-konvencióját követi: a függvények 0-t adnak vissza siker esetén, hiba esetén pedig `-ENODEV`, `-EBUSY`, `-EIO` és hasonló értékeket. A mérési ciklusban ezekre külön reagálhatunk: az `-EBUSY` és `-EAGAIN` átmeneti hibák, amelyek után érdemes rövid késleltetéssel újra próbálkozni.

6.1.3 Döntési szempontok

A szenzor API-n belül több tervezési döntés is felmerül:

Kérdés	Opciók	Mire érdemes figyelni
Fetch és get szétválasztása	a) `sensor_sample_fetch()` + több `sensor_channel_get()` b) `sensor_sample_fetch_chan(dev, chan)` csatornánként	a) hatékonyabb, ha egyszerre több csatornát olvasunk; b) egyszerűbb, de többszöri mérést indíthat
Érték kinyerése	`val1`/`val2` közvetlen feldolgozása vagy `sensor_value_to_double()`	Lebegőpontos lebegés pontossága elegendő; ha bitpontos átvitel kell, őrizzük meg a nyers `sensor_value` párt
Hiba újrapróbálkozás	azonnali visszatérés, fix késleltetés, exponenciális visszavonulás	Átmeneti `-EBUSY` esetén rövid késleltetés; tartós hiba esetén jelezzük a felügyeleti rétegnek
Szenzor kiválasztása	gyártó-specifikus driver vs. saját emulált driver	`native_sim`-en a labor saját `lab,emul-temp` driverje fut; éles hardverre a devicetree overlay és a `compatible` cseréje szükséges

A TerraSense esetében a fetch/get szétválasztást választjuk, mert egyetlen fetch-csel mindkét csatornát (hőmérséklet, páratartalom) be tudjuk tölteni. A talajnedvesség mérése későbbi laborban kerül hozzá.

6.1.4 Kidolgozott példa: `sensor_value` átváltás és minta kitöltése

Tegyük fel, hogy a hőmérséklet-szenzor a `sensor_value` struktúrában a következőt adja vissza:

- hőmérséklet: `val1 = 23`, `val2 = 450000` → 23,45 °C
- páratartalom: `val1 = 45`, `val2 = 200000` → 45,20 %RH

A `sensor_value_to_double()` ezt a két mezőt `val1 + val2 * 1e-6` módon összegzi. A mérési függvényünk ezeket az értékeket a `ts_sample` struktúrába írja:

```
sample->temperature_c = sensor_value_to_double(&temp_val);
sample->humidity_rh = sensor_value_to_double(&hum_val);
sample->soil_moisture_pct = 0.0f;
sample->soil_moisture_valid = false;   /* ebben a laborban nincs talajszenzor */
```

A nem mért talajnedvességet nem lehet `0.0`-val „kitölteni”: a nulla érvényes, csontszáraz talajérték, és egy későbbi öntözési szabály éppen ebből indítaná a locsolást. Ezért a rekord külön **validitásjelzőt** visz (`soil_moisture_valid`), és a feldolgozó réteg csak akkor használja a számértéket, ha a jelző igaz. A hiányzó mérés így nem téveszthető össze a mért nullával; a telemetriában a mező el is hagyható, vagy explicit `null` lehet.

A `ts_sample` `seq` (sorszám), `uptime_ms` és `event_ts_epoch_ms` mezőit a mérőszál tölti ki, mert azok nem a szenzor driver hatáskörébe tartoznak. A `seq` egy monoton növekvő egyedi azonosító, az `uptime_ms` jelen laborban a `k_uptime_get()` értéke (a rendszer indulása óta eltelt ezredmásodperc). Az `event_ts_epoch_ms` a szerződés szerinti UTC idő; amíg a `ts-node` órája nincs szinkronizálva, értéke 0, és a `ts-gw` a vétel idejével becsüli, ahogy azt az Idő az IoT-ban alfejezet tárgyalja.

6.1.5 Labor: `ts-node` emulált szenzorral

Munkakönyvtár:

```
mkdir -p ~/iot-labor/i013 && cd ~/iot-labor/i013
mkdir -p src boards
```

Előfeltétel: a Szinkronizáció és üzenetátadás: mutex, szemafor, üzenetsor, work queue laborban létrehozott `ts_sample` struktúra és `sample_msgq` mintáját itt újrahasznosítjuk. A teljes fájlt a Szenzorolvasás a Zephyr sensor API-val alfejezetben készítjük el, de a korábbi labor logikáját megtartjuk.

Fájlok:

1. `CMakeLists.txt`:

```
cmake_minimum_required(VERSION 3.20.0)
find_package(Zephyr REQUIRED HINTS $ENV{ZEPHYR_BASE})
project(ts-node-sensor)

target_sources(app PRIVATE src/main.c)
target_sources_ifdef(CONFIG_LAB_EMUL_TEMP app PRIVATE src/lab_emul_temp.c)
```

2. `prj.conf`:

```
CONFIG_LOG=y
CONFIG_SENSOR=y
CONFIG_SENSOR_LOG_LEVEL_DBG=y
# A saját, alkalmazásba épített lab,emul-temp driver bekapcsolása.
CONFIG_LAB_EMUL_TEMP=y
# A LOG_INF("%.2f", ...) lebegőpontos formázásához; enélkül a %f nem működik.
CONFIG_CBPRINTF_FP_SUPPORT=y
# A 64 bites uptime/epoch mezők %lld formázásához.
CONFIG_CBPRINTF_FULL_INTEGRAL=y
```

3. `Kconfig`:

```
menu "TerraSense lab drivers"

config LAB_EMUL_TEMP
    bool "Emulált hőmérséklet- és páratartalom-szenzor (lab,emul-temp)"
    default y
    depends on SENSOR
    help
      Az i013 labor saját, alkalmazásba épített szenzordriverét
      fordítja. A lab,emul-temp devicetree node-hoz tartozik; a
      laborban maradjon bekapcsolva, mert a main.c erre a node-ra
      hivatkozik.

endmenu
```

4. `boards/native_sim.overlay`:

```
/ {
    temp_sensor: temp_sensor@0 {
        compatible = "lab,emul-temp";
        reg = <0>;
        sample-interval-ms = <1000>;
        status = "okay";
    };
};
```

5. `dts/bindings/sensor/lab,emul-temp.yaml` – a saját node bindingja:

```
description: |
  TerraSense laboratóriumi, emulált hőmérséklet- és páratartalom-szenzor.
  A driver determinisztikus, az uptime-ból számított illusztratív értékeket
  ad, hogy a labor valós hardver nélkül is reprodukálható legyen.

compatible: "lab,emul-temp"

include: base.yaml

properties:
  sample-interval-ms:
    type: int
    required: true
    description: |
      A mérési periódus ezredmásodpercben; a 0 érték érvénytelen.

  report-die-temp:
    type: boolean
    description: |
      Ha jelen van, az adott példány a SENSOR_CHAN_DIE_TEMP csatornát
      szolgálja ki (a laborban a talajhőmérséklet helyettesítésére).
```

6. `src/lab_emul_temp.c` – a saját szenzordriver:

```
/*
 * Minimális, alkalmazásba épített szenzordriver a lab,emul-temp
 * compatible-hez. Nem a Zephyr faának része: az alkalmazás forrásából
 * fordul, és a Zephyr 3.7 LTS sensor API-ját használja.
 */
#define DT_DRV_COMPAT lab_emul_temp

#include <zephyr/device.h>
#include <zephyr/devicetree.h>
#include <zephyr/drivers/sensor.h>
#include <zephyr/kernel.h>
#include <zephyr/logging/log.h>
#include <errno.h>
#include <stdbool.h>
#include <stdint.h>

LOG_MODULE_REGISTER(lab_emul_temp, CONFIG_SENSOR_LOG_LEVEL);

struct lab_emul_temp_config {
    uint32_t sample_interval_ms;
    bool report_die_temp;
};

struct lab_emul_temp_data {
    struct sensor_value temp;
    struct sensor_value humidity;
    struct sensor_value die_temp;
};

static int lab_emul_temp_sample_fetch(const struct device *dev,
                                      enum sensor_channel chan)
{
    const struct lab_emul_temp_config *cfg = dev->config;
    struct lab_emul_temp_data *data = dev->data;
    uint32_t tick = (uint32_t)(k_uptime_get() / cfg->sample_interval_ms);
    int32_t milli;

    if (chan != SENSOR_CHAN_ALL && chan != SENSOR_CHAN_AMBIENT_TEMP &&
```

```
        chan != SENSOR_CHAN_HUMIDITY && chan != SENSOR_CHAN_DIE_TEMP) {
        return -ENOTSUP;
    }

    if (cfg->report_die_temp) {
        /* Illusztratív, determinisztikus fűrészjel 18,000-19,000 °C között. */
        milli = 18000 + (int32_t)((tick * 97U) % 1000U);
        data->die_temp.val1 = milli / 1000;
        data->die_temp.val2 = (milli % 1000) * 1000;
    } else {
        /* 23,000-25,000 °C és 45,000-50,000 %RH, tickenként növekedve. */
        milli = 23000 + (int32_t)((tick * 137U) % 2000U);
        data->temp.val1 = milli / 1000;
        data->temp.val2 = (milli % 1000) * 1000;

        milli = 45000 + (int32_t)((tick * 271U) % 5000U);
        data->humidity.val1 = milli / 1000;
        data->humidity.val2 = (milli % 1000) * 1000;
    }

    return 0;
}

static int lab_emul_temp_channel_get(const struct device *dev,
                                     enum sensor_channel chan,
                                     struct sensor_value *val)
{
    const struct lab_emul_temp_config *cfg = dev->config;
    const struct lab_emul_temp_data *data = dev->data;

    switch (chan) {
    case SENSOR_CHAN_AMBIENT_TEMP:
        if (cfg->report_die_temp) {
            return -ENOTSUP;
        }
        *val = data->temp;
        return 0;
    case SENSOR_CHAN_HUMIDITY:
        if (cfg->report_die_temp) {
            return -ENOTSUP;
        }
        *val = data->humidity;
        return 0;
    case SENSOR_CHAN_DIE_TEMP:
        if (!cfg->report_die_temp) {
            return -ENOTSUP;
        }
        *val = data->die_temp;
        return 0;
    default:
        return -ENOTSUP;
    }
}

static int lab_emul_temp_init(const struct device *dev)
{
    const struct lab_emul_temp_config *cfg = dev->config;

    if (cfg->sample_interval_ms == 0U) {
        LOG_ERR("sample-interval-ms must be greater than zero");
        return -EINVAL;
    }
```

```
    return 0;
}

static const struct sensor_driver_api lab_emul_temp_api = {
    .sample_fetch = lab_emul_temp_sample_fetch,
    .channel_get = lab_emul_temp_channel_get,
};

#define LAB_EMUL_TEMP_INIT(inst)                                            \
    static struct lab_emul_temp_data lab_emul_temp_data_##inst;             \
    static const struct lab_emul_temp_config lab_emul_temp_cfg_##inst = {   \
        .sample_interval_ms =                                               \
            (uint32_t)DT_INST_PROP(inst, sample_interval_ms),               \
        .report_die_temp = DT_INST_PROP_OR(inst, report_die_temp, 0),       \
    };                                                                      \
    DEVICE_DT_INST_DEFINE(inst, lab_emul_temp_init, NULL,                   \
                          &lab_emul_temp_data_##inst,                       \
                          &lab_emul_temp_cfg_##inst, POST_KERNEL,           \
                          CONFIG_SENSOR_INIT_PRIORITY, &lab_emul_temp_api);

DT_INST_FOREACH_STATUS_OKAY(LAB_EMUL_TEMP_INIT)
```

7. src/main.c:

```
#include <zephyr/kernel.h>
#include <zephyr/device.h>
#include <zephyr/devicetree.h>
#include <zephyr/drivers/sensor.h>
#include <zephyr/logging/log.h>
#include <stdbool.h>
#include <stdint.h>

LOG_MODULE_REGISTER(sensor_app, LOG_LEVEL_INF);

/* A mintavételi periódus ugyanaz a devicetree-property, amelyet a
 * driver a belső tick számításához használ. */
#define SAMPLE_INTERVAL_MS \
    ((uint32_t)DT_PROP(DT_NODELABEL(temp_sensor), sample_interval_ms))

/* Az i011-ből továbbvitt rekord; a mérési idő két külön mezőre bontva. */
struct ts_sample {
    uint32_t seq;
    int64_t uptime_ms;
    int64_t event_ts_epoch_ms;
    float temperature_c;
    float humidity_rh;
    float soil_moisture_pct;
    bool soil_moisture_valid;
};

K_MSGQ_DEFINE(sample_msgq, sizeof(struct ts_sample), 8, 4);

static uint32_t dropped_samples;

static int sensor_read_sample(const struct device *dev, struct ts_sample *sample)
{
    struct sensor_value temp_val, hum_val;
    int ret;

    if (!dev) {
        return -EINVAL;
    }
```

```
    ret = sensor_sample_fetch(dev);
    if (ret != 0) {
        LOG_ERR("fetch failed: %d", ret);
        return ret;
    }

    ret = sensor_channel_get(dev, SENSOR_CHAN_AMBIENT_TEMP, &temp_val);
    if (ret != 0) {
        LOG_ERR("temp get failed: %d", ret);
        return ret;
    }

    ret = sensor_channel_get(dev, SENSOR_CHAN_HUMIDITY, &hum_val);
    if (ret != 0) {
        LOG_ERR("humidity get failed: %d", ret);
        return ret;
    }

    sample->temperature_c = (float)sensor_value_to_double(&temp_val);
    sample->humidity_rh = (float)sensor_value_to_double(&hum_val);
    sample->soil_moisture_pct = 0.0f;
    sample->soil_moisture_valid = false; /* nincs talajszenzor ebben a laborban */

    return 0;
}

static void sensor_thread(void *arg1, void *arg2, void *arg3)
{
    const struct device *dev = DEVICE_DT_GET(DT_NODELABEL(temp_sensor));
    uint32_t seq = 0;

    if (!device_is_ready(dev)) {
        LOG_ERR("temp_sensor not ready, sensor thread stops");
        return;
    }

    while (1) {
        struct ts_sample sample = {0};
        int ret = sensor_read_sample(dev, &sample);

        if (ret == 0) {
            sample.seq = ++seq;
            sample.uptime_ms = k_uptime_get();
            sample.event_ts_epoch_ms = 0; /* nincs óraszinkron a laborban */
            if (k_msgq_put(&sample_msgq, &sample, K_NO_WAIT) != 0) {
                dropped_samples++;
                LOG_WRN("sample queue full, dropped=%u", dropped_samples);
            }
        } else if (ret == -EBUSY || ret == -EAGAIN) {
            LOG_WRN("sensor busy, retrying");
            k_sleep(K_MSEC(50));
            continue;
        } else {
            LOG_ERR("sensor read failed: %d, skipping cycle", ret);
        }

        k_sleep(K_MSEC(SAMPLE_INTERVAL_MS));
    }
}

static void telemetry_thread(void *arg1, void *arg2, void *arg3)
{
    struct ts_sample sample;
```

```
    uint32_t sent = 0;

    while (1) {
        k_msgq_get(&sample_msgq, &sample, K_FOREVER);
        sent++;
        LOG_INF("telemetry #%u: seq=%u uptime=%lld ms temp=%.2f C hum=%.2f %%",
                sent, sample.seq, (long long)sample.uptime_ms,
                (double)sample.temperature_c, (double)sample.humidity_rh);
    }
}

K_THREAD_DEFINE(sensor_tid, 2048, sensor_thread, NULL, NULL, NULL,
                K_PRIO_PREEMPT(8), 0, 0);
K_THREAD_DEFINE(telemetry_tid, 2048, telemetry_thread, NULL, NULL, NULL,
                K_PRIO_PREEMPT(9), 0, 0);

void main(void)
{
    LOG_INF("TerraSense sensor thread started");
}
```

Miért a szálban kérjük le az eszközt? A `K_THREAD_DEFINE` `arg1` mezője itt szándékosan `NULL`: a szál a `DEVICE_DT_GET(DT_NODELABEL(temp_sensor))` makróval maga oldja fel a devicetree-csomópontot. Így nem függ a `main` egy lokális változójától, amely a szál indulásakor már nem élne, és a `device_is_ready()` ellenőrzés a szál elején egyértelmű hibát naplóz, ha a driver nem példányosodott. Ha mégis paraméterként akarjuk átadni, a `K_THREAD_DEFINE` hívásban `(void *)DEVICE_DT_GET(...)` alak szükséges, a szálban pedig vissza kell castolni `const struct device *`-ra; a szálon belüli feloldás kevesebb átadási pontot hagy.

Fogyasztó és backpressure. A `telemetry_thread` a `sample_msgq` fogyasztója, amely ebben a laborban a hálózati küldést helyettesíti. A termelő `K_NO_WAIT`-tel ír, mert a mérőszálnak nem szabad a hálózatra várnia; ha a sor megtelik, a minta eldobódik, és a `dropped_samples` számláló naplózza a veszteséget. A fogyasztó prioritása (9) alacsonyabb a mérőszálénál (8), így a mérés nem csúszik el. A szándékos eldobás helyett a későbbi Store-and-forward: offline működés és visszatöltés alfejezet tartós puffert használ.

Saját driver és binding. A `lab,emul-temp` nem a Zephyr beépített emulátora: a fenti YAML-binding, a `Kconfig` `LAB_EMUL_TEMP` opciója és a `src/lab_emul_temp.c` együtt adja a teljes vertikális szeletet. A binding a node alakját írja le, a driver a `sensor_driver_api` két függvényét valósítja meg, a `DT_INST_FOREACH_STATUS_OKAY` pedig minden `okay` állapotú példányra létrehozza a `struct device`-ot. A `CONFIG_LAB_EMUL_TEMP` (alapértelmezésben `y`) a CMake `target_sources_ifdef` feltételeként működik, a `prj.conf` pedig explicit bekapcsolja. Ez a Zephyr 3.7 LTS sensor API-ján alapuló, önmagában zárt megoldás; nem függ a Zephyr fájában esetleg létező, kiadásonként változó emulátor-compatible-től vagy annak Kconfig-nevétől.

Build és futtatás:

```
cd ~/iot-labor/i013
west build -b native_sim
./build/zephyr/zephyr.exe
```

Várt kimenet (részlet, illusztratív):

```
[00:00:00.000,000] <inf> sensor_app: TerraSense sensor thread started
[00:00:00.010,000] <inf> sensor_app: telemetry #1: seq=1 uptime=10 ms temp=23.00 C hum=45.00 %
[00:00:01.010,000] <inf> sensor_app: telemetry #2: seq=2 uptime=1010 ms temp=23.14 C hum=45.27 %
[00:00:02.011,000] <inf> sensor_app: telemetry #3: seq=3 uptime=2011 ms temp=23.27 C hum=45.54 %
[00:00:03.011,000] <inf> sensor_app: telemetry #4: seq=4 uptime=3011 ms temp=23.41 C hum=45.81 %
[00:00:04.012,000] <inf> sensor_app: telemetry #5: seq=5 uptime=4012 ms temp=23.55 C hum=46.08 %
[00:00:05.012,000] <inf> sensor_app: telemetry #6: seq=6 uptime=5012 ms temp=23.69 C hum=46.36 %
```

```
[00:00:06.013,000] <inf> sensor_app: telemetry #7: seq=7 uptime=6013 ms temp=23.82 C hum=46.63 %
[00:00:07.013,000] <inf> sensor_app: telemetry #8: seq=8 uptime=7013 ms temp=23.96 C hum=46.90 %
[00:00:08.014,000] <inf> sensor_app: telemetry #9: seq=9 uptime=8014 ms temp=24.10 C hum=47.17 %
[00:00:09.014,000] <inf> sensor_app: telemetry #10: seq=10 uptime=9014 ms temp=24.23 C hum=47.44
↪ %
```

A saját driver determinisztikus fűrészjelet állít elő az uptime és a `sample-interval-ms` hányadosából, ezért az értékek nem véletlenszerűek, hanem a fenti táblázat szerint számíthatók. A `native_sim` ütemezése miatt a tényleges `uptime` néhány ms-mal eltérhet, és ezzel a `tick` is elcsúszhat; a tizedes értékek ezért illusztratívak, nem mért szenzoradatok. A tíz egymást követő `telemetry` sor igazolja, hogy a termelő–fogyasztó lánc folyamatosan működik, és a sor nem telik meg.

Takarítás:
A labor saját könyvtárának tartalmát törölhetjük, de a továbbvitt eredmény miatt célszerű megőrizni a `src/main.c`, a `boards/native_sim.overlay` és a `prj.conf` fájlokat. A build mappa és az ideiglenes állományok `rm -rf build` paranccsal távolíthatók el.

6.1.6 Ellenőrzés és hibakeresés

Az alábbi táblázat a labor során gyakran előforduló hibákat és azok javítását foglalja össze.

Tünet	Ok	Javítás
`device_is_ready` hamis, `temp_sensor not ready` log	A devicetree overlay nem töltődött be, a `lab,emul-temp` binding hiányzik, vagy a node `status` nem `okay`	Ellenőrizzük a `boards/native_sim.overlay` és a `dts/bindings/sensor/lab,emul-temp.yaml` fájlt, a node `status = "okay"` értékét, majd töröljük a `build` mappát, és futtassunk újra `west build`-et
`fetch failed: -22` (`-EINVAL`)	A `sample-interval-ms` értéke 0, ezt a driver initje elutasítja; vagy a device pointer érvénytelen	Adjunk pozitív `sample-interval-ms` értéket az overlay-ben, és ellenőrizzük a `compatible = "lab,emul-temp"` bejegyzést
`temp get failed: -95` (`-ENOTSUP`)	A driver a kért csatornát nem szolgálja ki; például a node-on `report-die-temp` szerepel, de a kód `SENSOR_CHAN_AMBIENT_TEMP`-et kér	A `report-die-temp` nélküli példányon kérjük az `AMBIENT_TEMP`/`HUMIDITY` csatornát, a jelölt példányon pedig a `DIE_TEMP`-et
`sample queue full, dropped=N`	A fogyasztó szál nem fut, vagy tartósan lassabb a termelőnél	Ellenőrizzük a `telemetry_tid` létrejöttét és prioritását; növeljük a `sample_msgq` méretét, vagy csökkentsük a mintavételi gyakoriságot

Tünet	Ok	Javítás
A logban mindig 0,00 értékek jelennek meg	A `sensor_value` `val1`/`val2` mezői nem töltődtek ki, mert a `sample_fetch` nem futott le, vagy másik példányt olvasunk	A `sensor_sample_fetch()` és a `sensor_channel_get()` párost ugyanarra a `dev`-re hívjuk; a `val1`/`val2` tartalmát debug loggal ellenőrizzük
A build nem találja a `lab,emul-temp` bindingot	A YAML nem a projekt `dts/bindings/` fájában van, vagy a `compatible` nem egyezik	Helyezzük a fájlt `dts/bindings/sensor/lab,emul-temp.yaml` néven, és a `compatible` pontosan `"lab,emul-temp"` legyen

6.1.7 Tipikus hibák és félreértések

1. **A `fetch` és a `get` összekeverése.**
   Gyakori hiba, hogy a fejlesztő minden `sensor_channel_get()` előtt új `fetch`-et hív, vagy éppen fordítva, elfelejti a `fetch`-et. Az első esetben a mérés feleslegesen lassul és nő az energiafogyasztás, a másodikban viszont ugyanazt az adatot olvassuk ki többször, vagy elavult értéket kapunk. A szabály: egy `fetch` után több `get` jön, amíg új mérésre nincs szükség.

2. **Lebegőpontos pontosság túlértékelése.**
   A `sensor_value_to_double()` kényelmes, de lebegőpontos reprezentációja nem alkalmas minden esetben. Ha például a hőmérsékletet tized fok pontossággal akarjuk elküldeni, a `double` értékből kerekítés után pontatlanság keletkezhet. Ilyenkor jobb a nyers `sensor_value` párt megőrizni, és csak a megjelenítésnél konvertálni.

3. **Hibakezelés hiánya az átmeneti hibáknál.**
   A `sensor_sample_fetch()` `-EBUSY` hibával is visszatérhet, ha a driver éppen egy másik mérést végez. Ha ezt a hibát tartós hibaként kezeljük és leállítjuk a mérést, a szenzor kiesik a rendszerből. A helyes megoldás a rövid késleltetés utáni újrapróbálkozás, ahogy a labor kódja teszi.

6.1.8 Feladatok

(a) Fogalmi kérdés
Mi a különbség a `sensor_sample_fetch()` és a `sensor_channel_get()` között? Miért választja szét a Zephyr ezt a két műveletet?

(b) Számolási/tervezési feladat
Egy szenzor a hőmérsékletet `sensor_value` formában adja vissza: `val1 = -5`, `val2 = 250000`. Hány °C az érték? Ha a relatív páratartalom `val1 = 99`, `val2 = 900000`, akkor hány százalék? Írd le az átváltás képletét.

(c) Implementációs feladat
Egészítsd ki a labor `sensor_read_sample()` függvényét úgy, hogy a hívás legfeljebb háromszor próbálkozzon meg a `sensor_sample_fetch()` hívással, ha az `-EBUSY` vagy `-EAGAIN` hibát ad. Minden próbálkozás között 50 ms-ot várjon. Ha a harmadik próbálkozás is sikertelen, a függvény adja vissza az utolsó hibakódot.

(d) Kiterjesztési feladat
A TerraSense rendszerben talajnedvesség-mérésre is szükség van. A fejlesztőkártyán egy külön szenzor méri a talajhőmérsékletet, amelyet a `SENSOR_CHAN_DIE_TEMP` csatornán ad. A labor saját `lab,emul-temp` driverje ezt a csatornát is kiszolgálja, ha a node-on szerepel a `report-die-temp` property. Egészítsd ki a devicetree overlay-t egy második, `soil_temp_sensor` labelű `lab,emul-temp` példánnyal, a `ts_sample` struktúrát egy `soil_temp_c` mezővel és egy `soil_temp_valid` jelzővel, a `sensor_read_sample()` függvényt pedig egy második eszközparaméterrel. A hívóoldalt (a szálat) is vezesd át: a talajszenzor hiánya ne bukassa el a teljes mérést, csak a `soil_temp_valid` legyen hamis. A feladat célja a több szenzorpéldány és a módosított függvényaláírás következetes átvezetése.

6.1.9 Megoldások

(a)
A `sensor_sample_fetch()` utasítja a szenzort a mérés elvégzésére, és megvárja, amíg az adat rendelkezésre áll. A `sensor_channel_get()` a már begyűjtött adatot olvassa ki a kiválasztott csatornából. A szétválasztás lehetővé teszi, hogy egyetlen fetch-csel több csatornát is lekérjünk, és a driver egyszerre töltse fel a belső pufferét.

(b)
A `sensor_value` értéke = `val1 + val2 * 1e-6`.
Hőmérséklet: $-5 + 250000 \times 10^{-6} = -4{,}75$ °C.
Páratartalom: $99 + 900000 \times 10^{-6} = 99{,}90$ %RH.

(c)

```
static int sensor_read_sample(const struct device *dev, struct ts_sample *sample)
{
    struct sensor_value temp_val, hum_val;
    int ret;
    int attempts;

    if (!dev) {
        return -EINVAL;
    }

    for (attempts = 0; attempts < 3; attempts++) {
        ret = sensor_sample_fetch(dev);
        if (ret == 0) {
            break;
        }
        if (ret == -EBUSY || ret == -EAGAIN) {
            LOG_WRN("fetch busy, attempt %d/3", attempts + 1);
            k_sleep(K_MSEC(50));
            continue;
        }
        LOG_ERR("fetch failed: %d", ret);
        return ret;
    }

    if (ret != 0) {
        LOG_ERR("fetch failed after retries");
        return ret;
    }

    ret = sensor_channel_get(dev, SENSOR_CHAN_AMBIENT_TEMP, &temp_val);
    if (ret != 0) {
        LOG_ERR("temp get failed: %d", ret);
```

```
        return ret;
    }

    ret = sensor_channel_get(dev, SENSOR_CHAN_HUMIDITY, &hum_val);
    if (ret != 0) {
        LOG_ERR("humidity get failed: %d", ret);
        return ret;
    }

    sample->temperature_c = (float)sensor_value_to_double(&temp_val);
    sample->humidity_rh = (float)sensor_value_to_double(&hum_val);
    sample->soil_moisture_pct = 0.0f;
    sample->soil_moisture_valid = false;

    return 0;
}
```

(d)
A teljes patch három részből áll: struktúra, devicetree és a függvény a hívóoldallal együtt.

A `ts_sample` bővítése:

```
struct ts_sample {
    uint32_t seq;
    int64_t uptime_ms;
    int64_t event_ts_epoch_ms;
    float temperature_c;
    float humidity_rh;
    float soil_moisture_pct;
    bool soil_moisture_valid;
    float soil_temp_c;
    bool soil_temp_valid;
};
```

A devicetree overlay bővítése (a második példány ugyanazt a drivert használja):

```
/ {
    temp_sensor: temp_sensor@0 {
        compatible = "lab,emul-temp";
        reg = <0>;
        sample-interval-ms = <1000>;
        status = "okay";
    };

    soil_temp_sensor: soil_temp@1 {
        compatible = "lab,emul-temp";
        reg = <1>;
        sample-interval-ms = <5000>;
        report-die-temp;
        status = "okay";
    };
};
```

A `sensor_read_sample()` függvény és a hívó szál:

```
static int sensor_read_sample(const struct device *dev,
                              const struct device *soil_dev,
                              struct ts_sample *sample)
{
    struct sensor_value temp_val, hum_val, soil_temp_val;
    int ret;

    if (!dev) {
```

```
        return -EINVAL;
    }

    sample->soil_temp_valid = false;

    ret = sensor_sample_fetch(dev);
    if (ret != 0) {
        LOG_ERR("fetch failed: %d", ret);
        return ret;
    }

    ret = sensor_channel_get(dev, SENSOR_CHAN_AMBIENT_TEMP, &temp_val);
    if (ret != 0) {
        LOG_ERR("temp get failed: %d", ret);
        return ret;
    }

    ret = sensor_channel_get(dev, SENSOR_CHAN_HUMIDITY, &hum_val);
    if (ret != 0) {
        LOG_ERR("humidity get failed: %d", ret);
        return ret;
    }

    sample->temperature_c = (float)sensor_value_to_double(&temp_val);
    sample->humidity_rh = (float)sensor_value_to_double(&hum_val);

    if (soil_dev != NULL) {
        ret = sensor_sample_fetch(soil_dev);
        if (ret != 0) {
            LOG_WRN("soil fetch failed: %d", ret);
        } else {
            ret = sensor_channel_get(soil_dev, SENSOR_CHAN_DIE_TEMP,
                                     &soil_temp_val);
            if (ret == 0) {
                sample->soil_temp_c =
                    (float)sensor_value_to_double(&soil_temp_val);
                sample->soil_temp_valid = true;
            } else {
                LOG_WRN("soil temp get failed: %d", ret);
            }
        }
    }

    return 0;
}

static void sensor_thread(void *arg1, void *arg2, void *arg3)
{
    const struct device *dev = DEVICE_DT_GET(DT_NODELABEL(temp_sensor));
    const struct device *soil_dev =
        DEVICE_DT_GET(DT_NODELABEL(soil_temp_sensor));
    uint32_t seq = 0;

    if (!device_is_ready(dev)) {
        LOG_ERR("temp_sensor not ready, sensor thread stops");
        return;
    }
    if (!device_is_ready(soil_dev)) {
        LOG_WRN("soil_temp_sensor not ready, soil temperature disabled");
        soil_dev = NULL;
    }

    while (1) {
```

```
        struct ts_sample sample = {0};
        int ret = sensor_read_sample(dev, soil_dev, &sample);

        if (ret == 0) {
            sample.seq = ++seq;
            sample.uptime_ms = k_uptime_get();
            sample.event_ts_epoch_ms = 0;
            if (k_msgq_put(&sample_msgq, &sample, K_NO_WAIT) != 0) {
                dropped_samples++;
                LOG_WRN("sample queue full, dropped=%u", dropped_samples);
            }
        } else if (ret == -EBUSY || ret == -EAGAIN) {
            LOG_WRN("sensor busy, retrying");
            k_sleep(K_MSEC(50));
            continue;
        } else {
            LOG_ERR("sensor read failed: %d, skipping cycle", ret);
        }

        k_sleep(K_MSEC(SAMPLE_INTERVAL_MS));
    }
}
```

A megoldás a meglévő driverrel működik, nem kell új `compatible`-et kitalálni: a `report-die-temp` boolean property a bindingban deklarált, a driver pedig a `SENSOR_CHAN_DIE_TEMP` csatornát szolgálja ki rajta. A talajszenzor hiánya nem állítja le a mérést: ilyenkor a `soil_temp_valid` hamis marad, és a telemetria kihagyhatja a mezőt. A hívóoldal is át van vezetve, mert a függvény szignatúrája megváltozott; a struktúra és a hívás együtt fordul.

6.1.10 Rövid összefoglaló

A Zephyr sensor API a mérést két fázisra bontja: `sensor_sample_fetch()` indítja az adatgyűjtést, `sensor_channel_get()` pedig kiolvassa a kiválasztott csatorna értékét. A mért érték `sensor_value` struktúrában érkezik, amelyet `sensor_value_to_double()` segítségével fizikai mennyiséggé alakítunk. A laborban saját, `lab,emul-temp` compatible-höz tartozó YAML-bindingot és szenzordrivert készítettünk, majd a `sensor_read_sample()` függvénnyel töltjük meg a `ts_sample` struktúrát. A mérőszál ezt a mintát az `sample_msgq` üzenetsoron továbbítja, `seq` sorszámmal, `uptime_ms` és `event_ts_epoch_ms` időbélyeggel, a fogyasztó `telemetry_thread` pedig kiveszi és naplózza. A hiányzó talajnedvességet validitásjelző különbözteti meg a mért nullától. **Továbbvitt eredmény:** a `sensor_read_sample()` függvény, a `ts_sample` struktúra, a `sample_msgq`, a `telemetry_thread`, valamint a `temp_sensor` devicetree node és a hozzá tartozó `lab,emul-temp` binding és driver; ezt használja az Mintavétel, kalibráció és szűrés alfejezet a szűrőlogika beépítéséhez, az Eszközszoftver-architektúra: rétegek és állapotgépek alfejezet a rétegek kialakításához, és az IP az eszközön: IPv6, 6LoWPAN, Thread és a Zephyr hálózati stack alfejezet a hálózati küldéshez. A laborfájlok: `~/iot-labor/i013/CMakeLists.txt`, `~/iot-labor/i013/prj.conf`, `~/iot-labor/i013/Kconfig`, `~/iot-labor/i013/boards/native_sim.overlay`, `~/iot-labor/i013/dts/bindings/sensor/lab,emul-temp.yaml`, `~/iot-labor/i013/src/lab_emul_temp.c`, `~/iot-labor/i013/src/main.c`.

6.1.11 Források és ellenőrzési hiányok

A Zephyr sensor API (`sensor_sample_fetch`, `sensor_channel_get`, `sensor_value`, `SENSOR_CHAN_*`) nevei és a `native_sim` build menete a Zephyr hivatalos dokumentációjából származnak. A könyv rögzített verziója Zephyr 3.7.0 LTS; ebben a kiadásban a

`sensor_driver_api` `sample_fetch`/`channel_get` függvénymutatói, a `DEVICE_DT_INST_DEFINE` és a `DT_INST_FOREACH_STATUS_OKAY` makró a fenti módon használható, de a labor első lefordításakor a `build/zephyr/include/generated/devicetree_generated.h` és a `build/zephyr/.config` alapján érdemes ellenőrizni a generált `lab_emul_temp` példányokat és a `CONFIG_CBPRINTF_FP_SUPPORT` beállítást. A `CONFIG_CBPRINTF_FP_SUPPORT` a `LOG_INF`/`printk` `%f` konverziójához kell; enélkül a lebegőpontos argumentum nem a várt tizedes értékként jelenik meg. A driver által generált értékek determinisztikus, illusztratív fűrészjelek, nem valós szenzor mérési eredményei; a labor a könyv készítésekor nem futott le, a lépések statikus ellenőrzésen alapulnak.

6.2 Mintavétel, kalibráció és szűrés

6.2.1 Motiváció

A TerraSense üvegházban a szenzorok jelei ritkán olyan tiszták, mint egy tankönyvi ábrán. A hőmérsékletmérésre rárakódik a szellőztető motor elektromos zaja, a talajnedvesség-érzékelő jele megugrik, ha éppen öntözés közben telítődik a közeg, a páratartalom-érzékelő pedig időnként a szaturációs tartomány szélére ér. Ha ezeket a nyers ADC-értékeket kalibrálás és szűrés nélkül küldené fel a `ts-node`, akkor a felhőben tárolt idősor tele lenne tüskékkel, téves riasztásokkal, és a vezérlő oldalon sem lehetne megbízhatóan eldönteni, mikor kell nyitni az öntözőszelepet.

Ebben az alfejezetben a mérési lánc feldolgozó részét építjük meg: megválasztjuk a mintavételi frekvenciát, a nyers mérésből fizikai mennyiséget számolunk kétpontos kalibrációval, majd a zajt mozgóátlaggal vagy exponenciális szűrővel csökkentjük. Végül megtanuljuk kezelni a mérési tartomány széleit és a kiugró értékeket úgy, hogy ne veszítsünk el valós, gyors eseményeket sem.

6.2.2 Fogalmak és működés

6.2.2.1 Mintavételi frekvencia

A mintavétel a folytonos fizikai jel időben diszkrét mérése. A digitális feldolgozás alapvető feltétele, hogy a mintavételi frekvencia legalább kétszerese legyen a jel legnagyobb értelmes frekvenciájának: $f_s \geq 2f_{\max}$. Az üvegházi környezeti jelek rendkívül lassúak: a hőmérséklet tipikusan tizedperces, a talajnedvesség órás skálán változik, ezért az elméleti minimum már 0,001 Hz–0,01 Hz körül van. Ilyen lassú jelnél a másodperces mintavétel is erős túlmintavételezésnek számít, ami lehetőséget ad a zaj átlagolására, de energiába és processzoridőbe kerül.

A gyakorlatban a mintavételi frekvencia megválasztása a következőktől függ:

- a szenzor konverziós ideje (például egy lassú talajnedvesség-érzékelő 1 ms–100 ms alatt ad új értéket),
- a jel változási sebessége és a szükséges válaszidő,
- az aktuátor-vezérlés reakcióideje,
- az energia- és adatátviteli költség.

6.2.2.2 Kétpontos kalibráció

Definíció. A kalibráció azt a leképezést állítja elő, amely a szenzor nyers, tipikusan ADC-kódban mért kimenetét a mért fizikai mennyiség becsült értékévé alakítja.

A legtöbb érzékelő a működési tartományában jó közelítéssel lineáris, vagy legalábbis üvegházi tartományban annak tekinthető. A kétpontos kalibráció két ismert referenciaponton keresztül egy egyenest illeszt:

$$y = mx + b,$$

ahol x a nyers ADC-kód, y a fizikai érték, m a meredekség, b a tengelymetszet.

6.2.2.3 A két pontból számított együtthatók levezetése

Legyen a két kalibrációs pont (x_1, y_1) és (x_2, y_2). A meredekség:

$$m = \frac{y_2 - y_1}{x_2 - x_1}.$$

Ezt behelyettesítve az első pontba:

$$b = y_1 - mx_1.$$

Számpélda. Tegyük fel, hogy a talajnedvesség-érzékelő ADC-kimenete 20 %RH referencia esetén 200, 80 %RH esetén 800. A mérést 0,01 %RH egységben tároljuk, tehát $y_1 = 2000$, $y_2 = 8000$. Ekkor:

$$m = \frac{8000 - 2000}{800 - 200} = \frac{6000}{600} = 10{,}0 \text{ (0,01 \%RH)/LSB},$$

$$b = 2000 - 10 \cdot 200 = 0.$$

A kalibrált érték ADC = 500 esetén:

$$y = 10 \cdot 500 + 0 = 5000,$$

azaz 50,00 %RH. Másik példa: ha az offset nem nulla, akkor is ugyanígy számolunk. A levezetés mutatja, hogy a két pont ne legyen túl közel, mert akkor a meredekség érzékeny a mérési hibára.

A fixpontos implementációban az együtthatókat egész számokkal, külön számlálóval és nevezővel adjuk meg, elkerülve a lebegőpontos számítást a célprocesszoron.

6.2.2.4 Mozgóátlag

Definíció. A mozgóátlag (moving average) az utolsó N minta számtani közepe:

$$y_n = \frac{1}{N} \sum_{k=0}^{N-1} x_{n-k}.$$

Ez egy véges impulzusválaszú szűrő, amely minden mintát egyenlő súllyal vesz figyelembe, és az ablak elhagyásakor a régi minta hirtelen kiesik. Előnye, hogy egyszerű, kiszámítható késleltetést ad, és jól csillapítja a fehér zajt. Hátránya, hogy N darab mintát tárolnia kell, és a lépcsőzetes válaszreakciója miatt gyors tranziensekre kevésbé alkalmas.

6.2.2.5 Exponenciális szűrő (EMA) fixpontos számítással

Definíció. Az exponenciális mozgóátlag (exponential moving average, EMA) egypólusú rekurzív szűrő:

$$y_n = \alpha x_n + (1 - \alpha) y_{n-1},$$

ahol $0 < \alpha \leq 1$ a simítási tényező. Kis α erősebb simítást, nagyobb késleltetést ad.

A beágyazott implementációban érdemes α-t úgy megválasztani, hogy $\alpha = 1/D$ egész nevezővel írható legyen, mert ekkor:

$$y_n = y_{n-1} + \frac{x_n - y_{n-1}}{D}.$$

Ez egyetlen egész osztást igényel, és nem kell tárolni a korábbi mintákat. A $D = 2^k$ esetén az osztást jobbra shiftre cserélhetjük, de a C szabvány szerint a negatív egész jobbra shift implementációfüggő, ezért hordozható kódban inkább osztást használunk.

Az implementációban az EMA nem nulláról indul: az első **érvényes** minta inicializálja a szűrőt (`initialized` jelző), így állandó bemenetnél az első kimenet maga a bemenet, és nincs hamis, lassú felkúszás. A kiugrókezelés viszonyítási alapja is csak inicializált szűrő esetén az EMA értéke; az első mintánál a minta saját kalibrált értéke.

6.2.2.6 Kiugró értékek, szaturáció, érvényességi tartomány

Definíció. Kiugró értéknek (outlier) nevezzük azt a mintát, amely nagyságrendileg eltér a szűrt jel aktuális értékétől, és valószínűleg mérési hiba, nem pedig valós fizikai változás.

A legegyszerűbb kiugrókezelés az előző szűrt értékhez viszonyított korlát: ha a delta túl nagy, a mintát vagy eldobjuk, vagy a korlátra húzzuk, és `suspect` jelzést adunk. Ez megakadályozza, hogy egyetlen tüske elrontsa az EMA állapotát, de megőrzi a gyors, korláton belüli változásokat.

Definíció. A szaturáció azt jelenti, hogy a szenzor vagy az ADC a mérési tartomány szélére ért, ezért a kimenet már nem hordoz érdemi információt a valós értékről.

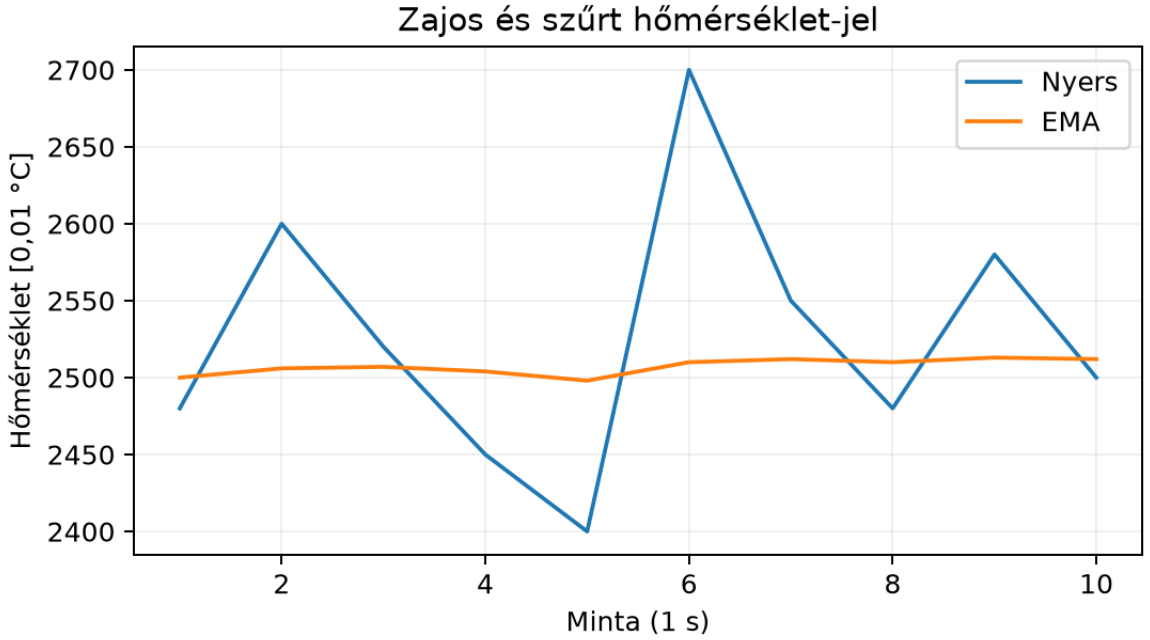
A kalibrálási intervallum széle és az ADC fizikai telítése nem azonos fogalom, ezért a két jelzés szemantikáját külön kell rögzíteni. A `calibration_apply()` három esetet különböztet meg:

Nyers kód	`value`	`valid`	`saturated`	Jelentés
`raw_min < raw < raw_max`	kalibrált érték	`true`	`false`	Normál mérés
`raw == raw_min` vagy `raw == raw_max`	a határra számolt érték	`true`	`true`	A mérés a tartomány szélén van, az érték csak korlát
`raw < raw_min` vagy `raw > raw_max`	`0`	`false`	`true`	A tartományon kívül, nincs használható mérés

Ha például a hőmérséklet-érzékelő 0–50 °C közötti tartományra van kalibrálva, és a nyers ADC a tartományon kívülre esik, akkor a valós hőmérséklet bizonytalan; ilyenkor `valid = false` és `saturated = true` a helyes jelzés. A határponton a konverzió még elvégezhető, ezért ott `valid = true` marad, de a `saturated = true` figyelmeztet, hogy az érték nem pontos mérés, hanem a tartomány korlátja. Az érvényességi jelzés a telemetria séma részévé válik, így a felhőben nem keveredik a valós mérés és a tartományon kívüli becslés.

A laborban a `raw_min`/`raw_max` viselkedését a kalibrációs struktúra szélső értékeivel érdemes ellenőrizni: `raw = 100` és `raw = 4000` esetén `valid = 1`, `saturated = 1`; `raw = 99` és `raw = 4001` esetén `valid = 0`, `saturated = 1`, `value = 0`. Hibás konfiguráció (`slope_den == 0` vagy `raw_max <= raw_min`) esetén szintén `valid = 0`, de `saturated = 0`, mert nem szenzortelítésről, hanem programhibáról van szó.

A következő ábra egy generált zajos jelet és annak EMA-szűrt változatát mutatja:



6.2. ábra: Zajos és szűrt hőmérséklet-jel - Illusztratív, generált zajjal; az EMA alpha=1/16.

A szaggatottan nem látszik, de a lényeg: az EMA kevésbé ingadozik, miközben a trendet követi. A mozgóátlag hasonló simítást ad, de nagyobb késleltetéssel és több memóriával.

6.2.3 Döntési szempontok

Szempont	Mozgóátlag	EMA	Medián / limit
Memóriaigény	N elem puffer	1 állapot	1–3 elem
Számításigény	Összegzés, osztás	1 kivonás, 1 osztás, 1 összeadás	1 összehasonlítás
Késleltetés	$N/2$ minta	$\approx (1-\alpha)/\alpha$ minta	0–1 minta
Kiugró-érzékenység	Elkeni, de nem dobja el	Elkeni	Kifejezetten kezeli
Tranziens válasz	Lépcsőzetes	Exponenciális	Azonnali
Jellemző használat	Rádió előtti átlag, lassú telemetria	Gyors állapotjelző, szabályozó bemenet	Érvényesség, hibadetektálás

Az üvegházi `ts-node` esetében a telemetria-küldés előtt egy 8–16 elemű mozgóátlag vagy $\alpha = 1/16$ EMA jó kompromisszum. A kapcsolódó aktuátor-vezérléshez viszont gyorsabb követést és kiugrókezelést használunk, mert a szelepnyitási döntést nem szabad fél perc késleltetéssel meghozni.

6.2.4 Kidolgozott labor: szűrőmodul a `ts-node` számára

A laborban a `sensor_filter` modult készítjük el, amely a későbbi telemetria- és edge-feldolgozó laborok közös alapja lesz. A modul a Zephyr alkalmazáson belül fut, de a szűrési logika független az operációs rendszertől, ezért később a gateway C++ implementációban is hasonló elven használható.

Munkakönyvtár:

```
mkdir -p ~/iot-labor/i014/src && cd ~/iot-labor/i014
```

A labor a következő fájlokat hozza létre: `src/sensor_filter.h`, `src/sensor_filter.c`, `src/main.c`, `CMakeLists.txt`, `prj.conf`.

Először a szűrőmodul fejlécállománya:

```
/* src/sensor_filter.h */
#ifndef SENSOR_FILTER_H
#define SENSOR_FILTER_H

#include <stdbool.h>
#include <stdint.h>

struct calibration {
    int32_t slope_num;
    int32_t slope_den;
    int32_t offset_scaled;
    int32_t raw_min;
    int32_t raw_max;
};

struct ema_filter {
    int32_t y;
    uint32_t denom;
    bool initialized;   /* hamis, amíg nincs érvényes minta */
};
```

```
struct moving_average {
    int32_t buf[8];
    uint8_t idx;
    uint8_t count;
    int64_t sum;        /* 8 db int32 összege is túlcsordulhat int32-ben */
    uint8_t n;
};

struct sensor_sample {
    int32_t raw;
    int32_t value;
    bool raw_valid;
    bool valid;
    bool saturated;
    bool suspect;
};
typedef struct sensor_sample sensor_sample_t;

void calibration_apply(const struct calibration *c, int32_t raw,
                       int32_t *value, bool *valid, bool *saturated);

void ema_init(struct ema_filter *f, uint32_t denom);
int32_t ema_update(struct ema_filter *f, int32_t x);

void ma_init(struct moving_average *f, uint8_t n);
int32_t ma_update(struct moving_average *f, int32_t x);
int32_t ma_value(const struct moving_average *f);

int32_t reject_outlier(int32_t x, int32_t previous, int32_t limit,
                       bool *suspect);

#endif
```

A `sensor_sample_t`-ben a `value` a kalibrált, szűrt érték 0,01 °C vagy 0,01 %RH skálán; a `valid` az érvényes mérést, a `saturated` a tartomány szélét, a `suspect` a kiugrógyanús mintát jelzi.

A modul implementációja:

```
/* src/sensor_filter.c */
#include "sensor_filter.h"
#include <limits.h>
#include <stddef.h>
#include <string.h>

static int32_t clamp_i32(int64_t v)
{
    if (v > INT32_MAX) {
        return INT32_MAX;
    }
    if (v < INT32_MIN) {
        return INT32_MIN;
    }
    return (int32_t)v;
}

void calibration_apply(const struct calibration *c, int32_t raw,
                       int32_t *value, bool *valid, bool *saturated)
{
    /* Minden kimenet definiált, akkor is, ha a bemenet érvénytelen. */
    if (value == NULL || valid == NULL || saturated == NULL) {
        return;
    }
    *value = 0;
```

```
    *valid = false;
    *saturated = false;

    if (c == NULL || c->slope_den == 0 || c->raw_max <= c->raw_min) {
        return;
    }
    if (raw < c->raw_min || raw > c->raw_max) {
        /* A kalibrált tartományon kívül: nincs használható érték. */
        *saturated = true;
        return;
    }

    /* 64 bites köztes eredmény: slope_num * raw 32 biten túlcsordulhatna. */
    int64_t scaled = (int64_t)c->slope_num * raw;
    int64_t converted = scaled / c->slope_den + c->offset_scaled;

    if (converted > INT32_MAX || converted < INT32_MIN) {
        *saturated = true;
        return;
    }
    *value = (int32_t)converted;
    *valid = true;
    /* A kalibrálási intervallum széle még számolható, de csak korlát. */
    *saturated = (raw == c->raw_min || raw == c->raw_max);
}

void ema_init(struct ema_filter *f, uint32_t denom)
{
    f->denom = denom == 0 ? 1 : denom;
    f->y = 0;
    f->initialized = false;
}

int32_t ema_update(struct ema_filter *f, int32_t x)
{
    int64_t delta;

    if (!f->initialized) {
        /* Az első érvényes minta definiálja a szűrő kezdőállapotát. */
        f->y = x;
        f->initialized = true;
        return f->y;
    }

    delta = (int64_t)x - f->y;
    f->y = clamp_i32((int64_t)f->y + delta / (int64_t)f->denom);
    return f->y;
}

void ma_init(struct moving_average *f, uint8_t n)
{
    if (n > 8) n = 8;
    f->n = n == 0 ? 1 : n;
    memset(f->buf, 0, sizeof(f->buf));
    f->idx = 0;
    f->count = 0;
    f->sum = 0;
}

int32_t ma_update(struct moving_average *f, int32_t x)
{
    if (f->count == f->n) {
        f->sum -= f->buf[f->idx];
```

```
    } else {
        f->count++;
    }
    f->buf[f->idx] = x;
    f->sum += x;
    f->idx = (f->idx + 1) % f->n;
    return (int32_t)(f->sum / f->count);
}

int32_t ma_value(const struct moving_average *f)
{
    return f->count == 0 ? 0 : (int32_t)(f->sum / f->count);
}

int32_t reject_outlier(int32_t x, int32_t previous, int32_t limit,
                       bool *suspect)
{
    int64_t delta = (int64_t)x - previous;

    if (limit < 0) {
        limit = 0;
    }
    if (delta > limit) {
        *suspect = true;
        return clamp_i32((int64_t)previous + limit);
    }
    if (delta < -limit) {
        *suspect = true;
        return clamp_i32((int64_t)previous - limit);
    }
    *suspect = false;
    return x;
}
```

A `main.c` egy mockolt érzékelőt futtat, amely 1 másodpercenként állít elő egy zajos hőmérsékleti ADC-jelet, 25,00 °C körül. A 30. mintánál mesterséges tüskét adunk hozzá:

```
/* src/main.c */
#include <zephyr/kernel.h>
#include <zephyr/sys/printk.h>
#include "sensor_filter.h"

static int32_t sim_raw(uint32_t step)
{
    /* 25,00 °C-nak megfelelő 2048-as kód körül; a zaj -32..+31 LSB,
     * a tüske kizárólag a 30. mintán jelenik meg. */
    int32_t noise = ((int32_t)(step * 1103515245u) & 0x3F) - 32;
    int32_t spike = (step == 30) ? 800 : 0;
    return 2048 + noise + spike;
}

int main(void)
{
    struct calibration cal = {
        .slope_num = 5000,    /* 50,00 °C = 5000, 0,01 °C skálán */
        .slope_den = 4095,    /* ADC tartomány 0..4095 */
        .offset_scaled = 0,
        .raw_min = 100,
        .raw_max = 4000,
    };
    struct ema_filter ema;
    struct moving_average ma;
    ema_init(&ema, 16);
```

```
    ma_init(&ma, 8);

    for (uint32_t step = 0;; step++) {
        int32_t raw = sim_raw(step);
        int32_t cal_value = 0;
        bool valid = false;
        bool saturated = false;
        calibration_apply(&cal, raw, &cal_value, &valid, &saturated);

        bool suspect = false;
        if (valid) {
            /* Az első érvényes mintánál az EMA még inicializálatlan;
             * ilyenkor a minta maga lesz a viszonyítási alap. */
            int32_t previous = ema.initialized ? ema.y : cal_value;
            int32_t limited = reject_outlier(cal_value, previous, 200,
                                             &suspect);
            ema_update(&ema, limited);
            ma_update(&ma, limited);
        }
        /* Érvénytelen mintánál a szűrők állapota változatlan marad. */

        printk("%u raw=%d cal=%d sat=%d suspect=%d ema=%d ma=%d valid=%d\n",
               step, raw, cal_value, saturated, suspect, ema.y,
               ma_value(&ma), valid);
        k_msleep(1000);
    }
}
```

A buildhez és a futtatáshoz a következő `CMakeLists.txt` és `prj.conf` kell:

```
# CMakeLists.txt
cmake_minimum_required(VERSION 3.20)
find_package(Zephyr REQUIRED HINTS $ENV{ZEPHYR_BASE})
project(sensor_filter_lab)

target_sources(app PRIVATE src/main.c src/sensor_filter.c)
target_include_directories(app PRIVATE src)

# prj.conf
CONFIG_PRINTK=y
CONFIG_SERIAL=y
```

A build és futtatás:

```
cd ~/iot-labor/i014
west build -b native_sim .
./build/zephyr/zephyr.exe
```

Várt kimenet (részlet, a végleges kódból determinisztikusan számolva):

```
0 raw=2016 cal=2461 sat=0 suspect=0 ema=2461 ma=2461 valid=1
1 raw=2061 cal=2516 sat=0 suspect=0 ema=2464 ma=2488 valid=1
2 raw=2042 cal=2493 sat=0 suspect=0 ema=2465 ma=2490 valid=1
3 raw=2023 cal=2470 sat=0 suspect=0 ema=2465 ma=2485 valid=1
...
29 raw=2041 cal=2492 sat=0 suspect=0 ema=2494 ma=2504 valid=1
30 raw=2822 cal=3445 sat=0 suspect=1 ema=2506 ma=2528 valid=1
31 raw=2067 cal=2523 sat=0 suspect=0 ema=2507 ma=2535 valid=1
```

Az első minta `ema=2461` értéke nem felkúszás eredménye: az `ema_update()` az első érvényes mintánál inicializálja a szűrőt, ezért azonnal definiált értéket ad. A 30. mintánál látható tüske kalibrált értéke 3445 lenne, de a `reject_outlier` az előző szűrt értékhez (2494) képest 200-zal korlátozza, azaz 2694-re, és `suspect=1`-et ad. Az EMA így alig mozdul. A kimenet

determinisztikus, mert a `sim_raw()` rögzített lineáris kongruenciás zajt és csak a 30. lépésen tüskét ad; a `native_sim` ütemezése csak a sorok időzítését befolyásolja, az értékeket nem.

Továbbvitt eredmény. A `~/iot-labor/i014/src/sensor_filter.h`-ban definiált `sensor_sample_t` struktúra (benne `raw`, `value`, `valid`, `saturated`, `suspect`), valamint a `calibration_apply`, `ema_update` és `reject_outlier` függvények alkotják a mérési lánc feldolgozó magját. A Adatformátumok: JSON, CBOR, Protocol Buffers, SenML alfejezet ebből a struktúrából származtatja a telemetria sémát, az Helyi feldolgozás: szűrés, aggregálás, eseménydetektálás pedig ezt a modult használja az edge-szűréshez.

6.2.5 Ellenőrzés és hibakeresés

A labor ellenőrzéséhez figyeljük a kimenet stabilitását. A 25,00 °C alapjel körül az EMA értéknek 1–2 tizedesnyi ingadozás után 2500 körül kell maradnia. A `suspect` jelzésnek minden 30. mintánál meg kell jelennie, és a `valid` jelzésnek `1`-nek kell maradnia, mert a generált tüske nem lépi át a `raw_max` határt.

Tünet	Ok	Javítás
Az EMA nem simít, hanem pontosan követi a nyers jelet	`denom` túl kicsi (pl. 1)	Állítsd `ema_init(&ema, 16)`-ra
A kimenet túl lassú, a hőmérsékletváltozás késik	`denom` túl nagy	Csökkentsd 4–8 közé
A `cal_value` negatív vagy irreálisan nagy	`slope_num/den` vagy `offset_scaled` hibás	Ellenőrizd a két kalibrációs pontot
A `valid` mindig 0	`raw_min`/`raw_max` szűkebb a generált jel tartományánál	Állítsd a határokat a mock jelhez
A `suspect` nem jelenik meg tüskénél	`limit` túl nagy	Csökkentsd 200 alá
A mozgóátlag eleinte jelentősen eltér	A puffert `0`-val inicializáltad, nem a jel kezdeti értékével	Első mintával töltsd fel a puffert

6.2.6 Tipikus hibák és félreértések

1. **A szaturált ADC-t valós mérésnek tekintik.** Ha a nyers kód a tartomány szélén áll, a kalibrált érték matematikailag kiszámolható, de fizikailag már nem megbízható. Mindig külön `saturated` vagy `valid` jelzést kell továbbítani.
2. **A kiugrók eldobásával elveszítik a gyors fizikai eseményt.** A `reject_outlier` vagy mediánszűrő önmagában nem tudja eldönteni, hogy a hirtelen változás zaj vagy valós folyamat-e. Ezért a `suspect` jelzést el kell menteni, és ha több egymást követő gyanús minta érkezik, az valószínűleg nem zaj.
3. **Az EMA shift implementálása negatív delta esetén.** A `delta >> shift` negatív `delta` mellett GCC-n tipikusan aritmetikai shift, de a C szabvány ezt nem garantálja. Hordozható kódban `delta / (1 << shift)` vagy az itt használt `delta / denom` alakot alkalmazzuk.
4. **A mintavételi frekvenciát csak a Nyquist-kritérium alapján választják meg.** A lassú környezeti jeleknél a hangsúly nem a Shannon–Nyquist feltételen van, hanem a zajcsökkentés, az energia és a reakcióidő közötti kompromisszumon. A másodperces

mintavétel ritkán szükséges, de ha van energiakeret, akkor a túlmintavételezés és átlagolás jobb jel-zaj viszonyt adhat.

6.2.7 Feladatok

1. **Fogalmi kérdés.** Miért nem elég a mintavételi frekvenciát a jel maximális frekvenciájához igazítani egy zajos IoT-érzékelőnél? Milyen további szempontokat kell figyelembe venni?
2. **Számolási feladat.** Egy hőmérséklet-érzékelő nyers ADC-kódja 1000-nél 10,00 °C-ot, 3000-nél 30,00 °C-ot jelent. Határozd meg a kétpontos kalibráció egyenletét 0,01 °C skálán, és add meg a hőmérsékletet ADC = 2200 esetén. Használd az $y = mx + b$ alakot.
3. **Implementációs feladat.** A labor `main.c`-jében módosítsd a `reject_outlier` hívást úgy, hogy gyanús minta esetén a kalibrált értéket ne korlátozza, hanem teljesen dobja el, és az EMA/MA ne kapjon új mintát. Ellenőrizd, hogy a tüske után a szűrt jel kevésbé mozdul-e el.
4. **Kiterjesztési feladat.** A labor mozgóátlag ablakát növeld 8-ról 16-ra, az EMA `denom`-ját pedig csökkentsd 16-ról 4-re. Futtasd mindkettőt ugyanazzal a mock jellel, és hasonlítsd össze a késleltetést egy 5,00 °C-os ugrásnál. Melyik szűrő követi gyorsabban a valós változást, és miért?

6.2.8 Megoldások

1. A Nyquist-kritérium csak az alulmintavételezésből fakadó torzítást kerüli el. Zajos jelnél túlmintavételezésre van szükség ahhoz, hogy a zajt átlagolással csökkenthessük. Emellett figyelembe kell venni a szenzor konverziós idejét, a rádióadás energiaigényét, a processzor alvó üzemmódjait és a vezérlő hurok válaszidejét.
2. Skála: 0,01 °C, tehát $y_1 = 1000$, $y_2 = 3000$, $x_1 = 1000$, $x_2 = 3000$.

$$m = \frac{3000 - 1000}{3000 - 1000} = \frac{2000}{2000} = 1,0 \text{ (0,01 °C)/LSB},$$

$$b = 1000 - 1 \cdot 1000 = 0.$$

ADC = 2200 esetén:

$$y = 1 \cdot 2200 + 0 = 2200,$$

azaz 22,00 °C. Ha a skála 0,1 °C lenne, az együtthatók tízszer nagyobbak lennének.

3. A lényeg: a gyanús mintánál `cal_value`-t nem adjuk át a szűrőnek. A `main.c` megfelelő blokkja így módosul:

```
if (valid) {
    /* Az első érvényes mintánál az EMA még inicializálatlan. */
    int32_t previous = ema.initialized ? ema.y : cal_value;
    /* Csak a gyanús minta tényét vizsgáljuk; a korlátozott értéket nem
     * használjuk, mert a gyanús mintát teljesen eldobjuk. */
    (void)reject_outlier(cal_value, previous, 200, &suspect);
    if (!suspect) {
        ema_update(&ema, cal_value);
        ma_update(&ma, cal_value);
    }
}
printk("%u raw=%d cal=%d suspect=%d ema=%d ma=%d valid=%d\n",
       step, raw, cal_value, suspect, ema.y, ma_value(&ma), valid);
```

Gyanús mintánál a `ma` belső állapota (puffer, összeg, index, elemszám) változatlan marad, és az EMA `y/initialized` mezője sem módosul. A naplózás az `ma_value()` olvasó getterrel történik, amely csak kiolvassa az átlagot; a szűrő állapotát a megfigyelés nem változtatja meg. A 30. lépés tüskéje így nem befolyásolja a szűrt jelet: az EMA a 30. mintánál is az előző értéken (2494) marad, szemben az eredeti, korlátozó változat 2506-os értékével. A nyers kalibrált értéket és a `suspect` jelzést továbbra is naplózzuk, hogy a tüske ténye ne tűnjön el.

4. A mozgóátlag 16 elemmel az ugrás után 16 mintán keresztül lineárisan éri el az új értéket; az EMA $D = 4$ esetén gyorsabban, körülbelül $1/\alpha = 4$ mintányi időállandóval reagál. A mozgóátlag késleltetése $N/2 = 8$ minta, az EMA késleltetése $\approx (1-\alpha)/\alpha = 3$ minta. Ezért a kisebb nevezőjű EMA gyorsabban követi a valós ugrást, de több zajt enged át.

6.2.9 Rövid összefoglaló

A mérési lánc feldolgozó része a nyers ADC-értékből kalibrált, szűrt, fizikai egységben értelmezett mérést állít elő. A kétpontos kalibráció $y = mx + b$ egyenletével a szenzor nyers kódját 0,01 °C vagy 0,01 %RH skálára képezzük le. A zaj csökkentésére mozgóátlagot vagy fixpontos exponenciális szűrőt használunk; az EMA $y_n = y_{n-1} + \Delta/D$ formában egyetlen egész osztással számolható. A kiugró értékeket delta-korláttal szűrjük, és `suspect` jelzést adunk, a tartomány szélén pedig `saturated` és/vagy `valid=false` jelzéssel óvjuk a fogyasztót a hamis méréstől. A laborban elkészült a `sensor_filter.h` modul a `sensor_sample_t` struktúrával, amelyet az Adatformátumok: JSON, CBOR, Protocol Buffers, SenML és az Helyi feldolgozás: szűrés, aggregálás, eseménydetektálás alfejezetek használnak tovább.

6.2.10 Források és ellenőrzési hiányok

- Zephyr Project dokumentáció (https://docs.zephyrproject.org/latest/): a `native_sim` board, a `printk` és a `k_msleep` API aktuális neveit ellenőrizni kell.
- A kalibrációs együtthatók és a mock jel ebben az alfejezetben illusztratívak; valós szenzorhoz az adatlapot és a gyártói kalibrációs ajánlást kell használni.
- Az EMA fixpontos kerekítésének viselkedése C szabvány szerint az osztással hordozható, de negatív operandusú jobbra shift esetén implementációfüggő.

6.3 Aktuátorok: biztonságos vezérlés és visszajelzés

Az aktuátorok vezérlése az a pont, ahol az IoT-rendszer visszahat a fizikai világra. Egy szenzor hibás olvasása kellemetlen, de egy hibásan nyitva maradt öntözőszelep elárasztható egy üvegházat,

vagy egy beragadt fűtőrelé tönkreteheti a termést. A TerraSense üvegházában a talajnedvesség-szenzorok adatai alapján akarjuk nyitni és zárni az öntözőszelepet; ehhez olyan szoftver kell, amelyik akkor is biztonságos állapotba kerül, ha egy parancs elvész, a tápfeszültség pillanatra kimarad, vagy a visszajelző érzékelő meghibásodik.

6.3.1 Aktuátorok és vezérlő kimenetek

Definíció. Az *aktuátor* (actuator) olyan eszköz, amely elektromos jelet fizikai hatássá alakít: mozgássá, hővé, fénnyé vagy áramlássá. IoT-kontextusban jellemzően szelep, relé, motor, fűtőszál vagy LED.

A mikrovezérlő kimenete általában logikai szintű GPIO jel, amely ritkán tud közvetlenül nagy teljesítményű terhelést kapcsolni. A leggyakoribb felépítés az, hogy a GPIO egy *relét* vagy egy *félvezető kapcsolót* (tranzisztort, MOSFET-et, SSR-t) vezérel, az pedig a tényleges eszközt kapcsolja. Egy mágnesszelep (solenoid valve) esetén a nyitáshoz és a záráshoz is lehet külön kimenet, de az ipari és kerti öntözőrendszerekben sokszor *normally closed* (NC) szelepet használnak: áram hatására nyit, áramtalanítva zár. Ez már egy passzív fail-safe tulajdonság.

```c
/* GPIO-kimenet egy szelep számára Zephyr-ben */
#include <zephyr/drivers/gpio.h>
#include <zephyr/device.h>
#include <errno.h>

#define VALVE_NODE DT_ALIAS(valve)
static const struct gpio_dt_spec valve_gpio =
    GPIO_DT_SPEC_GET(VALVE_NODE, gpios);

int valve_hw_open(const struct gpio_dt_spec *gpio, bool open)
{
    int rc;

    if (gpio == NULL || !device_is_ready(gpio->port)) {
        return -ENODEV;
    }
    /* NC szelep: 1 = nyit, 0 = zár */
    rc = gpio_pin_set_dt(gpio, open ? 1 : 0);
    return rc < 0 ? rc : 0;
}
```

Ha a szelep nem csak kétállapotú, hanem arányosan állítható (például motoros keverőszelep), akkor PWM kimenetet használunk. A PWM kitöltési tényezője határozza meg a motor pillanatnyi sebességét vagy egy fűtőelem teljesítményét, és a vezérlő szoftverben ugyancsak egy 0–100% közötti értékkel modellezhető.

A biztonságos alapállapot két szinten jelenik meg:

1. **Hardveres alapállapot:** a kimenet áramtalanítva milyen állapotba kerül. NC szelepnél ez zárt, egy fűtőrelénél kikapcsolt, egy motoros zsaluzónál nyitott vagy zárt attól függően, hogy mi a biztonságos az adott folyamatra nézve.
2. **Szoftveres alapállapot:** ha a vezérlő szoftver fut, de nem kap érvényes frissítést vagy hibát észlel, milyen állapotot ír elő. Ezt watchdoggal, időkorláttal és szándékos „safe state” függvénnyel valósítjuk meg.

A TerraSense öntözőszelepének biztonságos alapállapota **zárt**, tehát a szoftveres fail-safe is zárást kell, hogy előírjon, ha elveszett a kapcsolat vagy lejárt a maximális öntözési idő.

6.3.2 Szelepvezérlés időkorláttal és visszajelzéssel

Definíció. A *maximális bekapcsolási idő* (maximum on-time) az az időtartam, ameddig egy aktuátor megszakítás nélkül aktív állapotban maradhat. Ha a vezérlő eddig nem kapott érvényes leállító parancsot, akkor az aktuátort saját hatáskörben biztonságos állapotba állítja.

A maximális bekapcsolási idő véd a szoftverhibák, eltűnt hálózati parancsok és hibás szenzoradatok ellen. Például ha a talajnedvesség-szenzor beragad „száraz” értékre, az öntözés ne folyhasson órákig: a vezérlő legfeljebb 10 percig tarthatja nyitva a szelepet, utána automatikusan zár.

A visszajelzés (feedback) azt jelenti, hogy a rendszer nem csak kiadja a parancsot, hanem ellenőrzi is az aktuátor tényleges állapotát. Egy NC mágnesszelepnél a visszajelzés lehet:

- **Áramkör-megszakításos mikrokapcsoló** a szelep végállásáról.
- **Átfolyásmérő** vagy nyomáskapcsoló a vezetékben.
- **Áramérzékelő** a relé kimenetén, amely azt mutatja, hogy valóban folyik-e áram a tekercsbe.

Ha a parancsolt állapot (`requested`) és a visszajelzett állapot (`reported`) tartósan eltér, akkor hibát jelzünk: a szelep lehet, hogy mechanikusan beragadt, a relé nem kapcsolt, vagy a visszajelző érzékelő hibás. Az ilyen eltérésre a biztonságos válasz általában az, hogy a szelepet megkíséreljük zárni, a hibát pedig naplózzuk és jelentjük a backend felé. A visszajelzés nélküli nyílt hurkú vezérlés csak akkor elfogadható, ha a kockázat kicsi, és a rendszer más szinten (például túlfolyásgátló, mechanikus nyomáskapcsoló) már védve van.

```
/* Szelepvezérlő állapot időkorláttal és visszajelzéssel */
#define VALVE_MAX_ON_LIMIT_MS (60LL * 60LL * 1000LL)  /* 1 óra felső korlát */

struct valve_ctrl {
    const struct gpio_dt_spec gpio;   /* kimeneti GPIO */
    bool requested_open;              /* parancsolt állapot */
    bool reported_open;               /* visszajelzett állapot */
    int64_t on_deadline_ms;           /* mikor kell lezárni */
    int64_t max_on_time_ms;           /* maximális bekapcsolási idő */
    bool hw_present;                  /* szimulációban false */
};

int valve_init(struct valve_ctrl *v, int64_t max_on_time_ms, bool hw_present)
{
    int rc;

    if (!v) return -EINVAL;
    if (max_on_time_ms <= 0 || max_on_time_ms > VALVE_MAX_ON_LIMIT_MS) {
        return -EINVAL;
    }

    v->requested_open = false;
    v->reported_open = false;
    v->on_deadline_ms = 0;
    v->max_on_time_ms = max_on_time_ms;
    v->hw_present = hw_present;

    /* Ellenőrzött alapállapot: a kimenet zárt, mielőtt bármit nyitnánk. */
    if (hw_present) {
        rc = valve_hw_open(&v->gpio, false);
        if (rc < 0) {
            return rc;
        }
    }
    return 0;
}
```

```
int valve_set_target(struct valve_ctrl *v, bool open)
{
    int rc;

    if (!v) return -EINVAL;

    if (v->requested_open == open) {
        /* Idempotens: azonos parancs nem vált, és nem hosszabbítja meg
         * a biztonsági maximális nyitvatartási határidőt. */
        return 0;
    }

    if (!v->hw_present) {
        /* natív szimulációban csak naplózzuk */
        v->reported_open = open;
    } else {
        rc = valve_hw_open(&v->gpio, open);
        if (rc < 0) {
            /* Sikertelen hardverírás: a kért állapot nem lép életbe,
             * a biztonságos alapállapot zárt. */
            (void)valve_hw_open(&v->gpio, false);
            return rc;
        }
    }
    v->requested_open = open;
    if (open) {
        v->on_deadline_ms = k_uptime_get() + v->max_on_time_ms;
    } else {
        v->on_deadline_ms = 0;
    }
    return 0;
}

int valve_tick(struct valve_ctrl *v)
{
    if (!v) return -EINVAL;

    /* 1. időkorlát ellenőrzése */
    if (v->requested_open && v->on_deadline_ms != 0 &&
        k_uptime_get() >= v->on_deadline_ms) {
        printk("valve: max on-time exceeded, forcing close\n");
        return valve_set_target(v, false);
    }

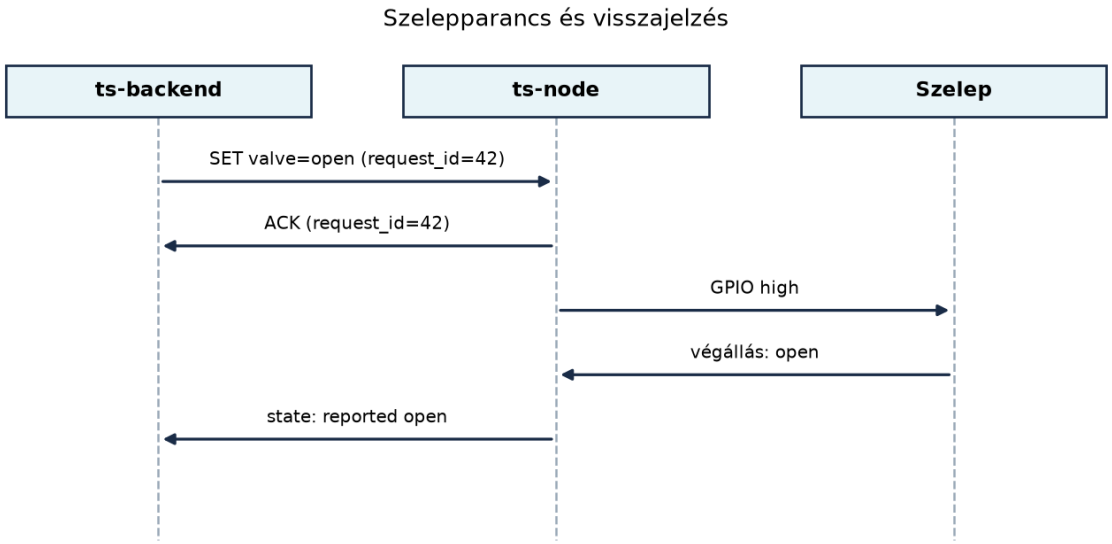
    /* 2. visszajelzés olvasása valós hardveren */
    if (v->hw_present) {
        /* pl. egy bemeneti GPIO-t olvasunk */
        /* v->reported_open = gpio_pin_get_dt(&sense_pin); */
    }

    /* 3. eltérés kezelése */
    if (v->reported_open != v->requested_open) {
        /* rövid türelmi idő után hiba; itt most azonnal jelezzük */
        printk("valve: feedback mismatch, req=%d rep=%d\n",
               v->requested_open, v->reported_open);
        return -EIO;
    }
    return 0;
}
```

A fenti kód a hardveres állapotgép vázát mutatja: a `valve_hw_open()` hibakódot ad vissza, a `valve_init()` pedig csak pozitív, egy óránál nem nagyobb maximális bekapcsolási időt fogad

el, és ellenőrzi a kimenet zárt alapállapotát. A visszajelző bemenet konkrét GPIO-ját a board devicetree-je és a későbbi labor adja meg; a 6.3.2-ben ezért szerepel kommentként a beolvasás. A laborban a `hw_present` értéke `false`, ezért a GPIO-írás és a visszajelzés-olvasás nem fut le: a labor a biztonsági logikát szimulált állapotátmenetekkel mutatja be, a valós hardveres zárást és visszajelzést éles eszközön kell igazolni.

A fenti `valve_set_target()` függvény **idempotens**: ugyanazzal az állapottal többször meghívva nem váltogat, hanem ugyanazt az eredményt adja. Ez döntő különbség a *váltás* (toggle) szemantikához képest. Ha az MQTT-kapcsolat rövid megszakadása miatt ugyanaz a „nyisd ki" parancs kétszer érkezik meg, a toggle parancs kétszeri végrehajtása hibásan zárt állapotot eredményezne; az idempotens `set_target(true)` viszont egyszer nyit, másodjára semmit sem változtat. Az ismételt OPEN parancs a biztonsági maximális nyitvatartási határidőt sem indítja újra: a parancsdeduplikáció és a védelmi időkorlát külön mechanizmus, különben sűrű ismétléssel korlátlanul nyitva lehetne tartani a szelepet. Az idempotencia tehát a parancs **megismételhetőségét** biztosítja, ami az at-least-once kézbesítés (QoS 1) természetes következménye.



6.3. ábra: Szelepparancs és visszajelzés - A tervezett MQTT-integráció előkészítése; a laborban a backend még nem fut

Az ábrán látható, hogy a parancsra a csomópont először nyugtát küld, majd a tényleges állapotot jelenti. A nyugta önmagában még nem bizonyítja, hogy a szelep mechanikusan is átváltott; ezért kell a külön visszajelző üzenet. A MQTT QoS, munkamenetek, retain, Last Will és MQTT 5 alfejezetben látjuk majd, hogyan lehet ezt a nyugtázást QoS 1-gyel megbízhatóvá tenni.

6.3.3 Döntési szempontok

Szempont	Relé	Félvezető kapcsoló (SSR/MOSFET)
Mechanikus élettartam	jellemzően 10^5–10^7 kapcsolás	nincs mozgó alkatrész
Kapcsolási sebesség	lassú (ms)	gyors (µs)
Galvanikus leválasztás	általában van	nincs, vagy külön optocsatoló kell

Szempont	Relé	Félvezető kapcsoló (SSR/MOSFET)
Terhelhetőség (AC/DC)	mindkettő	főleg DC, AC-hez TRIAC/SSR
Zaj, csattogás	hallható	nincs

A TerraSense öntözőszelepe jellemzően 24 V AC mágnesszelep, ezért relé vagy AC SSR jöhet szóba. A leggyakoribb hobbi és ipari megoldás a mechanikus relé, mert olcsó, egyszerű, és a szelep ritkán kapcsol (naponta néhányszor). Ha a kapcsolások száma nagy, vagy PWM-arányos vezérlés kell, akkor félvezetőt választunk.

A visszajelzés típusa szerint is dönthetünk:

- **Nincs visszajelzés:** elfogadható, ha a rendszer más módon védett (pl. mechanikus nyomáskapcsoló, túlfolyásgátló).
- **Végálláskapcsoló:** kétállapotú szelepnél elég.
- **Analóg visszajelzés:** arányos szelepnél potenciométer vagy 4–20 mA-es jeladó.

A TerraSense-ben a labor első változatában a visszajelzést szimuláljuk, de a szoftver interfészét úgy alakítjuk ki, hogy később valós GPIO-visszajelzésre cserélhető legyen.

6.3.4 Labor: `ts-node` szelepvezérlés szimulált GPIO-val

Munkakönyvtár: `~/iot-labor/i015`

```
mkdir -p ~/iot-labor/i015 && cd ~/iot-labor/i015
```

A labor célja egy olyan szelepvezérlő modul elkészítése, amelyet a későbbi alfejezetekben MQTT-parancsfeldolgozóval kötünk össze. Most a parancsot egy egyszerű szimulált bemenetről kapjuk, a GPIO-t pedig a `native_sim` célon csak naplózzuk.

Fájlok:

`valve.h`

```
#ifndef VALVE_H
#define VALVE_H

#include <zephyr/kernel.h>
#include <zephyr/device.h>
#include <zephyr/drivers/gpio.h>

#define VALVE_MAX_ON_LIMIT_MS (60LL * 60LL * 1000LL)  /* 1 óra felső korlát */
#define VALVE_FEEDBACK_GRACE_MS 2000                  /* türelmi idő */

struct valve_ctrl {
    const struct gpio_dt_spec *output;    /* kimenet; szimulációban NULL */
    const struct gpio_dt_spec *feedback;  /* visszajelzés; lehet NULL */
    bool requested_open;
    bool reported_open;
    bool hw_present;
    int64_t on_deadline_ms;
    int64_t max_on_time_ms;
    int64_t mismatch_since_ms;            /* példányonkénti türelmi idő */
};

int valve_init(struct valve_ctrl *v, int64_t max_on_time_ms,
```

```
                bool hw_present);
int valve_set_io(struct valve_ctrl *v, const struct gpio_dt_spec *output,
                 const struct gpio_dt_spec *feedback);
int valve_set_target(struct valve_ctrl *v, bool open);
bool valve_is_open(const struct valve_ctrl *v);
int valve_tick(struct valve_ctrl *v);
#endif
```

valve.c

```
#include "valve.h"
#include <zephyr/kernel.h>
#include <zephyr/logging/log.h>
#include <errno.h>

LOG_MODULE_REGISTER(valve, LOG_LEVEL_INF);

static int valve_hw_write(const struct valve_ctrl *v, bool open)
{
    int rc;

    if (v->output == NULL || !device_is_ready(v->output->port)) {
        return -ENODEV;
    }
    rc = gpio_pin_set_dt(v->output, open ? 1 : 0);
    if (rc < 0) {
        LOG_ERR("valve: GPIO write failed: %d", rc);
    }
    return rc;
}

int valve_init(struct valve_ctrl *v, int64_t max_on_time_ms,
               bool hw_present)
{
    if (!v) return -EINVAL;
    if (max_on_time_ms <= 0 || max_on_time_ms > VALVE_MAX_ON_LIMIT_MS) {
        return -EINVAL;
    }

    v->output = NULL;
    v->feedback = NULL;
    v->requested_open = false;
    v->reported_open = false;
    v->hw_present = hw_present;
    v->on_deadline_ms = 0;
    v->max_on_time_ms = max_on_time_ms;
    v->mismatch_since_ms = -1;
    return 0;
}

int valve_set_io(struct valve_ctrl *v, const struct gpio_dt_spec *output,
                 const struct gpio_dt_spec *feedback)
{
    int rc;

    if (!v) return -EINVAL;
    if (v->hw_present) {
        if (output == NULL || !device_is_ready(output->port)) {
            return -ENODEV;
        }
        /* Ellenőrzött alapállapot: zárás, mielőtt bármit nyitnánk. */
        rc = gpio_pin_set_dt(output, 0);
        if (rc < 0) {
```

```
            return rc;
        }
    }
    v->output = output;
    v->feedback = feedback;
    return 0;
}

int valve_set_target(struct valve_ctrl *v, bool open)
{
    int rc;

    if (!v) return -EINVAL;

    if (v->requested_open == open) {
        /* Idempotens: azonos parancs nem vált, és nem hosszabbítja meg
         * a biztonsági maximális nyitvatartási határidőt. */
        return 0;
    }

    if (!v->hw_present) {
        v->reported_open = open;
        LOG_INF("valve: simulated %s", open ? "open" : "closed");
    } else {
        rc = valve_hw_write(v, open);
        if (rc < 0) {
            /* Hardverhiba: a kért állapot nem lép életbe, a biztonságos
             * alapállapot pedig zárt; ezt megkíséreljük fenntartani. */
            (void)valve_hw_write(v, false);
            return rc;
        }
        if (v->feedback == NULL) {
            /* Nyílt hurkú hardver: a parancsot tekintjük állapotnak. */
            v->reported_open = open;
        }
    }
    v->requested_open = open;
    if (open) {
        v->on_deadline_ms = k_uptime_get() + v->max_on_time_ms;
    } else {
        v->on_deadline_ms = 0;
    }
    return 0;
}

bool valve_is_open(const struct valve_ctrl *v)
{
    return v ? v->reported_open : false;
}

int valve_tick(struct valve_ctrl *v)
{
    if (!v) return -EINVAL;

    if (v->requested_open && v->on_deadline_ms != 0 &&
        k_uptime_get() >= v->on_deadline_ms) {
        LOG_WRN("max on-time exceeded, forcing close");
        return valve_set_target(v, false);
    }

    /* A visszajelző GPIO olvasását és a türelmi időt a 3. feladat
     * valósítja meg; a labor szimulált módjában reported = requested. */
```

```
    if (v->reported_open != v->requested_open) {
        LOG_ERR("feedback mismatch: requested=%d reported=%d",
                v->requested_open, v->reported_open);
        return -EIO;
    }
    return 0;
}
```

main.c

```
#include <zephyr/kernel.h>
#include <zephyr/logging/log.h>
#include "valve.h"

LOG_MODULE_REGISTER(main, LOG_LEVEL_INF);

int main(void)
{
    struct valve_ctrl valve;
    int err;

    /* max 10 másodperc; hw_present = false: szimulált GPIO */
    err = valve_init(&valve, 10000, false);
    if (err) {
        LOG_ERR("valve_init failed: %d", err);
        return err;
    }

    /* szimulált parancs: nyitás */
    err = valve_set_target(&valve, true);
    if (err) {
        LOG_ERR("valve_set_target failed: %d", err);
    }

    while (1) {
        err = valve_tick(&valve);
        if (err == -EIO) {
            LOG_ERR("valve feedback error, closing");
            valve_set_target(&valve, false);
        }
        k_sleep(K_MSEC(1000));
    }
    return 0;
}
```

CMakeLists.txt

```
cmake_minimum_required(VERSION 3.20.0)
find_package(Zephyr REQUIRED HINTS $ENV{ZEPHYR_BASE})
project(ts_node_actuator)

target_sources(app PRIVATE src/main.c src/valve.c)
```

prj.conf

```
CONFIG_LOG=y
```

A `src/` alkönyvtárba másoljuk a forrást; mivel a Zephyr alapértelmezés szerint a `src/` könyvtárat keresi, a `CMakeLists.txt` explicit módon adja meg a fájlokat.

Build és futtatás:

```
west build -b native_sim -d build
./build/zephyr/zephyr.exe
```

Várt kimenet (illusztratív, a log időbélyegei nélkül):

```
[00:00:00.000,000] <inf> valve: simulated open
[00:00:10.000,000] <wrn> valve: max on-time exceeded, forcing close
[00:00:10.000,000] <inf> valve: simulated closed
```

A kimenet mutatja, hogy a szelep 10 másodperc után automatikusan zár, még akkor is, ha a `main` nem ad le záró parancsot. A szimulációban a visszajelzés azonnal követi a parancsot; valós hardveren a `valve_tick` olvasná a bemeneti GPIO-t, és eltérés esetén hibát jelezne. A labor `hw_present = false` értékkel fut, ezért a GPIO-írás és a hardveres zárás nem valósul meg: a biztonsági logika (időkorlát, állapotátmenetek) szimulált, a tényleges hardveres viselkedést éles eszközön, a `valve_set_io()` által beállított kimenettel és a 3. feladatban hozzáadott visszajelzés-olvasással kell igazolni.

6.3.5 Ellenőrzés és hibakeresés

A labor futtatásakor a következőket ellenőrizzük:

1. A nyitás után 10 másodperccel a `valve_tick` valóban lezár, és a logban `max on-time exceeded` üzenet jelenik meg.
2. Ha kétszer egymás után hívjuk meg a `valve_set_target(&valve, true)` függvényt, a második hívás nem vált, és a `requested_open` megmarad `true`-nak.
3. Ha a visszajelzést szándékosan eltérítjük (pl. a szimulált `reported_open` értékét átírjuk `false`-ra, miközben `requested_open` `true`), a `valve_tick` `-EIO` hibát ad, és a `main` lezárja a szelepet.

Hibatünet	Ok	Javítás
A szelep azonnal visszazár a nyitás után	A maximális bekapcsolási idő túl rövid, vagy a `max_on_time_ms` nullára inicializált	Ellenőrizzük a `valve_init` hívás paraméterét; tegyük nagyobbra, vagy javítsuk a kezdőértéket
A visszajelzés eltérés hibát jelez, bár a szelep működik	A visszajelző GPIO bemenete rosszul van konfigurálva, vagy a mechanikus végálláskapcsoló késlekedik	A 3. feladat példányonkénti türelmi ideje legyen legalább néhány másodperc, és csak ezután jelezzünk hibát
A GPIO-írás nem látszik a logban	A `hw_present` értéke hamis, pedig hardvert csatlakoztatnánk	Állítsuk `true`-ra a `valve_init` hívásban, hívjuk meg a `valve_set_io()`-t érvényes kimeneti GPIO-val, és a board overlay-jében adjuk meg a csomópontot
A `native_sim` build hibát ad a GPIO driver hiánya miatt	A `native_sim` nem támogat valós GPIO-t	Használjunk szimulált módot (`hw_present=false`) a natív teszthez, és a hardveres GPIO-t csak valós board overlay-jel fordítsuk

6.3.6 Tipikus hibák és félreértések

1. **Toggle parancs használata idempotens állapotbeállítás helyett.** Ha a backend `toggle_valve()` szemantikát küld, akkor egy elveszett nyugta miatti újraküldés kétszer

fordítja meg az állapotot, és a szelep éppen az ellenkezőjére kerül. A parancs mindig konkrét célállapotot írjon elő: `set_valve(true)` vagy `set_valve(false)`.

2. **A maximális bekapcsolási idő elhanyagolása.** Ha csak a backend parancsára hagyatkozunk, akkor egy hálózati kiesés vagy szoftverhiba esetén a szelep végtelen ideig nyitva maradhat. Az időkorlát az eszköz helyi védelme, és a backendtől függetlenül működnie kell.
3. **A visszajelzés azonnali ellenőrzése mechanikus rendszerben.** A relé és a szelep mechanikus átállása több tíz milliszekundum is lehet; ha azonnal olvassuk a végálláskapcsolót, hamis hibát kapunk. Türelmi időt kell beiktatni.
4. **A fail-safe alapállapot félreértése.** A szoftveres fail-safe nem helyettesíti a hardveresen biztonságos állapotot. Ha a mikrokontroller lefagy, a szoftveres zárás nem fut le; ezért kell az NC szelep vagy külső védőrelé.

6.3.7 Feladatok

1. **Fogalmi, döntési kérdés:** Miért idempotens a `valve_set_target(open)` parancs, és miért hibás az MQTT-alapú parancskiadásnál a `valve_toggle()`? Mondjon egy konkrét forgatókönyvet, ahol az utóbbi kárt okoz.
2. **Számolási, tervezési feladat:** Egy 24 V-os mágnesszelep tekercse 0,5 A áramot vesz fel nyitott állapotban. Ha a szelep naponta átlagosan 6 percet van nyitva, mennyi energiát fogyaszt havonta (30 nap)? Mennyi legyen a maximális bekapcsolási idő, ha egy öntözési ciklus legfeljebb 0,5 liter vizet engedhet át, és a szelep átfolyása 2 l/min? (Az energia számításához használja a $P = U \cdot I$ összefüggést.)
3. **Implementációs feladat a TerraSense-ben:** Bővítse a labor `valve_tick` függvényét a `valve_ctrl` példányonkénti `feedback` GPIO-bemenetével: ha a `hw_present` igaz és a `feedback` nem NULL, olvassa le a GPIO-t, és rendelje hozzá a `reported_open` mezőhöz. Adjon hozzá 2 másodperc türelmi időt, mielőtt eltérést jelezne.
4. **Hibakeresési, kiterjesztési feladat:** A laborban a `valve_set_target(false)` nem naplózza a zárást, ha a szelep már zárva volt. Módosítsa a függvényt úgy, hogy minden állapotváltozást (nyitás és zárás) logoljon, akkor is, ha az idempotens hívás miatt nincs tényleges váltás. Indokolja, miért fontos ez a nyomkövetés szempontjából.

6.3.8 Megoldások

1. A `set_target(open)` ugyanazt a célállapotot írja elő, így ha a parancs kétszer érkezik meg, másodszor nem változtat. A `toggle()` viszont minden híváskor ellenkező állapotba viszi a kimenetet. Tegyük fel, hogy a backend `toggle()` parancsot küld, mert a felhasználó rákattintott egy gombra, de a nyugta elveszik, és az MQTT QoS 1 újraküldés miatt a parancs másodszor is megérkezik. Az első hívás nyitja, a második zárja a szelepet, pedig a felhasználó nyitni akart. Ha a parancs `set_target(true)` lett volna, a második hívás már nem változtatott volna.

2. A szelep napi energiafogyasztása nyitott állapotban: $P = 24\ \text{V} \cdot 0{,}5\ \text{A} = 12\ \text{W}$. Naponta 6 perc = 0,1 óra, tehát $E_{\text{nap}} = 12\ \text{W} \cdot 0{,}1\ \text{h} = 1{,}2\ \text{Wh}$. Havonta $E_{\text{hónap}} = 1{,}2\ \text{Wh} \cdot 30 = 36\ \text{Wh}$. A maximális bekapcsolási idő: ha legfeljebb 0,5 liter vizet szabad átengedni, és az átfolyás 2 l/min, akkor $t_{\max} = \frac{0{,}5\ \text{l}}{2\ \text{l/min}} = 0{,}25\ \text{min} = 15\ \text{s}$.

3. A `valve_tick` bővítése türelmi idővel:

```
int valve_tick(struct valve_ctrl *v)
{
    int rc;
```

```
    if (!v) return -EINVAL;

    if (v->requested_open && v->on_deadline_ms != 0 &&
        k_uptime_get() >= v->on_deadline_ms) {
        LOG_WRN("max on-time exceeded, forcing close");
        return valve_set_target(v, false);
    }

    if (v->hw_present && v->feedback != NULL) {
        if (!device_is_ready(v->feedback->port)) {
            return -ENODEV;
        }
        rc = gpio_pin_get_dt(v->feedback);
        if (rc < 0) {
            /* Az olvasási hiba nem "nyitott" állapot. */
            LOG_ERR("feedback read failed: %d", rc);
            return rc;
        }
        v->reported_open = (rc != 0);
    }

    if (v->reported_open != v->requested_open) {
        int64_t now = k_uptime_get();

        if (v->mismatch_since_ms < 0) {
            v->mismatch_since_ms = now;  /* türelmi idő indul */
        } else if (now - v->mismatch_since_ms > VALVE_FEEDBACK_GRACE_MS) {
            LOG_ERR("feedback mismatch for >%d ms",
                    VALVE_FEEDBACK_GRACE_MS);
            v->mismatch_since_ms = -1;
            return -EIO;
        }
    } else {
        v->mismatch_since_ms = -1;
    }
    return 0;
}
```

A `mismatch_since_ms` a `valve_ctrl` példány mezője, ezért két szelep türelmi ideje nem keveredik; korábbi `static` változóként minden példány között közös lett volna. A `-1` érték azt jelzi, hogy nincs folyamatban lévő eltérés; az első eltérő mintánál indul a türelmi idő, és a `feedback` NULL pointere esetén nem történik GPIO-olvasás. A negatív `gpio_pin_get_dt()` visszatérési értéket külön hibaként kezeljük, nem alakítjuk `true`-ra.

4. A naplózás minden hívásnál akkor is fontos, ha nincs állapotváltozás, mert a parancs ismétlődése és a rendszer viselkedése így rekonstruálható. A módosított `valve_set_target`:

```
int valve_set_target(struct valve_ctrl *v, bool open)
{
    int rc;

    if (!v) return -EINVAL;

    if (v->requested_open == open) {
        /* Megfigyelés: a redundáns parancs is naplózva, de a biztonsági
         * határidő nem indul újra. */
        LOG_DBG("valve: redundant set_target(%s), no change",
                open ? "open" : "closed");
        return 0;
    }

    if (!v->hw_present) {
```

```
        LOG_INF("valve: simulated %s", open ? "open" : "closed");
        v->reported_open = open;
    } else {
        rc = valve_hw_write(v, open);
        if (rc < 0) {
            (void)valve_hw_write(v, false);
            return rc;
        }
        if (v->feedback == NULL) {
            v->reported_open = open;
        }
        LOG_INF("valve: hardware set to %s", open ? "open" : "closed");
    }

    v->requested_open = open;
    if (open) {
        v->on_deadline_ms = k_uptime_get() + v->max_on_time_ms;
    } else {
        v->on_deadline_ms = 0;
    }
    return 0;
}
```

A redundáns hívást `LOG_DBG` szinten naplózzuk; ehhez a `prj.conf`-ban a `CONFIG_LOG_LEVEL_DBG` (vagy a modul szintjének megfelelő beállítása) szükséges. Az idempotens ág a naplózás mellett sem nyúl a `requested_open` és `on_deadline_ms` mezőhöz, így a megfigyelés nem változtatja meg a biztonsági működést.

6.3.9 Rövid összefoglaló

Ebben az alfejezetben megterveztük a TerraSense öntözőszelepének vezérlését. Az aktuátor egy NC mágnesszelep, amelyet GPIO-val és relével kapcsolunk; a biztonságos alapállapot zárt. A `valve_ctrl` modul (fájlok: `valve.h`, `valve.c`, `main.c` a `~/iot-labor/i015/` könyvtárban) idempotens `valve_set_target(open)` parancsot valósít meg, maximális bekapcsolási időt tart, a `valve_tick` pedig a kért és jelentett állapot eltérését hibaként jelzi; a visszajelző GPIO tényleges olvasását és a türelmi időt a 3. feladat adja hozzá. A labor a `native_sim` célon fut, ahol a GPIO-t szimuláljuk; valós hardveren a visszajelző GPIO bemenetet kell bekötni. A továbbvitt eredmény a **`valve_set_target` idempotens parancsfüggvény**, amelyet az MQTT-parancsoknál (MQTT QoS, munkamenetek, retain, Last Will és MQTT 5), a lefelé irányuló parancsoknál (Parancsok és konfiguráció lefelé: kézbesítés, nyugtázás, időkorlát) és az eszközárnyéknál (Eszközárnyék és digitális iker: kívánt és jelentett állapot) használunk fel.

6.3.10 Források és ellenőrzési hiányok

A Zephyr GPIO API (`gpio_pin_set_dt`, `gpio_pin_get_dt`, `GPIO_DT_SPEC_GET`) stabil, a példa a Zephyr aktuális dokumentációjának mintáját követi, de a pontos függvénynevek és makrók verziófüggőek; használat előtt ellenőrizze a telepített Zephyr verziójának dokumentációját. A relé, szelep és félvezető kapcsoló paraméterei (élettartam, kapcsolási idő) nagyságrendi értékek, a konkrét adatlapoktól függenek. A `native_sim` board GPIO-szimulációs képessége korlátozott, ezért a labor szimulált módot használ; a valós hardver bekötése az Opcionális valós hardver: fejlesztőkártyák és szenzorok alfejezetben ismertetett eszközökhöz igazodik.

7 Energiagazdálkodás

7.1 Energiamodell és az elemélettartam kiszámítása

7.1.1 Motiváció

A TerraSense üvegházban a talajnedvesség- és klímaszenzorok elemes csomópontok, amelyeket a termelő nem akar havonta cserélni. A nem funkcionális követelmények között rögzítettük, hogy a `ts-node` eszközöknek legalább 12 hónapig kell működniük egyetlen elemszettel. Ha ezt a számot rosszul becsüljük, akkor a terepre telepített eszközök idő előtt elnémulnak, és a karbantartás költsége felemészti a rendszer előnyeit.

Az energiafogyasztást nem elég egyetlen pillanatban megmérni. Az IoT-csomópontok tipikusan rövid aktív fázisokból és hosszú alvó fázisokból álló ciklusokban működnek, ezért az átlagos áramfelvétel a különböző állapotok áramából és időtartamából kiindulva számítható. Ebben az alfejezetben felépítjük ezt a duty-cycle energiamodellt, kiszámítjuk a TerraSense csomópont élettartamát, és megvizsgáljuk, melyik fogyasztási tétel csökkentése éri meg leginkább.

7.1.2 Fogalmak és működés

A rádiós szenzorcsomópont munkaciklusa olyan állapotok sorozatára bontható, amelyek rendre ismétlődnek. A leggyakoribb állapotok:

Definíció (munkaciklus, duty cycle). A munkaciklus az aktív (nem alvó) idő aránya a teljes ciklusidőhöz képest:

$$D = \frac{\sum_{k \in \text{aktív}} t_k}{T},$$

ahol T a ciklusidő, t_k pedig az egyes aktív állapotok időtartama. Az IoT-eszközök gyakran $D \ll 1$ értékkel dolgoznak, például egy 5 percenként 1 másodpercig adó csomópont esetén $D \approx 0{,}0033$.

Definíció (állapotok áramfelvétele). Egy vezeték nélküli szenzorcsomópont jellemző fogyasztási állapotai:

- **Alvás (sleep):** a mikrovezérlő mély alvó üzemmódban, a rádió kikapcsolva, néhány µA nagyságrendű áramfelvétel (például 1–10 µA).
- **Mérés (measurement):** a szenzor beolvasása, ADC-konverzió, belső logika; jellemzően 1–10 mA, időtartama 10–100 ms.
- **Rádió TX:** adás, a legnagyobb fogyasztású állapot, jellemzően 20–50 mA, időtartama 100–500 ms.
- **Rádió RX:** vétel (például nyugtázás fogadása), jellemzően 10–25 mA, időtartama 50–200 ms.

Ezek az értékek nagyságrendi becslések, amelyek a konkrét mikrovezérlő és rádió adatlapjából ellenőrizendők.

A teljes ciklusidő az egyes állapotidők összege:

$$T = t_{\text{sleep}} + t_{\text{measure}} + t_{\text{TX}} + t_{\text{RX}}.$$

Az átlagáram a súlyozott középérték:

$$I_{\text{avg}} = \frac{\sum_k I_k \cdot t_k}{T}.$$

7.1.2.1 Levezetés: az átlagáram és az élettartam

Legyen a ciklus n állapotból álló sorozat, ahol az k-adik állapotban az eszköz I_k áramot vesz fel t_k ideig. A ciklus alatt elfogyasztott töltés:

$$Q_{\text{ciklus}} = \sum_{k=1}^{n} I_k \cdot t_k.$$

Ezt a $T = \sum t_k$ ciklusidővel elosztva kapjuk az átlagáramot:

$$I_{\text{avg}} = \frac{Q_{\text{ciklus}}}{T} = \frac{\sum_{k=1}^{n} I_k \cdot t_k}{\sum_{k=1}^{n} t_k}.$$

Ha az elem hasznos kapacitása C_{eff} (mAh), akkor a várható élettartam órában:

$$L_{\text{óra}} = \frac{C_{\text{eff}}}{I_{\text{avg}}},$$

ahol I_{avg} mA-ben adott. Az élettartam hónapban, ha egy hónapot 30 nappal, egy napot 24 órával számolunk:

$$L_{\text{hónap}} = \frac{C_{\text{eff}}}{I_{\text{avg}} \cdot 24 \cdot 30}.$$

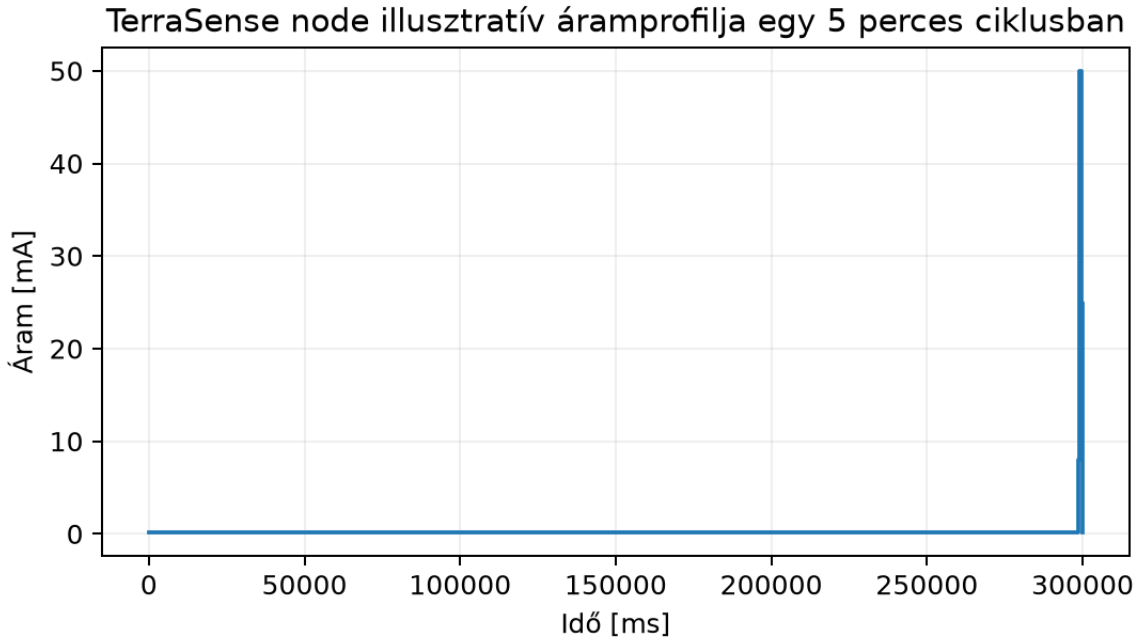
Az önkisülést kétféleképpen vehetjük figyelembe. Az egyszerűbb módszer a névleges kapacitást csökkenti egy biztonsági tényezővel: például egy 2500 mAh névleges kapacitású elemnél 20% tartalék levonásával $C_{\text{eff}} = 2000$ mAh-val számolunk. A pontosabb modell az önkisülést ekvivalens szivárgási áramként adja hozzá az átlagáramhoz:

$$I_{\text{total}} = I_{\text{avg}} + I_{\text{self}},$$

ahol I_{self} az elem kémiai önkisüléséből származó folyamatos veszteség. Évi 2% kapacitásveszteség egy 2500 mAh névleges elemnél körülbelül 50 mAh/év, azaz $\approx 5{,}7$ µA ekvivalens áram; ha ugyanezt a 2000 mAh effektív kapacitásra vetítjük, $\approx 4{,}6$ µA adódik. A továbbiakban a kapacitástartalékos módszert használjuk, mert az könnyebben kezelhető a gyakorlatban.

7.1.2.2 Áramprofil

Az áramfelvétel időben nem folytonos, hanem lépcsős függvény. Az alábbi ábra egy TerraSense-csomópont egyetlen ciklusának illusztratív áramprofilját mutatja.



7.1. ábra: TerraSense node illusztratív áramprofilja egy 5 perces ciklusban - Illusztratív, adatlapi nagyságrendekkel; a lépcsős ábrázolás az állapotváltásokat jelzi. A tényleges értékek a választott MCU és rádió adatlapjából ellenőrizendők.

Az ábrán az időtengely a ciklus végétől visszafelé mutatja az aktív szakaszokat: az eszköz az első 298,6 másodpercben (298600 ms) alszik 0,15 mA-rel, majd 400 ms-ig mér 8 mA-rel, 700 ms-ig ad 50 mA-rel, végül 300 ms-ig vétel üzemmódban van 25 mA-rel. A lépcsők magassága az állapotra jellemző áram, a szélessége az állapot időtartama; a lépcsők alatti terület a ciklus alatt elfogyasztott töltés.

7.1.3 Döntési szempontok

A domináns fogyasztási tag meghatározásához nem elég az áramértékeket nézni, az időtartamokkal súlyozott töltést kell összevetni. Az alábbi táblázat a TerraSense csomópont három állapotát hasonlítja össze.

Állapot	Áram (mA)	Időtartam (s)	Töltés (mA · s)	Részesedés
Alvás	0,15	298,6	44,79	49,5%
Mérés	8	0,4	3,20	3,5%
Rádió TX	50	0,7	35,00	38,7%
Rádió RX	25	0,3	7,50	8,3%
Összesen	–	**300**	**90,49**	**100%**

A táblázatból látható, hogy a legnagyobb áramú TX állapot csak a második legnagyobb tétel, mert az alvási szakasz hosszú időtartama miatt a kicsi alvási áram is jelentős töltést fogyaszt.

Ezért a gyakorlatban gyakran az alvási áram csökkentése (például mélyebb alvó üzemmód, perifériák lekapcsolása) éppolyan fontos, mint a rádió adásidejének rövidítése.

Kompromisszumok és tervezési döntések:

- **Rövidebb periódusidő:** gyorsabb adatfrissítés, de nagyobb átlagáram, mert az aktív állapotok gyakrabban ismétlődnek. A csomópontnak választania kell a frissesség és az élettartam között.
- **Hosszabb RX ablak:** nagyobb esély a parancsok fogadására, de több energia. Az energiatakarékos eszközök gyakran rövid, ütemezett vételi ablakokat használnak.
- **Nagyobb TX teljesítmény:** nagyobb hatótáv, de több áram; a rádió adóteljesítménye és az időtartam a hatótáv, a megbízhatóság és az energia között feszül.
- **Mérés gyakorisága:** a szenzorok beolvasása is energiát fogyaszt, ezért a helyi előfeldolgozás (például csak változás esetén küldés) csökkentheti a rádióhasználatot.

7.1.4 Kidolgozott példa: TerraSense node 5 perces és 10 perces küldéssel

A következő számításban a `ts-node` csomópont egy lehetséges, illusztratív paraméterkészletét használjuk. Az értékeket a kiválasztott mikrovezérlő és rádió adatlapjából kell ellenőrizni, de a nagyságrendek tipikusak.

Feltételezett állapotok és paraméterek:

- Alvás: $I_{\text{sleep}} = 0{,}15$ mA (150 µA)
- Mérés: $I_{\text{measure}} = 8$ mA, $t_{\text{measure}} = 0{,}4$ s
- Rádió TX: $I_{\text{TX}} = 50$ mA, $t_{\text{TX}} = 0{,}7$ s
- Rádió RX: $I_{\text{RX}} = 25$ mA, $t_{\text{RX}} = 0{,}3$ s
- Elem hasznos kapacitása: $C_{\text{eff}} = 2000$ mAh (2500 mAh névleges, 20% tartalék)

7.1.4.1 5 perces periódus

A ciklusidő $T = 5$ perc $= 300$ s. Az alvási idő:

$$t_{\text{sleep}} = 300 - (0{,}4 + 0{,}7 + 0{,}3) = 298{,}6 \text{ s}.$$

A ciklus alatt elfogyasztott töltés:

$$Q_{\text{ciklus}} = 0{,}15 \cdot 298{,}6 + 8 \cdot 0{,}4 + 50 \cdot 0{,}7 + 25 \cdot 0{,}3 = 44{,}79 + 3{,}20 + 35{,}00 + 7{,}50 = 90{,}49 \text{ mA} \cdot \text{s}.$$

Az átlagáram:

$$I_{\text{avg}} = \frac{90{,}49}{300} = 0{,}3016 \text{ mA}.$$

Az élettartam hónapban:

$$L_{\text{hónap}} = \frac{2000}{0{,}3016 \cdot 24 \cdot 30} = \frac{2000}{217{,}15} \approx 9{,}2 \text{ hónap}.$$

Ez nem éri el a 12 hónapos követelményt.

7.1.4.2 10 perces periódus

A ciklusidő $T = 10$ perc $= 600$ s. Az alvási idő:

$$t_{\text{sleep}} = 600 - 1{,}4 = 598{,}6 \text{ s}.$$

A töltés:

$$Q_{\text{ciklus}} = 0{,}15 \cdot 598{,}6 + 8 \cdot 0{,}4 + 50 \cdot 0{,}7 + 25 \cdot 0{,}3 = 89{,}79 + 3{,}20 + 35{,}00 + 7{,}50 = 135{,}49 \text{ mA} \cdot \text{s}.$$

Az átlagáram:

$$I_{\text{avg}} = \frac{135{,}49}{600} = 0{,}2258 \text{ mA}.$$

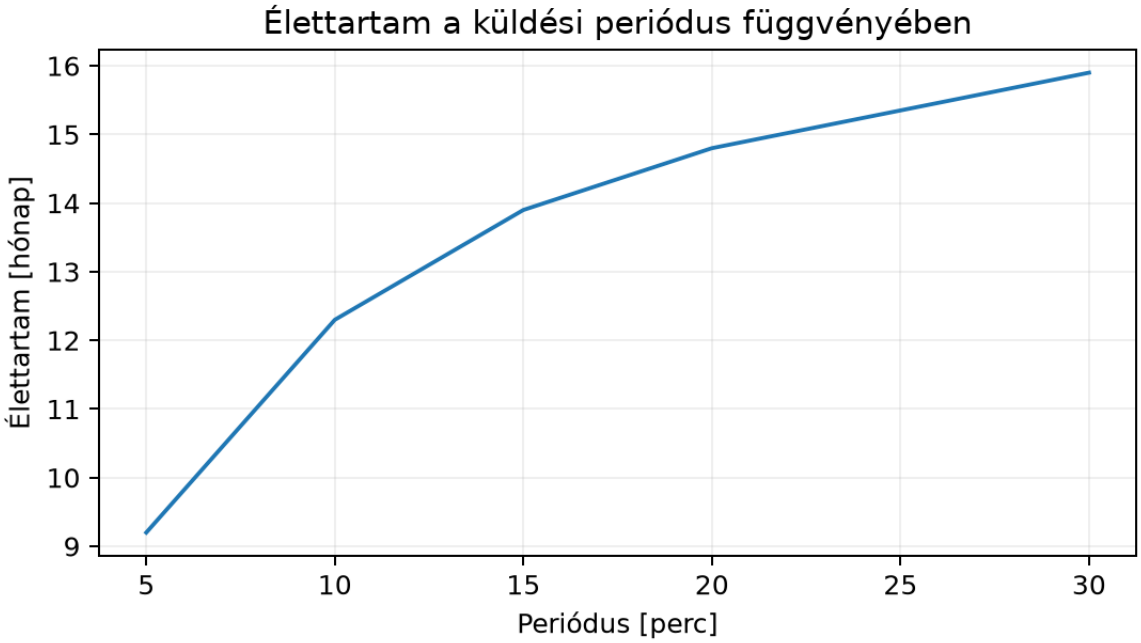
Az élettartam:

$$L_{\text{hónap}} = \frac{2000}{0{,}2258 \cdot 24 \cdot 30} = \frac{2000}{162{,}58} \approx 12{,}3 \text{ hónap}.$$

A 10 perces telemetriai ciklussal a számított élettartam 12,3 hónap, azaz a 12 hónapos követelményhez képest mindössze 0,3 hónap a tartalék. Ez az érték **becslés**, nem verifikáció: a bemenő áramok és idők adatlapi nagyságrendek, a modell pedig elhanyagolja a hőmérséklet hatását, az önkisülést, az elem terhelhetőségét és a tápfeszültség-átalakító veszteségeit. A 10 perces ciklus ezért tervezési döntés, amelyet méréssel kell igazolni; a követelmény teljesülését önmagában ez a számítás nem bizonyítja.

7.1.4.3 Érzékenységvizsgálat

Az alábbi ábra az élettartam alakulását mutatja a küldési periódus függvényében, változatlan aktív állapotidők mellett.



7.2. ábra: Élettartam a küldési periódus függvényében - Számított értékek az illusztratív paraméterekkel; a valós értékek a hardver adatlapjából ellenőrizendők.

A görbe ellaposodik: minél ritkább a küldés, annál inkább az alvási áram dominál, és a periódus további növelése egyre kevesebb élettartam-növekedést hoz. 5 percről 10 percre váltva közel 3 hónappal nő az élettartam, 20 percről 30 percre váltva viszont már csak alig több mint 1 hónappal.

Továbbvitt eredmény. A fenti paramétereket és a 10 perces periódust rögzítjük a `~/iot-labor/i016/energy-model.md` fájlban. Ezt a modellt használja az Alvómódok és energiatakarékos szoftvertervezés alfejezet az alvási állapotok finomításakor, a Rádiós technológiák összevetése alfejezet a rádió kiválasztásakor, és a LoRaWAN: nagy hatótáv kis energiával alfejezet a LoRa adás energiaigényének becslésekor. A fájl tartalma:

```
# TerraSense ts-node energiamodell (i016)
Periódus: 600 s (10 perc)
Állapotok:
  alvás: 0,15 mA, idő: 598,6 s
  mérés: 8 mA, idő: 0,4 s
  rádió TX: 50 mA, idő: 0,7 s
  rádió RX: 25 mA, idő: 0,3 s
Átlagáram: 0,2258 mA
Elem hasznos kapacitás: 2000 mAh
Számított élettartam: 12,3 hónap
Megjegyzés: az áramértékek illusztratívak, a választott hardver adatlapjából ellenőrizendők.
Megjegyzés: a 12,3 hónap becslés, nem verifikáció; a hiányzó
tényezőket lásd a becslés korlátai szakaszban.
```

7.1.4.4 A becslés korlátai és a verifikáció

A duty-cycle számítás akkor ad használható eredményt, ha a bemenő paraméterek méréssel igazoltak, és a modell minden lényeges fogyasztót tartalmaz. A 12,3 hónapos érték a következő tényezők nélkül készült:

Tényező	Hatás a becslésre	Kezelés
Hőmérséklet	Alacsony hőmérsékleten csökken az elem hasznos kapacitása és nő a belső ellenállása; a magas hőmérséklet gyorsítja az önkisülést	A legrosszabb üvegházi hőmérsékleten kell mérni és számolni
Önkisülés	A névleges kapacitás évente akár 2%-a is elveszhet; ez a 20%-os tartalék egy részét felemészti	Ekvivalens szivárgási árammal vagy nagyobb tartalékkal kell számolni
Elem terhelhetősége	Az 50 mA-es TX-csúcs a kis elem belső ellenállásán feszültségesést okozhat, ami újrainduláshoz vagy adáskimaradáshoz vezet	Ellenőrizni kell az adatlap pulzus-terhelhetőségét; szükség esetén nagyobb elem vagy pufferkondenzátor kell
Tápfeszültség-átalakító	A boost/buck átalakító hatásfoka jellemzően 80–90%, emellett saját nyugalmi árama van	A terhelési áramot a hatásfokkal osztva, a nyugalmi áramot hozzáadva kell számolni
Szelepszelep	A 24 V-os mágnesszelep 0,5 A-t vesz fel, ami nagyságrendekkel nagyobb a szenzoros csomópont fogyasztásánál	A szelepet külön, hálózati tápról kell működtetni; a 12 hónapos követelmény a szenzoros node-ra vonatkozik

Számszerűen: ha a csomópont 3,6 V-os elemről 3,3 V-ra konvertál 85%-os hatásfokkal, és az átalakító nyugalmi árama 2 µA, akkor az elemoldali átlagáram $(3{,}3/3{,}6) \cdot 0{,}2258/0{,}85 + 0{,}002 \approx 0{,}2455$ mA, amiből az élettartam $2000/(0{,}2455 \cdot 24 \cdot 30) \approx 11{,}3$ hónap. Az önkisülés 2%/év a 2500 mAh névleges kapacitásnál körülbelül 50 mAh/év, azaz hozzávetőleg 5,7 µA folyamatos ekvivalens áram; ezt hozzáadva az átlagáram 0,2512 mA, az élettartam pedig mintegy 11,0 hónap. Az átalakító vesztesége és az önkisülés tehát együtt a 12 hónapos határ alá viszi a becslést, holott a 12,3 hónapos alapérték még felette volt.

A **tervezési tartalék** (a 20%-os kapacitáscsökkentés) nem mérési eredmény, hanem tudatos biztonsági sáv. A **mérési bizonyíték** ezzel szemben csak valós eszközön, hiteles műszerrel felvett áramprofilból és kapacitástesztből származhat: oszcilloszkóp és söntellenállás az állapotidőkre és csúcsáramokra, energia-analizátor a ciklusonkénti töltésre, valamint hőkamrás vagy legalább szélső hőmérsékletű mérés a kapacitásra. A 12 hónapos követelmény teljesülését akkor lehet kijelenteni, ha a mért átlagáram a tervezési tartalékkal együtt is legfeljebb $2000/12/720 \approx 0{,}2315$ mA.

7.1.5 Ellenőrzés és hibakeresés

A számított élettartam csak annyira pontos, amennyire az állapotidők és az áramértékek helyesek. A valós hardveren végzett méréshez az alábbi eszközök használhatók:

- **Digitális multiméter:** átlagos áramfelvétel mérésére alkalmas, ha a mintavételezés elég gyors; lassú multiméter alulbecsülheti a rövid áramcsúcsokat.

- **Oszcilloszkóp és söntellenállás:** a feszültségesést mérve az áramprofil időbeli lefutása láthatóvá válik, így az egyes állapotok időtartama és árama pontosan meghatározható.
- **Energia-analizátor:** a nagy pontosságú, nagy mintavételi sebességű műszer közvetlenül a töltést (mAh) méri egy adott időablakban.

A `native_sim` környezet nem modellezi a valós hardver energiafogyasztását, ezért az energiamérést mindig valós fejlesztőkártyán vagy célhardveren kell elvégezni. A mérési összeállítás pontossága és az eszközök specifikációja ellenőrizendő.

Hibatünet	Lehetséges ok	Javítás
A mért élettartam lényegesen rövidebb	Rossz állapotidő (pl. a rádió inicializálása hosszabb)	Oszcilloszkóppal mérje meg a tényleges időket, és frissítse a modellt
A számított átlagáram alacsonyabb a mértnél	Elhanyagolt perifériák (pl. LED, szenzor fűtése)	Azonosítsa az összes fogyasztót, és vegye fel az állapotok közé
Az élettartam nem lineárisan nő a periódussal	Az alvási áram dominál, a rádióhatás kicsi	Csökkentse az alvási áramot vagy a rádió vételi ablakát
Az elem kapacitása idő előtt elfogy	Önkisülés vagy hőmérsékletfüggő kapacitáscsökkenés	Használjon nagyobb tartalékot, és mérje a hőmérsékleti profilt

7.1.6 Tipikus hibák és félreértések

1. **Csak az aktív áramot nézik, az alvási áramot elhanyagolják.** A fenti példában az alvási állapot adja a töltés felét, annak ellenére, hogy árama három nagyságrenddel kisebb a TX-áramnál. A hosszú időtartam miatt a kis alvási áram is domináns lehet.
2. **Az önkisülést figyelmen kívül hagyják.** Egy 2500 mAh névleges kapacitású elem évek alatt önkisüléssel is veszít kapacitásából. Ha a csomópontot 10 évre tervezik, az önkisülés akár nagyobb is lehet, mint a hasznos fogyasztás.
3. **A periódusidőt és az állapotidőket különböző mértékegységben adják meg.** Ha a periódus másodpercben, az állapotidőket pedig milliszekundumban számolják, az átlagáram több nagyságrenddel téves lesz. Ügyelni kell az egységes másodperc- vagy óra-alapú számításra.
4. **A rádió RX ablakát túl hosszúra veszik.** Az eszközök gyakran hosszú vételi ablakot nyitnak a parancsok fogadására, de ez az energiafogyasztás jelentős forrása. A vételi ablakot a parancs-késleltetési követelmény és az energiafogyasztás közötti kompromisszumként kell megválasztani.

7.1.7 Feladatok

7.1.7.1 1. Fogalmi kérdés

Miért nem elegendő a csúcsáram ismerete az elem élettartamának becsléséhez? Milyen további adatokra van szükség?

7.1.7.2 2. Számolási feladat

Egy szenzorcsomópont ciklusideje 2 perc. Az állapotok: alvás 1 mA 119,5 s, mérés 10 mA 0,2 s, TX 40 mA 0,2 s, RX 20 mA 0,1 s. Az elem hasznos kapacitása 1500 mAh. Számítsa ki az átlagáramot és az élettartamot hónapban!

7.1.7.3 3. Implementációs feladat

A TerraSense modellben változtassa a periódust 5 percről 15 percre, és frissítse a `~/iot-labor/i016/energy-model.md` fájlt! Számítsa ki az új átlagáramot és élettartamot, majd hasonlítsa össze a 10 perces eredménnyel!

7.1.7.4 4. Hibakeresési feladat

Egy valós mérés során az eszköz élettartama 6 hónap lett a számított 12 helyett. Felsorolja a lehetséges okokat és a javítási lépéseket! Melyik mérési módszerrel azonosítaná a hibát?

7.1.8 Megoldások

7.1.8.1 1. Fogalmi kérdés

A csúcsáram csak az egyik állapot pillanatnyi fogyasztását adja meg. Az élettartamhoz az összes állapot áramát és időtartamát ismerni kell, mert a hosszú alvási szakaszok kis árammal is jelentős töltést fogyasztanak. Képlet: $I_{\text{avg}} = \frac{\sum I_k t_k}{T}$.

7.1.8.2 2. Számolási feladat

Periódus: $T = 120$ s. Alvási idő: $t_{\text{sleep}} = 120 - (0{,}2 + 0{,}2 + 0{,}1) = 119{,}5$ s. Töltés: $1 \cdot 119{,}5 + 10 \cdot 0{,}2 + 40 \cdot 0{,}2 + 20 \cdot 0{,}1 = 119{,}5 + 2 + 8 + 2 = 131{,}5$ mA $\cdot$ s. Átlagáram: $I_{\text{avg}} = 131{,}5/120 = 1{,}096$ mA. Élettartam: $L = 1500/(1{,}096 \cdot 24 \cdot 30) = 1500/789{,}12 \approx 1{,}9$ hónap.

7.1.8.3 3. Implementációs feladat

15 perc = 900 s. Alvási idő: $900 - 1{,}4 = 898{,}6$ s. Töltés: $0{,}15 \cdot 898{,}6 + 8 \cdot 0{,}4 + 50 \cdot 0{,}7 + 25 \cdot 0{,}3 = 134{,}79 + 3{,}2 + 35 + 7{,}5 = 180{,}49$ mA $\cdot$ s. Átlagáram: $180{,}49/900 = 0{,}2005$ mA. Élettartam: $2000/(0{,}2005 \cdot 720) = 2000/144{,}36 \approx 13{,}9$ hónap. A 10 perces eredmény 12,3 hónap, tehát a 15 perces periódus 1,6 hónappal növeli az élettartamot. A fájl frissítése: a periódust 900 s-ra, az alvási időt 898,6 s-ra, az átlagáramot 0,2005 mA-re, az élettartamot 13,9 hónapra kell állítani.

7.1.8.4 4. Hibakeresési feladat

Lehetséges okok: (a) az alvási áram nagyobb a feltételezettnél (pl. nem kapcsolják le a perifériákat); (b) a rádió RX ablak hosszabb; (c) az elem kapacitása a hőmérséklet vagy a gyártási szórás miatt kisebb; (d) az önkisülés nagyobb. Az azonosításhoz oszcilloszkóppal és söntellenállással mérje meg az áramprofilt, hasonlítsa össze a modell állapotaival, és mérje az elem tényleges kapacitását. A hibás állapotokat egyesével korrigálva a számított élettartam a mért értékhez közelíthető.

7.1.9 Rövid összefoglaló

A duty-cycle energiamodell az eszköz ciklikus működését állapotokra bontja, és az átlagáramot az egyes állapotok áramának időtartammal súlyozott összegeként számítja: $I_{\text{avg}} = \frac{\sum I_k t_k}{T}$. A TerraSense csomópont illusztratív paramétereivel 5 perces periódusnál 9,2 hónap, 10 percesnél 12,3 hónap élettartam adódik. Ez azonban becslés, nem verifikáció: a 0,3 hónapos tartalék a modell bizonytalanságán belül van, és az átalakító veszteségével, valamint az önkisüléssel együtt az érték már 11 hónap körülire csökken. A 10 perces ciklus követelménye csak méréssel igazolható. A domináns tétel nem feltétlenül a legnagyobb áramú TX, hanem gyakran az alvási állapot. **Továbbvitt eredmény:** a modell paraméterei a `~/iot-labor/i016/energy-model.md` fájlban vannak; ezt használja az Alvómódok és energiatakarékos szoftvertervezés, a Rádiós technológiák összevetése és a LoRaWAN: nagy hatótáv kis energiával.

7.1.10 Források és ellenőrzési hiányok

Az alfejezetben használt áram- és időértékek illusztratív nagyságrendek, amelyeket a kiválasztott mikrovezérlő és rádió adatlapjából kell ellenőrizni. A mérőeszközök (multiméter, oszcilloszkóp, energia-analizátor) pontos specifikációja és a mérési összeállítás kalibrálása szintén ellenőrizendő. Az önkisülés mértéke az elem típusától és a hőmérséklettől függ, ezért a gyártói adatlap és a mérési környezet figyelembevétele szükséges. A Zephyr `native_sim` környezet energiafogyasztása nem reprezentatív, ezért az energiamérést valós hardveren kell elvégezni.

7.2 Alvómódok és energiatakarékos szoftvertervezés

7.2.1 Motiváció

A TerraSense üvegházban az elemes `ts-node` csomópontnak akár két évig is működnie kell egyetlen lítium-tionil-klorid (Li-SOCl) elemről. Az Energiamodell és az elemélettartam kiszámítása alfejezetben felírt átlagáram-egyenlet jól mutatja, hogy az élettartamot az aktív és alvó üzemmódok aránya határozza meg. Ha a mérőszál 100 ms-onként felébred, és a rádió minden egyes mintát külön küld, az átlagáram több mA is lehet; ha viszont a CPU az idő 99,9%-ában mély alvásban van, a rádió pedig csak ritkán, kötegelve ad, az átlagáram µA nagyságrendűre csökken. Ebben az alfejezetben a Zephyr energiagazdálkodási eszközeit és az energiatakarékos szoftvermintákat alkalmazzuk a `ts-node` mérési ciklusára.

7.2.2 Fogalmak és működés

Definíció. Alvómód (sleep mode): a mikrovezérlő olyan üzemállapota, amelyben a CPU mag órajele leáll vagy lelassul, a perifériák egy része kikapcsol, és a fogyasztás a normál aktív áram töredékére csökken. Mélyebb alvómód nagyobb energiát takarít meg, de hosszabb ébredési időt és kevesebb ébresztési forrást támogat.

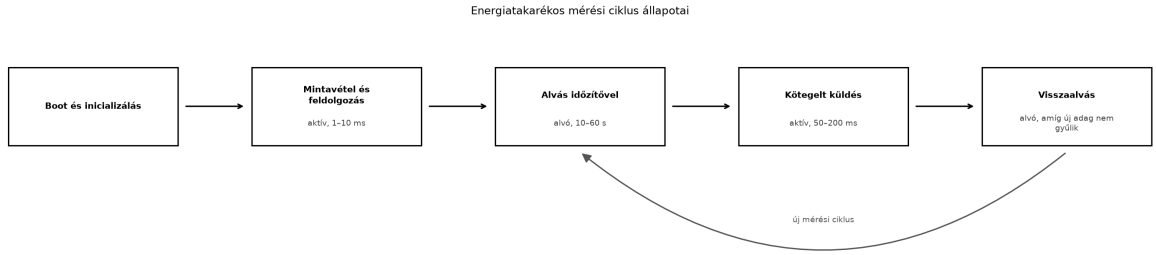
Az MCU-k jellemzően több alvómódot kínálnak, amelyek a kikapcsolt órajelű részekben és az ébredési latenciában térnek el. A Zephyr három, egymástól különböző PM-réteget használ: a rendszer-PM (`CONFIG_PM`) a teljes SoC alvási állapotát választja ki az idle szálban, a device PM (`CONFIG_PM_DEVICE`) az egyes perifériák suspend/resume műveleteit kezeli, a runtime PM (`CONFIG_PM_DEVICE_RUNTIME`) pedig a használaton kívüli eszközöket függeszti fel. A `k_sleep()` csak blokkolja a szálat; hogy közben a rendszer milyen mély állapotba kerül, az a board támogatásától, az engedélyezett ébresztőforrásoktól, a késleltetési korlátoktól és a PM policy-től

függ, nem pedig automatikusan a legmélyebb állapot. A `native_sim` szimulátor nem modellezi a valós fogyasztást, ezért a laborban szimulált energiaszámlálót használunk.

Egy konkrét, ellenőrizendő példa: a `nrf52840dk/nrf52840` boardon a `CONFIG_PM=y`, `CONFIG_PM_DEVICE=y` és `CONFIG_PM_DEVICE_RUNTIME=y` beállításokkal a kernel idle állapotban rendszer-PM suspendet hajthat végre, egy szenzor pedig `pm_device_runtime_put()` után függeszthető fel, ha a driver támogatja. Az ébresztés RTC/időzítő vagy GPIO lehet. A tényleges állapotot, az ébresztési késleltetést és a fogyasztást az adott boardon kell mérni; a `native_sim` ezt nem modellezi.

Az energiatakarékos szoftvertervezés legfontosabb szabálya: a CPU-nak eseményvezérelten kell működnie, és minden várakozást blokkoló hívással (`k_sleep`, `k_msgq_get`, `k_sem_take`) kell megvalósítani. A busy wait (`k_busy_wait`, `for` ciklusos várakozás) aktívan tartja a magot, és a teljes aktív áramot fogyasztja.

Az alábbi folyamatábra a mérési ciklus energiaállapotait mutatja:



7.3. ábra: Energiatakarékos mérési ciklus állapotai - A ts-node a mérés és a küldés között alvó állapotba kerül, és csak időzítő vagy külső esemény ébreszti.

Az ébresztési források a következők lehetnek:

- **Időzítő (timer):** a leggyakoribb, periodikus mintavételhez.
- **Külső megszakítás (GPIO):** például ajtónyitás-érzékelő vagy páratartalom-küszöb átlépése.
- **RTC (real-time clock):** hosszú, másodperc vagy perc pontosságú alváshoz.
- **Watchdog:** biztonsági ébresztés, ha a szoftver elakad.
- **Kommunikációs periféria:** UART, SPI vagy rádió bejövő adat esetén.

A Zephyr **tickless kernel** üzemmódjában a kernel nem ébred fel periodikus rendszer-tickre, csak akkor, amikor tényleges esemény (időzítő lejárata, megszakítás) történik. Ez csökkenti a felesleges ébredések számát, és hosszabb alvási periódusokat tesz lehetővé. A tickless üzemmód nem önmagában pontosságvesztési funkció: a `k_uptime_get()` és az időzítők felbontása a hardver timerétől és a kernel konfigurációjától függ, a gyakorlati kompromisszum pedig inkább a hosszabb ébredési latency és a timer-újraprogramozás költsége. A `CONFIG_TICKLESS_KERNEL=y` beállítás a legtöbb Zephyr-támogatású MCU-n elérhető, de a pontos viselkedés architektúrafüggő, ezért ellenőrizni kell a használt verzió dokumentációjában.

A rádióidő minimalizálása legalább olyan fontos, mint a CPU alvása. Egy BLE- vagy LoRa-adás jellemzően 10–100 mA-t fogyaszt, ami nagyságrendekkel több, mint a mély alvó mód. Ezért a `ts-node` nem minden minta után küld, hanem **kötegelt küldést** alkalmaz: 5 mérést összegyűjt, és egyetlen adásban továbbítja. Ez a stratégia csökkenti a rádió bekapcsolási idejét, a protokoll overheadet és a nyugták számát is. Hasonló elvet használ majd az Adatformátumok: JSON, CBOR, Protocol Buffers, SenML alfejezet, amikor a tömbös telemetria-sémát véglegesíti, valamint

a Store-and-forward: offline működés és visszatöltés alfejezet, ahol a gateway is kötegelve tölti vissza az offline adatokat.

A perifériák kikapcsolása szintén kritikus. A Zephyr eszközmodelljében minden bekapcsolt periféria (szenzor, UART, SPI, GPIO) fogyaszt. A `sensor_sample_fetch()` után a szenzor device PM-en keresztül függeszthető fel: ha a driver és a board támogatja, a runtime PM `pm_device_runtime_put()` hívással kapcsolja ki, illetve `pm_device_action_run()` adhat ki suspend kérést. A `sensor_attr_set()` nem általános szenzoralvó API, hanem driver-specifikus attribútumot állít; csak akkor használható erre, ha az adott driver dokumentálja. A nem használt GPIO-kat az MCU adatlapja szerint kell beállítani: analóg módba a bemeneti puffer tiltásával, vagy definiált fel-/lehúzású digitális bemenetbe, vagy rögzített kimeneti szintre. A lebegő digitális bemenet zajérzékeny, és a bemeneti puffer átbillenése miatt növelheti a fogyasztást, ezért önmagában nem jó energiatakarékos megoldás; a külső áramkört és a szivárgási utakat is figyelembe kell venni.

7.2.3 Döntési szempontok

Az alvómód kiválasztásánál a következő kompromisszumokat kell mérlegelni:

Szempont	Idle / tickless	Sleep	Deep sleep / Standby
Ébredési idő	azonnali	µs nagyságrend	ms nagyságrend
Fogyasztás	100 µA–1 mA	10–100 µA	0,1–10 µA
Megőrzött állapot	minden	CPU regiszterek, RAM	korlátozott vagy nincs
Ébresztési források	minden	időzítő, GPIO, RTC	RTC, külső megszakítás
Zephyr-ben	`k_sleep` + tickless	`CONFIG_PM` + `CONFIG_PM_DEVICE`	`CONFIG_PM` + board-specifikus mély állapot

A `ts-node` esetében a mérési ciklus 10 másodperces periódusával a **deep sleep** a megfelelő választás, mert az ébredési latencia (néhány ms) elhanyagolható a periódushoz képest, a fogyasztáscsökkenés viszont jelentős. Ha a csomópontnak valós időben kell reagálnia egy szelepvezérlő parancsra, akkor a rádió ébresztését is engedélyezni kell, ami a BLE esetén a kapcsolat fenntartását vagy hirdetési intervallumot jelent – ez növeli az alapfogyasztást, ezért ilyenkor az alvó mód sekélyebb lesz.

7.2.4 Kidolgozott labor: a `ts-node` ciklusának átalakítása szimulált energiaszámlálóval

Munkakönyvtár:

```
mkdir -p ~/iot-labor/i017 && cd ~/iot-labor/i017
```

A labor a Szálak és ütemezés alfejezetben létrehozott `ts-node` projektet alakítja át. Másoljuk át a korábbi forrást:

```
cp -r ../i010/ts-node ./ts-node
cd ts-node
```

A `src/main.c` fájlt cseréljük le az alábbi tartalomra. A kód két szálat használ: a `measure_thread_fn` 10 másodpercenként mintát vesz, a `send_thread_fn` 5 minta összegyűjtése után egy kötegelt küldést szimulál. A 60 másodperces ablakban így 6 minta készül, amelyből éppen egy teljes 5 mintás adag küldődik el. A szimulált energiaszámláló a CPU-aktív, a rádió-aktív és az alvó időt külön komponensként gyűjti, majd a program 60 másodperc után kiírja a becsült átlagáramot.

```
#include <zephyr/kernel.h>
#include <zephyr/random/random.h>
#include <zephyr/sys/atomic.h>
#include <stdio.h>
#include <string.h>

#define MEASURE_INTERVAL_MS   10000U
#define BATCH_SIZE            5U
#define CPU_ACTIVE_CURRENT_MA 10.0f
#define RADIO_CURRENT_MA      120.0f
#define SLEEP_CURRENT_UA      10.0f
#define MEASURE_ACTIVE_MS     2U
#define SEND_ACTIVE_MS        120U
#define MEASURE_STACK_SIZE    1024
#define SEND_STACK_SIZE       1024

static atomic_t cpu_active_ms;
static atomic_t radio_active_ms;
static atomic_t g_batch_dropped;

struct telemetry_data {
    uint32_t seq;
    float temperature_c;
    float humidity_rh;
};

static struct telemetry_data g_batch[BATCH_SIZE];
static uint8_t g_batch_count;
static struct k_mutex g_batch_mutex;
static struct k_sem g_batch_sem;

K_THREAD_STACK_DEFINE(measure_stack, MEASURE_STACK_SIZE);
K_THREAD_STACK_DEFINE(send_stack, SEND_STACK_SIZE);
static struct k_thread measure_thread_data;
static struct k_thread send_thread_data;

static void measure_thread_fn(void *p1, void *p2, void *p3);
static void send_thread_fn(void *p1, void *p2, void *p3);

static void add_cpu_active(uint32_t duration_ms)
{
    atomic_add(&cpu_active_ms, duration_ms);
}

static void add_radio_active(uint32_t duration_ms)
{
    atomic_add(&radio_active_ms, duration_ms);
}

static void simulate_measurement(struct telemetry_data *out, uint32_t seq)
{
    /* Szimulált szenzorolvasás: 2 ms CPU-aktív idő */
    k_busy_wait(MEASURE_ACTIVE_MS * 1000U);

    out->seq = seq;
```

```
    out->temperature_c = 20.0f + (sys_rand32_get() % 1000) / 100.0f;
    out->humidity_rh = 50.0f + (sys_rand32_get() % 2000) / 100.0f;

    add_cpu_active(MEASURE_ACTIVE_MS);
}

static void simulate_send_batch(const struct telemetry_data *batch,
                                uint8_t count)
{
    /* Szimulált rádióadás: 120 ms CPU- és rádió-aktív idő */
    k_busy_wait(SEND_ACTIVE_MS * 1000U);
    add_cpu_active(SEND_ACTIVE_MS);
    add_radio_active(SEND_ACTIVE_MS);

    printf("[send] %u minta elküldve, seq=%u..%u\n",
           count, batch[0].seq, batch[count - 1].seq);
}

static void measure_thread_fn(void *p1, void *p2, void *p3)
{
    uint32_t seq = 0;

    ARG_UNUSED(p1);
    ARG_UNUSED(p2);
    ARG_UNUSED(p3);

    while (1) {
        struct telemetry_data sample;

        simulate_measurement(&sample, ++seq);

        k_mutex_lock(&g_batch_mutex, K_FOREVER);
        if (g_batch_count < BATCH_SIZE) {
            g_batch[g_batch_count++] = sample;
            if (g_batch_count == BATCH_SIZE) {
                k_sem_give(&g_batch_sem);
            }
        } else {
            /* Definált túlcsordulási politika: a minta eldobása. */
            atomic_inc(&g_batch_dropped);
        }
        k_mutex_unlock(&g_batch_mutex);

        k_sleep(K_MSEC(MEASURE_INTERVAL_MS));
    }
}

static void send_thread_fn(void *p1, void *p2, void *p3)
{
    struct telemetry_data local[BATCH_SIZE];

    ARG_UNUSED(p1);
    ARG_UNUSED(p2);
    ARG_UNUSED(p3);

    while (1) {
        uint8_t count;

        k_sem_take(&g_batch_sem, K_FOREVER);

        /* A közös puffert csak a másolás idejére zárjuk. */
        k_mutex_lock(&g_batch_mutex, K_FOREVER);
        count = g_batch_count;
```

```
        memcpy(local, g_batch, count * sizeof(local[0]));
        g_batch_count = 0;
        k_mutex_unlock(&g_batch_mutex);

        if (count > 0) {
            simulate_send_batch(local, count);
        }
    }
}

int main(void)
{
    uint32_t start_ms;

    k_mutex_init(&g_batch_mutex);
    k_sem_init(&g_batch_sem, 0, 1);

    start_ms = k_uptime_get_32();

    k_thread_create(&measure_thread_data, measure_stack,
                    K_THREAD_STACK_SIZEOF(measure_stack),
                    measure_thread_fn, NULL, NULL, NULL,
                    K_PRIO_PREEMPT(5), 0, K_NO_WAIT);
    k_thread_create(&send_thread_data, send_stack,
                    K_THREAD_STACK_SIZEOF(send_stack),
                    send_thread_fn, NULL, NULL, NULL,
                    K_PRIO_PREEMPT(4), 0, K_NO_WAIT);

    k_sleep(K_SECONDS(60));

    uint32_t elapsed_ms = k_uptime_get_32() - start_ms;
    uint32_t cpu_ms = (uint32_t)atomic_get(&cpu_active_ms);
    uint32_t radio_ms = (uint32_t)atomic_get(&radio_active_ms);
    uint32_t sleep_ms = elapsed_ms > cpu_ms ? elapsed_ms - cpu_ms : 0;

    if (elapsed_ms == 0) {
        elapsed_ms = 1;
    }

    float avg_current_ma =
        (cpu_ms * CPU_ACTIVE_CURRENT_MA +
         radio_ms * RADIO_CURRENT_MA +
         sleep_ms * (SLEEP_CURRENT_UA / 1000.0f)) / elapsed_ms;

    printf("\n--- Szimulált energiaszámláló ---\n");
    printf("Aktív CPU idő:   %u ms\n", cpu_ms);
    printf("Rádió aktív idő: %u ms\n", radio_ms);
    printf("Alvó idő:        %u ms\n", sleep_ms);
    printf("Duty cycle (CPU): %.2f %%\n", 100.0f * cpu_ms / elapsed_ms);
    printf("Átlagáram:       %.3f mA\n", avg_current_ma);
    printf("Becsült élettartam 2400 mAh-s elemmel: %.1f nap\n",
           2400.0f / avg_current_ma / 24.0f);
    printf("Eldobott minták: %u\n", (uint32_t)atomic_get(&g_batch_dropped));
    return 0;
}
```

A `measure_thread_fn` az írás előtt ellenőrzi a `g_batch_count < BATCH_SIZE` feltételt; ha a fogyasztó lassabb, a minta eldobódik, és a `g_batch_dropped` atomi számláló nő, a tömb nem csordul túl. A `send_thread_fn` a mutex alatt csak lemásolja az adagot, majd elengedi a mutexet, és a 120 ms-os adást már a közös puffer zárása nélkül végzi, így a mérőszál nem blokkolódik a teljes adás idejére.

Az energiaszámláló nem szálankénti alvásidőket ad össze, mert a szálak alvása és aktivitása

átfedhet: a `send_thread_fn` akkor is aktív lehet, amikor a `measure_thread_fn` éppen alszik. A modell ezért rendszerszintű, egymást kizáró időfelosztást használ: a `cpu_active_ms` a tényleges CPU-aktív szakaszok (mérés és adás) összege, a `radio_active_ms` a rádió adási ideje, az alvó idő pedig az eltelt teljes időből a CPU-aktív idő levonásával adódik. A rádió adása alatt a CPU is aktív, ezért a két áram összeadódik (10 mA + 120 mA), de az idő nem számolódik kétszer. A `k_uptime_get_32()`-vel mért `elapsed_ms` biztosítja, hogy az ablak hossza a tényleges eltelt idő legyen, a záró, még be nem könyvelt alvásrészt is beleértve. Az atomi számlálók (`atomic_t`) elkerülik a versenyhelyzetet a két szál és a `main` között.

A `prj.conf` tartalmát egészítsük ki a tickless kernel engedélyezésével:

```
CONFIG_TICKLESS_KERNEL=y
CONFIG_PM=y
CONFIG_ENTROPY_GENERATOR=y
```

A Zephyr `native_sim` boardon a `CONFIG_PM` és a `CONFIG_TICKLESS_KERNEL` pontos hatása verziófüggő; a fenti beállítások ellenőrzési igénnyel szerepelnek, de a szimulált energiaszámláló ettől függetlenül működik.

Fordítás és futtatás:

```
west build -b native_sim
./build/zephyr/zephyr.exe
```

Várt kimenet a tervezett ütemezés szerint (a szimulátor időzítése néhány ms eltérést okozhat):

```
[send] 5 minta elküldve, seq=1..5
--- Szimulált energiaszámláló ---
Aktív CPU idő:   132 ms
Rádió aktív idő: 120 ms
Alvó idő:        59868 ms
Duty cycle (CPU): 0.22 %
Átlagáram:       0.272 mA
Becsült élettartam 2400 mAh-s elemmel: 367.7 nap
Eldobott minták: 0
```

A 60 másodperc alatt a mérőszál 6 mintát vesz (t = 0, 10, …, 50 s), ezért éppen egy teljes 5 mintás adag küldődik el; a 6. minta a következő ablakban kerül sorra. Az átlagáram 0,272 mA, amit a 120 mA-es rádiócsúcs 120 ms-os részaránya és a 10 mA-es CPU-aktív idő együttesen ad. Ez nem „tized mA alatti” érték: a korábbi 0,136 mA-es szám a rádióáram elhagyásából és a hibás időszámításból származott. Ha a szenzort minden másodpercben olvasnánk, a CPU-aktív idő és ezzel a duty cycle nőne, miközben a rádióadás kötegelt maradhat.

Takarítás:

```
cd ~/iot-labor
rm -rf i017
```

7.2.5 Ellenőrzés és hibakeresés

A labor működésének ellenőrzése:

- A `west build` hiba nélkül lefut, és a `zephyr.exe` elindul.
- A `send` sor a 40. másodperc körül jelenik meg (5 minta × 10 s), a 60 másodperces ablakban egyszer.
- Az energiaszámláló kiírja a CPU- és rádió-aktív időt, az alvó időt, a duty cycle-t (kb. 0,2%) és az átlagáramot (kb. 0,27 mA).

- A `k_sem_give` és `k_sem_take` párosítás helyes; a minta csak akkor íródik a tömbbe, ha van hely, különben az `Eldobott minták` számláló nő.
- Ha a `send_thread_fn`-t szándékosan késleltetjük, a `g_batch` tömb nem csordul túl: a `g_batch_count < BATCH_SIZE` ellenőrzés miatt a felesleges minták eldobódnak.

Hibatünet	Ok	Javítás
A program azonnal kilép, nincs kimenet	A `main` nem vár eleget	Növeljük a `k_sleep(K_SECONDS(60))` értékét
A `send` sorok sűrűbben jelennek meg	A `MEASURE_INTERVAL_MS` túl kicsi	Állítsuk 10000-re
Az átlagáram nagyobb a vártnál	A `SLEEP_CURRENT_UA` beépítése hibás	Ellenőrizzük az átváltást: `SLEEP_CURRENT_UA / 1000.0f`
A szálak nem indulnak el	A veremméret vagy a prioritás hibás	Növeljük a veremméretet 2048 bájtra, és ellenőrizzük a `k_thread_create` paramétereit

7.2.6 Tipikus hibák és félreértések

1. **Busy wait a várakozásra.** A `k_busy_wait()` a processzort aktívan dolgoztatja, ezért a teljes aktív áramot fogyasztja. Csak nagyon rövid, hardver-időzítésű késleltetésekhez használjuk; minden más esetben `k_sleep()`.
2. **Minden mintánál rádióadás.** Ha a `ts-node` minden mérés után azonnal PUBLISH-t küld, a rádió fogyasztása dominál, és az elem hetek alatt lemerül. A kötegelt küldés 5–10 mintát gyűjt, és egy adásban továbbítja, így a rádióidő töredékére csökken.
3. **A perifériák bekapcsolva hagyása.** A szenzor, az UART és a GPIO-k alvás közben is fogyasztanak, ha nem kapcsoljuk ki őket. A Zephyr device PM/runtime PM rétege csak akkor függesztheti fel az eszközt, ha az alkalmazás elengedi (`pm_device_runtime_put()`), a driver támogatja a suspend műveletet, és a board PM-konfigurációja engedélyezi azt. A `sensor_attr_set()` nem általános alvó API, ezért driverenként ellenőrizni kell, milyen attribútumok érhetők el.
4. **Túl gyakori tick.** Ha a tickless kernel nincs engedélyezve, a kernel minden ticknél felébred (jellemzően 10 ms). Ez évente több millió felesleges ébredést jelent, ami jelentősen növeli az átlagáramot.

7.2.7 Feladatok

1. **Fogalmi kérdés.** Magyarázd el, miért növeli a tickless kernel az alvási időt a hagyományos tick-alapú ütemezéshez képest, és milyen kompromisszummal jár.
2. **Számolási feladat.** A `ts-node` mérési ciklusa 10 s alvásból és 2 ms aktív időből áll. A rádió 120 ms-ig aktív, de csak minden 8. minta után kapcsol be. Számítsd ki az átlagáramot, ha a CPU aktív árama 8 mA, az alvó áram 5 µA, és a rádió adása alatt a rádióáram a CPU-áramhoz adódik (120 mA). Mennyi a becsült élettartam 2400 mAh-s elemmel? Ha az adatlap a TX alatti teljes rendszeráramot adja meg, hogyan változik a számítás?
3. **Implementációs feladat.** Módosítsd a labor `main.c`-jét úgy, hogy a `send_thread_fn` ne csak akkor küldjön, ha 5 minta összegyűlt, hanem legkésőbb 30 másodperc után is, még

akkor is, ha csak 3 minta van. Ezzel elkerüljük, hogy ritka mintavétel esetén az adat sokáig az eszközön maradjon.

4. **Hibakeresési feladat.** A laborban a `k_sem_take(&g_batch_sem, K_FOREVER)` blokkoló hívás a `send_thread_fn`-ben. Mi történik, ha a `measure_thread_fn` leáll (például egy szenzorhiba miatt)? Hogyan lehetne ezt watchdoggal vagy időkorláttal kezelni?

7.2.8 Megoldások

1. A tick-alapú kernel periodikusan (pl. 10 ms-onként) megszakítja a processzort, még akkor is, ha nincs tennivaló. Ez meggátolja a hosszú, egybefüggő alvást. A tickless kernel csak a következő időzítő lejártakor ébreszti fel a CPU-t, így a 10 másodperces mérési periódus alatt kevesebb ébredés történik. A kompromisszum nem önmagában pontosságvesztés: az időzítők felbontása a hardver timerétől és a kernel konfigurációjától függ, a gyakorlati költség pedig a hosszabb ébredési latency és a timer újraprogramozása lehet. A `k_uptime_get()` felbontását az adott boardon kell ellenőrizni.

2. Egy 8 mintás ciklusban a mérési periódusok 8×10 s $= 80\,000$ ms alvást adnak. A CPU a 8 mérés során 8×2 ms $= 16$ ms-ig aktív, ehhez jön a 120 ms-os rádióadás, amely alatt a CPU szintén aktív: összesen 136 ms. A rádióáram a CPU-áramhoz adódik, ezért adás közben a rendszeráram $8 + 120 = 128$ mA; ha az adatlap a TX alatti teljes rendszeráramot adja meg, a CPU-áramot nem kell még egyszer hozzáadni.

$$Q_{\text{ciklus}} = 136 \cdot 8 + 120 \cdot 120 + 80\,000 \cdot 0{,}005 = 1088 + 14\,400 + 400 = 15\,888 \text{ mA} \cdot \text{ms}.$$

Az összidő $80\,000 + 136 = 80\,136$ ms, ezért:

$$I_{\text{avg}} = \frac{15\,888}{80\,136} \approx 0{,}198 \text{ mA}.$$

Az élettartam: $2400/0{,}198 \approx 12\,121$ óra, azaz körülbelül 505 nap. Ha a 120 mA már a teljes TX alatti rendszeráram, akkor a CPU 120 ms-os járuléka nem adódik hozzá, és az átlagáram ennél valamivel alacsonyabb.

3. A `send_thread_fn` módosítása: használjunk `k_sem_take` helyett `k_poll`-t vagy egy időzítőt, amely 30 másodperc után is jelez. Egyszerűbb megoldás: a `send_thread_fn` ciklusában `k_sem_take(&g_batch_sem, K_SECONDS(30))`, majd ha a visszatérési érték `-EAGAIN`, akkor a jelenlegi `g_batch_count` értékkel is küldünk, ha az nagyobb nullánál.

4. Ha a `measure_thread_fn` leáll, a `g_batch_sem` soha nem kap jelet, így a `send_thread_fn` örökre blokkol. A megoldás: a `k_sem_take` időkorláttal történő hívása (`K_MSEC(30000)`), és a visszatérési érték vizsgálata. Ha időtúllépés történt, akkor is küldjük el a rendelkezésre álló mintákat, és esetleg növeljünk egy hibaszámlálót. A watchdog pedig újraindíthatja a teljes rendszert, ha a mérőszál hosszabb ideig nem ad életjelet.

7.2.9 Rövid összefoglaló

Az elemes IoT-csomópont élettartamát az alvó és aktív idő aránya határozza meg. A CPU-t eseményvezérelten, blokkoló hívásokkal kell várakoztatni, és a board által támogatott, a késleltetési igényeknek megfelelő alvómódba kell küldeni; a `CONFIG_PM` önmagában nem garantálja a legmélyebb állapotot. A Zephyr rendszer-, device- és runtime PM rétegei, valamint a tickless kernel beállításai csökkentik a felesleges ébredéseket, de a pontos hatás board- és verziófüggő. A rádióidő minimalizálása érdekében a `ts-node` **kötegelt küldést** alkalmaz: 5 mérést gyűjt, és egyetlen adásban továbbítja. Ezt a stratégiát a labor `~/iot-labor/i017/ts-node/src/main.c` fájlja valósítja meg a `g_batch` tömbbel, a `g_batch_count` számlálóval és a `g_batch_sem` szemaforral. A szimulált energiaszámláló a CPU-aktív, rádió-aktív és alvó időt komponensenként, egyetlen idővonalon számolja, és kiírja a duty cycle-t és a becsült élettartamot. **Továbbvitt eredmény.** A kötegelt küldés és az 5 mintás adagolás konvencióját használja az Adatformátumok: JSON, CBOR, Protocol Buffers, SenML alfejezet a tömbös telemetria-séma kialakításához, valamint a Store-and-forward: offline működés és visszatöltés alfejezet, amikor a gateway is kötegelve tölti vissza az offline adatokat.

7.2.10 Források és ellenőrzési hiányok

- Zephyr dokumentáció, power management és tickless kernel. URL: https://docs.zephyrproject.org/latest/ (nem ellenőriztem közvetlenül).
- A `CONFIG_TICKLESS_KERNEL`, `CONFIG_PM` pontos neve és hatása a használt Zephyr verzióban ellenőrizendő. A `native_sim` board PM-támogatása korlátozott lehet.
- A `k_uptime_get_32()` és a `k_sleep` viselkedése tickless módban a Zephyr verziójától függően változhat.
- A szimulált áramértékek (aktív 10 mA, alvó 10 µA, rádió 120 mA) illusztratív, tipikus nagyságrendek, nem konkrét adatlapról származnak.

8 Robusztus eszközszoftver

8.1 Eszközszoftver-architektúra: rétegek és állapotgépek

8.1.1 Motiváció

A TerraSense `ts-node` eszközszoftvere a korábbi fejezetekben egyetlen, lineárisan futó C program volt: inicializálás, szenzorolvasás, üzenetküldés, néhány másodperc várakozás. Ez a szerkezet akkor működik, amíg minden rendben megy, de éles környezetben az eszköz élete nem egyenes vonalú. Mi történik, ha a hálózati kapcsolat megszakad? Ha a felhőből parancs érkezik? Ha az eszköznek gyártás utáni beüzemelésre (provisioning) van szüksége? Ha egy hibát automatikusan helyre kell állítani? Az ilyen helyzetekre a lineáris kód nem ad választ: az if-else láncok gyorsan átláthatatlanná válnak, a hibakezelés beszivárog minden sorba, és a karbantartás lehetetlenné válik.

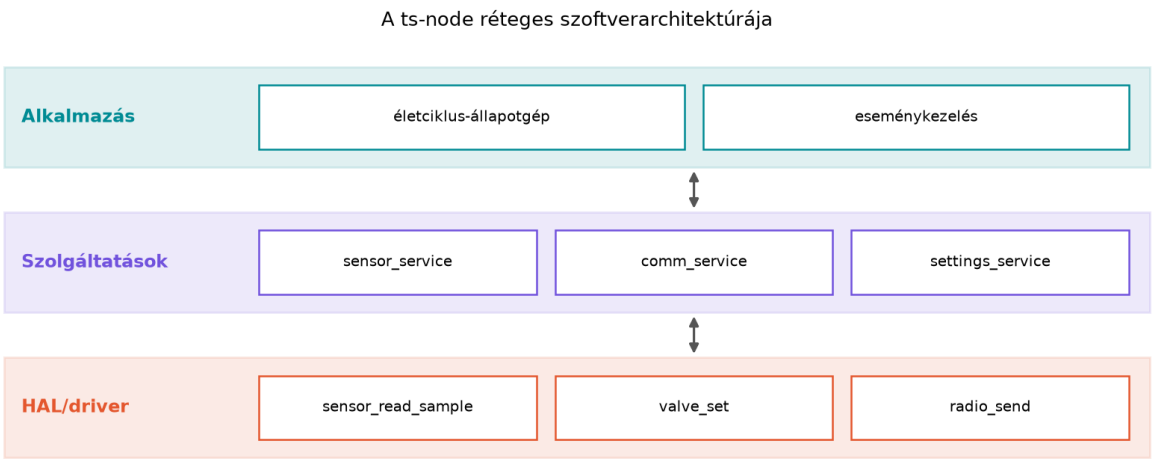
Ebben az alfejezetben a `ts-node` szoftverét rétegekre bontjuk, és az üzleti logikát egy explicit életciklus-állapotgéppel (state machine) vezéreljük. Ez lesz az alapja annak, hogy a későbbi OTA-frissítés, provisioning és hibatűrés jól strukturáltan illeszthető legyen a rendszerbe.

8.1.2 Fogalmak és működés

8.1.2.1 Réteges szoftverarchitektúra

A beágyazott eszközszoftverekben bevált gyakorlat a felelősségek függőleges rétegekre osztása. A rétegek közötti kapcsolat kizárólag interfészeken keresztül történik, így a hardverfüggő kód nem szivárog be az alkalmazási logikába, és a szolgáltatások tesztelhetők hardver nélkül is.

Definíció. A `ts-node` szoftverének rétegei a következők: - **HAL (Hardverabsztrakciós réteg, Hardware Abstraction Layer)** – a Zephyr periféria API-jaira épülő illesztőréteg, amely a konkrét hardverelemeket (szenzorok, szelep, rádió) egységes függvényhívásokkal teszi elérhetővé. Ez a réteg tartalmazza a `sensor_read_sample()` függvényt és a devicetree-ből származó eszközkötéseket. - **Szolgáltatások (services)** – az egyes feladatok önálló moduljai, például a mérési szolgáltatás (sensor service), a kommunikációs szolgáltatás (communication service) és a konfigurációs szolgáltatás (settings service). A szolgáltatások használják a HAL-t, de nem ismerik egymás belső működését. - **Alkalmazás (application)** – az üzleti logika, amely a szolgáltatásokat eseményvezérelt módon hívja, és az eszköz életciklus-állapotgépét futtatja.



8.1. ábra: A ts-node réteges szoftverarchitektúrája - A nyilak lefelé mutató függőséget jelentenek; az interfészek a réteghatárokon definiáltak.

A rétegek közötti kommunikáció úgy valósul meg, hogy az alsóbb rétegek függvényeit csak a közvetlenül felettük lévő réteg hívja, az események és adatok pedig üzenetsorokon vagy callback-függvényeken keresztül jutnak felfelé. Például a mérőszál (a korábbi Szenzorolvasás a Zephyr sensor API-val alfejezetből) továbbra is a `sensor_read_sample()` HAL-függvényt hívja, majd a mintát a `sample_msgq` üzenetsorra teszi; ezt az üzenetsort most a sensor service fogyasztja, nem pedig közvetlenül az alkalmazás.

8.1.2.2 Életciklus-állapotgép

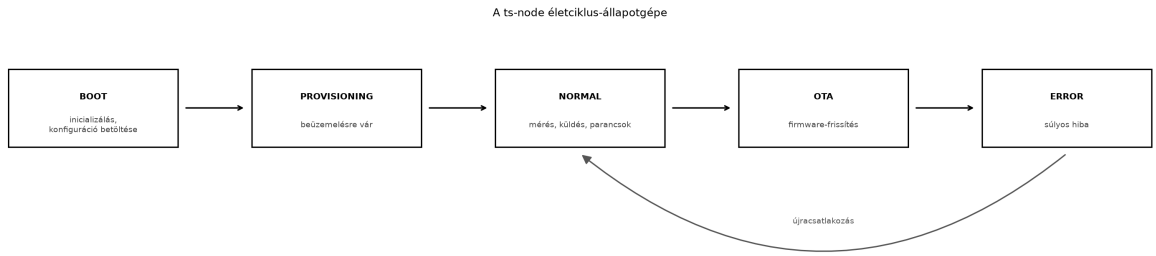
Az életciklus-állapotgép az eszköz lehetséges működési állapotait és azok közötti átmeneteket definiálja. A cél, hogy a rendszer minden pillanatban pontosan egy állapotban legyen, és csak előre meghatározott események hatására váltson állapotot. Ez megkönnyíti a hibakeresést, a tesztelést és a bővítést.

Definíció. Az életciklus-állapotgép egy olyan modell, amely: - **állapotokból** (state) áll, amelyek az eszköz aktuális működési módját írják le, - **eseményekből** (event) áll, amelyek belső vagy külső ingerek (pl. mérés elkészült, parancs érkezett, watchdog lejárt), - **átmeneti függvényből** (transition function) áll, amely az aktuális állapot és a bejövő esemény alapján meghatározza a következő állapotot és a végrehajtandó műveleteket.

A `ts-node` esetében az alábbi életciklus-állapotokat vezetjük be:

1. **BOOT** – bekapcsolás utáni inicializálás (hardver, rétegek, konfiguráció betöltése).
2. **PROVISIONING** – az eszköz még nem rendelkezik érvényes azonosítóval vagy hálózati konfigurációval; beüzemelésre vár.
3. **NORMAL** – normál működés: mérés, küldés, parancsfeldolgozás.
4. **ERROR** – súlyos hiba, amelyből csak újraindulással vagy manuális beavatkozással lehet kilépni.
5. **OTA** – firmware-frissítés végrehajtása (a későbbi OTA-frissítés: A/B képek, rollback, delta, fokozatos kiadás alfejezetben részletezzük).

Az állapotok közötti átmeneteket az alábbi flow ábra szemlélteti:



8.2. ábra: A ts-node életciklus-állapotgépe - A nyilak az események nevével vannak jelölve; az ERROR állapotból újraindítással lehet BOOT-ba kerülni.

Az átmeneteket a következő táblázat foglalja össze (a sorok az aktuális állapotok, az oszlopok az események):

Állapot/esemény	EV_BOOT_DONE	EV_PROVISIONED	EV_NET_DOWN	EV_NET_UP	EV_ENTER_OTA	EV_OTA_DONE	EV_ERROR
BOOT	→ PROVISIONING	–	–	–	–	–	→ ERROR
PROVISIONING	–	→ NORMAL	–	–	–	–	→ ERROR
NORMAL	–	–	→ ERROR (vagy retry)	–	→ OTA	–	→ ERROR
OTA	–	–	–	–	–	→ NORMAL	→ ERROR

A `BOOT` állapotból `EV_BOOT_DONE` után a rendszer megvizsgálja a mentett konfigurációt: ha a `provisioned` flag igaz, akkor `EV_PROVISIONED` eseményt generál, és közvetlenül `NORMAL`-ba lép; különben `PROVISIONING`-ba kerül. A `PROVISIONING` állapotban a beüzemelési szolgáltatás vár a felhő vagy a helyi konfigurációs interfész parancsára, majd `EV_PROVISIONED` eseményre vált.

A `NORMAL` állapot a legösszetettebb: itt fut a mérési ciklus, a hálózati küldés és a parancsfeldolgozás. Ha a hálózat megszakad, az eszköz nem ugrik azonnal hibába, hanem például újrapróbálkozási logikát futtat (`EV_NET_DOWN` → retry timer). Csak tartós hiba esetén vált `ERROR`-ba.

Az eseményeket egy központi üzenetsor, az `app_event_q` fogadja. Minden esemény egy `app_event_t` struktúra, amely tartalmazza az esemény típusát és opcionálisan adatokat (pl. hiba kódja, mérés eredménye). Az állapotgép feldolgozó függvénye a `while (1)` ciklusban blokkolva olvassa ezt az üzenetsort, és meghívja az aktuális állapot eseménykezelőjét.

8.1.2.3 Eseményvezérelt tervezés üzenetsorral

A Zephyr `k_msgq` üzenetsora (lásd Szinkronizáció és üzenetátadás: mutex, szemafor, üzenetsor, work queue) ideális eszköz az eseményvezérelt tervezéshez: a szálak és az ISR-ek eseményeket küldenek az alkalmazás-üzenetsorba, így az állapotgép egyetlen szálban, szinkron módon dolgozza

fel azokat, elkerülve a konkurens adat-hozzáférést. A `k_msgq` mérete és az üzenetek mérete rögzített, ami determinisztikus memória-felhasználást eredményez.

8.1.2.4 Konfiguráció tárolása settings/NVS-sel

Az eszköz konfigurációját (pl. site ID, node ID, provisioned flag) nem áramtalanítás-biztos memóriában, hanem flash-ben tároljuk. A Zephyr `settings` alrendszere erre egy egységes API-t ad, amely a háttérben NVS-t (Non-Volatile Storage) vagy más hátteret használ. A laborban a `settings` alrendszert konfiguráljuk, és az egyszerű kulcs-érték párok tárolását mutatjuk be. Az NVS réteg a flash feletti, ciklusbiztos fájlrendszerszerű tárolást valósít meg.

Definíció. Az NVS (Non-Volatile Storage) egy flash-alapú tároló, amelyet a Zephyr a konfigurációs adatok megőrzésére biztosít. A `settings` alrendszer az NVS fölé helyezett absztrakció, amely kulcs-érték párok tárolását és visszatöltését teszi lehetővé.

8.1.3 Döntési szempontok

A rétegezés és az állapotgép bevezetése nem mindig szükséges. Az alábbi táblázat a megfontolandó szempontokat foglalja össze:

Szempont	Egyszerű lineáris kód	Rétegezett + állapotgép
Komplexitás	Alacsony	Magasabb kezdő ráfordítás
Hibakezelés	Nehezen átlátható	Központosított, állapotonként elkülönített
Bővíthetőség	Korlátozott	Új állapotok/események hozzáadása lokális
Tesztelhetőség	Csak teljes integráció	Rétegenként és állapotonként izolálható
Memória/futás-idő	Kisebb	Kicsit nagyobb (üzenetsor, táblák)
Ajánlott, ha	Rövid életű, egyfunkciós eszköz	Hálózatba kötött, frissíthető, hibatűrő eszköz

A `ts-node` esetében a hálózati kapcsolat, a provisioning és az OTA miatt egyértelműen az állapotgépes megközelítés a helyes választás.

A konfiguráció tárolására használható NVS helyett közönséges változó is, de akkor az újraindítás után elvesznek a beállítások. Az NVS ciklusbiztos, és a `settings` API-val egyszerűen használható. Hátránya, hogy a flash írási ciklusai korlátozottak, ezért a gyakran változó adatokat nem szabad NVS-be írni.

8.1.4 Kidolgozott példa és labor: `ts-node` állapotgép

Munkakönyvtár: `mkdir -p ~/iot-labor/i018 && cd ~/iot-labor/i018`

A labor célja, hogy a korábbi Szenzorolvasás a Zephyr sensor API-val alfejezetből származó szenzorolvasó kódot rétegekre bontva, életciklus-állapotgéppel vezérelje. A kiinduló fájlokat átmásoljuk, majd módosítjuk.

8.1.4.1 1. lépés: Kiinduló fájlok másolása

```
cp -r ~/iot-labor/i013/src ~/iot-labor/i018/
cp ~/iot-labor/i013/CMakeLists.txt ~/iot-labor/i018/
cp ~/iot-labor/i013/prj.conf ~/iot-labor/i018/
mkdir -p ~/iot-labor/i018/boards
cp ~/iot-labor/i013/boards/native_sim.overlay ~/iot-labor/i018/boards/
```

8.1.4.2 2. lépés: A prj.conf kiegészítése a settings/NVS támogatásához

A `~/iot-labor/i018/prj.conf` fájlhoz adjuk hozzá a következő sorokat (a már meglévő `CONFIG_SENSOR_SIM=y` és egyéb beállítások mellé):

```
CONFIG_SETTINGS=y
CONFIG_SETTINGS_NVS=y
CONFIG_NVS=y
CONFIG_FLASH=y
CONFIG_FLASH_MAP=y
CONFIG_FLASH_SIMULATOR=y
```

A `native_sim` board flash-szimulációt biztosít (`CONFIG_FLASH_SIMULATOR`), így az NVS a szimulált flash-ben működik. A `CONFIG_MPU_ALLOW_FLASH_WRITE` ARM-specifikus beállítás, natív szimuláción nincs rá szükség. Az NVS backendnek szüksége van egy flash-partícióra és a `zephyr,settings-partition` chosen node-ra; ezt a `boards/native_sim.overlay` végére illesztjük:

```
/ {
    chosen {
        zephyr,settings-partition = &storage_partition;
    };
};
```

A `storage_partition` a `native_sim` board flash-szimulátorának devicetree-jében szerepel. Ha a telepített Zephyr-kiadásban más a partíció neve, a hivatkozást ahhoz kell igazítani. A `settings_subsys_init()` hívása kötelező a `settings_load()` és a `settings_save_one()` előtt; ezt a következő lépésben az állapotgép végzi el.

8.1.4.3 3. lépés: A HAL és az állapotgép interfészének definiálása

Először a közös adatmodellt és a HAL-t emeljük ki a korábbi `main.c`-ből. A `src/sensor_hal.h` tartalmazza a `struct ts_sample` pontos, Szenzorolvasás a Zephyr sensor API-val alfejezetből ismert elrendezését, a `sample_msgq` deklarációját és az olvasófüggvény prototípusát. Így az állapotgép, a szolgáltatások és a `main.c` ugyanazt a típust használja, és nincs eltérő struktúraelrendezés:

```
/* src/sensor_hal.h */
#ifndef SENSOR_HAL_H
#define SENSOR_HAL_H

#include <zephyr/device.h>
#include <zephyr/kernel.h>
#include <stdbool.h>
#include <stdint.h>

struct ts_sample {
    uint32_t seq;
    int64_t uptime_ms;
```

```
    int64_t event_ts_epoch_ms;
    float temperature_c;
    float humidity_rh;
    float soil_moisture_pct;
    bool soil_moisture_valid;
};

extern struct k_msgq sample_msgq;

const struct device *sensor_hal_device(void);
int sensor_read_sample(const struct device *dev, struct ts_sample *sample);

#endif /* SENSOR_HAL_H */
```

A `src/sensor_hal.c` a Szenzorolvasás a Zephyr sensor API-val alfejezetből átemelt olvasót tartalmazza változatlan működéssel; csak a `static` jelölés kerül le, hogy a szolgáltatás hívhassa. Az üzenetsor definíciója is ide kerül, így a `main.c`-ben nem kell extern módon elérni:

```
/* src/sensor_hal.c */
#include "sensor_hal.h"

#include <zephyr/devicetree.h>
#include <zephyr/drivers/sensor.h>
#include <zephyr/logging/log.h>

LOG_MODULE_REGISTER(sensor_hal, LOG_LEVEL_INF);

K_MSGQ_DEFINE(sample_msgq, sizeof(struct ts_sample), 8, 4);

const struct device *sensor_hal_device(void)
{
    return DEVICE_DT_GET(DT_NODELABEL(temp_sensor));
}

int sensor_read_sample(const struct device *dev, struct ts_sample *sample)
{
    struct sensor_value temp_val, hum_val;
    int ret;

    if (!dev) {
        return -EINVAL;
    }

    ret = sensor_sample_fetch(dev);
    if (ret != 0) {
        LOG_ERR("fetch failed: %d", ret);
        return ret;
    }

    ret = sensor_channel_get(dev, SENSOR_CHAN_AMBIENT_TEMP, &temp_val);
    if (ret != 0) {
        LOG_ERR("temp get failed: %d", ret);
        return ret;
    }

    ret = sensor_channel_get(dev, SENSOR_CHAN_HUMIDITY, &hum_val);
    if (ret != 0) {
        LOG_ERR("humidity get failed: %d", ret);
        return ret;
    }

    sample->temperature_c = (float)sensor_value_to_double(&temp_val);
    sample->humidity_rh = (float)sensor_value_to_double(&hum_val);
```

```
    sample->soil_moisture_pct = 0.0f;
    sample->soil_moisture_valid = false;

    return 0;
}
```

Ezután hozzuk létre a `src/app_state_machine.h` fájlt a következő tartalommal:

```
#ifndef APP_STATE_MACHINE_H
#define APP_STATE_MACHINE_H

#include <zephyr/kernel.h>

/* Állapotok */
typedef enum {
    APP_STATE_BOOT,
    APP_STATE_PROVISIONING,
    APP_STATE_NORMAL,
    APP_STATE_OTA,
    APP_STATE_ERROR
} app_state_t;

/* Események */
typedef enum {
    APP_EVT_BOOT_DONE,
    APP_EVT_PROVISIONED,
    APP_EVT_NET_DOWN,
    APP_EVT_NET_UP,
    APP_EVT_ENTER_OTA,
    APP_EVT_OTA_DONE,
    APP_EVT_ERROR
} app_event_type_t;

typedef struct {
    app_event_type_t type;
    int error_code;
} app_event_t;

/* Az alkalmazás-üzenetsor, amelyre az események érkeznek */
extern struct k_msgq app_event_q;

/* Az állapotgép inicializálása és fő ciklusa */
void app_state_machine_start(void);

#endif /* APP_STATE_MACHINE_H */
```

8.1.4.4 4. lépés: Az állapotgép és a rétegek implementálása

A `src/app_state_machine.c` fájl tartalmazza az állapotgép logikáját. A szolgáltatások külön modulokban vannak, de a laborban a rövidség kedvéért a szolgáltatásfüggvényeket is ugyanebben a fájlban definiáljuk (később szétválaszthatók). A HAL-t a korábbi `sensor_read_sample()` jelenti.

```
#include "app_state_machine.h"

#include <zephyr/kernel.h>
#include <zephyr/logging/log.h>
#include <zephyr/settings/settings.h>
#include <string.h>

#include "sensor_hal.h"
```

```
LOG_MODULE_REGISTER(app_sm, LOG_LEVEL_INF);

#define SETTINGS_KEY_PROVISIONED "provisioned"
#define SETTINGS_FULL_KEY "app/provisioned"
#define SENSOR_PERIOD_MS 5000

/* Alkalmazás-üzenetsor: 8 db app_event_t méretű üzenet tárolására */
K_MSGQ_DEFINE(app_event_q, sizeof(app_event_t), 8, 4);

/* A szolgáltatások és az állapotgép belső függvényei */
static void sensor_service_start(void);
static void comm_service_start(void);
static void services_stop(void);
static void enter_normal(void);
static int settings_service_init(void);
static int settings_service_load(void);
static int settings_service_save_provisioned(bool value);

static void sensor_thread(void *a, void *b, void *c);
static void comm_thread(void *a, void *b, void *c);

/* Állapotgép változó */
static app_state_t current_state = APP_STATE_BOOT;
static bool provisioned;
static bool settings_ready;

/* Szolgáltatás-szálak és állapotuk */
K_THREAD_STACK_DEFINE(sensor_stack, 2048);
K_THREAD_STACK_DEFINE(comm_stack, 2048);
static struct k_thread sensor_thread_data;
static struct k_thread comm_thread_data;
static bool sensor_service_running;
static bool comm_service_running;
static bool sensor_thread_started;
static bool comm_thread_started;
static uint32_t sensor_seq;

/* Állapot-eseménykezelő függvények (forward deklaráció) */
static void handle_state_boot(app_event_t *evt);
static void handle_state_provisioning(app_event_t *evt);
static void handle_state_normal(app_event_t *evt);
static void handle_state_ota(app_event_t *evt);
static void handle_state_error(app_event_t *evt);

/* Settings callback: a load során visszatölti az értéket */
static int settings_get_cb(const char *key, size_t len,
                           settings_read_cb read_cb, void *cb_arg)
{
    uint8_t val;
    ssize_t rc;

    ARG_UNUSED(len);

    if (!settings_name_steq(key, SETTINGS_KEY_PROVISIONED, NULL)) {
        return 0;
    }

    rc = read_cb(cb_arg, &val, sizeof(val));
    if (rc == (ssize_t)sizeof(val)) {
        provisioned = (val != 0U);
        LOG_INF("provisioned loaded: %d", provisioned);
    } else {
```

```
        LOG_ERR("invalid provisioned value length: %d", (int)rc);
    }

    return 0;
}

/* Settings backend regisztráció: az "app" handler alatt a "provisioned" kulcs */
SETTINGS_STATIC_HANDLER_DEFINE(app_settings, "app", NULL, settings_get_cb,
                               NULL, NULL);

static int settings_service_init(void)
{
    int ret;

    if (settings_ready) {
        return 0;
    }

    ret = settings_subsys_init();
    if (ret != 0) {
        LOG_ERR("settings_subsys_init failed: %d", ret);
        return ret;
    }

    settings_ready = true;
    return 0;
}

static int settings_service_load(void)
{
    int ret = settings_load();

    if (ret != 0) {
        LOG_ERR("settings_load failed: %d", ret);
        return ret;
    }

    LOG_INF("settings loaded");
    return 0;
}

static int settings_service_save_provisioned(bool value)
{
    uint8_t val = value ? 1U : 0U;
    int ret;

    ret = settings_save_one(SETTINGS_FULL_KEY, &val, sizeof(val));
    if (ret != 0) {
        LOG_ERR("settings_save_one failed: %d", ret);
        return ret;
    }

    /* A RAM-beli állapotot csak sikeres tartós mentés után frissítjük. */
    provisioned = value;
    return 0;
}

static void sensor_thread(void *a, void *b, void *c)
{
    const struct device *dev = sensor_hal_device();

    ARG_UNUSED(a);
    ARG_UNUSED(b);
```

```
    ARG_UNUSED(c);

    if (!device_is_ready(dev)) {
        LOG_ERR("temp_sensor not ready, sensor service stops");
        sensor_service_running = false;
        return;
    }

    while (sensor_service_running) {
        struct ts_sample sample = {0};

        if (sensor_read_sample(dev, &sample) == 0) {
            sample.seq = ++sensor_seq;
            sample.uptime_ms = k_uptime_get();
            sample.event_ts_epoch_ms = 0; /* nincs óraszinkron a laborban */
            if (k_msgq_put(&sample_msgq, &sample, K_NO_WAIT) != 0) {
                LOG_WRN("sample queue full");
            }
        } else {
            LOG_ERR("sensor read failed");
        }

        k_sleep(K_MSEC(SENSOR_PERIOD_MS));
    }
}

static void comm_thread(void *a, void *b, void *c)
{
    struct ts_sample sample;

    ARG_UNUSED(a);
    ARG_UNUSED(b);
    ARG_UNUSED(c);

    while (comm_service_running) {
        if (k_msgq_get(&sample_msgq, &sample, K_MSEC(200)) != 0) {
            continue;
        }

        /* A laborban a fogyasztó naplóz; később MQTT-publish lesz belőle. */
        LOG_INF("telemetry seq=%u temp=%d centiC hum=%d centi%%",
                sample.seq, (int)(sample.temperature_c * 100.0f),
                (int)(sample.humidity_rh * 100.0f));
    }
}

static void sensor_service_start(void)
{
    if (sensor_service_running) {
        return;
    }

    if (!device_is_ready(sensor_hal_device())) {
        LOG_ERR("temp_sensor not ready; sensor service not started");
        return;
    }

    sensor_service_running = true;
    sensor_thread_started = true;
    k_thread_create(&sensor_thread_data, sensor_stack,
                    K_THREAD_STACK_SIZEOF(sensor_stack),
                    sensor_thread, NULL, NULL, NULL,
                    K_PRIO_PREEMPT(8), 0, K_NO_WAIT);
```

```
    LOG_INF("sensor_service started");
}

static void comm_service_start(void)
{
    if (comm_service_running) {
        return;
    }

    comm_service_running = true;
    comm_thread_started = true;
    k_thread_create(&comm_thread_data, comm_stack,
                    K_THREAD_STACK_SIZEOF(comm_stack),
                    comm_thread, NULL, NULL, NULL,
                    K_PRIO_PREEMPT(9), 0, K_NO_WAIT);
    LOG_INF("comm_service started");
}

static void services_stop(void)
{
    sensor_service_running = false;
    comm_service_running = false;

    if (sensor_thread_started) {
        (void)k_thread_join(&sensor_thread_data, K_SECONDS(7));
        sensor_thread_started = false;
    }

    if (comm_thread_started) {
        (void)k_thread_join(&comm_thread_data, K_SECONDS(2));
        comm_thread_started = false;
    }
}

static void enter_normal(void)
{
    current_state = APP_STATE_NORMAL;
    sensor_service_start();
    comm_service_start();
}

/* Állapotkezelők */
static void handle_state_boot(app_event_t *evt)
{
    int ret;

    if (evt->type != APP_EVT_BOOT_DONE) {
        LOG_WRN("Unexpected event %d in BOOT", evt->type);
        return;
    }

    ret = settings_service_init();
    if (ret != 0) {
        LOG_ERR("Settings backend unavailable, entering ERROR");
        current_state = APP_STATE_ERROR;
        return;
    }

    ret = settings_service_load();
    if (ret != 0) {
        LOG_ERR("Configuration load failed, entering ERROR");
        current_state = APP_STATE_ERROR;
        return;
```

```
    }

    if (provisioned) {
        LOG_INF("Already provisioned, go to NORMAL");
        enter_normal();
    } else {
        LOG_INF("Not provisioned, go to PROVISIONING");
        current_state = APP_STATE_PROVISIONING;
    }
}

static void handle_state_provisioning(app_event_t *evt)
{
    int ret;

    if (evt->type != APP_EVT_PROVISIONED) {
        LOG_WRN("Unexpected event %d in PROVISIONING", evt->type);
        return;
    }

    ret = settings_service_save_provisioned(true);
    if (ret != 0) {
        LOG_ERR("Provisioning persist failed: %d, staying in PROVISIONING", ret);
        return;
    }

    LOG_INF("Provisioning persisted, go to NORMAL");
    enter_normal();
}

static void handle_state_normal(app_event_t *evt)
{
    switch (evt->type) {
    case APP_EVT_NET_DOWN:
        LOG_WRN("Network down, entering retry logic");
        /* Itt például elindítunk egy retry timer-t, de most nem teszünk semmit */
        break;
    case APP_EVT_NET_UP:
        LOG_INF("Network up");
        break;
    case APP_EVT_ENTER_OTA:
        LOG_INF("Entering OTA state, stopping services");
        services_stop();
        current_state = APP_STATE_OTA;
        break;
    case APP_EVT_ERROR:
        LOG_ERR("Fatal error, entering ERROR state");
        services_stop();
        current_state = APP_STATE_ERROR;
        break;
    default:
        LOG_WRN("Unexpected event %d in NORMAL", evt->type);
    }
}

static void handle_state_ota(app_event_t *evt)
{
    if (evt->type == APP_EVT_OTA_DONE) {
        LOG_INF("OTA done, go back to NORMAL");
        enter_normal();
    } else if (evt->type == APP_EVT_ERROR) {
        LOG_ERR("OTA error, entering ERROR state");
        current_state = APP_STATE_ERROR;
```

```
    } else {
        LOG_WRN("Unexpected event %d in OTA", evt->type);
    }
}

static void handle_state_error(app_event_t *evt)
{
    LOG_ERR("Device in ERROR state; only reboot helps");
    /* Nem teszünk mást, csak várunk a watchdogra vagy manuális resetre */
}

/* Fő állapotgép ciklus */
void app_state_machine_start(void)
{
    LOG_INF("State machine started in BOOT state");
    app_event_t evt;

    /* Boot esemény generálása */
    app_event_t boot_evt = { .type = APP_EVT_BOOT_DONE };
    k_msgq_put(&app_event_q, &boot_evt, K_NO_WAIT);

    while (1) {
        k_msgq_get(&app_event_q, &evt, K_FOREVER);
        LOG_INF("Processing event %d in state %d", evt.type, current_state);

        switch (current_state) {
        case APP_STATE_BOOT:
            handle_state_boot(&evt);
            break;
        case APP_STATE_PROVISIONING:
            handle_state_provisioning(&evt);
            break;
        case APP_STATE_NORMAL:
            handle_state_normal(&evt);
            break;
        case APP_STATE_OTA:
            handle_state_ota(&evt);
            break;
        case APP_STATE_ERROR:
            handle_state_error(&evt);
            break;
        default:
            LOG_ERR("Invalid state %d", current_state);
        }
    }
}

/* LAB-segédszál: a provisioning interfész hiányában 3 mp után szimulálja
 * a beérkező APP_EVT_PROVISIONED eseményt. Éles rendszerben ezt a
 * provisioning szolgáltatás vagy a felhő küldi. */
static void provisioning_sim_thread(void *a, void *b, void *c)
{
    ARG_UNUSED(a);
    ARG_UNUSED(b);
    ARG_UNUSED(c);

    k_sleep(K_SECONDS(3));

    if (current_state == APP_STATE_PROVISIONING) {
        app_event_t evt = { .type = APP_EVT_PROVISIONED };

        LOG_INF("LAB: simulated provisioning event");
        k_msgq_put(&app_event_q, &evt, K_NO_WAIT);
```

```
    }
}

K_THREAD_DEFINE(prov_sim_tid, 1024, provisioning_sim_thread, NULL, NULL, NULL,
                K_PRIO_PREEMPT(10), 0, K_NO_WAIT);
```

A fenti kódban a `sample_msgq` és a `sensor_read_sample` a `sensor_hal.h` közös interfészén keresztül érhető el, a definíciójuk a `sensor_hal.c`-ben van. A mérőszálat és a fogyasztót is az állapotgép indítja: a `sensor_service_start()` valódi Zephyr-szálat hoz létre, amely a `sample_msgq`-ra teszi a mintákat, a `comm_service_start()` pedig létrehozza a fogyasztó szálat, amely a mintákat naplózza (később MQTT-publish lesz belőle). Mindkét szolgáltatás csak `NORMAL` állapotban fut; `OTA` vagy `ERROR` állapotba lépéskor a `services_stop()` leállítja és joinolja a szálakat. A `PROVISIONING` állapotban egyik szolgáltatás sem indul el. A `provisioning_sim_thread` csak laborsegéd: 3 másodperc után szimulálja a provisioning eseményt, hogy a kimenet reprodukálható legyen.

8.1.4.5 5. lépés: A `main.c` egyszerűsítése

A `src/main.c` már csak az alkalmazás belépési pontját tartalmazza: elindítja az állapotgépet, és minden szolgáltatás indítását az állapotgépre bízza. Ezzel megszűnik az a korábbi hiba, hogy a mérőszál a provisioning állapotától függetlenül, automatikusan elindult.

```
#include <zephyr/kernel.h>
#include <zephyr/logging/log.h>
#include "app_state_machine.h"

LOG_MODULE_REGISTER(main, LOG_LEVEL_INF);

int main(void)
{
    LOG_INF("ts-node starting");
    app_state_machine_start();

    return 0;
}
```

A `main()` belép az állapotgép végtelen ciklusába. A mérőszál és a fogyasztó szál létrehozása a `NORMAL` állapotba lépéskor történik, a `sensor_hal.c` pedig a korábbi laborból változatlanul átemelt olvasót használja. A `CMakeLists.txt`-ben mindhárom forrás szerepel:

```
cmake_minimum_required(VERSION 3.20)
find_package(Zephyr REQUIRED HINTS $ENV{ZEPHYR_BASE})
project(ts_node_i018)

target_sources(app PRIVATE
  src/main.c
  src/app_state_machine.c
  src/sensor_hal.c
)
```

8.1.4.6 6. lépés: Build és futtatás

```
cd ~/iot-labor/i018
west build -b native_sim .
./build/zephyr/zephyr.exe --flash=i018_flash.bin
```

A `--flash=i018_flash.bin` kapcsoló a `native_sim` flash-szimulátorát fájlhoz köti, így a `settings` tartalma a folyamat újraindítása után is megmarad. E nélkül a szimulált flash csak RAM-ban él, és a „reboot” után elveszne a mentett érték. A második futtatás ezért már a mentett `provisioned` értéket tölti be.

Várt kimenet az első futásból (részlet):

```
[00:00:00.000,000] <inf> main: ts-node starting
[00:00:00.000,000] <inf> app_sm: State machine started in BOOT state
[00:00:00.000,000] <inf> app_sm: Processing event 0 in state 0
[00:00:00.000,000] <inf> app_sm: settings loaded
[00:00:00.000,000] <inf> app_sm: Not provisioned, go to PROVISIONING
[00:00:03.000,000] <inf> app_sm: LAB: simulated provisioning event
[00:00:03.000,000] <inf> app_sm: Processing event 1 in state 1
[00:00:03.000,000] <inf> app_sm: Provisioning persisted, go to NORMAL
[00:00:03.000,000] <inf> app_sm: sensor_service started
[00:00:03.000,000] <inf> app_sm: comm_service started
[00:00:03.000,000] <inf> app_sm: telemetry seq=1 temp=... centiC hum=... centi%
...
```

A második futásban a `settings_load()` már `provisioned = 1` értéket tölt be, ezért az eszköz közvetlenül `NORMAL`-ba lép, és a szimulált provisioning esemény már nem kerül feldolgozásra:

```
[00:00:00.000,000] <inf> app_sm: provisioned loaded: 1
[00:00:00.000,000] <inf> app_sm: Already provisioned, go to NORMAL
[00:00:00.000,000] <inf> app_sm: sensor_service started
[00:00:00.000,000] <inf> app_sm: comm_service started
...
```

A kimenetben az első futás a BOOT-ból PROVISIONING-ba lép, majd a szimulált esemény és a sikeres tartós mentés után NORMAL-ba. A második futás a mentett értékből közvetlenül NORMAL-ba jut; mindkét útvonal ugyanazt az `enter_normal()` függvényt hívja, így a mérési és kommunikációs szolgáltatás mindkét esetben elindul.

8.1.5 Ellenőrzés és hibakeresés

A labor működésének ellenőrzéséhez a log üzeneteket figyeljük. A logból láthatóvá kell válnia, hogy az állapotgép melyik eseményt milyen állapotban dolgozza fel, és hogyan vált át.

Tipikus hibatünetek és megoldások:

Hibatünet	Ok	Javítás
Az állapotgép nem vált át, az esemény “Unexpected event” logot ír	Az esemény típusa nem megfelelő, vagy az állapotkezelő nem ismeri a kombinációt	Ellenőrizd az esemény enum értékét, és hogy az adott állapotban milyen eseményeket várunk; javítsd az átmeneteket
Az NVS-ből nem töltődik be a provisioned érték	A settings kulcsa nem egyezik, vagy a regisztrált handler nem megfelelő	Ellenőrizd a `SETTINGS_STATIC_HANDLER_DEFINE` nevét és a kulcs formátumát (`"app/provisioned"`), valamint a `settings_get_cb` visszatérési értékét

Hibatünet	Ok	Javítás
A `sample_msgq` vagy a `sensor_read_sample` definiálatlan	A `sensor_hal.c` nincs a buildben	Vedd fel a `src/sensor_hal.c`-t a `CMakeLists.txt` `target_sources` listájára
A `settings_load` után nem töltődik be a `provisioned` érték	Hiányzó `zephyr,settings-partition` chosen node, NVS backend vagy flash-szimulátor; esetleg nem futott le a `settings_subsys_init()`	Ellenőrizd az overlay chosen node-ját és a `CONFIG_SETTINGS_NVS`, `CONFIG_FLASH_SIMULATOR` kapcsolókat; a `settings_subsys_init()`-nek a `settings_load()` előtt kell futnia
A második futás után is PROVISIONING-ba lép	A flash-szimulátor nincs fájlhoz kötve, ezért a mentés nem maradt meg	Indítsd a binárist `--flash=i018_flash.bin` kapcsolóval

8.1.6 Tipikus hibák és félreértések

1. **Az állapotgép futtatása ISR-ből.** Sokan megpróbálják az állapotátmeneteket közvetlenül megszakításból hívni. Ez veszélyes, mert az ISR-ekben nem szabad blokkoló hívást végezni, és a kontextusváltás korlátozott. A helyes megoldás, hogy az ISR csak egy eseményt küld az üzenetsorra (`k_msgq_put`), és az állapotgép szálon dolgozza fel.

2. **A rétegek közötti közvetlen hívás.** Gyakori hiba, hogy az alkalmazás közvetlenül eléri a HAL függvényeket, vagy a szolgáltatások egymás belső adatait manipulálják. Ez összekuszálja a függőségeket, és megnehezíti a tesztelést. Mindig interfészeken keresztül kommunikáljunk, és az adatokat üzenetsorokon vagy visszahívásokon keresztül adjuk át.

3. **A settings/NVS írásának gyakorisága.** A flash-nek korlátozott az írási ciklusa, ezért a provisioned flag-et csak akkor írjuk, amikor valóban megváltozik. Nem szabad minden ciklusban menteni például az aktuális hőmérsékletet NVS-be; az ilyen gyorsan változó adatokat RAM-ban vagy külső tárban kell tartani.

4. **Hiányzó várakozás a settings betöltésére.** A `settings_load()` aszinkron is lehet (bár alapértelmezésben szinkron), de a konfigurációra csak a betöltés befejezése után támaszkodhatunk. Ha a rendszer azelőtt használja a `provisioned` változót, mielőtt a callback lefutott volna, hibás döntést hozhat. A laborban a `settings_service_load()` hívása után közvetlenül olvassuk a változót, ami működik, de élesben érdemes szinkronizációs eseményt használni.

8.1.7 Feladatok

8.1.7.1 a) Fogalmi/döntési kérdés

Ismertesd röviden, miért előnyös az életciklus-állapotgép használata a `ts-node` esetében a lineáris kóddal szemben! Milyen hátrányai vannak az állapotgépes megközelítésnek egyszerű eszközökön?

8.1.7.2 b) Számolási vagy tervezési feladat

Tervezd meg a `NORMAL` állapot belső eseménykezelését egy retry logikával! Hányféleképpen lehet megszakadni a hálózatnak, és milyen állapotváltozók kellenek a retry számlálóhoz? Tervezd meg a retry állapotátmeneteket egy táblázatban.

8.1.7.3 c) Implementációs feladat

Egészítsd ki a labor kódját egy `APP_EVT_LOW_BATTERY` eseménnyel, amely a `NORMAL` állapotban `ERROR`-ba viszi az eszközt, és logolja ki az akkumulátor töltöttségét! Végül a `main()`-ban szimuláld az eseményt néhány másodperc múlva egy időzítővel.

8.1.7.4 d) Hibakeresési/kiterjesztési feladat

Vizsgáld meg, mi történik, ha a settings betöltése közben az NVS flash nem elérhető (pl. törölt flash). Hogyan lehetne kezelni ezt a hibát az állapotgépben? Javasolj egy `APP_EVT_CONFIG_ERROR` eseményt, és implementáld a kezelését!

8.1.8 Megoldások

8.1.8.1 a) Megoldás

Az életciklus-állapotgép központosítja a viselkedést: minden állapotban csak azok az események engedélyezettek, amelyek az adott állapotban értelmesek. Ez megakadályozza a váratlan átmeneteket, és a hibakeresést is strukturálja – ha egy esemény nem várt, azt logoljuk. A lineáris kódban az if-else láncok gyorsan átláthatatlanná válnak, és könnyű kihagyni egy kombinációt. Hátránya az állapotgépnek, hogy növeli a memória- és futásidő-igényt (táblák, üzenetsor), és kezdeti tervezést igényel. Egyszerű, egyfunkciós eszközön, ahol nincs sok bemenet, felesleges komplexitást jelent.

8.1.8.2 b) Megoldás

A hálózat megszakadása lehet átmeneti (pl. rövid interferencia) vagy tartós (pl. nincs térerő). A retry logika állapotváltozói: `retry_count`, `retry_max` (pl. 5), `retry_interval` (pl. 30 s). Átmenetek:

Állapot	Esemény	Következő	Művelet
NORMAL	NET_DOWN	RETRY	retry_count = 0, retry timer indítása

Állapot	Esemény	Következő	Művelet
RETRY	NET_UP	NORMAL	kommunikáció újraindítása
RETRY	RETRY_TIMEOUT	RETRY	retry_count++; ha retry_count >= retry_max → ERROR
RETRY	ERROR	ERROR	–

8.1.8.3 c) Megoldás

Az alábbi kódrészlet mutatja az `APP_EVT_LOW_BATTERY` esemény hozzáadását és kezelését. Az esemény az enum-hoz kerül, a `handle_state_normal` pedig bővül:

```
/* az enum-ban: */
    APP_EVT_LOW_BATTERY,

/* a handle_state_normal-ben: */
    case APP_EVT_LOW_BATTERY:
        LOG_ERR("Low battery, entering ERROR state");
        current_state = APP_STATE_ERROR;
        break;

/* a main()-ben egy időzítővel generáljuk az eseményt:
 * (a main végén, az app_state_machine_start() hívása előtt,
 * vagy egy külön szálban) */
static void battery_sim_thread(void*, void*, void*)
{
    k_sleep(K_SECONDS(10));
    app_event_t evt = { .type = APP_EVT_LOW_BATTERY, .error_code = -1 };
    k_msgq_put(&app_event_q, &evt, K_FOREVER);
}
K_THREAD_DEFINE(battery_sim_tid, 512, battery_sim_thread, NULL, NULL, NULL,
                K_PRIO_PREEMPT(8), 0, 0);
```

8.1.8.4 d) Megoldás

Ha a settings betöltése közben az NVS flash meghibásodik, a `settings_load()` hiba kódot ad vissza. Ilyenkor a rendszer nem tudja eldönteni, hogy érvényes konfigurációval rendelkezik-e. Az állapotgépnek `ERROR` állapotba kellene lépnie, de nehogy végtelen újraindulási ciklus alakuljon ki. Javaslat: vezessük be az `APP_EVT_CONFIG_ERROR` eseményt, amely a BOOT állapotból ERROR-ba visz, és a hiba naplózása után újraindítja az eszközt (a watchdog majd ezt megteszi). A megoldás a `settings_service_load()` visszatérési értékének ellenőrzése, és hiba esetén az esemény küldése.

8.1.9 Rövid összefoglaló

A `ts-node` eszközszoftverét három rétegre bontottuk: HAL (pl. `sensor_read_sample()`), szolgáltatások (sensor, comm, settings) és alkalmazás (életciklus-állapotgép). Az életciklus-állapotgép állapotai: `BOOT`, `PROVISIONING`, `NORMAL`, `OTA`, `ERROR`; az eseményeket az `app_event_q` üzenetsoron keresztül fogadja. A konfigurációval kapcsolatos adatokat (pl. `provisioned`

flag) a Zephyr settings/NVS rétegében tároljuk. A laborban elkészítettük a közös adatmodellt és HAL-t (`src/sensor_hal.h`, `src/sensor_hal.c`), az állapotgép és a szolgáltatások implementációját (`src/app_state_machine.c`), a minimális `main.c`-t, valamint beállítottuk a settings/NVS támogatást a `prj.conf`-ban és az overlay-ben. A mérési és kommunikációs szálat kizárólag az állapotgép indítja `NORMAL` állapotban, és állítja le `OTA/ERROR` állapotban. **Továbbvitt eredmény:** az `app_state_machine.h/.c` állapotgép interfész és implementáció, az `app_event_q` üzenetsor, az állapotok és események konvenciója; ezt használja az Gyártás és üzembe helyezés (provisioning, onboarding), az OTA-frissítés: A/B képek, rollback, delta, fokozatos kiadás és a Hibatűrés: watchdog, újraindulás, hibanaplózás alfejezet. A laborfájlok: `~/iot-labor/i018/CMakeLists.txt`, `~/iot-labor/i018/prj.conf`, `~/iot-labor/i018/boards/native_sim.overlay`, `~/iot-labor/i018/src/main.c`, `~/iot-labor/i018/src/sensor_hal.h`, `~/iot-labor/i018/src/sensor_hal.c`, `~/iot-labor/i018/src/app_state` `~/iot-labor/i018/src/app_state_machine.c`.

8.1.10 Források és ellenőrzési hiányok

A Zephyr `k_msgq`, settings/NVS API-k és a `native_sim` build menet a Zephyr hivatalos dokumentációjából származnak. A konkrét Kconfig opciók (`CONFIG_SETTINGS`, `CONFIG_SETTINGS_NVS`, `CONFIG_NVS`, `CONFIG_FLASH`, `CONFIG_FLASH_MAP`, `CONFIG_FLASH_SIMULATOR`) neveit a telepített Zephyr-változatban ellenőrizni kell, mert verziótól függően változhatnak. A `settings_subsys_init`, `settings_load` és `settings_save_one` API nevek stabilnak tekinthetők; az NVS backendhez a `zephyr,settings-partition` chosen node és egy flash-partíció szükséges, ezt a `boards/native_sim.overlay` rögzíti. A `native_sim` flash-szimulációjának fájlhoz kötését (`--flash`) és a `storage_partition` pontos nevét a Zephyr dokumentációban ellenőrizni érdemes. A laborban használt `sample_msgq` és `sensor_read_sample` függvények a Szenzorolvasás a Zephyr sensor API-val laborból származnak, a `struct ts_sample` mezőit és az olvasó működését a korábbi fejezet rögzíti.

8.2 Hibatűrés: watchdog, újraindulás, hibanaplózás

8.2.1 Motiváció

A TerraSense szenzorcsomópontja hosszú időn át, felügyelet nélkül fut az üvegházban. Egyetlen elakadt szál – például egy megszakítás miatt rosszul védett I2C-tranzakció vagy egy váratlan `NULL` pointer miatti lefagyás – azt eredményezheti, hogy a csomópont nem küld telemetriát, és a felhő nem látja, mi történik az adott node-on. A hibatűrési mechanizmusok célja, hogy az eszköz felépüljön az ilyen hibákból, és a felépülést nyomon lehessen követni a backendben. Ebben az alfejezetben azt tanítjuk meg, hogyan kell watchdogot kötni a TerraSense csomópont szálaihoz, hogyan kell kiolvasni és tárolni az újraindulás okát, és hogyan kell elkerülni a végtelen újraindulási hurkot.

8.2.2 Fogalmak és működés

Definíció. A watchdog (őrkutya) egy olyan hardveres vagy szoftveres mechanizmus, amely meghatározott időn belül „etetést” (feed) vár el a felügyelt szoftvertől. Ha az etetés elmarad, a watchdog reseteli vagy újraindítja a rendszert.

Definíció. A reset ok (reset cause) az újraindulás kiváltó okát leíró érték, amelyet a boot során ki lehet olvasni a hardverből. A crash-napló (crash log) az újraindulás okán kívül a hiba kontextusát (pl. szálazonosító, programcounter, hibakód) rögzítő, nem felejtő memóriába mentett adat.

A Zephyrben két féle watchdog-ot is használhatunk:

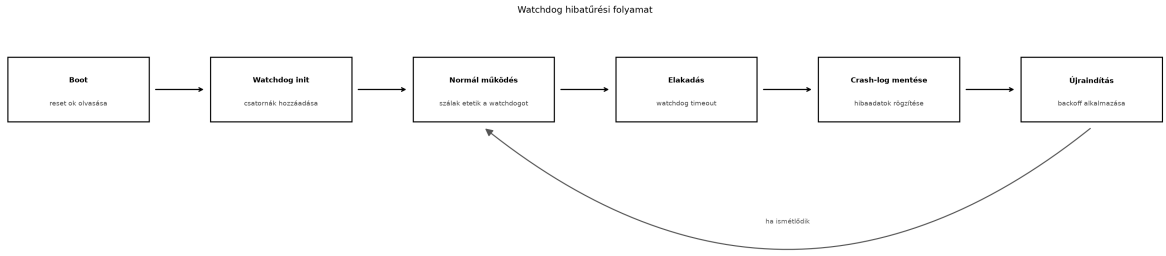
- **Hardveres watchdog:** dedikált periféria, amely a processzort reseteli, ha a szoftver nem eteti. A Zephyr `wdt` API-ján keresztül érhető el; valós fejlesztőkártyán a devicetree-ben kell engedélyezni.
- **Szoftveres (task) watchdog:** a Zephyr `task_wdt` szolgáltatása, amely kernel-szinten, szoftverből figyeli az egyes szálakat. Minden felügyelt szálhoz egy-egy csatorna tartozik, és a szálnak rendszeresen meg kell hívnia a `task_wdt_feed()` függvényt a csatorna azonosítójával.

A szoftveres watchdog előnye, hogy `native_sim` céllal is futtatható, ezért a laborban ezt használjuk. A hardveres watchdog a natív szimulációban nem elérhető, viszont éles hardveren mindig be kell kötni, mert a szoftveres watchdog nem véd akkor, ha a teljes kernel megáll.

A reset ok kiolvasása Zephyrben a hivatalos `hwinfo` API-val történik: a `<zephyr/drivers/hwinfo.h>` fejléc `int hwinfo_get_reset_cause(uint32_t *cause)` függvénye egy bitmezőbe írja a reset okát. A vizsgálandó flagok például a `RESET_POR` (power-on reset), a `RESET_SOFTWARE` (szoftveres újraindítás) és a `RESET_WATCHDOG` (watchdog által kiváltott reset). A `hwinfo_clear_reset_cause()` törli a tárolt okot, hogy a következő boot ne lássa újra. Az API-t a `CONFIG_HWINFO` Kconfig engedélyezi. Nem minden board – és a `native_sim` sem feltétlenül – implementálja: ilyenkor a függvény negatív hibakódot (`-ENOSYS`, `-ENOTSUP`) ad, amit a kódnak kezelnie kell, és „ismeretlen” okot kell naplózni.

A Zephyr `assert` és fatal error handler mechanizmusai a hibák korai észlelésére szolgálnak. A `__ASSERT(cond, msg)` makró hibás feltétel esetén a kernel fatal error handlerét hívja meg. Az alapértelmezett handler leállítja a rendszert, de saját függvénnyel felülírható, hogy a hiba adatait előbb crash-naplóba írjuk, majd `sys_reboot()`-ot hívjunk.

Újraindulási hurok akkor alakul ki, ha a hiba a boot után azonnal újra előjön, és az eszköz folyamatosan resetelek. A **backoff** (hátráló várakozás) lényege, hogy minden egymást követő hibás indulásnál egyre hosszabb ideig várakozunk, mielőtt újra megpróbálnánk a normál működést; ez időt ad a külső körülmények megváltozására (pl. rádiós zavar megszűnik), és csökkenti az energiafogyasztást.



8.3. ábra: Watchdog hibatűrési folyamat - A TerraSense csomópont boot utáni folyamata.

A fenti ábra a teljes folyamatot mutatja: boot után kiolvassuk a reset okot, elindítjuk a watchdogot, normál működés közben a szálak etetik, ha elakadás történik, a watchdog timeout bekövetkezik, a crash-log elmentődik, majd újraindul a rendszer, és ha a hiba ismétlődik, a backoff egyre hosszabb várakozást iktat be.

8.2.3 Döntési szempontok

Szempont	Hardveres watchdog	Szoftveres (task) watchdog
Véd a teljes kernel lefagyása ellen	igen	nem
Véd egyetlen elakadt szál ellen	csak ha a szál eteti a hardveres watchdogot	igen, szálankénti csatornával
Natív szimulációban (`native_sim`) futtatható	nem	igen
Többlet erőforrásigény	minimális	egy kernel-timer és néhány bájt memória
Konfigurálhatóság	devicetree + Kconfig	Kconfig + futásidejű API
Tipikus használat	éles hardveren mindenképp	Robusztus szállalapú rendszerekben, teszteléshez

A TerraSense csomópontban éles hardveren mindkettőt alkalmazzuk: a hardveres watchdog a teljes rendszer lefagyása ellen véd, a task watchdog pedig az egyes szálak elakadását figyeli. A labor a natív szimuláció miatt csak a task watchdogot mutatja be.

8.2.4 Kidolgozott példa: szándékos elakadás és helyreállás `native_sim`-en

Munkakönyvtár:

```
mkdir -p ~/iot-labor/i019/src && cd ~/iot-labor/i019
```

Korábbi laborból (Eszközszoftver-architektúra: rétegek és állapotgépek) átmásoljuk a már elkészített projektvázat, ha van. Ha nincs, üres Zephyr projektet hozunk létre.

```
cp -r ~/iot-labor/i018/src/* ~/iot-labor/i019/src/ 2>/dev/null || true
```

Hozzuk létre a `prj.conf` és `CMakeLists.txt` fájlokat.

`prj.conf`:

```
CONFIG_TASK_WDT=y
CONFIG_TASK_WDT_CHANNELS=4
CONFIG_PRINTK=y
CONFIG_REBOOT=y
CONFIG_HWINFO=y
```

`CMakeLists.txt`:

```
cmake_minimum_required(VERSION 3.20.0)
find_package(Zephyr REQUIRED HINTS $ENV{ZEPHYR_BASE})
project(ts_node_i019)

target_sources(app PRIVATE src/main.c)
```

A fő alkalmazás `src/main.c` tartalma:

```c
#include <zephyr/kernel.h>
#include <zephyr/drivers/hwinfo.h>
#include <zephyr/sys/printk.h>
#include <zephyr/sys/reboot.h>
#include <zephyr/sys/util.h>
#include <zephyr/task_wdt/task_wdt.h>

#define WORKER_STACK_SIZE 1024
#define FAULTY_STACK_SIZE 1024
#define WDT_TIMEOUT_MS 5000

K_THREAD_STACK_DEFINE(worker_stack, WORKER_STACK_SIZE);
K_THREAD_STACK_DEFINE(faulty_stack, FAULTY_STACK_SIZE);
static struct k_thread worker_thread_data;
static struct k_thread faulty_thread_data;

static void watchdog_callback(int channel_id, void *user_data)
{
    printk("Watchdog timeout on channel %d, rebooting...\n", channel_id);

    /* Valós hardveren itt mentenénk a crash-logot nem felejtő memóriába,
     * például a settings alrendszerrel. A laborban a kimenetre írjuk. */
    printk("Crash log: channel=%d, thread=%s\n",
           channel_id, (const char *)user_data);

    /* Szoftveres újraindítás. native_sim-en a folyamat kilép,
     * a watchdog-loop.sh indítja újra. */
    sys_reboot(SYS_REBOOT_WARM);
}

static void worker_thread(void *p1, void *p2, void *p3)
{
    int channel = POINTER_TO_INT(p1);

    ARG_UNUSED(p2);
    ARG_UNUSED(p3);

    printk("Worker thread started, feeding channel %d...\n", channel);
    while (1) {
        task_wdt_feed(channel);
        k_sleep(K_MSEC(1000));
    }
}

static void faulty_thread(void *p1, void *p2, void *p3)
{
    int channel = POINTER_TO_INT(p1);

    ARG_UNUSED(p2);
    ARG_UNUSED(p3);

    printk("Faulty thread started, will now block forever...\n");
    /* Egyszeri feed, utána elakadunk, így a watchdog lejár. */
    task_wdt_feed(channel);
    k_sleep(K_FOREVER);
}

int main(void)
{
    uint32_t reset_flags = 0U;
    int worker_ch;
    int faulty_ch;
    int ret;
```

```
    ret = hwinfo_get_reset_cause(&reset_flags);
    if (ret == 0) {
        printk("Boot: reset cause flags = 0x%08x\n", reset_flags);
        if ((reset_flags & RESET_WATCHDOG) != 0U) {
            printk("Boot: previous reset was caused by the watchdog\n");
        } else if ((reset_flags & RESET_POR) != 0U) {
            printk("Boot: power-on reset\n");
        } else if ((reset_flags & RESET_SOFTWARE) != 0U) {
            printk("Boot: software reset\n");
        } else {
            printk("Boot: reset cause unknown (no flag set)\n");
        }
        (void)hwinfo_clear_reset_cause();
    } else {
        printk("Boot: reset cause unavailable: %d\n", ret);
    }

    /* Szoftveres watchdog inicializálása.
     * A NULL paraméter azt jelenti, hogy nincs hardveres watchdog. */
    ret = task_wdt_init(NULL);
    if (ret != 0) {
        printk("task_wdt_init failed: %d\n", ret);
        return 0;
    }

    /* Két csatorna: a munkaszál és a hibás szál.
     * A visszaadott azonosítókat eltároljuk, nem feltételezzük a 0/1 értéket. */
    worker_ch = task_wdt_add(WDT_TIMEOUT_MS, watchdog_callback,
                             (void *)"worker");
    if (worker_ch < 0) {
        printk("task_wdt_add worker failed: %d\n", worker_ch);
        return 0;
    }

    faulty_ch = task_wdt_add(WDT_TIMEOUT_MS, watchdog_callback,
                             (void *)"faulty");
    if (faulty_ch < 0) {
        printk("task_wdt_add faulty failed: %d\n", faulty_ch);
        return 0;
    }

    printk("Watchdog running: worker channel %d, faulty channel %d\n",
           worker_ch, faulty_ch);

    k_thread_create(&worker_thread_data, worker_stack,
                    K_THREAD_STACK_SIZEOF(worker_stack),
                    worker_thread, INT_TO_POINTER(worker_ch), NULL, NULL,
                    K_PRIO_PREEMPT(8), 0, K_NO_WAIT);

    k_thread_create(&faulty_thread_data, faulty_stack,
                    K_THREAD_STACK_SIZEOF(faulty_stack),
                    faulty_thread, INT_TO_POINTER(faulty_ch), NULL, NULL,
                    K_PRIO_PREEMPT(9), 0, K_NO_WAIT);

    return 0;
}
```

A kódban a két szál teljes `k_thread_create` hívással jön létre, a `K_THREAD_STACK_DEFINE` által biztosított stackkel. A `task_wdt_add()` által visszaadott csatornaazonosítót nem feltételezzük 0-nak és 1-nek: a visszakapott értéket `INT_TO_POINTER`-rel adjuk át a szálnak, amely `POINTER_TO_INT`-tel olvassa vissza. Ha egy korábbi modul már foglalt csatornát, a két új azono-

sító ettől eltér, és a watchdog továbbra is a helyes csatornát figyeli. A `faulty_thread` egyszer etet, majd `K_FOREVER`-rel elakad, így az 5 másodperces timeout lejár, és meghívja a callbacket.

A laborhoz tartozó `watchdog-loop.sh` szkript:

```
#!/bin/bash
# A binárist újraindítja, ha kilép, és növekvő backoffot alkalmaz.
cd ~/iot-labor/i019
for attempt in 1 2 3; do
  echo "===== Attempt $attempt ====="
  ./build/zephyr/zephyr.exe
  rc=$?
  echo "Exited with code $rc"
  sleep $((attempt * 5))
done
```

Build és futtatás:

```
cd ~/iot-labor/i019
west build -b native_sim .
chmod +x watchdog-loop.sh
./watchdog-loop.sh
```

Várt kimenet (illusztratív, rövidítve):

```
===== Attempt 1 =====
Boot: reset cause unavailable: <hibakód>
Watchdog running: worker channel 0, faulty channel 1
Worker thread started, feeding channel 0...
Faulty thread started, will now block forever...
Watchdog timeout on channel 1, rebooting...
Crash log: channel=1, thread=faulty
Exited with code N
===== Attempt 2 =====
Boot: reset cause unavailable: <hibakód>
...
```

A kimenet illusztratív: a `hwinfo_get_reset_cause()` eredménye board- és szimulációfüggő. A `native_sim` nem feltétlenül szolgáltat valódi reset-okot, ezért ott a „reset cause unavailable” ág futhat; ilyenkor a labor a watchdog működését mutatja meg, nem a hardveres reset diagnosztikáját. Valós hardveren (például nRF52840 DK-n) a watchdog-reset után a `RESET_WATCHDOG` flag jelenik meg. A kilépési kód (`N`) szintén platformfüggő, mert a `native_sim` a `sys_reboot`-ot jellemzően a folyamat leállításával valósítja meg; a `watchdog-loop.sh` ezért nem a kód konkrét értékére, hanem a folyamat kilépésére reagál. A csatornaszámok a `task_wdt_add()` visszatérési értékei; a fenti 0 és 1 csak akkor jelenik meg, ha előttük más nem foglalt csatornát.

Továbbvitt eredmény. A reset ok és a watchdog timeout logolása a boot üzenetben, valamint a kilépési kód alapján a `watchdog-loop.sh` által kiírt `Exited with code N` sor a későbbi megfigyelhetőségi Metrikák, naplók, nyomkövetés és flottakezelési Flottakezelés alfejezetekben felhasználható: a gateway vagy a backend ebből az eseményből riasztást generálhat, ha egy node ismétlődő watchdog-resetet szenved.

8.2.5 Ellenőrzés és hibakeresés

Hibatünet	Lehetséges ok	Javítás
A watchdog nem indul el, a `task_wdt_init` hibát ad	`CONFIG_TASK_WDT` nincs engedélyezve	`prj.conf`-ban add hozzá a `CONFIG_TASK_WDT=y` sort

Hibatünet	Lehetséges ok	Javítás
A watchdog timeout sosem következik be	A hibás szál is rendszeresen etet, vagy a `K_FOREVER` nem blokkol	Ellenőrizd, hogy a `faulty_thread` csak egyszer etet, és utána valóban `K_FOREVER`-t hív
A `hwinfo_get_reset_cause()` `-ENOSYS` vagy `-ENOTSUP` hibát ad	A board vagy a `native_sim` nem implementálja a reset-ok lekérdezését	Kapcsold be a `CONFIG_HWINFO=y` beállítást, és kezeld a hibát „ismeretlen” okként; valós hardveren ellenőrizd, hogy a board támogatja-e a `hwinfo` API-t
A `native_sim` processz nem lép ki a `sys_reboot` után	A natív implementáció a folyamatot állítja le vagy újraindítja, nem hardveres resetet hajt végre	A `watchdog-loop.sh` kezelje a kilépést; ne a kilépési kód konkrét értékére, hanem a naplóüzenetre alapozz

8.2.6 Tipikus hibák és félreértések

1. **A szoftveres watchdogot hardveres resetként kezelik.** A task watchdog csak akkor véd a teljes kernel lefagyása ellen, ha a hardveres watchdog is aktív. Natív szimulációban a szoftveres watchdog megállítja a folyamatot, de valós hardveren a hardveres watchdog nélkül egy kernelpánik nem feltétlenül indít újra.
2. **A reset okot nem tárolják el tartósan.** Ha a crash-naplót nem írják nem felejtő memóriába, a reset után elveszik az ok, és a backend nem tudja diagnosztizálni a hibát. A Zephyr `settings` vagy közvetlen flash-írás szükséges a túléléshez.
3. **Nincs backoff az újraindulási hurok ellen.** Ha a hiba a boot után azonnal visszatér, az eszköz végtelen ciklusban resetel, és lemeríti az akkumulátort. Mindig alkalmazzunk növekvő várakozást vagy maximális reset-számlálót.

8.2.7 Feladatok

8.2.7.1 1. Feladat (fogalmi)

Magyarázd el, mi a különbség a hardveres és a szoftveres watchdog között, és miért nem véd a szoftveres watchdog a kernel teljes lefagyása ellen?

8.2.7.2 2. Feladat (tervezési)

A TerraSense csomópontjában a telemetria-szál 10 másodpercenként küld adatot. Tervezd meg a watchdog csatornák számát és az etetési periódusokat, ha a következő szálak futnak: telemetria, szenzorolvasás, parancsfeldolgozás, hálózati stack. Indokold, miért nem elegendő egyetlen csatorna.

8.2.7.3 3. Feladat (implementációs, TerraSense)

Egészítsd ki a labor `main.c`-jét úgy, hogy a `hwinfo_get_reset_cause()` által adott flagmezőt egy `reset_flags` nevű globális változóba mentse, és ha a `RESET_WATCHDOG` flag be van állítva, készítse elő a `{"reset_reason":"watchdog"}` JSON payloadot a `terrasense/gh1/n001/state` topic számára, majd naplózza azt. A tényleges MQTT-publikálást még ne valósítsd meg: az MQTT-klienst és a `publish_json()` segédfüggvényt a MQTT a gyakorlatban: Zephyr-kliens és Mosquitto alfejezet vezeti be, erre a laborra tehát még nem támaszkodhat.

8.2.7.4 4. Feladat (hibakeresési)

A tesztelés során a node folyamatosan újraindul, de a `hwinfo_get_reset_cause()` nem ad egyértelmű okot: vagy hibakóddal tér vissza, vagy egyetlen flag sincs beállítva. Mi lehet a hiba, és hogyan derítenéd ki, hogy a watchdog vagy valamilyen más ok okozza-e az újraindulást? Adj lépésről lépésre hibakeresési tervet.

8.2.8 Megoldások

8.2.8.1 1. Megoldás

A hardveres watchdog egy dedikált periféria, amely a CPU resetvonalát vezérli; ha a szoftver nem eteti, a hardver közvetlenül reseteli a processzort. A szoftveres watchdog ezzel szemben a kernel egy szolgáltatása (pl. Zephyr `task_wdt`), amely kernel-timerrel ellenőrzi az etetéseket. Ha a kernel teljesen lefagy, a szoftveres watchdog is lefagy, mert a kernel nem tudja futtatni a timert. A hardveres watchdog ekkor is működik, mert a CPU-tól független periféria.

8.2.8.2 2. Megoldás

Négy szálhoz négy csatornát érdemes rendelni. Minden szál a saját periódusánál rövidebb időközönként etessen, pl. ha a telemetria 10 s-onként küld, a watchdog timeout 20 s, az etetés 5 s-onként. Egyetlen csatorna nem elegendő, mert ha az egyik szál elakad, de a többi eteti a közös csatornát, a watchdog nem érzékeli az elakadást. A szálankénti csatorna lehetővé teszi, hogy pontosan az elakadt szálat azonosítsuk a callback `channel_id` paraméteréből.

8.2.8.3 3. Megoldás

A `main.c`-ben:

```
#include <zephyr/drivers/hwinfo.h>
#include <zephyr/sys/printk.h>
#include <stdbool.h>

static uint32_t reset_flags;
static bool reset_was_watchdog;

int main(void)
{
    int ret = hwinfo_get_reset_cause(&reset_flags);

    if (ret == 0) {
        reset_was_watchdog = (reset_flags & RESET_WATCHDOG) != 0U;
        printk("Boot: reset flags = 0x%08x\n", reset_flags);
```

```
        (void)hwinfo_clear_reset_cause();
    } else {
        printk("Boot: reset cause unavailable: %d\n", ret);
    }

    /* ... watchdog init ... */

    if (reset_was_watchdog) {
        const char *payload = "{\"reset_reason\":\"watchdog\"}";

        printk("state payload for terrasense/gh1/n001/state: %s\n", payload);
    }
}
```

A payload előállítása és naplózása önállóan működik. A `terrasense/gh1/n001/state` topicra való publikálást a MQTT a gyakorlatban: Zephyr-kliens és Mosquitto MQTT-alfejezet után, az ott bevezetett `publish_json()` helperrel lehet hozzáadni; így ez a labor nem függ egy későbbi fejezet függvényétől. A `reset_flags` flagmező, ezért a watchdog-ot nem `==` összehasonlítással, hanem bitmaszkkal kell vizsgálni.

8.2.8.4 4. Megoldás

A folyamatos újraindulás és a hiányzó reset-ok arra utal, hogy a `hwinfo` API-t vagy nem támogatja a board/szimuláció, vagy a reset oka nem különböztethető meg a rendelkezésre álló flagokkal. Hibakeresési terv:

1. Ellenőrizd a `watchdog-loop.sh` kimenetén a naplóüzeneteket; a „Watchdog timeout on channel …” sor egyértelműen a watchdogra utal, a kilépési kód önmagában nem.
2. Kapcsold be a Zephyr logot a kernelpánik és a fatal error handler számára, hogy lásd a pontos hibaüzenetet.
3. Tedd a `sys_reboot` callbacket naplózóvá, és figyeld, hogy a callback meghívódik-e.
4. Valós hardveren ellenőrizd a `hwinfo_get_reset_cause()` által adott flagmezőt; ott a hardveres watchdog `RESET_WATCHDOG` flaget ad.
5. Ha a `hwinfo_get_reset_cause()` továbbra is hibát ad, használj egy külső logoló rendszert (például host-oldali naplót), hogy lásd, mi okozza a kilépést, és a board dokumentációjában keresd a reset-ok támogatását.

8.2.9 Rövid összefoglaló

Ebben az alfejezetben a TerraSense csomópont hibatűrését tanítottuk. Bekötöttük a Zephyr szoftveres (task) watchdogját, amely szálankénti csatornákkal figyeli a futó szálakat, és elakadás esetén `sys_reboot`-ot hív. Megmutattuk a reset ok kiolvasását a `hwinfo_get_reset_cause()` függvénnyel, a flagmező vizsgálatát és a crash-napló tartós tárolásának szükségességét. A laborban natív szimulációban idézhető elő szándékos elakadás, amelyet a `watchdog-loop.sh` növekvő backoffal indít újra. A továbbvitt eredmény: a boot üzenetben megjelenő reset-flag és a watchdog-timeout naplója alapján a későbbi megfigyelhetőségi és flottakezelési alfejezetek riasztást generálhatnak. A kód a `~/iot-labor/i019/src/main.c`, a konfiguráció a `prj.conf`, a futtató szkript pedig a `watchdog-loop.sh`.

8.2.10 Források és ellenőrzési hiányok

- A Zephyr task watchdog API (`task_wdt_init`, `task_wdt_add`, `task_wdt_feed`) a 3.7.0 LTS dokumentációban; a csatornaazonosítót a `task_wdt_add()` visszatérési értékeként kell kezelni, nem szabad 0/1 értéket feltételezni.
- A `hwinfo_get_reset_cause()` és a `hwinfo_clear_reset_cause()` a `<zephyr/drivers/hwinfo.h>` fejlécben érhető el, `CONFIG_HWINFO` engedélyezésével; a `native_sim` és az egyes boardok támogatása eltér, ezért a hibakódot kezelni kell.
- A `native_sim` `sys_reboot`-viselkedése (kilépés vagy újraindítás, kilépési kód) a Zephyr kiadásától függ; a fejezet kódját statikusan ellenőriztük, futási eredményt nem mértünk.
- Az MQTT-publikálást tudatosan a MQTT a gyakorlatban: Zephyr-kliens és Mosquitto alfejezetre halasztottuk, hogy a labor ne hivatkozzon egy később bevezetett függvényre.

8.3 Eszközszoftver tesztelése a hoston

8.3.1 Motiváció

A TerraSense üvegházában a `ts-node` szenzorcsomópont szoftverét fejlesztve a leglassabb visszajelzést a valós hardverre történő flashelés és a terepi hibakeresés adja. Egy szenzorolvasási logikai hiba például csak akkor derülhet ki, amikor a csomópontot ténylegesen rákötjük a szenzorra, és megtörténik a mérés. Ha a hibát a fejlesztőasztalon, hardver nélkül, determinisztikusan reprodukálható tesztekkel már a build előtt elkapjuk, akkor a fejlesztési ciklus percekre rövidül, és a hibák nem jutnak ki a terepre. Az Eszközszoftver-architektúra: rétegek és állapotgépek alfejezetben a `ts-node` kódját rétegekre bontottuk: a hardverfüggő szenzorolvasást különválasztottuk a szűrő- és szelepvezérlési logikától. Ez a szétválasztás teszi lehetővé, hogy az üzleti logikát hoston, rögzített bemenetekkel teszteljük. Ebben az alfejezetben kiegészítjük a firmware-t Zephyr Ztest tesztekkel, és a tényleges modulokat a `native_sim` célon teszteljük.

8.3.2 Fogalmak és működés

A hoston történő eszközszoftver-tesztelés alapja, hogy a hardverfüggő részeket elkülönítjük a tiszta logikától. A `ts-node` esetében a hardverfüggő szenzorolvasás a Szenzorolvasás a Zephyr sensor API-val modulban marad, a `sensor_filter` (Mintavétel, kalibráció és szűrés) és a `valve` (Aktuátorok: biztonságos vezérlés és visszajelzés) pedig nem hívja közvetlenül a Zephyr `sensor` API-t. A `valve` a GPIO-t a `struct valve_ctrl` mezőin keresztül éri el, és `hw_present = false` esetén szimulált szelepként működik, így hardver nélkül tesztelhető.

Definíció. A **HAL interfész** olyan C nyelvű absztrakció, amely a hardverműveleteket függvénycsoportként írja le, és a felsőbb rétegek csak ezen a felületen keresztül érik el a hardvert. A konkrét implementáció lehet valódi szenzor-meghajtó vagy teszt-mock.

Definíció. A **függőség-befecskendezés C-ben** azt jelenti, hogy a modul nem fordítási időben köti be a hardverfüggő implementációt, hanem futásidőben kap egy függvénymutatókat tartalmazó struktúrát. Egy általános példa:

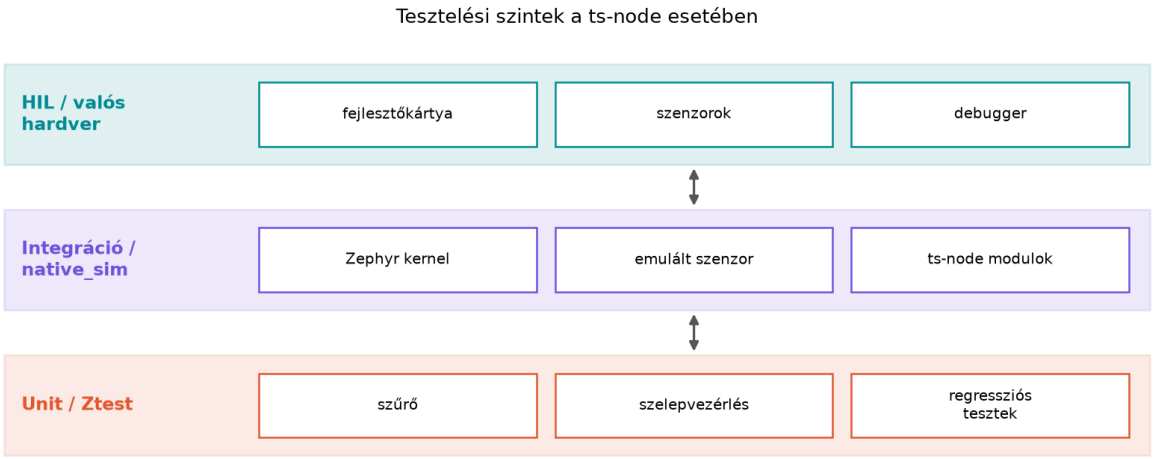
```
typedef struct {
    int (*read_temperature_celsius_milli)(int32_t *out);
    int (*read_humidity_percent_milli)(int32_t *out);
    int (*read_soil_moisture_percent)(int32_t *out);
} sensor_port_t;
```

A `ts-node` valós moduljaiban a hardverhatár ennél egyszerűbb: a `sensor_filter` tiszta számítás, bemenete és kimenete skalár, ezért közvetlenül tesztelhető; a `valve` a `struct valve_ctrl output`, `feedback` és `hw_present` mezőin keresztül éri el a GPIO-t, és `hw_present = false` esetén szimulált szelepként működik. A labor ezért nem vezet be új HAL-interfészt, hanem a két tényleges modult teszteli.

Ztest. A Ztest a Zephyr beépített egységteszt-keretrendszere. A tesztek ugyanabban a buildben fordulnak, mint a firmware, de nem a termelési `main` fut, hanem a teszt-suite-ok. A fejezet a Zephyr 3.7.0 LTS Ztest API-ját használja: a `ZTEST_SUITE`, `ZTEST` és `zassert_*` makrók suite-okba szervezett, assertelésen alapuló teszteket tesznek lehetővé.

Twister. A Twister a Zephyr tesztfuttató és riportáló eszköze. A `west twister` parancs bejárja a tesztfákat, lefordítja a teszteket a megadott boardra vagy szimulátorra, lefuttatja őket, és összesített jelentést ad. A Twister integrálható CI-rendszerbe, így a host-tesztek automatikusan lefutnak minden push után.

native_sim. A `native_sim` egy Zephyr board, amely nem mikrovezérlőt emulál, hanem a Zephyr kernelt és a modulokat natív x86-os Linux processzként futtatja. A perifériák nagy része ebben a környezetben nem elérhető, de a logikai tesztekhez nincs is rájuk szükség. A `native_sim` gyors, debuggerrel jól vizsgálható, és pontosan ez a cél: a hardverfüggetlen részeket hoston futtatni.



8.4. ábra: Tesztelési szintek a ts-node esetében - A rétegábra a host-tesztektől a valós hardverig mutatja a tesztelési spektrumot. Ebben az alfejezetben a két alsó rétegre fókuszálunk.

A **hardware-in-the-loop (HIL)** a tesztelés azon foka, ahol a szoftver valódi vagy nagy pontossággal emulált hardverrel együtt fut. Itt már a valódi időzítések, elektromos jelek és szenzorhibák is szerepet kapnak. Ez az alfejezet csak előkészíti a HIL-t: a logikai teszteket hoston futtatjuk, a HIL-t a Tesztelési szintek: unit, integráció, HIL, terheléses teszt alfejezetben részletezzük.

8.3.3 Döntési szempontok

A tesztelési stratégia kiválasztásakor a következő szempontokat érdemes mérlegelni.

Szempont	Unit teszt izolált modulokkal	native_sim integráció	HIL valós hardverrel
Sebesség	másodpercek	másodpercek–percek	percek–órák
Determinizmus	teljesen kontrollált	kontrollált, de időzítés szimulált	valós időzítés, környezet zajos
Hardverhibák modellezése	injektált bemenetekkel tetszőlegesen	korlátozott	valós, de nehezen reprodukálható
Futtatási környezet	Linux host	Linux host	fejlesztőkártya + labor
Legjobb felhasználás	szűrő, szelepvezérlés, szekvenciák	modulok együttműködése, kernel-szolgáltatások	szenzor-kalibráció, teljesítménymérés

A `ts-node` esetében a szűrő és a szelepvezérlés tiszta egységtesztet érdemel: a `sensor_filter` közvetlenül, a `valve` `hw_present = false` mellett hívható. Amikor a modulok együtt futnak a Zephyr szenzor-alrendszerrel, akkor `native_sim` integrációs teszt jöhet szóba. A valós hardvert csak akkor vesszük elő, ha a szenzor tényleges viselkedését, a bootolást vagy az energiafogyasztást kell ellenőrizni.

8.3.4 Kidolgozott példa és labor

Ebben a laborban a `ts-node` már meglévő moduljaihoz – a Szenzorfúzió és szűrés alfejezet `sensor_filter.c` és a Szelepvezérlés és biztonsági korlátok alfejezet `valve.c` moduljához – írunk Ztest teszteket, és futtatjuk őket `native_sim` alatt. Nem írunk új szűrőt vagy állapotgépet: a cél az, hogy a ténylegesen továbbvitt firmware viselkedését rögzítsük, beleértve a korábban javított hibák regressziós tesztjeit is.

Továbbvitt eredmény. Az itt létrehozott host-tesztek – `test_sensor_filter.c`, `test_valve.c`, valamint a hozzájuk tartozó `prj.conf` és `testcase.yaml` – a CI/CD a firmware-től a backendig alfejezetben a firmware-feladat részeként automatikusan lefutnak.

Munkakönyvtár:

```
mkdir -p ~/iot-labor/i020/src ~/iot-labor/i020/tests
cd ~/iot-labor/i020
```

A labor feltételezi, hogy a Mintavétel, kalibráció és szűrés és Aktuátorok: biztonságos vezérlés és visszajelzés labor elkészült, és a modulok a `~/iot-labor/i014/src/`, illetve `~/iot-labor/i015/src/` könyvtárban vannak. Másoljuk át a tényleges forrásokat a tesztprojektbe:

```
cp ~/iot-labor/i014/src/sensor_filter.c ~/iot-labor/i020/src/
cp ~/iot-labor/i014/src/sensor_filter.h ~/iot-labor/i020/src/
cp ~/iot-labor/i015/src/valve.c ~/iot-labor/i020/src/
cp ~/iot-labor/i015/src/valve.h ~/iot-labor/i020/src/
```

A `sensor_filter` fejlécének lényeges része (a teljes fájl a Mintavétel, kalibráció és szűrés alfejezetben van):

```
struct ema_filter {
    int32_t y;
    uint32_t denom;
    bool initialized;   /* hamis, amíg nincs érvényes minta */
};
```

```
void ema_init(struct ema_filter *f, uint32_t denom);
int32_t ema_update(struct ema_filter *f, int32_t x);
```

A `valve` fejlécének lényeges része (a teljes fájl az Aktuátorok: biztonságos vezérlés és visszajelzés alfejezetben van):

```
struct valve_ctrl {
    const struct gpio_dt_spec *output;
    const struct gpio_dt_spec *feedback;
    bool requested_open;
    bool reported_open;
    bool hw_present;
    int64_t on_deadline_ms;
    int64_t max_on_time_ms;
    int64_t mismatch_since_ms;
};

int valve_init(struct valve_ctrl *v, int64_t max_on_time_ms, bool hw_present);
int valve_set_target(struct valve_ctrl *v, bool open);
bool valve_is_open(const struct valve_ctrl *v);
int valve_tick(struct valve_ctrl *v);
```

A `valve_init(..., hw_present = false)` szimulált szelepet hoz létre: a GPIO-t nem éri el, a `reported_open` a kért állapotot tükrözi, így a biztonsági logika hardver nélkül tesztelhető. A `sensor_filter` egyáltalán nem függ a Zephyr hardver-API-tól, ezért a hoston közvetlenül hívható. Az egységtesztekhez nincs szükség külön mock HAL-ra; a hardverhatár a `valve_ctrl` `hw_present` mezője, illetve a tiszta függvények bemenete.

A tesztprojekt `tests/prj.conf` fájlja engedélyezi a Ztestet és a `valve.c` által használt alrendszereket:

```
CONFIG_ZTEST=y
CONFIG_LOG=y
CONFIG_GPIO=y
CONFIG_PRINTK=y
```

A `tests/testcase.yaml` írja le a Twisternek a tesztet. A `harness: ztest` beállítás azt jelenti, hogy a Twister a Ztest kimenetét értelmezi, és nem elégszik meg a folyamat nulla kilépési kódjával:

```
tests:
  ts_node.i020:
    type: unit
    platform_allow: native_sim
    integration_platforms:
      - native_sim
    tags: ztest
    harness: ztest
```

A `tests/CMakeLists.txt` a projekt `src/` könyvtárából fordítja a tényleges modulokat, és a tesztfájlokat is hozzáadja:

```
cmake_minimum_required(VERSION 3.20)
find_package(Zephyr REQUIRED HINTS $ENV{ZEPHYR_BASE})
project(ts_node_tests)

target_include_directories(app PRIVATE ${CMAKE_CURRENT_SOURCE_DIR}/../src)

target_sources(app PRIVATE
  test_sensor_filter.c
```

```
  test_valve.c
  ../src/sensor_filter.c
  ../src/valve.c
)
```

A teszteket suite-onként egy-egy fájlba tesszük. A `ZTEST_SUITE` regisztrációt pontosan egyszer, abban a fájlban kell elhelyezni, ahol a suite tesztjei vannak. Nincs szükség központi `test_main.c` fájlra: a Ztest a linker-szekciókból gyűjti össze a suite-okat, és a futtató keretrendszer saját belépési ponttal rendelkezik. Egy központi fájlban megismételt `ZTEST_SUITE(...)` ugyanannak a suite-nak a többszörös definícióját adná, amit a linker duplikált szimbólummal jelez.

A `tests/test_sensor_filter.c` az EMA első mintáját és a mozgóátlag kezdeti viselkedését ellenőrzi. Az első teszt egyben regressziós teszt: a korábbi, nulláról induló EMA-implementáció az első 3000-es mintára 750-et adott volna, mert a kezdőértéket nullának vette.

```c
/* tests/test_sensor_filter.c */
#include <zephyr/ztest.h>
#include "sensor_filter.h"

ZTEST(sensor_filter, test_ema_first_sample_initializes)
{
    struct ema_filter f;

    ema_init(&f, 4);

    /* Regresszió: a nulláról induló változat az első 3000-es mintára
     * 750-et adott volna, mert a kezdőértéket nullának vette. */
    zassert_equal(ema_update(&f, 3000), 3000,
                  "az EMA első mintája legyen a kezdőérték");
    zassert_equal(ema_update(&f, 3000), 3000,
                  "azonos minta ne módosítsa az EMA-t");
}

ZTEST(sensor_filter, test_ema_converges_towards_input)
{
    struct ema_filter f;

    ema_init(&f, 4);
    zassert_equal(ema_update(&f, 0), 0, "első minta 0");
    zassert_equal(ema_update(&f, 4000), 1000,
                  "a negyednyi lépés 1000 legyen");
    zassert_equal(ema_update(&f, 4000), 1750,
                  "a következő lépés 1750 legyen");
}

ZTEST(sensor_filter, test_moving_average_first_samples)
{
    struct moving_average f;

    ma_init(&f, 4);
    zassert_equal(ma_update(&f, 1000), 1000, "egy minta átlaga önmaga");
    zassert_equal(ma_update(&f, 3000), 2000, "két minta átlaga 2000");
    zassert_equal(ma_value(&f), 2000, "ma_value ugyanazt adja");
}

ZTEST_SUITE(sensor_filter, NULL, NULL, NULL, NULL, NULL);
```

A `tests/test_valve.c` a biztonsági logikát teszteli szimulált szeleppel (`hw_present = false`). A második teszt egyben regressziós teszt: a korábbi változat minden OPEN parancsra új határidőt számolt, és ezzel meghosszabbította a maximális nyitvatartást.

```c
/* tests/test_valve.c */
#include <zephyr/ztest.h>
#include <zephyr/kernel.h>
#include "valve.h"

ZTEST(valve, test_repeated_open_does_not_extend_deadline)
{
    struct valve_ctrl v;
    int64_t first_deadline;

    zassert_equal(valve_init(&v, 60000, false), 0, "init");
    zassert_equal(valve_set_target(&v, true), 0, "első OPEN");

    first_deadline = v.on_deadline_ms;
    zassert_true(first_deadline != 0, "nyitáskor legyen határidő");

    k_sleep(K_MSEC(5));

    zassert_equal(valve_set_target(&v, true), 0, "ismételt OPEN");
    zassert_equal(v.on_deadline_ms, first_deadline,
                  "ismételt OPEN ne hosszabbítsa meg a határidőt");
}

ZTEST(valve, test_max_on_time_forces_close)
{
    struct valve_ctrl v;

    zassert_equal(valve_init(&v, 1, false), 0, "init 1 ms korláttal");
    zassert_equal(valve_set_target(&v, true), 0, "OPEN");

    k_sleep(K_MSEC(2));

    zassert_equal(valve_tick(&v), 0, "tick a határidő után");
    zassert_false(v.requested_open, "a kérés zárásra vált");
    zassert_false(valve_is_open(&v), "a szelep zárva");
}

ZTEST_SUITE(valve, NULL, NULL, NULL, NULL, NULL);
```

A `ZTEST_SUITE` mindkét fájlban pontosan egyszer szerepel, és a suite neve (`sensor_filter`, `valve`) a tesztnevek előtagja. Nincs külön `test_main.c`: a futtató keretrendszer a linker-szekciókból gyűjti össze a regisztrált suite-okat.

A build és futtatás a `tests/` könyvtárból, `native_sim` célra:

```
cd ~/iot-labor/i020/tests
west build -b native_sim .
./build/zephyr/zephyr.exe
```

A Twister a `testcase.yaml` alapján fedezi fel a tesztet. A platformot `-p` kapcsolóval kell kiválasztani:

```
cd ~/iot-labor/i020
west twister -T tests -p native_sim
```

Ha a `west twister` parancs a telepített Zephyr-verzióban nem érhető el, a `zephyr/scripts/twister -T tests -p native_sim` hívás ugyanezt teszi. A Twister kimenetében ellenőrizd, hogy a kiválasztott tesztscenáriumok és a futtatott binárisok száma 1; a `harness: ztest` és a `--inline-logs` kapcsoló megmutatja, hogy valóban lefutott-e az öt teszt, és nem nulla tesztre adott-e látszólagos sikert.

A várt kimenetben minden teszt PASS jelölést kap, például:

```
Running test suite sensor_filter
===================================================================
START - test_ema_first_sample_initializes
 PASS - test_ema_first_sample_initializes
START - test_ema_converges_towards_input
 PASS - test_ema_converges_towards_input
START - test_moving_average_first_samples
 PASS - test_moving_average_first_samples
===================================================================
Running test suite valve
===================================================================
START - test_repeated_open_does_not_extend_deadline
 PASS - test_repeated_open_does_not_extend_deadline
START - test_max_on_time_forces_close
 PASS - test_max_on_time_forces_close
===================================================================
PROJECT EXECUTION SUCCESSFUL
```

A kimenet illusztratív; a pontos sorok a Zephyr kiadásától függnek. A tesztek a `tests/` könyvtárból a tényleges `../src/sensor_filter.c` és `../src/valve.c` forrásokat fordítják, nem külön, egyszerűsített modellt. A `valve` tesztek `hw_present = false` mellett futnak, így nem nyitnak valódi GPIO-t; a visszajelzés (feedback) ágát és a hardveres hibakezelést csak GPIO-emulációval vagy HIL-ben lehet lefedni.

8.3.5 Ellenőrzés és hibakeresés

A tesztek sikeres futtatása után a kernel `PROJECT EXECUTION SUCCESSFUL` sorral ér véget. Ha egy assert meghiúsul, a Ztest kiírja a fájlt, a sort és a feltételt. A Twister jelentése `twister.json` formában megőrzi az eredményeket, ami CI-ben is felhasználható.

Hibatünet	Lehetséges ok	Javítás
A Twister nulla tesztet futtat	A `testcase.yaml` hiányzik, rossz helyen van, vagy a `platform_allow` nem tartalmazza a `native_sim`-et	Tedd a `testcase.yaml`-t a `tests/` könyvtárba, és a `west twister -T tests -p native_sim` hívást használd
Duplikált szimbólum linkeléskor	Ugyanaz a `ZTEST_SUITE` több fájlban szerepel	Suite-onként egyszer regisztrálj; hagyd el a központi `test_main.c`-t
Assert hiba az EMA első mintájánál	Az `ema_update` nulláról indul, és nem az első mintát tekinti kezdőértéknek	Ellenőrizd az `initialized` flag kezelését a Mintavétel, kalibráció és szűrés `ema_update` függvényében
Az ismételt OPEN meghosszabbítja a határidőt	A `valve_set_target` az idempotencia-ellenőrzés előtt számol új határidőt	Azonos kérésre a határidő módosítása nélkül térj vissza
A `valve` teszt valódi GPIO-t próbál elérni	A teszt `hw_present = true` értékkel inicializál	Használj `false` értéket a szimulált szelephez

8.3.6 Tipikus hibák és félreértések

1. **Globális állapot a tesztek között.** Ha a teszt nem inicializálja a szűrőt vagy a szelepvezérlőt minden teszt előtt, a sorrendtől függő eredményt kaphat. Minden teszt elején hívjuk az `ema_init`, `ma_init` vagy `valve_init` függvényt.

2. **Hardverhívás a unit tesztben.** A `valve` modul csak `hw_present = true` esetén ér el GPIO-t; a tesztben ezért `false` értékkel kell inicializálni. A `sensor_filter` tiszta függvényekből áll, így közvetlenül hívható. A visszajelzés (feedback) ágához GPIO-emuláció vagy HIL szükséges.

3. **A `native_sim` időzítés valós hardverként kezelése.** A `native_sim` a Zephyr kernel szimulált időalapját futtatja, de ez nem reprezentálja a valós órajel- vagy periféria-időzítéseket. Az időzítésre épülő logikát hoston csak logikai szinten tesztelhetjük; a `k_sleep` a szimulált időt lépteti, nem valódi hardveres késleltetést mér.

4. **A teszt a saját egyszerűsített modelljét teszteli.** Ha a CMake nem a `../src/sensor_filter.c` és `../src/valve.c` tényleges forrásait fordítja, a zöld tesztek nem igazolnak semmit a továbbvitt firmware-ről. Ellenőrizd a CMake forráslistáját, és tartsd meg a korábbi hibákra visszatekintő regressziós teszteket.

8.3.7 Feladatok

a) **Fogalmi kérdés:** Miért érdemes a `sensor_filter` és a `valve` modulokat hoston, Ztesttel tesztelni, és mit nem bizonyítanak ezek a tesztek a valós hardveren?

b) **Tervezési feladat:** Tervezze meg a `valve` modul azon tesztjeit, amelyek az ismételt OPEN parancsot és a maximális nyitvatartási határidőt ellenőrzik. Adja meg a tesztlépéseket, a vizsgált mezőket és azt, hogy mikor kell `hw_present = false` értéket használni.

c) **Implementációs feladat:** Írjon Ztest tesztet a `sensor_filter` EMA első mintájára, amely 3000-es bemenetre 3000-et vár, és mutassa meg, hogy a teszt a nulláról induló implementáción elbukik. A tesztet a `tests/test_sensor_filter.c` fájlba helyezze.

d) **Hibakeresési feladat:** Az ismételt OPEN parancsot ellenőrző teszt azt várja, hogy a `on_deadline_ms` érték ne változzon, a teszt azonban azt tapasztalja, hogy a határidő meghosszabbodik. Keresse meg a hibát a `valve_set_target` függvényben, javítsa ki, és indokolja a javítást.

8.3.8 Megoldások

a) A host-teszt gyors, determinisztikus és hardver nélkül futtatható. A `sensor_filter` tiszta számítás, a `valve` pedig `hw_present = false` mellett szimulált szelepként működik, így a biztonsági logika (idempotencia, maximális nyitvatartás) jól izolálható. A tesztek nem bizonyítják a valós GPIO-időzítést, a visszajelzés elektromos viselkedését, a szenzor kalibrációját vagy a HIL-környezetet; ezekhez hardveres vagy emulált perifériás teszt kell.

b) A tesztlépések: inicializálás `valve_init(&v, 60000, false)` hívással; első `valve_set_target(&v, true)`; a `v.on_deadline_ms` elmentése; rövid `k_sleep`; ismételt OPEN; annak ellenőrzése, hogy a `v.on_deadline_ms` változatlan, a `v.requested_open` igaz, és a `valve_is_open(&v)` igaz. A maximális nyitvatartáshoz 1 ms korláttal inicializálunk, OPEN után `k_sleep`-pel túllépjük a határidőt, majd `valve_tick(&v)` után ellenőrizzük, hogy a kérés és a jelentett állapot is zárult. A visszajelzés ágához `hw_present = false` nem elegendő; ahhoz GPIO-emuláció vagy HIL kell.

c) A teszt kiegészítése:

```
ZTEST(sensor_filter, test_ema_first_sample)
{
    struct ema_filter f;

    ema_init(&f, 4);
    zassert_equal(ema_update(&f, 3000), 3000,
                  "az első minta legyen a kezdőérték");
}
```

A nulláról induló implementáció az első frissítésnél `0 + (3000 - 0) / 4 = 750` értéket adna, ezért a `zassert_equal` elbukna. A helyes Mintavétel, kalibráció és szűrés-implementáció az `initialized` flaggel az első mintát közvetlenül kezdőértékké teszi.

d) A hiba akkor fordul elő, ha a `valve_set_target` az idempotencia-ellenőrzés előtt frissíti a határidőt:

```
/* Hibás sorrend: minden OPEN új határidőt számol. */
v->on_deadline_ms = k_uptime_get() + v->max_on_time_ms;
if (v->requested_open == open) {
    return 0;
}
```

A helyes sorrend az azonos kérés korai visszaadása, a határidő módosítása nélkül:

```
if (v->requested_open == open) {
    /* Idempotens: nem vált és nem hosszabbítja meg a határidőt. */
    return 0;
}
```

Az Aktuátorok: biztonságos vezérlés és visszajelzés modul már ezt a helyes sorrendet tartalmazza; a regressziós teszt feladata, hogy a hiba visszacsúszását megakadályozza.

8.3.9 Rövid összefoglaló

Az alfejezetben a `ts-node` firmware hardverfüggetlen logikáját tettük hoston tesztelhetővé Zephyr Ztest és `native_sim` használatával. Nem új modellt írtunk, hanem a ténylegesen továbbvitt `sensor_filter.c` (Mintavétel, kalibráció és szűrés) és `valve.c` (Aktuátorok: biztonságos vezérlés és visszajelzés) modulokat teszteltük. A `sensor_filter` tiszta számításai közvetlenül hívhatók; a `valve` `hw_present = false` mellett szimulált szelepként működik, így a biztonsági logika GPIO nélkül vizsgálható. Létrehoztuk a `tests/test_sensor_filter.c` és `tests/test_valve.c` teszteket, a `tests/prj.conf`, `tests/testcase.yaml` és `tests/CMakeLists.txt` fájlokat, valamint a korábbi hibákra visszatekintő regressziós teszteket: az EMA első mintáját és az ismételt OPEN határidő-módosítását. A tesztek a `tests/` könyvtárból `west build -b native_sim .` paranccsal fordulnak, a Twister pedig `west twister -T tests -p native_sim` hívással fedezi fel őket; a várt, illusztratív kimenetben öt teszt kap PASS jelölést. **Továbbvitt eredmény:** ezek a host-tesztek a CI/CD a firmware-től a backendig alfejezetben automatikusan futnak a firmware-feladat részeként.

8.3.10 Források és ellenőrzési hiányok

- A Zephyr 3.7.0 LTS Ztest- és Twister-dokumentációja szerint a `ZTEST_SUITE` suite-onként egyszer, a tesztfájlban szerepel; a tesztprojektet a `testcase.yaml` és a `prj.conf` írja le, a platformot a Twister `-p` kapcsolóval választja, a `harness: ztest` pedig a Ztest kimenetét értelmezi.

- A fejezet teszthívásait és a várt kimenetet statikusan ellenőriztük; a laborban nem futtattunk Zephyr buildet vagy Twistert, ezért a kimenet illusztratív, és a pontos sorok a telepített Zephyr-kiadástól függnek.
- A tesztelt modulok a Mintavétel, kalibráció és szűrés `sensor_filter.c` és az Aktuátorok: biztonságos vezérlés és visszajelzés `valve.c`; a `valve` visszajelzés (feedback) ágát és a valós GPIO-viselkedést ez a host-teszt nem fedi le, ahhoz GPIO-emuláció vagy HIL szükséges.
- A `valve.c` a Zephyr `k_uptime_get` és logging API-ját használja, ezért a teszt `native_sim` boardon fut; a `sensor_filter.c` hardverfüggetlen.

III. RÉSZ – Kommunikáció: rádiók, hálózatok, protokollok, adatformátumok

9 Rádiós technológiák és hálózati topológiák

9.1 Rádiós technológiák összevetése: hatótáv, energia, sávszélesség, költség

9.1.1 Motiváció

A TerraSense csomópont tervezésekor el kell dönteni, hogyan jut el a szenzoradat a gatewayhez. A követelménytáblázat szerint a `ts-node` 10 percenként küld legfeljebb 200 B CBOR-telemetriát, legalább 12 hónapig kell elemből működnie, és a csomópontok egy üvegházon belül, tipikusan 10–50 m-re helyezkednek el a `ts-gw`-től. Emellett a csomópontnak helyben kell észlelnie a kritikus riasztási feltételeket, ezért az idő kritikus adatoknál alacsony késleltetéssel kell továbbítania. Ha rossz rádiós technológiát választunk, az elem hetek alatt lemerül, vagy a gateway nem kapja meg az üzeneteket. Ezért ebben az alfejezetben a leggyakoribb IoT-rádiós technológiákat vetjük össze, és kialakítunk egy ismételhető választási módszert.

9.1.2 Fogalmak és működés

Az IoT-rádiós technológiák két nagy csoportba sorolhatók a felhasznált spektrum alapján:

- **Licenc nélküli sávok (ISM-sávok):** ilyen a 2,4 GHz-es sáv (Wi-Fi, BLE, Zigbee/Thread) és a 868/915 MHz-es sáv (LoRa, egyes Zigbee-változatok). Ezekben bárki adhat, de a zavartűrés korlátozott, mert sok eszköz osztozik a spektrumon. Előnyük, hogy nincs licencdíj, és a telepítés gyors.
- **Licencelt sávok:** a mobilszolgáltatók spektruma, például az NB-IoT és az LTE-M esetében. Itt a hálózat koordinálja a hozzáférést, ezért kiszámíthatóbb a zavartűrés, de a hozzáférés SIM-kártyát és szolgáltatói díjat igényel.

A technológiaválasztást leggyakrabban három, egymással versengő paraméter feszíti: a hatótáv, az adatsebesség és az energiafogyasztás. Ezt nevezzük **hatótáv–adatsebesség–energia háromszögnek**.

Definíció (hatótáv–adatsebesség–energia háromszög). Egy rádiós technológia fizikai jellemzői közül a nagy hatótáv általában alacsonyabb adatsebességet vagy nagyobb adási energiát követel; a nagy adatsebesség általában nagyobb energiafelvételt és rövidebb hatótávot eredményez. Az IoT-eszközök tervezésekor ezt a három dimenziót együtt kell optimalizálni a költség és a rendelkezésre álló spektrum figyelembevételével.

A TerraSense-eszköz energiamodelljében az adási idő közvetlenül befolyásolja az átlagáramot, hiszen az adási állapot árama általában 10–100-szorosa az alvási állapotének. Minél nagyobb az adatsebesség, annál rövidebb az adási idő, de a nagyobb sebességű rádiók gyakran magasabb csúcsáramot és bonyolultabb modulációt használnak, ami önmagában növeli a fogyasztást. Ezért az optimális választás nem mindig a leggyorsabb technológia.

Az alábbi táblázat a leggyakoribb IoT-rádiós technológiákat hasonlítja össze nagyságrendi értékekkel. Az adatok tipikus értékek, és a konkrét implementáció, környezet és szabályozási tartomány

szerint jelentősen eltérhetnek; a konkrét értékeket mindig a kiválasztott chip adatlapjából kell ellenőrizni.

Technológia	Sáv	Adatsebesség	Hatótáv	Energiaigény	Topológia	Költség
Wi-Fi (802.11n/ac)	2,4/5 GHz	10–100 Mbit/s	30–100 m	nagyon magas	csillag	közepes
BLE (Bluetooth 5)	2,4 GHz	125–2000 kbit/s	10–100 m	alacsony	csillag, hirdetés	alacsony
Zigbee	2,4 GHz / 868 MHz	20–250 kbit/s	10–100 m	alacsony	mesh	alacsony
Thread (802.15.4)	2,4 GHz	250 kbit/s	10–100 m	alacsony	mesh, IPv6	alacsony
LoRaWAN	868/915 MHz	0,3–50 kbit/s	1–15 km	nagyon alacsony	csillag (gateway)	alacsony
NB-IoT / LTE-M	licencelt	20–1000 kbit/s	1–10 km (cellás lefedettség)	alacsony	csillag (bázisállomás)	közepes, havidíj

A táblázatból jól látszik a háromszög: - A Wi-Fi nagy adatsebességet ad, de az energiaigénye miatt elemes IoT-csomóponthoz ritkán alkalmas. - A BLE és a Thread/Zigbee hasonló sávban, hasonló hatótávval működik, de a BLE egyszerűbb csillagtopológiát, a Thread/Zigbee mesh-hálózatot ad. - A LoRaWAN nagy hatótávú, de adatsebessége erősen korlátozott, ezért kis üzenetek ritka küldésére való. - Az NB-IoT és az LTE-M mobilszolgáltatói hálózaton működik, ami nagy lefedettséget ad, de a SIM-kezelés és a szolgáltatói költségek miatt nagyobb flottánál jelentős tétel.

9.1.3 Döntési szempontok

A megfelelő technológiát a követelményekből kiindulva, négy fő kérdéssel választjuk ki:

1. **Mekkora a szükséges hatótáv és a telepítési környezet?** Ha a csomópontok és a gateway ugyanabban az épületben vagy közvetlenül egymás mellett vannak, a rövid hatótávú technológiák elegendők. Ha több kilométeres, nehezen elérhető területet kell lefedni, LoRaWAN vagy cellás technológia kell.
2. **Mennyi az adatmennyiség és a küldési gyakoriság?** A ritka, kis üzenetek (például 10 percenként 200 B) jól illenek a keskenysávú, alacsony sebességű technológiákhoz. A nagyobb adatmennyiség (például firmware-frissítés, képátvitel) nagyobb sávszélességet igényel, ami általában nagyobb energiafogyasztással jár.
3. **Mennyire szigorú az energia- és költségkorlát?** Elemes eszköznél a csúcsáram, az adási idő és az alvó üzemmód energiaigénye dominál. Hálózatról táplált eszköznél a fogyasztás kevésbé fontos, a sávszélesség és a késleltetés előbbre sorolható.
4. **Milyen topológiát és hálózatkezelést tudunk vállalni?** A csillagtopológia egyszerűbb, de a gateway hatótávolsága korlátozott. A mesh-hálózat nagyobb területet fed le, de több eszközön kell fenntartani az útvonalakat, ami növeli a firmware-bonyolultságot és az energiafogyasztást.

Ezen kívül figyelembe kell venni a szabályozási és tanúsítási követelményeket (például az EU-ban a rádiós eszközök RED-engedélyezése) és azt is, hogy a licencelt sávos technológiák hálózati lefedettsége földrajzilag korlátozott lehet.

9.1.4 Kidolgozott példa: LoRa idő a levegőben és energiaigény

A `ts-node` 200 B hasznos terhet (payload) küld CBOR formátumban. LoRaWAN-on a PHY payload a LoRaWAN fejléccel együtt utazik: MHDR 1 B, FHDR 7 B (DevAddr 4 B, FCtrl 1 B, FCnt 2 B), FPort 1 B és MIC 4 B, összesen 13 B, ha nincs FOpts. A 200 B alkalmazásadatból így $PL = 213\,\mathrm{B}$ PHY payload lesz.

A LoRa adási idejét nem az L/R hányados adja meg, mert a PHY bitráta mellett a preamble, a header, a CRC és a kódolás is időt vesz igénybe, a payload pedig egész számú szimbólumban utazik. A LoRa Alliance képleteivel a szimbólumidő:

$$T_{\mathrm{sym}} = \frac{2^{SF}}{BW},$$

a preamble ideje N_{pre} preamble-szimbólummal:

$$T_{\mathrm{pre}} = (N_{\mathrm{pre}} + 4{,}25)\,T_{\mathrm{sym}},$$

a payload szimbólumszáma pedig:

$$n_{\mathrm{pay}} = 8 + \left\lceil \frac{8PL - 4SF + 28 + 16CRC - 20IH}{4(SF - 2DE)} \right\rceil (CR + 4).$$

Itt $CRC = 1$ a bekapcsolt keretellenőrzés, $IH = 0$ az explicit header, $DE = 1$ az alacsony adatsebesség optimalizálása (SF11/SF12 mellett 125 kHz-en), $CR = 1$ pedig a 4/5 kódolási rátát jelöli. A teljes idő:

$$T_{\mathrm{ToA}} = T_{\mathrm{pre}} + n_{\mathrm{pay}} T_{\mathrm{sym}}.$$

SF7, BW = 125 kHz, CR 4/5, 8 szimbólum preambulum, CRC engedélyezve, explicit header és $DE = 0$ mellett:

$$n_{\mathrm{pay}} = 8 + \left\lceil \frac{1704 - 28 + 28 + 16}{28} \right\rceil \cdot 5 = 8 + 62 \cdot 5 = 318,$$

$$T_{\mathrm{sym}} = \frac{128}{125\,000} = 1{,}024\,\mathrm{ms},$$

$$T_{\mathrm{ToA}} = 12{,}25 \cdot 1{,}024 + 318 \cdot 1{,}024 \approx 338\,\mathrm{ms}.$$

Ugyanez SF12-vel, ahol $DE = 1$:

$$n_{\mathrm{pay}} = 8 + \left\lceil \frac{1704 - 48 + 28 + 16}{40} \right\rceil \cdot 5 = 8 + 43 \cdot 5 = 223,$$

$$T_{\text{sym}} = \frac{4096}{125\,000} = 32{,}768\,\text{ms},$$

$$T_{\text{ToA}} = 12{,}25 \cdot 32{,}768 + 223 \cdot 32{,}768 \approx 7{,}71\,\text{s}.$$

A két érték jól mutatja, hogy a nagyobb szórási tényező nagyobb hatótávot ad, de az adási idő több mint hússzorosára nő. Az L/R hányados csak ideális alsó becslés: a PHY bitráta például SF7-nél $7 \cdot 125\,000/2^7 \cdot 4/5 \approx 5{,}47\,\text{kbit/s}$, de a tényleges ToA a fenti szimbólumszámításból jön.

A BLE-összehasonlításban ugyanaz a 200 B hasznos adat GATT-notificationben utazik. A 200 B mellé 3 B ATT-notification fejléc, 4 B L2CAP fejléc, 2 B LL fejléc, titkosításnál 4 B MIC, 3 B CRC, valamint 2 B preamble és 4 B access address jön, ami összesen 222 B a levegőben. 2 Mbit/s-os PHY-n ez

$$t_{\text{air}} = \frac{222 \cdot 8}{2\,000\,000} = 0{,}888\,\text{ms},$$

amelyhez a nyugtázó csomag és a keretek közötti 150 mikroszekundumos várakozás miatt körülbelül 1,1 ms adási idő tartozik. 1 Mbit/s-on ugyanez körülbelül 2,0 ms. A számítás egy notificationt és annak nyugtázását egy connection eventben feltételezi; a kapcsolat felépítésének és a connection intervalnak az energiaigénye ettől külön számítandó. Data Length Extension nélkül a 207 B-os LL payload nem fér bele egy 27 B-os legacy LL PDU-ba, ezért töredezni kell; a fenti érték a DLE-t és a legalább 203 B-os ATT MTU-t feltételezi.

Technológia	Hasznos adat	Protokollfej-léc	PHY payload/keret	PHY-sebesség	Adási idő
LoRaWAN SF7, BW 125 kHz, CR 4/5	200 B	13 B LoRaWAN	213 B PHY payload	kb. 5,47 kbit/s effektív	kb. 338 ms
LoRaWAN SF12, BW 125 kHz, CR 4/5	200 B	13 B LoRaWAN	213 B PHY payload	kb. 0,293 kbit/s effektív	kb. 7,71 s
BLE 5, 2M PHY, GATT notification	200 B	3 B ATT + 4 B L2CAP + 2 B LL + 4 B MIC + 3 B CRC + 6 B preamble/AA	222 B a levegőben	2 Mbit/s	kb. 1,1 ms
BLE 5, 1M PHY, GATT notification	200 B	ugyanaz	222 B a levegőben	1 Mbit/s	kb. 2,0 ms

A táblázatban a hasznos adat és a teljes keret külön oszlopban szerepel. A BLE legacy hirdetés 31 B-os korlátja miatt a 200 B nem fér bele; a 200 B-os telemetria GATT-kapcsolatot és legalább 203 B-os ATT MTU-t igényel (a részleteket a BLE alfejezet tárgyalja). Az energiafogyasztás az adási állapot áramából és az adási időből számolható:

$$E_{\text{tx}} = U \cdot I_{\text{tx}} \cdot t_{\text{tx}}.$$

LoRaWAN SF7 esetén $I_{\text{tx}} = 90\,\text{mA}$, $U = 3{,}3\,\text{V}$ mellett:

$$E_{\text{tx}} = 3{,}3 \cdot 0{,}09 \cdot 0{,}338 \approx 0{,}100\,\text{J}.$$

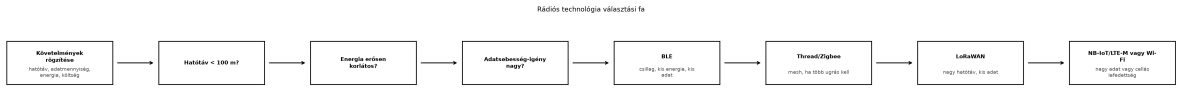
SF12 esetén ugyanez $3{,}3 \cdot 0{,}09 \cdot 7{,}71 \approx 2{,}29\,\text{J}$. Ha a `ts-node` 10 percenként ($T = 600\,\text{s}$) küld, az adás átlagáram-hozzájárulása SF7-nél:

$$I_{\text{avg,tx}} = \frac{0{,}09 \cdot 0{,}338}{600} \approx 0{,}0507\,\text{mA},$$

SF12-nél pedig körülbelül 1,16 mA. Ez a számítás illeszkedik az Energiamodell és az elemélettartam kiszámítása alfejezetben bemutatott duty-cycle modellhez: a rádiós állapot árama és időtartama az egyik összetevője az átlagáramnak. Az illusztratív értékekből is látszik, hogy a ritka küldésnél az alvó állapot dominál, ezért a technológiaválasztásnál nem csak az adási energia, hanem az alvó üzemmód és a felébredés költsége is döntő.

9.1.5 Döntési fa

A választási módszert az alábbi folyamatábra foglalja össze. A folyamatot a TerraSense kontextusában is végigvezetjük, de a logika általános.



9.1. ábra: Rádiós technológia választási fa - A TerraSense követelményeivel a BLE-re vezet; nagy hatótávú kiegészítésnél LoRaWAN.

A fa használata:

1. Először a hatótávot vizsgáljuk. Ha az eszközök egy épületen belül, 100 m-en belül vannak, akkor a rövid hatótávú technológiák jönnek szóba; ha ennél nagyobb területet kell lefedni, akkor LoRaWAN vagy cellás technológia szükséges.
2. Ha a hatótáv rövid, akkor az energia korlátosságát nézzük. Elemes eszközöknél a BLE vagy a Thread/Zigbee jöhet szóba; hálózatról táplált eszköznél a Wi-Fi is választható.
3. Rövid hatótáv és szigorú energiakorlát esetén a mesh-topológia szükségessége dönt: ha egyetlen gateway lefedi a területet, a BLE csillagtopológia egyszerűbb és energiatakarékosabb; ha több ugrás kell, a Thread vagy a Zigbee mesh-hálózatot ad.
4. Nagy hatótáv esetén az adatmennyiség és a lefedettség dönt: kis, ritka üzenetekhez LoRaWAN illik; nagyobb adatmennyiséghez vagy meglévő cellás lefedettséghez NB-IoT/LTE-M.

9.1.6 TerraSense választása és indoklása

A TerraSense-ben a csomópontok és a gateway közötti helyi szakaszra a **BLE** jó választás, de nem a hirdetési mód, hanem a **GATT-kapcsolat** miatt. A 200 B-os telemetria nem fér bele egy legacy hirdetési csomagba (31 B), ezért a `ts-node` perifériaként viselkedik, a `ts-gw` pedig központi (central) szerepben kapcsolódik hozzá; a telemetria GATT-notificationben, a parancsok GATT-write-ban utaznak.

- **Hatótáv:** az üvegházban a csomópontok és a `ts-gw` jellemzően 10–50 m-re vannak, amit a BLE 5 kapcsolati üzemmódja kényelmesen lefed.
- **Energia:** a `ts-node` elemes, és a 10 perces ciklusú telemetria kis adatmennyisége miatt a BLE alacsony csúcsárama és rövid adási ideje kedvező. A kapcsolat rövid ideig tart, vagy hosszú connection interval mellett fut, így a node a küldések között alhat.
- **Adatmennyiség és keret:** a 200 B alkalmazásadat ATT-notificationben 203 B ATT PDU-t, 207 B L2CAP/LL payloadot ad. Ezért az ATT MTU-t legalább 203 B-ra kell emelni (az alapértelmezett 23 B nem elég), és a Data Length Extension legyen engedélyezve (LL payload legfeljebb 251 B). Ha a gateway MTU-ja kisebb, az alkalmazásnak töredeznie kell.
- **Parancs-visszaút:** a `ts-gw` a parancsot egy GATT command characteristicre írja (write with response), a node a végrehajtás eredményét egy result characteristic notificationjében küldi vissza. Így a telemetria és a parancs ugyanabban a kapcsolatban, dokumentált szolgáltatásban fér el.
- **Összetettség:** a csillagtopológia egyszerű eszközszoftvert tesz lehetővé, és a Zephyr BLE-támogatása kiforrott. A mesh-topológia bonyolultsága itt nem indokolt, mert az üvegház mérete nem igényel több ugrást.

A hirdetés szerepe csak a kapcsolatfelvétel: a node connectable advertisinggel jelzi a szolgáltatását. A legacy hirdetés 31 B-jába a telemetria nem fér bele; az extended advertising (BLE 5) nagyobb payloadot tud, de önmagában nem ad kényelmes, nyugtázott parancsvisszautat, ezért a TerraSense nem azt választja. A GATT-szolgáltatás, az MTU-egyeztetés és a Zephyr konfiguráció részletei a Bluetooth Low Energy: hirdetés, GATT, kapcsolatok alfejezetben készülnek el.

A döntési fában ez az „energia erősen korlátos” és „mesh nem szükséges” ágon a BLE-t jelenti.

Opcionálisan, ha a TerraSense-t több, egymástól távoli üvegházból álló telepre kellene kiterjeszteni, egy **LoRaWAN** gateway is bevezethető lenne. Ebben az esetben a csomópontok egy része LoRaWAN-rádióval küldene a telepi gatewayhez, a BLE pedig a helyi, rövid hatótávú kommunikációra maradna. Ezt a lehetőséget a LoRaWAN: nagy hatótáv kis energiával alfejezet vizsgálja.

Továbbvitt eredmény. A fenti döntési fa és a négy fő szempont (hatótáv, adatmennyiség, energiakorlát, topológia) a TerraSense választási módszere. Ezt a módszert használja a Bluetooth Low Energy: hirdetés, GATT, kapcsolatok, a LoRaWAN: nagy hatótáv kis energiával és az IP az eszközön: IPv6, 6LoWPAN, Thread és a Zephyr hálózati stack alfejezet, amikor az egyes technológiák részleteit dolgozza ki. A TerraSense fő csomópont–gateway szakaszára a BLE-t választottuk; a konkrét BLE GATT-szolgáltatás, az MTU-egyeztetés és a parancsvisszaút a Bluetooth Low Energy: hirdetés, GATT, kapcsolatok alfejezetben készül el.

9.1.7 Ellenőrzés és hibakeresés

A rádiós technológia kiválasztása után az igazi ellenőrzés a valós környezetben végzett hatótáv- és energiafogyasztás-mérés. A `native_sim` rádiós rétege nem reprezentálja a valódi rádiós

viselkedést, ezért a méréseket mindig a kiválasztott fejlesztőkártyán, a tervezett antennával kell elvégezni.

Tünet	Lehetséges ok	Javítás
A gateway nem kapja az üzeneteket	Rossz technológiaválasztás, hatótávon kívüli csomópont	Hatótáv-ellenőrzés, nyereségesebb antenna, mesh vagy más technológia
Az elem hetek alatt lemerül	Túl gyakori adás, nagy csúcsáramú rádió, sok újraküldés	Csökkenteni a küldési gyakoriságot és az újraküldések számát; ha a link megengedi, nagyobb PHY-sebességet használni, mert az azonos adatmennyiségnél rövidebb adási időt ad; optimalizálni az alvó üzemmódot és a connection intervalt
Nagy késleltetés a riasztásnál	A technológia nagy átviteli késleltetést okoz	BLE vagy Thread a kritikus útra, az alvó ciklus lerövidítése
Zavart a 2,4 GHz-es sávban	Wi-Fi, BLE, Zigbee együtt fut	Csatornaválasztás, frekvenciaugrás, LoRaWAN 868 MHz-en

A mérési jegyzőkönyvben rögzíteni kell a környezetet (falak, távolság, interferencia), a rádió beállításait (teljesítmény, adatsebesség, csatorna) és a mért RSSI vagy csomagvesztési arányt. Ez alapján lehet a hatótávra és az energiafogyasztásra vonatkozó követelményeket igazolni.

9.1.8 Tipikus hibák és félreértések

1. **„A nagyobb adatsebesség mindig energiatakarékosabb."** Bár a nagyobb sebesség rövidebb adási időt ad, a nagyobb sebességű rádiómoduláció gyakran magasabb csúcsáramot és bonyolultabb jelfeldolgozást igényel, ami az összenergiát növelheti. A teljes energiamérleget a chip adatlapjából kell kiszámítani.
2. **„A licenc nélküli sáv mindig olcsóbb."** A licenc nélküli sávban nincs spektrumdíj, de a zavartűrés miatt több újraküldésre lehet szükség, ami növeli az energiafogyasztást és a késleltetést. A licencelt sáv havidíja egy prediktív, garantáltabb szolgáltatást fedezhet.
3. **„A mesh-hálózat mindenre jobb."** A mesh megnöveli a lefedettséget, de több eszköznek kell ébren maradnia az útvonalak fenntartásához, ami növeli a teljes hálózat energiafogyasztását és a firmware-bonyolultságot. Csillagtopológiával is lefedhető területen a mesh felesleges.
4. **„A hatótáv adatlapértéke laboratóriumi körülményekre vonatkozik."** A valós beltéri hatótáv a falak, az emberek és az interferencia miatt a gyártói érték töredéke is lehet. A tervezéskor mindig valós környezetben kell mérni.

9.1.9 Feladatok

1. Fogalmi kérdés: Magyarázd el a hatótáv–adatsebesség–energia háromszöget egy konkrét példán. Miért nem lehet egyszerre nagy hatótáv, nagy adatsebesség és alacsony energiafogyasztás?
2. Számítási feladat: Egy LoRaWAN eszköz 20 B hasznos terhet küld SF7 szórási tényezővel, 125 kHz sávszélességgel, 4/5 kódolási rátával, 8 szimbólum preambulummal, engedélyezett CRC-vel és explicit headerrel. A LoRaWAN fejléc 13 B, így a PHY payload 33 B. Számítsd ki a payload szimbólumszámát és az adási időt a LoRa idő a levegőben képletével, majd az adási energiát, ha a rádió csúcsárama 120 mA 3,3 V-on. Mennyi az adás átlagáram-hozzájárulása, ha az eszköz 15 percenként küld?
3. Implementációs feladat a TerraSense-ben: A `ts-node` BLE GATT-kapcsolati és LoRaWAN opciójához készíts egy egyszerű döntési táblázatot a `~/iot-labor/i021/radio-selection.md` fájlban, amely a TerraSense követelményeihez (10 perc ciklus, 200 B, 12 hó elem, 50 m hatótáv) hozzárendeli a választott technológiát és az indoklást. A táblázat a későbbi laborok bemenetéül szolgál.
4. Hibakeresési feladat: Egy üvegházban a `ts-node`-ok csatlakoztatható BLE-hirdetéseit a `ts-gw` csak az esetek 30%-ában látja, és a kapcsolódási kísérletek gyakran elakadnak. Sorold fel a lehetséges okokat, és adj mindegyikhez javítási javaslatot. Milyen mérésekkel igazolnád a javítást?

9.1.10 Megoldások

1. A háromszög fizikai kompromisszumot fejez ki. A nagy hatótávhoz több adási energiát vagy érzékenyebb, kisebb adatsebességű modulációt kell használni. A nagy adatsebesség több spektrumot igényel, és a rádió jelfeldolgozó áramkörei nagyobb csúcsáramot vesznek fel. Az alacsony energiafogyasztás rövid adási időt vagy alacsony csúcsáramot kíván, ami korlátozza a hatótávot vagy a sebességet. Például a LoRaWAN nagy hatótávja alacsony adatsebességen valósul meg, míg a Wi-Fi nagy sebessége rövid hatótávon, hálózati tápellátással működik.
2. A PHY payload $PL = 20 + 13 = 33\,\mathrm{B}$, $SF = 7$, $BW = 125\,000\,\mathrm{Hz}$, $CR = 1$, $CRC = 1$, $IH = 0$ és $DE = 0$. A payload szimbólumszáma:

$$n_{\mathrm{pay}} = 8 + \left\lceil \frac{8 \cdot 33 - 4 \cdot 7 + 28 + 16}{4 \cdot 7} \right\rceil \cdot 5 = 8 + 10 \cdot 5 = 58,$$

a szimbólumidő $T_{\mathrm{sym}} = 2^7/125\,000 = 1{,}024\,\mathrm{ms}$, a preamble ideje $12{,}25 \cdot 1{,}024 \approx 12{,}54\,\mathrm{ms}$. Az adási idő:

$$T_{\mathrm{ToA}} = 12{,}54 + 58 \cdot 1{,}024 \approx 71{,}9\,\mathrm{ms}.$$

Energia:

$$E_{\mathrm{tx}} = 3{,}3 \cdot 0{,}12 \cdot 0{,}0719 \approx 0{,}0285\,\mathrm{J}.$$

A periódus $T = 15\,\mathrm{perc} = 900\,\mathrm{s}$, így az átlagáram-hozzájárulás:

$$I_{\mathrm{avg}} = \frac{0{,}12 \cdot 0{,}0719}{900} \approx 0{,}0096\,\mathrm{mA}.$$

Az L/R osztás itt azért adna más értéket, mert a PHY bitráta nem számol a preamble, a header, a CRC és az egész szimbólumok idejével; a LoRa adásidejét a fenti szimbólumszámítás adja.

3. A fájl tartalma például:

```
# TerraSense rádiós választás

| Követelmény | Érték | Döntés |
|---|---|---|
| Hatótáv | 50 m beltér, üvegház | BLE GATT-kapcsolat megfelelő |
| Adatmennyiség | 200 B / 10 perc | GATT-notification, ATT MTU >= 203 B |
| Energia | 12 hó elemes | BLE alacsony fogyasztás, rövid kapcsolat |
| Topológia | csillag | ts-gw central, ts-node periféria |
| Parancs | write with response | GATT command + result notification |
| Döntés | BLE GATT | lásd i023 |
| Opció | LoRaWAN | nagyobb telepi kiterjesztéshez |
```

4. Lehetséges okok:

- Hirdetési csatorna zsúfoltsága a 2,4 GHz-en (Wi-Fi, más BLE eszközök). Javítás: hirdetési intervallum és csatorna módosítása, frekvenciaugrás.
- Hatótáv vagy antenna-probléma: a csomópont túl messze van, vagy az antenna nem megfelelő. Javítás: RSSI-mérés, jobb antenna, áthelyezés.
- A hirdetések ütköznek, mert sok csomópont azonos intervallumban küld. Javítás: véletlenített hirdetési intervallum.
- A `ts-gw` rádiója vagy BLE-verziója nem megfelelően van konfigurálva, vagy a hardver hibás. Javítás: ellenőrzés másik eszközzel, a central szerep és a scan beállításainak ellenőrzése.
- A 200 B-os telemetriát nem a hirdetésben kell küldeni: a legacy hirdetés legfeljebb 31 B, ezért a node connectable advertisinggel csak a kapcsolatfelvételt jelzi, az adat GATT-notificationben megy. Javítás: GATT-kapcsolat és megfelelő ATT MTU, nem nagyobb hirdetési adat.

A javítást a kapcsolódási sikerarány, a kapcsolódási idő, valamint csomagvesztési arány (PER) és RSSI-méréssel lehet igazolni, például a gateway naplójában rögzített értékekkel.

9.1.11 Rövid összefoglaló

A rádiós technológiát a hatótáv, az adatsebesség és az energiafogyasztás háromszögében, a költségek és a topológia figyelembevételével választjuk ki. A licenc nélküli 2,4 GHz-es és 868/915 MHz-es sávok gyors telepítést tesznek lehetővé, a licencelt NB-IoT/LTE-M pedig szolgáltatói lefedettséget ad. A TerraSense csomópont–gateway szakaszára a BLE-t választottuk: a csomópont GATT-periféria, a gateway GATT-central, a 200 B-os telemetria notificationben, a parancs write with response üzenetben utazik. Ehhez legalább 203 B-os ATT MTU és Data Length Extension szükséges, mert a legacy hirdetés 31 B-os korlátja miatt az adat nem fér el a hirdetésben. A döntési fa és a négy fő szempont a `~/iot-labor/i021/radio-selection.md` fájlban összegzett módszer, amelyet a BLE, a LoRaWAN és a Thread alfejezetek használnak. A LoRa adásidejét a preamble, a payload-szimbólumok és a szimbólumidő képletével kell számítani; az L/R hányados csak ideális alsó becslés. A konkrét rádiós értékeket mindig a chip adatlapjából és valós mérésekből kell ellenőrizni.

9.1.12 Források és ellenőrzési hiányok

- A táblázatban szereplő adatsebesség, hatótáv és energiafogyasztási értékek tipikus nagyságrendek, amelyeket az adott rádiómodul és a szabályozási tartomány adatlapjából kell ellenőrizni.
- A LoRa adásidejének képlete a LoRa Alliance LoRaWAN-specifikációjából származik (preamble, payload-szimbólumszám, szimbólumidő); a pontos szórási tényező, sávszélesség és kódolási ráta értékek a régió és a csatornaterv szerint ellenőrizendők.
- A BLE fizikai adatsebességek, az ATT MTU, a Data Length Extension és a hirdetési payload korlátai a Bluetooth SIG specifikációin alapulnak; a konkrét chip támogatása és a hirdetési üzemmód energiafogyasztása ellenőrizendő.
- A számítások statikus ellenőrzéssel készültek, Zephyr build vagy rádiós mérés nem történt; a laborban a mért ToA, RSSI és energiafogyasztás alapján kell a modellt igazolni.
- Az NB-IoT/LTE-M lefedettség és költség szolgáltatófüggő, ezért a jelenlegi csomagok és díjak a helyi szolgáltatók adatlapjairól ellenőrizendők.

9.2 Topológiák: csillag, mesh, gateway-alapú hálózatok

9.2.1 Motiváció

A TerraSense üvegházában nyolc `ts-node` szenzorcsomópont méri a hőmérsékletet, páratartalmat és talajnedvességet, és egy `ts-gw` gyűjti az adatokat. A tervezés első kérdése, hogy minden csomópont közvetlenül a gatewayhez küldjön-e, vagy a csomópontok egymás üzeneteit továbbítva, több ugráson keresztül érjék el a gatewayt. A válasz nemcsak a rádiós lefedettséget és a költséget határozza meg, hanem azt is, hogy egy elem lemerülése vagy kiesése után mennyi idő alatt áll helyre a rendszer, és mekkora akkumulátorra van szükség az egyes csomópontokban. Ebben az alfejezetben a három alapvető topológiát, a csillagot, a fát és a mesh-hálót hasonlítjuk össze, különös tekintettel a továbbító csomópontok energiaköltségére.

9.2.2 Fogalmak és működés

Definíció. A **hálózati topológia** a csomópontok közötti logikai összeköttetések szerkezete: azt írja le, hogy melyik eszköz melyik másik eszközzel kommunikálhat közvetlenül, és milyen útvonalon jut el egy üzenet a forrástól a célállomásig.

Definíció. Csillag topológia esetén minden végpont (szenzorcsomópont) közvetlenül egy központi eszközhöz, tipikusan gatewayhez vagy bázisállomáshoz kapcsolódik. A végpontok egymással nem kommunikálnak közvetlenül; minden forgalom a központon halad át.

Definíció. Mesh topológia esetén a csomópontok több szomszéddal is közvetlen összeköttetésben állnak, és legalább egy részük továbbítja más csomópontok csomagjait. A topológia ezzel *lehetőséget* ad arra, hogy egy üzenet több útvonalon is eljusson a célhoz, de hogy egy összeköttetés kiesésekor tényleg van-e működő alternatív út, azt az útválasztási protokoll, a továbbítók elhelyezése és azok táplálása dönti el. A mesh topológia önmagában nem garantál hibatűrést.

Definíció. Fa topológia esetén a csomópontok hierarchikus, gyökeres fastruktúrába szerveződnek. A gyökér általában a gateway, alatta router csomópontok, alattuk pedig végpontok helyezkednek el. Egy adott pillanatban minden csomópontnak pontosan egy aktív szülője van, és az üzenet a gyökér felé vagy onnan lefelé halad. A szülő kiesésekor a fa csak akkor áll helyre,

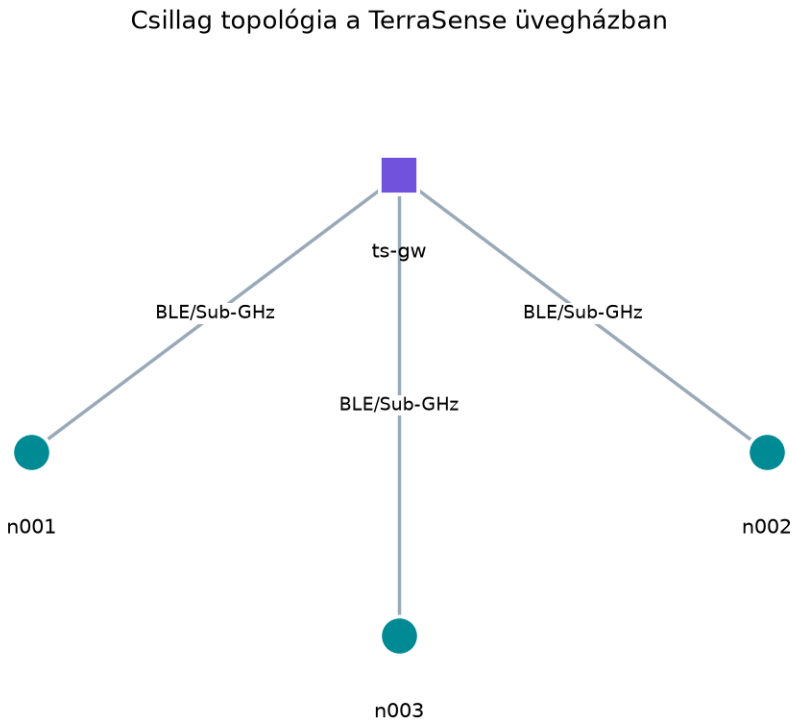
ha a protokoll támogatja az új szülő kiválasztását (például Thread vagy Zigbee esetén); a fix, statikusan konfigurált hierarchiában viszont az ág elveszik.

Definíció. Alvó végpont (sleepy end device) olyan csomópont, amely az idő nagy részében alacsony fogyasztású alvó állapotban van, csak saját mérési adatait küldi el, és nem továbbít más csomópontok forgalmát. Általában akkumulátorról működik.

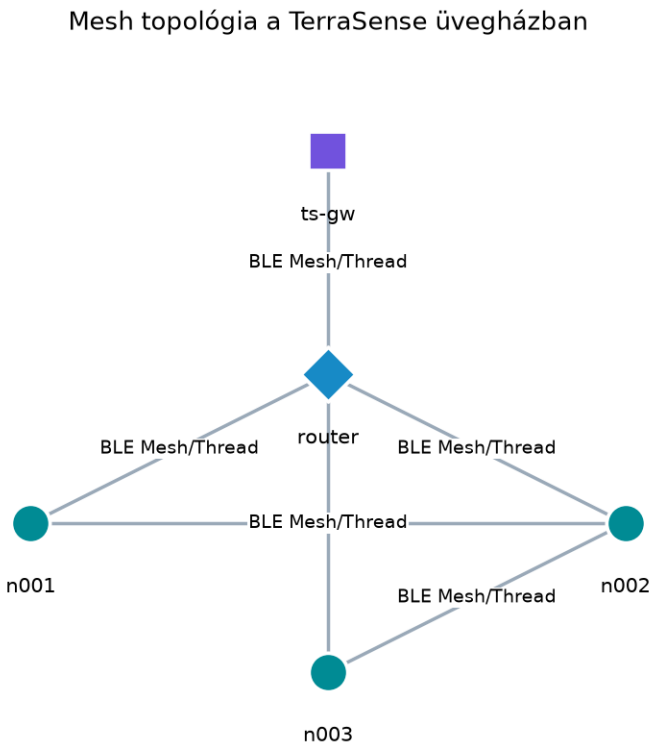
Definíció. Továbbító csomópont (router node) olyan mesh- vagy fa-hálózati csomópont, amely más csomópontok csomagjait is fogadja és továbbítja. Emiatt rádiójának hosszabb ideig aktívnak, vevő üzemmódban kell lennie, ezért energiafogyasztása nagyobb, mint egy alvó végponté.

Definíció. Gateway mindhárom topológiában az a csomópont, amely a helyi IoT-hálózatot egy másik hálózathoz, jellemzően IP-alapú felhőhöz vagy helyi szerverhez köti. Mesh-alapú IP-hálózatokban a gateway gyakran egyben **border router** is: a mesh-protokoll és az IP-hálózat közötti határon továbbít.

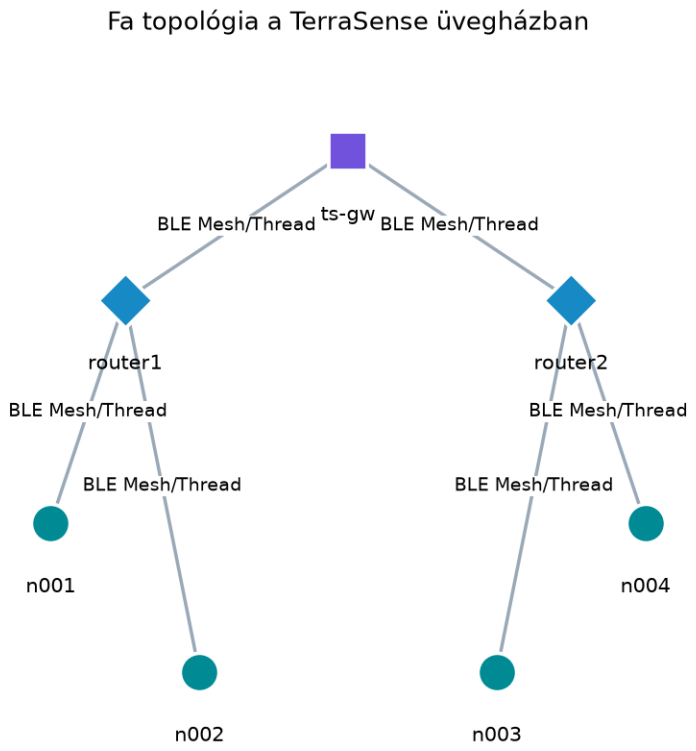
A három topológiát a következő ábrák szemléltetik.



9.2. ábra: Csillag topológia a TerraSense üvegházban - A ts-gw központi gateway, a három ts-node közvetlenül hozzá kapcsolódik; a node-ok nem kommunikálnak egymással. Illusztratív példa.



9.3. ábra: Mesh topológia a TerraSense üvegházban - A ts-gw mellett egy router csomópont továbbít; a topológia lehetőséget ad alternatív útra, de a tényleges hibatűréshez működő továbbítók és útválasztási protokoll kell. Illusztratív példa.



9.4. ábra: Fa topológia a TerraSense üvegházban - A gyökér a ts-gw, alatta két router, alattuk végpontok; minden végpontnak egy aktív szülője van, új szülő választása protokollfüggő. Illusztratív példa.

Az ábrák nélkül is érthetően: a csillagban a `ts-gw` a középpont, és minden `ts-node` pontosan egy rádiós összeköttetéssel éri el. A mesh-ben a `router` csomópont és a végpontok között több lehetséges út is van, így ha például az `n001` és a `router` közötti összeköttetés megszűnik, az `n001` az `n002` közbeiktatásával is elérheti a gatewayt – feltéve, hogy az `n002` továbbítóként működik, és az útválasztási protokoll az új utat ki is választja. A fa topológiában a két router hierarchiát alkot: az `n001` alapértelmezésben a `router1`-en keresztül kommunikál. Ha a `router1` kiesik, a viselkedés a protokolltól függ: új szülő választását támogató hálózatban (például Thread) az `n001` a `router2`-höz csatlakozhat újra, ami késleltetéssel jár; statikus szülő-beállításnál viszont az alatta lévő végpontok elvesztik a kapcsolatot.

A csillag topológia jellemző példája a LoRaWAN, ahol a végpontok egy vagy több gatewayhez küldenek, de egymással nem kommunikálnak, valamint a Wi-Fi infrastruktúra mód, ahol az állomások egy hozzáférési ponthoz kapcsolódnak. A mesh topológiát a Thread, a BLE Mesh és a Zigbee használja. A fa topológia számos LPWAN-hálózatnál és egyes ipari szenzorhálózatoknál jelenik meg, ahol a fix hierarchia egyszerűbbé teszi az útvonalválasztást.

A továbbító és alvó csomópontok megkülönböztetése az energia- és késleltetés-tervezés központi kérdése. Egy alvó végpont csak a saját mintavételi periódusában ébred fel, elküldi az adatot, majd visszaalszik. Egy továbbító csomópontnak azonban folyamatosan vagy legalább gyakran vevő üzemmódban kell lennie, hogy fogadni tudja a gyermekcsomópontjai vagy mesh-szomszédai csomagjait. Késleltetés szempontjából ez előnyös, mert a továbbító azonnal fogad, míg egy alvó végpont csak a következő felébredési ablakban elérhető.

A hibatűrés és az öngyógyítás nem a mesh topológiából automatikusan következő garancia, hanem a topológia és az útválasztási protokoll együttes eredménye. A mesh-topológia csak *lehetőséget* ad alternatív utakra; a hibatűréshez legalább két, egymástól független, működő továbbító kell, megfelelő protokollal, amely a linkek minőségét rendszeresen méri, és egy link megszűnésekor új útvonalat választ. Ha a mesh-ben csak egyetlen router van, akkor annak kiesése éppen úgy elvágja a hozzá kapcsolódó végpontokat, mint egy fa szülőjének kiesése; a border router (a mesh és az IP-hálózat határa) kiesése pedig az egész mesh forgalmát a felhő felé megszakíthatja, még ha a mesh belül továbbra is működik. A hibatűrést ezért a routerek számával, elhelyezésével, táplálásával és az útválasztási protokoll beállításaival kell megtervezni, és a kiesési forgatókönyveket külön tesztelni. A fa topológiában az újraszerveződés protokollfüggő: a Thread és a Zigbee támogatja az új szülő választását, a statikusan konfigurált fa viszont nem. A csillag topológiában a hibatűrés fő eszköze a redundáns gateway vagy az, hogy a végpont több gatewayt is elérhet, ahogyan a LoRaWAN-ban egy adott földrajzi ponton több gateway is veheti ugyanazt az üzenetet.

A gateway minden topológiában az IoT-adatok kilépési pontja. Csillag esetén a gateway maga a központi gyűjtő; mesh- és fa-hálózatban a gateway gyakran border routerként is működik, és a mesh-protokoll címzését IP-címzésre fordítja. A gateway szerepét, biztonsági határként és pufferpontként betöltött funkcióját a A gateway szerepe: híd, puffer, biztonsági határ alfejezet részletezi.

9.2.3 Döntési szempontok

A megfelelő topológia kiválasztása a következő szempontok együttes mérlegelését igényli. A táblázat a három alaptopológia tipikus jellemzőit foglalja össze.

Szempont	Csillag	Fa	Mesh
Lefedettség	A gateway hatósugarától függ; nagyobb területen több gateway kell	A routerek meghosszabbítják a hatótávot, de fix hierarchiában	Potenciálisan a legnagyobb, de a tényleges lefedettséget a routerek elhelyezése és a működő továbbítás adja
Energia a végponton	Alacsony, alvó üzemmód jól tervezhető	Alacsony a levél csomóponton, magasabb a routeren	A routereken magas, mert továbbítaniuk és hallgatniuk kell
Hibatűrés	Gyenge, a gateway single point of failure; több gateway vagy redundancia javít	A protokolltól függ; új szülő választásával helyreállhat, statikus hierarchiában egy szülő kiesése levágja az ágat	A protokolltól, a redundanciától és a routerek táplálásától függ; alternatív út és működő továbbítók esetén erős, egyetlen router kiesése helyi kiesést okoz
Késleltetés	Alacsony, ha a végpont ébren van	Közepes, több ugrás miatt	Közepes vagy magas, az ugrásszámtól és az ébredési ablakoktól függően
Konfiguráció és komplexitás	Alacsony	Közepes	Magas, útvonalválasztást és címzést kell kezelni

Szempont	Csillag	Fa	Mesh
Tipikus technológiák	LoRaWAN, Wi-Fi, BLE kapcsolat	Egyes ipari rádiós rendszerek, LPWAN-átjárók	Thread, BLE Mesh, Zigbee

Csillagot akkor érdemes választani, ha a csomópontok száma kicsi, a telepítési terület a gateway közvetlen rádiós hatósugarán belül van, és a fő cél az alacsony energiafogyasztás. Mesh akkor indokolt, ha a terület nagy, árnyékolt vagy a megbízhatóság és az öngyógyítás elsődleges követelmény, és a választott protokoll támogatja az alternatív útvonalat, valamint a routerek táplálása biztosított. A fa topológia kompromisszum: jobb lefedettséget ad, mint a csillag, de kisebb komplexitással és energiaigénnyel, mint a teljes mesh.

9.2.4 Kidolgozott példa: TerraSense üvegház topológiája és energiaszámítás

Tegyük fel, hogy a TerraSense `gh1` üvegháza 15 m × 30 m alapterületű, és nyolc `ts-node` csomópontot, valamint egy `ts-gw` gatewayt telepítünk. A rádiós linkek tervezési dokumentációja szerint minden `ts-node` látja a `ts-gw`-t legalább olyan jelerősséggel, amely a megbízható kapcsolathoz szükséges. Ebben az esetben a csillag topológia a legjobb választás: a csomópontok közvetlenül a gatewayhez küldenek, a gateway pedig MQTT-be fordítja az adatokat. Ez a döntés a A gateway szerepe alfejezetben később meghatározza a `ts-gw` BLE-központi szerepét.

Ha azonban a telepítés során kiderül, hogy a fém állványzat vagy a növényzet erős csillapítást okoz, és legalább két csomópont nem éri el megbízhatóan a gatewayt, akkor a csillag topológia önmagában nem elég. A BLE GATT-kapcsolat vagy a LoRaWAN alaphálózat nem válik több ugrásossá attól, hogy újabb csomópontot telepítünk; az adott technológiához illeszkedő megoldás kell: második gateway vagy hozzáférési pont, vagy pedig a technológia lecserélése több ugrást támogató hálózatra (BLE Mesh, Thread, Zigbee). Thread-alapú mesh esetén például két router csomópontot jelölünk ki, amelyek a többi végpont forgalmát továbbítják. A döntés energiaoldalát egy átlagáram-számítással mutatjuk be.

Egy alacsony fogyasztású végpont esetében a következő illusztratív paramétereket tételezzük fel:

- periódusidő: $T_{\text{total}} = 60\,\text{s}$,
- saját adás ideje: $T_{\text{tx}} = 10\,\text{ms}$,
- saját vételi idő: $T_{\text{rx}} = 10\,\text{ms}$,
- adási áramfelvétel: $I_{\text{tx}} = 30\,\text{mA}$,
- vételi áramfelvétel: $I_{\text{rx}} = 10\,\text{mA}$,
- alvó áramfelvétel: $I_{\text{sleep}} = 5\,\mu\text{A}$.

Az átlagos áramfelvétel:

$$I_{\text{avg}} = \frac{T_{\text{tx}} I_{\text{tx}} + T_{\text{rx}} I_{\text{rx}} + T_{\text{sleep}} I_{\text{sleep}}}{T_{\text{total}}}$$

ahol $T_{\text{sleep}} = 60\,\text{s} - 0{,}01\,\text{s} - 0{,}01\,\text{s} = 59{,}98\,\text{s}$.

Behelyettesítve:

$$I_{\text{avg}} = \frac{0{,}01 \cdot 0{,}03 + 0{,}01 \cdot 0{,}01 + 59{,}98 \cdot 0{,}000005}{60}$$

$$I_{\text{avg}} = \frac{0{,}0003 + 0{,}0001 + 0{,}0002999}{60} = \frac{0{,}0006999}{60} \approx 0{,}0000117\,\text{A} = 11{,}7\,\mu\text{A}.$$

Egy mesh-router csomópont esetében a továbbítás miatt a rádió ideje egy részében vételi üzemmódban kell, hogy legyen. Tételezzük fel, hogy a router óránként 3 másodpercet – azaz az idő 5%-át – aktívan hallgatja a szomszédos csomópontok forgalmát, és saját adása továbbra is 10 ms. Ekkor:

- aktív vételi idő: $T_{\text{rx,router}} = 3\,\text{s}$,
- alvó idő: $T_{\text{sleep,router}} = 60\,\text{s} - 3\,\text{s} - 0{,}01\,\text{s} = 56{,}99\,\text{s}$.

Az átlagos áram:

$$I_{\text{avg,router}} = \frac{0{,}01 \cdot 0{,}03 + 3 \cdot 0{,}01 + 56{,}99 \cdot 0{,}000005}{60}$$

$$I_{\text{avg,router}} = \frac{0{,}0003 + 0{,}03 + 0{,}00028495}{60} \approx 0{,}0005097\,\text{A} \approx 0{,}51\,\text{mA}.$$

Ez nagyságrendileg 40-szerese az alvó végpont átlagáramának. A példa jól mutatja, hogy a mesh-ben a továbbító szerepű csomópontoknak nagyobb akkumulátorra vagy vezetékes táplálásra van szükségük. A BLE Mesh-hez kapcsolódó energiaköltségeket a Bluetooth Low Energy: hirdetés, GATT, kapcsolatok alfejezet használja fel, amikor a hirdetési és kapcsolati üzemmódokat számszerűsíti.

Továbbvitt eredmény. A TerraSense csillag topológiára vonatkozó döntése és az itt bemutatott energiaszámítási módszer a gateway szerepének megtervezéséhez szükséges kiindulópont; a `ts-gw` központi gyűjtőként való elhelyezését a A gateway szerepe: híd, puffer, biztonsági határ alfejezet használja. A mesh-router energiaköltség-számítást a Bluetooth Low Energy: hirdetés, GATT, kapcsolatok alfejezet viszi tovább a konkrét BLE-üzemmódok elemzésében.

9.2.5 Ellenőrzés és hibakeresés

Topológia tervezésekor a legegyszerűbb ellenőrzési módszer a rádiós jelerősség (Received Signal Strength Indication, RSSI) és a csomagvesztési arány (packet loss rate) mérése a telepítési helyszínen. Csillag topológia esetén minden csomópont pozíciójában meg kell győződni arról, hogy a gatewayhez stabil link van. Mesh esetén azt is ellenőrizni kell, hogy a kiválasztott routerek legalább két, egymástól eltérő útvonalat kínálnak-e.

Hibatünet	Lehetséges ok	Javítás
Egy csillag-végpont rendszeresen kiesik	A gateway árnyékolt vagy a távolság túl nagy	A gateway áthelyezése, külső antenna, vagy az adott technológiához illeszkedő második gateway/hozzáférési pont; relay vagy mesh csak akkor, ha a protokoll támogatja (Thread, BLE Mesh, Zigbee). LoRaWAN vagy BLE GATT csillag esetén a végpont ettől nem válik több ugrásossá

Hibatünet	Lehetséges ok	Javítás
A mesh-router akkumulátora lényegesen gyorsabban lemerül	A router folyamatosan aktív, mert túl sok forgalmat továbbít	Routerek számának növelése, az aktív hallgatási idő csökkentése, vagy vezetékes táplálás
A fa-hálózat egy ága teljesen elveszik	A szülő csomópont kiesett, és nincs alternatív út	Mesh használata, vagy a szülő redundáns kialakítása
A mesh-hálózat késleltetése növekszik	Túl sok ugrás, vagy az útvonalat gyakran újraépíti a hálózat	Routerek elhelyezésének optimalizálása, a kritikus forgalom priorizálása

9.2.6 Tipikus hibák és félreértések

„A mesh mindig jobb, mert öngyógyító." A mesh hibatűrése *lehet* jobb, de csak akkor, ha van alternatív út, a továbbítók működnek és a protokoll valóban új útvonalat választ. Ezt nagyobb energiafogyasztással, nagyobb komplexitással és nagyobb rádiós terheléssel éri el. Kis területen, kis csomópontszámnál a csillag topológia energiahatékonyabb és egyszerűbben üzemeltethető.

„A mesh-csomópont is alhat, mert csak továbbít." A mesh-routernek fogadnia kell a szomszédos csomópontok csomagjait, ezért nem alhat tetszőlegesen hosszan. Ha a router túl sokat alszik, csomagok vesznek el, vagy a hálózat késleltetése megnő.

„A gateway csak protokollfordító, ezért a topológia nem befolyásolja." A gateway kapacitása, rádiós interfészeinek száma és elhelyezése éppen a topológiától függ. Csillag esetén a gatewaynek minden végponttal közvetlen rádiós kapcsolatban kell lennie, mesh esetén viszont elegendő, ha a routereket éri el.

9.2.7 Feladatok

9.2.7.1 (a) Fogalmi kérdés

Magyarázza el, miért fogyaszt többet egy mesh-router csomópont, mint egy ugyanolyan rádióval szerelt alvó végpont, ha mindkettő ugyanannyi saját mérési adatot küld.

9.2.7.2 (b) Számolási feladat

Egy Thread-hálózatban egy alvó végpont 120 másodpercenként ébred fel, 15 ms-ig ad, 15 ms-ig hallgat, alvó árama 3 µA, adási árama 25 mA, vételi árama 8 mA. Számítsa ki az átlagos áramfelvételét. Mennyi lenne az átlagos áram, ha ugyanez a csomópont routerként az idő 4%-ában aktív vételi üzemmódban lenne?

9.2.7.3 (c) Tervezési feladat

Tervezze meg a TerraSense `gh2` üvegházának topológiáját 12 `ts-node` és egy `ts-gw` gateway esetén, ha a terület két, rádiósan árnyékolt zónára oszlik, és a zónák között nincs közvetlen rálátás. Válasszon topológiát, és rajzolja fel a csomópontok és a gateway közötti logikai kapcsolatokat. Indokolja a döntést.

9.2.7.4 (d) Hibakeresési vagy kiterjesztési feladat

Egy mesh-hálózatban az egyik router csomópont kiesik. Írja le, hogyan viselkedik a hálózat öngyógyító útvonalválasztása, és milyen mérésekkel ellenőrizné, hogy a forgalom a kiesés után is eljut a gatewayhez.

9.2.8 Megoldások

9.2.8.1 (a) Fogalmi kérdés megoldása

A mesh-routernek más csomópontok csomagjait is fogadnia kell, ezért rádiójának rendszeresen vagy folyamatosan vevő üzemmódban kell lennie. A vételi üzemmód áramfelvétele lényegesen nagyobb, mint az alvó állapoté. Egy alvó végpont csak a saját adásának és szükséges vételének idejére ébred fel, az idő többi részében alvó üzemmódban van. A nagyobb aktív időarány miatt a router átlagos áramfelvétele többszöröse lehet.

9.2.8.2 (b) Számolási feladat megoldása

Az alvó végpont periódusideje $T = 120\,\mathrm{s}$. Az aktív idő:

$$T_{\mathrm{tx}} = 15\,\mathrm{ms} = 0{,}015\,\mathrm{s}, \quad T_{\mathrm{rx}} = 15\,\mathrm{ms} = 0{,}015\,\mathrm{s}.$$

Az alvó idő:

$$T_{\mathrm{sleep}} = 120 - 0{,}015 - 0{,}015 = 119{,}97\,\mathrm{s}.$$

Az átlagos áram:

$$I_{\mathrm{avg}} = \frac{0{,}015 \cdot 0{,}025 + 0{,}015 \cdot 0{,}008 + 119{,}97 \cdot 0{,}000003}{120}.$$

Számláló:

$$0{,}000375 + 0{,}00012 + 0{,}00035991 = 0{,}00085491\,\mathrm{A \cdot s}.$$

Ezt 120 másodperccel osztva:

$$I_{\mathrm{avg}} \approx 0{,}00000712\,\mathrm{A} = 7{,}12\,\mu\mathrm{A}.$$

Routerként az aktív vételi idő 4%: $0{,}04 \cdot 120 = 4{,}8\,\mathrm{s}$. Az alvó idő:

$$T_{\mathrm{sleep,router}} = 120 - 0{,}015 - 4{,}8 = 115{,}185\,\mathrm{s}.$$

Az átlagos routeráram:

$$I_{\mathrm{avg,router}} = \frac{0{,}015 \cdot 0{,}025 + 4{,}8 \cdot 0{,}008 + 115{,}185 \cdot 0{,}000003}{120}.$$

Számláló:

$$0{,}000375 + 0{,}0384 + 0{,}000345555 = 0{,}039120555\,\mathrm{A}\cdot\mathrm{s}.$$

Osztva 120-nal:

$$I_{\mathrm{avg,router}} \approx 0{,}000326\,\mathrm{A} = 326\,\mu\mathrm{A}.$$

Ez mintegy 46-szorosa az alvó végpont átlagáramának, ami jól mutatja a továbbító szerep többletköltségét.

9.2.8.3 (c) Tervezési feladat megoldása

Mivel a két zóna között nincs közvetlen rádiós összeköttetés, a tiszta csillag topológia nem működik, mert a gateway nem érné el mindkét zónát. A megoldás technológiafüggő. Több ugrást támogató hálózatnál (Thread, BLE Mesh, Zigbee) zónánként egy-egy router csomópontot helyezünk el, amelyek a zónán belüli végpontok forgalmát a gatewayhez továbbítják; a két router között és a gateway felé is legyen közvetlen rádiós link. A `router1` az A zóna, a `router2` a B zóna forgalmát gyűjti, a `ts-gw` pedig a fa gyökere. Ha a későbbi telepítés során a routerek közötti közvetlen link is elérhető, akkor mesh-hálózattá bővíthető a rendszer a nagyobb hibatűrés érdekében. Egyugrásos technológiánál (BLE GATT-kapcsolat, LoRaWAN csillag) viszont nem lehet routert beiktatni: zónánként külön gatewayt vagy hozzáférési pontot kell telepíteni, és a két gyűjtőpontot a háttérhálózatban összekötni.

9.2.8.4 (d) Hibakeresési feladat megoldása

Amikor egy router kiesik, a mesh-útválasztási protokoll – ha támogatja az öngyógyítást – észleli, hogy a routeren át vezető útvonalak elérhetetlenné váltak, és új útvonalat keres a szomszédos csomópontokon keresztül. Ehhez legalább egy másik, működő továbbítónak kell lennie; egyetlen routeres mesh-ben nincs alternatív út, a border router kiesése pedig a felhő felé szakítja meg a forgalmat. A folyamatot a hálózat forgalmának és a csomópontok routing-tábláinak figyelésével lehet ellenőrizni. Gyakorlati mérésként a gatewayen megszámoljuk a beérkező telemetriaüzeneteket, és megfigyeljük, hogy a router kiesése után rövid kiesési idővel ismét minden csomópont adata megérkezik-e. Emellett a végpontokon naplózott csomagküldési hibák és az újracsatlakozási események is jelzik, hogy az öngyógyítás megtörtént.

9.2.9 Rövid összefoglaló

Az alfejezet a csillag, a fa és a mesh topológiákat hasonlította össze a TerraSense üvegház példáján. A csillag a legegyszerűbb és a legenergiahatékonyabb, de lefedettsége a gateway hatósugarától függ. A fa hierarchikusan bővíti a lefedettséget, de egy szülő kiesése az alhálózat elvesztését okozhatja, kivéve, ha a protokoll támogatja az új szülő választását. A mesh topológiája lehetőséget ad öngyógyító alternatív útvonalakra, de a tényleges hibatűréshez működő továbbítók, redundancia és megfelelő útválasztási protokoll kell; a router csomópontoknak folyamatosan vagy gyakran hallgatniuk kell, ezért átlagos áramfelvételük nagyságrendekkel nagyobb lehet. A Gateway minden topológiában az IoT-forgalom kilépési pontja. Továbbvitt eredmények: a TerraSense csillag topológiára vonatkozó döntését a A gateway szerepe: híd, puffer, biztonsági határ, a mesh-router energiaköltség-számítást pedig a Bluetooth Low Energy: hirdetés, GATT, kapcsolatok alfejezet használja tovább.

9.2.10 Források és ellenőrzési hiányok

A fejezethez tartozó forráskiindulópontok a Bluetooth, a LoRa Alliance és az OpenThread dokumentációi, valamint az RFC 6282. Az alfejezetben használt áramfelvételi értékek illusztratívak, és valós rádiómodulok esetén a gyártó adatlapjából kell ellenőrizni azokat. A Thread, a BLE Mesh és a LoRaWAN pontos útvonalválasztási és energiatakarékossági mechanizmusai verziófüggők lehetnek, ezért az aktuális specifikációt ellenőrizni kell. A TerraSense topológiadöntéséhez mérési jegyzőkönyvet érdemes készíteni, amely az adott helyszínre érvényes RSSI- és csomagvesztési értékeket tartalmazza; ilyen mérés a tananyagban nem szerepel ténylegesen mért adatként.

9.3 Bluetooth Low Energy: hirdetés, GATT, kapcsolatok

9.3.1 Motiváció

A Rádiós technológiák összevetése alfejezetben a TerraSense csomópont–gateway szakaszára a BLE mellett döntöttünk. A ts-node egy üvegházban, legfeljebb néhány tíz méterre van a ts-gw-től, és 10 percenként küld néhány száz bájt telemetriát. A klasszikus Bluetooth folyamatos kapcsolata és nagyobb fogyasztása felesleges lenne, a Wi-Fi pedig az energia- és költségkeretet feszítené szét. A BLE alacsony fogyasztása, olcsó moduljai és az okostelefonos provisioning lehetősége illeszkedik a TerraSense céljaihoz. Ebben az alfejezetben megtervezzük a ts-node BLE-interfészét: a hirdetésből a gateway felismeri a csomópontot, a GATT-szolgáltatáson keresztül pedig a telemetriát, az állapotot és a parancsokat cseréljük. Ez az interfész lesz a későbbi gateway-fordítás és az üzembe helyezés alapja.

9.3.2 Fogalmak és működés

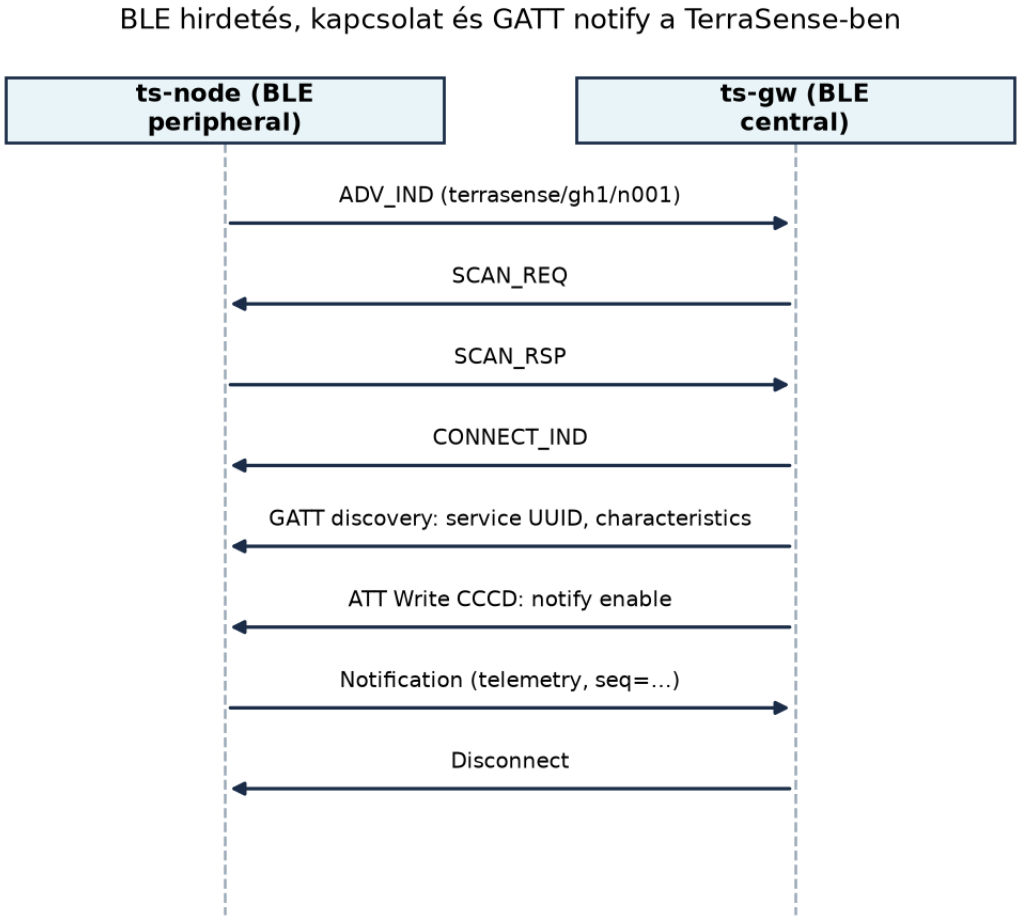
Definíció. A **Bluetooth Low Energy (BLE)** a Bluetooth 4.0-tól bevezetett, alacsony energiafogyasztású rádiós technológia. A 2,4 GHz-es sávban 40 csatornát használ, ebből 3 hirdetési csatorna (37, 38, 39) és 37 adatcsatorna; a kapcsolatban frekvenciaugrást (adaptive frequency hopping) alkalmaz. A fizikai adatsebesség tipikusan 1 Mb/s, opcionálisan 2 Mb/s vagy kódolt PHY.

Definíció. A **GAP** (Generic Access Profile) határozza meg a szerepeket. A **peripheral** hirdeti magát, és GATT-szerverként szolgál ki; a **central** pásztázik, kapcsolatot kezdeményez, és jellemzően GATT-kliens. A **broadcaster** csak hirdet, az **observer** csak pásztázik, kapcsolódás nélkül. Ugyanaz az eszköz több szerepet is felvehet, de a TerraSense-ben a ts-node peripheral, a ts-gw central.

A hirdetés (advertising) során a peripheral periodikusan, a három hirdetési csatornán ADV_IND csomagokat küld. A hirdetési adat legfeljebb 31 bájt hasznos adatot tartalmazhat (legacy advertising), amely szolgáltatás-UUID-t, nevet, gyártói adatot vagy beacon-formátumot hordozhat. A central a SCAN_REQ/SCAN_RSP váltással további adatot kérhet, vagy kapcsolatot kezdeményez (CONNECT_IND). A hirdetési intervallum 20 ms és 10,24 s között állítható; a rövidebb intervallum gyorsabb felfedezést, de nagyobb fogyasztást eredményez.

A **beacon-alapú telemetria** azt jelenti, hogy az eszköz a hirdetési csomagban közvetlenül adatot sugároz, kapcsolat nélkül. Például a BTHome vagy az iBeacon/Eddystone formátum egy hőmérséklet-értéket is elhelyezhet a 31 bájtban. Ez energiatakarékos, mert nincs kapcsolatfelépítés, viszont nincs nyugtázás, nincs titkosítás, és a hirdetési csatornák torlódhatnak. A TerraSense telemetriája 200 B körüli, ezért nem fér el a legacy hirdetésben; a hirdetést felderítésre és jelenlétre, a tényleges adatcserét GATT-kapcsolatra bízzuk.

Definíció. A **GATT** (Generic Attribute Profile) az ATT (Attribute Protocol) fölé épülő keretrendszer. A GATT-szerver egy attribútumtáblát tart fenn; az attribútumoknak 16 vagy 128 bites **UUID**-ja, értéke és jogosultságai vannak. A GATT-kliens felfedezi a szolgáltatásokat, olvas, ír, vagy feliratkozik a **notify**/**indicate** értesítésekre. A **notify** esetén a szerver elküldi az új értéket, de nem vár alkalmazásszintű nyugtát; az **indicate** esetén a kliens visszaigazol. A TerraSense telemetriájához a notify elegendő, mert a seq mező és a későbbi MQTT-ingest idempotenciája kezeli az elvesztést.



9.5. ábra: BLE hirdetés, kapcsolat és GATT notify a TerraSense-ben - A ts-node peripheral, a ts-gw central. A hirdetés periodikus, a kapcsolat csak adatcserére jön létre.

A kapcsolat felépítése után a gateway feliratkozik a telemetria karakterisztikára. A CCCD (Client Characteristic Configuration Descriptor) írásával engedélyezi a notify-t, így a ts-node minden új mintánál értesíti a gatewayt. A parancsok a cmd karakterisztikán lefelé, a visszaigazolás a cmd_result karakterisztikán felfelé halad.

Biztonság: párosítási szintek röviden. A BLE párosítás célja a kapcsolat titkosítása és a felek hitelesítése. A LE Security Mode 1 szintjei: **Level 1** – nincs biztonság; **Level 2** – nem hitelesített párosítás titkosítással; **Level 3** – hitelesített párosítás (pl. passkey); **Level 4** – hitelesített LE Secure Connections. A társítási módszerek: Just Works (nincs felhasználói beavatkozás, sebezhető passzív lehallgatásra), Passkey Entry, Numeric Comparison és Out-of-Band. A TerraSense valós telepítésénél a node és a gateway közötti kapcsolatot hitelesített LE

Secure Connections szintre állítjuk, különösen a cmd és a provisioning írásakor, mert a nyílt GATT-írás lehetővé tenné idegen parancs bejuttatását.

9.3.3 Döntési szempontok

A hirdetés alapú és a kapcsolat alapú adatküldés közötti választást az alábbi táblázat foglalja össze:

Szempont	Hirdetés (beacon)	GATT-kapcsolat
Előzetes kapcsolatfelépítés	nincs	van, néhány 10 ms–100 ms
Nyugtázás	nincs	ATT/alkalmazásszinten van
Hasznos adat legacy hirdetésben	31 B	néhány száz B attribútumonként
Energia	alacsony, ha a hirdetési intervallum nagy	magasabb a kapcsolati események miatt
Késleltetés	hirdetési intervallum függvénye	a kapcsolat fenntartása, de notify azonnal
Tipikus alkalmazás	jelenlét, kicsi státusz, beltéri pozíció	telemetria, parancs, firmware

A TerraSense-ben a hirdetést a felderítésre és a node jelenlétének jelzésére használjuk; a hirdetési adatban a szolgáltatás-UUID és a `ts-node/gh1/n001` azonosító jelenik meg. A telemetriát, az állapotot és a parancsokat GATT-on cseréljük, mert a 200 B-os telemetria és a megbízhatósági igény ezt indokolja. A beacon-alapú telemetria akkor lenne elegendő, ha a csomópont csak egyetlen hőmérsékletértéket küldene, de a három szenzor és a szelep visszajelzése túl nagy.

9.3.4 Kidolgozott példa és labor

Munkakönyvtár:

```
mkdir -p ~/iot-labor/i023 && cd ~/iot-labor/i023
```

Előfeltétel: a Zephyr első alkalmazás projektváz ismerete. A labor a GATT-interfész specifikációját és egy Zephyr BLE peripheral vázát hozza létre.

9.3.4.1 1. A TerraSense GATT-interfész specifikációja

Hozd létre a `~/iot-labor/i023/gatt-interface.md` fájlt az alábbi tartalommal. Ez a fájl a későbbi gateway-fordítás és provisioning kiindulópontja.

```
# TerraSense GATT-interfész (i023)

Szolgáltatás neve: TerraSense Service
Szolgáltatás UUID: a3f4c8d1-2b6e-4f1a-9c3e-7d8a5b1e0f42  (illusztratív, 128 bit)

Karakterisztikák:
- Telemetry UUID: a3f4c8d1-2b6e-4f1a-9c3e-7d8a5b1e0f43
  tulajdonságok: READ | NOTIFY
  érték: telemetria bájtok; séma az i032 alfejezetben
- State UUID: a3f4c8d1-2b6e-4f1a-9c3e-7d8a5b1e0f44
  tulajdonságok: READ | NOTIFY
  érték: node állapot, pl. "online", "watering"
```

```
- Cmd UUID: a3f4c8d1-2b6e-4f1a-9c3e-7d8a5b1e0f45
  tulajdonságok: WRITE (write with response)
  érték: parancs, pl. {"cmd":"valve","state":"open"}
- Cmd Result UUID: a3f4c8d1-2b6e-4f1a-9c3e-7d8a5b1e0f46
  tulajdonságok: NOTIFY
  érték: parancs eredménye, pl. {"cmd_id":7,"result":"ok"}
- Provision UUID: a3f4c8d1-2b6e-4f1a-9c3e-7d8a5b1e0f47
  tulajdonságok: WRITE | READ
  érték: site/node konfiguráció, pl. {"site":"gh1","node":"n001"}

Biztonság: valós telepítésnél LE Secure Connections; a laborban nincs párosítás.
```

A 128 bites UUID-k nem szabványosak, a laborhoz generáltuk őket. Valós termékben a gyártó saját regisztrált UUID-jait kell használni, hogy elkerüljük az ütközést.

9.3.4.2 2. Zephyr BLE peripheral váz

Hozd létre a `~/iot-labor/i023/ts-node-ble/` projektet. A `src/main.c` a GATT-szolgáltatást definiálja, elindítja a hirdetést, és működő notify-, illetve write-utat tartalmaz. A kód a Zephyr 3.7 API-t használja: a `BT_GATT_CHARACTERISTIC` hat argumentumot vár (UUID, tulajdonságok, jogosultság, read, write, user_data), a notify karakterisztikákhoz külön `BT_GATT_CCC` descriptor tartozik, a parancsot write callback dolgozza fel, az eredményt pedig notify küldi vissza.

```c
#include <string.h>

#include <zephyr/kernel.h>
#include <zephyr/bluetooth/bluetooth.h>
#include <zephyr/bluetooth/conn.h>
#include <zephyr/bluetooth/att.h>
#include <zephyr/bluetooth/gatt.h>
#include <zephyr/logging/log.h>
#include <zephyr/sys/byteorder.h>

LOG_MODULE_REGISTER(ts_node_ble, LOG_LEVEL_DBG);

#define DEVICE_NAME "ts-n001"
#define DEVICE_NAME_LEN (sizeof(DEVICE_NAME) - 1)
#define ADV_INT_MIN 160  /* 100 ms, az egység 0,625 ms */
#define ADV_INT_MAX 320  /* 200 ms */

/* Illusztratív TerraSense UUID-k, a BT_UUID_DECLARE_128 makró bájtsorrendje szerint. */
#define TS_UUID_SERVICE BT_UUID_DECLARE_128(0x42, 0x0f, 0x1e, 0x5b, 0x8a, 0x7d, \
    0x3e, 0x9c, 0x1a, 0x4f, 0x6e, 0x2b, 0xd1, 0xc8, 0xf4, 0xa3)
#define TS_UUID_TELEMETRY BT_UUID_DECLARE_128(0x43, 0x0f, 0x1e, 0x5b, 0x8a, 0x7d, \
    0x3e, 0x9c, 0x1a, 0x4f, 0x6e, 0x2b, 0xd1, 0xc8, 0xf4, 0xa3)
#define TS_UUID_STATE BT_UUID_DECLARE_128(0x44, 0x0f, 0x1e, 0x5b, 0x8a, 0x7d, \
    0x3e, 0x9c, 0x1a, 0x4f, 0x6e, 0x2b, 0xd1, 0xc8, 0xf4, 0xa3)
#define TS_UUID_CMD BT_UUID_DECLARE_128(0x45, 0x0f, 0x1e, 0x5b, 0x8a, 0x7d, \
    0x3e, 0x9c, 0x1a, 0x4f, 0x6e, 0x2b, 0xd1, 0xc8, 0xf4, 0xa3)
#define TS_UUID_CMD_RESULT BT_UUID_DECLARE_128(0x46, 0x0f, 0x1e, 0x5b, 0x8a, 0x7d, \
    0x3e, 0x9c, 0x1a, 0x4f, 0x6e, 0x2b, 0xd1, 0xc8, 0xf4, 0xa3)
#define TS_UUID_PROVISION BT_UUID_DECLARE_128(0x47, 0x0f, 0x1e, 0x5b, 0x8a, 0x7d, \
    0x3e, 0x9c, 0x1a, 0x4f, 0x6e, 0x2b, 0xd1, 0xc8, 0xf4, 0xa3)

static uint8_t telemetry_data[200];
static uint8_t state_data[] = "online";
static uint8_t cmd_data[64];
static uint8_t provision_data[64];
static uint16_t provision_len;

static const struct bt_gatt_attr *telemetry_attr;
```

```
static const struct bt_gatt_attr *cmd_result_attr;

static struct bt_conn *current_conn;
static struct k_work_delayable telemetry_work;
static struct k_work cmd_work;

static ssize_t telemetry_read(struct bt_conn *conn,
                              const struct bt_gatt_attr *attr,
                              void *buf, uint16_t len, uint16_t offset)
{
    return bt_gatt_attr_read(conn, attr, buf, len, offset,
                             telemetry_data, sizeof(telemetry_data));
}

static ssize_t state_read(struct bt_conn *conn,
                          const struct bt_gatt_attr *attr,
                          void *buf, uint16_t len, uint16_t offset)
{
    return bt_gatt_attr_read(conn, attr, buf, len, offset,
                             state_data, sizeof(state_data) - 1);
}

static ssize_t provision_read(struct bt_conn *conn,
                              const struct bt_gatt_attr *attr,
                              void *buf, uint16_t len, uint16_t offset)
{
    return bt_gatt_attr_read(conn, attr, buf, len, offset,
                             provision_data, provision_len);
}

static ssize_t provision_write(struct bt_conn *conn,
                               const struct bt_gatt_attr *attr,
                               const void *buf, uint16_t len,
                               uint16_t offset, uint8_t flags)
{
    ARG_UNUSED(conn);
    ARG_UNUSED(attr);
    ARG_UNUSED(flags);

    if (offset != 0) {
        return BT_GATT_ERR(BT_ATT_ERR_INVALID_OFFSET);
    }
    if (len > sizeof(provision_data)) {
        return BT_GATT_ERR(BT_ATT_ERR_INVALID_ATTRIBUTE_LEN);
    }

    memcpy(provision_data, buf, len);
    provision_len = len;
    LOG_INF("Provisioning adat frissítve (%u B)", len);
    return len;
}

static ssize_t cmd_write(struct bt_conn *conn,
                         const struct bt_gatt_attr *attr,
                         const void *buf, uint16_t len,
                         uint16_t offset, uint8_t flags)
{
    ARG_UNUSED(conn);
    ARG_UNUSED(attr);
    ARG_UNUSED(flags);

    if (offset != 0) {
        return BT_GATT_ERR(BT_ATT_ERR_INVALID_OFFSET);
```

```
    }
    if (len == 0 || len >= sizeof(cmd_data)) {
        return BT_GATT_ERR(BT_ATT_ERR_INVALID_ATTRIBUTE_LEN);
    }

    memcpy(cmd_data, buf, len);
    cmd_data[len] = '\0';
    k_work_submit(&cmd_work);
    return len;
}

static void ccc_changed(const struct bt_gatt_attr *attr, uint16_t value)
{
    ARG_UNUSED(attr);
    LOG_DBG("CCCD módosult: 0x%04x", (unsigned int)value);
}

static void telemetry_work_handler(struct k_work *work)
{
    static uint32_t seq;
    int err;

    if (current_conn != NULL &&
        bt_gatt_is_subscribed(current_conn, telemetry_attr, BT_GATT_CCC_NOTIFY)) {
        sys_put_le32(seq++, telemetry_data);
        sys_put_le32(k_uptime_get_32(), &telemetry_data[4]);
        err = bt_gatt_notify(current_conn, telemetry_attr,
                             telemetry_data, sizeof(telemetry_data));
        if (err) {
            LOG_ERR("Telemetry notify hiba: %d", err);
        }
    }

    /* A laborban 10 másodperc; a valós TerraSense-ben 10 perc. */
    k_work_schedule(&telemetry_work, K_SECONDS(10));
}

static void cmd_work_handler(struct k_work *work)
{
    static const char result_ok[] = "{\"cmd_id\":0,\"result\":\"ok\"}";
    static const char result_unknown[] = "{\"cmd_id\":0,\"result\":\"unknown\"}";
    const char *result;
    size_t result_len;
    int err;

    ARG_UNUSED(work);

    if (current_conn == NULL ||
        !bt_gatt_is_subscribed(current_conn, cmd_result_attr, BT_GATT_CCC_NOTIFY)) {
        LOG_WRN("Nincs feliratkozott gateway, az eredmény nem küldhető el");
        return;
    }

    if (strstr((const char *)cmd_data, "\"state\":\"open\"") != NULL) {
        result = result_ok;
    } else {
        result = result_unknown;
    }

    result_len = strlen(result);
    err = bt_gatt_notify(current_conn, cmd_result_attr, result, result_len);
    if (err) {
        LOG_ERR("Command result notify hiba: %d", err);
```

```
    }
}

static struct bt_gatt_attr attrs[] = {
    BT_GATT_PRIMARY_SERVICE(TS_UUID_SERVICE),
    BT_GATT_CHARACTERISTIC(TS_UUID_TELEMETRY,
                           BT_GATT_CHRC_READ | BT_GATT_CHRC_NOTIFY,
                           BT_GATT_PERM_READ, telemetry_read, NULL, NULL),
    BT_GATT_CCC(ccc_changed, BT_GATT_PERM_READ | BT_GATT_PERM_WRITE),
    BT_GATT_CHARACTERISTIC(TS_UUID_STATE,
                           BT_GATT_CHRC_READ | BT_GATT_CHRC_NOTIFY,
                           BT_GATT_PERM_READ, state_read, NULL, NULL),
    BT_GATT_CCC(ccc_changed, BT_GATT_PERM_READ | BT_GATT_PERM_WRITE),
    BT_GATT_CHARACTERISTIC(TS_UUID_CMD,
                           BT_GATT_CHRC_WRITE,
                           BT_GATT_PERM_WRITE, NULL, cmd_write, NULL),
    BT_GATT_CHARACTERISTIC(TS_UUID_CMD_RESULT,
                           BT_GATT_CHRC_NOTIFY,
                           BT_GATT_PERM_NONE, NULL, NULL, NULL),
    BT_GATT_CCC(ccc_changed, BT_GATT_PERM_READ | BT_GATT_PERM_WRITE),
    BT_GATT_CHARACTERISTIC(TS_UUID_PROVISION,
                           BT_GATT_CHRC_READ | BT_GATT_CHRC_WRITE,
                           BT_GATT_PERM_READ | BT_GATT_PERM_WRITE,
                           provision_read, provision_write, NULL),
};

static struct bt_gatt_service ts_svc = BT_GATT_SERVICE(attrs);

static const struct bt_data ad[] = {
    BT_DATA_BYTES(BT_DATA_FLAGS, (BT_LE_AD_GENERAL | BT_LE_AD_NO_BREDR)),
    BT_DATA_BYTES(BT_DATA_UUID128_ALL, 0x42, 0x0f, 0x1e, 0x5b, 0x8a, 0x7d,
                  0x3e, 0x9c, 0x1a, 0x4f, 0x6e, 0x2b, 0xd1, 0xc8, 0xf4, 0xa3),
    BT_DATA(BT_DATA_NAME_COMPLETE, DEVICE_NAME, DEVICE_NAME_LEN),
};

static const struct bt_le_adv_param *adv_param = BT_LE_ADV_PARAM(
    BT_LE_ADV_OPT_CONNECTABLE, ADV_INT_MIN, ADV_INT_MAX, NULL);

static void connected(struct bt_conn *conn, uint8_t err)
{
    if (err) {
        LOG_ERR("Kapcsolódási hiba: 0x%02x", err);
        return;
    }
    current_conn = bt_conn_ref(conn);
    LOG_INF("Gateway csatlakozott");
}

static void disconnected(struct bt_conn *conn, uint8_t reason)
{
    if (current_conn == conn) {
        bt_conn_unref(current_conn);
        current_conn = NULL;
    }
    LOG_INF("Gateway lecsatlakozott: 0x%02x", reason);
}

BT_CONN_CB_DEFINE(conn_callbacks) = {
    .connected = connected,
    .disconnected = disconnected,
};

int main(void)
```

```
{
    int err;

    k_work_init_delayable(&telemetry_work, telemetry_work_handler);
    k_work_init(&cmd_work, cmd_work_handler);

    err = bt_enable(NULL);
    if (err) {
        LOG_ERR("bt_enable hiba: %d", err);
        return 0;
    }

    telemetry_attr = bt_gatt_find_by_uuid(attrs, ARRAY_SIZE(attrs),
                                          TS_UUID_TELEMETRY);
    cmd_result_attr = bt_gatt_find_by_uuid(attrs, ARRAY_SIZE(attrs),
                                           TS_UUID_CMD_RESULT);
    if (telemetry_attr == NULL || cmd_result_attr == NULL) {
        LOG_ERR("GATT attribútum keresése sikertelen");
        return 0;
    }

    err = bt_gatt_service_register(&ts_svc);
    if (err) {
        LOG_ERR("GATT service regisztrációs hiba: %d", err);
        return 0;
    }

    err = bt_le_adv_start(adv_param, ad, ARRAY_SIZE(ad), NULL, 0);
    if (err) {
        LOG_ERR("Advertising indítási hiba: %d", err);
        return 0;
    }

    LOG_INF("Advertising elindult, ts-n001");
    k_work_schedule(&telemetry_work, K_SECONDS(10));
    return 0;
}
```

A `telemetry_data` 200 B-os tömb, mert a telemetria séma (Adatformátumok: JSON, CBOR, Protocol Buffers, SenML) ekkora; a laborban az első 8 bájton sorszám és uptime van, a többi nulla. A 200 B-os notificationhöz a gatewaynek legalább 203 B-os ATT MTU-t kell egyeztetnie; a `prj.conf` ezért 247 B-os L2CAP TX MTU-t és 251 B-os ACL puffert állít be. Ha a gateway kisebb MTU-t egyeztet, a notify rövidül vagy hibát ad, ezért a központi oldalon is kezelni kell a töredezést.

A `BT_GATT_CCC` descriptor nélkül a kliens nem tud feliratkozni, a `bt_gatt_notify` pedig nem küld értesítést. A `bt_gatt_is_subscribed` ellenőrzés kapcsolatonként azt vizsgálja, hogy a gateway tényleg beírta-e a CCCD-t notify módra. A `cmd` karakterisztika write with response, így a parancs átvétele ATT-szinten nyugtázott; a végrehajtás eredményét a `cmd_result` notification adja vissza, ami együtt teljes parancs–eredmény kört alkot. A valós termékben a `BT_GATT_PERM_WRITE` helyett `BT_GATT_PERM_WRITE_ENCRYPT` vagy `BT_GATT_PERM_WRITE_AUTHEN` kell.

A `prj.conf`:

```
CONFIG_BT=y
CONFIG_BT_PERIPHERAL=y
CONFIG_BT_DEVICE_NAME="ts-n001"
CONFIG_BT_L2CAP_TX_MTU=247
CONFIG_BT_BUF_ACL_TX_SIZE=251
CONFIG_BT_BUF_ACL_RX_SIZE=251
```

```
CONFIG_LOG=y
CONFIG_LOG_MODE_IMMEDIATE=y
```

A `CMakeLists.txt`:

```
cmake_minimum_required(VERSION 3.20.0)
find_package(Zephyr REQUIRED HINTS $ENV{ZEPHYR_BASE})
project(ts-node-ble)
target_sources(app PRIVATE src/main.c)
```

9.3.4.3 3. Build és futtatás

Ez a labor hardveres labor: a BLE-hirdetés és a GATT-kapcsolat valódi BLE-controllert igényel. A `native_sim` boardon nincs BLE-controller és nincs HCI-backend, ezért ott a `bt_enable` `-ENODEV` vagy `-ENOTSUP` hibát ad, és a hirdetés nem indul el; a `native_sim` build legfeljebb a BLE-től független kódrészek fordítására alkalmas, a BLE-viselkedés ellenőrzésére nem. A BabbleSim (`nrf52_bsim`) működő szimulációs út, de külön fejlesztői környezet telepítését igényli, ezért ebben a laborban nem használjuk.

A javasolt hardver nRF52840 DK (Cortex-M4F, beépített BLE-controller és J-Link debug probe). A build és a flash:

```
west build -d ~/iot-labor/i023/build-nrf -b nrf52840dk/nrf52840 ~/iot-labor/i023/ts-node-ble
west flash -d ~/iot-labor/i023/build-nrf
```

A `west flash` a kártya J-Link probe-ját használja; a felhasználónak jogosultsága kell a debug probe eléréséhez (jellemzően `plugdev` csoport és udev-szabály) és a BlueZ D-Bus szolgáltatásához (jellemzően `bluetooth` csoport vagy polkit-szabály). A labor nem módosít rendszerbeállítást; ha a `bluetoothctl` nem indul el, a jogosultságot a hálózati adminisztrátor állítja be.

Linux oldalon a BlueZ `bluetoothctl` kliensével pásztázhatunk és kapcsolódhatunk:

```
bluetoothctl

[bluetooth]# scan on
Discovery started
[NEW] Device 12:34:56:78:9A:BC ts-n001
[bluetooth]# connect 12:34:56:78:9A:BC
Connection successful
[ts-n001]# menu gatt
[ts-n001]# list-attributes
```

Az illusztratív kimenetben a MAC-cím kártyánként változik. A `list-attributes` a TerraSense szolgáltatást és az öt karakterisztikát (Telemetry, State, Cmd, Cmd Result, Provision) listázza. A további lépések:

```
[ts-n001]# select-attribute /org/bluez/hci0/dev_12_34_56_78_9A_BC/service0001/char0002
[ts-n001]# notify on
[ts-n001]# write "{\"cmd\":\"valve\",\"state\":\"open\"}"
```

A telemetria notification 10 másodpercenként érkezik (a laborban lerövidített periódus), a `cmd` write után pedig a `cmd_result` notification `{"cmd_id":0,"result":"ok"}` vagy `{"cmd_id":0,"result":"unknown"}` üzenetet ad. A `menu gatt`, a `select-attribute` és a `write` pontos elérhetősége BlueZ-verziófüggő; régebbi BlueZ-ben ezek a parancsok közvetlenül a fő menüből is működnek. A karakterisztikák objektumútvonalát mindig a saját `list-attributes` kimenetéből kell venni.

9.3.5 Ellenőrzés és hibakeresés

A GATT-interfész működését öt szinten ellenőrizzük:

1. **Logok:** a ts-node az `Advertising elindult, ts-n001` üzenetet naplózza. Ha a `bt_enable` hibát ad, a `CONFIG_BT` hiányzik, nincs controller, vagy `native_sim`-en futunk BLE-backend nélkül.
2. **Felfedezés:** a BlueZ `bluetoothctl scan on` listázza a ts-node-ot. Ha nem, ellenőrizd a hirdetési adatot és a `BT_LE_ADV_PARAM`-mal beállított 100–200 ms-os intervallumot; a szolgáltatás-UUID-nak benne kell lennie az `ad` tömbben.
3. **GATT-böngészés:** `connect` után a `list-attributes` mutatja a szolgáltatást, az öt karakterisztikát és a notify karakterisztikák CCCD descriptorát. Ha a szolgáltatás hiányzik, a `bt_gatt_service_register` visszatérési értéke vagy a `bt_enable` sorrendje hibás.
4. **Feliratkozás és notify:** a telemetria karakterisztikán `notify on` után 10 másodpercenként érkezik a notification, benne a növekvő `seq` és az uptime értékkel. Ha nem érkezik, a CCCD-t nem írta be a kliens, vagy a `bt_gatt_is_subscribed` feltétel hamis.
5. **Parancs–eredmény kör:** a `cmd` karakterisztikára írt `{"cmd":"valve","state":"open"}` után a `cmd_result` notification `{"cmd_id":0,"result":"ok"}` üzenetet ad; ismeretlen parancsnál `"unknown"` az eredmény. Ez igazolja a write callback és a result notify útját.

Tünet	Ok	Javítás
`bt_enable` -ENODEV vagy -ENOTSUP	Nincs BLE controller vagy nincs engedélyezve a stack	Ellenőrizd a `prj.conf`-ot; `native_sim` helyett valós nRF52840 DK-t használj
A node nem látható a scan listában	Hirdetési adat üres vagy az intervallum túl nagy	Adj szolgáltatás-UUID-t és nevet az `ad` tömbböz; ellenőrizd a `BT_LE_ADV_PARAM` intervallumát
A karakterisztikák nem jelennek meg	A GATT service regisztráció hibás vagy a `bt_enable` után történt	Ellenőrizd a `bt_gatt_service_register` visszatérési értékét és a sorrendet
A gateway nem kap notify-t	A CCCD descriptor hiányzik vagy a kliens nem írta be a notify engedélyt	Győződj meg arról, hogy a karakterisztikához tartozik `BT_GATT_CCC`, és a gateway feliratkozott
A notify `-EMSGSIZE` hibát ad	A gateway ATT MTU-ja kisebb 203 B-nál	A gateway egyeztessen nagyobb ATT MTU-t, vagy az alkalmazás töredezze a 200 B-os telemetriát
A `cmd` write hibát ad	Hibás offset, túl hosszú vagy üres üzenet, illetve hiányzó write engedély	Ellenőrizd a `cmd_write` offset- és hosszellenőrzését, valamint a `BT_GATT_PERM_WRITE` jogosultságot
A parancs eredménye nem érkezik meg	A gateway nem iratkozott fel a `cmd_result` CCCD-re	A gateway írja be a `cmd_result` CCCD-t notify módra

9.3.6 Tipikus hibák és félreértések

1. **ISR-ből hívott `bt_gatt_notify`.** A BLE-stack függvényei nem hívhatók megszakítás-kontextusból, mert belső szinkronizációt és memóriaallokációt használhatnak. A szenzor megszakításkezelője csak jelezzen egy szálnak vagy work queue-nak, és a notify-t abból küldjük.
2. **Túl kicsi hirdetési intervallum.** Az energiagazdálkodásnál gyakori hiba a 10–20 ms-os hirdetés állandó használata. A TerraSense 10 perces mintavételénél 100–200 ms hirdetési intervallum is elegendő a felfedezéshez, és a rádiós energia töredékét fogyasztja.
3. **Nyílt GATT-írás a parancsokhoz.** A cmd karakterisztika write engedélye `BT_GATT_PERM_WRITE` titkosítás nélkül lehetővé teszi, hogy bárki a hatótávon belül szelepet nyisson. Valós rendszerben `BT_GATT_PERM_WRITE_ENCRYPT` vagy `BT_GATT_PERM_WRITE_AUTHEN` szükséges.
4. **A hirdetési adat és a GATT-karakterisztika összekeverése.** A hirdetésben csak kis mennyiségű, nem titkosított adatot szabad elhelyezni. A telemetriát nem a hirdetési csomagba tesszük, hanem GATT notify-jal küldjük el a kapcsolat létrejötte után.

9.3.7 Feladatok

1. **Fogalmi kérdés.** Mi a különbség a GATT notify és indicate között, és miért elegendő a TerraSense telemetriájához a notify?
2. **Számolási feladat.** A ts-node legacy hirdetést használ 1 s intervallummal. Egy ADV_IND csomag küldése 376 µs-ig tart 1 Mb/s PHY mellett. Számítsd ki a hirdetési rádiós duty cycle-t, és becsüld meg az átlagos rádiós áramot, ha a TX csúcsáram 10 mA. Az eredményt illusztratívnak tekintjük.
3. **Implementációs feladat.** Egészítsd ki a labor GATT-szolgáltatását egy `Uptime` karakterisztikával (UUID `a3f4c8d1-2b6e-4f1a-9c3e-7d8a5b1e0f48`, READ | NOTIFY), amely a node boot óta eltelt másodperceket adja vissza. Frissítsd a `gatt-interface.md`-t is.
4. **Hibakeresési feladat.** A gateway sikeresen csatlakozik a ts-node-hoz, de a telemetria karakterisztika notify-ja nem érkezik meg. Sorold fel a lehetséges okokat, és add meg a javítás lépéseit.

9.3.8 Megoldások

1. A **notify** esetén a szerver elküldi az új értéket, de a kliens nem küld alkalmazásszintű visszaigazolást; az **indicate** esetén a kliens ATT-választ küld minden értesítésre. A notify gyorsabb és kisebb forgalmú, de egy értesítés elveszhet. A TerraSense telemetriája 10 percenként érkezik, a csomagban `seq` sorszám van, és az MQTT-ingest a Ingest: MQTT-ből a backendbe, idempotensen alfejezetben idempotensen fogadja a duplikátumokat, ezért a notify elegendő. A indicate akkor lenne indokolt, ha minden egyes értesítés megbízható kézbesítése alkalmazásszinten is kritikus lenne, és a lassabb nyugtázás nem jelent problémát.
2. A duty cycle:

$$D = \frac{376 \cdot 10^{-6}\ \text{s}}{1\ \text{s}} = 3{,}76 \cdot 10^{-4} = 0{,}0376\,\%.$$

   Az átlagos rádiós áram:

$$I_{\text{avg}} = D \cdot I_{\text{TX}} = 3{,}76 \cdot 10^{-4} \cdot 10 \text{ mA} = 3{,}76 \ \mu\text{A}.$$

Ez illusztratív, mert a valós átlagot az alvóáram, a hirdetési csatornák közötti időzítés és a rádió felébredési ideje is befolyásolja.

3. A `Uptime` karakterisztika hozzáadása a `gatt-interface.md`-hez, majd a Zephyr kódban:

```
#define TS_UUID_UPTIME BT_UUID_DECLARE_128(0x48, 0x0f, 0x1e, 0x5b, 0x8a, 0x7d, \
    0x3e, 0x9c, 0x1a, 0x4f, 0x6e, 0x2b, 0xd1, 0xc8, 0xf4, 0xa3)

static uint8_t uptime_value[4]; /* uint32 másodperc, little-endian */

static ssize_t uptime_read(struct bt_conn *conn,
                           const struct bt_gatt_attr *attr,
                           void *buf, uint16_t len, uint16_t offset)
{
    uint32_t uptime = (uint32_t)(k_uptime_get() / 1000);

    sys_put_le32(uptime, uptime_value);
    return bt_gatt_attr_read(conn, attr, buf, len, offset,
                             uptime_value, sizeof(uptime_value));
}
```

A táblában a `State` után:

```
    BT_GATT_CHARACTERISTIC(TS_UUID_UPTIME,
                           BT_GATT_CHRC_READ | BT_GATT_CHRC_NOTIFY,
                           BT_GATT_PERM_READ, uptime_read, NULL, NULL),
    BT_GATT_CCC(ccc_changed, BT_GATT_PERM_READ | BT_GATT_PERM_WRITE),
```

A `BT_GATT_CHARACTERISTIC` itt is hat argumentumot kap, és maga hozza létre az értékattribútumot, ezért külön `BT_GATT_DESCRIPTOR` nem kell. A notifyhoz a `BT_GATT_CCC` adja a CCCD descriptort; a read callback önmagában nem küld értesítést, ezért a telemetriához hasonlóan egy delayable workből kell `bt_gatt_notify`-t hívni. A `sys_put_le32` a `zephyr/sys/byteorder.h` include-ot igényli. A `gatt-interface.md`-ben rögzítsd az új UUID-t és a tulajdonságokat.

4. Lehetséges okok és javítások:

Ok	Javítás
A gateway a GATT felfedezés után nem írta a CCCD descriptorba a notify engedélyt	A gateway klienskódjában engedélyezze a CCCD-t: `0x01:00` notify, `0x02:00` indicate
A telemetria karakterisztikának nincs CCCD descriptorja	A GATT definícióban a karakterisztikához tartozzon `BT_GATT_CCC` descriptor
A ts-node nem hívja a `bt_gatt_notify`-t	Ellenőrizd, hogy a telemetria szál periodikusan küld notify-t, és a paraméterek helyesek
A kapcsolat megszakad a notify előtt	Hosszabbítsd a kapcsolati időtúllépést, vagy a gateway tartsa fenn a kapcsolatot

9.3.9 Rövid összefoglaló

A TerraSense ts-node BLE peripheral, a ts-gw BLE central. A node hirdetésben jelzi a jelenlétét, a szolgáltatás-UUID-ját és a nevét; a kapcsolat létrejötte után a GATT-szolgáltatáson keresztül cserél adatot. A hirdetés alapú beacon kevés, nem nyugtázott adatot szállít, míg a kapcsolat alapú GATT notify megbízhatóbb és nagyobb hasznos adatot enged. A laborban

definiáltuk a TerraSense GATT-szolgáltatást (`~/iot-labor/i023/gatt-interface.md`), a 128 bites UUID-kkal és a telemetry, state, cmd, cmd_result, provision karakterisztikákkal. A Zephyr 3.7-es váz (`~/iot-labor/i023/ts-node-ble/`) hat argumentumos `BT_GATT_CHARACTERISTIC` makrót, `BT_GATT_CCC` descriptorokat, `bt_gatt_is_subscribed` ellenőrzést, valamint működő `bt_gatt_notify` és write callback utat mutat; a hirdetési intervallumot a `BT_LE_ADV_PARAM` köti be. A BLE-viselkedés ellenőrzése valós nRF52840 DK-n történik, mert a `native_sim` boardon nincs BLE-controller. Ezt a GATT-interfészt használja a ts-gw protokollfordítás és az üzembe helyezés alfejezet.

9.3.10 Források és ellenőrzési hiányok

- Bluetooth Core Specification: https://www.bluetooth.com/specifications/specs/ – ellenőrizendő a legacy advertising 31 bájtos korlátja, a hirdetési intervallum tartománya, az ATT MTU és a Data Length Extension, valamint a CCCD notify szemantikája.
- Zephyr 3.7 BLE dokumentáció: https://docs.zephyrproject.org/3.7.0/connectivity/bluetooth/api.html – ellenőrizendő a hat argumentumos `BT_GATT_CHARACTERISTIC`, a `BT_GATT_CCC`, a `bt_gatt_notify`, a `bt_gatt_is_subscribed` és a `BT_LE_ADV_PARAM` pontos szintaxisa.
- BlueZ hivatalos dokumentáció: https://git.kernel.org/pub/scm/bluetooth/bluez.git – ellenőrizendő a `bluetoothctl menu gatt`, `select-attribute`, `notify` és `write` parancsainak verziófüggő viselkedése.
- A labor UUID-jai illusztratívak; valós termékben a Bluetooth SIG 16 bites UUID-jait vagy a gyártó által regisztrált 128 bites UUID-kat kell használni.
- A `native_sim` boardon nincs BLE-controller és HCI-backend, ezért a labor BLE-viselkedése valós nRF52840 DK-t igényel; a BabbleSim (`nrf52_bsim`) külön telepítést igénylő alternatíva, amelyet ez a labor nem használ. A kód, a buildparancsok és a BlueZ-lépések statikus ellenőrzéssel készültek; a tényleges build, flash és bluetoothctl-es végigtesztelés ebben a kiadásban nem történt meg, ezért a labor végrehajtásakor kell igazolni.

9.4 LoRaWAN: nagy hatótáv kis energiával

9.4.1 Motiváció: távoli üvegház gateway nélkül

A TerraSense alapértelmezett laborarchitektúrájában a `ts-node` szenzorcsomópont UDP-n küldi a mérési adatokat a `ts-gw` gatewaynek, amely aztán MQTT-n továbbítja a backend felé. Ez a modell akkor működik jól, ha a csomópont és a gateway között van megbízható helyi hálózat, például egy üvegházon belüli Wi-Fi vagy vezetékes Ethernet.

Más a helyzet, ha a monitorozott üvegház egy távoli, infrastruktúra nélküli területen van, vagy ha egyetlen gatewaynek több, egymástól több száz méterre lévő csomópontot kellene kiszolgálnia. Ilyenkor a rövid hatótávú rádiók (BLE, Wi-Fi) nem elegendők, a cellás kapcsolat pedig túl drága és energiaéhes lenne egy elemes szenzorhoz. A LoRaWAN ezt a rést tölti be: kilométeres hatótávot ad alacsony adatsebességgel és nagyon kis energiafogyasztással, cserébe szigorú adásgyakorisági korlátokkal. Ebben az alfejezetben megismerjük a LoRaWAN architektúráját, eszközosztályait, aktiválási módjait, és kiszámítjuk, milyen gyakran küldhet egy eszköz a duty cycle szabályok betartásával.

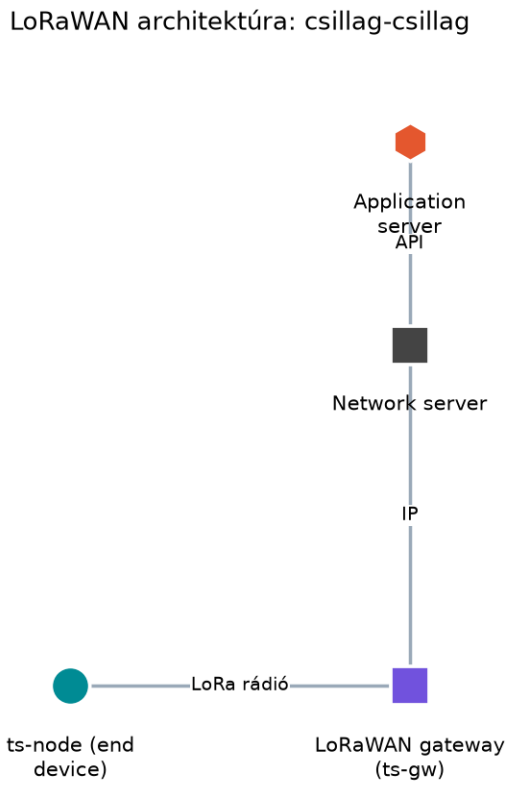
9.4.2 Fogalmak és működés

9.4.2.1 Architektúra és a csillag-csillag topológia

Definíció. A LoRaWAN (Long Range Wide Area Network) egy LPWAN (Low Power Wide Area Network) hálózati protokoll, amely a LoRa modulációt használja. A hálózat négy fő elemből áll:

- **End device (végberendezés):** a szenzor vagy aktuátor, például a `ts-node`.
- **Gateway (átjáró):** rádiós bázisállomás, amely a LoRa rádiójeleket IP-csomagokká alakítja. Egy gateway több ezer eszközt is kiszolgálhat. A gateway nem fejti vissza az application payloadot, csak a rádiókereteket és a vételi metaadatokat továbbítja.
- **Network server (hálózati szerver):** a LoRaWAN hálózati rétegét kezeli: munkamenet-állapot, `DevAddr`, frame counterek, MIC-ellenőrzés, duplikátumszűrés, adaptív adatsebesség és downlink-ütemezés. A hálózati kulccsal (`NwkSKey`) a hálózati réteg integritását és a hálózati mezőket védi; az application payload visszafejtése nem a feladata.
- **Application server (alkalmazásszerver):** az application payloadot az `AppSKey` kulccsal fejti vissza, dekódolja az alkalmazás sémáját, továbbítja az alkalmazás felé, és fogadja az alkalmazás parancsait. Gyakran MQTT-n vagy HTTP-n kommunikál a backenddel.
- **Join server (LoRaWAN 1.1):** a gyökérkulcsokat (`NwkKey`, `AppKey`) tárolja, és a join során származtatja, majd szétosztja a munkamenetkulcsokat a network server és az application server felé. Ezzel a hálózati és az alkalmazási kulcs szétválasztható.

A LoRaWAN rádiós oldala **csillag topológiájú**: minden end device közvetlenül egy vagy több gatewayhez küld, az eszközök egymással nem kommunikálnak. A gatewayek IP-hálózaton (Ethernet, Wi-Fi, cellás) csatlakoznak a network serverhez, ami egy második, hálózati csillagot alkot. Ezt a kétszintű felépítést nevezzük **csillag-csillag topológiának**.



9.6. ábra: LoRaWAN architektúra: csillag-csillag - Az end device-ok rádiós csillagban kapcsolódnak a gatewayhez, a gatewayek IP-hálózaton a network serverhez.

Az ábra azt mutatja, hogy a `ts-node` LoRa rádión küld a `ts-gw`-nek, amely IP-n továbbít a network servernek, onnan pedig az application serverhez jut az adat. A gateway nem értelmezi és nem fejti vissza az application payloadot, csak a rádiókereteket és a vételi metaadatokat továbbítja. A MIC-et és a hálózati mezőket a network server ellenőrzi `NwkSKey`-jel, az application payloadot pedig az application server fejti vissza `AppSKey`-jel. A LoRaWAN 1.1-ben a join server osztja szét a két kulcsot, ezért a network server nem kapja meg automatikusan az application payload visszafejtéséhez szükséges kulcsot. A LoRaWAN 1.0.x-ben a kulcsszármaztatás egyszerűbb, és a network server jellemzően mindkét munkamenetkulcsot ismeri; ilyenkor a két szerep szétválasztása elsősorban üzemeltetési és szerződéses, nem kriptográfiai garancia.

9.4.2.2 Class A, B és C: downlink-ablakok

Definíció. A LoRaWAN három eszközosztályt határoz meg, amelyek abban különböznek, hogy az eszköz mikor tud downlink üzenetet fogadni:

- **Class A:** minden uplink után két rövid downlink-ablakot nyit (RX1 és RX2). Az eszköz csak ezekben az ablakokban hallgat. Ez a legalacsonyabb energiafogyasztású mód, és minden LoRaWAN eszköznek támogatnia kell.
- **Class B:** az eszköz ütemezett időrésekben (ping slots) is hallgat, így a downlink késleltetése kisebb, de az energiafogyasztás nagyobb. A Class B eszközök szinkronizálódnak a gateway beaconjaival.
- **Class C:** az eszköz folyamatosan nyitva tartja a vevőjét, kivéve, amikor éppen ad. Ez a legkisebb downlink késleltetést adja, de csak hálózati táplálás mellett ésszerű.

A TerraSense elemes szenzorcsomópontja számára a **Class A** a természetes választás, mert a mérési ciklus (10 perc az Energiamodell és az elemélettartam kiszámítása alapján) ritkán igényel azonnali parancsot. A szelep vezérléséhez a parancs a következő uplink utáni RX1 vagy RX2 ablakban érkezhet meg, de ez csak a **következő fogadási lehetőség**, nem kézbesítési garancia: ha az uplink vagy a downlink elveszik, a parancs nem érkezik meg. Ezért a parancsnak TTL-t (lejárati időt) kell adni, az application server a TTL-en belül újrapróbálkozhat, és a szelep helyi biztonsági logikája (a Aktuátorok: biztonságos vezérlés és visszajelzés alfejezet `max_on_time` korlátja) a parancs hiányában is zár. A gyakorlati késleltetés így a következő uplinkig tartó várakozás plusz az esetleges újrapróbálkozások ideje, nem pedig garantáltan legfeljebb egy periódus.

9.4.2.3 OTAA vs. ABP aktiválás

Definíció. A LoRaWAN kétféle aktiválási módot kínál:

- **OTAA (Over-The-Air Activation):** az eszköz csatlakozási kérelmet (join request) küld, amelyre a join server (LoRaWAN 1.1) vagy a network server (1.0.x) join accept üzenettel válaszol. Ezután az eszköz és a hálózat megegyezik a `DevAddr` azonosítóban és a munkamenetkulcsokban. A LoRaWAN 1.1-ben a join server a gyökérkulcsokból (`NwkKey`, `AppKey`) származtatja a hálózati `NwkSKey` és az alkalmazási `AppSKey` munkamenetkulcsot, és külön adja át a network servernek, illetve az application servernek. Az eszköz az application payloadot `AppSKey`-jel titkosítja, a MIC számításához pedig a hálózati kulcsot használja. Az OTAA lehetővé teszi a kulcsok rotációját és a biztonságos újracsatlakozást.
- **ABP (Activation By Personalization):** a `DevAddr` és a munkamenetkulcsok előre be vannak égetve az eszközbe, így nincs join eljárás; az eszköz azonnal küldhet. Cserébe a kulcsok statikusak, a network server jellemzően mindkét kulcsot ismeri, és a frame counter elvesztése biztonsági kockázatot jelent.

A TerraSense esetében az **OTAA** javasolt, mert a join serverrel való időszakos újracsatlakozás lehetővé teszi a kulcsfrissítést és a biztonságos flottakezelést. Az ABP csak olyan zárt, felügyelt környezetben elfogadható, ahol a frame counter perzisztens tárolása garantált, és a network server és az application server szerepe nem válik szét kriptográfiailag.

9.4.2.4 Spreading factor, adatsebesség, rádióidő és duty cycle

Definíció. A LoRa modulációban a **spreading factor (SF)** azt adja meg, hogy egy szimbólum hány chipből áll: 2^{SF} chipből. A chip a chirp moduláció elemi időegysége, a szimbólum pedig SF bit információt hordoz; a kódolási ráta, a header és a CRC miatt a tényleges adatbitráta ennél kisebb. A szimbólumidő $T_{\text{sym}} = 2^{SF}/BW$, tehát az SF növelése exponenciálisan növeli a szimbólumidőt: 125 kHz-en SF7 esetén $T_{\text{sym}} = 1{,}024$ ms, SF12 esetén 32,768 ms, ami ugyanannyi szimbólumnál 32-szer hosszabb adást jelent. Nagyobb SF nagyobb érzékenységet és hatótávot ad, de kisebb adatsebességet és hosszabb **rádióidőt** (time on air) eredményez. A **duty cycle** (adáskihasználtság) azt a szabályozási korlátot jelenti, amely megadja, hogy egy adott frekvenciasávban vagy alsávban az eszköz az idő mekkora hányadában adhat.

A LoRaWAN régiós szabályozásai – például az EU868 sávban az ETSI előírásai – jellemzően 1% vagy annál kisebb duty cycle-t írnak elő egyes **alsávokra**. Ez az érték **ellenőrzési igényt** hordoz: a pontos számok a régiótól, az alsávtól és a szabályozás aktuális verziójától függenek. A következő számításban ezért illusztratív értéket használunk.

A rádióidő nem lineárisan arányos az SF-fel: a szimbólumidő 2^{SF}-fel nő, a payload szimbólumszáma pedig a payload méretétől, az SF-től és a kódolási rátától függ. A pontos LoRa idő

a levegőben képletet a Rádiós technológiák összevetése alfejezet mutatta be. Ha egy eszköz rádióideje T_{air}, és az adott szabályozott alsávban megengedett duty cycle d, akkor a minimális periódus:

$$T_{\text{min}} = \frac{T_{\text{air}}}{d}$$

Ebből a maximális üzenetgyakoriság:

$$f_{\text{max}} = \frac{1}{T_{\text{min}}} = \frac{d}{T_{\text{air}}}$$

A gyakorlatban az eszköz több csatornát is használhat, de az ugyanabba a szabályozott alsávba eső csatornák adásideje összeadódik; a csatornaváltás nem nullázza a felhasznált keretet. Az EU868 alapértelmezett 868,1/868,3/868,5 MHz-es csatornái például jellemzően ugyanabba a 868,0–868,6 MHz-es alsávba esnek, ezért a három csatorna együttes adásidejére kell a korlátot betartani. A különböző alsávok külön kerettel rendelkeznek, és a hálózati szerver fair-use szabálya ettől független, további korlát lehet.

9.4.3 Döntési szempontok

A LoRaWAN akkor a legjobb választás, ha:

- nagy hatótávra van szükség (1–10 km, nyílt terepen),
- az adatmennyiség kicsi (napi néhány tucat üzenet, egyenként néhány tíz bájt),
- az eszköz elemes, és az energiafogyasztás kritikus,
- a késleltetés másodperces vagy perces nagyságrendje elfogadható.

Nem jó választás, ha:

- nagy sávszélesség kell (például kép vagy hang átvitele),
- azonnali downlink parancsokra van szükség,
- az eszköz hálózati táplálású, és a fogyasztás nem szempont,
- az adatforgalom sűrű, mert a duty cycle korlátokba ütközünk.

A Rádiós technológiák összevetése alfejezetben részletesen összehasonlítottuk a LoRaWAN-t a BLE, Wi-Fi és cellás megoldásokkal. A TerraSense alapértelmezett laborhálózatában a rövid hatótávú BLE/UDP megoldást használjuk, de a LoRaWAN-t alternatív uplinkként tartjuk nyilván azokra a telephelyekre, ahol a `ts-gw` nem telepíthető a csomópont közelébe.

Továbbvitt eredmény. A LoRaWAN mint alternatív uplink döntését a TerraSense-ben a következőképpen rögzítjük: ha egy telephelyen nincs helyi `ts-gw`, a `ts-node` LoRaWAN Class A végberendezésként, OTAA aktiválással csatlakozik egy hálózati szerverhez. Az adat az application serveren keresztül MQTT-topicra kerül, és onnan a meglévő `ts-backend` ingest útvonalat használja. Ezt a lehetőséget a gateway szerepe: híd, puffer, biztonsági határ és a záróprojekt: a teljes TerraSense rendszer összeállítása és átvétele alfejezetek használják fel.

Az alábbi táblázat összefoglalja a LoRaWAN osztályok és aktiválási módok közötti választást.

Szempont	Class A	Class B	Class C	OTAA	ABP
Downlink késleltetés	A következő uplink ablakáig (nagy, nem garancia)	Ütemezett rések (közepes)	Azonnali (kicsi)	Első csatlakozáskor join szükséges	Nincs join, azonnal küldhet
Energiafogyasztás	Legalacsonyabb	Közepes	Magas	Join alatt kicsit több	Nincs join többlet
Tipikus alkalmazás	Elemes szenzor	Elemes eszköz, időzített parancsok	Hálózati táplálású aktuátor	Biztonságos, skálázható flotta	Zárt, egyszerű hálózat
Biztonság	Jó	Jó	Jó	Kulcsrotáció lehetséges	Statikus kulcsok, frame counter kockázat

9.4.4 Kidolgozott példa: maximális üzenetgyakoriság a duty cycle-ből

Tegyük fel, hogy a TerraSense egyik távoli üvegházában a `ts-node` LoRaWAN Class A eszközként működik az EU868 sávban. Az eszköz SF7 spreading faktort használ 125 kHz sávszélességgel, a payload mérete 10 bájt. A rádióidő ezzel a beállítással körülbelül $T_{\text{air}} = 50$ ms (illusztratív érték, a pontos szám a LoRa moduláció fejléc- és CRC-paramétereitől függ). A régiós szabályozás az adott szabályozott alsávra – például a 868,0–868,6 MHz-es alsávra – $d = 0{,}01$ (1%) duty cycle-t ír elő (ellenőrizendő az aktuális ETSI szabályozásban).

A minimális periódus az alsávban összesített adásidőre:

$$T_{\text{min}} = \frac{T_{\text{air}}}{d} = \frac{0{,}05\ \text{s}}{0{,}01} = 5\ \text{s}$$

A maximális üzenetgyakoriság:

$$f_{\text{max}} = \frac{d}{T_{\text{air}}} = \frac{0{,}01}{0{,}05\ \text{s}} = 0{,}2\ \text{üzenet/s}$$

Ez azt jelenti, hogy az eszköz az adott alsávban összesen legfeljebb 50 ms-ot adhat 5 másodpercenként, ami 50 ms-os uplink esetén átlagosan 5 másodperces küldési periódusnak felel meg. Ez a keret a csatornaváltással nem növelhető, ha a csatornák ugyanabba az alsávba esnek. A TerraSense 10 perces mérési ciklusa ($T = 600$ s) ennél két nagyságrenddel ritkább: egy 50 ms-os uplink a 600 s-os periódus 0,0083%-át használja, tehát a duty cycle korlát nem szűk keresztmetszet ebben az alkalmazásban.

Fontos megjegyezni, hogy a duty cycle az **alsávra** vonatkozik, nem az egyes csatornákra és nem az eszközre. Az ugyanabba az alsávba eső csatornák adásideje összeadódik, ezért a véletlenszerű csatornaváltás nem nullázza a felhasznált keretet. A különböző alsávok külön korláttal rendelkeznek, és a hálózati szerver fair-use szabálya (például egy közösségi hálózat napi adásidő-kerete) ettől független, további korlát lehet. A tervezés során a legszigorúbb érintett alsávra és a szerver fair-use szabályára is ellenőrizni kell a rádióidőt.

9.4.5 Ellenőrzés és hibakeresés

A LoRaWAN eszközök duty cycle betartását a network server naplózza: minden frame-hez tartozik egy időbélyeg és a rádióidő. A The Things Stack vagy más LoRaWAN hálózati szerver konzolján ellenőrizhető, hogy az eszköz adásai megfelelnek-e a régiós szabályoknak. A gatewayek is jelentik a vett jel erősségét (RSSI) és a jel-zaj viszonyt (SNR), ami a hatótáv és a spreading factor beállításának ellenőrzésére használható.

Hibatünet	Lehetséges ok	Javítás
A hálózati szerver nem fogadja a frame-eket	Túl gyakori adás, duty cycle túllépés	Növeljük a periódust, vagy váltsunk alacsonyabb SF-re, ha a hatótáv engedi
Az eszköz nem csatlakozik OTAA-val	Hibás AppKey vagy JoinEUI	Ellenőrizzük a regisztrált eszközazonosítókat és kulcsokat a network serverben
ABP eszköz újraindítás után nem működik	A frame counter nem lett perzisztensen mentve	Tároljuk a frame countert nem felejtő memóriában, vagy váltsunk OTAA-ra
Nagy SF mellett is rossz a hatótáv	Antenna probléma, árnyékolás, rossz gateway-pozíció	Ellenőrizzük az antennát, helyezzük magasabbra a gatewayt

9.4.6 Tipikus hibák és félreértések

1. **ABP használata frame counter mentése nélkül.** Ha az eszköz újraindul és a frame counter lenullázódik, a network server ismétlésnek vagy visszajátszásnak érzékeli a frame-eket, és elutasítja azokat. A frame countert nem felejtő memóriában kell tárolni, vagy egyszerűbb OTAA-t használni, ahol a join új kulcsokat generál.
2. **Class A eszköznél azonnali downlink elvárása.** Sok fejlesztő azt feltételezi, hogy a parancs azonnal eljut az eszközhöz, mint egy TCP-kapcsolatnál. A Class A eszköz csak az uplink utáni rövid RX1/RX2 ablakban hallgat, ezért a parancs legkorábban a következő uplink után érkezhet meg, és ez csak fogadási lehetőség, nem kézbesítési garancia. Uplink- vagy downlinkvesztés esetén a parancs a következő periódusban próbálható újra, ezért TTL-t, újrapróbálkozást és helyi biztonsági zárást kell tervezni. Ha másodperces garanciájú vezérlés kell, Class C vagy más technológia szükséges.
3. **A duty cycle figyelmen kívül hagyása.** A LoRaWAN sávok szigorú szabályozás alatt állnak; a túlzott adáskihasználtság a hálózati szerver frame-eldobásához vezethet, és akár jogi következményei is lehetnek. Az ugyanabba a szabályozott alsávba eső csatornák adásideje összeadódik, ezért a csatornaváltás nem ad több keretet. A gyakori adatküldés igénye esetén növelni kell a periódust, csökkenteni kell a rádióidőt, vagy más technológiát kell választani.
4. **A spreading factor növelése mindig javítja a rendszert.** Bár a nagyobb SF nagyobb hatótávot ad, a rádióidő is nő, ami növeli az energiafogyasztást és csökkenti a hálózati kapacitást. A cél a lehető legkisebb SF, amely még megbízható kapcsolatot ad.

9.4.7 Feladatok

9.4.7.1 (a) Fogalmi kérdés

Magyarázza el, miért nevezzük a LoRaWAN-t csillag-csillag topológiájúnak, és mi a különbség a network server és az application server szerepe között.

9.4.7.2 (b) Számolási feladat

Egy LoRaWAN eszköz az EU868 sávban SF7, 125 kHz beállítással 20 bájt payloadot küld. A rádióidő körülbelül 80 ms (illusztratív). A duty cycle korlát 1%. Számítsa ki a minimális periódust és a maximális üzenetgyakoriságot az adott szabályozott alsávra összesítve, és magyarázza el, miért nem növeli a keretet a csatornaváltás, ha a csatornák ugyanabba az alsávba esnek.

9.4.7.3 (c) Implementációs feladat a TerraSense-ben

A TerraSense távoli üvegházában a `ts-node` LoRaWAN-t használ, és a szelepet is vezérelnie kell. Válasszon eszközosztályt (Class A/B/C) és aktiválási módot (OTAA/ABP), és indokolja a döntést. Térjen ki arra, hogyan jut el a szelepnyitási parancs az eszközhöz.

9.4.7.4 (d) Hibakeresési feladat

Egy ABP-vel aktivált LoRaWAN eszköz működik, de áramszünet után újraindul, és ezután a hálózati szerver nem fogadja az uplinkjeit. Mi lehet az ok, és hogyan javítható?

9.4.8 Megoldások

9.4.8.1 (a)

A csillag-csillag topológia azt jelenti, hogy az end device-ok rádiós csillagban kapcsolódnak a gatewayekhez (minden eszköz közvetlenül a gatewayhez, nem egymáshoz), a gatewayek pedig IP-hálózaton egy másik csillagban a network serverhez. A **network server** a LoRaWAN hálózati rétegét kezeli: munkamenet-állapot, `DevAddr`, frame counterek, MIC-ellenőrzés, duplikátumszűrés, adaptív adatsebesség és downlink-ütemezés. A hálózati kulccsal (`NwkSKey`) a hálózati réteg integritását és a hálózati mezőket védi, de az application payloadot nem neki kell visszafejtenie. Az **application server** az application payloadot az `AppSKey` kulccsal fejti vissza, dekódolja az alkalmazás sémáját, továbbítja az alkalmazásnak, és fogadja az alkalmazás parancsait. A LoRaWAN 1.1-ben a **join server** a gyökérkulcsokból származtatja és külön osztja szét a két munkamenetkulcsot, ezért a network server nem kapja meg automatikusan az application payload kulcsát. A LoRaWAN 1.0.x-ben a kulcsszármaztatás egyszerűbb, és a network server jellemzően mindkét kulcsot ismeri; ilyenkor a hálózati és alkalmazási szerep szétválasztása üzemeltetési és szerződéses, nem kriptográfiai garancia. A MIC az integritást védi, az `AppSKey` pedig az alkalmazási adat bizalmasságát; a kettő külön kulcson és külön szerepkörön alapul.

9.4.8.2 (b)

A minimális periódus:

$$T_{\min} = \frac{T_{\text{air}}}{d} = \frac{0{,}08 \text{ s}}{0{,}01} = 8 \text{ s}$$

A maximális üzenetgyakoriság:

$$f_{\max} = \frac{1}{8 \text{ s}} = 0{,}125 \text{ üzenet/s}$$

Tehát az eszköz az adott alsávban összesítve legalább 8 másodpercenként küldhet, azaz legfeljebb 0,125 üzenetet másodpercenként. Ha a három alapértelmezett csatorna ugyanabba az alsávba esik, a csatornák közötti váltás nem növeli a keretet: a 80 ms-os adások együttes idejére kell a 8 másodperces periódust betartani.

9.4.8.3 (c)

A `ts-node` elemes szenzorcsomópont, ezért az energiafogyasztás kritikus. A szelep vezérlése nem igényel azonnali, másodpercnél pontosabb reakciót, ezért a **Class A** osztály elegendő: a parancs legkorábban a következő uplink utáni RX1 vagy RX2 ablakban érkezhet meg, ami legfeljebb 10 perces várakozást jelent. Ez azonban csak a következő fogadási lehetőség, nem kézbesítési garancia; elveszett uplink vagy downlink esetén a parancs nem érkezik meg. Ezért a parancsnak TTL-t kell adni, az application server a TTL-en belül újrapróbálkozhat, a szelep helyi `max_on_time` logikája pedig a parancs hiányában is zár. Aktiválási módként **OTAA**-t választunk, mert a join serverrel való újracsatlakozás kulcsrotációt és egyszerűbb flottakezelést tesz lehetővé. Ha később mégis gyorsabb downlink kellene, át lehet térni Class B-re az ütemezett ping résekkel, de az energiafogyasztás növekedésével számolni kell.

9.4.8.4 (d)

Az ok nagy valószínűséggel az, hogy a frame counter (`FCntUp`) nem lett perzisztensen tárolva, és az újraindulás után az eszköz nulláról kezdi a sorszámozást. A network server ezt visszajátszásnak vagy duplikátumnak érzékeli, és elutasítja a frame-eket. A javítás: a frame countert nem felejtő memóriában (például flashben) kell tárolni, és újraindulás után onnan kell folytatni. Hosszú távon jobb megoldás az OTAA-ra való áttérés, mert a join eljárás új kulcsokat és friss frame countereket generál.

9.4.9 Rövid összefoglaló

A LoRaWAN egy LPWAN protokoll, amely csillag-csillag topológiában nagy hatótávot és alacsony energiafogyasztást biztosít. Az end device-ok rádiós csillagban gatewayekhez kapcsolódnak, a gatewayek IP-n a network serverhez; a network server a hálózati réteget (munkamenet, frame counter, MIC, downlink-ütemezés) kezeli, az application server pedig az `AppSKey`-jel visszafejtett application payloadot dolgozza fel. A LoRaWAN 1.1-ben a join server osztja szét a hálózati és alkalmazási munkamenetkulcsot; 1.0.x-ben a network server jellemzően mindkét kulcsot ismeri. A három eszközosztály (A, B, C) a downlink-ablakok gyakoriságában tér el; a TerraSense elemes csomópontjához a Class A illik. A Class A csak a következő uplink utáni RX1/RX2 ablakban tud fogadni; ez a következő fogadási lehetőség, nem kézbesítési garancia. A tényleges késleltetés

a következő uplinkig tartó várakozás plusz az esetleges újrapróbálkozások ideje, ezért TTL, újrapróbálkozás és helyi biztonsági zárás kell. Az OTAA aktiválás biztonságosabb, mint az ABP, mert lehetővé teszi a kulcsrotációt. A spreading factor a szimbólumonkénti 2^{SF} chipet és a $T_{\text{sym}} = 2^{SF}/BW$ szimbólumidőt határozza meg, ezért a rádióidő az SF-fel exponenciálisan nő. A duty cycle szabályozási korlátot ad az adásgyakoriságra: $T_{\text{min}} = T_{\text{air}}/d$, és az ugyanabba a szabályozott alsávba eső csatornák adásideje összeadódik. A példában 50 ms rádióidő és 1% alsáv-duty-cycle mellett a minimális periódus 5 s, ami bőven lehetővé teszi a TerraSense 10 perces ciklusát. **Továbbvitt eredmény:** a LoRaWAN-t a TerraSense-ben alternatív uplinkként kezeljük: a `ts-node` Class A eszközként OTAA-val csatlakozik, az adat MQTT-n át a `ts-backend`-be jut; ezt a gateway szerepe: híd, puffer, biztonsági határ és a záróprojekt: a teljes TerraSense rendszer összeállítása és átvétele alfejezetek használják.

9.4.10 Források és ellenőrzési hiányok

A LoRaWAN protokoll alapfogalmai stabil tudás; a hivatalos specifikációt a LoRa Alliance adja ki (LoRaWAN Specification). A fejezet a LoRaWAN 1.1 szerepkiosztását követi: a join server a gyökérkulcsokból származtatja a `NwkSKey` és `AppSKey` munkamenetkulcsot, a network server a hálózati réteget és a MIC-et kezeli, az application server az `AppSKey`-jel fejti vissza az application payloadot. A LoRaWAN 1.0.x eltér: ott a network server jellemzően mindkét kulcsot ismeri, ezért a két szerep kriptográfiai szétválasztása nem garantált; ezt a szöveg külön jelzi. A rádióidő pontos számítása a LoRa moduláció paramétereitől (SF, sávszélesség, payload, coding rate, header, CRC) függ, ezért a példában használt 50 ms-os érték illusztratív. A duty cycle régiós értékei – például az EU868 868,0–868,6 MHz-es alsávjára vonatkozó 1% – az aktuális ETSI szabályozásból (EN 300 220 sorozat) ellenőrizendők; a pontos korlátok alsávonként, országonként és a szabályozás verziója szerint eltérhetnek, és a hálózati szerver fair-use szabálya külön korlát. A The Things Stack vagy más LoRaWAN hálózati szerver naplózási és konfigurációs funkcióinak nevei szintén ellenőrzést igényelnek. A fejezet számításai statikus ellenőrzéssel készültek; éles LoRaWAN-adás vagy hálózati szerveres mérés nem történt.

9.5 IP az eszközön: IPv6, 6LoWPAN, Thread és a Zephyr hálózati stack

9.5.1 Motiváció

A TerraSense `ts-node` eddig emulált szenzorait a saját processzén belül olvasta, de ahhoz, hogy az adat eljusson a gatewayhez vagy a felhőbe, az eszköznek hálózati kapcsolatra van szüksége. Amikor a rádió közvetlenül IP-alapú, a fejlesztő megszokott socket-absztrakciót használhat; amikor nem, protokollfordító gateway kell. Ez az alfejezet azt mutatja meg, hogyan kaphat egy korlátos mikrovezérlő IPv6-címet 6LoWPAN-on vagy Threaden, és hogyan nyithatunk Zephyrben BSD-szerű UDP-socketet, amely `native_sim`-en a Linux hosttal kommunikál.

9.5.2 Fogalmak és működés

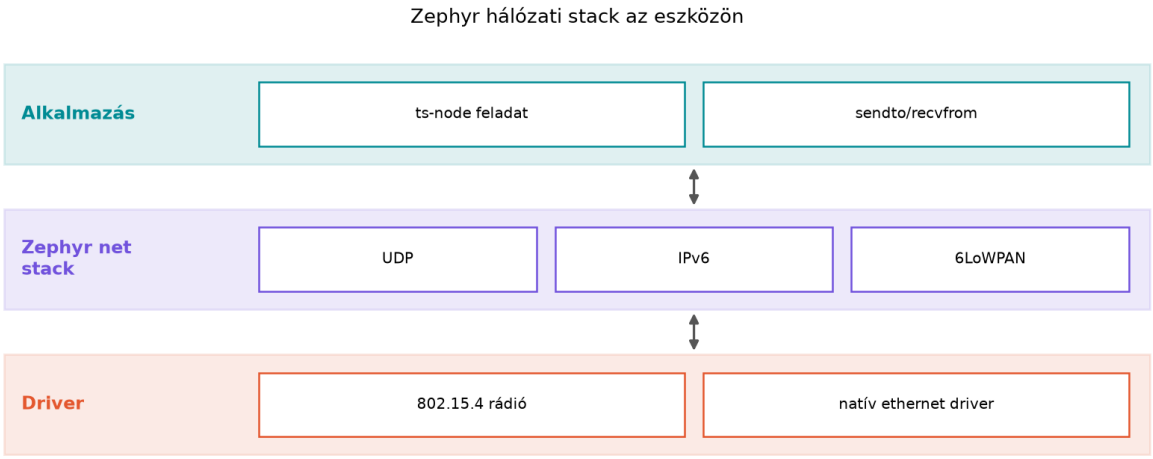
Definíció. Az **IPv6** a 128 bites címzésű hálózati protokoll (RFC 8200). Minden interfész rendelkezik link-local `fe80::/10` címmel; a globális címet általában router advertisement (RA) hirdeti meg, vagy statikusan adjuk meg.

Definíció. A **6LoWPAN** (IPv6 over Low-Power Wireless Personal Area Networks, RFC 6282) IPv6-csomagok szállítását teszi lehetővé IEEE 802.15.4 rádió felett fejléc-tömörítéssel és szükség

esetén fragmentálással, mert a 802.15.4 fizikai keret hasznos része sokszor mindössze 81–127 bájt nagyságrendű, szemben az IPv6 40 bájtos alapfejlécével.

Definíció. A **Thread** egy IPv6-alapú, alacsony fogyasztású mesh topológiájú hálózati protokoll, amely a 6LoWPAN-t használja a 802.15.4 fölött, és a csomópontok közötti útvonalakat automatikusan építi fel. A **border router** az a csomópont, amely a Thread mesh-t összeköti a nagyobb IP-hálózattal, például a helyi Ethernet/Wi-Fi hálózattal.

A 6LoWPAN fejléc-tömörítés lényege, hogy a link-local prefixeket, az ismert interfész-azonosítókat és a nullázható mezőket nem kell minden csomagban elküldeni. Például ha a forrás- és célcím is link-local, a forgalmi osztály és a flow label nulla, akkor a 40 bájtos IPv6 fejléc akár 2 bájtra csökkenthető. A UDP 8 bájtos fejléce is tömöríthető: a portszámokból csak a változó részt kell átvinni. A checksum elhagyása (RFC 6282 4.3.2) csak szigorú feltételekkel engedett: a linkrétegnek a teljes IPv6-datagramra kiterjedő integritásvédelmet kell adnia (például 802.15.4 FCS), és a fogadó félnek a 6LoWPAN-fejléc alapján rekonstruálnia kell a hiányzó UDP-checksumot; ha ez nem teljesül, a checksumot meg kell tartani. A fejezet laborpéldájában (UDP socket `native_sim` Ethernet fölött) a checksumot a Zephyr stack alapértelmezésben számolja, ezért a kezdő példában nem hagyjuk el. Az alábbi rétegábra a Zephyr hálózati vermet mutatja, amelyben a socket API az alkalmazás felől ugyanaz, míg alatta a rádióhoz igazodó adaptációs réteg dolgozik.



9.7. ábra: Zephyr hálózati stack az eszközön - A socket API elrejti a rádió- és adaptációs rétegek különbségeit.

Az alkalmazás tehát `socket()` hívással dolgozik; a Zephyr hálózati stack a `CONFIG_NET_*` Kconfig kapcsolókkal kapcsolható be, a driver pedig eldönti, hogy a keret ténylegesen 802.15.4, Thread vagy emulált Ethernet közegen megy-e ki. A Zephyr socket API BSD-szerű: a `socket`, `bind`, `connect`, `sendto`, `recvfrom`, `close` függvények neve és alapszemantikája megegyezik a Linux socketekkel, de a háttérben nincs külön címtér, a hálózati verem fut a mikrovezérlő processzén, és a pufferek, a csatlakozások száma, valamint a szálak veremigénye fordítási időben, Kconfiggal korlátozott.

9.5.3 Döntési szempontok

A fő kérdés: érdemes-e IP-t juttatni az eszközre, vagy inkább a gateway fordítson? A táblázat a TerraSense-hez is használható szempontokat foglalja össze.

Szempont	IPv6/6LoWPAN/Thread az eszközön	Nem-IP rádió (BLE, LoRaWAN) + gateway
Végpont-végpont biztonság	A TLS/DTLS az eszköz és a végpont között fut; a köztes IP-hopok csak továbbítják a csomagot	A rádiós link hop-by-hop védelme és az alkalmazási end-to-end titkosítás külön réteg; a gateway csak akkor látja a payloadot, ha szándékosan visszafejti
Energiaigény	nagyobb, folyamatosabb rádióaktivitás	kisebb lehet (pl. LoRaWAN)
Flash/RAM igény	magasabb (IP stack)	alacsonyabb
Hálózati topológia	mesh, öngyógyuló (Thread)	jellemzően csillag
Szabványos alkalmazásprotokoll (MQTT, CoAP)	közvetlenül fut az eszközön	gateway kell
Címzési modell	minden eszköznek saját IP-címe van	gateway mögötti azonosító

A Thread akkor előnyös, ha több tucatnyi, elemes eszköznek kell kis energiafogyasztású meshben, IPv6-végpontként működnie, és van egy border router a telephelyen. A TerraSense üvegházához például Thread mesh illik, ha a csomópontok egymás között is át tudják adni a csomagokat; ha viszont csak néhány BLE-szenzor van, a BLE + `ts-gw` egyszerűbb lehet. A Zephyr socket API akkor is ugyanúgy használható, ha a rádió IP-t beszél; a nem-IP rádiók esetében a protokollfordítás a gateway feladata.

A biztonsági rétegeket érdemes szétválasztani. A **rádiós linkvédelem** (BLE párosítás és titkosítás, 802.15.4 linkkulcs) csak a szomszédos hopot védi. A **hop-by-hop TLS** (például gateway→broker) a gateway és a broker közötti szakaszt védi. Az **alkalmazási end-to-end védelem** (az eszköz által titkosított payload, amelyet a gateway változatlanul továbbít) akkor is véd, ha a gateway vagy egy köztes hop nem megbízható. Nem-IP rádióknál a gateway protokollfordítást végez; az, hogy a payloadot visszafejti-e, tudatos tervezési döntés: a gateway értelmezheti csak a fejlécet és továbbíthatja a titkosított payloadot, vagy végpontként viselkedhet. A TerraSense alapértelmezett láncában a gateway a telemetriát MQTT-re fordítja, ezért az eszköz–backend end-to-end védelem külön alkalmazási réteg (például payload-szintű titkosítás vagy az eszköz és a backend közötti DTLS/TLS csatorna), nem azonos a gateway és a broker közötti TLS-sel.

9.5.4 Labor: `ts-node` UDP-üzenet a hostra

Munkakönyvtár:

```
mkdir -p ~/iot-labor/i025/src
cd ~/iot-labor/i025
```

A labor a Zephyr `native_sim` boardot használja, és a `ts-node` processz egy IPv6-os UDP-socketről küld `hello from ts-node` üzenetet a Linux hostra. Az előző Devicetree és Kconfig: a hardver és a konfiguráció leírása alfejezet `prj.conf`-ját bővítjük hálózati beállításokkal.

A `CMakeLists.txt`:

```
cmake_minimum_required(VERSION 3.20)
find_package(Zephyr REQUIRED HINTS $ENV{ZEPHYR_BASE})
project(ts_node_udp)

target_sources(app PRIVATE src/main.c)
```

A `prj.conf`:

```
CONFIG_NETWORKING=y
CONFIG_NET_IPV6=y
CONFIG_NET_UDP=y
CONFIG_NET_SOCKETS=y
CONFIG_NET_SOCKETS_POSIX_NAMES=y
CONFIG_NET_LOG=y
CONFIG_NET_L2_ETHERNET=y
CONFIG_ETH_NATIVE_POSIX=y
CONFIG_NET_CONFIG_SETTINGS=y
CONFIG_NET_CONFIG_NEED_IPV6=y
CONFIG_NET_CONFIG_MY_IPV6_ADDR="2001:db8::2"
CONFIG_NET_CONFIG_PEER_IPV6_ADDR="2001:db8::1"
CONFIG_SHELL=y
CONFIG_NET_SHELL=y
```

A Zephyr 3.7.0-ban a `CONFIG_ETH_NATIVE_POSIX` választja ki a natív Ethernet drivert: ez a hoston már létező `zeth` TAP-interfészhez csatlakozik (`CONFIG_ETH_NATIVE_POSIX_DRV_NAME` alapértéke `zeth`, a `CONFIG_ETH_NATIVE_POSIX_DEV_NAME` pedig `/dev/net/tun`); az interfészt tehát nem a Zephyr hozza létre, hanem a hostnak kell előkészítenie. A `CONFIG_NET_CONFIG_*` beállítások adják a Zephyr-oldali statikus IPv6-címet: az eszköz a `2001:db8::2`, a host a `2001:db8::1` címet használja. A `CONFIG_NET_SOCKETS_POSIX_NAMES` teszi elérhetővé a `socket`, `sendto`, `close`, `inet_pton` POSIX-neveket (ezek a `zsock_*` függvényekre képződnek le), a `CONFIG_SHELL` és a `CONFIG_NET_SHELL` pedig a `net iface` és `net route` ellenőrző parancsokat adja. A `CONFIG_NET_CONFIG_MY_IPV6_ADDR` és `CONFIG_NET_CONFIG_PEER_IPV6_ADDR` nevek a `subsys/net/lib/config/Kconfig` fájlban szerepelnek; a korábbi `CONFIG_NET_IPV6_PEER_ADDR` nem létező szimbólum volt.

A `src/main.c`:

```
#include <zephyr/kernel.h>
#include <zephyr/net/socket.h>
#include <errno.h>
#include <stdio.h>
#include <string.h>

#define PEER_IP "2001:db8::1"
#define PEER_PORT 4242

int main(void)
{
    int sock;
    struct sockaddr_in6 addr;
    const char *msg = "hello from ts-node";

    sock = socket(AF_INET6, SOCK_DGRAM, IPPROTO_UDP);
    if (sock < 0) {
        printf("socket failed: errno=%d\n", errno);
        return 0;
    }

    memset(&addr, 0, sizeof(addr));
    addr.sin6_family = AF_INET6;
    addr.sin6_port = htons(PEER_PORT);
```

```
    if (inet_pton(AF_INET6, PEER_IP, &addr.sin6_addr) != 1) {
        printf("inet_pton failed\n");
        close(sock);
        return 0;
    }

    printf("ts-node UDP sender started, peer [%s]:%d\n", PEER_IP, PEER_PORT);

    while (1) {
        ssize_t sent = sendto(sock, msg, strlen(msg), 0,
                              (struct sockaddr *)&addr, sizeof(addr));
        if (sent < 0) {
            printf("sendto failed: errno=%d\n", errno);
        } else {
            printf("sent %zd bytes\n", sent);
        }
        k_msleep(5000);
    }
}
```

A `strlen` miatt a `string.h`-t is be kell húzni, a `printf` a Zephyr konzoljára ír. A POSIX-neveken hívott socket-függvények hiba esetén `-1`-et adnak vissza, a hiba okát pedig az `errno` tartalmazza; ezért nem a visszatérési értéket, hanem az `errno`-t érdemes kiírni és értelmezni. Fontos különbség, hogy a Zephyr kernel-API-k (például a `k_msgq_put()` vagy a `k_thread_create()`) jellemzően negatív errnóval térnek vissza, míg a POSIX socket-hívások `-1`-et adnak és az `errno`-t állítják; a két hibajelzési módot ne keverjük. Gyakori értékek: `EINVAL` (22) – hibás család vagy címformátum; `ENETUNREACH` (101) – nincs útvonal a célhoz; `EHOSTUNREACH` (113) – a szomszéd nem érhető el; `EADDRNOTAVAIL` (99) – még nincs használható forrás-cím (például a DAD nem fejeződött be).

Build és futtatás:

```
west build -b native_sim -d build
```

A Zephyr 3.7 `ETH_NATIVE_POSIX` driver nem hozza létre a TAP-interfészt, hanem egy már létező `zeth` interfészhez csatlakozik. Ezért először a hoston, root joggal létrehozzuk és felhúzzuk a TAP-ot, majd a felhasználó tulajdonába adjuk; így magát a szimulátort már nem kell rootként futtatni:

```
sudo ip tuntap add dev zeth mode tap user "$USER"
sudo ip link set dev zeth up
sudo ip -6 addr add 2001:db8::1/64 dev zeth
ip -6 addr show dev zeth
```

Az `ip -6 addr show` kimenetében látszania kell a `2001:db8::1/64` címnek és az interfész `UP` állapotának. A cím a `2001:db8::/64` prefixre kerül, ezért a hostnak a `2001:db8::2` Zephyr-cím felé on-link útvonala van a `zeth`-en; külön `ip -6 route add` nem kell. Ezután indítjuk a szimulátort:

```
./build/zephyr/zephyr.exe
```

A Zephyr konzoljában a `net iface` és a `net route` shell-paranccsal ellenőrizhető, hogy az Ethernet-interfészen megjelent a `2001:db8::2` cím, és a `2001:db8::/64` prefix on-link útvonalként szerepel. A hoston egy másik terminálban indítunk UDP-listener-t, és szükség esetén `tcpdump`-pal figyeljük a csomagutat:

```
nc -6 -u -l 4242
# opcionális, root joggal:
sudo tcpdump -i zeth -n -6 udp port 4242
```

A `zeth` létrehozására a Zephyr `net-tools` tárolójának `net-setup.sh` szkriptje is használható; a fenti `ip` parancsok ugyanazt a hatást érik el, és a `netcat`/`iproute2` csomagokon kívül nem igényelnek telepítést. Várt kimenet a hoston:

```
hello from ts-node
```

A Zephyr konzol logjában pedig öt másodpercenként:

```
sent 18 bytes
```

Az időzítés `native_sim`-en nem valós hardverhű; ez a labor a socket API és a hálózati útvonal működését demonstrálja, nem pedig tényleges rádiós időzítést.

9.5.5 Ellenőrzés és hibakeresés

A működés legegyszerűbb ellenőrzése a `netcat` kimenete; ha a host megkapta az üzenetet, akkor a Zephyr socket, a hálózati stack és a natív driver is működött. Ha a csomag nem érkezik meg, az alábbi táblázat segít.

Tünet	Ok	Javítás
A Zephyr nem indul, a `zeth` interfészre panaszkodik	A TAP nem létezik, nincs `UP` állapotban, vagy nem a futtató felhasználó tulajdonában van	`sudo ip tuntap add dev zeth mode tap user "$USER"`, majd `sudo ip link set dev zeth up`; ellenőrizd a `CONFIG_ETH_NATIVE_POSIX=y` beállítást
A `net iface` nem mutat IPv6-címet	A `CONFIG_NET_CONFIG_*` beállítás hiányzik, vagy az Ethernet driver nem indult el	`CONFIG_NET_CONFIG_SETTINGS=y` és `CONFIG_NET_CONFIG_MY_IPV6_ADDR="2001:d` ellenőrizd a `zeth` link-up állapotát
`sendto failed: errno=22` (EINVAL)	A célcím nem érvényes IPv6, vagy a család nem AF_INET6	`inet_pton` visszatérési értékét és a `sin6_family` mezőt ellenőrizni
`sendto failed: errno=101` (ENETUNREACH)	A host oldali cím nincs a `zeth` interfészre állítva, vagy nem azonos a prefix	`sudo ip -6 addr add 2001:db8::1/64 dev zeth`, majd `ip -6 route show dev zeth`
A host nem kap csomagot, de a log szerint elküldte	A host oldali IP-cím nincs a `zeth` interfészre rendelve, a listener rossz porton figyel, vagy tűzfal szűri az UDP-portot	IP-cím és `nc -6 -u -l 4242` ellenőrzése, tűzfal engedélyezése
A build nem találja a hálózati szimbólumokat	A `prj.conf` hálózati kapcsolói hiányoznak	`CONFIG_NETWORKING=y`, `CONFIG_NET_IPV6=y`, `CONFIG_NET_SOCKETS_POSIX_NAMES=y`

9.5.6 Tipikus hibák és félreértések

1. **IPv6 link-local címhez nincs zone index.** Amikor egy alkalmazás `fe80::` címre küld, a Linux socketeknél meg kell adni a kimenő interfész azonosítóját (`fe80::1%zeth`). A példában globális `2001:db8::1` címet használunk, ezért nincs szükség zone indexre, de link-local kommunikációban ez gyakori hiba forrása.
2. **UDP-t megbízhatónak tekinteni.** A `sendto` sikere csak azt jelenti, hogy a csomag átadódott a hálózati stacknek; a csomag elveszhet, duplikálódhat vagy átrendeződhet. A TerraSense telemetriánál a monoton `seq` és a későbbi MQTT-kézbesítési garanciák oldják meg az at-least-once szemantikát.
3. **A hálózati stack inicializálását elfelejteni.** A Zephyrben a `socket()` hívás általában működik, ha a `CONFIG_NETWORKING` engedélyezett, de bonyolultabb beállításoknál (statikus IP, csatlakozás előtti várakozás, több interfész) explicit interfész- és IP-konfiguráció kell.

9.5.7 Feladatok

1. **Fogalmi kérdés.** Miért van szükség a 6LoWPAN fejléc-tömörítésre az IEEE 802.15.4 hálózatokon? Magyarázza el a 40 bájtos IPv6 fejléc és a 81 bájtos maximális hasznos terhelés viszonyát.
2. **Számítási feladat.** Egy 6LoWPAN-csomag 16 bájt alkalmazás-adatot visz. Számítsa ki a teljes hosszt IPv6 + UDP fejléccel tömörítés nélkül, majd ideális tömörítéssel, ha az IPv6 fejléc 2 bájtra, az UDP fejléc 4 bájtra csökken. Mennyi a megtakarítás százalékban?
3. **Implementációs feladat.** Egészítse ki a labort úgy, hogy a host egy UDP-választ küld vissza a `ts-node`-nak, és a `ts-node recvfrom`-mal fogadja, majd kiírja a választ. A hoston `nc -6 -u -l 4242` helyett például `socat` válaszoljon. A kódváltozást a `src/main.c`-ben tegye meg, és rögzítse az új függvényt.
4. **Hibakeresési/kiterjesztési feladat.** A `ts-node` logban `sendto failed: errno=22` jelenik meg. Milyen három lehetséges okot ellenőrizne először, és milyen sorrendben? Indokolja a sorrendet.

9.5.8 Megoldások

1. A 802.15.4 rádiókeret hasznos része tipikusan 81–127 bájt nagyságrendű. Az IPv6 fejléc önmagában 40 bájt, az UDP fejléc további 8 bájt, így már 48 bájt elmenne, mielőtt egyetlen alkalmazás-bájtot is átvinnénk. A fejléc-tömörítés az ismert prefixeket, az állandó mezőket és a checksumot elhagyja, ezért a 48 bájt akár 2–6 bájtra csökken, így marad hely hasznos adatnak.
2. Tömörítés nélkül: $40 + 8 + 16 = 64$ bájt. Ideális tömörítéssel: $2 + 4 + 16 = 22$ bájt. A megtakarítás: $(64 - 22)/64 \times 100 \approx 65{,}6\%$. Ezért a 6LoWPAN nem csak kényelem, hanem energiaszempontból is kritikus, mert minden fölösleges bájt rádióidőt jelent.
3. A `recvfrom` hozzáadása:

```
char buf[64];
struct sockaddr_in6 src;
socklen_t src_len = sizeof(src);
ssize_t recv = recvfrom(sock, buf, sizeof(buf) - 1, 0,
                        (struct sockaddr *)&src, &src_len);
if (recv >= 0) {
    buf[recv] = '\0';
    printf("got reply: %s\n", buf);
}
```

A fogadást érdemes külön szálba vagy a küldő ciklus elé tenni, mert a `recvfrom` blokkol. A host oldalon például `socat UDP6-LISTEN:4242,fork EXEC:'echo received'` használható. 4. Először a `sendto` paramétereit ellenőrizném: `sock` érvényes-e, az `addr` családja `AF_INET6`-e, és az `inet_pton` sikeres volt-e. A POSIX-socket hiba esetén `-1`-et ad vissza, a részletes okot az `errno` hordozza; a `22` (`EINVAL`) leggyakrabban a család vagy a cím formátumával kapcsolatos. Másodszor a Zephyr konfigurációt: engedélyezve van-e a `CONFIG_NET_IPV6`, az UDP és a `CONFIG_NET_SOCKETS_POSIX_NAMES`. Harmadszor a cél IP-cím érvényességét a host `zeth` interfészén: ha a `2001:db8::1` nincs hozzárendelve, a stack `ENETUNREACH` (101) hibát adhat. A három ellenőrzés sorrendje azért ez, mert a paraméterhiba a legvalószínűbb és leggyorsabban kizárható, a konfiguráció a következő szint, a hostoldali cím pedig csak azután értelmezhető, ha az eszköz már helyes címmel és socketkonfigurációval rendelkezik.

9.5.9 Rövid összefoglaló

Az alfejezet bemutatta, hogyan jut IP-kapcsolat egy korlátos eszközre IPv6, 6LoWPAN és Thread segítségével. A **6LoWPAN** fejléc-tömörítéssel csökkenti az IPv6/UDP fejlécek költségét a 802.15.4 rádiókon, a **Thread** pedig IPv6 mesh-hálózatot épít, amelyet border router köt össze a nagyobb hálózattal. A Zephyr BSD-szerű socket API-ja lehetővé teszi, hogy a `ts-node` `sendto`/`recvfrom` hívásokkal kommunikáljon, miközben a rádió- és adaptációs rétegeket a hálózati stack elrejti. A labor a `~/iot-labor/i025/` könyvtárban egy `native_sim` UDP-küldőt hozott létre, amely a host `2001:db8::1` címére küld `hello from ts-node` üzenetet. **Továbbvitt eredmény.** A `src/main.c` socket-inicializáló és üzenetküldő logikája, valamint a `prj.conf` hálózati beállításai alapját képezik a MQTT a gyakorlatban: Zephyr-kliens és Mosquitto és a CoAP: REST UDP fölött, Observe és blokkátvitel alfejezeteknek; az MQTT/CoAP laborok ugyanezt a socket-réteget fogják használni a TCP/UDP kapcsolatokhoz.

9.5.10 Források és ellenőrzési hiányok

- RFC 8200 (IPv6), RFC 6282 (6LoWPAN).
- Zephyr Project dokumentáció: Networking, socket API, `native_sim` board.
- Thread Group hivatalos leírása a Thread protokollról.
- A Zephyr 3.7.0 dokumentációja szerint a `native_sim` TAP-alapú Ethernet-driverje a `CONFIG_ETH_NATIVE_POSIX`; a driver a hoston már létező `zeth` interfészhez csatlakozik, ezért az interfészt a hostnak kell létrehoznia (kézzel vagy a `net-tools` `net-setup.sh` szkriptjével). A statikus címek a `CONFIG_NET_CONFIG_SETTINGS`, `CONFIG_NET_CONFIG_MY_IPV6_ADDR` és `CONFIG_NET_CONFIG_PEER_IPV6_ADDR` szimbólumokkal adhatók meg; ezek a `subsys/net/lib/config/Kconfig` fájlban szerepelnek.
- Ellenőrizendő: a `native_sim` hálózati driver és a `NET_CONFIG_*` szimbólumok viselkedése a telepített Zephyr-kiadásban, valamint a `nc` IPv6 UDP-módjának pontos kapcsolói az Ubuntu 24.04 rendszeren. A fejezet laborját statikusan ellenőriztük, éles TAP-os futtatást nem végeztünk.
- Ellenőrizendő: a `socat` és `nc` IPv6 UDP-módjának pontos kapcsolói az Ubuntu 24.04 rendszeren.

10 Üzenetkezelési protokollok

10.1 MQTT: publish/subscribe, broker, topicok

10.1.1 Motiváció

A TerraSense üvegházban több tucat `ts-node` mér hőmérsékletet, páratartalmat és talajnedvességet. Az adatokat egy központi backendnek rögzítenie kell, a riasztási szabályoknak azonnal értesülniük kell, a kezelői dashboardnak pedig friss értékeket kell mutatnia. Egyetlen adatforrásnak tehát több fogyasztója van. Ha minden eszköz közvetlenül, kérés–válasz stílusban kapcsolódna a backendhez, akkor a backendnek ismernie kellene minden eszköz címét, az eszközöknek pedig a backend elérhetőségét, és minden új fogyasztó belépése új kapcsolatokat igényelne. Ráadásul az eszközök gyakran alszanak, és nem várhatnak arra, hogy a backend éppen elérhető legyen.

Az MQTT (Message Queuing Telemetry Transport) ezt a problémát a **publish/subscribe** (publikálás/feliratkozás) modell segítségével oldja meg: az eszközök nem egymásnak küldenek üzenetet, hanem egy központi **brokernek**, aki a címzés helyett **topic** alapján továbbítja az üzenetet minden érdeklődő félnek. Ebben az alfejezetben megismerjük a modellt, a topic-hierarchiát és a wildcardokat, majd a TerraSense rendszerhez rögzítjük a konkrét topic-konvenciót, amelyet a későbbi MQTT-laborok, a gateway és a backend is használni fog.

10.1.2 Fogalmak és működés

Az MQTT egy alkalmazásrétegbeli, TCP fölött futó, alacsony szigorú üzenetkezelő protokoll. A résztvevők szerepei:

- **Client (kliens):** minden eszköz, szenzor, backend szolgáltatás, amely csatlakozik a brokerhez. A TerraSense-ben a `ts-node`, a `ts-gw` és a `ts-backend` is MQTT-kliens.
- **Broker (üzenetközvetítő):** az a központi szerver, amely fogadja a kliensek bejövő üzeneteit, és a topic-előfizetések alapján továbbítja azokat a többi kliensnek. A TerraSense rendszerben ez az Eclipse Mosquitto.
- **Topic (téma):** egy hierarchikus, UTF-8 karakterekből álló, `/` jellel szintekre tagolt címke, amelyre az üzenetet publikálják, vagy amelyre fel lehet iratkozni.
- **Publish (publikálás):** a kliens üzenetet küld egy adott topicra.
- **Subscribe (feliratkozás):** a kliens kifejezi a broker felé, hogy egy topicra (vagy topic-mintára) érkező üzeneteket meg szeretne kapni.

Definíció. A **publish/subscribe** (pub/sub) kommunikációs modellben a küldő (publisher) nem ismeri a fogadók (subscriber) kilétét; az üzeneteket egy topicra publikálja, a broker pedig az aktuális feliratkozások alapján kézbesíti minden érdeklődőnek. A feliratkozás aszinkron: a subscriber nem kérdez, hanem értesítést kap, amikor üzenet érkezik.

A pub/sub három fontos szétcsatolást biztosít:

- **Térbeli szétcsatolás:** a publikálónak nem kell tudnia a feliratkozók címét vagy létezését.

- **Időbeli szétcsatolás:** a publikáló akkor is elküldheti az üzenetet, ha a subscriber éppen nem csatlakozik; a broker az MQTT QoS beállításoktól és munkamenet-tulajdonságoktól függően tárolhatja, vagy eldobhatja az üzenetet.
- **Szinkronizációs szétcsatolás:** az üzenetküldés nem blokkolja a publishert, amíg a subscriber feldolgozza az adatot.

10.1.2.1 A broker szerepe

A broker a rendszer központi eleme. Feladatai:

1. **Kapcsolatkezelés:** fogadja a kliensek TCP-kapcsolatait, kezeli a CONNECT/CONNACK kézfogást, a keep-alive időzítőt és a lecsatlakozást.
2. **Topic-szűrés:** minden bejövő PUBLISH üzenet topicját összehasonlítja a kliensek által regisztrált topic-előfizetésekkel, beleértve a wildcardokat is.
3. **Továbbítás:** minden egyező előfizetésnek továbbítja az üzenetet, figyelembe véve az adott kliens által kért QoS-t.
4. **Munkamenet-állapot:** opcionálisan megőrzi a kliens session-jét (előfizetések, QoS 1/2 üzenetek állapota), ha a kliens tiszta munkamenet nélkül csatlakozik.
5. **Retained üzenetek kezelése:** a legutóbbi retained üzenetet eltárolja egy topicra, és új feliratkozónak azonnal elküldi.

A broker **nem** értelmezi a payloadot, nem módosítja a topicot, és nem végez tartalmi útválasztást. Az MQTT 5 bevezeti a shared subscription és a topic alias fogalmát is, de ezekre a későbbi alfejezetekben térünk ki.

10.1.2.2 Topic-hierarchia és wildcardok

A topic egy tetszőleges hosszúságú, de nem üres karakterlánc, amelyben a szinteket a `/` karakter választja el. Például:

```
terrasense/gh1/n001/telemetry
```

Ez négy szint: `terrasense`, `gh1`, `n001`, `telemetry`. A topicok **case-sensitive**ek, azaz `terrasense` és `TerraSense` különböző topicok. A topicokban szóköz, `+` és `#` karakter nem szerepelhet szintnévként, mert ezek speciális jelentésűek.

A feliratkozáskor **wildcard**okat használhatunk:

- **Single-level wildcard (`+`):** pontosan egy szintet helyettesít. Például `terrasense/+/n001/telemetry` illeszkedik minden `terrasense/gh1/n001/telemetry` és `terrasense/gh2/n001/telemetry` topicra, de nem illeszkedik `terrasense/gh1/n001/state`-re, mert az utolsó szint eltér.
- **Multi-level wildcard (`#`):** nulla vagy több szintet helyettesít, de csak a topic **utolsó szint-jeként** állhat. Például `terrasense/gh1/#` illeszkedik a `terrasense/gh1/n001/telemetry`, `terrasense/gh1/n001/state`, sőt maga a `terrasense/gh1` topicra is (nulla további szint), viszont nem illeszkedik `terrasense/gh2/n001/telemetry`-re.

A wildcardok használatának szabályai:

- A `#` önállóan is használható (`#` minden topicra illeszkedik), de ezt csak nagyon indokolt esetben szabad használni, mert a forgalom és a jogosultságkezelés átláthatatlanná válik.
- A `+` és a `#` csak feliratkozásban használható, **publikáláskor tilos**.
- A broker a feliratkozási szűrőket nem módosítja, de a `#` utáni további szintek (`foo/#/bar`) érvénytelenek.

Jó topic-tervezés:

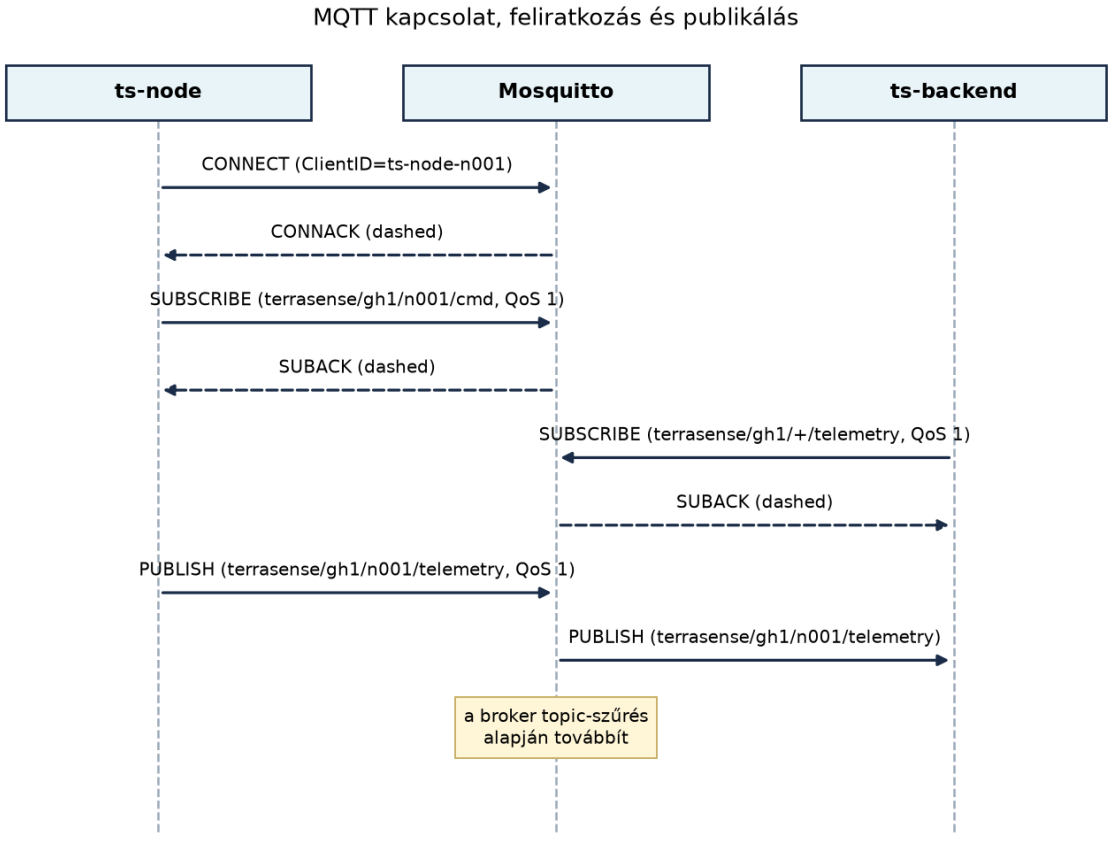
- A topicot balról jobbra specifikustól az általános felé, vagy fordítva, de **következetesen** építsük fel.
- Az első szint legyen a rendszert vagy a tenantot azonosító névtér.
- Az utolsó szint legyen az üzenettípus vagy a művelet, például `telemetry`, `state`, `cmd`.
- A wildcard előfizetéseket úgy alakítsuk ki, hogy egyetlen `+` vagy `#` csak egy jól meghatározott csoportot fedjen le.

Rossz példák:

- `gh1+n001` – a `+` nem használható szint közepén.
- `terrasense/` – üres szint a végén, értelmezhetetlen.
- `/terrasense/gh1` – vezető `/`, üres első szint.
- `terrasense/#/telemetry` – a `#` nem lehet közbülső.

10.1.2.3 Az MQTT kapcsolatfelépítés és üzenetcsere

Az alábbi diagram a TerraSense tipikus üzenetcseréjét mutatja: a `ts-node` felcsatlakozik a brokerre, feliratkozik a neki szóló parancsok topicjára, a `ts-backend` feliratkozik az összes telemetriára, majd a `ts-node` publikál egy telemetria-üzenetet.



10.1. ábra: MQTT kapcsolat, feliratkozás és publikálás - A ts-node és a ts-backend ugyanahhoz a Mosquitto brokerhez csatlakozik; a nyugták szaggatott vonallal jelölve.

Az ábra lépései:

1. A `ts-node` TCP-kapcsolatot nyit a brokerre, majd `CONNECT` csomagot küld a kliensazonosítóval (`ClientID`). A broker `CONNACK` csomaggal nyugtázza a kapcsolatot.
2. A `ts-node` `SUBSCRIBE` csomagot küld a `terrasense/gh1/n001/cmd` topicra, QoS 1-gyel. A broker `SUBACK` csomagban visszaadja a feliratkozás eredményét.
3. A `ts-backend` szintén `SUBSCRIBE` üzenetet küld, de a `terrasense/gh1/+/telemetry` wildcard topicra, így minden ezen a telephelyen lévő node telemetriáját megkapja.
4. A `ts-node` `PUBLISH` csomagban elküldi a telemetriát. A broker a topic-szűrők alapján továbbítja a `ts-backend`nek, mert annak előfizetése illeszkedik a topicra.

A payload formátumát ebben a fejezetben még nem szabjuk meg; a Adatformátumok: JSON, CBOR, Protocol Buffers, SenML alfejezetben fogjuk véglegesíteni. A lényeg most a címzési logika: a broker nem a payload alapján, hanem kizárólag a topic alapján dönt.

10.1.3 Döntési szempontok

Az MQTT pub/sub modellje nem minden esetben ideális. A kérés–válasz (például HTTP REST) és a pub/sub közötti választásnál a következő szempontokat érdemes mérlegelni.

Szempont	Kérés–válasz (pl. REST)	Publish/subscribe (MQTT)
Irány	A kliens kér, a szerver válaszol	A publisher egyirányúan küld, a broker szétoszt
Címzés	A kliens ismeri a szerver címét	A publisher nem ismeri a subscribereket
Több fogyasztó	Egy válasz egy kliensnek	Egy üzenet sok subscribernek
Időbeli csatolás	A szervernek elérhetőnek kell lennie a kéréskor	A broker pufferezhet (QoS > 0, session szerint)
Erőforrásigény	Nagyobb, ha sok eszköz folyamatosan polloz	Alacsony, mert nincs per-request HTTP overhead
Válasz szükségessége	Természetes	Külön reply-to topic + correlation ID szükséges

Az MQTT-t akkor válasszuk, ha:

- az eszközök **push** jelleggel küldenek adatot (telemetria, események),
- több fogyasztóra van szükség (backend, dashboard, riasztás),
- a hálózat megbízhatatlan, és az eszközök időnként offline-ok,
- a sávszélesség és az energia szűkös (a protokoll fejléc kicsi).

A kérés–választ akkor válasszuk, ha:

- egyetlen fogyasztó kér konkrét adatot, és a válasz azonnal szükséges,
- az üzenetváltás tranzakciós, és a válasz mindig kötelező,
- a rendszer már rendelkezik REST API-val, és az újrafelhasználás fontos.

A TerraSense-ben a rendszer magja MQTT, de a kezelői felület és a backend közti kommunikáció továbbra is REST (a A ts-backend váza: REST API és egészségellenőrzés alfejezetben). A kettő jól kiegészíti egymást: az eszközök MQTT-n küldenek, a felhasználók pedig HTTP-n kérdeznek.

10.1.4 Kidolgozott példa: a TerraSense topic-konvenció véglegesítése

Az A TerraSense mintarendszer architektúrája alfejezetben már bevezettük a `terrasense/<site>/<node>/...` alapstruktúrát. Most azt specifikáljuk pontosan, hogy a későbbi alfejezetek egyértelműen hivatkozhassanak rá. A konvenció a következő:

Topic	Irány	Tartalom	Retained?
`terrasense/<site>/<node>/telemetry`	eszköz → broker	mérési adatok (hőmérséklet, páratartalom, talajnedvesség)	nem
`terrasense/<site>/<node>/state`	eszköz → broker	aktuátorok és belső állapot (pl. szelep nyitva/zárva)	igen
`terrasense/<site>/<node>/cmd`	broker → eszköz	parancsok (pl. öntözés indítása)	nem
`terrasense/<site>/<node>/cmd/result`	eszköz → broker	parancs végrehajtás eredménye	nem
`terrasense/<site>/<node>/status`	eszköz → broker	jelenlét, kapcsolati állapot	igen (Last Will)

Azonosítók:

- `site`: a telephely rövid kódja, például `gh1` (1-es üvegház).
- `node`: az eszköz egyedi azonosítója, például `n001`.

A telemetria-payload a későbbi séma szerint fog `node_id`, `seq` és `ts` mezőket is tartalmazni, de ez a topic szempontjából nem releváns.

Továbbvitt eredmény. Ez a topic-konvenció a TerraSense rendszer végleges konvenciója. Ezt használja a MQTT QoS, munkamenetek, retain, Last Will és MQTT 5 alfejezet a QoS és retained beállításoknál, a MQTT a gyakorlatban: Zephyr-kliens és Mosquitto alfejezet a Zephyr-kliens implementációnál, a A ts-gw szolgáltatás C++-ban: protokollfordítás MQTT-re alfejezet a gateway MQTT-kimeneténél, valamint a Ingest: MQTT-ből a backendbe, idempotensen alfejezet a Spring Boot MQTT-betöltésénél.

Példa üzenetváltás wildcarddal:

Tegyük fel, hogy a `gh1` telephelyen három node (`n001`, `n002`, `n003`) működik. A `ts-backend` a `terrasense/gh1/+/telemetry` topicra iratkozik fel, így mindhárom node telemetriáját megkapja, de nem kapja meg a `state` vagy `cmd` üzeneteket. A riasztási szolgáltatás viszont a `terrasense/gh1/#` szűrőre iratkozik fel, hogy minden eseményt lásson. A `ts-gw` csak a saját node-jai parancsaira iratkozik fel, például `terrasense/gh1/+/cmd`, hogy minden node parancsát kézbesítse.

10.1.5 Labor: Mosquitto konzolkliensekkel, publish/subscribe és wildcardok

Ebben a laborban a Mosquitto broker és a hozzá tartozó `mosquitto_pub` és `mosquitto_sub` parancssori kliensek segítségével próbáljuk ki a pub/sub modellt. A labor célja, hogy megfigyeljük a broker topic-szűrését és a wildcardok működését.

Munkakönyvtár:

```
mkdir -p ~/iot-labor/i026 && cd ~/iot-labor/i026
```

Előfeltétel: Docker és Docker Compose v2 telepítve, valamint a 4. fejezetben létrehozott közös labor-stack (`~/iot-labor/stack/compose.yaml`) elérhető. A brokert nem indítjuk újra, és nem hozunk létre külön konténert: a stack `lab-broker` szolgáltatása (`lab-broker` konténer) már figyel a host 1883-as portján. Ellenőrizzük, hogy fut:

```
docker compose -f ~/iot-labor/stack/compose.yaml ps lab-broker
```

Ha még nem fut, indítsuk el a stack `lab-broker` szolgáltatását:

```
docker compose -f ~/iot-labor/stack/compose.yaml up -d lab-broker
```

A broker konfigurációja a stack `mosquitto/mosquitto.conf` fájlja. Ezt a labor nem módosítja, mert a későbbi fejezetek és a párhuzamos laborok ugyanezt a példányt használják; a hitelesítés kipróbálásához a (c) feladat megoldása külön, izolált példányt indít a 1884-es porton.

Telepítsük a `mosquitto-clients` csomagot, ha még nincs:

```
sudo apt update && sudo apt install -y mosquitto-clients
```

Most nyissunk egy új terminált, és iratkozzunk fel a `terrasense/gh1/#` topicra:

```
mosquitto_sub -h localhost -p 1883 -t 'terrasense/gh1/#' -v
```

A második terminálban publikáljunk egy telemetria-üzenetet:

```
mosquitto_pub -h localhost -p 1883 -t 'terrasense/gh1/n001/telemetry' -m
↪  '{"node_id":"n001","temp":23.5}'
```

A feliratkozó terminálban várhatóan megjelenik:

```
terrasense/gh1/n001/telemetry {"node_id":"n001","temp":23.5}
```

Most iratkozzunk fel egy szűkebb wildcarddal, például `terrasense/gh1/+/telemetry`, egy másik terminálban. Publikáljunk egy `status` üzenetet is, és figyeljük meg, hogy az első wildcard (`#`) megkapja, a második (`+` a telemetry-re) nem:

```
mosquitto_pub -h localhost -p 1883 -t 'terrasense/gh1/n001/status' -m 'online'
```

Az első feliratkozó (`terrasense/gh1/#`) ezt is megkapja, a második (csak telemetry) nem. Ez jól mutatja a wildcardok hatókörét.

Takarítás: A közös brokert nem állítjuk le, mert a többi labor és fejezet is használja. Ha csak a saját munkamenetünket fejezzük be, a `mosquitto_sub` folyamatot szakítsuk meg (`Ctrl+C`), a stack konténerét hagyjuk futni.

10.1.6 Ellenőrzés és hibakeresés

A fenti labor során ellenőrizhetjük, hogy a broker helyesen szűri-e a topicokat. Hasznos parancsok:

- `mosquitto_sub -v` kiírja a topicot is a payload elé.
- `mosquitto_sub -d` debug módban mutatja a kapcsolatfelvételt és a feliratkozást is.
- `docker logs lab-broker` megmutatja a broker naplóját, benne a klienscsatlakozásokat és a publikációkat.

Hibatünet	Lehetséges ok	Javítás
A `mosquitto_sub` nem kap üzenetet	A broker nem fut, vagy a port nem elérhető	Ellenőrizzük a `docker compose -f ~/iot-labor/stack/compose.yaml ps lab-broker` kimenetét, és a `-h localhost -p 1883` paramétereket
A `mosquitto_sub` kapcsolódik, de nem jelenik meg semmi	A topic kifejezés eltér, vagy a wildcard rossz	Ellenőrizzük a topic pontos helyesírását, használjuk a `-v` kapcsolót, és próbáljuk ki a `#` szűrőt
A `mosquitto_pub` hibát ír	A broker nem engedélyezi az anonim kapcsolatot	Ellenőrizzük a stack `mosquitto/mosquitto.conf` fájlját, majd indítsuk újra a brokert: `docker compose -f ~/iot-labor/stack/compose.yaml restart lab-broker`
A `#` wildcard nem működik	A `#` karaktert a shell értelmezi, vagy a topicban középen használtuk	Használjunk idézőjelet: `'terrasense/gh1/#'`, és győződjünk meg róla, hogy a `#` a topic végén áll

10.1.7 Tipikus hibák és félreértések

1. **A `#` wildcard használata publikáláskor:** a `#` és a `+` csak feliratkozásban érvényes; publikáláskor a MQTT-specifikáció tiltja. Ha mégis megpróbáljuk, a broker visszautasítja a kapcsolatot. Ez egy gyakori kezdő hiba, mert a felhasználó azt gondolja, hogy a wildcarddal „több helyre” publikálhat.
2. **A retained üzenet félreértése:** a retained flaggel jelölt üzenet a brokerben tárolódik, és minden új feliratkozó azonnal megkapja, még akkor is, ha a publikáló már lecsatlakozott. Ez hasznos a `status` topicnál, de ha nem tudatosan használjuk, elavult állapotot okozhat. A retained üzenet törléséhez üres payloadú retained üzenetet kell küldeni a topicra.
3. **A topicok esetén a helyes hierarchia elhanyagolása:** ha a topicokat nem következetesen tervezzük, a wildcard előfizetések nem fognak megfelelően működni. Például ha az egyik eszköz `gh1/n001/telemetry`, a másik `gh1_n001_telemetry`, akkor a `gh1/+/telemetry` nem illeszkedik a másodikra.

10.1.8 Feladatok

(a) Fogalmi kérdés. Magyarázza el, miért nem tud egy MQTT-kliens közvetlenül egy másik kliensnek üzenetet küldeni a broker megkerülésével. Hogyan lehet mégis kérés–válasz jellegű kommunikációt megvalósítani pub/sub fölött?

(b) Tervezési feladat. A TerraSense rendszerben egy új telephelyet (`gh2`) adunk hozzá, két nodedal (`n004`, `n005`). Írja fel az összes topicot, amelyre a `ts-backend`nek fel kell iratkoznia ahhoz, hogy megkapja: (i) minden telemetriát mindkét telephelyről, (ii) minden parancs-eredményt, de csak a `gh2` telephelyről. Adja meg a wildcard kifejezéseket.

(c) Implementációs feladat. A laborban használt Mosquitto konfigurációt egészítse ki úgy, hogy a `ts-backend` által használt user (`backend`) és jelszó (`labor-jelszo-2024`, csak labor célra) hitelesítve legyen, és ne engedjen anonim hozzáférést. Tesztelje a `mosquitto_sub` és `mosquitto_pub` kliensekkel a bejelentkezést.

(d) Hibakeresési feladat. Egy fejlesztő a következő `mosquitto_sub` parancsot futtatja: `mosquitto_sub -t 'terrasense/#/cmd'`, de nem kap semmilyen üzenetet. Mi a hiba? Javítsa a parancsot, és magyarázza el, miért nem működik.

10.1.9 Megoldások

(a) Az MQTT-specifikáció szerint a kliensek nem címezhetnek meg közvetlenül más klienseket; minden üzenet a brokeren keresztül halad. Ennek oka a szétcsatolás fenntartása. Kérés–válasz jellegű kommunikáció úgy valósítható meg, hogy a kérés küldője egy válasz-topicot (`reply-to`) ad meg a payloadban vagy az MQTT 5 tulajdonságokban, a fogadó pedig erre a topicra publikálja a választ. A korrelációhoz egyedi azonosítót (`correlation ID`) is mellékelni kell.

(b) A `ts-backend` feliratkozásai:

- (i) Minden telemetria: `terrasense/+/+/telemetry` (két `+`, mert a site és a node is változó).
- (ii) Csak `gh2` telephely parancs-eredményei: `terrasense/gh2/+/cmd/result`.

A wildcard kifejezések a megadott szinteknek felelnek meg. Például a `terrasense/gh2/+/cmd/result` illeszkedik `terrasense/gh2/n004/cmd/result`-re, de nem illeszkedik `terrasense/gh1/n004/cmd/result`-re.

(c) A közös brokert nem konfiguráljuk át, mert azzal a többi labor elérhetetlenné válna. Készítsünk izolált hitelesítési kísérletet a 1884-es porton. Először hozzuk létre a jelszófájlt a Mosquitto image-ből futtatott `mosquitto_passwd` segítségével, így nem kell a hostra külön csomagot telepíteni:

```
mkdir -p ~/iot-labor/i026/auth && cd ~/iot-labor/i026/auth
docker run --rm --user "$(id -u):$(id -g)" -v "$PWD:/work" \
  --entrypoint sh eclipse-mosquitto:2 \
  -c 'touch /work/passwd && mosquitto_passwd -b /work/passwd backend labor-jelszo-2024'
```

A `--user` beállítás miatt a létrehozott `passwd` a host felhasználó tulajdona lesz. A `mosquitto_passwd -b` batch módban a jelszót a parancssorból veszi, ezért ez a megoldás csak labor célra alkalmas; éles környezetben interaktívan vagy titokkezelőből adjuk át a jelszót.

Ezután hozzuk létre a `mosquitto-auth.conf` fájlt:

```
listener 1884
allow_anonymous false
password_file /mosquitto/config/passwd
```

és a `compose.yaml` fájlt, amely a létrehozott `passwd` fájlt a konténerbe mountolja:

```
services:
  broker-auth:
    image: eclipse-mosquitto:2
    container_name: lab-broker-auth-i026
    ports:
      - "1884:1884"
    volumes:
      - ./mosquitto-auth.conf:/mosquitto/config/mosquitto.conf:ro
      - ./passwd:/mosquitto/config/passwd:ro
```

Indítsuk el az izolált brokert, majd a másik terminálban iratkozzunk fel:

```
docker compose up -d
mosquitto_sub -h localhost -p 1884 -u backend -P labor-jelszo-2024 -t 'terrasense/gh1/#' -v
```

A harmadik terminálban publikáljunk ugyanezzel a felhasználóval és jelszóval:

```
mosquitto_pub -h localhost -p 1884 -u backend -P labor-jelszo-2024 \
  -t 'terrasense/gh1/n001/telemetry' -m '{"node_id":"n001","temp":23.5}'
```

Rossz jelszóval a broker elutasítja a kapcsolatot (`Connection Refused: not authorised`). Ugyanezt kell kapnunk névtelen klienssel is, mert az `allow_anonymous false` beállítás miatt a broker nem enged hitelesítés nélküli kapcsolatot:

```
mosquitto_pub -h localhost -p 1884 \
  -t 'terrasense/gh1/n001/telemetry' -m '{"node_id":"n001"}'
```

A kísérlet után az izolált példányt állítsuk le:

```
docker compose down
```

A közös, 1883-as `lab-broker` érintetlen marad. A részletes hitelesítési beállítások a MQTT a gyakorlatban: Zephyr-kliens és Mosquitto alfejezetben lesznek.

(d) A hiba az, hogy a `#` wildcard a topic közepén áll: `terrasense/#/cmd`. A multi-level wildcard csak az utolsó szinten lehet. Helyes kifejezés például `terrasense/+/+/cmd` (ha a site és a node is tetszőleges), vagy ha minden `cmd`-re kíváncsi, akkor `terrasense/#` és szűrjön a kliens. Az eredeti parancsot a MQTT-specifikáció érvénytelen feliratkozási szűrőként utasítja el.

10.1.10 Rövid összefoglaló

Az MQTT a publish/subscribe modellre épül: a kliensek topicokra publikálnak, a broker pedig az aktuális feliratkozások alapján továbbítja az üzeneteket. A topicok `/` jellel tagolt hierarchikus címkék, amelyekben a `+` egy szintet, a `#` nulla vagy több szintet helyettesít feliratkozáskor. A pub/sub térbeli, időbeli és szinkronizációs szétcsatolást biztosít, de kérés–válaszhoz külön reply-to topicra és korrelációs azonosítóra van szükség. **Továbbvitt eredmény:** a TerraSense végleges topic-konvenciója: `terrasense/<site>/<node>/telemetry`, `state`, `cmd`, `cmd/result` és `status`; a `status` retained + Last Will használatú. Ezt a konvenciót használják a MQTT QoS, munkamenetek, retain, Last Will és MQTT 5, a MQTT a gyakorlatban: Zephyr-kliens és Mosquitto, a A ts-gw szolgáltatás C++-ban: protokollfordítás MQTT-re és a Ingest: MQTT-ből a backendbe, idempotensen alfejezetek. A laborban Mosquitto konzolkliensekkel ellenőriztük a publikálást, feliratkozást és a wildcardok működését.

10.1.11 Források és ellenőrzési hiányok

- MQTT 5.0 specifikáció: OASIS, `https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html`
- MQTT 3.1.1 specifikáció: OASIS, `https://docs.oasis-open.org/mqtt/mqtt/v3.1.1/mqtt-v3.1.1.html`
- Mosquitto dokumentáció: `https://mosquitto.org/documentation/`
- Az MQTT topic-szintaxis és wildcard-szabályok a fenti specifikációkból származnak, ezek stabil, jól ismert definíciók. A Mosquitto konténer pontos verzióját és a `mosquitto-clients` csomag elérhetőségét a labor környezetben telepítés előtt ellenőrizni kell. A laborban használt jelszó és felhasználónév kizárólag helyi, laborcélú; éles rendszerben nem használható.

10.2 MQTT QoS, munkamenetek, retain, Last Will és MQTT 5

10.2.1 Motiváció

A TerraSense üvegházban a `ts-node` percenként hőmérsékletet, páratartalmat és talajnedvességet küld MQTT-n. Ha a hálózat pillanatnyi kimaradása miatt elveszik egy mérés, az nem katasztrófa; ha viszont a backend kétszer dolgoz fel egy öntözőparancsot, a szelep kétszer nyit, felesleges vízfogyasztást okozva. Hasonlóan, ha a felhasználói felület lekérdezi az eszköz aktuális állapotát, nem szeretné megvárni a következő telemetriai üzenetet: az utolsó jelentett állapotnak azonnal rendelkezésre kell állnia. Végül az üzemeltetőnek látnia kell, ha egy csomópont váratlanul leállt – ehhez a brokernek a kapcsolat megszakadásakor automatikusan jeleznie kell.

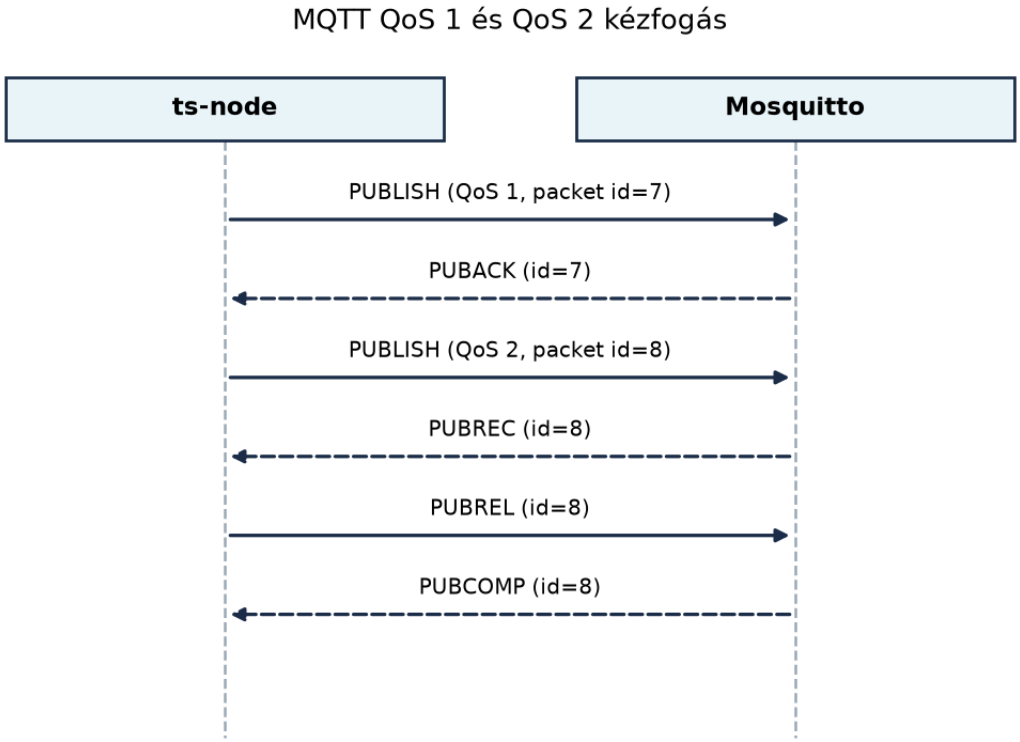
Ezeket a problémákat az MQTT kézbesítési garanciái (Quality of Service, QoS), a megőrzött üzenet (retained message) és a Last Will Testament (LWT) oldják meg. Az MQTT 5 ezen felül finomabb visszajelzéseket, tulajdonságokat és további szolgáltatásokat ad, amelyek a robusztus IoT-rendszerekben ma már gyakran alapkövetelmények.

10.2.2 Fogalmak és működés

Definíció (MQTT QoS). Az MQTT QoS a kiadványozótól a brokerig tartó, illetve a brokertől a feliratkozóig tartó üzenetkézbesítés garanciaszintje. A szabvány három szintet definiál:

- **QoS 0 – legfeljebb egyszer (at most once):** az üzenetet egyszer elküldik, nyugtázás nincs.
- **QoS 1 – legalább egyszer (at least once):** az üzenetet addig küldik, amíg meg nem érkezik a PUBACK; a hálózati hiba miatt az üzenet **duplikálódhat**.
- **QoS 2 – pontosan egyszer (exactly once):** négylépéses kézfogás (PUBLISH, PUBREC, PUBREL, PUBCOMP) biztosítja, hogy a broker logikailag egyszer kézbesíti az üzenetet a feliratkozónak.

Fontos különbség, hogy a QoS a két szakaszra (publisher→broker, broker→subscriber) **külön** értelmezett: a publisher QoS 2-vel küldhet, de a feliratkozó QoS 0-val kaphatja, ha úgy iratkozik fel. A broker a továbbított üzenet QoS-ét a publikált üzenet QoS-e és a feliratkozásban megkapott (granted) QoS közül a **kisebbikből** képzi: `min(publish QoS, granted subscription QoS)`. A SUBACK-ban a broker a kértnél alacsonyabb QoS-t is megadhat, például ha a topichoz csak QoS 0-t engedélyez; ezért a kliensnek a SUBACK tényleges értékét kell figyelembe vennie, nem a kérését.



10.2. ábra: MQTT QoS 1 és QoS 2 kézfogás - Illusztrálja a QoS 1 (kétlépéses) és QoS 2 (négylépéses) üzenetváltást; a szaggatott nyilak nyugták.

A diagramon látható: QoS 1 esetén a PUBACK elvesztése után a `ts-node` a DUP jelzővel újraküldi a PUBLISH-t, ez okozza a duplikációt. A QoS 2 négylépéses kézfogása ezt a hálózati szintű duplikációt kiküszöböli, de a feldolgozó oldali oldalhatás (pl. szelepnyitás) duplikálódását már nem tudja megakadályozni, ahogy később látni fogjuk.

Definíció (munkamenet, clean start). Az MQTT-kapcsolat állapota: a kliens és a broker által tárolt feliratkozások, ki nem kézbesített QoS 1/2 üzenetek és a klienshez tartozó munkamenet-flag. Az MQTT 3.1.1-ben a `cleanSession=true` azt jelenti, hogy a kapcsolat bontásakor a broker mindent eldob; `false` esetén a munkamenet megmarad, és a kliens újracsatlakozáskor folytathatja (offline üzenetek, meglévő feliratkozások). Az MQTT 5 a `clean start` fogalmat használja: minden csatlakozáskor eldönthető, hogy új munkamenetet kezdünk-e, a `session expiry` intervallum pedig azt szabja meg, hogy a broker a kapcsolat megszakadása után mennyi ideig őrizze meg a munkamenetet.

Definíció (retained message). A broker az utolsó retained flaggel küldött üzenetet a topicra elmenti, és új feliratkozó csatlakozásakor azonnal elküldi neki. Ez a mechanizmus az állapot-jellegű adatok (pl. aktuális hőmérséklet, eszközstátusz) közzétételére való: a feliratkozó nem vár a következő időszakos publikálásra, hanem azonnal kap egy „utolsó ismert értéket”. Retained üzenet törlése üres payload-jú retained PUBLISH küldésével lehetséges.

Definíció (Last Will Testament, LWT). A kliens a csatlakozáskor megadhat egy will-üzenetet: topic, payload, QoS és retain flag. Ha a kliens nem szabályos DISCONNECT-tel bontja a kapcsolatot (pl. hálózati hiba, akkumulátor lemerülés, processz-összeomlás), a broker ezt az üzenetet közzéteszi a megadott topicra. Így a rendszer többi része értesül a váratlan kiesésről.

Az MQTT 5 legfontosabb újdonságai a TerraSense szempontjából:

- **Reason codes:** a PUBACK, PUBREC, SUBACK stb. csomagokban részletes okkód (pl. `0x10` – no matching subscribers, `0x87` – not authorized), amely alapján a kliens megkülönböztetheti a sikert a hiba típusától.
- **User properties:** tetszőleges kulcs-érték párok az üzenet fejlécében, pl. `trace-id` vagy `ingest-version` átvitele.
- **Message expiry:** az üzenetre beállítható élettartam (másodperc); ha a broker nem tudja kézbesíteni ennyi időn belül, eldobja, és a kliens opcionálisan értesítést kap.
- **Shared subscriptions:** több kliens közösen iratkozhat fel ugyanarra a topicra `$share/csoport/topic` formában; a broker az üzenetet a csoport egy tagjának továbbítja. Ezzel a feldolgozás terhelése megosztható, de az üzenetek sorrendje az egyes kliensek között nem garantált.
- **Request–response minta:** a `response topic` és `correlation data` tulajdonságokkal a válaszüzenet visszaköthető egy adott kéréshez anélkül, hogy a válasz topicját a payloadba kellene ágyazni.

10.2.3 Döntési szempontok

A következő táblázat összefoglalja a QoS-szintek kompromisszumait:

Szempont	QoS 0	QoS 1	QoS 2
Kézbesítési garancia	Legfeljebb egyszer	Legalább egyszer	Egyszer
Nyugtázás	Nincs	PUBACK	PUBREC/PUBREL/PUBCOMP
Üzenetduplikáció	Nincs	Lehetséges	Hálózati szinten nincs
Feldolgozói ismétlés	Nincs	Van, ha újra feldolgozzák	Van, ha a feldolgozó oldalhatása nem idempotens
Overhead	Legalacsonyabb	Közepes	Legmagasabb
Jellemző használat	Műszerfalra küldött telemetria, ahol néhány elvesztés elfogadható	Parancsok, riasztások, telemetria	Pénzügyi tranzakciók, olyan esetek, ahol a duplikáció súlyos

A retain és LWT választásánál az alábbiakat érdemes mérlegelni:

- **Retain** akkor indokolt, ha a topic tartalma állapotot ír le, és a feliratkozóknak azonnal szükségük van az utolsó értékre. Ilyen a TerraSense `state` és `status` topicja. Telemetriai idősoroknál a retain felesleges: az adott pillanat mérési értéke elavulttá válik, és a feldolgozó a későbbi üzenetekkel dolgozik.
- **LWT** minden olyan eszköznél kötelező, amelynek a jelenlétét figyelni kell. A `status` topicra érkező will-üzenetet retained flaggel érdemes küldeni, így a kiesést a később csatlakozó feliratkozó is azonnal látja.

A QoS 2 használata csábító, de a gyakorlatban több buktatót rejt: a broker és a kliens állapotgépének bonyolultsága nő, a kézfogás késleltetést ad, és ami a legfontosabb: **a teljes rendszerre vetítve nem jelent „pontosan egyszeri” feldolgozást**, mert az üzenet fogyasztója az adatbázisba írást vagy a szelepnyitást akkor is megismételheti, ha az üzenet maga egyszer érkezett. A helyes megoldás az at-least-once (QoS 1) és az idempotens feldolgozás kombinációja.

10.2.4 Kidolgozott példa

Tervezzük meg a TerraSense MQTT-topicjainak QoS, retain és LWT beállításait. A könyv korábbi konvenciója szerint a topicok:

- `terrasense/<site>/<node>/telemetry`
- `terrasense/<site>/<node>/state`
- `terrasense/<site>/<node>/cmd`
- `terrasense/<site>/<node>/cmd/result`
- `terrasense/<site>/<node>/status`

A döntési táblázat ehhez:

Topic	QoS	Retain	Indoklás
`telemetry`	1	Nem	Idősor, ahol az elvesztés ritka, de a duplikáció kezelhető; retain felesleges
`state`	1	Igen	Aktuális állapot; új feliratkozó azonnal látja
`cmd`	1	Nem	Parancs; nem szabad retained üzenetként újra kiküldeni korábbi feliratkozóknak
`cmd/result`	1	Nem	Válasz a parancsra; nem állapot, hanem esemény
`status`	1 (LWT)	Igen	Jelenlét; a retained garantálja, hogy új megfigyelő is tudja, mi az utolsó állapot

A telemetria üzenetben már definiált `seq` monoton növekvő sorszám az idempotens feldolgozás alapja. A backend a `(node_id, seq)` párt egy rövid lejáratú cache-ben tárolja; ha az adott pár már feldolgozásra került, az ismétlődő üzenetet eldobja. Ezt a mintát fogja használni az Ingest: MQTT-ből a backendbe, idempotensen alfejezet, valamint a Eseményvezérelt feldolgozás: Kafka és a kézbesítési szemantika és a Parancsok és konfiguráció lefelé: kézbesítés, nyugtázás, időkorlát alfejezet.

Példa a deduplikációhoz a ts-backendben (Java, Spring Boot környezetben, de csak a logika):

```
private final Set<DedupKey> processed = Collections.synchronizedSet(
    new LinkedHashMap<DedupKey, Boolean>() {
        @Override
        protected boolean removeEldestEntry(Map.Entry<DedupKey, Boolean> eldest) {
            return size() > 10_000;
        }
    });
private final Set<DedupKey> inFlight = ConcurrentHashMap.newKeySet();

public void onTelemetry(String nodeId, long seq, String payloadJson) {
    DedupKey key = new DedupKey(nodeId, seq);
    if (processed.contains(key)) {
```

```
        // Már sikeresen feldolgozott üzenet: QoS 1 újraküldés miatt érkezett.
        return;
    }
    if (!inFlight.add(key)) {
        // Ugyanez az üzenet másik szálon éppen feldolgozás alatt áll.
        return;
    }
    try {
        // Tényleges feldolgozás: TimescaleDB-be írás, riasztásvizsgálat, stb.
        processTelemetry(payloadJson);
        // Csak sikeres feldolgozás után kerül a tartós deduplikációs halmazba.
        processed.add(key);
    } finally {
        inFlight.remove(key);
    }
}
```

A kulcsot tehát nem a feldolgozás előtt, hanem annak sikeres befejezése után tesszük a `processed` halmazba; hiba esetén a `finally` eltávolítja az `inFlight` bejegyzést, így az újraküldött üzenet újra feldolgozható. Az `inFlight` halmaz a párhuzamos szálakon érkező azonos üzenetek egyidejű feldolgozását is megakadályozza. A tartós idempotenciához az adatbázis-írásnak is idempotensnek kell lennie (például egyedi kulcs vagy `ON CONFLICT DO NOTHING`), mert a feldolgozás sikere és a `processed.add` között a folyamat megszakadhat; ilyenkor az újraküldött üzenet ismét lefut, de nem hoz létre második adatbázissort.

A `node_id` és a `seq` együttesen egyedi kulcs; a `processed` halmaz csak a sikeres feldolgozás után bővül, a cache mérete pedig korlátozza a memóriahasználatot, és a legrégebbi bejegyzések törlődnek. Mivel a `seq` a csomópontonként monoton növekvő, a csomópont újraindulásakor a `node_id` és a `seq` kombinációja megismétlődhet, ezért a gyakorlatban a `boot_id`-t (az adott bootolási munkamenet azonosítóját) is érdemes a kulcs részévé tenni, vagy a `seq`-et nem felejtő memóriában tartani. A `boot_id` a későbbi, verziózott telemetriai sémában is szerepel.

A retained üzenet és LWT működését a következőképp demonstrálhatjuk Mosquitto-val (a 2.x verzió alapértelmezetten csak localhost-on engedélyezi az anonim hozzáférést):

```
# 1. terminál: figyelő a státuszra
mosquitto_sub -h localhost -t 'terrasense/gh1/n001/status' -v

# 2. terminál: az eszközt imitáló, hosszan futó kliens will-lel
mosquitto_sub -h localhost -t 'terrasense/gh1/n001/cmd' -v \
  --will-topic 'terrasense/gh1/n001/status' \
  --will-payload '{"online":false,"ts":1717000000000}' \
  --will-qos 1 --will-retain

# 3. terminál: az induló állapot retained publikálása (szabályos kliens)
mosquitto_pub -h localhost -t 'terrasense/gh1/n001/status' \
  -m '{"online":true,"ts":1717000000000}' -r

# A 2. terminál folyamatát külsőleg, SIGKILL-lel szakítsuk meg,
# így nem küld DISCONNECT csomagot, és a broker publikálja a will-t:
pkill -9 -f "mosquitto_sub -h localhost -t terrasense/gh1/n001/cmd"
```

A 2. terminálban futó kliens szabályos leállítása (`Ctrl+C`) DISCONNECT csomagot küldene, amire a broker **nem** publikálja a will-t. Ezért a kiesést `SIGKILL` jelzéssel vagy a folyamat összeomlásával kell szimulálni. A `mosquitto_pub` rövid életű, szabályosan lezáruló kliens, ezért a kilépése nem will-publikálást, hanem DISCONNECT-et jelent; a will csak a hosszan futó kliens nem szabályos megszakadásakor aktiválódik.

10.2.5 Ellenőrzés és hibakeresés

A QoS, retain és LWT beállítások helyességét a legegyszerűbben `mosquitto_sub` és `mosquitto_pub` eszközökkel ellenőrizhetjük. A fenti példában a retained státusz azonnal megjelenik a feliratkozónál, amint csatlakozik; ezt a `mosquitto_sub` kimenetén azonnal látjuk. A LWT csak akkor ellenőrizhető, ha a will-lel csatlakozó, hosszan futó klienst `SIGKILL`-lel megszakítjuk, mert a `Ctrl+C` szabályos DISCONNECT-et küld, és a broker ilyenkor nem publikálja a will-t.

Hibatünet	Lehetséges ok	Javítás
A feliratkozó nem kap retained üzenetet	A feliratkozás a retained publish után történt, de a broker nem tárolta (retain flag hiányzik)	Ellenőrizze a publish `-r` kapcsolóját és a broker `retain_available` beállítását
A LWT nem jelenik meg a kapcsolat megszakadásakor	A kliens szabályos DISCONNECT-et küldött, vagy a will-topic hibás	Tesztelje a folyamat `SIGKILL` jelzésével, ellenőrizze a will paramétereket
A QoS 1 üzenet duplikátumként érkezik, de a backend nem dobja el	Hiányzik a deduplikációs kulcs (pl. `seq`) vagy a cache nem tartalmazza a feldolgozott kulcsokat	Vezessen be `node_id`+`seq` (és `ts`) alapú deduplikációt
Az MQTT 5 kliens nem kap reason code-ot	A broker vagy a kliens MQTT 3.1.1-re visszaminősít	Győződjön meg róla, hogy mindkét fél MQTT 5-öt tárgyal, és a CONNECT protokollszintje 5

10.2.6 Tipikus hibák és félreértések

1. **QoS 1 használata idempotens feldolgozás nélkül.** A QoS 1 pontosan azért okozhat duplikátumot, mert a PUBACK hiánya miatt az üzenetet újraküldik. Ha a fogadó oldal ezt nem kezeli, a szelepparancs kétszer is végrehajtódik. Az Eszközszoftver-architektúra: rétegek és állapotgépek alfejezetben tárgyalt `valve_set_target` függvény idempotens, de a parancs rétegnek is deduplikálnia kell.

2. **A QoS 2-t „pontos kézbesítésnek” hiszik teljes rendszerre.** A QoS 2 csak a broker és a kliens közötti üzenetszintű duplikációt szünteti meg, nem a fogyasztói oldalhatást. Aki azt állítja, hogy a QoS 2 pontosan egyszeri végponttól végpontig, az a kézbesítést összekeveri a feldolgozással.

3. **A retained üzenetet telemetriára alkalmazzák.** A telemetria idősor, ahol az utolsó érték nem releváns egy új feliratkozó számára; a retained üzenet ráadásul felesleges tárolást és hálózati forgalmat generál.

4. **A LWT hiánya a jelenlétfigyelésnél.** Ha az eszköz váratlanul leáll, a rendszer nem tudja megkülönböztetni a csendes működést a kieséstől. A LWT a legegyszerűbb módja a jelenlét kezelésének; enélkül a felhő csak a timeoutra hagyatkozhat, ami lassabb és pontatlanabb.

10.2.7 Feladatok

1. **Fogalmi kérdés.** Magyarázza el, miért nem garantálja a QoS 2 a pontosan egyszeri feldolgozást egy parancs végrehajtásánál. Milyen QoS-t érdemes választani a TerraSense öntözőparancsához, ha a parancs idempotens?

2. **Tervezési feladat.** Adott egy vízmérő, amely naponta egyszer küld egy aggregátumot és egy riasztást, ha a szivárgás észlelhető. Milyen QoS-t és retain beállítást választana az aggregátumhoz és a riasztáshoz? Indokolja.

3. **Implementációs feladat a TerraSense-ben.** Készítsen konfigurációt a `ts-node` MQTT-klienséhez, amely a `telemetry` topicra QoS 1-retain false, a `state` topicra QoS 1-retain true, a `cmd/result` topicra QoS 1-retain false beállítással publikál, és a kapcsolódáskor LWT-t regisztrál a `status` topicra. Írja le a pszeudokódot vagy a Zephyr MQTT API hívásokat.

4. **Hibakeresési feladat.** A teszt során a `ts-backend` kétszer dolgozza fel ugyanazt a telemetriai üzenetet: a `seq` azonos, de a `ts` időbélyeg is azonos. A deduplikációs cache 10 000 bejegyzést tartalmaz, és az üzenetek közötti idő néhány másodperc. Mi lehet a hiba? (Tipp: a `DedupKey` osztály `equals` metódusát vizsgálja.)

10.2.8 Megoldások

1. A QoS 2 a kommunikációs rétegben kiküszöböli a hálózati duplikációt, de a parancsot feldolgozó alkalmazás ettől még kétszer is kiválthatja a szelepnyitást, ha például a PUBCOMP elvesztése miatt a parancsfeldolgozó újraindul és újraolvassa az üzenetet, vagy ha a feldolgozó oldali retry logika megismétli a végrehajtást. A helyes választás at-least-once (QoS 1) idempotens parancsvégrehajtással: a `valve_set_target` függvényt ismételten hívva ugyanazt az állapotot állítja be, és a parancsfeldolgozó a `seq` alapján eldobja a duplikátumot.

2. Az aggregátumhoz QoS 1-retain false: az adat idősor, ahol a napi érték elvesztése ritka, de ha elveszne, másnap újabb jön; a retain felesleges. A riasztáshoz QoS 1-retain true: a riasztás állapotjellegű, a feliratkozóknak azonnal tudniuk kell a legutóbbi riasztási állapotot, akár akkor csatlakoznak, amikor a riasztás már aktív. A retain miatt a riasztás törlését is explicit üres retained üzenettel kell megoldani.

3. Pszeudokód a Zephyr MQTT klienshez (a konkrét API függvénynevek verziófüggőek, ellenőrizze a telepített Zephyr dokumentációt):

```
struct mqtt_publish_param telemetry = {
    .message.topic.qos = MQTT_QOS_1_AT_LEAST_ONCE,
    .message.topic.topic.utf8 = "terrasense/gh1/n001/telemetry",
    .message.topic.topic.size = sizeof("terrasense/gh1/n001/telemetry") - 1,
    .message.payload.data = buf,
    .message.payload.len = len,
    .message_id = mqtt_allocate_msg_id(&client),
    .dup_flag = 0,
    .retain_flag = 0,
};
mqtt_publish(&client, &telemetry);

struct mqtt_publish_param state = telemetry;
state.message.topic.topic.utf8 = "terrasense/gh1/n001/state";
state.message.topic.topic.size = sizeof("terrasense/gh1/n001/state") - 1;
state.retain_flag = 1;
```

```
state.message_id = mqtt_allocate_msg_id(&client);
mqtt_publish(&client, &state);
```

A LWT regisztrálás a CONNECT paraméterben: a `will_topic` a `status` topic, a will payload `{"online":false}` és a `will_retain` beállítás 1. A will QoS szintén 1.

4. A hiba valószínű oka, hogy a `DedupKey` osztály `equals` és `hashCode` metódusa nincs összhangban: ha az `equals` a `node_id`-et és a `seq`-et hasonlítja, de a `hashCode` csak a `node_id`-et használja, a `Set` nem találja meg a meglévő bejegyzést, így minden üzenetet újként kezel. Megoldás: mindkét mezőt szerepeltesse a `hashCode`-ban, vagy használjon rekord típust (Java 21-ben `record DedupKey(String nodeId, long seq) {}`), amely automatikusan helyesen implementálja ezeket.

10.2.9 Rövid összefoglaló

Az MQTT QoS 0, 1 és 2 szintjei a publisher-broker-subscriber szakaszokra külön-külön adnak kézbesítési garanciát. A QoS 1 legalább egyszer kézbesít, ezért a fogadó oldalon idempotens feldolgozás szükséges; a QoS 2 a hálózati duplikációt szünteti meg, de a feldolgozói ismétlést nem. A retained üzenetek az állapottopicokhoz valók, a Last Will a váratlan kiesés jelzésére szolgál. A TerraSense konvenciója: `telemetry` QoS 1, retain false; `state` QoS 1, retain true; `cmd` és `cmd/result` QoS 1, retain false; `status` QoS 1, LWT retained true. Az MQTT 5 reason code-jai, user properties, message expiry és shared subscriptionjei finomabb vezérlést adnak. **Továbbvitt eredmény:** az at-least-once és idempotens feldolgozás kombinációját használja az Ingest: MQTT-ből a backendbe, idempotensen, az Eseményvezérelt feldolgozás: Kafka és a kézbesítési szemantika és a Parancsok és konfiguráció lefelé: kézbesítés, nyugtázás, időkorlát alfejezet; a retained és LWT beállításokat az Eszközárnyék és digitális iker: kívánt és jelentett állapot és a Megfigyelhetőség végponttól végpontig: metrikák, naplók, nyomkövetés alfejezet.

10.2.10 Források és ellenőrzési hiányok

A QoS-szintek, munkamenet, retained és Last Will definíciói az MQTT 3.1.1 (OASIS) és MQTT 5.0 (OASIS) szabványokon alapulnak, amelyek stabil részei az alapfogalmaknak. Az MQTT 5 reason code-jainak pontos listája és a shared subscription szintaxisa verziófüggő: használat előtt ellenőrizze a telepített Mosquitto 2.x verziójának dokumentációját. A Zephyr MQTT API pontos függvénynevei és struktúrái a Zephyr aktuális verziójától függenek; a példában jelzett pszeudokódot a telepített Zephyr dokumentációja alapján kell adaptálni. A Mosquitto `mosquitto_pub` és `mosquitto_sub` kapcsolói (`--will-*`, `-r`) a 2.x verzióban elérhetők, de az anonim hozzáférés és a hálózati beállítások a helyi konfigurációtól függnek.

10.3 MQTT a gyakorlatban: Zephyr-kliens és Mosquitto

10.3.1 Motiváció

A MQTT QoS, munkamenetek, retain, Last Will és MQTT 5 alfejezetben elméletben már ismerjük az MQTT fogalmait, de a TerraSense-rendszer egyetlen rétege sem működik, ha a `ts-node` nem tud valódi MQTT-üzenetet eljuttatni az üzenetközvetítőhöz (brokerhez). Ebben az alfejezetben végre összekötjük a korábban felépített labor-elemeket: a Zephyr `native_sim` célon futtatott `ts-node` felépít egy újracsatlakozó MQTT-publikálót, a Dockerben futó Mosquitto brokert a A labor felállítása: Linux host, Docker, Zephyr, Mosquitto, Java alfejezetben létrehozott

compose-vázból indítjuk, és Linux oldalon egy C-ből használt Paho-előfizetővel ellenőrizzük a teljes láncot. Ezzel a lánccal később a gateway (A ts-gw szolgáltatás C++-ban: protokollfordítás MQTT-re), a biztonságos kommunikáció (Biztonságos kapcsolat: TLS 1.3, mTLS és DTLS az MQTT/CoAP alatt) és a záróprojekt (Záróprojekt: a teljes TerraSense rendszer összeállítása és átvétele) is erre épít.

10.3.2 Fogalmak és működés

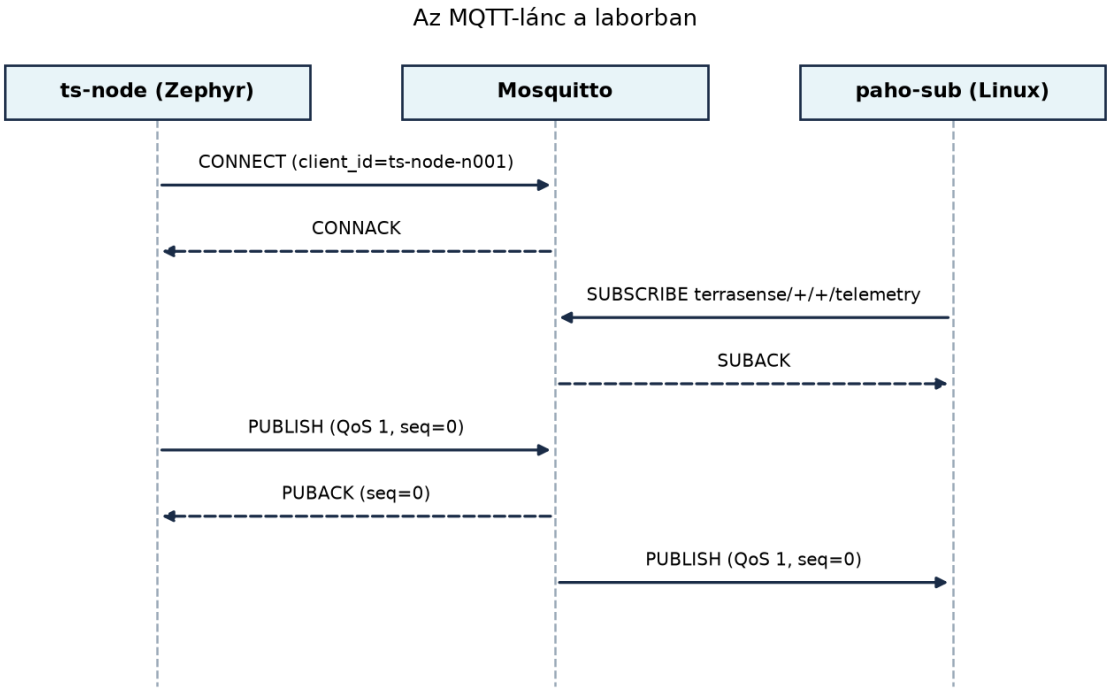
A Zephyr MQTT-kliens API-ja a Zephyr saját hálózati stackjére és a socket API-ra épül. A kliens nem külön szálat indít, hanem az alkalmazás felelős az úgynevezett *input/live loop* futtatásáért. A kapcsolat felépítése után a `mqtt_input()` feldolgozza a bejövő MQTT-vezérlő csomagokat, a `mqtt_live()` pedig a keepalive időzítést és az újraküldést kezeli.

Definíció. Input/live loop: az MQTT-kliens könyvtár által megkövetelt, az alkalmazás ciklusában ismételten hívott eljáráspár. Az `mqtt_input()` olvassa a socketről a bejövő bájtokat, és meghívja az eseménykezelőt; az `mqtt_live()` karbantartja a kapcsolat állapotát: keepalive ping, PUBACK-várakozás, újracsatlakozás előkészítése. Ha a loop kimarad, a broker a keepalive-időtúllépés után lezárja a kapcsolatot.

Definíció. Exponenciális backoff: újracsatlakozási stratégia, amelyben két próba közötti várakozási idő a sikertelen kísérletek számával exponenciálisan nő, általában felső korláttal. Alakja: $\Delta t = \min(2^n \cdot base, max)$, ahol n a sikertelen próbák száma. Ez megelőzi a hálózati vihart és kíméli az akkumulátort.

A Paho MQTT C kliens ezzel szemben egy Linux-host könyvtár, amely szinkron vagy aszinkron módon is használható. A laborban az aszinkron callbackes módot vesszük igénybe: `MQTTClient_create()`, `MQTTClient_setCallbacks()`, `MQTTClient_connect()`, `MQTTClient_subscribe()`, majd a callback szálán érkező üzenetre várunk. Ebben a módban a Paho saját fogadószálat indít, ezért a `MQTTClient_receive()` nem használható.

Az alábbi sorrenddiagram a labor MQTT-láncát mutatja.



10.3. ábra: Az MQTT-lánc a laborban - A ts-node a Zephyr native_sim-ben fut, a Mosquitto konténerben, a Paho-sub a Linux hoston. A dashed nyilak nyugták.

Az ábra szöveges összefoglalása: a `ts-node` csatlakozik a brokerhez, a Linux `paho-sub` feliratkozik a `terrasense/+/+/telemetry` wildcard topicra. A `ts-node` publikál egy QoS 1 üzenetet, a broker nyugtázza, majd továbbítja a feliratkozott kliensnek.

10.3.3 Döntési szempontok

A laborban az MQTT 3.1.1-et használjuk, mert a Zephyr kliens ezt stabilan támogatja (ellenőrzési igény: a konkrét API és a támogatott protokollverzió a Zephyr kiadásától függhet). A production rendszerben az MQTT 5 bevezethető, ha a property-k és a munkamenet-kezelés előnyeit ki akarjuk használni, de a jelen tananyag a 3.1.1-re épül.

Döntés	Választás a laborban	Indoklás
Hol fut az MQTT-kliens?	A `ts-node`-on	A IP az eszközön alfejezetben bemutatott modell szerint a csomópont közvetlenül a brokerhez beszél.
MQTT protokollverzió	3.1.1	Széles körű kompatibilitás; a Zephyr kliens ezt biztosítja.
QoS telemetriához	1 (at least once)	A későbbi idempotens ingest (Ingest: MQTT-ből a backendbe, idempotensen) megköveteli az üzenetek legalább egyszeri kézbesítését.

Döntés	Választás a laborban	Indoklás
Clean session	true	A `native_sim`-eszköznek nincs szüksége a munkamenet újrafelvételére; a valós eszköznél ezt a MQTT QoS, munkamenetek, retain, Last Will és MQTT 5 alfejezetben tárgyalt szempontok szerint állítjuk be.
Autentikáció	anonim (labor)	TLS és jelszó/tanúsítvány a Biztonságos kapcsolat: TLS 1.3, mTLS és DTLS az MQTT/CoAP alatt alfejezetben kerül be.

10.3.4 Kidolgozott példa és labor

A labor célja: a `ts-node` Zephyrben MQTT-n publikál a `terrasense/gh1/n001/telemetry` topicra, a Mosquitto a Dockerben fogadja, és egy Linuxon futó Paho C előfizető megjeleníti az üzenetet. A labor nem használ valós hardvert; minden a `native_sim` boardon és a hoston fut.

Munkakönyvtár:

```
mkdir -p ~/iot-labor/i028/ts-node-mqtt/src
mkdir -p ~/iot-labor/i028/paho-sub
cd ~/iot-labor/i028
```

10.3.4.1 1. Mosquitto-konfiguráció a Compose-ban

A korábban létrehozott `~/iot-labor/stack/mosquitto/mosquitto.conf` fájlt használjuk; a labor nem módosítja, mert a közös stack más fejezetek laborjaival is fut. A fájl releváns tartalma:

```
listener 1883
allow_anonymous true
log_dest stdout
log_type all
persistence false
```

A `~/iot-labor/stack/compose.yaml` releváns részlete (a szolgáltatás és a konténer neve is `lab-broker`):

```
services:
  lab-broker:
    image: eclipse-mosquitto:2
    container_name: lab-broker
    ports:
      - "1883:1883"
    volumes:
      - ./mosquitto/mosquitto.conf:/mosquitto/config/mosquitto.conf:ro
    restart: unless-stopped
```

Indítsuk el a brokert:

```
cd ~/iot-labor/stack
docker compose up -d lab-broker
docker ps | grep lab-broker
```

Várt kimenet: a `lab-broker` konténer fut, és a 1883-as port elérhető a hoston.

10.3.4.2 2. Zephyr MQTT-publikáló a `native_sim` célon

Hozzuk létre a `ts-node-mqtt` alkalmazást. A `prj.conf`:

```
CONFIG_NETWORKING=y
CONFIG_NET_IPV4=y
CONFIG_NET_TCP=y
CONFIG_NET_SOCKETS=y
CONFIG_NET_DRIVERS=y
CONFIG_NET_SOCKETS_OFFLOAD=y
CONFIG_NET_NATIVE_OFFLOADED_SOCKETS=y
CONFIG_POSIX_API=y
CONFIG_MQTT_LIB=y
CONFIG_MAIN_STACK_SIZE=2048
```

A `CONFIG_NET_NATIVE_OFFLOADED_SOCKETS=y` a `native_sim` úgynevezett NSOS (native simulator offloaded sockets) módját kapcsolja be. Ebben a módban a Zephyr socket-hívásai a host Linux BSD socket API-ját használják, ezért a `127.0.0.1` a **host** loopback interfésze, és a Dockerben futó, host 1883-as portra publikált broker elérhető. NSOS nélkül a `native_sim` a Zephyr saját hálózati stackjét használná, amelynek nincs automatikus útja a host loopbackjére: ahhoz TAP-interfész (`zeth`) vagy más hostoldali hálózati összekötés kellene, ahogy az IP az eszközön alfejezet tárgyalja. Ezért itt nem adunk meg `CONFIG_NET_CONFIG_MY_IPV4_ADDR` értéket: a címet nem a Zephyr stackje, hanem a host socketrétege kezeli.

A `CMakeLists.txt`:

```
cmake_minimum_required(VERSION 3.20.0)
find_package(Zephyr REQUIRED HINTS $ENV{ZEPHYR_BASE})
project(ts-node-mqtt)

target_sources(app PRIVATE src/main.c)
```

A `src/main.c`:

```
#include <zephyr/kernel.h>
#include <zephyr/net/socket.h>
#include <zephyr/net/mqtt.h>
#include <errno.h>
#include <stdbool.h>
#include <string.h>

#define SERVER_ADDR "127.0.0.1"
#define SERVER_PORT 1883
#define CLIENT_ID "ts-node-n001"
#define TOPIC_TELEMETRY "terrasense/gh1/n001/telemetry"
#define TOPIC_STATE "terrasense/gh1/n001/status"
#define PUBLISH_INTERVAL_MS 5000
#define CONNECT_TIMEOUT_MS 5000
#define BACKOFF_BASE_MS 1000
#define BACKOFF_MAX_MS 32000
#define BACKOFF_MAX_SHIFT 5

static uint8_t rx_buffer[512];
static uint8_t tx_buffer[512];
```

```c
static struct mqtt_client client;
static struct sockaddr_storage broker;
static struct pollfd fds[1];
static int nfds;
static bool connected;
static uint32_t seq;
static uint16_t next_message_id = 1;

static void broker_init(void)
{
    struct sockaddr_in *addr = (struct sockaddr_in *)&broker;

    addr->sin_family = AF_INET;
    addr->sin_port = htons(SERVER_PORT);
    inet_pton(AF_INET, SERVER_ADDR, &addr->sin_addr);
}

static void clear_fds(void)
{
    nfds = 0;
}

static void mqtt_evt_handler(struct mqtt_client *const c,
                             const struct mqtt_evt *evt)
{
    ARG_UNUSED(c);

    switch (evt->type) {
    case MQTT_EVT_CONNACK:
        if (evt->result == 0) {
            connected = true;
            printk("MQTT connected (CONNACK)\n");
        } else {
            printk("CONNACK error: %d\n", evt->result);
        }
        break;
    case MQTT_EVT_DISCONNECT:
        printk("MQTT disconnected: %d\n", evt->result);
        connected = false;
        clear_fds();
        break;
    default:
        break;
    }
}

static uint16_t alloc_message_id(void)
{
    uint16_t id = next_message_id++;

    if (next_message_id == 0) {
        next_message_id = 1;
    }
    return id;
}

static void prepare_fds(void)
{
    fds[0].fd = client.transport.tcp.sock;
    fds[0].events = POLLIN;
    nfds = 1;
}
```

```
static int connect_mqtt(void)
{
    int rc;

    mqtt_client_init(&client);
    client.broker = &broker;
    client.evt_cb = mqtt_evt_handler;
    client.client_id.utf8 = (const uint8_t *)CLIENT_ID;
    client.client_id.size = strlen(CLIENT_ID);
    client.protocol_version = MQTT_VERSION_3_1_1;
    client.clean_session = 1;
    client.keepalive = 60;
    client.rx_buf = rx_buffer;
    client.rx_buf_size = sizeof(rx_buffer);
    client.tx_buf = tx_buffer;
    client.tx_buf_size = sizeof(tx_buffer);
    client.transport.type = MQTT_TRANSPORT_NON_SECURE;

    rc = mqtt_connect(&client);
    if (rc != 0) {
        return rc;
    }

    prepare_fds();
    rc = poll(fds, nfds, CONNECT_TIMEOUT_MS);
    if (rc < 0) {
        rc = -errno;
    } else if (rc == 0) {
        rc = -ETIMEDOUT;
    } else {
        rc = mqtt_input(&client);
    }

    if (rc != 0 || !connected) {
        mqtt_abort(&client);
        clear_fds();
        return rc != 0 ? rc : -ECONNREFUSED;
    }
    return 0;
}

static int publish_telemetry(void)
{
    char payload[128];
    int len = snprintk(payload, sizeof(payload),
                       "{\"node_id\":\"n001\",\"seq\":%u,\"ts\":0}",
                       (unsigned int)seq);
    struct mqtt_publish_param param;
    int rc;

    if (len < 0 || (size_t)len >= sizeof(payload)) {
        return -ENOMEM;
    }

    memset(&param, 0, sizeof(param));
    param.message.topic.qos = MQTT_QOS_1_AT_LEAST_ONCE;
    param.message.topic.topic.utf8 = (const uint8_t *)TOPIC_TELEMETRY;
    param.message.topic.topic.size = strlen(TOPIC_TELEMETRY);
    param.message.payload.data = payload;
    param.message.payload.len = len;
    param.message_id = alloc_message_id();
    param.dup_flag = 0;
    param.retain_flag = 0;
```

```
    rc = mqtt_publish(&client, &param);
    if (rc != 0) {
        printk("Publish failed: %d\n", rc);
        return rc;
    }
    printk("Published: %s\n", payload);
    seq++;
    return 0;
}

static uint32_t backoff_delay(unsigned int attempt)
{
    unsigned int shift = MIN(attempt, BACKOFF_MAX_SHIFT);
    uint32_t delay = (uint32_t)BACKOFF_BASE_MS << shift;
    uint32_t jitter;

    delay = MIN(delay, (uint32_t)BACKOFF_MAX_MS);
    /* Legfeljebb 25%-os, olcsó jitter: több node ne próbálkozzon egyszerre. */
    jitter = (k_uptime_get_32() >> 10) % (delay / 4U + 1U);
    return MIN(delay + jitter, (uint32_t)BACKOFF_MAX_MS);
}

void main(void)
{
    int attempt = 0;
    int64_t next_publish = 0;

    printk("ts-node MQTT publisher starting\n");
    broker_init();

    while (1) {
        if (!connected) {
            int rc = connect_mqtt();

            if (rc != 0) {
                uint32_t backoff = backoff_delay(attempt);

                printk("Connect failed (%d), retry in %u ms\n", rc, backoff);
                k_sleep(K_MSEC(backoff));
                if (attempt < BACKOFF_MAX_SHIFT) {
                    attempt++;
                }
                continue;
            }
            attempt = 0;
            next_publish = k_uptime_get() + PUBLISH_INTERVAL_MS;
        }

        if (nfds > 0) {
            int rc = poll(fds, nfds, 100);

            if (rc < 0) {
                printk("poll failed: %d\n", errno);
                mqtt_abort(&client);
                connected = false;
                clear_fds();
                continue;
            }
            if (rc > 0 && mqtt_input(&client) != 0) {
                printk("mqtt_input failed\n");
                mqtt_abort(&client);
                connected = false;
```

```
                clear_fds();
                continue;
            }
        }

        if (connected) {
            int rc = mqtt_live(&client);

            if (rc != 0 && rc != -EAGAIN) {
                printk("mqtt_live failed: %d\n", rc);
                mqtt_abort(&client);
                connected = false;
                clear_fds();
                continue;
            }
        }

        if (connected && k_uptime_get() >= next_publish) {
            if (publish_telemetry() != 0) {
                mqtt_abort(&client);
                connected = false;
                clear_fds();
                continue;
            }
            next_publish = k_uptime_get() + PUBLISH_INTERVAL_MS;
        }
    }
}
```

A kód magyarázata:

- A `broker_init()` a `127.0.0.1:1883` végpontot tölti fel a kliensstruktúrába. Az NSOS miatt ez a host loopbackje, ahol a Docker broker publikál.
- A `connect_mqtt()` beállítja a client id-t, a `clean_session`-t és a keepalive-ot, majd meghívja az `mqtt_connect()`-et. Az `mqtt_connect()` csak a CONNECT csomag elküldéséig tér vissza; a kapcsolat akkor jön létre, amikor a broker CONNACK csomagját az `mqtt_input()` feldolgozza. Ezért a függvény `poll()`-lal vár a socket olvashatóságára, majd a CONNACK után tér vissza sikerrel.
- A `connected` jelzőt kizárólag az eseménykezelő állítja: `MQTT_EVT_CONNACK` esetén igazra, `MQTT_EVT_DISCONNECT` esetén hamisra. A `struct mqtt_client` Zephyr 3.7-ben nem tartalmaz `connected` mezőt.
- A `publish_telemetry()` egy minimális JSON payloadot készít. A `ts` mezőt most nullával töltjük, hogy a laborban a lánc működését igazoljuk; a tényleges időbélyeg-konvenciót a Következő lépések tárgyalja. A message ID-t saját, 1-től 65535-ig körbeforduló számláló adja, mert a Zephyr 3.7 MQTT API-ban nincs `mqtt_client_get_next_id()`.
- A főciklus `poll()`-lal vár a bejövő adatra, futtatja az `mqtt_input()`/`mqtt_live()` párost, és a `k_uptime_get()` 64 bites órájához kötött határidővel publikál 5 másodpercenként. A `k_uptime_get_32() % 5000 == 0` típusú feltétel azért hibás, mert a ciklus ritkán találja el pontosan az osztható értéket, és a 32 bites számláló 49 nap után túlcsordul.
- Az újracsatlakozási várakozás exponenciálisan nő, de a `BACKOFF_MAX_SHIFT` miatt a shift és a szorzás soha nem csordul túl, és a várakozás legfeljebb 32 másodperc. A számított várakozás legfeljebb 25%-os jitterrel egészül ki, amelyet a `k_uptime_get_32()` alacsonyabb bitjeiből képezünk, és a végén ismét a 32 másodperces plafonhoz vágunk; így száz egymást követő hiba után sem lesz negatív vagy rövidre forduló várakozás, és a node-ok nem próbálkoznak egyszerre. A `seq` monoton nő, így minden üzenet egyedi sorszámmal megy ki.
- A QoS 1 függő (PUBACK-ra váró) publikálásokat a `mqtt_abort()` eldobja, amikor a

kapcsolat megszakad. Telemetriánál ez a rendszerszintű helyes viselkedés: a következő minta új `seq`-csel megy ki, az ingest pedig idempotens. Parancsoknál viszont a függő kéréseket külön, tartós outboxban kell vezetni, különben a PUBACK elvesztésekor a parancs nyugtázatlan marad.

Build és futtatás:

```
cd ~/iot-labor/i028/ts-node-mqtt
west build -b native_sim -p always
./build/zephyr/zephyr.exe
```

Várt kimenet:

```
ts-node MQTT publisher starting
MQTT connected (CONNACK)
Published: {"node_id":"n001","seq":0,"ts":0}
Published: {"node_id":"n001","seq":1,"ts":0}
...
```

10.3.4.3 3. Paho C előfizető Linuxon

A `paho-sub/subscriber.c`:

```c
#include <stdio.h>
#include <stdlib.h>
#include <stdatomic.h>
#include <unistd.h>
#include <MQTTClient.h>

#define ADDRESS "tcp://127.0.0.1:1883"
#define CLIENTID "lab-paho-sub"
#define TOPIC "terrasense/+/+/telemetry"
#define QOS 1

static atomic_int delivered;

int msgarrvd(void *context, char *topicName, int topicLen,
             MQTTClient_message *message)
{
    (void)context;
    (void)topicLen;

    printf("topic=%s payload=%.*s\n",
           topicName, message->payloadlen, (char *)message->payload);
    MQTTClient_freeMessage(&message);
    MQTTClient_free(topicName);
    atomic_store(&delivered, 1);
    return 1;
}

void connlost(void *context, char *cause)
{
    (void)context;
    printf("connection lost: %s\n", cause ? cause : "unknown");
}

int main(void)
{
    MQTTClient client;
    MQTTClient_connectOptions conn_opts = MQTTClient_connectOptions_initializer;
    int rc;
```

```
    if ((rc = MQTTClient_create(&client, ADDRESS, CLIENTID,
                                MQTTCLIENT_PERSISTENCE_NONE, NULL))
        != MQTTCLIENT_SUCCESS) {
        fprintf(stderr, "create failed: %d\n", rc);
        return 1;
    }
    conn_opts.keepAliveInterval = 20;
    conn_opts.cleansession = 1;

    if ((rc = MQTTClient_setCallbacks(client, NULL, connlost, msgarrvd,
                                      NULL)) != MQTTCLIENT_SUCCESS) {
        fprintf(stderr, "setCallbacks failed: %d\n", rc);
        MQTTClient_destroy(&client);
        return 1;
    }
    if ((rc = MQTTClient_connect(client, &conn_opts)) != MQTTCLIENT_SUCCESS) {
        fprintf(stderr, "connect failed: %d\n", rc);
        MQTTClient_destroy(&client);
        return 1;
    }
    if ((rc = MQTTClient_subscribe(client, TOPIC, QOS))
        != MQTTCLIENT_SUCCESS) {
        fprintf(stderr, "subscribe failed: %d\n", rc);
        MQTTClient_disconnect(client, 1000);
        MQTTClient_destroy(&client);
        return 1;
    }

    printf("waiting on %s\n", TOPIC);
    while (!atomic_load(&delivered)) {
        sleep(1);
    }

    MQTTClient_disconnect(client, 1000);
    MQTTClient_destroy(&client);
    return 0;
}
```

A `MQTTClient_setCallbacks()` aszinkron üzemmódba kapcsolja a Paho C klienst: a könyvtár saját fogadószálat indít, és az `msgarrvd` callbacket ezen a szálon hívja meg. Ebben az üzemmódban a főszálnak **tilos** `MQTTClient_receive()`-et hívnia; a szinkron API ötparaméteres `MQTTClient_receive(client, topicName, topicLen, message, timeout)` formája csak a callback nélküli módhoz tartozik. A `delivered` jelző `atomic_int` típusú, mert a callback és a főszál külön szálon fut; a `volatile int` nem garantál szálak közötti láthatóságot.

Telepítés és build (ha a `libpaho-mqtt-dev` még nincs fent):

```
sudo apt install libpaho-mqtt-dev   # ellenőrizendő a pontos csomagnév
cd ~/iot-labor/i028/paho-sub
gcc -o sub subscriber.c -lpaho-mqtt3a
```

A `-lpaho-mqtt3a` az aszinkron Paho C könyvtár, amely a `MQTTClient_setCallbacks()` callbackes üzemmódját tartalmazza. A `-lpaho-mqtt3c` szinkron változat csak a `MQTTClient_receive()` alapú használathoz elegendő; callbackek mellett linkelési hibát ad.

Futtatás egy másik terminálban, miközben a Zephyr alkalmazás fut:

```
cd ~/iot-labor/i028/paho-sub
./sub
```

Várt kimenet:

```
waiting on terrasense/+/+/telemetry
topic=terrasense/gh1/n001/telemetry payload={"node_id":"n001","seq":12,"ts":0}
```

Továbbvitt eredmény. Ebben a laborban létrejött a működő MQTT-lánc: a `ts-node` a `terrasense/gh1/n001/telemetry` topicra publikál QoS 1-gyel, `node_id` és monoton `seq` mezőkkel, `clean_session=1` beállítással. A broker a `stack/compose.yaml` `broker` szolgáltatása (`lab-broker` konténer), a Paho C előfizető a hivatkozott fájl. Ezt a láncot használja a ts-gw szolgáltatás C++-ban: protokollfordítás MQTT-re alfejezet, a Biztonságos kapcsolat: TLS 1.3, mTLS és DTLS az MQTT/CoAP alatt alfejezet, és a Záróprojekt: a teljes TerraSense rendszer összeállítása és átvétele alfejezet.

10.3.5 Ellenőrzés és hibakeresés

Ellenőrzéshez használhatjuk a parancssori `mosquitto_sub`-ot is:

```
mosquitto_sub -h 127.0.0.1 -t 'terrasense/+/+/telemetry' -v
```

Ennek ugyanazt az üzenetfolyamot kell mutatnia, mint a Paho-előfizetőnek. A Zephyr logban a „Published” sorok jelennek meg minden egyes küldésnél.

Hibatünet	Lehetséges ok	Javítás
A Zephyr alkalmazás nem csatlakozik	A broker nem fut, vagy a 1883-as port foglalt	`docker ps` és `ss -tlnp` ellenőrzése, a Mosquitto indítása
A Zephyr log „MQTT connect failed”	Hálózati konfiguráció hiányzik vagy helytelen	`prj.conf` ellenőrzése, `CONFIG_NETWORKING=y` és IP-beállítások
A Paho-sub nem kap üzenetet	A topic wildcard hibás, vagy a feliratkozás nem járt sikerrel	Ellenőrizd a topicot: `terrasense/+/+/telemetry`, és a `MQTTClient_subscribe()` visszatérési értékét
A kapcsolat megszakad, és nem áll vissza	A CONNACK-ra nem várunk, vagy a `mqtt_connect()` visszatérési értékét nem kezeljük	A `poll()` + `mqtt_input()` várakozás és az exponenciális backoff kódrész beépítése
A broker a Paho-subból kizárja a ts-node-ot	Az MQTT 3.1.1-ben azonos client id miatt a régebbi kapcsolat lezárul	Egyedi `CLIENT_ID` használata minden kliensnél

10.3.6 Tipikus hibák és félreértések

1. **Az input/live loop elhagyása.** A Zephyr MQTT-kliens nem külön szál, és a keepalive-ot az alkalmazásnak kell karbantartania. Ha a `mqtt_live()` nem fut, a broker a keepalive-időtúllépés után lezárja a kapcsolatot.
2. **Újracsatlakozás fix késleltetéssel.** Ha minden sikertelen próba után ugyanannyit várakozunk, akkor a hálózati hiba elhárulásakor a csomópont feleslegesen sokat alszik. Az exponenciális backoff a kezdeti gyors újrapróbálkozást és a későbbi energiatakarékos ritmust kombinálja.

3. **A seq nem monoton növekvő.** A `seq` sorszámmal a későbbi idempotens ingest (Ingest: MQTT-ből a backendbe, idempotensen) azonosítja az üzeneteket. Ha a számláló újraindulás után nullázódik, a duplikátumok észrevétlenek maradnak.
4. **A Last Will elhagyása.** A `ts-node` normál leállásakor a `status` topic retained üzenete nem frissül; a rendszer nem tudja, hogy az eszköz offline lett. A Last Will beállítása a MQTT QoS, munkamenetek, retain, Last Will és MQTT 5 alfejezetben leírtak szerint kötelező.
5. **Időbélyeg nélküli vagy uptime-ot használó payload.** A `ts` mező most placeholder nulla, de éles rendszerben UTC epoch ms kell. Az uptime nem hordoz globális sorrendet, ezért a telemetria-sémát a Idő az IoT-ban: időbélyeg, időszinkron, sorrend alfejezet szerint kell véglegesíteni.

10.3.7 Feladatok

10.3.7.1 (a) Fogalmi kérdés

Miért nem elegendő, ha a Zephyr MQTT-kliensben csak a `mqtt_connect()` és a `mqtt_publish()` hívásokat használjuk? Mi a szerepe az `mqtt_input()` és az `mqtt_live()` hívásoknak?

10.3.7.2 (b) Számolási feladat

Egy eszköz a laborban használt exponenciális backoffot használja: `base = 1000 ms`, `max = 32000 ms`. Hány másodperc a teljes várakozási idő, ha az első öt újracsatlakozási kísérlet mind sikertelen, és a hatodik kísérlet sikerült? (A várakozás mindig a sikertelen próba után következik be.)

10.3.7.3 (c) Implementációs feladat

Módosítsd a `ts-node-mqtt/src/main.c` fájlt úgy, hogy a `mqtt_connect()` előtt beállítod a Last Will üzenetet a `terrasense/gh1/n001/status` topicra, retained flaggel és `"offline"` payloaddal. Tedd hozzá a megfelelő MQTT-paramétereket, és ellenőrizd a brokeren.

10.3.7.4 (d) Hibakeresési/kiterjesztési feladat

Írj egy új Paho C előfizetőt `paho-sub/cmd_sub.c` néven, amely a `terrasense/gh1/n001/cmd` topicra iratkozik fel, és minden bejövő parancsot kiír a szabványos kimenetre. Ezzel előkészíted a Parancsok és konfiguráció lefelé: kézbesítés, nyugtázás, időkorlát alfejezetet.

10.3.8 Megoldások

10.3.8.1 (a)

Az `mqtt_input()` olvassa a socketet és feldolgozza a CONNACK, SUBACK, PUBACK stb. vezérlőcsomagokat. Enélkül a kliens nem tudná meg a kapcsolatfelvétel eredményét, és a bejövő üzeneteket sem dolgozná fel. Az `mqtt_live()` a keepalive-ot küldi és a PUBACK-várakozást kezeli. Ha ezek a hívások hiányoznak, a kapcsolat lejár, és a kliens nem érzékeli a broker válaszait.

10.3.8.2 (b)

Az első öt sikertelen próba után a várakozások:

Kísérlet sorszáma	`attempt`	Várakozás (ms)
1	0	$2^0 \cdot 1000 = 1000$
2	1	$2^1 \cdot 1000 = 2000$
3	2	$2^2 \cdot 1000 = 4000$
4	3	$2^3 \cdot 1000 = 8000$
5	4	$2^4 \cdot 1000 = 16000$

Összesen: $1000 + 2000 + 4000 + 8000 + 16000 = 31000$ ms $= 31$ s. A hatodik kísérlet sikeres, tehát nem várakozunk utána. (Ha a hatodik is sikertelen lenne, a `max` miatt a várakozás 32 s lenne.)

10.3.8.3 (c)

A `connect_mqtt()` függvényben a `mqtt_client_init()` után, de a `mqtt_connect()` hívása előtt beállítjuk a Last Will-et. A Zephyr 3.7 MQTT API-ban a will nem külön `mqtt_will_param` struktúra, hanem a kliens `will_topic`, `will_message` és `will_retain` mezői; a will QoS-e a `will_topic.qos` mezőben van:

```
static const char will_topic_str[] = TOPIC_STATE;
static const char will_payload_str[] = "offline";
static struct mqtt_topic will_topic = {
    .topic = {
        .utf8 = (const uint8_t *)will_topic_str,
        .size = sizeof(will_topic_str) - 1,
    },
    .qos = MQTT_QOS_1_AT_LEAST_ONCE,
};
static struct mqtt_utf8 will_message = {
    .utf8 = (const uint8_t *)will_payload_str,
    .size = sizeof(will_payload_str) - 1,
};

static void set_last_will(void)
{
    client.will_topic = &will_topic;
    client.will_message = &will_message;
    client.will_retain = 1;
}
```

A `set_last_will()` hívást a `connect_mqtt()`-on belül közvetlenül a `mqtt_client_init(&client)` után helyezzük el, mert az inicializálás nullázza a kliens mezőit. A `will_topic` és a `will_message` statikus élettartamú, mert a mutatóiknak a kapcsolat teljes ideje alatt érvényeseknek kell maradniuk. Ellenőrzés:

```
mosquitto_sub -h 127.0.0.1 -t 'terrasense/gh1/n001/status' -v -W 5
```

A Zephyr alkalmazás `native_sim` folyamatát `Ctrl+C`-vel leállítva a folyamat nem küld MQTT DISCONNECT csomagot, ezért a broker a will-t publikálja, és az `offline` üzenetnek meg kell jelennie.

10.3.8.4 (d)

A `paho-sub/cmd_sub.c` hasonló a `subscriber.c`-hez, de a topic `terrasense/gh1/n001/cmd`, és a callback minden üzenetet kiír. Mivel a `subscriber.c` callbackes, aszinkron módot használ, itt sem szabad `MQTTClient_receive()`-et hívni: a callback ne állítson le egy üzenet után, a főszál pedig `while (1) { sleep(1); }` ciklussal várjon a folyamat megszakításáig. Build:

```
cd ~/iot-labor/i028/paho-sub
gcc -o cmd_sub cmd_sub.c -lpaho-mqtt3a
```

Teszteléshez publikáljunk egy tesztparancsot:

```
mosquitto_pub -h 127.0.0.1 -t 'terrasense/gh1/n001/cmd' -m '{"cmd":"open_valve","seq":100}'
```

A `cmd_sub` kiírja a parancsot.

10.3.9 Rövid összefoglaló

Ebben az alfejezetben működő MQTT-láncot építettünk a TerraSense laborban. A Zephyr `native_sim` célon futó `ts-node` a `~/iot-labor/i028/ts-node-mqtt/` alatt egy újracsatlakozó MQTT-publikálót valósít meg: `mqtt_connect()` a brokerhez, CONNACK-várakozás `poll()` + `mqtt_input()` hívással, majd `mqtt_input()`/`mqtt_live()` az input/live loopban és `mqtt_publish()` QoS 1-gyel. A `connected` állapotot az eseménykezelő tartja, a message ID-t saját számláló adja. Az újracsatlakozás exponenciális backoffot használ `base=1000 ms`, `max=32000 ms` paraméterekkel, legfeljebb 5 shift mellett. A broker a `stack/compose.yaml` `broker` szolgáltatása (`lab-broker` konténer), a konfiguráció a `stack/mosquitto/mosquitto.conf`-ban hallgat a 1883-as porton. A Linux oldali Paho C előfizető a `~/iot-labor/i028/paho-sub/subscriber.c` fájlban aszinkron callbackes módban figyel a `terrasense/+/+/telemetry` topicra. A továbbvitt eredmény: a `terrasense/gh1/n001/telemetry` topic, a `node_id`/`seq`/`ts` mezőkkel ellátott JSON payload és a QoS 1 publikálás; ezt építi tovább a gateway (A ts-gw szolgáltatás C++-ban: protokollfordítás MQTT-re), a biztonság (Biztonságos kapcsolat: TLS 1.3, mTLS és DTLS az MQTT/CoAP alatt) és a záróprojekt (Záróprojekt: a teljes TerraSense rendszer összeállítása és átvétele).

10.3.10 Források és ellenőrzési hiányok

- Zephyr MQTT-kliens API dokumentáció: https://docs.zephyrproject.org/latest/connectivity/networking/api/mqtt.html – ellenőrizendő a `mqtt_connect`, `mqtt_publish`, `mqtt_input`, `mqtt_live` függvények pontos szignatúrája és a `CONFIG_MQTT_LIB` Kconfig-név az aktuális Zephyr-kiadásban.
- Zephyr `native_sim` networking: https://docs.zephyrproject.org/latest/boards/native/native_sim/doc/index.html – ellenőrizendő az offloaded sockets (NSOS) Kconfig-nevei (`CONFIG_NET_SOCKETS_OFFLOAD`, `CONFIG_NET_NATIVE_OFFLOADED_SOCKETS`) és a host loopback használata.
- Mosquitto dokumentáció: https://mosquitto.org/documentation/ – ellenőrizendő a `allow_anonymous`, `log_dest` és `persistence` direktívák pontos szintaxisa a Mosquitto 2.x verzióban.
- Paho MQTT C kliens API: https://eclipse.dev/paho/files/paho.mqtt.c/doc/ – ellenőrizendő az aszinkron API függvénynevei és a `libpaho-mqtt3a` (aszinkron), illetve `libpaho-mqtt3c` (szinkron) könyvtárak elérhetősége Ubuntu 24.04-en.
- A Zephyr MQTT-kliens 3.1.1-es protokollverziójának támogatása: ellenőrizendő a hivatalos Zephyr release notes alapján.

10.4 CoAP: REST UDP fölött, Observe és blokkátvitel

10.4.1 Motiváció

A TerraSense rendszerben a ts-node szenzorcsomópontok alapértelmezés szerint MQTT-t használnak, mert az üvegházban kiépített Wi-Fi-hálózaton egy központi broker gyűjti a telemetriát. Vannak azonban olyan telepítési helyek, ahol nincs megbízható TCP-alapú kapcsolat, vagy a csomópont erőforrásai annyira szűkösek, hogy a TCP-verem és az MQTT-kliens együttes memóriaigénye már nem fér be. Ilyenkor egy UDP fölött működő, kérés–válasz jellegű protokoll sokkal kisebb lábnyommal is képes együttműködni a háttérrendszerrel. Ugyanez a helyzet a firmware-frissítésnél: a ts-backendnek nagy adatblokkokat kell megbízhatóan átvinnie olyan eszközökre, amelyek nem tudnak TCP-alapú kapcsolatot fenntartani.

A CoAP (Constrained Application Protocol) ezt a rést tölti be: a HTTP-hez hasonló REST-modellt kínál, de kifejezetten korlátos eszközökre tervezve, UDP fölött, kis üzenetméretekkel és opcionális megbízhatósággal. A TerraSense-ben a ts-gw és a ts-backend közötti észak–déli forgalom MQTT marad, de a nagyon kis eszközök felé, vagy ahol a kérés–válasz minta fontosabb a folyamatos publikálásnál, a CoAP életképes alternatíva.

10.4.2 Fogalmak és működés

Definíció. A CoAP az RFC 7252 szabványban definiált, REST-alapú alkalmazásprotokoll korlátos erőforrású eszközök és hálózatok számára. UDP fölött üzemel, üzenetmodellje a CON (Confirmable), NON (Non-confirmable), ACK (Acknowledgement) és RST (Reset) típusokra épül. Minden üzenet egyedi Message ID-t (MID) és opcionális tokent hordoz, az utóbbi a kérés–válasz párosítást szolgálja.

A CoAP-erőforrások URI-kkal címezhetők, például `coap://ts-node.local/sensors/temperature`. A fő metódusok megegyeznek a HTTP-évl: GET, PUT, POST, DELETE. A válaszkódok is hasonló felépítésűek, de tömörebbek: a 2.xx osztály a sikeres, a 4.xx a klienshiba, az 5.xx a szerverhiba jele. A leggyakoribb kódok: 2.04 Changed, 2.05 Content, 4.04 Not Found, 4.13 Request Entity Too Large, 5.00 Internal Server Error.

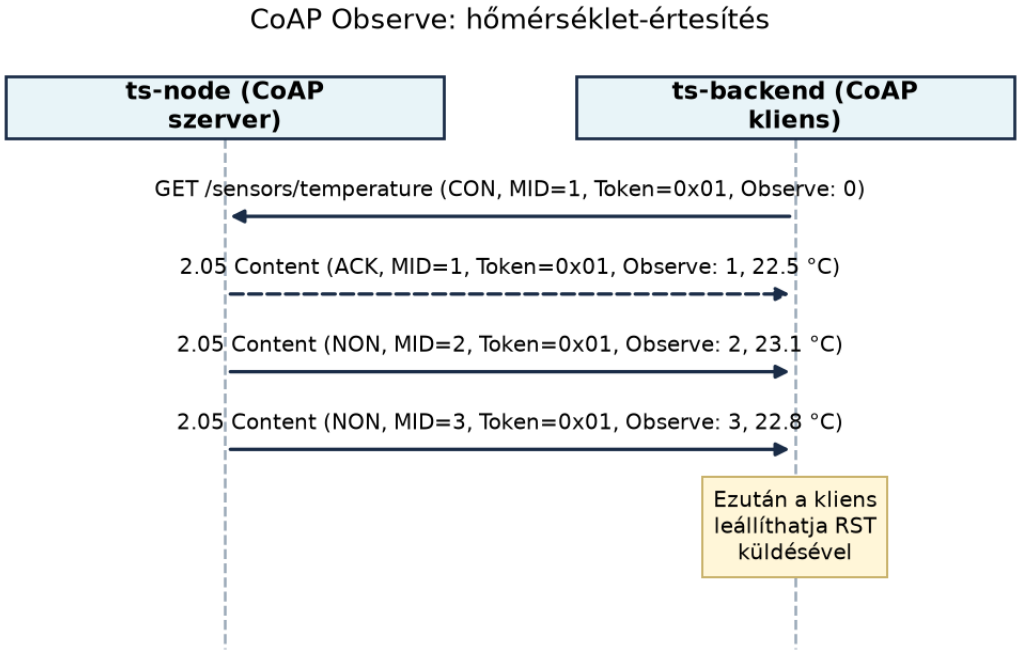
A CoAP megbízhatósági modellje eltér a TCP-től. A CON üzenetet a fogadó ACK-kal nyugtázza, ugyanazzal a MID-del. Ha az ACK nem érkezik meg egy véletlenített kezdeti időn belül, a küldő exponenciálisan növekvő késleltetéssel újraküldi az üzenetet (alapértelmezés szerint 2 és 3 másodperc közötti kezdeti érték, majd duplázódás). A NON üzenetet nem nyugtázza a fogadó, így a kézbesítés nem garantált, viszont nincs retransmission, ezért például periodikus telemetria küldésére alkalmas. A kérés–válasz lehet *piggybacked* (a válasz az ACK-ban utazik) vagy *separate* (ha a szerver nem tud azonnal válaszolni, külön CON/NON üzenetben küldi el a választ).

Az Observe (RFC 7641) kiterjesztés a kérés–válasz modellt egészíti ki értesítés-alapú frissítéssel. A kliens a GET kérésben Observe opciót helyez el, ezzel jelzi, hogy az erőforrás értékének változásairól folyamatosan értesítést szeretne kapni. A szerver a kezdeti értéket elküldi, majd minden változáskor egy újabb üzenetet küld ugyanazzal a tokennel, így a kliens össze tudja kapcsolni a frissítéseket. A frissítések lehetnek CON vagy NON típusúak; ha NON, akkor elveszhetnek, de a következő frissítés a legfrissebb állapotot hordozza.

A Block (RFC 7959) kiterjesztés nagyobb adategységek átvitelét teszi lehetővé kisebb blokkokban. A blokk méretét az SZX hárombites méretkitevő adja meg: a blokkméret $2^{\mathrm{SZX}+4}$ bájt, ahol SZX 0 és 6 közötti érték, így a lehetséges blokkméretek 16, 32, 64, 128, 256, 512 és 1024 bájt. A 64 bájtos blokkhoz tehát SZX=2 tartozik, nem 64. A NUM a blokk sorszáma, az M flag pedig azt jelzi, hogy van-e további blokk.

A Block1 a **kérés** törzsének darabolására szolgál (feltöltés, például PUT vagy POST): a kliens minden blokkot a Block1 opcióval küld, a szerver a köztes blokkokra 2.31 Continue választ ad, az utolsó blokkra pedig a művelet végső kódját (például 2.04 Changed). A Block2 a **válasz** törzsének darabolására szolgál (letöltés, például GET): a kliens a Block2 opcióval kéri a következő blokkot, a szerver pedig minden blokkra 2.05 Content választ ad a Block2 opcióval és az M flaggel; Block2 esetén nincs 2.31 Continue. A 2.31 Continue tehát nem általános „blokknyugta”, hanem kizárólag a Block1 köztes állapotának válasza.

Az alábbi ábra egy Observe regisztrációt és két frissítést mutat be a ts-node és a ts-backend között.



10.4. ábra: CoAP Observe: hőmérséklet-értesítés - A ts-node regisztrál a /sensors/temperature erőforrásra, majd a backend két frissítést kap a változó hőmérsékletről. A frissítések NON típusúak, ezért nem nyugtázottak.

A diagramon látható, hogy a kezdeti GET CON üzenet, mert a regisztrációt megbízhatóan kell eljuttatni. A válasz piggybacked ACK, tehát ugyanabban az üzenetben utazik. A későbbi frissítések NON típusúak, így nincs nyugtázási overhead, de a kliensnek számolnia kell az esetleges elvesztéssel.

10.4.3 Döntési szempontok

A CoAP és az MQTT közötti választás a TerraSense-ben az alábbi szempontokon múlik:

Szempont	CoAP	MQTT
Kommunikációs minta	Kérés–válasz, REST	Publish/subscribe
Szállítás	UDP (DTLS biztonsággal)	TCP (TLS biztonsággal)
Központi komponens	Nem igényel brokert, peer-to-peer	Broker szükséges

Szempont	CoAP	MQTT
Üzenetméret	Kicsi, bináris fejléc (4 B alap)	Kicsi, de TCP + MQTT fejléc
Megbízhatósági szintek	CON/NON, ACK, retransmission	QoS 0, 1, 2
Üzenetmegőrzés	Nincs retained	Retained, Last Will
Több fogadó	Egyedi címzés, multicast lehetséges	Több feliratkozó topic-onként
Erőforrás-igény (eszköz)	Alacsonyabb (UDP, nincs TCP-verem)	Magasabb (TCP-verem, MQTT-kliens)
Energiaprofil	Rövid adás, nincs kézfogás TCP-szinten	TCP kézfogás, kapcsolattartás

A TerraSense-ben a ts-node és a ts-gw közötti déli irány (rövid hatótáv, nagy megbízhatóság, eszközök sokasága) tipikusan BLE vagy LoRaWAN, ezért a CoAP itt nem közvetlenül jelenik meg. Azonban a ts-backend felé irányuló IP-alapú kapcsolatnál felmerülhet: ha egy eszköz közvetlenül UDP-t használ (például 6LoWPAN/Thread hálózaton), akkor a CoAP Observe hatékonyabb, mint egy MQTT-kliens TCP-kapcsolattal. Hasonlóan, a firmware-frissítésnél a CoAP blokkátvitel kifejezetten előnyös, mert a HTTP-hez vagy MQTT-hez képest kisebb overheaddel, szabványos módon, folytathatóan visz át nagy adatot.

10.4.4 Kidolgozott példa

A példában a ts-node egy 300 bájtos firmware-képet kap a ts-backendtől CoAP blokkátvitellel. A blokkméretet 64 bájtban állapítjuk meg, mert a szűkös pufferű eszköz így egyszerre csak kis darabot kell, hogy kezeljen. A 300 bájtos képet 5 blokkra osztjuk: négy darab 64 bájtos és egy darab 44 bájtos blokkra.

A kliens (ts-node) a `PUT /fw` erőforrásra küldi a blokkokat. Minden CON üzenetben a Block1 opció jelzi az aktuális blokk számát és az M flaget. Az első blokkban `NUM=0, M=1` (van több), az utolsóban `NUM=4, M=0` (nincs több). A szerver (ts-backend) minden köztes blokkra `2.31 Continue` választ ad, az utolsóra `2.04 Changed`-et. Az üzenetváltást a fenti blokkátviteli sequence diagram szemlélteti; itt a részleteket bontjuk ki.

Az első kérés üzenetformátuma (Csak a releváns mezők):

```
CoAP Header: Ver=1, T=CON, Code=0.03 (PUT), MID=100
Token: 0xAA
Option Block1: NUM=0, M=1, SZX=2 (64 bájt)
Payload: a firmware első 64 bájtja
```

Byte-szintű kódolás: a `PUT /fw` kérésben a token után előbb a Uri-Path opció (száma 11, értéke `fw`, 2 bájt) áll: fejléce `0xB2` (delta=11, length=2), majd `0x66 0x77`. A Block1 opció száma 27, az előző opcióhoz képest a delta 16, ami 4 biten nem fér el, ezért kiterjesztett delta-kódolás kell: a fejléc `0xD1` (delta=13, length=1), a kiterjesztett delta bájt `0x03` (13+3=16), az érték pedig `0x0A` (NUM=0, M=1, SZX=2, azaz `(0<<4)|(1<<3)|2`). Az utolsó, NUM=4-es blokk értéke `0x42` (`(4<<4)|(0<<3)|2`). A Block1 értékbájtban tehát a felső négy bit a NUM (egy bájtban legfeljebb 15), a 3. bit az M, az alsó három bit az SZX.

A szerver válasza `ACK` típusú, ugyanazzal a `MID=100`-zal, a `Token=0xAA`-val, `2.31 Continue` kóddal, és a Block1 opciót `NUM=0, M=1, SZX=2` értékkel visszhangozza, jelezve, hogy a 0. blokkot elfogadta, és további blokkokat vár. A kliens ezután `MID=101`-gyel küldi az 1. blokkot, és így tovább. Az utolsó blokk (`NUM=4`, `M=0`, 44 bájt) után a szerver `2.04 Changed`-et ad, amivel a teljes

kép átvitele sikeresen befejeződik. Ha bármelyik ACK elveszik, a kliens a CON retransmission szabályai szerint újraküldi az adott blokkot ugyanazzal a MID-del. Mivel a szerver idempotensen kezeli a blokkokat (a `Block1 NUM` alapján felismeri a duplikátumot), az újraküldés nem okoz adatsérülést. Ez a példa egyetlen, megszakítás nélküli feltöltést mutat; az újraindulás után is folytatható, tartós változatot a 3. feladat megoldása írja le.

Továbbvitt eredmény. A példában bevezetett blokkátviteli paramétereket (64 bájtos blokkméret, Block1 opció, a Block1 köztes válaszaként adott 2.31 Continue kezelése) használja a LwM2M: eszközmenedzsment objektummodellel alfejezet a firmware-frissítési objektum tárgyalásakor, valamint az OTA-frissítés: A/B képek, rollback, delta, fokozatos kiadás alfejezet a TerraSense OTA-folyamatának tervezésekor. Az itt kialakított konvenció: a firmware-kép erőforrása `coap://ts-node/fw`, a blokkméret 64 bájt, és a szerver a Block1 feltöltés minden köztes blokkjára 2.31 Continue-val válaszol.

10.4.5 Ellenőrzés és hibakeresés

A CoAP működését Linux hoston a `libcoap` `coap-client` és `coap-server` eszközeivel, vagy Pythonban az `aiocoap` könyvtárral ellenőrizhetjük. A laborban a ts-node-ot egy egyszerű CoAP-szerverrel szimulálhatjuk, a ts-backendet pedig egy CoAP-klienssel. A hálózati forgalmat Wiresharkkal foghatjuk el (`udp.port == 5683` szűrő).

Tünet	Ok	Javítás
A CON üzenetre nem érkezik ACK, a kliens folyamatosan újraküld	A szerver nem fut, vagy a port blokkolt	Indítsd el a szervert, ellenőrizd a tűzfalat (`netstat -uap`)
A kliens nem kap Observe frissítéseket	A regisztráció elmaradt, vagy a token nem egyezik	Ellenőrizd, hogy a GET Observe opciót tartalmazott-e, és a token azonos-e
A Block1 feltöltés elakad a 2. blokk után	A szerver nem küld 2.31 Continue-t, vagy a kliens nem kezeli	Vizsgáld a Block1 opciót a válaszban; a szervernek vissza kell adnia a NUM-ot
Duplikált blokkok érkeznek a szerverre	A CON retransmission és a blokkátvitel együtt	A szerver idempotensen kezelje a Block1 NUM alapján, és ne írja felül az adatot

A teszteléshez használjunk `coap-client -m get coap://127.0.0.1/sensors/temperature -s 5` parancsot, amely 5 másodpercig figyeli az Observe értesítéseket.

10.4.6 Tipikus hibák és félreértések

1. **A CON üzenet garantálja a kézbesítést.** A CON csupán azt jelenti, hogy a fogadó ACK-kal nyugtázza az üzenet vételét. Ez nem jelenti azt, hogy az alkalmazáslogika sikeresen feldolgozta a kérést. Gondoljunk a TCP-hez hasonlóan: az ACK a hálózati réteg nyugtája, nem az alkalmazásé. Ezért a kéréseket idempotensen kell megtervezni, különösen a PUT és DELETE műveleteknél.

2. **A CoAP Observe helyettesíti az MQTT pub/sub-ot.** Az Observe egyetlen kliens számára nyújt értesítést egyetlen erőforrásról. Nem támogat több feliratkozót, nincs broker,

ami a topicok alapján továbbítana. Ha sok eszköznek kell egyszerre látnia ugyanazt az adatot, az Observe nem skálázódik: minden kliens külön regisztrál, és a szerver mindegyiknek külön küld frissítést. Ilyenkor az MQTT a megfelelő választás.

3. **A blokkátvitelnél minden blokkra 2.31 Continue kell.** Ez csak a Block1 feltöltés köztes blokkjaira igaz (atomikus feldolgozás esetén); az utolsó Block1 blokkra a művelet végső kódja (például 2.04 Changed) érkezik. A Block2 letöltésnél egyáltalán nincs 2.31 Continue: minden válasz 2.05 Content, a Block2 opcióval és az M flaggel jelezve, hogy van-e további letöltendő blokk.

10.4.7 Feladatok

1. **Fogalmi kérdés:** Magyarázd el, mi a különbség a CoAP piggybacked és separate válasza között, és adj példát arra, mikor van szükség separate válaszra a TerraSense-ben.

2. **Számolási feladat:** A ts-backend `PUT /config` kérésekkel feltölt egy 1 500 bájtos konfigurációs fájlt a ts-node-ra, 128 bájtos blokkmérettel, Block1 opcióval. Hány blokkra oszlik az átvitel? Mennyi a teljes CoAP-overhead (UDP és IP fejléc nélkül), ha minden blokk CON üzenet, a válaszok piggybacked ACK-ok, és a szerver a Block1 opciót minden válaszban visszhangozza? Számold bele a 4 bájtos CoAP alapfejlécet, az 1 bájtos tokent, a `PUT /config` Uri-Path opcióját (6 bájtos érték), a Block1 opciót a delta-kódolással együtt, valamint a payload markert (0xFF, 1 bájt) minden kérésben; a válaszokban csak az alapfejléc, a token és a visszhangzott Block1 opció van, payload nélkül.

3. **Implementációs feladat:** Tervezd meg a TerraSense CoAP-erőforrásait a firmware-frissítéshez. Add meg az URI-struktúrát, a metódusokat, a szükséges opciókat, és írd le, hogyan kezelné a ts-node a blokkátvitel közbeni újraindulást (például hány blokk után menti el az állapotot). Ez a terv közvetlenül előkészíti a OTA-frissítés alfejezetet.

4. **Hibakeresési feladat:** Egy ts-node Observe regisztrációt küld a `/sensors/humidity` erőforrásra, de a ts-backend csak az első értéket kapja meg, utána nem érkezik frissítés, pedig a páratartalom változik. Sorolj fel legalább három lehetséges okot, és add meg mindegyikre a javítást.

10.4.8 Megoldások

1. Piggybacked válasz esetén a szerver a CON kérésre érkező ACK-ban azonnal elküldi a választ, mert a feldolgozás rövid. Separate válaszra akkor van szükség, ha a szerver nem tud azonnal válaszolni (például a ts-backendnek egy külső adatbázisból kell lekérnie az eszközárnyékot). Ekkor a szerver először egy üres ACK-ot küld, majd később egy külön CON/NON üzenetben a tényleges választ, ugyanazzal a tokennel.

2. A fájl 1 500 bájt, blokkméret 128 bájt. A blokkok száma $\lceil 1500/128 \rceil = 12$ (11 teljes blokk és egy 92 bájtos maradék). SZX 128 bájt esetén 3.

Egy kérés CoAP-overheadje: 4 bájt alapfejléc + 1 bájt token + 7 bájt Uri-Path (a `config` érték 6 bájtja, valamint a `0xB6` opciófejléc) + 3 bájt Block1 (a `0xD1` fejléc, a `0x03` kiterjesztett delta és az 1 bájtos érték, mert a NUM 11 alatt van) + 1 bájt payload marker = 16 bájt. A piggybacked válaszban csak a 4 bájtos alapfejléc, az 1 bájtos token és a visszhangzott Block1 opció van (első opcióként delta 27: `0xD1 0x0E` és 1 bájt érték), azaz 8 bájt; payload és payload marker nincs. Összesen 12 kérés és 12 válasz: $12 \times 16 + 12 \times 8 = 192 + 96 = 288$ bájt. A 11 köztes válasz 2.31 Continue, a tizenkettedik a végső 2.04 Changed kód; az utolsó kérés payloadja 92 bájt, de a

payload bájtjai nem részei az overheadnek. Ha a válasz is hordozna payloadot, azonként további 1 bájt payload marker jönne.

3. A firmware-frissítéshez a ts-node a `coap://ts-node/fw` erőforrást használja, de a folytatható feltöltéshez többrészes erőforrás-modell kell. A puszta `PUT /fw` nem elég, mert a szervernek ismernie kell az átvitel azonosítóját, a teljes hosszt és a teljes kép hash-ét is:

   - `POST /fw/transfers` `Content-Format: application/json` testtel: `{"transfer_id":"fw-2026-...", "total_size":..., "sha256":"...", "version":"..."}`. A válasz `2.01 Created`, a `Location` fejléc pedig `/fw/transfers/<id>`.
   - `PUT /fw/transfers/<id>?offset=0` Block1 opcióval, rögzített SZX-szel (a laborban SZX=2, azaz 64 bájt). A szerver a köztes blokkokra `2.31 Continue`, az utolsóra `2.04 Changed` választ ad; a válasz Block1 opciója a szerver által elfogadott NUM/M/SZX állapotot tükrözi.
   - `GET /fw/transfers/<id>` a szerver hiteles állapotát adja vissza: `{"next_offset":..., "received_bytes":..., "state":"receiving|verified|active"}`.
   - `POST /fw/transfers/<id>/commit` kéri a teljes SHA-256 ellenőrzését és az inaktív A/B slot aktiválását.

   A ts-node újraindulás után nem a saját NUM-jából és a részleges adat hash-éből következtet a folytatás helyére, hanem `GET`-tel lekérdezi a szerver `next_offset` értékét, és onnan folytatja. A `PUT` `offset` paramétere és a Block1 NUM a szerver állapotához igazodik; a blokkméretet az átvitel rögzíti, a NUM-ot az offset és a rögzített blokkméret adja. Ha a kliens nem a várt blokkot küldi, a szerver `4.08 Request Entity Incomplete` választ ad, mire a kliens újra lekérdezi a státuszt. Ha a szerver elvesztette az átvitel állapotát, `4.04 Not Found`-ot ad, és a kliens új `transfer_id`-vel elölről kezdi. A szerver a fogadott előtagot blokkonként tartósan (fsync vagy atomikus rename) tárolja, a teljes képet pedig csak a teljes SHA-256 egyezése után aktiválja; a részleges hash csak integritás-ellenőrzésre szolgál, nem a folytatás helyének meghatározására. Az aktiválás az inaktív A/B slotba ír, és ha az új kép nem indul el, a bootloader az előző slotra áll vissza. Ez a mechanizmus a OTA-frissítés alfejezetben kerül részletes kidolgozásra.

4. Lehetséges okok:

   - A ts-node frissítései NON típusúak, és a hálózaton elvesznek. Javítás: használjon CON frissítéseket, vagy alkalmazzon nyugtázott Observe módot (ha a kliens implementálja).
   - A ts-backend nem figyeli a bejövő frissítéseket, mert a kezdeti regisztráció után lezárta a socketet. Javítás: ellenőrizni kell, hogy a kliens a tokenhez társított állapotot fenntartja-e.
   - A ts-node erőforrásérték-változás észlelése hibás: a szenzor ugyan változik, de a CoAP szerver nem generál Observe eseményt. Javítás: az erőforrás frissítését helyesen kell jelölni a szerverben.
   - A hálózati tűzfal vagy NAT blokkolja a szervertől a kliens felé irányuló UDP csomagokat. Javítás: nyisson ki a megfelelő portot, vagy használjon DTLS-t a NAT átjáráshoz.

10.4.9 Rövid összefoglaló

A CoAP egy korlátos eszközökre tervezett REST-protokoll UDP fölött. Négy üzenettípust használ: CON, NON, ACK, RST. A CON üzeneteket ACK nyugtázza, elvesztés esetén exponenciális visszalépéssel újraküldés történik. A kérés–válasz párosítást a token biztosítja. Az Observe kiterjesztéssel a kliens folyamatos értesítést kap egy erőforrás változásáról. A Block kiterjesztéssel

nagy adatok vihetők át kis blokkokban: a Block1 a kérés, a Block2 a válasz darabolására szolgál. A CoAP és az MQTT közötti döntés a kommunikációs mintától, a broker szükségességétől és az eszköz erőforrásaitól függ. **Továbbvitt eredmény:** a TerraSense firmware-frissítési konvenciója CoAP Block1 feltöltés a `coap://ts-node/fw` alatti erőforrásokkal: 64 bájtos blokkméret, köztes válasz 2.31 Continue, utolsó 2.04 Changed, a folytatást pedig a szerver `next_offset` állapota vezérli. Ezt használja a LwM2M: eszközmenedzsment objektummodellel és az OTA-frissítés: A/B képek, rollback, delta, fokozatos kiadás alfejezet. A tervezett erőforrás-séma és az idempotens blokkkezelés kulcsfontosságú a megbízható frissítéshez.

10.4.10 Források és ellenőrzési hiányok

- CoAP alap: RFC 7252, `https://www.rfc-editor.org/rfc/rfc7252` (stabil szabvány).
- Observe kiterjesztés: RFC 7641, `https://www.rfc-editor.org/rfc/rfc7641` (stabil szabvány).
- Block kiterjesztés: RFC 7959, `https://www.rfc-editor.org/rfc/rfc7959` (stabil szabvány).
- A leírásban szereplő CoAP-kódok és opciók a fenti RFC-kből származnak, ezek jól ismert, nem változó definíciók. A `libcoap` és `aiocoap` eszközök pontos verzióját és CLI-kapcsolóit a felhasználó rendszerén ellenőrizni kell, mert azok verziófüggőek lehetnek.

10.5 LwM2M: eszközmenedzsment objektummodellel

Az LwM2M (Lightweight Machine-to-Machine) akkor válik szükségessé, amikor a TerraSense üvegházban már nem elég egyedi, saját fejlesztésű topicokon és REST-hívásokon keresztül menedzselni a csomópontokat. Ha a flottát üzemeltető rendszernek egységesen kell látnia az eszközök állapotát, konfigurációját, firmware-verzióját, és szabványos módon kell parancsokat kiadnia (öntözőszelep nyitás, újraindítás, frissítés indítása), akkor a CoAP-ra épülő, szabványos objektummodellel dolgozó LwM2M jelentős előnyt kínál: a backend és az eszköz között nem kell minden egyes művelethez egyedi protokollt és adatsémát tervezni. Ebben az alfejezetben az LwM2M-et mint a TerraSense menedzsmentrétegét vezetjük be, és az eszköz adatait szabványos objektumokhoz rendeljük.

10.5.1 Motiváció

A TerraSense rendszerben a telemetria már MQTT-n keresztül érkezik a backendbe, de az üzemeltetés során felmerülő kérdések – hány eszköz aktív, melyik milyen firmware-verziót futtat, hogyan lehet egy adott csomópontot újraindítani vagy firmware-frissítést indítani – túlmutatnak az MQTT-topicok egyszerű publish/subscribe modelljén. Egyedi megoldást írhatnánk: minden eszközhöz saját topicokat, JSON-sémákat és parancsformátumot definiálhatnánk, de ez gyorsan átláthatatlanná válik, és a különböző gyártók, eszközosztályok integrációja is nehézkes. Az LwM2M ezt a rést tölti be: egy szabványos, CoAP-alapú menedzsmentprotokollt ad, amelyben az eszköz erőforrásai objektum–példány–erőforrás hierarchiában címezhetők, és a műveletek (olvasás, írás, végrehajtás, megfigyelés) a CoAP módszereire épülnek. Így a TerraSense-ben a ts-node Zephyr-firmware-je LwM2M-klienst futtathat, a backend pedig egy LwM2M-szerverrel (például nyílt forrású megoldással) menedzselheti a flottát.

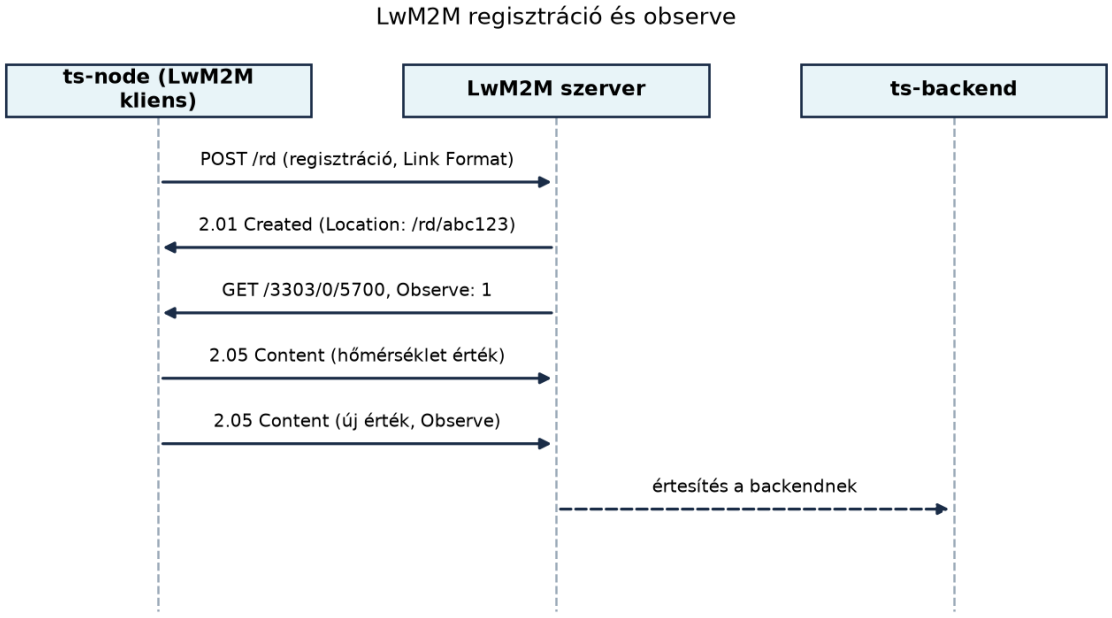
10.5.2 Fogalmak és működés

Definíció. Az OMA Lightweight M2M (LwM2M) egy szabványos eszközmenedzsment- és telemetriaprotokoll, amely a CoAP-ra épül, és az eszközök erőforrásait objektum–példány–erőforrás (Object–Instance–Resource) hierarchiában modellezi. Minden erőforrás URI-ja a `/objektum/példány/erőforrás` sémát követi, például `/3303/0/5700` a hőmérséklet-érzékelő aktuális értékét jelenti.

Definíció. Az LwM2M három fő szereplője: - **LwM2M-ügyfél (client):** az eszközön futó komponens, amely az erőforrásokat tárolja, és a CoAP-kéréseket fogadja. A TerraSense-ben ez a ts-node Zephyr-alkalmazása. - **LwM2M-szerver (server):** a központi menedzsmentkomponens, amely regisztrálja az ügyfelet, és műveleteket kezdeményez (olvasás, írás, megfigyelés, végrehajtás). A TerraSense-ben ez a backend része lehet, vagy külön szolgáltatás. - **Bootstrap-szerver (bootstrap server):** opcionális szereplő, amely az ügyfél kezdeti konfigurációját adja meg (például a menedzsmentszerver címét, biztonsági kulcsokat). A tiszta, nulláról indított eszközök ezen keresztül kapják meg a csatlakozáshoz szükséges paramétereket.

Az LwM2M-ben az eszköz állapotát nem önkényes JSON-mezőkkel, hanem szabványos objektumokkal írjuk le. A leggyakoribb objektumok: - `/3` – Device: gyártó, modell, sorozatszám, firmware-verzió, újraindítás. - `/5` – Firmware Update: frissítési kép, állapot, indítás. - A szenzorokhoz és aktuátorokhoz az IPSO Smart Objects katalógusa biztosít szabványos azonosítókat, például `/3303` hőmérséklet, `/3304` páratartalom, `/3305` teljesítménymérés és `/3306` aktuátor (Actuation). A szelepvezérléshez a `/3306` objektum `5850` On/Off erőforrása tartozik; a `/3305` objektum teljesítményadatot mér, nem aktuátort vezérel.

A hőmérséklet-érzékelő `5700` erőforrása az aktuális mért értéket adja, míg a Device objektum `0` erőforrása a gyártó nevét, `3` erőforrása a firmware-verziót.



10.5. ábra: LwM2M regisztráció és observe - A ts-node regisztrál az LwM2M-szervernél, majd a hőmérséklet erőforrást megfigyelés alá vonják.

A regisztráció: az ügyfél a szerver `/rd` erőforrására küldött CoAP POST-tal regisztrál. A payload a támogatott objektumpéldányok listája CoRE Link Format formátumban, például:

```
</3/0>,</3303/0>,</3304/0>,</3306/0>,</5/0>
```

A szerver válasza `2.01 Created`, és a Location fejlécben visszaadja a regisztráció azonosítóját (`/rd/abc123`). Ez az azonosító szükséges a későbbi frissítéshez és a lejárat kezeléséhez.

A regisztráció élettartamát az ügyfél a `lt` (lifetime) lekérdezési paraméterrel jelezheti, és a lejárat előtt frissítenie kell egy `POST /rd/abc123` üzenettel. Ha ez elmarad, a szerver törli az eszközt, és az nem menedzselhető tovább.

A **megfigyelés (observe)** a CoAP Observe opcióját használja: a szerver `GET` kérést küld az erőforrásra, az Observe opcióval. Az ügyfél válasza után minden értékváltozásnál újabb `2.05 Content` üzenetet küld, egészen addig, amíg a szerver le nem iratkozik. Ezzel a backend valós időben értesül a hőmérséklet változásáról, anélkül, hogy folyamatosan lekérdezné.

A **write** (írás) egy művelet, amellyel a szerver egy erőforrás értékét módosítja, például az aktuátor be- vagy kikapcsolására. CoAP `PUT` vagy `POST` üzenettel történik, a payload az új érték. A **execute** (végrehajtás) egy erőforráshoz rendelt művelet indítására szolgál (például `/3/0/4` reboot), CoAP `POST` üzenettel, payload nélkül.

A **Firmware Update objektum** (`/5`) előzetese: az eszköz firmware-frissítését szabványos módon menedzseli. A `/5/0/0` erőforrás a firmware-csomag, `/5/0/1` az URI, ahonnan a kép letölthető, `/5/0/2` az update művelet, `/5/0/3` pedig az állapot. Egy frissítés folyamata: a szerver beírja a kép URI-ját (`/5/0/1`), majd execute-et küld a `2` erőforrásra (Update). Az eszköz letölti, ellenőrzi és telepíti a firmware-t, majd frissíti az állapotot (`/5/0/3`). Ezt a TerraSense-ben az OTA-frissítés: A/B képek, rollback, delta, fokozatos kiadás alfejezet fogja részletesen felhasználni, az LwM2M mint a frissítési parancsok szállítórétege jelenik meg.

A Zephyr RTOS tartalmaz LwM2M-kliens támogatást: a Zephyr hálózati stack CoAP alapú, és létezik LwM2M kliens könyvtár. Azonban a pontos Kconfig-opciók, API-k és az aktuális LwM2M-verzió támogatása verziófüggő, ezért a konkrét neveket ellenőrzési igénnyel említjük. A labor ebben az alfejezetben nem a Zephyr-be építést mutatja be, hanem a TerraSense eszköz LwM2M-modelljét készíti el, amelyet később a Zephyr-kliens és a menedzsmentréteg használ.

10.5.3 Döntési szempontok

Az LwM2M nem helyettesíti az MQTT-t a telemetria nagy sebességű, sok eszközt érintő adatfolyamában, hanem a menedzsmentréteget egészíti ki. A következő táblázat összefoglalja, mikor melyik megoldás előnyös a TerraSense-ben.

Szempont	LwM2M	Egyedi MQTT-topicok + JSON
Szabványos eszközmodell	Igen, IPSO és OMA objektumok	Nem, minden sémát magunk definiálunk
Interoperabilitás más gyártókkal	Jó, a szabványt követő eszközök azonnal integrálhatók	Korlátozott, alkalmazásszintű adaptáció szükséges
Kis erőforrásigény	Alacsony, CoAP felett fut	Közepes, MQTT-kliens és JSON-parszer kell
Menedzsmentműveletek (reboot, OTA)	Beépített szabványos objektumok	Egyedi parancsok tervezése szükséges
Telemetria nagy gyakorisággal	Observe-szal hatékony, de sok erőforrásnál összetettebb	MQTT-ben természetes, kötegelt publish
Biztonság	DTLS a CoAP alatt	TLS a MQTT alatt

A TerraSense esetében a javasolt architektúra: a telemetria továbbra is MQTT-n fut (a meglévő topic-konvencióval), míg az eszközmenedzsment – flottaállapot, konfiguráció, firmware-frissítés, újraindítás – LwM2M-en keresztül történik. Ezzel a Flottakezelés: nyilvántartás, konfiguráció, állapot alfejezet szabványos alapokra épülhet, és az OTA-frissítés is az `/5` objektumot használja.

10.5.4 Kidolgozott példa vagy labor

Ebben a laborban a TerraSense csomópont LwM2M-modelljét készítjük el, és egy Python-szkriptet írunk, amely strukturálisan ellenőrzi a modellt, valamint előállítja a regisztrációs üzenet CoAP-payloadját. A labor célja, hogy az olvasó kézzel is lássa a címzési sémát, és a későbbi Zephyr-konfigurációhoz pontos specifikációt kapjon.

Munkakönyvtár: hozzuk létre a labor könyvtárát és lépjünk be.

```
mkdir -p ~/iot-labor/i030 && cd ~/iot-labor/i030
```

Az eszközmodellt egy JSON-fájlban adjuk meg, amely tartalmazza a TerraSense releváns LwM2M objektumait, példányait és erőforrásait. A `terrasense_lwm2m.json` fájl tartalma:

```
{
  "endpoint": "terrasense-gh1-n001",
  "objects": [
    {
      "id": 3,
      "name": "Device",
      "instances": [0],
      "resources": [
        {"id": 0, "name": "Manufacturer", "type": "string"},
        {"id": 1, "name": "Model", "type": "string"},
        {"id": 2, "name": "Serial Number", "type": "string"},
        {"id": 3, "name": "Firmware Version", "type": "string"}
      ]
    },
    {
      "id": 3303,
      "name": "Temperature",
      "instances": [0],
      "resources": [
        {"id": 5700, "name": "Sensor Value", "type": "float"}
      ]
    },
    {
      "id": 3304,
      "name": "Humidity",
      "instances": [0],
      "resources": [
        {"id": 5700, "name": "Sensor Value", "type": "float"}
      ]
    },
    {
      "id": 3306,
      "name": "Actuation",
      "instances": [0],
      "resources": [
        {"id": 5850, "name": "On/Off", "type": "boolean"}
      ]
    },
    {
      "id": 5,
      "name": "Firmware Update",
```

```
      "instances": [0],
      "resources": [
        {"id": 1, "name": "Package URI", "type": "string"},
        {"id": 2, "name": "Update", "type": "executable"},
        {"id": 3, "name": "State", "type": "integer"}
      ]
    }
  ]
}
```

A talajnedvességhez egyéni objektumot is felvehetünk, de ebben a laborban a hőmérsékletre, páratartalomra és a szelepvezérlésre koncentrálunk; az egyéni objektum később a flottakezelésben jelenik meg.

Most írjuk meg a `lwm2m_model.py` Python-szkriptet, amely beolvassa ezt a modellt, elvégzi a szerkezeti ellenőrzést (kulcsok, típusok, egyedi azonosítók), majd kiírja a regisztrációs payloadot és néhány példaművelet URI-ját.

```
import json

MODEL_FILE = "terrasense_lwm2m.json"
RESOURCE_TYPES = {"string", "float", "integer", "boolean", "executable"}

def load_model(path: str) -> dict:
    with open(path, encoding="utf-8") as f:
        return json.load(f)

def validate_model_structure(model: dict) -> None:
    if "endpoint" not in model or not isinstance(model["endpoint"], str):
        raise ValueError("Hiányzik vagy hibás az endpoint azonosító.")
    if "objects" not in model or not isinstance(model["objects"], list):
        raise ValueError("A modelnek objects listát kell tartalmaznia.")
    seen_objects = set()
    for obj in model["objects"]:
        for key in ["id", "name", "instances", "resources"]:
            if key not in obj:
                raise ValueError(f"Hiányzó kulcs az objektumban: {key}")
        if not isinstance(obj["id"], int):
            raise ValueError(f"Az objektum ID nem egész szám: {obj['id']}")
        if obj["id"] in seen_objects:
            raise ValueError(f"Duplikált objektum ID: {obj['id']}")
        seen_objects.add(obj["id"])
        if not isinstance(obj["instances"], list) or not all(
            isinstance(inst, int) for inst in obj["instances"]
        ):
            raise ValueError(f"Hibás instances lista: {obj['id']}")
        seen_resources = set()
        for res in obj["resources"]:
            for key in ["id", "name", "type"]:
                if key not in res:
                    raise ValueError(
                        f"Hiányos erőforrás az objektumban: {obj['id']}"
                    )
            if not isinstance(res["id"], int):
                raise ValueError(
                    f"Az erőforrás ID nem egész szám: {obj['id']}/{res['id']}"
                )
            if res["id"] in seen_resources:
                raise ValueError(
                    f"Duplikált erőforrás ID: {obj['id']}/{res['id']}"
                )
            if res["type"] not in RESOURCE_TYPES:
                raise ValueError(
```

```
                    f"Ismeretlen erőforrás-típus: {res['type']}"
                )
            seen_resources.add(res["id"])

def registration_payload(model: dict) -> str:
    entries = []
    for obj in model["objects"]:
        for inst in obj["instances"]:
            entries.append(f"</{obj['id']}/{inst}>")
    return ",".join(entries)

def main() -> None:
    model = load_model(MODEL_FILE)
    validate_model_structure(model)
    print(f"Endpoint: {model['endpoint']}")
    print("Regisztrációs payload (Link Format):")
    print(registration_payload(model))
    print("\nPéldaműveletek:")
    print(f"  Hőmérséklet olvasás: CoAP GET /3303/0/5700")
    print(f"  Szelep nyitás: CoAP PUT /3306/0/5850 payload: \"1\"")
    print(f"  Újraindítás: CoAP POST /3/0/4")

if __name__ == "__main__":
    main()
```

A `validate_model_structure()` szándékosan csak szerkezeti ellenőrzést végez: a kötelező kulcsokat, az azonosítók egész típusát és egyediségét, az `instances` listát, valamint az erőforrás-típusok ismert készletét vizsgálja. Nem ellenőrzi, hogy az objektum- és erőforrás-azonosítók megfelelnek-e az OMA/IPSO katalógusnak, sem az értéktartományokat és a hozzáférési jogokat; az egyéni objektumok miatt ezt séma nélkül nem is teheti. A szemantikai ellenőrzést a későbbi LwM2M-szerver vagy egy külön sémaellenőrző végzi.

A szkript futtatása:

```
python3 lwm2m_model.py
```

Várt kimenet (a `python3` a rendszerben elérhető, a modellfájl a munkakönyvtárban található):

```
Endpoint: terrasense-gh1-n001
Regisztrációs payload (Link Format):
</3/0>,</3303/0>,</3304/0>,</3306/0>,</5/0>

Példaműveletek:
  Hőmérséklet olvasás: CoAP GET /3303/0/5700
  Szelep nyitás: CoAP PUT /3306/0/5850 payload: "1"
  Újraindítás: CoAP POST /3/0/4
```

Továbbvitt eredmény. A laborban elkészített `terrasense_lwm2m.json` modell és a Python-szkript által generált regisztrációs payload a későbbi Flottakezelés: nyilvántartás, konfiguráció, állapot alfejezetben a flottanyilvántartás alapja lesz: az LwM2M objektumok és az endpoint-azonosító (`terrasense-gh1-n001`) egyértelműen meghatározza az eszköz menedzsmentmodelljét. Az OTA-frissítés: A/B képek, rollback, delta, fokozatos kiadás alfejezet pedig a `/5` Firmware Update objektum erőforrásait (különösen a `/5/0/1`, `/5/0/2`, `/5/0/3`) fogja használni a frissítési folyamat vezérlésére.

Takarítás: a labor fájljai törölhetők a `rm ~/iot-labor/i030/terrasense_lwm2m.json ~/iot-labor/i030/lwm2m_model.py` paranccsal, de a továbbvitt eredmények miatt érdemes megtartani őket.

10.5.5 Ellenőrzés és hibakeresés

A labor kimenetét úgy ellenőrizhetjük, hogy a regisztrációs payloadot összevetjük a JSON-fájlban szereplő objektumpéldányok listájával: minden objektum minden példánya pontosan egyszer jelenjen meg `<...>` formában, vesszővel elválasztva. A strukturális ellenőrzést a Python-szkript automatikusan elvégzi a kötelező kulcsokra, a típusokra és a duplikált azonosítókra; az IPSO/OMA szemantikát nem.

Gyakori hibatünetek és megoldásuk:

Hibatünet	Lehetséges ok	Javítás
A regisztrációs payloadban hiányzik egy objektum	Az objektum `instances` listája üres vagy rossz	Ellenőrizzük a JSON `instances` értékét
A Python-szkript `ValueError` hibát ad a duplikált erőforrás-ID miatt	Két erőforrás ugyanazzal az ID-vel szerepel	Az erőforrás-azonosítóknak egy objektumon belül egyedieknek kell lenniük
A kimenetben nem jelenik meg az endpoint	A JSON-ból hiányzik az `endpoint` kulcs	Adjuk hozzá az endpoint mezőt
A Link Format payload szintaktikailag helytelen	Hiányzik a `</` és `>` záró karakter	A generátor formázását ellenőrizzük; a mintakód helyesen zárja az elemeket

10.5.6 Tipikus hibák és félreértések

1. **A regisztráció élettartamának figyelmen kívül hagyása.** Ha az LwM2M-kliens nem frissíti a regisztrációt a lejárat előtt, a szerver törli, és az eszköz nem menedzselhető. A laborban ez nem jelenik meg, de éles rendszerben a lifetime kezelése kötelező.
2. **Az Observe opció elhagyása.** Sokan lekérdezik az erőforrást Observe nélkül, így a szerver csak pollingolással kapja az új értékeket, ami felesleges forgalmat és késleltetést okoz. Az LwM2M-ben a változásvezérelt adatszállításhoz mindig Observe-ot kell használni.
3. **A DTLS hiánya.** Az LwM2M CoAP-alapú, ezért ha nem használunk DTLS-t, a menedzsmentforgalom titkosítatlan. Ez különösen veszélyes az execute és write műveleteknél, például a szelep vezérlésénél.

10.5.7 Feladatok

10.5.7.1 a) Fogalmi kérdés

Magyarázza el, mi a szerepe a bootstrap-szervernek az LwM2M-ben, és adjon egy TerraSense-példát, amikor hasznos lehet.

10.5.7.2 b) Tervezési feladat

A TerraSense egy új talajnedvesség-szenzorral bővül. Tervezze meg az egyéni objektum azonosítóját (pl. 33001) és erőforrásait, majd egészítse ki a modellt a 0–100% relatív talajnedvesség érték számára. Indokolja a választott erőforrás-azonosítót.

10.5.7.3 c) Implementációs feladat

Módosítsa a `lwm2m_model.py` szkriptet úgy, hogy a regisztrációs payload mellett kiírja az összes erőforrás teljes URI-ját is, megjelölve, hogy melyik olvasható, írható vagy végrehajtható.

10.5.7.4 d) Hibakeresési feladat

Egy LwM2M-kliens regisztrál, de a szerver rendszeresen törli a regisztrációját. Mi lehet a hiba? Adjon két lehetséges okot és a javításukat.

10.5.8 Megoldások

10.5.8.1 a)

A bootstrap-szerver a kezdeti konfigurációt adja meg az LwM2M-ügyfélnek, mielőtt az bármilyen menedzsmentszerverhez csatlakozna. A TerraSense-ben például a gyártás után, az első beüzemeléskor a bootstrap-szerver megadhatja az üzemeltető LwM2M-szerverének URI-ját, a biztonsági tanúsítványokat és az eszköz egyedi azonosítóját. Így az eszköz tömeggyártásakor még nem kell minden egyes példányba fixen beégetni a szerver címét.

10.5.8.2 b)

Az egyéni talajnedvesség-objektum azonosítójaként válasszunk egy olyan számot, amely nem ütközik az OMA és IPSO szabványosított tartományával. A 33001 például az IPSO privát tartományához illeszkedik, de az üzemeltetőnek dokumentálnia kell. Az erőforrás-azonosító a 5700 legyen, mert az IPSO-konvenció szerint ez a szenzor aktuális értékének szabványos erőforrása. A modellben hozzáadhatjuk:

```
{
  "id": 33001,
  "name": "Soil Moisture",
  "instances": [0],
  "resources": [
    {"id": 5700, "name": "Sensor Value", "type": "float"}
  ]
}
```

Ezzel a relatív talajnedvesség 0–100% között olvasható.

10.5.8.3 c)

A szkript kiegészítése a teljes URI-k kiírására:

```
def print_resource_uris(model: dict) -> None:
    for obj in model["objects"]:
        for inst in obj["instances"]:
            for res in obj["resources"]:
                uri = f"/{obj['id']}/{inst}/{res['id']}"
                print(f"  {uri} ({res['name']})")

# A main()-ben:
print("Erőforrások:")
print_resource_uris(model)
```

A teljes módosított kód a korábbi főprogramba illeszthető, a fenti függvény meghívásával.

10.5.8.4 d)

Két lehetséges ok: 1. **A lifetime (`lt`) túl rövid, és az ügyfél nem frissíti időben a regisztrációt.** Javítás: növeljük a lifetime értéket, vagy gondoskodjunk a rendszeres regisztrációfrissítésről (CoAP POST a Location URI-ra) a lejárat előtt. 2. **Az ügyfél IP-címe vagy hálózati kapcsolata megváltozik, és a szerver nem ismeri fel.** Javítás: a kapcsolat visszaállítása után az ügyfél küldjön új regisztrációt, és a szerver törölje a régi rekordot.

10.5.9 Rövid összefoglaló

Az LwM2M egy CoAP-alapú, szabványos eszközmenedzsment-protokoll, amely az eszköz erőforrásait objektum–példány–erőforrás hierarchiában címezi. A TerraSense-ben az MQTT-telemetria mellett ez a menedzsmentréteg: a regisztráció `/rd`-n, a megfigyelés CoAP Observe-tal, a vezérlés write és execute műveletekkel történik. A laborban elkészítettük a `terrasense_lwm2m.json` modellt és a `lwm2m_model.py` szkriptet, amely strukturálisan ellenőrzi a modellt és generálja a regisztrációs payloadot. **Továbbvitt eredmény:** a modell endpoint-azonosítója és az objektumok (3, 3303, 3304, 3306, 5) a Flottakezelés: nyilvántartás, konfiguráció, állapot alapját képezik, az `/5` Firmware Update objektum pedig az OTA-frissítés: A/B képek, rollback, delta, fokozatos kiadás alfejezetben kerül felhasználásra. A Zephyr LwM2M-kliens pontos konfigurációja ellenőrzési igényű, a következő fejezetekben dolgozzuk ki.

10.5.10 Források és ellenőrzési hiányok

- OMA Lightweight M2M (LwM2M) specifikációk: a regisztráció, objektummodell és a Firmware Update objektum definíciói az OMA hivatalos dokumentumaiban találhatók.
- IETF RFC 7252 (CoAP) és RFC 7641 (Observe) – az LwM2M alapjául szolgáló protokollok.
- Ellenőrzési igény: a Zephyr LwM2M-kliens aktuális Kconfig-nevei, API-jai és a támogatott LwM2M-verzió a Zephyr adott kiadásától függ; a pontos neveket a hivatalos Zephyr-dokumentációban vagy a forráskódon keresztül kell ellenőrizni a build előtt.

10.6 Ipari protokollok: Modbus és OPC UA

10.6.1 Motiváció

A TerraSense üvegházban a levegő hőmérsékletét, páratartalmát és a talajnedvességet nemcsak a saját Zephyr-alapú `ts-node` eszköz méri, hanem gyakran meglévő ipari berendezések, például PLC-k, klímavezérlők vagy öntözőszivattyú-vezérlők is előállítják. Az ilyen eszközök többsége a Modbus protokollt beszéli, amely egyszerű regiszteralapú adatcserét kínál. Nagyobb telephelyeken pedig egyre gyakoribb az OPC UA, amely gazdag információs modelljével egységes képet ad a teljes gyártási vagy épületautomatizálási rendszerről. A `ts-gw` gateway feladata az lesz, hogy ezekből az ipari adatforrásokból adatot nyerjen ki, és azt a TerraSense MQTT topic-konvenciója szerint továbbítsa a broker felé. Ebben az alfejezetben a Modbus és az OPC UA alapjait tanuljuk meg, majd egy Modbus TCP-szimulátorból olvasunk ki regisztereket. A kapott adatforrás-kódot a későbbi gateway-implementáció használja fel.

10.6.2 Fogalmak és működés

10.6.2.1 Modbus: master/slave, regiszterek, függvénykódok

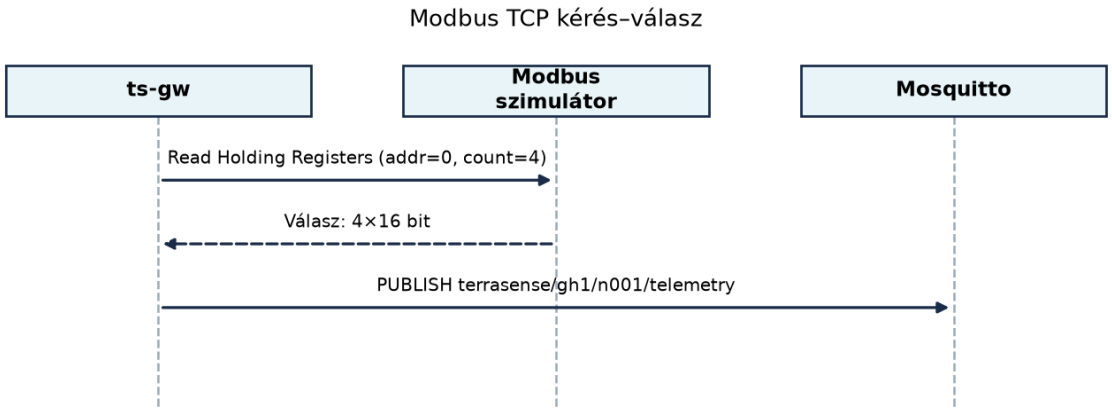
Definíció. A Modbus egy alkalmazási szintű üzenetprotokoll, amely mester–szolga (master–slave) viszonyt valósít meg. A mester – tipikusan egy PLC, számítógép vagy gateway – kezdeményezi a kérést, a szolga pedig válaszol. A szolga adatmodellje címzett 1 bites és 16 bites regiszterekből áll.

A klasszikus elnevezés master/slave, de Modbus TCP esetén gyakran kliens/szerver párt mondunk: a TCP-kliens a mester, a TCP-szerver a szolga. A protokoll ugyanaz, csak a szállítási réteg változik.

A Modbus négy alapvető adattípust különböztet meg:

Adattípus	Méret	Irány a mester felől	Tipikus funkciók
Coil (kapcsoló kimenet)	1 bit	olvasható/írható	FC01, FC05, FC15
Discrete input (diszkrét bemenet)	1 bit	csak olvasható	FC02
Input register (bemeneti regiszter)	16 bit	csak olvasható	FC04
Holding register (tartóregiszter)	16 bit	olvasható/írható	FC03, FC06, FC16

A gyakorlatban a mérési értékeket általában holding registerekben vagy input registerekben tárolják. A függvénykód (function code, FC) mondja meg, hogy milyen műveletet kér a mester. Például az FC03 *Read Holding Registers* egy kezdőcímet és darabszámot kap, a szolga pedig a regiszterek értékét adja vissza. Az FC16 *Write Multiple Registers* egyszerre több regisztert ír.



10.6. ábra: Modbus TCP kérés–válasz - A ts-gw Modbus TCP-kliensként olvas a szimulátorból, majd a dekódolt értéket MQTT-re továbbítja.

A Modbus regiszter 16 bites, a bájtsorrend a protokollban big-endian: a magasabb helyiértékű bájt érkezik előbb. A probléma akkor jelentkezik, ha egy 32 bites értéket – például egy lebegőpontos hőmérsékletet – két regiszterben tárolnak. A Modbus specifikáció nem rögzíti a regiszterek sorrendjét, ezért a gyártói konvenciók eltérhetnek: az egyik eszköz *high word first* (a kisebb

című regiszter tartalmazza a felső 16 bitet), a másik *low word first* sorrendet használ. Ezt a regisztertérképet olvasva mindig ellenőrizni kell, és szükség esetén meg kell cserélni a két regisztert.

10.6.2.2 OPC UA: csomópontok, címtér, szolgáltatások

Definíció. Az OPC UA platformfüggetlen, szolgáltatásorientált adatcsere- és információs modell. A szerver címtérben tárolja az elérhető adatokat; a címtér csomópontokból (Node) áll, amelyek attribútumokkal és referenciákkal írják le az objektumokat, változókat és metódusokat.

Minden csomópontot egyedi azonosítóval, a NodeId-val azonosítunk. A NodeId egy névtér-indexet (namespace) és egy azonosítót tartalmaz, például `ns=2;s=Temperature` vagy `ns=2;i=1001`. A címtér hierarchikus: a csomópontokra referenciák mutatnak, így lehet bejárni a gépek, szenzorok és azok paramétereinek faszerkezetét. Egy csomópont attribútumai között van például a megjelenítendő név, az adattípus, a hozzáférési szint és a tényleges érték.

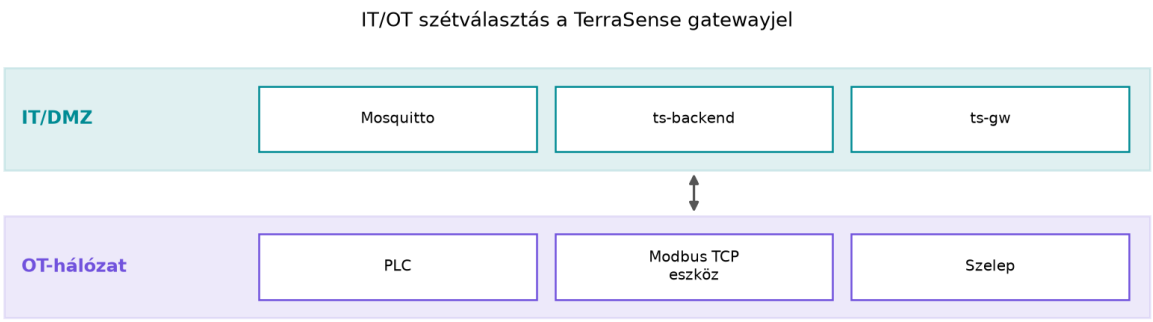
Az OPC UA szolgáltatások közé tartozik a Read (érték kiolvasása), a Write (írás), a Browse (címtér bejárása), a Subscription/ Monitoring (értesítés értékváltozáskor) és a Call (távoli metódus hívása). A biztonsági módok három szintje: `None` (csak zárt, fizikailag védett hálózaton), `Sign` (az üzenetek aláírtak, de nem titkosítottak) és `SignAndEncrypt` (aláírt és titkosított). A biztonságos üzemhez X.509 tanúsítványokat használnak, így a kliens és a szerver kölcsönösen azonosítja magát.

10.6.2.3 OPC UA Pub/Sub és MQTT

Az OPC UA klasszikus kliens–szerver modellje mellett a szabvány 1.04-es verziójától létezik **Pub/Sub** üzemmód is. Ebben a modellben a Publisher a beállított adatkészleteket üzenetekben teszi közzé, a Subscriberek pedig ezeket fogadják. A szállítási réteg lehet például UDP, AMQP vagy MQTT. MQTT esetén az OPC UA JSON-kódolású hálózati üzeneteket publikál egy topicra, és a fogadó oldalon az OPC UA alkalmazás visszaállítja az információs modellt. Az MQTT-transzport tűzfalbarát és jól illeszkedik a TerraSense brokeréhez. Hátránya, hogy a QoS-t, a munkamenet-tartást és a retenciót az MQTT-tulajdonságok biztosítják, ezért az OPC UA Pub/Sub rendszerben is pontosan kell beállítani a topicokat és a minőségi paramétereket.

10.6.2.4 IT/OT szétválasztás és biztonsági megfontolások

Az üzemi hálózat (Operational Technology, OT) és az irodai, illetve felhőalapú hálózat (Information Technology, IT) szétválasztása alapvető ipari védelmi elv. A Modbus sem hitelesítést, sem titkosítást nem tartalmaz, ezért csak zárt, védett OT-hálózatban szabad használni. Az OPC UA már kínál biztonsági réteget, de a `None` módot csak lokálisan, a védett szegmensben szabad engedélyezni. A `ts-gw` a DMZ-ben helyezkedik el: az OT-hálózat felé kliensként olvassa a PLC-ket, az IT-hálózat felé pedig kifelé irányuló, mTLS-alapú MQTT-kapcsolatot nyit. Így az OT-hálózatnak nincs közvetlen internetelérése.



10.7. ábra: IT/OT szétválasztás a TerraSense gatewayjel - Az OT-hálózat ipari protokolljai zárt VLAN-ban futnak; a ts-gw a DMZ-ben ül, és csak kifelé indít MQTT/TLS-t.

10.6.3 Döntési szempontok

A három protokoll közötti választás elsősorban a modellezési igénytől, a rendelkezésre álló erőforrásoktól és az integrációs környezettől függ.

Szempont	Modbus	OPC UA	MQTT
Modell	lapos regisztercímek	gazdag, szemantikus címtér	topic-hierarchia, szabad séma
Erőforrásigény	nagyon alacsony	közepes–magas	alacsony–közepes
Biztonság	nincs a protokollban	X.509, aláírás, titkosítás	TLS/mTLS a szállítás alatt
Üzenetminta	kérés–válasz	kérés–válasz, subscription	publish/subscribe
Tipikus hely	PLC-k, kis szenzorok, régi eszközök	gépek, cellák, MES-ek	felhőintegráció, eszköz–backend

A TerraSense-ben a `ts-node` közvetlenül MQTT-t használ, mert energiatakarékos és a brokerrel szemben kérés nélkül is közzétehet. A meglévő PLC-ket viszont Modbus-on kell lekérdezni, ezért a gateway egy Modbus TCP-kliens komponenst kap. Ha a telephely egyszer már OPC UA-ban szabványosított, akkor a gateway OPC UA-kliensként olvashat, és az így kapott modellt leképezi MQTT-re.

10.6.4 Kidolgozott példa és labor: Modbus TCP-regiszterek kiolvasása

10.6.4.1 Regisztertérkép és dekódolás

Tegyük fel, hogy a szimulált PLC a következő holding registereket szolgáltatja a 0. címtől kezdve:

Cím	Regiszter	jelentés	méretezés
0	HR0	hőmérséklet	0,1 °C egység (value / 10)

Cím	Regiszter	jelentés	méretezés
1	HR1	relatív páratartalom	0,1 % egység (value / 10)
2–3	HR2, HR3	talajnedvesség	32 bites IEEE 754 float, high word first

A szimulátor a következő nyers értékeket adja: HR0=234, HR1=650, HR2=0x41A0, HR3=0x0000. A dekódolás lépései:

- Hőmérséklet: $T = 234/10 = 23{,}4\,^{\circ}\mathrm{C}$.
- Páratartalom: $H = 650/10 = 65{,}0\,\%\mathrm{RH}$.
- Talajnedvesség: a két 16 bites regisztert big-endian sorrendben összeillesztve $0x41A00000$ értéket kapunk, amely IEEE 754 single-precision floatként éppen 20,0 (%). Ha valaki low word first-ként értelmezné, $0x000041A0$ érték jönne ki, ami közel nulla, nyilvánvalóan hibás.

Ez a példa mutatja, miért kell a gyártói regisztertérképet és a szó-sorrendet pontosan ismerni.

10.6.4.2 Labor: Modbus TCP-szimulátor konténerben

Munkakönyvtár: hozd létre és lépj be:

```
mkdir -p ~/iot-labor/i031 && cd ~/iot-labor/i031
```

A telemetria-séma ebben a laborban még csak a kimeneti JSON-szinten jelenik meg; a végleges sémát az Adatformátumok: JSON, CBOR, Protocol Buffers, SenML alfejezet rögzíti. A laborban a `ts-gw` gatewayt egy Python-prototípussal helyettesítjük, amely kiolvassa a Modbus-regisztereket, és az eredményt JSON-ként írja ki.

Először a közös `~/iot-labor/stack/compose.yaml` fájlba vegyünk fel egy `modbus-sim` szolgáltatást. A `volumes` útvonala a compose-fájl könyvtárához (`~/iot-labor/stack`) viszonyított, ezért a `../i031` a `~/iot-labor/i031` munkakönyvtárra mutat. A `pymodbus` verzióját kötjük, mert a 3.10.0-ban az API több ponton átneveződött (`ModbusDeviceContext`, `devices=`, `device_id=`); az alábbi kód a 3.9.2-es API-t használja:

```
services:
  modbus-sim:
    image: python:3.12-slim
    container_name: lab-modbus-sim
    ports:
      - "1502:502"
    volumes:
      - ../i031:/app:ro
    command: sh -c "pip install --no-cache-dir pymodbus==3.9.2 && python /app/simulator.py"
```

Hozd létre a `~/iot-labor/i031/simulator.py` fájlt:

```
from pymodbus.server import StartTcpServer
from pymodbus.datastore import ModbusSlaveContext, ModbusServerContext
from pymodbus.datastore import ModbusSequentialDataBlock

# A pymodbus 3.9.2 a kliens 0-s PDU-címéhez +1-et ad, ezért a blokk
# báziscíme 1: így a 0-s cím a lista első elemére (234) képződik le.
store = ModbusSlaveContext(
    di=ModbusSequentialDataBlock(1, [0] * 10),
    co=ModbusSequentialDataBlock(1, [0] * 10),
    hr=ModbusSequentialDataBlock(1, [234, 650, 0x41A0, 0x0000]),
```

```
    ir=ModbusSequentialDataBlock(1, [0] * 10),
)
context = ModbusServerContext(slaves=store, single=True)

print("Modbus TCP szimulátor a 0.0.0.0:502 porton...")
StartTcpServer(context=context, address=("0.0.0.0", 502))
```

Indítsd el a szimulátort a stack könyvtárából:

```
cd ~/iot-labor/stack && docker compose up -d modbus-sim
```

A kliens a hoston fut. Telepítsd a `pymodbus` csomagot egy virtuális környezetbe; a szerverrel azonos, 3.9.2-es verziót használj, mert a kliens `slave=` paramétere a 3.10.0-tól `device_id=` néven él tovább. Ezután hozd létre a `~/iot-labor/i031/modbus_client.py` fájlt:

```
import struct
from pymodbus.client import ModbusTcpClient

client = ModbusTcpClient("127.0.0.1", port=1502)
client.connect()

rr = client.read_holding_registers(address=0, count=4, slave=1)
if rr.isError():
    print(f"Modbus hiba: {rr}")
else:
    temp = rr.registers[0] / 10.0
    hum = rr.registers[1] / 10.0
    raw_float = struct.pack(">HH", rr.registers[2], rr.registers[3])
    soil = struct.unpack(">f", raw_float)[0]
    result = {
        "node_id": "n001",
        "ts": None,
        "seq": 0,
        "temperature_c": temp,
        "humidity_rh": hum,
        "soil_moisture_pct": soil,
    }
    print(result)

client.close()
```

Futtatás:

```
cd ~/iot-labor/i031
python3 -m venv venv
venv/bin/pip install pymodbus==3.9.2
venv/bin/python modbus_client.py
```

Várt kimenet (a `ts` és `seq` most még semleges érték):

```
{'node_id': 'n001', 'ts': None, 'seq': 0, 'temperature_c': 23.4, 'humidity_rh': 65.0,
↪  'soil_moisture_pct': 20.0}
```

A labor takarítása a konténer leállítása és a hálózat eltávolítása, de a létrehozott szkriptek a későbbi felhasználáshoz megmaradhatnak:

```
cd ~/iot-labor/stack && docker compose stop modbus-sim && docker compose rm -f modbus-sim
```

Továbbvitt eredmény. A `modbus_client.py` által előállított JSON-szerkezet – `node_id`, `ts`, `seq`, valamint a mértékegység-azonosító kulcsok, mint `temperature_c`, `humidity_rh`, `soil_moisture_pct` – a gateway felőli ipari adatforrás prototípusa. A A ts-gw szolgáltatás C++-ban: protokollfordítás MQTT-re alfejezet ezt a mintát valósítja meg C++17-ben, és köti össze a TerraSense `terrasense/gh1/n001/telemetry` topicjával.

10.6.5 Ellenőrzés és hibakeresés

A Modbus-kliens működését legegyszerűbben a dekódolt értékekkel ellenőrizzük. Ha a kimenet 23,4 °C, 65,0 %RH és 20,0 % talajnedvesség, akkor a regiszterek címe és a szó-sorrend helyes. A regisztertérkép érvényességét a szimulátor `hr` blokkjának módosításával is tesztelhetjük.

Tünet	Ok	Javítás
A kliens nem csatlakozik	a szimulátor nem fut, vagy a port mapping hibás	`docker compose ps` ellenőrzés; a host-port 1502, konténer 502
`Modbus Error` vagy timeout	a slave ID nem egyezik, vagy a cím érvénytelen	a `slave=1` és az `address=0` ellenőrzése
A hőmérséklet 2340 °C-nak látszik	a méretezést (÷10) elfelejtetted	használd a `/ 10.0` osztást
A talajnedvesség közel nulla	a két regiszter szó-sorrendje fel van cserélve	a `struct.pack(">HH", ...)` big-endian sorrend megtartása

10.6.6 Tipikus hibák és félreértések

32 bites érték regiszter-sorrendjének felcserélése. A Modbus csak 16 bites regisztereket szállít, a gyártói dokumentációk pedig gyakran nem írják egyértelműen, hogy a kisebb című regiszter a felső vagy az alsó 16 bitet tartalmazza. A téves sorrend nem mindig azonnal látszik, mert az érték formailag érvényes float, de teljesen hibás. Mindig egy ismert értékkel (például a szimulátor 20,0-es értékével) kell ellenőrizni a dekódolást.

Modbus TCP-nél a slave ID elhagyása. Sok fejlesztő azt gondolja, hogy TCP/IP esetén a slave ID felesleges, pedig a Modbus TCP átvitelben a Unit Identifier mező ugyanúgy jelen van. Ha a szerver több szolga-címet szolgál ki, vagy a gyártói implementáció megköveteli, a `slave=1` elhagyása hibás választ vagy timeoutot okoz.

10.6.7 Feladatok

1. Fogalmi kérdés: mi a különbség a coil és a holding register között, és melyiket használnád egy szelep beállításához?
2. Tervezési feladat: egy PLC dokumentációja szerint a hőmérséklet a 40001-es címen található, 0,01 °C felbontással. Hányas 0-alapú Modbus-címet kell kiolvasnod, és milyen osztóval dekódolod az értéket?
3. Implementációs feladat: bővítsd a szimulátort egy új holding registerrel, amely az öntöző-szelep állapotát jelzi: 0 = zárva, 1 = nyitva. Frissítsd a kliens JSON-kiírását is.
4. Hibakeresési/kiterjesztési feladat: az OPC UA Pub/Sub MQTT-transzporttal hogyan képeznéd le a TerraSense topic-konvencióra egy OPC UA Publisher által küldött adatkészletet? Mit tennél, ha a Publisher topicja nem illeszkedik a `terrasense/<site>/<node>/telemetry` sémához?

10.6.8 Megoldások

1. A coil 1 bites, írható/olvasható kimenet; a holding register 16 bites, olvasható/írható. A szelep vezérlésére coil is elég, ha kétállapotú, de ha analóg pozíciót vagy több állapotot akarunk küldeni, holding registert választunk.

2. A Modbus egyezmény szerint a 40001-es emberi címen a holding register 0. címét kell olvasni (`address=0`). A felbontás 0,01 °C, tehát a nyers értéket 100-zal kell osztani.
3. A szimulátor `hr` blokkjában a 4. regiszter legyen `valve` érték, például `ModbusSequentialDataBlock(1, [234, 650, 0x41A0, 0x0000, 1])` (a báziscím 1 marad). A kliens `read_holding_registers(address=0, count=5, slave=1)` hívása után a `valve_state = "open" if rr.registers[4] == 1 else "closed"` mező kerül a JSON-ba.
4. Az OPC UA Pub/Sub MQTT-transzport esetén a Publisher általában saját topicot használ. A TerraSense-konvenció betartásához a `ts-gw` gatewayre van szükség, amely feliratkozik a Publisher topicjára, az üzenetet normalizálja, és a `terrasense/<site>/<node>/telemetry` topicra publikál tovább. Így az ipari forrás belső topic-elrendezése nem szivárog be a backendbe, és a gateway egységes sémát biztosít.

10.6.9 Rövid összefoglaló

A Modbus egy egyszerű, regiszteralapú master–slave protokoll, amely 1 bites coilokat és 16 bites regisztereket használ. A regisztertérkép és a függvénykódok ismerete nélkülözhetetlen a helyes dekódoláshoz, különösen 32 bites értékeknél a szó-sorrend megállapításához. Az OPC UA gazdag, node-alapú címteret, szolgáltatásokat és erős biztonsági módokat kínál, Pub/Sub üzemmódja pedig MQTT-vel is összekapcsolható. Az IT/OT szétválasztás alapelv: a Modbus és az OPC UA az OT-hálózaton fut, a DMZ-ben lévő gateway fordítja le az adatokat MQTT-re. **Továbbvitt eredmény:** a `~/iot-labor/i031/modbus_client.py` kimeneti JSON-szerkezete (`node_id`, `ts`, `seq`, mértékegység-azonosító mezők) és a `modbus-sim` compose-szolgáltatás a A ts-gw szolgáltatás C++-ban: protokollfordítás MQTT-re alfejezet ipari adatforrás-mintája.

10.6.10 Források és ellenőrzési hiányok

- Modbus Application Protocol Specification V1.1b3, Modbus Organization.
- OPC UA szabványcsalád, IEC 62541.
- MQTT 5.0 specifikáció: OASIS, `https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html`.
- A labor a `python:3.12-slim` base image-re és a `pymodbus==3.9.2` csomagra épül; a kód a 3.9.2 `ModbusSlaveContext`/`ModbusServerContext`/`slave=` API-ját használja. A 3.10.0 ezeket `ModbusDeviceContext`, `devices=` és `device_id=` névre nevezte át, ezért a verziót a szerveren és a kliensen is rögzíteni kell. A konténer-alapú szimulátor és a compose-szolgáltatás viselkedését a helyi környezetben ellenőrizni kell.

11 Adatformátumok, sémák és idő

11.1 Adatformátumok: JSON, CBOR, Protocol Buffers, SenML

11.1.1 Motiváció

A TerraSense csomópontja a mérést bináris struktúrában, `ts_sample`-ként állítja elő, a gateway pedig MQTT-n továbbítja a backendnek. A topic-konvenció már rögzíti, hogy a telemetria a `terrasense/<site>/<node>/telemetry` topicra kerül, de az üzenet törzsének formátumáról még nem döntöttünk. A választás hosszú távon meghatározza a sávszélességet, a feldolgozási költséget, az eszközoldali memóriaigényt és a séma evolúcióját. Egy LoRaWAN-os csomópontnál minden bájt fizetős, míg a backend fejlesztésekor a JSON gyors hibakeresést tesz lehetővé. Ebben az alfejezetben összevetjük a négy legfontosabb formátumot, és véglegesítjük a TerraSense telemetria-sémáját.

11.1.2 Fogalmak és működés

11.1.2.1 JSON: olvasható, de nagy

Definíció. A JSON (JavaScript Object Notation) szöveges adatcsere-formátum, amely objektumokat kulcs–érték párokkal, tömböket és skalárokat (szám, string, logikai, null) ábrázol. Az RFC 8259 rögzíti, hogy a számok a JavaScripthez hasonlóan IEEE 754 double pontosságú lebegőpontos értékként kezelendők, hacsak a fogyasztó másképp nem állapodik meg.

A JSON előnye, hogy minden platformon azonnal olvasható és könnyen logolható. A hátránya a méret és a számábrázolás: nincs külön egész és lebegőpontos típus, a 0,1 nem pontosan reprezentálható, és a 2^{3}-nél nagyobb egészek elveszíthetik a pontosságot. Az epoch ms időbélyegünk $1{,}73 \cdot 10^{12}$ nagyságrendű, tehát még belefér a double egész pontosságú tartományába, de a mérési értékeknél a `22.5` mögött a fogyasztó `22.499999`-et is láthat.

11.1.2.2 CBOR: bináris JSON-modell

Definíció. A CBOR (Concise Binary Object Representation) az RFC 8949-ben definiált bináris formátum, amely a JSON adatmodellt bővíti ki: bájtstring, nem szöveges map-kulcsok, címkék (tag) és lebegőpontos félpontosság is elérhető.

A CBOR ugyanazt a szemantikát kódolja, de kompakt bájtokba. Például egy rövid szöveget egy hossz- és típusbájttal, egy uint64-et pedig legfeljebb 9 bájttal ábrázol. A Zephyrben a `zcbor` könyvtár elérhető CBOR kódoláshoz és dekódoláshoz, de a konkrét Kconfig-nevét és API-ját a telepített Zephyr-változatban ellenőrizni kell.

11.1.2.3 Protocol Buffers: séma és kódgenerálás

Definíció. A Protocol Buffers (protobuf) a Google által fejlesztett, sémaalapú bináris formátum. A `.proto` fájlban mezőszámokkal és típusokkal írjuk le az üzenetet, a `protoc` generátor pedig célnyelvi osztályokat és sorosító kódot készít.

A protobuf wire-formátuma a mezőket nem névvel, hanem mezőszámmal azonosítja. A mezőazonosító egy bájtba sűríti a mezőszámot és a wire típust: `tag = (field_number << 3) | wire_type`. A varint egész kódolás a kis számokat 1 bájtba kódolja, a lebegőpontos `float` 4 bájt, a string és a beágyazott üzenet hosszprefixes. Mikrokontrollerekre a `nanopb` implementáció használható, amely Zephyrben modulként érhető el; a pontos verzió és az integráció ellenőrzést igényel.

11.1.2.4 SenML: szabványos mérési rekord

Definíció. A SenML (Sensor Measurement Lists) az RFC 8428-ban definiált formátum, amely mérési rekordok listáját írja le. Minden rekordnak van neve (`n`), egysége (`u`), értéke (`v`) és időbélyege (`t`), valamint opcionális bázismezei (`bn`, `bt`, `bu`).

A SenML előnye, hogy az egységek egy szabványos regisztrációból származnak, például `Cel`, `%RH`, `%`. Ezért a TerraSense akkor is értelmezhető marad, ha a mérést nem a saját backendünk, hanem egy másik fél fogadja. A SenML kódolható JSON-ban, CBOR-ban vagy EXI-ben is.

11.1.3 Döntési szempontok

A négy formátum közötti választást a következő táblázat foglalja össze.

Szempont	JSON	CBOR	Protobuf	SenML
Méret	nagy	közepes	kicsi	közepes
Olvashatóság	igen	nem	nem	igen
Séma	nincs, de JSON Schema	nincs, de CDDL	van, `.proto`	van, RFC 8428
Feldolgozási költség	közepes	alacsony	nagyon alacsony	közepes
Eszközök	mindenhol	Zephyr `zcbor`	`nanopb`	kevés
Interoperabilitás	jó	jó	csak protobuf-fal	kiváló

A TerraSense döntése: a telemetria elsődlegesen **JSON** lesz a gateway és a backend közötti MQTT-üzenetekben, mert ezen a szakaszon a sávszélesség nem szűkös, a Java/Spring Boot és a C++ gateway is jól kezeli, és a hibakeresés során olvasható marad. A `.proto` sémát is definiáljuk, mert később bináris optimalizáláshoz és a nanopb-s eszközoldalhoz szükséges. A SenML-t az egységek és a mérési rekordok interoperábilis leírására használjuk.

11.1.4 Kidolgozott példa

Legyen a TerraSense mérés: `schema_version=1`, `node_id="n001"`, `seq=1`, `ts=1735689600000` (UTC, epoch ms), és egy hőmérséklet-mérés `22,5 °C`. Ugyanaz az üzenet JSON-ban, olvashatóság kedvéért tagolva:

```
{
  "schema_version": 1,
  "node_id": "n001",
  "seq": 1,
  "ts": 1735689600000,
  "measurements": {
    "temperature": {"value": 22.5, "unit": "Cel", "valid": true}
  }
}
```

A script a fenti objektumot `separators=(",", ":")` beállítással tömöríti, így a JSON kódolás hossza 136 bájt. A CBOR ugyanezt 101 bájtba kódolja, a protobuf pedig 33 bájtba. A CBOR bájtok teljes hex dumpja (a sorok csak a nyomtatás miatt tördeltek):

```
a56e736368656d615f76657273696f6e01676e6f64655f6964646e3030316373
6571016274731b000001941f297c006c6d6561737572656d656e7473a16b7465
6d7065726174757265a36576616c7565fa41b4000064756e69746343656c6576
616c6964f5
```

A protobuf kanonikus sémája a következő. A mezőszámok a fejezet teljes hosszában rögzítettek: a későbbi verziók csak új mezőszámot adhatnak hozzá, a meglévőket nem számozzák át.

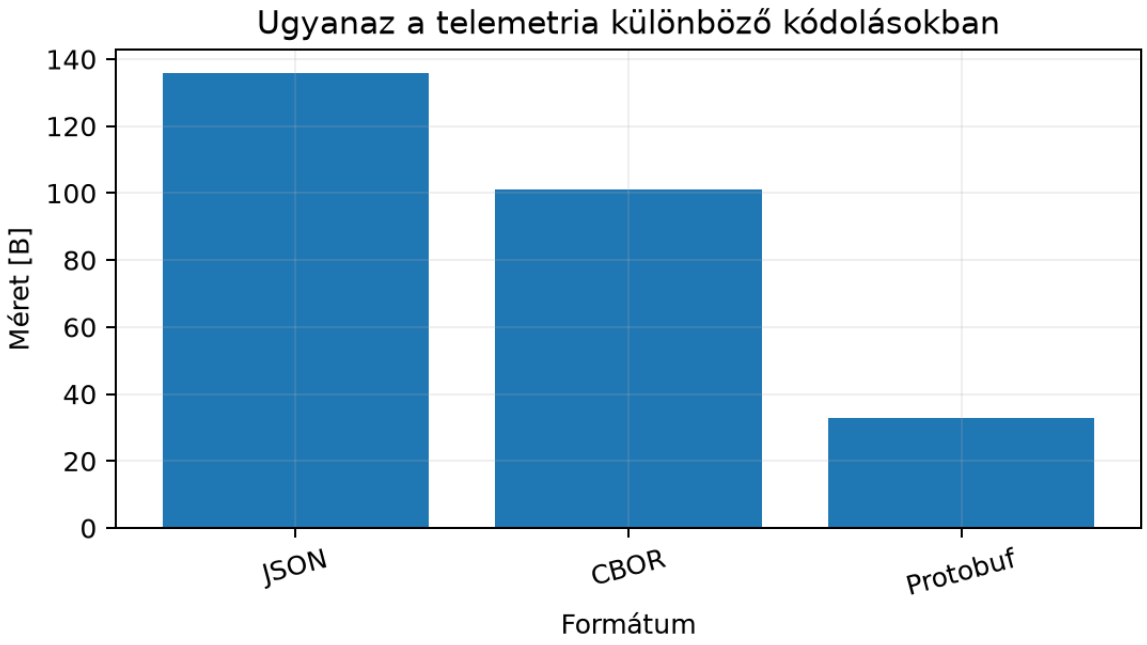
```
syntax = "proto3";
package terrasense;

message Telemetry {
  uint32 schema_version = 1;
  string node_id = 2;
  uint64 seq = 3;
  int64 ts = 4;
  Measurements measurements = 5;
}

message Measurements {
  Measurement temperature = 1;
  Measurement humidity = 2;
  Measurement soil_moisture = 3;
}

message Measurement {
  float value = 1;
  string unit = 2;
  bool valid = 3;
  string reason = 4;
}
```

A példaüzenet protobuf kódolásában a `schema_version` 2 bájt, a `node_id` 6 bájt (tag + length + adat), a `seq` 2 bájt, a `ts` 7 bájt, a beágyazott `measurements` pedig 16 bájt, összesen 33 bájt. A v1 séma öt felső szintű mezőt definiál; a későbbi verziók új mezőszámot toldanak hozzá, a meglévőket nem számozzák át.



11.1. ábra: Ugyanaz a telemetria különböző kódolásokban - A labor telemetry_codec.py által kódolt, schema_version=1 mezőt és egy hőmérséklet-mérést tartalmazó TerraSense példaüzenet mérete; a Python json.dumps, valamint a saját CBOR- és Protobuf-kódoló kimenete.

A méretkülönbség jól mutatja, hogy a bináris formátumok ott előnyösek, ahol a bájtok száma energia- vagy költségkorlátot jelent.

11.1.5 Labor: formátumok összemérése

Munkakönyvtár:

```
mkdir -p ~/iot-labor/i032 && cd ~/iot-labor/i032
```

Hozd létre a `telemetry_codec.py` fájlt az alábbi tartalommal. A script ugyanazt a mérést kódolja JSON-ban, CBOR-ban és protobufban, és kiírja a méreteket.

```
#!/usr/bin/env python3
import json
import struct

MAX_UINT64 = (1 << 64) - 1


def require_uint(value, maximum=MAX_UINT64):
    if not isinstance(value, int) or isinstance(value, bool):
        raise TypeError(f"egész érték kell: {value!r}")
    if not 0 <= value <= maximum:
        raise ValueError(f"tartományon kívüli egész: {value}")


def cbor_head(major, length):
    if not 0 <= length <= MAX_UINT64:
        raise ValueError(f"CBOR hossz túl nagy: {length}")
    if length < 24:
```

```
        return bytes([(major << 5) | length])
    if length <= 0xFF:
        return bytes([(major << 5) | 24]) + length.to_bytes(1, "big")
    if length <= 0xFFFF:
        return bytes([(major << 5) | 25]) + length.to_bytes(2, "big")
    if length <= 0xFFFFFFFF:
        return bytes([(major << 5) | 26]) + length.to_bytes(4, "big")
    return bytes([(major << 5) | 27]) + length.to_bytes(8, "big")


def cbor_encode(value):
    if isinstance(value, bool):
        return b"\xf5" if value else b"\xf4"
    if value is None:
        return b"\xf6"
    if isinstance(value, int):
        if value < 0:
            raise ValueError("negatív egész nem kódolható uintként")
        return cbor_head(0, value)
    if isinstance(value, float):
        return b"\xfa" + struct.pack(">f", value)
    if isinstance(value, str):
        data = value.encode("utf-8")
        return cbor_head(3, len(data)) + data
    if isinstance(value, dict):
        out = bytearray(cbor_head(5, len(value)))
        for key, item in value.items():
            if not isinstance(key, str):
                raise TypeError("a CBOR map kulcsa csak string lehet")
            out.extend(cbor_encode(key))
            out.extend(cbor_encode(item))
        return bytes(out)
    if isinstance(value, (list, tuple)):
        out = bytearray(cbor_head(4, len(value)))
        for item in value:
            out.extend(cbor_encode(item))
        return bytes(out)
    raise TypeError(f"nem támogatott CBOR típus: {type(value).__name__}")


class CborReader:
    def __init__(self, data):
        self.data = data
        self.pos = 0

    def take(self, count):
        if self.pos + count > len(self.data):
            raise ValueError("csonka CBOR")
        chunk = self.data[self.pos:self.pos + count]
        self.pos += count
        return chunk

    def decode(self):
        first = self.take(1)[0]
        major = first >> 5
        info = first & 0x1F
        if major == 7:
            if info == 20:
                return False
            if info == 21:
                return True
            if info == 22:
                return None
```

```
            if info == 26:
                return struct.unpack(">f", self.take(4))[0]
            raise ValueError(f"nem támogatott egyszerű érték: {info}")
        if info < 24:
            arg = info
        elif info == 24:
            arg = int.from_bytes(self.take(1), "big")
        elif info == 25:
            arg = int.from_bytes(self.take(2), "big")
        elif info == 26:
            arg = int.from_bytes(self.take(4), "big")
        elif info == 27:
            arg = int.from_bytes(self.take(8), "big")
        else:
            raise ValueError(f"fenntartott CBOR additional info: {info}")
        if major == 0:
            return arg
        if major == 3:
            return self.take(arg).decode("utf-8")
        if major == 4:
            return [self.decode() for _ in range(arg)]
        if major == 5:
            result = {}
            for _ in range(arg):
                key = self.decode()
                if not isinstance(key, str):
                    raise TypeError("a CBOR map kulcsa csak string lehet")
                if key in result:
                    raise ValueError(f"duplikált CBOR kulcs: {key}")
                result[key] = self.decode()
            return result
        raise ValueError(f"nem támogatott CBOR major type: {major}")


def cbor_decode(data):
    reader = CborReader(data)
    value = reader.decode()
    if reader.pos != len(data):
        raise ValueError("a CBOR után feldolgozatlan bájt maradt")
    return value


def encode_varint(value):
    require_uint(value)
    out = bytearray()
    while value > 0x7F:
        out.append((value & 0x7F) | 0x80)
        value >>= 7
    out.append(value)
    return bytes(out)


def pb_key(field_number, wire_type):
    require_uint(field_number, 0x1FFFFFFF)
    if field_number < 1 or 19000 <= field_number <= 19999:
        raise ValueError(f"fenntartott protobuf mezőszám: {field_number}")
    return encode_varint((field_number << 3) | wire_type)


def pb_varint(field_number, value):
    return pb_key(field_number, 0) + encode_varint(value)
```

```
def pb_bytes(field_number, data):
    return pb_key(field_number, 2) + encode_varint(len(data)) + data


def pb_text(field_number, text):
    return pb_bytes(field_number, text.encode("utf-8"))


def pb_float(field_number, value):
    return pb_key(field_number, 5) + struct.pack("<f", value)


def proto_encode_measurement(measurement):
    out = bytearray()
    out.extend(pb_float(1, measurement["value"]))
    out.extend(pb_text(2, measurement["unit"]))
    out.extend(pb_varint(3, 1 if measurement["valid"] else 0))
    reason = measurement.get("reason")
    if reason is not None:
        out.extend(pb_text(4, reason))
    return bytes(out)


def proto_encode_telemetry(message):
    measurements = bytearray()
    names = ("temperature", "humidity", "soil_moisture")
    for index, name in enumerate(names, 1):
        if name in message["measurements"]:
            encoded = proto_encode_measurement(message["measurements"][name])
            measurements.extend(pb_bytes(index, encoded))
    out = bytearray()
    out.extend(pb_varint(1, message["schema_version"]))
    out.extend(pb_text(2, message["node_id"]))
    out.extend(pb_varint(3, message["seq"]))
    out.extend(pb_varint(4, message["ts"]))
    out.extend(pb_bytes(5, bytes(measurements)))
    return bytes(out)


def hex_lines(data, width=64):
    digits = data.hex()
    return [digits[i:i + width] for i in range(0, len(digits), width)]


def check_cbor_roundtrip(message, label):
    encoded = cbor_encode(message)
    decoded = cbor_decode(encoded)
    if decoded != message:
        raise AssertionError(f"CBOR round-trip eltérés: {label}")
    print(f"CBOR round-trip OK: {label} ({len(encoded)} B)")


def main():
    msg = {
        "schema_version": 1,
        "node_id": "n001",
        "seq": 1,
        "ts": 1735689600000,
        "measurements": {
            "temperature": {"value": 22.5, "unit": "Cel", "valid": True}
        },
    }
    json_bytes = json.dumps(msg, separators=(",", ":")).encode("utf-8")
```

```
    cbor_bytes = cbor_encode(msg)
    proto_bytes = proto_encode_telemetry(msg)
    print(f"JSON      {len(json_bytes):3d} B")
    print(f"CBOR      {len(cbor_bytes):3d} B")
    print(f"Protobuf  {len(proto_bytes):3d} B")
    print("CBOR hex:")
    for line in hex_lines(cbor_bytes):
        print(f"  {line}")
    print("Protobuf hex:")
    for line in hex_lines(proto_bytes):
        print(f"  {line}")

    for seq in (0, 23, 24, 255, 256, 65535, 65536):
        check_cbor_roundtrip(dict(msg, seq=seq), f"seq={seq}")
    long_id = "n001-eszak-uveghaz-01-hosszu-azonosito"
    check_cbor_roundtrip(dict(msg, node_id=long_id), "hosszú node_id")
    for bad in (-1, 1 << 64):
        try:
            cbor_encode(dict(msg, seq=bad))
        except (ValueError, TypeError) as exc:
            print(f"CBOR elutasítva: seq={bad} ({exc})")
        else:
            raise AssertionError(f"a hibás seq átment: {bad}")


if __name__ == "__main__":
    main()
```

Futtasd a scriptet:

```
python3 telemetry_codec.py
```

Várt kimenet:

```
JSON      136 B
CBOR      101 B
Protobuf   33 B
CBOR hex:
  a56e736368656d615f76657273696f6e01676e6f64655f6964646e3030316373
  6571016274731b000001941f297c006c6d6561737572656d656e7473a16b7465
  6d7065726174757265a36576616c7565fa41b4000064756e69746343656c6576
  616c6964f5
Protobuf hex:
  080112046e30303118012080f8a5f9c1322a0e0a0c0d0000b441120343656c18
  01
CBOR round-trip OK: seq=0 (101 B)
CBOR round-trip OK: seq=23 (101 B)
CBOR round-trip OK: seq=24 (102 B)
CBOR round-trip OK: seq=255 (102 B)
CBOR round-trip OK: seq=256 (103 B)
CBOR round-trip OK: seq=65535 (103 B)
CBOR round-trip OK: seq=65536 (105 B)
CBOR round-trip OK: hosszú node_id (136 B)
CBOR elutasítva: seq=-1 (negatív egész nem kódolható uintként)
CBOR elutasítva: seq=18446744073709551616 (CBOR hossz túl nagy: 18446744073709551616)
```

A program a mintát és a határeseteket is ellenőrzi: a `seq` 0, 23, 24, 255, 256, 65 535 és 65 536 értékére, valamint egy 23 bájtnál hosszabb `node_id`-ra a CBOR round-tripnek egyeznie kell. A negatív és a 64 bites tartományon kívüli egész kódolása kivételt ad, nem csendben csonkol.

Hozd létre a végleges telemetria-sémát is. A `telemetry-schema.json` egy JSON Schema (2020-12), amely a kanonikus v1 üzenet kötelező mezőit és a mérési objektum szerkezetét rögzíti:

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$id": "https://terrasense.example/schemas/telemetry-v1.json",
  "title": "TerraSense telemetry v1",
  "type": "object",
  "required": ["schema_version", "node_id", "seq", "ts", "measurements"],
  "properties": {
    "schema_version": {"const": 1},
    "node_id": {"type": "string", "minLength": 1, "maxLength": 64},
    "seq": {"type": "integer", "minimum": 0},
    "ts": {"type": "integer", "minimum": 0},
    "boot_id": {"type": "string", "minLength": 1},
    "measurements": {
      "type": "object",
      "minProperties": 1,
      "additionalProperties": {"$ref": "#/$defs/measurement"}
    }
  },
  "additionalProperties": true,
  "$defs": {
    "measurement": {
      "type": "object",
      "required": ["value", "unit", "valid"],
      "properties": {
        "value": {"type": "number"},
        "unit": {"type": "string", "minLength": 1},
        "valid": {"type": "boolean"},
        "reason": {"type": ["string", "null"]}
      },
      "additionalProperties": true
    }
  }
}
```

A `telemetry.proto` fájl tartalma a fenti kanonikus protobuf definíció, változtatás nélkül. A JSON Schema és a protobuf ugyanazokat a mezőneveket használja; a protobufban a mezőszámok rögzítik az azonosítást. A JSON Schema `additionalProperties: true` beállítása szándékos: a v1 feldolgozó a jövőbeli verziók ismeretlen mezőit figyelmen kívül hagyja, és naplózza őket.

A korábbi, lapos JSON-formátum (`temperature_c`, `humidity_rh`, `soil_moisture_pct`) nem része a kanonikus v1 sémának. Az ilyen üzeneteket a backend a 0-es verziójú, dokumentált adapterrel fogadja el, és a lapos mezőket `measurements` objektummá alakítja; erről a Sémák és verziózás: kompatibilis változtatások és az Ingest: MQTT-ből a backendbe, idempotensen alfejezet részletesen szól.

Továbbvitt eredmény. A `~/iot-labor/i032/telemetry-schema.json` fájlban rögzített JSON Schema a TerraSense hivatalos telemetria-formátumának v1 definíciója: `schema_version`, `node_id`, `seq`, `ts` és `measurements` mezőkkel, SenML-egységrövidítésekkel (`Cel`, `%RH`, `%`) és `valid`/`reason` érvényességi jelöléssel. A `~/iot-labor/i032/telemetry.proto` ugyanennek a bináris változata. Ezt a sémát használja a Sémák és verziózás: kompatibilis változtatások, a A ts-gw szolgáltatás C++-ban: protokollfordítás MQTT-re, az Ingest: MQTT-ből a backendbe, idempotensen és az Idősoros adatbázisok és adatmodellezés alfejezet.

11.1.6 Ellenőrzés és hibakeresés

A labor kimenetét a hex dump alapján ellenőrizhetjük. A CBOR első bájtja `a5`, ami egy 5 elemből álló map-et jelent; a protobuf első bájtja `08`, ami az 1-es mező (`schema_version`) varint típusú tagje. A protobuf wire-formátum nem garantálja a mezők sorrendjét, ezért a feldolgozó a

tagből olvassa ki a mezőszámot, nem a bájtpozícióból. A méreteket a `len()` függvény pontosan adja meg.

Tünet	Ok	Javítás
A JSON-ben a `22.5` után `22.499999` jelenik meg	A fogyasztó `double` ábrázolása	Kerekíts a megjelenítésnél, vagy küldj fixpontos egész értéket
A protobuf kódolt bájt hossza nagyobb a vártnál	A mezőszám nagyobb, mint 15, vagy a string hossza nagyobb, mint 127	Használj kis mezőszámokat a gyakori mezőkhöz, és ellenőrizd a varint hosszakat
A CBOR dekódoló `major type 7` hibát ad	A `ts` uint64 kódolásnál rossz a kiegészítő bájt	Ellenőrizd a `0x1B` utáni 8 bájt big-endian sorrendjét
A SenML-ben a `v` mező ütközik a verzióval	A SenML a `v`-t a mért értéknek tartja fenn	A TerraSense JSON-ban `schema_version` mezőt használj, ne `v`-t

11.1.7 Tipikus hibák és félreértések

1. **Lebegőpontos mérés JSON-ban.** A JSON számok IEEE 754 double-ként viselkednek, ezért a `0,1` nem pontos. A fizikai méréseket érdemes fixpontos egészre skálázva küldeni, például a hőmérsékletet `0,01 °C` egységben.
2. **Protobuf mezőszám és mezőnév összekeverése.** A mezőszám az üzenet azonosítója a wire-formátumban; a mezőnév megváltoztatása biztonságos, de a mezőszám megváltoztatása sématörést okoz.
3. **SenML `v` mező félreértése.** A SenML-ben a `v` az érték, nem a verzió; a saját sémánkban a verziót `schema_version`-nek nevezzük.
4. **CBOR mint „tömörített JSON”.** A CBOR sokkal több, mint tömörített JSON: bájtstringet, nem szöveges map-kulcsot és címkéket is támogat, ezért a JSON-modelltől eltérő adatokat is tud kódolni.
5. **Beégetett hosszak és egybájtos length mezők.** Egy kézi kódoló könnyen feltételezi, hogy a `seq` 0 és 23 közé esik, a szöveg hossza egy bájtba fér, vagy a protobuf length mezője mindig egy bájt. Ez csak a példaüzenetre igaz. A fenti kód ezért a CBOR fejléceket a `cbor_head`, a protobuf length mezőket a `pb_bytes` és az `encode_varint` segítségével készíti, és a `seq` 24, 255, 256, valamint a hosszabb `node_id` esetét round-trip teszttel ellenőrzi.

11.1.8 Feladatok

1. **Fogalmi/döntési kérdés.** Egy LoRaWAN-os TerraSense csomópont napi 100 üzenetet küld, és a havi adatkeret 1 MiB. Miért nem a JSON-t választanád erre a rádióra? Melyik formátumot és miért?
2. **Számolási feladat.** A protobuf sémában a `seq` értéke 300, a `ts` pedig 1735689600000. Hány bájtot foglal a két mező varint kódolása külön-külön?
3. **Implementációs feladat.** Bővítsd a `telemetry_codec.py` mintáját páratartalom-méréssel (`humidity`, `61.2`, `%RH`, `valid=true`). A kanonikus protobuf séma már tartalmazza a `humidity` mezőt, ezért csak a minta `measurements` objektumát kell kiegészíteni. Ez a módosítás előkészíti a Sémák és verziózás: kompatibilis változtatások alfejezetet.

4. **Hibakeresési feladat.** A backend régi eszközöktől lapos JSON-t kap (`temperature_c`, `humidity_rh`, `soil_moisture_pct`), új eszközöktől pedig `measurements` objektumot. Tervezz olyan adaptert és verziószabályt, amely a két formátumot egyetlen kanonikus feldolgozási útra tereli, és nem veszíti el a `valid`/`reason` információt.

11.1.9 Megoldások

1. A LoRaWAN-os csomópontnak minden bájt számít, mert a rádióidő és az adatforgalom energiát és pénzt fogyaszt. A JSON önleíró, de terjengős, ezért inkább CBOR-t vagy protobufot érdemes használni. A SenML CBOR jó interoperabilitást ad, a protobuf pedig a legkisebb méretet. A TerraSense-nél a LoRaWAN-os eszközre a CBOR + SenML egységmodellt választanánk.
2. A `seq=300` varint kódolása 2 bájt, mert $300 > 127$, de $300 < 16384$. A `ts=1735689600000` varint kódolása 6 bájt, mert 2^{2}-nél kisebb, de 2^{3}-nél nagyobb.
3. A mintában a `measurements` objektum bővül; a generikus CBOR-kódoló a map méretét és a kulcsok hosszát automatikusan számolja:

```
"measurements": {
    "temperature": {"value": 22.5, "unit": "Cel", "valid": True},
    "humidity": {"value": 61.2, "unit": "%RH", "valid": True},
}
```

A protobuf kódoló a `Measurements` üzenet mezőit 1-től 3-ig sorrendben dolgozza fel, ezért a `humidity` a 2-es mezőre kerül (`0x12` tag). A `reason` hiányzik, ezért a 4-es mező nem képződik le. A 61,2 float32-ként nem ábrázolható pontosan, ezért a CBOR round-trip ellenőrzésnél a mérési értéket tűréssel kell összehasonlítani, vagy pontosan ábrázolható értéket (például 61,25) kell használni.

4. A megoldás a 0-es verziójú adapter: ha a `schema_version` hiányzik vagy 0, és a payload lapos mérési mezőket tartalmaz, a backend a `temperature_c`, `humidity_rh`, `soil_moisture_pct` mezőkből `temperature`, `humidity`, `soil_moisture` mérési objektumokat készít. A mértékegység a mezőnevből rögzített (`Cel`, `%RH`, `%`), a `valid` értéke `true`, a `reason` értéke `null`. A tárolt kanonikus payload `schema_version` értéke 0 marad, így később is azonosítható, hogy adapterrel készült. Az 1-es verziójú üzenetek változatlanul, közvetlenül a `measurements` objektumot használják. Ez a stratégia alapozza meg a Sémák és verziózás: kompatibilis változtatások alfejezetet.

11.1.10 Rövid összefoglaló

A JSON olvasható, de nagy; a CBOR ugyanazt a szemantikát kompakt bájtokban tárolja; a protobuf sémaalapú, kódgenerálást és minimális wire-méretet ad; a SenML pedig szabványos mérési rekord és egységmodell. A TerraSense telemetria elsődlegesen JSON, mert a gateway–backend szakaszon a sávszélesség nem korlát, és a hibakeresés fontos. A végleges séma a `~/iot-labor/i032/telemetry-schema.json` fájlban található: `schema_version`, `node_id`, `seq`, `ts` és `measurements` mezőkkel, SenML-egységekkel, `valid` jelzővel. A `~/iot-labor/i032/telemetry.proto` a bináris alternatíva. A `telemetry_codec.py` labor megmutatja, hogy a példaüzenet JSON-ben 136 B, CBOR-ban 101 B, protobufban 33 B; a korábbi, lapos JSON-formátumot a backend 0-es verziójú adapterrel fogadja el. **Továbbvitt eredmény:** ezt a sémát használja az Sémák és verziózás: kompatibilis változtatások, a A ts-gw szolgáltatás C++-ban: protokollfordítás MQTT-re, a Ingest: MQTT-ből a backendbe, idempotensen és az Idősoros adatbázisok és adatmodellezés alfejezet.

11.1.11 Források és ellenőrzési hiányok

- RFC 8949 (CBOR), RFC 8428 (SenML), RFC 8259 (JSON) — a protokollok és a formátumok alapfogalmai stabilak.
- Protocol Buffers dokumentáció: `https://protobuf.dev/` — a `protoc` pontos parancsa és a nanopb verziófüggő API-ja ellenőrizendő.
- A Zephyr `zcbor` és `nanopb` moduljainak Kconfig-neveit és a telepített verziókat a felhasználónak a saját Zephyr-telepítésében kell ellenőriznie.
- A labor méret- és hexadatai a `telemetry_codec.py` lefuttatásából származnak a könyvben szereplő példaüzenetre; más mezőnevek, mezősorrend vagy kódolóbeállítások eltérő bájtokat adnak. A CBOR- és protobuf-kódoló oktatási célú, nem termelési könyvtár; éles rendszerben `zcbor`, illetve `nanopb` vagy `protoc` használatos.

11.2 Sémák és verziózás: kompatibilis változtatások

11.2.1 Motiváció

A TerraSense rendszer első változata már működik: a `ts-node` eszközök MQTT-n keresztül küldik a telemetria-adatokat, a `ts-backend` pedig feldolgozza és eltárolja azokat. A fejlesztés során azonban új igény merül fel: a csomópontok egy részén napelemes táplálást vezetünk be, ezért az akkumulátorfeszültséget is mérni és továbbítani kell. A backendet frissítjük, hogy kezelje az új mezőt, de a flottában több száz eszköz van, amelyeket nem lehet egyetlen pillanat alatt OTA-frissítéssel ellátni. Ráadásul néhány régi eszköz még hosszú hónapokig a régi firmware-rel fut, mert a frissítési ablakuk csak később érkezik el.

Ez a mindennapos helyzet rávilágít a sémaverziózás központi problémájára: hogyan változtathatjuk meg az üzenetformátumot úgy, hogy a régi és az új komponensek (eszköz, gateway, backend) együtt tudjanak működni? Ebben az alfejezetben megtanuljuk az előre és visszafelé kompatibilitás fogalmát, a protobuf mezőszám-szabályait, valamint a verzió elhelyezésének lehetőségeit.

11.2.2 Fogalmak és működés

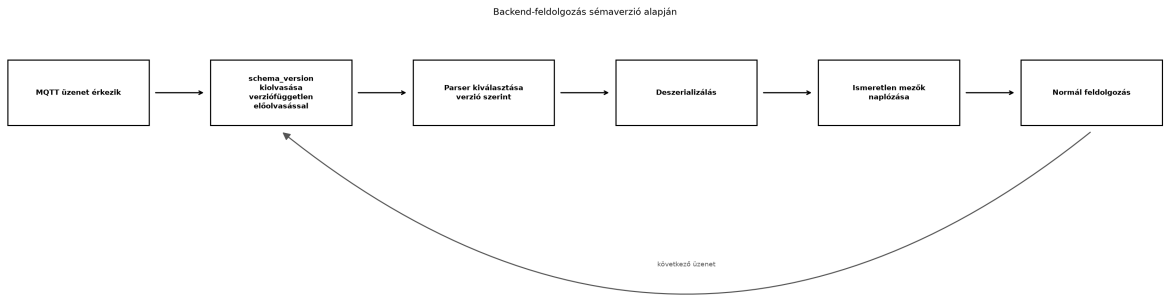
Definíció. Egy séma (schema) az üzenetek szerkezetének formális leírása: milyen mezők léteznek, milyen típusúak, melyek kötelezőek vagy opcionálisak, és milyen azonosítókkal (pl. JSON-kulcs, protobuf mezőszám) hivatkozunk rájuk.

Definíció. Egy sémamódosítás **visszafelé kompatibilis**, ha az új séma szerint előállított üzenetet a régi séma szerint működő fogyasztó is képes értelmezni. Például ha hozzáadunk egy opcionális mezőt, a régi kód figyelmen kívül hagyja az ismeretlen mezőt, és a régi mezőket továbbra is megtalálja.

Definíció. Egy sémamódosítás **előre kompatibilis**, ha a régi séma szerint előállított üzenetet az új séma szerint működő fogyasztó is képes értelmezni. Ez általában azt jelenti, hogy az új séma nem követel meg olyan mezőt, amely a régi üzenetben nem szerepel, vagy az új mező opcionális és alapértelmezett értéke van.

Definíció. Ismeretlen mező kezelése alatt azt értjük, hogy a deszerializáló hogyan bánik azokkal a mezőkkel, amelyeket a saját sémája nem ismer. A protobuf megőrzi ezeket a mezőket (uninterpreted data), hogy egy proxy vagy továbbító komponens ne veszítse el őket. A JSON- és CBOR-alapú feldolgozók gyakran egyszerűen eldobják az ismeretlen kulcsokat, ami adatvesztéshez vezethet, ha az üzenet több komponensen halad át.

A következő folyamatábra azt mutatja, hogy a `ts-backend` hogyan dolgozza fel a bejövő MQTT-üzeneteket a sémaverzió alapján:



11.2. ábra: Backend-feldolgozás sémaverzió alapján - A ts-backend minden támogatott sémaverzióhoz külön deszerializálót tart, az ismeretlen mezőket nem dobja el, hanem naplózza. A verziót név vagy mezőszám alapján, nem bájtpozíció alapján olvassa ki.

A folyamat lényege: az üzenetben lévő `schema_version` mező alapján a backend kiválasztja a megfelelő deszerializálót. A verziót nem a payload első bájtjából vagy első mezőjéből olvassa ki: JSON-ban és CBOR-ban név szerint keresi az objektumban, protobufban a tag alapján olvassa be az 1-es mezőszámot, a többi mező sorrendjétől függetlenül. Így egy átrendezett JSON vagy eltérő mezősorrendű protobuf üzenet ugyanahhoz a parserhez jut. Az ismeretlen mezőket (például egy jövőbeli verzió új mezője) nem dobja el véglegesen, hanem naplózza, így a fejlesztők értesülnek arról, hogy a flottában már feltűnt egy újabb verzió.

11.2.3 Döntési szempontok

Az első fontos döntés: hová kerüljön a sémaverzió? Két elterjedt lehetőség van: a topicba vagy az üzenetbe. A TerraSense esetében a topic-konvenció már rögzített, és a gateway-nek nem szabad a topicot a sémaverzió alapján átírnia. Ezért az üzenetben hordozott verziót választjuk.

Szempont	Verzió a topicban	Verzió az üzenetben
Előny	A fogyasztó már a topic alapján eldöntheti, hogyan értelmezze az üzenetet; nincs szükség a payload előzetes vizsgálatára.	A séma és az adat együtt utazik, nincs külön topic-hierarchia; a gateway nem módosítja a topicot.
Hátrány	A topic-struktúra bonyolultabbá válik; a feliratkozóknak több topicot kell figyelniük; a gateway-nek ismernie kell a verziót.	A fogyasztónak először ki kell csomagolnia a payloadot, hogy megtudja a verziót; a séma nélkül nem tudja deszerializálni.
TerraSense döntés	–	`schema_version` mező az üzenetben; a feldolgozó név vagy mezőszám alapján keresi, nem a payload pozíciója alapján.

A JSON objektumok kulcssorrendje szemantikailag jelentéktelen, a protobuf wire-formátuma pedig a mezőket taggel azonosítja, és a mezők sorrendjét nem garantálja. Ezért a `schema_version` „payload elejére” helyezése nem működési elv, csak szerkesztési konvenció. A feldolgozó vagy egy verziófüggetlen borítékot (például egy külön fejlécet) olvas, vagy a teljes üzenetet beolvassa egy minimális, verziófüggetlen parserrel, és abból emeli ki a `schema_version` mezőt. A TerraSense JSON- és CBOR-üzeneteiben a mezőt név szerint, protobufban az 1-es mezőszámmal kell megkeresni; a bájtpozícióra építő parser hibás.

Továbbvitt eredmény. A TerraSense telemetria-sémájának verzióját az üzenetben hordozzuk, a topic nem tartalmaz verziót. A backend minden támogatott verziót fogadja, az ismeretlen mezőket eldobja (JSON/CBOR) vagy megőrzi (protobuf), de mindenképpen naplózza. Ezt a szabályt használja az Ingest: MQTT-ből a backendbe, idempotensen, az OTA-frissítés: A/B képek, rollback, delta, fokozatos kiadás és a Flottakezelés: nyilvántartás, konfiguráció, állapot alfejezet.

A séma-nyilvántartás (schema registry) egy központi szolgáltatás, amely nyilvántartja a sémaverziókat, és a fogyasztók innen töltik be a deszerializáláshoz szükséges sémát. Előnye, hogy egyetlen helyen lehet nyomon követni a változásokat és a kompatibilitási szabályokat; hátránya, hogy újabb komponenst kell üzemeltetni, és a fogyasztó függ a registry elérhetőségétől. A TerraSense-ben a sémaverziók száma kicsi, ezért a backendbe fordításkor beágyazott parser-osztályokkal dolgozunk, és nem vezetünk be külön külső szolgáltatást. Ez a döntés a build vs. buy elvhez is illeszkedik.

A protobuf esetében a kompatibilitási szabályok a mezőszámokon alapulnak. A mezőszám 1 és 536 870 911 között lehet, de a 19 000–19 999 tartomány a protobuf implementációk számára van fenntartva. A legfontosabb szabályok:

- Új mező hozzáadásakor mindig új, korábban nem használt mezőszámot adjunk.
- A mezőszám soha nem hasznosítható újra, még akkor sem, ha a mező törlésre került.
- Opcionális mezők hozzáadása visszafelé és előre is kompatibilis, ha a fogyasztó kezeli a hiányzó mezőt (alapértelmezett értékkel vagy `optional` kulcsszóval).
- Kötelező mező hozzáadása nem visszafelé kompatibilis, mert a régi üzenetben nem szerepel, és az új fogyasztó elutasíthatja.

A flottában vegyes firmware-verziók jelenléte elkerülhetetlen. Az OTA-frissítések fokozatos kiadása, a kihelyezett eszközök offline időszakai és a gyártási sorozatok különböző kiinduló verziói miatt mindig több szoftververzió fut párhuzamosan. Ezért a backend nem támaszkodhat arra, hogy minden eszköz azonos sémaverziót küld; ellenkezőleg, a backendnek addig kell támogatnia egy sémaverziót, amíg a legrégebbi eszköz is frissítve nem lett. Ezt a támogatási időt a flottakezelési irányelvekben rögzíteni kell.

11.2.4 Kidolgozott példa

A kanonikus v1 séma a Adatformátumok: JSON, CBOR, Protocol Buffers, SenML alfejezetben rögzített `measurements` objektumot használja. A v1 JSON-üzenet:

```
{
  "schema_version": 1,
  "node_id": "n001",
  "seq": 123,
  "ts": 1710000000000,
  "measurements": {
    "temperature": {"value": 22.5, "unit": "Cel", "valid": true},
    "humidity": {"value": 60.0, "unit": "%RH", "valid": true},
    "soil_moisture": {"value": 35.0, "unit": "%", "valid": true}
```

```
  }
}
```

A második verzió (v2) hozzáadja az akkumulátorfeszültséget (`battery_mv`) opcionális, felső szintű mezőként, mert nem minden csomópont rendelkezik napelemes táplálással:

```
{
  "schema_version": 2,
  "node_id": "n001",
  "seq": 124,
  "ts": 1710000060000,
  "measurements": {
    "temperature": {"value": 22.8, "unit": "Cel", "valid": true},
    "humidity": {"value": 59.5, "unit": "%RH", "valid": true},
    "soil_moisture": {"value": 34.0, "unit": "%", "valid": true}
  },
  "battery_mv": 3650
}
```

A v1-es backend, amely csak az 1. sémaverziót ismeri, a v2-es üzenetet is sikeresen feldolgozza, ha a JSON-feldolgozó figyelmen kívül hagyja az ismeretlen `battery_mv` kulcsot. A `measurements` objektum és a benne lévő `valid`/`reason` mezők változatlanok, ezért nem vész el érvényességi információ. Ez **visszafelé kompatibilis** változás.

A v2-es backend a v1-es üzenetet is fogadja, mert a hiányzó `battery_mv` mezőt „nincs adat” állapotként kezeli. Ez **előre kompatibilis** változás. A két verzió között nincs mező-átnúmerozás és nincs mező-átnevezés; a v2 csak egy új, opcionális mezőszámot használ.

A régi, lapos JSON-formátum külön verzióhatárt kap. Az ilyen üzenetben nincs `measurements` objektum, hanem `temperature_c`, `humidity_rh` és `soil_moisture_pct` mezők vannak, és a `schema_version` hiányzik vagy 0:

```
{
  "node_id": "n001",
  "seq": 125,
  "ts": 1710000120000,
  "temperature_c": 23.0,
  "humidity_rh": 58.0,
  "soil_moisture_pct": 33.0
}
```

Ezt a formátumot a backend 0-es verziójú adaptere fogadja el: a lapos mezőkből `measurements` objektumot készít, a mértékegységet a mezőnévből rögzíti (`Cel`, `%RH`, `%`), a `valid` értékét `true`-ra, a `reason` értékét `null`-ra állítja, a tárolt payloadban pedig `schema_version=0`-t jelöl. Az 1-es és 2-es verzió kizárólag a `measurements` objektumot használja; az a payload, amelyben `schema_version=1` mellett lapos mérési mező szerepel, érvénytelen, és a dead-letter útvonalra kerül. Így egyetlen kanonikus feldolgozási út van, a variánsok pedig explicit verziót és adaptert kapnak.

Most nézzük ugyanezt protobuf sémával. A v1 protobuf definíció a későbbi mezőszámokat is rögzíti, hogy a bővítés ne számozza át a meglévő mezőket:

```
syntax = "proto3";

message Telemetry {
  uint32 schema_version = 1;
  string node_id = 2;
  uint64 seq = 3;
  int64 ts = 4;
  Measurements measurements = 5;
```

```
}

message Measurements {
  Measurement temperature = 1;
  Measurement humidity = 2;
  Measurement soil_moisture = 3;
}

message Measurement {
  float value = 1;
  string unit = 2;
  bool valid = 3;
  string reason = 4;
}
```

A v2 protobuf definíció ugyanezt használja, és hozzáadja az opcionális `battery_mv` mezőt a 6-os mezőszámmal:

```
syntax = "proto3";

message Telemetry {
  uint32 schema_version = 1;
  string node_id = 2;
  uint64 seq = 3;
  int64 ts = 4;
  Measurements measurements = 5;
  optional uint32 battery_mv = 6;
}
```

A `Measurements` és `Measurement` üzenet definíciója változatlan; a v2 példában csak a felső szintű `Telemetry` bővül. A v1-es deszerializáló a 6-os mezőszámot ismeretlenként kezeli, és a protobuf specifikáció szerint megőrzi azt az ismeretlen mezők puffereben. A v2-es deszerializáló a v1-es üzenetben egyszerűen nem találja a 6-os mezőt, és az `optional` kulcsszó miatt a `battery_mv` mező `has_battery_mv()` metódusa hamisat ad, így a fogyasztó felismeri a hiányzó adatot.

A `ts-backend` Java-oldali feldolgozása a következő elvet követi: a `schema_version` alapján választja ki a megfelelő deszerializálót, és az ismeretlen mezőket nem hagyja figyelmen kívül, hanem naplózza. A cél az, hogy az újabb eszközverziók által küldött ismeretlen mezők ne vesszenek el, és a fejlesztők időben értesüljenek a sémabővítésről.

11.2.5 Ellenőrzés és hibakeresés

A sémakompatibilitást legegyszerűbben úgy ellenőrizzük, hogy a régi és az új deszerializálót is lefuttatjuk a másik verzió által előállított mintára. A tesztelés során az alábbi lépéseket érdemes követni:

1. Készítsünk v1-es üzenetet, és deszerializáljuk v2-es parserrel. Ellenőrizzük, hogy a közös mezők helyesen értelmeződnek, és a hiányzó `battery_mv` ne okozzon hibát.
2. Készítsünk v2-es üzenetet, és deszerializáljuk v1-es parserrel. Ellenőrizzük, hogy a v1-es kód az ismeretlen mezőt figyelmen kívül hagyja (JSON/CBOR) vagy megőrzi (protobuf).
3. Protobuf esetén vizsgáljuk meg, hogy az ismeretlen mezők megőrzése után egy újbóli serializálás nem változtatja-e meg az eredeti mezőszámokat és értékeket.
4. Rendezzük át a JSON kulcsait, illetve adjuk meg a protobuf mezőket más sorrendben, és ellenőrizzük, hogy a `schema_version` kiolvasása és a deszerializálás eredménye ugyanaz. A parser nem támaszkodhat a `schema_version` pozíciójára.
5. A backendben ellenőrizzük a naplókat: az ismeretlen mezők megjelenjenek figyelmeztetésként.

Hibatünet	Lehetséges ok	Javítás
A backend elutasítja az üzenetet „unknown field” hibával.	A JSON-deszerializáló szigorú konfigurációja nem engedi az ismeretlen mezőket.	Kapcsoljuk be a lenient módot, vagy vezessünk be explicit mezőszűrést a séma alapján.
A v1-es eszköz üzenete a v2-es backendben hibát okoz, mert hiányzik a `battery_mv`.	Az új mezőt kötelezőként kezeltük, nem opcionálisként.	Tegyük opcionálissá a mezőt, vagy adjunk neki alapértelmezett értéket.
Protobufban az ismeretlen mezők elvesznek, pedig a gateway továbbítja őket.	A gateway újra-serializálja az üzenetet, de nem őrzi meg az ismeretlen mezőket.	Használjunk olyan protobuf API-t, amely megőrzi az ismeretlen mezőket (pl. Java-ban `UnknownFieldSet`).
A topic-verzió nem frissül, amikor az eszköz új sémára vált.	A gateway a bejövő UDP csomag alapján generálja a topicot, de nem olvassa a sémaverziót.	Az üzenetben hordozott verziót választottuk, ezért a topic nem változik; a gateway-nek nincs feladata.

11.2.6 Tipikus hibák és félreértések

1. **Mezőszám újrahasznosítása protobufban.** A fejlesztők néha törölnek egy mezőt, majd később ugyanazt a mezőszámot adják egy másik mezőhöz. Ez súlyos adatkárosodást okozhat, mert a régi üzenetekben az adott mezőszám más jelentéssel bír, és a feldolgozó rossz típussal értelmezi. A mezőszámot soha nem hasznosítjuk újra, még akkor sem, ha a mező törlésre került.
2. **Ismeretlen mezők eldobása proxy esetén.** Ha a rendszerben egy köztes komponens (például a ts-gw) újra-serializálja az üzenetet, és a JSON/CBOR feldolgozó eldobja az ismeretlen mezőket, az adatok elvesznek. A gateway ezért csak akkor alakíthatja át a payloadot, ha garantálni tudja a mezők megőrzését, vagy ha a backend számára felesleges mezőket szándékosan szűri.
3. **Kötelező mező hozzáadása visszafelé kompatibilisnek gondolása.** Sokan azt hiszik, hogy ha az új fogyasztó el tudja olvasni a régi üzenetet, akkor a változás automatikusan kompatibilis. Valójában a visszafelé kompatibilitás azt követeli meg, hogy a régi fogyasztó is értse az új üzenetet. Kötelező mező hozzáadása ezt megsérti, mert a régi üzenetben nincs jelen.
4. **Sémaverzió nélküli JSON-mezők.** Ha az üzenetben nincs explicit verziószám, a fogyasztó csak heurisztikával (pl. mező jelenléte) tudja eldönteni a verziót. Ez törékeny és hibákhoz vezet. A kanonikus üzenetekben mindig szerepeljen `schema_version`; a régi, lapos formátum csak a 0-es verziójú adapter bemenete lehet, és az adapter után a tárolt payload már explicit verziót hordoz.

11.2.7 Feladatok

1. **Fogalmi/döntési kérdés.** Döntse el az alábbi sémamódosításokról, hogy visszafelé, előre vagy egyik irányban sem kompatibilisek. Indokolja válaszát.
   - a) Hozzáadunk egy kötelező `device_type` mezőt a telemetria-üzenethez.
   - b) Töröljük a `measurements.humidity` bejegyzést, mert a szenzor már nem méri.

- c) Átnevezzük a `measurements.temperature` kulcsot `temp`-re.
- d) Hozzáadunk egy opcionális `battery_mv` mezőt a 6-os mezőszámmal protobufban.

2. **Számolási/tervezési feladat.** A TerraSense v1 telemetria-üzenet öt felső szintű mezőt tartalmaz. Becsülje meg, hogy az opcionális `battery_mv` mező hozzáadása mennyivel növeli a JSON- és a protobuf-üzenet méretét, ha az akkumulátorfeszültség tipikusan 3 600 mV. A JSON-ban a mezőnév `"battery_mv": 3650,` hossza hány bájt? A protobufban a mező fejléce (tag) és a varint érték hány bájt? (A protobuf uint32 mező tagje 1 bájt, ha a mezőszám kisebb 16-nál; a varint 3 650 értékhez 2 bájt szükséges.)

3. **Implementációs feladat.** Terjessze ki a TerraSense telemetria-sémáját egy új opcionális mezővel, `rssi_dbm` (sint32) a 7-es mezőszámmal protobufban. Írja meg a v3-as protobuf definíciót, és mutassa be, hogyan deszerializálja a v2-es backend ezt az üzenetet (a hiányzó `rssi_dbm` kezelése).

4. **Hibakeresési/kiterjesztési feladat.** A `ts-backend` naplójában a következő hiba jelenik meg: `Unrecognized field "battery_mv" (class TelemetryV1), not marked as ignorable`. Mi a hiba oka, és hogyan javítható anélkül, hogy a v1-es parser sémáját módosítanánk?

11.2.8 Megoldások

1. **Kompatibilitási döntések.**

   - a) **Nem visszafelé kompatibilis**, mert a régi üzenetből hiányzik a `device_type`, és a régi fogyasztó nem tudja értelmezni. Előre lehet kompatibilis, ha az új fogyasztó a hiányzó mezőt alapértelmezett értékkel kezeli, de a kötelezőség ezt megnehezíti.
   - b) **Nem előre kompatibilis**, mert az új fogyasztó hiába keresi a `measurements.humidity` bejegyzést, az a régi üzenetben szerepel, de az új fogyasztó már nem ismeri. Visszafelé lehet kompatibilis, ha a régi fogyasztó az új üzenetben egyszerűen nem találja a bejegyzést, és ez nem okoz hibát. Azonban a törlés általában adatvesztéssel jár.
   - c) **Egyik irányban sem kompatibilis**, mert a mérési kulcs megváltoztatása azt jelenti, hogy a régi és az új fogyasztó különböző kulcsot keres, így az adat elveszik.
   - d) **Visszafelé és előre is kompatibilis**, mert az opcionális mezőt a régi fogyasztó figyelmen kívül hagyja, az új fogyasztó pedig a hiányzó mezőt kezelni tudja.

2. **Méretnövekedés.**

   - JSON: a mezőnév `"battery_mv":` 14 bájt, az érték `3650` 4 bájt, a vessző 1 bájt, összesen 19 bájt. A teljes JSON növekedése tehát 19 bájt (a szám pontos hossza a formázástól függ).
   - Protobuf: a mező tagje 1 bájt (mezőszám 6, wire type 0 -> `0x30`), a varint 3 650 értéke 2 bájt, összesen 3 bájt. A protobuf tehát jelentősen kisebb növekedést okoz.

3. **Protobuf v3 séma.**

```
syntax = "proto3";

message Telemetry {
  uint32 schema_version = 1;
  string node_id = 2;
  uint64 seq = 3;
  int64 ts = 4;
  Measurements measurements = 5;
  optional uint32 battery_mv = 6;
  optional sint32 rssi_dbm = 7;
}
```

A `Measurements` és `Measurement` üzenet definíciója változatlan. A v2-es backend deszerializálja a v3-as üzenetet: a 7-es mezőszámot ismeretlenként kezeli, és megőrzi az ismeretlen mezők pufferében. A v2-es sémában nincs `rssi_dbm`, így a fogyasztó nem fér hozzá, de az üzenet többi része hibátlanul feldolgozható.

4. **Hiba oka és javítása.** A hiba azt jelzi, hogy a v1-es Java deszerializáló (Jackson) szigorú módban fut, és az ismeretlen `battery_mv` kulcsot nem engedi figyelmen kívül hagyni. A javítás: állítsuk be a Jackson `ObjectMapper`-en a `FAIL_ON_UNKNOWN_PROPERTIES` értéket `false`-ra, vagy a `@JsonIgnoreProperties(ignoreUnknown = true)` annotációt használjuk a v1-es DTO osztályon. Ezzel a v1-es parser el tudja fogadni a v2-es üzenetet, és az ismeretlen mezőt egyszerűen eldobja (vagy naplózza, ha konfiguráljuk).

11.2.9 Rövid összefoglaló

A sémaváltozások kompatibilitását két irányban vizsgáljuk: visszafelé (az új üzenetet a régi fogyasztó is érti) és előre (a régi üzenetet az új fogyasztó is érti). Az ismeretlen mezők kezelése kulcsfontosságú: a protobuf megőrzi, a JSON/CBOR gyakran eldobja azokat, ezért proxy esetén figyelni kell. A TerraSense-ben a sémaverziót az üzenetben hordozzuk (`schema_version` mező), a topic nem tartalmaz verziót. A kanonikus v1 üzenet `measurements` objektumot használ; a korábbi lapos JSON-formátumot a backend 0-es verziójú adapterrel fogadja el. A sémaverziót a feldolgozó név vagy mezőszám alapján keresi, nem a payload pozíciója alapján. A protobuf mezőszámokat soha nem hasznosítjuk újra, az új mezőket opcionálisan vesszük fel. A backend minden támogatott verziót fogad, az ismeretlen mezőket naplózza. Ezt a szabályt használja az Ingest: MQTT-ből a backendbe, idempotensen, az OTA-frissítés: A/B képek, rollback, delta, fokozatos kiadás és a Flottakezelés: nyilvántartás, konfiguráció, állapot alfejezet.

11.2.10 Források és ellenőrzési hiányok

A protobuf mezőszám-szabályai és az ismeretlen mezők megőrzése a Protocol Buffers hivatalos dokumentációjában (https://protobuf.dev/programming-guides/proto3/) találhatók. A JSON- és CBOR-ismeretlen mezők kezelése az RFC 8949 (CBOR) és az általános JSON-gyakorlat szerint történik. A séma-nyilvántartás fogalma a Confluent Schema Registry dokumentációjában (https://docs.confluent.io/platform/current/schema-registry/) érhető el; a TerraSense-ben nem vezettünk be ilyen szolgáltatást. A fenti URL-ek kiindulópontok, az ellenőrzés a megírás időpontjában nem történt meg.

11.3 Idő az IoT-ban: időbélyeg, időszinkron, sorrend

11.3.1 Motiváció

A TerraSense üvegházban a `ts-node` reggel 7:00-kor méri a talajnedvességet, és a háttérrendszernek pontosan tudnia kell, mikor nyitotta ki az öntözőszelepet. A mikrovezérlő órajele azonban nem atomóra: egy olcsó, 20 ppm-es kvarc már naponta több mint másfél másodpercet tévedhet, és a `native_sim` vagy egy valós kártya kikapcsolás után nem őrzi meg a pontos időt. Ráadásul az adat útja a rádiótól a gatewayen át a backendig késleltetésekkel terhelt, így az „időbélyeg” önmagában nem egyértelmű. Ebben az alfejezetben megtervezzük a TerraSense időkezelését: hogyan jelöljük az esemény bekövetkeztét, hogyan szinkronizáljuk az órákat, és hogyan biztosítjuk a mérések helyes sorrendjét akkor is, ha az üzenetek késve vagy átmenetileg elveszve érkeznek.

11.3.2 Fogalmak és működés

Definíció. Az **óraelcsúszás (clock drift)** az a jelenség, amikor egy valós órajel frekvenciája a névlegestől eltér; mértékét **ppm**-ben (parts per million, milliomodrész) adják meg. Az **esemény-idő** az az időpont, amikor a mérés vagy beavatkozás ténylegesen megtörtént; a **feldolgozási idő** az az időpont, amikor egy komponens (gateway, backend) az adatot fogadta; a **monoton óra** egy olyan időforrás, amely soha nem csökken, és független a rendszer valós idejű órájától, mert csak a felindulás óta eltelt időt méri.

Egy tipikus MCU, mint a Zephyr által támogatott ARM Cortex-M, gyakran nem rendelkezik akkumulátoros RTC-vel (Real-Time Clock). Bekapcsoláskor a belső számláló nulláról indul, és a rendszer csupán azt tudja, mennyi ideje fut (`uptime`). A pontos abszolút időt hálózati szinkronizációval kell megszerezni. A legegyszerűbb megoldás az **SNTP** (Simple Network Time Protocol), az NTP (RFC 5905) egyszerűsített kliense, amely UDP/123 porton kérdezi le a szervertől a UTC-t. Az eszköz ritkán (pl. óránként vagy boot után) szinkronizál, mert a rádió bekapcsolása energiát fogyaszt; a két szinkronizálás között az óra a kvarc pontosságától függően driftel.

Az emberi hiba elkerülésére az **UTC**-t használjuk (nincs téli/nyári idő, nincs helyi időzóna), és minden időbélyeget ISO 8601 formátumban vagy epoch milliszekundumban (UTC) jelenítünk meg. A TerraSense konvenciója az **epoch ms**, ahogyan azt az Adatformátumok: JSON, CBOR, Protocol Buffers, SenML alfejezet rögzítette.

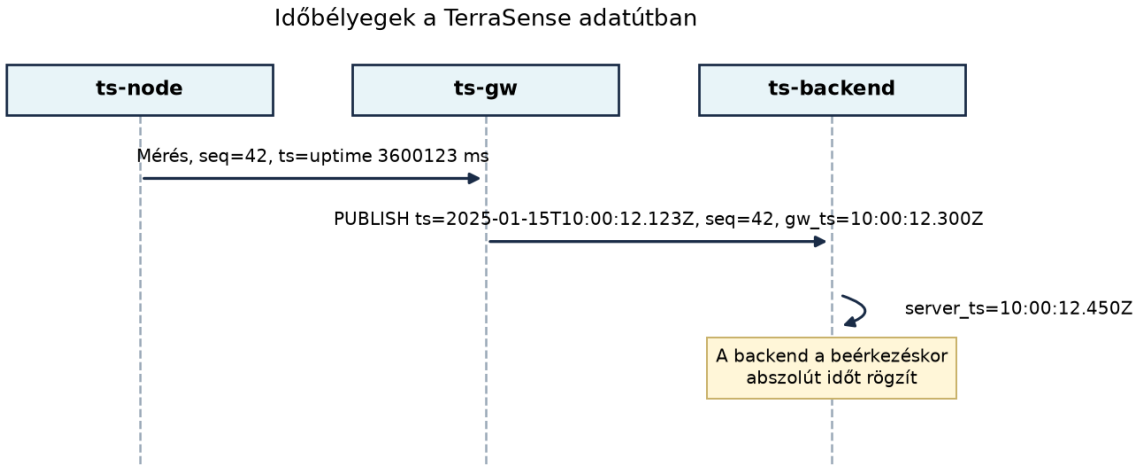
Az adatút során háromféle időbélyeg keletkezhet:

Időbélyeg típus	Ki állítja elő?	Jellemzője
Eszközidő (`ts`)	`ts-node`	Az esemény tényleges időpontja; pontatlan, ha az óra még nem szinkronizált, de a legközelebb áll az igazsághoz
Gateway-idő (`gw_ts`)	`ts-gw`	Az üzenet fogadásának időpontja a gatewayen; pontosabb, de magában foglalja a rádiós késleltetést
Szerveridő (`server_ts`)	`ts-backend`	Az üzenet beérkezésének időpontja a backendben; a legmegbízhatóbb, de a legtávolabb van az eseménytől

Definíció. A **relatív időbélyeg** az esemény időpontja az eszköz monoton órája szerint (uptime milliszekundumban), amelyet a gateway vagy a backend abszolút idővé számíthat vissza, ha ismeri az eszköz rendszerindítási idejét (pl. egy korábbi szinkronizációból vagy a gateway `boot_time` nyilvántartásából): $t_{\text{abs}} = t_{\text{boot}} + \text{uptime}$.

A sorrend egyértelműsítésére a `seq` (sequence number) monoton növekvő sorszám szolgál. Még ha két mérés időbélyege azonos vagy az óra visszaáll (pl. újraindulás után), a `seq` megadja, melyik üzenet keletkezett korábban. A `seq` a `ts-node` által, bootonként 0-tól indul, és minden sikeresen elküldött üzenetnél eggyel nő; a gateway és a backend a `seq` alapján ismeri fel a duplikátumot vagy a hiányt.

A következő diagram a TerraSense tipikus adatútját mutatja a három időbélyeggel:



11.3. ábra: Időbélyegek a TerraSense adatútban - Az eszköz relatív uptime-ot és sorszámot küld; a gateway abszolút időt pecsétel, a backend feldolgozási időt ad. A példaértékek illusztratívak.

Az ábrán a `ts-node` a mérés pillanatában a saját belső uptime-ját (`ts=uptime 3600123 ms`) és a `seq=42` sorszámot küldi. A `ts-gw` hozzáteszi a saját fogadási időpontját (`gw_ts`), majd a `ts-backend` a beérkezéskor elmenti a szerveridőt (`server_ts`). A későbbi feldolgozás során az `eseményidő` az üzenet `ts` mezője, ha az eszköz már szinkronizált; különben a `gw_ts` (vagy a `server_ts`) használandó, de ezt a Ingest: MQTT-ből a backendbe, idempotensen és a Store-and-forward: offline működés és visszatöltés alfejezetek pontosítják.

11.3.3 Döntési szempontok

Az időforrás kiválasztása az energia, a pontosság és a rendelkezésre álló hardver függvénye:

Időforrás	Pontosság	Energiaigény	Használat a TerraSense-ben
Belső kvarc, nincs szinkron	10–100 ppm, naponta több másodperc hiba	minimális	Alapértelmezett a költségtakarékos csomópontoknál

Időforrás	Pontosság	Energiaigény	Használat a TerraSense-ben
SNTP/NTP szinkron	néhány ms–s a hálózati késleltetéstől függően	rádióhasználat miatt közepes	A `ts-node` ritkán (pl. boot után) szinkronizál; a `ts-gw` folyamatosan NTP-t futtat
RTC külső kristállyal	néhány ppm, de akkumulátort igényel	nagyon alacsony	Opcionális, drágább hardvereken
Gateway-idő	1 ms alatt a helyi RTC-ből	nem terheli az eszközt	Minden továbbított üzenetre rákerül
Szerveridő	ms-os pontosság a fogadási oldalon	nem terheli az eszközt	A backend mindig feljegyzi

A döntési szabály: a pontos eseményidőhöz **először próbáljuk az eszközidőt használni**, de csak akkor, ha az eszköz órája szinkronizált. Ha ez nem elérhető vagy megbízhatatlan, akkor a **gateway-idő** a másodlagos, a **szerveridő** pedig a végső tartalék. A `seq` minden esetben kötelező, mert a sorrendet és az esetleges duplikátumokat ez alapján lehet eldönteni, függetlenül az órák pontosságától.

11.3.4 Kidolgozott példa: a TerraSense időbélyeg-stratégiája

Tegyük fel, hogy a `ts-node` órája 20 ppm-es kvarcon fut, és a gateway állandóan pontos NTP-szinkronnal rendelkezik. A mérés 2025-01-15 10:00:00 UTC-kor történik, de az eszköz órája az utolsó szinkronizálás óta 0,8 másodpercet siet. A `ts-node` ezért `ts=2025-01-15T10:00:00.800Z`-t, `seq=42`-t és `uptime=3600123` ms-ot küld. A `ts-gw` a fogadáskor hozzáadja a saját `gw_ts=2025-01-15T10:00:12.300Z`-t (a rádiós átvitel és a gateway sorban állása 11,5 mp késést okozott). A backendben a beérkezés ideje `server_ts=2025-01-15T10:00:12.450Z`. A feldolgozó komponens látja, hogy a `ts` mező 0,8 mp-vel későbbi a gateway időbélyegénél, tehát az eszköz órája siet; mivel a `seq=42` és a korábbi `seq=41` üzenet időbélyege közötti különbség 10 másodperc (a mintavételi periódus), a rendszer a `ts` mezőt 0,8 mp-cel korrigálja, és az eseményidőt 10:00:00.000 UTC-ként tárolja el. Ezt a korrekciót azonban csak akkor végezzük el, ha a hiba kisebb, mint egy előre definiált küszöb (pl. 5 másodperc); nagyobb eltérés esetén a `gw_ts`-t használjuk, és riasztást adunk az óra újrakalibrálására.

Ez a példa bemutatja a továbbvitt konvenciót: - **Eseményidő:** az `ts` mező, epoch ms UTC-ben; ha hiányzik vagy érvénytelen, a `gw_ts` lép a helyébe. - **Sorszám:** a `seq` mező, `uint32_t`, monoton növekvő, bootonként újraindulhat, de a gateway és a backend párosítja a `seq`-et a node azonosítójával, és észleli a nem monoton vagy hiányzó értékeket. - **Boot azonosító:** opcionálisan a `boot_id` mező, amely megkülönbözteti az eszköz újraindulásait, így a `seq` újraindulását is.

11.3.5 Labor: monoton óra és sorszám a `ts-node`-ban

Munkakönyvtár: `~/iot-labor/i034`. Hozzuk létre, majd a következő Zephyr `native_sim` alkalmazást valósítjuk meg, amely bemutatja a relatív időbélyeg és a sorszám használatát.

```
mkdir -p ~/iot-labor/i034 && cd ~/iot-labor/i034
```

Hozzuk létre a `CMakeLists.txt` fájlt:

```
cmake_minimum_required(VERSION 3.20)
find_package(Zephyr REQUIRED HINTS $ENV{ZEPHYR_BASE})
project(ts_node_time)

target_sources(app PRIVATE src/main.c)
```

A `prj.conf` fájl:

```
CONFIG_SYS_CLOCK_TICKS_PER_SEC=1000
CONFIG_PRINTK=y
CONFIG_CBPRINTF_FULL_INTEGRAL=y
```

A `src/main.c` fájlban két szál fut: a mérőszál periodikusan „mérést” végez, és növeli a `seq`-et, a `ts` mezőt pedig a 64 bites `k_uptime_get()` és a legutóbbi szinkronizáció alapján számolja. Szinkronizáció előtt a `ts` értéke nem hamis epoch 0, hanem `n/a`, és a `ts_source` mező jelzi, hogy a relatív uptime áll rendelkezésre.

```
#include <zephyr/kernel.h>
#include <zephyr/sys/printk.h>
#include <stdbool.h>
#include <stdint.h>

#define SENSOR_STACK_SIZE 1024
#define MIN_VALID_EPOCH_MS 1577836800000LL  /* 2020-01-01 UTC */

K_THREAD_STACK_DEFINE(sensor_thread_stack, SENSOR_STACK_SIZE);
static struct k_thread sensor_thread_data;

static uint32_t seq;
static uint32_t boot_id = 0x5EED0001u;  /* szimulált; élesben RNG/RTC */
static bool clock_synced;
static int64_t sync_uptime_ms;
static int64_t sync_epoch_ms;
static int64_t sync_uncertainty_ms;

static bool sync_with_gateway(int64_t gateway_epoch_ms,
                              int64_t gateway_uptime_ms,
                              int64_t rtt_ms) {
    if (gateway_epoch_ms < MIN_VALID_EPOCH_MS) {
        printk("sync rejected: invalid epoch=%lld\n",
               (long long)gateway_epoch_ms);
        return false;
    }
    if (gateway_uptime_ms < 0 || gateway_uptime_ms > k_uptime_get()) {
        printk("sync rejected: invalid uptime=%lld\n",
               (long long)gateway_uptime_ms);
        return false;
    }
    if (rtt_ms < 0 || rtt_ms > 5000) {
        printk("sync rejected: invalid rtt=%lld\n", (long long)rtt_ms);
        return false;
    }
    sync_epoch_ms = gateway_epoch_ms;
    sync_uptime_ms = gateway_uptime_ms;
    sync_uncertainty_ms = rtt_ms / 2;
    clock_synced = true;
    return true;
}

void sensor_thread(void *arg1, void *arg2, void *arg3) {
    (void)arg1;
    (void)arg2;
    (void)arg3;
```

```
    while (1) {
        int64_t uptime_ms = k_uptime_get();
        uint32_t temp_x10 = 225u + (seq % 10u);
        uint32_t hum_x10 = 550u + (seq % 5u) * 5u;

        if (!clock_synced && seq == 3u) {
            if (sync_with_gateway(1735689600000LL, uptime_ms, 4)) {
                printk("clock synced at uptime_ms=%lld\n",
                       (long long)uptime_ms);
            }
        }
        if (clock_synced) {
            int64_t event_ms = sync_epoch_ms + (uptime_ms - sync_uptime_ms);
            printk("seq=%u boot=%08x src=device ts=%lld up=%lld unc=%lld "
                   "temp=%u.%u hum=%u.%u\n",
                   seq, (unsigned int)boot_id, (long long)event_ms,
                   (long long)uptime_ms, (long long)sync_uncertainty_ms,
                   temp_x10 / 10u, temp_x10 % 10u,
                   hum_x10 / 10u, hum_x10 % 10u);
        } else {
            printk("seq=%u boot=%08x src=uptime ts=n/a up=%lld "
                   "temp=%u.%u hum=%u.%u\n",
                   seq, (unsigned int)boot_id, (long long)uptime_ms,
                   temp_x10 / 10u, temp_x10 % 10u,
                   hum_x10 / 10u, hum_x10 % 10u);
        }
        seq++;
        k_msleep(1000);
    }
}

int main(void) {
    printk("ts-node time example\n");
    k_thread_create(&sensor_thread_data, sensor_thread_stack,
                    K_THREAD_STACK_SIZEOF(sensor_thread_stack),
                    sensor_thread, NULL, NULL, NULL,
                    K_PRIO_PREEMPT(5), 0, K_NO_WAIT);
    return 0;
}
```

A `K_THREAD_STACK_DEFINE` egy 1024 bájtos, statikusan foglalt vermet hoz létre, a `struct k_thread sensor_thread_data` pedig a szál vezérlőblokkja; a `k_thread_create` mindkettőt megkapja. A `CONFIG_CBPRINTF_FULL_INTEGRAL=y` teszi lehetővé, hogy a `printk` a 64 bites `%lld` értékeket is helyesen jelentse meg; e nélkül a formázás csonkolhat. A 64 bites `k_uptime_get()` nem fordul körbe 49,7 nap után, mint a `k_uptime_get_32()`. A `sync_with_gateway` csak érvényes epochot és a jelenlegi uptime-nál nem nagyobb gateway-uptime-ot fogad el; a szinkronizáció tényét külön `clock_synced` jelző tárolja, így a szinkronizálatlan induló állapot nem jelenik meg epoch 0-ként. A szinkronizáció után az eseményidő $t_{\text{event}} = t_{\text{sync,epoch}} + (t_{\text{uptime}} - t_{\text{sync,uptime}})$, amely a kvarc driftje miatt a szinkronizáció óta eltelt idővel arányosan bizonytalanabb lesz. A példában a 4 ms-os round-trip fele, azaz 2 ms kerül a `sync_uncertainty_ms` mezőbe és az `unc` kimenetre; valós SNTP-nél a négy időbélyegből számolt offset hibája legfeljebb a round-trip késleltetés fele, amelyet az eseményidő mellett kell tárolni, és a fogadási időtől külön kell kezelni. A gateway `gw_ts` és a backend `server_ts` továbbra is külön mező; a `ts_source=device` csak a szinkronizált eszközidőre utal. A `boot_id` minden újraindulásnál változik, ezért a `(node_id, boot_id, seq)` hármas akkor sem ütközik, ha a `seq` nulláról indul újra. A példában a szinkronizáció ugyanabban a szálban történik; külön szinkronizáló szál esetén a `clock_synced`, `sync_epoch_ms` és `sync_uptime_ms` együttes olvasását mutexszel vagy atomi pillanatképpel kell védeni.

Build és futtatás:

```
west build -b native_sim . -d build
./build/zephyr/zephyr.exe
```

Illusztratív kimenet (a pontos uptime-értékek a szimulátor ütemezésétől függnek; a programot a lektor nem futtatta):

```
ts-node time example
seq=0 boot=5eed0001 src=uptime ts=n/a up=0 temp=22.5 hum=55.0
seq=1 boot=5eed0001 src=uptime ts=n/a up=1000 temp=22.6 hum=55.5
seq=2 boot=5eed0001 src=uptime ts=n/a up=2000 temp=22.7 hum=56.0
clock synced at uptime_ms=3000
seq=3 boot=5eed0001 src=device ts=1735689600000 up=3000 unc=2 temp=22.8 hum=56.5
seq=4 boot=5eed0001 src=device ts=1735689601000 up=4000 unc=2 temp=22.9 hum=57.0
...
```

Látható, hogy szinkronizáció előtt a `ts` nem hamis epoch, hanem `n/a`, miközben az `up` (uptime) monoton nő; a szinkronizáció után a `ts` a 64 bites uptime és a rögzített offset különbségéből számolódik, a `boot` az újraindítási sorozatot azonosítja. A `seq` minden mérésnél növekszik. Ez a hármas (`boot_id`, `seq`, eseményidő) elegendő ahhoz, hogy a gateway vagy a backend akkor is azonosítsa a sorrendet, ha az abszolút idő bizonytalan. **Továbbvitt eredmény.** A `boot_id` és a `seq` konvenció kerül be a telemetria-üzenet sémájába, és a Store-and-forward: offline működés és visszatöltés alfejezet az újraküldés során erre az azonosítóra támaszkodik; a Idősoros adatbázisok és adatmodellezés alfejezet a TimescaleDB táblába ezt a `seq` mezőt is elmenti.

11.3.6 Ellenőrzés és hibakeresés

A laborban futtassuk a programot 10 másodpercig, és ellenőrizzük, hogy a `seq` és az `uptime_ms` értékek szigorúan monoton növekednek. A `native_sim` időalapja a host órájára támaszkodik, így a lépésköz 1000 ms ± néhány ms pontossággal megjelenik; valós hardveren a drift méréséhez külső időforrásra van szükség.

Tipikus hibakeresési táblázat:

Hibatünet	Lehetséges ok	Javítás
A `seq` nem monoton növekvő	Az eszköz újraindult, és a számláló nulláról indult	Adjunk hozzá `boot_id` mezőt, és a gateway/backend a `(node_id, boot_id, seq)` hármast használja a sorrendhez
Az `up` érték ugrik vagy visszaáll	A szál újraindult, vagy régi kódban a `k_uptime_get_32()` túlcsordult	Használjunk 64 bites `k_uptime_get()`-et; az újraindulást a `boot_id` különbözteti meg
A `ts` tartósan `n/a`	A szinkronizáció elmaradt, vagy érvénytelen epochot/RTT-t kapott	Ellenőrizzük a gateway epochját és a round-trip késleltetést; szinkronizációig a gateway `gw_ts`-t használjuk
Az eseményidő új szinkronizáció után visszaugrik	Az új offset kisebb a korábbinál; a már tárolt eseményeket nem szabad átírni	Tároljuk külön az esemény- és fogadási időt, a sorrendhez a `(boot_id, seq)`-et használjuk, és őrizzük meg a `ts_source`/bizonytalanság mezőt

Hibatünet	Lehetséges ok	Javítás
A `ts` mező abszolút időt tartalmaz, de helytelen időzónát	Az eszköz helyi időt küldött	Kötelező az UTC; az eszközön ne tároljunk időzónát, a konverziót a backend végezze
A gateway `gw_ts` és a backend `server_ts` között nagy a késés	A broker bemeneti sora túlterhelt	Figyeljük a broker queue hosszát, és a backend skálázásával csökkentsük a feldolgozási időt; a mező puszta jelenléte nem okoz hibát, de a riasztásoknál figyelembe kell venni

11.3.7 Tipikus hibák és félreértések

1. **Helyi idő használata UTC helyett.** Ha a `ts-node` a beépített RTC-ből helyi időt olvas, a nyári/téli időszámítás váltásakor az időbélyegek kétértelművé vagy hibássá válnak. Mindig UTC-t használjunk, és a megjelenítéskor alakítsuk a felhasználó időzónájára.
2. **A `seq` hiánya és az időre hagyatkozás a sorrendhez.** Ha két mérés ugyanabban a milliszekundumban történik, vagy az óra visszaállítása miatt az időbélyeg csökken, a backend nem tudja eldönteni a sorrendet. A monoton sorszám (és opcionálisan a `boot_id`) kötelező.
3. **Abszolút idő használata szinkronizáció nélkül.** Egy frissen bootolt eszköz RTC-hiányában a `ts` mező értéke 0 vagy az 1970-es epoch, ami hamis riasztásokat okoz. Ilyenkor a `ts` legyen `n/a` vagy hiányzó, az esemény monoton `uptime`-ját külön mezőben küldjük, és a gateway a fogadáskor `gw_ts`-t pecsétel.
4. **A monoton óra és a valós idő összekeverése.** A `k_uptime_get()` a rendszer indulásától eltelt időt adja, nem az epoch időt. A két fogalom nem helyettesíthető: a sorrendhez a monoton, a rögzített eseményidőhöz a valós (szinkronizált) idő kell.

11.3.8 Feladatok

1. Fogalmi: Melyik időbélyeget használja a TerraSense backend, ha a `ts-node` órája 3 másodpercet késik a gatewayhez képest, és a mérési periódus 60 másodperc? Indokolja a választ.
2. Számolási: Egy MCU órajele 30 ppm-es pontosságú. Számítsa ki, mekkora hibát halmoz fel egy hónap (30 nap) alatt, ha nincs közben szinkronizálás.
3. Implementációs: Módosítsa a labor `sensor_thread` függvényét úgy, hogy a `ts` mező a legutóbbi szinkronizációs offset segítségével számított `epoch_ms` UTC legyen, szinkronizáció előtt pedig `n/a`. A kiíratásban jelenjen meg az esemény abszolút és monoton ideje, a szinkronizáció bizonytalansága (`uncertainty_ms`), valamint a gateway fogadási ideje (`gw_ts`) külön mezőben.
4. Hibakeresés: A backend naplójában a `seq` értékek 0, 1, 2, 3, 0, 1, 2… szekvenciát mutatnak ugyanattól a `node_id`-től. Mi lehet a probléma, és hogyan javítaná?

11.3.9 Megoldások

1. A backend a `ts` mezőt használja, ha a késés (3 mp) kisebb, mint a mintavételi periódus (60 másodperc) fele, tehát az események sorrendje egyértelmű, és a `ts` továbbra is a legközelebbi

közelítése az eseményidőnek. Azonban a rendszer riasztást adhat az óra kalibrálására. Ha a késés nagyobb lenne, mint a periódus fele, akkor a `gw_ts`-t kell választani, mert az eszközidő már megbízhatatlan. A `seq` mindenképpen megőrzi a sorrendet.

2. Az óra elcsúszása havonta:

$$\text{drift} = 30 \cdot 86400 \cdot \frac{30}{10^6} = 77760 \cdot 10^{-2} = 77.76 \text{ s}$$

   Tehát egy hónap alatt 77,76 másodpercet késik, ha a drift pozitív (az óra lassabb). Ha siet, akkor ennyivel előrébb jár.

3. A labor megoldásában a `ts` csak érvényes szinkronizáció után epoch; a lényeges részek:

```
#define MIN_VALID_EPOCH_MS 1577836800000LL  /* 2020-01-01 UTC */
#define DRIFT_PPM 20

static bool clock_synced;
static int64_t sync_epoch_ms;
static int64_t sync_uptime_ms;
static int64_t sync_uncertainty_ms;
static uint32_t boot_id = 0x5EED0001u;
static uint32_t seq;

bool sync_with_gateway(int64_t gateway_epoch_ms,
                       int64_t gateway_uptime_ms,
                       int64_t rtt_ms) {
    if (gateway_epoch_ms < MIN_VALID_EPOCH_MS ||
        gateway_uptime_ms < 0 ||
        gateway_uptime_ms > k_uptime_get() ||
        rtt_ms < 0 || rtt_ms > 5000) {
        clock_synced = false;
        return false;
    }
    sync_epoch_ms = gateway_epoch_ms;
    sync_uptime_ms = gateway_uptime_ms;
    sync_uncertainty_ms = rtt_ms / 2;
    clock_synced = true;
    return true;
}

void sensor_thread(void *arg1, void *arg2, void *arg3) {
    (void)arg1; (void)arg2; (void)arg3;
    while (1) {
        int64_t uptime_ms = k_uptime_get();
        if (clock_synced) {
            int64_t elapsed_ms = uptime_ms - sync_uptime_ms;
            int64_t event_epoch_ms = sync_epoch_ms + elapsed_ms;
            int64_t uncertainty_ms = sync_uncertainty_ms +
                (elapsed_ms / 1000) * DRIFT_PPM / 1000;
            printk("seq=%u boot=%08x ts=%lld up=%lld unc=%lld\n",
                   seq, (unsigned int)boot_id,
                   (long long)event_epoch_ms, (long long)uptime_ms,
                   (long long)uncertainty_ms);
        } else {
            printk("seq=%u boot=%08x ts=n/a up=%lld\n",
                   seq, (unsigned int)boot_id, (long long)uptime_ms);
        }
        seq++;
        k_msleep(1000);
    }
}
```

Az `event_epoch_ms` az esemény ideje; a gateway a fogadáskor ettől független `gw_ts` mezőt, a backend a `server_ts` mezőt rögzíti. Az `uncertainty_ms` a szinkronizációs offset hibájából (legfeljebb RTT/2) és a szinkronizáció óta eltelt idő driftjéből adódik. A `boot_id` a `seq` újraindulását különíti el; a már tárolt eseményeket egy újabb szinkronizáció nem írja át, a sorrendet a `(boot_id, seq)` adja. A szinkronizáció indítása történhet a `main`-ből vagy külön szálból; külön szál esetén a közös offsetet mutexszel vagy atomi pillanatképpel kell védeni.

4. A `seq` újraindulása azt jelzi, hogy az eszköz újraindult (pl. watchdog, áramkimaradás), és a sorszám számláló lenullázódott. A backend ezt az `(node_id, seq)` párosban észleli, és ha a `seq` hirtelen kisebb lesz, mint az előző, újraindulást feltételez. A javítás: az üzenetbe foglaljunk `boot_id` mezőt (pl. a boot időbélyege vagy egy véletlen szám), és a backend a `(node_id, boot_id, seq)` hármas alapján azonosítsa a sorozatot. Az újraindulás után a `boot_id` változik, így a `seq` újraindulása nem okoz hamis duplikátum- vagy hiányérzékelést.

11.3.10 Rövid összefoglaló

Az alfejezet bevezette az IoT-rendszerek időkezelési alapjait: az óraelcsúszást (ppm), az SNTP/NTP-szinkronizációt, az eseményidő és a feldolgozási idő különbségét, valamint a relatív időbélyeg visszaszámítását. A TerraSense stratégiája három időbélyegre épül: az eszköz `ts` mezője (epoch ms UTC, ha az óra szinkronizált; egyébként `n/a`, és az esemény monoton uptime-ja külön mezőben utazik), a gateway `gw_ts` mezője, és a backend `server_ts` mezője. A sorrendet a monoton növekvő `seq` sorszám és opcionálisan a `boot_id` biztosítja. A labor egy `native_sim` Zephyr-alkalmazást mutat be, amely a 64 bites `k_uptime_get()`, a `clock_synced` jelző, a `boot_id` és a `seq` használatával generál időbélyegzett telemetriát; szinkronizáció előtt a `ts` értéke `n/a`, a fogadási idő pedig külön `gw_ts`/`server_ts` mező marad. **Továbbvitt eredmény.** Az eseményidő (`ts`) és a sorszám (`seq`) mezőket, valamint a `boot_id` kiegészítést használja a Store-and-forward: offline működés és visszatöltés az újraküldések deduplikálására, az Idősoros adatbázisok és adatmodellezés a TimescaleDB sémában, és a Eseményvezérelt feldolgozás: Kafka és a kézbesítési szemantika az üzenetek sorrendjének és idempotenciájának biztosítására.

11.3.11 Források és ellenőrzési hiányok

- RFC 5905 (NTP) és RFC 4330 (SNTP); az offset és a round-trip késleltetés számítása az RFC 4330 5. szakaszában.
- Zephyr 3.7.0 dokumentáció: Kernel timing (`k_uptime_get`, `k_uptime_get_32`), valamint a `CONFIG_CBPRINTF_FULL_INTEGRAL` beállítás a 64 bites `%lld` formázáshoz.
- A `native_sim` board időzítési viselkedése.
- Ellenőrizendő: a `native_sim` időbázisának pontossága a telepített Zephyr-verzióban. A labort a lektor nem futtatta; a kód statikusan ellenőrzött, a 64 bites `k_uptime_get()` és a `printk` `%lld` támogatása a Zephyr 3.7.0 dokumentációján alapul. A `k_uptime_get_32` 32 bites felső határa miatt 49,7 naponként túlcsordul, ezért a labor 64 bites változatot használ; a 10 másodperces futás a túlcsordulást nem mutatná meg.

IV. RÉSZ – Edge és gateway

12 A gateway

12.1 A gateway szerepe: híd, puffer, biztonsági határ

12.1.1 Motiváció

A TerraSense üvegházban a Topológiák: csillag, mesh, gateway-alapú hálózatok alfejezet döntése szerint a `n001`, `n002`, … szenzorcsomópontok csillag topológiában kapcsolódnak egy központi egységhez. Az eszközök BLE GATT-tel, a labor szimulációban UDP-vel küldik a mérési adatokat. A Mi az IoT? Rétegek, szereplők, adatút alfejezetben bevezetett rétegmodell alapján ez a központi egység az edge/gateway réteg: ez az a hely, ahol a helyi eszközhálózat kilép az internetező világba.

Ha minden `ts-node` közvetlenül MQTT/TLS-t beszélne a brokerrel, akkor mindegyik eszköznek TCP/IP hálózati stackre, TLS-tanúsítványra, IP-címre és folyamatos vagy gyakori rádióidőre lenne szüksége. Ez energia, memória, provisioning és karbantartási költség. A gateway ezt a komplexitást egyetlen, jobban erőforrás-ellátott komponensbe emeli ki.

Emellett a felhőkapcsolat nem garantált: az üvegház internetkapcsolata kieshet, a `ts-backend` karbantartás alatt lehet, a Mosquitto broker rövid időre elérhetetlenné válhat. Ilyenkor a rendszernek akkor is működnie kell: a mérési adatokat nem szabad elveszíteni, és a kritikus öntözésvezérlésnek helyben kell maradnia. Ezért a gateway nem egyszerű „drótnélküli híd”, hanem üzleti és biztonsági szempontból is önálló komponens.

12.1.2 Fogalmak és működés

Definíció. Az IoT gateway (átjáró) az edge rétegben elhelyezkedő szoftver- és hardverkomponens, amely egy vagy több helyi/peremi protokollt és adatmodellt (például BLE GATT, Modbus, UDP, LoRaWAN) alakít át egy vagy több felhő- vagy vállalati protokollra (például MQTT, OPC UA, Kafka), elválasztja a helyi eszközhálózatot a felhőhálózattól, és emellett pufferelést, helyi feldolgozást, valamint biztonsági határfunkciót is megvalósít.

A gateway tehát nem router: a router a hálózati rétegben csomagokat továbbít anélkül, hogy értené az alkalmazási adatot. A gateway ezzel szemben alkalmazási rétegbeli határon dolgozik, ismeri a telemetria sémáját, és képes azon döntést, átalakítást, szűrést végezni.

A TerraSense gateway fő feladatai a következők.

Protokollfordítás. A `ts-gw` a labor szimulációban UDP datagramokat fogad, valós hardveren pedig BLE GATT értesítéseket olvas. Mindkettőből a TerraSense MQTT topicjaira kerülő, egységes telemetriaüzenet lesz. Ipari környezetben ugyanez a modell alkalmazható Modbus RTU/TCP vagy OPC UA bemenetre is.

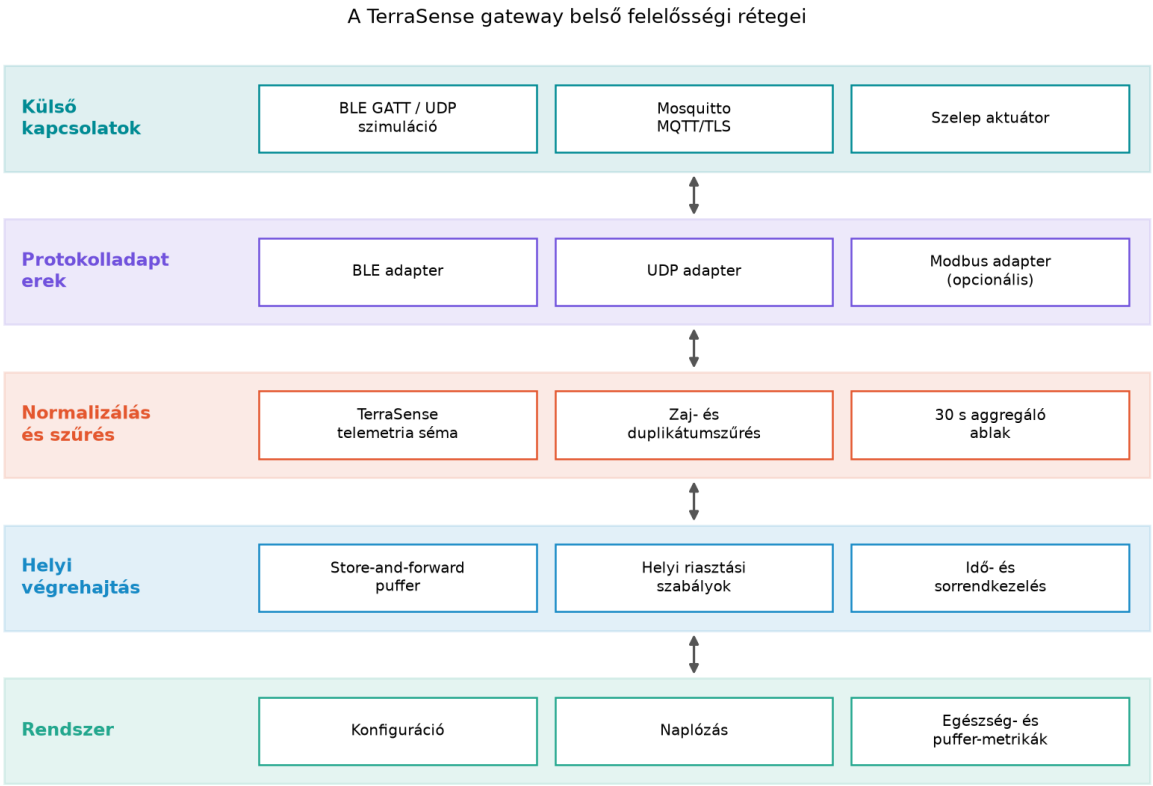
Aggregálás és szűrés. Az eszköz általában gyakrabban mér, mint ahogy a felhőnek az adatra szüksége van. A gateway időablakokban átlagot, minimumot, maximumot vagy mintadarabszámot számolhat, és csak az aggregált eredményt küldi tovább. A zajos, tartományon kívüli vagy duplikált mérést is itt érdemes eltávolítani.

Offline puffer. Ha az MQTT-kapcsolat megszakad, a gatewaynek az üzeneteket tartósan, jellemzően lemezen kell tárolnia, majd a kapcsolat visszatérésekor sorrendben visszaküldenie. Ez a Store-and-forward: offline működés és visszatöltés alfejezet témája lesz.

Helyi riasztás. Vannak olyan szabályok, amelyeknek akkor is futniuk kell, ha a felhő nem érhető el. Ilyen például a talajnedvesség alsó küszöbértéke: ha a `ts-node` mérése szerint a talaj túl száraz, a gatewaynek helyben kell kezdeményeznie az öntözőszelep nyitását, és riasztást kell küldenie.

Biztonsági határ. A gateway a helyi eszközhálózatot elválasztja a felhő felé nyitott hálózattól. Az eszközök nem kapnak közvetlen internetelérést, és nem is fut rajtuk teljes TLS/MQTT stack; a gateway viszont a kifelé irányuló kapcsolatban mTLS-tanúsítvánnyal azonosítja magát, és titkosított csatornán kommunikál a brokerrel.

A következő ábra a gateway belső felelősségi rétegeit mutatja.



12.1. ábra: A TerraSense gateway belső felelősségi rétegei - A nyilak az adat- és vezérlési utat egyaránt jelölik. Az alsóbb rétegek a helyi eszközök felé, a felső réteg a felhő felé dolgozik.

A rétegek leírása:

- **Külső kapcsolatok:** ide tartozik minden tényleges hálózati vagy fizikai bemenet és kimenet. A TerraSense-ben a bemenet BLE GATT vagy a laborban használt UDP-szimuláció, a kimenet pedig MQTT a Mosquitto felé. Az aktuátor felé irányuló vezérlés is itt jelenik meg.

- **Protokolladapterek:** minden protokoll kap egy saját adaptert, amely leválasztja a protokoll sajátosságait a közös belső modellről. Így egy új protokoll hozzáadása nem érinti a szűrő- vagy pufferlogikát.
- **Normalizálás és szűrés:** ebben a rétegben keletkezik az egységes `terrasense/<site>/<node>/telemetr` üzenet. Itt ellenőrizzük a mérési tartományt, a `seq` sorszám alapján kiszűrjük a duplikátumokat, és elvégezzük az aggregálást.
- **Helyi végrehajtás:** itt fut a store-and-forward puffer és a helyi riasztási szabályrendszer. Ez a réteg akkor is működik, ha a felhő nem elérhető.
- **Rendszer:** a konfiguráció, a naplózás és az egészségállapot-megfigyelés. Ezek nem az adatút részei, de üzemeltetés nélkül nincs megbízható gateway.

Egy konkrét adatút a következőképpen néz ki: a BLE-adapter fogad egy `23.1` hőmérséklet-értesítést, amely a `site=gh1`, `node=n001`, `boot=7`, `seq=42` és `ts=...` azonosítókat is tartalmazza. A normalizáló ellenőrzi ezeket, szükség esetén `gateway_estimate` időbélyeget és `estimated_time` minőségjelzőt állít be, a szűrő eldönti, hogy az érték érvényes-e, az aggregáló pedig 30 s-os ablakban átlagot számol. Az elkészült üzenet bekerül a store-and-forward pufferbe, majd az MQTT-kapcsolaton eljut a brokerre. Ha a kapcsolat éppen nincs, az üzenet a lemezen marad, és visszatöltésre vár.

12.1.3 Döntési szempontok

A legfontosabb tervezési kérdés, hogy egy adott funkció az eszközön, a gatewayen vagy a felhőben fusson-e. A válasz mindig energia, késleltetés, sávszélesség, offline tűrőképesség és karbantarthatóság függvénye.

Funkció	Eszközön	Gatewayen	Felhőben	TerraSense döntés és indok
Protokollfordítás	Nem, az eszköznek egy protokollt kell beszélnie	Igen, ez az elsődleges feladat	Nem értelmezhető	Gatewayen: a `ts-node` BLE/UDP marad, a `ts-gw` MQTT-t beszél
Azonnali szelepvédelem	Kritikus határérték, hardverközeli védelem	Megerősítő szabály, időablak, helyi riasztás	Policy, audit, flottaszintű elemzés	Eszközön és gatewayen is, mert a szelepvezérlésnek akkor is működnie kell, ha a gateway vagy a felhő kiesik
Zaj- és duplikátumszűrés	Alapvető tartományellenőrzés	Részletes, `seq` alapú szűrés	Statisztikai utófeldolgozás	Gatewayen a részletes szűrés, mert az eszközön korlátozott a memória
Aggregálás	Csak akkor, ha az energiaköltség indokolja	30 s ablak, min/max/átlag	Hosszú távú aggregálás az adatbázisban	Gatewayen, mert így csökken a felhő felé irányuló üzenetszám

Funkció	Eszközön	Gatewayen	Felhőben	TerraSense döntés és indok
Offline puffer	Nem, az eszköz memóriája kicsi	Igen, lemezen	A felhő kiesésekor a gatewaynek kell pufferelnie	Gatewayen, ez a store-and-forward alapja
Helyi riasztás	Életvédelmi/berendezésvédelmi határ	Részletes szabályok, timeout és nyugtázás	Globális szabályok, értesítés, tanulás	Gatewayen, mert a helyi döntésnek felhő nélkül is futnia kell
Hálózati biztonsági határ	Eszközoldali védelem, például BLE-párosítás	Határzóna, mTLS a broker felé	Központi PKI, audit	A gateway a biztonsági határ, az eszközöknek nincs közvetlen internetelérésük
Hosszú távú analitika, gépi tanulás	Nem	Később edge-feldolgozás lehet	Igen, itt van elegendő erőforrás	Felhőben, később a gatewayen csak előszűrt jellemzők

A táblázatból kiolvasható az az általános elv, hogy **az eszközön a hardverközeli, gyors védelem fut, a gatewayen az üzleti és offline feldolgozás, a felhőben pedig a hosszú távú, flottaszintű elemzés**.

12.1.4 Kidolgozott példa: TerraSense gateway-feladatlista

A későbbi alfejezetek a `ts-gw` szolgáltatást fokozatosan építik fel. Ahhoz, hogy a fejlesztési lépések egymásra épüljenek, most rögzítjük a gateway feladatlistáját egy verziózott dokumentumban.

Munkakönyvtár:

```
mkdir -p ~/iot-labor/i035 && cd ~/iot-labor/i035
```

A `gateway-feladatlista.md` tartalma a következő:

```
# TerraSense gateway-feladatlista (i035)
# Állapot: jóváhagyott alap az i036, i037 és i039 számára.

R1 Protokollfordítás: BLE GATT (valós hardver) / UDP (szimuláció) -> MQTT.
   Labor bemenet: udp:0.0.0.0:4000 (konfigurálható), a datagram hordozza a
   site és node azonosítót; kimenet: terrasense/<site>/<node>/telemetry.
   A gateway egyszerre több site/node forgalmát fogadja, ezért a topicot
   mintánként kell képezni, nem rögzített gh1/n001 értékkel.

R2 Normalizálás: az üzenetben site, node_id, boot_id, monoton seq és ts
   epoch_ms mezők szerepelnek. Mértékegység-szerződés: hőmérséklet °C
   (a ts-node által már konvertálva), relatív páratartalom %RH 0..100,
   talajnedvesség térfogati % 0..100. Ezek nem SI-alapegységek; a JSON
   mezőnevek temperature_c, humidity_rh, soil_moisture_pct, a SenML
   megfelelőjük Cel, %RH, %. A hiányzó mérés JSON null, nem 0 és nem NaN.

R3 Aggregálás: 30 s ablakonként per node min/max/átlag és minta-darabszám,
   ha az adott mérési típusra az aggregálás engedélyezett.

R4 Szűrés: tartományon kívüli minta elvetése; site/node azonosító és
```

```
   numerikus mező ellenőrzése; a (site, node_id, boot_id, seq) kulcs
   monotonitásának ellenőrzése. Hiányzó vagy érvénytelen device ts esetén
   a gateway gateway_estimate időbélyeget és estimated_time minőségjelzőt
   használ (nem csendben device időt), opcionálisan karanténba teszi a
   mintát. A gateway saját órájának szinkronállapotát naplózza.

R5 Store-and-forward: offline puffer lemezen, FIFO sorrend. A gateway
   három állapotot különböztet meg: helyben rögzített (spooled), broker
   által nyugtázott (broker_acked, QoS 1 PUBACK) és backend által
   feldolgozott (backend_committed). A puffer idempotens újraküldése a
   (site, node_id, boot_id, seq) eseménykulcsot használja; a backend
   külön idempotencia-kulccsal védi az üzleti hatást. boot_id nélküli
   örökölt üzenetnél a kulcs gyengébb, ezt a gateway naplózza.

R6 Helyi riasztás: talajnedvesség < 20 % -> öntözőszelep nyitás + riasztás;
   talajnedvesség > 80 % vagy hőmérséklet > 40 °C -> öntözés blokkolása
   és figyelmeztetés.

R7 Biztonsági határ: az eszközhálózat nem érheti el közvetlenül az
   internetet; a ts-gw kifelé mTLS-tanúsítvánnyal csatlakozzon a brokerhez.

R8 Üzemeltetés: puffer mélysége, uptime, MQTT kapcsolat állapota és
   naplózás a gateway egészségállapotához.
```

Továbbvitt eredmény. A `~/iot-labor/i035/gateway-feladatlista.md` R1–R8 követelményeit használja az A ts-gw szolgáltatás C++-ban: protokollfordítás MQTT-re alfejezet a szolgáltatás vázának kialakításához, a Store-and-forward: offline működés és visszatöltés alfejezet a puffer implementációjához, valamint a Helyi feldolgozás: szűrés, aggregálás, eseménydetektálás alfejezet a szűrő- és aggregálólogikához.

12.1.5 Ellenőrzés és hibakeresés

A feladatlista akkor jó, ha minden rétegbeli döntés visszavezethető a TerraSense követelményeire. Ellenőrizhetjük a dokumentum szerkezetét:

```
grep -n '^R[0-9]' ~/iot-labor/i035/gateway-feladatlista.md
```

A kimenetben mind a nyolc követelménynek szerepelnie kell. Funkcionális ellenőrzést a későbbi alfejezetek adnak, de már most érdemes egy képzeletbeli offline tesztet végigjátszani: kapcsoljuk ki a brokert, generáljunk hőmérséklet-üzeneteket, és ellenőrizzük, hogy az R5 szerinti puffer fogadja azokat, az R6 szerinti szabály pedig a talajnedvesség-csökkenésre helyben reagál.

Hibatünet	Lehetséges ok	Javítás
Offline tesztben nincs pufferelt üzenet	A gateway csak memóriában tárol, újraindításkor elvesznek az adatok	R5 szerint lemezen tárolni, indításkor visszatölteni
A helyi riasztás csak a felhő visszatérése után fut le	A szabálylogika a backendben van	R6 szerint a gatewayen is futtatni
A dashboardon ismeretlen topic jelenik meg	A normalizálás nem a TerraSense topic-konvencióját követi	R1–R2 szerint egységes site/node és topicnevek
Ugyanaz a mérés kétszer érkezik meg	Hiányzik a `(site, node_id, boot_id, seq)` alapú duplikátumszűrés	R4 szerint ellenőrizni az eseménykulcs monotonitását

12.1.6 Tipikus hibák és félreértések

1. **A gateway csak egy „buta” protokollfordító.** Ha a gateway pufferelés és helyi szabályok nélkül csak átalakítja az üzeneteket, akkor a rendszer a felhő kiesésekor elveszíti az adatait, és a kritikus vezérlés is megszakadhat. A gateway akkor értékes, ha offline is hasznos marad.
2. **A biztonsági határt az eszközre vagy a felhőbe helyezzük.** Ha minden `ts-node` közvetlen internetelérést kap, akkor minden eszköz támadási felület lesz, és a TLS-kulcsokat is minden eszközön kezelni kell. A gatewayre helyezett határ csökkenti a támadási felületet és a provisioning költségét.
3. **A QoS 1 önmagában elég az offline működéshez.** A QoS 1 azt garantálja, hogy a broker és a kliens között legalább egyszer megérkezik az üzenet, de ha a gateway nem tárolja tartósan az üzenetet, akkor a broker kiesése alatt az adat még a gatewayen belül elveszhet.

12.1.7 Feladatok

(a) Sorolj fel három olyan TerraSense-helyzetet, amelyben a gatewaynek akkor is döntenie kell, ha a felhő nem elérhető.

(b) Számítsd ki, hogy mennyi MQTT `PUBLISH` üzenetet takarít meg a gateway egy óra alatt, ha 20 `ts-node` csomópont 5 másodpercenként mér, a gateway pedig percenként egy aggregált üzenetet küld minden csomópontról.

(c) Készítsd el a `~/iot-labor/i035/gateway-feladatlista.md` fájlt a fenti nyolc követelménnyel, majd egészítsd ki az R8 követelményt egy új sorral: a gateway naplózza a store-and-forward puffer legutóbbi sikeres visszatöltésének UTC időpontját.

(d) Egy offline teszt után a gateway újraindul, majd ugyanazokat az üzeneteket másodszor is elküldi az MQTT brokernak. Melyik követelményt kell módosítani vagy kiegészíteni ahhoz, hogy ez ne okozzon duplikátumot a backendben? Mit írnál a feladatlistába?

12.1.8 Megoldások

(a) Három lehetséges helyzet:

1. A talajnedvesség 20 % alá csökken, ezért a gatewaynek helyben kell nyitnia az öntözőszelepet, még akkor is, ha a felhő-parancs nem érkezik meg.
2. Az üvegház hőmérséklete 40 °C fölé emelkedik, ezért a gatewaynek blokkolnia kell az öntözést és riasztást kell küldenie.
3. Az MQTT-kapcsolat megszakad, ezért a gatewaynek a mérési üzeneteket lemezen kell pufferelnie, amíg a kapcsolat vissza nem tér.

(b) A nyers üzenetszám:

$$N_{\text{raw}} = 20 \cdot \frac{3600}{5} = 14400 \text{ msg/h}$$

Az aggregált üzenetszám, ha minden csomópontról percenként egy üzenet megy:

$$N_{\text{agg}} = 20 \cdot 60 = 1200 \text{ msg/h}$$

A megtakarítás:

$$\eta_{\text{save}} = 1 - \frac{N_{\text{agg}}}{N_{\text{raw}}} = 1 - \frac{1200}{14400} \approx 0{,}917$$

Tehát a gateway körülbelül 91,7 %-kal csökkenti az MQTT `PUBLISH` üzenetek számát ebben a példában.

(c) A fájl létrehozása után az R8 alá beillesztendő sor:

```
R8 Üzemeltetés: puffer mélysége, uptime, MQTT kapcsolat állapota és
   naplózás a gateway egészségállapotához.
   Utolsó sikeres puffervisszatöltés: UTC epoch_ms időbélyeggel,
   hibás visszatöltés esetén a hiba okával együtt.
```

(d) Az R5 követelményt kell kiegészíteni a kézbesítési állapotok és az eseménykulcs pontos leírásával. A módosított sor például:

```
R5 Store-and-forward: offline puffer lemezen, FIFO sorrend. A gateway
   külön tartja nyilván a spooled, broker_acked (QoS 1 PUBACK) és
   backend_committed állapotot; a PUBACK önmagában nem jelenti, hogy a
   backend feldolgozta az üzenetet. Minden pufferelt üzenet megtartja az
   eredeti site/node_id/boot_id/seq/ts ötöst. Az újraküldés és a helyi
   duplikátumszűrés kulcsa (site, node_id, boot_id, seq); a backend
   ugyanezt kapja idempotencia-kulcsként, és külön tranzakcióban dönti el,
   hogy az esemény már feldolgozott-e. Újraindulás után a puffer és a
   backend ezt a kulcsot használja; a boot_id nélküli örökölt üzenet
   kulcsa (site, node_id, 0, seq), és ezt a gateway figyelmeztetéssel
   naplózza.
```

Így egy node újraindulása után az új `boot_id`-dal érkező `seq=1` új esemény, míg ugyanazon boot ugyanazon `seq` értékének replayje nem hoz létre új üzleti hatást.

12.1.9 Rövid összefoglaló

Az alfejezet definiálta az IoT gatewayt mint alkalmazási rétegbeli protokoll- és adatmodell-határt, és bemutatta a TerraSense `ts-gw` komponens feladatait: protokollfordítás BLE/UDP-ről MQTT-re, normalizálás, aggregálás, szűrés, store-and-forward, helyi riasztás, biztonsági határ és üzemeltetés. A rétegábra rögzítette a gateway belső felelősségi köreit. A döntési táblázat elválasztotta az eszközön, a gatewayen és a felhőben futtatandó funkciókat. **Továbbvitt eredmény:** a `~/iot-labor/i035/gateway-feladatlista.md` R1–R8 követelményei, amelyeket az A ts-gw szolgáltatás C++-ban: protokollfordítás MQTT-re, a Store-and-forward: offline működés és visszatöltés és a Helyi feldolgozás: szűrés, aggregálás, eseménydetektálás alfejezetek használnak tovább.

12.1.10 Források és ellenőrzési hiányok

Az alfejezet a gateway fogalmát és a TerraSense-feladatlistát általános, stabil rendszertervezési elvekből vezeti le. A fejezethez tartozó kiindulópontok a systemd, a Docker, a Paho MQTT és a Mosquitto dokumentációi, azonban ezeknek a konkrét CLI-kapcsolóit, verzióit és API-it nem ellenőriztük ebben az alfejezetben. A `ts-gw` tényleges C++ implementációjához a Paho C++ kliens és a Mosquitto aktuális verziójának dokumentációját kell ellenőrizni; a laborban rögzített verzió Paho C++ 1.3.2. A TerraSense telemetria-sémájának és topicjainak részleteit a Adatformátumok: JSON, CBOR, Protocol Buffers, SenML alfejezet rögzíti; az itt megadott `30 s` aggregálási ablak a mintarendszer kiinduló, később méréssel finomítható értéke.

12.2 A ts-gw szolgáltatás C++-ban: protokollfordítás MQTT-re

12.2.1 Motiváció

A TerraSense architektúrájában a `ts-node` szenzorcsomópont önmagában nem mindig képes közvetlenül MQTT-t küldeni a brokernek. Valós hardveren gyakran BLE GATT-értesítéssel adja át az adatot, a szimulációs laborokban pedig egy egyszerű UDP-datagrammal. A `ts-gw` az a Linux-gateway, amely ezt a heterogén eszközoldali forgalmat egységes MQTT-üzenetté alakítja a felhő felé. Ez a protokollfordítás nem csupán szintaktikai konverzió: a gateway ellenőrzi és normalizálja a mezőneveket, kezeli az eszköz- és gateway-időbélyeg különbségét, elvégzi az alapvető duplikátum- és sorrendellenőrzést, majd a i028-ban már bevezetett `terrasense/<site>/<node>/telemetry` topicra publikál. Az alábbiakban egy adapter-alapú, RAII-t használó C++17 gatewayt építünk fel, amelyet a későbbi alfejezetek a store-and-forward, az üzemeltetés, a helyi feldolgozás, a biztonság és a záróprojekt során bővítenek.

A fejezet célja, hogy a bemásolt projekt ne egy hiányos váz legyen: minden hivatkozott `.cpp` fájl teljes forrással szerepel, a CMake-konfiguráció a ténylegesen használt Paho C++ 1.3.2 könyvtárat linkeli, és a UDP-adapter egyszerre több site/node forgalmát képes fogadni. A hálózati és brokeres végigfuttatás a labor feladata; a fejezet a statikusan ellenőrizhető részeket külön jelöli.

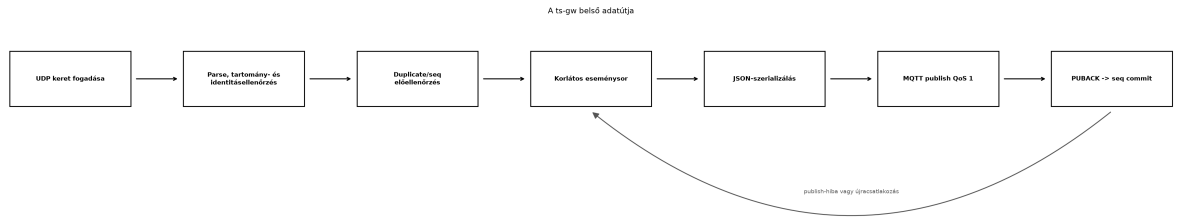
12.2.2 Fogalmak és működés

Definíció (input adapter). Az input adapter a gateway bemeneti protokollját elszigetelő, lecserélhető programkomponens. Feladata, hogy egy adott forrásból (UDP, BLE GATT, soros port) nyers kereteket fogadjon, és az egységes belépési pontnak átadja. A többi komponens csak az adapter által kikényszerített absztrakciót ismeri, így a protokollcsere nem érinti a normalizálót vagy az MQTT-publikálót. Ebben a fejezetben az adapter még nyers `std::string` keretet ad át, a normalizálás a fő szál tulajdona; így az adapter nem függ a telemetria sémájától.

Definíció (normalizált telemetria-minta). A gateway belső eseménye egy olyan, a nyers protokolltól független adatstruktúra, amely már a TerraSense telemetria-sémát tükrözi: `site`, `node_id`, `boot_id`, monoton `seq`, UTC eseményidő epoch ms-ban, az időbélyeg forrása és minőségjelzője, valamint a mért értékek. A mérési mezők `std::optional<double>` típusúak: a hiányzó érték JSON `null`, nem nulla és nem NaN.

Definíció (korlátos belső eseménysor). A belső eseménysor egy szálbiztos FIFO puffer, amely az egyik szálon érkező normalizált mintákat átadja a publikáló szálnak. A sornak explicit kapacitása és eldobási politikája van: teljes sor esetén a legújabb minta kimarad, a gateway pedig számlálót és naplóbejegyzést készít. A memória így nem nő korlátlanul. Ez a fejezet átmeneti puffere; a tartós, újraindulást is túlélő puffer a store-and-forward alfejezetben kerül a helyére.

A `ts-gw` belső adatútja az alábbi lépésekből áll:



12.2. ábra: A ts-gw belső adatútja - Az UDP-adaptertől az MQTT-publikálásig tartó lánc. A seq állapot csak sikeres PUBACK után commitálódik; hiba esetén a minta visszakerül a korlátos sor elejére.

A folyamatot a következő komponensek valósítják meg:

Komponens	Felelősség	Megjegyzés
`InputAdapter`	absztrakt bemeneti interfész	a `UdpInputAdapter` ebből származik
`UdpInputAdapter`	UDP-socket, `poll`, fogadó szál	200 ms-os poll-timeout, tiszta `stop()`
`Normalizer`	nyers szöveg -> `TelemetrySample`	mezők, tartományok, időbélyeg, `seq`
`Json`	ellenőrzött JSON-író	escape, véges számok, `null`
`EventQueue<T>`	korlátos, szálbiztos sor	kapacitás, drop-számláló, visszahelyezés
`MqttPublisher`	MQTT-kapcsolat és publikálás	Paho C++ 1.3.2, QoS 1, PUBACK
`Config`	alapérték, TOML, env, CLI	egy helyen, determinisztikus prioritás
`main`	életciklus, SIGINT, újracsatlakozás	adapter indítása, fő eseményhurok

A szálak viszonya a következő. Az UDP-adapter saját `std::thread`-ben fut, `poll()`-lal legfeljebb 200 ms-ot vár, majd `recvfrom()`-mal olvassa a datagramot. A nyers keretet a `FrameHandler` adja át a fő szálnak, amely a `Normalizer`-rel feldolgozza, majd a korlátos sorba teszi. A fő szál `wait_pop_for()` hívással várakozik, szerializál, publikál, és csak a sikeres PUBACK után commitálja a `seq` állapotot. A hálózati küldés idejére az UDP-fogadó nem blokkolódik. A `stop()` nem zárja be a socketet a fogadó szál alatt: előbb `running_ = false`, majd `join()`, végül `close()`. Ez elkerüli azt a versenyhelyzetet, amikor egy másik szál egy már lezárt fájlleírót használ.

A C++-os RAII itt nem szlogen, hanem a gateway-hibakezelés alapja. A socket lezárása a `UdpInputAdapter` destruktorában, a MQTT-kliens felszabadítása a `MqttPublisher` destruktorában, a szál joinolása a `stop()` metódusban történik. A `SIGINT` és `SIGTERM` csak egy atomi jelzőt állít; a tényleges erőforrás-felszabadítás a normál kilépési úton fut le.

A laborban UDP-adaptert használunk, mert a `native_sim` ts-node szimulátorból a legegyszerűbben UDP-datagramot küldhetünk. Valós hardveren a `BleInputAdapter` ugyanezt az interfészt implementálhatja a BlueZ D-Bus API-jával: a GATT-értesítések adatait kell nyers keretté alakítani. A D-Bus-kód bonyolultsága miatt itt csak a vázat és a döntési szempontot tárgyaljuk; a labor a szimulált UDP-adapterre épül.

12.2.3 Döntési szempontok

Az alábbi táblázat a gateway implementációjának fő dilemmáit foglalja össze.

Kérdés	Választás a laborban	Indoklás / alternatíva
Szálak vagy async I/O	`std::thread` + `poll` + blokkoló Paho-hívás	egyszerű, átlátható; nagy forgalomnál Boost.Asio vagy `epoll`
Paho C vagy C++ kliens	Paho C++ 1.3.2, `<mqtt/client.h>`	típusos RAII, a QoS 1 tokenje visszaadja a PUBACK eredményét; a C API külön kézi életciklust igényelne
JSON-előállítás	saját, ellenőrzött writer	nincs plusz függőség; az escape és a véges szám ellenőrzött. Nagy sémánál `nlohmann/json` is választható
UDP vagy BLE	UDP a laborban	hardver nélkül BLE-t nem tudunk futtatni; az absztrakció lehetővé teszi a cserét
Sor mérete	1024 elem, drop-newest + számláló	éles rendszerben a tartós spool veszi át a szerepét; a korlátlan sor memóriaszivárgás
Hibás device idő	`gateway_estimate` + `estimated_time` jelző	nem hazudunk device-időt; a backend az időbélyeg forrását is látja
Újracsatlakozás	exponenciális backoff 1-8 s	a gateway hosszú életű; a broker kiesése nem állítja le az UDP-fogadást

A konfiguráció prioritása rögzített: beépített alapérték < TOML-fájl < `TS_GW_*` környezeti változó < parancssori kapcsoló. Ugyanezt a sorrendet használja a systemd-telepítés és a konténer is, ezért a `--config` fájl, a környezeti változó és a CLI nem egymást felülíró, rejtett mechanizmusok. A TOML itt szándékosan csak lapos `kulcs = érték` részhalmaz; a szekciós TOML-t a teljes parser nélkül nem támogatjuk, és az ismeretlen kulcsot hibaüzenettel utasítjuk el.

12.2.4 Kidolgozott példa: a teljes ts-gw projekt

Munkakönyvtár és könyvtárstruktúra:

```
mkdir -p ~/iot-labor/i036/ts-gw/{src,scripts,config}
cd ~/iot-labor/i036/ts-gw
```

```
ts-gw/
  CMakeLists.txt
  config/ts-gw.toml
  scripts/send_udp.py
  src/
      Config.h, Config.cpp
      EventQueue.h
      InputAdapter.h
      Json.h
```

```
MqttPublisher.h, MqttPublisher.cpp
Normalizer.h, Normalizer.cpp
TelemetryTypes.h
UdpInputAdapter.h, UdpInputAdapter.cpp
main.cpp
```

A labor előfeltétele, hogy a MQTT a gyakorlatban: Zephyr-kliens és Mosquitto alfejezetben bevezetett `lab-broker` Mosquitto fusson a `~/iot-labor/stack/compose.yaml` alapján, és a 1883-as porton hallgasson. A projekt a rögzített laborprofil szerinti Paho C++ 1.3.2-t használja. Ubuntu 24.04 alatt a fejlesztői csomag neve `libpaho-mqttpp3-dev`; a `pkg-config --modversion paho-mqttpp3` parancsnak a rögzített verziót kell kiírnia. Ha a telepített csomag eltér, a CMake konfigurálása a verzióellenőrzésnél álljon meg, ne csendben linkeljen más API-t.

A konfigurációs fájl:

```
# TerraSense ts-gw - lapos TOML részhalmaz
broker_uri = "tcp://localhost:1883"
client_id = "ts-gw"
udp_bind = "0.0.0.0"
udp_port = 4000
queue_capacity = 1024
qos = 1
log_level = "info"
```

A belső adattípus a `TelemetrySample`:

```
#pragma once
#include <cstdint>
#include <optional>
#include <string>

namespace terrasense {

// A gateway belso, protokollfuggetlen esemenye. A meresi mezok optionalisak:
// a kulonbozo szenzorvaltozatok nem feltetlenul mindharom erteket kuldik.
struct TelemetrySample {
  int schema_version{1};
  std::string site;
  std::string node_id;
  uint32_t boot_id{0};
  uint32_t seq{0};
  int64_t ts_epoch_ms{0};
  std::string ts_source{"device"};
  std::string quality{"good"};
  std::optional<double> temperature_c;
  std::optional<double> humidity_rh;
  std::optional<double> soil_moisture_pct;
};

inline bool has_any_measurement(const TelemetrySample& s) {
  return s.temperature_c.has_value() || s.humidity_rh.has_value() ||
         s.soil_moisture_pct.has_value();
}

} // namespace terrasense
```

Az input adapter absztrakciója nyers keretet ad át:

```
#pragma once
#include <functional>
#include <string>
```

```
namespace terrasense {

class InputAdapter {
public:
  using FrameHandler = std::function<void(std::string)>;
  virtual ~InputAdapter() = default;
  virtual bool start(FrameHandler handler, std::string& err) = 0;
  virtual void stop() = 0;
};

} // namespace terrasense
```

A `UdpInputAdapter` a `start()`-ban nyitja a socketet, a `run()` a háttérszálban `poll()`-lal vár, a `stop()` pedig előbb leállítja és joinolja a szálat, majd lezárja a socketet. A datagram maximum 2048 bájt; a ennél hosszabb keretet eldobja és naplózza. A socket `SOCK_CLOEXEC`, hogy egy későbbi `exec` ne örökölje.

```
#pragma once
#include "InputAdapter.h"
#include <atomic>
#include <cstdint>
#include <string>
#include <thread>

namespace terrasense {

class UdpInputAdapter final : public InputAdapter {
public:
  UdpInputAdapter(std::string bind_ip, uint16_t port);
  ~UdpInputAdapter() override;
  bool start(FrameHandler handler, std::string& err) override;
  void stop() override;
  std::uint64_t loop_ticks() const { return loop_ticks_.load(); }

private:
  void run();

  int fd_{-1};
  std::string bind_ip_;
  uint16_t port_;
  std::thread worker_;
  std::atomic<bool> running_{false};
  std::atomic<std::uint64_t> loop_ticks_{0};
  FrameHandler handler_;
};

} // namespace terrasense
```

```
#include "UdpInputAdapter.h"
#include <arpa/inet.h>
#include <cerrno>
#include <cstring>
#include <iostream>
#include <netinet/in.h>
#include <poll.h>
#include <sys/socket.h>
#include <unistd.h>
#include <utility>

namespace terrasense {

UdpInputAdapter::UdpInputAdapter(std::string bind_ip, uint16_t port)
    : bind_ip_(std::move(bind_ip)), port_(port) {}
```

```
UdpInputAdapter::~UdpInputAdapter() { stop(); }

bool UdpInputAdapter::start(FrameHandler handler, std::string& err) {
  if (running_.load()) {
    err = "UDP adapter already running";
    return false;
  }
  fd_ = ::socket(AF_INET, SOCK_DGRAM | SOCK_CLOEXEC, 0);
  if (fd_ < 0) {
    err = std::string("socket: ") + std::strerror(errno);
    return false;
  }
  int yes = 1;
  if (::setsockopt(fd_, SOL_SOCKET, SO_REUSEADDR, &yes, sizeof(yes)) < 0) {
    err = std::string("setsockopt: ") + std::strerror(errno);
    ::close(fd_);
    fd_ = -1;
    return false;
  }
  sockaddr_in address{};
  address.sin_family = AF_INET;
  address.sin_port = htons(port_);
  if (::inet_pton(AF_INET, bind_ip_.c_str(), &address.sin_addr) != 1) {
    err = "invalid IPv4 bind address: " + bind_ip_;
    ::close(fd_);
    fd_ = -1;
    return false;
  }
  if (::bind(fd_, reinterpret_cast<sockaddr*>(&address), sizeof(address)) < 0) {
    err = "bind " + bind_ip_ + ":" + std::to_string(port_) + ": " +
          std::strerror(errno);
    ::close(fd_);
    fd_ = -1;
    return false;
  }
  handler_ = std::move(handler);
  running_.store(true);
  worker_ = std::thread(&UdpInputAdapter::run, this);
  return true;
}

void UdpInputAdapter::stop() {
  running_.store(false);
  if (worker_.joinable()) {
    worker_.join();
  }
  if (fd_ >= 0) {
    ::close(fd_);
    fd_ = -1;
  }
}

void UdpInputAdapter::run() {
  char buffer[2048];
  while (running_.load()) {
    pollfd descriptor{fd_, POLLIN, 0};
    const int ready = ::poll(&descriptor, 1, 200);
    loop_ticks_.fetch_add(1);
    if (ready < 0) {
      if (errno == EINTR) {
        continue;
      }
```

```
      std::cerr << "[ERROR] UDP poll: " << std::strerror(errno) << "\n";
      break;
    }
    if (ready == 0) {
      continue;
    }
    sockaddr_storage peer{};
    socklen_t peer_length = sizeof(peer);
    const ssize_t received =
        ::recvfrom(fd_, buffer, sizeof(buffer), 0,
                   reinterpret_cast<sockaddr*>(&peer), &peer_length);
    if (received < 0) {
      if (errno == EINTR) {
        continue;
      }
      std::cerr << "[ERROR] UDP recvfrom: " << std::strerror(errno) << "\n";
      break;
    }
    if (received == static_cast<ssize_t>(sizeof(buffer))) {
      std::cerr << "[WARN] oversized UDP datagram discarded\n";
      continue;
    }
    if (handler_) {
      handler_(std::string(buffer, static_cast<std::size_t>(received)));
    }
  }
}

} // namespace terrasense
```

A `Normalizer` a `site=gh1;node=n001;boot=7;seq=42;ts=...;T=23.4;RH=61.2;SM=34` formátumú keretet alakítja `TelemetrySample`-lé. A számokat `std::from_chars` konvertálja, így a tizedesjel nem függ a locale-tól. Az azonosítók `[A-Za-z0-9][A-Za-z0-9_-]{0,31}` alakúak; az idézőjelet vagy pontosvesszőt tartalmazó azonosítót a gateway elutasítja, nem próbálja „megtisztítani”. A hiányzó vagy érvénytelen `ts`, illetve a `clock=0` esetén a gateway `gateway_estimate` időbélyeget és `estimated_time` minőségjelzőt állít be. A mérési tartományok: hőmérséklet -40..85 °C, páratartalom 0..100 %RH, talajnedvesség 0..100 térfogati %. A `seq` állapot kulcsa a `(site, node_id, boot_id)` hármas; a commit csak a sikeres publikálás után történik.

```
#pragma once
#include "TelemetryTypes.h"
#include <cstdint>
#include <map>
#include <mutex>
#include <string>

namespace terrasense {

class Normalizer {
public:
  bool parse(const std::string& raw, TelemetrySample& out,
             std::string& err) const;
  bool is_duplicate(const TelemetrySample& sample, std::string& err) const;
  void commit_seq(const TelemetrySample& sample);
  void reset();

private:
  struct SeqState {
    uint32_t last_seq{0};
    bool have_last{false};
  };
  static std::string key_for(const TelemetrySample& sample);
```

```
  mutable std::mutex mutex_;
  std::map<std::string, SeqState> seq_state_;
};

} // namespace terrasense

#include "Normalizer.h"
#include <charconv>
#include <chrono>
#include <cmath>
#include <cstddef>
#include <string_view>
#include <vector>

namespace terrasense {
namespace {

std::string trim(std::string_view value) {
  const auto first = value.find_first_not_of(" \t\r\n");
  if (first == std::string_view::npos) {
    return {};
  }
  const auto last = value.find_last_not_of(" \t\r\n");
  return std::string(value.substr(first, last - first + 1));
}

bool valid_identifier(std::string_view value) {
  if (value.empty() || value.size() > 32) {
    return false;
  }
  for (std::size_t i = 0; i < value.size(); ++i) {
    const char c = value[i];
    const bool alnum = (c >= 'a' && c <= 'z') || (c >= 'A' && c <= 'Z') ||
                       (c >= '0' && c <= '9');
    if (i == 0 && !alnum) {
      return false;
    }
    if (!alnum && c != '_' && c != '-') {
      return false;
    }
  }
  return true;
}

template <typename T>
bool parse_integer(std::string_view text, T& value) {
  if (text.empty()) {
    return false;
  }
  T result = 0;
  const auto parsed = std::from_chars(text.data(), text.data() + text.size(),
                                      result);
  if (parsed.ec != std::errc{} || parsed.ptr != text.data() + text.size()) {
    return false;
  }
  value = result;
  return true;
}

bool parse_number(std::string_view text, double& value) {
  if (text.empty()) {
    return false;
  }
  double result = 0.0;
```

```
  const auto parsed = std::from_chars(text.data(), text.data() + text.size(),
                                      result);
  if (parsed.ec != std::errc{} || parsed.ptr != text.data() + text.size() ||
      !std::isfinite(result)) {
    return false;
  }
  value = result;
  return true;
}

int64_t now_ms() {
  using namespace std::chrono;
  return duration_cast<milliseconds>(system_clock::now().time_since_epoch())
      .count();
}

// A 2000-01-01 es 2100-01-01 kozotti idobelyeget tekintjuk ertelmesnek.
constexpr int64_t kMinEpochMs = 946684800000LL;
constexpr int64_t kMaxEpochMs = 4102444800000LL;

bool valid_metric_range(const char* name, double value, double low,
                        double high, std::string& err) {
  if (value < low || value > high) {
    err = std::string(name) + " out of range [" +
          std::to_string(static_cast<int>(low)) + ", " +
          std::to_string(static_cast<int>(high)) + "]";
    return false;
  }
  return true;
}

} // namespace

bool Normalizer::parse(const std::string& raw, TelemetrySample& out,
                       std::string& err) const {
  out = TelemetrySample{};
  bool have_site = false;
  bool have_node = false;
  bool have_seq = false;
  bool have_metric = false;
  bool clock_unsynchronized = false;
  bool have_device_ts = false;
  int64_t device_ts = 0;

  std::size_t begin = 0;
  while (begin <= raw.size()) {
    const auto end = raw.find(';', begin);
    const std::string_view field(
        raw.data() + begin,
        (end == std::string::npos ? raw.size() : end) - begin);
    const std::string token = trim(field);
    if (!token.empty()) {
      const auto equal = token.find('=');
      if (equal == std::string::npos) {
        err = "field without '=': " + token;
        return false;
      }
      const std::string key = trim(std::string_view(token).substr(0, equal));
      const std::string value = trim(std::string_view(token).substr(equal + 1));
      if (key == "site") {
        if (!valid_identifier(value)) {
          err = "invalid site identifier";
          return false;
```

```
  }
  out.site = value;
  have_site = true;
} else if (key == "node") {
  if (!valid_identifier(value)) {
    err = "invalid node identifier";
    return false;
  }
  out.node_id = value;
  have_node = true;
} else if (key == "boot") {
  if (!parse_integer(value, out.boot_id)) {
    err = "invalid boot id";
    return false;
  }
} else if (key == "seq") {
  if (!parse_integer(value, out.seq)) {
    err = "invalid seq";
    return false;
  }
  have_seq = true;
} else if (key == "ts") {
  if (!parse_integer(value, device_ts) || device_ts < kMinEpochMs ||
      device_ts >= kMaxEpochMs) {
    have_device_ts = false;
  } else {
    have_device_ts = true;
  }
} else if (key == "clock") {
  clock_unsynchronized = (value == "0");
} else if (key == "schema") {
  if (!parse_integer(value, out.schema_version) ||
      out.schema_version < 1 || out.schema_version > 100) {
    err = "invalid schema version";
    return false;
  }
} else if (key == "T") {
  double v = 0.0;
  if (!parse_number(value, v)) {
    err = "invalid temperature";
    return false;
  }
  if (!valid_metric_range("temperature_c", v, -40.0, 85.0, err)) {
    return false;
  }
  out.temperature_c = v;
  have_metric = true;
} else if (key == "RH") {
  double v = 0.0;
  if (!parse_number(value, v)) {
    err = "invalid humidity";
    return false;
  }
  if (!valid_metric_range("humidity_rh", v, 0.0, 100.0, err)) {
    return false;
  }
  out.humidity_rh = v;
  have_metric = true;
} else if (key == "SM") {
  double v = 0.0;
  if (!parse_number(value, v)) {
    err = "invalid soil moisture";
    return false;
```

```
        }
        if (!valid_metric_range("soil_moisture_pct", v, 0.0, 100.0, err)) {
          return false;
        }
        out.soil_moisture_pct = v;
        have_metric = true;
      } else {
        err = "unknown field: " + key;
        return false;
      }
    }
    if (end == std::string::npos) {
      break;
    }
    begin = end + 1;
  }

  if (!have_site || !have_node || !have_seq || !have_metric) {
    err = "missing required field (site, node, seq or measurement)";
    return false;
  }
  if (clock_unsynchronized || !have_device_ts) {
    out.ts_epoch_ms = now_ms();
    out.ts_source = "gateway_estimate";
    out.quality = "estimated_time";
  } else {
    out.ts_epoch_ms = device_ts;
    out.ts_source = "device";
    out.quality = "good";
  }
  return true;
}

std::string Normalizer::key_for(const TelemetrySample& sample) {
  return sample.site + "\x1f" + sample.node_id + "\x1f" +
         std::to_string(sample.boot_id);
}

bool Normalizer::is_duplicate(const TelemetrySample& sample,
                              std::string& err) const {
  std::lock_guard<std::mutex> lock(mutex_);
  const auto it = seq_state_.find(key_for(sample));
  if (it == seq_state_.end() || !it->second.have_last) {
    return false;
  }
  if (sample.seq <= it->second.last_seq) {
    err = "seq " + std::to_string(sample.seq) + " <= last committed " +
          std::to_string(it->second.last_seq);
    return true;
  }
  return false;
}

void Normalizer::commit_seq(const TelemetrySample& sample) {
  std::lock_guard<std::mutex> lock(mutex_);
  auto& state = seq_state_[key_for(sample)];
  if (!state.have_last || sample.seq > state.last_seq) {
    state.last_seq = sample.seq;
    state.have_last = true;
  }
}

void Normalizer::reset() {
```

```
  std::lock_guard<std::mutex> lock(mutex_);
  seq_state_.clear();
}

} // namespace terrasense
```

A JSON-író nem egyszerű string-összefűzés: a string mezőket escape-eli, a számokat csak akkor írja ki, ha végesek, a hiányzó mérést pedig `null`-ként jeleníti meg. Így egy idézőjelet tartalmazó naplóüzenet vagy egy NaN nem törli el a JSON szerkezetét. A `to_json()` hibát ad vissza, ha valamelyik szám nem véges; a hívó ilyenkor a mintát eldobja és naplóz.

```
#pragma once
#include "TelemetryTypes.h"
#include <charconv>
#include <cmath>
#include <cstdio>
#include <optional>
#include <string>
#include <string_view>
#include <system_error>

namespace terrasense {

// Idezojelet, visszaleptet es vezérlőkaraktert is biztonságosan kezelő
// JSON-string writer. Nem teljes JSON-konyvtar: a gateway sajat, fix semajahoz
// elegendo, de a string hatarolo karaktereket nem hagyja kiszokni.
inline void json_append_escaped(std::string& out, std::string_view text) {
  out.push_back('"');
  for (const unsigned char c : text) {
    switch (c) {
      case '"': out += "\\\""; break;
      case '\\': out += "\\\\"; break;
      case '\b': out += "\\b"; break;
      case '\f': out += "\\f"; break;
      case '\n': out += "\\n"; break;
      case '\r': out += "\\r"; break;
      case '\t': out += "\\t"; break;
      default:
        if (c < 0x20) {
          char buf[7];
          std::snprintf(buf, sizeof(buf), "\\u%04x", static_cast<unsigned>(c));
          out += buf;
        } else {
          out.push_back(static_cast<char>(c));
        }
    }
  }
  out.push_back('"');
}

inline bool json_append_number(std::string& out, double value) {
  if (!std::isfinite(value)) {
    return false;
  }
  char buf[32];
  const auto result = std::to_chars(buf, buf + sizeof(buf), value,
                                    std::chars_format::general);
  if (result.ec != std::errc{}) {
    return false;
  }
  out.append(buf, result.ptr);
  return true;
}
```

```
inline bool to_json(const TelemetrySample& sample, std::string& out,
                    std::string& err) {
  out.clear();
  out.reserve(256);
  out += "{\"schema_version\":";
  out += std::to_string(sample.schema_version);
  out += ",\"site\":";
  json_append_escaped(out, sample.site);
  out += ",\"node_id\":";
  json_append_escaped(out, sample.node_id);
  out += ",\"boot_id\":";
  out += std::to_string(sample.boot_id);
  out += ",\"seq\":";
  out += std::to_string(sample.seq);
  out += ",\"ts\":";
  out += std::to_string(sample.ts_epoch_ms);
  out += ",\"ts_source\":";
  json_append_escaped(out, sample.ts_source);
  out += ",\"quality\":";
  json_append_escaped(out, sample.quality);

  const auto append_metric = [&](const char* name,
                                 const std::optional<double>& value) {
    out += ",\"";
    out += name;
    out += "\":";
    if (!value.has_value()) {
      out += "null";
      return true;
    }
    if (!json_append_number(out, *value)) {
      err = std::string("non-finite metric: ") + name;
      return false;
    }
    return true;
  };
  if (!append_metric("temperature_c", sample.temperature_c) ||
      !append_metric("humidity_rh", sample.humidity_rh) ||
      !append_metric("soil_moisture_pct", sample.soil_moisture_pct)) {
    out.clear();
    return false;
  }
  out += "}";
  return true;
}

} // namespace terrasense
```

A korlátos eseménysor:

```
#pragma once
#include <chrono>
#include <condition_variable>
#include <cstddef>
#include <cstdint>
#include <deque>
#include <mutex>

namespace terrasense {

// Korlatos, szalbiztos FIFO. A producer nem blokkol: ha a sor megtelt, az uj
// elemet eldobja es szamolja. A sikertelen publish utani visszahelyezes a sor
```

```
// elejere kerul; ehhez szukseg eseten a legfiatalabb elem esik ki, hogy a mar
// megkezdett, regi mintat ne veszitsuk el.
template <typename T>
class EventQueue {
public:
  explicit EventQueue(std::size_t capacity) : capacity_(capacity) {}

  bool try_push(const T& item) {
    std::lock_guard<std::mutex> lock(mutex_);
    if (closed_ || queue_.size() >= capacity_) {
      ++dropped_;
      return false;
    }
    queue_.push_back(item);
    cv_.notify_one();
    return true;
  }

  void requeue_front(const T& item) {
    std::lock_guard<std::mutex> lock(mutex_);
    if (closed_) {
      return;
    }
    if (queue_.size() >= capacity_) {
      queue_.pop_back();
      ++dropped_;
    }
    queue_.push_front(item);
    cv_.notify_one();
  }

  bool wait_pop_for(T& item, std::chrono::milliseconds timeout) {
    std::unique_lock<std::mutex> lock(mutex_);
    if (!cv_.wait_for(lock, timeout,
                      [this] { return closed_ || !queue_.empty(); })) {
      return false;
    }
    if (queue_.empty()) {
      return false;
    }
    item = queue_.front();
    queue_.pop_front();
    return true;
  }

  void close() {
    std::lock_guard<std::mutex> lock(mutex_);
    closed_ = true;
    cv_.notify_all();
  }

  std::size_t size() const {
    std::lock_guard<std::mutex> lock(mutex_);
    return queue_.size();
  }

  std::uint64_t dropped() const {
    std::lock_guard<std::mutex> lock(mutex_);
    return dropped_;
  }

private:
  mutable std::mutex mutex_;
```

```
  std::condition_variable cv_;
  std::deque<T> queue_;
  std::size_t capacity_;
  std::uint64_t dropped_{0};
  bool closed_{false};
};

} // namespace terrasense
```

A `MqttPublisher` a Paho C++ szinkron kliense felett ad QoS 1 publikálást. A `publish()` nem csak azt nézi, hogy a hívás nem dobott kivételt: megvárja a tokent, és ellenőrzi a visszatérési kódot. QoS 1 esetén a token sikeres befejezése a broker PUBACK-je. A `connect()` és `publish()` 5, illetve 10 másodperces korlátot használ, hogy egy félig nyitott TCP-kapcsolat ne blokkolja örökre a fő szálat.

```
#pragma once
#include <mqtt/client.h>
#include <string>

namespace terrasense {

class MqttPublisher {
public:
  MqttPublisher(std::string broker_uri, std::string client_id);
  ~MqttPublisher();
  MqttPublisher(const MqttPublisher&) = delete;
  MqttPublisher& operator=(const MqttPublisher&) = delete;

  bool connect(std::string& err);
  void disconnect();
  bool publish(const std::string& topic, const std::string& payload, int qos,
               std::string& err);
  bool is_connected() const;

private:
  mqtt::client client_;
  std::string broker_uri_;
  std::string client_id_;
};

} // namespace terrasense
```

```
#include "MqttPublisher.h"
#include <chrono>
#include <mqtt/connect_options.h>
#include <mqtt/exception.h>
#include <mqtt/message.h>
#include <mqtt/token.h>
#include <utility>

namespace terrasense {

MqttPublisher::MqttPublisher(std::string broker_uri, std::string client_id)
    : client_(broker_uri, client_id),
      broker_uri_(std::move(broker_uri)),
      client_id_(std::move(client_id)) {}

MqttPublisher::~MqttPublisher() { disconnect(); }

bool MqttPublisher::connect(std::string& err) {
  if (client_.is_connected()) {
    return true;
  }
```

```
  try {
    mqtt::connect_options options;
    options.set_clean_session(true);
    options.set_keep_alive_interval(20);
    options.set_connect_timeout(5);
    auto token = client_.connect(options);
    if (!token->wait_for_completion(std::chrono::seconds(5))) {
      err = "MQTT connect timeout";
      return false;
    }
    if (token->get_return_code() != 0) {
      err = "MQTT connect refused, rc=" +
            std::to_string(token->get_return_code());
      return false;
    }
    return client_.is_connected();
  } catch (const mqtt::exception& e) {
    err = std::string("MQTT connect: ") + e.what();
    return false;
  }
}

void MqttPublisher::disconnect() {
  if (!client_.is_connected()) {
    return;
  }
  try {
    auto token = client_.disconnect();
    if (token) {
      token->wait_for_completion(std::chrono::seconds(2));
    }
  } catch (const mqtt::exception&) {
    // A leallitas nem dobhat: a hivo mar a hibautet naplozza.
  }
}

bool MqttPublisher::publish(const std::string& topic,
                            const std::string& payload, int qos,
                            std::string& err) {
  if (!client_.is_connected()) {
    err = "MQTT not connected";
    return false;
  }
  try {
    auto message = mqtt::make_message(topic, payload);
    message->set_qos(qos);
    auto token = client_.publish(message);
    if (!token->wait_for_completion(std::chrono::seconds(10))) {
      err = "MQTT publish timeout (no PUBACK)";
      return false;
    }
    if (token->get_return_code() != 0) {
      err = "MQTT publish failed, rc=" +
            std::to_string(token->get_return_code());
      return false;
    }
    return true;
  } catch (const mqtt::exception& e) {
    err = std::string("MQTT publish: ") + e.what();
    return false;
  }
}
```

```
bool MqttPublisher::is_connected() const { return client_.is_connected(); }

} // namespace terrasense
```

A `Config` a négy konfigurációs forrást egy helyen, dokumentált sorrendben dolgozza fel. A TOML-parser lapos kulcs-érték párokat, teljes soros `#` kommentet és idézőjeles stringet ismer; az ismeretlen kulcsot hibaként jelzi. A környezeti változók nevei `TS_GW_MQTT_BROKER`, `TS_GW_CLIENT_ID`, `TS_GW_UDP_BIND`, `TS_GW_UDP_PORT`, `TS_GW_QUEUE_CAPACITY`, `TS_GW_QOS`, `TS_GW_LOG_LEVEL`, `TS_GW_SPOOL_PATH`, `TS_GW_MAX_SPOOL_RECORDS`. A `spool_path` és `max_spool_records` mezőt ebben a fejezetben még nem használjuk, de a store-and-forward és az üzemeltetés ugyanezt a konfigurációt viszi tovább.

```
#pragma once
#include <cstddef>
#include <cstdint>
#include <string>

namespace terrasense {

struct Config {
  std::string broker_uri{"tcp://localhost:1883"};
  std::string client_id{"ts-gw"};
  std::string udp_bind{"0.0.0.0"};
  uint16_t udp_port{4000};
  std::size_t queue_capacity{1024};
  int qos{1};
  std::string log_level{"info"};
  std::string spool_path{"./var/spool.db"};
  std::size_t max_spool_records{100000};
};

// A konfiguracio forrasa: beepitett alap < TOML-fajl < kornyezeti valtozo <
// parancssor. A fuggveny false-t ad vissza hiba eseten, a hiba szoveget az
// err tartalmazza. A --help eseten a help_requested igaz lesz.
bool load_config(int argc, char** argv, Config& cfg, bool& help_requested,
                 std::string& err);
void print_usage(const char* program);

} // namespace terrasense
```

```
#include "Config.h"
#include <cstdlib>
#include <fstream>
#include <iostream>
#include <limits>
#include <sstream>
#include <string_view>
#include <vector>

namespace terrasense {
namespace {

std::string trim(std::string_view value) {
  const auto first = value.find_first_not_of(" \t\r\n");
  if (first == std::string_view::npos) {
    return {};
  }
  const auto last = value.find_last_not_of(" \t\r\n");
  return std::string(value.substr(first, last - first + 1));
}

bool parse_unsigned(std::string_view text, std::size_t& value) {
```

```
  if (text.empty()) {
    return false;
  }
  std::size_t result = 0;
  for (const char c : text) {
    if (c < '0' || c > '9') {
      return false;
    }
    const auto digit = static_cast<std::size_t>(c - '0');
    if (result > (std::numeric_limits<std::size_t>::max() - digit) / 10) {
      return false;
    }
    result = result * 10 + digit;
  }
  value = result;
  return true;
}

bool parse_string(std::string_view text, std::string& value) {
  const std::string t = trim(text);
  if (t.size() < 2 || t.front() != '"' || t.back() != '"') {
    return false;
  }
  value = t.substr(1, t.size() - 2);
  return value.find('"') == std::string::npos;
}

bool apply_value(Config& cfg, const std::string& key, const std::string& raw,
                 std::string& err) {
  std::size_t number = 0;
  if (key == "broker_uri" || key == "client_id" || key == "udp_bind" ||
      key == "log_level" || key == "spool_path") {
    std::string value;
    if (!parse_string(raw, value)) {
      err = "string value expected for key " + key;
      return false;
    }
    if (key == "broker_uri") cfg.broker_uri = value;
    if (key == "client_id") cfg.client_id = value;
    if (key == "udp_bind") cfg.udp_bind = value;
    if (key == "log_level") cfg.log_level = value;
    if (key == "spool_path") cfg.spool_path = value;
    return true;
  }
  if (!parse_unsigned(trim(raw), number)) {
    err = "non-negative integer expected for key " + key;
    return false;
  }
  if (key == "udp_port") {
    if (number == 0 || number > 65535) {
      err = "udp_port out of range";
      return false;
    }
    cfg.udp_port = static_cast<uint16_t>(number);
  } else if (key == "queue_capacity") {
    if (number == 0) {
      err = "queue_capacity must be positive";
      return false;
    }
    cfg.queue_capacity = number;
  } else if (key == "qos") {
    if (number > 1) {
      err = "qos must be 0 or 1 in this gateway";
```

```
      return false;
    }
    cfg.qos = static_cast<int>(number);
  } else if (key == "max_spool_records") {
    if (number == 0) {
      err = "max_spool_records must be positive";
      return false;
    }
    cfg.max_spool_records = number;
  } else {
    err = "unknown configuration key: " + key;
    return false;
  }
  return true;
}

bool load_file(const std::string& path, Config& cfg, std::string& err) {
  std::ifstream input(path);
  if (!input) {
    err = "cannot open config file: " + path;
    return false;
  }
  std::string line;
  std::size_t line_no = 0;
  while (std::getline(input, line)) {
    ++line_no;
    const std::string t = trim(line);
    if (t.empty() || t.front() == '#') {
      continue;
    }
    const auto equal = t.find('=');
    if (equal == std::string::npos) {
      err = path + ":" + std::to_string(line_no) + ": key = value expected";
      return false;
    }
    const std::string key = trim(t.substr(0, equal));
    std::string value = trim(t.substr(equal + 1));
    // A mintaban csak a teljes soros kommentet tamogatjuk, a " karakter
    // utan allo # nem komment. Ez szandekosan egyszeru TOML-reszhalmaz.
    if (value.size() >= 2 && value.front() == '"') {
      const auto closing = value.find('"', 1);
      if (closing == std::string::npos) {
        err = path + ":" + std::to_string(line_no) + ": unterminated string";
        return false;
      }
      const std::string after = trim(value.substr(closing + 1));
      if (!after.empty() && after.front() != '#') {
        err = path + ":" + std::to_string(line_no) + ": trailing text";
        return false;
      }
      value = value.substr(0, closing + 1);
    } else {
      const auto hash = value.find('#');
      if (hash != std::string::npos) {
        value = trim(value.substr(0, hash));
      }
    }
    if (!apply_value(cfg, key, value, err)) {
      err = path + ":" + std::to_string(line_no) + ": " + err;
      return false;
    }
  }
  return true;
```

```
}

bool apply_env(Config& cfg, std::string& err) {
  struct EnvKey { const char* name; const char* key; };
  const EnvKey keys[] = {
      {"TS_GW_MQTT_BROKER", "broker_uri"},
      {"TS_GW_CLIENT_ID", "client_id"},
      {"TS_GW_UDP_BIND", "udp_bind"},
      {"TS_GW_UDP_PORT", "udp_port"},
      {"TS_GW_QUEUE_CAPACITY", "queue_capacity"},
      {"TS_GW_QOS", "qos"},
      {"TS_GW_LOG_LEVEL", "log_level"},
      {"TS_GW_SPOOL_PATH", "spool_path"},
      {"TS_GW_MAX_SPOOL_RECORDS", "max_spool_records"},
  };
  for (const auto& entry : keys) {
    const char* raw = std::getenv(entry.name);
    if (raw == nullptr) {
      continue;
    }
    std::string value = raw;
    const std::string key = entry.key;
    if (key == "broker_uri" || key == "client_id" || key == "udp_bind" ||
        key == "log_level" || key == "spool_path") {
      value = "\"" + value + "\"";
    }
    if (!apply_value(cfg, key, value, err)) {
      err = std::string(entry.name) + ": " + err;
      return false;
    }
  }
  return true;
}

} // namespace

bool load_config(int argc, char** argv, Config& cfg, bool& help_requested,
                 std::string& err) {
  help_requested = false;
  std::string config_path;
  std::vector<std::pair<std::string, std::string>> overrides;
  for (int i = 1; i < argc; ++i) {
    const std::string arg = argv[i];
    if (arg == "--help" || arg == "-h") {
      help_requested = true;
      return true;
    }
    if (arg.rfind("--", 0) != 0) {
      err = "unexpected positional argument: " + arg;
      return false;
    }
    if (i + 1 >= argc) {
      err = "missing value for " + arg;
      return false;
    }
    const std::string value = argv[++i];
    if (arg == "--config") {
      config_path = value;
    } else if (arg == "--broker") {
      overrides.emplace_back("broker_uri", value);
    } else if (arg == "--client-id") {
      overrides.emplace_back("client_id", value);
    } else if (arg == "--udp-bind") {
```

```
      overrides.emplace_back("udp_bind", value);
    } else if (arg == "--udp-port") {
      overrides.emplace_back("udp_port", value);
    } else if (arg == "--queue-capacity") {
      overrides.emplace_back("queue_capacity", value);
    } else if (arg == "--qos") {
      overrides.emplace_back("qos", value);
    } else if (arg == "--log-level") {
      overrides.emplace_back("log_level", value);
    } else if (arg == "--spool-path") {
      overrides.emplace_back("spool_path", value);
    } else if (arg == "--max-spool-records") {
      overrides.emplace_back("max_spool_records", value);
    } else {
      err = "unknown option: " + arg;
      return false;
    }
  }

  if (!config_path.empty() && !load_file(config_path, cfg, err)) {
    return false;
  }
  if (!apply_env(cfg, err)) {
    return false;
  }
  for (const auto& override : overrides) {
    std::string value = override.second;
    if (override.first != "broker_uri" && override.first != "client_id" &&
        override.first != "udp_bind" && override.first != "log_level" &&
        override.first != "spool_path") {
      // A parancssori szamokat ugyanazzal a validacioval dolgozzuk fel.
    } else {
      value = "\"" + value + "\"";
    }
    if (!apply_value(cfg, override.first, value, err)) {
      err = "--" + override.first + ": " + err;
      return false;
    }
  }
  return true;
}

void print_usage(const char* program) {
  std::cout << "usage: " << program << " [options]\n"
            << "  --config <path>          TOML subset configuration\n"
            << "  --broker <uri>           MQTT broker URI\n"
            << "  --client-id <id>         MQTT client identifier\n"
            << "  --udp-bind <ip>          UDP bind address (default 0.0.0.0)\n"
            << "  --udp-port <port>        UDP port (default 4000)\n"
            << "  --queue-capacity <n>     bounded queue capacity\n"
            << "  --qos <0|1>              MQTT publish QoS\n"
            << "  --log-level <level>      debug, info, warn or error\n"
            << "  --spool-path <path>      durable spool path (i037)\n"
            << "  --max-spool-records <n>  spool retention limit (i037)\n";
}

} // namespace terrasense
```

A `main()` összerakja a komponenseket. Induláskor a broker elérhetetlensége nem végzetes hiba: a gateway tovább fogad, a minták a korlátos átmeneti sorba kerülnek, és a fő ciklus backoffal próbál újracsatlakozni. Sikertelen publish esetén a minta visszakerül a sor elejére; ha a sor megtelt, a legfiatalabb minta esik ki, és a drop-számláló nő. A `seq` commit csak sikeres PUBACK

után történik, ezért egy sikertelen publish nem „foglalja le” a sorszámot. Ez a viselkedés még nem tartós: a folyamat leállásakor a sor tartalma elveszik, ezt oldja meg a következő alfejezet spoolja.

```
#include "Config.h"
#include "EventQueue.h"
#include "Json.h"
#include "MqttPublisher.h"
#include "Normalizer.h"
#include "TelemetryTypes.h"
#include "UdpInputAdapter.h"
#include <algorithm>
#include <atomic>
#include <chrono>
#include <csignal>
#include <cstdint>
#include <exception>
#include <iostream>
#include <string>
#include <thread>

using namespace terrasense;

namespace {

std::atomic<bool> g_running{true};
std::atomic<std::uint64_t> g_main_loop_ticks{0};

void handle_signal(int) { g_running.store(false); }

int64_t now_ms() {
  using namespace std::chrono;
  return duration_cast<milliseconds>(system_clock::now().time_since_epoch())
      .count();
}

} // namespace

int main(int argc, char** argv) {
  Config config;
  bool help = false;
  std::string err;
  if (!load_config(argc, argv, config, help, err)) {
    std::cerr << "[ERROR] configuration: " << err << "\n";
    return 2;
  }
  if (help) {
    print_usage(argv[0]);
    return 0;
  }
  std::signal(SIGINT, handle_signal);
  std::signal(SIGTERM, handle_signal);

  try {
    Normalizer normalizer;
    EventQueue<TelemetrySample> queue(config.queue_capacity);
    MqttPublisher publisher(config.broker_uri, config.client_id);
    UdpInputAdapter udp(config.udp_bind, config.udp_port);

    std::cout << "[INFO] config broker=" << config.broker_uri
              << " udp=" << config.udp_bind << ":" << config.udp_port
              << " capacity=" << config.queue_capacity << "\n";
    if (publisher.connect(err)) {
```

```
  std::cout << "[INFO] MQTT connected\n";
} else {
  std::cout << "[WARN] MQTT unavailable: " << err
            << "; collecting in the transitional queue\n";
}

if (!udp.start(
        [&](std::string raw) {
          TelemetrySample sample;
          std::string parse_error;
          if (!normalizer.parse(raw, sample, parse_error)) {
            std::cout << "[WARN] rejected frame: " << parse_error << "\n";
            return;
          }
          std::string duplicate_error;
          if (normalizer.is_duplicate(sample, duplicate_error)) {
            std::cout << "[WARN] duplicate " << sample.site << "/"
                      << sample.node_id << " seq=" << sample.seq << ": "
                      << duplicate_error << "\n";
            return;
          }
          if (!queue.try_push(sample)) {
            std::cout << "[WARN] queue full; dropped " << sample.site
                      << "/" << sample.node_id << " seq=" << sample.seq
                      << " (dropped=" << queue.dropped() << ")\n";
          }
        },
        err)) {
  std::cerr << "[ERROR] UDP adapter: " << err << "\n";
  return 1;
}
std::cout << "[INFO] UDP listening on " << config.udp_bind << ":"
          << config.udp_port << "\n";

int backoff_ms = 1000;
int64_t next_reconnect_ms = 0;
while (g_running.load()) {
  g_main_loop_ticks.fetch_add(1);
  const int64_t now = now_ms();
  if (!publisher.is_connected() && now >= next_reconnect_ms) {
    if (publisher.connect(err)) {
      std::cout << "[INFO] MQTT connected\n";
      backoff_ms = 1000;
      next_reconnect_ms = 0;
    } else {
      std::cout << "[WARN] MQTT reconnect failed: " << err << "\n";
      next_reconnect_ms = now + backoff_ms;
      backoff_ms = std::min(backoff_ms * 2, 8000);
    }
  }

  TelemetrySample sample;
  if (!queue.wait_pop_for(sample, std::chrono::milliseconds(200))) {
    continue;
  }
  if (!publisher.is_connected()) {
    queue.requeue_front(sample);
    continue;
  }
  std::string duplicate_error;
  if (normalizer.is_duplicate(sample, duplicate_error)) {
    std::cout << "[WARN] duplicate before publish " << sample.site << "/"
              << sample.node_id << " seq=" << sample.seq << ": "
```

```
                << duplicate_error << "\n";
      continue;
    }
    std::string payload;
    std::string json_error;
    if (!to_json(sample, payload, json_error)) {
      std::cout << "[WARN] serialization failed: " << json_error << "\n";
      continue;
    }
    const std::string topic = "terrasense/" + sample.site + "/" +
                              sample.node_id + "/telemetry";
    if (publisher.publish(topic, payload, config.qos, err)) {
      normalizer.commit_seq(sample);
      std::cout << "[INFO] published " << topic << " seq=" << sample.seq
                << "\n";
    } else {
      std::cout << "[WARN] publish failed: " << err
                << "; sample kept in the transitional queue\n";
      queue.requeue_front(sample);
      publisher.disconnect();
      next_reconnect_ms = 0;
    }
  }

  udp.stop();
  queue.close();
  std::cout << "[INFO] gateway stopped; pending=" << queue.size()
            << " dropped=" << queue.dropped() << "\n";
 } catch (const std::exception& e) {
  std::cerr << "[ERROR] fatal: " << e.what() << "\n";
  return 1;
 }
 return 0;
}
```

A CMake-konfiguráció a Paho C++ 1.3.2 `paho-mqttpp3` pkg-config modulját keresi. Ha a modul neve a telepítésben eltér (például `paho-mqttpp3-static`), a `pkg_check_modules` sort kell javítani, nem a forráskódot.

```
cmake_minimum_required(VERSION 3.28)
project(ts-gw CXX)

set(CMAKE_CXX_STANDARD 17)
set(CMAKE_CXX_STANDARD_REQUIRED ON)
set(CMAKE_CXX_EXTENSIONS OFF)

find_package(PkgConfig REQUIRED)
pkg_check_modules(PAHO_MQTTPP3 REQUIRED IMPORTED_TARGET paho-mqttpp3)
find_package(Threads REQUIRED)

add_executable(ts-gw
  src/main.cpp
  src/Config.cpp
  src/Normalizer.cpp
  src/UdpInputAdapter.cpp
  src/MqttPublisher.cpp)

target_include_directories(ts-gw PRIVATE src)
target_link_libraries(ts-gw PRIVATE
  PkgConfig::PAHO_MQTTPP3
  Threads::Threads)
target_compile_options(ts-gw PRIVATE -Wall -Wextra -Wpedantic)
```

A labor UDP-küldője egyszerre több mintát, növekvő `seq`-vel tud küldeni; a `--site`, `--node`, `--boot` és `--no-ts` kapcsolókkal a többeszközös és a gateway-időbélyeges eset is gyakorolható:

```
#!/usr/bin/env python3
"""TerraSense UDP telemetry generator for the i036 gateway lab."""
import argparse
import socket
import time


def main():
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--host", default="127.0.0.1")
    parser.add_argument("--port", type=int, default=4000)
    parser.add_argument("--site", default="gh1")
    parser.add_argument("--node", default="n001")
    parser.add_argument("--boot", type=int, default=7)
    parser.add_argument("--seq", type=int, default=42)
    parser.add_argument("--count", type=int, default=1)
    parser.add_argument("--interval", type=float, default=1.0)
    parser.add_argument("--temp", type=float, default=23.4)
    parser.add_argument("--humidity", type=float, default=61.2)
    parser.add_argument("--soil", type=float, default=34.0)
    parser.add_argument("--no-ts", action="store_true",
                        help="omit ts so the gateway must estimate it")
    args = parser.parse_args()

    sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
    for index in range(args.count):
        seq = args.seq + index
        fields = [f"site={args.site}", f"node={args.node}",
                  f"boot={args.boot}", f"seq={seq}"]
        if not args.no_ts:
            fields.append(f"ts={int(time.time() * 1000)}")
        fields.extend([f"T={args.temp:g}", f"RH={args.humidity:g}",
                       f"SM={args.soil:g}"])
        message = ";".join(fields)
        sock.sendto(message.encode("utf-8"), (args.host, args.port))
        print(f"sent: {message}")
        if index + 1 < args.count:
            time.sleep(args.interval)


if __name__ == "__main__":
    main()
```

A build és a futtatás:

```
cd ~/iot-labor/i036/ts-gw
cmake -S . -B build -DCMAKE_BUILD_TYPE=Release
cmake --build build -j"$(nproc)"
./build/ts-gw --config config/ts-gw.toml
```

Ezután egy másik terminálban indítsd el az előfizetőt a labor brokerében, majd küldj öt mintát növekvő `seq`-vel:

```
docker exec -it lab-broker mosquitto_sub -t 'terrasense/#' -v
python3 ~/iot-labor/i036/ts-gw/scripts/send_udp.py --count 5 --seq 42 --interval 0.2
```

A `ts-gw` termináljának illusztratív kimenete (az időbélyeg és a PUBACK időzítése futásonként változik):

```
[INFO] config broker=tcp://localhost:1883 udp=0.0.0.0:4000 capacity=1024
[INFO] MQTT connected
[INFO] UDP listening on 0.0.0.0:4000
[INFO] published terrasense/gh1/n001/telemetry seq=42
[INFO] published terrasense/gh1/n001/telemetry seq=43
[INFO] published terrasense/gh1/n001/telemetry seq=44
[INFO] published terrasense/gh1/n001/telemetry seq=45
[INFO] published terrasense/gh1/n001/telemetry seq=46
```

A `mosquitto_sub` kimenete egy üzenetre (a `ts` értéke futásonként változik):

```
terrasense/gh1/n001/telemetry
↪  {"schema_version":1,"site":"gh1","node_id":"n001","boot_id":7,"seq":42,"ts":1793462683777,"ts_source":"devic
```

A fenti `ts` konkrét érték; a laborban az aktuális epoch ms jelenik meg. A `ts_source` és a `quality` mező akkor is látható, ha a node nem küldött időbélyeget: ilyenkor `gateway_estimate`, illetve `estimated_time` szerepel, és a `ts` a gateway fogadási ideje.

Továbbvitt eredmény. A `ts-gw` teljes projektje a `~/iot-labor/i036/ts-gw/` könyvtárban jön létre: `InputAdapter`, `UdpInputAdapter`, `Normalizer`, `Json`, `EventQueue`, `MqttPublisher`, `Config` és `main` komponensekkel. A `terrasense/<site>/<node>/telemetry` topicot mintánként képezzük, a QoS 1 publikálás PUBACK-ig tart, a `seq` commit a sikeres publish után történik, a hiányzó mérés `null`, a hibás idő pedig minőségjelölt gateway-idő. Erre a projektre épül a store-and-forward, az üzemeltetés, a helyi feldolgozás, a biztonságos kapcsolat és a záróprojekt.

12.2.5 Ellenőrzés és hibakeresés

A teljes lánc ellenőrzése a fenti `mosquitto_sub` előfizetéssel történik. A Paho C++ és a broker nélkül is ellenőrizhető a normalizáló, a JSON-író, a konfiguráció és a korlátos sor: ezek a komponensek külső könyvtár nélkül fordíthatók. A fejezet kódjából a `Config`, `Normalizer`, `Json` és `EventQueue` részt a laborban önállóan is le lehet fordítani; a teljes `ts-gw` linkeléséhez a Paho C++ fejlesztői csomag kell.

Az alábbi bemenetek szándékosan hibásak; a gateway a normalizáló hibaüzenetével utasítja el őket, és nem ír ki érvénytelen JSON-t:

```
python3 scripts/send_udp.py --node 'n"001' --seq 43
# [WARN] rejected frame: invalid node identifier
python3 scripts/send_udp.py --temp nan --seq 44
# [WARN] rejected frame: invalid temperature
python3 scripts/send_udp.py --humidity 101 --seq 45
# [WARN] rejected frame: humidity_rh out of range [0, 100]
```

A `--no-ts` kapcsolóval küldött minta nem device-időt kap, hanem `gateway_estimate` forrást és `estimated_time` minőséget. Ez a viselkedés a backend számára látható; a gateway nem tesz úgy, mintha a node órája szinkronban lenne.

Tünet	Lehetséges ok	Javítás
A `ts-gw` nem indul	hiányzik a `paho-mqttpp3` pkg-config modul	telepítsd a Paho C++ 1.3.2 fejlesztői csomagját; ellenőrizd a `pkg-config --modversion paho-mqttpp3` kimenetét
`[ERROR] UDP adapter: bind ...`	a 4000-es port foglalt, vagy rossz a bind-cím	`ss -lun | grep 4000`; állítsd le a foglaló folyamatot, vagy adj más `--udp-port` értéket

Tünet	Lehetséges ok	Javítás
A subscriber nem lát adatot	a topic vagy a broker-cím hibás	ellenőrizd a `--broker` értékét és a `terrasense/<site>/<node>/telemetry` topicot
A gateway nem ír `[INFO] published` sort	az UDP-küldő nem a megfelelő portra küld	`ss -lun | grep 4000`; a `send_udp.py --port` értéke egyezzen a konfigurációval
`[WARN] queue full`	a broker tartósan elérhetetlen, a sor megtelt	ez az átmeneti viselkedés; a tartós megoldás a store-and-forward
Duplikált `seq` üzenet	a normalizáló állapota elveszett újraindításkor	a következő fejezet tartós spoolja és a backend eseménykulcsa kezeli; a gateway naplózza a `boot_id` hiányát

A hálózati forgalom nyomkövetéséhez a `tcpdump -i lo udp port 4000` parancs használható; ezzel a bejövő UDP-keret pontos tartalma ellenőrizhető. A `seq` commit helyét a naplóban a `[INFO] published` sor utáni állapot mutatja: sikertelen publish esetén nincs commit, és a minta visszakerül a sorba.

12.2.6 Tipikus hibák és félreértések

A `seq` ellenőrzés a fogadáskor commitál. Ha a gateway már a sorba helyezéskor elfogadottnak jelöli a sorszámot, akkor egy sikertelen MQTT-publish után a minta elveszhet, miközben a `seq` állapota már előrelépett. A helyes tranzakciós pont a sikeres PUBACK (később a tartós spoolba írás). Ezért a `Normalizer` külön `is_duplicate()` és `commit_seq()` műveletet ad; a `main()` csak a publish sikere után commitál.

A locale-függő `std::stod`. A magyar vagy más európai rendszerbeállítás mellett a `std::stod("23.4")` értelmezése locale-függő lehet. Az IoT-üzenetekben mindig a pont a tizedesjel, ezért a normalizáló `std::from_chars`-t használ, nem a globális locale-t.

A blokkoló `recvfrom()` leállítása socketzárással. Ha a `stop()` a fogadó szál alatt zárja le a socketet, a fájlleíró újra felhasználhatóvá válhat, és a `recvfrom()` egy másik kapcsolat adatát olvashatja. A helyes sorrend: `running_ = false`, `join()`, majd `close()`. A 200 ms-os `poll()` timeout biztosítja, hogy a join ne blokkoljon sokáig.

A MQTT-publikálás visszatérési értékének figyelmen kívül hagyása. A QoS 1 publikálás csak akkor tekinthető a broker által nyugtázottnak, ha a token befejeződött és a visszatérési kód sikeres. A Paho C++ `publish()` önmagában nem garantálja, hogy a PUBACK megérkezett; a kód ezért `wait_for_completion()`-t és `get_return_code()`-ot is ellenőriz.

A JSON kézi összefűzése ellenőrizetlen mezőkkel. Ha a `node_id`-t vagy egy hibaüzenetet escape nélkül írunk a JSON-ba, egy idézőjel érvénytelen payloadot okoz. A `Json.h` ezért minden stringet escape-el, a számokat pedig csak véges érték esetén írja ki; az azonosítókat ezen felül a normalizáló is validálja.

12.2.7 Feladatok

1. **Fogalmi kérdés.** Magyarázd el, miért jobb az `InputAdapter` absztrakció, mint ha a `main()` közvetlenül az UDP-socketből olvasna, és onnan hívná a MQTT-publikálót.
2. **Tervezési feladat.** Tegyük fel, hogy egy valós `ts-node` BLE GATT-értesítésekkel küldi az adatokat. Írd le lépésenként, hogyan implementálnál egy `BleInputAdapter` osztályt a meglévő `InputAdapter` interfész alapján, milyen BlueZ D-Bus objektumokat kellene figyelni.
3. **Implementációs feladat.** Módosítsd a `Normalizer`-t úgy, hogy a hiányzó `seq` mezőt ne pótolja kitalált értékkel, hanem utasítsa el a keretet, és a hibaüzenet nevezze meg a hiányzó mezőt. Teszteld a `send_udp.py` egy olyan változatával, amely kihagyja a `seq`-et. Indokold, miért veszélyes a gateway által kitalált `seq`.
4. **Hibakeresési feladat.** Indítsd el a `ts-gw`-t, majd küldj folyamatosan növekvő `seq`-ű UDP-kereteket. Állítsd le a `lab-broker` szolgáltatást, küldj további mintákat, majd indítsd újra. Figyeld meg a `[WARN] queue full`, a drop-számlálót és az újracsatlakozást. Magyarázd el, miért nem elég ez a viselkedés tartós offline működéshez, és hogyan oldja meg a store-and-forward.

12.2.8 Megoldások

1. **Megoldás.** Az adapter leválasztja a gateway magját a bemeneti protokollról. Ha később BLE-re vagy soros portra váltunk, csak az adaptert kell kicserélni, a normalizáló és a publikáló réteg érintetlen marad. Emellett az adapter határozza meg a nyers adatok fogadásának és leállításának életciklusát, így a `main()` nem keveredik a socket-kezelés részleteibe. A nyers keret átadása azt is lehetővé teszi, hogy a normalizálás és a `seq` állapot egy helyen, a fő szálban maradjon.

2. **Megoldás.** A `BleInputAdapter` indulásakor csatlakozik a BlueZ D-Bus rendszerbuszhoz, majd az `org.bluez Adapter1` interfészen keresztül a megfelelő adaptert, a `Device1` interfészen a `ts-node` eszközt, végül a `GattCharacteristic1` interfészen a telemetria-karakterisztikát keresi meg. Ezután `StartNotify` hívással iratkozik fel az értesítésekre. Minden értesítés nyers bájtjait `std::string` keretként adja át a `FrameHandler`-nek; a normalizálás ugyanaz marad, mint UDP esetén. A leállítás `StopNotify` hívással és a D-Bus kapcsolat lezárásával történik.

3. **Megoldás.** A `parse()` végén a kötelező mezők ellenőrzése már tartalmazza a `seq`-et: `if (!have_site || !have_node || !have_seq || !have_metric)`. A hibaüzenet pontosan megnevezi a hiányzó mezőt, például `missing required field (site, node, seq or measurement)`. A gateway azért nem pótolhatja a `seq`-et `last_seq + 1` értékkel, mert a `seq` az eszköz eseményazonosítója: egy újraindult node új `boot_id`-dal újra 1-től számozhat, és a gateway által kitalált érték összeütközhet egy később érkező valódi eseménnyel. A hiányzó `seq` protokollhiba; a helyes válasz az elutasítás és a naplózás, nem a csendben történő kiegészítés.

4. **Megoldás.** A broker leállása után a `publish()` hívás `MQTT not connected` vagy tokenhiba miatt false-t ad. A minta visszakerül a sor elejére, a publisher lecsatlakozik, a fő ciklus pedig backoffal újrapróbálkozik. A sor korlátos: ha a broker hosszabb ideig nem elérhető, a `try_push()` a legújabb mintákat eldobja, és a `dropped` számláló nő. A folyamat leállásakor a sor tartalma elveszik, mert az csak memóriában van. Tartós offline működéshez a mintát a fogadás után azonnal lemezre kell írni, és a PUBACK után törölni; ezt valósítja meg a store-and-forward SQLite-spoolja.

12.2.9 Rövid összefoglaló

A `ts-gw` egy adapter-alapú, C++17-ben megvalósított Linux-gateway. Az `InputAdapter` absztrakció mögött a `UdpInputAdapter` fogadja a szimulált `ts-node` UDP-kereteit; valós hardveren ugyanezt az interfészt valósíthatja meg egy BlueZ D-Bus `BleInputAdapter`. A `Normalizer` a nyers `site=...;node=...;seq=...;T=...` szöveget `TelemetrySample`-lé alakítja, ellenőrzi az azonosítókat, a tartományokat és a `seq`-et, a hiányzó időbélyeget pedig minőségjelölt gateway-idővel pótolja. A `Json` modul escape-elt, véges számokat tartalmazó payloadot készít. A szálbiztos, korlátos `EventQueue` elválasztja a blokkoló fogadást a publikálástól. A `MqttPublisher` Paho C++ 1.3.2 klienst használ QoS 1-gyel, PUBACK-ellenőrzéssel és újracsatlakozással. A build CMake-kel, a labor teljes lánca UDP-küldéstől a `terrasense/<site>/<node>/telemetry` topicig terjed. **Továbbvitt eredmény:** a `~/iot-labor/i036/ts-gw/` alatt létrejött teljes projekt a store-and-forward, az üzemeltetés, a helyi feldolgozás, a biztonság és a záróprojekt alapja.

12.2.10 Források és ellenőrzési hiányok

- Paho MQTT C++ kliens dokumentáció: https://eclipse.dev/paho/ – a Paho C++ 1.3.2 `client`, `connect_options`, `token` és `message` API-ját, valamint a `paho-mqttpp3` pkg-config modul nevét a telepített csomagon ellenőrizni kell. A fejezet kódja ezekre az API-kra épül, de a Paho-könyvtár nélküli környezetben csak a saját komponensek fordíthatók le.
- Mosquitto dokumentáció: https://mosquitto.org/documentation/ – a `mosquitto_sub` kapcsolói (pl. `-v`, `-t`) ellenőrizendők a használt verzióban; a labor a `lab-broker` konténer klienseit használja.
- BlueZ D-Bus API: a `BleInputAdapter` vázlatában említett `Adapter1`, `Device1`, `GattCharacteristic1`, `StartNotify`, `StopNotify` objektumnevek a hivatalos BlueZ dokumentációban ellenőrizendők.
- CMake dokumentáció: a `pkg_check_modules(IMPORTED_TARGET)` és a `Threads::Threads` használata a rögzített CMake 3.28.3-ban ellenőrizendő.
- A `libpaho-mqttpp3-dev` csomagnév és a benne lévő Paho C++ verzió az Ubuntu 24.04 LTS csomaglistájában ellenőrizendő; a laborprofil a Paho C++ 1.3.2 verziót rögzíti.

12.3 Store-and-forward: offline működés és visszatöltés

A hálózatkiesés az IoT-edge rétegben nem ritka esemény, hanem tervezési feltétel. A TerraSense üvegházában a `ts-node` szenzorok továbbra is mérik a hőmérsékletet, páratartalmat és talajnedvességet, a `ts-gw` gateway viszont nem tudja az MQTT-üzeneteket a brokerhez eljuttatni, mert a helyi hálózat vagy a felhőkapcsolat megszakadt. Ha ilyenkor a gateway egyszerűen eldobná a beérkező telemetriát, értékes adatok vesznének el, és a későbbi elemzések, riasztások hiányosak lennének. A megoldás a **store-and-forward** (tárolás és továbbítás): az offline időszakban a gateway perzisztens puffert használ, majd a kapcsolat helyreállása után sorrendhelyesen visszatölti az adatokat, miközben jelzi az esetleges duplikációkat.

12.3.1 Fogalmak és működés

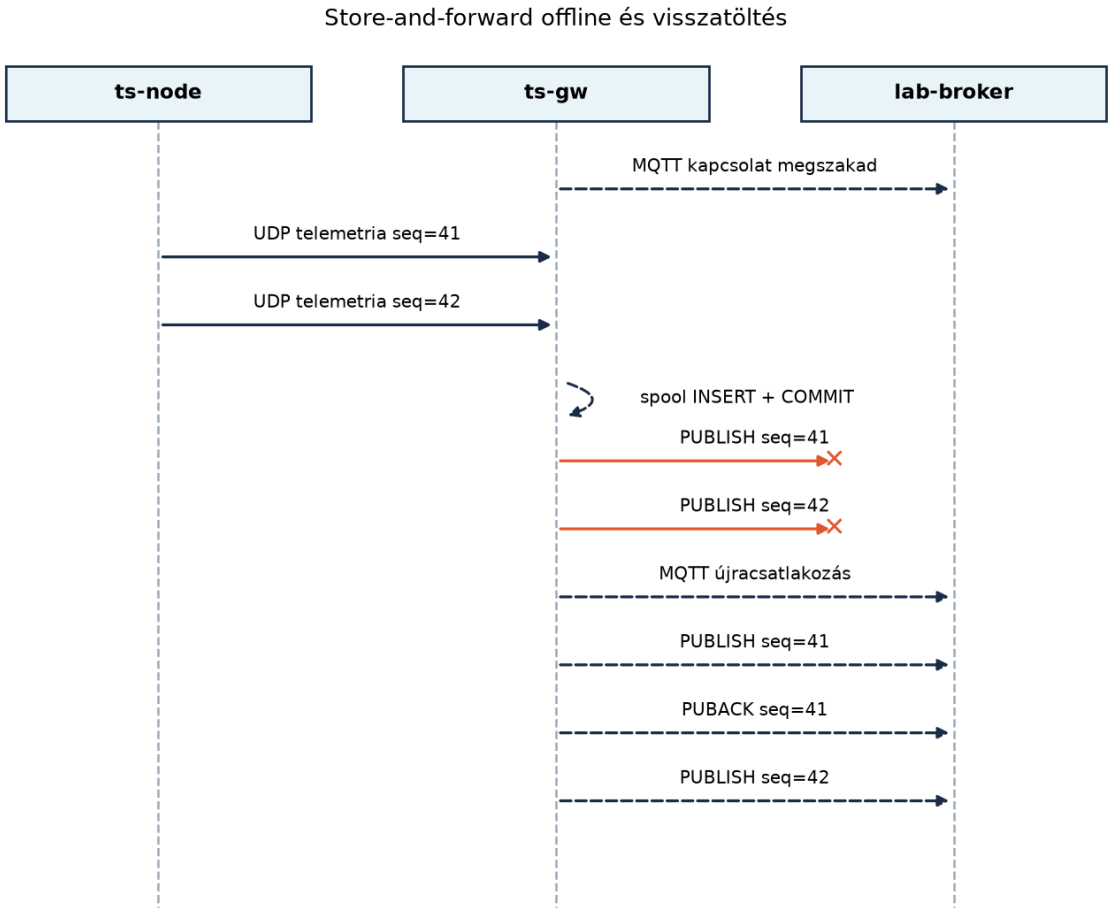
Definíció (store-and-forward). A store-and-forward olyan üzenetkezelési minta, amelyben a köztes csomópont (itt a gateway) az üzenetet először tartósan tárolja, majd a hálózati kapcsolat rendelkezésre állásakor továbbítja a cél felé. A tárolás akkor is megtörténik, ha a cél nem elérhető; így az adat nem vész el, legfeljebb késik.

Definíció (perzisztens puffer). A perzisztens puffer olyan, lemezen (vagy más nem felejtő közegen) tárolt várakozási sor, amely túléli a folyamat újraindulását is. A memóriapufferrel szemben nem törlődik áramszünet vagy processzkilépés esetén.

A `ts-gw` C++ szolgáltatás már rendelkezik az alapvető protokollfordítással: a bejövő UDP/BLE üzeneteket MQTT publish-csomaggá alakítja, és elküldi a brokerre. A store-and-forward réteg beillesztése a következő logikával bővíti ezt:

1. A beérkező eszközüzenetből kinyerjük a **deduplikációs kulcsot**: a `(site, node_id, boot_id, seq)` négyest. Ez azonosítja az eszköz egy adott mérési csomagját. A `boot_id` nélkül egy újraindult node `seq=1` értéke összeütközne a korábbi boot azonos sorszámával; a site nélkül két üvegház azonos node- és seq-értéke ütközne.
2. Az üzenetet beszúrjuk a perzisztens pufferbe, `UNIQUE(site, node_id, boot_id, seq)` megkötéssel, hogy ugyanaz a csomag ne kerülhessen be kétszer.
3. Az UDP-fogadó szál a beszúrás után azonnal commitálja a gateway oldali `seq` állapotot. A fogadás és a spoolba írás így a brokertől függetlenül működik; a broker elérhetetlensége nem blokkolja az adatfelvételt.
4. Egy külön **publisher** ciklus figyeli az MQTT-kapcsolatot. Ha van kapcsolat, a pufferből legfeljebb 50 rekordot olvas `id` szerinti sorrendben, és egyesével továbbítja őket.
5. Minden sikeres QoS 1 `PUBLISH` (a `PUBACK` megérkezése) után a rekord törlődik a pufferből. Ha a publish hibát ad, a rekord a pufferben marad, a publisher lecsatlakozik, és a ciklus backoffal újrapróbálkozik.

A gateway három állapotot különböztet meg. A **spooled** azt jelenti, hogy a minta tartósan a helyi SQLite-pufferben van. A **broker_acked** azt jelenti, hogy a broker a QoS 1 PUBACK-kel nyugtázta a PUBLISH-t. A **backend_committed** azt jelenti, hogy a backend az eseményt idempotensen feldolgozta és tartósan eltárolta. A PUBACK tehát nem azonos a backend feldolgozásával; a gateway a spoolból a PUBACK után törölhet, de a backend oldali idempotencia ettől függetlenül kötelező. Ezt a különbséget a Kafka és a kézbesítési szemantika alfejezet is hangsúlyozza.



12.3. ábra: Store-and-forward offline és visszatöltés - A ts-gw az offline időszakban a helyi SQLite-spoolba ír, majd a kapcsolat visszaállása után id szerint visszatölt. A PUBLISH akkor sikeres, ha a broker PUBACK-kel nyugtázza.

A puffer megvalósításához SQLite-ot használunk, **WAL** (Write-Ahead Logging) módban. A WAL lehetővé teszi, hogy az olvasó műveletek ne blokkolják az írót, és fordítva, ami akkor fontos, ha a publisher szál folyamatosan olvassa a sort, miközben az UDP-fogadó szál írja. A tranzakciók biztosítják, hogy a beszúrás, a korlát érvényesítése és a törlés atomi legyen, és a folyamat váratlan leállása esetén sem sérüljön a tábla. A `synchronous=NORMAL` beállítás WAL mellett jó kompromisszum: a leállás után az adatbázis konzisztens, de áramszünet esetén az utolsó néhány tranzakció elveszhet. Ha a puffer minden egyes mintát áramszünet után is kötelezően meg akar őrizni, `synchronous=FULL` szükséges, nagyobb írási késleltetéssel.

Tábla (spool séma).

Oszlop	Típus	Megkötés	Leírás
`id`	INTEGER	PRIMARY KEY AUTOINCREMENT	Visszatöltési sorrend
`site`	TEXT	NOT NULL	Telephely-azonosító
`node_id`	TEXT	NOT NULL	Eszközazonosító (pl. `n001`)
`boot_id`	INTEGER	NOT NULL	Eszköz boot-epoch azonosítója

Oszlop	Típus	Megkötés	Leírás
`seq`	INTEGER	NOT NULL	Monoton sorszám az adott booton belül
`ts`	INTEGER	NOT NULL	Eredeti eseményidő, epoch ms UTC
`topic`	TEXT	NOT NULL	Cél MQTT topic
`payload`	TEXT	NOT NULL	Sorosított üzenet
`created_ms`	INTEGER	NOT NULL	A spoolba írás ideje (megfigyeléshez)
		UNIQUE(`site`, `node_id`, `boot_id`, `seq`)	Deduplikáció

A `payload` mező a teljes JSON-szöveget tartalmazza; a gateway a visszatöltéskor nem építi újra az üzenetet, hanem bitazonos szöveget küld. Ha később CBOR-ra váltunk, a payload BLOB lehet. A `created_ms` nem az esemény ideje, hanem a gateway fogadási ideje; a kettőt a diagnosztikában külön kell kezelni.

12.3.2 Döntési szempontok

Memóriapuffer vs. lemezes puffer. A memóriapuffer gyors, de nem vészeli át a folyamat újraindulását. Mivel a gateway kritikus adatútközvetítő, a lemezes puffer az alapértelmezés. A memóriapuffert legfeljebb a lemez előtti gyors átmeneti sorként érdemes használni; a ts-gw szolgáltatás C++-ban: protokollfordítás MQTT-re korlátos `EventQueue`-ja ilyen átmeneti szerepet tölt be, de tartós tárolásra nem alkalmas.

SQLite vs. append-only fájl. Az append-only napló egyszerűbb, de a törlés utáni üresedés, a töredezettség és a konzisztens tranzakciók kezelése kézi munkát igényel. Az SQLite beépített tranzakciókat, lekérdezéseket és WAL-t ad, és C++-hoz is egyszerűen linkelhető. A hátrány az SQLite-függőség és a nagyobb bináris méret; a gatewayen ez elfogadható.

Korlát és eldobási stratégia. A lemez véges. Meg kell határozni a puffer maximális méretét (például rekordszámban vagy bájtban). Ha a puffer megtelik, két gyakori stratégia van:

- **drop-oldest**: a legrégebbi üzenet eldobása, az új megtartása, ami friss adatot biztosít, de lyukat hagy a múltban;
- **drop-newest**: a beérkező üzenet eldobása, ami a múltat őrzi meg, de a legfrissebbet veszíti el.

A TerraSense-ben a **drop-oldest** a választott stratégia, mert a riasztásoknak és a valós idejű megfigyelésnek jobban kedvez a friss adat, a történeti elemzés pedig ritkán igényel teljes offline időszakot másodperc pontossággal. A korlát rekordszám-alapú; a bájtalapú korlát a Feladatok egyik kiterjesztése. A korlát érvényesítése ugyanabban a tranzakcióban történik, mint a beszúrás, különben két párhuzamos beszúrás túlléphetné a limitet. A legrégebbi rekordokat az `id` sorrendje jelöli ki, így az eldobás determinisztikus.

Szempont	SQLite WAL	Append-only fájl	Memóriapuffer
Túlélés újraindításkor	igen	igen	nem
Tranzakciós biztonság	igen	kézzel	nem értelmezett
Olvasás írás közben	jól támogatott	nehézkes	triviális

Szempont	SQLite WAL	Append-only fájl	Memóriapuffer
Lekérdezés és korlát	SQL, kötött paraméterrel	saját kód	saját kód
Bonyolultság	közepes	kisebb kezdetben	kicsi

A szülőkönyvtár és a jogosultságok. A spool fájlja nem írható olyan könyvtárba, amely nem létezik, vagy amelybe a szolgáltatás felhasználója nem írhat. A laborban a projekt alatti `./var/spool.db` az alapértelmezés, és a gateway induláskor létrehozza a `./var` könyvtárat. Éles rendszerben a spool `/var/lib/ts-gw/spool.db`; ezt a systemd-telepítés `install -d -o tsgw -g tsgw -m 0750 /var/lib/ts-gw` paranccsal hozza létre. A gateway nem ír a `/etc` alá, és nem fut rootként.

A visszatöltés kötegelése. A `pending()` nem tölti be a teljes spoolt, hanem legfeljebb 50 rekordot ad vissza. Hosszú offline időszak után így a memóriahasználat és a brokerre zúduló első hullám is korlátozott. A rekordok között 100 ms szünet van; ez lassítja a visszatöltést, de védi a brokert és a backendet.

12.3.3 Kidolgozott példa: SQLite-alapú store-and-forward

Tegyük fel, hogy a broker 10:00:00-kor leáll. A `ts-node` továbbra is küldi a telemetriát 10 másodpercenként. A gateway a 10:00:10 és 10:00:30 között érkező üzeneteket a pufferbe írja. 10:01:00-kor a broker újra elérhető, a publisher ciklus pedig megkezdi a visszatöltést.

Idő (UTC)	Esemény	Puffer tartalom (id, site, node, boot, seq)
10:00:00	broker leáll	üres
10:00:10	UDP telemetria `gh1/n001`, boot=7, seq=41	(1, gh1, n001, 7, 41)
10:00:20	UDP telemetria `gh1/n001`, boot=7, seq=42	(1..2, gh1, n001, 7, 41..42)
10:00:30	UDP telemetria `gh1/n001`, boot=7, seq=43	(1..3, gh1, n001, 7, 41..43)
10:01:00	broker újra elindul	(1..3, gh1, n001, 7, 41..43)
10:01:00.2	PUBLISH seq=41 sikeres, törlés	(2, 42), (3, 43)
10:01:00.4	PUBLISH seq=42 sikeres, törlés	(3, 43)
10:01:00.6	PUBLISH seq=43 sikeres, törlés	üres

A visszatöltés sorrendjét az `id` oszlop garantálja, így a backend az eseményidőtől függetlenül a gateway általi fogadási sorrendben kapja az üzeneteket. Az `seq` mező a backend felé megőrzi az eszköz szerinti sorrendet, az `ts` a mért időt, a `created_ms` pedig a gateway oldali fogadási időt.

Munkakönyvtár és a projekt másolása. A ts-gw szolgáltatás C++-ban: protokollfordítás MQTT-re projektje a `~/iot-labor/i036/ts-gw` könyvtárban van. A másolás célja a `~/iot-labor/i037/ts-gw`, ezért a forrás útvonala abszolút:

```
mkdir -p ~/iot-labor/i037
cp -r ~/iot-labor/i036/ts-gw ~/iot-labor/i037/ts-gw
cd ~/iot-labor/i037/ts-gw
mkdir -p var
```

A másolás után a `Config`, `Normalizer`, `Json`, `EventQueue`, `UdpInputAdapter`, `MqttPublisher` és a többi A ts-gw szolgáltatás C++-ban: protokollfordítás MQTT-re-os fájl változatlanul a helyén van. Az alábbiakban csak a megváltozott vagy új fájlokat adjuk meg teljes terjedelemben: a `store_forward.hpp`, a `store_forward.cpp`, a `main.cpp`, a `CMakeLists.txt` és a konfiguráció. Ez a projekt így önállóan buildelhető; nem marad hiányzó `.cpp` vagy be nem fejezett osztály.

A konfiguráció a laborban a projekt alatti spoolt használja, hogy ne kelljen root jogosultság:

```
# config/ts-gw.toml
broker_uri = "tcp://localhost:1883"
client_id = "ts-gw"
udp_bind = "0.0.0.0"
udp_port = 4000
queue_capacity = 1024
qos = 1
log_level = "info"
spool_path = "./var/spool.db"
max_spool_records = 100000
```

A `StoreForwardDb` fejlécfájlja tartalmazza az `<sqlite3.h>` fejlécet, így a `sqlite3*` típus nem függ attól, hogy a felhasználó előbb más fejlécet includeolt-e. A `SpoolRecord` a teljes deduplikációs kulcsot és a payloadot is tartalmazza. A `pending()` legfeljebb `limit` rekordot ad vissza; nincs olyan metódus, amely a teljes spoolt egyetlen vektorba töltené.

```
#pragma once
#include <cstddef>
#include <cstdint>
#include <mutex>
#include <string>
#include <vector>
#include <sqlite3.h>

namespace terrasense {

struct SpoolRecord {
  int64_t id{0};
  std::string site;
  std::string node_id;
  uint32_t boot_id{0};
  uint32_t seq{0};
  int64_t ts{0};
  std::string topic;
  std::string payload;
};

// SQLite-alapu tartos spool. Egy kapcsolatot hasznal, a metodusokat mutex
// sorosítja; a WAL mod a folyamat leallasa után is konzisztens allapotot ad.
class StoreForwardDb {
public:
  StoreForwardDb(const std::string& path, std::size_t max_records);
  ~StoreForwardDb();
  StoreForwardDb(const StoreForwardDb&) = delete;
  StoreForwardDb& operator=(const StoreForwardDb&) = delete;

  void initialize();
  // A beszuras deduplikalt a (site, node_id, boot_id, seq) kulcson. Az
  // inserted akkor igaz, ha uj rekord kerult a spoolba.
```

```
  void enqueue(const std::string& site, const std::string& node_id,
               uint32_t boot_id, uint32_t seq, int64_t ts,
               const std::string& topic, const std::string& payload,
               bool& inserted);
  // Legfeljebb limit rekordot ad vissza id szerinti sorrendben.
  std::vector<SpoolRecord> pending(std::size_t limit);
  bool remove(int64_t id);
  std::size_t count();
  uint64_t dropped_total() const;

private:
  void enforce_limit_locked();
  void exec_locked(const char* sql);
  [[noreturn]] void fail_locked(const std::string& context) const;

  sqlite3* db_{nullptr};
  std::string path_;
  std::size_t max_records_;
  mutable std::mutex mutex_;
  uint64_t dropped_total_{0};
};

} // namespace terrasense
```

A megvalósítás a beszúrást és a korlát érvényesítését egyetlen `BEGIN IMMEDIATE` tranzakcióba fogja. A korlát lekérdezése és a törlés kötött paraméterrel fut, nem string-összefűzéssel; a `sqlite3_changes()` adja a ténylegesen törölt rekordok számát, amelyet a `spool_stats` tábla tartósan számlál. A `fail_locked()` minden SQLite-hibát kontextussal és az adatbázis útvonalával dob tovább, így a naplóban nem marad néma „prepare failed”.

```
#include "store_forward.hpp"
#include <chrono>
#include <memory>
#include <stdexcept>
#include <string>

namespace terrasense {
namespace {

using StmtPtr = std::unique_ptr<sqlite3_stmt, decltype(&sqlite3_finalize)>;

StmtPtr prepare(sqlite3* db, const char* sql) {
  sqlite3_stmt* stmt = nullptr;
  if (sqlite3_prepare_v2(db, sql, -1, &stmt, nullptr) != SQLITE_OK) {
    const std::string message = sqlite3_errmsg(db);
    sqlite3_finalize(stmt);
    throw std::runtime_error("sqlite prepare: " + message);
  }
  return StmtPtr(stmt, &sqlite3_finalize);
}

int64_t now_ms() {
  using namespace std::chrono;
  return duration_cast<milliseconds>(system_clock::now().time_since_epoch())
      .count();
}

} // namespace

StoreForwardDb::StoreForwardDb(const std::string& path,
                               std::size_t max_records)
    : path_(path), max_records_(max_records) {
  if (max_records_ == 0) {
```

```
    throw std::runtime_error("max_records must be positive");
  }
  sqlite3* db = nullptr;
  const int rc = sqlite3_open_v2(
      path_.c_str(), &db,
      SQLITE_OPEN_READWRITE | SQLITE_OPEN_CREATE | SQLITE_OPEN_FULLMUTEX,
      nullptr);
  if (rc != SQLITE_OK) {
    const std::string message = db != nullptr ? sqlite3_errmsg(db)
                                              : "cannot allocate sqlite handle";
    if (db != nullptr) {
      sqlite3_close_v2(db);
    }
    throw std::runtime_error("sqlite open " + path_ + ": " + message);
  }
  db_ = db;
  sqlite3_busy_timeout(db_, 5000);
}

StoreForwardDb::~StoreForwardDb() {
  if (db_ != nullptr) {
    sqlite3_close_v2(db_);
  }
}

void StoreForwardDb::fail_locked(const std::string& context) const {
  const char* message = db_ != nullptr ? sqlite3_errmsg(db_) : "null database";
  throw std::runtime_error(context + " (" + path_ + "): " + message);
}

void StoreForwardDb::exec_locked(const char* sql) {
  char* message = nullptr;
  if (sqlite3_exec(db_, sql, nullptr, nullptr, &message) != SQLITE_OK) {
    const std::string text = message != nullptr ? message : "unknown error";
    sqlite3_free(message);
    throw std::runtime_error("sqlite exec (" + path_ + "): " + text);
  }
}

void StoreForwardDb::initialize() {
  std::lock_guard<std::mutex> lock(mutex_);
  exec_locked("PRAGMA journal_mode=WAL;");
  exec_locked("PRAGMA synchronous=NORMAL;");
  exec_locked(
      "CREATE TABLE IF NOT EXISTS spool ("
      " id INTEGER PRIMARY KEY AUTOINCREMENT,"
      " site TEXT NOT NULL,"
      " node_id TEXT NOT NULL,"
      " boot_id INTEGER NOT NULL,"
      " seq INTEGER NOT NULL,"
      " ts INTEGER NOT NULL,"
      " topic TEXT NOT NULL,"
      " payload TEXT NOT NULL,"
      " created_ms INTEGER NOT NULL,"
      " UNIQUE(site, node_id, boot_id, seq)"
      ");"
      "CREATE INDEX IF NOT EXISTS idx_spool_id ON spool(id);"
      "CREATE TABLE IF NOT EXISTS spool_stats ("
      " key TEXT PRIMARY KEY, value INTEGER NOT NULL"
      ");"
      "INSERT INTO spool_stats(key, value) VALUES('dropped_total', 0)"
      " ON CONFLICT(key) DO NOTHING;");
  auto statement = prepare(
```

```
      db_, "SELECT value FROM spool_stats WHERE key='dropped_total';");
  const int rc = sqlite3_step(statement.get());
  if (rc == SQLITE_ROW) {
    dropped_total_ = static_cast<uint64_t>(sqlite3_column_int64(
        statement.get(), 0));
  } else if (rc != SQLITE_DONE) {
    fail_locked("read dropped_total");
  }
}

void StoreForwardDb::enqueue(const std::string& site,
                             const std::string& node_id, uint32_t boot_id,
                             uint32_t seq, int64_t ts,
                             const std::string& topic,
                             const std::string& payload, bool& inserted) {
  std::lock_guard<std::mutex> lock(mutex_);
  inserted = false;
  exec_locked("BEGIN IMMEDIATE;");
  try {
    auto statement = prepare(
        db_,
        "INSERT INTO spool"
        " (site, node_id, boot_id, seq, ts, topic, payload, created_ms)"
        " VALUES (?, ?, ?, ?, ?, ?, ?, ?)"
        " ON CONFLICT(site, node_id, boot_id, seq) DO NOTHING;");
    sqlite3_stmt* stmt = statement.get();
    if (sqlite3_bind_text(stmt, 1, site.c_str(), -1, SQLITE_TRANSIENT) !=
            SQLITE_OK ||
        sqlite3_bind_text(stmt, 2, node_id.c_str(), -1, SQLITE_TRANSIENT) !=
            SQLITE_OK ||
        sqlite3_bind_int64(stmt, 3, static_cast<sqlite3_int64>(boot_id)) !=
            SQLITE_OK ||
        sqlite3_bind_int64(stmt, 4, static_cast<sqlite3_int64>(seq)) !=
            SQLITE_OK ||
        sqlite3_bind_int64(stmt, 5, static_cast<sqlite3_int64>(ts)) !=
            SQLITE_OK ||
        sqlite3_bind_text(stmt, 6, topic.c_str(), -1, SQLITE_TRANSIENT) !=
            SQLITE_OK ||
        sqlite3_bind_text(stmt, 7, payload.c_str(), -1, SQLITE_TRANSIENT) !=
            SQLITE_OK ||
        sqlite3_bind_int64(stmt, 8, now_ms()) != SQLITE_OK) {
      fail_locked("bind enqueue");
    }
    if (sqlite3_step(stmt) != SQLITE_DONE) {
      fail_locked("insert spool record");
    }
    inserted = sqlite3_changes(db_) > 0;
    enforce_limit_locked();
    exec_locked("COMMIT;");
  } catch (...) {
    sqlite3_exec(db_, "ROLLBACK;", nullptr, nullptr, nullptr);
    throw;
  }
}

void StoreForwardDb::enforce_limit_locked() {
  auto count_statement = prepare(db_, "SELECT COUNT(*) FROM spool;");
  if (sqlite3_step(count_statement.get()) != SQLITE_ROW) {
    fail_locked("count spool");
  }
  const auto total = static_cast<uint64_t>(
      sqlite3_column_int64(count_statement.get(), 0));
  if (total <= static_cast<uint64_t>(max_records_)) {
```

```
    return;
  }
  const uint64_t to_drop =
      total - static_cast<uint64_t>(max_records_);
  auto delete_statement = prepare(
      db_,
      "DELETE FROM spool WHERE id IN"
      " (SELECT id FROM spool ORDER BY id ASC LIMIT ?);");
  if (sqlite3_bind_int64(delete_statement.get(), 1,
                         static_cast<sqlite3_int64>(to_drop)) != SQLITE_OK) {
    fail_locked("bind spool limit");
  }
  if (sqlite3_step(delete_statement.get()) != SQLITE_DONE) {
    fail_locked("enforce spool limit");
  }
  const auto dropped = static_cast<uint64_t>(sqlite3_changes(db_));
  if (dropped == 0) {
    return;
  }
  dropped_total_ += dropped;
  auto update_statement = prepare(
      db_,
      "UPDATE spool_stats SET value = value + ?"
      " WHERE key='dropped_total';");
  if (sqlite3_bind_int64(update_statement.get(), 1,
                         static_cast<sqlite3_int64>(dropped)) != SQLITE_OK) {
    fail_locked("bind dropped_total");
  }
  if (sqlite3_step(update_statement.get()) != SQLITE_DONE) {
    fail_locked("update dropped_total");
  }
}

std::vector<SpoolRecord> StoreForwardDb::pending(std::size_t limit) {
  std::lock_guard<std::mutex> lock(mutex_);
  std::vector<SpoolRecord> result;
  if (limit == 0) {
    return result;
  }
  auto statement = prepare(
      db_,
      "SELECT id, site, node_id, boot_id, seq, ts, topic, payload"
      " FROM spool ORDER BY id ASC LIMIT ?;");
  if (sqlite3_bind_int64(statement.get(), 1,
                         static_cast<sqlite3_int64>(limit)) != SQLITE_OK) {
    fail_locked("bind pending limit");
  }
  int rc = SQLITE_ROW;
  while ((rc = sqlite3_step(statement.get())) == SQLITE_ROW) {
    SpoolRecord record;
    record.id = sqlite3_column_int64(statement.get(), 0);
    const auto* site = sqlite3_column_text(statement.get(), 1);
    const auto* node = sqlite3_column_text(statement.get(), 2);
    record.site = site != nullptr
                      ? reinterpret_cast<const char*>(site)
                      : "";
    record.node_id = node != nullptr
                         ? reinterpret_cast<const char*>(node)
                         : "";
    record.boot_id = static_cast<uint32_t>(
        sqlite3_column_int64(statement.get(), 3));
    record.seq = static_cast<uint32_t>(
        sqlite3_column_int64(statement.get(), 4));
```

```
    record.ts = sqlite3_column_int64(statement.get(), 5);
    const auto* topic = sqlite3_column_text(statement.get(), 6);
    const auto* payload = sqlite3_column_text(statement.get(), 7);
    record.topic = topic != nullptr
                       ? reinterpret_cast<const char*>(topic)
                       : "";
    record.payload = payload != nullptr
                         ? reinterpret_cast<const char*>(payload)
                         : "";
    result.push_back(std::move(record));
  }
  if (rc != SQLITE_DONE) {
    fail_locked("read pending records");
  }
  return result;
}

bool StoreForwardDb::remove(int64_t id) {
  std::lock_guard<std::mutex> lock(mutex_);
  auto statement = prepare(db_, "DELETE FROM spool WHERE id=?;");
  if (sqlite3_bind_int64(statement.get(), 1,
                         static_cast<sqlite3_int64>(id)) != SQLITE_OK) {
    fail_locked("bind remove id");
  }
  if (sqlite3_step(statement.get()) != SQLITE_DONE) {
    fail_locked("remove spool record");
  }
  return sqlite3_changes(db_) > 0;
}

std::size_t StoreForwardDb::count() {
  std::lock_guard<std::mutex> lock(mutex_);
  auto statement = prepare(db_, "SELECT COUNT(*) FROM spool;");
  if (sqlite3_step(statement.get()) != SQLITE_ROW) {
    fail_locked("count spool records");
  }
  return static_cast<std::size_t>(
      sqlite3_column_int64(statement.get(), 0));
}

uint64_t StoreForwardDb::dropped_total() const {
  std::lock_guard<std::mutex> lock(mutex_);
  return dropped_total_;
}

} // namespace terrasense
```

A `main.cpp` a bemeneti oldalt és a publisher oldalt egy folyamatban, de egymástól függetlenül működteti. Az UDP-visszahívás normalizál, JSON-t készít, majd a spoolba ír; a spoolba írás sikere után commitálja a `seq`-et. A publisher ciklus csak a spoolból olvas, ezért a broker kiesése nem akadályozza az adatfelvételt. A `pending(50)` kötegelt olvasás, a `remove()` csak sikeres PUBACK után fut. A `main` a spool szülőkönyvtárát létrehozza, és a spool útvonalát a konfigurációból veszi, nem hardcode-olja a `/var/lib/ts-gw` címet.

```
#include "Config.h"
#include "Json.h"
#include "MqttPublisher.h"
#include "Normalizer.h"
#include "TelemetryTypes.h"
#include "UdpInputAdapter.h"
#include "store_forward.hpp"
#include <algorithm>
```

```
#include <atomic>
#include <chrono>
#include <csignal>
#include <cstdint>
#include <exception>
#include <filesystem>
#include <iostream>
#include <string>
#include <thread>

using namespace terrasense;

namespace {

std::atomic<bool> g_running{true};
std::atomic<std::uint64_t> g_main_loop_ticks{0};

void handle_signal(int) { g_running.store(false); }

int64_t now_ms() {
  using namespace std::chrono;
  return duration_cast<milliseconds>(system_clock::now().time_since_epoch())
      .count();
}

} // namespace

int main(int argc, char** argv) {
  Config config;
  bool help = false;
  std::string err;
  if (!load_config(argc, argv, config, help, err)) {
    std::cerr << "[ERROR] configuration: " << err << "\n";
    return 2;
  }
  if (help) {
    print_usage(argv[0]);
    return 0;
  }
  std::signal(SIGINT, handle_signal);
  std::signal(SIGTERM, handle_signal);

  try {
    const std::filesystem::path spool_path(config.spool_path);
    const std::filesystem::path parent = spool_path.parent_path();
    if (!parent.empty()) {
      std::error_code ec;
      std::filesystem::create_directories(parent, ec);
      if (ec) {
        throw std::runtime_error("cannot create spool directory " +
                                 parent.string() + ": " + ec.message());
      }
    }
    StoreForwardDb spool(config.spool_path, config.max_spool_records);
    spool.initialize();
    std::cout << "[INFO] spool " << config.spool_path
              << " max_records=" << config.max_spool_records
              << " current=" << spool.count() << "\n";

    Normalizer normalizer;
    MqttPublisher publisher(config.broker_uri, config.client_id);
    UdpInputAdapter udp(config.udp_bind, config.udp_port);
```

```
std::cout << "[INFO] config broker=" << config.broker_uri
          << " udp=" << config.udp_bind << ":" << config.udp_port
          << " qos=" << config.qos << "\n";
if (publisher.connect(err)) {
  std::cout << "[INFO] MQTT connected\n";
} else {
  std::cout << "[WARN] MQTT unavailable: " << err
            << "; input continues into the durable spool\n";
}

std::uint64_t last_dropped = spool.dropped_total();
if (!udp.start(
        [&](std::string raw) {
          TelemetrySample sample;
          std::string parse_error;
          if (!normalizer.parse(raw, sample, parse_error)) {
            std::cout << "[WARN] rejected frame: " << parse_error << "\n";
            return;
          }
          std::string duplicate_error;
          if (normalizer.is_duplicate(sample, duplicate_error)) {
            std::cout << "[WARN] duplicate " << sample.site << "/"
                      << sample.node_id << " boot=" << sample.boot_id
                      << " seq=" << sample.seq << ": "
                      << duplicate_error << "\n";
            return;
          }
          std::string payload;
          std::string json_error;
          if (!to_json(sample, payload, json_error)) {
            std::cout << "[WARN] serialization failed: " << json_error
                      << "\n";
            return;
          }
          const std::string topic = "terrasense/" + sample.site + "/" +
                                    sample.node_id + "/telemetry";
          bool inserted = false;
          try {
            spool.enqueue(sample.site, sample.node_id, sample.boot_id,
                          sample.seq, sample.ts_epoch_ms, topic, payload,
                          inserted);
          } catch (const std::exception& e) {
            std::cout << "[ERROR] spool enqueue failed: " << e.what()
                      << "\n";
            return;
          }
          // A tartos spoolba kerules a dedup elfogadasi pontja: innen a
          // seq commitálhato, fuggetlenul a broker elerhetosegetol.
          normalizer.commit_seq(sample);
          if (inserted) {
            std::cout << "[INFO] spooled " << topic
                      << " seq=" << sample.seq << "\n";
          } else {
            std::cout << "[INFO] duplicate already in spool " << topic
                      << " seq=" << sample.seq << "\n";
          }
          const std::uint64_t dropped = spool.dropped_total();
          if (dropped != last_dropped) {
            std::cout << "[WARN] spool limit reached; dropped="
                      << dropped << "\n";
            last_dropped = dropped;
          }
        },
```

```
        err)) {
  std::cerr << "[ERROR] UDP adapter: " << err << "\n";
  return 1;
}
std::cout << "[INFO] UDP listening on " << config.udp_bind << ":"
          << config.udp_port << "\n";

int backoff_ms = 1000;
int64_t next_reconnect_ms = 0;
constexpr std::size_t kBatchSize = 50;
while (g_running.load()) {
  g_main_loop_ticks.fetch_add(1);
  const int64_t now = now_ms();
  if (!publisher.is_connected() && now >= next_reconnect_ms) {
    if (publisher.connect(err)) {
      std::cout << "[INFO] MQTT connected\n";
      backoff_ms = 1000;
      next_reconnect_ms = 0;
    } else {
      std::cout << "[WARN] MQTT reconnect failed: " << err << "\n";
      next_reconnect_ms = now + backoff_ms;
      backoff_ms = std::min(backoff_ms * 2, 8000);
    }
  }
  if (!publisher.is_connected()) {
    std::this_thread::sleep_for(std::chrono::milliseconds(200));
    continue;
  }

  auto records = spool.pending(kBatchSize);
  if (records.empty()) {
    std::this_thread::sleep_for(std::chrono::milliseconds(200));
    continue;
  }
  bool publish_failed = false;
  for (const auto& record : records) {
    if (!g_running.load()) {
      break;
    }
    if (!publisher.publish(record.topic, record.payload, config.qos, err)) {
      std::cout << "[WARN] publish failed for spool id=" << record.id
                << ": " << err << "; record stays in spool\n";
      publisher.disconnect();
      next_reconnect_ms = 0;
      publish_failed = true;
      break;
    }
    if (!spool.remove(record.id)) {
      std::cout << "[WARN] spool id=" << record.id
                << " disappeared before removal\n";
    }
    std::cout << "[INFO] forwarded spool id=" << record.id << " "
              << record.topic << " boot=" << record.boot_id
              << " seq=" << record.seq << "\n";
    std::this_thread::sleep_for(std::chrono::milliseconds(100));
  }
  if (publish_failed) {
    continue;
  }
}

udp.stop();
std::cout << "[INFO] gateway stopped; spool=" << spool.count()
```

```
              << " dropped=" << spool.dropped_total() << "\n";
  } catch (const std::exception& e) {
    std::cerr << "[ERROR] fatal: " << e.what() << "\n";
    return 1;
  }
  return 0;
}
```

A CMake a Paho C++ mellett az SQLite-ot is pkg-configgal keresi, és mindkettőt `IMPORTED_TARGET`-ként linkeli. A `store_forward.cpp` bekerül az executable forrásai közé; a `paho-mqttpp3` target nem keveredik a Paho C `paho-mqtt3c` könyvtárral.

```
cmake_minimum_required(VERSION 3.28)
project(ts-gw CXX)

set(CMAKE_CXX_STANDARD 17)
set(CMAKE_CXX_STANDARD_REQUIRED ON)
set(CMAKE_CXX_EXTENSIONS OFF)

find_package(PkgConfig REQUIRED)
pkg_check_modules(PAHO_MQTTPP3 REQUIRED IMPORTED_TARGET paho-mqttpp3)
pkg_check_modules(SQLITE3 REQUIRED IMPORTED_TARGET sqlite3)
find_package(Threads REQUIRED)

add_executable(ts-gw
  src/main.cpp
  src/Config.cpp
  src/Normalizer.cpp
  src/UdpInputAdapter.cpp
  src/MqttPublisher.cpp
  src/store_forward.cpp)

target_include_directories(ts-gw PRIVATE src)
target_link_libraries(ts-gw PRIVATE
  PkgConfig::PAHO_MQTTPP3
  PkgConfig::SQLITE3
  Threads::Threads)
target_compile_options(ts-gw PRIVATE -Wall -Wextra -Wpedantic)
```

A build és az első, online futtatás:

```
cd ~/iot-labor/i037/ts-gw
cmake -S . -B build -DCMAKE_BUILD_TYPE=Release
cmake --build build -j"$(nproc)"
docker compose -f ~/iot-labor/stack/compose.yaml up -d lab-broker
./build/ts-gw --config config/ts-gw.toml
```

Egy másik terminálban figyeld a topicokat, majd küldj öt mintát. A küldő a ts-gw szolgáltatás C++-ban: protokollfordítás MQTT-re projekt abszolút útvonalán van, nem a régi `i017/udp_sender.py` példány:

```
docker exec -it lab-broker mosquitto_sub -t 'terrasense/#' -v
python3 ~/iot-labor/i036/ts-gw/scripts/send_udp.py --count 5 --seq 41 --interval 0.2
```

A gateway illusztratív kimenete:

```
[INFO] spool ./var/spool.db max_records=100000 current=0
[INFO] config broker=tcp://localhost:1883 udp=0.0.0.0:4000 qos=1
[INFO] MQTT connected
[INFO] UDP listening on 0.0.0.0:4000
[INFO] spooled terrasense/gh1/n001/telemetry seq=41
[INFO] forwarded spool id=1 terrasense/gh1/n001/telemetry boot=7 seq=41
```

```
[INFO] spooled terrasense/gh1/n001/telemetry seq=42
[INFO] forwarded spool id=2 terrasense/gh1/n001/telemetry boot=7 seq=42
```

Az offline szakasz ellenőrzése:

```
docker compose -f ~/iot-labor/stack/compose.yaml stop lab-broker
python3 ~/iot-labor/i036/ts-gw/scripts/send_udp.py --count 3 --seq 46 --interval 0.2
sqlite3 ~/iot-labor/i037/ts-gw/var/spool.db   'SELECT id, site, node_id, boot_id, seq, ts FROM
↪  spool ORDER BY id;'
```

A várt, illusztratív kimenet (a `ts` értéke futásonként változik):

```
1|gh1|n001|7|46|1793462683777
2|gh1|n001|7|47|1793462683977
3|gh1|n001|7|48|1793462684177
```

Ezután indítsd újra a brokert, és figyeld a visszatöltést:

```
docker compose -f ~/iot-labor/stack/compose.yaml start lab-broker
```

A publisher néhány másodpercen belül feldolgozza a három rekordot, a `spool` tábla kiürül, a `mosquitto_sub` pedig sorrendben mutatja a `seq=46`, `47`, `48` üzeneteket. Újabb lekérdezéssel ellenőrizhető, hogy a tábla üres:

```
sqlite3 ~/iot-labor/i037/ts-gw/var/spool.db 'SELECT COUNT(*) FROM spool;'
```

A korlát és az eldobás ellenőrzése. Indíts külön spoolt 3 rekordos korláttal, állítsd le a brokert, majd küldj öt mintát:

```
docker compose -f ~/iot-labor/stack/compose.yaml stop lab-broker
./build/ts-gw --config config/ts-gw.toml   --spool-path ./var/spool-limit.db
↪  --max-spool-records 3
python3 ~/iot-labor/i036/ts-gw/scripts/send_udp.py --count 5 --seq 41 --interval 0.1
sqlite3 ~/iot-labor/i037/ts-gw/var/spool-limit.db   'SELECT id, seq FROM spool ORDER BY id;
↪  SELECT value FROM spool_stats WHERE key="dropped_total";'
```

A várt, illusztratív eredmény: a legrégebbi két rekord kiesik, a táblában a `seq=43`, `44`, `45` marad, a `dropped_total` értéke 2. Az `id` értékek nem feltétlenül 3, 4, 5, mert az `AUTOINCREMENT` a törölt sorok után is növeli a számlálót; a sorrend és a `seq` a lényeg. A korlát érvényesítése ugyanabban a tranzakcióban fut, ezért nem fordulhat elő, hogy a beszúrás commitálódik, de a törlés nem.

A deduplikációs kulcs ellenőrzése. Ugyanazzal a spool-lal, offline állapotban küldj egy ismételt `boot=7, seq=43` mintát, majd egy új bootot jelképező `boot=8, seq=1` mintát:

```
python3 ~/iot-labor/i036/ts-gw/scripts/send_udp.py --boot 7 --seq 43
python3 ~/iot-labor/i036/ts-gw/scripts/send_udp.py --boot 8 --seq 1
sqlite3 ~/iot-labor/i037/ts-gw/var/spool-limit.db   'SELECT boot_id, seq, COUNT(*) FROM spool
↪  GROUP BY boot_id, seq ORDER BY boot_id, seq;'
```

A `boot=7, seq=43` rekordból egy marad, a `boot=8, seq=1` viszont új rekord, mert a node újraindulása új boot-epochot indít. Ez a viselkedés akkor is helyes, ha a node ugyanazon a site-on újraindul. A backend idempotenciája a `(site, node_id, boot_id, seq)` kulcsot kapja; a gateway helyi spoolja csak az első védelmi vonal.

12.3.4 Ellenőrzés és hibakeresés

Tünet	Lehetséges ok	Javítás
A puffer nem ürül, a broker elérhető	a publisher nem fut, vagy a publish eredményét nem ellenőrzi	nézd meg a `[INFO] forwarded spool` sorokat; ellenőrizd a `publisher.is_connected()` és a publish visszatérési értékét
A visszatöltött üzenetek sorrendje nem megfelelő	a `pending()` nem `ORDER BY id`	a lekérdezésben `ORDER BY id ASC` szerepeljen; a kötegelt olvasás is ezt tartja
Duplikált sorok a pufferben	hiányzik a négyrészes `UNIQUE` kulcs	`UNIQUE(site, node_id, boot_id, seq)`; a régi adatbázist új sémával kell létrehozni
„database is locked” induláskor	másik folyamat írja a fájlt, vagy nincs WAL	ellenőrizd, hogy egy gateway fut; a WAL bekapcsolása az `initialize()`-ban történik
„unable to open database file”	a szülőkönyvtár nem létezik vagy nem írható	a `main` létrehozza a szülőkönyvtárat; élesben a systemd `install -d` parancsa ad jogosultságot
A spool mérete nem csökken a limitnél	a `enforce_limit_locked()` nem fut le, vagy nincs kötve a paraméter	a beszúrás és a limit egy tranzakcióban van; ellenőrizd a `dropped_total` növekedését
A gateway a broker leállásakor sem vesz fel adatot	a régi `main` a publish sikerétől tette függővé a feldolgozást	a spoolba írás a broker állapotától független; a publish külön ciklus

A naplóban a `[INFO] spooled` a tartós elfogadást, a `[INFO] forwarded` a broker felé történő továbbítást, a `[WARN] spool limit reached` pedig az eldobást jelzi. A `spool_stats` tábla `dropped_total` értéke újraindulás után is megmarad, így a metrika nem tűnik el a folyamat újraindulásakor. A `created_ms` és a `ts` különbsége a gateway fogadási késleltetésére utalhat.

12.3.5 Tipikus hibák és félreértések

1. **A deduplikációs kulcs csak `(node_id, seq)`.** Két site ugyanazon node-azonosítóval és sorszámozással ütközik, és egy node újraindulása után az új `seq=1` hamis duplikátumnak tűnik. A helyes kulcs `(site, node_id, boot_id, seq)`. A `boot_id` nélküli örökölt üzenetnél a gateway a 0 értéket használja, és ezt figyelmeztetéssel naplózza; ez gyengébb védelem, nem tekinthető teljes idempotenciának.
2. **A PUBACK-et backend-commitként kezelik.** A broker a QoS 1 PUBACK-kel csak azt nyugtázza, hogy az üzenetet megkapta, nem azt, hogy a backend feldolgozta. A gateway a spoolból a PUBACK után törölhet, de a backendnek saját idempotencia-kulccsal kell védenie az üzleti hatást. A Kafka és a kézbesítési szemantika ezt a különbséget a fogyasztói oldal felől tárgyalja.
3. **A `pending()` az egész spoolt betölti.** Hosszú offline időszak után ez memóriacsúcsot és brokerterhelést okoz. A megoldás a kötegelt `LIMIT ?`, amelyet a `pending(50)` használ.
4. **A korlát SQL-je csak komment.** Ha a `DELETE` nincs előkészítve, paraméterezve és végrehajtva, a limit nem érvényesül. A fejezet kódjában az `enforce_limit_locked()`

a `sqlite3_prepare_v2`, `sqlite3_bind_int64` és `sqlite3_step` hívásokkal fut, a törölt rekordok számát pedig a `spool_stats` tábla tartósan számlálja.

5. **A spool a `/var/lib/ts-gw` alatt van, de a könyvtár nem létezik.** A `sqlite3_open_v2` ilyenkor nem tud fájlt létrehozni. A laborban a `./var/spool.db` relatív út a projekt alatt van; élesben a systemd-telepítés hozza létre a `/var/lib/ts-gw` könyvtárat a `tsgw` felhasználó tulajdonában.
6. **A `store_forward.hpp` nem includeolja a `sqlite3.h`-t.** A `sqlite3*` típus e nélkül nem ismert, és a hiba a fordítóban későn, félrevezető helyen jelenik meg. A fejléc közvetlenül tartalmazza a szükséges include-ot.
7. **A Paho C és a Paho C++ keveredik.** A projekt `<mqtt/client.h>`-t használ, ezért a CMake `paho-mqttpp3` modult keres, nem a `paho-mqtt3c`-t. A két könyvtár egy binárisban való keverése linkelési és ABI-problémákat okoz.

12.3.6 Feladatok

1. **Döntési kérdés.** Milyen esetben választanál memóriapuffert a lemezes puffer helyett a gatewayen? Indokold meg, és nevezd meg azt a kockázatot, amelyet a választás bevezet.
2. **Számolási feladat.** A `ts-node` 5 mintát küld 1 perc alatt (minden minta 120 B hasznos adat). A gateway offline 24 órán át. Hány rekord keletkezik, és mekkora a minimális lemezterület, ha a rekordonkénti overhead 80 B? Add meg a választ KiB-ban (1 KiB = 1024 B), és értékeld, hogy indokolt-e a lemez.
3. **Implementáció.** Egészítsd ki a `StoreForwardDb` osztályt egy második, bájtalapú korláttal (`max_spool_bytes`). Döntsd el, hogy a rekordszám- vagy a bájtkorlát érvényesüljön-e előbb, és a törlést ugyanabban a tranzakcióban végezd. Írd le, hogyan számlálod a `payload`, a `topic` és a metaadatok méretét.
4. **Hibakeresés.** A broker visszatérte után a puffer nem ürül, pedig a broker elérhető, és a `mosquitto_sub` látja a bejövő üzeneteket. A gateway naplója nem mutat publish hibát. Milyen állapotokat és metrikákat ellenőriznél? Készíts diagnosztikai lépéssort.
5. **Adatmodell.** Egy node újraindul, és a `boot_id`-ja megváltozik, de a `seq` értéke újra 1-től indul. Mutasd meg, miért nem tekinthető ez duplikátumnak, és hogyan különbözteti meg a spool és a backend a két eseményt.

12.3.7 Megoldások

1. **Megoldás.** A memóriapuffer akkor indokolt, ha a gateway csak rövid hálózati kiesést simít ki, az adatok újraküldése nem kritikus, és a folyamat újraindulásakor az adatvesztés elfogadható. Kockázat: áramszünet vagy processzkilépés esetén a teljes puffer elveszik. A TerraSense telemetriájánál ez nem elfogadható, ezért a lemezes SQLite-spool az alapértelmezés; a memória csak a lemez előtti rövid átmeneti sor lehet.

2. **Megoldás.** Az üzenetek száma: $5 \cdot 60 \cdot 24 = 7200$ rekord. Rekordonkénti méret: $120 + 80 = 200$ B. Teljes méret: $7200 \cdot 200 = 1\,440\,000$ B, ami $1\,440\,000/1024 = 1406{,}25$ KiB, azaz körülbelül 1,37 MiB. Ez kényelmesen elfér egy modern gateway lemezén; a valós mérethez a SQLite WAL és a lapok overheadje is hozzáadódik, ezért a tervezésnél érdemes 2-3× biztonsági tartalékkal számolni.

3. **Megoldás.** A bájtkorlátot a rekordok hozzávetőleges méretével lehet számolni: `payload` és `topic` UTF-8 bájtjai, valamint a fix mezők (körülbelül 40-60 B) összege. A `spool` táblába érdemes egy `size_bytes` oszlopot felvenni, amelyet beszúráskor számolunk. A limit érvényesítésekor először a rekordszámot, majd a bájtösszeget kell ellenőrizni; a két limit közül a szigorúbbat kell alkalmazni. A törlés ugyanabban a `BEGIN IMMEDIATE` tranzakcióban

történik, mint a beszúrás, és a törölt rekordok számát/metrikáját a `spool_stats` táblában kell tartósítani. A bájtalapú limitet a `main` konfigurációjában és a `StoreForwardDb` konstruktorában is át kell adni; a részleges implementáció nem elfogadható.

4. **Megoldás.** Először a `publisher.is_connected()` értékét és a `[INFO] MQTT connected` / `[WARN] MQTT reconnect failed` naplósorokat kell nézni. Másodszor a `spool.pending(50)` visszatérési értékét és a `spool.count()` értékét kell naplózni: ha a puffer üres, de a subscribe látja az üzeneteket, akkor a gateway nem a várt spoolból dolgozik, vagy egy másik példány publishol. Harmadszor a `remove()` visszatérési értékét és a `spool_stats dropped_total` értékét kell ellenőrizni. A „nincs publish hiba" önmagában nem bizonyíték: a publish visszatérési értékét és a token visszatérési kódját is naplózni kell. Végül ellenőrizni kell, hogy a `main` ciklus nem ragadt-e be a `pending()` vagy a publish hívásban; a `g_main_loop_ticks` számláló és a watchdog a következő alfejezet eszköze.

5. **Megoldás.** A node újraindulása után a `boot_id` új értéket kap, ezért a `(site, node_id, boot_id, seq)` kulcs különbözik, noha a `seq` értéke ismét 1. A spool és a backend ezt új eseményként fogadja el. A régi boot ugyanazon `seq` értékének ismételt megérkezése viszont ugyanazt a kulcsot adja, ezért az `UNIQUE` megkötés és a backend idempotencia-kulcsa kiszűri. A `boot_id` nélküli örökölt üzenetnél ez a védelem hiányos; a gateway ilyenkor 0-t használ, és figyelmeztetést naplóz.

12.3.8 Rövid összefoglaló

Az alfejezet bemutatta a gateway store-and-forward rétegét SQLite WAL-módú perzisztens pufferrel. A `StoreForwardDb` `enqueue()`, `pending(limit)`, `remove()`, `count()` és `dropped_total()` metódusokkal kezeli a sort; a séma `UNIQUE(site, node_id, boot_id, seq)` kulccsal biztosítja a deduplikációt, a `spool_stats` tábla pedig a limit miatti eldobásokat számlálja. Az UDP-visszahívás a brokertől függetlenül ír a spoolba, a publisher ciklus csak sikeres QoS 1 PUBACK után töröl. A visszatöltés `id` szerinti sorrendben, 50 rekordos kötegekben, 100 ms-os késleltetéssel történik. **Továbbvitt eredmény:** a `~/iot-labor/i037/ts-gw` projekt teljes `store_forward.hpp`/`.cpp` és `main.cpp` forrással, a `(site, node_id, boot_id, seq)` eseménykulcs pedig a backend idempotens fogadásának és a Kafka kézbesítési szemantikájának alapja.

12.3.9 Források és ellenőrzési hiányok

- SQLite dokumentáció: WAL mód, `PRAGMA journal_mode`, `INSERT ... ON CONFLICT DO NOTHING`, `sqlite3_bind_*`, `sqlite3_changes`. URL: https://sqlite.org/docs.html – a fejezet SQL-sémáját és a drop-oldest lekérdezést a laborban külön ellenőrizni kell; a C++ API pontos viselkedése a használt SQLite-verziótól függ.
- Eclipse Paho C++ kliens dokumentáció: `mqtt::client`, `mqtt::token`, QoS 1 PUBACK. URL: https://eclipse.dev/paho/ – a `wait_for_completion()` és `get_return_code()` szemantikáját a Paho C++ 1.3.2 verzióhoz kell ellenőrizni.
- Mosquitto dokumentáció: QoS 1 PUBACK viselkedés. URL: https://mosquitto.org/documentation/
- **Ellenőrizendő:** a Paho C++ 1.3.2 `publish()` tokenjének blokkolási garanciája, az SQLite `synchronous=NORMAL` tartóssági szintje áramszünet esetén, valamint a `paho-mqttpp3` és `sqlite3` pkg-config modulok neve az Ubuntu 24.04 telepítésen. A fejezet C++ forrása a Paho és SQLite fejlesztői csomagokkal fordítható; a teljes brokeres végigfuttatás a labor feladata.

12.4 A gateway üzemeltetése: systemd, konténerek, naplózás

12.4.1 Motiváció

Az A ts-gw szolgáltatás C++-ban: protokollfordítás MQTT-re alfejezetben elkészített `ts-gw` már lefordítható és kézzel elindítható, a Store-and-forward: offline működés és visszatöltés alfejezet pedig tartós SQLite-pufferrel egészítette ki. Éles környezetben ez még kevés. Az üvegházban futó gatewaynek akkor is élnie kell, ha a folyamat összeomlik, a gép újraindul, vagy egy frissítés hibásnak bizonyul. Emellett a logokból gyorsan ki kell derülnie, hogy melyik csomópont üzenete veszett el, és a telepítésnek reprodukálhatónak kell lennie a fejlesztői géptől az edge hardverig.

Ebben az alfejezetben a `ts-gw`-t Linux-szolgáltatásként és konténerben is üzemeltethetővé tesszük. A systemd biztosítja az automatikus újraindítást és a watchdog-felügyeletet, a strukturált JSON-naplózás a journald-ben kereshetővé teszi az eseményeket, a többlépcsős Docker-build pedig kicsi, jól korlátozott futásidejű image-et eredményez. A watchdog nem vakon küld életjelet: a gateway csak akkor jelez, ha a fő adatút és az UDP-bemenet is előrehaladt.

12.4.2 Fogalmak és működés

Definíció. A *systemd unit* a systemd által kezelt erőforrás (szolgáltatás, socket, timer, mount stb.) deklaratív leírása. A szolgáltatások unitfájlja `.service` kiterjesztésű, és a `[Unit]`, `[Service]`, `[Install]` szekciókban adja meg, hogy mit, hogyan és mikor indítson a rendszer.

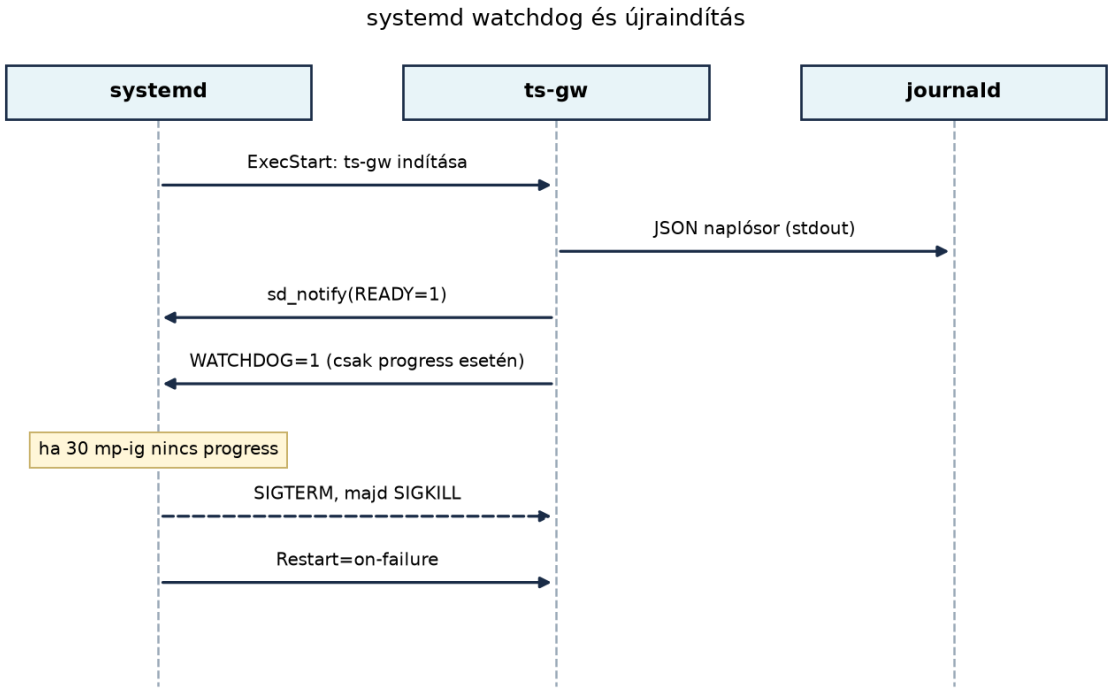
Definíció. A systemd *watchdog* egy szoftveres felügyelet: ha a szolgáltatás `WatchdogSec` értéken belül nem küld életjelet, a systemd megöli és az újraindítási politika szerint újraindítja a folyamatot. A folyamat a `WATCHDOG=1` üzenetet az `sd_notify` interfészen keresztül, a `$NOTIFY_SOCKET` Unix-socketre írva jelzi. A TerraSense gatewayben ez az életjel *progress-alapú*: csak akkor megy ki, ha a figyelt számlálók az előző ellenőrzés óta elmozdultak.

Definíció. A *Type=notify* indítási mód azt jelenti, hogy a szolgáltatás nem akkor tekinthető sikeresen elindultnak, amikor a fork visszatér, hanem amikor a folyamat `READY=1` üzenetet küld. Ugyanezen a csatornán megy a watchdog-életjel is. Ha a folyamat nem küld `READY=1`-et, a systemd az indítást hibásnak tekinti, és a `Restart` politika szerint újrapróbálhat.

Definíció. A *többlépcsős Docker-build* olyan image-építés, ahol a buildhez szükséges eszközöket (compiler, fejlesztői fejlécek) egy külön építő konténerbe tesszük, majd a kész binárist és a minimális futásidejű függőségeket egy második, jóval kisebb image-be másoljuk.

Definíció. A systemd *StateDirectory* beállítása egy szolgáltatás számára létrehozott, a szolgáltatás felhasználója által írható állapotkönyvtárat jelent. A `ts-gw` esetében ez a `/var/lib/ts-gw`, ahol a tartós spool adatbázisa van. A systemd a könyvtárat az indítás előtt hozza létre, a `User`/`Group` beállításnak megfelelő tulajdonossal, ezért a folyamatnak nem kell rootként írnia a `/var/lib` alá.

Az alábbi sequence-diagram a systemd és a `ts-gw` együttműködését mutatja progress-alapú watchdog mellett:



12.4. ábra: systemd watchdog és újraindítás - A ts-gw Type=notify szolgáltatásként indul. A WATCHDOG=1 csak akkor megy ki, ha a fő adatút és az UDP-bemenet is előrehaladt.

A szolgáltatás tehát három dolgot ír: a szabványos kimenetre JSON-sorokat (napló), a `$NOTIFY_SOCKET`-re `READY=1`-et és csak előrehaladás esetén `WATCHDOG=1`-et (életjel). A journald ezeket együtt tárolja, és `journalctl -u ts-gw` paranccsal lekérdezhető.

A progress-alapú watchdog lényege, hogy a watchdog-szál nem a saját életét figyeli, hanem a munkaszálakét. A `ts-gw` két számlálót tart: a fő ciklus iterációnként és minden feldolgozott spool-rekordnál növeli az egyiket, az UDP-fogadó szál pedig minden `poll` körben a másikat. Ha a fő ciklus beszorul egy SQLite-zárba, vagy az UDP-szál nem jut el a következő `poll`-ig, a watchdog észleli, hogy az adott számláló nem mozdult, és nem küld `WATCHDOG=1`-et. Így a rendszer nem „gyógyítja meg” azt a hibát, amelytől a szolgáltatás valójában nem működik.

12.4.3 Döntési szempontok

A gateway üzemeltetési módjának kiválasztása több szemponttól függ:

Szempont	systemd szolgáltatás	Docker-konténer
Újraindítás, watchdog	natív, jól integrált (`Restart`, `WatchdogSec`, `sd_notify`)	a konténer újraindítása Docker/Compose szinten; systemd-watchdog alapból nincs a konténerben
Erőforrás-limit	`MemoryMax`, `CPUQuota`, `TasksMax`	`mem_limit`, `cpus`, `pids_limit`
Függőségek kezelése	OS-szintű csomagok, verzióütközések lehetnek	az image-ben szigetelt, reprodukálható

Szempont	systemd szolgáltatás	Docker-konténer
Frissítés	bináris csere + `systemctl restart`	image-tag csere + `docker compose up -d`, rollback egyszerű
Naplózás	közvetlenül journald-be	konténerlogból gyűjtőbe, vagy journald-driverrel
Konfiguráció	`/etc/ts-gw/config.toml` + `EnvironmentFile`	image-be épített alapértelmezés + Compose `environment`

Egyetlen edge-gatewayen általában a systemd a természetes választás, mert nincs plusz réteg, és a watchdog közvetlenül a folyamatot figyeli. A konténer akkor előnyös, ha a teljes edge-szoftvercsomagot (gateway + MQTT-kliens + konfiguráció) egyetlen egységként akarjuk szállítani és több gépre telepíteni. A TerraSense laborban mindkettőt bemutatjuk: a systemd a hoston futó natív telepítés, a Docker a hordozható változat. A két mód ugyanazt a `Config` betöltési sorrendet használja, ezért a viselkedés nem függ attól, hogy éppen melyikben fut a gateway.

12.4.4 Kidolgozott példa és labor

Munkakönyvtár: `~/iot-labor/i038`

```
mkdir -p ~/iot-labor/i038
cp -r ~/iot-labor/i037/ts-gw ~/iot-labor/i038/ts-gw
cd ~/iot-labor/i038/ts-gw
```

A labor előfeltétele a Store-and-forward alfejezetben elkészült `ts-gw`, amely már tartalmazza a tartós SQLite-spoolt és a teljes MQTT-publikálót. A gateway üzemeltetése: systemd, konténerek, naplózás nem írja újra ezeket a részeket: a `Config`, a `Normalizer`, az `UdpInputAdapter`, a `MqttPublisher`, a `Json` és a `store_forward` változatlanul a Store-and-forward: offline működés és visszatöltés alfejezetből származik. Új elem a `Logger`, a progress-alapú `Watchdog`, az ezeket bekötő `main.cpp`, valamint a systemd- és Docker-telepítés.

A projekt a következő fájlokból áll:

```
ts-gw/
  CMakeLists.txt
  Dockerfile
  .dockerignore
  config/ts-gw.toml
  deploy/ts-gw.service
  deploy/ts-gw.default
  scripts/send_udp.py
  src/
    Config.h
    Config.cpp
    EventQueue.h
    InputAdapter.h
    Json.h
    Logger.h
    Logger.cpp
    MqttPublisher.h
    MqttPublisher.cpp
    Normalizer.h
    Normalizer.cpp
    TelemetryTypes.h
```

```
UdpInputAdapter.h
UdpInputAdapter.cpp
Watchdog.h
Watchdog.cpp
store_forward.hpp
store_forward.cpp
main.cpp
```

12.4.4.1 A konfiguráció és a prioritási sorrend

A gateway beállításait egy lapos TOML-részhalmaz, a `TS_GW_*` környezeti változók és a parancssori kapcsolók adják. A sorrend: beépített alapérték, majd a `--config` fájl, majd a környezeti változók, végül a parancssor. A későbbi forrás mindig felülírja a korábbit.

```
# config/ts-gw.toml
# A fájl a labor alapértelmezése; a rendszerd unit és a Compose felülírhatja.
broker_uri = "tcp://127.0.0.1:1883"
client_id = "ts-gw-gh1"
udp_bind = "0.0.0.0"
udp_port = 4000
queue_capacity = 1024
qos = 1
log_level = "info"
spool_path = "/var/lib/ts-gw/spool.db"
max_spool_records = 100000
```

A `Config` struktúra és a betöltő teljes forrása a ts-gw szolgáltatás C++-ban: protokollfordítás MQTT-re alfejezetben található; itt csak a szerződést ismételjük meg:

```
struct Config {
  std::string broker_uri{"tcp://localhost:1883"};
  std::string client_id{"ts-gw"};
  std::string udp_bind{"0.0.0.0"};
  uint16_t udp_port{4000};
  std::size_t queue_capacity{1024};
  int qos{1};
  std::string log_level{"info"};
  std::string spool_path{"./var/spool.db"};
  std::size_t max_spool_records{100000};
};
```

A környezeti változók nevei a `Config` mezőihez rendelődnek:

Környezeti változó	Mező	Megjegyzés
`TS_GW_MQTT_BROKER`	`broker_uri`	teljes URI, például `tcp://lab-broker:1883`
`TS_GW_CLIENT_ID`	`client_id`	MQTT kliensazonosító
`TS_GW_UDP_BIND`	`udp_bind`	konténerben `0.0.0.0` kell
`TS_GW_UDP_PORT`	`udp_port`	1–65535
`TS_GW_QUEUE_CAPACITY`	`queue_capacity`	pozitív egész
`TS_GW_QOS`	`qos`	ebben a gatewayben 0 vagy 1
`TS_GW_LOG_LEVEL`	`log_level`	`debug`, `info`, `warn`, `error`
`TS_GW_SPOOL_PATH`	`spool_path`	systemdben `/var/lib/ts-gw/spool.db`
`TS_GW_MAX_SPOOL_RECORDS`	`max_spool_records`	a drop-oldest korlát

A parancssori kapcsolók közvetlenül ugyanezeket a mezőket állítják: `--config`, `--broker`, `--client-id`, `--udp-bind`, `--udp-port`, `--queue-capacity`, `--qos`, `--log-level`, `--spool-path`, `--max-spool-records`. A `--help` a teljes listát kiírja.

A `load_config()` önálló, MQTT-könyvtár nélküli tesztjében a prioritási sorrend ténylegesen megfigyelhető. Ugyanarra a TOML-fájlra először a fájl `udp_port = 4000` értéke érvényesült, majd a `TS_GW_UDP_PORT=4100` környezeti változó, végül a `--udp-port 4200` parancssori kapcsoló nyert:

```
TOML:                   port=4000
TOML + env:             port=4100
TOML + env + CLI:       port=4200
```

A laborban a teljes gateway futtatásakor ugyanez a napló első `config ...` sorában látható. Ha a TOML-ban átírjuk az `udp_port` értékét, és nem adunk meg felülíró env-változót vagy kapcsolót, a gateway más porton figyel; ez az a viselkedés, amit a rendszerd unit `--config` kapcsolója elvár.

12.4.4.2 Naplózás: egy JSON-sor egy esemény

A naplózó a `stdout`-ra ír, soronként egy JSON-objektumot. A rendszerd a szabványos kimenetet a journald-be irányítja, így a napló nem függ attól, hogy a folyamat milyen könyvtárból indul. A három kötelező mező a `ts`, a `level` és a `msg`; a TerraSense későbbi naplófeldolgozása is ezekre épül.

```
// src/Logger.h
#pragma once
#include <string>

namespace terrasense {

// ISO 8601 UTC idobelyeg ezredmasodperccel, pl. 2026-10-31T12:34:56.789Z.
std::string current_iso8601_utc();

// A kimenet minimalis szintje: debug, info, warn vagy error. Az ismeretlen
// ertek az info szintet jelenti.
void set_log_level(const std::string& level);

// Egy JSON-sor a stdoutra. A msg es a level escape-elve kerul a kimenetbe.
// A beallitott szint alatti uzenetek nem íródnak ki.
void log_json(const std::string& level, const std::string& msg);

} // namespace terrasense
```

```
// src/Logger.cpp
#include "Logger.h"
#include "Json.h"
#include <atomic>
#include <chrono>
#include <cstdio>
#include <ctime>
#include <iostream>
#include <mutex>

namespace terrasense {
namespace {
std::mutex g_log_mutex;
std::atomic<int> g_min_level{1};
```

```
int level_rank(const std::string& level) {
  if (level == "debug") return 0;
  if (level == "info") return 1;
  if (level == "warn") return 2;
  if (level == "error") return 3;
  return 1;
}
} // namespace

std::string current_iso8601_utc() {
  using namespace std::chrono;
  const auto now = system_clock::now();
  const auto milliseconds_part =
      duration_cast<milliseconds>(now.time_since_epoch()) % 1000;
  const std::time_t seconds = system_clock::to_time_t(now);
  std::tm utc{};
  gmtime_r(&seconds, &utc);
  char base[32];
  std::strftime(base, sizeof(base), "%Y-%m-%dT%H:%M:%S", &utc);
  char result[40];
  std::snprintf(result, sizeof(result), "%s.%03lldZ", base,
                static_cast<long long>(milliseconds_part.count()));
  return result;
}

void set_log_level(const std::string& level) {
  g_min_level.store(level_rank(level));
}

void log_json(const std::string& level, const std::string& msg) {
  if (level_rank(level) < g_min_level.load()) {
    return;
  }
  std::string line;
  line.reserve(msg.size() + 96);
  line += "{\"ts\":";
  json_append_escaped(line, current_iso8601_utc());
  line += ",\"level\":";
  json_append_escaped(line, level);
  line += ",\"msg\":";
  json_append_escaped(line, msg);
  line += "}";
  std::lock_guard<std::mutex> lock(g_log_mutex);
  std::cout << line << '\n' << std::flush;
}

} // namespace terrasense
```

A `json_append_escaped()` a ts-gw szolgáltatás C++-ban: protokollfordítás MQTT-re JSON-írójából származik, és a `"`, a `\`, valamint a vezérlőkarakterek escape-elését végzi. Ez azért kötelező, mert egy hibaüzenet tartalmazhat idézőjelet vagy sortörést; escape nélkül a naplósor nem lenne érvényes JSON, és a journald-alapú feldolgozó eldobná. A naplózó önálló, g++-os tesztjének tényleges kimenete:

```
{"ts":"2026-10-31T16:13:13.101Z","level":"info","msg":"quote=\" and backslash=\\ and
↪  newline=\n"}
{"ts":"2026-10-31T16:13:13.101Z","level":"warn","msg":"ez latszik"}
```

A `set_log_level("warn")` után az `info` szintű sor már nem jelent meg, a `warn` igen. Az időbélyeg futásonként változik; a fenti érték egy helyi, Paho nélküli tesztből származik. A naplószint a TOML `log_level` mezőjéből vagy a `TS_GW_LOG_LEVEL` környezeti változóból állítható.

A journald-ben a sorok a `MESSAGE` mezőben jelennek meg; a rendszerd metaadatai (unit, PID, prioritás) külön mezők. A napló olvasása:

```
journalctl -u ts-gw -b -o cat
journalctl -u ts-gw -b -f
```

Az `-o cat` a JSON-sort adja vissza, a `-f` pedig élőben követi. A JSON-t külön eszközzel, például `jq`-val lehet szűrni, de a gatewaynek nem kell naplófájlt kezelnie; a fájlforgatás és a megőrzés a journald feladata.

12.4.4.3 Progress-alapú watchdog

A watchdog két számlálót kap: a fő ciklusét és az UDP-bemenetét. A `Watchdog` a `WatchdogSec` felének megfelelő periódussal ellenőriz; ha mindkét számláló elmozdult, elküldi a `WATCHDOG=1`-et, ha valamelyik megakadt, visszatartja. A rendszerd a `WatchdogSec` lejárta után `SIGTERM`-mel, majd szükség esetén `SIGKILL`-lel leállítja és újraindítja a folyamatot.

```
// src/Watchdog.h
#pragma once
#include <atomic>
#include <chrono>
#include <cstdint>
#include <functional>
#include <thread>

namespace terrasense {

// A watchdog csak akkor kuldi a WATCHDOG=1 uzenetet, ha a figyelt
// progress-szamlalok az elozo ellenorzes ota elmozdultak. Igy egy elakadt
// fő ciklus vagy input szál nem kap vakon életjelet.
class Watchdog {
public:
  using TickSource = std::function<std::uint64_t()>;

  Watchdog(std::chrono::milliseconds interval, TickSource main_ticks,
           TickSource input_ticks);
  ~Watchdog();
  Watchdog(const Watchdog&) = delete;
  Watchdog& operator=(const Watchdog&) = delete;

  void start();
  void stop();

private:
  void run();

  std::chrono::milliseconds interval_;
  TickSource main_ticks_;
  TickSource input_ticks_;
  std::atomic<bool> running_{false};
  std::thread worker_;
};

} // namespace terrasense

// src/Watchdog.cpp
#include "Watchdog.h"
#include "Logger.h"
#include <chrono>
#include <stdexcept>
#include <systemd/sd-daemon.h>
```

```
#include <thread>
#include <utility>

namespace terrasense {

Watchdog::Watchdog(std::chrono::milliseconds interval, TickSource main_ticks,
                   TickSource input_ticks)
    : interval_(interval),
      main_ticks_(std::move(main_ticks)),
      input_ticks_(std::move(input_ticks)) {
  if (interval_.count() <= 0) {
    throw std::invalid_argument("watchdog interval must be positive");
  }
  if (!main_ticks_ || !input_ticks_) {
    throw std::invalid_argument("watchdog tick sources must be set");
  }
}

Watchdog::~Watchdog() { stop(); }

void Watchdog::start() {
  if (running_.exchange(true)) {
    return;
  }
  worker_ = std::thread(&Watchdog::run, this);
}

void Watchdog::stop() {
  running_.store(false);
  if (worker_.joinable()) {
    worker_.join();
  }
}

void Watchdog::run() {
  std::uint64_t last_main = main_ticks_();
  std::uint64_t last_input = input_ticks_();
  const auto sleep_for = interval_ / 2;
  bool healthy = true;
  while (running_.load()) {
    std::this_thread::sleep_for(sleep_for);
    const std::uint64_t current_main = main_ticks_();
    const std::uint64_t current_input = input_ticks_();
    const bool main_progress = current_main != last_main;
    const bool input_progress = current_input != last_input;
    if (main_progress && input_progress) {
      if (!healthy) {
        log_json("info", "watchdog: progress resumed");
        sd_notify(0, "STATUS=running");
        healthy = true;
      }
      if (sd_notify(0, "WATCHDOG=1") < 0) {
        log_json("warn", "sd_notify(WATCHDOG=1) failed");
      }
    } else {
      if (healthy) {
        log_json("warn", "watchdog: no progress, withholding WATCHDOG=1");
        sd_notify(0, "STATUS=no progress");
        healthy = false;
      }
    }
    last_main = current_main;
    last_input = current_input;
```

```
  }
}

} // namespace terrasense
```

A watchdog-periódus nem fix 15 másodperc: a `main.cpp` a rendszerd által beállított `WATCHDOG_USEC` környezeti változóból számolja a `WatchdogSec` felét, és 100 ms és 300 s között korlátozza. Ha nincs `WATCHDOG_USEC`, a 15 másodperces alapérték érvényes. A `WatchdogSec=30s` tehát 15 másodperces ellenőrzést jelent; ha a uniton később 60 másodpercre növeljük a `WatchdogSec`-et, a kód változatlan marad, és az ellenőrzés 30 másodpercesre vált.

A watchdog-szál csak akkor indul el, ha a `NOTIFY_SOCKET` környezeti változó létezik, azaz a folyamatot rendszerd indította. Konténerben, ahol nincs rendszerd, a `sd_notify` hívás no-op lenne; a felesleges szál helyett a program naplózza, hogy foreground/container módban fut, és a watchdog kikapcsol. A konténer életképességét a Docker `restart` politikája és a folyamat kilépési kódja kezeli.

12.4.4.4 A main integrációja

A `main.cpp` a korábbi Store-and-forward: offline működés és visszatöltés alfejezetbeli változat kibővítése. A naplózás minden korábbi `std::cout`-ot kivált; a spool, a normalizálás, a deduplikáció és a visszatöltés változatlan. A watchdog a spool és az UDP-bemenet elindítása után kap életjelet, a `READY=1` pedig akkor megy ki, amikor a tartós puffer, az UDP-bemenet és a watchdog már aktív. A broker elérhetetlensége nem akadálya a `READY=1`-nek, mert a gateway offline is gyűjti az adatot.

```
// src/main.cpp
#include "Config.h"
#include "Json.h"
#include "Logger.h"
#include "MqttPublisher.h"
#include "Normalizer.h"
#include "TelemetryTypes.h"
#include "UdpInputAdapter.h"
#include "Watchdog.h"
#include "store_forward.hpp"
#include <algorithm>
#include <atomic>
#include <charconv>
#include <chrono>
#include <csignal>
#include <cstdint>
#include <cstdlib>
#include <exception>
#include <stdexcept>
#include <filesystem>
#include <optional>
#include <string>
#include <systemd/sd-daemon.h>
#include <thread>

using namespace terrasense;

namespace {

std::atomic<bool> g_running{true};
std::atomic<std::uint64_t> g_main_loop_ticks{0};

void handle_signal(int) { g_running.store(false); }
```

```
int64_t now_ms() {
  using namespace std::chrono;
  return duration_cast<milliseconds>(system_clock::now().time_since_epoch())
      .count();
}

std::chrono::milliseconds watchdog_interval_from_env() {
  using namespace std::chrono;
  const char* raw = std::getenv("WATCHDOG_USEC");
  if (raw == nullptr) {
    return milliseconds(15000);
  }
  std::uint64_t usec = 0;
  const std::string text(raw);
  const auto result = std::from_chars(text.data(), text.data() + text.size(),
                                      usec);
  if (result.ec != std::errc{} || result.ptr != text.data() + text.size() ||
      usec == 0) {
    return milliseconds(15000);
  }
  const auto half_ms = static_cast<std::uint64_t>(usec / 2000);
  return milliseconds(std::clamp<std::uint64_t>(half_ms, 100, 300000));
}

} // namespace

int main(int argc, char** argv) {
  Config config;
  bool help = false;
  std::string err;
  if (!load_config(argc, argv, config, help, err)) {
    log_json("error", "configuration: " + err);
    return 2;
  }
  if (help) {
    print_usage(argv[0]);
    return 0;
  }
  set_log_level(config.log_level);
  std::signal(SIGINT, handle_signal);
  std::signal(SIGTERM, handle_signal);

  try {
    const std::filesystem::path spool_path(config.spool_path);
    const std::filesystem::path parent = spool_path.parent_path();
    if (!parent.empty()) {
      std::error_code ec;
      std::filesystem::create_directories(parent, ec);
      if (ec) {
        throw std::runtime_error("cannot create spool directory " +
                                 parent.string() + ": " + ec.message());
      }
    }
    StoreForwardDb spool(config.spool_path, config.max_spool_records);
    spool.initialize();
    log_json("info", "spool " + config.spool_path +
                         " max_records=" +
                         std::to_string(config.max_spool_records) +
                         " current=" + std::to_string(spool.count()));

    Normalizer normalizer;
    MqttPublisher publisher(config.broker_uri, config.client_id);
```

```
UdpInputAdapter udp(config.udp_bind, config.udp_port);

log_json("info", "config broker=" + config.broker_uri + " udp=" +
                     config.udp_bind + ":" +
                     std::to_string(config.udp_port) + " qos=" +
                     std::to_string(config.qos) + " log=" +
                     config.log_level);
if (publisher.connect(err)) {
  log_json("info", "MQTT connected");
} else {
  log_json("warn", "MQTT unavailable: " + err +
                       "; input continues into the durable spool");
}

std::uint64_t last_dropped = spool.dropped_total();
if (!udp.start(
        [&](std::string raw) {
          TelemetrySample sample;
          std::string parse_error;
          if (!normalizer.parse(raw, sample, parse_error)) {
            log_json("warn", "rejected frame: " + parse_error);
            return;
          }
          std::string duplicate_error;
          if (normalizer.is_duplicate(sample, duplicate_error)) {
            log_json("warn", "duplicate " + sample.site + "/" +
                                 sample.node_id + " boot=" +
                                 std::to_string(sample.boot_id) +
                                 " seq=" + std::to_string(sample.seq) +
                                 ": " + duplicate_error);
            return;
          }
          std::string payload;
          std::string json_error;
          if (!to_json(sample, payload, json_error)) {
            log_json("warn", "serialization failed: " + json_error);
            return;
          }
          const std::string topic = "terrasense/" + sample.site + "/" +
                                    sample.node_id + "/telemetry";
          bool inserted = false;
          try {
            spool.enqueue(sample.site, sample.node_id, sample.boot_id,
                          sample.seq, sample.ts_epoch_ms, topic, payload,
                          inserted);
          } catch (const std::exception& e) {
            log_json("error", "spool enqueue failed: " +
                                  std::string(e.what()));
            return;
          }
          // A tartós spoolba kerülés a dedup elfogadási pontja: innen a
          // seq commitálható, függetlenül a broker elérhetőségétől.
          normalizer.commit_seq(sample);
          if (inserted) {
            log_json("info", "spooled " + topic +
                                 " seq=" + std::to_string(sample.seq));
          } else {
            log_json("info", "duplicate already in spool " + topic +
                                 " seq=" + std::to_string(sample.seq));
          }
          const std::uint64_t dropped = spool.dropped_total();
          if (dropped != last_dropped) {
            log_json("warn", "spool limit reached; dropped=" +
```

```
                                std::to_string(dropped));
            last_dropped = dropped;
          }
        },
        err)) {
  throw std::runtime_error("UDP adapter: " + err);
}
log_json("info", "UDP listening on " + config.udp_bind + ":" +
                     std::to_string(config.udp_port));

// A READY=1 akkor megy ki, amikor a tartós puffer és az UDP-bemenet már
// elindult. A brokerhoz csatlakozás nem feltétele a ready állapotnak,
// mert a gateway offline is gyűjti az adatot a spoolba.
const bool notify_enabled = std::getenv("NOTIFY_SOCKET") != nullptr;
std::optional<Watchdog> watchdog;
if (notify_enabled) {
  const auto interval = watchdog_interval_from_env();
  log_json("info", "systemd watchdog enabled interval_ms=" +
                       std::to_string(interval.count()));
  watchdog.emplace(interval,
                   [] { return g_main_loop_ticks.load(); },
                   [&udp] { return udp.loop_ticks(); });
  watchdog->start();
  // A READY=1 azt jelenti, hogy a tartós puffer, az UDP-bemenet és a
  // watchdog-felügyelet elindult. A brokerhoz csatlakozás nem feltétele,
  // mert a gateway offline is gyűjti az adatot a spoolba.
  sd_notify(0, "READY=1");
} else {
  log_json("info", "NOTIFY_SOCKET not set; foreground/container mode, "
                   "systemd watchdog disabled");
}

int backoff_ms = 1000;
int64_t next_reconnect_ms = 0;
constexpr std::size_t kBatchSize = 50;
while (g_running.load()) {
  g_main_loop_ticks.fetch_add(1);
  const int64_t now = now_ms();
  if (!publisher.is_connected() && now >= next_reconnect_ms) {
    if (publisher.connect(err)) {
      log_json("info", "MQTT connected");
      backoff_ms = 1000;
      next_reconnect_ms = 0;
    } else {
      log_json("warn", "MQTT reconnect failed: " + err);
      next_reconnect_ms = now + backoff_ms;
      backoff_ms = std::min(backoff_ms * 2, 8000);
    }
  }
  if (!publisher.is_connected()) {
    std::this_thread::sleep_for(std::chrono::milliseconds(200));
    continue;
  }

  auto records = spool.pending(kBatchSize);
  if (records.empty()) {
    std::this_thread::sleep_for(std::chrono::milliseconds(200));
    continue;
  }
  bool publish_failed = false;
  for (const auto& record : records) {
    g_main_loop_ticks.fetch_add(1);
    if (!g_running.load()) {
```

```
          break;
        }
        if (!publisher.publish(record.topic, record.payload, config.qos, err)) {
          log_json("warn", "publish failed for spool id=" +
                           std::to_string(record.id) + ": " + err +
                           "; record stays in spool");
          publisher.disconnect();
          next_reconnect_ms = 0;
          publish_failed = true;
          break;
        }
        if (!spool.remove(record.id)) {
          log_json("warn", "spool id=" + std::to_string(record.id) +
                           " disappeared before removal");
        }
        log_json("info", "forwarded spool id=" + std::to_string(record.id) +
                         " " + record.topic + " boot=" +
                         std::to_string(record.boot_id) + " seq=" +
                         std::to_string(record.seq));
        std::this_thread::sleep_for(std::chrono::milliseconds(100));
      }
      if (publish_failed) {
        continue;
      }
    }

    if (watchdog.has_value()) {
      watchdog->stop();
    }
    udp.stop();
    if (notify_enabled) {
      sd_notify(0, "STOPPING=1");
    }
    log_json("info", "gateway stopped; spool=" + std::to_string(spool.count()) +
                     " dropped=" +
                     std::to_string(spool.dropped_total()));
  } catch (const std::exception& e) {
    log_json("error", "fatal: " + std::string(e.what()));
    return 1;
  }
  return 0;
}
```

A fő ciklus minden iterációban és minden feldolgozott spool-rekordnál növeli a `g_main_loop_ticks` számlálót. Ez fontos: egy hosszú visszatöltési sorozat közben is látszik az előrehaladás, miközben egy SQLite-zár vagy egy végtelen ciklus valóban megakasztja a számlálót. Az UDP-szál `loop_ticks()` számlálója minden `poll` körben nő, akkor is, ha éppen nincs datagram; a szál életképességét tehát nem az üzenetforgalom, hanem a ciklus előrehaladása igazolja.

12.4.4.5 Build és a rendszerd-függőség

A hoston a következő csomagok kellenek. A Paho C++ fejlesztői csomagja mellett a `libsqlite3-dev` a spoolhoz, a `libsystemd-dev` az `sd_notify` fejlécéhez és linkeléséhez szükséges. A csomagnevek Ubuntu 24.04-re vonatkoznak; a laborban az `apt-cache policy` parancs megerősíti őket.

```
sudo apt-get update
sudo apt-get install -y build-essential cmake pkg-config \
  libpaho-mqttpp-dev libsqlite3-dev libsystemd-dev
```

A `CMakeLists.txt` a korábbi Store-and-forward: offline működés és visszatöltés alfejezetbeli változat kibővítése a `Logger.cpp` és a `Watchdog.cpp` forrásokkal, valamint a `libsystemd` pkg-config modullal:

```
cmake_minimum_required(VERSION 3.28)
project(ts-gw CXX)

set(CMAKE_CXX_STANDARD 17)
set(CMAKE_CXX_STANDARD_REQUIRED ON)
set(CMAKE_CXX_EXTENSIONS OFF)

find_package(PkgConfig REQUIRED)
pkg_check_modules(PAHO_MQTTPP3 REQUIRED IMPORTED_TARGET paho-mqttpp3)
pkg_check_modules(SQLITE3 REQUIRED IMPORTED_TARGET sqlite3)
pkg_check_modules(SYSTEMD REQUIRED IMPORTED_TARGET libsystemd)
find_package(Threads REQUIRED)

add_executable(ts-gw
  src/main.cpp
  src/Config.cpp
  src/Normalizer.cpp
  src/UdpInputAdapter.cpp
  src/MqttPublisher.cpp
  src/store_forward.cpp
  src/Logger.cpp
  src/Watchdog.cpp)

target_include_directories(ts-gw PRIVATE src)
target_link_libraries(ts-gw PRIVATE
  PkgConfig::PAHO_MQTTPP3
  PkgConfig::SQLITE3
  PkgConfig::SYSTEMD
  Threads::Threads)
target_compile_options(ts-gw PRIVATE -Wall -Wextra -Wpedantic)
```

A build és egy foreground próba:

```
cmake -S . -B build -DCMAKE_BUILD_TYPE=Release
cmake --build build -j"$(nproc)"
./build/ts-gw --config config/ts-gw.toml --spool-path ./var/spool.db
```

A `--spool-path ./var/spool.db` azért kell, mert a TOML éles útvonalat (`/var/lib/ts-gw/spool.db`) tartalmaz, a laborban viszont nem root felhasználóként futtatjuk a gatewayt. A parancssori felülírás a prioritási sorrend utolsó eleme, ezért érvényesül. A broker elérhetetlensége ebben a módban nem állítja le a programot: a naplóban `MQTT unavailable` látható, az UDP-bemenet pedig a spoolba ír.

12.4.4.6 systemd-unit és telepítés

A unit a következő. A `Type=notify`, a `WatchdogSec=30s`, a `StateDirectory=ts-gw` és a `User=tsgw` együtt biztosítja, hogy a folyamat ne rootként fusson, a spool a megfelelő helyre kerüljön, és a rendszerd a `READY=1` valamint a `WATCHDOG=1` üzeneteket figyelje.

```
# deploy/ts-gw.service
[Unit]
Description=TerraSense Gateway (ts-gw)
Documentation=man:ts-gw(1)
After=network-online.target
Wants=network-online.target
StartLimitIntervalSec=60
```

```
StartLimitBurst=3

[Service]
Type=notify
NotifyAccess=main
ExecStart=/usr/local/bin/ts-gw --config /etc/ts-gw/config.toml
Restart=on-failure
RestartSec=5s
WatchdogSec=30s
TimeoutStartSec=30s
TimeoutStopSec=20s
KillSignal=SIGTERM
User=tsgw
Group=tsgw
StateDirectory=ts-gw
StateDirectoryMode=0750
WorkingDirectory=/var/lib/ts-gw
Environment=TS_GW_SPOOL_PATH=/var/lib/ts-gw/spool.db
EnvironmentFile=-/etc/default/ts-gw
UMask=0027
NoNewPrivileges=true
ProtectSystem=strict
ProtectHome=true
PrivateTmp=true
PrivateDevices=true
ProtectKernelTunables=true
ProtectKernelModules=true
ProtectControlGroups=true
RestrictAddressFamilies=AF_UNIX AF_INET AF_INET6
LockPersonality=true
CapabilityBoundingSet=
AmbientCapabilities=
MemoryMax=256M
CPUQuota=50%
TasksMax=64
LimitNOFILE=4096

[Install]
WantedBy=multi-user.target
```

Az `Environment=TS_GW_SPOOL_PATH=...` felülírja a TOML-ban szereplő útvonalat, és a `StateDirectory` által létrehozott, a `tsgw` felhasználó számára írható könyvtárra mutat. A `WorkingDirectory` ugyanez a könyvtár, így a relatív útvonalak is kiszámíthatók. A `ProtectSystem=strict` a rendszer nagy részét csak olvashatóvá teszi; a `StateDirectory` kivétel, ezért a spool írható marad. A `ProtectHome=true` miatt a szolgáltatás nem látja a `/home` alatti fájlokat, ezért a binárist nem a labor `~/iot-labor` könyvtárából, hanem a verziózott `/opt/ts-gw` alól kell futtatni.

Az opcionális környezeti fájl:

```
# deploy/ts-gw.default
# A konfigurációs fájlt felülbíráló környezeti változók.
TS_GW_LOG_LEVEL=info
```

A telepítés lépései. A felhasználót és a csoportot csak akkor hozzuk létre, ha még nem léteznek:

```
getent group tsgw >/dev/null || sudo groupadd --system tsgw
id -u tsgw >/dev/null 2>&1 || sudo useradd --system --gid tsgw \
  --home-dir /var/lib/ts-gw --no-create-home \
  --shell /usr/sbin/nologin tsgw

sudo install -d -m 0755 /etc/ts-gw /opt/ts-gw/v1.3.0
```

```
sudo install -m 0644 config/ts-gw.toml /etc/ts-gw/config.toml
sudo install -m 0644 deploy/ts-gw.service /etc/systemd/system/ts-gw.service
sudo install -m 0644 deploy/ts-gw.default /etc/default/ts-gw
sudo install -m 0755 build/ts-gw /opt/ts-gw/v1.3.0/ts-gw
sudo ln -sfn /opt/ts-gw/v1.3.0/ts-gw /usr/local/bin/ts-gw
sudo systemctl daemon-reload
sudo systemctl enable --now ts-gw
```

A `/usr/local/bin/ts-gw` szimbolikus link a verziózott binárisra mutat; a frissítés és a rollback ezt a linket állítja át. A `StateDirectory=ts-gw` a unit első indításakor létrehozza a `/var/lib/ts-gw` könyvtárat a `tsgw:tsgw` tulajdonossal és `0750` módban. Ellenőrzés:

```
systemctl status ts-gw --no-pager
systemctl show ts-gw -p WatchdogUSec -p StateDirectory -p User
journalctl -u ts-gw -b -o cat -n 30
systemd-analyze verify /etc/systemd/system/ts-gw.service
```

A `systemctl show` a `WatchdogUSec=30s` értéket mutatja; ebből a program a `WATCHDOG_USEC` környezeti változón keresztül 15 másodperces ellenőrzési periódust számol. A `systemd-analyze verify` szintaktikai ellenőrzést végez; a tényleges indítás a `systemctl start ts-gw` után a journald-ben követhető.

12.4.4.7 Docker-image többlépcsős builddel

A Dockerfile az építő image-ben fordít, a futásidejű image-be csak a binárist és a szükséges futásidejű könyvtárakat másolja. A Paho C++ 1.3.2 és a SQLite rendszerkönyvtárai a futásidejű image-ben is jelen vannak; a `libsystemd0` az `sd_notify` szimbólumhoz kell, még akkor is, ha a konténerben nincs rendszerd, mert a bináris linkelése ezt igényli.

```
# Dockerfile
# syntax=docker/dockerfile:1
FROM ubuntu:24.04 AS build
ENV DEBIAN_FRONTEND=noninteractive
RUN apt-get update && apt-get install -y --no-install-recommends \
    build-essential cmake pkg-config \
    libpaho-mqttpp-dev libsqlite3-dev libsystemd-dev \
 && rm -rf /var/lib/apt/lists/*
WORKDIR /src
COPY CMakeLists.txt ./
COPY src ./src
RUN cmake -S . -B build -DCMAKE_BUILD_TYPE=Release \
 && cmake --build build -j"$(nproc)"

FROM ubuntu:24.04 AS runtime
ENV DEBIAN_FRONTEND=noninteractive
RUN apt-get update && apt-get install -y --no-install-recommends \
    libpaho-mqttpp3-1 libsqlite3-0 libsystemd0 ca-certificates \
 && rm -rf /var/lib/apt/lists/* \
 && groupadd --system tsgw \
 && useradd --system --gid tsgw --home-dir /var/lib/ts-gw \
      --no-create-home --shell /usr/sbin/nologin tsgw \
 && install -d -o tsgw -g tsgw -m 0750 /var/lib/ts-gw \
 && install -d -m 0755 /etc/ts-gw
COPY --from=build /src/build/ts-gw /usr/local/bin/ts-gw
COPY config/ts-gw.toml /etc/ts-gw/config.toml
USER tsgw
WORKDIR /var/lib/ts-gw
EXPOSE 4000/udp
ENTRYPOINT ["/usr/local/bin/ts-gw"]
CMD ["--config", "/etc/ts-gw/config.toml"]
```

A `.dockerignore` csökkenti a build kontextusát, és megakadályozza, hogy a helyi spool vagy a build könyvtár bekerüljön az image-be:

```
# .dockerignore
build/
var/
*.db
*.o
.git/
```

A futásidejű image-ben a `TS_GW_SPOOL_PATH` a `/var/lib/ts-gw/spool.db`, a `TS_GW_UDP_BIND` pedig `0.0.0.0`. Utóbbi kötelező: a `127.0.0.1` csak a konténer saját hálózati névterében lenne elérhető, más konténerek nem tudnák elérni a gatewayt. A Compose a hálózatot a szolgáltatások között hozza létre, ezért a `lab-broker` DNS-név a `ts-gw` konténerből feloldható.

12.4.4.8 Compose-integráció

A `~/iot-labor/stack/compose.yaml` fájlban a broker szolgáltatás neve a 4. fejezet óta `lab-broker`, így a `ts-gw` `tcp://lab-broker:1883` címe közvetlenül feloldódik. A következő részlet a meglévő fájlba illesztendő; a top-level `volumes` szakasz a fájl végére kerül.

```yaml
services:
  lab-broker:
    image: eclipse-mosquitto:2.0.18
    container_name: lab-broker
    ports:
      - "127.0.0.1:1883:1883"
    volumes:
      - ./mosquitto/mosquitto.conf:/mosquitto/config/mosquitto.conf:ro
    healthcheck:
      test: ["CMD", "mosquitto_sub", "-t", "$$SYS/#", "-C", "1", "-W", "3"]
      interval: 10s
      timeout: 5s
      retries: 5
      start_period: 5s

  ts-gw:
    build:
      context: ../i038/ts-gw
    container_name: ts-gw
    environment:
      TS_GW_MQTT_BROKER: tcp://lab-broker:1883
      TS_GW_UDP_BIND: 0.0.0.0
      TS_GW_UDP_PORT: "4000"
      TS_GW_SPOOL_PATH: /var/lib/ts-gw/spool.db
      TS_GW_LOG_LEVEL: info
    ports:
      - "4000:4000/udp"
    volumes:
      - ts-gw-spool:/var/lib/ts-gw
    depends_on:
      lab-broker:
        condition: service_healthy
    restart: unless-stopped
    mem_limit: 256m
    cpus: 0.5
    pids_limit: 128
    stop_grace_period: 20s

volumes:
  ts-gw-spool:
```

A `depends_on condition: service_healthy` beállítása megvárja, hogy a broker healthcheckje sikeres legyen. Ez nem helyettesíti az MQTT-kliens újracsatlakozási logikáját: ha a broker később újraindul, a `MqttPublisher` a spoolban lévő rekordokkal együtt újracsatlakozik. A `restart: unless-stopped` a konténer szintű újraindítási politika; a systemd `Restart=on-failure` beállításának konténeres megfelelője.

Build és futtatás:

```
cd ~/iot-labor/stack
docker compose up -d --build ts-gw
docker compose ps
docker compose logs -f ts-gw
```

A várható, szemléltető naplórészlet (az időbélyeg és a spool aktuális száma eltérhet):

```
{"ts":"...","level":"info","msg":"config broker=tcp://lab-broker:1883 udp=0.0.0.0:4000 qos=1
↪  log=info"}
{"ts":"...","level":"info","msg":"UDP listening on 0.0.0.0:4000"}
{"ts":"...","level":"info","msg":"NOTIFY_SOCKET not set; foreground/container mode, systemd
↪  watchdog disabled"}
```

A konténerben a `NOTIFY_SOCKET` hiánya miatt a watchdog-szál nem indul el; ez szándékos. A `READY=1` helyett a Docker a konténer futását figyeli, a folyamat összeomlását pedig a `restart` politika kezeli. A `ts-gw` nem nyújt HTTP health végpontot; a mélyebb, alkalmazásszintű életképesség-ellenőrzés a Megfigyelhetőség végponttól végpontig alfejezet témája.

A konténer és a broker kapcsolatának ellenőrzése:

```
docker compose exec ts-gw getent hosts lab-broker
docker compose exec ts-gw env | grep '^TS_GW_'
docker compose port ts-gw 4000/udp
```

A `getent hosts lab-broker` a Compose-hálózat DNS-feloldását igazolja. A `docker compose port` a host felé publikált UDP-portot mutatja. A konténerből konténerbe irányuló UDP-próba a `lab-broker` image-ben elérhető `nc` segítségével:

```
docker compose run --rm --no-deps --entrypoint sh lab-broker -c \
  'printf "site=gh1;node=n001;boot=7;seq=500;T=22.0;RH=50;SM=30" \
   | nc -u -w1 ts-gw 4000'
```

Sikeres próba esetén a `ts-gw` naplójában `spooled terrasense/gh1/n001/telemetry seq=500` jelenik meg. A `send_udp.py` továbbra is használható a hostról a publikált `127.0.0.1:4000/udp` porton keresztül.

12.4.4.9 Frissítés és visszaállítás

Natív systemd-telepítésnél a binárist verziózott könyvtárakba telepítjük, és a `/usr/local/bin/ts-gw` szimbolikus link mutat az aktív verzióra. Frissítéskor az új binárist például a `/opt/ts-gw/v1.4.0/` könyvtárba másoljuk, majd `sudo ln -sfn /opt/ts-gw/v1.4.0/ts-gw /usr/local/bin/ts-gw` és `sudo systemctl restart ts-gw`. Ha az új verzió nem indul el, a linket visszaállítjuk az előző verzióra, és újraindítjuk a szolgáltatást. A spool a `/var/lib/ts-gw` alatt marad, ezért a visszaállítás nem veszíti el a még el nem küldött rekordokat. Ez az A/B vagy szimbolikus linkes rollback az OTA-frissítés: A/B képek, rollback, delta, fokozatos kiadás alfejezetben tér vissza részletesen.

Konténeres telepítésnél a rollback az image-tag cseréje: a `ts-gw` szolgáltatást korábbi taggel indítjuk újra, például `ts-gw:1.2.0` helyett `ts-gw:1.1.0`, majd `docker compose up -d ts-gw`. A névvel ellátott `ts-gw-spool` kötet megmarad, ezért a visszaállított image ugyanazt a spoolt

látja. A spool séma visszafelé kompatibilisnek kell maradnia; ha az új verzió migrációt vezet be, a rollback előtt azt külön kell kezelni.

12.4.5 Ellenőrzés és hibakeresés

Továbbvitt eredmény. A `ts-gw.service` unit, a `Dockerfile` és a Compose-részlet a Megfigyelhetőség végponttól végpontig: metrikák, naplók, nyomkövetés alfejezetben a log- és metrikagyűjtés alapjául szolgál; a frissítési stratégia az OTA-frissítés: A/B képek, rollback, delta, fokozatos kiadás alfejezetbe csatlakozik, a teljes stack pedig a Záróprojekt: a teljes TerraSense rendszer összeállítása és átvétele alfejezetben áll össze.

A következő ellenőrzések a laborban elvégzendők. A kézirat készítésekor a Paho, a SQLite és a rendszerd fejlesztői csomagjai nem voltak telepítve, ezért a teljes gateway buildje és a rendszerd/Docker labor nem futott le; a források statikus, helyi stub-fordítása és a konfiguráció, valamint a naplózó önálló tesztje igen. A táblázat jelöli, melyik eredmény helyi teszt és melyik laborfeladat.

Ellenőrzés	Módszer	Elvárt eredmény	Státusz
TOML tényleg hat	`config/ts-gw.toml` `udp_port` átírása, majd foreground indítás	a `config ... udp=...` naplósorban az új port	laborfeladat
Prioritási sorrend	TOML + `TS_GW_UDP_PORT` + `--udp-port`	4000 → 4100 → 4200	helyi `load_config` teszt
JSON escape	idézőjel, backslash és sortörés a `msg`-ben	érvényes JSON, `\"`, `\\`, `\n`	helyi `Logger` teszt
Naplószint	`set_log_level("warn")` után `info`, majd `warn`	az `info` elmarad, a `warn` megjelenik	helyi `Logger` teszt
Watchdog előrehaladás	az egyik tickszámláló leállítása a tesztben	`WATCHDOG=1` visszatartva, `watchdog: no progress`	helyi `Watchdog` teszt
Watchdog rendszerd alatt	a `main` ciklusába ideiglenesen `sleep(40s)` beiktatása, majd `systemctl restart ts-gw`	30 s után rendszerd-újraindítás, journald-bejegyzés	laborfeladat
Konténer UDP	`nc -u` próba a `lab-broker` konténerből	`spooled ... seq=500` a `ts-gw` naplójában	laborfeladat
Broker DNS	`docker compose exec ts-gw getent hosts lab-broker`	a broker konténer IP-címe	laborfeladat

A watchdog helyi tesztjében a fő- és az input-számláló eleinte együtt nőtt, majd az input-számláló leállt. A `WATCHDOG=1` hívások száma a következőképpen alakult: az első fázisban 6, a határértéken átcsúszó ellenőrzés miatt a második fázis elején még 7, a harmadik fázisban, az input újraindulása után 13. A lényeg a második fázisban a `watchdog: no progress, withholding WATCHDOG=1`

naplósor és az, hogy a steady state alatt nem érkezett újabb `WATCHDOG=1`. A rendszerd-es próbát a labornak kell elvégeznie; a `sleep(40s)` ideiglenes módosítás, amelyet a teszt után el kell távolítani.

A rendszerd-es watchdog-próba menete:

1. A laborban a `main.cpp` fő ciklusának elejére ideiglenesen tegyünk egy `std::this_thread::sleep_for(std` sort.
2. `cmake --build build`, majd a bináris telepítése és `sudo systemctl restart ts-gw`.
3. `journalctl -u ts-gw -f -o cat` figyelése. A watchdog 15 másodperces ellenőrzései először `watchdog: no progress, withholding WATCHDOG=1`-et írnak, majd 30 másodperc után a rendszerd leállítja és újraindítja a szolgáltatást.
4. A teszt után a `sleep` sort el kell távolítani, újra kell építeni, és a szolgáltatást újraindítani.

A hibakeresési táblázat:

Hibatünet	Lehetséges ok	Javítás
A szolgáltatás nem indul el, `status=203/EXEC`	rossz bináris útvonal, hiányzó jogosultság vagy nem létező `User`	ellenőrizzük az `ExecStart` és a `/usr/local/bin/ts-gw` linket, a `tsgw` felhasználót és a `journalctl -u ts-gw -b` kimenetét
A watchdog újraindítja a folyamatot	a fő ciklus vagy az UDP-szál nem halad előre, esetleg a `WATCHDOG=1` nem érkezik	nézzük a `watchdog: no progress` bejegyzést, a `WATCHDOG_USEC` értékét és a spool/SQLite zárakat; a vak watchdog helyett a progress-számlálókat kell javítani
A JSON-naplósorok nem jelennek meg a journald-ben	a program fájlba ír, vagy a kimenet nincs a stdouton	a `log_json` a `std::cout`-ra írjon; ellenőrizzük a `StandardOutput` beállítást és a `journalctl -u ts-gw -o cat` kimenetét
A Docker-konténer nem éri el a brokert	a `TS_GW_MQTT_BROKER` értéke `localhost`, vagy a szolgáltatásnév nem `lab-broker`	Compose-ban `tcp://lab-broker:1883`, `getent hosts lab-broker`, a broker healthcheckje és a `depends_on service_healthy` feltétele
Más konténer nem éri el a gateway UDP-portját	a bind `127.0.0.1`, hiányzó portpublikálás vagy eltérő hálózat	`TS_GW_UDP_BIND=0.0.0.0`, `4000:4000/udp`, közös Compose-hálózat; ellenőrizzük a `docker compose port` kimenetét
A spool írása `SQLITE_CANTOPEN` hibával meghal	a `/var/lib/ts-gw` nem a `tsgw` tulajdona, vagy a `ProtectSystem` miatt nem írható	a `StateDirectory=ts-gw` és a `User=tsgw` együtt legyen beállítva; ellenőrizzük a `ls -ld /var/lib/ts-gw` tulajdonosát

Hibatünet	Lehetséges ok	Javítás
A TOML módosítása nem hat	a `--config` rossz fájlra mutat, vagy egy env-változó felülírja	`--config` útvonal, `TS_GW_*` változók, `journalctl` első `config ...` sora

12.4.6 Tipikus hibák és félreértések

1. **A `Type=notify` elhagyása watchdoggal együtt.** Ha a unit `Type=simple`, a rendszerd nem várja a `READY=1` üzenetet, és a `WatchdogSec` csak akkor működik, ha a program mégis küld `WATCHDOG=1`-et. A `Type=notify` és a watchdog együtt értelmes; a `READY=1` elmaradása esetén a rendszerd az indítást hibásnak látja.
2. **A watchdog „vakon” küldi az életjelet.** Ha a watchdog-szál függetlenül, a munkafolyamattól elszakadva küldi a `WATCHDOG=1`-et, egy teljesen megakadt gateway is egészségesnek látszik. A progress-számlálók ehhez kapcsolják az életjelet; a `Watchdog` csak akkor küld, ha a fő adatút és az UDP-bemenet is előrehaladt.
3. **JSON-naplózás escape nélkül.** Ha a `msg` idézőjelet vagy sortörést tartalmaz, és a naplózó nem escape-eli, a sor nem érvényes JSON. A `json_append_escaped()` használata kötelező; a `jq` és a journald-alapú feldolgozó is érvényes JSON-t vár.
4. **A rendszeróra és a naplóidő keverése.** A gateway helyi ideje és a UTC közötti különbség gyakran okoz sorrendi hibákat. Minden naplóbejegyzés UTC ISO 8601 formátumú legyen, ahogy a telemetria is.
5. **`localhost` a konténerben.** A `localhost` a konténer saját hálózati névterét jelenti, nem a hostot és nem a szomszédos konténert. A Compose-hálózatban a szolgáltatásnevet (`lab-broker`) kell használni; a `depends_on` önmagában csak az indítási sorrendet befolyásolja, a broker fogadókészségét a healthcheck `service_healthy` feltétele garantálja.
6. **A `ProtectHome=true` és a laborútvonal keverése.** A rendszerd unit szándékosan nem látja a `/home` alatti fájlokat. A binárist a `/opt/ts-gw` alól, a konfigurációt a `/etc/ts-gw` alól kell futtatni; a `~/iot-labor` csak a build és a kísérletezés helye.
7. **Erőforráskorlát nélküli konténer.** A `mem_limit` és a `cpus` elhagyása lehetővé teszi, hogy egy hibás gateway az egész edge-eszköz erőforrásait lefoglalja. A `pids_limit` a szálak és segédfolyamatok számát korlátozza; a `ts-gw` alapból kevés szálat használ, de egy hiba esetén a korlát véd.

12.4.7 Feladatok

1. **Fogalmi kérdés.** Miért előnyösebb a `Type=notify` a `Type=simple`-höz képest, ha watchdogot is használunk? Milyen hibát kerülünk el vele?
2. **Tervezési feladat.** Egy gateway szolgáltatás `WatchdogSec=30s`. Mekkora legyen a watchdog-ellenőrzések periódusa, és miért nem elég a `WatchdogSec` felét használni akkor, ha a fő ciklus egy hosszú spool-visszatöltés miatt több másodpercig nem ér körbe? Hogyan segít ezen a spool-rekordonkénti tickszámláló?

3. **Implementációs feladat.** Egészítse ki a `Watchdog` osztályt egy harmadik tickszámlálóval a MQTT-publikáló ciklus számára, és kösse be a `main.cpp`-ben. A konstruktor vegyen át egy harmadik `TickSource`-t, és csak akkor küldjön `WATCHDOG=1`-et, ha mindhárom számláló elmozdult. Írjon olyan helyi tesztet, amelyben a publikáló számláló leáll, és ellenőrizze, hogy a watchdog visszatartja az életjelet.

4. **Hibakeresési feladat.** A Docker-konténer `ts-gw` nem csatlakozik a `lab-broker` brokerhez. A logban a `connection refused` üzenet látható. Hogyan deríti ki, hogy a `TS_GW_MQTT_BROKER` környezeti változó helyes-e, a `lab-broker` név feloldódik-e, és a `depends_on` miért nem elég a kapcsolat biztosításához? Javítsa a Compose-fájlt, és adja meg a healthchecket.

12.4.8 Megoldások

1. A `Type=notify` esetén a rendszerd csak akkor tekinti a szolgáltatást sikeresen elindultnak, ha a folyamat explicit `READY=1` üzenetet küld. Ha a program elindul, de valamiért nem éri el a működő állapotot (például nem tud csatlakozni a brokerhez), a `Type=simple` mellett a rendszerd már sikeresnek látná, és nem indítaná újra. A `Type=notify` ezt a hibát felfedi, és a `Restart=on-failure` beavatkozik.

2. A watchdog-ellenőrzések periódusának kisebbnek kell lennie a `WatchdogSec` felénél, hogy egyetlen késés ne okozzon újraindítást. `WatchdogSec=30s` esetén 10–15 másodperc ésszerű. A hosszú spool-visszatöltés nem akadályozhatja meg az életjelet: a fő ciklus a spool-rekordonkénti tickszámlálót is növeli, így a watchdog látja az előrehaladást. Ha a visszatöltés egyetlen rekordnál akad meg, a tickszámláló sem nő, és a watchdog helyesen jelez.

3. A harmadik tickszámláló bevezetése a `Watchdog` osztály kibővítésével történik. A fejlécben a konstruktor és az adattag kap egy harmadik forrást:

```
// Watchdog.h (részlet)
using TickSource = std::function<std::uint64_t()>;

Watchdog(std::chrono::milliseconds interval, TickSource main_ticks,
         TickSource input_ticks, TickSource publish_ticks);
// ...
TickSource publish_ticks_;
```

A `Watchdog.cpp`-ben a konstruktor átveszi és eltárolja a harmadik forrást, a `run()` pedig mindhárom számlálót ellenőrzi:

```
// Watchdog.cpp (részlet)
Watchdog::Watchdog(std::chrono::milliseconds interval, TickSource main_ticks,
                   TickSource input_ticks, TickSource publish_ticks)
    : interval_(interval),
      main_ticks_(std::move(main_ticks)),
      input_ticks_(std::move(input_ticks)),
      publish_ticks_(std::move(publish_ticks)) {
  if (!main_ticks_ || !input_ticks_ || !publish_ticks_) {
    throw std::invalid_argument("watchdog tick sources must be set");
  }
}

void Watchdog::run() {
  std::uint64_t last_main = main_ticks_();
  std::uint64_t last_input = input_ticks_();
  std::uint64_t last_publish = publish_ticks_();
  const auto sleep_for = interval_ / 2;
  while (running_.load()) {
```

```
    std::this_thread::sleep_for(sleep_for);
    const std::uint64_t current_main = main_ticks_();
    const std::uint64_t current_input = input_ticks_();
    const std::uint64_t current_publish = publish_ticks_();
    const bool progressed =
        current_main != last_main && current_input != last_input &&
        current_publish != last_publish;
    if (progressed) {
      sd_notify(0, "WATCHDOG=1");
    } else {
      log_json("warn", "watchdog: no progress, withholding WATCHDOG=1");
    }
    last_main = current_main;
    last_input = current_input;
    last_publish = current_publish;
  }
}
```

A `main.cpp`-ben a publikáló számláló minden sikeres PUBACK után nő, és a watchdog létrehozásakor harmadikként kapja meg:

```
std::atomic<std::uint64_t> g_publish_ticks{0};

// a sikeres publish után:
g_publish_ticks.fetch_add(1);

// a watchdog létrehozásakor:
watchdog.emplace(interval,
                 [] { return g_main_loop_ticks.load(); },
                 [&udp] { return udp.loop_ticks(); },
                 [] { return g_publish_ticks.load(); });
```

A helyi tesztben a publikáló számlálót le kell állítani, miközben a másik kettő tovább nő; a várt eredmény a `watchdog: no progress, withholding WATCHDOG=1` naplósor, majd a publikáló ciklus újraindulása után a `watchdog: progress resumed`.

4. Ellenőrizzük a konténer környezeti változóját: `docker compose exec ts-gw env | grep TS_GW`. Ha a `TS_GW_MQTT_BROKER` értéke `localhost` vagy hiányzik, a konténer a saját hálózatát használja, nem a `lab-broker` szolgáltatást. A `getent hosts lab-broker` mutatja, hogy a név feloldható-e. A `depends_on` csak az indítási sorrendet biztosítja, nem várja meg, amíg a broker ténylegesen fogadja a kapcsolatokat. Megoldás: a broker szolgáltatásnév egységesítése `lab-broker`-re, `depends_on condition: service_healthy` beállítással, a broker `mosquitto_sub` healthcheckjével, valamint a `ts-gw` újracsatlakozási logikájának megtartása.

12.4.9 Rövid összefoglaló

A `ts-gw` üzemeltetéséhez systemd-unitot készítettünk `Type=notify`, `Restart=on-failure` és `WatchdogSec=30s` beállításokkal; a program `sd_notify`-on keresztül küld `READY=1`-et és csak előrehaladás esetén `WATCHDOG=1`-et. A naplózást JSON-sorokként a stdoutra írjuk, a `Logger` escape-eli a szöveget és a `log_level` szűri a kimenetet; a journald gyűjti a sorokat. A Docker-többlépcsős build külön építő és futásidejű konténert használ; a Compose-ban `mem_limit`, `cpus` és `pids_limit` erőforráskorlátokat adtunk meg, a broker healthcheckje és a `service_healthy` feltétel pedig a kapcsolat rendelkezésre állását segíti. A konfigurációt fájlból, környezeti változókból és parancssorból töltjük be, a prioritási sorrend alap, TOML, env, CLI. A frissítés és visszaállítás natív rendszeren szimbolikus linkkel, konténerben image-tag cserével történik. **Továbbvitt**

eredmények: `ts-gw.service`, `Dockerfile`, a Compose-részlet és a JSON-naplózási konvenció a Megfigyelhetőség, az OTA és a Záróprojekt alfejezetekhez csatlakozik.

12.4.10 Források és ellenőrzési hiányok

- systemd dokumentáció: `man systemd.service`, `man systemd.exec`, `man systemd.resource-control`, `man sd_notify`. Az elérhető opciók (például `MemoryMax`, `CPUQuota`, `StateDirectory`, `ProtectSystem`) pontos szintaxisa a rendszer verziójától függ; Ubuntu 24.04 rendszerd 255-ön ellenőrizendő.
- Docker Compose specifikáció: a `mem_limit`, `cpus`, `pids_limit` a Compose v2 nem-swarm mezői; a `deploy.resources.limits` a Swarm módhoz tartozik. A healthcheck `condition: service_healthy` csak a Compose v2-ben használható.
- Paho MQTT C++ csomagnevek Ubuntu 24.04-en: `libpaho-mqttpp-dev` (fejlesztői) és `libpaho-mqttpp3-1` (futásidejű); a laborban `apt-cache policy libpaho-mqttpp-dev libpaho-mqttpp3-1` ellenőrzi. A Paho C++ verziója a rögzített profil szerint 1.3.2.
- Az `sd_notify`, a `$NOTIFY_SOCKET` és a `WATCHDOG_USEC` viselkedését a `sd_notify(3)` és a `systemd.service(5)` man-oldal írja le.
- A könyv készítésekor a Paho, a SQLite és a rendszerd fejlesztői csomagjai nem voltak telepítve, ezért a teljes `cmake` build, a rendszerd-es és a Docker-labor nem futott le. A `Config` betöltő, a `Logger` és a `Watchdog` önálló, helyi g++ 13.3-as fordítása és tesztje megtörtént; a `main.cpp`, a `Logger.cpp`, a `Watchdog.cpp`, a `Config.cpp`, a `Normalizer.cpp`, az `UdpInputAdapter.cpp`, a `MqttPublisher.cpp` és a `store_forward.cpp` pedig stub fejlécekkel, fordítási szinten ellenőrzött. A rendszerd-es watchdog-próbát és a konténeres UDP-próbát a labornak kell elvégeznie.

13 Edge-feldolgozás

13.1 Helyi feldolgozás: szűrés, aggregálás, eseménydetektálás

13.1.1 Motiváció

A TerraSense üvegházban a `ts-node` másodpercenként vagy akár gyakrabban küld mérési adatot a `ts-gw` gateway-nek. A gateway feladata nem csak a protokollfordítás: ha minden nyers mintát azonnal továbbítana MQTT-n a felhőbe, az uplink folyamatosan terhelt lenne, a mobil adatkeret hamar elfogyna, és a backend is nagyobb számítási kapacitást igényelne. Ugyanakkor a felhasználót nem feltétlenül érdekli, hogy a hőmérséklet 22,1 °C-ról 22,2 °C-ra változott; elég, ha a jelentős változásról azonnal értesül, a lassú trendről pedig percenkénti összegzést kap. A gateway ezért ideális hely a helyi feldolgozásra: közel van az eszközökhöz, rendelkezik számítási kapacitással, és csökkentheti a felhőbe küldött adatmennyiséget, miközben a kritikus riasztások késleltetését alacsonyan tartja.

13.1.2 Fogalmak és működés

13.1.2.1 Deadband és hiszterézis

Definíció. A deadband (holtsáv) olyan szűrési szabály, amely szerint egy új mérési értéket csak akkor továbbítunk, ha az utolsó továbbított értéktől legalább egy előre megadott abszolút küszöbértékkel eltér. Ha az eltérés kisebb, az értéket eldobjuk vagy egy későbbi aggregálásba gyűjtjük.

Példa: ha a hőmérséklet-deadband 0,5 °C, akkor a 22,0 °C után csak akkor küldünk új értéket, ha a hőmérséklet 21,5 °C alá csökken vagy 22,5 °C fölé emelkedik. Ez a mechanizmus C++-ban egyszerűen egy előző-érték tárolással valósítható meg.

Egy gatewayen egyszerre több eszköz és több mérésfajta van, ezért a holtsáv állapotát nem elég egyetlen globális „utolsó értékben” tárolni. A laborban a szűrő eszközönként és mérésfajtánként külön tartja az utolsó továbbított értéket és annak időbélyegét: a hőmérséklet holtsávja 0,5 °C, a relatív páratartalomé 2 %RH, a talajnedvességé 1%. Egy minta akkor kerül továbbításra, ha valamelyik mérésfajta első értéke megérkezik, a saját holtsávjánál nagyobbat változik, vagy lejár a saját maximális hallgatási ideje (heartbeat). A heartbeat akkor is továbbít, ha az érték egyáltalán nem változott, így a backend a lassú vagy állandó folyamatokról is kap életjelet. A döntés indokát a modul mérésfajtánként naplózza, például `deadband:temperature` vagy `heartbeat:soil_moisture`; a tényleges MQTT-üzenet továbbra is egyetlen telemetria-minta, amely az összes rendelkezésre álló mérésfajtát tartalmazza. Fontos, hogy a deadband csak az azonnali továbbítást szűri: az ablakos aggregálás a nem továbbított nyers mintákat is feldolgozza, különben a minimum, a maximum és az átlag torzulna.

A hiszterézis (hysteresis) ennek általánosítása: a be- és kikapcsolási küszöb nem ugyanaz. Például egy fűtési riasztásnál a magas hőmérséklet riasztást 30,0 °C felett vált ki, de a riasztás csak 28,0 °C alatt szűnik meg. Ezzel elkerüljük, hogy az érték a küszöb körül lebegve másodpercenként kapcsolgassa a riasztást, ami zajos telemetriát és felesleges értesítéseket okozna.

13.1.2.2 Ablakos aggregálás

Definíció. Az ablakos aggregálás során a beérkező mintákat időablakokba csoportosítjuk, és minden ablak végén egyetlen összegző üzenetet küldünk, amely statisztikai jellemzőket tartalmaz: minimum, maximum, átlag, esetleg szórás vagy minta darabszám.

Két fő ablaktípust különböztetünk meg:

- **Tumbling (ugró) ablak:** az ablakok nem fedik egymást. Például minden egész perckor kezdődő 60 másodperces ablak; az ablak végén kiértékeljük, majd az aggregátumot publikáljuk, és új ablakot kezdünk. A tumbling ablak egyszerű, de a határokon késleltetés léphet fel.
- **Sliding (csúszó) ablak:** az ablak hossza rögzített, de az indítás gyakori, például 60 másodperces ablakot 10 másodpercenként indítunk. Ez nagyobb számítási és adatforgalmi költséget jelent, de frissebb eredményt ad.

A TerraSense gateway-ben az első változatban tumbling ablakot használunk: 60 másodperces ablakban számolunk minimumot, maximumot és átlagot.

Az ablak határait nem a feldolgozás pillanata, hanem a minta eseményideje (`ts`, epoch ms, UTC) jelöli ki. Az ablak azonosítója a `window_start = floor(ts / window_ms) * window_ms` érték, az intervallum pedig fél-nyílt: `[window_start, window_start + window_ms)`. Ezért a 60. másodpercben érkező minta már a következő, `[60 s, 120 s)` ablakba tartozik, nem az előző utolsó elemeként. Az ablak lezárását egy vízjel vezérli: a legnagyobb látott eseményidő mínusz a megengedett késés (`allowed_lateness_ms`). Ha egy ablak vége a vízjel alá esik, az ablak lezárul; a már lezárt ablakba érkező minta késői adat, amelyet a modul eldob és külön számlál. Azok az időszakok, amelyekben egyáltalán nincs minta, nem hoznak létre üres aggregátumot: a következő minta a saját, eseményidő szerinti ablakát nyitja meg. Leállításkor a `flush_all()` a még nyitott, részleges ablakokat is kiadja, és `final_window` jelöléssel látja el, hogy a fogyasztó meg tudja különböztetni a vízjel miatt lezárt ablaktól.

13.1.2.3 Helyi riasztás

Definíció. A helyi riasztás olyan, a gateway-en futó szabály, amely a bejövő mérési értékekből azonnal, felhő-kapcsolat nélkül is képes döntést hozni, és állapotváltozást jelezni. A riasztás lehet:

- **Küszöbalapú:** az érték átlép egy felső vagy alsó határt.
- **Időtartam-alapú:** a küszöbátlépésnek egy minimális ideig fenn kell állnia, hogy elkerüljük a kiugró értékekből adódó téves riasztást.
- **Trendalapú:** az érték rövid idő alatt gyorsan változik (például 2 °C emelkedés 10 perc alatt), ami közelgő problémára utalhat.

A laborban a küszöb- és időtartam-alapú riasztás négyállapotú állapotgépet használ: `NORMAL`, `PENDING`, `ALERTING` és `RECOVERING`. A magas küszöb elérésekor a riasztás `PENDING` állapotba kerül, és csak akkor vált `ALERTING` állapotba, ha a magas érték az `activate_after_ms` ideig folyamatosan fennáll. Ha a jel a várakozás közben a magas küszöb alá esik, a `PENDING` állapot megszűnik, tehát egy rövid kiugrás nem vált ki riasztást. Az `ALERTING` állapotból az alacsony küszöb elérése `RECOVERING` állapotba visz; innen csak akkor lesz ismét `NORMAL`, ha az érték a `recover_after_ms` ideig folyamatosan az alacsony küszöb alatt marad. Ha közben az alacsony küszöb fölé emelkedik, a riasztás visszatér `ALERTING` állapotba. A `min_duration` tehát nem csak a kikapcsolást késlelteti: a bekapcsoláshoz is tartós küszöbátlépés kell, a hiszterézis pedig a két küszöb közötti sávban megakadályozza a lebegést.

Ezek a riasztások a TerraSense-ben a `terrasense/<site>/<node>/state` retained topicra kerülnek, hogy a backend és a felhasználói felület is lássa a jelenlegi állapotot.

13.1.3 Döntési szempontok

Az alábbi táblázat azt foglalja össze, mikor érdemes a feldolgozást a gateway-en végezni a nyers felhőbe küldés helyett.

Szempont	Nyers adat felhőbe	Deadband	Ablakos aggregálás
Sávszélesség-igény	Magas	Közepes	Alacsony
Információvesztés	Nincs	Kicsi (küszöb alatti változás)	Nagyobb (részletek elvesznek)
Riasztási késleltetés	Alacsony	Alacsony	Akár az ablak hossza
Tárolási költség	Magas	Közepes	Alacsony
Gateway CPU-terhelés	Alacsony	Alacsony	Közepes
Tipikus felhasználás	Hibakeresés, kalibráció	Hirtelen változások	Trend, élettartam, költség

A TerraSense-ben a telemetria-topicra a deadband-szűrt azonnali értékek és a percenkénti aggregátumok is felkerülnek, de eltérő séma-mezőkkel; a részleteket az Adatformátumok: JSON, CBOR, Protocol Buffers, SenML alfejezet rögzíti.

13.1.4 Kidolgozott példa: megtakarítás-számítás

Tegyük fel, hogy a `ts-node` 10 másodpercenként küld egy nyers telemetria-üzenetet, amelynek mérete JSON-ban 86 B. A gateway deadband-szűrést és 60 másodperces tumbling ablakot alkalmaz. Az aggregált üzenet – amely tartalmazza a minimumot, maximumot és átlagot – mérete 96 B, mert több mező van benne, de csak percenként egy ilyen üzenet keletkezik.

Jelölje:

- $T_m = 10$ s a mérési periódus,
- $S_{\text{nyers}} = 86$ B a nyers üzenet mérete,
- $T_w = 60$ s az ablak hossza,
- $S_{\text{agg}} = 96$ B az aggregált üzenet mérete.

A naponta küldött üzenetek száma nyers esetben:

$$N_{\text{nyers}} = \frac{86\,400 \text{ s}}{10 \text{ s}} = 8\,640$$

A napi nyers adatmennyiség:

$$D_{\text{nyers}} = N_{\text{nyers}} \cdot S_{\text{nyers}} = 8\,640 \times 86 \text{ B} = 743\,040 \text{ B} \approx 743 \text{ kB}$$

Az aggregálással percenként egy üzenet keletkezik, tehát naponta:

$$N_{\text{agg}} = \frac{86\,400 \text{ s}}{60 \text{ s}} = 1\,440$$

A napi aggregált adatmennyiség:

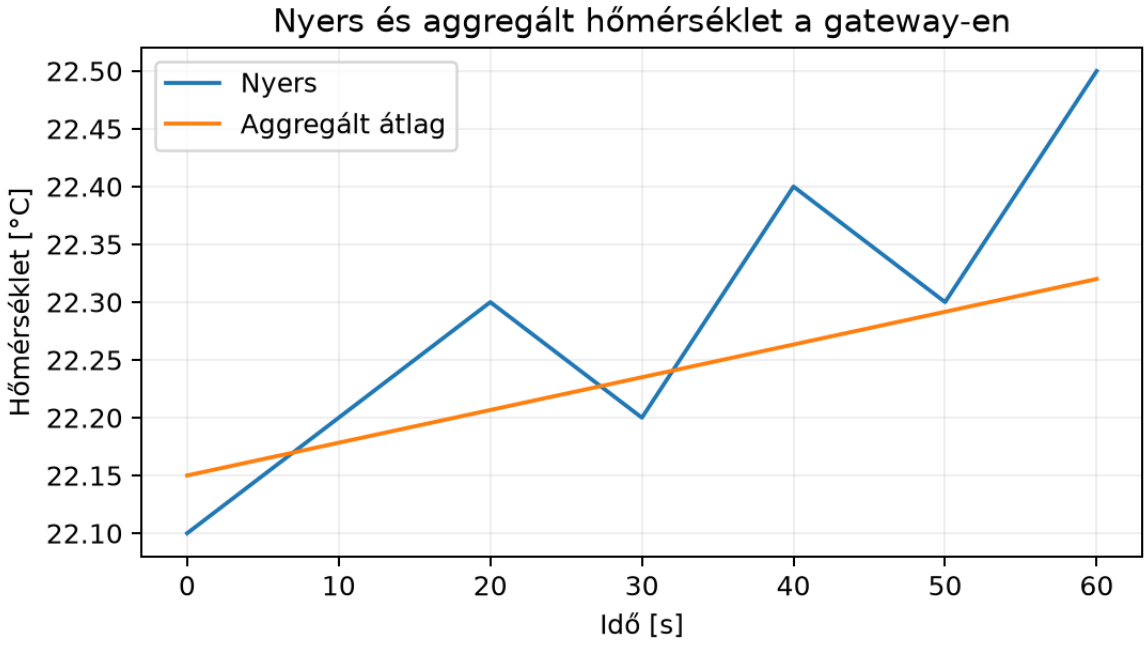
$$D_{\text{agg}} = N_{\text{agg}} \cdot S_{\text{agg}} = 1\,440 \times 96 \text{ B} = 138\,240 \text{ B} \approx 138 \text{ kB}$$

A megtakarítás:

$$\Delta = 1 - \frac{D_{\text{agg}}}{D_{\text{nyers}}} = 1 - \frac{138\,240}{743\,040} \approx 0{,}814$$

Tehát a gateway-en végzett ablakos aggregálás körülbelül **81,4%** sávszélesség-megtakarítást eredményez. Természetesen az információ egy része elvész: a percen belüli ingadozások csak a minimum- és maximumértékben jelennek meg, az átlag ezeket elrejti. Ezért az aggregátumboz mindig érdemes csatolni a `min` és `max` mezőket is.

Az alábbi ábra illusztrálja a nyers és az aggregált adat megjelenését.



13.1. ábra: Nyers és aggregált hőmérséklet a gateway-en - Illusztratív, számított mintaadatok; az aggregált pont az első 60 s átlagát mutatja.

13.1.5 Labor: aggregáló modul a `ts-gw`-ben

Munkakönyvtár:

```
mkdir -p ~/iot-labor/i039 && cd ~/iot-labor/i039
```

Ebben a laborban egy önálló, külső könyvtáraktól mentes C++17 modult készítünk, amelyet a `ts-gw` későbbi változata a `Normalizer` után, a JSON-szerializálás és a spool elé illeszthet. A mintákat a `Sample` struktúra reprezentálja: a `site`, `node_id`, `boot_id`, `seq` és `ts` mellett a három mérésfajta `std::optional<double>` mezőként szerepel, mert egy eszköz egy adott körben nem feltétlenül küld minden értéket. A modul három alaposztálya a `DeadbandFilter`, a `TumblingWindowAggregator` és a `ThresholdAlert`; ezeket az `EdgeAggregator` fogja össze,

amely eszközönként külön szűrő-, ablak- és riasztásállapotot tart. A kód teljes: nem függ Paho-tól, SQLite-tól, systemd-től vagy más külső könyvtártól, ezért önállóan fordítható és futtatható.

A labor konfigurációja: hőmérséklet-deadband 0,5 °C, páratartalom-deadband 2 %RH, talajnedvesség-deadband 1%, heartbeat 60 s, ablakhossz 60 s, megengedett késés 5 s, riasztási magas küszöb 30 °C, alacsony küszöb 28 °C, aktiválási és feloldási idő 300 s. A `main.cpp` három, egymástól jól elkülöníthető demonstrációt futtat: a teljes pipeline-t két eszközzel, az eseményidő-ablakot késői adattal, valamint a riasztás állapotgépét.

Fájl: `edge_aggregator.h`

```
#pragma once
#include <cstdint>
#include <map>
#include <optional>
#include <string>
#include <vector>

namespace terrasense {

struct Sample {
  std::string site;
  std::string node_id;
  std::uint32_t boot_id{0};
  std::uint32_t seq{0};
  std::uint64_t ts{0};  // epoch ms, UTC
  std::optional<double> temperature;     // °C
  std::optional<double> humidity;        // %RH
  std::optional<double> soil_moisture;   // %
};

struct MetricAggregate {
  double min{0.0};
  double max{0.0};
  double avg{0.0};
  std::uint32_t count{0};
};

struct AggregatedSample {
  std::string site;
  std::string node_id;
  std::uint64_t window_start{0};
  std::uint64_t window_end{0};
  std::uint32_t sample_count{0};
  bool final_window{false};  // flush_all() zárta le, nem vízjel
  MetricAggregate temperature;
  MetricAggregate humidity;
  MetricAggregate soil_moisture;
};

struct DeadbandConfig {
  double temperature{0.5};
  double humidity{2.0};
  double soil_moisture{1.0};
  std::uint64_t heartbeat_ms{60000};
};

struct FilterResult {
  bool forward{false};
  std::vector<std::string> reasons;
};
```

```
class DeadbandFilter {
public:
  explicit DeadbandFilter(const DeadbandConfig& config);

  // Mérésfajtánként dönt: változás, első érték vagy heartbeat esetén jelez.
  FilterResult update(const Sample& sample);

private:
  enum class Metric { kTemperature = 0, kHumidity = 1, kSoilMoisture = 2 };

  double threshold(Metric metric) const;
  static const char* metric_name(Metric metric);
  void check(Metric metric, const std::optional<double>& value,
             std::uint64_t ts, FilterResult& result);

  struct LastValue {
    bool valid{false};
    double value{0.0};
    std::uint64_t forwarded_ts{0};
  };

  DeadbandConfig config_;
  LastValue last_[3];
};

class TumblingWindowAggregator {
public:
  TumblingWindowAggregator(std::uint64_t window_ms,
                           std::uint64_t allowed_lateness_ms);

  // A mintát az eseményideje szerinti fél-nyílt [start, end) ablakba teszi.
  // Visszatér a lezárt ablakokkal, a vízjel a legnagyobb látott eseményidő
  // mínusz a megengedett késés.
  std::vector<AggregatedSample> add(const Sample& sample);
  std::vector<AggregatedSample> flush_all();
  std::uint64_t late_dropped() const { return late_dropped_; }

private:
  struct Accumulator {
    bool has_value{false};
    double min{0.0};
    double max{0.0};
    double sum{0.0};
    std::uint32_t count{0};

    void add(double value);
    MetricAggregate finish() const;
  };

  struct Window {
    AggregatedSample meta;
    Accumulator temperature;
    Accumulator humidity;
    Accumulator soil_moisture;
  };

  std::vector<AggregatedSample> close_ready();

  std::uint64_t window_ms_;
  std::uint64_t allowed_lateness_ms_;
  std::uint64_t max_seen_ts_{0};
  std::uint64_t closed_before_{0};
```

```
  bool has_seen_{false};
  std::uint64_t late_dropped_{0};
  std::map<std::uint64_t, Window> windows_;
};

enum class AlertState { NORMAL, PENDING, ALERTING, RECOVERING };

const char* alert_state_name(AlertState state);

struct AlertEvent {
  AlertState from{AlertState::NORMAL};
  AlertState to{AlertState::NORMAL};
  std::uint64_t ts{0};
  std::string reason;
};

struct AlertConfig {
  double high{30.0};
  double low{28.0};
  std::uint64_t activate_after_ms{300000};
  std::uint64_t recover_after_ms{300000};
};

class ThresholdAlert {
public:
  explicit ThresholdAlert(const AlertConfig& config);

  // Csak állapotváltozáskor ad eseményt. A tartós magas állapotot az
  // aktiválás előtt, a tartós alacsony állapotot a feloldás előtt méri.
  std::optional<AlertEvent> update(const Sample& sample);
  AlertState state() const { return state_; }
  bool is_active() const {
    return state_ == AlertState::ALERTING ||
           state_ == AlertState::RECOVERING;
  }

private:
  AlertConfig config_;
  AlertState state_{AlertState::NORMAL};
  std::uint64_t candidate_since_{0};
};

struct AggregatorConfig {
  DeadbandConfig deadband;
  std::uint64_t window_ms{60000};
  std::uint64_t allowed_lateness_ms{5000};
  AlertConfig alert;
};

class EdgeAggregator {
public:
  explicit EdgeAggregator(const AggregatorConfig& config);

  struct Result {
    std::string device;
    bool forwarded{false};
    std::vector<std::string> reasons;
    std::vector<AggregatedSample> completed_windows;
    std::optional<AlertEvent> alert_event;
  };

  // Eszközönként külön deadband-, ablak- és riasztásállapotot tart.
  Result process(const Sample& sample);
```

```
  std::vector<AggregatedSample> flush_all();
  std::uint64_t late_dropped(const std::string& device) const;

private:
  struct DeviceState {
    explicit DeviceState(const AggregatorConfig& config)
        : filter(config.deadband),
          windows(config.window_ms, config.allowed_lateness_ms),
          alert(config.alert) {}

    DeadbandFilter filter;
    TumblingWindowAggregator windows;
    ThresholdAlert alert;
  };

  static std::string device_key(const Sample& sample);

  AggregatorConfig config_;
  std::map<std::string, DeviceState> devices_;
};

} // namespace terrasense
```

Fájl: edge_aggregator.cpp

```
#include "edge_aggregator.h"
#include <algorithm>
#include <cmath>
#include <stdexcept>
#include <utility>

namespace terrasense {

DeadbandFilter::DeadbandFilter(const DeadbandConfig& config)
    : config_(config) {
  if (config_.temperature < 0.0 || config_.humidity < 0.0 ||
      config_.soil_moisture < 0.0) {
    throw std::invalid_argument("deadband threshold must be non-negative");
  }
  if (config_.heartbeat_ms == 0) {
    throw std::invalid_argument("heartbeat_ms must be positive");
  }
}

double DeadbandFilter::threshold(Metric metric) const {
  switch (metric) {
    case Metric::kTemperature:
      return config_.temperature;
    case Metric::kHumidity:
      return config_.humidity;
    case Metric::kSoilMoisture:
      return config_.soil_moisture;
  }
  return 0.0;
}

const char* DeadbandFilter::metric_name(Metric metric) {
  switch (metric) {
    case Metric::kTemperature:
      return "temperature";
    case Metric::kHumidity:
      return "humidity";
    case Metric::kSoilMoisture:
```

```
      return "soil_moisture";
  }
  return "unknown";
}

void DeadbandFilter::check(Metric metric, const std::optional<double>& value,
                           std::uint64_t ts, FilterResult& result) {
  if (!value.has_value() || !std::isfinite(*value)) {
    return;
  }
  LastValue* last_ptr = nullptr;
  switch (metric) {
    case Metric::kTemperature:
      last_ptr = &last_[0];
      break;
    case Metric::kHumidity:
      last_ptr = &last_[1];
      break;
    case Metric::kSoilMoisture:
      last_ptr = &last_[2];
      break;
  }
  if (last_ptr == nullptr) {
    return;
  }
  LastValue& last = *last_ptr;
  if (!last.valid) {
    last = LastValue{true, *value, ts};
    result.forward = true;
    result.reasons.push_back(std::string("first:") + metric_name(metric));
    return;
  }
  const double delta = std::fabs(*value - last.value);
  if (delta >= threshold(metric)) {
    last = LastValue{true, *value, ts};
    result.forward = true;
    result.reasons.push_back(std::string("deadband:") + metric_name(metric));
    return;
  }
  if (ts >= last.forwarded_ts && ts - last.forwarded_ts >=
                                     config_.heartbeat_ms) {
    last = LastValue{true, *value, ts};
    result.forward = true;
    result.reasons.push_back(std::string("heartbeat:") + metric_name(metric));
  }
}

FilterResult DeadbandFilter::update(const Sample& sample) {
  FilterResult result;
  check(Metric::kTemperature, sample.temperature, sample.ts, result);
  check(Metric::kHumidity, sample.humidity, sample.ts, result);
  check(Metric::kSoilMoisture, sample.soil_moisture, sample.ts, result);
  return result;
}

void TumblingWindowAggregator::Accumulator::add(double value) {
  if (!has_value) {
    has_value = true;
    min = value;
    max = value;
  } else {
    min = std::min(min, value);
    max = std::max(max, value);
```

```
  }
  sum += value;
  ++count;
}

MetricAggregate TumblingWindowAggregator::Accumulator::finish() const {
  MetricAggregate out;
  if (!has_value || count == 0) {
    return out;
  }
  out.min = min;
  out.max = max;
  out.avg = sum / static_cast<double>(count);
  out.count = count;
  return out;
}

TumblingWindowAggregator::TumblingWindowAggregator(
    std::uint64_t window_ms, std::uint64_t allowed_lateness_ms)
    : window_ms_(window_ms), allowed_lateness_ms_(allowed_lateness_ms) {
  if (window_ms_ == 0) {
    throw std::invalid_argument("window_ms must be positive");
  }
}

std::vector<AggregatedSample> TumblingWindowAggregator::add(
    const Sample& sample) {
  if (sample.site.empty() || sample.node_id.empty()) {
    throw std::invalid_argument("sample site and node_id must be set");
  }
  const std::uint64_t window_start =
      (sample.ts / window_ms_) * window_ms_;
  if (has_seen_ && window_start < closed_before_) {
    ++late_dropped_;
    return {};
  }
  if (!has_seen_ || sample.ts > max_seen_ts_) {
    max_seen_ts_ = sample.ts;
  }
  has_seen_ = true;

  Window& window = windows_[window_start];
  if (window.meta.sample_count == 0) {
    window.meta.site = sample.site;
    window.meta.node_id = sample.node_id;
    window.meta.window_start = window_start;
    window.meta.window_end = window_start + window_ms_;
  }
  ++window.meta.sample_count;
  if (sample.temperature.has_value() &&
      std::isfinite(*sample.temperature)) {
    window.temperature.add(*sample.temperature);
  }
  if (sample.humidity.has_value() && std::isfinite(*sample.humidity)) {
    window.humidity.add(*sample.humidity);
  }
  if (sample.soil_moisture.has_value() &&
      std::isfinite(*sample.soil_moisture)) {
    window.soil_moisture.add(*sample.soil_moisture);
  }
  return close_ready();
}
```

```
std::vector<AggregatedSample> TumblingWindowAggregator::close_ready() {
  std::vector<AggregatedSample> result;
  const std::uint64_t watermark =
      max_seen_ts_ >= allowed_lateness_ms_
          ? max_seen_ts_ - allowed_lateness_ms_
          : 0;
  for (auto it = windows_.begin(); it != windows_.end();) {
    Window& window = it->second;
    if (window.meta.window_end > watermark) {
      ++it;
      continue;
    }
    window.meta.temperature = window.temperature.finish();
    window.meta.humidity = window.humidity.finish();
    window.meta.soil_moisture = window.soil_moisture.finish();
    closed_before_ = std::max(closed_before_, window.meta.window_end);
    result.push_back(window.meta);
    it = windows_.erase(it);
  }
  return result;
}

std::vector<AggregatedSample> TumblingWindowAggregator::flush_all() {
  std::vector<AggregatedSample> result;
  for (auto& entry : windows_) {
    Window& window = entry.second;
    window.meta.temperature = window.temperature.finish();
    window.meta.humidity = window.humidity.finish();
    window.meta.soil_moisture = window.soil_moisture.finish();
    window.meta.final_window = true;
    closed_before_ = std::max(closed_before_, window.meta.window_end);
    result.push_back(window.meta);
  }
  windows_.clear();
  return result;
}

const char* alert_state_name(AlertState state) {
  switch (state) {
    case AlertState::NORMAL:
      return "NORMAL";
    case AlertState::PENDING:
      return "PENDING";
    case AlertState::ALERTING:
      return "ALERTING";
    case AlertState::RECOVERING:
      return "RECOVERING";
  }
  return "UNKNOWN";
}

ThresholdAlert::ThresholdAlert(const AlertConfig& config) : config_(config) {
  if (config_.low > config_.high) {
    throw std::invalid_argument("alert low threshold must not exceed high");
  }
  if (config_.activate_after_ms == 0 || config_.recover_after_ms == 0) {
    throw std::invalid_argument("alert durations must be positive");
  }
}

std::optional<AlertEvent> ThresholdAlert::update(const Sample& sample) {
  if (!sample.temperature.has_value() ||
      !std::isfinite(*sample.temperature)) {
```

```
    return std::nullopt;
  }
  const double value = *sample.temperature;
  std::optional<AlertEvent> event;
  switch (state_) {
    case AlertState::NORMAL:
      if (value >= config_.high) {
        state_ = AlertState::PENDING;
        candidate_since_ = sample.ts;
        event = AlertEvent{AlertState::NORMAL, state_, sample.ts,
                           "magas küszöb elérve"};
      }
      break;
    case AlertState::PENDING:
      if (value < config_.high) {
        const AlertState from = state_;
        state_ = AlertState::NORMAL;
        event = AlertEvent{from, state_, sample.ts,
                           "kiugrás az aktiválás előtt megszűnt"};
      } else if (sample.ts >= candidate_since_ &&
                 sample.ts - candidate_since_ >=
                     config_.activate_after_ms) {
        const AlertState from = state_;
        state_ = AlertState::ALERTING;
        event = AlertEvent{from, state_, sample.ts,
                           "magas küszöb tartósan fennáll"};
      }
      break;
    case AlertState::ALERTING:
      if (value <= config_.low) {
        const AlertState from = state_;
        state_ = AlertState::RECOVERING;
        candidate_since_ = sample.ts;
        event = AlertEvent{from, state_, sample.ts,
                           "alacsony küszöb elérve"};
      }
      break;
    case AlertState::RECOVERING:
      if (value > config_.low) {
        const AlertState from = state_;
        state_ = AlertState::ALERTING;
        event = AlertEvent{from, state_, sample.ts,
                           "az érték a feloldási küszöb fölé emelkedett"};
      } else if (sample.ts >= candidate_since_ &&
                 sample.ts - candidate_since_ >=
                     config_.recover_after_ms) {
        const AlertState from = state_;
        state_ = AlertState::NORMAL;
        event = AlertEvent{from, state_, sample.ts,
                           "a feloldás tartósan fennáll"};
      }
      break;
  }
  return event;
}

EdgeAggregator::EdgeAggregator(const AggregatorConfig& config)
    : config_(config) {
  if (config_.window_ms == 0) {
    throw std::invalid_argument("window_ms must be positive");
  }
}
```

```
std::string EdgeAggregator::device_key(const Sample& sample) {
  return sample.site + "/" + sample.node_id;
}

EdgeAggregator::Result EdgeAggregator::process(const Sample& sample) {
  if (sample.site.empty() || sample.node_id.empty()) {
    throw std::invalid_argument("sample site and node_id must be set");
  }
  const std::string key = device_key(sample);
  auto inserted = devices_.try_emplace(key, config_);
  DeviceState& state = inserted.first->second;

  Result result;
  result.device = key;
  const FilterResult filter = state.filter.update(sample);
  result.forwarded = filter.forward;
  result.reasons = filter.reasons;
  result.completed_windows = state.windows.add(sample);
  result.alert_event = state.alert.update(sample);
  return result;
}

std::vector<AggregatedSample> EdgeAggregator::flush_all() {
  std::vector<AggregatedSample> result;
  for (auto& entry : devices_) {
    auto windows = entry.second.windows.flush_all();
    result.insert(result.end(), windows.begin(), windows.end());
  }
  return result;
}

std::uint64_t EdgeAggregator::late_dropped(const std::string& device) const {
  const auto it = devices_.find(device);
  if (it == devices_.end()) {
    return 0;
  }
  return it->second.windows.late_dropped();
}

} // namespace terrasense
```

Fájl: `main.cpp`

```
#include "edge_aggregator.h"
#include <cstddef>
#include <cstdint>
#include <iomanip>
#include <iostream>
#include <string>
#include <utility>
#include <vector>

using namespace terrasense;

namespace {

constexpr std::uint64_t kStart = 1'700'000'040'000ULL;

std::string join_reasons(const std::vector<std::string>& reasons) {
  std::string out;
  for (std::size_t i = 0; i < reasons.size(); ++i) {
    if (i != 0) {
      out += ",";
```

```
    }
    out += reasons[i];
  }
  return out.empty() ? "-" : out;
}

Sample make_sample(const std::string& site, const std::string& node,
                   std::uint64_t offset_ms, double temperature,
                   double humidity, double soil_moisture) {
  Sample sample;
  sample.site = site;
  sample.node_id = node;
  sample.boot_id = 7;
  sample.seq = static_cast<std::uint32_t>(offset_ms / 10000 + 1);
  sample.ts = kStart + offset_ms;
  sample.temperature = temperature;
  sample.humidity = humidity;
  sample.soil_moisture = soil_moisture;
  return sample;
}

void print_metric(const char* name, const MetricAggregate& metric) {
  std::cout << "    " << name << "[n=" << metric.count;
  if (metric.count > 0) {
    std::cout << " min=" << std::fixed << std::setprecision(1) << metric.min
              << " max=" << metric.max << " avg=" << metric.avg;
  }
  std::cout << "]\n";
}

void print_window(const AggregatedSample& window) {
  std::cout << "  ablak " << window.site << "/" << window.node_id << " +"
            << (window.window_start - kStart) / 1000 << "s..+"
            << (window.window_end - kStart) / 1000 << "s"
            << " minták=" << window.sample_count
            << " final=" << (window.final_window ? "igen" : "nem") << "\n";
  print_metric("hőmérséklet", window.temperature);
  print_metric("páratartalom", window.humidity);
  print_metric("talaj", window.soil_moisture);
}

void print_alert(const AlertEvent& event) {
  std::cout << "  riasztás " << alert_state_name(event.from) << " -> "
            << alert_state_name(event.to) << " +"
            << (event.ts - kStart) / 1000 << "s-nál: " << event.reason << "\n";
}

void print_result(const EdgeAggregator::Result& result,
                  std::uint64_t offset_ms) {
  std::cout << result.device << " +" << offset_ms / 1000
            << "s továbbítás=" << (result.forwarded ? "igen" : "nem")
            << " okok=" << join_reasons(result.reasons) << "\n";
}

void demo_pipeline() {
  std::cout << "== pipeline: eszközönkénti deadband, ablak, riasztás ==\n";
  AggregatorConfig config;
  config.deadband.temperature = 0.5;
  config.deadband.humidity = 2.0;
  config.deadband.soil_moisture = 1.0;
  config.deadband.heartbeat_ms = 60000;
  config.window_ms = 60000;
  config.allowed_lateness_ms = 5000;
```

```
  config.alert.high = 30.0;
  config.alert.low = 28.0;
  config.alert.activate_after_ms = 300000;
  config.alert.recover_after_ms = 300000;

  EdgeAggregator aggregator(config);
  const double temperatures[] = {22.0, 22.1, 22.2, 22.3, 22.4, 22.2};
  for (int i = 0; i < 6; ++i) {
    const auto offset = static_cast<std::uint64_t>(i) * 10000ULL;
    const Sample sample = make_sample("alpha", "n001", offset,
                                      temperatures[i], 55.0, 40.0);
    const auto result = aggregator.process(sample);
    print_result(result, offset);
    for (const auto& window : result.completed_windows) {
      print_window(window);
    }
  }

  for (const std::uint64_t offset : {0ULL, 10000ULL}) {
    const Sample sample = make_sample("alpha", "n002", offset, 22.0, 55.0,
                                      offset == 0 ? 40.0 : 20.0);
    const auto result = aggregator.process(sample);
    print_result(result, offset);
    for (const auto& window : result.completed_windows) {
      print_window(window);
    }
  }

  for (const auto& item : std::vector<std::pair<std::uint64_t, double>>{
           {60000ULL, 22.1}, {70000ULL, 23.0}, {80000ULL, 22.4}}) {
    const Sample sample = make_sample("alpha", "n001", item.first, item.second,
                                      55.0, 40.0);
    const auto result = aggregator.process(sample);
    print_result(result, item.first);
    for (const auto& window : result.completed_windows) {
      print_window(window);
    }
    if (result.alert_event.has_value()) {
      print_alert(*result.alert_event);
    }
  }

  for (const auto& window : aggregator.flush_all()) {
    print_window(window);
  }
  std::cout << "késői adat eldobva: alpha/n001="
            << aggregator.late_dropped("alpha/n001")
            << " alpha/n002=" << aggregator.late_dropped("alpha/n002")
            << "\n";
}

void demo_late_data() {
  std::cout << "\n== eseményidő-ablak és késői adat ==\n";
  TumblingWindowAggregator aggregator(60000, 5000);
  for (int i = 0; i < 6; ++i) {
    const auto offset = static_cast<std::uint64_t>(i) * 10000ULL;
    const Sample sample = make_sample("alpha", "n003", offset, 20.0, 50.0,
                                      30.0);
    for (const auto& window : aggregator.add(sample)) {
      print_window(window);
    }
  }
```

```
  const Sample at_seventy = make_sample("alpha", "n003", 70000, 21.0, 50.0,
                                        30.0);
  for (const auto& window : aggregator.add(at_seventy)) {
    print_window(window);
  }

  const Sample late = make_sample("alpha", "n003", 55000, 99.0, 50.0, 30.0);
  for (const auto& window : aggregator.add(late)) {
    print_window(window);
  }
  std::cout << "késői minta +55s-nél: eldobva=" << aggregator.late_dropped()
            << "\n";

  const Sample out_of_order = make_sample("alpha", "n003", 68000, 19.0, 50.0,
                                          30.0);
  for (const auto& window : aggregator.add(out_of_order)) {
    print_window(window);
  }
  for (const auto& window : aggregator.flush_all()) {
    print_window(window);
  }
  std::cout << "késői adat összesen eldobva=" << aggregator.late_dropped() << "\n";
}

void demo_alert() {
  std::cout << "\n== riasztás: NORMAL/PENDING/ALERTING/RECOVERING ==\n";
  AlertConfig config;
  config.high = 30.0;
  config.low = 28.0;
  config.activate_after_ms = 300000;
  config.recover_after_ms = 300000;
  ThresholdAlert alert(config);

  const std::vector<std::pair<std::uint64_t, double>> timeline{
      {0, 29.0},     {60000, 31.0},  {120000, 31.0}, {180000, 29.0},
      {240000, 31.0}, {300000, 31.0}, {360000, 31.0}, {420000, 31.0},
      {480000, 31.0}, {540000, 31.0}, {600000, 29.0}, {660000, 27.0},
      {720000, 27.0}, {780000, 27.0}, {840000, 27.0}, {900000, 27.0},
      {960000, 27.0}};
  std::cout << "  magas=30 alacsony=28 aktiválás=300s feloldás=300s\n";
  for (const auto& entry : timeline) {
    Sample sample;
    sample.ts = kStart + entry.first;
    sample.temperature = entry.second;
    const auto event = alert.update(sample);
    if (event.has_value()) {
      print_alert(*event);
    }
  }
  std::cout << "  végállapot=" << alert_state_name(alert.state())
            << " aktív=" << (alert.is_active() ? "igen" : "nem") << "\n";
}

} // namespace

int main() {
  demo_pipeline();
  demo_late_data();
  demo_alert();
  return 0;
}
```

Build és futtatás:

```
g++ -std=c++17 -Wall -Wextra -Wpedantic -Werror -O2 \
    -o aggregator_demo edge_aggregator.cpp main.cpp
./aggregator_demo
```

Tényleges kimenet. A kódot a szerzői környezetben GCC 13.3.0-val, C++17 módban lefuttattuk; az alábbi kimenet ebből a futásból származik, nem becsült érték. A program nem használ rendszeridőt: a `kStart` rögzített, szintetikus epoch-érték, ezért a kimenet ismételhető.

```
== pipeline: eszközönkénti deadband, ablak, riasztás ==
alpha/n001 +0s továbbítás=igen okok=first:temperature,first:humidity,first:soil_moisture
alpha/n001 +10s továbbítás=nem okok=-
alpha/n001 +20s továbbítás=nem okok=-
alpha/n001 +30s továbbítás=nem okok=-
alpha/n001 +40s továbbítás=nem okok=-
alpha/n001 +50s továbbítás=nem okok=-
alpha/n002 +0s továbbítás=igen okok=first:temperature,first:humidity,first:soil_moisture
alpha/n002 +10s továbbítás=igen okok=deadband:soil_moisture
alpha/n001 +60s továbbítás=igen
 ↪  okok=heartbeat:temperature,heartbeat:humidity,heartbeat:soil_moisture
alpha/n001 +70s továbbítás=igen okok=deadband:temperature
  ablak alpha/n001 +0s..+60s minták=6 final=nem
    hőmérséklet[n=6 min=22.0 max=22.4 avg=22.2]
    páratartalom[n=6 min=55.0 max=55.0 avg=55.0]
    talaj[n=6 min=40.0 max=40.0 avg=40.0]
alpha/n001 +80s továbbítás=igen okok=deadband:temperature
  ablak alpha/n001 +60s..+120s minták=3 final=igen
    hőmérséklet[n=3 min=22.1 max=23.0 avg=22.5]
    páratartalom[n=3 min=55.0 max=55.0 avg=55.0]
    talaj[n=3 min=40.0 max=40.0 avg=40.0]
  ablak alpha/n002 +0s..+60s minták=2 final=igen
    hőmérséklet[n=2 min=22.0 max=22.0 avg=22.0]
    páratartalom[n=2 min=55.0 max=55.0 avg=55.0]
    talaj[n=2 min=20.0 max=40.0 avg=30.0]
késői adat eldobva: alpha/n001=0 alpha/n002=0

== eseményidő-ablak és késői adat ==
  ablak alpha/n003 +0s..+60s minták=6 final=nem
    hőmérséklet[n=6 min=20.0 max=20.0 avg=20.0]
    páratartalom[n=6 min=50.0 max=50.0 avg=50.0]
    talaj[n=6 min=30.0 max=30.0 avg=30.0]
késői minta +55s-nél: eldobva=1
  ablak alpha/n003 +60s..+120s minták=2 final=igen
    hőmérséklet[n=2 min=19.0 max=21.0 avg=20.0]
    páratartalom[n=2 min=50.0 max=50.0 avg=50.0]
    talaj[n=2 min=30.0 max=30.0 avg=30.0]
késői adat összesen eldobva=1

== riasztás: NORMAL/PENDING/ALERTING/RECOVERING ==
  magas=30 alacsony=28 aktiválás=300s feloldás=300s
  riasztás NORMAL -> PENDING +60s-nál: magas küszöb elérve
  riasztás PENDING -> NORMAL +180s-nál: kiugrás az aktiválás előtt megszűnt
  riasztás NORMAL -> PENDING +240s-nál: magas küszöb elérve
  riasztás PENDING -> ALERTING +540s-nál: magas küszöb tartósan fennáll
  riasztás ALERTING -> RECOVERING +660s-nál: alacsony küszöb elérve
  riasztás RECOVERING -> NORMAL +960s-nál: a feloldás tartósan fennáll
  végállapot=NORMAL aktív=nem
```

A kimenet első részében az `alpha/n001` első mintája mindhárom mérésfajtát továbbítja. A +10 s és +50 s közötti kis változások egyik holtsávot sem érik el, ezért nem kerülnek ki. A +60 s-os minta három `heartbeat` indokot ad: 60 s telt el az előző továbbítás óta, tehát akkor is megy életjel, ha az érték nem változott. A +70 s-en a 23,0 °C már 0,9 °C-kal tér el az utolsó továbbított 22,1 °C-tól, ezért `deadband:temperature`; a +80 s-en a 22,4 °C a 23,0 °C-hoz képest

0,6 °C-kal csökken, ezért ismét továbbítódik. Ez a H158-ban hiányolt, 23,0 °C utáni 22,4 °C-os deadband-továbbítás.

Az `alpha/n002` külön eszközállapotot kap: a +10 s-en csak a talajnedvesség változik 40%-ról 20%-ra, ezért csak `deadband:soil_moisture` szerepel az okok között. Ez mutatja, hogy az `n001` hőmérséklet-állapota és az `n002` talajnedvesség-állapota nem keveredik össze. Az `alpha/n001` 0–60 s ablakában 6 minta van, a hőmérséklet minimuma 22,0 °C, maximuma 22,4 °C, átlaga 22,2 °C. A minimum azért 22,0 és nem 0,0, mert az `Accumulator` az első tényleges értékből inicializál, a mérésfajtánkénti darabszám pedig külön számolódik. A 60–120 s ablak a +80 s utáni `flush_all()` hívásból kerül ki, ezért `final=igen`; ez a részleges ablak a leállítási szabályt szemlélteti.

Az eseményidő-ablakos részben a +70 s-os minta vízjele zárja le a 0–60 s ablakot, mert a megengedett késés 5 s. A +55 s-os késői minta már nem fér bele a lezárt ablakba: a modul eldobja, és a `late_dropped` számlálót 1-re növeli. A +68 s-os minta viszont még a nyitott 60–120 s ablakba kerül, tehát rendezetlen érkezésű, de nem késői adat; a végső, kétmintás ablakot itt is a `flush_all()` adja ki `final=igen` jelöléssel.

A riasztási részben a +60 s-os 31 °C rövid kiugrás csak `PENDING` állapotot indít; a +180 s-os 29 °C visszaállítja `NORMAL` állapotba, tehát nem lesz riasztás. A +240 s-től tartósan 31 °C következik, és az aktiválási idő letelte után, +540 s-nál vált `ALERTING` állapotba. A +660 s-os 27 °C `RECOVERING` állapotot indít, amely a feloldási idő leteltével, +960 s-nál tér vissza `NORMAL` állapotba. A +600 s-os 29 °C a két küszöb között van, ezért az `ALERTING` állapotot nem szünteti meg: ez a hiszterézis működése.

Integráció a `ts-gw`-be. A `Normalizer::parse` által előállított `TelemetrySample` mezőit kell a `Sample` struktúrába átmásolni, majd az `EdgeAggregator::process` hívását a spoolba írás előtt elvégezni. A `forwarded` jelzésű minták a meglévő `terrasense/<site>/<node>/telemetry` útvonalon, a szokásos JSON-szerializálással és spool-lal mennek tovább. A `completed_windows` elemei ugyanerre a telemetria-topicra kerülhetnek `agg` objektummal; a `final_window` mezőt a fogyasztónak érdemes továbbítania, hogy meg tudja különböztetni a részleges záróablakot. Az `alert_event` alapján a `terrasense/<site>/<node>/state` retained topicra kerülhet a `local_alert` állapot. A teljes `ts-gw` Paho-, SQLite- és systemd-integrációját ez a labor nem futtatja; az itt bemutatott modul önálló, determinisztikus egység.

13.1.6 Ellenőrzés és hibakeresés

A modul működését unit tesztekkel ellenőrizhetjük, például a Google Test keretrendszerrel, de a fenti egyszerű `main` is alkalmas a kézi validálásra.

Tünet	Lehetséges ok	Javítás
A deadband nem továbbít, pedig nagy a változás	Nem a megfelelő eszköz vagy mérésfajta állapota frissül, esetleg a heartbeat értéke nulla	Ellenőrizd az `EdgeAggregator` `site/node` kulcsát és a `DeadbandFilter` mérésfajtánkénti `last_` tárolását; a heartbeat legyen pozitív
Az aggregátor nem ad ki eredményt	A vízjel még nem érte el az ablak végét, vagy a minta késői	Ellenőrizd a `max_seen_ts_`, `allowed_lateness_ms_` és `window_end <= watermark` viszonyt; leállításkor hívd a `flush_all()`-t

Tünet	Lehetséges ok	Javítás
A riasztás nem szűnik meg	`RECOVERING` állapotban a `recover_after_ms` nem telt el, vagy az érték ismét az alacsony küszöb fölé emelkedett	Ellenőrizd a `candidate_since_`, `recover_after_ms` és `state_` értékeket; ha az érték a `low` fölé kerül, a helyes állapot ismét `ALERTING`
Az átlag irreálisan magas	A mérésfajtánkénti `count` és az ablak `sample_count` összekeveredik, vagy hiányzó mérésnél is osztasz	A `MetricAggregate` csak a saját `count`-jával osszon; `count == 0` esetén ne legyen átlag; hiányzó méréshez ne rendelj nullát
A késői minta nyom nélkül eltűnik	A minta már lezárt ablakba érkezett	Növeld az `allowed_lateness_ms_` értékét, vagy javítsd az eseményidőt; a `late_dropped` számláló jelzi az eldobást

13.1.7 Tipikus hibák és félreértések

1. **Túl nagy deadband:** ha a küszöböt 5 °C-ra állítjuk, egy lassú, de fontos 4 °C-os emelkedés soha nem kerül továbbításra, amíg a küszöböt át nem lépi. A deadband értékét a fizikai folyamat időállandójához kell igazítani.
2. **Hiszterézis nélküli riasztás:** ha a riasztás be- és kikapcsolási küszöbe azonos, a hőmérséklet a küszöb körül ingadozva másodpercenként kapcsolgatja a riasztást, ami riasztási fáradtsághoz vezet. Mindig használjunk legalább 1–2 °C széles hiszterézis-sávot.
3. **Helyi idő UTC helyett:** ha a gateway a helyi időzónát használja az ablakokhoz, a nyári időszámítás váltásakor az ablakok eltolódnak. Mindig UTC epoch ms-t használjunk.
4. **Az aggregált átlag elrejti a kiugró értéket:** ha csak átlagot küldünk, egy rövid ideig tartó 40 °C-os kiugrás az átlagban alig látszik. Mindig küldjünk min/max értéket is.
5. **Feldolgozási idő az eseményidő helyett:** ha az ablak kezdetét az első minta érkezési idejéből számítjuk, a hálózati késés vagy az újraindítás eltolja az ablakokat, és ugyanaz a minta más csomóponton másik ablakba kerülhet. A `window_start` mindig a minta `ts` mezőjéből, fix ablakhatárral számoljon.
6. **Globális deadband-állapot:** ha egyetlen `last_` értéket használunk minden eszközre és minden mérésfajtára, az egyik eszköz változása elnyomhatja egy másik eszköz fontos értékét. Az állapot kulcsa legalább a `site`, a `node_id` és a mérésfajta legyen.
7. **A záró részleges ablak elfelejtése:** leállításkor a még nyitott ablak adatai elveszhetnek, ha csak a vízjelre bízunk. A `flush_all()` adja ki ezeket, és a `final_window` mező tegye egyértelművé, hogy az ablak nem a normál vízjel miatt zárult le.

13.1.8 Feladatok

1. **Fogalmi kérdés:** Miért előnyös a deadband-szűrést a gateway-en végezni, ha a szenzor már küld nyers adatot?
2. **Számolási feladat:** Egy eszköz 5 másodpercenként küld 100 B méretű nyers üzenetet. A gateway 120 másodperces tumbling ablakkal aggregál, az aggregált üzenet mérete 150 B. Számítsd ki a napi adatforgalmat nyers és aggregált esetben, valamint a megtakarítás százalékát!

3. **Implementációs feladat:** Módosítsd a `TumblingWindowAggregator` osztályt úgy, hogy 60 másodperces sliding ablakot valósítson meg 10 másodperces csúszással! A laborban készített modulban add hozzá a `SlidingWindowAggregator` osztályt, és teszteld le a `main.cpp`-ben.
4. **Hibakeresési feladat:** A riasztás nem szűnik meg, pedig a hőmérséklet 25 °C-ra csökkent. Mit néznél meg először a kódban és a konfigurációban?

13.1.9 Megoldások

1. A deadband-szűrés a gateway-en azért előnyös, mert a szenzor és a gateway közötti kis hatótávú, alacsony költségű kapcsolaton (például BLE vagy UDP) olcsó a nyers adat továbbítása, de a gateway és a felhő közötti mobil vagy nagy távolságú kapcsolat drága és korlátozott. A gateway a nyers adatból csak a jelentős változást engedi tovább, ezzel csökkenti a felhőbe küldött forgalmat anélkül, hogy a szenzor energiáját vagy a mérési gyakoriságot csökkentené.
2. Nyers esetben a minták száma naponta: $86\,400/5 = 17\,280$. A nyers adatforgalom: $17\,280 \times 100\ \text{B} = 1\,728\,000\ \text{B} \approx 1\,728\ \text{kB}$. Aggregált esetben az ablak hossza 120 s, tehát naponta $86\,400/120 = 720$ ablak. Az aggregált adatforgalom: $720 \times 150\ \text{B} = 108\,000\ \text{B} = 108\ \text{kB}$. A megtakarítás: $1 - 108/1728 \approx 93{,}75\%$.
3. A sliding ablak megvalósításához olyan puffert kell fenntartani, amely a mintákat időrendben tárolja, és minden 10. másodpercben eltávolítja a 60 másodpercnél régebbi elemeket, majd kiszámítja az aktuális ablak statisztikáit. Egy lehetséges megoldás: a `SlidingWindowAggregator` rendelkezik egy `std::deque<Sample>` pufferrel, és minden `slide` hívásnál a `now - window_ms_`-nél régebbi elemeket eldobja, majd a maradékból aggregál.
4. Először ellenőrizném a `ThresholdAlert::update` metódusban, hogy az állapotgép helyese: a hőmérsékletnek a `low` küszöb alá kell esnie, ekkor `RECOVERING` állapotba kerül, és a `candidate_since_` óta el kell telnie a `recover_after_ms` időtartamnak. Ha a hőmérséklet 25 °C, az valószínűleg a `low` (például 28 °C) alatt van, de lehet, hogy a `recover_after_ms` még nem telt le, vagy közben az érték a `low` fölé emelkedett, és az állapot visszaállt `ALERTING`-ba. Azt is meg kell nézni, hogy az `update` metódus rendszeresen megkapja-e az új mintákat, mert esemény nélkül a `RECOVERING` állapot nem tud továbblépni.

13.1.10 Rövid összefoglaló

A gateway-en végzett helyi feldolgozás csökkenti a felhőbe irányuló adatforgalmat és költséget, miközben a kritikus riasztások késleltetése alacsony marad. A deadband-szűrés eszközönként és mérésfajtánként csak a jelentős változásokat továbbítja, a heartbeat pedig állandó érték mellett is ad életjelet. A hiszterézis és a négyállapotú riasztás megakadályozza a küszöb körüli lebegést, ugyanakkor a tartós küszöbátlépéshez időtartamot is megkövetel. A tumbling ablakos aggregálás eseményidőben, fix, fél-nyílt ablakokkal dolgozik; a vízjel kezeli a késői adatot, a záró részleges ablakot pedig `flush_all()` adja ki. Az adott példában a percenkénti aggregálás körülbelül 81%-os megtakarítást eredményez. A laborban elkészült az `edge_aggregator.h/cpp` modul, amely a `DeadbandFilter`, `TumblingWindowAggregator`, `ThresholdAlert` és `EdgeAggregator` osztályokat tartalmazza, és a bemutatott kimenet valódi, lefuttatott C++17 programból származik. **Továbbvitt eredmény.** Az aggregált telemetria továbbra is a `terrasense/<site>/<node>/telemetry` topicra kerül, de az üzenet tartalmaz egy `agg` objektumot `window_start`, `window_end`, `min`, `max`, `avg`, `count` és `final_window` mezőkkel; ezt a konvenciót használja az Idősoros adatbázisok és adatmodellezés és az Eseményvezérelt feldolgozás: Kafka. A helyi riasztás állapota a `terrasense/<site>/<node>/state` retained

topicra kerül `local_alert` mezővel, amelyet az Riasztás: szabályok, értesítés, riasztási fáradtság alfejezet használ fel.

13.1.11 Források és ellenőrzési hiányok

- A modul C++17 szabványt használ (`std::optional`, `std::map`, `std::vector`, `std::uint64_t`), külső könyvtár nélkül. A `main.cpp` által használt epoch ms értékek szintetikusak és rögzítettek, de a közölt kimenet valódi, GCC 13.3.0-val lefuttatott programból származik; a program nem olvas rendszeridőt, ezért a kimenet ismételhető.
- A teljes `ts-gw` Paho-, SQLite- és systemd-integrációja nem futott ebben a laborban. A bemutatott modul önállóan fordult és futott; a gateway éles bekötése, a spool- és MQTT-konfiguráció, valamint a systemd alatti futás ellenőrzése a későbbi laborok feladata.
- A gateway és a felhő közötti MQTT-topicok és az `agg` séma a jelen alfejezetben rögzített konvenció, amelyet az Adatformátumok: JSON, CBOR, Protocol Buffers, SenML alfejezetnek kell véglegesítenie. A `final_window` mező az aggregált üzenet része, ezért a JSON-sémában is szerepelnie kell.
- A sávszélesség-megtakarítási példa számított, illusztratív érték; valós rendszerben a tényleges méretek a séma, a kódolás és a tömörítés függvényében változhatnak.

13.2 Gépi tanulás az eszközön és az edge-en: TinyML áttekintés

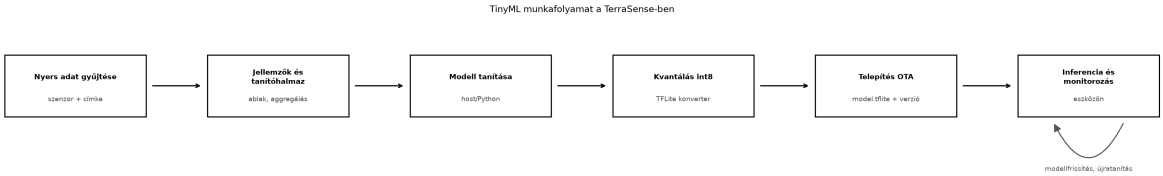
Az üvegházban a Helyi feldolgozás: szűrés, aggregálás, eseménydetektálás alfejezetben megismert determinisztikus szabályok sok esetben elégségesek: ha a talajnedvesség hirtelen csökken, az egyszerű küszöb is jól működik. Vannak azonban olyan hibamódok, amelyek több szenzor együttes, nem nyilvánvaló mintázatában jelennek meg: például egy eltömődött csepegtetőnél a talajnedvesség és a hőmérséklet együtt mutat anomáliát, de egyenként még a normál tartományban maradhat. Ilyenkor a gépi tanulás kiegészítheti a szabályalapú logikát. A kihívás az, hogy az eszköz egy Mikrovezérlő vs. Linuxos processzor: memória, OS nélküli világ alfejezetben tárgyalt erőforrás-korlátos környezet: néhány tíz KiB RAM, néhány száz KiB flash, és nincs virtuális memória. A TinyML pontosan erre a környezetre optimalizálja a modelleket.

13.2.1 TinyML fogalmak és munkafolyamat

Definíció. A *TinyML* olyan gépi tanulási modellek tervezése, betanítása és futtatása, amelyek erőforrás-korlátos beágyazott eszközökön – jellemzően MCU-n vagy kis teljesítményű edge-gatewayen – futnak, gyakran néhány tíz–száz KiB modellmérettel és néhány milliszekundumos vagy az alatti inferenciaidővel.

Definíció. Az *inferencia* a betanított modell futtatása új bemeneti adaton; a *kvantálás* pedig a modell lebegőpontos súlyainak és aktivációinak alacsonyabb bitszélességű, gyakran 8 bites egész számmá alakítása úgy, hogy a pontosság lehetőleg alig csökkenjen.

A TinyML-munkafolyamat két részre válik: a tanítás és a modellátalakítás tipikusan hoston vagy felhőben fut, a telepített modell pedig az eszközön. Az alábbi folyamatábra a TerraSense szempontjából mutatja a lépéseket.



13.2. ábra: TinyML munkafolyamat a TerraSense-ben - A tanítás hoston fut, a kvantált modell OTA-tartalomként kerül az eszközre.

A folyamat a címkézett vagy múltból ismert nyers adatok gyűjtésével indul. Ebből a Helyi feldolgozás: szűrés, aggregálás, eseménydetektálás alfejezetben megismert ablakos jellemzőket számítjuk, például átlagot, szórást, minimumot, maximumot és trendet. A tanítás hoston, Python-környezetben történik; majd a modellt int8-as TensorFlow Lite formátumba konvertáljuk. A kvantált modellt verziózott artefaktumként telepítjük az eszközre OTA-n keresztül. Az eszközön az inferencia fut, és a kimenetet a szabályalapú logika vagy riasztási küszöb használja fel.

13.2.2 Mikor éri meg a TinyML?

A TinyML nem helyettesíti automatikusan a szabályokat. A döntésnél a késleltetés, az adatvédelem, a sávszélesség és az energiaigény mellett a fejlesztési költséget is figyelembe kell venni.

Szempont	Szabályalapú / egyszerű küszöb	TinyML az eszközön
Fejlesztési idő	rövid, azonnal ellenőrizhető	adatgyűjtés, tanítás, kiértékelés kell
Determinisztikusság	teljes	közelítő, statisztikai
Erőforrásigény	minimális	modell + arena + CPU
Sok jellemző, nem lineáris kapcsolat	nehezen karbantartható	jól kezeli
Frissítés	kódváltozás, firmware-OTA	modell-OTA lehetséges
Kritikus biztonsági döntés	előnyös	csak kiegészítő lehet

A lokális inferencia legfontosabb előnyei:

- **Késleltetés:** egy szelepzárás a felhőn keresztül másodperceket vehet igénybe; lokális modellnél milliszekundum.
- **Adatvédelem:** a nyers szenzoradat nem hagyja el az eszközt, csak az eredmény, például egy osztálycímke vagy anomáliajelzés.
- **Sávszélesség:** óránkénti nyers telemetriai csomagok helyett csak anomália esetén küldünk riasztást.
- **Energia:** a rádióadás elhagyása gyakran nagyobb energiamegtakarítás, mint maga a modell futtatása.

Ugyanakkor a modell becslési hibája nem okozhat közvetlen veszélyes állapotot; a végrehajtó logika maradjon determinisztikus.

13.2.3 Modellméret, kvantálás és inferenciaidő

Egy kis, teljesen kapcsolt neurális háló paraméterszáma könnyen számítható. Ha a bemeneti jellemzők száma n_{in}, a rejtett rétegben n_h neuron van, a kimeneti osztályok száma pedig n_{out}, akkor a paraméterszám:

$$N_p = n_{\text{in}} \cdot n_h + n_h + n_h \cdot n_{\text{out}} + n_{\text{out}}$$

A műveletigényt a szorzás-összegzés (MAC) műveletek száma közelíti:

$$N_{\text{MAC}} = n_{\text{in}} \cdot n_h + n_h \cdot n_{\text{out}}$$

A kvantálásnál egy lebegőpontos r értékből egész q értéket képzünk skála és nullapont segítségével:

$$q = \text{round}\left(\frac{r}{s}\right) + z, \quad r \approx s \cdot (q - z)$$

A skála s az értéktartomány és a 8 bites tartomány hányadosa, a z nullapont pedig azt mondja meg, hogy a nulla lebegőpontos érték melyik egész számra képződik le. Az int8-as modell a 32 bites lebegőpontos modellnél körülbelül négyszer kisebb tárhelyet igényel, és az MCU fixpontos vagy DSP-utasításokkal gyorsabban dolgozza fel.

A Cortex-M osztályú MCU-kon egy egyszerű teljesen kapcsolt háló inferenciaideje jellemzően mikroszekundumos–milliszekundumos nagyságrendű. A pontos érték az órajeltől, a DSP-kiterjesztéstől és az interpreter overheadjétől függ; a `native_sim` időzítése nem hardverhű.

13.2.4 TFLite Micro / LiteRT röviden

A Google által korábban *TensorFlow Lite for Microcontrollers* néven ismert keretrendszer – amelyet az újabb dokumentáció *LiteRT for Microcontrollers* néven említ – kifejezetten MCU-kra tervezett interpretált futtatókörnyezet. A modell FlatBuffers formátumú, amelyet C-tömbként vagy fájlból beolvasva lehet beágyazni. Az interpreter fix méretű, statikusan allokált memóriaarénát használ; ez illeszkedik a Mikrovezérlő vs. Linuxos processzor: memória, OS nélküli világ alfejezetben tárgyalt statikus allokációs elvhez.

Az alábbi API-vázlat a TFLite Micro/LiteRT hívási szerződését mutatja: ellenőrzi az `AllocateTensors` és az `Invoke` eredményét, a tenzor típusát, alakját, méretét és kvantálási paramétereit, a bemeneti puffer hosszát, majd tényleges argmaxot számol. A vázlat **nem lefordított példa**, mert a TFLite Micro könyvtár és a `model.h` nincs a labor környezetében; a pontos függvény- és mezőneveket a telepített kiadásban ellenőrizni kell. A vázlat célja annak rögzítése, hogy egy termelési wrappernek mely hibákat kell kontrolláltan kezelnie, nem egy működő osztályozó helyettesítése.

```
#include <cstddef>
#include <cstdint>
#include "model.h"  // g_model: a konvertált, int8 modell
#include "tensorflow/lite/micro/micro_interpreter.h"
#include "tensorflow/lite/micro/micro_mutable_op_resolver.h"

namespace {
constexpr std::size_t kFeatureCount = 6;
constexpr std::size_t kClassCount = 3;
```

```
constexpr std::size_t kArenaSize = 8 * 1024;
alignas(16) std::uint8_t arena[kArenaSize];
tflite::MicroInterpreter* interpreter = nullptr;
TfLiteTensor* input = nullptr;
TfLiteTensor* output = nullptr;

bool check_tensor(const TfLiteTensor* tensor, std::size_t expected,
                  const char* name, const char** error) {
  if (tensor == nullptr) {
    *error = "hiányzó tensor";
    return false;
  }
  if (tensor->type != kTfLiteInt8) {
    *error = "a tensor nem int8";
    return false;
  }
  if (tensor->dims->size != 2 || tensor->dims->data[1] != expected) {
    *error = "váratlan tensor-alak";
    return false;
  }
  if (tensor->bytes != expected) {
    *error = "váratlan tensor-méret";
    return false;
  }
  if (tensor->params.scale <= 0.0F) {
    *error = "nem pozitív kvantálási skála";
    return false;
  }
  if (tensor->params.zero_point < -128 ||
      tensor->params.zero_point > 127) {
    *error = "a nullapont kívül esik az int8 tartományon";
    return false;
  }
  (void)name;
  return true;
}
}  // namespace

bool ml_init(const char** error) {
  if (error == nullptr) {
    return false;
  }
  static tflite::MicroMutableOpResolver<2> resolver;
  if (resolver.AddFullyConnected() != kTfLiteOk ||
      resolver.AddSoftmax() != kTfLiteOk) {
    *error = "az operátor nem regisztrálható";
    return false;
  }
  static tflite::MicroInterpreter static_interpreter(
      tflite::GetModel(g_model), resolver, arena, kArenaSize);
  interpreter = &static_interpreter;
  if (interpreter->AllocateTensors() != kTfLiteOk) {
    *error = "AllocateTensors sikertelen";
    return false;
  }
  input = interpreter->input(0);
  output = interpreter->output(0);
  if (!check_tensor(input, kFeatureCount, "input", error) ||
      !check_tensor(output, kClassCount, "output", error)) {
    return false;
  }
  return true;
}
```

```
bool ml_classify(const std::int8_t* features, std::size_t count,
                 int* class_id, const char** error) {
  if (error == nullptr) {
    return false;
  }
  if (features == nullptr || class_id == nullptr) {
    *error = "nullptr bemenet vagy kimenet";
    return false;
  }
  if (count != kFeatureCount) {
    *error = "rövid feature-puffer";
    return false;
  }
  if (interpreter == nullptr || input == nullptr || output == nullptr) {
    *error = "az interpreter nincs inicializálva";
    return false;
  }
  for (std::size_t i = 0; i < count; ++i) {
    input->data.int8[i] = features[i];
  }
  if (interpreter->Invoke() != kTfLiteOk) {
    *error = "Invoke sikertelen";
    return false;
  }
  int best = 0;
  for (std::size_t i = 1; i < kClassCount; ++i) {
    if (output->data.int8[i] > output->data.int8[best]) {
      best = static_cast<int>(i);
    }
  }
  *class_id = best;
  return true;
}
```

A `check_tensor` a hiányzó tenzort, a nem int8 típust, a váratlan alakot és méretet, a nem pozitív skálát és az int8 tartományon kívüli nullapontot egyaránt hibává teszi. Az `ml_classify` a `nullptr`, a rövid feature-puffer és az inicializálatlan interpreter esetét is elutasítja, majd az `Invoke` hibáját továbbadja. Az argmax az int8 kimeneti tenzor értékein fut; a kimeneti valószínűséghez a `scale` és a `zero_point` ismeretében lehet visszakvantálni.

13.2.4.1 Önálló ellenőrző runner

A TFLite Micro telepítése nélkül is ellenőrizhető ugyanez a szerződés egy minimális, önálló C++17 runnerrel. A `tinyml_runner.cpp` nem TFLite-modellt futtat, hanem egy `Model` interfészen keresztül ugyanazokat az ellenőrzéseket gyakorolja: tensor típus és hossz, skála, nullapont, bemeneti hossz, `invoke` eredménye és argmax. A jó modell két különböző bemenetre két különböző osztályt ad; a szándékosan hibás modellek és a rövid puffer kontrollált hibát eredményeznek.

```
#include <cstddef>
#include <cstdint>
#include <iomanip>
#include <iostream>
#include <string>
#include <vector>

namespace terrasense {

enum class TensorType { kInt8, kFloat32 };
```

```
struct Tensor {
  TensorType type{TensorType::kInt8};
  std::size_t length{0};
  float scale{0.0F};
  int zero_point{0};
  std::vector<std::int8_t> data;
};

struct Model {
  Tensor input;
  Tensor output;
  bool (*invoke)(Model&, std::string&) = nullptr;
};

bool invoke_ok(Model& model, std::string&) {
  if (model.input.data.size() != 6 || model.output.data.size() != 3) {
    return false;
  }
  const bool negative = model.input.data[0] < 0;
  model.output.data[0] = static_cast<std::int8_t>(negative ? 100 : -100);
  model.output.data[1] = 0;
  model.output.data[2] = static_cast<std::int8_t>(negative ? -100 : 100);
  return true;
}

bool invoke_fail(Model&, std::string& error) {
  error = "szándékos operátorhiba";
  return false;
}

bool valid_tensor(const Tensor& tensor, std::size_t expected,
                  const char* name, std::string& error) {
  if (tensor.type != TensorType::kInt8) {
    error = std::string(name) + ": nem int8 tensor";
    return false;
  }
  if (tensor.length != expected) {
    error = std::string(name) + ": elvárt " + std::to_string(expected) +
            " elem, kapott " + std::to_string(tensor.length);
    return false;
  }
  if (tensor.scale <= 0.0F) {
    error = std::string(name) + ": a skála nem pozitív";
    return false;
  }
  if (tensor.zero_point < -128 || tensor.zero_point > 127) {
    error = std::string(name) + ": a nullapont kívül esik az int8 tartományon";
    return false;
  }
  return true;
}

bool classify(Model& model, const std::vector<std::int8_t>& features,
              int& label, float& score, std::string& error) {
  if (!valid_tensor(model.input, 6, "input", error)) {
    return false;
  }
  if (!valid_tensor(model.output, 3, "output", error)) {
    return false;
  }
  if (features.size() != model.input.length) {
    error = "input hossz: elvárt " + std::to_string(model.input.length) +
            ", kapott " + std::to_string(features.size());
```

```
    return false;
  }
  if (model.invoke == nullptr) {
    error = "hiányzó invoke operátor";
    return false;
  }
  model.input.data = features;
  if (!model.invoke(model, error)) {
    return false;
  }
  int best = 0;
  for (int i = 1; i < 3; ++i) {
    if (model.output.data[i] > model.output.data[best]) {
      best = i;
    }
  }
  label = best;
  score = model.output.scale *
          static_cast<float>(model.output.data[best] - model.output.zero_point);
  return true;
}

Model make_model() {
  Model model;
  model.input.length = 6;
  model.input.scale = 0.1F;
  model.input.data.assign(6, 0);
  model.output.length = 3;
  model.output.scale = 0.05F;
  model.output.data.assign(3, 0);
  model.invoke = &invoke_ok;
  return model;
}

void run_case(const char* name, Model& model,
              const std::vector<std::int8_t>& features) {
  int label = -1;
  float score = 0.0F;
  std::string error;
  const bool ok = classify(model, features, label, score, error);
  std::cout << name << ": ";
  if (ok) {
    std::cout << "ok label=" << label << " score=" << std::fixed
              << std::setprecision(1) << score;
  } else {
    std::cout << "hiba: " << error;
  }
  std::cout << "\n";
}

} // namespace terrasense

int main() {
  using namespace terrasense;
  const std::vector<std::int8_t> normal{-10, 0, 0, 0, 0, 0};
  const std::vector<std::int8_t> valve{10, 0, 0, 0, 0, 0};

  Model good = make_model();
  run_case("jó modell, normal", good, normal);
  run_case("jó modell, valve", good, valve);

  Model short_input = make_model();
  run_case("rövid feature-puffer", short_input,
```

```
            std::vector<std::int8_t>(5, 0));

  Model bad_type = make_model();
  bad_type.input.type = TensorType::kFloat32;
  run_case("rossz tensor típus", bad_type, normal);

  Model bad_output = make_model();
  bad_output.output.length = 2;
  run_case("rossz kimeneti hossz", bad_output, normal);

  Model bad_operator = make_model();
  bad_operator.invoke = &invoke_fail;
  run_case("hibás operátor", bad_operator, normal);
  return 0;
}
```

Build és futtatás:

```
g++ -std=c++17 -Wall -Wextra -Wpedantic -Werror -O2 \
    -o tinyml_runner tinyml_runner.cpp
./tinyml_runner
```

Tényleges kimenet (GCC 13.3.0, C++17):

```
jó modell, normal: ok label=0 score=5.0
jó modell, valve: ok label=2 score=5.0
rövid feature-puffer: hiba: input hossz: elvárt 6, kapott 5
rossz tensor típus: hiba: input: nem int8 tensor
rossz kimeneti hossz: hiba: output: elvárt 3 elem, kapott 2
hibás operátor: hiba: szándékos operátorhiba
```

A kimenet első két sora igazolja, hogy a két bemenet két külön osztályt kap (`label=0` és `label=2`). A további sorok a rövid feature-puffert, a rossz tensor típust, a rossz kimeneti hosszt és a szándékosan hibás operátort külön, olvasható hibaüzenettel utasítják el. Ez a runner a TFLite API viselkedésének szerződéses modellje, nem helyettesíti a valódi TFLite Micro buildet és a hardveres mérést.

13.2.5 Anomáliadetektálás a TerraSense-ben ötletszinten

A TerraSense-ben érdemes két szinten gondolkodni:

1. **ts-node szintjén:** egy 6 bemenetes osztályozó, amely a Helyi feldolgozás: szűrés, aggregálás, eseménydetektálás alfejezetben kiszámított ablakjellemzőkből három kimeneti osztályt becsül: `normal`, `sensor_fault`, `valve_fault`. A kimeneti valószínűség alapján a determinisztikus állapotgép dönthet arról, hogy azonnal riaszt, vagy csak megnövelt mintavételi gyakorisággal folytatja a megfigyelést.
2. **ts-gw szintjén:** a gateway több csomópont adatát összevonva detektálhat üvegház-szintű anomáliát, például térben inkonzisztens hőmérsékleti mintázatot. Itt C++-ban ugyanaz a TFLite-runtime futtatható Linux-processzon, nagyobb arénával és nagyobb modellel.

A modell kimenete nem helyettesíti az állapotgépet: a szelepzárás és a hibajelzés szabályai maradjanak ellenőrizhető, determinisztikus feltételekhez kötve. A TinyML a gyanú detektálására szolgál, nem a biztonsági döntés közvetlen meghozatalára.

13.2.6 Modellfrissítés mint OTA-tartalom

A modell nem kód, hanem adat: egy `model.tflite` fájl, amely verzióval, mérettel és kriptográfiai hashel rendelkezik. Ugyanaz az aláírási és visszagörgetési mechanizmus alkalmazható rá, mint a firmware-re, ezért a OTA-frissítés: A/B képek, rollback, delta, fokozatos kiadás alfejezetben tárgyalt artefaktumkezeléshez csatlakozik. A modellverziót a bemeneti jellemzők vektorának verziójával együtt kell kezelni: ha a jellemzőkészlet változik, a régi modellt nem szabad az új firmware-en futtatni, és fordítva.

Továbbvitt eredmény. A TerraSense-modellt a laborban `~/iot-labor/i040/model/model.tflite` néven, a metaadatokat pedig `model.json` néven tároljuk. A `model.json` tartalmazza a `version`, a `feature_version`, a `hash_sha256`, a `model_size_bytes` és az `input_scale`/`input_zero_point` mezőket. A `hash_sha256` és a `model_size_bytes` értékét nem kézzel írjuk be: a csomagoló lépés a tényleges `model.tflite` fájlból számítja ki, és a `null` értéket az OTA-kapu elutasítja. Ezt a konvenciót és a fájlnevet használja majd a OTA-frissítés: A/B képek, rollback, delta, fokozatos kiadás alfejezet.

13.2.7 Kidolgozott példa: 6 bemenetes anomáliaosztályozó

Tegyük fel, hogy a ts-node modellje a következő: 6 bemeneti jellemző, 16 neuron egy rejtett rétegben, és 3 kimeneti osztály. A paraméterszám:

$$N_p = 6 \cdot 16 + 16 + 16 \cdot 3 + 3 = 96 + 16 + 48 + 3 = 163$$

A szükséges MAC-műveletek száma:

$$N_{\text{MAC}} = 6 \cdot 16 + 16 \cdot 3 = 96 + 48 = 144$$

Az int8-as súlyok és torzítások együttes mérete 163 B, míg float32 esetén ugyanez közelítőleg 652 B. A FlatBuffers-modellhez hozzáadódik a gráf és a tenzorleírók overheadje, így a teljes int8-as modell gyakorlatban 1–2 KiB nagyságrendű lehet, a float32 pedig 3–4 KiB. Az SRAM-aréna a rejtett réteg 16 int8-as aktivációja, a 6 bemeneti és 3 kimeneti tenzor, valamint az interpreter belső állapota miatt 4 KiB-tal kényelmesen kiszolgálható. Ezek a paraméterszámból és a kvantálási bitszélességből számított értékek; a tényleges `.tflite` fájlméretet és a modell hibaarányát a konverzió és a kiértékelés után kell megmérni. A következő szakasz ehhez ad egy szintetikus referenciaadatot és egy futtatható baseline-t, amely nem TFLite-modell, hanem a jellemző- és kvantálási lánc reprodukálhatóságát ellenőrzi.

Időbecslés illusztratív célból: egy 64 MHz-es Cortex-M4 mag 144 MAC műveletet elvileg néhány mikroszekundum alatt elvégez, de a memória-elérés, az aktiváció és az interpreter overheadje miatt a gyakorlatban inkább 50–500 µs nagyságrendű futásidő várható. Ez nem mért adat, és a `native_sim` futtatás nem helyettesíti a valós hardveres mérést. A lényeg az, hogy a modell az óránkénti vagy eseményvezérelt mintavétel mellett energia szempontból elhanyagolható a rádióadáshoz képest, ha miatta egy MQTT-csomag elhagyható.

13.2.8 Referenciaadat és baseline kiértékelés

A fejezet eddigi modellméretei analitikus becslések, nem egy betanított háló mért értékei. A H161-ben hiányolt reprodukálhatóság érdekében ez a szakasz egy kis, szintetikus referenciaadatot és egy csak a Python standard könyvtárát használó baseline-t ad. Ez **nem TFLite-modell**, és nem állítja, hogy valódi üvegházas tanítóadat vagy éles pontosság áll rendelkezésre; a cél a jellemzővektor, a kvantálás, a centroidok és az argmax szerződésének ellenőrzése. A 18 minta osztályonként 6 sort tartalmaz; osztályonként az első 4 tanító, az utolsó 2 független teszt. A szintetikus adat szándékosan elkülöníthető, ezért a baseline eredménye nem általánosítási mérés.

Fájl: `data/reference.csv`

```
temp_mean,temp_min,temp_max,humidity_mean,soil_mean,soil_delta,label
22.1,21.2,23.1,56.0,42.0,0.2,normal
22.4,21.5,23.3,57.5,41.5,-0.1,normal
22.0,21.0,23.0,55.0,43.0,0.4,normal
22.8,21.8,23.8,58.0,40.5,-0.2,normal
22.2,21.1,23.2,56.5,44.0,0.3,normal
22.6,21.6,23.6,57.0,41.0,0.0,normal
22.5,21.4,23.5,8.0,42.0,0.1,sensor_fault
22.3,21.3,23.3,6.5,41.5,0.0,sensor_fault
22.7,21.6,23.7,9.0,43.0,-0.1,sensor_fault
22.2,21.2,23.2,7.5,40.5,0.2,sensor_fault
22.4,21.5,23.4,5.5,42.5,0.0,sensor_fault
22.6,21.7,23.6,8.5,41.0,0.1,sensor_fault
25.4,24.4,27.0,47.0,25.0,-8.0,valve_fault
25.8,24.8,27.4,46.0,23.0,-7.5,valve_fault
25.2,24.2,26.8,48.0,27.0,-8.5,valve_fault
25.6,24.6,27.2,45.0,22.0,-7.0,valve_fault
25.0,24.0,26.6,49.0,28.0,-9.0,valve_fault
25.9,24.9,27.5,46.5,24.0,-8.2,valve_fault
```

Fájl: `scripts/baseline_eval.py`

```python
#!/usr/bin/env python3
"""Kis, csak standard könyvtárat használó int8 nearest-centroid baseline."""
import csv
import math
import sys
from collections import Counter, defaultdict

FEATURES = ["temp_mean", "temp_min", "temp_max",
            "humidity_mean", "soil_mean", "soil_delta"]
LABELS = ["normal", "sensor_fault", "valve_fault"]
SCALE = 0.1
ZERO_POINT = 0


def load_rows(path):
    rows = []
    with open(path, newline="", encoding="utf-8") as handle:
        for row in csv.DictReader(handle):
            rows.append({
                "features": [float(row[name]) for name in FEATURES],
                "label": row["label"],
            })
    return rows


def mean(values):
    return sum(values) / len(values)
```

```
def pstdev(values):
    m = mean(values)
    return math.sqrt(sum((value - m) ** 2 for value in values) / len(values))


def quantize(value, feature_mean, feature_std):
    z = (value - feature_mean) / feature_std
    q = round(z / SCALE) + ZERO_POINT
    return max(-128, min(127, q))


def main(path):
    rows = load_rows(path)
    by_label = defaultdict(list)
    for row in rows:
        by_label[row["label"]].append(row)

    train = []
    test = []
    for label in LABELS:
        class_rows = by_label[label]
        if len(class_rows) < 6:
            raise SystemExit(f"kevés minta a {label} osztályban")
        train.extend(class_rows[:4])
        test.extend(class_rows[4:])

    feature_means = [mean([row["features"][i] for row in train])
                     for i in range(len(FEATURES))]
    feature_stds = [pstdev([row["features"][i] for row in train])
                    for i in range(len(FEATURES))]
    if any(std == 0 for std in feature_stds):
        raise SystemExit("nulla szórású jellemző a tanítóhalmazban")

    def qrow(row):
        return [quantize(value, feature_means[i], feature_stds[i])
                for i, value in enumerate(row["features"])]

    centroids = {}
    for label in LABELS:
        class_q = [qrow(row) for row in train if row["label"] == label]
        centroids[label] = [round(mean([vec[i] for vec in class_q]))
                            for i in range(len(FEATURES))]

    predictions = []
    confusion = Counter()
    for row in test:
        q = qrow(row)
        distances = {}
        for label in LABELS:
            distances[label] = sum(abs(q[i] - centroids[label][i])
                                   for i in range(len(FEATURES)))
        predicted = min(LABELS, key=lambda label: (distances[label], label))
        predictions.append((row["label"], predicted))
        confusion[(row["label"], predicted)] += 1

    correct = sum(1 for expected, predicted in predictions if expected == predicted)
    accuracy = correct / len(predictions)
    print(f"tanító minták: {len(train)}, teszt minták: {len(test)}")
    print(f"jellemző-skála: {SCALE}, zero_point: {ZERO_POINT}")
    print("int8 centroidok:")
    for label in LABELS:
        print(f"  {label}: {centroids[label]}")
```

```
    print(f"centroid tárhely: {len(LABELS) * len(FEATURES)} B")
    print(f"teszt pontosság: {correct}/{len(predictions)} = {accuracy:.3f}")
    print("konfúziós mátrix (sor=várt, oszlop=becsült):")
    print("  " + " ".join(f"{label:>12}" for label in LABELS))
    for expected in LABELS:
        counts = [confusion[(expected, predicted)] for predicted in LABELS]
        print(f"  {expected:>12} " + " ".join(f"{count:>12}" for count in counts))
    if accuracy < 0.5:
        return 1
    return 0


if __name__ == "__main__":
    if len(sys.argv) != 2:
        raise SystemExit("használat: baseline_eval.py data/reference.csv")
    sys.exit(main(sys.argv[1]))
```

Futtatás:

```
python3 scripts/baseline_eval.py data/reference.csv
```

Tényleges kimenet (Python 3, szintetikus referenciaadat):

```
tanító minták: 12, teszt minták: 6
jellemző-skála: 0.1, zero_point: 0
int8 centroidok:
  normal: [-7, -7, -7, 10, 7, 7]
  sensor_fault: [-6, -7, -6, -14, 7, 7]
  valve_fault: [14, 14, 14, 4, -14, -14]
centroid tárhely: 18 B
teszt pontosság: 6/6 = 1.000
konfúziós mátrix (sor=várt, oszlop=becsült):
        normal sensor_fault  valve_fault
        normal            2            0            0
  sensor_fault            0            2            0
   valve_fault            0            0            2
```

A baseline a tanítóhalmazból jellemzőnként átlagot és szórást számol, a jellemzőket z-score után 0,1-es skálával int8-ra kvantálja, majd osztályonként int8 centroidot képez. A becslés a centroidoktól vett L1-távolság minimuma, tehát a kimenet argmaxa; a `centroid tárhely: 18 B` a három osztály hat jellemzőjének int8 mérete. A 6/6 tesztpontosság azt igazolja, hogy a három szintetikus osztály ezen a kis mintán megkülönböztethető, nem azt, hogy egy valódi TFLite modell vagy egy éles üvegház adatán is jó lenne. A valódi `.tflite` méretét, hibaarányát és MCU-s inferenciaidejét a modell konvertálása és hardveres mérése után kell rögzíteni; ezek ebben a fejezetben nem mért értékek.

13.2.9 Ellenőrzés és hibakeresés

A modell méretét a konvertálás után `ls -l model.tflite`, a beágyazott C-tömb méretét pedig a bináris `size` kimenete mutatja. A pontos tenzorméreteket a TFLite interpreter debug-kimenete vagy a modellvizsgáló eszköz írja ki. A gyakori hibákat az alábbi táblázat foglalja össze.

Tünet	Lehetséges ok	Javítás
`AllocateTensors()` hibát ad	túl kicsi arena vagy hibás modell	növeld a `kArenaSize` értéket, vagy ellenőrizd a modell tenzorméreteit; a hibát ne nyeld el

Tünet	Lehetséges ok	Javítás
A modell mindig ugyanazt az osztályt adja	kiegyensúlyozatlan tanítóhalmaz, rossz küszöb vagy hiányzó argmax	tanítóadatok átsúlyozása, küszöb kalibrálása; a kimeneti tenzor argmaxát számold ki
Hoston jó, eszközön rossz	kvantálási hiba, nem egyező bemeneti normalizálás	reprezentatív kalibráció, ugyanaz a skála a jellemzőknél
Rövid feature-puffer csendben átmásolódik	nincs bemeneti hosszellenőrzés	a `count == kFeatureCount` feltétel legyen kötelező; rövid pufferre kontrollált hiba
Túl nagy a bináris méret	nem használt műveletek is belinkelődtek	`MicroMutableOpResolver` csak a szükséges op-kkal
Frissített modell rossz eredményt ad	jellemzőverzió-eltérés	`feature_version` ellenőrzése OTA előtt
A manifest hash-e helyőrző	kézzel írt szöveg vagy `null` került a csomagba	a csomagoló számítsa ki a SHA-256-ot; az OTA-kapu a `null` vagy nem 64 hexadecimális karakteres hash-t utasítsa el

13.2.10 Tipikus hibák és félreértések

1. **A float32 modell közvetlen futtatása MCU-n.** A 32 bites lebegőpontos műveletek lassúak és sok memóriát foglalnak; int8-ra kvantálás nélkül a modell gyakran nem fér be, vagy túl lassú lesz.
2. **A modell kimenetét közvetlen biztonsági végrehajtásra használni.** A gépi modell statisztikai becslés, nem garantált döntési logika. A vészleállító, watchdog és szelepvezérlő állapotgép maradjon determinisztikus.
3. **Adatszivárgás a tanítás és a tesztelés között.** Ha ugyanaz az ablak szerepel a tanító- és a tesztadatokban, a pontosság hamisan magas lesz; a TerraSense-ben idősor esetén időalapú felosztást kell használni.
4. **A modellfrissítést nem kezelik OTA-artefaktumként.** A modell is tartalmazhat rosszindulatú vagy hibás tartalmat, ezért aláírás, verzió és visszagörgetés nélkül ugyanolyan kockázatot jelent, mint egy rossz firmware.
5. **A wrapper nem ellenőrzi a tensor alakját, típusát és a bemenet hosszát.** A rövid feature-puffer vagy egy float32 tenzor csendben hibás memóriaterületet olvashat, és látszólag működő, de értelmetlen osztályt adhat. A típus-, alak-, méret- és kvantálási ellenőrzés legyen az első művelet, az `Invoke` eredményét pedig kötelező vizsgálni.
6. **A manifestbe helyőrző hash kerül.** A `labor-only-placeholder-hash` vagy a `null` nem SHA-256; ha az OTA ezt elfogadja, a letöltött modell integritása nem ellenőrizhető. A hash-t a csomagoló lépés számítsa ki a tényleges fájlból, és a kapu a hiányzó vagy hibás hash-t utasítsa el.

13.2.11 Feladatok

1. **Fogalmi/döntési kérdés.** Sorolj fel két TerraSense-helyzetet, ahol a szabályalapú logika elegendő, és egyet, ahol a TinyML valódi előnyt ad. Indokold röviden.
2. **Számítási feladat.** Egy modell bemenete 8 jellemző, rejtett rétege 20 neuron, kimenete 4 osztály. Számítsd ki a paraméterszámot, a MAC-ek számát és a súlyok int8-as, illetve float32-es tárhelyét (FlatBuffers-overhead nélkül).

3. **Implementációs feladat.** Készíts a TerraSense számára egy `model.json` leírót a Kidolgozott példa 6 bemenetes modelljéhez. A fájl tartalmazzon `version`, `feature_version`, `hash_sha256`, `model_size_bytes`, `model_file` és kvantálási paramétereket. A hash és a méret még ne legyen kitalált érték: a forrássablonban `null` szerepeljen, és a csomagoló lépés töltse ki a tényleges `model.tflite` fájlból. Ezt a fájlformátumot előkészíti a OTA-frissítés: A/B képek, rollback, delta, fokozatos kiadás alfejezet.
4. **Hibakeresési feladat.** A ts-node-on a modell a laborban jól működik, de a valós hardveren mindig `normal` osztályt ad. Nevezz meg négy lehetséges okot, és mindegyikhez egy-egy diagnosztikai lépést.

13.2.12 Megoldások

1. A küszöbalapú logika elég például a talajnedvesség abszolút alsó határának figyeléséhez és a hőmérséklet fagyásjelzéséhez. A TinyML akkor előnyös, ha egy hibás szelep és egy enyhén emelkedő talajnedvesség-mintázat együttesen utal eltömődésre, de a külön-külön vett értékek még normálnak tűnnek.
2. $N_p = 8 \cdot 20 + 20 + 20 \cdot 4 + 4 = 160 + 20 + 80 + 4 = 264$ paraméter. $N_{\text{MAC}} = 8 \cdot 20 + 20 \cdot 4 = 160 + 80 = 240$. Int8: 264 B; float32: 1 056 B.
3. Egy lehetséges `model.json`:

```
{
  "schema_version": 1,
  "name": "ts-node-anomaly-v1",
  "version": "1.0.0",
  "feature_version": 1,
  "format": "tflite-int8",
  "model_file": "model.tflite",
  "model_size_bytes": null,
  "hash_sha256": null,
  "hash_status": "pending_packaging",
  "input_scale": 0.235,
  "input_zero_point": -43,
  "output_labels": ["normal", "sensor_fault", "valve_fault"]
}
```

A `null` itt nem helyettesítő hash, hanem annak jelölése, hogy a forrássablon még nem tartalmaz csomagolt modellt. A csomagoló lépés a `model.tflite` fájlból számítja ki a `sha256sum` kimenetét és a `stat -c%s` méretet, majd csak akkor írja be a manifestbe, ha a hash 64 hexadecimális karakter és a méret pozitív. Az OTA-kapu a `null`, a `hash_status != "packaged"` vagy a nem megfelelő hash esetét elutasítja, így helyőrző soha nem kerülhet a frissítési folyamatba. Az `input_scale` és `input_zero_point` szintén a konvertált modellből származó érték; a fenti számok szemléltetők, nem egy létező modell mérési eredményei.

4. Négy lehetséges ok: (a) a bemeneti jellemzők normalizálása a hoston más skálát használ, mint az eszközön – ellenőrizd a bemeneti tenzor statisztikáját; (b) a modell int8-ra kvantálása rossz kalibrációs adatokkal történt – kvantálás előtt gyűjts reprezentatív valós mintákat; (c) a kimeneti küszöb túl magas – csökkentsd az anomáliaosztály valószínűségi küszöbét; (d) a szenzorok nyers értékei eltérnek a labor-szimulációtól – hasonlítsd össze a nyers ADC- vagy szenzorértékeket a tanító-adatokkal.

13.2.13 Rövid összefoglaló

A TinyML lehetővé teszi, hogy a TerraSense ts-node és ts-gw rétege lokálisan, kis késleltetéssel, adatvédelem és nagyobb rádióforgalom nélkül detektáljon anomáliákat. A modell méretét

a paraméterszám és az int8 kvantálás határozza meg; egy 6 bemenetes, 16 rejtett neuronos osztályozó mindössze 144 MAC műveletet és néhány KiB-ot igényel. A munkafolyamat a címkézett adatgyűjtéstől a tanításon és kvantáláson át a verziózott OTA-telepítésig tart. A fejezet TFLite API-vázlata nem lefordított példa, hanem a wrapper szerződését mutatja; a külön futtatott `tinyml_runner` a típus-, méret-, kvantálási- és argmax-ellenőrzéseket gyakorolja, a szintetikus referenciaadaton futó baseline pedig a jellemző- és kvantálási láncot teszi reprodukálhatóvá. A modellek valódi mérete, hibaaránya és MCU-s inferenciaideje továbbra is mérési feladat; a modell kimenete csak gyanút ad, a biztonsági végrehajtás determinisztikus marad. **Továbbvitt eredmény:** a `~/iot-labor/i040/model/model.tflite` és `model.json` elnevezés, valamint a `feature_version`, `hash_sha256`, `model_size_bytes` és `hash_status` metaadat-konvenció, amelyet a OTA-frissítés: A/B képek, rollback, delta, fokozatos kiadás alfejezet használ fel.

13.2.14 Források és ellenőrzési hiányok

- Google LiteRT / TensorFlow Lite for Microcontrollers dokumentáció: https://ai.google.dev/edge/litert/microcontrollers/overview
- Zephyr `tflite-micro` integrációs dokumentáció
- TFLite modellkonverter és kvantálási útmutató

Ellenőrzési igény: a LiteRT/TFLite Micro aktuális termék- és API-nevei, a `MicroInterpreter`, `MicroMutableOpResolver` és a `data.int8` mező pontos aláírása a használt könyvtári verzióban ellenőrizendő. A TFLite API-vázlat nem része a lefordított laborpéldának; a `tinyml_runner.cpp` önállóan, GCC 13.3.0-val fordult és futott, a `baseline_eval.py` pedig Python 3 alatt futott. A szintetikus referenciaadat és a 6/6 baseline-pontosság nem valós üvegházi mérés és nem általánosítási teljesítmény; a valódi TFLite modell, a `.tflite` méret, a hibaarány és a hardveres inferenciaidő továbbra is mérési feladat. A példában szereplő modellméretek és futásidők számított vagy illusztratív jellegűek; valós hardveres mérést nem helyettesítenek.

V. RÉSZ – Backend és enterprise integráció

14 Backend-architektúra és a JEE-híd

14.1 Az IoT-backend felépítése: ingest, tárolás, API, alkalmazások

14.1.1 Motiváció

A TerraSense rendszerben a `ts-node` eszközök MQTT-n keresztül publikálnak telemetriát, a `ts-gw` átjáró pedig ezt a brokerbe továbbítja. A felhőben futó `ts-backend` feladata, hogy az üzeneteket fogadja, feldolgozza, eltárolja, lekérdezhetővé tegye, és szükség esetén értesítéseket küldjön. Emellett a rendszernek parancsokat is el kell juttatnia az eszközökhöz, például szelep nyitására. A backend tehát nem egyszerű adatfogadó, hanem a rendszer központi idegrendszere, ahol az adat és a vezérlés találkozik. Az alábbiakban felvázoljuk a backend logikai komponenseit, a köztük lévő adatfolyamot, és azt a döntési keretet, amely alapján a TerraSense esetén moduláris monolitot választunk mikroszolgáltatások helyett.

14.1.2 Fogalmak és működés

Az IoT-backend több logikai felelősséget hordoz. Ezeket érdemes külön modulként kezelni, még akkor is, ha kezdetben egyetlen futtatható egységben (monolitban) valósulnak meg.

Definíció. Ingest (beviteli réteg): az a komponens, amely a külső rendszerekből (jelen esetben MQTT brokerből, opcionálisan UDP-ből) érkező adatokat fogadja, protokoll-szintű dekódolást és kezdeti validációt végez, majd normalizált formában továbbítja a feldolgozó rétegnek.

Definíció. Feldolgozás (processing): az üzleti logika része, amely a bejövő eseményeket értelmezi, séma szerint validálja, szükség esetén szűri, gazdagítja, és kiváltja a levezetett eseményeket (például riasztásokat, parancs-válaszokat).

Definíció. Tárolás (storage): a perzisztencia réteg, amely felelős az idősoros adatok és az eszközök metaadatainak, állapotának megőrzéséért. A TerraSense-ben ez PostgreSQL + TimescaleDB.

Definíció. API (alkalmazásprogramozási felület): a kifelé nyújtott, jellemzően szinkron interfész, amelyen keresztül más alkalmazások (Grafana, mobil app, külső rendszerek) lekérdezhetik az adatokat és parancsokat küldhetnek.

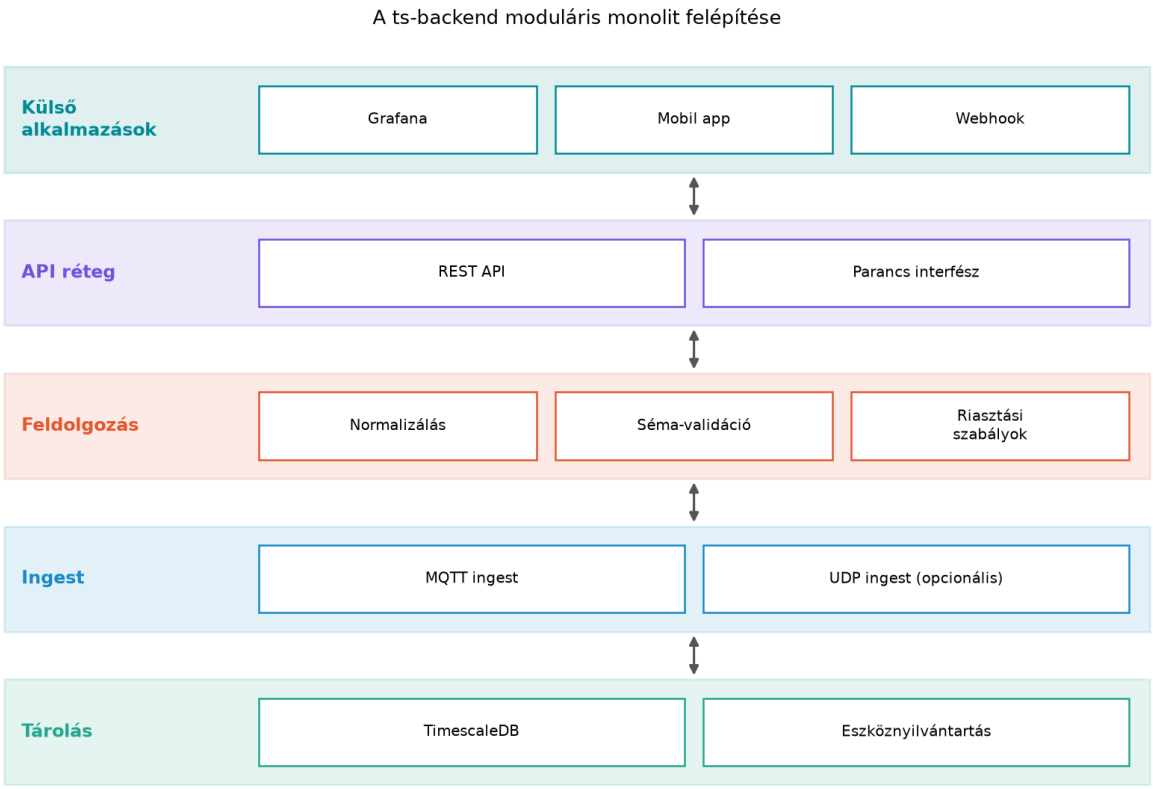
Definíció. Értesítés (notification): a rendszerből kifelé irányuló aszinkron üzenetek, például riasztási e-mailek, webhookok, push értesítések.

Definíció. Eszköznyilvántartás (device registry): az eszközök regisztrációját, identitását, konfigurációját és aktuális állapotát kezelő modul.

Ezek a komponensek a következő adatfolyamban vesznek részt: az eszközök az MQTT: publish/subscribe, broker, topicok alfejezetben rögzített topic-konvenció szerint publikálnak a `terrasense/.../telemetry` topicokra; a `ts-backend` ingest modulja feliratkozik a brokerre, és minden üzenetet továbbít a feldolgozó modulnak. A feldolgozó ellenőrzi a séma szerinti érvényességet, majd a tároló rétegbe írja az idősoros adatot. Az API modul a tárolóból olvas, és REST végpontokat biztosít. Az értesítés modul a feldolgozás által kiváltott eseményeket

juttatja el az üzemeltetőkhöz. A lefelé irányuló parancsok az API-ból indulnak, és a feldolgozáson keresztül MQTT publishként kerülnek a brokerre.

Az alábbi rétegábra a `ts-backend` komponenseit foglalja össze.



14.1. ábra: A ts-backend moduláris monolit felépítése - A rétegek felülről lefelé: külső alkalmazások, API, feldolgozás, ingest, tárolás. A nyilak a telemetria útját jelzik az ingest felől a tárolás és az API felé.

Az ábra szöveges leírása: a legfelső rétegben a backendet használó alkalmazások vannak; ezek az API rétegen keresztül érhetik el az adatokat. Az API a feldolgozási logikát hívja, a feldolgozás az ingest és a tárolás között közvetít. Az ingest a brokerből érkező üzeneteket fogadja, a tárolás pedig a TimescaleDB-t és az eszköznyilvántartást fedi. A rétegek közötti függőség egyirányú: a felsőbb rétegek hívnak lefelé, a visszafelé irányuló kommunikáció eseményekkel vagy callbackekkel valósul meg.

14.1.3 Döntési szempontok

A backend architektúrájának kiválasztása során az első kérdés a **bevetési egység** mérete: monolit, moduláris monolit vagy mikroszolgáltatások.

A **monolit** egyetlen folyamat, benne az összes komponenssel. Előnye az egyszerű fejlesztés és üzemeltetés, hátránya a nagyobb kódösszefonódás. Fontos azonban különválasztani a **telepítési egységet** a **skálázás lehetőségétől**: a monolit egy folyamatként települ, de ez nem jelenti azt, hogy csak vertikálisan skálázható. Ha a folyamat állapotmentes, vagy az állapotot külső tárolóba (adatbázis, Redis, megosztott cache) tesszük, akkor a monolitból több példány is

futtatható egy load balancer mögött. A **moduláris monolit** ugyanígy egyetlen folyamat, de a belső komponensek világosan elhatárolt modulokba szervezettek, köztük szigorú függőségi szabályokkal; ez a forma is replikázható, és a modulhatárok később lehetővé teszik egy-egy modul önálló folyamatba emelését. A **mikroszolgáltatás-architektúra** több függetlenül telepíthető szolgáltatásból áll, amelyek hálózaton keresztül kommunikálnak; itt a skálázási egység szolgáltatásonként külön választható, ami nagyobb szervezeti és operatív szabadságot ad, de nem az egyetlen módja a horizontális skálázásnak.

A gyakorlati tapasztalat – amelyet a szakirodalom is megerősít – az, hogy **ne kezdj mikroszolgáltatásokkal**, különösen akkor, ha a csapat kicsi, a domain határai még kiforratlanok, és a terhelés mérsékelt. A mikroszolgáltatások ugyanis jelentős operatív terhet hoznak (konténerorchesztráció, szolgáltatás-felfedezés, elosztott tranzakciók, naplózás és nyomkövetés), miközben a fejlesztési sebesség csökken. A moduláris monolit megtartja az egyszerű üzemeltetést, de a belső modulhatárok lehetővé teszik a későbbi szétválasztást, ha a skálázási vagy szervezeti igény úgy kívánja.

Az alábbi táblázat összefoglalja a kompromisszumokat:

Szempont	Monolit	Moduláris monolit	Mikroszolgáltatások
Fejlesztési sebesség	Gyors	Gyors	Lassabb, több koordináció
Operatív komplexitás	Alacsony	Alacsony	Magas
Kódösszefonódás	Magas	Alacsony (szabályokkal)	Alacsony
Skálázási egység	egy folyamat	egy folyamat, belső modulokkal	szolgáltatásonként külön
Horizontális skálázás	állapotmentes replikákkal; az állapotot externalizálni kell	ugyanaz, de a replikák a modulokat együtt skálázzák	szolgáltatásonként független
Adatbázis-korlát	közös adatbázis; gyakran ez a szűk keresztmetszet	ugyanaz, tudatos connection pool-lal	szolgáltatásonként külön séma/adatbázis, elosztott tranzakciók
Hibaelkülönítés	Rossz	Jobb	Jó
Tranzakciókezelés	Könnyű (lokális)	Könnyű (lokális)	Nehéz (elosztott)
Ideális csapatméret	Kicsi	Kicsi–közepes	Nagy

A monolit horizontális skálázásának egyik gyakorlati útja az MQTT-fogyasztói munkamegosztás. Ha a broker támogatja a megosztott feliratkozást (például MQTT 5 `$share/<csoport>/terrasense/+/+/telemetry` formában), akkor a `ts-backend` több ingest-példánya ugyanabba a csoportba iratkozhat fel, és a broker osztja el közöttük az üzeneteket. A példányok állapotmentesek maradhatnak, ha a deduplikációs kulcsot és a feldolgozási állapotot közös adatbázisban tárolják; a QoS 1 miatti ismételt kézbesítés így idempotensen

kezelhető. A megosztott feliratkozás nem változtatja meg a TerraSense adat-topicjait, csak a broker oldali fogyasztói csoportot használja.

A másik gyakori szűk keresztmetszet nem a monolit folyamat, hanem az adatbázis: a TimescaleDB írási sebessége, a connection pool mérete és a nagy időtartományú lekérdezések terhelése. Ilyenkor a horizontális skálázás része lehet a write-batching, a hypertable particionálás, a retention policy, az olvasási replikák használata az API számára, vagy az ingest és az API külön folyamatba emelése. A mikroszolgáltatás tehát nem az egyetlen horizontális skálázási lehetőség: egy jól megtervezett monolit is replikázható, amíg a megosztott állapotot és az adatbázis-korlátot tudatosan kezeljük.

A TerraSense backendje a jelenlegi követelmények (egy laboratóriumi rendszer, néhány tucat eszköz, egy-két fejlesztő) alapján **moduláris monolit** legyen. Az eszközszám növekedésével először a monolit replikái és az adatbázis-korlát kezelése jön szóba; az ingest és a feldolgozás külön folyamatba emelése csak ezután, a mért terhelés alapján indokolt.

A második fontos döntés a **szinkron** és **aszinkron** kommunikáció határainak kijelölése. Az eszközök felől érkező telemetria természeténél fogva aszinkron: a broker push-olja az üzeneteket, a backendnek nem kell választ adnia. Az ingest és a feldolgozás közötti hívás kezdetben lehet szinkron (a feldolgozó egyszerű metódus hívása), mert a terhelés kicsi, és ez egyszerűbbé teszi a hibakezelést. Az API felé mutató interfész viszont szinkron REST, mert a kliensek (Grafana, mobil) adatlekérdezést végeznek, és válaszra várnak. A lefelé irányuló parancsok is szinkron kezdeményezésűek, de maga a kézbesítés aszinkron MQTT-n történik; a parancs eredményét a backend aszinkron módon, a `cmd/result` topicról kapja meg.

A **nem funkcionális követelmények leképezése**:

- **Skálázhatóság**: a moduláris monolit vertikálisan (több erőforrás) és horizontálisan is skálázható: állapotmentes vagy külső tárolóba tett állapot mellett több replika futtatható load balancer mögött, illetve MQTT megosztott feliratkozással a fogyasztói terhelés is eloszthető. Az ingest modul ezen felül külön folyamatba emelhető, ha a mért üzenetsebesség meghaladja a replikált monolit kapacitását.
- **Rendelkezésre állás**: az állapotmentes modulok (ingest, API) replikázhatók; az állapotos tárolás (TimescaleDB) redundáns klaszterrel védhető, de ez a laborban nem követelmény.
- **Karbantarthatóság**: a modulhatárok és a függőségi szabályok megakadályozzák a spagetti kód kialakulását, és lehetővé teszik a modulok külön tesztelését.
- **Megfigyelhetőség**: minden modul hozzáfér a közös naplózási és metrika-infrastruktúrához, így a hibakeresés végponttól végpontig lehetséges.

14.1.4 Kidolgozott példa

A `ts-backend` moduláris monolit csomagszerkezete a következő lehet:

```
ts-backend/
  src/main/java/com/terrasense/backend/
      ingest/            # MQTT ingest, topic-feliratkozás, idempotencia
      processing/        # normalizálás, séma-validáció, riasztási szabályok
      storage/           # TimescaleDB repository, eszköznyilvántartó repository
      api/               # REST kontrollerek, DTO-k, parancs interfész
      notification/      # e-mail, webhook, riasztási csatornák
      deviceregistry/    # eszközök, konfiguráció, állapot
      common/            # domain események, kivételek, idők, séma-verzió
  pom.xml
```

A modulok közötti függőségi szabály: a `common` csomagot mindenki használhatja; az `ingest` a `processing`-ot hívja; a `processing` a `storage`-ot és a `notification`-t; az `api` a `processing`-ot és a `storage`-ot; a `deviceregistry` a `storage`-ot és a `common`-t. Az `ingest` nem hívhat közvetlenül `storage`-ot, hogy az idempotencia és a séma-kezelés a feldolgozásban maradjon.

Az adatfolyam a kidolgozott példában: a `ts-node` a `terrasense/gh1/n001/telemetry` topicra küld egy hőmérséklet-üzenetet. A Mosquitto továbbítja a `ts-backend` ingest moduljának, amely feliratkozott a `terrasense/+/+/telemetry` topicra. Az ingest ellenőrzi a topic formátumát, majd a `processing` modulnak adja át. A `processing` validálja a JSON/CBOR sémát, kinyeri az eseményidőt és a mérési értékeket, majd a `storage` rétegbe írja a TimescaleDB-be. Ezzel egyidejűleg, ha a hőmérséklet meghalad egy küszöböt, a `notification` modul riasztást küld. A Grafana az API modulon keresztül kérdezi le az idősoros adatokat.

Továbbvitt eredmény. A `ts-backend` **moduláris monolit** architektúrájú, a fenti csomagszerkezettel és modulhatárokkal. Ez a felépítés szolgál alapul a JEE-ből Spring Bootba: fogalmi híd C/C++ fejlesztőknek alfejezetben bemutatott fogalmi átvezetéshez, a Ingest: MQTT-ből a backendbe, idempotensen alfejezetben az ingest konkrét implementációjához, és a Felhős és nyílt forrású IoT-platformok: mikor építs, mikor vásárolj alfejezetben a build vs. buy döntéshez. A modulok elnevezése és a függőségi szabályok a későbbi laborokban is érvényesek, ezért azokat nem szabad megváltoztatni.

14.1.5 Ellenőrzés és hibakeresés

Az architektúra helyességét nem futtatással, hanem **kódátvizsgálással** és **függőségi elemzéssel** lehet ellenőrizni. A Spring Boot alkalmazásban például az ArchUnit vagy hasonló eszköz képes a csomagfüggőségi szabályok betartását tesztelni. Kézzel is ellenőrizhető, hogy az `ingest` csomag nem importál a `storage`-ból, és az `api` nem hívja közvetlenül az MQTT klienst.

Tipikus hibatünetek és okok:

Tünet	Ok	Javítás
Az ingest közvetlenül az adatbázisba ír, megkerülve a feldolgozást	A modulhatárok nem egyértelműek, vagy a fejlesztő sietett	Az adatbázis-hozzáférést csak a `storage` modulon keresztül engedélyezzük, kódátvizsgálással és statikus elemzéssel
A backend szinkron MQTT hívást vár az eszköztől	A fejlesztő a REST mintát ültette át az eszközkommunikációra	Az eszköz felé irányuló parancsokat aszinkron módon, a `cmd` topicon keresztül kezeljük
A monolit egyetlen óriási csomaggá nő, ahol minden függőség keresztezhető	A modulhatárokat nem tartják be	Az ArchUnit-tesztek és a rendszeres refaktorálás bevezetése
A mikroszolgáltatásokat túl korán vezetik be	A csapat túlértékeli a jövőbeli skálázási igényt	Vissza kell vonni a szolgáltatások számát, amíg a terhelés nem indokolja

14.1.6 Tipikus hibák és félreértések

1. **„Mikroszolgáltatásokkal kell kezdeni, mert az skálázható.”** Ez a leggyakoribb félreértés. A mikroszolgáltatás-architektúra elsősorban szervezeti és skálázási problémát old meg, nem pedig a kezdeti fejlesztést gyorsítja. Kevés eszköz és kis csapat esetén a szolgáltatások szétválasztása csak növeli a komplexitást, és lassítja a szállítást. A moduláris monolit később is szétválasztható.
2. **„A backend és az eszköz közötti kommunikáció legyen REST, mert az egyszerű.”** Az eszköz hálózati megbízhatósága és az alvó üzemmódok miatt a REST nem megfelelő; az MQTT aszinkron publish/subscribe modellje sokkal jobban illeszkedik az IoT tűréshatáraihoz. A backend felől érkező parancsok is aszinkron módon, nyugtázással működnek, nem szinkron HTTP-hívásként.
3. **„A feldolgozó modul közvetlenül a brokerhez csatlakozzon, ne az ingest modul.”** Ezzel a modulhatárok elmosódnak, és a feldolgozási logika összekapcsolódik a protokollfüggő MQTT-klienssel. Később, ha új ingest csatornát (például CoAP vagy UDP) kell bevezetni, az egész feldolgozást módosítani kell.

14.1.7 Feladatok

(a) Fogalmi/döntési kérdés. Miért választunk a TerraSense `ts-backend`jéhez moduláris monolitot mikroszolgáltatások helyett? Sorolj fel legalább három indokot.

(b) Tervezési feladat. Tegyük fel, hogy a TerraSense rendszerben 100 eszköz működik, mindegyik percenként egy telemetria üzenetet küld. Számítsd ki az átlagos üzenetsebességet (üzenet/másodperc), és becsüld meg, hogy ez mekkora adatmennyiséget jelent naponta, ha egy üzenet átlagosan 200 bájt. Indokold meg, hogy ezen terhelés mellett szükség van-e az ingest modul külön folyamatba történő kiszervezésére, vagy elegendő a monolit folyamaton belüli szinkron hívás.

(c) Implementációs feladat (előkészíti a továbbvitt eredményt). Hozd létre a `~/iot-labor/i041/ts-backend` Spring Boot projektet a fenti csomagszerkezettel. A projekt egyelőre üres osztályokat tartalmazzon a modulok határvonalain. A `pom.xml`-ben add meg a Spring Boot 3.x szülőt és a minimális függőségeket. Ezt a vázat fogja felhasználni az A ts-backend váza: REST API és egészségellenőrzés alfejezet.

(d) Hibakeresési/kiterjesztési feladat. Új követelmény: a rendszernek támogatnia kell egy olyan eszköztípust, amely nem MQTT-n, hanem UDP-n küld telemetriát. Hol kell módosítani a moduláris monolit architektúrát? Melyik modul mit kap? Indokold, hogy miért nem kell megváltoztatni a feldolgozó és tároló modulokat.

14.1.8 Megoldások

(a) A moduláris monolit mellett szól: (1) kis csapat és laboratóriumi méret, ahol a mikroszolgáltatások operatív terhe nem térül meg; (2) a domain határok még kiforratlanok, így kockázatos lenne rögzíteni a szolgáltatáshatárokat; (3) a belső modulhatárok lehetővé teszik a későbbi szétválasztást, ha a terhelés nő; (4) a tranzakciókezelés lokális, nem kell elosztott tranzakciókkal foglalkozni.

(b) Az üzenetsebesség: 100 eszköz $\times$ 1 üzenet/perc = 100 üzenet/perc $\approx$ 1,67 üzenet/másodperc. Napi adatmennyiség: $100 \times 60 \times 24 \times 200$ bájt = 28 800 000 bájt = 28,8 MB. Ez nagyon

alacsony terhelés, egyetlen monolit folyamat és szinkron hívás bőven elegendő. Az ingest külön folyamatba emelése csak legalább ezres nagyságrendű üzenetsebesség felett indokolt.

(c) A projekt létrehozása a `spring-boot-starter-web` és a `spring-boot-starter-actuator` függőségekkel. A csomagokat üres osztályokkal, például `IngestService`, `ProcessingService`, `StorageService`, `ApiController`, `NotificationService`, `DeviceRegistryService` nevekkel hozzuk létre. A `pom.xml` a Spring Boot 3.x szülőt használja, a Java 21-es verzióval. A pontos Maven-parancs és a teljes `pom.xml` az A ts-backend váza: REST API és egészségellenőrzés alfejezetben kerül kidolgozásra, itt a váz felépítése a cél.

(d) Új UDP ingest csatorna esetén csak az `ingest` modul bővül: hozzáadunk egy `UdpIngestService` osztályt, amely az UDP csomagokat fogadja, és ugyanazt a normalizált eseményt állítja elő, mint az MQTT ingest. A `processing`, `storage` és a többi modul változatlan marad, mert azok a normalizált eseményen dolgoznak, nem a szállítási protokollon. Ez a modulhatárok előnye: a szállítási csatorna hozzáadása nem érinti az üzleti logikát.

14.1.9 Rövid összefoglaló

A TerraSense `ts-backend`je moduláris monolit architektúrával épül fel. A logikai komponensek: ingest (MQTT bemenet), feldolgozás (normalizálás, séma-validáció, riasztások), tárolás (TimescaleDB, eszköznyilvántartás), API (REST), értesítés és eszköznyilvántartás. A modulok határai egyirányú függőséget követnek, a `common` csomag mindenki által használható. A döntés indoka a kiscsapatos fejlesztés, az alacsony terhelés és a későbbi szétválaszthatóság. Az eszköz felől érkező telemetria aszinkron, az API szinkron. A `~/iot-labor/i041/ts-backend` projekt a vázat adja, amelyet az A ts-backend váza: REST API és egészségellenőrzés alfejezet fejleszt tovább. Ezt a csomagstruktúrát és modulhatárokat használják a JEE-ből Spring Bootba: fogalmi híd C/C++ fejlesztőknek, Ingest: MQTT-ből a backendbe, idempotensen és Felhős és nyílt forrású IoT-platformok: mikor építs, mikor vásárolj alfejezetek is.

14.1.10 Források és ellenőrzési hiányok

- Spring Boot dokumentáció: `https://docs.spring.io/spring-boot/` – a pontos verziók (3.x, Java 21) ellenőrzési igényt jelentenek a későbbi alfejezetekben.
- Jakarta EE specifikációk: `https://jakarta.ee/specifications/` – a JEE fogalmi híd forrása.
- Hohpe, G., & Woolf, B. *Enterprise Integration Patterns*: `https://www.enterpriseintegrationpatterns` – a szinkron és aszinkron kommunikációs minták általános háttere.
- A TerraSense architektúra saját koncepció, ezért külső forrásra nem hivatkozik. A Spring Boot és a TimescaleDB konkrét verzióit a Laborkörnyezet alfejezet rögzíti. A “ne kezdj mikroszolgáltatásokkal” elv jól ismert iparági gyakorlat, de a konkrét számok (pl. az okos eszközök száma) a TerraSense laboratóriumi körülményeire vonatkoznak, nem általánosíthatók mérés nélkül.

14.2 JEE-ből Spring Bootba: fogalmi híd C/C++ fejlesztőknek

14.2.1 Motiváció

A Az IoT-backend felépítése: ingest, tárolás, API, alkalmazások alfejezetben áttekintettük a TerraSense backend rétegeit. A ts-backendet Spring Bootban készítjük el, de az olvasó eddig főleg a Jakarta EE (korábbi nevén Java Platform, Enterprise Edition, JEE) fogalmaival találkozott:

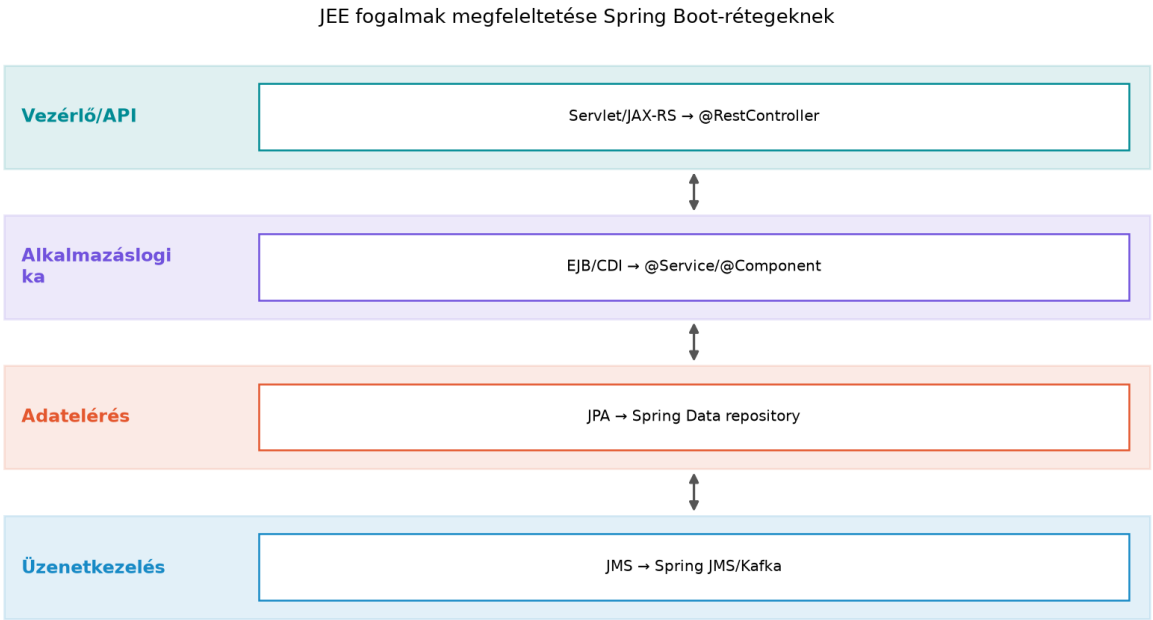
servlet, EJB, JPA, JMS. Ezek a fogalmak hasznosak, mert a Spring Boot ugyanazokat a problémákat oldja meg, csak eltérő, modernebb eszközökkel. Ebben az alfejezetben felépítjük a fogalmi hidat, és közben a szükséges Java-minimumot is átadjuk C++-os szemmel, hogy a ts-backend kódját ne kelljen „vakon” olvasni.

14.2.2 Fogalmak és működés

Először tisztázzuk a két világ viszonyát. A **Jakarta EE** szabványosított specifikációk gyűjteménye: servlet, JAX-RS, EJB, CDI, JPA, JMS. A specifikáció önmagában nem futtatható; implementáció kell hozzá, például WildFly, Payara, Open Liberty vagy Quarkus. A hagyományos Jakarta EE telepítési modell a WAR-csomag és egy külső alkalmazásszerver, de a modern implementációk futtatható JAR-t és konténerbarát csomagolást is kínálnak (például Payara Micro, WildFly bootable JAR, Open Liberty). Ezzel szemben a **Spring Boot** egy keretrendszer, amely a Spring-projekt konvencióira épít, és jellemzően beágyazott szervert (Tomcat, Jetty vagy Undertow) tartalmazó futtatható JAR-t állít elő. A két világ különbsége tehát nem a „külső szerver kontra karcsúság” sztereotípiája, hanem a specifikáció és a keretrendszer, valamint a csomagolási és telepítési mód eltérése.

Definíció. A **Spring Boot** olyan Java-keretrendszer, amely a Spring-ökoszisztémát előre konfigurált indítókkal, beágyazott webkiszolgálóval és „futtatható JAR” csomagolással teszi használhatóvá. A **Spring-bean** pedig egy olyan objektum, amelynek életciklusát a Spring-konténer kezeli: példányosítja, függőségekkel ellátja, és szükség szerint felszabadítja.

Az alábbi ábra a JEE-komponensek és a Spring Boot komponensek megfeleltetését foglalja össze rétegenként.



14.2. ábra: JEE fogalmak megfeleltetése Spring Boot-rétegeknek - A táblázat grafikus összefoglalása: a felső réteg a vezérlő, alatta a szolgáltatás és az adatelérés.

A rétegekhez tartozó fogalmak:

- **Servlet / JAX-RS → @RestController.** A servlet alacsony szintű HTTP-kezelő; a JAX-RS erőforrás-osztályokat annotációkkal köt URL-ekhez. Spring Bootban a `@RestController` osztály metódusai `@GetMapping`, `@PostMapping` stb. annotációval közvetlenül REST-végpontokat adnak. A HTTP-konverzió (JSON stb.) automatikus.
- **EJB / CDI → @Service / @Component.** Az EJB-k, illetve a CDI-beanek az alkalmazáslogika hordozói, tranzakcióval, biztonsággal. Spring Bootban a `@Service` és a általánosabb `@Component` annotációjú osztályok a bean-ek; a tranzakciót `@Transactional` jelöli.
- **JPA → Spring Data.** A JPA az objektum-relációs leképezés szabványa. A Spring Data repository-interfészekkel még a JPA lekérdezések nagy részét is elkerüli: például `findByNodeId(String nodeId)` metódusból a keretrendszer lekérdezést generál.
- **JMS → Spring JMS / Kafka.** A JMS a Java-üzenetkezelés szabványa. A Spring Boot ehhez `JmsTemplate`-et vagy magasabb szintű `@KafkaListener`-t kínál; az IoT-ban gyakran Kafka vagy MQTT az üzenethordozó.

Dependency injection és inversion of control. A kulcsfogalom a IoC (Inversion of Control, a vezérlés megfordítása): a komponens nem maga hozza létre a függőségeit, hanem kívülről kapja. Spring Bootban ezt a DI (Dependency Injection, függőség-injektálás) valósítja meg. C++-os analógiaként képzeljük el a konstruktorinjektálást:

```
// C++: a függőséget a konstruktor paraméterként kapja, a hívó hozza létre
auto repo = std::make_shared<NodeStateRepository>();
auto service = std::make_shared<NodeStateService>(repo);
StatusController controller{service};
```

A Spring ugyanezt deklaratívan csinálja. A bean-definíciók alapján a keretrendszer felépíti az objektumgráfot, és a konstruktorba injektálja a szükséges példányokat. C++-ban ezt a hívást gyakran egy kompozíciós gyökér (composition root) végzi; Spring Bootban a konténer tölti be ezt a szerepet.

14.2.3 Döntési szempontok

A TerraSense háttérrendszeréhez Spring Bootot választottunk, de a döntést így is érdemes megindokolni. Ehhez négy dolgot kell különválasztani: a **specifikációt** (mi a szerződés), az **implementációt** (melyik runtime valósítja meg), a **csomagolást** (WAR vagy futtatható JAR) és a **telepítést** (külső szerver, beágyazott runtime vagy konténer).

A **Jakarta EE** formális specifikációgyűjtemény, amelyhez több kompatibilis implementáció választható. A **Spring Boot** nem specifikáció, hanem egy konkrét keretrendszer és runtime: a Spring-konténer, az automatikus konfiguráció és a beágyazott webkiszolgáló együttese. Csomagolásban a hagyományos Jakarta EE alkalmazás WAR, amelyet külső alkalmazásszerverre telepítenek, de a modern implementációk futtatható JAR-t is adnak; a Spring Boot alapértelmezése a beágyazott szervert tartalmazó futtatható JAR. Telepítésben mindkét világ kerülhet konténerbe: a hagyományos Jakarta EE úton a szervert külön kell üzemeltetni, a Spring Bootnál az alkalmazás hozza magával a szervert, a modern Jakarta EE runtime-ok pedig szintén tudnak önállóan induló csomagot adni.

Szempont	Jakarta EE	Spring Boot
Mi ez?	specifikációgyűjtemény	keretrendszer és runtime
Implementációk	WildFly, Payara, Open Liberty, Quarkus	maga a Spring Boot, beágyazott Tomcat/Jetty/Undertow

Szempont	Jakarta EE	Spring Boot
Csomagolás	hagyományosan WAR; modern runtime-nal futtatható JAR is	jellemzően futtatható JAR (WAR is lehetséges)
Telepítés	külső alkalmazásszerver vagy önálló/beágyazott runtime	önálló JAR beágyazott szerverrel
Szabványosság	formális specifikáció	de facto konvenciók
Fejlesztési modell	Jakarta annotációk, CDI, gyakran deployment descriptor	Spring annotációk, Spring-konténer, konvenció a konfiguráció fölött
IoT-mikroszolgáltatás	a választott implementációtól függ: lehet karcsú (Payara Micro, Quarkus) vagy teljes szerver	jellemzően karcsú, konténerbarát
Tanulási görbe	Jakarta EE ismeret kell	Spring-ismeret kell
Enterprise környezet	gyakran nagyvállalati standard	szintén elterjedt, cloud-native

A TerraSense esetében a ts-backend konténerben fut, REST API-t és MQTT-ingestet valósít meg. A Spring Bootot nem azért választottuk, mert a Jakarta EE feltétlenül nehézkes lenne, hanem mert a csapat Spring-ismerettel rendelkezik, a Spring Web, a Spring Data és az MQTT-integráció egy ökoszisztémában elérhető, és a beágyazott szerveres futtatható JAR egyszerű konténeres telepítést ad. Ugyanez a szolgáltatás Jakarta EE implementációval is megvalósítható lenne (például Open Liberty vagy Quarkus alatt); a döntés tehát csapat-, ökoszisztéma- és üzemeltetési szempont, nem a Jakarta EE alkalmatlansága.

14.2.4 Kidolgozott példa: teljes Spring Boot projekt

Készítsünk egy teljes, fordítható Spring Boot projektet a TerraSense fogalmaival. A cél a `GET /api/nodes/{nodeId}/status` végpont, amely egy csomópont legutóbbi állapotát adja vissza. A projekt gyökere `~/iot-labor/i042/ts-backend/`, a fájlstruktúra pedig a következő:

```
~/iot-labor/i042/ts-backend/
  pom.xml
  src/
      main/
          java/com/terrasense/backend/
              TsBackendApplication.java
              NodeStatus.java
              NodeStateService.java
              StatusController.java
          resources/
              application.properties
              application-dev.properties
```

A `pom.xml` teljes tartalmát a következő, Build szakasz mutatja. A Java-források és a konfigurációs fájlok a következők.

1. A főosztály. Minden Spring Boot alkalmazásnak van egy belépési pontja, amely a `SpringApplication.run` hívást végzi. A `@SpringBootApplication` három dolgot fog össze: konfigurációs osztály, automatikus konfiguráció engedélyezése és komponenspásztázás. A pásztázás annak a csomagnak a gyökerétől indul, amelyben ez az osztály van, ezért a főosztálynak a `com.terrasense.backend` gyökérben kell lennie.

```java
package com.terrasense.backend;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class TsBackendApplication {

    public static void main(String[] args) {
        SpringApplication.run(TsBackendApplication.class, args);
    }
}
```

2. A telemetriastátusz record. A Java 16-tól elérhető `record` egy immutable adattartó, nagyjából olyan, mint egy C++-os `struct`, amelynek minden tagja `const`, és a konstruktor a paraméterlistából generálódik. A JSON-szerializációhoz használhatjuk közvetlenül.

```java
package com.terrasense.backend;

public record NodeStatus(
    String nodeId,
    long seq,
    double temperatureC,
    boolean irrigationValveOpen
) {}
```

3. A szolgáltatás- vagy üzleti logika. A `@Service` annotáció jelzi a Springnek, hogy ebből az osztályból bean-t hozzon létre. A DI-t itt konstruktorinjektálással kapja; ebben a példában még nem függ külső komponenstől, később azonban ez lesz a belépési pont az adatbázis-felé.

```java
package com.terrasense.backend;

import org.springframework.stereotype.Service;

@Service
public class NodeStateService {

    public NodeStatus latestFor(String nodeId) {
        // Illusztratív állapot: a valódi lekérdezés az idősoros
        // adatbázisból dolgozik majd, amelyet az i045 alfejezet tárgyal.
        return new NodeStatus(nodeId, 42, 22.5, false);
    }
}
```

4. A REST-vezérlő. A `@RestController` a HTTP-réteget képviseli. A `@GetMapping` az URL-re köti a metódust. A `NodeStateService`-t a konstruktorban kapja, így a Spring létrehozza a függőségi láncot.

```java
package com.terrasense.backend;

import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.RestController;

@RestController
public class StatusController {

    private final NodeStateService nodeStateService;

    public StatusController(NodeStateService nodeStateService) {
        this.nodeStateService = nodeStateService;
    }
```

```
    @GetMapping("/api/nodes/{nodeId}/status")
    public NodeStatus status(@PathVariable String nodeId) {
        return nodeStateService.latestFor(nodeId);
    }
}
```

5. Az alapkonfiguráció. Az `application.properties` a közös beállításokat tartalmazza. A `application-dev.properties` csak akkor lép életbe, ha a `dev` Spring-profil aktív; erről a Build szakaszban lesz szó.

```
# src/main/resources/application.properties
spring.application.name=ts-backend
server.port=8080

# src/main/resources/application-dev.properties
logging.level.com.terrasense.backend=DEBUG
```

6. Indítás és ellenőrzés. A projekt gyökeréből:

```
cd ~/iot-labor/i042/ts-backend
mvn clean package
java -jar target/ts-backend-0.1.0.jar
```

A másik terminálban:

```
curl -s http://localhost:8080/api/nodes/n001/status
```

A várt, illusztratív kimenet:

```
{"nodeId":"n001","seq":42,"temperatureC":22.5,"irrigationValveOpen":false}
```

A C++-os olvasónak ez a kód a következőt jelenti: a Spring Boot elindul, a `@SpringBootApplication` alatti csomagot pásztázza, létrehoz egy `NodeStateService` példányt (bean), majd egy `StatusController` példányt, és a `StatusController` konstruktorába injektálja a `NodeStateService` referenciát. Ezt a kapcsolatot a `@Service` és a `@RestController` annotációk írják le. A `@GetMapping` metódus hívása olyan, mintha a HTTP-keretrendszer a megfelelő URL hívásakor meghívná a C++-os vezérlő metódusát.

Továbbvitt eredmény. Ez a teljes projekt – a főosztály, a bean-ek, a DI, a `@RestController`/`@Service` páros és a `NodeStatus` record – közvetlenül megalapozza az A ts-backend váza: REST API és egészségellenőrzés alfejezetet, ahol a `StatusController` és a `NodeStateService` már a ts-backend projektbe szervezett komponensekké válnak. Ugyanezt a DI-mintát és a bean-modellt használja majd az Ingest: MQTT-ből a backendbe, idempotensen alfejezet MQTT-kliens komponense, valamint az Parancsok és konfiguráció lefelé: kézbesítés, nyugtázás, időkorlát alfejezet parancskezelője is.

14.2.5 Java-minimum C++-os szemmel

A Spring Boot kód olvasásához néhány alapvető Java-fogalmat érdemes rögzíteni.

GC vs. RAII. C++-ban az objektum destruktora determinisztikus: a hatókör elhagyásakor fut. Javában nincs destruktor; a szemétgyűjtő (garbage collector, GC) bármikor felszabadíthatja a nem használt objektumokat, de ez nem garantált időpont. A determinisztikus erőforrás-felszabadításhoz a Java 7-ben bevezetett **try-with-resources** szolgál: az `AutoCloseable` interfészt megvalósító objektumot a `try` blokk erőforrás-deklarációjában létrehozva a blokk végén a `close()` automatikusan hívódik, hasonlóan a C++ RAII-hoz, de csak lexikális hatókörhöz kötötten.

```
try (var client = new MqttClient("tcp://localhost:1883", "ts-backend")) {
    client.connect();
    // ... publish / subscribe műveletek
} // A client.close() itt automatikusan meghívódik
```

Kivételek. A Javában kétféle kivétel van: **ellenőrzött (checked)** és **nem ellenőrzött (unchecked)**. Az előbbit a metódus szignatúrájában deklarálni kell (`throws IOException`), az utóbbit nem. C++-ban nincs ilyen megkülönböztetés; a `noexcept` specifikáció a Java `throws`-hoz hasonló, de nem azonos szerepet tölt be. A Spring Boot általában unchecked kivételeket használ, a checked kivételeket gyakran becsomagolja.

Record. A fenti `NodeStatus` példa mutatja: a `record` egy immutable adatosztály, amely a konstruktorból automatikusan létrehozza a mezőket, az `equals`, `hashCode` és `toString` metódusokat. C++-os szemmel ez egy `const` tagokból álló `struct`, amelynek nincs settere, és a tagjait a konstruktorban kell megadni.

Változóinferencia. A lokális változók típusa a `var` kulcsszóval elhagyható, a fordító a jobb oldalból következtet. Ez a C++11 `auto` megfelelője.

14.2.6 Build: Maven, futtatható JAR, profilok

A Spring Boot projekt standard buildeszköze a Maven. A `pom.xml` a projekt objektummodellje, nagyjából a C++ `CMakeLists.txt`-hez hasonló szerepű: függőségek, build-lépések, profilok. A teljes `pom.xml` a ts-backendhez:

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
             https://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>

    <parent>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-parent</artifactId>
        <version>3.3.5</version>
        <relativePath/>
    </parent>

    <groupId>com.terrasense</groupId>
    <artifactId>ts-backend</artifactId>
    <version>0.1.0</version>

    <properties>
        <java.version>21</java.version>
    </properties>

    <dependencies>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-web</artifactId>
        </dependency>
    </dependencies>

    <build>
        <plugins>
            <plugin>
                <groupId>org.springframework.boot</groupId>
                <artifactId>spring-boot-maven-plugin</artifactId>
            </plugin>
```

```
        </plugins>
    </build>

    <profiles>
        <profile>
            <id>dev</id>
            <properties>
                <spring-boot.run.profiles>dev</spring-boot.run.profiles>
            </properties>
        </profile>
    </profiles>
</project>
```

A futtatható JAR előállítása és futtatása:

```
cd ~/iot-labor/i042/ts-backend
mvn clean package
java -jar target/ts-backend-0.1.0.jar --spring.profiles.active=dev
# vagy környezeti változóval:
SPRING_PROFILES_ACTIVE=dev java -jar target/ts-backend-0.1.0.jar
```

A `mvn spring-boot:run` fejlesztés közben gyorsabb, mert nem igényli a csomagolást; a `dev` Maven-profil a `spring-boot.run.profiles` tulajdonságon keresztül adja át a Spring profil nevét:

```
mvn -Pdev spring-boot:run
```

Két külön profilfogalom. A Maven-profil (`-Pdev`) build-időben választ ki build-beállításokat, plugin-konfigurációt és tulajdonságokat; a Spring-profil (`spring.profiles.active`) futásidőben választja ki az `application-dev.properties`/`application-dev.yaml` fájlt és a profilra feltételes bean-eket. A kettő nem ugyanaz: a POM-ban egy `<properties><spring.profiles.active>dev</spring.p` bejegyzés önmagában nem állítja be a futó alkalmazás Spring-profilját, mert a Spring Boot Maven plugin nem olvassa ezt a tulajdonságot automatikusan. A helyes átadás a fenti `spring-boot.run.profiles` (a `mvn spring-boot:run` esetén), illetve a `--spring.profiles.active=dev` vagy a `SPRING_PROFILES_ACTIVE=dev` (a `java -jar` esetén). A futó alkalmazás naplójában ellenőrizhető a ténylegesen aktív profil, például: `The following 1 profile is active: "dev"`.

14.2.7 Ellenőrzés és hibakeresés

Egy ilyen fogalmi alfejezetben a „futtatás” inkább azt jelenti, hogy az olvasó a fenti kódot elhelyezi egy Spring Boot projektben, majd a `curl` paranccsal ellenőrzi a REST-végpontot:

```
curl -s http://localhost:8080/api/nodes/n001/status
```

A várt kimenet:

```
{"nodeId":"n001","seq":42,"temperatureC":22.5,"irrigationValveOpen":false}
```

Gyakori hibatünetek:

Tünet	Lehetséges ok	Javítás
`UnsatisfiedDependencyExceptionHiányzó`	`@Service`/`@Component`, vagy a konstruktorban olyan típust várunk, amelyből nincs bean	Adjuk meg a hiányzó annotációt, vagy hozzunk létre bean-t

Tünet	Lehetséges ok	Javítás
`404 Not Found` a `curl`-re	A `@RestController` nincs felderítve, vagy rossz a `@GetMapping` útvonal	Ellenőrizzük a `@SpringBootApplication` csomagszkennelését, és a path-t
`NoSuchBeanDefinitionException`	A konstruktorinjektálás típusa nem illik a bean-típushoz	Ellenőrizzük a csomagokat és a típusokat
Az alkalmazás nem indul	Portütközés vagy hiányzó függőség	`--server.port=8081` vagy a `pom.xml` függőségek

A Spring naplója bőbeszédű: a bean-ek létrejöttét, a beágyazott Tomcat indulását és a hiba okát is kiírja. Érdemes a konzolt figyelni.

14.2.8 Tipikus hibák és félreértések

1. **Az annotáció nem konfiguráció.** A `@Service` önmagában nem varázsol; a Spring a classpath-on lévő annotált osztályokat pásztázza, és a kontextusba regisztrálja. Ha az osztály nincs a komponens-kereső csomag alatt, nem jön létre bean.
2. **Mezőinjektálás helyett konstruktorinjektálás.** A `@Autowired` mezőinjektálás kényelmes, de rejtett függőségeket hoz létre, és nehezebben tesztelhető. C++-os analógiaként a konstruktorinjektálás olyan, mintha a függőséget a konstruktor `const&` paramétereként adnánk át, nem pedig egy globális singletonból vennénk elő.
3. **A GC nem RAII.** A C++-os fejlesztő hajlamos azt hinni, hogy a `finalize()` vagy a `close()` automatikusan lefut, amikor az objektumra már nincs hivatkozás. A Javában a `close()` csak akkor fut, ha try-with-resources-t használunk, vagy explicit módon hívjuk. Így a hálózati kapcsolatokat, fájlokat, MQTT-klienseket mindig try-with-resources-ba kell tenni.
4. **A `record` nem POJO.** A `record` immutable, nincs setter, és az öröklődés korlátozott. Ha JPA-entitásként akarjuk használni, az nem megy közvetlenül; JPA-entitáshoz osztály kell.

14.2.9 Feladatok

1. **Fogalmi/döntési kérdés.** Magyarázza el egy C++-os kollégának, mi a különbség a Jakarta EE és a Spring Boot között, és miért a Spring Boot a TerraSense ts-backendjének jobb választása.
2. **Tervezési feladat.** A ts-backend egy `TelemetryIngestService` komponense MQTT-üzeneteket fogad be, és idősoros adatbázisba ír. Tervezze meg a DI-alapú osztályszerkezetet: milyen interfészt/függőséget kapjon a konstruktorban? Rajzolja le szövegesen az objektumgráfot.
3. **Implementációs feladat.** Bővítse a fenti példát egy `TelemetryController` osztállyal, amely a `POST /api/nodes/{nodeId}/telemetry` végponton fogad egy `NodeTelemetry` recordot. A record tartalmazza a `seq`, `temperatureC`, `humidityRh`, `soilMoisturePercent` mezőket. A vezérlő hívja a `TelemetryIngestService.ingest(...)` metódust.
4. **Hibakeresési feladat.** A Spring Boot alkalmazás indulásakor a következő hibaüzenet jelenik meg: `Parameter 0 of constructor in com.terrasense.backend.StatusController required a bean of type 'com.terrasense.backend.NodeStateService' that could not be found.` Mi a hiba oka, és hogyan javítja?

14.2.10 Megoldások

1. A Jakarta EE szabványokra épül, és alkalmazásszervert igényel; a Spring Boot egy keretrendszer, amely beágyazott szerverrel, előre konfigurált modulokkal és gyors fejlesztési ciklussal dolgozik. A ts-backend konténerben fut, nem kell teljes Java-EE-szervert üzemeltetni, és a Spring Data, a `@RestController` valamint a Spring integrációk (MQTT, Kafka) pontosan a szükséges rétegeket fedik le. A Spring Boot tehát egyszerűbb üzemeltetést és kevesebb boilerplate-et ad.

2. A `TelemetryIngestService` konstruktorában egy `TelemetryRepository` interfészt (vagy Spring Data repository-t) és egy `ObjectMapper`-t kapjon. A DI révén a hívó, jelen esetben a Spring-konténer adja át ezeket a függőségeket. Az objektumgráf: `TelemetryController → TelemetryIngestService → TelemetryRepository`. Ezt a gráfot a bean-definíciók alapján a Spring építi fel, hasonlóan ahhoz, ahogy egy C++ kompozíciós gyökér explicit módon létrehozza a `repo`, majd `service`, majd `controller` példányokat.

3. A kód:

```
package com.terrasense.backend;

public record NodeTelemetry(
    long seq,
    double temperatureC,
    double humidityRh,
    double soilMoisturePercent
) {}

@RestController
public class TelemetryController {

    private final TelemetryIngestService ingestService;

    public TelemetryController(TelemetryIngestService ingestService) {
        this.ingestService = ingestService;
    }

    @PostMapping("/api/nodes/{nodeId}/telemetry")
    public void ingest(@PathVariable String nodeId,
                       @RequestBody NodeTelemetry telemetry) {
        ingestService.ingest(nodeId, telemetry);
    }
}
```

A `@RequestBody` a JSON-t automatikusan `NodeTelemetry` objektummá alakítja. A `TelemetryIngestService`-t `@Service` annotációval kell ellátni, hogy a Spring bean-ként létrehozza.

4. A hiba oka, hogy a `NodeStateService` osztályból nem lett Spring-bean; valószínűleg hiányzik a `@Service` annotáció, vagy az osztály a komponens-kereső csomagon kívül található. A javítás: adjuk hozzá a `@Service` annotációt a `NodeStateService` osztályhoz, és győződjünk meg arról, hogy a `StatusController`-rel azonos csomagban vagy annak alcsomagjában van. Ellenőrizzük, hogy a fő `@SpringBootApplication` osztály a `com.terrasense.backend` csomagban van, mert a komponens-pásztázás innen indul.

14.2.11 Rövid összefoglaló

Az alfejezet hidat épített a Jakarta EE és a Spring Boot között: a servlet/JAX-RS a `@RestController`, az EJB/CDI a `@Service`/`@Component`, a JPA a Spring Data, a JMS pedig a Spring üzenetkezelés megfelelője. A DI és IoC C++-os analógiája a konstruktorinjektálás: a függőségeket a Spring-konténer adja át, hasonlóan egy kompozíciós gyökérhez. A Java-minimum részeként tisztáztuk a GC vs. RAII, a kivételek, a `record` és a try-with-resources szerepét, valamint a Mavenbuild, a futtatható JAR és a profilok alapjait. **Továbbvitt eredmény:** a `com.terrasense.backend` csomag, a `NodeStatus` record, a `NodeStateService`/`StatusController` példa és a DI-fogalmak közvetlenül megalapozzák az A ts-backend váza: REST API és egészségellenőrzés, az Ingest: MQTT-ből a backendbe, idempotensen és a Parancsok és konfiguráció lefelé: kézbesítés, nyugtázás, időkorlát alfejezeteket.

14.2.12 Források és ellenőrzési hiányok

- Spring Boot dokumentáció: https://docs.spring.io/spring-boot/ – a `@RestController`, a `@Service`, a beágyazott webkiszolgáló és a Maven-integráció leírása. A pontos Spring Boot 3.x verziószám és a Java 21 kompatibilitás ellenőrzendő.
- Jakarta EE specifikációk: https://jakarta.ee/specifications/ – a servlet, JAX-RS, EJB, CDI, JPA és JMS fogalmak aktuális specifikációi. A specifikációk verziói a kiadott dokumentációban ellenőrizendők.
- Enterprise Integration Patterns: https://www.enterpriseintegrationpatterns.com/ – az üzenetkezelési minták háttere. A konkrét Spring implementációk nevei (pl. `JmsTemplate`, `@KafkaListener`) a Spring-dokumentációban ellenőrizendők.

Ellenőrzési hiányok: a példakódban használt Spring Boot 3.3.5 verziószám, a `spring-boot-starter-parent` pontos verziója, a `record` JSON-szerializációjának viselkedése a használt Jackson-verzióval, valamint a `mvn spring-boot:run` pontos CLI-kapcsolója a kiadott Maven-pluginban ellenőrzendő.

14.3 A ts-backend váza: REST API és egészségellenőrzés

14.3.1 Motiváció

Az előző alfejezetben bevezettük a Spring Bootot mint a TerraSense backendjének keretrendszerét. A `ts-gw` már képes MQTT-n telemetriát küldeni a `lab-broker` felé, de a rendszernek még nincs olyan szolgáltatása, amely a felhő oldalon fogadja, tárolja és kiszolgálja ezeket az adatokat. Ebben az alfejezetben elkészítjük a `ts-backend` válogatóját: egy olyan minimális, de konténerben futtatható Spring Boot alkalmazást, amely már most fel tud mutatni REST végpontokat, egészségellenőrzést és konfigurációt. A memóriabeli adattárolót szándékosan ideiglenesnek szánjuk – a későbbi alfejezetekben kicseréljük MQTT-ingestre, TimescaleDB-re és eszközárnyékra. A váz azonban már most meghatározza a csomagszerkezetet, a konfigurációs stílust és a konténeres futtatás módját, amelyet a Ingest: MQTT-ből a backendbe, idempotensen, a Idősoros adatbázisok és adatmodellezés, a Parancsok és konfiguráció lefelé: kézbesítés, nyugtázás, időkorlát és a Megfigyelhetőség végponttól végpontig: metrikák, naplók, nyomkövetés alfejezetek is használni fognak.

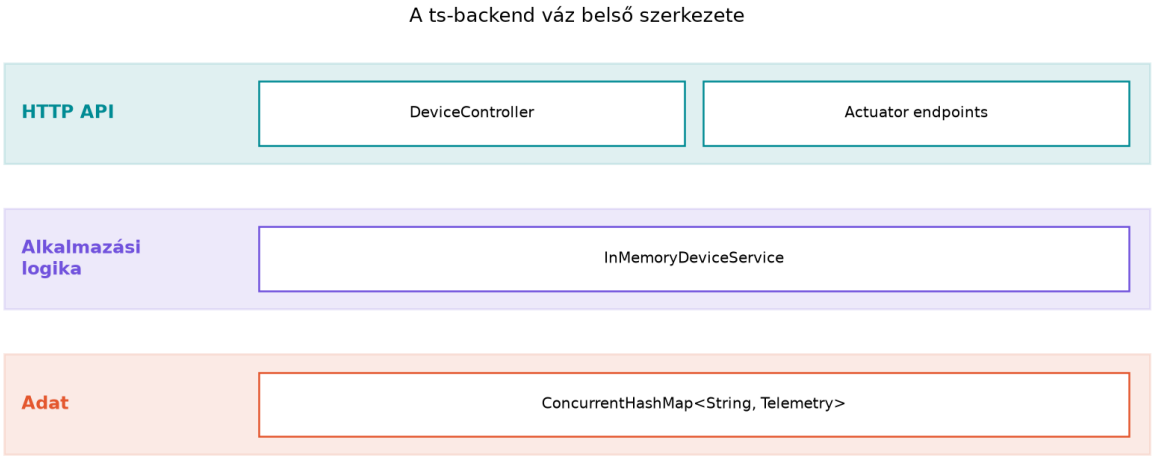
14.3.2 Fogalmak és működés

Definíció. A *REST API* (Representational State Transfer) HTTP-alapú, erőforrásokra épülő interfész, ahol az erőforrásokat URI-k azonosítják, a műveleteket pedig a HTTP-metódusok (GET, POST, PUT, DELETE) fejezik ki. A válasz általában JSON, a státusz pedig a szabványos HTTP-állapotkód.

Definíció. Az *egészségellenőrzés* (health check) olyan olvasási pont, amely jelzi, hogy a szolgáltatás él-e, és képes-e az alapvető funkcióira. A Spring Bootban ezt a *Spring Boot Actuator* modul biztosítja, amely további üzemeltetési végpontokat is kínál, például metrikákat (`/actuator/metrics`), környezeti infókat (`/actuator/info`) és konfigurációs részleteket.

Definíció. Az *alkalmazásprofil* (profile) a konfiguráció egy nevesített változata. A `dev` profil például részletes naplózást és memóriabeli tárolót, a `prod` profil szigorúbb beállításokat és valódi adatbázis-kapcsolatot használhat.

A háromrétegű belső felépítés elválasztja a HTTP-kezelést, az alkalmazási logikát és az adatelérést. Jelenleg minden réteg egyetlen osztályból vagy memóriabeli mapból áll:



14.3. ábra: A ts-backend váz belső szerkezete - A HTTP-API-t a Spring Web réteg, az üzleti logikát a service, az adatokat jelenleg egy memóriabeli map szolgálja ki. Az Actuator a HTTP-API mellett kap helyet.

A `DeviceController` fogadja a kéréseket, a `InMemoryDeviceService` elvégzi a keresést és a validálást, a `ConcurrentHashMap` pedig a legutolsó méréseket tárolja kulcs-érték párként. Ez a felosztás lehetővé teszi, hogy később a memóriabeli tárolót adatbázisra cseréljük anélkül, hogy a REST végpontok kódját meg kellene változtatni.

14.3.3 Döntési szempontok

A következő táblázat összefoglalja a váz kialakításakor hozott főbb döntéseket és azok indoklását:

Döntés	Választás	Indoklás / kompromisszum
Nyelvi/framework verzió	Java 21, Spring Boot 3.x	A verzióprofil szerint modern, hosszú távú támogatású Java; a 3.x natívan támogatja a Jakarta EE 10+ API-kat.
REST útvonal előtag	`/api/`	Elkülöníti a nyilvános API-t az Actuator végpontjaitól, és előkészíti a verziózást (`/api/v1/...`).
Adattárolás	`ConcurrentHashMap` a service-ben	Szándékosan ideiglenes; így a REST réteg már most tesztelhető, a perzisztencia később cserélhető.
Health check	Actuator `/actuator/health`	Szabványos, konténer-orchestrátorok (Docker, Kubernetes) által is felismert végpont.
Konfiguráció	`application.yaml` + profilok + környezeti változók	A YAML olvashatóbb, a profilok külön környezetet adnak, a környezeti változók lehetővé teszik a konténerben való felülbírálást.
Konténerkép	Multi-stage Docker build	Csak a futáshoz szükséges JRE kerül a végleges image-be, kisebb méret és kevesebb támadási felület.

A REST jó választás a backend és a dashboard/API-fogyasztók közötti szinkron, lekérdezésalapú kommunikációra. Az MQTT-ingesthez viszont aszinkron, eseményvezérelt feldolgozás kell majd, ezért a REST réteg és az MQTT-ingest réteg szét lesz választva a Ingest: MQTT-ből a backendbe, idempotensen alfejezetben.

14.3.4 Kidolgozott labor: a `ts-backend` váz a Compose-ban

Munkakönyvtár: hozzuk létre a labor könyvtárát, és dolgozzunk benne:

```
mkdir -p ~/iot-labor/i043 && cd ~/iot-labor/i043
```

A labor célja, hogy a `ts-backend` konténerben fusson, REST végpontokat szolgáljon ki, és a `/actuator/health` elérhető legyen. A labor végén a `~/iot-labor/stack/compose.yaml` fájlban új szolgáltatásként fog szerepelni.

14.3.4.1 1. lépés: Maven projekt és függőségek

Készítsünk `pom.xml` fájlt a következő tartalommal:

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
                             https://maven.apache.org/xsd/maven-4.0.0.xsd">
```

```
    <modelVersion>4.0.0</modelVersion>

    <parent>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-parent</artifactId>
        <version>3.4.1</version> <!-- ellenőrizendő: a legfrissebb stabil 3.x -->
        <relativePath/>
    </parent>

    <groupId>com.terrasense</groupId>
    <artifactId>ts-backend</artifactId>
    <version>0.1.0</version>
    <name>ts-backend</name>
    <description>TerraSense backend váz</description>

    <properties>
        <java.version>21</java.version>
    </properties>

    <dependencies>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-web</artifactId>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-actuator</artifactId>
        </dependency>
    </dependencies>

    <build>
        <plugins>
            <plugin>
                <groupId>org.springframework.boot</groupId>
                <artifactId>spring-boot-maven-plugin</artifactId>
            </plugin>
        </plugins>
    </build>
</project>
```

A `spring-boot-starter-web` a REST-kezeléshez szükséges (beágyazott Tomcat, Jackson JSON-szerializáció), a `spring-boot-starter-actuator` pedig az üzemeltetési végpontokat adja.

14.3.4.2 2. lépés: A fő alkalmazásosztály

Hozzuk létre a `src/main/java/com/terrasense/backend/TsBackendApplication.java` fájlt:

```
package com.terrasense.backend;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class TsBackendApplication {
    public static void main(String[] args) {
        SpringApplication.run(TsBackendApplication.class, args);
    }
}
```

A `@SpringBootApplication` a komponenskeresést, az automatikus konfigurációt és a konfigurációs osztályok beolvasását kapcsolja be.

14.3.4.3 3. lépés: Modellosztály

Készítsünk `Telemetry` modellt a `src/main/java/com/terrasense/backend/model/Telemetry.java` fájlban:

```
package com.terrasense.backend.model;

public record Telemetry(
    String nodeId,
    long seq,
    long ts,
    double temperature,
    double humidity,
    double soilMoisture
) {}
```

A `record` tömör, immutable adatmodellt ad; a JSON-szerializáció automatikus. Ez a REST DTO a kanonikus telemetria egy szűkített nézete: a `seq` a booton belüli monoton sorszám, a `ts` az eseményidő. A teljes ingest-modell (`site`, `boot_id`, `schema_version`, `ts_source`, `quality`) az Ingest alfejezetben jön; a REST réteg addig is ezt a DTO-t használja, és a későbbi ingest egy explicit adapterrel képezi le a kanonikus modellre.

14.3.4.4 4. lépés: Memóriabeli service

Hozzuk létre az `InMemoryDeviceService` osztályt a `src/main/java/com/terrasense/backend/service/InMem` fájlban:

```
package com.terrasense.backend.service;

import com.terrasense.backend.model.Telemetry;
import org.springframework.stereotype.Service;

import java.util.List;
import java.util.Map;
import java.util.Optional;
import java.util.concurrent.ConcurrentHashMap;

@Service
public class InMemoryDeviceService {

    private final Map<String, Telemetry> latestTelemetry = new ConcurrentHashMap<>();

    public InMemoryDeviceService() {
        // Labor-minta; élesben ezt az MQTT-ingest tölti majd fel.
        latestTelemetry.put("n001", new Telemetry("n001",
            41L, 1734567890123L, 22.5, 61.0, 34.0));
        latestTelemetry.put("n002", new Telemetry("n002",
            17L, 1734567895123L, 23.1, 58.5, 31.0));
    }

    public List<String> listDeviceIds() {
        return latestTelemetry.keySet().stream().sorted().toList();
    }

    public Optional<Telemetry> findLatest(String nodeId) {
```

```
        return Optional.ofNullable(latestTelemetry.get(nodeId));
    }
}
```

A konstruktorban feltöltött mintaadatok lehetővé teszik a REST-végpontok azonnali tesztelését. Az `Optional` használatával a controller később kényelmesen kezeli a hiányzó eszközt.

14.3.4.5 5. lépés: REST controller

Készítsük el a `DeviceController` osztályt a `src/main/java/com/terrasense/backend/controller/DeviceCo` fájlban:

```
package com.terrasense.backend.controller;

import com.terrasense.backend.model.Telemetry;
import com.terrasense.backend.service.InMemoryDeviceService;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

import java.util.List;

@RestController
@RequestMapping("/api/devices")
public class DeviceController {

    private final InMemoryDeviceService deviceService;

    public DeviceController(InMemoryDeviceService deviceService) {
        this.deviceService = deviceService;
    }

    @GetMapping
    public List<String> listDevices() {
        return deviceService.listDeviceIds();
    }

    @GetMapping("/{nodeId}/latest")
    public ResponseEntity<Telemetry> latest(@PathVariable String nodeId) {
        return deviceService.findLatest(nodeId)
            .map(ResponseEntity::ok)
            .orElse(ResponseEntity.notFound().build());
    }
}
```

A `@GetMapping` és a `@PathVariable` a Spring Web deklaratív REST-kezelését mutatja: a metódus visszatérési értéke automatikusan JSON-szerializálódik.

14.3.4.6 6. lépés: Konfiguráció

Hozzuk létre az `src/main/resources/application.yaml` fájlt:

```yaml
server:
  port: 8080

spring:
  application:
    name: ts-backend

management:
  endpoints:
    web:
      exposure:
        include: health,info,metrics
  endpoint:
    health:
      show-details: always
```

A `management.endpoints.web.exposure.include` beállítás nélkül az Actuator csak a `/actuator/health` végpontot teszi elérhetővé; a `metrics` most azért kerül be, hogy a Megfigyelhetőség végponttól végpontig: metrikák, naplók, nyomkövetés alfejezetben rögtön láthassuk az első adatokat.

Hozzunk létre egy `application-dev.yaml` profilt is, amely a fejlesztés során hasznos részletes naplózást kapcsol be:

```yaml
logging:
  level:
    com.terrasense.backend: DEBUG
```

A profil aktiválása a `SPRING_PROFILES_ACTIVE=dev` környezeti változóval történik a konténerben, vagy parancssorból `--spring.profiles.active=dev` kapcsolóval.

14.3.4.7 7. lépés: Build és helyi futtatás

Ellenőrizzük a buildet:

```
mvn verify
```

Indítsuk el lokálisan (hoston, nem konténerben), hogy meggyőződjünk a működésről:

```
mvn spring-boot:run
```

Ekkor a logban megjelenik a `Tomcat started on port 8080` üzenet. Nyissunk egy másik terminálablakot, és próbáljuk ki:

```
curl -s http://localhost:8080/actuator/health
```

Várható kimenet:

```
{"status":"UP"}
```

14.3.4.8 8. lépés: Dockerfile

Állítsuk le a lokális folyamatot (`Ctrl+C`), majd hozzuk létre a `Dockerfile` fájlt az `~/iot-labor/i043` könyvtárban:

```
FROM maven:3.9-eclipse-temurin-21 AS build
WORKDIR /app
COPY pom.xml .
RUN mvn dependency:go-offline
COPY src ./src
RUN mvn package -DskipTests

FROM eclipse-temurin:21-jre
RUN apt-get update \
 && apt-get install -y --no-install-recommends curl \
 && rm -rf /var/lib/apt/lists/*
WORKDIR /app
COPY --from=build /app/target/ts-backend-*.jar app.jar
EXPOSE 8080
HEALTHCHECK --interval=10s --timeout=3s --start-period=30s --retries=5 \
  CMD curl -fsS http://localhost:8080/actuator/health || exit 1
ENTRYPOINT ["java", "-jar", "app.jar"]
```

A multi-stage build először Maven-környezetben felépíti a futtatható JAR-t, majd azt egy minimális Java-futásidejű image-be másolja. A `mvn dependency:go-offline` előtölti a függőségeket, így a build gyorsabban és megbízhatóbban fut konténerben. A `HEALTHCHECK` a futásidejű image-be telepített `curl`-lel hívja meg az Actuator `/actuator/health` végpontját; a `--start-period=30s` adja a Spring-kontextus felállásához szükséges időt. A `docker ps (healthy)` állapota tehát tényleges HTTP-ellenőrzésből származik, nem a folyamat futásából; az Actuator health csak a Spring-kontextus és a beépített indikátorok állapotát tükrözi, a későbbi saját health indikátorok (MQTT, adatbázis) külön liveness/readiness szemantikát adnak.

14.3.4.9 9. lépés: Compose-integráció

A `~/iot-labor/stack/compose.yaml` fájlt bővítsük ki a `ts-backend` szolgáltatással. A meglévő `lab-broker` szolgáltatás mellett adjuk hozzá a következő blokkot (a teljes fájl később, a Záróprojekt: a teljes TerraSense rendszer összeállítása és átvétele alfejezetben lesz kiírva):

```
  ts-backend:
    build: ../i043
    container_name: ts-backend
    ports:
      - "8080:8080"
    environment:
      - SPRING_PROFILES_ACTIVE=dev
    depends_on:
      - lab-broker
```

A `build: ../i043` a labor könyvtárban lévő Dockerfile-ra mutat. A `depends_on` csak az indítási sorrendet befolyásolja, nem várja meg, amíg a broker teljesen fel nem áll – ezt a Gateway üzemeltetése: systemd, konténerek, naplózás és későbbi alfejezetek részletesen tárgyalják.

14.3.4.10 10. lépés: Futtatás a Compose-ban

A `~/iot-labor/stack` könyvtárból indítsuk el a szolgáltatást:

```
cd ~/iot-labor/stack
docker compose up -d --build ts-backend
```

Ellenőrizzük, hogy a konténer fut:

```
docker ps --filter name=ts-backend --format '{{.Names}} {{.Status}}'
```

Várható kimenet (a pontos uptime eltérhet):

```
ts-backend Up 5 seconds (healthy)
```

Végül próbáljuk le a REST API-t a hostról:

```
curl -s http://localhost:8080/api/devices
```

Várható kimenet:

```
["n001","n002"]
```

A legutolsó mérés lekérdezése:

```
curl -s http://localhost:8080/api/devices/n001/latest
```

Várható kimenet:

```
{"nodeId":"n001","ts":1734567890123,"temperature":22.5,"humidity":61.0,"soilMoisture":34.0}
```

Ismeretlen eszköz esetén a válasz `404 Not Found`:

```
curl -i http://localhost:8080/api/devices/n999/latest
```

A válasz első sora: `HTTP/1.1 404`.

14.3.5 Ellenőrzés és hibakeresés

A labor helyes végrehajtását a következő tesztekkel ellenőrizhetjük:

1. **Build sikeressége:** `mvn verify` hiba nélkül lefut, és létrejön a `target/ts-backend-0.1.0.jar` fájl.
2. **Actuator health:** `curl -s http://localhost:8080/actuator/health` `{"status":"UP"}` értéket ad.
3. **REST végpontok:** mindkét eszközlista és a legutolsó mérés visszatér a várt JSON-nel.
4. **Konténer-indítás:** a `docker compose up -d --build ts-backend` nem hibázik, a konténer `healthy` vagy `Up` állapotban van.

Tipikus hibatünetek és javításuk:

Tünet	Ok	Javítás
`curl: (7) Failed to connect`	A Spring Boot nem fut, vagy nem a 8080-as porton	Ellenőrizd a `docker ps` kimenetét; ha a konténer nem fut, nézd meg a `docker logs ts-backend` naplót.
`404 Not Found` a `/actuator/health`-en	Az Actuator nincs a classpath-on, vagy az exposure nem tartalmazza a `health`-et	Győződj meg arról, hogy a `spring-boot-starter-actuator` függőség benne van, és az `application.yaml` exposure listája helyes.

Tünet	Ok	Javítás
`No qualifying bean of type 'InMemoryDeviceService'`	A service nincs `@Service` annotációval ellátva, vagy nem a komponenskeresés alatti package-ben van	Ellenőrizd a package nevet (`com.terrasense.backend`) és az annotációt.
A konténer elindul, de a REST nem érhető el a hostról	A port-hozzárendelés hiányzik a Compose-ból	A `ts-backend` szolgáltatásban legyen `ports: - "8080:8080"`.
A Docker build lassú vagy elakad	A `mvn dependency:go-offline` letölti az összes függőséget, és a hálózat lassú	Használj helyi Maven repository cache-t (`~/.m2` bindolása), vagy várd meg a letöltést.

14.3.6 Tipikus hibák és félreértések

1. **A health check nem egyenlő a rendszer egészségével.** Az Actuator `/actuator/health` végpontja csak azt igazolja, hogy a Spring kontextus felállt, és a beépített indikátorok nem jeleztek hibát. Azt nem tudja megmondani, hogy a MQTT-kapcsolat él, a TimescaleDB elérhető, vagy a legutóbbi mérés friss-e. A későbbi alfejezetekben ezért saját health indikátorokat és alkalmazásszintű metrikákat adunk hozzá.

2. **A memóriabeli tároló nem perzisztens.** Ha a konténer újraindul, a konstruktorban betöltött mintaadatok visszaállnak, a futás közben hozzáadott mérések eltűnnek. Ez a váz szándékos része, de élesben azonnal megjelenne az adatvesztés. A Idősoros adatbázisok és adatmodellezés alfejezetben a memóriabeli service-t TimescaleDB-alapú repository-ra cseréljük.

3. **A környezeti változók felülbírálják a YAML-t, de nem mindig közvetlenül.** A Spring Bootban a `SPRING_PROFILES_ACTIVE` például a profil aktiválását szabályozza, míg a `SERVER_PORT` a portot. A property-k kötésekor a Spring relaxed bindingot használ: a YAML-ben `server.port`, a környezeti változóban `SERVER_PORT` ugyanazt a beállítást éri el. Nem minden környezeti változó ilyen; a `management.endpoints.web.exposure.include` például nem rövidíthető `MANAGEMENT_ENDPOINTS_WEB_EXPOSURE_INCLUDE`-ra a pontok miatt.

4. **A 404 nem feltétlenül hibát jelent.** A `ResponseEntity.notFound().build()` szándékosan 404-et ad ismeretlen eszköz esetén. Ez gyakori félreértés: a kliens hibásan hibakeresést indít, pedig a szerver helyesen jelzi, hogy az erőforrás nem létezik. A megfelelő státuszkódok használata fontos az API-fogyasztók számára.

14.3.7 Feladatok

1. **Fogalmi/döntési kérdés:** Miért használunk a REST API-ban `/api` előtagot, és miért hasznos, ha az Actuator végpontjait nem ugyanazon az útvonalon tesszük közzé? Fogalmazz meg egy olyan helyzetet, amikor a `/api` nélküli nyílt Actuator kockázatot jelent.

2. **Számítási/tervezési feladat:** Tegyük fel, hogy az eszközlista 1000 eszközből áll, és minden eszköz nevét `n001`, `n002`, … formátumban tároljuk. Becsüld meg a `GET /api/devices` válasz JSON-méretét, ha minden azonosító átlagosan 4 karakter, a JSON tömb szintaxisa pedig `["n001","n002",...]`. A vessző és az idézőjelek is karakterek. Nagyságrendileg

hány bájt lesz? (A tényleges méret a UTF-8 kódolástól és a szerializációtól függ, itt elég a közelítő becslés.)

3. **Implementációs feladat a TerraSense-ben:** Bővítsd az `InMemoryDeviceService` osztályt egy `updateTelemetry(Telemetry)` metódussal, amely beilleszti vagy felülírja a legutóbbi mérést. Ez a metódus később az MQTT-ingest feldolgozásakor fog kelleni. Írj hozzá egy `@PostMapping` végpontot is a `DeviceController`-be, amely fogad egy új mérést, és beírja a service-be. Teszteld `curl` segítségével.

4. **Hibakeresési/kiterjesztési feladat:** A `ts-backend` konténer elindul, de a `curl http://localhost:8080/actuator/health` hosszú ideig nem ad választ, majd `Connection timed out` jelenik meg. Sorolj fel legalább három lehetséges okot, és mindegyikre adj egy-egy diagnosztikai lépést a `docker logs` vagy a `docker inspect` parancs segítségével.

14.3.8 Megoldások

14.3.8.1 1. Fogalmi kérdés

Az `/api` előtag elkülöníti a publikus, üzleti REST erőforrásokat az üzemeltetési belső végpontoktól. Ha az Actuator végpontjai is nyilvánosan, `api` előtag nélkül lennének elérhetőek, egy támadó könnyebben felderíthetné a belső állapotot (`/actuator/health`, `/actuator/env`, `/actuator/metrics`). A `security` azt is lehetővé teszi, hogy az `/actuator/**` útvonalra szigorúbb hozzáférés-vezérlést alkalmazzunk, míg az `/api` útvonalakra más szabályok vonatkoznak. A TerraSense esetében a dashboard és a külső kliensek az `/api`-t, a monitoring pedig az `/actuator`-t használja.

14.3.8.2 2. Méretbecslés

Egy azonosító JSON-ban így néz ki: `"n001"` = 1 idézőjel + 4 karakter + 1 idézőjel = 6 karakter. A tömb elemei között vessző van. 1000 elemnél a vesszők száma 999, egyenként 1 karakter. A nyitó és záró szögletes zárójel 2 karakter. Így a teljes hossz:

$$1000 \cdot 6 + 999 \cdot 1 + 2 = 6000 + 999 + 2 = 7001 \text{ karakter}$$

UTF-8 kódolással, ha minden karakter 1 bájt (a példa karakterei ASCII), a válasz mérete körülbelül **7,0 kB**. Ha a szerializáció még szóközöket vagy sortöréseket ad hozzá, a méret kissé nőhet, de nagyságrendileg 7 kB marad. Ez jól mutatja, hogy a JSON mérete sok kis azonosítónál a szintaxis miatt gyorsan nő; nagyobb listáknál érdemes lapozást vagy rövidebb kódolást használni.

14.3.8.3 3. Implementációs feladat

Az `InMemoryDeviceService` bővítése:

```
public boolean updateTelemetry(Telemetry telemetry) {
    java.util.concurrent.atomic.AtomicBoolean updated =
        new java.util.concurrent.atomic.AtomicBoolean(false);
    latestTelemetry.compute(telemetry.nodeId(), (id, current) -> {
        boolean newer = current == null
            || telemetry.seq() > current.seq()
            || (telemetry.seq() == current.seq()
```

```
                && telemetry.ts() > current.ts());
        if (newer) {
            updated.set(true);
            return telemetry;
        }
        return current;
    });
    return updated.get();
}
```

A `DeviceController` bővítése:

```
@PostMapping("/{nodeId}/latest")
public ResponseEntity<Void> updateLatest(@PathVariable String nodeId,
                                         @RequestBody Telemetry telemetry) {
    if (!nodeId.equals(telemetry.nodeId())) {
        return ResponseEntity.badRequest().build();
    }
    if (telemetry.seq() < 0 || telemetry.ts() <= 0
            || !Double.isFinite(telemetry.temperature())
            || !Double.isFinite(telemetry.humidity())
            || !Double.isFinite(telemetry.soilMoisture())
            || telemetry.temperature() < -40.0
            || telemetry.temperature() > 85.0
            || telemetry.humidity() < 0.0 || telemetry.humidity() > 100.0
            || telemetry.soilMoisture() < 0.0
            || telemetry.soilMoisture() > 100.0) {
        return ResponseEntity.badRequest().build();
    }
    if (!deviceService.updateTelemetry(telemetry)) {
        // Elavult vagy duplikált esemény: a frissebb latest érték marad.
        return ResponseEntity.status(HttpStatus.CONFLICT).build();
    }
    return ResponseEntity.noContent().build();
}
```

A `@PostMapping` és `@RequestBody` a beérkező JSON-t automatikusan `Telemetry` objektummá alakítja. A `noContent()` a 204-es státuszt adja vissza, jelezve, hogy a kérés sikeres, de nincs választest. A `compute` a `(seq, ts)` párost használja a legfrissebb esemény kiválasztására: nagyobb `seq` mindig frissebb, azonos `seq` esetén a nagyobb `ts` nyer; így egy késve érkező régi minta nem cseréli le a frissebb latest értéket, és 409-cel jelez.

Tesztelés:

```
curl -i -X POST http://localhost:8080/api/devices/n003/latest \
  -H 'Content-Type: application/json' \
  -d
   ↪  '{"nodeId":"n003","seq":1,"ts":1734567900000,"temperature":21.0,"humidity":55.0,"soilMoisture":29.5}'
```

Ezután a `GET /api/devices/n003/latest` a friss adatot adja vissza. Egy kisebb `seq`-ű vagy régebbi `ts`-ű kérés `409 Conflict` választ kap, és a store-ban a frissebb érték marad.

14.3.8.4 4. Hibakeresési feladat

Lehetséges okok:

- **A konténer nem is fut.** `docker ps -a | grep ts-backend` megmutatja az állapotot. Ha `Exited`, nézd meg a `docker logs ts-backend` naplót, valószínűleg a JAR nem indul el (pl. rossz package vagy port).

- **A port-hozzárendelés hiányzik vagy ütközik.** `docker port ts-backend` mutatja a port-hozzárendelést. Ha nincs `0.0.0.0:8080`, akkor a hostról nem érhető el. Ha a 8080-as port már foglalt, a Compose indításkor hibát ad; cseréld a host portot, például `8081:8080`.
- **A Spring Boot a konténeren belül nem a 8080-as porton figyel.** A Dockerfile `EXPOSE 8080`-at ad, de a `application.yaml`-ben `server.port: 8080` van; ha ezt felülbírálod `SERVER_PORT` környezeti változóval, a port-hozzárendelés elavul. Ellenőrizd a `docker logs ts-backend` első sorait: a `Tomcat started on port ...` mutatja a tényleges portot.
- **A konténer hálózata nem éri el a hostot.** Ha `network_mode: host` vagy más hálózati beállítás van, a port-hozzárendelés másként működik. `docker inspect ts-backend` mutatja a hálózati módot; a Compose-ban a `ports` beállítás bridge hálózatot feltételez.

14.3.9 Rövid összefoglaló

Elkészítettük a `ts-backend` Spring Boot vázát a `~/iot-labor/i043` könyvtárban. A projekt Maven-alapú, Java 21 és Spring Boot 3.x verzióval. A csomagszerkezet: `com.terrasense.backend` alatt `controller`, `service`, `model`. A `DeviceController` két REST végpontot kínál: `GET /api/devices` és `GET /api/devices/{nodeId}/latest`. A `InMemoryDeviceService` memóriabeli `ConcurrentHashMap`-ben tárolja a legutolsó méréseket, és konstruktorban mintaadatokat tölt be. Az Actuator `/actuator/health`, `/actuator/info` és `/actuator/metrics` végpontokat tesz elérhetővé. A konfiguráció `application.yaml`-ben, a fejlesztői naplózás `application-dev.yaml` profilban található. A multi-stage `Dockerfile` futtatható JAR-t épít. A `~/iot-labor/stack/compose.yaml`-ben a `ts-backend` szolgáltatás a `../i043` build contextet használja, a 8080-as portot publikálja, és a `dev` profilt aktiválja. **Továbbvitt eredmény.** Ez a váz adja a csomagszerkezetet, a konfigurációs stílust, a REST API-t és a konténerizációt a Ingest: MQTT-ből a backendbe, idempotensen, a Idősoros adatbázisok és adatmodellezés, a Parancsok és konfiguráció lefelé: kézbesítés, nyugtázás, időkorlát és a Megfigyelhetőség végponttól végpontig: metrikák, naplók, nyomkövetés alfejezetekhez.

14.3.10 Források és ellenőrzési hiányok

- Spring Boot dokumentáció: https://docs.spring.io/spring-boot/ – ellenőrizendő a 3.4.1 pontos verzió és a `spring-boot-starter-parent` kompatibilitás Java 21-gyel.
- Spring Boot Actuator dokumentáció: https://docs.spring.io/spring-boot/reference/actuator/ – ellenőrizendő a `management.endpoints.web.exposure.include` property aktuális neve és a `show-details` lehetséges értékei.
- Maven image a Docker Hubon: `maven:3.9-eclipse-temurin-21` – ellenőrizendő, hogy ez a tag elérhető és aktuális.
- Eclipse Temurin JRE 21 image: `eclipse-temurin:21-jre` – ellenőrizendő a pontos tag.
- A `mvn dependency:go-offline` parancs a Maven 3.9-ben – ellenőrizendő, hogy a használt Maven verzióval kompatibilis-e a Docker build során.
- A Spring Boot `@SpringBootApplication` alapértelmezett komponenskeresése a fő osztály package-ére és annak alpackage-eire terjed ki – ez a jelen példában `com.terrasense.backend`.

15 Ingest és adattárolás

15.1 Ingest: MQTT-ből a backendbe, idempotensen

15.1.1 Motiváció

A TerraSense rendszerben a telemetria a `terrasense/<site>/<node>/telemetry` topicra érkezik, és a MQTT QoS, munkamenetek, retain, Last Will és MQTT 5 alfejezetben meghatározott konvenció szerint QoS 1-es publikálással utazik. A QoS 1 garantálja, hogy az üzenet legalább egyszer megérkezik a brokerhez, de a hálózati hibák utáni újraküldés miatt ugyanazt az üzenetet többször is kézbesítheti. Ha a backend minden egyes érkezést új sorként ír az adatbázisba, a duplikátumok torzítják az átlagot, a riasztási szabályokat és a későbbi idősoros elemzést. Ebben az alfejezetben megépítjük a ts-backend MQTT-ingest komponensét, amely a kanonikus sémát és a korábbi lapos formátumot is értelmezi, ellenőrzi a topic és a payload azonosságát, és a `(site, node_id, boot_id, seq)` eseménykulcs segítségével idempotensen dolgozza fel az üzeneteket.

15.1.2 Fogalmak és működés

Definíció. (Idempotens feldolgozás) Egy feldolgozási lépés akkor idempotens, ha ugyanazt a bemeneti üzenetet többször végrehajtva ugyanazt a külső hatást érjük el, mint egyszeri végrehajtáskor. MQTT-ingest esetén ez azt jelenti, hogy a duplikált telemetriaüzenet nem hoz létre új adatsort, és nem vált ki többszörös riasztást vagy más mellékhatást.

Definíció. (Shared subscription) Az MQTT 5 shared subscription egy broker-szolgáltatás, amely lehetővé teszi, hogy egy topicüzenetet egy feliratkozási csoport több kliense közül csak egy kapjon meg. A szintaxis: `$share/<csoportnév>/<topic>`. Ha például két ts-backend példány fut, mindkettő a `$share/ingest/terrasense/+/+/telemetry` topicra iratkozik fel, és a Mosquitto egy-egy üzenetet csak az egyiküknek továbbít. Ezzel elkerülhető a párhuzamos példányok közötti duplikált feldolgozás anélkül, hogy közös állapotot kellene fenntartaniuk.

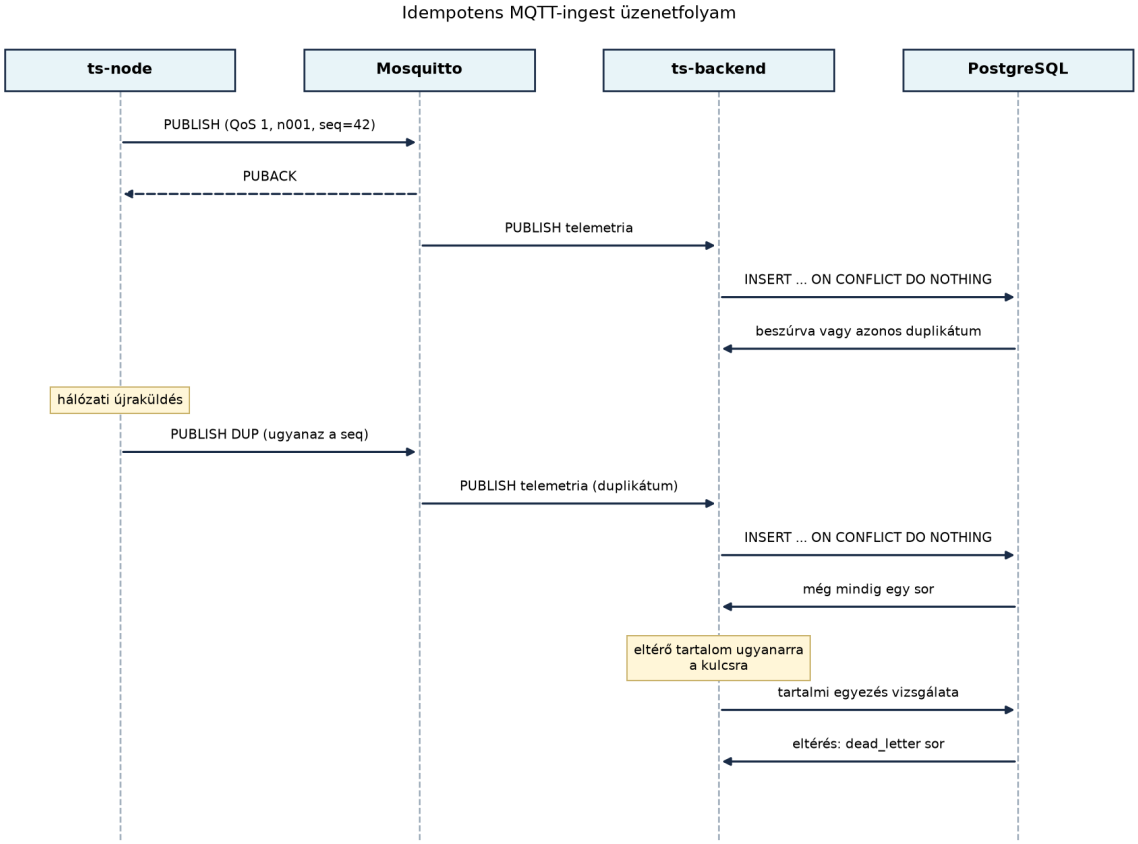
Definíció. (Dead letter) A dead letter olyan tároló (tipikusan topic vagy adatbázistábla), ahová azok az üzenetek kerülnek, amelyeket a rendszer nem tud értelmezni vagy feldolgozni. Az ingest-komponensben ez az érvénytelen JSON-üzenetek és a tartalmilag ütköző események célhelye, hogy az üzemeltető később kivizsgálhassa őket anélkül, hogy a hibás üzenet blokkolná a teljes adatcsatornát.

Definíció. (Kanonikus eseménykulcs) A TerraSense telemetria idempotencia-kulcsa a `(site, node_id, boot_id, seq)` négyes. A `site` és a `node_id` a topicból származó identitás, a `boot_id` az eszköz adott bootolási ciklusát azonosítja, a `seq` pedig az adott booton belül monoton növekvő sorszám. A `boot_id` hiányában a `boot-0` alapértéket használjuk; így a régi, bootazonosítót nem küldő eszközök eseményei is egyértelműen kulcsolhatók.

Definíció. (Retry spool) A retry spool egy tartós, helyi fájlpuffer, amely azokat az üzeneteket őrzi meg, amelyeket átmeneti infrastruktúrahiba (például adatbázis-leállás) miatt nem sikerült feldolgozni. A spool a hálózati vagy adatbázis-helyreállítás után újrapróbálja az üzeneteket, így azok nem vesznek el pusztán azért, mert a feldolgozó háttértár átmenetileg nem elérhető. A

spool csak a tartós adatvesztés elleni védelem; nem helyettesíti a DLQ-t, amely a véglegesen feldolgozhatatlan vagy tartalmilag ütköző üzeneteké.

Az ingest adatútja a következő:



15.1. ábra: Idempotens MQTT-ingest üzenetfolyam - A duplikált QoS 1 üzenetet a ts-backend a (site, node_id, boot_id, seq) kulccsal fogadja; az ON CONFLICT DO NOTHING nem hoz létre új sort, az eltérő tartalmú ütközés pedig a DLQ-ba kerül.

A fenti diagramon a ts-node QoS 1-es üzenetet küld, amelyet a Mosquitto PUBACK-kel nyugtáz. Ha a PUBACK elveszik, a kliens újraküldi ugyanazt az üzenetet; a hálózati duplikátum ismét eljut a backendhez. A backend mindkét esetben ugyanazt az `INSERT ... ON CONFLICT DO NOTHING` utasítást futtatja. Ha a meglévő sor tartalma azonos, a művelet csendben visszatér; ha a tartalom eltér, a backend nem írja felül az első beérkezést, hanem a `dead_letter` táblába menti az ütközést.

15.1.3 Döntési szempontok

Az MQTT-kliens kiválasztásakor a Spring világban két tipikus út van: az Eclipse Paho Java kliens közvetlen használata, vagy a Spring Integration MQTT-adaptere. A TerraSense oktatási céljaihoz a **Paho Java kliens** mellett döntünk, mert:

- a callback-alapú kód átláthatóan mutatja az MQTT 5 tulajdonságait (például a `messageArrived` metódusban elérhető `MqttMessage`-et);
- a C++-os Paho kliensből ismert fogalmak (publisher, connection options) közvetlenül megjelennek;

- a Spring Integration absztrakció kényelmes, de a jelen alfejezet tanulási célja nem a keretrendszer-konfiguráció, hanem az idempotens feldolgozás és a protokollszintű viselkedés megértése;
- a Paho 5-ös kliense shared subscriptiont is tud, ami a többpéldányos telepítés alapja.

A választás kompromisszumait az alábbi táblázat foglalja össze.

Szempont	Paho Java kliens	Spring Integration MQTT
MQTT 5 támogatás	közvetlen, verziófüggő	újabb verziókban, absztrakció mögött
Tanulási átláthatóság	magas	közepes
Rugalmasság (callback, manuális ack)	teljes	korlátozott
Konfigurációs kód	explicit	deklaratív
TerraSense-ben	**választott**	nem használt

A shared subscription használatáról akkor érdemes dönteni, ha a ts-backend több példányban fut. Egy példány esetén a normál `terrasense/+/+/telemetry` feliratkozás elegendő, és a topic-wildcard biztosítja, hogy minden üzenet eljusson az ingesthez. Több példánynál a `$share/ingest/terrasense/+/+/telemetry` használata megakadályozza, hogy ugyanazt az üzenetet mindegyik példány feldolgozza – azonban az idempotens adatmodell így is védőhálót ad, mert ha valamelyik példány mégis kétszer kapná az üzenetet, az `ON CONFLICT DO NOTHING` akkor sem hoz létre duplikátumot. Több példány esetén a kliensazonosítónak példányonként egyedinek kell lennie (például `ts-backend-ingest-1`, `ts-backend-ingest-2`), különben az MQTT 5 munkamenet-átvétel miatt a broker folyamatosan lecsatlakoztatja az egyik klienst.

A validálás során a backend a Sémák és verziózás: kompatibilis változtatások alfejezetben rögzített elvet követi: a sémaverziót az üzenet hordozza (`schema_version` mező). A TerraSense kanonikus, 1-es verziójú üzenete a Adatmodell: eszköz, telemetria, telemetriai séma alfejezet szerinti `measurements` objektumot tartalmazza; a Backend és gateway közötti adatszerződés alfejezetben bemutatott korábbi, lapos `temperature_c`/`humidity_rh`/`soil_moisture_pct` mezőket a backend 0-es verziójú, dokumentált adapterrel fogadja el. Az ismeretlen mezők nem okoznak deszerializációs hibát, de a kanonikus payloadot a repository normalizálja, így a tárolt JSON szerkezete egységes. Az érvénytelen üzeneteket a `dead_letter` táblába írjuk. A `terrasense/ingest/dead-letter` MQTT topicra történő tükrözés lehetséges üzemeltetési kiegészítés, de ezt a labor nem építi meg; a labor DLQ-ja az adatbázistábla.

15.1.4 Kidolgozott példa vagy labor

Munkakönyvtár: `~/iot-labor/i044`

```
mkdir -p ~/iot-labor/i044 && cd ~/iot-labor/i044
cp -r ~/iot-labor/i043 ./ts-backend
cd ts-backend
```

A A ts-backend váza: REST API és egészségellenőrzés alfejezetben a Maven-projekt gyökere közvetlenül `~/iot-labor/i043`, ezért a teljes könyvtárat másoljuk `ts-backend` néven. Így a `pom.xml`, a `Dockerfile` és a `src/` a várt `~/iot-labor/i044/ts-backend/` alá kerül, és a későbbi Compose build context `../i044/ts-backend` lehet.

Előfeltétel: a közös `~/iot-labor/stack/compose.yaml`-ben futó Mosquitto (`lab-broker`) és TimescaleDB (`lab-timescaledb`) szolgáltatás. A 14.3-ban a Compose csak a brokert tartalmazta;

most adjuk hozzá az adatbázist, a backendet és a broker perzisztens kötetét. A `compose.yaml` releváns részlete:

```
services:
  lab-timescaledb:
    image: timescale/timescaledb:2.17.2-pg16
    container_name: lab-timescaledb
    environment:
      POSTGRES_DB: terrasense
      POSTGRES_USER: terra
      POSTGRES_PASSWORD: lab-password
    ports:
      - "5432:5432"
    volumes:
      - lab-timescaledb-data:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U terra -d terrasense"]
      interval: 5s
      timeout: 3s
      retries: 20

  ts-backend:
    build: ../i044/ts-backend
    container_name: ts-backend
    ports:
      - "8080:8080"
    environment:
      MQTT_BROKER_URI: tcp://lab-broker:1883
      SPRING_DATASOURCE_URL: jdbc:postgresql://lab-timescaledb:5432/terrasense
      SPRING_DATASOURCE_USERNAME: terra
      SPRING_DATASOURCE_PASSWORD: lab-password
      INGEST_SPOOL_PATH: /app/spool/ingest-retry.jsonl
    depends_on:
      lab-broker:
        condition: service_healthy
      lab-timescaledb:
        condition: service_healthy

volumes:
  lab-timescaledb-data:
    name: lab-timescaledb-data
  lab-broker-data:
    name: lab-broker-data
```

A meglévő `lab-broker` szolgáltatás `volumes` listájához vegyük fel a perzisztens adatkönyvtárat:

```
    volumes:
      - ./mosquitto/mosquitto.conf:/mosquitto/config/mosquitto.conf:ro
      - lab-broker-data:/mosquitto/data
```

A Mosquitto konfigurációjába pedig kerüljön be a perzisztencia, hogy a broker újraindítása után a QoS 1-es munkamenet és a várakozó üzenetek megmaradjanak:

```
cat >> ~/iot-labor/stack/mosquitto/mosquitto.conf <<'EOF'
persistence true
persistence_location /mosquitto/data/
autosave_interval 30
EOF
```

A labor indítása:

```
docker compose -f ~/iot-labor/stack/compose.yaml up -d lab-timescaledb lab-broker
```

1. A Maven-projekt és a függőségek. A 14.3-ban létrehozott `pom.xml` szülőverzióját a könyv rögzített verzióprofja szerint 3.3.4-re állítjuk, majd kiegészítjük a JDBC-, PostgreSQL-, Lombok- és Paho-függőségekkel. A teljes, működő `pom.xml`:

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
                             https://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>

    <parent>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-parent</artifactId>
        <version>3.3.4</version>
        <relativePath/>
    </parent>

    <groupId>com.terrasense</groupId>
    <artifactId>ts-backend</artifactId>
    <version>0.1.0</version>
    <name>ts-backend</name>
    <description>TerraSense backend</description>

    <properties>
        <java.version>21</java.version>
        <paho.version>1.2.5</paho.version>
    </properties>

    <dependencies>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-web</artifactId>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-actuator</artifactId>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-jdbc</artifactId>
        </dependency>
        <dependency>
            <groupId>org.postgresql</groupId>
            <artifactId>postgresql</artifactId>
            <scope>runtime</scope>
        </dependency>
        <dependency>
            <groupId>org.eclipse.paho</groupId>
            <artifactId>org.eclipse.paho.mqttv5.client</artifactId>
            <version>${paho.version}</version>
        </dependency>
        <dependency>
            <groupId>org.projectlombok</groupId>
            <artifactId>lombok</artifactId>
            <optional>true</optional>
        </dependency>
    </dependencies>

    <build>
        <plugins>
            <plugin>
                <groupId>org.springframework.boot</groupId>
```

```
                <artifactId>spring-boot-maven-plugin</artifactId>
                <configuration>
                    <excludes>
                        <exclude>
                            <groupId>org.projectlombok</groupId>
                            <artifactId>lombok</artifactId>
                        </exclude>
                    </excludes>
                </configuration>
            </plugin>
            <plugin>
                <groupId>org.apache.maven.plugins</groupId>
                <artifactId>maven-compiler-plugin</artifactId>
                <configuration>
                    <annotationProcessorPaths>
                        <path>
                            <groupId>org.projectlombok</groupId>
                            <artifactId>lombok</artifactId>
                            <version>${lombok.version}</version>
                        </path>
                    </annotationProcessorPaths>
                </configuration>
            </plugin>
        </plugins>
    </build>
</project>
```

A `spring-boot-starter-jdbc` hozza a `JdbcTemplate`-et és a HikariCP kapcsolatkészletet, a `postgresql` driver runtime hatókörben van, a Lombok pedig fordításidejű annotációfeldolgozó. A Spring Boot szülő a Lombok verzióját a `${lombok.version}` tulajdonságon keresztül kezeli; a `maven-compiler-plugin` `annotationProcessorPaths` bejegyzése nélkül a `@Slf4j` a friss Maven-cache mellett fordítási hibát adhat.

A labor végén a projektfa a következő (a 14.3-ból származó vezérlő, modell és memóriabeli service megmarad):

```
ts-backend/
  pom.xml
  Dockerfile
  src/main/
      java/com/terrasense/backend/
          TsBackendApplication.java
          controller/DeviceController.java
          model/Telemetry.java
          service/InMemoryDeviceService.java
          TelemetryMessage.java
          Measurement.java
          TelemetryIngestService.java
          TelemetryRepository.java
          IngestRetrySpool.java
          SchedulingConfig.java
      resources/
           application.yaml
           schema.sql
```

2. Telemetria DTO a kanonikus és a régi formátumhoz. A `src/main/java/com/terrasense/backend/Te` és a mellette lévő `Measurement.java`:

```
package com.terrasense.backend;

import com.fasterxml.jackson.annotation.JsonIgnoreProperties;
import java.util.LinkedHashMap;
```

```java
import java.util.Map;

@JsonIgnoreProperties(ignoreUnknown = true)
public record TelemetryMessage(
    Integer schema_version,
    String node_id,
    Long seq,
    Long ts,
    String site,
    String boot_id,
    Map<String, Measurement> measurements,
    Double temperature_c,
    Double humidity_rh,
    Double soil_moisture_pct
) {
    public static final int CANONICAL_SCHEMA_VERSION = 1;

    public int effectiveSchemaVersion() {
        if (schema_version != null) {
            return schema_version;
        }
        return isLegacyFlat() ? 0 : -1;
    }

    public String effectiveBootId() {
        return (boot_id == null || boot_id.isBlank()) ? "boot-0" : boot_id;
    }

    public boolean isLegacyFlat() {
        return (measurements == null || measurements.isEmpty())
            && (temperature_c != null || humidity_rh != null || soil_moisture_pct != null);
    }

    public Map<String, Measurement> normalizedMeasurements() {
        if (measurements != null && !measurements.isEmpty()) {
            return measurements;
        }
        Map<String, Measurement> legacy = new LinkedHashMap<>();
        if (temperature_c != null) {
            legacy.put("temperature", new Measurement(temperature_c, "Cel", true, null));
        }
        if (humidity_rh != null) {
            legacy.put("humidity", new Measurement(humidity_rh, "%RH", true, null));
        }
        if (soil_moisture_pct != null) {
            legacy.put("soil_moisture", new Measurement(soil_moisture_pct, "%", true, null));
        }
        return legacy;
    }
}

package com.terrasense.backend;

import com.fasterxml.jackson.annotation.JsonIgnoreProperties;

@JsonIgnoreProperties(ignoreUnknown = true)
public record Measurement(Double value, String unit, Boolean valid, String reason) {}
```

A mezők `Integer`, `Long` és `Double` típusúak, nem primitívek: így a hiányzó `schema_version`, `seq` vagy mérési érték `null` marad, és a validáció észleli, nem pedig csendben `0`-ra konvertálódik. A `normalizedMeasurements()` a 0-es verziójú lapos üzenetet alakítja át a kanonikus mérési objektummá; az 1-es verziójú üzenetet változatlanul adja tovább.

3. Ingest szolgáltatás MQTT 5 munkamenettel, validációval és retry spool-lal. A src/main/java/com/terrasense/backend/TelemetryIngestService.java:

```
package com.terrasense.backend;

import com.fasterxml.jackson.core.JsonProcessingException;
import com.fasterxml.jackson.databind.ObjectMapper;
import jakarta.annotation.PostConstruct;
import jakarta.annotation.PreDestroy;
import java.nio.charset.StandardCharsets;
import java.util.Map;
import java.util.regex.Matcher;
import java.util.regex.Pattern;
import lombok.extern.slf4j.Slf4j;
import org.eclipse.paho.mqttv5.client.IMqttToken;
import org.eclipse.paho.mqttv5.client.MqttCallback;
import org.eclipse.paho.mqttv5.client.MqttClient;
import org.eclipse.paho.mqttv5.client.MqttConnectionOptions;
import org.eclipse.paho.mqttv5.client.MqttDisconnectResponse;
import org.eclipse.paho.mqttv5.common.MqttException;
import org.eclipse.paho.mqttv5.common.MqttMessage;
import org.eclipse.paho.mqttv5.common.packet.MqttProperties;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.dao.DataAccessException;
import org.springframework.scheduling.annotation.Scheduled;
import org.springframework.stereotype.Service;

@Slf4j
@Service
public class TelemetryIngestService {

    private static final int MAX_SPOOL_ATTEMPTS = 60;
    private static final Pattern TOPIC_PATTERN =
        Pattern.compile("^terrasense/([^/]+)/([^/]+)/telemetry$");

    private final TelemetryRepository repository;
    private final ObjectMapper objectMapper;
    private final IngestRetrySpool retrySpool;
    private final String brokerUri;
    private final String clientId;
    private final String topicFilter;

    private MqttClient client;
    private MqttConnectionOptions options;

    public TelemetryIngestService(
            TelemetryRepository repository,
            ObjectMapper objectMapper,
            IngestRetrySpool retrySpool,
            @Value("${ingest.broker-uri}") String brokerUri,
            @Value("${ingest.client-id}") String clientId,
            @Value("${ingest.topic-filter}") String topicFilter) {
        this.repository = repository;
        this.objectMapper = objectMapper;
        this.retrySpool = retrySpool;
        this.brokerUri = brokerUri;
        this.clientId = clientId;
        this.topicFilter = topicFilter;
    }

    @PostConstruct
    public void start() throws MqttException {
        client = new MqttClient(brokerUri, clientId);
```

```
        options = new MqttConnectionOptions();
        options.setAutomaticReconnect(true);
        options.setCleanStart(false);
        options.setSessionExpiryInterval(3600L);
        client.setCallback(new MqttCallback() {
            @Override
            public void disconnected(MqttDisconnectResponse response) {
                log.warn("MQTT kapcsolat megszakadt: {}", response.getException());
            }

            @Override
            public void mqttErrorOccurred(MqttException exception) {
                log.warn("MQTT hiba: {}", exception.getMessage());
            }

            @Override
            public void messageArrived(String topic, MqttMessage message) {
                handleMessage(topic, message);
            }

            @Override
            public void deliveryComplete(IMqttToken token) {
                // QoS 1 kimenő üzenet nyugtázva; az ingest csak bejövő üzeneteket dolgoz fel.
            }

            @Override
            public void connectComplete(boolean reconnect, String serverURI) {
                log.info("MQTT kapcsolat kész: reconnect={}, broker={}", reconnect, serverURI);
                try {
                    subscribe();
                } catch (MqttException e) {
                    log.error("Az újrafeliratkozás sikertelen", e);
                }
            }

            @Override
            public void authPacketArrived(int reasonCode, MqttProperties properties) {
                // A labor broker anonymous kapcsolatot enged; hitelesítésnél itt dolgozható fel
                ↪  a válasz.
            }
        });
        client.connect(options);
        subscribe();
    }

    private void subscribe() throws MqttException {
        client.subscribe(topicFilter, 1, this::handleMessage);
        log.info("Feliratkozva: {}", topicFilter);
    }

    private void handleMessage(String topic, MqttMessage message) {
        String payload = new String(message.getPayload(), StandardCharsets.UTF_8);
        try {
            process(topic, payload);
        } catch (DataAccessException e) {
            spoolOrLog(topic, payload, e);
        } catch (RuntimeException e) {
            spoolOrLog(topic, payload, e);
        }
    }

    private void process(String topic, String payload) {
        Matcher matcher = TOPIC_PATTERN.matcher(topic);
```

```
        if (!matcher.matches()) {
            repository.insertDeadLetter(topic, payload, "váratlan topic");
            log.warn("Váratlan topic: {}", topic);
            return;
        }
        String site = matcher.group(1);
        String topicNode = matcher.group(2);

        TelemetryMessage telemetry;
        try {
            telemetry = objectMapper.readValue(payload, TelemetryMessage.class);
        } catch (JsonProcessingException e) {
            repository.insertDeadLetter(topic, payload, "érvénytelen JSON: " +
↪   e.getOriginalMessage());
            log.warn("Érvénytelen JSON a {} topicon", topic);
            return;
        }

        String validationError = validate(site, topicNode, telemetry);
        if (validationError != null) {
            repository.insertDeadLetter(topic, payload, validationError);
            log.warn("Érvénytelen telemetria ({}): {}", validationError, payload);
            return;
        }

        IngestOutcome outcome = repository.insertIfAbsent(site, telemetry, topic);
        if (outcome == IngestOutcome.CONTENT_MISMATCH) {
            log.error("Eltérő tartalom ugyanarra az eseménykulcsra: site={}, node={}, seq={}",
                site, telemetry.node_id(), telemetry.seq());
        } else if (outcome == IngestOutcome.DUPLICATE) {
            log.debug("Azonos duplikátum: site={}, node={}, seq={}",
                site, telemetry.node_id(), telemetry.seq());
        } else {
            log.info("Ingest OK: site={}, node={}, seq={}", site, telemetry.node_id(),
↪   telemetry.seq());
        }
    }

    private String validate(String site, String topicNode, TelemetryMessage m) {
        int version = m.effectiveSchemaVersion();
        if (version != 0 && version != TelemetryMessage.CANONICAL_SCHEMA_VERSION) {
            return "nem támogatott schema_version: " + m.schema_version();
        }
        if (m.node_id() == null || m.node_id().isBlank()) {
            return "hiányzó node_id";
        }
        if (m.seq() == null || m.seq() < 0) {
            return "hiányzó vagy negatív seq";
        }
        if (m.ts() == null || m.ts() < 1_600_000_000_000L || m.ts() > 4_000_000_000_000L) {
            return "érvénytelen ts";
        }
        if (!m.node_id().equals(topicNode)) {
            return "a topic node azonosítója eltér a payloadétól";
        }
        if (m.site() != null && !m.site().isBlank() && !m.site().equals(site)) {
            return "a payload site mezője eltér a topictól";
        }
        if (version == 0 && m.measurements() != null && !m.measurements().isEmpty()) {
            return "schema_version=0 mellett measurements nem lehet jelen";
        }
        if (version == 0 && !m.isLegacyFlat()) {
            return "schema_version=0 mellett hiányoznak a lapos mérési mezők";
```

```
    }
    if (version == 1 && (m.measurements() == null || m.measurements().isEmpty())) {
        return "schema_version=1 mellett hiányzó measurements";
    }

    Map<String, Measurement> measurements = m.normalizedMeasurements();
    if (measurements.isEmpty()) {
        return "nincs mérési adat";
    }
    for (Map.Entry<String, Measurement> entry : measurements.entrySet()) {
        if (entry.getKey() == null || entry.getKey().isBlank()) {
            return "üres méréstípus";
        }
        Measurement value = entry.getValue();
        if (value == null || value.value() == null) {
            return "hiányzó mérési érték: " + entry.getKey();
        }
        if (version == 1) {
            if (value.unit() == null || value.unit().isBlank()) {
                return "hiányzó mértékegység: " + entry.getKey();
            }
            if (value.valid() == null) {
                return "hiányzó valid jelző: " + entry.getKey();
            }
            if (!value.valid() && (value.reason() == null || value.reason().isBlank())) {
                return "érvénytelen mérés indoklás nélkül: " + entry.getKey();
            }
        }
    }
    return null;
}

private void spoolOrLog(String topic, String payload, Exception cause) {
    try {
        retrySpool.append(topic, payload);
        log.error("Átmeneti hiba; az üzenet a retry-spoolba került", cause);
    } catch (RuntimeException spoolError) {
        log.error("KRITIKUS: az üzenet sem az adatbázisba, sem a spoolba nem került: {}",
            payload, spoolError);
    }
}

@Scheduled(fixedDelay = 5000)
public void drainRetrySpool() {
    for (IngestRetrySpool.SpooledMessage spooled : retrySpool.readAll()) {
        try {
            process(spooled.topic(), spooled.payload());
            retrySpool.remove(spooled);
        } catch (DataAccessException e) {
            if (spooled.attempts() + 1 >= MAX_SPOOL_ATTEMPTS) {
                moveSpooledToDeadLetter(spooled, e);
            } else {
                retrySpool.replace(spooled, spooled.nextAttempt());
            }
            return;
        } catch (RuntimeException e) {
            try {
                repository.insertDeadLetter(spooled.topic(), spooled.payload(),
                    "feldolgozási hiba: " + e.getClass().getSimpleName());
            } catch (DataAccessException dlqError) {
                log.error("A spool-bejegyzés DLQ-ba mozgatása sem sikerült", dlqError);
                return;
            }
```

```
                retrySpool.remove(spooled);
            }
        }
    }

    private void moveSpooledToDeadLetter(IngestRetrySpool.SpooledMessage spooled, Exception
    ↪  cause) {
        try {
            repository.insertDeadLetter(spooled.topic(), spooled.payload(),
                "tartós infrastruktúrahiba " + MAX_SPOOL_ATTEMPTS + " kísérlet után");
            retrySpool.remove(spooled);
            log.error("A spool-bejegyzés a DLQ-ba került {} kísérlet után", MAX_SPOOL_ATTEMPTS,
↪   cause);
        } catch (DataAccessException e) {
            log.error("A DLQ sem írható; a spool-bejegyzés megmarad", e);
        }
    }

    @PreDestroy
    public void stop() throws MqttException {
        if (client != null) {
            try {
                if (client.isConnected()) {
                    client.disconnect();
                }
            } finally {
                client.close();
            }
        }
    }
}
```

A munkamenet-stratégia lényege: `cleanStart=false` és `sessionExpiryInterval=3600` esetén a broker a kliens átmeneti kiesése alatt is fenntartja a feliratkozást és a QoS 1-es várakozó üzeneteket; a `connectComplete` callback minden újracsatlakozás után újra feliratkozik, így a feliratkozás nem csak az első `connect` után jön létre. A `clientId`-nak stabilnak kell lennie, különben a broker nem találja a korábbi munkamenetet. A perzisztens broker beállítás (a fenti `persistence true`) nélkül a broker újraindítása elveszítheti a munkamenetet és a várakozó üzeneteket.

A hibakezelés szétválasztja az adat- és az infrastruktúrahibákat. Az érvénytelen JSON, a hiányzó kötelező mező, a topic–payload eltérés és a tartalmi ütközés adathiba: azonnal a `dead_letter` táblába kerül, és nem blokkolja a csatornát. Az adatbázis-kivétel (`DataAccessException`) infrastruktúrahiba: az üzenet a helyi spoolba kerül, és a háttérben öt másodpercenként újrapróbáljuk. A kísérletszám a spool-bejegyzésben tárolódik; 60 sikertelen kísérlet után a bejegyzés a DLQ-ba kerül, hogy az üzemeltető kézzel visszajátszhassa. A rövid adatbázis-leállás így nem veszít adatot, a hosszú leállás pedig nem okoz végtelen újrapróbálkozást.

4. Retry spool. A `src/main/java/com/terrasense/backend/IngestRetrySpool.java` egy soronkénti JSON-fájlt ír, amely a topicot, a nyers payloadot és a kísérletszámot tartalmazza:

```
package com.terrasense.backend;

import com.fasterxml.jackson.databind.ObjectMapper;
import java.io.IOException;
import java.nio.charset.StandardCharsets;
import java.nio.file.Files;
import java.nio.file.Path;
import java.nio.file.StandardCopyOption;
import java.nio.file.StandardOpenOption;
import java.util.ArrayList;
```

```
import java.util.List;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.stereotype.Component;

@Component
public class IngestRetrySpool {

    public record SpooledMessage(String topic, String payload, int attempts) {
        public SpooledMessage nextAttempt() {
            return new SpooledMessage(topic, payload, attempts + 1);
        }
    }

    private final Path path;
    private final ObjectMapper objectMapper;

    public IngestRetrySpool(
            ObjectMapper objectMapper,
            @Value("${ingest.spool-path:spool/ingest-retry.jsonl}") String spoolPath) {
        this.objectMapper = objectMapper;
        this.path = Path.of(spoolPath);
    }

    public synchronized void append(String topic, String payload) {
        try {
            Files.createDirectories(path.toAbsolutePath().getParent());
            String line = objectMapper.writeValueAsString(
                new SpooledMessage(topic, payload, 0)) + System.lineSeparator();
            Files.writeString(path, line, StandardCharsets.UTF_8,
                StandardOpenOption.CREATE, StandardOpenOption.APPEND);
        } catch (IOException e) {
            throw new IllegalStateException("A retry-spool nem írható: " + path, e);
        }
    }

    public synchronized List<SpooledMessage> readAll() {
        if (!Files.exists(path)) {
            return List.of();
        }
        try {
            List<SpooledMessage> result = new ArrayList<>();
            for (String line : Files.readAllLines(path, StandardCharsets.UTF_8)) {
                if (!line.isBlank()) {
                    result.add(objectMapper.readValue(line, SpooledMessage.class));
                }
            }
            return result;
        } catch (IOException e) {
            throw new IllegalStateException("A retry-spool nem olvasható: " + path, e);
        }
    }

    public synchronized void replace(SpooledMessage oldMessage, SpooledMessage newMessage) {
        rewrite(line -> line.equals(oldMessage) ? newMessage : null);
    }

    public synchronized void remove(SpooledMessage message) {
        rewrite(line -> line.equals(message) ? null : line);
    }

    private void rewrite(java.util.function.UnaryOperator<SpooledMessage> mapping) {
        if (!Files.exists(path)) {
            return;
```

```
        }
        try {
            List<String> remaining = new ArrayList<>();
            for (String line : Files.readAllLines(path, StandardCharsets.UTF_8)) {
                if (line.isBlank()) {
                    continue;
                }
                SpooledMessage current = objectMapper.readValue(line, SpooledMessage.class);
                SpooledMessage mapped = mapping.apply(current);
                if (mapped != null) {
                    remaining.add(objectMapper.writeValueAsString(mapped));
                }
            }
            Path temporary = path.resolveSibling(path.getFileName() + ".tmp");
            Files.write(temporary, remaining, StandardCharsets.UTF_8);
            Files.move(temporary, path,
                StandardCopyOption.REPLACE_EXISTING, StandardCopyOption.ATOMIC_MOVE);
        } catch (IOException e) {
            throw new IllegalStateException("A retry-spool nem írható: " + path, e);
        }
    }
}
```

A spool írása és törlése egy ideiglenes fájlon keresztül, atomikus `Files.move` művelettel történik, így áramszünet esetén nem marad félig írt sor. A spool csak az átmeneti hibákat őrzi; az adathibák továbbra is a DLQ-ba mennek. A spool fájlját a Compose-ban a backend konténer `/app/spool` kötetére érdemes tenni, különben a konténer újraépítésekor elveszhet; a laborban a `INGEST_SPOOL_PATH` erre a kötetre mutat.

5. Repository az idempotens beszúrással. A `src/main/java/com/terrasense/backend/TelemetryRepos`

```
package com.terrasense.backend;

import com.fasterxml.jackson.core.JsonProcessingException;
import com.fasterxml.jackson.databind.ObjectMapper;
import java.util.LinkedHashMap;
import java.util.Map;
import org.springframework.jdbc.core.JdbcTemplate;
import org.springframework.stereotype.Repository;
import org.springframework.transaction.annotation.Transactional;

@Repository
public class TelemetryRepository {

    public enum IngestOutcome {
        INSERTED, DUPLICATE, CONTENT_MISMATCH
    }

    private final JdbcTemplate jdbc;
    private final ObjectMapper objectMapper;

    public TelemetryRepository(JdbcTemplate jdbc, ObjectMapper objectMapper) {
        this.jdbc = jdbc;
        this.objectMapper = objectMapper;
    }

    @Transactional
    public IngestOutcome insertIfAbsent(String site, TelemetryMessage message, String topic) {
        String canonicalJson = canonicalJson(message);
        int inserted = jdbc.update("""
            INSERT INTO telemetry
                (site, node_id, boot_id, seq, received_at, ts, schema_version, payload)
            VALUES (?, ?, ?, ?, NOW(), ?, ?, ?::jsonb)
```

```
            ON CONFLICT (site, node_id, boot_id, seq) DO NOTHING
            """,
            site,
            message.node_id(),
            message.effectiveBootId(),
            message.seq(),
            message.ts(),
            message.effectiveSchemaVersion(),
            canonicalJson);

        if (inserted == 0) {
            Boolean same = jdbc.queryForObject("""
                SELECT (payload = ?::jsonb AND ts = ? AND schema_version = ?)
                FROM telemetry
                WHERE site = ? AND node_id = ? AND boot_id = ? AND seq = ?
                """,
                Boolean.class,
                canonicalJson,
                message.ts(),
                message.effectiveSchemaVersion(),
                site,
                message.node_id(),
                message.effectiveBootId(),
                message.seq());
            if (Boolean.TRUE.equals(same)) {
                return IngestOutcome.DUPLICATE;
            }
            insertDeadLetter(topic, canonicalJson,
                "eltérő tartalom ugyanarra az eseménykulcsra");
            return IngestOutcome.CONTENT_MISMATCH;
        }
        return IngestOutcome.INSERTED;
    }

    public void insertDeadLetter(String topic, String payload, String reason) {
        jdbc.update("""
            INSERT INTO dead_letter (topic, payload, reason, received_at)
            VALUES (?, ?, ?, NOW())
            """, topic, payload, reason);
    }

    private String canonicalJson(TelemetryMessage message) {
        Map<String, Object> canonical = new LinkedHashMap<>();
        canonical.put("schema_version", message.effectiveSchemaVersion());
        canonical.put("node_id", message.node_id());
        canonical.put("seq", message.seq());
        canonical.put("ts", message.ts());
        canonical.put("boot_id", message.effectiveBootId());
        canonical.put("measurements", message.normalizedMeasurements());
        try {
            return objectMapper.writeValueAsString(canonical);
        } catch (JsonProcessingException e) {
            throw new IllegalStateException("A kanonikus payload nem szerializálható", e);
        }
    }
}
```

Az `ON CONFLICT ... DO NOTHING` nem módosítja a már meglévő sor `received_at`, `ts`, `schema_version` vagy `payload` értékét: az első beérkezés változatlan marad. A tartalmi egyezést a JSONB `=` operátorával ellenőrizzük, amely a JSON-objektumoknál a szemantikus egyezést vizsgálja, nem a szöveges formázást. Ha a kulcs ütközik, de a tartalom eltér, az esemény nem írja felül az előzményt; a repository ugyanabban a tranzakcióban a `dead_letter` táblába menti

a kanonikus payloadot. A `@Transactional` gondoskodik arról, hogy az esemény beszúrása és az esetleges DLQ-bejegyzés együtt sikerüljön vagy együtt hiúsuljon meg.

A `schema.sql` a `src/main/resources/` alatt hozza létre a fogadótáblát és a DLQ-t:

```
CREATE TABLE IF NOT EXISTS telemetry (
    site           TEXT        NOT NULL,
    node_id        TEXT        NOT NULL,
    boot_id        TEXT        NOT NULL DEFAULT 'boot-0',
    seq            BIGINT      NOT NULL,
    received_at    TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    ts             BIGINT      NOT NULL,
    schema_version INT         NOT NULL,
    payload        JSONB       NOT NULL,
    PRIMARY KEY (site, node_id, boot_id, seq)
);

CREATE TABLE IF NOT EXISTS dead_letter (
    id          BIGSERIAL   PRIMARY KEY,
    topic       TEXT        NOT NULL,
    payload     TEXT        NOT NULL,
    reason      TEXT,
    received_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
```

A `telemetry` tábla ebben az alfejezetben még a nyers fogadóeseményeket tárolja. A Idősoros adatbázisok és adatmodellezés alfejezet verziózott migrációval alakítja át hosszú idősoros táblává, és a nyers eseménytáblát `telemetry_ingest` néven megtartja az idempotencia és az audit számára.

Az `application.yaml` a környezeti változókat használja, hogy ugyanaz az alkalmazáskód hostról és Compose-ból is működjön:

```
server:
  port: 8080

spring:
  application:
    name: ts-backend
  datasource:
    url: ${SPRING_DATASOURCE_URL:jdbc:postgresql://localhost:5432/terrasense}
    username: ${SPRING_DATASOURCE_USERNAME:terra}
    password: ${SPRING_DATASOURCE_PASSWORD:lab-password}
    driver-class-name: org.postgresql.Driver
  sql:
    init:
      mode: always
      schema-locations: classpath:schema.sql

ingest:
  broker-uri: ${MQTT_BROKER_URI:tcp://localhost:1883}
  client-id: ${MQTT_CLIENT_ID:ts-backend-ingest}
  topic-filter: ${MQTT_TOPIC_FILTER:terrasense/+/+/telemetry}
  spool-path: ${INGEST_SPOOL_PATH:spool/ingest-retry.jsonl}
```

Hostról futtatva a `localhost` alapértékek érvényesek, mert a Compose publikálja az 5432 és 1883 portokat; a `ts-backend` konténerben a fenti Compose-környezeti változók felülírják őket a `lab-timescaledb`, illetve `lab-broker` szolgáltatásnévre. A `SPRING_DATASOURCE_*` változók a Spring Boot relaxed bindingja miatt közvetlenül a `spring.datasource` property-ket állítják be.

6. Ütemezés engedélyezése. A `src/main/java/com/terrasense/backend/SchedulingConfig.java` kapcsolja be a `@Scheduled` feldolgozást:

```
package com.terrasense.backend;

import org.springframework.context.annotation.Configuration;
import org.springframework.scheduling.annotation.EnableScheduling;

@Configuration
@EnableScheduling
public class SchedulingConfig {}
```

7. Build és futtatás.

```
cd ~/iot-labor/i044/ts-backend
mvn clean verify
mvn spring-boot:run
```

Hostról futtatva a `lab-timescaledb` és a `lab-broker` szolgáltatásnak futnia kell. A konténeres változat:

```
docker compose -f ~/iot-labor/stack/compose.yaml up -d --build ts-backend
```

Egy másik terminálban kanonikus, 1-es verziójú tesztüzenet küldése QoS 1-gyel:

```
mosquitto_pub -h localhost -t terrasense/gh1/n001/telemetry -q 1 -m
↪  '{"schema_version":1,"node_id":"n001","seq":42,"ts":1710000000000,"boot_id":"b01","measurements":{"temperatu
```

Várt kimenet a backend logjában (a naplózási szinttől függően a Spring Boot indítási sorai is megjelennek):

```
Feliratkozva: terrasense/+/+/telemetry
MQTT kapcsolat kész: reconnect=false, broker=tcp://localhost:1883
Ingest OK: site=gh1, node=n001, seq=42
```

Duplikáció-teszt. Küldjük el ugyanezt a parancsot még egyszer (a `seq` ugyanaz). A táblában továbbra is egy sor van, és a `received_at` nem változik:

```
docker compose -f ~/iot-labor/stack/compose.yaml exec lab-timescaledb \
  psql -U terra -d terrasense \
  -c "SELECT count(*), min(received_at) FROM telemetry WHERE site='gh1' AND node_id='n001' AND
   ↪  seq=42;"
```

Várt kimenet:

```
 count |          min
-------+-----------------------
     1 | 2026-10-31 10:00:00+00
```

A pontos időbélyeg a futtatás idejétől függ; a lényeg, hogy a második üzenet nem írja át az első `received_at` értékét.

Tartalmi ütközés tesztje. Ugyanarra a `(site, node_id, boot_id, seq)` kulcsra küldjünk eltérő értéket:

```
mosquitto_pub -h localhost -t terrasense/gh1/n001/telemetry -q 1 -m
↪  '{"schema_version":1,"node_id":"n001","seq":42,"ts":1710000000000,"boot_id":"b01","measurements":{"temperatu
```

A `telemetry` tábla egyetlen sora változatlan marad, a `dead_letter` táblában pedig megjelenik az ütközés:

```
docker compose -f ~/iot-labor/stack/compose.yaml exec lab-timescaledb \
  psql -U terra -d terrasense \
  -c "SELECT topic, reason FROM dead_letter ORDER BY id DESC LIMIT 1;"
```

Várt kimenet:

```
             topic              |                       reason
--------------------------------+-----------------------------------------------------
 terrasense/gh1/n001/telemetry  | eltérő tartalom ugyanarra az eseménykulcsra
```

Legacy, 0-es verziójú üzenet tesztje. A Backend és gateway közötti adatszerződés alfejezet lapos formátuma adapterrel továbbra is beérkezik:

```
mosquitto_pub -h localhost -t terrasense/gh1/n001/telemetry -q 1 -m
↪  '{"node_id":"n001","seq":44,"ts":1710000300000,"temperature_c":22.8,"humidity_rh":61.2,"soil_moisture_pct":3
```

A backend `Ingest OK: site=gh1, node=n001, seq=44` sort naplóz, a `telemetry` táblában pedig a `schema_version` értéke 0, a payload `measurements` objektuma pedig a lapos mezőkből képződik.

Topic–payload eltérés tesztje. Küldjünk `n002` payloadot az `n001` topicra:

```
mosquitto_pub -h localhost -t terrasense/gh1/n001/telemetry -q 1 -m
↪  '{"schema_version":1,"node_id":"n002","seq":45,"ts":1710000600000,"measurements":{"temperature":{"value":21
```

Az üzenet nem íródik az `n001` eszközhöz; a DLQ-ban a `a topic node azonosítója eltér a payloadétól` indok jelenik meg.

Érvénytelen üzenet tesztje (hiányzó `node_id`):

```
mosquitto_pub -h localhost -t terrasense/gh1/n002/telemetry -q 1 -m
↪  '{"schema_version":1,"seq":43}'
```

A backend logban `Érvénytelen telemetria`, és a `dead_letter` táblában egy sor jelenik meg.

Adatbázis-leállás tesztje. Állítsuk le az adatbázist, küldjünk egy üzenetet, majd indítsuk újra:

```
docker compose -f ~/iot-labor/stack/compose.yaml stop lab-timescaledb
mosquitto_pub -h localhost -t terrasense/gh1/n001/telemetry -q 1 -m
↪  '{"schema_version":1,"node_id":"n001","seq":46,"ts":1710000900000,"measurements":{"temperature":{"value":23
docker compose -f ~/iot-labor/stack/compose.yaml start lab-timescaledb
```

A leállás alatt a backend naplójában `Átmeneti hiba; az üzenet a retry-spoolba került` jelenik meg, és a `spool/ingest-retry.jsonl` fájl tartalmazza a `seq=46` üzenetet. Az adatbázis visszatérése után legkésőbb öt másodperccel a spool kiürül, és a `telemetry` táblában megjelenik a `seq=46` sor. Ez a viselkedés a labor elvárt működése; a leállás és az újraindítás tényleges időzítését a környezet határozza meg.

15.1.5 Ellenőrzés és hibakeresés

A labor működését a fenti duplikáció-, tartalmi ütközés-, legacy-, topic-eltérés-, érvénytelenüzenet- és adatbázis-leállás-teszttel ellenőrizzük. Gyakori hibák:

Tünet	Lehetséges ok	Javítás
A backend nem csatlakozik a brokerhez	A Mosquitto nem fut a stackben, vagy a szolgáltatás neve nem `lab-broker`	`docker compose -f ~/iot-labor/stack/compose.yaml up -d lab-broker`; ellenőrizd a `docker ps` kimenetét
A `terrasense/+/+/telemetry` topicra nem érkezik üzenet	A publisher más topicra küld, vagy a backend topic-mintája hibás	Ellenőrizd a topic pontos nevét; a wildcard `+` pontosan egy szintet helyettesít
A duplikátumok mégis bekerülnek a `telemetry` táblába	A `PRIMARY KEY (site, node_id, boot_id, seq)` hiányzik, vagy a `schema.sql` nem fut le	Ellenőrizd a tábla definícióját: `\d telemetry`
A `dead_letter` tábla üres érvénytelen üzenet után is	Az alkalmazás nem hívja az `insertDeadLetter`-t, vagy a tranzakció hibát dob	Nézd a backend logját; a `validate` metódus adott-e vissza hibaszöveget
A Paho kliens `NoClassDefFoundError`	A Maven függőség hiányzik vagy rossz verzió	Ellenőrizd a `pom.xml`-t és a `mvn clean verify` kimenetét
A `@Slf4j` fordítási hiba	A Lombok annotation processor konfigurációja hiányzik	Ellenőrizd a `maven-compiler-plugin` `annotationProcessorPaths` bejegyzését
A backend a konténerből nem éri el az adatbázist	A `localhost` a konténer saját hálózatát jelenti	Használd a `SPRING_DATASOURCE_URL=jdbc:postgresql://timescaledb:5432/terrasense` környezeti változót
A spool nem ürül ki az adatbázis visszatérése után	Az ütemezés nincs engedélyezve, vagy a spool útvonala nem írható	Ellenőrizd a `SchedulingConfig` `@EnableScheduling` annotációját és az `INGEST_SPOOL_PATH` kötetét
Az újracsatlakozás után nem jönnek üzenetek	A `clientId` megváltozott, vagy a `cleanStart=true` eldobta a munkamenetet	Használj stabil `clientId`-t, `cleanStart=false` és `sessionExpiryInterval` beállítást

15.1.6 Tipikus hibák és félreértések

1. **QoS 1 idempotencia nélkül.** Sokan azt hiszik, hogy a QoS 1 „legalább egyszer” kézbesítése csak elméleti duplikációval jár. A PUBACK elvesztése valós hálózati hiba, ezért a backendnek mindenképp idempotensnek kell lennie; az egyedi kulcs nélküli `INSERT` felesleges sorokat szül.
2. **Helyi idő UTC helyett.** A `received_at`-et az adatbázis `NOW()`-val állítja, ami a szerver időzónájától függhet. A TerraSense konvenció szerint mindig UTC-t használunk; a `TIMESTAMPTZ` típus és a Postgres UTC-re normalizálása ezt biztosítja, de az alkalmazásszintű időbélyegzőknél ügyelni kell az `Instant` használatára.
3. **ISR-hez hasonló blokkoló hívás a callbackben.** Az MQTT callback nem megszakítás, de ha hosszú adatbázis- vagy hálózati műveletet végzünk benne, a broker felé késleltetjük a nyugtázást, és nő a memóriaigény. Nagy terhelésnél a callback csak a gyors validálást és queue-ba helyezést végezze.

4. **Shared subscription hiánya több példány esetén.** MQTT 3.1.1-ben nincs shared subscription, így két backend példány ugyanazt az üzenetet mindketten megkapja. Az idempotens adatmodell véd a duplikált soroktól, de a külső mellékhatások (például értesítések) akkor is kétszer futhatnak le, hacsak nem használunk egyedi feldolgozási azonosítót.
5. **A DLQ és a retry spool összekeverése.** A DLQ a véglegesen feldolgozhatatlan vagy tartalmilag ütköző üzeneteké; a spool az átmeneti infrastruktúrahiba miatt várakozó üzeneteké. Ha az adatbázis-leállás alatt minden üzenetet DLQ-ba tennénk, a DLQ maga is elérhetetlen lenne, és az üzenetek elvesznének. A spool ezért helyi fájl, nem adatbázistábla.

15.1.7 Feladatok

1. (Fogalmi) Miért nem elegendő a QoS 2 használata az idempotens feldolgozás kiváltására a teljes rendszerben?
2. (Tervezési) Tegyük fel, hogy a hálózaton 0,5% annak a valószínűsége, hogy egy PUBACK elveszik, és az eszköz 10 000 üzenetet küld naponta. Mennyi a várható duplikált üzenetek száma naponta, ha az eszköz minden elveszett PUBACK esetén pontosan egyszer újraküldi az üzenetet?
3. (Implementációs) Egészítsd ki a `TelemetryIngestService` `validate` metódusát úgy, hogy a kanonikus `measurements` objektumban csak számértékeket fogadjon el, a `temperature` értéke -40 és 80 °C közé essen, és a `soil_moisture` értéke 0 és 100% közé essen.
4. (Hibakeresési) A `telemetry` táblában ugyanarra a `(site, node_id, boot_id, seq)` kulcsra két sor van, de a `PRIMARY KEY` definíció helyes. Hol lehet a hiba a feldolgozási láncban?
5. (Tervezési) Miért kerül a tartalmilag eltérő, de azonos kulcsú esemény a DLQ-ba a felülírás helyett? Milyen üzleti következménye lenne annak, ha az utolsó beérkezés nyerne?

15.1.8 Megoldások

1. A QoS 2 a broker és a kliens közötti hálózati duplikációt szünteti meg, de nem véd a feldolgozói szintű ismétléstől (például ha a backend egy tranzakciót visszagörget, majd újra futtatja), és nem oldja meg a különböző kiadók által küldött ugyanolyan adatokat sem. Az idempotencia a feldolgozó oldal saját felelőssége.
2. naponta 10 000 üzenet, 0,005 veszteség $\rightarrow$ várhatóan $10\,000 \times 0{,}005 = 50$ PUBACK-vész, tehát várhatóan 50 duplikált üzenet. Az idempotens ingest ezeket mind egyetlen sorba írja.
3. Példakód a validáció kiegészítésére:

```
private String validateMeasurements(Map<String, Measurement> measurements) {
    for (Map.Entry<String, Measurement> entry : measurements.entrySet()) {
        Measurement value = entry.getValue();
        if (value == null || value.value() == null) {
            return "hiányzó mérési érték: " + entry.getKey();
        }
        double number = value.value();
        if (entry.getKey().equals("temperature") && (number < -40.0 || number > 80.0)) {
            return "tartományon kívüli temperature: " + number;
        }
        if (entry.getKey().equals("soil_moisture") && (number < 0.0 || number > 100.0)) {
            return "tartományon kívüli soil_moisture: " + number;
        }
    }
    return null;
}
```

A metódust a `validate` a `measurements` ellenőrzése után hívja meg, és az eredményt adja vissza.

4. A hiba valószínűleg a kulcs valamelyik összetevőjében van: a topicból származó `site` eltér a payload `site` mezőjétől, a `boot_id` az egyik üzenetben hiányzik (és a `boot-0` alapértékre esik), vagy a `seq` a JSON-ben karakterláncként érkezik, és a deszerializáció másképp értelmezi. Ellenőrizd a tényleges értékeket:

```
SELECT site, node_id, boot_id, seq, count(*)
FROM telemetry
GROUP BY 1, 2, 3, 4
HAVING count(*) > 1;
```

A tartalmi eltérés önmagában nem hoz létre második sort: az `ON CONFLICT DO NOTHING` a meglévő sort hagyja, és a DLQ-ba ír.

5. Az esemény első beérkezése auditálható és visszajátszható kell maradjon; a csendes felülírás elrejtené, hogy ugyanarra a kulcsra két különböző tartalom érkezett. Ez adatminőségi és biztonsági kockázat: például egy hibás vagy rosszindulatú eszköz felülírhatná a korábbi mérést. A DLQ-ba helyezett ütközés emberi döntést tesz lehetővé: eldönthető, melyik tartalom a hiteles, és szükség esetén új eseménykulccsal korrigálható.

15.1.9 Rövid összefoglaló

A laborban elkészült a ts-backend MQTT-ingest komponense, amely Paho MQTT 5 Java klienssel, tartós munkamenettel és újrafeliratkozással figyeli a `terrasense/+/+/telemetry` topicot. A szolgáltatás a kanonikus `measurements` objektumot és a 0-es verziójú lapos formátumot is értelmezi, ellenőrzi a topic és a payload site/node azonosságát, és a `(site, node_id, boot_id, seq)` eseménykulccsal `ON CONFLICT DO NOTHING` művelettel idempotensen dolgozza fel az üzeneteket. Az azonos duplikátum nem változtat az első soron, a tartalmi ütközés a `dead_letter` táblába kerül. Az átmeneti adatbázis-hibák a helyi retry spoolba jutnak, ahonnan öt másodpercenként, legfeljebb 60 kísérletig újrapróbálhatók. **Továbbvitt eredmény.** A `TelemetryRepository.insertIfAbsent` metódus és a `telemetry` tábla sémája az Idősoros adatbázisok és adatmodellezés alfejezetben verziózott migrációval hypertable-lé alakul, a Eseményvezérelt feldolgozás: Kafka és a kézbesítési szemantika alfejezetben a Kafka-producer tranzakciókban, a Záróprojekt: a teljes TerraSense rendszer összeállítása és átvétele alfejezetben pedig a végpontok közötti átvételi tesztekben használt idempotens ingest alapjaként szolgál. A labor kimenete a `~/iot-labor/i044/ts-backend` projekt, benne a `TelemetryIngestService.java`, `TelemetryMessage.java`, `Measurement.java`, `TelemetryRepository.java`, `IngestRetrySpool.java`, `SchedulingConfig.java`, `schema.sql` és a módosított `application.yaml`.

15.1.10 Források és ellenőrzési hiányok

- MQTT 5 shared subscription szintaxis és szemantika: OASIS MQTT 5.0 szabvány, 4.8. fejezet. Ellenőrzendő a pontos fejezetszám.
- MQTT 5 munkamenet-lejárat (`Session Expiry Interval`) és `Clean Start`: OASIS MQTT 5.0 szabvány, 3.1.2.11 és 4.1. fejezet. A labor 3600 másodperces lejáratot használ; a termelési értéket az üzemeltetési követelményekhez kell igazítani.
- Eclipse Paho MQTT 5 Java kliens használata: https://eclipse.dev/paho/ (a példa az `org.eclipse.paho.mqttv5.client` 1.2.5 artifactra épül; a laborban a Maven-fordítás és a brokeres végigfuttatás a lektorálási környezetben nem történt meg).

- Spring Boot `spring.sql.init`, `JdbcTemplate` és környezeti változós konfiguráció: https://docs.spring.io/spring-boot/ (a példák a Spring Boot 3.3.4 dokumentációja alapján készültek; a teljes `mvn clean verify` futtatása a lektorálási környezetben nem történt meg).
- PostgreSQL UPSERT (`INSERT ... ON CONFLICT`) és JSONB egyenlőség: https://www.postgresql.org/docs/ (stabil funkció, ellenőrizendő a pontos hivatkozás).
- A 14.3-ban létrehozott `pom.xml` szülőverziója a könyv rögzített verzióprofja szerint 3.3.4; ha a helyi projektben 3.4.1 szerepel, a labor első lépésében arra a verzióra kell állítani.

15.2 Idősoros adatbázisok és adatmodellezés

15.2.1 Motiváció

A ts-backend az Ingest: MQTT-ből a backendbe, idempotensen alfejezetben már beérkező üzenetenként feldolgozza a TerraSense telemetriát, de ez önmagában kevés: a méréseket úgy kell tárolni, hogy a későbbi lekérdezések ne lassuljanak le a több millió soros tábla növekedésével. Egy hagyományos PostgreSQL-tábla eleinte működik, de az idő szerinti szűrés, az óránkénti átlag, a percenként beérkező ezres írási sebesség és a hosszú megőrzési idő hamar szétfeszíti a naiv megoldást. Az üvegházakban elhelyezett szenzorok folyamatosan küldenek adatot; ha a rendszer nem skálázódik, a dashboard lassú lesz, a riasztások késnek, és a tárolási költség megnő.

Ez az alfejezet azt mutatja be, hogyan tervezzük meg a TerraSense mérések idősoros tábláit, hogy a TimescaleDB hatékonyan kezelje az írásintenzív terhelést, az időablakos lekérdezéseket és az eszközönkénti utolsó értékeket. A 15.1-ben létrehozott fogadótáblát verziózott migrációval alakítjuk át: a nyers eseményeket `telemetry_ingest` néven megtartjuk az idempotencia és az audit számára, a méréseket pedig hosszú formátumú hypertable-be írjuk.

15.2.2 Fogalmak és működés

Definíció. Az idősoros adat olyan rekord, amelyben minden sorhoz egy időbélyeg tartozik, és a lekérdezések dominánsan időintervallumokra, sorrendre, csúcsértékre vagy aggregált értékre vonatkoznak. Az IoT-mérések tipikus idősoros adatok: a hőmérsékletet, páratartalmat és talajnedvességet időpontonként rögzítjük, és a kérdés gyakran az, hogy „mi volt az átlaghőmérséklet az elmúlt 24 órában” vagy „mikor lépte át utoljára a 30 °C-ot”.

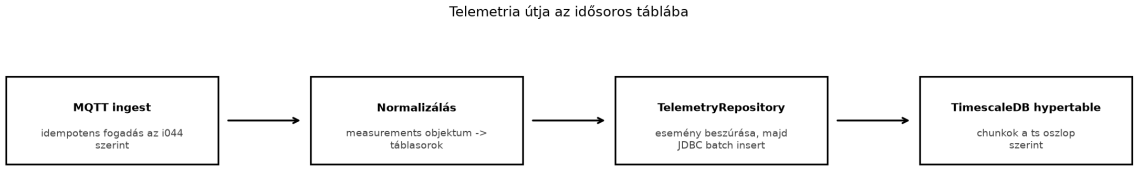
Definíció. A TimescaleDB a PostgreSQL egy kiterjesztése, amely egy hagyományos táblából ún. hypertable-t hoz létre. A hypertable-t a TimescaleDB automatikusan kisebb, idő szerint particionált részekre (chunk) bontja, miközben a felhasználó továbbra is szabványos SQL-lekérdezéseket ír. A particionálás gyorsítja az időablakos szűrést, a tömörítést és a későbbi adatmegőrzési műveleteket.

A TimescaleDB-ben a hypertable-t jellemzően az eseményidő oszlopra hozzuk létre. A `ts` oszlop tehát nem csak egy mező, hanem a tárolás elsődleges szerveződési tengelye. A beérkező üzenetek az append-only jelleget követik: ritkán frissítünk, gyakran szúrunk be, a múltbeli adatot inkább tömörítjük vagy ritkítjuk.

A TerraSense telemetria a Adatformátumok: JSON, CBOR, Protocol Buffers, SenML alfejezetben véglegesített séma szerint egy `measurements` objektumot tartalmaz, amelynek kulcsai a méréstípusok. Egyetlen MQTT-üzenet tehát több mérést is hozhat:

```
{
  "schema_version": 1,
  "node_id": "n001",
  "seq": 42,
  "ts": 1735689600000,
  "boot_id": "boot-1",
  "measurements": {
    "temperature": {"value": 22.5, "unit": "Cel", "valid": true},
    "humidity": {"value": 60.2, "unit": "%RH", "valid": true},
    "soil_moisture": {"value": 35.0, "unit": "%", "valid": true}
  }
}
```

Ebből a normalizálás három táblasort készít. A `ts` értéke epoch ms UTC, amit az adatbázisban `timestamptz` típusra alakítunk. Emellett tároljuk a `server_ts` feldolgozási időt is, hogy később vizsgálható legyen a hálózati késleltetés vagy a gateway offline pufferelésének hatása.



15.2. ábra: Telemetria útja az idősoros táblába - Az ingest réteg a kanonikus JSON-t normalizálja, a repository előbb az eseménytáblába ír, majd kötegelve a hypertable-be.

Az ábrán látható folyamatban a `TelemetryRepository` először a `telemetry_ingest` eseménytáblába próbálja beszúrni az üzenetet az `(site, node_id, boot_id, seq)` kulccsal. Ha ez az esemény még nem létezett, akkor írja be a méréseket a `telemetry` hypertable-be, kötegelt beszúrással: nem soronként, hanem néhány száz elemű batch-ekben, így csökken a hálózati és tranzakciós overhead. A két művelet ugyanabban a tranzakcióban fut.

15.2.3 Döntési szempontok

Az idősoros tároló kiválasztásakor a TerraSense esetében a TimescaleDB mellett az InfluxDB a leggyakrabban felmerülő alternatíva.

Szempont	TimescaleDB	InfluxDB
Adatmodell	PostgreSQL-táblák, SQL	saját adatmodell, bucket/measurement/tag/field
Lekérdezési nyelv	SQL, PostgreSQL-ökoszisztéma	InfluxQL vagy Flux (verziófüggő)
Tranzakciók	teljes ACID a PostgreSQL miatt	korlátozottabb
Üzemeltetés	meglévő PostgreSQL-ismeret előny	külön motor, saját működtetés
Skálázás	vertikális + kiterjesztések, chunkok	elosztott klaszterezés lehetősége
Integráció	Spring JDBC/JPA egyszerű	saját kliens vagy API

A TerraSense-ben a TimescaleDB-t választjuk, mert a ts-backend Java/Spring környezetben amúgy is PostgreSQL felé integrál, a csapat ismeri az SQL-t, és a TimescaleDB a mérési adatok mellett az eszköznyilvántartást is ugyanabban az adatbázisban tarthatja. Az InfluxDB erős versenyző, ha nagy elosztott klaszterre vagy speciális idősoros funkciókra van szükség, de jelen üvegházas méretben a bevált PostgreSQL-alap a fontosabb. A pontos verzió- és funkcióösszevetést a telepített TimescaleDB- és InfluxDB-verzió dokumentációjában ellenőrizni kell.

Az adatmodell tervezésénél két fő forma van:

- **Széles tábla**: minden méréstípus külön oszlop (`temperature`, `humidity`, `soil_moisture`). Előnye, hogy egy sor egy időpontot ír le, és a lekérdezés egyszerű. Hátránya, hogy új szenzortípus esetén sémamódosítás kell, és a hiányzó mérések NULL-okat szülnek.
- **Hosszú tábla**: egy `measurement_type` és egy `value` oszlop van, minden mérés egy sor. Előnye a rugalmasság és az idősoros indexelés hatékonysága. Hátránya, hogy egy eszköz időpillanatnyi állapotának összeállítása több sor egyesítését igényli.

A TerraSense a hosszú táblát választja, mert a telemetria JSON kanonikus `measurements` objektuma méréstípusonként egy-egy bejegyzést tartalmaz, és a jövőben új mérések (például széndioxid-koncentráció) séma-módosítás nélkül illeszthetők be. A `site` és `node_id` címkeként (label), a `measurement_type` és `value` pedig mérésként (metric) viselkedik. Az eseményszintű azonosítást nem a hosszú tábla egyedi kulcsa, hanem a külön `telemetry_ingest` eseménytábla végzi: így az ugyanazon esemény eltérő `ts`-sel történő visszajátszása sem hoz létre új méréssorokat.

15.2.4 Kidolgozott példa: a TerraSense táblaszerkezete

A 15.1-ben a `telemetry` tábla még a nyers, kanonikus JSON-payloadot tárolta `(site, node_id, boot_id, seq)` kulccsal. Ebben az alfejezetben ezt a táblát `telemetry_ingest` néven megtartjuk, és létrehozzuk mellette a hosszú formátumú `telemetry` hypertable-t. A két tábla szerepe:

- `telemetry_ingest`: eseményszintű deduplikáció és audit; egy MQTT-üzenet egy sor, a kanonikus JSON-payloadtal.
- `telemetry`: mérésszintű idősoros adat; egy MQTT-üzenet annyi sor, ahány mérés van benne.

A végleges táblaséma:

```
CREATE EXTENSION IF NOT EXISTS timescaledb;

CREATE TABLE IF NOT EXISTS telemetry_ingest (
    site           text        NOT NULL,
    node_id        text        NOT NULL,
    boot_id        text        NOT NULL DEFAULT 'boot-0',
    seq            bigint      NOT NULL,
    received_at    timestamptz NOT NULL DEFAULT now(),
    ts             bigint      NOT NULL,
    schema_version integer     NOT NULL,
    payload        jsonb       NOT NULL,
    PRIMARY KEY (site, node_id, boot_id, seq)
);

CREATE TABLE IF NOT EXISTS telemetry (
    server_ts        timestamptz NOT NULL DEFAULT now(),
    ts               timestamptz NOT NULL,
    site             text        NOT NULL,
    node_id          text        NOT NULL,
    seq              bigint      NOT NULL,
    boot_id          text        NOT NULL DEFAULT 'boot-0',
```

```
    schema_version   integer     NOT NULL,
    measurement_type text        NOT NULL,
    value            double precision NOT NULL,
    unit             text,
    valid            boolean     NOT NULL DEFAULT true,
    UNIQUE (site, node_id, boot_id, seq, measurement_type, ts)
);

SELECT create_hypertable('telemetry', 'ts',
       if_not_exists => TRUE,
       chunk_time_interval => interval '7 days');
CREATE INDEX IF NOT EXISTS telemetry_site_node_type_ts_idx
    ON telemetry (site, node_id, measurement_type, ts DESC);
```

A `telemetry_ingest` elsődleges kulcsa a `(site, node_id, boot_id, seq)` eseménykulcs. A `telemetry` egyedi kényszere a `(site, node_id, boot_id, seq, measurement_type, ts)` hatos, amely tartalmazza a hypertable partíciós oszlopát (`ts`), így a TimescaleDB elfogadja. A `ts` bevonása a mérési sor kulcsába azért szükséges, mert a TimescaleDB minden egyedi indexben megköveteli a partíciós oszlopot; az eseményszintű deduplikációt viszont nem ez a kulcs, hanem a `telemetry_ingest` végzi. Ez a szétválasztás akkor is véd, ha az esemény ismételt küldésekor a `ts` megváltozik: a `telemetry_ingest` ütközése miatt a repository nem ír új méréssorokat.

Továbbvitt eredmény. Ez a `telemetry` táblaszerkezet, a `(site, node_id, boot_id, seq, measurement_type, ts)` egyedi kulcs, a `telemetry_ingest` eseménytábla és a `hypertable` particionálás a Megőrzés, ritkítás és tárolási költség alapját adja. Ugyanezt a táblát használja a Dashboardok Grafanával a grafikonok mögötti lekérdezésekhez, és a Záróprojekt: a teljes TerraSense rendszer összeállítása és átvétele a végleges adatrétegként.

A beszúrást a Spring `JdbcTemplate` kötegelt API-jával végezzük. Az alábbi repository a 15.1-es DTO-t használja, de már a két táblába ír, egy tranzakcióban:

```
package com.terrasense.backend;

import com.fasterxml.jackson.core.JsonProcessingException;
import com.fasterxml.jackson.databind.ObjectMapper;
import java.sql.Timestamp;
import java.time.Instant;
import java.util.ArrayList;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;
import org.springframework.jdbc.core.JdbcTemplate;
import org.springframework.stereotype.Repository;
import org.springframework.transaction.annotation.Transactional;

@Repository
public class TelemetryRepository {

    public enum IngestOutcome {
        INSERTED, DUPLICATE, CONTENT_MISMATCH
    }

    private final JdbcTemplate jdbc;
    private final ObjectMapper objectMapper;

    public TelemetryRepository(JdbcTemplate jdbc, ObjectMapper objectMapper) {
        this.jdbc = jdbc;
        this.objectMapper = objectMapper;
    }

    @Transactional
```

```
public IngestOutcome insertIfAbsent(String site, TelemetryMessage message, String topic) {
    String canonicalJson = canonicalJson(message);
    int inserted = jdbc.update("""
        INSERT INTO telemetry_ingest
            (site, node_id, boot_id, seq, received_at, ts, schema_version, payload)
        VALUES (?, ?, ?, ?, now(), ?, ?, ?::jsonb)
        ON CONFLICT (site, node_id, boot_id, seq) DO NOTHING
        """,
        site,
        message.node_id(),
        message.effectiveBootId(),
        message.seq(),
        message.ts(),
        message.effectiveSchemaVersion(),
        canonicalJson);

    if (inserted == 0) {
        Boolean same = jdbc.queryForObject("""
            SELECT (payload = ?::jsonb AND ts = ? AND schema_version = ?)
            FROM telemetry_ingest
            WHERE site = ? AND node_id = ? AND boot_id = ? AND seq = ?
            """,
            Boolean.class,
            canonicalJson,
            message.ts(),
            message.effectiveSchemaVersion(),
            site,
            message.node_id(),
            message.effectiveBootId(),
            message.seq());
        if (Boolean.TRUE.equals(same)) {
            return IngestOutcome.DUPLICATE;
        }
        insertDeadLetter(topic, canonicalJson,
            "eltérő tartalom ugyanarra az eseménykulcsra");
        return IngestOutcome.CONTENT_MISMATCH;
    }

    List<TelemetryRow> rows = toRows(site, message);
    jdbc.batchUpdate("""
        INSERT INTO telemetry
            (server_ts, ts, site, node_id, seq, boot_id,
             schema_version, measurement_type, value, unit, valid)
        VALUES (now(), ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
        ON CONFLICT (site, node_id, boot_id, seq, measurement_type, ts)
        DO NOTHING
        """, rows, 500, (ps, row) -> {
            ps.setTimestamp(1, Timestamp.from(Instant.ofEpochMilli(row.tsEpochMs())));
            ps.setString(2, row.site());
            ps.setString(3, row.nodeId());
            ps.setLong(4, row.seq());
            ps.setString(5, row.bootId());
            ps.setInt(6, row.schemaVersion());
            ps.setString(7, row.measurementType());
            ps.setDouble(8, row.value());
            ps.setString(9, row.unit());
            ps.setBoolean(10, row.valid());
        });
    return IngestOutcome.INSERTED;
}

public void insertDeadLetter(String topic, String payload, String reason) {
    jdbc.update("""
```

```
            INSERT INTO dead_letter (topic, payload, reason, received_at)
            VALUES (?, ?, ?, now())
            """, topic, payload, reason);
    }

    private List<TelemetryRow> toRows(String site, TelemetryMessage message) {
        List<TelemetryRow> rows = new ArrayList<>();
        for (Map.Entry<String, Measurement> entry : message.normalizedMeasurements().entrySet())
        ↪ {
            Measurement measurement = entry.getValue();
            rows.add(new TelemetryRow(
                message.ts(),
                site,
                message.node_id(),
                message.seq(),
                message.effectiveBootId(),
                message.effectiveSchemaVersion(),
                entry.getKey(),
                measurement.value(),
                measurement.unit(),
                Boolean.TRUE.equals(measurement.valid())));
        }
        return rows;
    }

    private String canonicalJson(TelemetryMessage message) {
        Map<String, Object> canonical = new LinkedHashMap<>();
        canonical.put("schema_version", message.effectiveSchemaVersion());
        canonical.put("node_id", message.node_id());
        canonical.put("seq", message.seq());
        canonical.put("ts", message.ts());
        canonical.put("boot_id", message.effectiveBootId());
        canonical.put("measurements", message.normalizedMeasurements());
        try {
            return objectMapper.writeValueAsString(canonical);
        } catch (JsonProcessingException e) {
            throw new IllegalStateException("A kanonikus payload nem szerializálható", e);
        }
    }
}
```

A `TelemetryRow` egy külön rekord, amely a normalizált mérési sort írja le:

```
package com.terrasense.backend;

public record TelemetryRow(
    long tsEpochMs,
    String site,
    String nodeId,
    long seq,
    String bootId,
    int schemaVersion,
    String measurementType,
    double value,
    String unit,
    boolean valid
) {}
```

A `batchUpdate` 500 soronként fűz ki egy adagot, és a JDBC-driver általában ezt egyetlen round-tripben küldi. A `TelemetryRow` rekord mezői pontosan a `telemetry` tábla oszlopainak felelnek meg, a `ts` kivételével, amelyet a Java oldal `Instant.ofEpochMilli`-vel alakít `Timestamp`-re.

A fenti konkrét JSON-üzenetből a repository három sort hoz létre, például:

site	node_id	seq	boot_id	ts	measurement_type	value	unit	valid
gh1	n001	42	boot-1	2025-01-01 00:00:00+00	temperature	22.5	Cel	true
gh1	n001	42	boot-1	2025-01-01 00:00:00+00	humidity	60.2	%RH	true
gh1	n001	42	boot-1	2025-01-01 00:00:00+00	soil_moisture	35.0	%	true

Az időbélyeg `1735689600000` ms, ami `2025-01-01T00:00:00Z`-nek felel meg.

15.2.5 Verziózott migráció a 15.1 adatbázisából

A 15.1-ben létrehozott adatbázis frissítéséhez nem elegendő a `schema.sql` kicserélése: a meglévő `telemetry` táblát át kell nevezni, az új hypertable-t létre kell hozni, és a korábbi JSON-payloadokból ki kell nyerni a méréssorokat. A migrációt egy verziózott SQL-fájl tartalmazza, amelyet a backend újraindítása előtt egyszer kell lefuttatni:

```
-- ~/iot-labor/i045/02_normalized_telemetry.sql
BEGIN;

CREATE TABLE IF NOT EXISTS schema_migration (
    version     integer     PRIMARY KEY,
    description text        NOT NULL,
    applied_at  timestamptz NOT NULL DEFAULT now()
);

DO $$
BEGIN
    IF to_regclass('public.telemetry') IS NULL THEN
        RAISE EXCEPTION 'Hiányzik a 15.1 telemetry tábla; előbb futtasd a 15.1 labort.';
    END IF;
    IF to_regclass('public.telemetry_ingest') IS NULL THEN
        ALTER TABLE telemetry RENAME TO telemetry_ingest;
    END IF;
END $$;

CREATE TABLE IF NOT EXISTS telemetry (
    server_ts        timestamptz NOT NULL DEFAULT now(),
    ts               timestamptz NOT NULL,
    site             text        NOT NULL,
    node_id          text        NOT NULL,
    seq              bigint      NOT NULL,
    boot_id          text        NOT NULL DEFAULT 'boot-0',
    schema_version   integer     NOT NULL,
    measurement_type text        NOT NULL,
    value            double precision NOT NULL,
    unit             text,
    valid            boolean     NOT NULL DEFAULT true,
    UNIQUE (site, node_id, boot_id, seq, measurement_type, ts)
);

SELECT create_hypertable('telemetry', 'ts',
```

```
       if_not_exists => TRUE,
       chunk_time_interval => interval '7 days');
CREATE INDEX IF NOT EXISTS telemetry_site_node_type_ts_idx
    ON telemetry (site, node_id, measurement_type, ts DESC);

INSERT INTO telemetry
    (server_ts, ts, site, node_id, seq, boot_id, schema_version,
     measurement_type, value, unit, valid)
SELECT i.received_at,
       to_timestamp(i.ts / 1000.0),
       i.site,
       i.node_id,
       i.seq,
       i.boot_id,
       i.schema_version,
       m.key,
       (m.value ->> 'value')::double precision,
       m.value ->> 'unit',
       coalesce((m.value ->> 'valid')::boolean, true)
FROM telemetry_ingest i
CROSS JOIN LATERAL jsonb_each(i.payload -> 'measurements') AS m(key, value)
WHERE i.payload ? 'measurements'
ON CONFLICT (site, node_id, boot_id, seq, measurement_type, ts) DO NOTHING;

INSERT INTO schema_migration (version, description)
VALUES (2, 'normalized telemetry and event dedup table')
ON CONFLICT (version) DO NOTHING;

COMMIT;
```

A migráció feltételezi, hogy a 15.1-es `telemetry` tábla a kanonikus `payload` oszloppal létezik. A `DO` blokk csak akkor nevezi át a táblát, ha a `telemetry_ingest` még nem létezik; így a migráció újrafuttatása nem nevezi át kétszer a táblát. A backfill a JSONB `jsonb_each` függvényével bontja ki a `measurements` objektumot, és a méréseket a hosszú táblába írja. A `WHERE i.payload ? 'measurements'` feltétel kihagyja azokat a régi sorokat, amelyek nem kanonikus payloadot tartalmaznak; ezeket a migráció után kézzel kell átnézni. A `schema_migration` tábla naplózza, hogy a 2-es verziójú migráció lefutott.

A migráció után a backend `src/main/resources/schema.sql` fájlját is a végleges sémára kell cserélni, különben a `spring.sql.init.mode: always` a következő indításkor újra a régi `telemetry` táblát próbálná létrehozni. A végleges `schema.sql`:

```
CREATE EXTENSION IF NOT EXISTS timescaledb;

CREATE TABLE IF NOT EXISTS telemetry_ingest (
    site           text        NOT NULL,
    node_id        text        NOT NULL,
    boot_id        text        NOT NULL DEFAULT 'boot-0',
    seq            bigint      NOT NULL,
    received_at    timestamptz NOT NULL DEFAULT now(),
    ts             bigint      NOT NULL,
    schema_version integer     NOT NULL,
    payload        jsonb       NOT NULL,
    PRIMARY KEY (site, node_id, boot_id, seq)
);

CREATE TABLE IF NOT EXISTS telemetry (
    server_ts        timestamptz NOT NULL DEFAULT now(),
    ts               timestamptz NOT NULL,
    site             text        NOT NULL,
    node_id          text        NOT NULL,
    seq              bigint      NOT NULL,
```

```
    boot_id          text        NOT NULL DEFAULT 'boot-0',
    schema_version   integer     NOT NULL,
    measurement_type text        NOT NULL,
    value            double precision NOT NULL,
    unit             text,
    valid            boolean     NOT NULL DEFAULT true,
    UNIQUE (site, node_id, boot_id, seq, measurement_type, ts)
);

SELECT create_hypertable('telemetry', 'ts',
       if_not_exists => TRUE,
       chunk_time_interval => interval '7 days');
CREATE INDEX IF NOT EXISTS telemetry_site_node_type_ts_idx
    ON telemetry (site, node_id, measurement_type, ts DESC);

CREATE TABLE IF NOT EXISTS dead_letter (
    id          bigserial   PRIMARY KEY,
    topic       text        NOT NULL,
    payload     text        NOT NULL,
    reason      text,
    received_at timestamptz NOT NULL DEFAULT now()
);
```

A `dead_letter` tábla a 15.1-ből változatlanul megmarad; a fenti `schema.sql` a teljes, friss telepítéshez is használható, de a meglévő 15.1-es adatbázison előbb a migrációt kell lefuttatni. A migráció és a `schema.sql` együtt biztosítja, hogy a 15.1-ből induló adatbázis frissítése után a telemetria folyamatosan beérkezzen: a repository ugyanazt a DTO-t kapja, de már a `telemetry_ingest` és a `telemetry` táblába ír.

15.2.6 Labor: TimescaleDB-táblák létrehozása és lekérdezése

Munkakönyvtár:

```
mkdir -p ~/iot-labor/i045 && cd ~/iot-labor/i045
```

Előfeltétel: a Ingest: MQTT-ből a backendbe, idempotensen laborban elkészített ingest és a Adatformátumok: JSON, CBOR, Protocol Buffers, SenML alfejezetben elfogadott telemetriaséma.

A 15.1-ben a közös `~/iot-labor/stack/compose.yaml` már tartalmazza a `lab-timescaledb` szolgáltatást a `terra` felhasználóval, a `lab-password` jelszóval és a `terrasense` adatbázissal. Ebben az alfejezetben **nem hozunk létre új adatbázis-szolgáltatást**, mert az ugyanarra az 5432-es hostportra publikálva portütközést és rejtett adatbázisváltást okozna. A meglévő szolgáltatást használjuk; ellenőrizzük, hogy a stackben csak egy 5432-es publikáció van:

```
docker compose -f ~/iot-labor/stack/compose.yaml config | grep -n "5432"
docker compose -f ~/iot-labor/stack/compose.yaml ps
```

A szolgáltatás referenciaként a következő (ez már a 15.1-ben szerepel):

```
  lab-timescaledb:
    image: timescale/timescaledb:2.17.2-pg16
    container_name: lab-timescaledb
    environment:
      POSTGRES_USER: terra
      POSTGRES_PASSWORD: lab-password
      POSTGRES_DB: terrasense
    ports:
      - "5432:5432"
```

```
    volumes:
      - lab-timescaledb-data:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U terra -d terrasense"]
      interval: 10s
      timeout: 5s
      retries: 5
```

A `volumes:` szekcióban a kötet explicit nevet kap, hogy a takarításnál ne kelljen a Compose projektelőtagjával számolni:

```
volumes:
  lab-timescaledb-data:
    name: lab-timescaledb-data
```

A jelszó kizárólag a helyi laborhoz használt, nem éles környezetbe való. Ha a 15.1-es stackben még szerepel egy korábbi `postgres` szolgáltatás, azt ne indítsuk el, és a `ts-backend` `SPRING_DATASOURCE_URL` értéke a `lab-timescaledb` szolgáltatásnévre mutasson. A 15.1 laborban ezt már így állítottuk be.

Indítsuk el a szolgáltatást:

```
docker compose -f ~/iot-labor/stack/compose.yaml up -d lab-timescaledb
```

Másoljuk be a migrációt a labor könyvtárába, és futtassuk a 15.1-es adatbázison:

```
docker compose -f ~/iot-labor/stack/compose.yaml exec -T lab-timescaledb \
  psql -U terra -d terrasense -v ON_ERROR_STOP=1 < 02_normalized_telemetry.sql
```

A migráció után ellenőrizzük a táblákat és az átvitt sorok számát:

```
SELECT hypertable_name, num_chunks
FROM timescaledb_information.hypertables
WHERE hypertable_name = 'telemetry';

SELECT count(*) AS event_rows FROM telemetry_ingest;
SELECT count(*) AS measurement_rows FROM telemetry;
```

A `measurement_rows` értéke a 15.1-ben fogadott események méréseinek összege; ha a 15.1 laborban nem küldtünk üzenetet, mindkét érték nulla. Ez nem hiba: a migráció a meglévő adatokat viszi át, az új üzeneteket pedig a frissített repository írja.

Ezután írjuk meg a `~/iot-labor/i045/schema.sql` fájlt a fenti végleges sémával (a friss telepítéshez), a `seed.sql` fájlt a tesztadatokkal, és a `queries.sql` fájlt a lekérdezésekkel. A seed a 15.1-es migráció után is futtatható, mert a `telemetry` tábla már a hosszú formátumot használja:

```
-- ~/iot-labor/i045/seed.sql
SET TIME ZONE 'UTC';

INSERT INTO telemetry_ingest
    (site, node_id, boot_id, seq, received_at, ts, schema_version, payload)
VALUES
    ('gh1', 'n001', 'boot-1', 42, now(), 1735689600000, 1,

      ↪  '{"schema_version":1,"node_id":"n001","seq":42,"ts":1735689600000,"boot_id":"boot-1","measurements":{"t
      ↪  ::jsonb),
    ('gh1', 'n001', 'boot-1', 43, now(), 1735689900000, 1,

      ↪  '{"schema_version":1,"node_id":"n001","seq":43,"ts":1735689900000,"boot_id":"boot-1","measurements":{"t
      ↪  ::jsonb),
```

```
    ('gh1', 'n001', 'boot-1', 44, now(), 1735690200000, 1,
      ↪  '{"schema_version":1,"node_id":"n001","seq":44,"ts":1735690200000,"boot_id":"boot-1","measurements":{"t
      ↪  ::jsonb),
    ('gh1', 'n001', 'boot-1', 45, now(), 1735690500000, 1,
      ↪  '{"schema_version":1,"node_id":"n001","seq":45,"ts":1735690500000,"boot_id":"boot-1","measurements":{"t
      ↪  ::jsonb)
ON CONFLICT (site, node_id, boot_id, seq) DO NOTHING;

INSERT INTO telemetry
    (ts, site, node_id, seq, boot_id, schema_version,
     measurement_type, value, unit, valid)
VALUES
    (to_timestamp(1735689600.0), 'gh1', 'n001', 42, 'boot-1', 1,
     'temperature', 22.5, 'Cel', true),
    (to_timestamp(1735689600.0), 'gh1', 'n001', 42, 'boot-1', 1,
     'humidity', 60.2, '%RH', true),
    (to_timestamp(1735689600.0), 'gh1', 'n001', 42, 'boot-1', 1,
     'soil_moisture', 35.0, '%', true),
    (to_timestamp(1735689900.0), 'gh1', 'n001', 43, 'boot-1', 1,
     'temperature', 22.8, 'Cel', true),
    (to_timestamp(1735690200.0), 'gh1', 'n001', 44, 'boot-1', 1,
     'temperature', 23.1, 'Cel', true),
    (to_timestamp(1735690500.0), 'gh1', 'n001', 45, 'boot-1', 1,
     'temperature', 22.9, 'Cel', true)
ON CONFLICT (site, node_id, boot_id, seq, measurement_type, ts) DO NOTHING;
```

A `seed.sql` bejátszása után futtassuk a `~/iot-labor/i045/queries.sql` lekérdezéseit:

```
-- ~/iot-labor/i045/queries.sql
SET TIME ZONE 'UTC';

-- 1) óránkénti átlaghőmérséklet az n001 eszközre
SELECT time_bucket('1 hour', ts) AS bucket,
       round(avg(value)::numeric, 2) AS avg_temp,
       count(*) AS sample_count
FROM telemetry
WHERE site = 'gh1'
  AND node_id = 'n001'
  AND measurement_type = 'temperature'
GROUP BY bucket
ORDER BY bucket;

-- 2) utolsó érték minden méréstípusra, eszközönként
SELECT DISTINCT ON (node_id, measurement_type)
       node_id,
       measurement_type,
       value,
       ts
FROM telemetry
WHERE site = 'gh1'
ORDER BY node_id, measurement_type, ts DESC;

-- 3) hiányzó órás intervallumok kitöltése standard SQL-lel
WITH hourly AS (
    SELECT time_bucket('1 hour', ts) AS bucket,
           avg(value) AS avg_temp
    FROM telemetry
    WHERE site = 'gh1'
      AND node_id = 'n001'
      AND measurement_type = 'temperature'
    GROUP BY bucket
```

```
),
grid AS (
    SELECT generate_series(
                date_trunc('hour', min(bucket)),
                date_trunc('hour', max(bucket)),
                interval '1 hour') AS bucket
    FROM hourly
)
SELECT grid.bucket, hourly.avg_temp
FROM grid
LEFT JOIN hourly USING (bucket)
ORDER BY grid.bucket;
```

Futtassuk a szkripteket a Compose szolgáltatásnevén keresztül:

```
docker compose -f ~/iot-labor/stack/compose.yaml exec -T lab-timescaledb \
  psql -U terra -d terrasense -v ON_ERROR_STOP=1 < schema.sql
docker compose -f ~/iot-labor/stack/compose.yaml exec -T lab-timescaledb \
  psql -U terra -d terrasense -v ON_ERROR_STOP=1 < seed.sql
docker compose -f ~/iot-labor/stack/compose.yaml exec -T lab-timescaledb \
  psql -U terra -d terrasense -v ON_ERROR_STOP=1 < queries.sql
```

Várt kimenet (a harmadik lekérdezésnél rövidítve):

```
        bucket         | avg_temp | sample_count
-----------------------+----------+--------------
 2025-01-01 00:00:00+00 |    22.83 |            4

 node_id | measurement_type | value |           ts
---------+------------------+-------+------------------------
 n001    | humidity         |  60.2 | 2025-01-01 00:00:00+00
 n001    | soil_moisture    |  35.0 | 2025-01-01 00:00:00+00
 n001    | temperature      |  22.9 | 2025-01-01 00:15:00+00
```

A seed négy hőmérsékleti értéke (22,5; 22,8; 23,1; 22,9) ugyanabba az órába esik, átlaguk 22,825, két tizedesre kerekítve 22,83. Az utolsó hőmérséklet értéke 22,9, időpontja 00:15, nem 00:05. A harmadik lekérdezés a jelenlegi seednél egyetlen órás bucketet ad vissza, ezért nincs `NULL` sora; ha a seed több órára terjedne ki, és valamelyik óra hiányozna, a `grid` és a `LEFT JOIN` ahhoz a sorhoz `NULL` átlagot adna. A dashboard ezt később megszakításként vagy külön jelöléssel jelenítheti meg, de a hiányzó adatot nem szabad automatikusan nullának vagy folyamatos értéknek tekinteni.

Idempotencia-tesztek a két táblán. A laborban küldjünk ugyanarra az eseménykulcsra ismételt üzeneteket. Először ugyanazzal a `ts`-sel és tartalommal:

```
mosquitto_pub -h localhost -t terrasense/gh1/n001/telemetry -q 1 -m
↪  '{"schema_version":1,"node_id":"n001","seq":42,"ts":1735689600000,"boot_id":"boot-1","measurements":{"tempe
```

A `telemetry_ingest` száma nem változik, a `telemetry` táblában sem keletkezik új sor. Ezután ugyanaz az esemény eltérő `ts`-sel:

```
mosquitto_pub -h localhost -t terrasense/gh1/n001/telemetry -q 1 -m
↪  '{"schema_version":1,"node_id":"n001","seq":42,"ts":1735689900000,"boot_id":"boot-1","measurements":{"tempe
```

Az eseménytábla `(site, node_id, boot_id, seq)` kulcsa ütközik, a payload pedig eltér, ezért a repository nem ír új méréssort, hanem a `dead_letter` táblába menti az ütközést. Ezután ugyanazt a `seq`-et küldjük egy másik `boot_id`-val:

```
mosquitto_pub -h localhost -t terrasense/gh1/n001/telemetry -q 1 -m
↪  '{"schema_version":1,"node_id":"n001","seq":42,"ts":1735693200000,"boot_id":"boot-2","measurements":{"tempe
```

Ez új esemény: az eszköz újraindult, a `seq` újra 42-től indul, de a `boot_id` eltér. Végül ugyanazt a `seq`-et küldjük egy másik site alatt:

```
mosquitto_pub -h localhost -t terrasense/gh2/n001/telemetry -q 1 -m
↪  '{"schema_version":1,"node_id":"n001","seq":42,"ts":1735693500000,"boot_id":"boot-2","measurements":{"tempe
```

Ez is új esemény, mert a `site` része az eseménykulcsnak. A várt számok a három teszt után: a `telemetry_ingest` és a `telemetry` tábla a 15.1-ből átvitt sorokhoz képest a `boot-2` eseményekkel bővül, a `gh1/boot-1/seq=42` esemény pedig változatlan marad. A `dead_letter` táblában egy új sor jelenik meg a `gh1/boot-1/seq=42` eltérő `ts`-ű változatára.

Takarítás. A `docker compose stop` után a leállított konténer még hivatkozik a volume-ra, ezért a `docker volume rm` önmagában „volume is in use” hibát adhat. A helyes sorrend:

```
# 1) adatmegőrző leállítás
docker compose -f ~/iot-labor/stack/compose.yaml stop lab-timescaledb

# 2) a konténer eltávolítása; a volume még megmarad
docker compose -f ~/iot-labor/stack/compose.yaml rm -f lab-timescaledb

# 3) tudatos adatvesztés: csak akkor, ha tényleg törölni akarjuk az adatbázist
docker volume rm lab-timescaledb-data
```

A `lab-timescaledb` szolgáltatásnév és a `lab-timescaledb-data` explicit volume-név a 15.1-ből származik; más szolgáltatást és más volume-ot ne töröljünk. A `docker compose down -v` a teljes stack összes volume-ját törölné, ezért azt csak tudatos teljes visszaállításkor használjuk.

15.2.7 Ellenőrzés és hibakeresés

Ellenőrizzük, hogy a TimescaleDB kiterjesztés aktív, a hypertable létrejött, és a migráció naplózva van:

```
SELECT extname FROM pg_extension WHERE extname = 'timescaledb';
SELECT hypertable_name, num_chunks
FROM timescaledb_information.hypertables
WHERE hypertable_name = 'telemetry';
SELECT version, description, applied_at FROM schema_migration ORDER BY version;
```

Hibatünetek és megoldások:

Tünet	Lehetséges ok	Javítás
`function create_hypertable does not exist`	nincs betöltve a kiterjesztés	`CREATE EXTENSION IF NOT EXISTS timescaledb;`
`relation "telemetry" does not exist`	a schema.sql vagy a migráció nem futott le	futtassuk a migrációt, majd a végleges `schema.sql`-t a megfelelő adatbázisban
`relation "telemetry_ingest" does not exist`	a 15.1-es táblát nem nevezte át a migráció	ellenőrizzük a `02_normalized_telemetry.sql` lefutását és a `schema_migration` táblát
duplikált méréssorok jelennek meg	az eseménytábla beszúrása kimaradt a repository-ból	a repository először a `telemetry_ingest`-be írjon, és csak sikeres beszúrás után a `telemetry`-be

Tünet	Lehetséges ok	Javítás
a `telemetry` beszúrás `ON CONFLICT` hibát ad	a `ts` hiányzik az egyedi kulcsból	az egyedi kulcs `(site, node_id, boot_id, seq, measurement_type, ts)` legyen
portütközés a 5432-es porton	egy régi `postgres` és az új `lab-timescaledb` is fut	csak a `lab-timescaledb` szolgáltatást indítsuk; a régit ne indítsuk el
a `docker volume rm` „volume is in use” hibát ad	a leállított konténer még létezik	előbb `docker compose rm -f lab-timescaledb`, csak utána a volume törlése
lassú az idő szerinti szűrés	nincs hypertable vagy rossz az index	ellenőrizzük a `create_hypertable` hívást és a `ts` oszlopot

15.2.8 Tipikus hibák és félreértések

1. **Az epoch ms értéket másodpercként értelmezik.** A `to_timestamp(1735689600000)` hibás, mert a PostgreSQL az argumentumot másodpercnek veszi. Mindig osszuk el 1000-rel, vagy a Java oldalon már `Instant.ofEpochMilli`-vel alakítsuk `Timestamp`-re.
2. **Minden mérést külön táblába tesznek.** Ez felrobbantja a sémát és a lekérdezések bonyolultságát. A hosszú tábla egy `measurement_type` oszloppal rugalmasabb és jobban illeszkedik a TimescaleDB-hez.
3. **Nincs eseményszintű egyedi kulcs az idempotens beszúráshoz.** Ha az MQTT-ingest legalább egyszer kézbesít, az újra érkező üzenetek duplikátumokat szülnek. Az `telemetry_ingest (site, node_id, boot_id, seq)` elsődleges kulcsa és az `ON CONFLICT DO NOTHING` az idempotencia adatbázis-szintű biztosítéka. A `telemetry` egyedi kulcsa csak a méréssorok védelme; az esemény eltérő `ts`-sel történő visszajátszását az eseménytábla fogja meg.
4. **A `server_ts`-t használják eseményidőként.** A `server_ts` a beérkezés ideje, a `ts` az eseményé. A gateway offline pufferelésekor a kettő eltérhet; a mérési grafikonokat és az aggregációkat a `ts` szerint kell készíteni.
5. **A migráció után a régi `schema.sql`-t hagyják a projektben.** A backend indításakor a `spring.sql.init` újra létrehozná a régi `telemetry` táblát, és a repository már nem oda ír. A migrációval együtt a `src/main/resources/schema.sql`-t is a végleges sémára kell cserélni.

15.2.9 Feladatok

1. **Fogalmi/döntési feladat.** Milyen szempontok alapján választanál InfluxDB-t a TimescaleDB helyett, és milyen esetben lenne a széles tábla előnyös a TerraSense-ben?
2. **Számolási feladat.** Tételezz fel 1000 csomópontot, csomópontonként percenként 3 mérést. Hány telemetria-sor keletkezik naponta? Mennyi tárhelyet foglal ez egy év alatt, ha egy sor átlagosan 120 B, és az első évben nincs tömörítés?
3. **Implementációs feladat.** Egészítsd ki a labor `queries.sql` fájlját egy olyan lekérdezéssel, amely az utolsó órában a talajnedvesség maximumát adja csomópontonként.
4. **Hibakeresési/kiterjesztési feladat.** A dashboard azt mutatja, hogy az n001 hőmérséklet-görbéje órás bontásban hiányos, mert a szenzor néha kimarad. Készíts lekérdezést, amely a

hiányzó órákat `NULL` helyett az előző mért értékkel tölti ki, és magyarázd el, miért lehet ez félrevezető riasztások esetén.

5. **Migrációs feladat.** A 15.1-es adatbázisban egy esemény payloadja a régi, lapos formátumú `temperature_c` mezőket tartalmazza, nem `measurements` objektumot. Hogyan egészítenéd ki a migrációt, hogy ezek a sorok se vesszenek el?

15.2.10 Megoldások

1. Az InfluxDB előnyös, ha elosztott, nagy skálázású idősoros klasztert akarunk, vagy a csapat kifejezetten a Flux/InfluxQL nyelvet preferálja. A széles tábla akkor lehet előnyös, ha minden eszköz minden mérést mindig ugyanabban az időpontban küld, és az oszlopkészlet stabil; ekkor egy sorba kerül az összes érték, és az összetartozó mérések lekérése gyorsabb.

2. A napi sorok száma:

$$1000 \times 3 \times 60 \times 24 = 4\,320\,000$$

   Az éves nyers méret:

$$4\,320\,000 \times 365 \times 120 \text{ B} \approx 189 \text{ GB}$$

   Ezért fontos a TimescaleDB tömörítése és a ritkítás, amit a Megőrzés, ritkítás és tárolási költség tárgyal.

3. Például:

```
SELECT node_id,
       max(value) AS max_humidity
FROM telemetry
WHERE site = 'gh1'
  AND measurement_type = 'soil_moisture'
  AND ts >= now() - interval '1 hour'
GROUP BY node_id
ORDER BY node_id;
```

4. A valódi last observation carried forward (LOCF) nem elégszik meg az egy lépéses `lag` függvénnyel: az csak egyetlen hiányzó intervallumot töltene ki, két vagy több hiányzó óra után ismét `NULL` lenne. Az alábbi megoldás minden rácspontra a legutolsó nem `NULL` mért órát keresi meg, és megkülönbözteti a mért, a továbbvitt és az elavult értéket:

```
WITH hourly AS (
    SELECT time_bucket('1 hour', ts) AS bucket,
           avg(value) AS avg_temp
    FROM telemetry
    WHERE site = 'gh1'
      AND node_id = 'n001'
      AND measurement_type = 'temperature'
    GROUP BY bucket
),
grid AS (
    SELECT generate_series(
               date_trunc('hour', min(bucket)),
               date_trunc('hour', max(bucket)),
               interval '1 hour') AS bucket
    FROM hourly
)
SELECT g.bucket,
       h.avg_temp AS measured_avg,
       CASE
           WHEN h.avg_temp IS NOT NULL THEN h.avg_temp
           WHEN g.bucket - src.bucket > interval '3 hours' THEN NULL
```

```
            ELSE src.avg_temp
        END AS filled_temp,
        src.bucket AS source_bucket,
        CASE
            WHEN h.avg_temp IS NOT NULL THEN 'MERT'
            WHEN src.bucket IS NULL THEN 'NINCS'
            WHEN g.bucket - src.bucket > interval '3 hours' THEN 'ELAVULT'
            ELSE 'LOCF'
        END AS fill_status,
        CASE
            WHEN src.bucket IS NULL THEN NULL
            ELSE g.bucket - src.bucket
        END AS staleness
FROM grid g
LEFT JOIN hourly h USING (bucket)
LEFT JOIN LATERAL (
    SELECT h2.bucket, h2.avg_temp
    FROM hourly h2
    WHERE h2.bucket <= g.bucket
      AND h2.avg_temp IS NOT NULL
    ORDER BY h2.bucket DESC
    LIMIT 1
) src ON TRUE
ORDER BY g.bucket;
```

A `MAX_STALENESS` itt 3 óra: ha az utolsó mért érték ennél régebbi, a kitöltött érték `NULL`, és a `fill_status` `ELAVULT` lesz. A `MERT`, `LOCF` és `ELAVULT` jelző segít elkerülni, hogy a dashboard a becsült adatot valódi mérésként mutassa. A riasztási logikának a `fill_status` értékét is figyelembe kell vennie: LOCF mellett a riasztás „nincs friss adat” típusú lehet, nem pedig egy interpolált értékre kiadott riasztás.

A TimescaleDB `time_bucket_gapfill` és a `locf` függvény kényelmesebb alternatíva, de a `locf` a TimescaleDB Toolkit kiterjesztés része, amelyet az alap `timescale/timescaledb:2.17.2-pg16` image nem feltétlenül tartalmaz. Ezért a labor hordozható, standard SQL-megoldást használ; ha a Toolkit telepítve van, a `time_bucket_gapfill('1 hour', ts)` és a `locf(avg(value))` ugyanazt a célt szolgálja.

Teszt három egymást követő hiányzó órára: a 00:00 és 01:00 órában legyen mért érték, a 02:00, 03:00 és 04:00 hiányozzon, majd 05:00-kor ismét legyen mérés. A fenti lekérdezésben a 02:00 és 03:00 `LOCF`, a 04:00 szintén `LOCF` (3 órás staleness), az 05:00 pedig `MERT`; ha a küszöböt 2 órára állítjuk, a 04:00 már `ELAVULT` és `NULL` lesz. A riasztás csak a `MERT` sorokra támaszkodjon, a LOCF sorokat pedig jelölje becsültnek.

5. A migrációt ki kell egészíteni egy második `SELECT` ággal, amely a lapos mezőket alakítja méréssorokká. A `telemetry_ingest.payload` a 15.1-ben normalizált, de ha történt korábbi, közvetlenül lapos payloadot író beszúrás, akkor a `payload ? 'measurements'` feltétel helyett a következő unió használható:

```
INSERT INTO telemetry
    (server_ts, ts, site, node_id, seq, boot_id, schema_version,
     measurement_type, value, unit, valid)
SELECT i.received_at,
       to_timestamp(i.ts / 1000.0),
       i.site, i.node_id, i.seq, i.boot_id, i.schema_version,
       v.type, v.value, v.unit, true
FROM telemetry_ingest i
CROSS JOIN LATERAL (
    VALUES
        ('temperature', (i.payload ->> 'temperature_c')::double precision, 'Cel'),
        ('humidity',    (i.payload ->> 'humidity_rh')::double precision, '%RH'),
```

```
        ('soil_moisture', (i.payload ->> 'soil_moisture_pct')::double precision, '%')
) AS v(type, value, unit)
WHERE NOT (i.payload ? 'measurements')
  AND v.value IS NOT NULL
ON CONFLICT (site, node_id, boot_id, seq, measurement_type, ts) DO NOTHING;
```

A két ág együtt minden korábbi eseményt lefed: a kanonikus `measurements` objektumot tartalmazókat az első, a lapos mezőket tartalmazókat a második ág. A `schema_version` értéke a lapos eseményeknél 0 lehet, ezt a 15.1-es DTO `effectiveSchemaVersion()` metódusa állítja elő.

15.2.11 Rövid összefoglaló

Az alfejezet bevezette az idősoros adat tárolásának alapelveit, és a TerraSense-hez a TimescaleDB-t választotta, mert az a PostgreSQL-re épül, és a meglévő Spring/JDBC-környezetbe jól illeszkedik. A `telemetry` tábla hosszú formát használ: minden mérés külön sor, `measurement_type` és `value` oszlopokkal. Az eseményszintű idempotenciát a `telemetry_ingest` tábla `(site, node_id, boot_id, seq)` kulcsa biztosítja, a `telemetry` egyedi kulcsa pedig `(site, node_id, boot_id, seq, measurement_type, ts)`. A `create_hypertable` a `ts` eseményidőre particionálja a táblát. A 15.1-es adatbázist verziózott SQL-migráció frissíti: a régi `telemetry` tábla `telemetry_ingest` néven megmarad, a mérések pedig backfill-lel a hosszú táblába kerülnek. A laborban létrehoztuk a `~/iot-labor/i045/schema.sql`, `seed.sql` és `queries.sql` fájlokat, valamint a közös compose-fájlban a `lab-timescaledb` szolgáltatást használtuk. **Továbbvitt eredmény.** Ezt a táblaszerkezetet, indexeit és lekérdezéseit használja a Megőrzés, ritkítás és tárolási költség, a Dashboardok Grafanával és a Záróprojekt: a teljes TerraSense rendszer összeállítása és átvétele.

15.2.12 Források és ellenőrzési hiányok

- TimescaleDB dokumentáció: hypertable, `time_bucket`, `time_bucket_gapfill`, `create_hypertable` és a folyamatos aggregátumok — az aktuális verzióban elérhető pontos függvénynevek és paraméterek ellenőrizendők.
- InfluxDB dokumentáció: az aktuális 2.x/3.x query nyelv és a TimescaleDB-vel való összevetés részletei verziófüggők.
- Spring Framework dokumentáció: `JdbcTemplate.batchUpdate` pontos aláírása a használt Spring Boot 3.3.4 verzióban ellenőrizendő.
- A `timescale/timescaledb:2.17.2-pg16` konténerkép elérhetősége ellenőrizendő a labor futtatása előtt; a `latest-pg16` helyett a rögzített verzió használatos.
- A migráció, a seed és a lekérdezések futtatása a lektorálási környezetben nem történt meg; a várt eredmények a seedből számított értékek.
- A laborban használt `lab-password` kizárólag helyi, eldobható jelszó; éles rendszerben soha ne használjuk.

15.3 Megőrzés, ritkítás és tárolási költség

Az IoT-rendszer adatai folyamatosan, monoton módon keletkeznek: a TerraSense telemetriája másodpercenként vagy percenként érkezik, és soha nem „fogy el”. Ha minden nyers mintát korlátlan ideig tárolunk, a tárolási költség hamar meghaladja a rendszer értékét. Ugyanakkor a nyers adatok rövid távon fontosak a hibakereséshez, a hosszú távú trendekhez pedig elegendő az aggregált forma. Ebben az alfejezetben megtervezzük a TerraSense többszintű megőrzési

stratégiáját, és bemutatjuk, hogyan automatizálja ezt TimescaleDB-ben a megőrzési házirend, a folyamatos aggregátum és a tömörítés.

15.3.1 Motiváció: a folyamatosan növekvő adatmennyiség

Tegyük fel, hogy a TerraSense üvegház-telepén 1000 `ts-node` működik, mindegyik 1 percenként küld egy telemetria-üzenetet három méréssel: hőmérséklet, relatív páratartalom és talajnedvesség. A ts-backend ezt a idempotens ingest után az idősoros adatbázisba írja. A 15.2-ben bevezetett hosszú táblában egy üzenet három mérési sort eredményez, így egy év alatt több mint másfél milliárd mérési sor keletkezik. Ha a nyers sorokat örökké tartjuk, a tárolási költség lineárisan nő, és a lekérdezések is lassulnak. A megoldás nem az adat eldobása, hanem az **adattömörítés időben és tömörítési formában**: a régi nyers adatot rövid megőrzés után tömörítjük, majd később aggregált formába sűrítjük.

15.3.2 Fogalmak és működés

Definíció (megőrzési stratégia, retention strategy). Az idősoros adatbázisban előírt szabályrendszer, amely meghatározza, hogy az egyes adatfelbontások (nyers, órás, napi) mennyi ideig őrződnek meg, és mi történik a lejárt adatokkal (eldobás, archiválás, további aggregálás).

Definíció (chunk és megőrzési házirend). A TimescaleDB a hypertable-t belső, időben partícionált darabokra, úgynevezett `chunk`okra bontja. A `retention_policy drop_chunks` művelete a lejárt intervallumú chunkokat törli. Ez a törlés fájlszintű, ezért gyors, és nem igényel soronkénti `DELETE`-et.

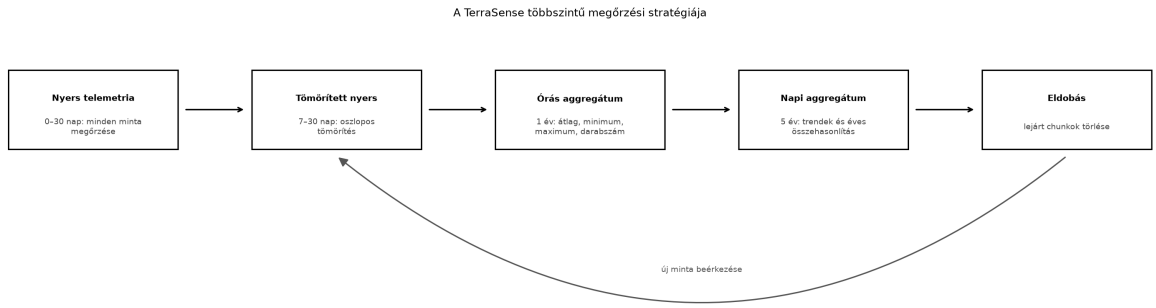
Definíció (folyamatos aggregátum, continuous aggregate). A TimescaleDB materializált nézete, amelyet a rendszer automatikusan, inkrementálisan frissít az új nyers adatok beérkezésekor. A felhasználó úgy kérdezheti le, mint egy normál nézetet, de a mögöttes aggregált tárolás miatt sokkal gyorsabb, mint a nyers adatokból való mindenkori `GROUP BY`.

Definíció (tömörítés, compression). A TimescaleDB oszlopos, deduplikáló tömörítése, amely a chunkok tárolási méretét gyakran 5–15-szörösére csökkenti. A tömörített chunkok olvashatók, de írni nem lehet őket.

A megőrzési stratégia rétegei a TerraSense számára:

- **Nyers telemetria:** 30 napig megőrzés, 7 nap után tömörítés.
- **Órás aggregátum:** 1 évig megőrzés, 3 hónap után opcionális tömörítés (ha a TimescaleDB verziója támogatja a folyamatos aggregátum tömörítését).
- **Napi aggregátum:** 5 évig megőrzés, 1 év után opcionális tömörítés.

Ezt a rétegződést mutatja az alábbi folyamatábra:



15.3. ábra: A TerraSense többszintű megőrzési stratégiája - Az időhöz kötött szabályok a példa szerint; a konkrét értékek üzemeltetői döntés.

A folyamatábra leírása: minden beérkező nyers minta először a 30 napos nyers rétegbe kerül. A 7. nap után a nyers chunk tömörödik, de továbbra is lekérdezhető. Az órás aggregátum folyamatosan frissül, és 1 évig őrzi a köztes statisztikákat. A napi aggregátum 5 évig ad hosszú távú trendet, például a Grafana-dashboardhoz. A lejárt chunkok törlődnek.

15.3.3 Döntési szempontok

Kérdés	Lehetőségek	TerraSense-döntés	Indoklás
Nyers adat megőrzési ideje	7 nap, 30 nap, 1 év	30 nap	Hibakereséshez elegendő, de nem növeli túlzottan a tárolást
Aggregátumok időablaka	1 óra, 1 nap	mindkettő	Az órás a rövid távú riasztáshoz, a napi a hosszú távú trendhez
Aggregátum megőrzési ideje	1 év, 5 év, korlátlan	1 év (órás), 5 év (napi)	Jogi és üzleti trendek, de a tárolási költség kontroll alatt
Tömörítés engedélyezése	nincs, csak nyers, minden réteg	nyers 7 nap után, aggregátumok később	A tömörített chunk csak olvasható, ezért ne tömörítsünk túl korán
Időbélyeg típusa	`timestamptz`, `timestamp`	`timestamptz`	UTC, időzóna-helyes; epoch ms a kódban, az adatbázisban `timestamptz`
Későn érkező adat kezelése	eldobás, LOCF, visszamenőleges refresh	visszamenőleges refresh 8/9 napos ablakkal	A gateway legfeljebb 7 napot pufferel offline; az ennél régebbi adat csak külön eljárással korrigálható

15.3.4 Kidolgozott példa: adatmennyiség és tárolási igény

Számítsuk ki az 1000 csomópontos TerraSense flotta éves nyers adatmennyiségét a 15.2-ben bevezetett hosszú táblában. A telemetria-üzenetek 1 percenként érkeznek, és üzenetenként három mérés van, tehát a mérési sorok száma:

$$N_{\text{minta}} = 1000 \cdot 3 \cdot 60 \cdot 24 \cdot 365 = 1\,576\,800\,000 \text{ sor/év}$$

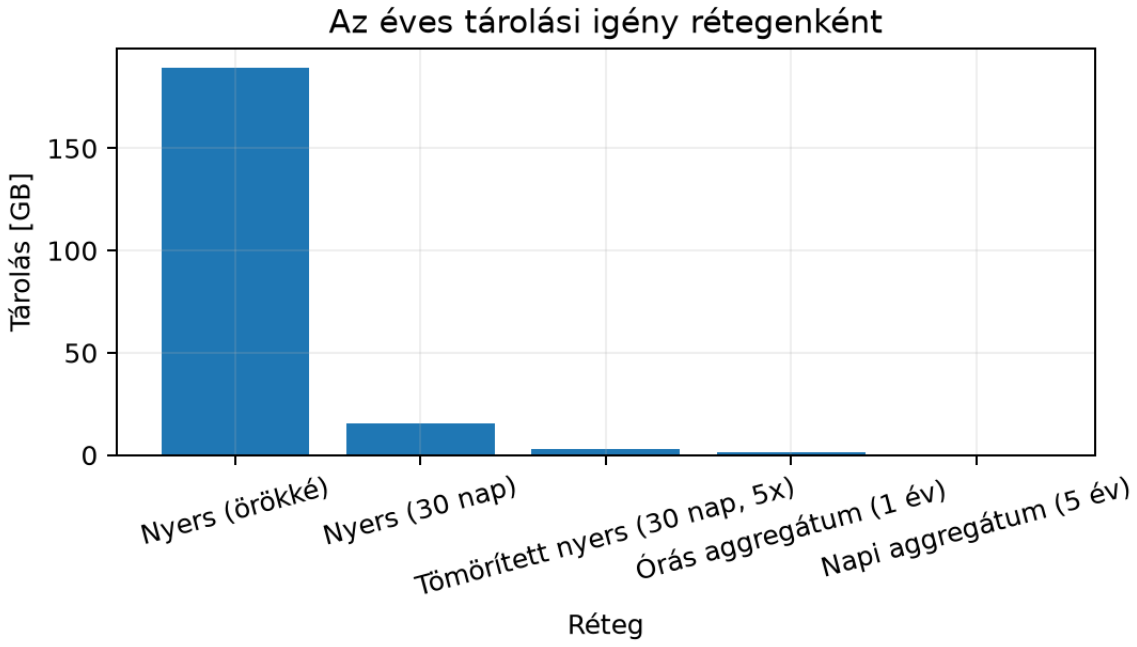
Feltételezzük, hogy egy mérési sor a hosszú táblában a `ts` (8 B), `site` (4 B), `node_id` (8 B), `seq` (8 B), `boot_id` (8 B), `measurement_type` (átlagosan 12 B), `value` (8 B), `unit` (4 B) és a sorindex overhead (kb. 60 B) miatt összesen körülbelül 120 B helyet foglal. Az éves nyers tárolási igény:

$$S_{\text{nyers}} = 1\,576\,800\,000 \cdot 120 \text{ B} = 189\,216\,000\,000 \text{ B} \approx 189{,}2 \text{ GB}$$

A 30 napos megőrzéssel a nyers réteg csak az év 30/365 részét tartja:

$$S_{\text{nyers, 30 nap}} = 189{,}2 \cdot \frac{30}{365} \approx 15{,}55 \text{ GB}$$

Az órás aggregátum 1 évig csomópontonként és méréstípusonként 8 760 sort tartalmaz, azaz $1000 \cdot 3 \cdot 8760 = 26\,280\,000$ sort, soronként kb. 60 B, összesen 1,58 GB. A napi aggregátum 5 évre $1000 \cdot 3 \cdot 365 \cdot 5 = 5\,475\,000$ sort tartalmaz, szintén kb. 60 B/sor mellett 0,33 GB. A tömörítés a TimescaleDB-ben jellemzően 5–10-szeres csökkentést ad (illusztratív, a konkrét mérés az adatok redundanciájától függ). Ha csak a nyers tömörített 30 napos rétegét nézzük, a 15,55 GB 5-szörös tömörítéssel 3,11 GB-ra csökken.



15.4. ábra: Az éves tárolási igény rétegenként - Illusztratív, a hosszú tábla 120 B-os mérési sorából és 5-szörös tömörítési arányból számított.

A diagram leírása: a nyers adat korlátlan ideig tartása 189,2 GB-ot igényelne évente. A 30 napos megőrzés ezt 15,55 GB-ra csökkenti, a tömörítés pedig 3,11 GB-ra. Az órás és napi aggregátumok hosszabb megőrzési idejük ellenére elhanyagolható méretűek. A számítás a mérési sorokkal dolgozik, nem az MQTT-üzenetekkel: egy üzenet a hosszú táblában három sor.

15.3.5 Labor: megőrzés, folyamatos aggregátum, tömörítés

Munkakönyvtár:

```
mkdir -p ~/iot-labor/i046 && cd ~/iot-labor/i046
```

Előfeltétel: a Idősoros adatbázisok és adatmodellezés laborjában létrehozott `terrasense` adatbázis, `telemetry` hypertable és `telemetry_ingest` eseménytábla. A közös stackben az adatbázis szolgáltatásneve `lab-timescaledb`, a felhasználó `terra`, az adatbázis `terrasense`; ezeket a 15.2 laborban már egységesen beállítottuk. Nem hozunk létre új `ts-timescaledb` konténert és nem használunk `ts` felhasználót.

Hozzunk létre egy SQL scriptet:

```
-- ~/iot-labor/i046/01_retention_aggregates.sql
-- Feltételezi: telemetry hypertable és telemetry_ingest létezik (lásd i045 labor)

-- 1) Folyamatos órás aggregátum a hosszú táblából
CREATE MATERIALIZED VIEW telemetry_hourly
WITH (timescaledb.continuous) AS
SELECT
  time_bucket('1 hour', ts) AS bucket,
  site,
  node_id,
  measurement_type,
  avg(value)   AS avg_value,
  min(value)   AS min_value,
  max(value)   AS max_value,
  count(*)     AS sample_count
FROM telemetry
WHERE valid = true
GROUP BY time_bucket('1 hour', ts), site, node_id, measurement_type;

-- 2) Folyamatos napi aggregátum az órásból, mintaszám szerint súlyozott átlaggal
CREATE MATERIALIZED VIEW telemetry_daily
WITH (timescaledb.continuous) AS
SELECT
  time_bucket('1 day', bucket) AS bucket,
  site,
  node_id,
  measurement_type,
  sum(avg_value * sample_count) / nullif(sum(sample_count), 0) AS avg_value,
  min(min_value) AS min_value,
  max(max_value) AS max_value,
  sum(sample_count) AS sample_count
FROM telemetry_hourly
GROUP BY time_bucket('1 day', bucket), site, node_id, measurement_type;

-- 3) Induló backfill: a már meglévő nyers adatokból feltöltjük a nézeteket
CALL refresh_continuous_aggregate('telemetry_hourly', NULL, NULL);
CALL refresh_continuous_aggregate('telemetry_daily', NULL, NULL);

-- 4) Nyers adat tömörítése 7 nap után, szegmentálás site/node/méréstípus szerint
ALTER TABLE telemetry SET (
  timescaledb.compress,
```

```
  timescaledb.compress_segmentby = 'site, node_id, measurement_type',
  timescaledb.compress_orderby = 'ts DESC'
);
SELECT add_compression_policy('telemetry', INTERVAL '7 days');

-- 5) Nyers adat megőrzése: 30 nap után chunkok eldobása
SELECT add_retention_policy('telemetry', INTERVAL '30 days');

-- 6) Folyamatos aggregátumok megőrzése
SELECT add_retention_policy('telemetry_hourly', INTERVAL '1 year');
SELECT add_retention_policy('telemetry_daily',  INTERVAL '5 years');

-- 7) Folyamatos aggregátumok automatikus frissítése
--    A 7 napos offline pufferéléshez illeszkedő 8/9 napos visszamenőleges ablak.
SELECT add_continuous_aggregate_policy('telemetry_hourly',
  start_offset => INTERVAL '8 days',
  end_offset   => INTERVAL '1 hour',
  schedule_interval => INTERVAL '1 hour');

SELECT add_continuous_aggregate_policy('telemetry_daily',
  start_offset => INTERVAL '9 days',
  end_offset   => INTERVAL '1 hour',
  schedule_interval => INTERVAL '1 hour');

-- 8) Opcionális: az órás aggregátum tömörítése 90 nap után
-- ALTER MATERIALIZED VIEW telemetry_hourly SET (
--   timescaledb.compress,
--   timescaledb.compress_segmentby = 'site, node_id, measurement_type'
-- );
-- SELECT add_compression_policy('telemetry_hourly', INTERVAL '90 days');
```

A script egy kezdeti labor-adatbázison fut, ezért a policy-k létrehozása nem használ `if_not_exists` feltételt. Ha a scriptet újra kell futtatni, előbb távolítsuk el a korábbi policy-ket, vagy használjunk friss adatbázist. A folyamatos aggregátumok a hosszú `telemetry` tábla `measurement_type`/`value` modelljére épülnek: nincs bennük `temperature`, `humidity` vagy `soil_moisture` oszlop, hanem méréstípusonként külön sorok, a Dashboardok Grafanával alfejezet által elvárt `bucket`, `site`, `node_id`, `measurement_type`, `avg_value`, `min_value`, `max_value`, `sample_count` oszlopokkal. Az aggregátumok csak az érvényes méréseket (`valid = true`) számítják be, mert egy szenzorhibát jelző, érvénytelen érték nem torzíthatja a dashboard trendjét és a `sample_count`-ot.

A napi aggregátum átlaga nem az óránkénti átlagok egyszerű átlaga, hanem mintaszám szerint súlyozott:

$$\text{avg_value}_{\text{napi}} = \frac{\sum_i \text{avg_value}_i \cdot \text{sample_count}_i}{\sum_i \text{sample_count}_i}$$

Ez akkor is helyes, ha az egyes órákban eltérő számú minta érkezett. Ha például egy órában 1 minta van 0 °C-on, egy másikban 9 minta 10 °C-on, a helyes napi átlag $(0 \cdot 1 + 10 \cdot 9)/10 = 9{,}0$, nem pedig az órás átlagok egyszerű átlaga, az 5,0. Az idővel súlyozott átlag ettől külön fogalom: az a minták időbeli eloszlását is figyelembe veszi, és a jelen labor nem azt számítja.

Futtatás a Compose szolgáltatásnevén és a `terra` felhasználóval:

```
docker compose -f ~/iot-labor/stack/compose.yaml exec -T lab-timescaledb \
  psql -U terra -d terrasense -v ON_ERROR_STOP=1 \
  < ~/iot-labor/i046/01_retention_aggregates.sql
```

Várt kimenet: a `SELECT` parancsok visszaadják a policy-azonosítókat, a `CREATE MATERIALIZED VIEW` nem ad vissza sort, de létrehozza a nézetet. A lekérdezés:

```
SELECT * FROM timescaledb_information.jobs;
SELECT view_name, compression_enabled
FROM timescaledb_information.continuous_aggregates;
```

Ellenőrzéshez szúrjunk be néhány hosszú formátumú teszt sort. A három különböző órában három méréstípus szerepel, ezért óránként három sor keletkezik:

```
INSERT INTO telemetry
    (ts, site, node_id, seq, boot_id, schema_version,
     measurement_type, value, unit, valid)
VALUES
 (date_trunc('hour', now()) - interval '3 hours' + interval '5 minutes',
  'gh1', 'n001', 101, 'boot-1', 1, 'temperature', 22.4, 'Cel', true),
 (date_trunc('hour', now()) - interval '3 hours' + interval '5 minutes',
  'gh1', 'n001', 101, 'boot-1', 1, 'humidity', 60.1, '%RH', true),
 (date_trunc('hour', now()) - interval '3 hours' + interval '5 minutes',
  'gh1', 'n001', 101, 'boot-1', 1, 'soil_moisture', 31.0, '%', true),
 (date_trunc('hour', now()) - interval '2 hours' + interval '5 minutes',
  'gh1', 'n001', 102, 'boot-1', 1, 'temperature', 22.8, 'Cel', true),
 (date_trunc('hour', now()) - interval '2 hours' + interval '5 minutes',
  'gh1', 'n001', 102, 'boot-1', 1, 'humidity', 59.4, '%RH', true),
 (date_trunc('hour', now()) - interval '2 hours' + interval '5 minutes',
  'gh1', 'n001', 102, 'boot-1', 1, 'soil_moisture', 30.8, '%', true),
 (date_trunc('hour', now()) - interval '1 hour' + interval '5 minutes',
  'gh1', 'n001', 103, 'boot-1', 1, 'temperature', 23.1, 'Cel', true),
 (date_trunc('hour', now()) - interval '1 hour' + interval '5 minutes',
  'gh1', 'n001', 103, 'boot-1', 1, 'humidity', 58.2, '%RH', true),
 (date_trunc('hour', now()) - interval '1 hour' + interval '5 minutes',
  'gh1', 'n001', 103, 'boot-1', 1, 'soil_moisture', 30.5, '%', true)
ON CONFLICT (site, node_id, boot_id, seq, measurement_type, ts) DO NOTHING;

CALL refresh_continuous_aggregate('telemetry_hourly', now() - interval '1 day', now());
SELECT * FROM telemetry_hourly
ORDER BY bucket DESC, measurement_type
LIMIT 9;
```

Várt kimenet: három órás bucket, melyek mindegyikében három méréstípus szerepel, méréstípusonként `sample_count = 1`. A napi aggregátum manuális frissítése után hasonlóan lekérdezhető:

```
CALL refresh_continuous_aggregate('telemetry_daily', now() - interval '2 days', now());
SELECT * FROM telemetry_daily
ORDER BY bucket DESC, measurement_type
LIMIT 9;
```

A `sample_count` méréstípusonként értendő; egy órás bucketben a fenti tesztben három sor van, mérésenként egy-egy mintával.

Induló backfill, későn visszatöltött adatok és a retention–refresh sorrend. A fenti script a nézetek létrehozása után `NULL, NULL` ablakkal végzi el az induló backfillt, hogy a 15.2-ből megmaradt nyers adatok azonnal megjelenjenek az aggregátumokban. Ez a művelet nagy adatmennyiségnél hosszú ideig tarthat, ezért üzemeltetésben terhelésmentes időszakban kell futtatni. A későn érkező adatok kezelésének lépései:

1. A gateway legfeljebb 7 napot pufferel offline (lásd a Gateway üzemeltetése: systemd, konténerek, naplózás alfejezetet).
2. Az órás aggregátum frissítési ablaka `start_offset => INTERVAL '8 days'`, ami lefedi a 7 napos maximális késést és a feldolgozási időt.

3. A napi aggregátum `start_offset => INTERVAL '9 days'`, mert a napi bucket a teljes napot újraszámolja, és az órás nézet későbbi korrekcióit is át kell vennie.
4. Az `end_offset => INTERVAL '1 hour'` szándékosan kihagyja a legutolsó, még nem teljes órát; a későn érkező adat a következő órás frissítéskor, a 8 napos ablakon belül még bekerül.
5. A nyers adat megőrzése 30 nap, ami hosszabb, mint a 8/9 napos refresh-ablak, ezért a frissítéshez szükséges nyers adat mindig rendelkezésre áll.
6. Az órás aggregátum 1 éves megőrzése hosszabb, mint a napi 9 napos refresh-ablaka, ezért a napi aggregátum frissítése nem ütközik az órás adat törlésével. A napi aggregátum saját sorai 5 évig megmaradnak, mert a materializált sorok nem függnek az órás nézet 1 éves megőrzésétől.

A 7 napnál régebbi, de 30 napnál fiatalabb minta visszatöltése a következőképpen ellenőrizhető. Szúrjunk be egy 7 nappal korábbi hőmérsékletet, majd futtassuk a két nézet frissítését:

```
INSERT INTO telemetry
    (ts, site, node_id, seq, boot_id, schema_version,
     measurement_type, value, unit, valid)
VALUES
    (now() - interval '7 days', 'gh1', 'n001', 900, 'boot-1', 1,
     'temperature', 21.0, 'Cel', true);

CALL refresh_continuous_aggregate('telemetry_hourly',
    now() - interval '8 days', now());
CALL refresh_continuous_aggregate('telemetry_daily',
    now() - interval '9 days', now());

SELECT bucket, avg_value, sample_count
FROM telemetry_hourly
WHERE site = 'gh1' AND node_id = 'n001'
  AND measurement_type = 'temperature'
  AND bucket >= date_trunc('hour', now() - interval '7 days')
  AND bucket <  date_trunc('hour', now() - interval '6 days')
ORDER BY bucket;
```

A várt eredmény: a 7 nappal korábbi minta megjelenik az órás és a napi nézetben is. A visszamenőleges refresh csak a megadott ablakot számolja újra; a korábbi aggregátumok érintetlenek maradnak. Ezt úgy ellenőrizhetjük, hogy a frissítés előtt és után lementjük egy 10 napnál régebbi bucket értékét:

```
SELECT bucket, avg_value, sample_count
FROM telemetry_hourly
WHERE bucket < now() - interval '10 days'
ORDER BY bucket
LIMIT 5;
```

A két lekérdezés eredményének azonosnak kell lennie. Ha egy 30 napnál régebbi minta visszatöltése szükséges, a nyers chunk már eldobásra kerülhetett; ilyenkor a minta csak archívumból, a retention policy ideiglenes felfüggesztésével tölthető vissza, és a 30 napnál régebbi, már eldobott nyers adatot a folyamatos aggregátum sem tudja visszamenőleg korrigálni. Ezért fontos, hogy a gateway offline puffere (7 nap) és a nyers megőrzés (30 nap) összhangban legyen.

Takarítás. A labor SQL scriptjét nem kell törölni; a reprodukálhatóság miatt hagyjuk a `~/iot-labor/i046/` könyvtárban. A létrehozott policy-k és nézetek az `terra` fejlesztői sémájában maradnak, mert a dashboard használni fogja őket.

15.3.6 Ellenőrzés és hibakeresés

A működést a `timescaledb_information.jobs` táblában látható jobokkal ellenőrizzük. A `retention_policy` jobok futási ideje és a `continuous_aggregate_policy` frissítési állapota itt látható. Tipikus hibatünetek:

Tünet	Ok	Javítás
`drop_chunks` nem törli a régi chunkokat	A policy job még nem futott le, vagy a chunk tartalmaz a lejárati időnél újabb sort is	Futtassuk manuálisan: `CALL run_job(policy_job_id);` és ellenőrizzük a `drop_chunks` intervallumot
A folyamatos aggregátum nem tartalmazza a legújabb adatokat	Az aggregátum frissítése a policy `end_offset` miatt késik	Csökkentsük az `end_offset` értékét, vagy hívjuk a `refresh_continuous_aggregate`-et manuálisan
A tömörített chunkra `INSERT` hibaüzenet	A beillesztés a tömörítési intervallumnál régebbi időbélyeggel próbál írni	Helyesbítsük az időbélyeget, vagy ideiglenesen kapcsoljuk ki a tömörítést
A `CREATE MATERIALIZED VIEW` a `telemetry_hourly`-ra hivatkozó második aggregátumnál hibát ad	A folyamatos aggregátum láncolása csak bizonyos TimescaleDB verziókban támogatott	Készítsük a napi aggregátumot közvetlenül a nyers `telemetry` táblából
Az SQL `column "temperature" does not exist` hibát ad	A régi, széles táblára írt aggregátum-definíció maradt a scriptben	A 15.2 hosszú modelljében `measurement_type`/`value` oszlopok vannak; a nézetet a fenti módon kell definiálni
A napi átlag eltér a várt értéktől	Az órás átlagok egyszerű átlagát használjuk	`sum(avg_value * sample_count) / sum(sample_count)` képlet kell
A 7 nappal korábbi adat nem jelenik meg	A refresh `start_offset` értéke 1 nap, vagy a kézi refresh ablaka túl szűk	Állítsuk 8/9 napra, és a kézi refreshnél is legalább 8/9 napos ablakot használjunk
Az aggregátum érvénytelen méréseket is beszámít	Hiányzik a `WHERE valid = true` szűrő a folyamatos aggregátumból	A hosszú tábla `valid` oszlopa szerint szűrjünk; az órás és a közvetlen napi nézet is csak érvényes értéket aggregáljon
A `docker exec ts-timescaledb` konténert keres	A 15.2-ben a szolgáltatás neve `lab-timescaledb`	`docker compose -f ~/iot-labor/stack/compose.yaml exec -T lab-timescaledb ...`

15.3.7 Tipikus hibák és félreértések

1. **A megőrzési házirend a tömörítés előtt törölheti a régi nyers adatokat.** Ha a `retention_policy` intervalluma rövidebb, mint a `compression_policy` kezdete, a régi nyers chunkok tömörítetlenül elvesznek. Az általunk beállított 7 napos tömörítés és 30 napos megőrzés biztosítja, hogy minden adat legalább 7 napig nyersen, utána 23 napig tömörítve elérhető.

2. **A folyamatos aggregátumot nem frissíti automatikusan semmi.** A `CREATE MATERIALIZED VIEW WITH (timescaledb.continuous)` csak a nézetet hozza létre; az automatikus frissítéshez külön `add_continuous_aggregate_policy` szükséges. A hiánya miatt az aggregátum adatai elavulnak, és a dashboard hibás trendeket mutat.
3. **Az UTC és a `timestamptz` figyelmen kívül hagyása.** Ha a tábla oszlopa `timestamp` (time zone nélkül), a különböző időzónájú eszközök időbélyegei keverednek. A TerraSense-ben minden `ts` érték epoch ms, és a backend `timestamptz` oszlopba írja UTC-ben.
4. **A tömörített adatot utólag módosítani akarjuk.** A TimescaleDB tömörített chunkja csak olvasható. A korrekciókat a tömörítési intervallum előtt kell elvégezni, vagy a hibás adatot kompenzáló új sort kell beilleszteni.
5. **A napi átlagot az órás átlagok átlagaként számolják.** Ez csak akkor lenne helyes, ha minden órában ugyanannyi minta lenne. A valóságban a kieső szenzorok miatt az óránkénti mintaszám erősen eltérhet, ezért a mintaszám szerint súlyozott képlet szükséges. Az idővel súlyozott átlag egy további, ettől különböző fogalom.
6. **A refresh-ablak rövidebb, mint a megengedett késés.** Ha a gateway 7 napot lehet offline, egy 1 napos `start_offset` mellett a 7 nappal később érkező minta már nem kerül be az aggregátumba. A 8/9 napos ablak ezt lefedi, és összhangban van a 30 napos nyers megőrzéssel.

15.3.8 Feladatok

15.3.8.1 (a) Fogalmi kérdés

Miért nem elegendő egyetlen, nagy felbontású nyers adatréteg 5 évre, ha a TimescaleDB tömörítés mellett is kezelhető a mérete?

15.3.8.2 (b) Számolási feladat

Egy másik TerraSense-telep 5000 csomópontot üzemeltet, a mintavételi intervallum 30 másodperc, és minden üzenet három mérést tartalmaz. Egy mérési sor becsült tárolási igénye 90 B. Számítsa ki az éves nyers adatmennyiséget GB-ban, majd a 30 napos megőrzés, 5-szörös tömörítés és 1 éves órás aggregátum (60 B/sor) melletti teljes tárigényt.

15.3.8.3 (c) Implementációs feladat

Készítsen SQL scriptet a labor bővítésére: hozzon létre napi aggregátumot közvetlenül a `telemetry` táblából, és állítson be rá 5 éves megőrzést. A script mentése: `~/iot-labor/i046/02_daily_direct.sql`.

15.3.8.4 (d) Hibakeresési feladat

A `telemetry_hourly` folyamatos aggregátum a frissítési policy beállítása után sem tartalmazza a legutóbbi 45 perc adatait. Mi lehet az ok, és hogyan javítaná?

15.3.9 Megoldások

15.3.9.1 (a) Fogalmi kérdés

Bár a tömörítés csökkenti a tárolási méretet, a nyers adatok lekérdezése akkor is lassú, ha több milliárd sort kell átvizsgálni. A hosszú távú trendekhez, például az éves hőmérséklet-összehasonlításhoz az előre kiszámított napi aggregátum sokkal hatékonyabb. Emellett a jogi adatvédelmi követelmények (például a GDPR adatminimalizálási elve) is a felesleges nyers adatok korlátozott idejű tárolását írják elő. Az aggregátumok a nyers adatoktól függetlenül is megőrizhetők, mert kevesebb személyes vagy szenzitív részletet tartalmaznak.

15.3.9.2 (b) Számolási feladat

A mérési sorok száma évente:

$$N_{\text{minta}} = 5000 \cdot \frac{3600}{30} \cdot 24 \cdot 365 \cdot 3 = 15\,768\,000\,000 \text{ sor/év}$$

Éves nyers méret:

$$S_{\text{nyers}} = 15\,768\,000\,000 \cdot 90 \text{ B} = 1\,419\,120\,000\,000 \text{ B} \approx 1\,419{,}1 \text{ GB} \approx 1{,}42 \text{ TB}$$

30 napos nyers réteg:

$$S_{\text{nyers, 30 nap}} = 1\,419{,}1 \cdot \frac{30}{365} \approx 116{,}6 \text{ GB}$$

5-szörös tömörítéssel:

$$S_{\text{tömörített}} = \frac{116{,}6}{5} \approx 23{,}33 \text{ GB}$$

Az órás aggregátum 1 évre $5000 \cdot 3 \cdot 8760 = 131\,400\,000$ sor, soronként 60 B:

$$S_{\text{órás}} = 131\,400\,000 \cdot 60 \text{ B} = 7\,884\,000\,000 \text{ B} \approx 7{,}88 \text{ GB}$$

Teljes tárterület: $23{,}33 + 7{,}88 = 31{,}21$ GB. Ez jól mutatja, hogy a tömörített nyers plusz aggregátum kombináció drasztikusan csökkenti az éves tárigényt a korlátlan nyers tároláshoz (1,42 TB/év) képest.

15.3.9.3 (c) Implementációs feladat

`~/iot-labor/i046/02_daily_direct.sql`:

```
-- Közvetlen napi aggregátum a nyers telemetry táblából, hosszú formátumban
CREATE MATERIALIZED VIEW telemetry_daily_direct
WITH (timescaledb.continuous) AS
SELECT
  time_bucket('1 day', ts) AS bucket,
  site,
  node_id,
  measurement_type,
  avg(value) AS avg_value,
  min(value) AS min_value,
  max(value) AS max_value,
  count(*)   AS sample_count
FROM telemetry
WHERE valid = true
GROUP BY time_bucket('1 day', ts), site, node_id, measurement_type;

SELECT add_retention_policy('telemetry_daily_direct', INTERVAL '5 years');
SELECT add_continuous_aggregate_policy('telemetry_daily_direct',
  start_offset => INTERVAL '9 days',
  end_offset   => INTERVAL '1 hour',
  schedule_interval => INTERVAL '1 hour');
```

A futtatás hasonló a laborhoz:

```
docker compose -f ~/iot-labor/stack/compose.yaml exec -T lab-timescaledb \
  psql -U terra -d terrasense -v ON_ERROR_STOP=1 \
  < ~/iot-labor/i046/02_daily_direct.sql
```

15.3.9.4 (d) Hibakeresési feladat

A leggyakoribb ok: a `add_continuous_aggregate_policy` `end_offset` paramétere 1 óra, ami azt jelenti, hogy a legutolsó 1 óra adatait szándékosan kihagyja, hogy a későn érkező vagy módosuló nyers sorok ne ronják az aggregátumot. A `start_offset` 8 nap, tehát a rendszer minden futásnál az utolsó 8 napot nézi, de csak az 1 óránál régebbi adatokat aggregálja. Ha a 45 perces adat hiányzik, ez a beállítás okozza. Javítás: a valós idejű panelhez közvetlenül a nyers `telemetry` táblát kérdezzük; vagy csökkentsük az `end_offset` értékét például 10 percre, és fogadjuk el, hogy a legfrissebb bucket a következő frissítéskor még változhat; vagy hívjuk a `refresh_continuous_aggregate`-et manuálisan a frissítéshez. A másik lehetőség: a policy job még nem futott le, ekkor `CALL run_job(<job_id>);`.

15.3.10 Rövid összefoglaló

A TerraSense-ben a `telemetry` hypertable nyers adatait 30 napig őrizzük meg, a 7. nap után automatikus tömörítéssel. A `telemetry_hourly` folyamatos aggregátum 1 évig, a `telemetry_daily` 5 évig őrzi a mérésenkénti átlag, minimum, maximum és darabszám statisztikákat. A nézetek a 15.2 hosszú táblamodelljére épülnek, `measurement_type`/`value` oszlopokkal, és a napi átlagot mintaszám szerint súlyozva számítják. Az induló backfill `NULL, NULL` ablakkal tölti fel a nézeteket, a 7 napos offline pufferéléshez pedig 8/9 napos visszamenőleges refresh-ablak és 30 napos nyers megőrzés illeszkedik. A laborban létrehozott SQL script (`~/iot-labor/i046/01_retention_aggregates.sql`) beállítja a megőrzési házirendeket, a tömörítést és a folyamatos aggregátumokat. **Továbbvitt eredmény.** A `telemetry_hourly` és

`telemetry_daily` nézeteket a Dashboardok Grafanával alfejezet használja a hosszú távú trendek megjelenítésére; a napi aggregátum a `sample_count` mezővel az adathiányok észlelését is támogatja. A tömörítési és megőrzési paraméterek (7 nap, 30 nap, 1 év, 5 év) a TerraSense üzemeltetői konvenciói.

15.3.11 Források és ellenőrzési hiányok

A TimescaleDB folyamatos aggregátumaira, tömörítésére és megőrzési házirendjeire vonatkozó SQL-szintaxis a TimescaleDB dokumentációján alapul (https://docs.timescale.com/). A pontos függvénynevek, paraméterek és viselkedés verziófüggő lehet; a fenti kód a TimescaleDB 2.17.2 általános API-ját mutatja, éles használat előtt ellenőrizni kell a telepített verzió dokumentációját. A hierarchikus folyamatos aggregátum (órából napi) támogatása verziófüggő; ha a telepített verzió nem támogatja, a (c) feladat közvetlen napi aggregátuma használható. A tömörítési arányra vonatkozó 5–15-szörös érték illusztratív, a tényleges csökkenés az adatok redundanciájától függ. A labor SQL scriptjének, a backfillnek és a késő adatok tesztjének futtatása a lektorálási környezetben nem történt meg; a várt eredmények a sémából és a számításokból származnak. A GDPR adatminimalizálási követelményeit az IoT-biztonság és szabályozás alfejezet tárgyalja; a konkrét jogi megőrzési határidők mindig az adott szektor és joghatóság szerint változnak, ezért jogi ellenőrzést igényelnek.

16 Streamfeldolgozás és integrációs minták

16.1 Eseményvezérelt feldolgozás: Kafka és a kézbesítési szemantika

16.1.1 Motiváció

A TerraSense eddigi adatútja lineáris: a `ts-node` BLE/UDP-n küld a `ts-gw`-nek, a gateway MQTT-n publikál a Mosquitto brokerre, a `ts-backend` pedig feliratkozik a `terrasense/...` topicokra. Ez a felépítés jól működik egy üvegház, néhány tíz node és egyetlen backend esetén. A következő igények azonban gyorsan szűkössé teszik:

- **Visszajátszás:** ha a `ts-backend` órákra leáll, az MQTT retained üzenetek nem tartalmazzák a múltbeli adatfolyamot; a leállás alatt elveszett telemetriát nem lehet utólag visszaolvasni.
- **Több, független fogyasztó:** a riasztási modul (Riasztás: szabályok, értesítés, riasztási fáradtság), az idősoros betöltő (Megőrzés, ritkítás és tárolási költség) és egy későbbi analitikai modul ugyanazt az adatfolyamot akarja feldolgozni anélkül, hogy egymást zavarnák.
- **Tartós puffer:** a gateway store-and-forwardja (Store-and-forward: offline működés és visszatöltés) megoldja a peremhálózati kiesést, de a felhőben is hasznos, ha a fogyasztó sebessége időszakosan elmarad a termeléstől.

Ezekre a problémákra az MQTT broker nem elosztott napló, hanem üzenetközvetítő. A megoldás egy elosztott napló, például Apache Kafka bevezetése az MQTT mögé, amely pufferként és újraolvasható eseményfolyamként szolgál. Ebben az alfejezetben megtervezzük a hidat, és tisztázzuk a kézbesítési szemantikát. **A labor szimuláció:** nem indít MQTT-t, Kafkát, partíciót vagy consumer groupot, hanem egy JSON-listán modellezi a fogyasztó offset-, deduplikálási, ablak- és checkpoint-viselkedését. A valós Kafka-infrastruktúra kiépítése külön integrációs feladat, amelyet ez a fejezet nem bizonyít.

16.1.2 Fogalmak és működés

16.1.2.1 A Kafka alapfogalmai

Definíció. A **Kafka topic** egy elnevezett, append-only log, amelybe a producerek üzeneteket (rekordokat) írnak, és amelyből a fogyasztók olvasnak. Minden rekord egy kulcs-érték párt, időbélyeget és metaadatokat tartalmaz; a sorrend a topic szintjén nem garantált, csak partíción belül.

Definíció. A **partíció** a topic fizikai felosztása: egy topic egy vagy több partícióból áll. A producer által megadott kulcs (vagy annak hiányában egy round-robin algoritmus) határozza meg, melyik partícióba kerül a rekord. Azonos kulcsú rekordok mindig ugyanabba a partícióba íródnak, így ott a beérkezési sorrend megőrződik.

Definíció. Az **offset** egy monoton növekvő, partíción belüli pozíció azonosító, amely a rekord helyét jelöli. A fogyasztó az offset segítségével tudja, hol tart, és újraindításkor a legutóbb elkötelezett (committed) offsettől folytathatja.

Definíció. A **consumer group** fogyasztók halmaza, amely közösen dolgozza fel a topic partícióit. Kafka garantálja, hogy egy partíciót egy adott pillanatban a csoporton belül legfeljebb egy fogyasztó olvas, így a feldolgozási terhelés párhuzamosítható, miközben elkerülhető a duplikált olvasás.

Az MQTT és a Kafka szerepe gyakran összemosódik, pedig alapvetően különbözik:

Szempont	MQTT (broker)	Kafka (napló)
Elsődleges cél	üzenetközvetítés eszközöknek	tartós, visszajátszható eseményfolyam
Olvasási modell	előfizetők push alapon kapják az üzenetet	fogyasztók pull alapon kérnek rekordokat
Tárolás	retained üzenetek kivételével rövid életű	konfigurálható megőrzés (napok, hetek)
Sorrend	topic szinten általában nincs garancia	partíción belül erős garancia
Visszajátszás	nem jellemző	alapvető képesség offset alapján

16.1.2.2 MQTT → Kafka híd

A TerraSense-ben a Mosquitto brokerre érkező telemetriát továbbítani kell a Kafka topicba. Erre két általános megoldás létezik:

1. **Kafka Connect forrás-konnektor:** például egy MQTT-forrás konnektor (a pontos elérhetőségét ellenőrizni kell), amely feliratkozik az MQTT topicokra, és minden üzenetet a konfigurált Kafka topicba ír. Előnye a konfigurációalapú üzemeltetés, hátránya a korlátozott rugalmasság.
2. **Saját híd:** a `ts-backend` vagy egy külön szolgáltatás, például `ts-ingest`, MQTT-klienssel feliratkozik, majd Kafka-producerként publikál. Ez teljes kontrollt ad a séma, a particionálási kulcs és a hibakezelés felett; ezt a mintát követjük a laborban.

A híd legfontosabb tervezési döntése a **particionálási kulcs**. A TerraSense üzeneteiben a `node_id` az azonosító, amely alapján az összes node-hoz tartozó rekordnak időrendben kell feldolgozhatónak lennie. Ha a kulcs a `node_id`, akkor minden node adata ugyanabba a partícióba kerül, és a fogyasztó oldalon a `seq` mező monotonitása ellenőrizhető. Ha nem adnánk meg kulcsot, a rekordok véletlenszerű partícióba kerülnének, és a node-ra vonatkozó sorrend felborulna. A valós, több telephelyes rendszerben a kulcs `site/node_id`, hogy két telephely azonos nevű node-jai ne ütközzenek.

A híd a Kanonikus telemetria-séma szerinti JSON-t továbbítja változtatás nélkül. A laborban az események egyszerűsített, lapos nézetben szerepelnek: a `temperature` mező a kanonikus `measurements.temperature.value` vetülete, hogy a kód rövid és olvasható maradjon. A valós híd nem vetíti el a `schema_version`, `site`, `boot_id` és `measurements` mezőket.

16.1.2.3 At-least-once és idempotens fogyasztó

A Kafka alapértelmezett feldolgozási szemantikája **at-least-once**: a fogyasztó poll-ozza a rekordokat, feldolgozza őket, majd elkötelezi az offsetet. Ha a feldolgozás és a commit között a fogyasztó összeomlik, akkor újraindulás után a legutóbbi commitnál folytatja, így a már feldolgozott, de nem commit-olt rekordokat újraolvassa. Ez duplikátumokat eredményezhet.

A duplikátumok kezelésére **idempotens fogyasztót** építünk: a feldolgozás során minden rekordhoz egy egyedi kulcsot rendelünk, és egy perzisztens vagy memóriabeli halmazban nyilvántartjuk a már feldolgozott kulcsokat. Ha egy kulcs másodszor érkezik, a rekordot eldobjuk. A TerraSense-ben erre természetes kulcs a `site`, a `node_id` és a `seq` hármas, kiegészítve a `boot_id`-vel, mert újraindulás után a `seq` nulláról indulhat (lásd Idő az IoT-ban: időbélyeg, időszinkron, sorrend). Így a deduplikálási kulcs: `(site, node_id, boot_id, seq)`, ahol a hiányzó `boot_id` helyére a 15.1 szerinti `boot-0` kerül. A labor egyetlen telephelyet modellez, ezért ott a kulcs `(node_id, boot_id, seq)`; a `site` elhagyása csak a labor egyszerűsítése, nem a kanonikus séma megváltoztatása.

Az offset-kezelésnél két hibás stratégia létezik:

- **Commit a feldolgozás előtt** → at-most-once: ha a feldolgozás meghiúsul, a rekord elveszik, mert a fogyasztó újraindításkor már a következő offsettől olvas.
- **Nem commit-olunk** → a fogyasztói csoport nem tudja nyomon követni az állapotot, és újraindításkor a topic elejéről kell olvasni.

A helyes stratégia: **feldolgozás után commit**, vagyis at-least-once + idempotens feldolgozás. Ez a kombináció gyakorlatilag exactly-once hatást ad, ha a feldolgozás mellékhatása (például adatbázis-írás) is idempotens, és ha az offset, a deduplikálási állapot és az aggregátumállapot egyetlen atomi checkpointba kerül. A laborban ezt egy `checkpoint.json` fájl valósítja meg.

16.1.2.4 Ablakok, watermark, késve érkező adat

A valós idejű feldolgozásban az **ablak** (window) az események időalapú csoportosítása. A Helyi feldolgozás: szűrés, aggregálás, eseménydetektálás alfejezetben a gateway percenkénti tumbling ablakot állít elő; itt ugyanezt a fogalmat a Kafka-fogyasztó oldalán alkalmazzuk. A tumbling ablak rögzített hosszúságú (például 60 s), és az ablakok nem fedik át egymást.

Definíció. A **watermark** az a pillanat az eseményidő-tengelyen, amelynél korábbi időbélyegű eseményeket a feldolgozó már nem vár. Ha a legutóbb látott esemény időbélyege `T`, és a watermark késleltetése `W`, akkor a `T - W`-nél korábbi eseményeket késve érkezettnek tekintjük, és vagy eldobjuk, vagy oldalági kimenetre (dead letter queue, DLQ) irányítjuk.

Definíció (ablakazonosító és eszköz szerint kulcsolt állapot). Az ablakállapot nem egyetlen `current_window_start` változó, hanem olyan struktúra, amelynek kulcsa az ablak kezdete és az eszköz azonosítója. Így a nem teljesen időrendben érkező események is a saját ablakukba kerülnek, és egy késői esemény nem zárja le visszafelé az éppen aktuális ablakot.

A TerraSense-ben a késés forrása lehet a gateway store-and-forward üzemmódja: hálózati kiesés után órákkal később érkezik egy köteg régi telemetriai üzenet. A fogyasztó ezért:

1. Az ablakokat `(ablak_kezdet, site, node_id)` kulccsal tárolja.
2. A watermarkot a látott maximális eseményidőből (`max_seen_ts`) és a megengedett késésből számítja: `watermark = max_seen_ts - W`.
3. A watermarkot elért ablakokat lezárja és kiírja.

4. A watermarknál régebbi eseményt nem tesz be a már lezárt ablakba, hanem `[késő]` jelzéssel a DLQ-ba irányítja, és az offsetjét is elkötelezi, hogy ne induljon végtelen újraolvasás.
5. A watermarknál nem régebbi, de a legutóbbi maximumnál korábbi eseményt (egy még nyitott ablakba tartozó, enyhén késői eseményt) elfogad, és a saját ablakába tesz.

A watermark beállítása kompromisszum: túl kicsi érték adatvesztést okoz, túl nagy érték pedig növeli a kimenet késleltetését, mert az ablak csak a watermark átlépése után zárul le. A TerraSense-ben a 7 napos offline pufferelés miatt a watermarkot nem 5 percben, hanem a mért késés percentiliséhez igazítva kell beállítani; a laborban a 10 perces alapérték és az 5 perces kísérleti érték csak szemléltetés.

16.1.2.5 Mikor nem kell Kafka

Kis rendszerekben, ahol:

- egyetlen backend fogyasztó van,
- az adatmennyiség percenként néhány tíz üzenet,
- a fogyasztó leállásának időtartama alatt keletkező adatok elvesztése elfogadható,
- nincs szükség visszajátszásra vagy több, független fogyasztóra,

a Kafka üzemeltetése (KRaft vagy külön kontrollsík, replikáció, monitoring) aránytalan komplexitást jelent. Ilyenkor a Mosquitto retained üzenetek és a gateway store-and-forward elegendő puffert ad. A TerraSense oktatási célú, egy üvegházas példájában ez a helyzet, ezért a Kafka bevezetését ebben a könyvben csak a nagyobb, több telephelyes kiterjesztésben indokoljuk.

16.1.3 Döntési szempontok

Kritérium	MQTT önmagában	MQTT + Kafka
Fogyasztók száma	kevés, ismert	sok, dinamikusan változó
Visszajátszás igénye	nincs	van (hibakeresés, újraindexelés)
Adatmennyiség	alacsony	magas, terhelési csúcsok
Sorrendi garancia	topiconként nem garantált	partíción belül erős
Üzemeltetési komplexitás	alacsony	közepes/magas
Költség	broker csak	broker + Kafka klaszter

A TerraSense alapváltozatában az MQTT elegendő. Ha a rendszer több üvegházra, több ezer node-ra és több backend szolgáltatásra nő, akkor a Kafka bevezetése indokolt.

16.1.4 Kidolgozott példa és labor

16.1.4.1 Példa: particionálási terv és adatmennyiség

Tegyük fel, hogy 1000 node percenként 1 telemetriai üzenetet küld. Egy üzenet JSON mérete körülbelül 200 B (a mezők: `node_id`, `seq`, `ts`, `boot_id`, `schema_version`, valamint a hőmérséklet, páratartalom és talajnedvesség a `measurements` objektumban). Napi adatmennyiség:

$$1000 \cdot 60 \cdot 24 \cdot 200 \text{ B} = 288\,000\,000 \text{ B} \approx 274 \text{ MiB}.$$

Ha a Kafka topicot 4 partícióra osztjuk, minden node a `site/node_id` kulcs hash alapján kerül partícióba, így egy partícióra naponta körülbelül 69 MiB jut. Ez a mennyiség egyetlen fogyasztóval is kezelhető, de a partíciók lehetővé teszik a párhuzamos feldolgozást, ha több fogyasztót indítunk ugyanabban a consumer groupban.

A labor egy **szimulált** Kafka-fogyasztót valósít meg Pythonban, mert a teljes Kafka-stack telepítése meghaladja ennek az alfejezetnek a kereteit, és a lektorálási környezetben tilos telepíteni. A szimuláció a következőket mutatja be:

1. **Offset-kezelés:** a fogyasztó a beolvasott rekordok után commit-ol, és ha újraindul, a legutóbb commit-olt pozíciótól folytatja.
2. **Idempotens feldolgozás:** a `(node_id, boot_id, seq)` kulcsot használja a duplikátumok eldobására.
3. **Eseményidő szerinti tumbling ablak:** 60 másodperces ablakokat számol, ablakonként és node-onként kulcsolt állapottal.
4. **Watermark és késő esemény:** a maximális látott időbélyegből számított watermarkmal zárja az ablakokat, a túl régi eseményeket pedig DLQ-ba irányítja.
5. **Atomi checkpoint:** az offsetet, a dedup kulcsokat, a maximális időbélyeget, a nyitott ablakokat és a késő eseményeket egyetlen JSON-fájlba menti.

16.1.4.2 Labor: MQTT → Kafka híd szimulációja

Munkakönyvtár:

```
mkdir -p ~/iot-labor/i047 && cd ~/iot-labor/i047
```

Hozd létre az `events.json` fájlt, amely a bridge-ből a Kafka topicba kerülő rekordokat szimulálja:

```
[
  {"node_id": "n001", "boot_id": "b01", "seq": 1, "ts": 1700000000000, "temperature": 23.5},
  {"node_id": "n002", "boot_id": "b02", "seq": 1, "ts": 1700000000000, "temperature": 24.1},
  {"node_id": "n001", "boot_id": "b01", "seq": 2, "ts": 1700000030000, "temperature": 23.8},
  {"node_id": "n001", "boot_id": "b01", "seq": 3, "ts": 1700000060000, "temperature": 24.2},
  {"node_id": "n002", "boot_id": "b02", "seq": 2, "ts": 1700000040000, "temperature": 24.5},
  {"node_id": "n001", "boot_id": "b01", "seq": 2, "ts": 1700000030000, "temperature": 23.8},
  {"node_id": "n001", "boot_id": "b01", "seq": 4, "ts": 1700000090000, "temperature": 24.6}
]
```

A negyedik elemtől kezdve az események `ts` mezője nem monoton: a negyedik esemény 1700000060000, az ötödik 1700000040000 időbélyegű. Ez szándékos, hogy a feldolgozó ne az érkezési sorrendre, hanem az eseményidőre és az ablakazonosítóra támaszkodjon. A hatodik elem a harmadik duplikátuma. A `boot_id` az egyszerűség kedvéért állandó.

Hozd létre a `consumer.py` fájlt:

```
import json
import os
import tempfile

WINDOW_SIZE_MS = 60_000
MAX_LATENESS_MS = 600_000
CHECKPOINT_FILE = "checkpoint.json"
CRASH_AFTER_OFFSET = os.environ.get("I047_CRASH_AFTER_OFFSET")

def load_events(path):
    with open(path, encoding="utf-8") as f:
        return json.load(f)
```

```
def empty_state():
    return {
        "offset": 0,
        "seen": [],
        "max_seen_ts": None,
        "windows": {},
        "late_events": [],
    }

def load_checkpoint():
    if not os.path.exists(CHECKPOINT_FILE):
        return empty_state()
    try:
        with open(CHECKPOINT_FILE, encoding="utf-8") as f:
            state = json.load(f)
    except json.JSONDecodeError as exc:
        raise SystemExit(f"Hibás checkpoint: {CHECKPOINT_FILE}: {exc}") from exc
    state.setdefault("offset", 0)
    state.setdefault("seen", [])
    state.setdefault("max_seen_ts", None)
    state.setdefault("windows", {})
    state.setdefault("late_events", [])
    return state

def save_checkpoint(state):
    tmp_fd, tmp_path = tempfile.mkstemp(dir=".", prefix=".checkpoint-", suffix=".tmp")
    try:
        with os.fdopen(tmp_fd, "w", encoding="utf-8") as f:
            json.dump(state, f, ensure_ascii=False, sort_keys=True)
            f.flush()
            os.fsync(f.fileno())
        os.replace(tmp_path, CHECKPOINT_FILE)
        try:
            dir_fd = os.open(".", os.O_RDONLY)
            try:
                os.fsync(dir_fd)
            finally:
                os.close(dir_fd)
        except OSError:
            pass
    finally:
        if os.path.exists(tmp_path):
            os.unlink(tmp_path)

def checkpoint_and_maybe_crash(state, i):
    save_checkpoint(state)
    if CRASH_AFTER_OFFSET and i == int(CRASH_AFTER_OFFSET):
        print(f"  [szimulált összeomlás] offset={i} checkpointja után")
        raise SystemExit(2)

def window_start(ts):
    return (ts // WINDOW_SIZE_MS) * WINDOW_SIZE_MS

def add_to_window(state, event):
    ws_key = str(window_start(event["ts"]))
    node = event["node_id"]
    windows = state["windows"].setdefault(ws_key, {})
    agg = windows.setdefault(node, {"min": None, "max": None, "sum": 0.0, "count": 0})
    value = float(event["temperature"])
    if agg["count"] == 0:
        agg["min"] = agg["max"] = value
    else:
```

```
            agg["min"] = min(agg["min"], value)
            agg["max"] = max(agg["max"], value)
        agg["sum"] += value
        agg["count"] += 1

def close_windows(state, watermark_ms):
    for ws_key in sorted(list(state["windows"]), key=int):
        ws = int(ws_key)
        if ws + WINDOW_SIZE_MS > watermark_ms:
            continue
        for node in sorted(state["windows"][ws_key]):
            agg = state["windows"][ws_key][node]
            if agg["count"] == 0:
                continue
            avg = agg["sum"] / agg["count"]
            print(f"  [ablak] start={ws} node={node} "
                  f"min={agg['min']:.1f} max={agg['max']:.1f} avg={avg:.1f} "
                  f"count={agg['count']}")
        del state["windows"][ws_key]

def process(events):
    state = load_checkpoint()
    seen = {tuple(x) for x in state["seen"]}
    print(f"Indulás offset={state['offset']}, dedup kulcsok: {len(seen)}")

    for i, event in enumerate(events):
        if i < state["offset"]:
            continue

        key = (event["node_id"], event["boot_id"], int(event["seq"]))
        ts = int(event["ts"])

        if key in seen:
            print(f"  [duplikátum] offset={i} key={key}")
            state["offset"] = i + 1
            checkpoint_and_maybe_crash(state, i)
            continue

        max_seen_ts = state["max_seen_ts"]
        if max_seen_ts is not None and ts < max_seen_ts - MAX_LATENESS_MS:
            print(f"  [késő] offset={i} key={key} ts={ts} (max_ts={max_seen_ts})")
            seen.add(key)
            state["seen"] = [list(k) for k in sorted(seen)]
            state["late_events"].append({
                "offset": i,
                "node_id": key[0],
                "boot_id": key[1],
                "seq": key[2],
                "ts": ts,
                "max_seen_ts": max_seen_ts,
                "reason": "late",
            })
            state["offset"] = i + 1
            checkpoint_and_maybe_crash(state, i)
            continue

        seen.add(key)
        state["seen"] = [list(k) for k in sorted(seen)]
        state["max_seen_ts"] = ts if max_seen_ts is None else max(max_seen_ts, ts)
        add_to_window(state, event)
        print(f"  [feldolgozva] offset={i} key={key} ts={ts} temp={event['temperature']}")
        state["offset"] = i + 1
```

```
        if state["max_seen_ts"] is not None:
            close_windows(state, state["max_seen_ts"] - MAX_LATENESS_MS)

        checkpoint_and_maybe_crash(state, i)

    if state["max_seen_ts"] is not None:
        close_windows(state, state["max_seen_ts"] + WINDOW_SIZE_MS)
    save_checkpoint(state)
    print(f"Végső commit offset={state['offset']}, feldolgozott kulcsok: {len(state['seen'])}")

if __name__ == "__main__":
    process(load_events("events.json"))
```

Futtatás:

```
python3 consumer.py
```

Várt kimenet. A kód ellenőrző futása Python 3-mal, a fenti `events.json` bemenettel és üres checkpointból indítva a következő kimenetet adta; ez egyben megfelel az M04 numerikus ellenőrzés elvárt értékeinek (végső offset 7, hat egyedi kulcs, első ablak 1699999980000, első n001 átlag 23.6, utolsó n001 átlag 24.4):

```
Indulás offset=0, dedup kulcsok: 0
  [feldolgozva] offset=0 key=('n001', 'b01', 1) ts=1700000000000 temp=23.5
  [feldolgozva] offset=1 key=('n002', 'b02', 1) ts=1700000000000 temp=24.1
  [feldolgozva] offset=2 key=('n001', 'b01', 2) ts=1700000030000 temp=23.8
  [feldolgozva] offset=3 key=('n001', 'b01', 3) ts=1700000060000 temp=24.2
  [feldolgozva] offset=4 key=('n002', 'b02', 2) ts=1700000040000 temp=24.5
  [duplikátum] offset=5 key=('n001', 'b01', 2)
  [feldolgozva] offset=6 key=('n001', 'b01', 4) ts=1700000090000 temp=24.6
  [ablak] start=1699999980000 node=n001 min=23.5 max=23.8 avg=23.6 count=2
  [ablak] start=1699999980000 node=n002 min=24.1 max=24.1 avg=24.1 count=1
  [ablak] start=1700000040000 node=n001 min=24.2 max=24.6 avg=24.4 count=2
  [ablak] start=1700000040000 node=n002 min=24.5 max=24.5 avg=24.5 count=1
Végső commit offset=7, feldolgozott kulcsok: 6
```

Az ablakok `ts` alapján zárulnak, a duplikátumot a rendszer eldobta. A két ablak kezdete 1699999980000 és 1700000040000; az első ablakba a 1700000000000 és 1700000030000 időbélyegű események, a másodikba a 1700000040000, 1700000060000 és 1700000090000 időbélyegűek tartoznak. Az ötödik (duplikált) esemény nem növeli a `n001` darabszámát.

A `checkpoint.json` a teljes futás után:

```
{
  "late_events": [],
  "max_seen_ts": 1700000090000,
  "offset": 7,
  "seen": [["n001", "b01", 1], ["n001", "b01", 2], ["n001", "b01", 3],
           ["n001", "b01", 4], ["n002", "b02", 1], ["n002", "b02", 2]],
  "windows": {}
}
```

A `seen` hat kulcsot tartalmaz, mert a hét bemeneti rekordból egy duplikátum. A `windows` a futás végén üres, mert az összes ablak lezárult. Ha újraindítod a szkriptet, a checkpoint miatt az összes rekordot kihagyja:

```
Indulás offset=7, dedup kulcsok: 6
Végső commit offset=7, feldolgozott kulcsok: 6
```

Újrafuttatás. A labor nem igényel törlést. Újrafuttatáshoz használj új munkakönyvtárat, vagy nevezd át a checkpointot: `mv checkpoint.json checkpoint.json.bak`. A `checkpoint.json` a labor értékes állapota, ezért ne töröld.

16.1.5 Ellenőrzés és hibakeresés

A labor futtatása után ellenőrizd:

- a `checkpoint.json` `offset` értéke 7, a `seen` hat egyedi kulcsot tartalmaz;
- a négy `[ablak]` sor értékei a fenti kimenetnek felelnek meg;
- a duplikátum (az ötödik esemény) nem jelenik meg a `[feldolgozva]` sorok között, csak a `[duplikátum]` sorban;
- újraindításkor a fogyasztó nem dolgozza fel újra a rekordokat;
- a `windows` a futás végén üres, a `late_events` pedig üres az eredeti bemeneten.

Összeomlás-teszt. A kód tartalmaz egy laboratóriumi crash-hookot: az `I047_CRASH_AFTER_OFFSET` környezeti változóval a megadott offset checkpointja után a program szándékosan leáll. Ez nem éles funkció, csak a tartósság szemléltetésére szolgál. Futtasd:

```
I047_CRASH_AFTER_OFFSET=3 python3 consumer.py
```

A kimenet a negyedik rekord után megáll, a `checkpoint.json` `offset` értéke 4, a `windows` pedig tartalmazza az addig feldolgozott két ablak részaggregátumait. Indítsd újra a programot:

```
python3 consumer.py
```

A folytatás kimenete:

```
Indulás offset=4, dedup kulcsok: 4
  [feldolgozva] offset=4 key=('n002', 'b02', 2) ts=1700000040000 temp=24.5
  [duplikátum] offset=5 key=('n001', 'b01', 2)
  [feldolgozva] offset=6 key=('n001', 'b01', 4) ts=1700000090000 temp=24.6
  [ablak] start=1699999980000 node=n001 min=23.5 max=23.8 avg=23.6 count=2
  [ablak] start=1699999980000 node=n002 min=24.1 max=24.1 avg=24.1 count=1
  [ablak] start=1700000040000 node=n001 min=24.2 max=24.6 avg=24.4 count=2
  [ablak] start=1700000040000 node=n002 min=24.5 max=24.5 avg=24.5 count=1
Végső commit offset=7, feldolgozott kulcsok: 6
```

Az első ablak `n001` sorában a 23.5 és 23.8 érték megmaradt, holott azokat a megszakítás előtt dolgozta fel a program. Ez bizonyítja, hogy az aggregátumállapot a checkpointban van, nem csak a memóriában. A folytatás nem dolgozta újra az offset 0–3 rekordjait, mert a checkpoint offsetje 4 volt.

Hibatünet	Ok	Javítás
A fogyasztó újraindúláskor mindent újra feldolgoz	A checkpoint nem jött létre, vagy a commit a feldolgozás előtt történt	Ellenőrizd a `save_checkpoint` hívás helyét; a commit csak a feldolgozás és az ablakfrissítés után menjen
Duplikátumok jelennek meg az adatbázisban	A dedup kulcs nem tartalmazza a `boot_id` mezőt	Egészítsd ki a kulcsot `(node_id, boot_id, seq)`, a valós rendszerben `(site, node_id, boot_id, seq)` formátumra
Az ablakok rossz időponthoz tartoznak	Az elemzés a feldolgozási időt használja az eseményidő helyett	Mindig az üzenet `ts` mezőjét használd az ablakok számításához

Hibatünet	Ok	Javítás
Egy késői esemény visszafelé lezárja az aktuális ablakot	Egyetlen `current_window_start` változó van	Kulcsold az állapotot `(ablak_kezdete, node_id)` szerint
Késve érkező adat beszámít a régi ablakba	Nincs watermark, vagy a watermarknál régebbi eseményt is elfogad a kód	Vezess be watermarkot, és a watermarknál régebbi eseményeket irányítsd DLQ-ba
Az offset előrehalad, de az aggregátum elveszik	Az offsetet és a dedup/aggregátum állapotot külön fájlba mented	Egyetlen atomi checkpointba írd az offsetet, a `seen` halmazt, a `max_seen_ts`-t, a nyitott ablakokat és a késő eseményeket
A késő esemény végtelen újraolvasást okoz	A késő eseményt eldobod, de az offsetjét nem commit-olod	A késő esemény offsetjét is commit-old, és a DLQ-beírást is vedd bele a checkpointba
A checkpoint félkész JSON	Nem atomikusan írod a fájlt	Írj ideiglenes fájlba, `fsync` után `os.replace`-szel nevezd át
A `seen` halmaz korlátlanul nő	A valós fogyasztó minden kulcsot örökké tárol	Éles rendszerben compacted Kafka topic vagy TTL-es adatbázis-tábla kell; a labor checkpointja csak oktatási célú

16.1.6 Tipikus hibák és félreértések

1. **At-most-once a feldolgozás előtti commit miatt.** Ha a fogyasztó azonnal commit-ol a poll után, de a feldolgozás meghiúsul, az üzenet végleg elveszik. A TerraSense-ben ez elfogadhatatlan, mert a szenzoradatok nem pótolhatók. Mindig a feldolgozás után commit-olj, és gondoskodj idempotens feldolgozásról.

2. **A particionálási kulcs elhagyása.** Ha a híd nem ad meg kulcsot, a Kafka producere véletlenszerűen osztja el a rekordokat a partíciók között. Ekkor az azonos `node_id`-hez tartozó rekordok sorrendje felborul, és a `seq` mező alapján történő ellenőrzés hamis hibát jelez. A kulcs mindig a `site/node_id` legyen.

3. **A watermark túl szorosra állítása.** Ha a watermark késleltetése kisebb, mint a valós késés (például 5 perc, miközben a gateway offline üzemmódban órákig pufferel), akkor a legtöbb késve érkező adat kiesik. A watermarkot a mért késés percentiliséhez kell igazítani, nem az átlaghoz. A labor 10 perces alapértéke és 5 perces kísérleti értéke csak a mechanizmust szemlélteti, nem üzemi javaslat.

4. **Az MQTT és a Kafka fogalmainak keverése.** Az MQTT-ben a topic hierarchikus és joker karakterekkel rendelkezik; a Kafka topic lapos, és a partíciók adják a párhuzamosságot. A kettőt nem lehet közvetlenül megfeleltetni egymásnak, a hídnak explicit módon kell leképeznie az MQTT topicstruktúrát a Kafka topicra és a kulcsra.

5. **Egyetlen `current_window_start` használata.** A nem teljesen időrendű eseménysor mellett az egyetlen aktuális ablakváltozó visszafelé lezárja az éppen nyitott ablakot, majd újranyit egy régit. Az eredmény ugyanarra az ablakra több, szétesett részeredmény. Az állapotot ablakazonosító és eszköz szerint kell kulcsolni.

6. **Az offset és a dedup/aggregátum állapot külön mentése.** Ha az offsetet az egyik fájlba, a `seen` halmazt a másikba, az aggregátumot pedig sehova nem mented, akkor egy összeomlás után az offset már előrébb mutat, de a korábbi kulcsok és részaggregátumok hiányoznak. Az eredmény hamis „kész” offset és elveszett aggregátum. A megoldás az egyetlen, atomikusan cserélt checkpoint.
7. **A `seen` halmaz örök növekedése.** A laborban a checkpoint a teljes `seen` halmazt tárolja, mert kevés esemény van. Éles rendszerben ez korlátlanul nő; ott compacted Kafka topicot, adatbázis-táblát TTL-lel vagy ablakos deduplikációt kell használni. A labor megoldása nem skálázható minta, hanem a szemantika bemutatása.

16.1.7 Feladatok

16.1.7.1 a) Fogalmi/döntési kérdés

Melyik esetben indokolt Kafkát bevezetni a TerraSense-be, és melyikben elegendő a Mosquitto? Válaszodat a visszajátszás, a fogyasztók száma és az adatmennyiség szempontjából indokold.

16.1.7.2 b) Számolási/tervezési feladat

Tegyük fel, hogy 1000 node percenként 1 üzenetet küld, egy üzenet 150 B. Hány partícióra osszuk a `ts-telemetry` topicot, ha a fogyasztói csoport 4 fogyasztót futtat, és a cél, hogy minden fogyasztó közel azonos terhelést kapjon? Mekkora a napi tárhelyigény 7 napos megőrzés mellett? (A replikációs faktor 2.)

16.1.7.3 c) Implementációs feladat

A labor kódja már tartalmazza a watermarkot és a késő események eldobását 10 perces késleltetéssel. Módosítsd a `MAX_LATENESS_MS` értékét 300 000-re (5 perc), egészítsd ki az `events.json`-t a következő eseménnyel, és igazold, hogy a korábbi négy ablak változatlan, a késő esemény `[késő]` jelzést kap, az offset viszont 8-ra nő:

```
{"node_id": "n001", "boot_id": "b01", "seq": 5, "ts": 1699999700000, "temperature": 21.0}
```

A labor futtatásakor ellenőrizd, hogy a késő esemény nem kerül be egyik ablakba sem, és hogy a checkpoint `late_events` listája tartalmazza.

16.1.7.4 d) Hibakeresési/kiterjesztési feladat

A jelenlegi fogyasztó egyetlen `checkpoint.json` fájlba menti az offsetet, a `seen` halmazt, a `max_seen_ts`-t, a nyitott ablakokat és a késő eseményeket. Tervezd meg, hogyan biztosítja ez az összeomlás utáni helyes folytatást. Miért nem elegendő az offsetet és a dedup halmazt külön fájlban tárolni? Mit jelent az, hogy a kimenet legalább egyszeres (at-least-once), és hogyan tehető idempotenssé a fogadó oldal?

16.1.8 Megoldások

16.1.8.1 a) Megoldás

Kafka akkor indokolt, ha:

- a `ts-backend` leállása alatt keletkező adatok utólagos visszajátszása követelmény;
- több, független fogyasztó (például riasztás, idősoros tárolás, analitika) dolgozza fel ugyanazt az adatfolyamot;
- a bejövő adatmennyiség meghaladja az egyetlen fogyasztó feldolgozási sebességét, és a pufferelés elengedhetetlen.

Egyetlen üvegház, néhány tíz node és egy backend esetén a Kafka üzemeltetése felesleges költséggel jár, a Mosquitto retained üzenetek és a gateway store-and-forward elegendő. A döntést a visszajátszási igény, a fogyasztók száma és az adatmennyiség együttesen határozza meg; egyik tényező önmagában nem elegendő.

16.1.8.2 b) Megoldás

A partíciók számának legalább a fogyasztók számával egyenlőnek kell lennie, hogy minden fogyasztó kapjon partíciót. 4 fogyasztó esetén a minimális partíciószám 4, de a terhelés kiegyensúlyozásához és a jövőbeli skálázhatósághoz érdemes 8 vagy 16 partíciót választani. A gyakorlatban a partíciószámot a csúcsterhelés és a fogyasztói párhuzamosság alapján választjuk; 8 partíció elegendő. A partíciószámot utólag nehéz csökkenteni, ezért érdemes a várható növekedést is figyelembe venni.

Napi üzenetmennyiség: $1000 \cdot 60 \cdot 24 = 1\,440\,000$ üzenet. Egy üzenet 150 B, tehát napi nyers adat:

$$1\,440\,000 \cdot 150 \text{ B} = 216\,000\,000 \text{ B} \approx 206 \text{ MiB}.$$

Replikációs faktor 2 miatt a tényleges tárigény kétszeres: 412 MiB. 7 napos megőrzés:

$$412 \text{ MiB} \cdot 7 \approx 2{,}8 \text{ GiB}.$$

Ez nem tartalmazza a Kafka belső indexeit, ezért a valós igény ennél 10–20%-kal nagyobb lehet, azaz körülbelül 3,1–3,4 GiB.

16.1.8.3 c) Megoldás

A módosítás két részből áll. Először a késleltetési állandó:

```
MAX_LATENESS_MS = 300_000
```

Másodszor az `events.json` kiegészítése. Az utolsó előtti elem után vessző kell, az új elem a lista vége:

```
  {"node_id": "n001", "boot_id": "b01", "seq": 4, "ts": 1700000090000, "temperature": 24.6},
  {"node_id": "n001", "boot_id": "b01", "seq": 5, "ts": 1699999700000, "temperature": 21.0}
```

A kód érintetlen része már tartalmazza a szükséges logikát: a `max_seen_ts` inicializálása `None`, minden elfogadott eseménynél `max_seen_ts = max(max_seen_ts, ts)`, a késő esemény ága pedig a `seen` halmazba veszi a kulcsot, a `late_events` listába írja a rekordot, majd az offsetet is előreléptei és checkpointol. A késő esemény tehát nem okoz végtelen újraolvasást, mert az offsetje commitolódik, és a döntés a checkpointban is rögzítődik.

Az ellenőrző futás kimenete a négy ablakra változatlan, a késő eseményre pedig:

```
  [késő] offset=7 key=('n001', 'b01', 5) ts=1699999700000 (max_ts=1700000090000)
  [ablak] start=1699999980000 node=n001 min=23.5 max=23.8 avg=23.6 count=2
  [ablak] start=1699999980000 node=n002 min=24.1 max=24.1 avg=24.1 count=1
  [ablak] start=1700000040000 node=n001 min=24.2 max=24.6 avg=24.4 count=2
  [ablak] start=1700000040000 node=n002 min=24.5 max=24.5 avg=24.5 count=1
Végső commit offset=8, feldolgozott kulcsok: 7
```

A `late_events` tartalma:

```
[{"boot_id": "b01", "max_seen_ts": 1700000090000, "node_id": "n001",
  "offset": 7, "reason": "late", "seq": 5, "ts": 1699999700000}]
```

A 10 perces alapértékkel ugyanez az esemény még elfogadható lenne, és egy új, 1699999680000 kezdetű ablakot nyitna 21.0 átlaggal; az 5 perces értékre váltás mutatja, hogy a watermark szigorítása a késő eseményt DLQ-ba irányítja. A négy eredeti ablak egyikében sem változik meg az átlag vagy a darabszám, mert a késő esemény nem kerül be egyik már lezárt vagy nyitott ablakba sem. A késő esemény kulcsa bekerül a `seen` halmazba, ezért a záró „feldolgozott kulcsok” száma 7; ez a deduplikációs állapot darabszáma, nem az ablakokba beszámított események száma.

16.1.8.4 d) Megoldás

A `checkpoint.json` egyetlen dokumentumban tartalmazza mindazt az állapotot, amelyre a folytatáshoz szükség van:

Mező	Jelentés	Miért kell a checkpointba
`offset`	a következő feldolgozandó rekord sorszáma	ebből tudja a fogyasztó, hol folytassa
`seen`	a már látott `(node_id, boot_id, seq)` kulcsok	a commit utáni újraolvasás deduplikálásához
`max_seen_ts`	a látott legnagyobb eseményidő	a watermark és a késő események küszöbének újraszámításához
`windows`	a nyitott ablakok részaggregátumai	a már feldolgozott, de még le nem zárt ablakok megőrzéséhez
`late_events`	a DLQ-ba irányított késő események	a döntés naplózásához és az offset előreléptetésének igazolásához

A mentés atomi: a kód először egy ideiglenes fájlba ír, `flush` és `fsync` után `os.replace`-szel cseréli le a `checkpoint.json`-t, majd a könyvtárat is `fsync`-eli. A `os.replace` ugyanazon a fájlrendszeren atomi, ezért a fogyasztó vagy a régi, vagy az új checkpointot látja, félkész állapotot nem. Ha az összeomlás a mentés előtt történik, a régi checkpoint marad érvényben, és a rekord

újra feldolgozódik; ha a mentés után, akkor az új offset és a hozzá tartozó ablakállapot együtt érvényes. A kettő nem csúszhat el egymáshoz képest.

Az offset és a dedup halmaz külön fájlban nem elegendő, mert a két fájl írása nem atomi együtt: az egyik már az új állapotot tartalmazhatja, a másik még a régit. Ha például az offset fájlja előbb frissül, majd a fogyasztó összeomlik a `seen` mentése előtt, akkor újraindításkor az offset már a következő rekordra mutat, de a korábbi kulcsok hiányoznak a `seen` halmazból, így a duplikátumok átszivároghatnak. Ugyanez a probléma súlyosabb az aggregátumokkal: ha az offset előrébb jár, mint a részaggregátum, a már commitolt események hozzájárulása végleg elveszik. A megoldás vagy egyetlen atomi állapotfájl, vagy tranzakciós adatbázis-tároló, vagy az offset és az állapot együtt írása ugyanabba a tranzakcióba.

A kimenet at-least-once: összeomlás esetén előfordulhat, hogy egy már kiszámított ablakot a folytatás újra kiír, mert a kimenet és a checkpoint írása nem egyetlen atomi művelet. Ez nem adatvesztés, hanem duplikált kimenet. A fogadó oldal ezért idempotens: az ablakokat `(ablak_kezdete, site, node_id)` kulccsal upserteli, vagy a kimeneti rekordot egyedi kulccsal látja el, így az ismételt kiírás nem duplikálja az adatot. Ha a fogadó oldal nem idempotens, a kimenet és a checkpoint együttes tranzakciója szükséges, például egy adatbázisban.

A crash-teszt ezt szemlélteti: az `I047_CRASH_AFTER_OFFSET=3` futtatás után a checkpoint `offset` értéke 4, a `windows` pedig tartalmazza az első ablak 23.5 és 23.8 értékét. A folytatás az offset 4-től indul, nem dolgozza újra a 0–3 rekordokat, és az első ablakot a megszakítás előtti értékekkel zárja le. Ha a kimenet újra kiíródik, a fogadó oldal idempotens kulcsa gondoskodik arról, hogy ne keletkezzen duplikált aggregátum.

16.1.9 Rövid összefoglaló

Az MQTT broker üzenetközvetítő, a Kafka pedig elosztott, visszajátszható napló. A TerraSenseben nagyobb rendszer esetén a Mosquitto mögé Kafka topic kerül, ahol a particionálási kulcs a `site/node_id`, így minden node üzenetei sorrendben maradnak. A híd a `ts-ingest` vagy a `ts-backend` MQTT-klienséből és Kafka-produceréből áll. A fogyasztó at-least-once szemantikát használ, commit-ot a feldolgozás után, és idempotens deduplikálást `(site, node_id, boot_id, seq)` kulccsal. Az eseményidő szerinti tumbling ablakok 60 másodpercesek, az állapot `(ablak_kezdete, node_id)` szerint kulcsolt, a watermark pedig a maximális látott időbélyegből és a megengedett késésből számított. A késő események DLQ-ba kerülnek, az offsetjük mégis commitolódik. Az offset, a dedup halmaz, a vízjel és a nyitott ablakok egyetlen atomi `checkpoint.json`-ba kerülnek. **Továbbvitt eredmény.** A labor szimuláció, amely nem indít MQTT-t, Kafkát, partíciót vagy consumer groupot, és nem bizonyítja a valós Kafka-híd létét; a `ts-telemetry` topic, a 4 partíció és a `ts-backend` consumer group a tervezett éles konfiguráció. A labor kódjának ellenőrző futása az M04 numerikus ellenőrzés értékeit adta: végső offset 7, hat egyedi kulcs, első ablak 1699999980000, első n001 átlag 23.6, utolsó n001 átlag 24.4. A valós Kafka-integráció külön labor lenne, benne a broker, a partíciók, a consumer group és az offset/ack tesztek tényleges futtatásával.

16.1.10 Források és ellenőrzési hiányok

- Apache Kafka dokumentáció: topicok, partíciók, consumer groupok és offset-kezelés (alapfogalmak, stabil).
- Enterprise Integration Patterns: messaging minták, idempotens fogyasztó (a könyvben hivatkozott, de nem ellenőrzött forrás).
- MQTT 5 specifikáció: topicok és retain viselkedés (lásd MQTT: publish/subscribe, broker, topicok–MQTT QoS, munkamenetek, retain, Last Will és MQTT 5).

- A laborban használt Python kód szimuláció, nem kapcsolódik tényleges Kafka-hoz. Az ellenőrző futás Python 3-mal, a fenti `events.json` bemenettel, üres checkpointból, valamint az `I047_CRASH_AFTER_OFFSET=3` megszakítási teszttel megtörtént; a könyvben közölt kimenetek ebből a futásból származnak. A valós Kafka-kliens API (például Java `KafkaConsumer`) és a Kafka Connect MQTT-konnektor pontos konfigurációja gyorsan változik; ellenőrizendő a telepített verzió dokumentációja. A particionálási kulcs hash algoritmusa a producer beállításától függ, ezt a TerraSense-ben explicit módon kell konfigurálni.
- A 10 perces és az 5 perces watermark-érték oktatási szemléltetés, nem üzemi javaslat. A TerraSense 7 napos offline pufferelése miatt a valós watermarkot a mért késés percentiliséhez kell igazítani.
- A `seen` halmaz és a teljes ablakállapot checkpointja a labor méretében működik; éles rendszerben compacted topic, TTL-es adatbázis-tábla vagy elosztott állapottároló szükséges. Ennek méretezését és üzemeltetését ez a fejezet nem tárgyalja.
- A valós MQTT → Kafka híd, a Kafka broker, a partíciók és a consumer group működése nem került kiépítésre és nem került tesztelésre; a labor kizárólag a feldolgozási szemantikát modellezi.

16.2 Enterprise integrációs minták IoT-ban

A TerraSense eddig befelé dolgozott: MQTT-ből olvasott, idősorba írt, REST API-t adott. Amint vállalati környezetbe kerül, megjelennek a kifelé irányuló kapcsolatok: a karbantartási rendszernek (CMMS) munkalapot kell nyitni túl magas hőmérséklet esetén, az ERP-nek összesítőt kell küldeni, egy riasztási felület webhookot vár. Ha ezeket a kapcsolatokat egyedi, pontonkénti kóddal oldjuk meg, a backend hamar áttekinthetetlenné válik, a hibakezelés pedig szétszóródik. Az Enterprise Integration Patterns (EIP) célja, hogy az ilyen üzenetfeldolgozási lépéseket újrafelhasználható, jól elnevezett építőelemekre bontsa.

Ebben az alfejezetben az Eseményvezérelt feldolgozás: Kafka és a kézbesítési szemantika alfejezetben bevezetett csatorna- és kézbesítési szemléletet visszük tovább: a mintákat a TerraSense konkrét adatútján alkalmazzuk, és külön hangsúlyt kap az outbox, amely a JEE-ből Spring Bootba: fogalmi híd C/C++ fejlesztőknek alfejezetben megismert Spring-tranzakciókkal dolgozik együtt. A fejezet kódja egy minimális, önállóan forduló Spring Boot 3.3.4 modul; a Maven-futtatás a lektorálási környezetben nem történt meg, a kód statikusan ellenőrzött. A labor a hallgatói környezetben futtatható.

16.2.1 Fogalmak és működés

Definíció (üzenet). Az üzenet (*message*) önálló, továbbítható adatcsomag, amely fejlécből (*header*) és törzsből (*body*) áll. A fejléc hordozza az azonosítókat, az időbélyeget, a típust és a kézbesítési metaadatokat; a törzs a tényleges adat.

Definíció (csatorna). A csatorna (*message channel*) logikai cím, amelyen keresztül az alkalmazások üzenetet küldenek és fogadnak. A csatorna elválasztja a küldőt a fogadótól: a küldő nem ismeri a fogadó címét, csak a csatorna nevét. MQTT-ben a topic, Kafka-ban a topic, JMS-ben a destination tölti be ezt a szerepet.

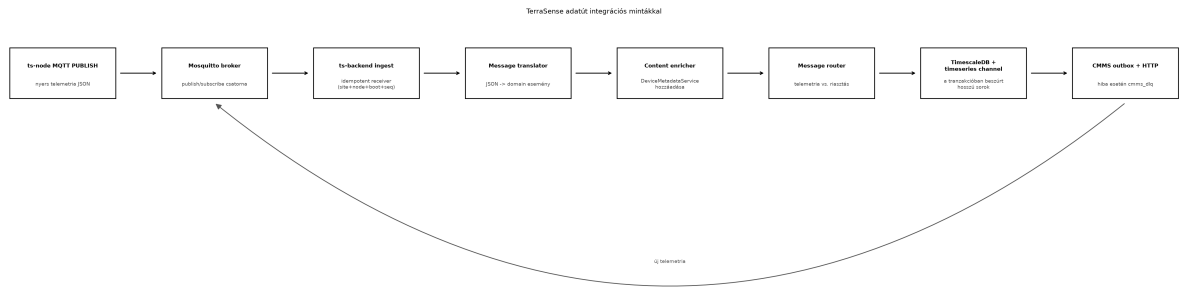
Definíció (Message és MessageBuilder). A Spring Integration `MessageChannel.send` metódusa `Message<?>` objektumot vár, nem nyers domain objektumot. A `MessageBuilder` állítja elő a `Message` példányt a törzsből és a fejlécekből; a routerben ezért minden kimenet `MessageBuilder`-rel készül. A `DirectChannel` szinkron hívja a feliratkozott handlert, ezért a hívó szál tranzakciója a handlerig terjed.

A gyakorlati integrációkban az üzenetek ritkán érkeznek pontosan abban a formában, ahogy a fogadó várja. A leggyakoribb feldolgozási minták:

- **Üzenetfordító (*message translator*):** az üzenet reprezentációját alakítja át két rendszer között. Például a ts-node által küldött JSON telemetriát a ts-backend domain `TelemetryEvent` rekorddá fordítja, vagy a CMMS felé egy másik JSON-sémát állít elő.
- **Üzenetirányító (*message router*):** egy bejövő üzenetet több kimeneti csatorna valamelyikére irányít feltétel alapján, anélkül hogy a törzset módosítaná. A TerraSense-ben a router dönti el, hogy egy telemetria-üzenet csak az idősoros tárolóba, vagy riasztásként is a megfelelő csatornára kerül.
- **Tartalmi kiegészítő (*content enricher*):** a bejövő üzenetet külső adatforrásból származó információval egészíti ki. A ts-node csak `node_id` és `site` azonosítót küld; a backend a `DeviceMetadataService`-ből kiegészíti az üzenetet a telepítési hellyel, az eszköztípussal és a karbantartási felelőssel.
- **Holtlevél-csatorna (*dead letter channel*):** olyan csatorna vagy tábla, ahová a sikeresen feldolgozhatatlan vagy kézbesíthetetlen üzenetek kerülnek. Nem a hibák elrejtésére való, hanem arra, hogy az üzenet ne vesszen el, és utólag elemezhető vagy újra feldolgozható legyen. A TerraSense-ben a CMMS-hez tartozó DLQ a `cmms_dlq` adatbázistábla, nem külön MQTT-topic.
- **Idempotens fogadó (*idempotent receiver*):** olyan fogadó, amely az üzenet azonosítóját megőrzi, és a duplikátumokat eldobja vagy egyesíti. Ez azért kritikus, mert az at-least-once kézbesítés — amint az Eseményvezérelt feldolgozás: Kafka és a kézbesítési szemantika alfejezetben láttuk — többszöri kézbesítést okozhat.
- **Outbox minta:** a szolgáltatás a saját tranzakciós adatbázisában, ugyanabban a tranzakcióban, mint az üzleti adatot, egy kimenő üzenettáblába írja a küldendő üzenetet. Egy külön folyamat olvassa ki a táblát, továbbítja az üzenetet a csatornára, és csak ezután jelöli kézbesítettnek. Így elkerülhető a kettős írás (*dual write*): nem fordulhat elő, hogy az adatbázis-írás sikeres, de a HTTP-hívás vagy üzenetküldés meghiúsul, és a két rendszer állapota elválik.

Az outbox tábla állapotgépe a TerraSense-ben: `PENDING` → `PROCESSING` → `SENT`, illetve `PENDING` → `PROCESSING` → `PENDING` (újrapróbálkozás) vagy `DEAD` (végleges hiba, DLQ). A `PROCESSING` állapot azért kell, hogy a publisher összeomlása után a rekord ne maradjon örökre „kész” állapotban: egy helyreállító lekérdezés az öt percnél régebbi `PROCESSING` rekordokat `PENDING`-re állítja, és a következő kör újra elküldi őket. Az újraküldés biztonságos, mert a `work_order_id` stabil idempotencia-kulcs.

Ezek a minták egymásra épülnek. A TerraSense adatútját az alábbi ábra foglalja össze.



16.1. ábra: TerraSense adatút integrációs mintákkal - Az üzenet útja az eszköztől a vállalati rendszerig; a zárójelben az EIP-minta neve szerepel.

A folyamatábra lépései: az eszköz a brokerre küld, ahonnan a backend fogadja; az ingest idempotensen dolgozik a `(site, node_id, boot_id, seq)` kulccsal; a translator a nyers JSON-ból domain eseményt készít; az enricher hozzáadja a metaadatot; a router szétválasztja a normál telemetriát és a riasztást; az idősoros beszúrás és a CMMS-outbox ugyanabban a tranzakcióban történik; végül a publisher HTTP-n továbbítja a munkalapot, hiba esetén a `cmms_dlq` táblába.

16.2.2 Döntési szempontok

A minták nem mindig szükségesek, és a túlzott használatuk is káros. Az alábbi táblázat a leggyakoribb döntési helyzeteket foglalja össze.

Helyzet	Ajánlott minta	Indoklás és kompromisszum
Két rendszer eltérő adatformátumot vár	Message translator	A fordítás egy helyen marad; a küldő és a fogadó nem változik, de a translator újabb karbantartandó komponens.
Több fogadó eltérő érdeklődéssel	Message router vagy publish-subscribe	A broker vagy router szétválasztja a felelősségeket; a router feltételes logikája egyszerűbb, de a szabályok növekedésével külön szabálymotor kellhet.
Az üzenetből hiányzik a feldolgozáshoz szükséges adat	Content enricher	Külső hívás szükséges, ezért gyorsítótár és időtúllépés nélkül lassíthatja az adatutat.
Külső HTTP hívás sikertelen	Dead letter channel + újrapróbálkozás	Nem veszítjük el az üzenetet, de a DLQ-t figyelni kell, különben csendben gyűlik a hiba.
Duplikátumok at-least-once kézbesítésnél	Idempotent receiver	Az üzenetazonosító alapú deduplikáció garantálja az egyszeri hatást; a kulcs megválasztása üzleti döntés.
A backend adatbázis-írása és üzenetküldése konzisztens kell legyen	Outbox	Ugyanabban a tranzakcióban írjuk az adatot és a kimenő üzenetet; a késleltetés nő, de a konzisztencia garantált.
A csatorna szinkron vagy aszinkron	DirectChannel, illetve ExecutorChannel	A `DirectChannel` megtartja a tranzakciót, de blokkolja a hívót; az `ExecutorChannel` leválasztja, de a tranzakció nem öröklődik, ezért külön tranzakciós outbox kell.

A teljes értékű ESB-termékek bemutatása kívül esik ezen alfejezet keretein; a fenti minták könnyű súlyú, alkalmazásba ágyazott megoldások, amelyek a TerraSense méretéhez illeszkednek.

16.2.3 Kidolgozott példa

Vegyünk egy konkrét telemetria-üzenetet, amely a `terrasense/gh1/n001/telemetry` topicra érkezik. A Ingest: MQTT-ből a backendbe, idempotensen alfejezet kanonikus sémáját használjuk; a hőmérséklet 31,2 °C, a talajnedvesség 18,0%, tehát mindkét riasztási feltétel teljesül:

```
{
  "schema_version": 1,
  "site": "gh1",
  "node_id": "n001",
  "seq": 42,
  "ts": 1717171200000,
  "boot_id": "b01",
  "measurements": {
    "temperature": {"value": 31.2, "unit": "Cel", "valid": true},
    "humidity": {"value": 55.0, "unit": "%RH", "valid": true},
    "soil_moisture": {"value": 18.0, "unit": "%", "valid": true}
  }
}
```

A `ts-backend` MQTT ingest komponense először idempotens fogadóként működik: a `(site, node_id, boot_id, seq)` kulcsot a `telemetry_ingest` táblában tárolja. Ha a kulcs már szerepel, az üzenet duplikátum, és a feldolgozás leáll. A Idősoros adatbázisok és adatmodellezés alfejezet repositorya a `telemetry_ingest` eseménytáblába és a hosszú formátumú `telemetry` hypertable-be ír, egy tranzakcióban. A translator ezután egy `TelemetryEvent` domain rekorddá alakítja az üzenetet, az enricher kiegészíti a `DeviceMetadataService` metaadataival, a router pedig a csatornákra irányít.

16.2.3.1 A minimális projekt és a függőségek

A példa a 15.1-es `ts-backend` projektre épül. A `pom.xml`-ben a Spring Boot 3.3.4 parent már meghatározza a verziókat; a Spring Integrationhöz és a tesztekhez az alábbi két függőség kell:

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-integration</artifactId>
</dependency>
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-test</artifactId>
    <scope>test</scope>
</dependency>
```

A projektfa a lényeges új fájlokkal:

```
ts-backend/
  pom.xml
  src/main/java/com/terrasense/backend/
    TelemetryMessage.java          (15.1, változatlan DTO)
    Measurement.java               (15.1, változatlan)
    TelemetryRepository.java       (15.2, hosszú tábla + telemetry_ingest)
    TelemetryEvent.java
    EnrichedTelemetry.java
    AlarmEvent.java
    DeviceMetadata.java
    DeviceMetadataService.java
    JdbcDeviceMetadataService.java
    TelemetryTranslator.java
    TelemetryEnricher.java
```

```
    TelemetryRouter.java
    TelemetryFlowConfiguration.java
    TimeseriesOutputHandler.java
    TelemetryPipelineService.java
    CmmsWorkOrder.java
    CmmsWorkOrderPayload.java
    CmmsOutboxRepository.java
    CmmsOutboxHandler.java
    CmmsOutboxService.java
    CmmsDlqRepository.java
    OutboxRecord.java
    OutboxPublisher.java
    CmmsClientConfiguration.java
  src/main/resources/
    application.yaml
    03_integration.sql
  src/test/java/com/terrasense/backend/
    TelemetryTranslatorTest.java
    TelemetryEnricherTest.java
    TelemetryRouterTest.java
    CmmsWorkOrderTest.java
```

16.2.3.2 Domain rekordok

A `TelemetryEvent` a kanonikus üzenet domain változata. A `schemaVersion`, `site`, `nodeId`, `seq`, `ts` és `bootId` mezők mellett a `measurements` térkép a 15.1-es `Measurement` rekordot használja, így a repository `TelemetryMessage` DTO-ja közvetlenül előállítható belőle. A `TelemetryEvent.java`:

```java
package com.terrasense.backend;

import java.util.Map;

public record TelemetryEvent(
        int schemaVersion,
        String site,
        String nodeId,
        long seq,
        long ts,
        String bootId,
        Map<String, Measurement> measurements) {

    public TelemetryEvent {
        measurements = Map.copyOf(measurements);
    }

    public Double temperature() {
        return validValue("temperature");
    }

    public Double soilMoisture() {
        return validValue("soil_moisture");
    }

    private Double validValue(String type) {
        Measurement measurement = measurements.get(type);
        if (measurement == null
                || !Boolean.TRUE.equals(measurement.valid())
                || measurement.value() == null) {
            return null;
        }
```

```
        return measurement.value();
    }

    public TelemetryMessage toMessage() {
        return new TelemetryMessage(
            schemaVersion, nodeId, seq, ts, site, bootId,
            measurements, null, null, null);
    }
}
```

A `DeviceMetadata`, az `EnrichedTelemetry` és az `AlarmEvent` rekordok:

```
package com.terrasense.backend;

public record DeviceMetadata(
        String site,
        String nodeId,
        String siteName,
        String deviceType,
        String maintenanceOwner) {}

package com.terrasense.backend;

public record EnrichedTelemetry(TelemetryEvent telemetry, DeviceMetadata metadata) {
    public String eventKey() {
        return telemetry.site() + "/" + telemetry.nodeId() + "/"
            + telemetry.bootId() + "/" + telemetry.seq();
    }
}

package com.terrasense.backend;

public record AlarmEvent(
        TelemetryEvent telemetry,
        DeviceMetadata metadata,
        String faultType,
        String severity,
        double value,
        String unit) {

    public String eventKey() {
        return telemetry.site() + "/" + telemetry.nodeId() + "/"
            + telemetry.bootId() + "/" + telemetry.seq() + "/" + faultType;
    }

    public static AlarmEvent highTemperature(EnrichedTelemetry enriched, double value) {
        return new AlarmEvent(enriched.telemetry(), enriched.metadata(),
            "high_temperature", "critical", value, "Cel");
    }

    public static AlarmEvent lowSoilMoisture(EnrichedTelemetry enriched, double value) {
        return new AlarmEvent(enriched.telemetry(), enriched.metadata(),
            "low_soil_moisture", "warning", value, "%");
    }
}
```

A metaadatszolgáltatás interfésze és JDBC-implementációja:

```
package com.terrasense.backend;

@FunctionalInterface
public interface DeviceMetadataService {
    DeviceMetadata get(String site, String nodeId);
}
```

```
package com.terrasense.backend;

import org.springframework.jdbc.core.JdbcTemplate;
import org.springframework.stereotype.Repository;

@Repository
public class JdbcDeviceMetadataService implements DeviceMetadataService {

    private final JdbcTemplate jdbc;

    public JdbcDeviceMetadataService(JdbcTemplate jdbc) {
        this.jdbc = jdbc;
    }

    @Override
    public DeviceMetadata get(String site, String nodeId) {
        return jdbc.queryForObject("""
            SELECT site, node_id, site_name, device_type, maintenance_owner
            FROM device_metadata
            WHERE site = ? AND node_id = ?
            """,
            (rs, rowNum) -> new DeviceMetadata(
                rs.getString("site"),
                rs.getString("node_id"),
                rs.getString("site_name"),
                rs.getString("device_type"),
                rs.getString("maintenance_owner")),
            site, nodeId);
    }
}
```

16.2.3.3 Translator, enricher és router

A translator a 15.1-es `TelemetryMessage` DTO-t használja, így a lapos, 0-es verziójú formátum adaptere is működik. A topic és a payload site/node azonosságát, a `seq`/`ts` pozitivitását és a mérési értékek meglétét is ellenőrzi. A `TelemetryTranslator.java`:

```
package com.terrasense.backend;

import com.fasterxml.jackson.core.JsonProcessingException;
import com.fasterxml.jackson.databind.ObjectMapper;
import java.util.Map;
import java.util.regex.Matcher;
import java.util.regex.Pattern;
import org.springframework.stereotype.Component;

@Component
public class TelemetryTranslator {

    private static final Pattern TOPIC_PATTERN =
        Pattern.compile("^terrasense/([^/]+)/([^/]+)/telemetry$");

    private final ObjectMapper objectMapper;

    public TelemetryTranslator(ObjectMapper objectMapper) {
        this.objectMapper = objectMapper;
    }

    public TelemetryEvent translate(String topic, String payload) {
        Matcher matcher = TOPIC_PATTERN.matcher(topic);
        if (!matcher.matches()) {
```

```
    throw new IllegalArgumentException("Érvénytelen telemetria-topic: " + topic);
}

TelemetryMessage message;
try {
    message = objectMapper.readValue(payload, TelemetryMessage.class);
} catch (JsonProcessingException e) {
    throw new IllegalArgumentException("Érvénytelen telemetria JSON", e);
}

int version = message.effectiveSchemaVersion();
if (version != 0 && version != 1) {
    throw new IllegalArgumentException("Nem támogatott schema_version: " + version);
}
if (version == 1 && (message.measurements() == null
        || message.measurements().isEmpty())) {
    throw new IllegalArgumentException(
        "schema_version=1 mellett hiányzó measurements");
}
if (version == 0 && message.measurements() != null
        && !message.measurements().isEmpty()) {
    throw new IllegalArgumentException(
        "schema_version=0 mellett measurements nem lehet jelen");
}
if (version == 0 && !message.isLegacyFlat()) {
    throw new IllegalArgumentException(
        "schema_version=0 mellett hiányoznak a lapos mérési mezők");
}
if (message.node_id() == null
        || !message.node_id().equals(matcher.group(2))) {
    throw new IllegalArgumentException("A topic node és a payload node eltér");
}
if (message.site() != null && !message.site().isBlank()
        && !message.site().equals(matcher.group(1))) {
    throw new IllegalArgumentException("A topic site és a payload site eltér");
}
if (message.seq() == null || message.seq() < 0) {
    throw new IllegalArgumentException("Hibás seq");
}
if (message.ts() == null
        || message.ts() < 1_600_000_000_000L
        || message.ts() > 4_000_000_000_000L) {
    throw new IllegalArgumentException("Hibás ts");
}

Map<String, Measurement> measurements = message.normalizedMeasurements();
if (measurements.isEmpty()) {
    throw new IllegalArgumentException("Hiányzó measurements");
}
for (Map.Entry<String, Measurement> entry : measurements.entrySet()) {
    if (entry.getKey() == null || entry.getKey().isBlank()) {
        throw new IllegalArgumentException("Üres méréstípus");
    }
    Measurement measurement = entry.getValue();
    if (measurement == null || measurement.value() == null) {
        throw new IllegalArgumentException(
            "Hiányzó érték a(z) " + entry.getKey() + " mérésben");
    }
    if (version == 1) {
        if (measurement.unit() == null || measurement.unit().isBlank()) {
            throw new IllegalArgumentException(
                "Hiányzó mértékegység: " + entry.getKey());
        }
```

```
                if (measurement.valid() == null) {
                    throw new IllegalArgumentException(
                        "Hiányzó valid jelző: " + entry.getKey());
                }
                if (!measurement.valid() && (measurement.reason() == null
                        || measurement.reason().isBlank())) {
                    throw new IllegalArgumentException(
                        "Érvénytelen mérés indoklás nélkül: " + entry.getKey());
                }
            }
        }

        return new TelemetryEvent(
            version, matcher.group(1), matcher.group(2),
            message.seq(), message.ts(), message.effectiveBootId(), measurements);
    }
}
```

Az enricher konstruktorinjektálással kapja a szolgáltatást. A `TelemetryEnricher.java`:

```
package com.terrasense.backend;

import org.springframework.stereotype.Component;

@Component
public class TelemetryEnricher {

    private final DeviceMetadataService metadataService;

    public TelemetryEnricher(DeviceMetadataService metadataService) {
        this.metadataService = metadataService;
    }

    public EnrichedTelemetry enrich(TelemetryEvent event) {
        DeviceMetadata metadata = metadataService.get(event.site(), event.nodeId());
        return new EnrichedTelemetry(event, metadata);
    }
}
```

A router a Spring Integration `MessageChannel` interfészét használja, a `MessageBuilder`-rel készít `Message` objektumot, és a hőmérséklet, illetve a talajnedvesség alapján dönt. A `TelemetryRouter.java`:

```
package com.terrasense.backend;

import org.springframework.beans.factory.annotation.Qualifier;
import org.springframework.integration.support.MessageBuilder;
import org.springframework.messaging.Message;
import org.springframework.messaging.MessageChannel;
import org.springframework.stereotype.Component;

@Component
public class TelemetryRouter {

    private final MessageChannel timeseriesOutput;
    private final MessageChannel alarmOutput;

    public TelemetryRouter(
            @Qualifier("timeseriesOutput") MessageChannel timeseriesOutput,
            @Qualifier("alarmOutput") MessageChannel alarmOutput) {
        this.timeseriesOutput = timeseriesOutput;
        this.alarmOutput = alarmOutput;
    }
```

```
    public void route(EnrichedTelemetry enriched) {
        Message<EnrichedTelemetry> telemetryMessage = MessageBuilder.withPayload(enriched)
            .setHeader("site", enriched.telemetry().site())
            .setHeader("nodeId", enriched.telemetry().nodeId())
            .setHeader("seq", enriched.telemetry().seq())
            .setHeader("eventKey", enriched.eventKey())
            .build();
        timeseriesOutput.send(telemetryMessage);

        Double temperature = enriched.telemetry().temperature();
        if (temperature != null && temperature > 30.0) {
            sendAlarm(AlarmEvent.highTemperature(enriched, temperature));
        }

        Double soilMoisture = enriched.telemetry().soilMoisture();
        if (soilMoisture != null && soilMoisture < 20.0) {
            sendAlarm(AlarmEvent.lowSoilMoisture(enriched, soilMoisture));
        }
    }

    private void sendAlarm(AlarmEvent alarm) {
        Message<AlarmEvent> message = MessageBuilder.withPayload(alarm)
            .setHeader("eventKey", alarm.eventKey())
            .setHeader("faultType", alarm.faultType())
            .setHeader("site", alarm.telemetry().site())
            .setHeader("nodeId", alarm.telemetry().nodeId())
            .setHeader("seq", alarm.telemetry().seq())
            .build();
        alarmOutput.send(message);
    }
}
```

A csatornák `DirectChannel` példányok, amelyek szinkron hívják a feliratkozókat. A `@ServiceActivator` handlerek és a konfiguráció:

```
package com.terrasense.backend;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.integration.channel.DirectChannel;
import org.springframework.integration.config.EnableIntegration;
import org.springframework.messaging.MessageChannel;

@Configuration
@EnableIntegration
public class TelemetryFlowConfiguration {

    @Bean
    public MessageChannel timeseriesOutput() {
        return new DirectChannel();
    }

    @Bean
    public MessageChannel alarmOutput() {
        return new DirectChannel();
    }
}
```

```
package com.terrasense.backend;

import lombok.extern.slf4j.Slf4j;
import org.springframework.integration.annotation.ServiceActivator;
```

```
import org.springframework.messaging.Message;
import org.springframework.stereotype.Component;

@Slf4j
@Component
public class TimeseriesOutputHandler {

    @ServiceActivator(inputChannel = "timeseriesOutput")
    public void handle(Message<EnrichedTelemetry> message) {
        EnrichedTelemetry enriched = message.getPayload();
        log.debug("Idősoros kimenet: site={}, node={}, seq={}",
            enriched.telemetry().site(),
            enriched.telemetry().nodeId(),
            enriched.telemetry().seq());
    }
}
```

A `TimeseriesOutputHandler` a példában naplóz; a tényleges idősoros beszúrást a `TelemetryRepository` végzi a pipeline-tranzakcióban. A csatorna így egy későbbi analitikai fogyasztó számára is nyitva marad.

16.2.3.4 A tranzakciós pipeline

A `TelemetryPipelineService` fogja össze a fordítást, az idempotens beszúrást, a kiegészítést és az irányítást. A `@Transactional` miatt a `telemetry_ingest/telemetry` beszúrás és a router által kiváltott CMMS-outbox írás ugyanabban a tranzakcióban történik. A `TelemetryPipelineService.java`:

```
package com.terrasense.backend;

import lombok.extern.slf4j.Slf4j;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

@Slf4j
@Service
public class TelemetryPipelineService {

    private final TelemetryTranslator translator;
    private final TelemetryRepository telemetryRepository;
    private final TelemetryEnricher enricher;
    private final TelemetryRouter router;

    public TelemetryPipelineService(
            TelemetryTranslator translator,
            TelemetryRepository telemetryRepository,
            TelemetryEnricher enricher,
            TelemetryRouter router) {
        this.translator = translator;
        this.telemetryRepository = telemetryRepository;
        this.enricher = enricher;
        this.router = router;
    }

    @Transactional
    public void process(String topic, String payload) {
        TelemetryEvent event;
        try {
            event = translator.translate(topic, payload);
        } catch (IllegalArgumentException e) {
            telemetryRepository.insertDeadLetter(topic, payload, e.getMessage());
```

```
            log.warn("Érvénytelen telemetria a pipeline-ban: {}", e.getMessage());
            return;
        }
        TelemetryRepository.IngestOutcome outcome =
            telemetryRepository.insertIfAbsent(event.site(), event.toMessage(), topic);
        if (outcome != TelemetryRepository.IngestOutcome.INSERTED) {
            log.debug("Az esemény nem új beillesztés ({}), a pipeline kihagyja: {}",
                outcome, event);
            return;
        }
        EnrichedTelemetry enriched = enricher.enrich(event);
        router.route(enriched);
    }
}
```

A pipeline-t a 15.1-es MQTT-ingest a validálás után hívja:

```
pipeline.process(topic, payload);
```

A 15.1-es `TelemetryIngestService.process` metódusában a közvetlen `repository.insertIfAbsent(...)` hívást és az `IngestOutcome` kezelését ez a hívás váltja fel; a `validate` metódus az ingestben maradhat, de a pipeline translatora ugyanezeket az ellenőrzéseket is elvégzi, és a hibás adatot ugyanúgy a `dead_letter` táblába irányítja. Az átmeneti adatbázis-hibák továbbra is az ingest retry spooljába kerülnek, mert a kivétel a `pipeline.process` hívásból kifelé halad.

A `TelemetryRepository.insertIfAbsent` a 15.2-es implementáció, amely a `telemetry_ingest` eseménytáblába és a hosszú `telemetry` táblába ír. Ha az esemény duplikátum, a pipeline nem irányít tovább, mert az első feldolgozás már megtette. Ha a tartalom ütközik, a repository a `dead_letter` táblába menti a payloadot, és a pipeline szintén nem irányít. Az outbox írása csak új esemény esetén történik.

Fontos tranzakciós feltétel. A `DirectChannel` szinkron, ezért a `TelemetryRouter send` hívása még a `process` tranzakciójában fut, és a `CmmsOutboxHandler` által végzett outbox-beszúrás ugyanabba a tranzakcióba tartozik. Ha a csatorna `ExecutorChannel` vagy `QueueChannel` lenne, a handler másik szálon futna, a tranzakció nem öröklődne, és az outbox beszúrása külön tranzakcióba kerülne. Ilyen esetben a riasztást előbb egy `alarm` táblába kellene írni ugyanabban a tranzakcióban, majd abból egy külön folyamat töltené fel az outboxot.

16.2.4 Az outbox és a DLQ teljes implementációja

A CMMS felé történő továbbítás outbox-szal valósul meg. A `cmms_outbox` tábla rekordja ugyanabban a Spring-tranzakcióban jön létre, mint az idősoros beszúrás. Az alábbi SQL a `device_metadata`, a `cmms_outbox` és a `cmms_dlq` táblát hozza létre:

```
-- ~/iot-labor/i048/03_integration.sql

CREATE TABLE IF NOT EXISTS device_metadata (
    site              TEXT NOT NULL,
    node_id           TEXT NOT NULL,
    site_name         TEXT NOT NULL,
    device_type       TEXT NOT NULL,
    maintenance_owner TEXT NOT NULL,
    PRIMARY KEY (site, node_id)
);

INSERT INTO device_metadata
    (site, node_id, site_name, device_type, maintenance_owner)
VALUES
```

```
    ('gh1', 'n001', 'Greenhouse 1', 'ts-node', 'gardener@example.com')
ON CONFLICT (site, node_id) DO NOTHING;

CREATE TABLE IF NOT EXISTS cmms_outbox (
    id              BIGSERIAL PRIMARY KEY,
    event_key       TEXT NOT NULL UNIQUE,
    work_order_id   UUID NOT NULL UNIQUE,
    payload         JSONB NOT NULL,
    status          TEXT NOT NULL DEFAULT 'PENDING'
                    CHECK (status IN ('PENDING', 'PROCESSING', 'SENT', 'DEAD')),
    attempt_count   INT NOT NULL DEFAULT 0,
    next_attempt_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    claimed_at      TIMESTAMPTZ,
    last_error      TEXT,
    created_at      TIMESTAMPTZ NOT NULL DEFAULT now(),
    sent_at         TIMESTAMPTZ
);

CREATE INDEX IF NOT EXISTS cmms_outbox_due_idx
    ON cmms_outbox (status, next_attempt_at);
CREATE INDEX IF NOT EXISTS cmms_outbox_claim_idx
    ON cmms_outbox (status, claimed_at);

CREATE TABLE IF NOT EXISTS cmms_dlq (
    id            BIGSERIAL PRIMARY KEY,
    outbox_id     BIGINT NOT NULL REFERENCES cmms_outbox(id),
    work_order_id UUID NOT NULL,
    payload       JSONB NOT NULL,
    reason        TEXT NOT NULL,
    created_at    TIMESTAMPTZ NOT NULL DEFAULT now(),
    replayed_at   TIMESTAMPTZ
);

CREATE INDEX IF NOT EXISTS cmms_dlq_open_idx
    ON cmms_dlq (created_at) WHERE replayed_at IS NULL;
```

A `work_order_id` a küldő oldalán keletkező, stabil idempotencia-kulcs. A `CmmsWorkOrder.from` a riasztás eseménykulcsából determinisztikus UUID-t képez, így ugyanahhoz a riasztáshoz minden újrapróbálkozás ugyanazt a munkalap-azonosítót küldi. A `CmmsWorkOrderPayload` a CMMS-szerződés szerinti JSON-mezőneveket használja. A `CmmsWorkOrder.java` és a `CmmsWorkOrderPayload.java`:

```
package com.terrasense.backend;

import java.nio.charset.StandardCharsets;
import java.util.UUID;

public record CmmsWorkOrder(
        String eventKey,
        UUID workOrderId,
        CmmsWorkOrderPayload payload) {

    public static CmmsWorkOrder from(AlarmEvent alarm) {
        String eventKey = alarm.eventKey();
        UUID workOrderId = UUID.nameUUIDFromBytes(
            eventKey.getBytes(StandardCharsets.UTF_8));
        CmmsWorkOrderPayload payload = new CmmsWorkOrderPayload(
            workOrderId.toString(),
            alarm.telemetry().site(),
            alarm.telemetry().nodeId(),
            alarm.faultType(),
            alarm.severity(),
            alarm.telemetry().ts(),
```

```
            alarm.value(),
            alarm.unit(),
            alarm.metadata().maintenanceOwner());
        return new CmmsWorkOrder(eventKey, workOrderId, payload);
    }
}
```

```
package com.terrasense.backend;

import com.fasterxml.jackson.annotation.JsonProperty;

public record CmmsWorkOrderPayload(
        @JsonProperty("work_order_id") String workOrderId,
        String site,
        @JsonProperty("node_id") String nodeId,
        @JsonProperty("fault_type") String faultType,
        String severity,
        long timestamp,
        double value,
        String unit,
        @JsonProperty("maintenance_owner") String maintenanceOwner) {}
```

A `CmmsOutboxRepository` végzi a tranzakciós beszúrást, a rekordok igénylését (`claim`), a `SENT/PENDING/DEAD` állapotátmeneteket és az elakadt `PROCESSING` rekordok helyreállítását. A `SELECT ... FOR UPDATE SKIP LOCKED` több publisher-példány esetén is megakadályozza, hogy ugyanazt a rekordot egyszerre ketten küldjék. A `CmmsOutboxRepository.java`:

```
package com.terrasense.backend;

import com.fasterxml.jackson.core.JsonProcessingException;
import com.fasterxml.jackson.databind.ObjectMapper;
import java.sql.Timestamp;
import java.time.Instant;
import java.util.List;
import java.util.UUID;
import org.springframework.jdbc.core.JdbcTemplate;
import org.springframework.stereotype.Repository;

@Repository
public class CmmsOutboxRepository {

    private final JdbcTemplate jdbc;
    private final ObjectMapper objectMapper;

    public CmmsOutboxRepository(JdbcTemplate jdbc, ObjectMapper objectMapper) {
        this.jdbc = jdbc;
        this.objectMapper = objectMapper;
    }

    public boolean insertIfAbsent(CmmsWorkOrder workOrder) {
        String payload;
        try {
            payload = objectMapper.writeValueAsString(workOrder.payload());
        } catch (JsonProcessingException e) {
            throw new IllegalStateException("A CMMS payload nem szerializálható", e);
        }
        int rows = jdbc.update("""
            INSERT INTO cmms_outbox
                (event_key, work_order_id, payload, status, attempt_count, next_attempt_at)
            VALUES (?, ?::uuid, ?::jsonb, 'PENDING', 0, now())
            ON CONFLICT (event_key) DO NOTHING
            """,
            workOrder.eventKey(),
```

```
            workOrder.workOrderId().toString(),
            payload);
        return rows == 1;
    }

    public int recoverStaleClaims() {
        return jdbc.update("""
            UPDATE cmms_outbox
            SET status = 'PENDING', claimed_at = NULL, next_attempt_at = now()
            WHERE status = 'PROCESSING'
              AND claimed_at < now() - interval '5 minutes'
            """);
    }

    public List<OutboxRecord> claimDue(int limit) {
        return jdbc.query("""
            UPDATE cmms_outbox
            SET status = 'PROCESSING',
                claimed_at = now(),
                attempt_count = attempt_count + 1
            WHERE id IN (
                SELECT id
                FROM cmms_outbox
                WHERE status = 'PENDING' AND next_attempt_at <= now()
                ORDER BY next_attempt_at
                LIMIT ?
                FOR UPDATE SKIP LOCKED
            )
            RETURNING id, event_key, work_order_id, payload, attempt_count
            """,
            (rs, rowNum) -> new OutboxRecord(
                rs.getLong("id"),
                rs.getString("event_key"),
                UUID.fromString(rs.getString("work_order_id")),
                rs.getString("payload"),
                rs.getInt("attempt_count")),
            limit);
    }

    public int markSent(long id) {
        return jdbc.update("""
            UPDATE cmms_outbox
            SET status = 'SENT', sent_at = now(), claimed_at = NULL, last_error = NULL
            WHERE id = ? AND status = 'PROCESSING'
            """, id);
    }

    public int markRetry(long id, Instant nextAttemptAt, String error) {
        return jdbc.update("""
            UPDATE cmms_outbox
            SET status = 'PENDING', claimed_at = NULL,
                next_attempt_at = ?, last_error = ?
            WHERE id = ? AND status = 'PROCESSING'
            """, Timestamp.from(nextAttemptAt), error, id);
    }

    public int markDead(long id, String error) {
        return jdbc.update("""
            UPDATE cmms_outbox
            SET status = 'DEAD', claimed_at = NULL, last_error = ?
            WHERE id = ? AND status = 'PROCESSING'
            """, error, id);
    }
```

```
}
```

Az `OutboxRecord` a claim eredménye:

```
package com.terrasense.backend;

import java.util.UUID;

public record OutboxRecord(
        long id,
        String eventKey,
        UUID workOrderId,
        String payload,
        int attemptCount) {}
```

A `CmmsOutboxHandler` a `alarmOutput` csatornára iratkozik fel, és a riasztásból outbox-rekordot készít. Mivel a `DirectChannel` szinkron, ez a beszúrás a `TelemetryPipelineService.process` tranzakciójában fut:

```
package com.terrasense.backend;

import org.springframework.integration.annotation.ServiceActivator;
import org.springframework.messaging.Message;
import org.springframework.stereotype.Component;

@Component
public class CmmsOutboxHandler {

    private final CmmsOutboxRepository outboxRepository;

    public CmmsOutboxHandler(CmmsOutboxRepository outboxRepository) {
        this.outboxRepository = outboxRepository;
    }

    @ServiceActivator(inputChannel = "alarmOutput")
    public void handle(Message<AlarmEvent> message) {
        outboxRepository.insertIfAbsent(CmmsWorkOrder.from(message.getPayload()));
    }
}
```

A DLQ-írást és a `DEAD` állapotot egy tranzakciós szolgáltatás végzi, hogy a kettő együtt sikerüljön vagy együtt hiúsuljon meg. A `CmmsOutboxService.java` és a `CmmsDlqRepository.java`:

```
package com.terrasense.backend;

import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

@Service
public class CmmsOutboxService {

    private final CmmsOutboxRepository outboxRepository;
    private final CmmsDlqRepository dlqRepository;

    public CmmsOutboxService(
            CmmsOutboxRepository outboxRepository,
            CmmsDlqRepository dlqRepository) {
        this.outboxRepository = outboxRepository;
        this.dlqRepository = dlqRepository;
    }

    @Transactional
    public void markDeadAndDlq(OutboxRecord outboxRecord, String error) {
```

```
        int updated = outboxRepository.markDead(outboxRecord.id(), error);
        if (updated == 1) {
            dlqRepository.insert(outboxRecord, error);
        }
    }
}
```

```
package com.terrasense.backend;

import org.springframework.jdbc.core.JdbcTemplate;
import org.springframework.stereotype.Repository;

@Repository
public class CmmsDlqRepository {

    private final JdbcTemplate jdbc;

    public CmmsDlqRepository(JdbcTemplate jdbc) {
        this.jdbc = jdbc;
    }

    public void insert(OutboxRecord outboxRecord, String reason) {
        jdbc.update("""
            INSERT INTO cmms_dlq
                (outbox_id, work_order_id, payload, reason)
            VALUES (?, ?::uuid, ?::jsonb, ?)
            """,
            outboxRecord.id(),
            outboxRecord.workOrderId().toString(),
            outboxRecord.payload(),
            reason);
    }
}
```

A publisher a `RestClient`-tel küldi a kérést, az `Idempotency-Key` fejlécben a stabil `work_order_id`-val. A 2xx és a 409 Conflict egyaránt siker: a 409 azt jelenti, hogy a CMMS már ismeri ezt a kulcsot, tehát nem készül duplikált munkalap. A 408, a 429, az 5xx és a hálózati hiba újrapróbálkozást indít; a többi 4xx végleges hiba, amelyet a DLQ-ba teszünk. A `OutboxPublisher.java`:

```
package com.terrasense.backend;

import java.time.Instant;
import lombok.extern.slf4j.Slf4j;
import org.springframework.http.MediaType;
import org.springframework.scheduling.annotation.Scheduled;
import org.springframework.stereotype.Component;
import org.springframework.web.client.HttpClientErrorException;
import org.springframework.web.client.HttpServerErrorException;
import org.springframework.web.client.RestClient;
import org.springframework.web.client.RestClientException;

@Slf4j
@Component
public class OutboxPublisher {

    private static final int BATCH_SIZE = 50;
    private static final int MAX_ATTEMPTS = 8;

    private final CmmsOutboxRepository outboxRepository;
    private final CmmsOutboxService outboxService;
    private final RestClient cmmsRestClient;
```

```
    public OutboxPublisher(
            CmmsOutboxRepository outboxRepository,
            CmmsOutboxService outboxService,
            RestClient cmmsRestClient) {
        this.outboxRepository = outboxRepository;
        this.outboxService = outboxService;
        this.cmmsRestClient = cmmsRestClient;
    }

    @Scheduled(fixedDelayString = "${terrasense.cmms.publisher-delay:5000}")
    public void publishDue() {
        int recovered = outboxRepository.recoverStaleClaims();
        if (recovered > 0) {
            log.warn("{} elakadt PROCESSING outbox rekordot visszaállítottunk PENDING
↪   állapotba",
                recovered);
        }
        for (OutboxRecord outboxRecord : outboxRepository.claimDue(BATCH_SIZE)) {
            try {
                publish(outboxRecord);
            } catch (RuntimeException e) {
                log.error("Váratlan hiba a(z) {} outbox rekord küldésekor",
                    outboxRecord.id(), e);
                retryOrDead(outboxRecord, "váratlan hiba: " + e.getMessage());
            }
        }
    }

    private void publish(OutboxRecord outboxRecord) {
        try {
            cmmsRestClient.post()
                .uri("/api/work-orders")
                .header("Idempotency-Key", outboxRecord.workOrderId().toString())
                .contentType(MediaType.APPLICATION_JSON)
                .body(outboxRecord.payload())
                .retrieve()
                .toBodilessEntity();
            outboxRepository.markSent(outboxRecord.id());
            log.info("CMMS munkalap elküldve: outbox={}, workOrder={}",
                outboxRecord.id(), outboxRecord.workOrderId());
        } catch (HttpClientErrorException.Conflict e) {
            outboxRepository.markSent(outboxRecord.id());
            log.info("A CMMS már ismeri a(z) {} munkalapot, SENT állapot",
                outboxRecord.workOrderId());
        } catch (HttpClientErrorException e) {
            int status = e.getStatusCode().value();
            if (status == 408 || status == 429) {
                retryOrDead(outboxRecord, e.getMessage());
            } else {
                outboxService.markDeadAndDlq(outboxRecord, e.getMessage());
                log.error("A CMMS {} választ adott, a rekord DLQ-ba került: {}",
                    status, outboxRecord.workOrderId());
            }
        } catch (HttpServerErrorException e) {
            retryOrDead(outboxRecord, e.getMessage());
        } catch (RestClientException e) {
            retryOrDead(outboxRecord, e.getMessage());
        }
    }

    private void retryOrDead(OutboxRecord outboxRecord, String error) {
        if (outboxRecord.attemptCount() >= MAX_ATTEMPTS) {
            outboxService.markDeadAndDlq(outboxRecord,
```

```
                "a maximális kísérletszám elfogyott: " + error);
            log.error("A(z) {} rekord a maximális kísérletszám után DLQ-ba került",
                outboxRecord.workOrderId());
            return;
        }
        long delaySeconds = Math.min(300, 1L << Math.min(outboxRecord.attemptCount(), 8));
        Instant nextAttemptAt = Instant.now().plusSeconds(delaySeconds);
        outboxRepository.markRetry(outboxRecord.id(), nextAttemptAt, error);
        log.warn("A(z) {} rekord újrapróbálkozása {} másodperc múlva (kísérlet: {})",
            outboxRecord.workOrderId(), delaySeconds, outboxRecord.attemptCount());
    }
}
```

A `RestClient`-et időtúllépésekkel konfiguráljuk, hogy a publisher ne akadjon el egy nem válaszoló CMMS miatt:

```
package com.terrasense.backend;

import org.springframework.beans.factory.annotation.Value;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.http.client.SimpleClientHttpRequestFactory;
import org.springframework.web.client.RestClient;

@Configuration
public class CmmsClientConfiguration {

    @Bean
    public RestClient cmmsRestClient(
            RestClient.Builder builder,
            @Value("${terrasense.cmms.base-url}") String baseUrl) {
        SimpleClientHttpRequestFactory factory = new SimpleClientHttpRequestFactory();
        factory.setConnectTimeout(2_000);
        factory.setReadTimeout(5_000);
        return builder.baseUrl(baseUrl).requestFactory(factory).build();
    }
}
```

A szükséges `application.yaml` részlet:

```
terrasense:
  cmms:
    base-url: ${CMMS_BASE_URL:http://localhost:8081}
    publisher-delay: 5000
```

A `SchedulingConfig` a 15.1-es projektből már tartalmazza az `@EnableScheduling`-et. Ha nem, a publisher `@Scheduled` annotációja nem fut le.

A CMMS fogadóoldali szerződése:

Elem	Érték
Metódus és útvonal	`POST /api/work-orders`
Fejléc	`Content-Type: application/json`, `Idempotency-Key: <work_order_id>`
Törzs	a `CmmsWorkOrderPayload` JSON-ja
201 Created / 200 OK	új munkalap létrejött
409 Conflict	a kulcs már létezik; a publisher sikerként kezeli
400 / 422	validációs hiba; `DEAD` + `cmms_dlq`, emberi javítás kell
408 / 429 / 5xx / időtúllépés	átmeneti hiba; exponenciális újrapróbálkozás

Elem	Érték

A fogadó oldalnak a `work_order_id`-t egyedi kulcsként kell tárolnia. Ha a CMMS nem deduplikál, a hálózati újrapróbálkozás duplikált munkalapot hoz létre, hiába küldjük az `Idempotency-Key` fejlécet.

A commit és a HTTP-válasz közötti megszakítás. A megoldás három esetet kezel:

1. **Megszakítás a tranzakció commitja előtt:** a `telemetry` és a `cmms_outbox` sor együtt gördül vissza; a MQTT-ingest retry spoolja újra feldolgozhatja az eseményt. Nincs elveszett esemény és nincs outbox-sor.
2. **Megszakítás a commit után, a HTTP-hívás előtt:** az outbox-sor `PENDING`, a publisher a következő körben elküldi. Nincs elveszett esemény.
3. **Megszakítás a HTTP-válasz után, a `markSent` előtt:** az outbox-sor `PROCESSING` marad. Öt perc után a `recoverStaleClaims` visszaállítja `PENDING`-re, a publisher ugyanazzal a `work_order_id`-val újraküldi, a CMMS 200-at vagy 409-et ad, és a sor `SENT` lesz. Nem készül duplikált munkalap, mert a fogadó oldal a stabil idempotencia-kulcsot egyediként kezeli.

16.2.5 Tesztek

A translator, az enricher és a router egységtesztjei a minimális projektben lefuttathatók. A `TelemetryTranslatorTest` a kanonikus payloadot és a topic/payload ütközést vizsgálja:

```
package com.terrasense.backend;

import static org.junit.jupiter.api.Assertions.assertEquals;
import static org.junit.jupiter.api.Assertions.assertThrows;

import com.fasterxml.jackson.databind.ObjectMapper;
import org.junit.jupiter.api.Test;

class TelemetryTranslatorTest {

    private final ObjectMapper objectMapper = new ObjectMapper();
    private final TelemetryTranslator translator = new TelemetryTranslator(objectMapper);

    private static final String CANONICAL = """
        {
          "schema_version": 1,
          "site": "gh1",
          "node_id": "n001",
          "seq": 42,
          "ts": 1717171200000,
          "boot_id": "b01",
          "measurements": {
            "temperature": {"value": 31.2, "unit": "Cel", "valid": true},
            "soil_moisture": {"value": 18.0, "unit": "%", "valid": true}
          }
        }
        """;

    @Test
    void canonicalPayloadIsTranslated() {
        TelemetryEvent event = translator.translate(
            "terrasense/gh1/n001/telemetry", CANONICAL);
```

```
        assertEquals(1, event.schemaVersion());
        assertEquals("gh1", event.site());
        assertEquals("n001", event.nodeId());
        assertEquals(42L, event.seq());
        assertEquals(31.2, event.temperature(), 1e-9);
        assertEquals(18.0, event.soilMoisture(), 1e-9);
    }

    @Test
    void topicAndPayloadNodeMismatchIsRejected() {
        assertThrows(IllegalArgumentException.class, () ->
            translator.translate("terrasense/gh1/n002/telemetry", CANONICAL));
    }
}
```

A `TelemetryEnricherTest` a metaadat hozzáadását ellenőrzi:

```
package com.terrasense.backend;

import static org.junit.jupiter.api.Assertions.assertEquals;

import java.util.Map;
import org.junit.jupiter.api.Test;

class TelemetryEnricherTest {

    @Test
    void metadataIsAddedToTheEvent() {
        DeviceMetadataService metadataService = (site, nodeId) ->
            new DeviceMetadata(site, nodeId, "Greenhouse 1", "ts-node",
                "gardener@example.com");
        TelemetryEnricher enricher = new TelemetryEnricher(metadataService);
        TelemetryEvent event = new TelemetryEvent(
            1, "gh1", "n001", 42L, 1717171200000L, "b01",
            Map.of("temperature", new Measurement(31.2, "Cel", true, null)));

        EnrichedTelemetry enriched = enricher.enrich(event);

        assertEquals(event, enriched.telemetry());
        assertEquals("Greenhouse 1", enriched.metadata().siteName());
        assertEquals("gardener@example.com", enriched.metadata().maintenanceOwner());
    }
}
```

A `TelemetryRouterTest` `QueueChannel`-ekkel fogja el a kimeneteket, és ellenőrzi, hogy a router nem módosítja a törzset, valamint hogy a küszöbök felett mindkét riasztás kimegy:

```
package com.terrasense.backend;

import static org.junit.jupiter.api.Assertions.assertEquals;
import static org.junit.jupiter.api.Assertions.assertNotNull;
import static org.junit.jupiter.api.Assertions.assertNull;

import java.util.Map;
import org.junit.jupiter.api.Test;
import org.springframework.integration.channel.QueueChannel;
import org.springframework.messaging.Message;

class TelemetryRouterTest {

    private static EnrichedTelemetry enriched(double temperature, double soilMoisture) {
        TelemetryEvent event = new TelemetryEvent(
            1, "gh1", "n001", 42L, 1717171200000L, "b01",
```

```
            Map.of(
                "temperature", new Measurement(temperature, "Cel", true, null),
                "soil_moisture", new Measurement(soilMoisture, "%", true, null)));
        DeviceMetadata metadata = new DeviceMetadata(
            "gh1", "n001", "Greenhouse 1", "ts-node", "gardener@example.com");
        return new EnrichedTelemetry(event, metadata);
    }

    @Test
    void highTemperatureAndDrySoilProduceAlarms() {
        QueueChannel timeseries = new QueueChannel();
        QueueChannel alarm = new QueueChannel();
        TelemetryRouter router = new TelemetryRouter(timeseries, alarm);

        router.route(enriched(31.2, 18.0));

        Message<?> timeseriesMessage = timeseries.receive(0);
        assertNotNull(timeseriesMessage);
        assertEquals(EnrichedTelemetry.class, timeseriesMessage.getPayload().getClass());

        Message<?> firstAlarm = alarm.receive(0);
        Message<?> secondAlarm = alarm.receive(0);
        assertNotNull(firstAlarm);
        assertNotNull(secondAlarm);
        assertEquals("high_temperature",
            ((AlarmEvent) firstAlarm.getPayload()).faultType());
        assertEquals("low_soil_moisture",
            ((AlarmEvent) secondAlarm.getPayload()).faultType());
    }

    @Test
    void normalValuesOnlyGoToTimeseries() {
        QueueChannel timeseries = new QueueChannel();
        QueueChannel alarm = new QueueChannel();
        TelemetryRouter router = new TelemetryRouter(timeseries, alarm);

        router.route(enriched(24.0, 30.0));

        assertNotNull(timeseries.receive(0));
        assertNull(alarm.receive(0));
    }
}
```

A `CmmsWorkOrderTest` a determinisztikus idempotencia-kulcsot ellenőrzi:

```
package com.terrasense.backend;

import static org.junit.jupiter.api.Assertions.assertEquals;

import java.util.Map;
import org.junit.jupiter.api.Test;

class CmmsWorkOrderTest {

    @Test
    void workOrderIdIsStableForTheSameAlarm() {
        TelemetryEvent event = new TelemetryEvent(
            1, "gh1", "n001", 42L, 1717171200000L, "b01",
            Map.of("temperature", new Measurement(31.2, "Cel", true, null)));
        DeviceMetadata metadata = new DeviceMetadata(
            "gh1", "n001", "Greenhouse 1", "ts-node", "gardener@example.com");
        AlarmEvent alarm = AlarmEvent.highTemperature(
            new EnrichedTelemetry(event, metadata), 31.2);
```

```
        CmmsWorkOrder first = CmmsWorkOrder.from(alarm);
        CmmsWorkOrder second = CmmsWorkOrder.from(alarm);

        assertEquals(first.workOrderId(), second.workOrderId());
        assertEquals("gh1/n001/b01/42/high_temperature", first.eventKey());
        assertEquals("gh1", first.payload().site());
        assertEquals("high_temperature", first.payload().faultType());
    }
}
```

A tesztek a Maven `test` fázisában futnak. A lektorálási környezetben Maven nem állt rendelkezésre, ezért a kód statikusan ellenőrzött, a tesztek futtatása a hallgatói környezet feladata.

16.2.6 Ellenőrzés és hibakeresés

A minták helyes működését rétegenkénti tesztekkel ellenőrizzük. Az enricher egységtesztje egy rögzített `DeviceMetadata` objektummal dolgozik, és azt vizsgálja, hogy a kiegészített mezők megjelennek. A router tesztje egy 31,2 °C-os és 18,0%-os eseményt ad be, és ellenőrzi, hogy a timeseries csatornára egy üzenet, az alarm csatornára két üzenet kerül; egy 24,0 °C-os és 30,0%-os esemény csak a timeseries csatornára megy. Az outbox tesztje egy H2 vagy Testcontainers PostgreSQL adatbázisban futtatja a tranzakciót, majd a `cmms_outbox` táblában ellenőrzi a rekordot. Az idempotens ingest tesztje ugyanazt a `(site, node_id, boot_id, seq)` kulcsot kétszer küldi be, és meggyőződik róla, hogy a hosszú `telemetry` táblában csak egyszer keletkeznek mérési sorok.

A `03_integration.sql` futtatása a laborban:

```
docker compose -f ~/iot-labor/stack/compose.yaml exec -T lab-timescaledb \
  psql -U terra -d terrasense -v ON_ERROR_STOP=1 \
  < ~/iot-labor/i048/03_integration.sql
```

A CMMS-szerződés kézi ellenőrzése `curl`-lel:

```
curl -i -X POST http://localhost:8081/api/work-orders \
  -H 'Content-Type: application/json' \
  -H 'Idempotency-Key: 0f2c7e4a-0000-3000-8000-000000000000' \
  -d
   ↪  '{"work_order_id":"0f2c7e4a-0000-3000-8000-000000000000","site":"gh1","node_id":"n001","fault_type":"high_
```

Ugyanazzal az `Idempotency-Key`-jel ismételve a CMMS-nek 200-at vagy 409-et kell adnia, és nem hozhat létre második munkalapot.

Az összeomlás tesztje SQL-lel. A commit és a HTTP-válasz közötti megszakítás szimulálható anélkül, hogy a hálózati hívást időzítenénk. Ha egy rekord `PROCESSING` állapotban marad, állítsuk a `claimed_at` értékét öt percnél régebbre, majd futtassuk a publisher következő körét:

```
UPDATE cmms_outbox
SET status = 'PROCESSING', claimed_at = now() - interval '6 minutes'
WHERE id = 1;

SELECT id, status, attempt_count, work_order_id FROM cmms_outbox WHERE id = 1;
```

A következő publisher-futás `recoverStaleClaims` lépése `PENDING`-re állítja a rekordot, a `claimDue` újra igényli, és ugyanazzal a `work_order_id`-val elküldi. A CMMS 200/409 válasza után a rekord `SENT` lesz. A várt eredmény: egyetlen munkalap, `status = 'SENT'`, növekedett `attempt_count`. Ha a CMMS nem idempotens, a teszt duplikált munkalapot mutat, ami a fogadóoldali szerződés hiányára utal.

Tünet	Ok	Javítás
A CMMS nem kap eseményeket, de a DLQ üres	A publisher nem fut, vagy a `PENDING` rekordok `next_attempt_at` értéke a jövőben van	Ellenőrizzük a `@Scheduled` futását, a `cmms_outbox.status` és `next_attempt_at` oszlopát; a `SchedulingConfig` tartalmazza-e az `@EnableScheduling`-et
Az ingest `EmptyResultDataAccessException`-t dob, és nem ír telemetriát	A `device_metadata` táblában nincs sor az adott `site`/`node_id` párra	Vegyük fel a metaadatot; a hiányzó metaadat konfigurációs hiba, ezért a tranzakció visszagördül, és az esemény nem vész el
A DLQ tele van ugyanazzal az üzenettel	A CMMS 4xx hibát ad (pl. érvénytelen séma), és a feldolgozó újrapróbálja	Javítsuk a `CmmsWorkOrderPayload` sémáját, majd a DLQ-ról játsszuk vissza az üzenetet
Duplikált munkalapok a CMMS-ben	Az `Idempotency-Key` hiányzik, vagy a CMMS nem deduplikál	A `work_order_id` legyen a küldő oldalán generált stabil kulcs, és a CMMS egyedi kényszerrel tárolja
Az idősoros táblában duplikált sorok	Az idempotens receiver nem tárolja a `(site, node_id, boot_id, seq)` kulcsot	A `telemetry_ingest` elsődleges kulcsa és az `ON CONFLICT DO NOTHING` együtt véd
`MessageDeliveryException: no subscriber`	A `DirectChannel`-nek nincs feliratkozója	Ellenőrizzük a `@ServiceActivator(inputChannel = ...)` handlert és a csatorna bean nevét
`ClassNotFoundException: org.springframework.integration...`	Hiányzik a `spring-boot-starter-integration`	Vegyük fel a függőséget a `pom.xml`-be
A `send(event)` nem fordul le	A `MessageChannel.send` `Message<?>`-t vár, nem domain objektumot	A router `MessageBuilder.withPayload(event)...bu` hívást használjon
A `final` csatornamezők inicializálatlanok	Nincs konstruktor	Adjunk explicit konstruktort a `@Qualifier` annotált paraméterekkel
Az outbox rekord örökre `PROCESSING`	A publisher a HTTP-hívás vagy a `markSent` előtt összeomlott	A `recoverStaleClaims` öt perc után visszaállítja; ellenőrizzük a `claimed_at` értékét
A telemetria commitolódik, de az outbox nem	A csatorna aszinkron (`ExecutorChannel`), ezért a tranzakció nem öröklődik	Használjunk `DirectChannel`-t, vagy írjunk külön `alarm` táblát ugyanabban a tranzakcióban

16.2.7 Tipikus hibák és félreértések

1. **Dual write outbox nélkül.** Ha a backend előbb beszúr a TimescaleDB-be, majd közvetlenül hívja a CMMS REST API-t, akkor a kettő közötti hiba esetén a két rendszer

állapota eltér. Az outbox minta éppen ezt a konzisztenciaproblémát oldja meg: a telemetria és az outbox ugyanabban a tranzakcióban commitol.

2. **Nem idempotens webhook.** A hálózati újrapróbálkozás duplikált munkalapokat hoz létre, ha a CMMS nem kap egyedi idempotencia-kulcsot. A `work_order_id` nem lehet a CMMS által generált, hanem a küldő oldalán kell keletkeznie, és a fogadó oldalnak egyedi kulcsként kell tárolnia.

3. **A content enricher túl sok külső hívást végez.** Ha minden telemetria-üzenethez lekérdezzük a metaadatot, a rendszer késleltetése megnő. A `DeviceMetadataService`-t érdemes rövid TTL-es cache-sel ellátni, mert a metaadat ritkán változik.

4. **A dead letter channel-t senki nem figyeli.** A DLQ nem a megoldás, hanem a hiba tárolója. Riasztást kell kötni a `cmms_dlq` nyitott sorainak számához, különben az integráció csendben leáll.

5. **A `MessageChannel.send` nyers objektummal hívása.** A Spring Integration csatornája `Message<?>`-t vár. A `send(event)` típushiba vagy `MessageDeliveryException`; a helyes minta a `MessageBuilder.withPayload(event).setHeader(...).build()`.

6. **Az outbox és a telemetria külön tranzakcióban.** Ha a repository hívása után a tranzakció már commitolt, az outbox beszúrása már nem tud visszagördülni vele. A `TelemetryPipelineService` ezért fogja össze a kettőt, és a csatorna `DirectChannel`.

7. **A `SENT` beállítása a HTTP-válasz előtt.** Ha a publisher a kérés elküldése előtt vagy a válasz megérkezése előtt `SENT`-re állítja a rekordot, egy sikertelen hívás után az esemény elveszik. A `SENT` csak 2xx vagy 409 után állítható.

8. **A `PROCESSING` rekordok örökös elakadása.** Ha nincs helyreállítás, egy összeomlott publisher után a rekord soha nem kerül újra sorra. A `recoverStaleClaims` és a `claimed_at` timeout kötelező része a megoldásnak.

16.2.8 Feladatok

16.2.8.1 a) Fogalmi kérdés

Nevezzen meg legalább három EIP-mintát, amely a TerraSense adatútjában megjelenik, és adja meg, melyik komponensben vagy lépésben alkalmazzuk.

16.2.8.2 b) Tervezési feladat

Tervezze meg egy CMMS felé irányuló integráció üzenetformátumát. Milyen mezőket tartalmaz a CMMS-nek küldött esemény? Hogyan biztosítja az idempotenciát? Mi kerül a dead letter channelre?

16.2.8.3 c) Implementációs feladat

Írjon egy Java `SoilMoistureRouter` osztályt, amely az `EnrichedTelemetry` üzeneteket a `timeseriesOutput` kimenetre mindig, az `alarmOutput` kimenetre pedig akkor irányítja, ha a talajnedvesség `soil_moisture` értéke 20 alá csökken. A router `MessageBuilder`-rel készítsen `Message` objektumot, ne adja át közvetlenül a domain objektumot. A csatornákat `@Qualifier` annotációval injektálja, és a `low_soil_moisture` riasztást az `AlarmEvent.lowSoilMoisture` segítségével állítsa elő.

16.2.8.4 d) Hibakeresési/kiterjesztési feladat

Az outbox táblában sok `PENDING` rekord gyűlt össze, és a CMMS nem kap új eseményeket. Milyen lehetséges okokat vizsgál? Hogyan folytatja az újrapróbálkozást úgy, hogy ne keletkezzenek duplikált munkalapok? Hogyan játszaná vissza a `cmms_dlq` egy rekordját?

16.2.9 Megoldások

16.2.9.1 a) Megoldás

Például: - **Idempotent receiver** a ts-backend MQTT ingest lépésében, a `(site, node_id, boot_id, seq)` kulcs alapján. - **Content enricher** a backendben, amikor a `DeviceMetadataService` hozzáadja a helyszín- és eszközmetaadatot. - **Message router** a telemetria és riasztás szétválasztásakor. - **Outbox** a CMMS felé történő munkalap-küldéskor. - **Dead letter channel** a sikertelen CMMS-kézbesítés esetén, a `cmms_dlq` táblában.

16.2.9.2 b) Megoldás

A CMMS-nek küldött esemény minimális sémája:

```
{
  "work_order_id": "gh1-n001-1717171200000-42",
  "node_id": "n001",
  "site": "gh1",
  "fault_type": "high_temperature",
  "severity": "critical",
  "timestamp": 1717171200000,
  "value": 31.2,
  "unit": "Cel",
  "maintenance_owner": "gardener@example.com"
}
```

A példában a `work_order_id` emberi szemmel olvasható, de a kód determinisztikus UUID-t használ, amely ugyanarra az eseménykulcsra mindig ugyanaz. Az idempotenciát a `work_order_id` biztosítja: a CMMS egyedi kulcsként kezeli, és a duplikált kéréseket 200 OK vagy 409 Conflict válasszal, de új munkalap nélkül fogadja. A dead letter channelre a 4xx validációs hibák és a maximális kísérletszámot elért 5xx vagy hálózati hibák kerülnek; a 400-as validációs hiba esetén emberi javítás szükséges, míg az 5xx esetén az automatikus újrapróbálkozás a DLQ előtt történik.

16.2.9.3 c) Megoldás

```
package com.terrasense.backend;

import org.springframework.beans.factory.annotation.Qualifier;
import org.springframework.integration.support.MessageBuilder;
import org.springframework.messaging.Message;
import org.springframework.messaging.MessageChannel;
import org.springframework.stereotype.Component;

@Component
public class SoilMoistureRouter {

    private final MessageChannel timeseriesOutput;
    private final MessageChannel alarmOutput;
```

```
    public SoilMoistureRouter(
            @Qualifier("timeseriesOutput") MessageChannel timeseriesOutput,
            @Qualifier("alarmOutput") MessageChannel alarmOutput) {
        this.timeseriesOutput = timeseriesOutput;
        this.alarmOutput = alarmOutput;
    }

    public void route(EnrichedTelemetry enriched) {
        Message<EnrichedTelemetry> telemetryMessage = MessageBuilder.withPayload(enriched)
            .setHeader("site", enriched.telemetry().site())
            .setHeader("nodeId", enriched.telemetry().nodeId())
            .setHeader("seq", enriched.telemetry().seq())
            .setHeader("eventKey", enriched.eventKey())
            .build();
        timeseriesOutput.send(telemetryMessage);

        Double soilMoisture = enriched.telemetry().soilMoisture();
        if (soilMoisture != null && soilMoisture < 20.0) {
            AlarmEvent alarm = AlarmEvent.lowSoilMoisture(enriched, soilMoisture);
            Message<AlarmEvent> alarmMessage = MessageBuilder.withPayload(alarm)
                .setHeader("eventKey", alarm.eventKey())
                .setHeader("faultType", alarm.faultType())
                .setHeader("site", alarm.telemetry().site())
                .setHeader("nodeId", alarm.telemetry().nodeId())
                .setHeader("seq", alarm.telemetry().seq())
                .build();
            alarmOutput.send(alarmMessage);
        }
    }
}
```

A router nem módosítja az `enriched` objektumot, csak a feltétel alapján irányít. A `MessageBuilder` a törzset és a fejléceket együtt csomagolja `Message` objektummá, így a `MessageChannel.send` típushelyesen hívható. A `QueueChannel`-es teszt a `TelemetryRouterTest` mintájára írható meg: 19,9%-nál egy alarm üzenet, 20,1%-nál nulla alarm üzenet várható.

16.2.9.4 d) Megoldás

Lehetséges okok: az `OutboxPublisher` ütemezett szála leállt, a `SchedulingConfig` nem tartalmazza az `@EnableScheduling`-et, a CMMS endpoint nem érhető el, a hálózati tűzfal blokkolja a kimenő hívást, vagy a `next_attempt_at` értékek a jövőben vannak a exponenciális backoff miatt. Ellenőrizzük a publisher logját, a `cmms_outbox.status`, `attempt_count`, `next_attempt_at` és `last_error` oszlopát, valamint a CMMS elérhetőségét.

Az újrapróbálkozást idempotens módon úgy folytatjuk, hogy minden rekordhoz tartozik egy stabil `work_order_id`, és a HTTP-kérésben elküldjük az `Idempotency-Key` fejlécet. A publisher csak 2xx vagy 409 után állítja `SENT` állapotúra a rekordot; átmeneti hiba esetén `PENDING` marad, és a `next_attempt_at` szerint újrapróbálkozik. Ha a rekord `PROCESSING` állapotban elakadt, a `recoverStaleClaims` öt perc után visszaállítja. Ha a rekord `DEAD`, a `cmms_dlq` tartalmazza a hibát; a javítás után a következő tranzakcióban játszható vissza:

```
BEGIN;
UPDATE cmms_outbox
SET status = 'PENDING',
    attempt_count = 0,
    next_attempt_at = now(),
    claimed_at = NULL,
    last_error = NULL
```

```
WHERE id = (
    SELECT outbox_id FROM cmms_dlq
    WHERE id = ? AND replayed_at IS NULL
);
UPDATE cmms_dlq SET replayed_at = now() WHERE id = ? AND replayed_at IS NULL;
COMMIT;
```

A visszajátszás ugyanazt a `work_order_id`-t tartja meg, ezért a CMMS a már létező munkalapot 200/409 válasszal fogadja, és nem hoz létre duplikátumot. Új `work_order_id` generálása tilos lenne, mert az valóban új munkalapot hozna létre.

16.2.10 Rövid összefoglaló

Az alfejezet az Enterprise Integration Patterns alapmintáit alkalmazta a TerraSense adatútjára: a message translator a kanonikus JSON-t domain eseménnyé alakítja, a content enricher a `DeviceMetadataService`-ből hiányzó metaadatot ad hozzá, a message router a telemetriát és a riasztást választja szét, a dead letter channel a sikertelen CMMS-kézbesítést fogadja, az idempotent receiver a `(site, node_id, boot_id, seq)` kulccsal szűri a duplikátumokat, az outbox pedig a TimescaleDB-írás és a CMMS REST-hívás közötti konzisztenciát biztosítja. A `MessageChannel.send` `Message<?>`-t vár, ezért a router `MessageBuilder`-rel készít üzenetet; a `DirectChannel` szinkron, így a telemetria-beszúrás és az outbox-írás ugyanabban a tranzakcióban fut. A `cmms_outbox` állapotgépe `PENDING`/`PROCESSING`/`SENT`/`DEAD`, a `cmms_dlq` tábla a végleges hibákat őrzi, a `work_order_id` pedig stabil idempotencia-kulcs a CMMS felé. **Továbbvitt eredmény:** a `cmms_outbox` tábla, az `OutboxPublisher`, a `work_order_id` idempotencia-kulcs és az enricher minta közvetlenül hasznosul a Parancsok és konfiguráció lefelé: kézbesítés, nyugtázás, időkorlát alfejezetben és a Felhős és nyílt forrású IoT-platformok: mikor építs, mikor vásárolj alfejezet build-vs-buy értékelésénél. A bemutatott kód egy minimális, statikusan ellenőrzött Spring Boot 3.3.4 modul; a Maven-futtatás és a Testcontainers-es integrációs teszt a hallgatói környezet feladata.

16.2.11 Források és ellenőrzési hiányok

- Enterprise Integration Patterns (Hohpe, Woolf): a message translator, content enricher, dead letter channel, idempotent receiver és outbox minták leírása. A pontos oldalszámok a nyomtatott kiadásban ellenőrizendők.
- Spring Integration dokumentáció: `MessageChannel`, `DirectChannel`, `QueueChannel`, `MessageBuilder`, `@ServiceActivator`, `@EnableIntegration`.
- Spring Boot 3.3.4 dokumentáció: `@Scheduled`, `@Transactional`, `RestClient`, `JdbcTemplate` és a `spring-boot-starter-integration` auto-konfiguráció.
- PostgreSQL dokumentáció: `FOR UPDATE SKIP LOCKED`, `UPDATE ... RETURNING`, `JSONB`, `UUID`.
- A fejezet kódja statikusan ellenőrzött, de a lektorálási környezetben Maven, Spring Boot és PostgreSQL nem futott; a tesztek és a labor a hallgatói környezetben futtathatók. A `cmms_outbox` és a `cmms_dlq` tényleges integrációs tesztje Testcontainers PostgreSQL-lel történne, ez a fejezet nem futtatta.
- A CMMS fogadóoldali implementációja nem része a TerraSense projektnek; a fejezet a szerződést és a kliensoldali idempotenciát adja meg. A CMMS-nek a `work_order_id` egyedi kulcsként való tárolása és a 409-es válasz a szerződés része, amelyet a CMMS csapatának kell betartania.
- A `DirectChannel` tranzakció-örökítése a Spring Integration szinkron hívási modelljén alapul; ha a csatorna aszinkron vagy hálózati, a tranzakciós outbox mintát külön `alarm`

táblával kell megvalósítani. Ennek részletes tranzakciós elemzése a JEE-ből Spring Bootba: fogalmi híd C/C++ fejlesztőknek alfejezetre épül.

17 Parancsok, digitális iker és megjelenítés

17.1 Parancsok és konfiguráció lefelé: kézbesítés, nyugtázás, időkorlát

17.1.1 Motiváció

A TerraSense üvegházban a `ts-node` szenzorcsomópont nemcsak mér, hanem beavatkozik is: az öntözőszelepet nyitja és zárja. A felhasználó a felhőből szeretne például egy 10 perces öntözést elindítani. Ez egy **lefelé irányuló parancs**: a backend felől az eszköz felé halad. Az MQTT publish/subscribe modellje alapvetően az eszköztől a felhő felé tartó telemetriára kényelmes; lefelé a kézbesítés, a nyugtázás és az időkorlát kezelése külön tervezést igényel. Ha a parancs elveszik, a felhasználó azt hiheti, hogy öntözött; ha kétszer hajtódik végre, túlöntözés lehet a vége. Az alvó eszköz pedig éppen nem figyel a hálózatra, ezért a parancsot késleltetve kell kézbesíteni.

Ebben az alfejezetben megtervezzük a `ts-backend` és `ts-node` közötti parancsutat: szabványos parancsüzenetet, nyugtázási és eredményjelentési protokollt, backend oldali állapotgépet, és az alvó eszközök kezelését.

17.1.2 Parancsmodell és üzenetformátum

Definíció. A **parancs** (command) a backend által kibocsátott, egy adott eszköznek címzett üzenet, amely egy művelet végrehajtását kéri. A TerraSense parancsüzenete legalább az alábbi mezőket tartalmazza:

- `id`: a parancs egyedi azonosítója, amelyet a backend generál (UUID vagy monoton növekvő szám). Ezzel hivatkozunk a nyugtára és az eredményre, és ezzel biztosítjuk az idempotenciát.
- `type`: a művelet neve, például `valve.open` vagy `valve.close`.
- `params`: típusfüggő paraméterobjektum, például `{"duration_ms": 600000, "max_on_time_ms": 600000}`.
- `expiry`: a parancs **legkésőbbi elkezdési ideje**, UTC epoch ms. Ha az eszköz eddig nem tudta elkezdeni a végrehajtást, hagyja figyelmen kívül, és `EXPIRED` eredményt küldhet. A már elkezdett művelet befejezését nem ez a mező szabályozza, hanem a művelet saját időtartama (például `duration_ms`).
- `execution_timeout_ms`: a backend által adott **végrehajtási határidő** a nyugta után. Ha az eszköz az `ACCEPTED` nyugta után ennyi időn belül nem küld eredményt, a backend `TIMEOUT` állapotot állapít meg. Ez nem azonos az `expiry`-vel: az `expiry` a legkésőbbi start, az `execution_timeout_ms` a nyugta utáni eredményre várakozás.
- `timestamp` (opcionális): a kibocsátás ideje, hibakereséshez.

A topic-konvenció az MQTT QoS, munkamenetek, retain, Last Will és MQTT 5 alfejezetben rögzített: a parancs az

```
terrasense/gh1/n001/cmd
```

topicra kerül, az eredmény pedig a

```
terrasense/gh1/n001/cmd/result
```

topicra. Mindkét topic QoS 1, `retain=false`.

Példa parancs JSON-üzenetre:

```
{
  "id": "c7d2f9a0-3b21-4f15-9e2f-6f9d2a3c1e4b",
  "type": "valve.open",
  "params": {
    "duration_ms": 600000,
    "max_on_time_ms": 600000
  },
  "expiry": 1735689600000,
  "execution_timeout_ms": 610000,
  "timestamp": 1735689000000
}
```

A parancsazonosító akkor is egyedi marad, ha a backend újraindul; UUID használata esetén nincs szükség központi számlálóra. Ha egy parancsot újra kell küldeni (például a backend újraindul, vagy az eszköz nem nyugtázta), ugyanazt az `id`-t kell megtartani.

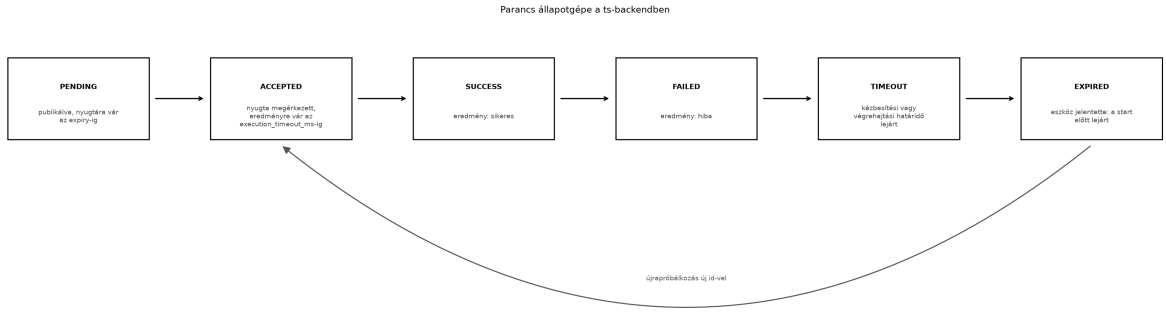
17.1.3 Nyugtázás és eredmény: backend állapotgép

A parancs életciklusa a backendben egy állapotgéppel írható le. A **nyugta** (acknowledgement, ACK) azt jelenti, hogy az eszköz megkapta és értelmezni tudta a parancsot; az **eredmény** (result) pedig a végrehajtás utáni állapotot jelenti. A kettő szétválasztása azért fontos, mert a végrehajtás hosszú lehet (például a szelep 10 percig nyitva marad), és a felhasználó előbb szeretné tudni, hogy a parancs eljutott-e.

A `ts-backend` a következő állapotokat tartja nyilván minden parancsra:

- `PENDING`: a parancsot a REST API fogadta, de még nem publikáltuk MQTT-n, vagy publikáltuk, de nem érkezett nyugta.
- `ACCEPTED`: az eszköz nyugtázta a parancsot; a végrehajtás folyamatban van.
- `SUCCESS`: az eszköz eredményt küldött, a végrehajtás sikeres volt.
- `FAILED`: az eszköz eredményt küldött, de a végrehajtás hibára futott.
- `TIMEOUT`: a backend saját határideje lejárt. `PENDING` állapotban ez az `expiry` (legkésőbbi start) elmúlását jelenti; `ACCEPTED` állapotban az `execution_timeout_ms` letelte után nem érkezett eredmény.
- `EXPIRED`: az eszköz jelezte, hogy a parancsot már a kézbesítéskor lejártnak találta, ezért el sem kezdte (például alvásból ébredve látta, hogy az `expiry` már elmúlt). Ez eszközoldali döntés, nem a backend időzítője.

Az átmeneteket az alábbi `flow` ábra mutatja.



17.1. ábra: Parancs állapotgépe a ts-backendben - A nyugta és az eredmény MQTT üzenetek formájában érkezik; a terminális állapotot késői üzenet nem írja vissza.

Átmenettáblázattal:

Jelenlegi állapot	Esemény	Következő állapot
`PENDING`	ACK érkezik az eszköztől	`ACCEPTED`
`PENDING`	eredmény érkezik ACK nélkül (rövid parancs)	`SUCCESS` vagy `FAILED`
`PENDING`	`expiry` (legkésőbbi start) lejár	`TIMEOUT`
`ACCEPTED`	eredmény érkezik	`SUCCESS` vagy `FAILED`
`ACCEPTED`	`acceptedAt + execution_timeout_ms` lejár	`TIMEOUT`
`PENDING` vagy `ACCEPTED`	az eszköz `EXPIRED`-et küld	`EXPIRED`
`SUCCESS`, `FAILED`, `TIMEOUT`, `EXPIRED`	bármely késői ACK vagy eredmény	változatlan (eldobva)

A `ts-backend` a parancsokat a laborban memóriában tárolja (`ConcurrentHashMap`), és ütemezetten ellenőrzi a határidőket. Ez a labor-hatókör: éles rendszerben a parancs- és eredményállapotot tartós tárolóba (például egy JDBC-táblába) kell menteni, különben egy backend-újraindítás elveszíti a függő parancsokat. A nyugta és az eredmény ugyanazon a `cmd/result` topicon érkezik, és a parancsazonosító alapján azonosítja a backend, melyik parancshoz tartozik. A terminális állapotot késői, duplikált vagy sorrenden kívüli üzenet nem írhatja vissza. A nyugta és az eredmény szétválasztható a `status` mezővel:

```
{
  "id": "c7d2f9a0-3b21-4f15-9e2f-6f9d2a3c1e4b",
  "status": "ACCEPTED",
  "node_id": "n001",
  "timestamp": 1735689001000
}
```

Egy későbbi eredményüzenet:

```
{
  "id": "c7d2f9a0-3b21-4f15-9e2f-6f9d2a3c1e4b",
  "status": "SUCCESS",
  "result": {
    "valve_opened_ms": 600000
  },
  "node_id": "n001",
  "timestamp": 1735689605000
}
```

Definíció. Az **idempotens parancsvégrehajtás** azt jelenti, hogy ugyanaz a parancsazonosító többszöri kézbesítés esetén is legfeljebb egyszer vált ki hatást. Az eszköz oldalán ezt a parancsazonosító megjegyzésével és a már feldolgozott azonosítók halmazával érjük el (a `valve_set_target` már idempotens a Aktuátorok: biztonságos vezérlés és visszajelzés alfejezetből). Ha az eszköz egy `SUCCESS` eredményt adott, és ugyanaz a parancs ismét megérkezik (például MQTT QoS 1 miatt), az eszköz nem hajtja végre újra, csak újraküldi az eredményt.

17.1.4 MQTT 5 request–response vs. topic-alapú konvenció

Az MQTT 5 szabvány bevezet egy explicit request–response mintát: a `Response Topic` és a `Correlation Data` user property-k segítségével a kliens megadhatja, hova várja a választ, és hogyan korrelálja a kéréssel. Ez különösen akkor hasznos, ha sok parancs fut párhuzamosan, és nincs természetes topic-szétválasztás.

A TerraSense-ben azonban az eszközönkénti topic-struktúra (`cmd` és `cmd/result`) eleve biztosítja a korrelációt, és a parancsüzenetben szereplő `id` mező erre elég. Az MQTT 5 feature-öket akkor érdemes bevetni, ha:

- többféle parancstípusra eltérő választ várunk,
- a válasz nem egy fix topicra jön, hanem például a backend egy kérésazonosítóra szűrt,
- a rendszer sok eszközt és sok párhuzamos parancsot kezel, és a topic-struktúra bonyolulttá válna.

A döntést a következő táblázat foglalja össze:

Szempont	Topic-alapú `cmd`/`cmd/result`	MQTT 5 response topic
Egyszerűség	egyszerű, konvencióval rögzíthető	több metaadatot igényel
Skálázhatóság	eszközönként jó, központi válaszgyűjtőnél szűrés kell	korrelációs adat megkönnyíti a szűrést
MQTT verzió	3.1.1 is támogatja	csak MQTT 5 kliens és broker
Fejlesztési hiba lehetősége	elgépelt topic, hiányzó feliratkozás	helytelen correlation data
TerraSense-ben	választott megoldás	nem használjuk a TerraSense-ben

17.1.5 Alvó eszközök és parancskézbesítés

Az energiatakarékos `ts-node` az idő nagy részében alvó állapotban van, és csak mintavételre vagy ébredési időzítőre kapcsolja be a rádiót. Ilyenkor egy azonnali parancs nem ér el hozzá. Két alapvető stratégia létezik:

1. **Retained parancs a `cmd` topicra.** Az MQTT retained üzenet az utolsó értéket tárolja a broker, és minden új feliratkozó megkapja. Ha a parancs retained, az alvó eszköz ébredéskor feliratkozik, és azonnal megkapja. Hátránya: csak egyetlen parancs tárolható, és ha új parancs érkezik, felülírja a régit. Ráadásul a lejárt parancs retained maradhat, és az eszköz feleslegesen kap egy elavult parancsot. Az `expiry` mezőt ilyenkor muszáj ellenőrizni.

2. **Backend oldali parancssor.** A backend eltárolja a függő parancsokat, és amikor az eszköz felébred (például MQTT `status` üzenettel jelzi), a backend leküldi a következő érvényes parancsot. Ez rugalmasabb, több parancsot is tárolhat, és lejárat szerint rendezhet. Hátránya: a backendnek tudnia kell az eszköz ébredéséről, és a kézbesítés késleltetése a backend–eszköz ciklusból adódik.

A TerraSense-ben a **retained parancsot** használjuk a következő ébredéskor végrehajtandó parancsra, de a `cmd` topic `retain=false` alapbeállítását **nem** módosítjuk; ehelyett a backend figyeli az eszköz `status` jelenlétét, és akkor publikálja a függő parancsot. Ezzel elkerüljük az elavult retained parancsokat, és megtartjuk a parancsok sorrendjét. Ennek a teljes, alvó eszközöket kiszolgáló változatnak a megvalósítása (a `status` üzenetre reagáló outbox és sorositó) nem része ennek a labornak: a laborban az eszköz online, ezért a `CommandService` közvetlenül publikál. Az alvó eszközök sora a `CommandService` későbbi kiterjesztéseként építhető meg ugyanarra a parancsmodellre.

Továbbvitt eredmény. A parancs- és eredménytopicok, valamint a parancsazonosítóval történő korreláció később az Eszközárnyék és digitális iker: kívánt és jelentett állapot alfejezetben a kívánt állapot (`desired`) kézbesítésére, a Záróprojekt: a teljes TerraSense rendszer összeállítása és átvétele alfejezetben pedig a teljes parancsút integrációjára épül.

17.1.6 Labor: szelepnyitás parancs a ts-backendtől a ts-node-ig

Ebben a laborban a `ts-backend` REST API-ján keresztül küldünk egy `valve.open` parancsot. A backend MQTT-n publikálja a `terrasense/gh1/n001/cmd` topicra. A `ts-node`-t egy egyszerű Python MQTT-kliens szimulálja, mert a Zephyr `native_sim` MQTT-stack integrációját az MQTT a gyakorlatban: Zephyr-kliens és Mosquitto alfejezetben már elvégeztük; a labort a backendre és a protokollra fókuszáljuk. A szimulátor feliratkozik a `cmd` topicra, végrehajtja a szelepnyitást (szimulált GPIO), nyugtát küld, majd a végrehajtás után eredményt a `cmd/result` topicra.

Munkakönyvtár:

```
mkdir -p ~/iot-labor/i049 && cd ~/iot-labor/i049
cp -r ../i043 ts-backend
mkdir -p ts-backend/src/main/java/com/terrasense/backend/{command,mqtt,controller,config}
```

A `ts-backend` a A ts-backend váza: REST API és egészségellenőrzés alfejezetben elkészített Spring Boot váza. A Maven `pom.xml`-hez adjuk hozzá az Eclipse Paho Java kliens függőséget:

```
<dependency>
    <groupId>org.eclipse.paho</groupId>
    <artifactId>org.eclipse.paho.client.mqttv3</artifactId>
    <version>1.2.5</version>
</dependency>
```

A Paho Java 1.2.5 a rögzített verzió; a backend a klasszikus v3 API-t használja. A Python szimulátor ezzel szemben Paho 2.x-et használ, mert a Python kliens 2.x callback API-ja eltér az 1.x-től; a két kliens verziója tudatosan különbözik.

A parancsmodell és az állapotgép osztályai. A `DeviceCommand` a backend belső és REST-nézetű modellje; az MQTT-n a szűkebb `CommandPayload` megy ki, hogy a belső állapotmezők ne kerüljenek a huzalformátumba.

```java
// src/main/java/com/terrasense/backend/command/CommandState.java
package com.terrasense.backend.command;

public enum CommandState {
    PENDING, ACCEPTED, SUCCESS, FAILED, TIMEOUT, EXPIRED;

    public boolean isTerminal() {
        return this == SUCCESS || this == FAILED || this == TIMEOUT || this == EXPIRED;
    }
}

// src/main/java/com/terrasense/backend/command/DeviceCommand.java
package com.terrasense.backend.command;

import java.util.Collections;
import java.util.LinkedHashMap;
import java.util.Map;
import java.util.UUID;

public record DeviceCommand(
        String id,
        String site,
        String nodeId,
        String type,
        Map<String, Object> params,
        long expiry,
        long executionTimeoutMs,
        CommandState state,
        long createdAt,
        long acceptedAt,
        long finishedAt,
        Map<String, Object> result
) {
    public DeviceCommand {
        if (id == null || site == null || nodeId == null || type == null) {
            throw new IllegalArgumentException("A parancs id/site/node/type mezője kötelező");
        }
        params = params == null ? Map.of()
                : Collections.unmodifiableMap(new LinkedHashMap<>(params));
        result = result == null ? Map.of()
                : Collections.unmodifiableMap(new LinkedHashMap<>(result));
    }

    public static DeviceCommand create(String site, String nodeId, String type,
                                       Map<String, Object> params,
                                       long ttlMs, long executionTimeoutMs) {
        long now = System.currentTimeMillis();
        return new DeviceCommand(UUID.randomUUID().toString(), site, nodeId, type,
                params, now + ttlMs, executionTimeoutMs,
                CommandState.PENDING, now, 0L, 0L, Map.of());
    }

    public DeviceCommand withState(CommandState next, long at, Map<String, Object> newResult) {
        if (state.isTerminal()) {
            return this;
        }
        long accepted = next == CommandState.ACCEPTED ? at : acceptedAt;
        long finished = next.isTerminal() ? at : finishedAt;
        return new DeviceCommand(id, site, nodeId, type, params, expiry,
                executionTimeoutMs, next, createdAt, accepted, finished,
                newResult == null ? result : newResult);
    }

    public boolean isTerminal() {
```

```
        return state.isTerminal();
    }
}

// src/main/java/com/terrasense/backend/command/CommandPayload.java
package com.terrasense.backend.command;

import com.fasterxml.jackson.annotation.JsonProperty;

import java.util.Map;

/** Az MQTT-n ténylegesen elküldött parancs huzalformátuma. */
public record CommandPayload(
        String id,
        String type,
        Map<String, Object> params,
        long expiry,
        @JsonProperty("execution_timeout_ms") long executionTimeoutMs,
        long timestamp
) {
    public static CommandPayload from(DeviceCommand cmd) {
        return new CommandPayload(cmd.id(), cmd.type(), cmd.params(), cmd.expiry(),
                cmd.executionTimeoutMs(), cmd.createdAt());
    }
}

// src/main/java/com/terrasense/backend/command/CommandResult.java
package com.terrasense.backend.command;

import com.fasterxml.jackson.annotation.JsonIgnoreProperties;
import com.fasterxml.jackson.annotation.JsonProperty;

import java.util.Map;

@JsonIgnoreProperties(ignoreUnknown = true)
public record CommandResult(
        String id,
        CommandState status,
        @JsonProperty("node_id") String nodeId,
        Long timestamp,
        Map<String, Object> result
) {
}

// src/main/java/com/terrasense/backend/command/CommandRequest.java
package com.terrasense.backend.command;

import com.fasterxml.jackson.annotation.JsonProperty;

import java.util.Map;

public record CommandRequest(
        String type,
        Map<String, Object> params,
        @JsonProperty("ttl_ms") Long ttlMs,
        @JsonProperty("execution_timeout_ms") Long executionTimeoutMs,
        String site
) {
}

// src/main/java/com/terrasense/backend/command/CommandResultHandler.java
package com.terrasense.backend.command;

@FunctionalInterface
```

```java
public interface CommandResultHandler {
    void onResult(CommandResult result);
}
```

Az MQTT-kapcsolatot kezelő, Spring beanként regisztrált gateway. A Jackson szerializálja a `CommandPayload` rekordot, így a `params` `Map` helyesen JSON-objektummá válik. A gateway a `cmd/result` topicra iratkozik fel, és az eredményt a regisztrált handlernek adja át; a handler beállítása miatt nincs szükség arra, hogy a gateway ismerje a `CommandService`-t.

```java
// src/main/java/com/terrasense/backend/mqtt/MqttCommandGateway.java
package com.terrasense.backend.mqtt;

import com.fasterxml.jackson.databind.ObjectMapper;
import com.terrasense.backend.command.CommandPayload;
import com.terrasense.backend.command.CommandResult;
import com.terrasense.backend.command.CommandResultHandler;
import com.terrasense.backend.command.DeviceCommand;
import jakarta.annotation.PostConstruct;
import jakarta.annotation.PreDestroy;
import org.eclipse.paho.client.mqttv3.IMqttDeliveryToken;
import org.eclipse.paho.client.mqttv3.MqttCallbackExtended;
import org.eclipse.paho.client.mqttv3.MqttClient;
import org.eclipse.paho.client.mqttv3.MqttConnectOptions;
import org.eclipse.paho.client.mqttv3.MqttException;
import org.eclipse.paho.client.mqttv3.MqttMessage;
import org.eclipse.paho.client.mqttv3.persist.MemoryPersistence;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.stereotype.Service;

import java.nio.charset.StandardCharsets;
import java.util.concurrent.atomic.AtomicReference;

@Service
public class MqttCommandGateway implements MqttCallbackExtended {
    private static final Logger log = LoggerFactory.getLogger(MqttCommandGateway.class);
    private static final String RESULT_TOPIC = "terrasense/+/+/cmd/result";

    private final MqttClient client;
    private final MqttConnectOptions options;
    private final ObjectMapper objectMapper;
    private final AtomicReference<CommandResultHandler> resultHandler = new AtomicReference<>();

    public MqttCommandGateway(
            ObjectMapper objectMapper,
            @Value("${terrasense.mqtt.broker-url:tcp://localhost:1883}") String brokerUrl,
            @Value("${terrasense.mqtt.client-id:ts-backend-cmd}") String clientId) {
        this.objectMapper = objectMapper;
        try {
            this.client = new MqttClient(brokerUrl, clientId, new MemoryPersistence());
        } catch (MqttException e) {
            throw new IllegalStateException("Nem hozható létre az MQTT kliens", e);
        }
        this.options = new MqttConnectOptions();
        options.setAutomaticReconnect(true);
        options.setCleanSession(true);
        options.setConnectionTimeout(10);
        options.setKeepAliveInterval(30);
        client.setCallback(this);
    }

    public void setResultHandler(CommandResultHandler handler) {
```

```
        resultHandler.set(handler);
    }

    @PostConstruct
    public void connect() {
        try {
            ensureConnected();
        } catch (MqttException e) {
            // A broker később is elindulhat; a publikálás előtt újra próbálkozunk.
            log.error("Az MQTT broker a startupkor nem érhető el: {}", e.getMessage());
        }
    }

    private synchronized void ensureConnected() throws MqttException {
        if (!client.isConnected()) {
            client.connect(options);
        }
    }

    public void publishCommand(DeviceCommand command) throws MqttException {
        ensureConnected();
        String topic = "terrasense/" + command.site() + "/" + command.nodeId() + "/cmd";
        final String payload;
        try {
            payload = objectMapper.writeValueAsString(CommandPayload.from(command));
        } catch (Exception e) {
            throw new IllegalStateException("A parancs nem szerializálható JSON-ná", e);
        }
        MqttMessage message = new MqttMessage(payload.getBytes(StandardCharsets.UTF_8));
        message.setQos(1);
        client.publish(topic, message).waitForCompletion(5_000);
        log.info("Parancs publikálva: topic={}, id={}, type={}", topic, command.id(),
↪   command.type());
    }

    @Override
    public void connectComplete(boolean reconnect, String serverURI) {
        try {
            client.subscribe(RESULT_TOPIC, 1);
            log.info("Feliratkozás kész: {} (reconnect={})", RESULT_TOPIC, reconnect);
        } catch (MqttException e) {
            log.error("A cmd/result feliratkozás nem sikerült", e);
        }
    }

    @Override
    public void messageArrived(String topic, MqttMessage message) {
        String json = new String(message.getPayload(), StandardCharsets.UTF_8);
        try {
            CommandResult result = objectMapper.readValue(json, CommandResult.class);
            CommandResultHandler handler = resultHandler.get();
            if (handler == null) {
                log.warn("Nincs eredménykezelő, eldobott üzenet: {}", json);
                return;
            }
            handler.onResult(result);
        } catch (Exception e) {
            log.warn("Érvénytelen cmd/result üzenet a(z) {} topicon: {}", topic, json, e);
        }
    }

    @Override
    public void connectionLost(Throwable cause) {
```

```
        log.warn("Az MQTT kapcsolat megszakadt; automatikus újracsatlakozás", cause);
    }

    @Override
    public void deliveryComplete(IMqttDeliveryToken token) {
        // A QoS 1 kézbesítési nyugta nem azonos a parancs végrehajtásának eredményével.
    }

    @PreDestroy
    public void close() {
        try {
            if (client.isConnected()) {
                client.disconnect();
            }
            client.close();
        } catch (MqttException e) {
            log.warn("Az MQTT kapcsolat lezárása nem sikerült", e);
        }
    }
}
```

A `CommandService` a parancsok tárolója, az eredményfeldolgozó és a határidőfigyelő. A `ConcurrentHashMap` és a `computeIfPresent` biztosítja, hogy egy terminális állapotot későbbi `ACCEPTED` vagy ismételt eredmény ne írjon vissza. A `PENDING` a kézbesítési határidőig (`expiry`), az `ACCEPTED` a végrehajtási határidőig (`execution_timeout_ms`) vár; a kettő külön időkorlát.

```
// src/main/java/com/terrasense/backend/command/CommandService.java
package com.terrasense.backend.command;

import com.terrasense.backend.mqtt.MqttCommandGateway;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.http.HttpStatus;
import org.springframework.scheduling.annotation.Scheduled;
import org.springframework.stereotype.Service;
import org.springframework.web.server.ResponseStatusException;

import java.util.Comparator;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;
import java.util.Optional;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.ConcurrentMap;

@Service
public class CommandService implements CommandResultHandler {
    private static final Logger log = LoggerFactory.getLogger(CommandService.class);
    private static final long DEFAULT_TTL_MS = 30_000;
    private static final long DEFAULT_DURATION_MS = 600_000;
    private static final long MAX_DURATION_MS = 600_000;
    private static final long MAX_EXECUTION_TIMEOUT_MS = 3_600_000;

    private final ConcurrentMap<String, DeviceCommand> commands = new ConcurrentHashMap<>();
    private final MqttCommandGateway gateway;

    public CommandService(MqttCommandGateway gateway) {
        this.gateway = gateway;
        gateway.setResultHandler(this);
    }

    public DeviceCommand createAndPublish(String site, String nodeId, String type,
                                          Map<String, Object> params,
```

```
                                    long ttlMs, long executionTimeoutMs) {
        String safeSite = (site == null || site.isBlank()) ? "gh1" : site.trim();
        if (nodeId == null || nodeId.isBlank()) {
            throw badRequest("A nodeId kötelező");
        }
        if (type == null || type.isBlank()) {
            throw badRequest("A type kötelező");
        }
        Map<String, Object> safeParams = params == null
                ? new LinkedHashMap<>() : new LinkedHashMap<>(params);

        long ttl = ttlMs > 0 ? ttlMs : DEFAULT_TTL_MS;
        if (ttl < 1_000 || ttl > 600_000) {
            throw badRequest("A ttl_ms 1000 és 600000 közé essen");
        }

        long duration = 0;
        if ("valve.open".equals(type)) {
            duration = longParam(safeParams, "duration_ms", DEFAULT_DURATION_MS);
            if (duration < 1_000 || duration > MAX_DURATION_MS) {
                throw badRequest("A duration_ms 1000 és 600000 közé essen");
            }
            long maxOnTime = longParam(safeParams, "max_on_time_ms", duration);
            if (maxOnTime < duration || maxOnTime > MAX_DURATION_MS) {
                throw badRequest("A max_on_time_ms nem lehet kisebb a duration_ms-nál");
            }
            safeParams.put("duration_ms", duration);
            safeParams.put("max_on_time_ms", maxOnTime);
        } else if ("valve.close".equals(type)) {
            safeParams.remove("duration_ms");
            safeParams.remove("max_on_time_ms");
        } else {
            throw badRequest("Ismeretlen parancstípus: " + type);
        }

        long execution = executionTimeoutMs > 0 ? executionTimeoutMs
                : ("valve.open".equals(type) ? duration + 10_000 : 10_000);
        long minimumExecution = "valve.open".equals(type) ? duration + 1_000 : 1_000;
        if (execution < minimumExecution || execution > MAX_EXECUTION_TIMEOUT_MS) {
            throw badRequest("Az execution_timeout_ms érvénytelen");
        }

        DeviceCommand command = DeviceCommand.create(safeSite, nodeId, type, safeParams,
                ttl, execution);
        commands.put(command.id(), command);
        try {
            gateway.publishCommand(command);
            log.info("Parancs létrehozva: id={}, type={}, state={}",
                    command.id(), command.type(), command.state());
            return command;
        } catch (Exception e) {
            DeviceCommand failed = command.withState(CommandState.FAILED,
                    System.currentTimeMillis(),
                    Map.<String, Object>of("error", "publish_failed",
                            "message", String.valueOf(e.getMessage())));
            commands.put(command.id(), failed);
            log.error("A parancs publikálása nem sikerült: id={}", command.id(), e);
            throw new ResponseStatusException(HttpStatus.BAD_GATEWAY,
                    "A parancs publikálása nem sikerült", e);
        }
    }

    @Override
```

```
public void onResult(CommandResult result) {
    if (result.id() == null || result.status() == null) {
        log.warn("Hiányos cmd/result üzenet: {}", result);
        return;
    }
    DeviceCommand updated = commands.computeIfPresent(result.id(), (id, current) -> {
        if (current.isTerminal()) {
            log.info("Késői vagy ismételt eredmény eldobva: id={}, state={}, érkezett={}",
                    id, current.state(), result.status());
            return current;
        }
        if (!isAllowed(current.state(), result.status())) {
            log.warn("Érvénytelen állapotátmenet: id={}, {} -> {}",
                    id, current.state(), result.status());
            return current;
        }
        log.info("Parancs állapot: id={}, {} -> {}", id, current.state(), result.status());
        return current.withState(result.status(), System.currentTimeMillis(),
                result.result() == null ? Map.<String, Object>of() : result.result());
    });
    if (updated == null) {
        log.warn("Ismeretlen parancs eredménye: id={}", result.id());
    }
}

private static boolean isAllowed(CommandState from, CommandState to) {
    return switch (from) {
        case PENDING -> to == CommandState.ACCEPTED || to == CommandState.SUCCESS
                || to == CommandState.FAILED || to == CommandState.EXPIRED;
        case ACCEPTED -> to == CommandState.SUCCESS || to == CommandState.FAILED
                || to == CommandState.EXPIRED;
        default -> false;
    };
}

@Scheduled(fixedDelay = 1_000)
public void checkDeadlines() {
    long now = System.currentTimeMillis();
    commands.replaceAll((id, command) -> {
        if (command.state() == CommandState.PENDING && now > command.expiry()) {
            log.warn("A parancs kézbesítési határideje lejárt: id={}", id);
            return command.withState(CommandState.TIMEOUT, now,
                    Map.<String, Object>of("reason", "latest_start_expired"));
        }
        if (command.state() == CommandState.ACCEPTED
                && now > command.acceptedAt() + command.executionTimeoutMs()) {
            log.warn("A parancs végrehajtási határideje lejárt: id={}", id);
            return command.withState(CommandState.TIMEOUT, now,
                    Map.<String, Object>of("reason", "execution_timeout"));
        }
        return command;
    });
}

public Optional<DeviceCommand> find(String id) {
    return Optional.ofNullable(commands.get(id));
}

public List<DeviceCommand> listForNode(String nodeId) {
    return commands.values().stream()
            .filter(command -> command.nodeId().equals(nodeId))
            .sorted(Comparator.comparingLong(DeviceCommand::createdAt))
            .toList();
```

```
    }

    private static long longParam(Map<String, Object> params, String name, long defaultValue) {
        Object value = params.get(name);
        if (value == null) {
            return defaultValue;
        }
        if (!(value instanceof Number number)) {
            throw badRequest(name + " nem szám");
        }
        return number.longValue();
    }

    private static ResponseStatusException badRequest(String message) {
        return new ResponseStatusException(HttpStatus.BAD_REQUEST, message);
    }
}
```

Az ütemezett határidőfigyelőhöz engedélyezni kell a Spring ütemezést. Ez a `ts-backend` induló alkalmazásosztályán vagy egy külön konfigurációs osztályon is elhelyezhető; itt utóbbit használjuk.

```
// src/main/java/com/terrasense/backend/config/SchedulingConfig.java
package com.terrasense.backend.config;

import org.springframework.context.annotation.Configuration;
import org.springframework.scheduling.annotation.EnableScheduling;

@Configuration
@EnableScheduling
public class SchedulingConfig {
}
```

A REST vezérlő a parancs létrehozását, állapotának lekérdezését és egy node parancsainak listázását kínálja. A POST `202 Accepted` választ ad, mert a parancs végrehajtása aszinkron; a válasz `Location` fejlécében a parancs lekérdezési URL-je szerepel.

```
// src/main/java/com/terrasense/backend/controller/CommandController.java
package com.terrasense.backend.controller;

import com.terrasense.backend.command.CommandRequest;
import com.terrasense.backend.command.CommandService;
import com.terrasense.backend.command.DeviceCommand;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;
import org.springframework.web.server.ResponseStatusException;

import java.net.URI;
import java.util.List;

@RestController
@RequestMapping("/api/devices/{nodeId}/commands")
public class CommandController {
    private final CommandService commandService;

    public CommandController(CommandService commandService) {
        this.commandService = commandService;
```

```
    }

    @PostMapping
    public ResponseEntity<DeviceCommand> create(@PathVariable String nodeId,
                                                @RequestBody CommandRequest request) {
        long ttl = request.ttlMs() == null ? 30_000 : request.ttlMs();
        long execution = request.executionTimeoutMs() == null ? 0 :
         ↪  request.executionTimeoutMs();
        DeviceCommand command = commandService.createAndPublish(request.site(), nodeId,
                request.type(), request.params(), ttl, execution);
        return ResponseEntity.accepted()
                .location(URI.create("/api/devices/" + nodeId + "/commands/" + command.id()))
                .body(command);
    }

    @GetMapping("/{commandId}")
    public DeviceCommand get(@PathVariable String nodeId, @PathVariable String commandId) {
        return commandService.find(commandId)
                .filter(command -> command.nodeId().equals(nodeId))
                .orElseThrow(() -> new ResponseStatusException(HttpStatus.NOT_FOUND,
                        "A parancs nem található"));
    }

    @GetMapping
    public List<DeviceCommand> list(@PathVariable String nodeId) {
        return commandService.listForNode(nodeId);
    }
}
```

Az `application.yaml` MQTT-beállítása (a Spring Boot váz konfigurációjába illesztve):

```
terrasense:
  mqtt:
    broker-url: tcp://localhost:1883
    client-id: ts-backend-cmd
```

A szimulált `ts-node` Pythonban. A szimulátor Paho 2.x-et használ, a callback csak validál és sorba ad; a hosszú szelepnyitást külön munkaszál és időzítő végzi, így a hálózati ciklus és a keepalive nem blokkol. A parancsot a `expiry`, a típus, a cél, a paramétertartomány és a korábbi parancsazonosító alapján ellenőrzi még az `ACCEPTED` előtt. A `valve.close` hosszú nyitás közben is azonnal feldolgozódik, és a nyitás eredménye `closed_early` jelzéssel zárul.

```
#!/usr/bin/env python3
"""ts-node-sim: a 17.1 parancsút szimulátora (paho-mqtt 2.x)."""
import json
import queue
import threading
import time

import paho.mqtt.client as mqtt

BROKER = "localhost"
PORT = 1883
SITE = "gh1"
NODE = "n001"
CMD_TOPIC = f"terrasense/{SITE}/{NODE}/cmd"
RESULT_TOPIC = f"terrasense/{SITE}/{NODE}/cmd/result"
MIN_DURATION_MS = 1_000
MAX_DURATION_MS = 600_000

work_queue: "queue.Queue[dict]" = queue.Queue()
processed: dict[str, dict] = {}
in_progress: set[str] = set()
```

```
state_lock = threading.Lock()
valve_lock = threading.Lock()
valve_open = False
open_cmd = None
open_started = 0.0
open_timer = None
shutdown = threading.Event()
client = None


def now_ms() -> int:
    return int(time.time() * 1000)


def publish_result(payload: dict) -> None:
    if client is not None:
        client.publish(RESULT_TOPIC, json.dumps(payload), qos=1)


def command_expired(cmd: dict) -> bool:
    expiry = cmd.get("expiry")
    return (isinstance(expiry, (int, float)) and not isinstance(expiry, bool)
            and now_ms() > int(expiry))


def valid_command(cmd: object) -> tuple[bool, str]:
    if not isinstance(cmd, dict):
        return False, "a parancs nem JSON-objektum"
    if not isinstance(cmd.get("id"), str) or not cmd["id"]:
        return False, "hiányzó vagy érvénytelen id"
    if cmd.get("node_id") not in (None, NODE):
        return False, "a parancs más node-nak szól"
    if cmd.get("type") not in ("valve.open", "valve.close"):
        return False, "ismeretlen parancstípus"
    if not isinstance(cmd.get("expiry"), (int, float)) or isinstance(cmd.get("expiry"), bool):
        return False, "hiányzó vagy érvénytelen expiry"
    params = cmd.get("params", {})
    if not isinstance(params, dict):
        return False, "a params nem objektum"
    if cmd["type"] == "valve.open":
        duration = params.get("duration_ms")
        if not isinstance(duration, int) or isinstance(duration, bool):
            return False, "a duration_ms nem egész szám"
        if duration < MIN_DURATION_MS or duration > MAX_DURATION_MS:
            return False, "a duration_ms a megengedett tartományon kívül van"
        max_on_time = params.get("max_on_time_ms", duration)
        if not isinstance(max_on_time, int) or isinstance(max_on_time, bool):
            return False, "a max_on_time_ms nem egész szám"
        if max_on_time < duration or max_on_time > MAX_DURATION_MS:
            return False, "a max_on_time_ms érvénytelen"
    return True, ""


def result_payload(cmd: dict, status: str, result: dict | None = None) -> dict:
    payload = {
        "id": cmd["id"],
        "status": status,
        "node_id": NODE,
        "site": SITE,
        "timestamp": now_ms(),
    }
    if result is not None:
        payload["result"] = result
```

```
    return payload


def store_result(command_id: str, payload: dict) -> None:
    with state_lock:
        processed[command_id] = payload
        in_progress.discard(command_id)


def finish_open(cmd: dict, started: float) -> None:
    global valve_open, open_cmd, open_timer
    with valve_lock:
        if open_cmd is not cmd:
            return
        open_cmd = None
        open_timer = None
        valve_open = False
    elapsed = int((time.monotonic() - started) * 1000)
    payload = result_payload(cmd, "SUCCESS", {
        "valve_opened_ms": elapsed,
        "closed_early": False,
        "reason": "duration_elapsed",
    })
    store_result(cmd["id"], payload)
    publish_result(payload)
    print(f"[ts-node-sim] szelep: CLOSED (időzítő, {elapsed} ms)")


def handle_open(cmd: dict) -> None:
    global valve_open, open_cmd, open_started, open_timer
    duration = int(cmd["params"]["duration_ms"])
    max_on_time = int(cmd["params"].get("max_on_time_ms", duration))
    effective = min(duration, max_on_time)
    with valve_lock:
        previous = open_cmd
        previous_started = open_started
        if open_timer is not None:
            open_timer.cancel()
            open_timer = None
        open_cmd = cmd
        open_started = time.monotonic()
        valve_open = True
        timer = threading.Timer(effective / 1000.0, finish_open,
                                args=(cmd, open_started))
        timer.daemon = True
        open_timer = timer
        timer.start()
    publish_result(result_payload(cmd, "ACCEPTED"))
    print(f"[ts-node-sim] szelep: OPEN (max {effective} ms)")
    if previous is not None and previous["id"] != cmd["id"]:
        elapsed = int((time.monotonic() - previous_started) * 1000)
        old_payload = result_payload(previous, "SUCCESS", {
            "valve_opened_ms": elapsed,
            "closed_early": True,
            "superseded_by": cmd["id"],
        })
        store_result(previous["id"], old_payload)
        publish_result(old_payload)


def handle_close(cmd: dict) -> None:
    global valve_open, open_cmd, open_started, open_timer
    with valve_lock:
```

```
            previous = open_cmd
            previous_started = open_started
            valve_open = False
            if open_timer is not None:
                open_timer.cancel()
                open_timer = None
            open_cmd = None
            open_started = 0.0
        publish_result(result_payload(cmd, "ACCEPTED"))
        close_payload = result_payload(cmd, "SUCCESS", {"valve_open": False})
        store_result(cmd["id"], close_payload)
        publish_result(close_payload)
        if previous is not None:
            elapsed = int((time.monotonic() - previous_started) * 1000)
            open_payload = result_payload(previous, "SUCCESS", {
                "valve_opened_ms": elapsed,
                "closed_early": True,
                "closed_by": cmd["id"],
            })
            store_result(previous["id"], open_payload)
            publish_result(open_payload)


    def handle_command(cmd: dict) -> None:
        command_id = cmd["id"]
        with state_lock:
            previous = processed.get(command_id)
            if previous is None and command_id in in_progress:
                previous = result_payload(cmd, "ACCEPTED")
            if previous is None:
                in_progress.add(command_id)
        if previous is not None:
            print(f"[ts-node-sim] duplikált parancs, eredmény újraküldve: {command_id}")
            publish_result(previous)
            return
        if command_expired(cmd):
            payload = result_payload(cmd, "EXPIRED",
                                     {"reason": "a parancs a kézbesítéskor már lejárt"})
            store_result(command_id, payload)
            publish_result(payload)
            print(f"[ts-node-sim] lejárt parancs: {command_id}")
            return
        if cmd["type"] == "valve.open":
            handle_open(cmd)
        else:
            handle_close(cmd)


    def execute() -> None:
        while not shutdown.is_set():
            try:
                cmd = work_queue.get(timeout=0.5)
            except queue.Empty:
                continue
            if cmd is None:
                work_queue.task_done()
                break
            try:
                handle_command(cmd)
            except Exception as exc:
                print(f"[ts-node-sim] hiba: {exc}")
                with state_lock:
                    in_progress.discard(cmd.get("id"))
```

```
            if isinstance(cmd, dict) and isinstance(cmd.get("id"), str):
                payload = result_payload(cmd, "FAILED", {"error": str(exc)})
                store_result(cmd["id"], payload)
                publish_result(payload)
        finally:
            work_queue.task_done()


def on_connect(client, userdata, flags, reason_code, properties) -> None:
    if reason_code.is_failure:
        print(f"[ts-node-sim] kapcsolódási hiba: {reason_code}")
        return
    client.subscribe(CMD_TOPIC, qos=1)
    print(f"[ts-node-sim] feliratkozva: {CMD_TOPIC}")


def on_disconnect(client, userdata, disconnect_flags, reason_code, properties) -> None:
    print(f"[ts-node-sim] kapcsolat bontva: {reason_code}")


def on_message(client, userdata, msg) -> None:
    try:
        cmd = json.loads(msg.payload.decode("utf-8"))
    except (UnicodeDecodeError, json.JSONDecodeError) as exc:
        print(f"[ts-node-sim] érvénytelen JSON: {exc}")
        return
    ok, reason = valid_command(cmd)
    if not ok:
        print(f"[ts-node-sim] elutasítva ({reason})")
        if isinstance(cmd, dict) and isinstance(cmd.get("id"), str):
            publish_result(result_payload(cmd, "FAILED", {"error": reason}))
        return
    work_queue.put(cmd)


def main() -> None:
    global client
    client = mqtt.Client(mqtt.CallbackAPIVersion.VERSION2, client_id="ts-node-sim")
    client.on_connect = on_connect
    client.on_disconnect = on_disconnect
    client.on_message = on_message
    client.reconnect_delay_set(min_delay=1, max_delay=30)
    worker = threading.Thread(target=execute, daemon=True)
    worker.start()
    try:
        client.connect(BROKER, PORT, keepalive=60)
        client.loop_forever()
    except OSError as exc:
        raise SystemExit(f"[ts-node-sim] nem érhető el a broker: {exc}") from exc
    except KeyboardInterrupt:
        print("[ts-node-sim] leállítás")
    finally:
        shutdown.set()
        work_queue.put(None)
        try:
            client.disconnect()
        except Exception:
            pass


if __name__ == "__main__":
    main()
```

A Python-környezetet a könyv egészében ugyanígy hozzuk létre; a Paho 2.x kötelező, mert a callback-szignatúrák eltérnek az 1.x verziótól. A `CallbackAPIVersion.VERSION2` használatával a `on_connect` ötparaméteres, a `reason_code` pedig `ReasonCode` objektum.

```
cd ~/iot-labor/i049
python3 -m venv .venv
. .venv/bin/activate
python -m pip install --upgrade pip
python -m pip install "paho-mqtt==2.1.0"
```

A futtatáshoz indítsuk el a Mosquitto brokert a közös stackből (a szolgáltatás és a konténer neve is `lab-broker`), majd a backendet és a szimulátort. A backend a `ts-backend` könyvtárból fut, a szimulátor a saját virtuális környezetéből.

```
# Mosquitto indítása a ~/iot-labor/stack segítségével
docker compose -f ~/iot-labor/stack/compose.yaml up -d lab-broker
# Backend build és futtatás
cd ~/iot-labor/i049/ts-backend && mvn spring-boot:run
# Szimulátor egy másik terminálban
cd ~/iot-labor/i049 && . .venv/bin/activate && python ts-node-sim.py
# Parancs küldése
curl -i -X POST http://localhost:8080/api/devices/n001/commands \
  -H "Content-Type: application/json" \
  -d '{"type":"valve.open","params":{"duration_ms":10000},"ttl_ms":30000}'
```

A POST válasza `202 Accepted`, a törzsben a `PENDING` állapotú parancs szerepel, a `Location` fejlécben pedig a lekérdezési útvonal. A parancs állapotát ezen az útvonalon vagy a lista-végponton kérdezhetjük le:

```
curl -s http://localhost:8080/api/devices/n001/commands/<parancs-id>
curl -s http://localhost:8080/api/devices/n001/commands
```

Illusztratív kimenet a backend naplójából (a pontos időbélyegek és azonosítók eltérnek):

```
Parancs létrehozva: id=..., type=valve.open, state=PENDING
Parancs publikálva: topic=terrasense/gh1/n001/cmd, id=..., type=valve.open
Parancs állapot: id=..., PENDING -> ACCEPTED
Parancs állapot: id=..., ACCEPTED -> SUCCESS
```

A szimulátor terminálján a parancs, az `ACCEPTED` nyugta és a szelepállapot jelenik meg; a 10 másodperces nyitás után a `SUCCESS` eredmény a `cmd/result` topicra kerül. A naplóban a `TIMEOUT` akkor jelenik meg, ha a parancs a `PENDING` fázisban a `ttl_ms` alatt nem kap nyugtát, vagy az `ACCEPTED` után az `execution_timeout_ms` alatt nem érkezik eredmény. A `TIMEOUT` és az `EXPIRED` különbözik: előbbi a backend saját határideje, utóbbi az eszköz jelzése arról, hogy a parancsot már a kézbesítéskor lejártnak találta.

Takarítás:

```
docker compose -f ~/iot-labor/stack/compose.yaml down
```

A labor során létrehozott fájlok: `ts-backend/src/main/java/com/terrasense/backend/command/` alatt `CommandState.java`, `DeviceCommand.java`, `CommandPayload.java`, `CommandResult.java`, `CommandRequest.java`, `CommandResultHandler.java`, `CommandService.java`; `mqtt/MqttCommandGateway.ja` `controller/CommandController.java`; `config/SchedulingConfig.java`; valamint a `ts-node-sim.py`. A `~/iot-labor/i049` könyvtár marad.

17.1.7 Ellenőrzés és hibakeresés

A parancsút működését a backend naplójában a parancsazonosítóval és az állapotváltásokkal ellenőrizhetjük. A Mosquitto konténer logja (`docker logs lab-broker`) mutatja a publikált üzenetek topicjait és QoS-ét. A szimulátor `print` kimenete igazolja, hogy a nyugta és az eredmény megérkezett. A `docker logs lab-broker` a közös stack `lab-broker` szolgáltatásának/konténerének naplóját mutatja; más laborfelállásban a saját szolgáltatás- és konténernevet kell használni.

Tipikus hibatünetek:

Hibatünet	Lehetséges ok	Javítás
A parancs `PENDING` marad	Az eszköz nincs feliratkozva a `cmd` topicra, a topic elgépelt, vagy a gateway nem tudott csatlakozni a brokerhez	Ellenőrizze a szimulátor feliratkozását, a topic prefixet és a `docker ps` / `docker logs lab-broker` kimenetét
A parancs a publikálás után azonnal `FAILED`	A broker nem érhető el, vagy a `publishCommand` kivételt dobott	Indítsa el a `broker` szolgáltatást, és nézze meg a backend `publish_failed` naplóbejegyzését
A nyugta megérkezik, de nincs eredmény	A szimulátor munkaszála kivételt dobott, az időzítő nem hívta meg a `finish_open`-t, vagy a backend nem figyeli a `cmd/result` topicot	Vizsgálja a szimulátor stderr-jét, ellenőrizze a `connectComplete` wildcard feliratkozását
Duplikált parancs végrehajtás	Az eszköz nem jegyzi meg a feldolgozott azonosítókat	A `processed` szótár és az `in_progress` halmaz már kezeli az ismételt `id`-t; ellenőrizze, hogy a szimulátor ezt a két adatszerkezetet használja
A parancs `PENDING`-ben marad, majd `TIMEOUT` lesz	Az `expiry` (legkésőbbi start) lejárt, mert az eszköz nem vette át a parancsot	Növelje a `ttl_ms`-t az eszköz ébredési idejéhez, vagy használjon alvó eszközökhöz outboxot
A parancs `ACCEPTED` után `TIMEOUT` lesz	Az `execution_timeout_ms` túl rövid, vagy az eredmény elveszett	Növelje az `execution_timeout_ms`-t, és ellenőrizze a `cmd/result` kézbesítést
Az alvó eszköz nem kap parancsot	A laborban az eszköz online; az alvó eszközök outboxa nem része a labornak	Az alvó eszközökhöz a `status` üzenetre reagáló outboxot kell megvalósítani

17.1.8 Tipikus hibák és félreértések

1. **A nyugta és az eredmény összemosása.** Ha csak egy üzenetet várunk a végrehajtás végén, a felhasználó hosszú ideig nem tudja, hogy a parancs eljutott-e. A szelepnyitásnál ez különösen zavaró: a `ACCEPTED` nyugta nélkül a backend nem tudja megkülönböztetni a hálózati hibát a lassú végrehajtástól.

2. **Az idempotencia elhanyagolása.** Az MQTT QoS 1 legalább egyszer kézbesít, tehát duplikátumok előfordulhatnak. Ha az eszköz minden `valve.open` parancsra újra kinyitja a szelepet, a duplikátum túlöntözést okoz. A parancsazonosító alapú deduplikáció kötelező.
3. **Elavult retained parancs.** Ha valaki mégis retained `cmd` üzenetet használ, és az eszköz ébredéskor feltétel nélkül végrehajtja, egy régi, lejárt parancs váratlan hatást válthat ki. Az `expiry` mezőt mindig ellenőrizni kell.
4. **A két időkorlát összemosása.** Az `expiry` a legkésőbbi start, az `execution_timeout_ms` a nyugta utáni eredményre várakozás. Ha az `expiry` túl rövid, az alvó eszköz ébredése előtt lejár; ha az `execution_timeout_ms` túl rövid, a lassú végrehajtás miatt a backend hamarabb ad `TIMEOUT`-ot, mint hogy az eszköz eredményt küldene. A két értéket külön kell az eszköz viselkedéséhez igazítani.

17.1.9 Feladatok

17.1.9.1 1. Fogalmi kérdés

Miért nem elegendő a parancs `cmd` topicra történő QoS 1 publikálása a megbízható parancskézbesítéshez? Milyen további mechanizmusokra van szükség a végrehajtás igazolásához és a duplikátumok kezeléséhez?

17.1.9.2 2. Tervezési feladat

Egy `ts-node` percenként egyszer ébred fel, és 5 másodpercig hallgatja a rádiót. A backend egy `valve.open` parancsot akar kézbesíteni. Az átlagos hálózati késleltetés MQTT-ben 200 ms, a maximális késleltetés 2 s. Mekkora legyen a parancs `expiry` értéke, ha a parancsot a lehető leghamarabb végre akarjuk hajtani, de nem akarunk feleslegesen hosszú várakozást? Indokoljon.

17.1.9.3 3. Implementációs feladat

Egészítse ki a `ts-backend` `CommandService` osztályát úgy, hogy a `cmd/result` topicról érkező `EXPIRED` állapotú eredmény esetén naplózza az eseményt, és a parancsot `EXPIRED` állapotba állítsa. Készítsen egységtesztet a `CommandService` állapotátmenetére.

17.1.9.4 4. Hibakeresési feladat

A labor során a parancs `ACCEPTED` állapotban marad, de `SUCCESS` soha nem érkezik. A szimulátor naplója szerint a nyugta elküldésre került. Hol keresse a hibát? Soroljon fel legalább három lehetséges okot, és mindegyikhez adjon ellenőrzési lépést.

17.1.10 Megoldások

17.1.10.1 1. Fogalmi kérdés

A QoS 1 csak a brokerbe történő kézbesítést garantálja, de nem garantálja, hogy az eszköz megkapta, és azt sem, hogy végrehajtotta. A nyugta (ACK) igazolja a kézbesítést az eszközig, az eredményüzenet pedig a végrehajtás sikerességét. A duplikátumok ellen a parancsazonosító alapú deduplikáció véd. Az `expiry` a legkésőbbi startot korlátozza, az `execution_timeout_ms` pedig a nyugta utáni eredményre várakozást; a kettő együtt jelzi, ha a parancs nem hajtható végre, illetve nem fejeződik be értelmes időn belül.

17.1.10.2 2. Tervezési feladat

Az eszköz legfeljebb 60 s + 5 s = 65 s alatt ébred fel és hallgatja a rádiót. A hálózati késleltetés maximuma 2 s, tehát a legrosszabb esetben a parancs 65 s + 2 s = 67 s múlva juthat el az eszközhöz. A végrehajtás (például a szelepnyitás) további időt vehet igénybe, de ez már nem az `expiry`, hanem az `execution_timeout_ms` hatásköre. Az `expiry`-t legalább 68 s-ra kell állítani; érdemes 70 s környékére választani, hogy a normál kézbesítés ne ütközzön a lejáratba. Az `execution_timeout_ms` ettől függetlenül a tényleges végrehajtási időhöz igazodjon: rövid szelepnyitásnál néhány másodperc, hosszú nyitásnál a `duration_ms` plusz tartalék. Ugyanakkor ha a parancs a start előtt lejár, a felhasználót értesíteni kell.

17.1.10.3 3. Implementációs feladat

A `CommandService` már tartalmazza az `EXPIRED` eredmény kezelését: az `onResult` a `CommandResult` rekordot dolgozza fel, és a `PENDING` vagy `ACCEPTED` állapotból engedi az `EXPIRED` átmenetet. A feladat megoldása egy JUnit 5 + Mockito teszt, amely a nyilvános `onResult` és `checkDeadlines` API-n keresztül ellenőrzi az állapotot, a késői nyugta és az ismételt eredmény eldobását, valamint a két külön határidőt. A `spring-boot-starter-test` függőség a Spring Boot vázban már szerepel.

```
// src/test/java/com/terrasense/backend/command/CommandServiceTest.java
package com.terrasense.backend.command;

import com.terrasense.backend.mqtt.MqttCommandGateway;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.extension.ExtendWith;
import org.mockito.Mock;
import org.mockito.junit.jupiter.MockitoExtension;

import java.util.Map;

import static org.junit.jupiter.api.Assertions.assertEquals;

@ExtendWith(MockitoExtension.class)
class CommandServiceTest {
    @Mock
    private MqttCommandGateway gateway;

    @Test
    void expiredResultIsStored() {
        CommandService service = new CommandService(gateway);
        DeviceCommand command = service.createAndPublish("gh1", "n001", "valve.close",
                Map.of(), 30_000, 10_000);
        service.onResult(new CommandResult(command.id(), CommandState.EXPIRED,
```

```
            "n001", System.currentTimeMillis(),
            Map.<String, Object>of("reason", "device_expired")));
    assertEquals(CommandState.EXPIRED,
            service.find(command.id()).orElseThrow().state());
}

@Test
void acceptedThenSuccessIsStored() {
    CommandService service = new CommandService(gateway);
    DeviceCommand command = service.createAndPublish("gh1", "n001", "valve.close",
            Map.of(), 30_000, 10_000);
    service.onResult(new CommandResult(command.id(), CommandState.ACCEPTED,
            "n001", System.currentTimeMillis(), Map.of()));
    assertEquals(CommandState.ACCEPTED,
            service.find(command.id()).orElseThrow().state());
    service.onResult(new CommandResult(command.id(), CommandState.SUCCESS,
            "n001", System.currentTimeMillis(), Map.of("valve_open", false)));
    assertEquals(CommandState.SUCCESS,
            service.find(command.id()).orElseThrow().state());
}

@Test
void lateAcceptedDoesNotOverrideSuccess() {
    CommandService service = new CommandService(gateway);
    DeviceCommand command = service.createAndPublish("gh1", "n001", "valve.close",
            Map.of(), 30_000, 10_000);
    service.onResult(new CommandResult(command.id(), CommandState.SUCCESS,
            "n001", System.currentTimeMillis(), Map.of()));
    service.onResult(new CommandResult(command.id(), CommandState.ACCEPTED,
            "n001", System.currentTimeMillis(), Map.of()));
    assertEquals(CommandState.SUCCESS,
            service.find(command.id()).orElseThrow().state());
}

@Test
void repeatedResultDoesNotChangeTerminalState() {
    CommandService service = new CommandService(gateway);
    DeviceCommand command = service.createAndPublish("gh1", "n001", "valve.close",
            Map.of(), 30_000, 10_000);
    service.onResult(new CommandResult(command.id(), CommandState.SUCCESS,
            "n001", System.currentTimeMillis(), Map.of()));
    service.onResult(new CommandResult(command.id(), CommandState.FAILED,
            "n001", System.currentTimeMillis(), Map.of("error", "kesoi_hiba")));
    assertEquals(CommandState.SUCCESS,
            service.find(command.id()).orElseThrow().state());
}

@Test
void pendingExpiryBecomesTimeout() throws InterruptedException {
    CommandService service = new CommandService(gateway);
    DeviceCommand command = service.createAndPublish("gh1", "n001", "valve.close",
            Map.of(), 1_000, 10_000);
    Thread.sleep(1_100);
    service.checkDeadlines();
    DeviceCommand timedOut = service.find(command.id()).orElseThrow();
    assertEquals(CommandState.TIMEOUT, timedOut.state());
    assertEquals("latest_start_expired", timedOut.result().get("reason"));
}

@Test
void acceptedExecutionTimeoutBecomesTimeout() throws InterruptedException {
    CommandService service = new CommandService(gateway);
    DeviceCommand command = service.createAndPublish("gh1", "n001", "valve.close",
```

```
                Map.of(), 30_000, 1_000);
        service.onResult(new CommandResult(command.id(), CommandState.ACCEPTED,
                "n001", System.currentTimeMillis(), Map.of()));
        Thread.sleep(1_100);
        service.checkDeadlines();
        DeviceCommand timedOut = service.find(command.id()).orElseThrow();
        assertEquals(CommandState.TIMEOUT, timedOut.state());
        assertEquals("execution_timeout", timedOut.result().get("reason"));
    }
}
```

A teszt a `gateway` Mockito-mockját használja, így nem kell valódi broker; a `createAndPublish` a mock `publishCommand` hívásán keresztül sikeresen létrehozza a parancsot. A határidős tesztek rövid `Thread.sleep`-t használnak, mert a `CommandService` valós rendszerórát olvas; a laborban így a két időkorlát is végigpróbálható. A tesztek a laborban futtathatók (`mvn test`), a könyv nem állítja, hogy itt lefutottak.

17.1.10.4 4. Hibakeresési feladat

Lehetséges okok és ellenőrzésük:

- **A backend nincs feliratkozva a `cmd/result` topicra.** Ellenőrizze a `MqttCommandGateway.connectComp` metódusában a `subscribe` hívást; a wildcard `terrasense/+/+/cmd/result` lefedi-e a `terrasense/gh1/n001/cmd/result` topicot. A `cleanSession=true` miatt újracsatlakozáskor a `connectComplete` fut le ismét, nem elég egyszer a konstruktorban feliratkozni.
- **A szimulátor más `id`-t küld a nyugtában, mint az eredményben.** Hasonlítsa össze a nyugta és az eredmény `id` mezőjét; ha nem egyezik, a backend nem tudja a parancshoz kötni.
- **A szimulátor munkaszála kivételt dob, vagy az időzítő nem hívja meg a `finish_open`-t.** Nézze meg a szimulátor stderr-jét; a `handle_command` kivételkezelése ilyenkor `FAILED` eredményt küld, de az időzítő hibáját külön kell vizsgálni.
- **Az `execution_timeout_ms` lejárt, mielőtt az eredmény megérkezett volna, és a backend `TIMEOUT`-ra állította a parancsot.** Ellenőrizze az `@Scheduled checkDeadlines` futását, és hogy az eredmény később érkezik-e a nyugta utáni határidőnél. Ez nem az `expiry` lejárata: az `expiry` a start előtti határidő.

17.1.11 Rövid összefoglaló

A `ts-backend` parancsútja a `terrasense/<site>/<node>/cmd` topicra publikál, az eszköz pedig a `cmd/result` topicra küld nyugtát és eredményt. A huzalformátum az `id`, `type`, `params`, `expiry`, `execution_timeout_ms` és `timestamp` mezőket tartalmazza; a `CommandPayload` szűkíti a belső `DeviceCommand` modellt erre a formára. A backend állapotgépe `PENDING`, `ACCEPTED`, `SUCCESS`, `FAILED`, `TIMEOUT`, `EXPIRED` állapotokat kezel, a terminális állapotokat késői üzenet nem írja vissza, és az `@Scheduled checkDeadlines` külön kezeli a `PENDING` kézbesítési (`expiry`) és az `ACCEPTED` végrehajtási (`execution_timeout_ms`) határidejét. A laborban az eszköz online, ezért a `CommandService` közvetlenül publikál; az alvó eszközök `status` üzenetre reagáló outboxa későbbi kiterjesztés. A parancsok tárolása a laborban memóriában történik; éles rendszerben tartós tárra van szükség. A laborban a `ts-backend` REST API-val fogad parancsot, MQTT-n publikál, és egy Paho 2.x-es Python szimulátor hajtja végre a szelepnyitást, nem blokkoló callbackkel, `expiry`-, típus-, paraméter- és duplikátumellenőrzéssel. A kulcsfájlok: `CommandState.java`, `DeviceCommand.java`, `CommandPayload.java`, `CommandResult.java`, `CommandRequest.java`,

`CommandResultHandler.java`, `CommandService.java`, `MqttCommandGateway.java`, `CommandController.java` `SchedulingConfig.java`, `ts-node-sim.py`. **Továbbvitt eredmény.** A parancsazonosítóval történő korreláció, a `cmd` és `cmd/result` topicok, valamint az állapotgép az Eszközárnyék és digitális iker: kívánt és jelentett állapot alfejezetben a kívánt állapot kézbesítésére, a Záróprojekt: a teljes TerraSense rendszer összeállítása és átvétele alfejezetben a teljes parancsút integrációjára épül.

17.1.12 Források és ellenőrzési hiányok

Az MQTT publish/subscribe és QoS 1 definíciói az MQTT 3.1.1 és MQTT 5.0 szabványokon alapulnak. A labor a rögzített verziómátrix szerint Spring Boot 3.3.4-et, Java 21-et, Paho Java 1.2.5-öt és Python `paho-mqtt` 2.1.0-t használ; a Java gateway a klasszikus v3 API-t, a Python szimulátor a 2.x `CallbackAPIVersion.VERSION2` API-t használja. Az MQTT 5 request–response minta a szabvány `Response Topic` és `Correlation Data` user property-ire épül, de a TerraSense-ben a saját topic-konvenciót használjuk. A laborban szimulált eszköz nem helyettesíti a valós Zephyr `native_sim` eszközt; a teljes eszközszoftvert az MQTT a gyakorlatban: Zephyr-kliens és Mosquitto alfejezet szerint kell integrálni.

17.2 Eszközárnyék és digitális iker: kívánt és jelentett állapot

17.2.1 Motiváció

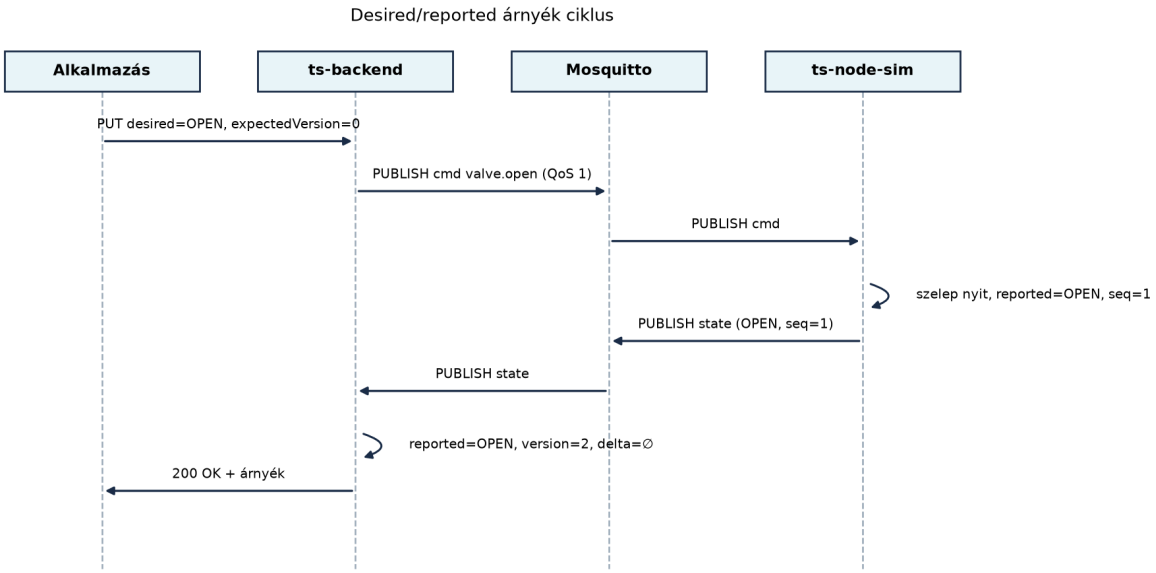
Az Parancsok és konfiguráció lefelé: kézbesítés, nyugtázás, időkorlát alfejezetben a `ts-backend` parancsot publikált a `cmd` topicra, az eszköz pedig a `cmd/result` topicon nyugtázott. A parancsút azonban önmagában nem ad választ egy alapvető kérdésre: mit tegyen a felhasználó vagy a szabálymotor, amíg az eszköz nem nyugtáz? A TerraSense-ben például a kezelő beállítja, hogy a `gh1/n001` csomópont öntözőszelepe legyen nyitva. A parancs elmegy MQTT-n, de az eszköz éppen alszik vagy rossz a rádiós kapcsolat. A `cmd/result` csak később, esetleg sosem érkezik meg. Az eszközárnyék (device shadow) ezt a problémát úgy oldja meg, hogy a backendben külön tárolja a **kívánt** (desired) és az eszköz által **jelentett** (reported) állapotot, a kettő eltéréséből pedig **deltát** számít. Így a felhasználó azonnal látja, hogy a kívánt állapot még nem teljesült, és az eszköz következő felébredésekor a delta újra kézbesíthető.

17.2.2 Fogalmak és működés

Definíció. Az **eszközárnyék (device shadow)** az IoT-eszköz backendben tárolt, szerializálható, üzemeltetési nézetű reprezentációja, amely legalább a kívánt állapotot (`desired`), az eszköz által jelentett tényleges állapotot (`reported`) és ezek eltérését (`delta`) tartalmazza. A **desired** a rendszer vagy a felhasználó által célként megadott állapot, a **reported** az eszköz legutolsó állapotüzenetéből nyert tényleges állapot. A **delta** a desired és a reported rendezett összehasonlítása; ha a kettő nem egyezik, a delta a kívánt értéket hordozza, és azt jelzi, hogy van még végrehajtandó munka.

A digitális iker ennél több. **Definíció.** A **digitális iker** az eszköz teljes, sokszor strukturált, térbeli, fizikai vagy viselkedési modellje, amely szimulációra, előrejelzésre és elemzésre is alkalmas. Az eszközárnyék gyakorlatorientált, kis erőforrású üzemeltetési nézet; a digitális iker a tervezési, üzemeltetési és elemzési rétegek közös modellje. A TerraSense-ben az eszközárnyék a szelep desired/reported állapotát tárolja, a digitális iker pedig például az üvegház hőmodelljét, az öntözés hatását és a várható termést is magában foglalná.

A működést a következő üzenetsor mutatja:



17.2. ábra: Desired/reported árnyék ciklus - A kívánt szelepállapot beállítása, MQTT parancs, eszköz jelentése és a delta frissülése.

Az árnyék három forrásból frissül. A **desired** frissítése jellemzően REST API-n, riasztási szabályból vagy parancsrétegből érkezik, és a `cmd` topicra publikált parancsot váltja ki. A **reported** frissítése az eszköz `state` topicra küldött üzenetéből származik. A delta nem önálló üzenet, hanem a tárolt desired és reported összehasonlításából számított, származtatott mező.

Verziószám és konfliktus. Mivel a desired állapotot több alkalmazás is módosíthatja, az árnyék minden íráskor növekvő `version` értéket kap. A TerraSense laborban **optimista konkurenciavezérlést** alkalmazunk: az ügyfél a `PUT` hívásban elküldi az általa ismert `expectedVersion` értéket. Ha az eltér a tárolt verziótól, a backend `409 Conflict` választ ad, és az ügyfélnek újra kell olvasnia az árnyékot. Ez megakadályozza, hogy egy elavult kliens felülírjon egy időközben beérkezett, újabb desired állapotot.

Retained üzenet vs. backend-tárolt árnyék. Az MQTT retained üzenet a brokerben tárolja az utolsó, adott topicra publikált üzenetet, és az újonnan feliratkozó kliensek azonnal megkapják. Ez egyszerű megoldás az utolsó `state` vagy akár `desired` érték megőrzésére. A backend-tárolt árnyék azonban többet tud: együtt tárolja a desired és reported párokat, verziószámot, időbélyeget, delta számítást, REST API-t és tranzakcionális frissítést. A retained üzenetnek nincs delta fogalma és nem tudja kezelni a konkurens írásokat.

Szempont	Retained üzenet	Backend-tárolt árnyék
Desired és reported együtt	csak üzenetenként	egy objektumban
Delta számítás	nincs	van
Verzió és konfliktuskezelés	nincs	van
REST/API elérés	nincs	van
Broker-függőség	erős	gyenge
Tárolt adatmennyiség	egy üzenet	rendezett rekord

A döntés TerraSense-ben egyértelmű: mivel a desired állapotot REST API-n akarjuk kezelni,

és a delta alapján újra kell kézbesíteni a parancsot, a backend-tárolt árnyékot választjuk. A retained `state` üzenetet továbbra is használhatjuk a frissen csatlakozó eszközök gyors felzárkóztatására, de az árnyék igazságforrása a backend tartós tárolója. A laborban ez memóriabeli `ConcurrentHashMap`, éles rendszerben adatbázis; a laborhatárt a fejezet külön jelzi.

17.2.3 Digitális iker fogalmi szinten és Eclipse Ditto

A digitális iker (digital twin) fogalma nem egyetlen technológia, hanem architekturális minta. A TerraSense üvegházában a digitális iker tartalmazhatná a csomópontok fizikai elhelyezkedését, a szenzorok kalibrációs paramétereit, a szelepek hidraulikai jellemzőit, valamint a szimulációs modellt, amely például a hőmérséklet és a talajnedvesség alakulását előrejelzi. Az eszközárnyék ennek csak egyetlen, üzemeltetés közeli szelete.

Az **Eclipse Ditto** egy nyílt forrású, digitális iker platform, amely REST API-val és WebSocket-csatornákon keresztül teszi elérhetővé a dolgok állapotát. A Ditto a dolgokat JSON-szerű digitális ikerként modellezi, támogatja a desired/reported propertyket, a delta-értesítéseket, a policy-alapú hozzáférés-vezérlést és a HTTP/MQTT/AMQP kapcsolatokat. A nagy felhős szolgáltatások közül az AWS IoT Device Shadow, az Azure IoT Hub Device Twin és a Google Digital Twins hasonló desired/reported modellt kínál. **Ellenőrzési igény.** A Ditto aktuális verziója, pontos Java API-ja és a felhős szolgáltatások konkrét árnyék-semái folyamatosan változnak; a fenti rövid leírás stabil fogalmi összehasonlítás, nem adott verzióra vonatkozó specifikáció.

17.2.4 Kidolgozott példa

Legyen a `gh1/n001` csomópont szelepárnyéka kezdetben:

Mező	Érték	Magyarázat
`desired`	`CLOSED`	a kezelő még nem kért nyitást
`reported`	`CLOSED`	az eszköz `state` üzenete szerint zárva
`delta`		nincs eltérés
`version`	`0`	még nem frissült
`reportedSeq`	`0`	még nem érkezett state

A kezelő azt kéri, hogy a szelep legyen nyitva. A `ts-backend` `PUT /api/shadows/gh1/n001/desired {"valveState":"OPEN","expectedVersion":0,"durationMs":600000}` hívást kap. A `ShadowService` ellenőrzi a verziót, növeli `version`-t 1-re, eltárolja a desired `OPEN` állapotot, beállítja a `desiredUntil` lease-t, és a `cmd` topicra publikál egy `valve.open` parancsot a `duration_ms` és a `max_on_time_ms` paraméterrel. Az árnyék ekkor:

Mező	Érték
`desired`	`OPEN`
`reported`	`CLOSED`
`delta`	`OPEN`
`version`	`1`
`desiredUntil`	a kérés ideje + 660 s

Amint az eszköz végrehajtja a parancsot, a `state` topicra publikálja a `{"site":"gh1","node_id":"n001","valv` üzenetet. A backend a `reported` mezőt frissíti, a `reportedSeq`-et 1-re állítja, és a verzió 2-re nő.
A delta kiürül:

Mező	Érték
desired	OPEN
reported	OPEN
delta	
version	2
reportedSeq	1

A `desiredUntil` lease és a `max_on_time_ms` együtt véd a végtelen öntözés ellen. Ha a szelep a `duration_ms` letelte után is nyitva maradna, az eszköz helyi védelme a `max_on_time_ms` elérésekor zár, és a state üzenetben `fault: "max_on_time"` értéket küld; a backend ekkor a desired állapotot a reported értékre állítja, és nem küld újabb nyitó parancsot. Ha az eszköz egyáltalán nem válaszol, a backend `desiredUntil` lease-e lejár, a desired `CLOSED`-re áll, és a reconciler záró parancsot publikál. A nyitás így legfeljebb a kért időtartamig, illetve a lease lejártáig tarthat.

Ha közben egy másik kliens `expectedVersion=0` értékkel próbált volna írni, a backend `409 Conflict` választ adott volna, mert a verzió már 1 vagy 2. Ez a minta garantálja, hogy a desired állapot változásai követhetők maradnak.

17.2.5 Labor: árnyék a ts-backendben a szelepállapotra

A laborban a `ts-backend` Spring Boot projektet bővítjük árnyékszolgáltatással. A cél: a desired állapot REST PUT-tal beállítható legyen, a parancs MQTT-n a szimulátorhoz jusson, a szimulátor a `state` topicra publikáljon, a backend pedig frissítse a reported mezőt, a verziót és a deltát. A kiindulás a Parancsok és konfiguráció lefelé: kézbesítés, nyugtázás, időkorlát labor `ts-backend` projektje, mert abban már működik a `CommandService` és a `MqttCommandGateway`; így nem kell újra megírni a parancsút alsó rétegét.

Munkakönyvtár:

```
mkdir -p ~/iot-labor/i050
cp -r ~/iot-labor/i049/ts-backend ~/iot-labor/i050/ts-backend
mkdir -p ~/iot-labor/i050/ts-backend/src/main/java/com/terrasense/backend/shadow
```

A Python-környezetet külön hozzuk létre; a Paho 2.x itt is kötelező.

```
cd ~/iot-labor/i050
python3 -m venv .venv
. .venv/bin/activate
python -m pip install --upgrade pip
python -m pip install "paho-mqtt==2.1.0"
```

Előfeltétel. A Mosquitto fut a közös stackben, és a `ts-backend` MQTT konfigurációja elérhető. A backend és a szimulátor két külön MQTT-kliensazonosítót használ (`ts-backend-cmd`, `ts-backend-state`, illetve `ts-node-sim-i050`), mert az azonos kliensazonosítójú kapcsolatokat a broker egymás mellett bontaná.

Első lépésként a `ValveState` enumot, a `ShadowKey` kulcsot és a `DeviceShadow` rekordot hozzuk létre. A rekord a verzió mellett a reported üzenet `seq` értékét, a reported időbélyegét, a desired OPEN lease-határidejét (`desiredUntil`) és az utolsó eszközoldali védelmi hibát is tárolja. A delta származtatott mező; a `delta(long now)` a lejárt OPEN lease esetén üres deltát ad, hogy a reconciler ne nyissa újra a szelepet.

```java
// src/main/java/com/terrasense/backend/shadow/ValveState.java
package com.terrasense.backend.shadow;

public enum ValveState {
    OPEN, CLOSED, UNKNOWN
}

// src/main/java/com/terrasense/backend/shadow/ShadowKey.java
package com.terrasense.backend.shadow;

public record ShadowKey(String site, String node) {
    public ShadowKey {
        if (site == null || site.isBlank() || node == null || node.isBlank()) {
            throw new IllegalArgumentException("A site és a node kötelező");
        }
    }
}

// src/main/java/com/terrasense/backend/shadow/DeviceShadow.java
package com.terrasense.backend.shadow;

import com.fasterxml.jackson.annotation.JsonInclude;
import com.fasterxml.jackson.annotation.JsonProperty;

import java.util.Optional;

public record DeviceShadow(
        ShadowKey key,
        ValveState desired,
        ValveState reported,
        long version,
        long reportedSeq,
        long reportedAt,
        long desiredUntil,
        String lastFault
) {
    public static DeviceShadow initial(ShadowKey key) {
        return new DeviceShadow(key, ValveState.UNKNOWN, ValveState.UNKNOWN,
                0L, 0L, 0L, 0L, null);
    }

    public Optional<ValveState> delta() {
        return delta(System.currentTimeMillis());
    }

    public Optional<ValveState> delta(long now) {
        if (desired == ValveState.UNKNOWN || desired == reported) {
            return Optional.empty();
        }
        if (desired == ValveState.OPEN && desiredUntil > 0 && now > desiredUntil) {
            return Optional.empty();
        }
        return Optional.of(desired);
    }

    @JsonProperty("delta")
    @JsonInclude(JsonInclude.Include.NON_ABSENT)
    public Optional<ValveState> deltaView() {
        return delta();
    }
}
```

A `ShadowService` `ConcurrentHashMap`-ben tárolja az árnyékokat. A desired és a reported frissítése is `compute`-tel történik: a verzióellenőrzés és az írás egy atomi művelet, így egy

párhuzamos GET nem láthat félkész állapotot, és egy elavult `expectedVersion` nem írhat felül egy újabbat. A reported frissítés csak újabb `seq` esetén növeli a verziót; a régi state eldobódik. A desired OPENhez `desiredUntil` lease tartozik, amely a `duration_ms` és egy 60 másodperces ráhagyás összege. A `@Scheduled` reconciler a lejárt lease-t `desired=CLOSED` értékre állítja, és a delta alapján újrapublikálja a parancsot; a publikálási hiba nem veszíti el a desired állapotot, mert a következő kör újrapróbálja. Ez tulajdonképpen memóriabeli outbox: a desired a kézbesítendő munka, a reconciler a kézbesítő; a PUT 200 OK választ ad, mert a desired rögzítése sikerült, a parancs tényleges kézbesítése pedig aszinkron, legfeljebb 30 másodperces késleltetéssel.

```
// src/main/java/com/terrasense/backend/shadow/ShadowService.java
package com.terrasense.backend.shadow;

import com.terrasense.backend.command.CommandService;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.http.HttpStatus;
import org.springframework.scheduling.annotation.Scheduled;
import org.springframework.stereotype.Service;
import org.springframework.web.server.ResponseStatusException;

import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;
import java.util.Optional;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.ConcurrentMap;

@Service
public class ShadowService {
    private static final Logger log = LoggerFactory.getLogger(ShadowService.class);
    private static final long DEFAULT_DURATION_MS = 600_000;
    private static final long MAX_DURATION_MS = 600_000;
    private static final long LEASE_GRACE_MS = 60_000;
    private static final long TTL_MS = 30_000;

    private final ConcurrentMap<ShadowKey, DeviceShadow> shadows = new ConcurrentHashMap<>();
    private final CommandService commandService;

    public ShadowService(CommandService commandService) {
        this.commandService = commandService;
    }

    public DeviceShadow updateDesired(ShadowKey key, ValveState desired,
                                      long expectedVersion, Long durationMs) {
        if (desired != ValveState.OPEN && desired != ValveState.CLOSED) {
            throw new ResponseStatusException(HttpStatus.BAD_REQUEST,
                    "A desired csak OPEN vagy CLOSED lehet");
        }
        long duration = durationMs == null ? DEFAULT_DURATION_MS : durationMs;
        if (desired == ValveState.OPEN
                && (duration < 1_000 || duration > MAX_DURATION_MS)) {
            throw new ResponseStatusException(HttpStatus.BAD_REQUEST,
                    "A durationMs 1000 és 600000 közé essen");
        }
        long now = System.currentTimeMillis();
        long leaseUntil = desired == ValveState.OPEN
                ? now + duration + LEASE_GRACE_MS : 0L;
        DeviceShadow updated = shadows.compute(key, (k, current) -> {
            DeviceShadow base = current == null ? DeviceShadow.initial(k) : current;
            if (base.version() != expectedVersion) {
                throw new ResponseStatusException(HttpStatus.CONFLICT,
                        "Shadow version mismatch: expected " + expectedVersion
```

```
                            + ", actual " + base.version());
        }
        return new DeviceShadow(k, desired, base.reported(),
                base.version() + 1, base.reportedSeq(), base.reportedAt(),
                leaseUntil, base.lastFault());
    });
    publishDesired(updated);
    return updated;
}

public DeviceShadow updateReported(ShadowKey key, ValveState reported,
                                   long seq, long ts, String fault) {
    return shadows.compute(key, (k, current) -> {
        DeviceShadow base = current == null ? DeviceShadow.initial(k) : current;
        if (seq <= base.reportedSeq()) {
            log.info("Elavult state eldobva: key={}, seq={}, utolsó={}",
                    k, seq, base.reportedSeq());
            return base;
        }
        long reportedAt = ts > 0 ? ts : System.currentTimeMillis();
        ValveState desired = base.desired();
        long desiredUntil = base.desiredUntil();
        if (fault != null && !fault.isBlank()) {
            // Az eszköz helyi védelme lezárta a szelepet; ne harcoljunk vele.
            desired = reported;
            desiredUntil = 0L;
            log.warn("Az eszköz védelmi hibát jelentett: key={}, fault={}", k, fault);
        }
        return new DeviceShadow(k, desired, reported, base.version() + 1,
                seq, reportedAt, desiredUntil, fault);
    });
}

public Optional<DeviceShadow> get(ShadowKey key) {
    return Optional.ofNullable(shadows.get(key));
}

public List<ShadowKey> listAllDeltas() {
    long now = System.currentTimeMillis();
    return shadows.values().stream()
            .filter(shadow -> shadow.delta(now).isPresent())
            .map(DeviceShadow::key)
            .toList();
}

@Scheduled(fixedDelay = 30_000)
public void reconcile() {
    long now = System.currentTimeMillis();
    for (Map.Entry<ShadowKey, DeviceShadow> entry : shadows.entrySet()) {
        ShadowKey key = entry.getKey();
        DeviceShadow shadow = entry.getValue();
        if (shadow.desired() == ValveState.OPEN
                && shadow.desiredUntil() > 0 && now > shadow.desiredUntil()) {
            shadows.computeIfPresent(key, (k, current) -> {
                if (current.version() != shadow.version()) {
                    return current;
                }
                log.info("A desired OPEN lease lejárt, biztonsági zárás: {}", k);
                return new DeviceShadow(k, ValveState.CLOSED, current.reported(),
                        current.version() + 1, current.reportedSeq(),
                        current.reportedAt(), 0L, current.lastFault());
            });
            continue;
```

```
            }
            if (shadow.delta(now).isPresent()) {
                publishDesired(shadow);
            }
        }
    }

    private void publishDesired(DeviceShadow shadow) {
        if (shadow.desired() == ValveState.UNKNOWN) {
            return;
        }
        long now = System.currentTimeMillis();
        Map<String, Object> params = new LinkedHashMap<>();
        String type;
        long executionTimeout;
        if (shadow.desired() == ValveState.OPEN) {
            long remaining = shadow.desiredUntil() - now - LEASE_GRACE_MS;
            if (remaining < 1_000) {
                log.warn("A desired OPEN lease-ből nincs biztonságos nyitási idő: {}",
                        shadow.key());
                return;
            }
            long duration = Math.min(MAX_DURATION_MS, remaining);
            params.put("duration_ms", duration);
            params.put("max_on_time_ms", duration);
            type = "valve.open";
            executionTimeout = duration + 10_000;
        } else {
            type = "valve.close";
            executionTimeout = 10_000;
        }
        try {
            commandService.createAndPublish(shadow.key().site(), shadow.key().node(),
                    type, params, TTL_MS, executionTimeout);
        } catch (RuntimeException e) {
            // A desired megmarad; a reconciler a következő körben újrapróbálja.
            log.warn("A desired parancs publikálása nem sikerült: {}", shadow.key(), e);
        }
    }
}
```

A REST vezérlő a desired beállítását és az árnyék lekérdezését teszi elérhetővé. A PUT a `ShadowKey`-t az útvonalból, a kért állapotot és az `expectedVersion`-t a kérésből veszi; a `durationMs` opcionális, és csak OPEN esetén értelmezett.

```
// src/main/java/com/terrasense/backend/shadow/DesiredRequest.java
package com.terrasense.backend.shadow;

public record DesiredRequest(ValveState valveState, long expectedVersion, Long durationMs) {
}

// src/main/java/com/terrasense/backend/shadow/ShadowController.java
package com.terrasense.backend.shadow;

import org.springframework.http.HttpStatus;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PutMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;
import org.springframework.web.server.ResponseStatusException;
```

```
@RestController
@RequestMapping("/api/shadows")
public class ShadowController {
    private final ShadowService shadowService;

    public ShadowController(ShadowService shadowService) {
        this.shadowService = shadowService;
    }

    @GetMapping("/{site}/{node}")
    public DeviceShadow get(@PathVariable String site, @PathVariable String node) {
        return shadowService.get(new ShadowKey(site, node))
                .orElseThrow(() -> new ResponseStatusException(HttpStatus.NOT_FOUND,
                        "Shadow not found"));
    }

    @PutMapping("/{site}/{node}/desired")
    public DeviceShadow updateDesired(@PathVariable String site, @PathVariable String node,
                                      @RequestBody DesiredRequest request) {
        return shadowService.updateDesired(new ShadowKey(site, node),
                request.valveState(), request.expectedVersion(), request.durationMs());
    }
}
```

A `state` topic fogyasztója külön MQTT-kliens. A `MqttStateSubscriber` a `terrasense/+/+/state` wildcardra iratkozik fel, ellenőrzi, hogy a topicban szereplő site/node megegyezik a payload `site`/`node_id` mezőjével, majd a `valve_open` értéket `ValveState`-re képezve hívja a `ShadowService.updateReported` metódust. A `seq` nélküli vagy nem növekvő state üzenetet a szolgáltatás eldobja; a `fault` mező helyi védelmi zárást jelez.

```
// src/main/java/com/terrasense/backend/shadow/StateMessage.java
package com.terrasense.backend.shadow;

import com.fasterxml.jackson.annotation.JsonIgnoreProperties;
import com.fasterxml.jackson.annotation.JsonProperty;

@JsonIgnoreProperties(ignoreUnknown = true)
public record StateMessage(
        String site,
        @JsonProperty("node_id") String nodeId,
        @JsonProperty("valve_open") Boolean valveOpen,
        long seq,
        long ts,
        String fault
) {
}

// src/main/java/com/terrasense/backend/shadow/MqttStateSubscriber.java
package com.terrasense.backend.shadow;

import com.fasterxml.jackson.databind.ObjectMapper;
import jakarta.annotation.PostConstruct;
import jakarta.annotation.PreDestroy;
import org.eclipse.paho.client.mqttv3.IMqttDeliveryToken;
import org.eclipse.paho.client.mqttv3.MqttCallbackExtended;
import org.eclipse.paho.client.mqttv3.MqttClient;
import org.eclipse.paho.client.mqttv3.MqttConnectOptions;
import org.eclipse.paho.client.mqttv3.MqttException;
import org.eclipse.paho.client.mqttv3.MqttMessage;
import org.eclipse.paho.client.mqttv3.persist.MemoryPersistence;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.beans.factory.annotation.Value;
```

```
import org.springframework.stereotype.Service;

import java.nio.charset.StandardCharsets;

@Service
public class MqttStateSubscriber implements MqttCallbackExtended {
    private static final Logger log = LoggerFactory.getLogger(MqttStateSubscriber.class);
    private static final String STATE_TOPIC = "terrasense/+/+/state";

    private final MqttClient client;
    private final MqttConnectOptions options;
    private final ObjectMapper objectMapper;
    private final ShadowService shadowService;

    public MqttStateSubscriber(
            ObjectMapper objectMapper,
            ShadowService shadowService,
            @Value("${terrasense.mqtt.broker-url:tcp://localhost:1883}") String brokerUrl,
            @Value("${terrasense.mqtt.state-client-id:ts-backend-state}") String clientId) {
        this.objectMapper = objectMapper;
        this.shadowService = shadowService;
        try {
            this.client = new MqttClient(brokerUrl, clientId, new MemoryPersistence());
        } catch (MqttException e) {
            throw new IllegalStateException("Nem hozható létre a state MQTT kliens", e);
        }
        this.options = new MqttConnectOptions();
        options.setAutomaticReconnect(true);
        options.setCleanSession(true);
        options.setConnectionTimeout(10);
        options.setKeepAliveInterval(30);
        client.setCallback(this);
    }

    @PostConstruct
    public void connect() {
        try {
            if (!client.isConnected()) {
                client.connect(options);
            }
        } catch (MqttException e) {
            log.error("A state MQTT broker a startupkor nem érhető el: {}", e.getMessage());
        }
    }

    @Override
    public void connectComplete(boolean reconnect, String serverURI) {
        try {
            client.subscribe(STATE_TOPIC, 1);
            log.info("State feliratkozás kész: {} (reconnect={})", STATE_TOPIC, reconnect);
        } catch (MqttException e) {
            log.error("A state feliratkozás nem sikerült", e);
        }
    }

    @Override
    public void messageArrived(String topic, MqttMessage message) {
        String[] parts = topic.split("/");
        if (parts.length != 4 || !"terrasense".equals(parts[0])
                || !"state".equals(parts[3])) {
            log.warn("Váratlan state topic: {}", topic);
            return;
        }
```

```
        String json = new String(message.getPayload(), StandardCharsets.UTF_8);
        try {
            StateMessage state = objectMapper.readValue(json, StateMessage.class);
            if (state.site() == null || state.nodeId() == null
                    || state.valveOpen() == null) {
                log.warn("Hiányos state üzenet: {}", json);
                return;
            }
            if (!parts[1].equals(state.site()) || !parts[2].equals(state.nodeId())) {
                log.warn("A state topic és payload site/node nem egyezik: {} / {}",
                        topic, json);
                return;
            }
            shadowService.updateReported(new ShadowKey(parts[1], parts[2]),
                    state.valveOpen() ? ValveState.OPEN : ValveState.CLOSED,
                    state.seq(), state.ts(), state.fault());
        } catch (Exception e) {
            log.warn("Érvénytelen state üzenet a(z) {} topicon: {}", topic, json, e);
        }
    }

    @Override
    public void connectionLost(Throwable cause) {
        log.warn("A state MQTT kapcsolat megszakadt; automatikus újracsatlakozás", cause);
    }

    @Override
    public void deliveryComplete(IMqttDeliveryToken token) {
        // A state fogyasztó nem publikál, így nincs kézbesítési nyugta.
    }

    @PreDestroy
    public void close() {
        try {
            if (client.isConnected()) {
                client.disconnect();
            }
            client.close();
        } catch (MqttException e) {
            log.warn("A state MQTT kapcsolat lezárása nem sikerült", e);
        }
    }
}
```

Az `application.yaml` kiegészítése a state kliens azonosítójával (a `terrasense.mqtt.broker-url` és a `client-id` már a Parancsok és konfiguráció lefelé: kézbesítés, nyugtázás, időkorlát laborból megvan):

```
terrasense:
  mqtt:
    broker-url: tcp://localhost:1883
    client-id: ts-backend-cmd
    state-client-id: ts-backend-state
```

A szimulátor a `ts-node-sim.py` fájlban egy teljes, Paho 2.x-es kliens. A `cmd` üzenet feldolgozása után a szelep tényleges állapotát a `state` topicra publikálja, növekvő `seq` értékkel. A `duration_ms` letelte után a normál időzítő zárja le a szelepet, és a state üzenet `fault` nélkül jelenti a CLOSED állapotot. Ha a normál zárás elmaradna, és a szelep a `max_on_time_ms` eléréséig nyitva maradna, a biztonsági időzítő helyi védelmi zárást hajt végre, és a state üzenetben `fault: "max_on_time"` értéket küld. A callback csak validál és sorba ad, a munkaszál végzi a szelepműveletet, ezért a hálózati ciklus nem blokkol.

```
#!/usr/bin/env python3
"""ts-node-sim: az i050 árnyéklabor szelepszimulátora (paho-mqtt 2.x)."""
import json
import queue
import threading
import time

import paho.mqtt.client as mqtt

BROKER = "localhost"
PORT = 1883
SITE = "gh1"
NODE = "n001"
CMD_TOPIC = f"terrasense/{SITE}/{NODE}/cmd"
RESULT_TOPIC = f"terrasense/{SITE}/{NODE}/cmd/result"
STATE_TOPIC = f"terrasense/{SITE}/{NODE}/state"
MIN_DURATION_MS = 1_000
MAX_DURATION_MS = 600_000

work_queue: "queue.Queue[dict]" = queue.Queue()
processed: dict[str, dict] = {}
in_progress: set[str] = set()
state_lock = threading.Lock()
valve_lock = threading.Lock()
valve_open = False
open_cmd = None
open_timer = None
safety_timer = None
seq = 0
shutdown = threading.Event()
client = None


def now_ms() -> int:
    return int(time.time() * 1000)


def publish(topic: str, payload: dict) -> None:
    if client is not None:
        client.publish(topic, json.dumps(payload), qos=1)


def publish_result(payload: dict) -> None:
    publish(RESULT_TOPIC, payload)


def publish_state(open_state: bool, fault: str | None = None) -> None:
    global seq
    with state_lock:
        seq += 1
        current_seq = seq
    payload = {
        "site": SITE,
        "node_id": NODE,
        "valve_open": open_state,
        "seq": current_seq,
        "ts": now_ms(),
    }
    if fault is not None:
        payload["fault"] = fault
    publish(STATE_TOPIC, payload)
    print(f"[ts-node-sim] state: valve_open={open_state}, seq={current_seq}"
          + (f", fault={fault}" if fault else ""))
```

```
def result_payload(cmd: dict, status: str, result: dict | None = None) -> dict:
    payload = {
        "id": cmd["id"],
        "status": status,
        "node_id": NODE,
        "site": SITE,
        "timestamp": now_ms(),
    }
    if result is not None:
        payload["result"] = result
    return payload


def store_result(command_id: str, payload: dict) -> None:
    with state_lock:
        processed[command_id] = payload
        in_progress.discard(command_id)


def command_expired(cmd: dict) -> bool:
    expiry = cmd.get("expiry")
    return (isinstance(expiry, (int, float)) and not isinstance(expiry, bool)
            and now_ms() > int(expiry))


def valid_command(cmd: object) -> tuple[bool, str]:
    if not isinstance(cmd, dict):
        return False, "a parancs nem JSON-objektum"
    if not isinstance(cmd.get("id"), str) or not cmd["id"]:
        return False, "hiányzó vagy érvénytelen id"
    if cmd.get("node_id") not in (None, NODE):
        return False, "a parancs más node-nak szól"
    if cmd.get("type") not in ("valve.open", "valve.close"):
        return False, "ismeretlen parancstípus"
    if not isinstance(cmd.get("expiry"), (int, float)) or isinstance(cmd.get("expiry"), bool):
        return False, "hiányzó vagy érvénytelen expiry"
    params = cmd.get("params", {})
    if not isinstance(params, dict):
        return False, "a params nem objektum"
    if cmd["type"] == "valve.open":
        duration = params.get("duration_ms")
        if not isinstance(duration, int) or isinstance(duration, bool):
            return False, "a duration_ms nem egész szám"
        if duration < MIN_DURATION_MS or duration > MAX_DURATION_MS:
            return False, "a duration_ms a megengedett tartományon kívül van"
        max_on_time = params.get("max_on_time_ms", duration)
        if not isinstance(max_on_time, int) or isinstance(max_on_time, bool):
            return False, "a max_on_time_ms nem egész szám"
        if max_on_time < duration or max_on_time > MAX_DURATION_MS:
            return False, "a max_on_time_ms érvénytelen"
    return True, ""


def close_valve(cmd: dict, fault: str | None) -> None:
    global valve_open, open_cmd, open_timer, safety_timer
    with valve_lock:
        if open_cmd is not cmd:
            return
        valve_open = False
        open_cmd = None
        for timer in (open_timer, safety_timer):
```

```
            if timer is not None:
                timer.cancel()
        open_timer = None
        safety_timer = None
    publish_state(False, fault)
    if fault is not None:
        print(f"[ts-node-sim] helyi védelmi zárás: {fault}")


def finish_normal(cmd: dict) -> None:
    close_valve(cmd, None)


def finish_safety(cmd: dict) -> None:
    close_valve(cmd, "max_on_time")


def handle_open(cmd: dict) -> None:
    global valve_open, open_cmd, open_timer, safety_timer
    duration = int(cmd["params"]["duration_ms"])
    max_on_time = int(cmd["params"].get("max_on_time_ms", duration))
    with valve_lock:
        for timer in (open_timer, safety_timer):
            if timer is not None:
                timer.cancel()
        valve_open = True
        open_cmd = cmd
        normal = threading.Timer(duration / 1000.0, finish_normal, args=(cmd,))
        normal.daemon = True
        open_timer = normal
        safety = None
        if max_on_time > duration:
            safety = threading.Timer(max_on_time / 1000.0, finish_safety, args=(cmd,))
            safety.daemon = True
        safety_timer = safety
        normal.start()
        if safety is not None:
            safety.start()
    print(f"[ts-node-sim] szelep: OPEN (max {max_on_time} ms)")
    publish_result(result_payload(cmd, "ACCEPTED"))
    publish_state(True)
    payload = result_payload(cmd, "SUCCESS",
                             {"valve_open": True, "max_on_time_ms": max_on_time})
    store_result(cmd["id"], payload)
    publish_result(payload)


def handle_close(cmd: dict) -> None:
    global valve_open, open_cmd, open_timer, safety_timer
    with valve_lock:
        valve_open = False
        for timer in (open_timer, safety_timer):
            if timer is not None:
                timer.cancel()
        open_timer = None
        safety_timer = None
        open_cmd = None
    publish_result(result_payload(cmd, "ACCEPTED"))
    publish_state(False)
    payload = result_payload(cmd, "SUCCESS", {"valve_open": False})
    store_result(cmd["id"], payload)
    publish_result(payload)
    print("[ts-node-sim] szelep: CLOSED")
```

```
def handle_command(cmd: dict) -> None:
    command_id = cmd["id"]
    with state_lock:
        previous = processed.get(command_id)
        if previous is None and command_id in in_progress:
            previous = result_payload(cmd, "ACCEPTED")
        if previous is None:
            in_progress.add(command_id)
    if previous is not None:
        print(f"[ts-node-sim] duplikált parancs, eredmény újraküldve: {command_id}")
        publish_result(previous)
        return
    if command_expired(cmd):
        payload = result_payload(cmd, "EXPIRED",
                                 {"reason": "a parancs a kézbesítéskor már lejárt"})
        store_result(command_id, payload)
        publish_result(payload)
        print(f"[ts-node-sim] lejárt parancs: {command_id}")
        return
    if cmd["type"] == "valve.open":
        handle_open(cmd)
    else:
        handle_close(cmd)


def execute() -> None:
    while not shutdown.is_set():
        try:
            cmd = work_queue.get(timeout=0.5)
        except queue.Empty:
            continue
        if cmd is None:
            work_queue.task_done()
            break
        try:
            handle_command(cmd)
        except Exception as exc:
            print(f"[ts-node-sim] hiba: {exc}")
            with state_lock:
                in_progress.discard(cmd.get("id"))
            if isinstance(cmd, dict) and isinstance(cmd.get("id"), str):
                payload = result_payload(cmd, "FAILED", {"error": str(exc)})
                store_result(cmd["id"], payload)
                publish_result(payload)
        finally:
            work_queue.task_done()


def on_connect(client, userdata, flags, reason_code, properties) -> None:
    if reason_code.is_failure:
        print(f"[ts-node-sim] kapcsolódási hiba: {reason_code}")
        return
    client.subscribe(CMD_TOPIC, qos=1)
    print(f"[ts-node-sim] feliratkozva: {CMD_TOPIC}")


def on_disconnect(client, userdata, disconnect_flags, reason_code, properties) -> None:
    print(f"[ts-node-sim] kapcsolat bontva: {reason_code}")


def on_message(client, userdata, msg) -> None:
```

```
    try:
        cmd = json.loads(msg.payload.decode("utf-8"))
    except (UnicodeDecodeError, json.JSONDecodeError) as exc:
        print(f"[ts-node-sim] érvénytelen JSON: {exc}")
        return
    ok, reason = valid_command(cmd)
    if not ok:
        print(f"[ts-node-sim] elutasítva ({reason})")
        if isinstance(cmd, dict) and isinstance(cmd.get("id"), str):
            publish_result(result_payload(cmd, "FAILED", {"error": reason}))
        return
    work_queue.put(cmd)


def main() -> None:
    global client
    client = mqtt.Client(mqtt.CallbackAPIVersion.VERSION2,
                         client_id="ts-node-sim-i050")
    client.on_connect = on_connect
    client.on_disconnect = on_disconnect
    client.on_message = on_message
    client.reconnect_delay_set(min_delay=1, max_delay=30)
    worker = threading.Thread(target=execute, daemon=True)
    worker.start()
    try:
        client.connect(BROKER, PORT, keepalive=60)
        client.loop_forever()
    except OSError as exc:
        raise SystemExit(f"[ts-node-sim] nem érhető el a broker: {exc}") from exc
    except KeyboardInterrupt:
        print("[ts-node-sim] leállítás")
    finally:
        shutdown.set()
        work_queue.put(None)
        try:
            client.disconnect()
        except Exception:
            pass


if __name__ == "__main__":
    main()
```

Futtatás. Indítsuk a közös stack brokert, majd a backendet a Parancsok és konfiguráció lefelé: kézbesítés, nyugtázás, időkorlát alfejezetből örökölt `ts-backend` könyvtárból, végül a szimulátort:

```
docker compose -f ~/iot-labor/stack/compose.yaml up -d lab-broker
cd ~/iot-labor/i050/ts-backend && mvn spring-boot:run
# másik terminálban
cd ~/iot-labor/i050 && . .venv/bin/activate && python ts-node-sim.py
```

Egy harmadik terminálban állítsuk be a desired állapotot. A `durationMs` opcionális; ha elhagyjuk, a 10 perces alapérték érvényes. A válasz a frissített árnyékot adja vissza.

```
curl -s -X PUT -H 'Content-Type: application/json' \
  -d '{"valveState":"OPEN","expectedVersion":0,"durationMs":600000}' \
  http://localhost:8080/api/shadows/gh1/n001/desired
```

Illusztratív PUT-válasz (a labor friss árnyéka még nem kapott state-et, ezért a reported `UNKNOWN`; a `delta` a desired és a reported eltérése miatt `OPEN`; az időbélyegek eltérnek):

```
{"key":{"site":"gh1","node":"n001"},"desired":"OPEN","reported":"UNKNOWN","version":1,
↪  "reportedSeq":0,"reportedAt":0,"desiredUntil":1735689660000,"lastFault":null,"delta":"OPEN"}
```

A szimulátor terminálján ehhez hasonló sorok jelennek meg:

```
[ts-node-sim] szelep: OPEN (max 600000 ms)
[ts-node-sim] state: valve_open=True, seq=1
```

A `reported` a `state` üzenetből frissül, a `reportedSeq` az eszköz monoton `seq` értéke, a `desiredUntil` pedig a kért nyitás lease-határideje. A `delta` csak akkor jelenik meg a JSON-ben, ha a desired és a reported eltér; a `@JsonInclude(NON_ABSENT)` miatt üres delta esetén kimarad. A state `seq` nélkül vagy csökkenő `seq`-csel érkező üzenetét a backend eldobja, így egy késve érkező régi state nem írja felül a frissebb reported értéket.

A `reconcile` 30 másodpercenként lefut. Ha a desired OPEN lease-e lejár, a desired `CLOSED`-re áll, és a következő kör záró parancsot publikál; ha a szimulátor a `max_on_time_ms` miatt helyi védelmi zárást hajt végre, a state `fault: "max_on_time"` mezője miatt a desired azonnal a reported értékre áll, és a delta megszűnik. A két mechanizmus együtt biztosítja, hogy egy elakadt vagy nem válaszoló eszköz ne öntözzön a végtelenségig: a nyitás időtartama a `duration_ms` és a `max_on_time_ms` által korlátos, a backend lease-e pedig a kért idő után záró parancsot kényszerít.

Ezután kérdezzük le az árnyékot:

```
curl -s http://localhost:8080/api/shadows/gh1/n001
```

Illusztratív GET-válasz a state feldolgozása után (a `delta` most üres, ezért hiányzik):

```
{"key":{"site":"gh1","node":"n001"},"desired":"OPEN","reported":"OPEN","version":2,
↪  "reportedSeq":1,"reportedAt":1735689001000,"desiredUntil":1735689660000,"lastFault":null}
```

A `delta` akkor jelenik meg, ha a desired és a reported eltér. Ha a szimulátor leállítása előtt ismét `CLOSED` desired állapotot állítunk be, az árnyékban a reported még `OPEN`, a delta `CLOSED` lesz, amíg a szimulátor nem jelenti az új állapotot. A következő PUT már a megnövelt `version`-t várja el `expectedVersion`-ként; elavult verzióval `409 Conflict` a válasz.

17.2.6 Ellenőrzés és hibakeresés

Hibatünet	Ok	Javítás
A `PUT` hívás 409 Conflict választ ad	A `expectedVersion` nem egyezik a tárolt verzióval	GET-tel kérdezzük le az aktuális verziót, majd azzal frissítsünk; a PUT a verziót mindig növeli
A `PUT` hívás 200 OK, de a parancs nem ment el a brokerre	A `CommandService` publikálása átmenetileg meghiúsult	Ellenőrizzük a `broker` szolgáltatást és a backend „publikálása nem sikerült” naplósorát; a desired megmarad, a reconciler legkésőbb 30 másodperc múlva újrapróbálja

Hibatünet	Ok	Javítás
A szimulátor nem nyitja a szelepet	A backend nem publikál a `cmd` topicra, a topic neve hibás, vagy a parancs a `ttl_ms` alatt lejárt	Ellenőrizzük a `CommandService` naplóját, a `MqttCommandGateway` topic-összeállítását és a `docker logs lab-broker` kimenetét
A reported mező nem frissül	A szimulátor nem publikál a `state` topicra, a backend nem iratkozott fel rá, vagy a state `seq` értéke nem nő	Ellenőrizzük a `state` topic nevét, a `connectComplete` feliratkozását és a szimulátor `seq` növelését
A state megérkezik, de a reported változatlan	A `seq` nem nagyobb a tárolt `reportedSeq`-nél, ezért a backend elavult state-ként eldobta	Nézzük meg a backend „Elavult state eldobva” naplósorát és a szimulátor `seq` értékét
A delta mindig a desired értéket mutatja	A reported `UNKNOWN`, mert az eszköz még sosem jelentett	Ez lehet helyes működés; várjuk meg az első `state` üzenetet
A desired OPEN nem tűnik el a `duration_ms` után	Az eszköz nem küldött `fault` state-et, és a lease még nem járt le	Ellenőrizzük a `desiredUntil` értékét, a szimulátor `max_on_time_ms` időzítőjét és a `reconcile` naplóját; a lejárt lease után a desired `CLOSED` lesz
A state fogyasztó nem csatlakozik	A broker a backend indítása után indult, vagy a state kliens azonosítója ütközik	Indítsuk a brokert a backend előtt, és ellenőrizzük a `ts-backend-state` kliensazonosítót

17.2.7 Tipikus hibák és félreértések

1. **Retained üzenet használata árnyékként.** A retained üzenet csak az utolsó state-et tárolja, de nincs desired, delta és verzió. Ha a desired is retained üzenetben lenne, akkor a broker újraindulásakor a delta elveszne, és a konkurens frissítések felülírnák egymást.
2. **A reported hiányának figyelmen kívül hagyása.** Ha a delta számítás `desired == reported` módon történik, és a `reported` értéke `UNKNOWN`, a program hamis deltát adhat, vagy egy `null` értékre hivatkozva kivételt dobhat. A `ValveState` enum használata és a hiányzó reported külön kezelése ezt elkerüli; a delta csak akkor ad értéket, ha a desired ismert és eltér a reportedtól.
3. **A verzió növelésének elmaradása vagy túl gyakori növelése.** Ha a reported frissítésekor nem növeljük a verziót, a desired frissítésnél alkalmazott optimista konkurenciavezérlés tévesen azt hiheti, hogy nincs változás. Ha viszont egy elavult vagy duplikált state is növeli a verziót, a verzió hamis lépéseket mutat, és a kliens feleslegesen kap 409-et. A `reportedSeq` ellenőrzés ezért a verzió növelése előtt történik.
4. **Az árnyék és a parancs összekeverése.** Az árnyék a kívánt és a tényleges állapot tárolója, nem pedig a parancsüzenet. A desired változás kiváltja a parancsot, de a parancs kézbesítése nem módosítja automatikusan a reported mezőt; azt mindig az eszköz `state` üzenetéből kell frissíteni.
5. **Az OPEN desired végtelen újraküldése.** Ha a delta alapján a reconciler korlátlanul újraküldi a nyitó parancsot, az eszköz hibája vagy offline állapota végtelen öntözést okozhat.

A labor ezért `desiredUntil` lease-t és `max_on_time_ms` helyi védelmet használ; a lejárt lease után a desired `CLOSED` lesz, és a reconciler záró parancsot küld.

17.2.8 Feladatok

17.2.8.1 1. Fogalmi és döntési kérdés

Melyik megoldást választaná egy olyan TerraSense-üzemeltetéshez, ahol a kezelőnek azonnal látnia kell, ha a szelep kívánt és tényleges állapota eltér, és a parancsot esetleg újra kell küldeni? Indokolja a retained üzenet és a backend-tárolt árnyék közötti választást.

17.2.8.2 2. Számolási vagy tervezési feladat

Egy üvegházban 40 csomópont van. Minden csomópont percenként küld új `seq`-ű `state` üzenetet a `state` topicra. A backend árnyéktárolása minden új reported `seq` esetén növeli a verziót, a duplikált vagy elavult `seq` esetén nem. Hány verziólépés keletkezik egy óra alatt a teljes flottában, ha a desired frissítés átlagosan óránként 2 alkalommal történik csomópontonként? Tegyük fel, hogy a verzió nem csökken, és nincs csomagvesztés.

17.2.8.3 3. Implementációs feladat a TerraSense-ben

A labor `ShadowService` már tartalmazza a `listAllDeltas()` metódust és a `@Scheduled reconcile()`-t. Egészítse ki a reconcilert a lejárt OPEN lease kezelésével: a lease lejárta után a desired állapot `CLOSED` legyen, és a következő körben záró parancs menjen az eszköznek. Magyarázza el, miért nem elég a `delta()` önmagában az újraküldéshez, és miért kell a lease-t a delta számításában is figyelembe venni.

17.2.8.4 4. Hibakeresési vagy kiterjesztési feladat

A labor szimulátora már küld `seq` mezőt a `state` üzenetben. Magyarázza el, mi történne, ha ezt elhagynánk, és hogyan kapcsolódik a probléma az Ingest: MQTT-ből a backendbe, idempotensen alfejezet idempotens ingest-jéhez. Mutassa be, hogy a backend `reportedSeq` ellenőrzése hogyan dobja el a duplikált és a csökkenő `seq`-ű state üzenetet, és miért nem növeli ilyenkor a verziót.

17.2.9 Megoldások

17.2.9.1 1. megoldás

Backend-tárolt árnyékot kell választani, mert a retained üzenet nem tartalmaz desired/reported párokat, sem delta, sem verzió fogalmat. A backend-tárolt árnyék REST API-val elérhető, és a delta alapján a backend újra tudja kézbesíteni a parancsot. A retained üzenet legfeljebb a frissen csatlakozó eszközök gyors felzárkóztatására szolgál.

17.2.9.2 2. megoldás

A reported frissítések száma óránként csomópontonként 60, a 40 csomópontra összesen $60 \times 40 = 2400$ verziólépés. A desired frissítések száma $2 \times 40 = 80$. A teljes verziólépés óránként $2400 + 80 = 2480$. Ez nem egyetlen 2480-as számlálót jelent: a verzió csomópontonként külön, monoton növekszik, így csomópontonként átlagosan 62 lépés történik óránként.

17.2.9.3 3. megoldás

```
public List<ShadowKey> listAllDeltas() {
    long now = System.currentTimeMillis();
    return shadows.values().stream()
            .filter(shadow -> shadow.delta(now).isPresent())
            .map(DeviceShadow::key)
            .toList();
}
```

A `delta(now)` a lejárt OPEN lease esetén üres `Optional`-t ad, ezért a lista nem tartalmazza a lejárt lease-ű árnyékot, és a reconciler nem küld rá új nyitó parancsot. A `reconcile` a lista elemeihez a `publishDesired`-en keresztül `commandService.createAndPublish`-ot hív; ha a publikálás hibára fut, a desired megmarad, és a következő 30 másodperces kör újrapróbálja. A lejárt lease-t a reconciler `CLOSED` desired-re állítja, így a következő kör záró parancsot küld. A `ConcurrentHashMap` weakly consistent iterációja miatt a lista készítése nem blokkolja a párhuzamos desired-frissítéseket; a `compute`-tel végzett írások továbbra is atomiak.

17.2.9.4 4. megoldás

A `seq` hiánya esetén az ingest nem tud idempotensen deduplikálni: ha a `state` üzenet kétszer érkezik meg, a reported mező kétszer frissülne, és a verzió kétszer nőne. Ez hamis verziólépéshez és felesleges parancs-újraküldéshez vezetne, a régi és az új state pedig nem lenne megkülönböztethető. A labor szimulátora ezért monoton `seq` értéket küld; a backend a `DeviceShadow.reportedSeq` mezőben tárolja az utolsó feldolgozott értéket, és a `ShadowService.updateReported` csak akkor növeli a verziót, ha az új `seq` nagyobb. A `seq <= reportedSeq` esetén a state eldobódik, így a késve érkező régi üzenet nem írja felül a frissebb reported állapotot. Ez ugyanaz az idempotencia-elv, amelyet az Ingest: MQTT-ből a backendbe, idempotensen alfejezet a telemetriánál használ.

17.2.10 Rövid összefoglaló

Az eszközárnyék a desired és reported állapot, valamint a delta tárolója a backendben. A desired frissítése parancsot vált ki, a reported frissítése az eszköz `state` üzenetéből származik, a delta pedig a kettő eltérése. A verziószám optimista konkurenciavezérlést tesz lehetővé: az elavult írás 409 Conflict választ kap. A `reportedSeq` a state üzenetek idempotens feldolgozását biztosítja; a régi `seq` nem növeli a verziót és nem írja felül a frissebb reported állapotot. A desired OPENhez `desiredUntil` lease tartozik, amely a lejárat után `CLOSED` desired-re áll, és a reconciler záró parancsot publikál; az eszköz `max_on_time_ms` védelme és a `fault` state jelzése ugyanezt a biztonsági célt szolgálja. A backend-tárolt árnyék jobb a retained üzenetnél, mert deltát, verziót és REST API-t ad. A digitális iker ennél bővebb modell, szimulációval és előrejelzéssel; az Eclipse Ditto és a felhős szolgáltatások ilyen platformok. A laborban a `ts-backend` `ShadowService`-e a szelep desired/reported árnyékát kezeli `ConcurrentHashMap`-ben, a `CommandService`-en keresztül publikál, az `MqttStateSubscriber` pedig a `state` topicot fogyasztja; a Paho 2.x-es Python szimulátor a `state` topicon jelenti a tényleges szelepállapotot

`seq`-csel. A labor tárolása memóriabeli; éles rendszerben az árnyékot tartós adatbázisba kell menteni. **Továbbvitt eredmény.** A `DeviceShadow`, a `ShadowService` delta-számítása és a verziókezelés az Flottakezelés: nyilvántartás, konfiguráció, állapot alfejezetben a flottaállapot-nyilvántartáshoz, valamint a Záróprojekt: a teljes TerraSense rendszer összeállítása és átvétele alfejezetben a végponttól végpontig tartó átvételi tesztekhez használható.

17.2.11 Források és ellenőrzési hiányok

Az eszközárnyék desired/reported/delta modelljének fogalmi alapja az AWS IoT Device Shadow dokumentációjában és az Eclipse Ditto fogalmi leírásában található. A labor kódja a Spring Boot 3.3.4, a Java 21 és a Paho MQTT kliens stabil fogalmaira épül; a Java oldalon `org.eclipse.paho:org.eclipse.paho.client.mqttv3:1.2.5`, a Python oldalon `paho-mqtt==2.1.0` a rögzített verzió. **Ellenőrzési igény.** Az Eclipse Ditto aktuális verziója, Java API-ja, a felhős árnyékszolgáltatások pontos sémája és a Python `paho-mqtt` callback API verziója gyorsan változik; használat előtt ellenőrizze az aktuális dokumentációt. **Statikus ellenőrzés.** A fejezet kódját és a szimulátor logikáját a szerző lokálisan, telepítés nélkül, Paho-stubbal és `py_compile`-lal ellenőrizte; a teljes Maven/Spring Boot/MQTT labort nem futtatta, ezért a `state` fogyasztás és a lease-kezelés végponttól végpontig történő igazolása a labor olvasójának feladata.

17.3 Dashboardok Grafanával

17.3.1 Motiváció

A TerraSense backend már fogadja a telemetriát, a TimescaleDB tárolja a nyers és aggregált adatokat. Az üzemeltető azonban nem fog minden kérdésre SQL-lel válaszolni: gyorsan látni akarja, hogy az üvegházban csökkent-e a hőmérséklet, mikor volt adathiány, és hogy az öntöző-szelep állapota követi-e a talajnedvességet. A Grafana erre a megjelenítési rétegre épül rá: nem tárol, és nem módosítja az adatokat, hanem a meglévő forrásokra csatlakozik, és dashboardokat kínál. A dashboard ráadásul nemcsak vizualizáció: az Riasztás: szabályok, értesítés, riasztási fáradtság alfejezetben ugyanazokat a panel-lekérdezéseket használjuk majd riasztási szabályként, ezért érdemes a lekérdezéseket itt is gondosan kialakítani.

17.3.2 Fogalmak és működés

Definíció. (Provisioning.) A Grafana azon mechanizmusa, amely a konfigurációs állományokat (YAML/JSON) beolvasva automatikusan létrehozza az adatforrásokat, dashboardokat és más erőforrásokat. Ezzel a néhány kattintással összeállított dashboard is reprodukálható, verziókövethető és konténer-újraindítás után is azonos lesz.

Definíció. (Panel.) A dashboard egy vizualizációs egysége, amely legalább egy adatforrás-lekérdezésből származó adatsort jelenít meg kiválasztott típusban: idősor (time series), állapot, hőtérkép, stat vagy műszer (gauge).

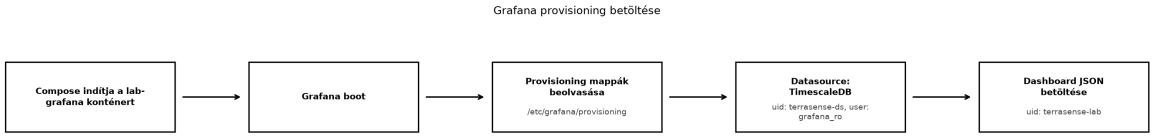
Definíció. (Változó.) A dashboard szintű paraméter, amelyet a Grafana a lekérdezésekbe `$változó` formában illeszt be. A felhasználó a dashboard tetején választhat értéket, például konkrét node-ot vagy site-ot.

A TerraSense esetében a Grafana a TimescaleDB adatforráson keresztül éri el a `telemetry`, `telemetry_hourly` és `telemetry_daily` táblákat/nézeteket. A Grafana SQL-makrói közül kettőt használunk folyamatosan:

- `$__timeFilter(ts)` a dashboard időtartományát adja át `WHERE` feltételként;
- `$__timeGroup(ts,'5m')` időbeli bucketet (például 5 perces) hoz létre.

A Grafana nem az adminisztrátori `terra` felhasználóval csatlakozik, hanem egy külön, csak olvasási jogú `grafana_ro` szerepkörrel. Ez a legkisebb jogosultság elve: a dashboard nem tud véletlenül írni a telemetriába, és egy elrontott panel nem módosíthatja a táblákat. A szerepkört a labor SQL-szkriptje hozza létre; éles rendszerben a jelszót titokkezelőből kell betölteni, nem a compose-fájlból.

Az alábbi ábra a provisioning betöltési útvonalát mutatja.



17.3. ábra: Grafana provisioning betöltése - A konténer indításakor a Grafana a megadott provisioning könyvtárakból automatikusan létrehozza az adatforrást és a dashboardot.

A konténer a host `~/iot-labor/i051/grafana-provisioning` mappáját mountolja a konténer `/etc/grafana/provisioning` pontjára. A Grafana a `datasources` és `dashboards` alkönyvtárakat figyeli, így a dashboard és az adatforrás a konténer indulásakor automatikusan létrejön.

17.3.3 Döntési szempontok

A megfelelő vizualizáció kiválasztása az adat jellegétől és az üzemeltetői kérdéstől függ. A TerraSense-ben az alábbi tipikus döntéseket hozzuk meg:

Vizualizáció	Adat jellege	TerraSense-példa
Idősor (time series)	Folyamatos mérés, trend	Hőmérséklet, páratartalom
Stat	Egyetlen aktuális vagy utolsó érték	Utolsó hőmérséklet node-onként
Állapotsor (state timeline)	Diszkrét állapotváltozás	Szelep nyitva/zárva
Műszer (gauge)	Érték tartománnyal, küszöbbel	Talajnedvesség a 30–70% tartományban
Hőtérkép (heatmap)	Eloszlás vagy hiányzó adat	Napi mintaszám, adathiány

Az idősor a leggyakoribb; a stat panel akkor hasznos, ha az utolsó érték gyorsan kiolvasható, például egy vezénylői nézetben. A hőtérképet akkor érdemes bevetni, amikor nem egyetlen érték, hanem a sűrűség vagy a hiányzó adat a kérdés. A Megőrzés, ritkítás és tárolási költség alfejezetben bevezetett `telemetry_hourly` és `telemetry_daily` nézetek a hosszú távú trendekhez és adathiány-észleléshez használandók, a nyers `telemetry` tábla pedig a rövid távú, finom felbontású panelekhez.

17.3.4 Kidolgozott példa / labor: TerraSense dashboard

A labor előfeltétele az Idősoros adatbázisok és adatmodellezés és a Megőrzés, ritkítás és tárolási költség alfejezet elvégzése. A `telemetry` hypertable, valamint a `telemetry_hourly` és `telemetry_daily` nézetek meglévő, seedelt adatokkal rendelkeznek. A nézetek oszlopai a TerraSense hosszú táblamodellje szerint: `bucket`, `site`, `node_id`, `measurement_type`, `avg_value`, `min_value`, `max_value`, `sample_count`. A `bucket` mindkét nézetben a TimescaleDB `time_bucket` kimenete; a napi nézet is ezt a nevet használja, nem külön `day` oszlopot.

Munkakönyvtár:

```
mkdir -p ~/iot-labor/i051/grafana-provisioning/datasources
mkdir -p ~/iot-labor/i051/grafana-provisioning/dashboards
cd ~/iot-labor/i051
```

Először az adathiány-észleléshez szükséges két segédtáblát hozzuk létre a `~/iot-labor/i051/01_dashboard_inventory.sql` fájlban. A `sensor_nodes` az üzemeltetési nyilvántartás: akkor is tartalmazza a node-ot, ha az éppen néma, mert a `telemetry` táblából a teljesen hiányzó napok nem derülnek ki. A `sensor_expected_samples` méréstípusonként adja meg az elvárt napi mintaszámot és a megengedett eltérést; a 10 perces alapciklus napi 144 mintát jelent. A `gh1/n002` szándékosan szerepel a nyilvántartásban, de nincs telemetriája: ezzel a teljesen néma eszköz viselkedését tudjuk ellenőrizni.

```
-- ~/iot-labor/i051/01_dashboard_inventory.sql
CREATE TABLE IF NOT EXISTS sensor_nodes (
    site    text NOT NULL,
    node_id text NOT NULL,
    PRIMARY KEY (site, node_id)
);

CREATE TABLE IF NOT EXISTS sensor_expected_samples (
    measurement_type text    PRIMARY KEY,
    samples_per_day  integer NOT NULL CHECK (samples_per_day > 0),
    tolerance_ratio  numeric(3,2) NOT NULL DEFAULT 0.10
        CHECK (tolerance_ratio >= 0 AND tolerance_ratio < 1)
);

INSERT INTO sensor_nodes (site, node_id)
VALUES ('gh1', 'n001'), ('gh1', 'n002')
ON CONFLICT (site, node_id) DO NOTHING;

INSERT INTO sensor_expected_samples (measurement_type, samples_per_day, tolerance_ratio)
VALUES ('temperature',   144, 0.10),
       ('humidity',      144, 0.10),
       ('soil_moisture', 144, 0.10)
ON CONFLICT (measurement_type) DO UPDATE
SET samples_per_day = EXCLUDED.samples_per_day,
    tolerance_ratio = EXCLUDED.tolerance_ratio;
```

Ezután a `~/iot-labor/i051/02_grafana_ro.sql` fájl létrehozza a csak olvasási jogú szerepkört. A jelszó kizárólag a helyi laborhoz tartozik. A `GRANT SELECT` kizárólag a dashboardhoz szükséges öt objektumra vonatkozik; a szerepkör nem kap írási jogot, és nem láthatja a többi táblát.

```
-- ~/iot-labor/i051/02_grafana_ro.sql
DO $$
BEGIN
    IF NOT EXISTS (SELECT 1 FROM pg_roles WHERE rolname = 'grafana_ro') THEN
        CREATE ROLE grafana_ro LOGIN PASSWORD 'grafana-lab-only';
    ELSE
        ALTER ROLE grafana_ro WITH LOGIN PASSWORD 'grafana-lab-only';
```

```
    END IF;
END
$$;

GRANT CONNECT ON DATABASE terrasense TO grafana_ro;
GRANT USAGE ON SCHEMA public TO grafana_ro;
GRANT SELECT ON TABLE telemetry,
                      telemetry_hourly,
                      telemetry_daily,
                      sensor_nodes,
                      sensor_expected_samples
TO grafana_ro;
```

A két szkriptet a közös stack adatbázisán futtatjuk. A `-v ON_ERROR_STOP=1` biztosítja, hogy egy hibás lépés után a szkript ne folytatódjon csendben. A második parancs után ellenőrizzük a jogosultságokat, majd egy pozitív és egy negatív olvasási/írási próbát is végzünk.

```
docker compose -f ~/iot-labor/stack/compose.yaml exec -T lab-timescaledb \
  psql -U terra -d terrasense -v ON_ERROR_STOP=1 < 01_dashboard_inventory.sql
docker compose -f ~/iot-labor/stack/compose.yaml exec -T lab-timescaledb \
  psql -U terra -d terrasense -v ON_ERROR_STOP=1 < 02_grafana_ro.sql
docker compose -f ~/iot-labor/stack/compose.yaml exec -T lab-timescaledb \
  psql -U terra -d terrasense -c '\dp telemetry*'
docker compose -f ~/iot-labor/stack/compose.yaml exec -T \
  -e PGPASSWORD=grafana-lab-only lab-timescaledb \
  psql -h 127.0.0.1 -U grafana_ro -d terrasense -c 'SELECT count(*) FROM telemetry;'
docker compose -f ~/iot-labor/stack/compose.yaml exec -T \
  -e PGPASSWORD=grafana-lab-only lab-timescaledb \
  psql -h 127.0.0.1 -U grafana_ro -d terrasense \
  -c "INSERT INTO sensor_nodes (site, node_id) VALUES ('gh1', 'n999');"
```

Az utolsó parancsnak `permission denied for table sensor_nodes` hibával kell megszakadnia; ez igazolja, hogy a dashboard felhasználója valóban csak olvashat. A `\dp telemetry*` kimenetében a `grafana_ro` számára csak az `r` (SELECT) jog látszik.

Ezután hozzuk létre a három provisioning fájlt. Az adatforrás definíciója a `grafana-provisioning/datasources/datasources.yml` fájlba kerül. Az `url` a közös compose-hálózat `lab-timescaledb` hostnevét használja; a stack szolgáltatásai ugyanazon az alapértelmezett compose-hálózaton vannak, ezért nincs szükség külön `lab` hálózat definiálására. A `grafana_ro` felhasználó és a laborjelszó a fenti SQL-szkripttel egyezik.

```
apiVersion: 1

datasources:
  - name: TimescaleDB
    type: postgres
    access: proxy
    uid: terrasense-ds
    url: lab-timescaledb:5432
    database: terrasense
    user: grafana_ro
    secureJsonData:
      password: grafana-lab-only
    jsonData:
      sslmode: disable
      postgresVersion: 1600
      timescaledb: true
```

A dashboard-provider a `grafana-provisioning/dashboards/dashboards.yml` fájlba kerül:

```yaml
apiVersion: 1

providers:
  - name: TerraSense
    orgId: 1
    folder: ""
    type: file
    disableDeletion: false
    updateIntervalSeconds: 30
    options:
      path: /etc/grafana/provisioning/dashboards
```

A dashboard JSON-t a `grafana-provisioning/dashboards/dashboard.json` fájlba mentsd. A `$site` és `$node` változó a `sensor_nodes` nyilvántartásból kérdez, így a néma node is kiválasztható. A nyers telemetriát használó panelek `valid = true` feltétellel szűrnek, és `node_id AS metric` oszlopnévvel node-onként külön sorozatot rajzolnak, nem egyetlen átlaggörbét. Az utolsó hőmérséklet stat panel a `$node` szerint ismétlődik, ezért minden kiválasztott node saját aktuális értéket kap. A hosszú távú panelek a Megőrzés, ritkítás és tárolási költség nézeteit használják; a napi nézet oszlopa `bucket`, az átlag pedig `avg_value`.

```json
{
  "title": "TerraSense - Üvegház monitor",
  "uid": "terrasense-lab",
  "refresh": "30s",
  "time": {"from": "now-7d", "to": "now"},
  "timezone": "utc",
  "schemaVersion": 39,
  "version": 1,
  "templating": {
    "list": [
      {
        "name": "site",
        "label": "Site",
        "type": "query",
        "datasource": {"type": "postgres", "uid": "terrasense-ds"},
        "definition": "SELECT DISTINCT site FROM sensor_nodes ORDER BY 1",
        "query": "SELECT DISTINCT site FROM sensor_nodes ORDER BY 1",
        "refresh": 1,
        "multi": true,
        "includeAll": true,
        "sort": 1
      },
      {
        "name": "node",
        "label": "Node",
        "type": "query",
        "datasource": {"type": "postgres", "uid": "terrasense-ds"},
        "definition": "SELECT DISTINCT node_id FROM sensor_nodes WHERE site IN ($site) ORDER BY
        ↪  1",
        "query": "SELECT DISTINCT node_id FROM sensor_nodes WHERE site IN ($site) ORDER BY 1",
        "refresh": 1,
        "multi": true,
        "includeAll": true,
        "sort": 1
      }
    ]
  },
  "panels": [
    {
      "id": 1,
      "type": "timeseries",
      "title": "Hőmérséklet node-onként",
```

```
      "gridPos": {"h": 8, "w": 12, "x": 0, "y": 0},
      "fieldConfig": {
        "defaults": {"unit": "celsius", "custom": {}},
        "overrides": []
      },
      "targets": [
        {
          "refId": "A",
          "datasource": {"type": "postgres", "uid": "terrasense-ds"},
          "format": "time_series",
          "rawQuery": true,
          "editorMode": "code",
          "rawSql": "SELECT $__timeGroup(ts,'5m') AS time, node_id AS metric, avg(value) AS
          ↪  \"avg_temp\" FROM telemetry WHERE $__timeFilter(ts) AND site IN ($site) AND
          ↪  node_id IN ($node) AND measurement_type = 'temperature' AND valid = true GROUP BY
          ↪  1, 2 ORDER BY 1"
        }
      ]
    },
    {
      "id": 2,
      "type": "timeseries",
      "title": "Páratartalom node-onként",
      "gridPos": {"h": 8, "w": 12, "x": 12, "y": 0},
      "fieldConfig": {
        "defaults": {"unit": "percent", "custom": {}},
        "overrides": []
      },
      "targets": [
        {
          "refId": "A",
          "datasource": {"type": "postgres", "uid": "terrasense-ds"},
          "format": "time_series",
          "rawQuery": true,
          "editorMode": "code",
          "rawSql": "SELECT $__timeGroup(ts,'5m') AS time, node_id AS metric, avg(value) AS
          ↪  \"avg_humi\" FROM telemetry WHERE $__timeFilter(ts) AND site IN ($site) AND
          ↪  node_id IN ($node) AND measurement_type = 'humidity' AND valid = true GROUP BY 1,
          ↪  2 ORDER BY 1"
        }
      ]
    },
    {
      "id": 3,
      "type": "timeseries",
      "title": "Talajnedvesség node-onként",
      "gridPos": {"h": 8, "w": 12, "x": 0, "y": 8},
      "fieldConfig": {
        "defaults": {"unit": "percent", "custom": {}},
        "overrides": []
      },
      "targets": [
        {
          "refId": "A",
          "datasource": {"type": "postgres", "uid": "terrasense-ds"},
          "format": "time_series",
          "rawQuery": true,
          "editorMode": "code",
          "rawSql": "SELECT $__timeGroup(ts,'5m') AS time, node_id AS metric, avg(value) AS
          ↪  \"avg_soil\" FROM telemetry WHERE $__timeFilter(ts) AND site IN ($site) AND
          ↪  node_id IN ($node) AND measurement_type = 'soil_moisture' AND valid = true GROUP
          ↪  BY 1, 2 ORDER BY 1"
        }
```

```
    ]
  },
  {
    "id": 4,
    "type": "stat",
    "title": "Utolsó hőmérséklet - $node",
    "repeat": "node",
    "repeatDirection": "h",
    "maxPerRow": 4,
    "gridPos": {"h": 8, "w": 12, "x": 12, "y": 8},
    "fieldConfig": {"defaults": {"unit": "celsius"}},
    "options": {
      "reduceOptions": {"calcs": ["lastNotNull"], "fields": "", "values": false}
    },
    "targets": [
      {
        "refId": "A",
        "datasource": {"type": "postgres", "uid": "terrasense-ds"},
        "format": "table",
        "rawQuery": true,
        "editorMode": "code",
        "rawSql": "SELECT value AS \"last_temp\" FROM telemetry WHERE $__timeFilter(ts) AND
        ↪  site IN ($site) AND node_id = '$node' AND measurement_type = 'temperature' AND
        ↪  valid = true ORDER BY ts DESC LIMIT 1"
      }
    ]
  },
  {
    "id": 5,
    "type": "timeseries",
    "title": "Napi átlaghőmérséklet (telemetry_daily)",
    "gridPos": {"h": 8, "w": 24, "x": 0, "y": 16},
    "fieldConfig": {
      "defaults": {"unit": "celsius", "custom": {}},
      "overrides": []
    },
    "targets": [
      {
        "refId": "A",
        "datasource": {"type": "postgres", "uid": "terrasense-ds"},
        "format": "time_series",
        "rawQuery": true,
        "editorMode": "code",
        "rawSql": "SELECT bucket AS time, node_id AS metric, avg_value AS \"daily_avg_temp\"
        ↪  FROM telemetry_daily WHERE $__timeFilter(bucket) AND site IN ($site) AND node_id
        ↪  IN ($node) AND measurement_type = 'temperature' ORDER BY 1"
      }
    ]
  },
  {
    "id": 6,
    "type": "heatmap",
    "title": "Napi adathiány (elvárt minta 90%-a alatt)",
    "timeFrom": "7d",
    "gridPos": {"h": 8, "w": 24, "x": 0, "y": 24},
    "fieldConfig": {"defaults": {"unit": "short", "min": 0, "max": 1}},
    "targets": [
      {
        "refId": "A",
        "datasource": {"type": "postgres", "uid": "terrasense-ds"},
        "format": "time_series",
        "rawQuery": true,
        "editorMode": "code",
```

```
          "rawSql": "WITH days AS (SELECT generate_series(date_trunc('day',
          ↪  $__timeFrom()::timestamptz), date_trunc('day', $__timeTo()::timestamptz), interval
          ↪  '1 day') AS day), expected AS (SELECT n.site, n.node_id, e.measurement_type,
          ↪  e.samples_per_day, e.tolerance_ratio FROM sensor_nodes n CROSS JOIN
          ↪  sensor_expected_samples e WHERE n.site IN ($site) AND n.node_id IN ($node) AND
          ↪  e.measurement_type = 'temperature'), actual AS (SELECT bucket::date AS day, site,
          ↪  node_id, measurement_type, sum(sample_count)::bigint AS sample_count FROM
          ↪  telemetry_daily WHERE $__timeFilter(bucket) AND measurement_type = 'temperature'
          ↪  GROUP BY 1, 2, 3, 4) SELECT d.day AS time, x.node_id AS metric, CASE WHEN
          ↪  coalesce(a.sample_count, 0) < x.samples_per_day * (1 - x.tolerance_ratio) THEN 1
          ↪  ELSE 0 END AS \"adathiany\" FROM days d CROSS JOIN expected x LEFT JOIN actual a
          ↪  ON a.day = d.day::date AND a.site = x.site AND a.node_id = x.node_id AND
          ↪  a.measurement_type = x.measurement_type WHERE d.day::date < current_date ORDER BY
          ↪  1, 2"
        }
      ]
    }
  ]
}
```

Egészítsd ki a közös `~/iot-labor/stack/compose.yaml` compose-fájlt a Grafana-szolgáltatással. A `lab-grafana` konténer a `../i051/grafana-provisioning` útvonalat mountolja, mert a compose-fájl a `stack` mappában van, a laborfájlok pedig az `i051` mappában. A szolgáltatás nem kap külön `networks` bejegyzést: a stack összes szolgáltatása az alapértelmezett compose-hálózaton van, így a `lab-grafana` a `lab-timescaledb` szolgáltatásnevet közvetlenül feloldja. A `GF_SECURITY_ADMIN_USER` és `GF_SECURITY_ADMIN_PASSWORD` kizárólag laborcélra szolgáló érték.

```
  lab-grafana:
    image: grafana/grafana:11.2.0
    container_name: lab-grafana
    ports: ["3000:3000"]
    environment:
      GF_SECURITY_ADMIN_USER: lab-admin
      GF_SECURITY_ADMIN_PASSWORD: lab-pass
    volumes:
      - ../i051/grafana-provisioning:/etc/grafana/provisioning
      - lab-grafana-data:/var/lib/grafana
    depends_on:
      - lab-timescaledb
```

A compose-fájl `volumes` szekciójához add hozzá a `lab-grafana-data` névvel jelölt kötetet. Az explicit `name` megadásával a kötet neve nem kap compose-projekt előtagot, összhangban a `lab-timescaledb-data` kötet Idősoros adatbázisok és adatmodellezés alfejezetbeli definíciójával.

```
volumes:
  lab-grafana-data:
    name: lab-grafana-data
```

Indítsd el a Grafanát, és ellenőrizd a konténer állapotát, majd a health endpointot:

```
cd ~/iot-labor/stack
docker compose up -d lab-grafana
sleep 5
docker compose ps lab-grafana
curl -s http://localhost:3000/api/health
```

Várt kimenet: a health-válasz JSON-objektum, amelyben a `database` mező `ok` értékű. Ezután böngészőben nyisd meg a `http://localhost:3000` címet, jelentkezz be a `lab-admin` / `lab-pass` hitelesítővel. A `Dashboards` menüben válaszd a `TerraSense - Üvegház monitor` dashboardot. A felső `Site` és `Node` választóban például a `gh1` site és az `n001` node választható; a hőmérséklet-,

páratartalom- és talajnedvesség-panel node-onként külön sorozatot rajzol, az utolsó hőmérséklet panel pedig node-onként ismétlődik.

A `gh1/n002` node a nyilvántartásban szerepel, de nincs telemetriája. Ha a dashboard időtartományát 7 napra állítod, a hőtérkép minden lezárt napra `1` értéket mutat rá, míg az `n001` a normál 144 mintás napokon `0` marad. Ez a teljesen néma eszköz és a teljesen hiányzó nap észlelésének ellenőrzése; a `telemetry_daily`-ben ugyanis ezekre a napokra egyáltalán nincs sor, ezért kell a `sensor_nodes` nyilvántartás és a dátumrács.

A labor takarítása a saját, névvel jelölt objektumokra korlátozódik:

```
cd ~/iot-labor/stack
docker compose rm -f lab-grafana
docker volume rm lab-grafana-data
```

17.3.5 Ellenőrzés és hibakeresés

A provisioning hibáit a Grafana logjában érdemes keresni:

```
docker compose logs lab-grafana | grep -i provisioning
```

A dashboard betöltését a böngészőn kívül hitelesítéssel is ellenőrizhetjük, de a laborban elegendő a health-check és a UI. Gyakori hibák és javításuk:

Tünet	Ok	Javítás
A dashboard nem jelenik meg	A provisioning mappa mountolása rossz, vagy a dashboard JSON érvénytelen	Ellenőrizd a compose `volumes`-bejegyzést és a JSON-érvényességet; `docker compose logs lab-grafana`
„permission denied for relation telemetry”	A `grafana_ro` nem kapott `SELECT`-et a szükséges objektumokra	Futtasd újra a `02_grafana_ro.sql` szkriptet, és ellenőrizd a `\dp` kimenetét
„PostgreSQL connection refused”	A Grafana nem éri el a `lab-timescaledb` konténert	Ellenőrizd, hogy a két szolgáltatás ugyanazon az alapértelmezett compose-hálózaton van, és a `lab-timescaledb` fut
„No data” az idősor panelen	A `$__timeFilter` makró hiányzik, a `valid = true` szűrő kizárja az összes mintát, vagy az időtartomány túl rövid	Győződj meg arról, hogy a query-ben szerepel `$__timeFilter(ts)`, és bővítsd a dashboard `time` tartományát
A panel üres, a Query Inspectorban `$__timeFilter` látszik	A target `rawSql` helyett `query` mezőt használ, vagy a `rawQuery`/`editorMode` hiányzik	Használd a `rawSql` mezőt `rawQuery: true` és `editorMode: "code"` mellett; ellenőrizd a Query Inspectorban a tényleges SQL-t
A hőtérkép minden napra piros	A mai, még nem lezárt napot is vizsgálja, vagy a `telemetry_daily` még nem frissült	A minta kizárja a `current_date` napját; frissítsd a folyamatos aggregátumot, és várd meg a zárt napot

Tünet	Ok	Javítás
A hőmérséklet nyers számként jelenik meg	A `unit` hiányzik a panel `fieldConfig` részéből	Állítsd be a `unit: "celsius"` vagy `"percent"` értéket

17.3.6 Tipikus hibák és félreértések

1. A Grafana SQL-szerkesztőben makróként megadott `$__timeFilter(ts)` nem kerül be a dashboard JSON `rawSql` mezőjébe, vagy a makró neve `$timeFilter` (nem `__`) lesz. A Grafana így nyers `$` jelet küld a PostgreSQL-nek, amely szintaxishibát ad.
2. Multiváltozós `$node` esetén a lekérdezés `node_id = '$node'` formát használ. Ahelyett, hogy a Grafana automatikusan `IN ('n001','n002')`-vé bővítené, a string interpoláció törése miatt hibás SQL jön létre. Helyesen: `node_id IN ($node)` idézőjelek nélkül. Kivétel a `repeat`-tel ismételt stat panel, ahol a `$node` éppen az aktuális ismétlés egyetlen értéke, ezért ott szándékosan `node_id = '$node'` szerepel.
3. A hosszú távú panelek mindig a nyers `telemetry` táblát kérdezik, és nem a Megőrzés, ritkítás és tárolási költség folyamatos aggregátumait. Ez lassú és feleslegesen sok pontot rajzol ki. A napi panel ezért a `telemetry_daily` `bucket` és `avg_value` oszlopát használja.
4. A Grafana időzónája eltér az adatbázis `ts` mezőjének UTC-jétől. A dashboard `timezone` beállítása ezért `utc`; a böngésző időzónája csak a megjelenítést befolyásolhatja, a tárolt adat továbbra is UTC.
5. A napi adathiányt csak a `telemetry_daily` soraiból számolni hibás, mert a teljesen néma node/nap egyáltalán nem hoz létre sort. A helyes minta a `sensor_nodes` nyilvántartás, a `sensor_expected_samples` elvárt mintaszáma és egy dátumrács összekapcsolása.

17.3.7 Feladatok

1. **Fogalmi/döntési kérdés.** Miért használ a TerraSense hosszú távú trendjeihez `telemetry_daily` nézetet a nyers `telemetry` tábla helyett? Milyen vizualizációt választanál a napi adathiány megjelenítésére?
2. **Tervezési feladat.** Írj a Grafana `time_series` formátumához SQL-lekérdezést, amely a Megőrzés, ritkítás és tárolási költség `telemetry_hourly` nézetéből 7 napos átlaghőmérsékletet ad vissza a `gh1/n001` node-ra, a Grafana dashboard időtartományát alkalmazva. A nézet oszlopnevei `bucket`, `site`, `node_id`, `measurement_type` és `avg_value`.
3. **Implementációs feladat.** Bővítsd a `dashboard.json` fájlt egy heatmap-panelnel, amely a `sensor_expected_samples` méréstípusonkénti elvárt napi mintaszáma (a 10 perces ciklus miatt 144) és a `telemetry_daily` `sample_count` értéke alapján jelzi a 90% alatti napokat. A teljesen néma node/nap is jelenjen meg; használd a `sensor_nodes` nyilvántartást és dátumrácsot, és zárd ki a még nem lezárt mai napot.
4. **Hibakeresési feladat.** A dashboard panel „No data” állapotot mutat, a PostgreSQL logban pedig `syntax error at or near "$"` hiba látható. Mi a hiba valószínű oka, és hogyan javítanád?

17.3.8 Megoldások

1. A `telemetry_daily` nézet már előre aggregálja az átlagokat, így egy 1 éves trend sokkal kisebb adatmennyiségből jeleníthető meg. A nyers tábla 30 napig őrzi csak meg a finom felbontású adatokat. Az adathiányhoz heatmap-panel illik: az x tengely az idő, az y tengely a node, a cella színe pedig az elvárt mintaszámtól való elmaradás. A teljesen

hiányzó napokat a `sensor_nodes` nyilvántartásból és dátumrácsból kell előállítani, mert az aggregátumban ezekre nincs sor.

2. A lekérdezés:

```
SELECT bucket AS time, avg_value AS "avg_temp"
FROM telemetry_hourly
WHERE $__timeFilter(bucket)
  AND site = 'gh1'
  AND node_id = 'n001'
  AND measurement_type = 'temperature'
ORDER BY 1
```

A panel `format` mezője `time_series`, a target pedig `rawSql` mezőt, `rawQuery: true` és `editorMode: "code"` beállítást használ. A hosszabb, 7 napos időtartományt a dashboard `time` beállítása adja; a nézetből így órás bontásban rajzolódik a trend.

3. A heatmap-panel mintája. A `sensor_expected_samples` táblában a `samples_per_day` 144, a `tolerance_ratio` 0.10; a query a `sensor_nodes` nyilvántartásból és egy dátumrácsból indul, ezért a teljesen néma node/nap is megjelenik. A `current_date` kizárja a még nem lezárt mai napot.

```
{
  "id": 6,
  "type": "heatmap",
  "title": "Napi adathiány (elvárt minta 90%-a alatt)",
  "timeFrom": "7d",
  "gridPos": {"h": 8, "w": 24, "x": 0, "y": 24},
  "fieldConfig": {"defaults": {"unit": "short", "min": 0, "max": 1}},
  "targets": [
    {
      "refId": "A",
      "datasource": {"type": "postgres", "uid": "terrasense-ds"},
      "format": "time_series",
      "rawQuery": true,
      "editorMode": "code",
      "rawSql": "WITH days AS (SELECT generate_series(date_trunc('day',
      ↪  $__timeFrom()::timestamptz), date_trunc('day', $__timeTo()::timestamptz),
      ↪  interval '1 day') AS day), expected AS (SELECT n.site, n.node_id,
      ↪  e.measurement_type, e.samples_per_day, e.tolerance_ratio FROM sensor_nodes n
      ↪  CROSS JOIN sensor_expected_samples e WHERE n.site IN ($site) AND n.node_id IN
      ↪  ($node) AND e.measurement_type = 'temperature'), actual AS (SELECT bucket::date
      ↪  AS day, site, node_id, measurement_type, sum(sample_count)::bigint AS
      ↪  sample_count FROM telemetry_daily WHERE $__timeFilter(bucket) AND
      ↪  measurement_type = 'temperature' GROUP BY 1, 2, 3, 4) SELECT d.day AS time,
      ↪  x.node_id AS metric, CASE WHEN coalesce(a.sample_count, 0) < x.samples_per_day *
      ↪  (1 - x.tolerance_ratio) THEN 1 ELSE 0 END AS \"adathiany\" FROM days d CROSS
      ↪  JOIN expected x LEFT JOIN actual a ON a.day = d.day::date AND a.site = x.site
      ↪  AND a.node_id = x.node_id AND a.measurement_type = x.measurement_type WHERE
      ↪  d.day::date < current_date ORDER BY 1, 2"
    }
  ]
}
```

Az `n001` normál 10 perces ciklusban napi 144 mintát küld, ami nem kisebb a 144 × 0,9 = 129,6 határnál, ezért nem ad riasztást. A nyilvántartott, de néma `n002` minden lezárt napra 0 mintát ad, ezért a hőtérkép 1-et mutat. A `sample_count` a `telemetry_daily` méréstípusonkénti darabszáma; a napi nézet a `sample_count` értékeket összegzi.

4. A hiba oka általában az, hogy a Grafana nem helyettesíti a `$__timeFilter` vagy `$__timeGroup` makrót, mert a dashboard JSON targetjében `query` mező szerepel a helyes

`rawSql` helyett, hiányzik a `rawQuery: true`, vagy a query nem a Postgres adatforrás `uid`-jét használja. A PostgreSQL a nyers `$` karaktert elutasítja. Javítás: a target `rawSql` mezőt, `rawQuery: true` és `editorMode: "code"` beállítást használjon, a `datasource.uid` értéke `terrasense-ds` legyen, a makró neve pedig pontosan `$__timeFilter(ts)` vagy `$__timeGroup(ts,'5m')`. A Query Inspectorban ellenőrizd, hogy a Grafana a makrót már feloldott SQL-ként küldi a PostgreSQL-nek.

17.3.9 Rövid összefoglaló

Az alfejezetben a TerraSense adataira építettünk Grafana-dashboardot. A provisioning fájlok a `~/iot-labor/i051/grafana-provisioning` mappában találhatók: `datasources/datasources.yml` definiálja a TimescaleDB-adatforrást `terrasense-ds` UID-del és a csak olvasási jogú `grafana_ro` felhasználóval, `dashboards/dashboards.yml` a fájlalapú dashboard-betöltést, `dashboards/dashboard.json` pedig a `TerraSense – Üvegház monitor` dashboardot (`uid: terrasense-lab`). A dashboard `$site` és `$node` multiváltozót használ; a `$node` a `sensor_nodes` nyilvántartásból táplálkozik, ezért a néma node is kiválasztható. A nyers telemetriát használó panelek `valid = true` szűréssel és `node_id AS metric` csoportosítással node-onként külön sorozatot rajzolnak, az utolsó hőmérséklet stat panel pedig node-onként ismétlődik. A hosszú távú panel a `telemetry_daily` `bucket`, `avg_value` és `sample_count` oszlopait használja; a hőtérkép a `sensor_expected_samples` elvárt mintaszáma, a `sensor_nodes` nyilvántartás és egy dátumrács alapján a teljesen néma node/napot is jelzi. A labor SQL-szkriptjei létrehozzák a csak olvasási jogú szerepkört és a segédtáblákat. **Továbbvitt eredmény.** A `dashboard.json` panel-lekérdezéseit használja az Riasztás: szabályok, értesítés, riasztási fáradtság alfejezet, a teljes TerraSense stack pedig a Záróprojekt során ezzel a Grafana-szolgáltatással áll össze.

17.3.10 Források és ellenőrzési hiányok

- Grafana provisioning dokumentáció: adatforrás- és dashboard-provider YAML felépítése; a `schemaVersion` és a panel JSON-mezők pontos neve a használt Grafana-verziótól függ, ellenőrizendő.
- Grafana PostgreSQL data source: a `rawSql`, `rawQuery` és `editorMode` targetmezők, valamint a `$__timeFilter`, `$__timeGroup` és multiváltozós interpoláció pontos szintaxisa a használt Grafana 11.2.0 verzióban ellenőrizendő; a fejezet kódját a szerző statikusan, JSON- és SQL-szintaktikailag ellenőrizte, valódi Grafana-példányon nem futtatta.
- A Grafana `11.2.0`, a TimescaleDB `2.17.2` és a PostgreSQL `16.4` rögzített laborverzió; a `grafana/grafana:11.2.0` konténerkép létezése és frissítési állapota használat előtt ellenőrizendő.
- TimescaleDB: a `telemetry_hourly` és `telemetry_daily` nézetek oszlopnevei a Megőrzés, ritkítás és tárolási költség alfejezetben létrehozott `bucket`, `site`, `node_id`, `measurement_type`, `avg_value`, `min_value`, `max_value`, `sample_count` mezők. A folyamatos aggregátumok jelenleg nem tartalmaznak `valid` oszlopot, ezért a hosszú távú panelek minden ingestált mintát beszámítanak; éles rendszerben a nézetet a `valid = true` feltétellel kell létrehozni, vagy a dashboardnak a nyers táblát kell használnia.
- A `lab-admin` / `lab-pass` Grafana-admin és a `grafana_ro` / `grafana-lab-only` adatbázis-hitelesítő kizárólag a helyi laborrendszerhez tartozik; éles környezetben használni tilos.

17.4 Riasztás: szabályok, értesítés, riasztási fáradtság

17.4.1 Motiváció

A TerraSense üvegházban a növények állapota akkor is romolhat, ha senki sem nézi a Grafanát. A hőmérséklet túllépheti a kritikus értéket, a talajnedvesség túl alacsony lehet, vagy ami még rosszabb: a szenzorcsomópont egyszerűen leáll, és nem érkezik több adat. Az operátorok nem figyelhetik folyamatosan a dashboardot, ezért a rendszernek magának kell észlelnie a rendellenességeket, és értesítenie kell a megfelelő személyt. A rosszul beállított riasztások azonban hamis pozitívokkal áraszthatják el az üzemeltetőt, ami riasztási fáradtsághoz vezet, és a valódi krízisek elvesznek a zajban. Ebben az alfejezetben olyan riasztási szabályokat tervezünk, amelyek megbízhatóan jelzik a problémákat anélkül, hogy felesleges zajt generálnának.

17.4.2 Fogalmak és működés

Definíció. A *riasztási szabály* (alert rule) egy kiértékelhető feltétel, egy időtartam és egy súlyossági szint együttese. A feltétel lehet küszöbátlépés, változási sebesség vagy hiányzó adat; az időtartam határozza meg, hogy a feltételnek mennyi ideig kell folyamatosan fennállnia a riasztás kiváltásához; a súlyosság pedig az értesítési csatornát és az eszkalációt befolyásolja.

A TerraSense-ben három alapvető riasztási osztályt különböztetünk meg:

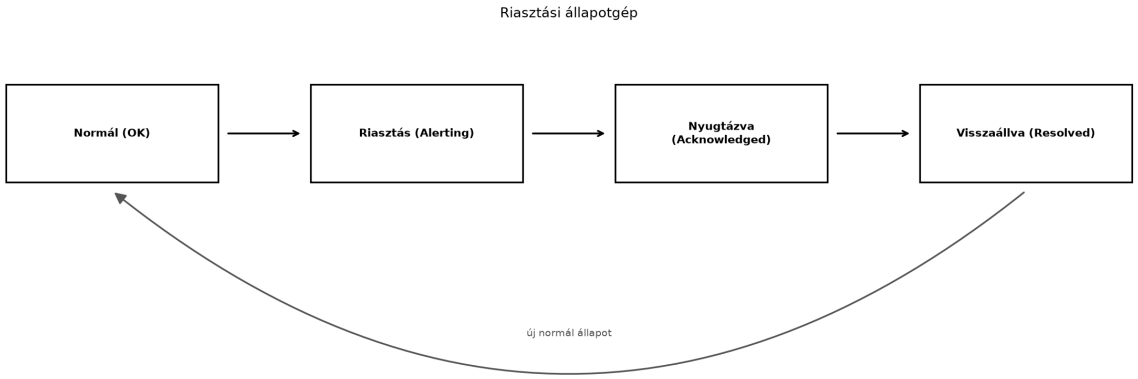
1. **Folyamatriasztás** (process alert): a mért fizikai érték nem megfelelő. Például a hőmérséklet meghaladja a 35 °C-ot, vagy a talajnedvesség 20 % alá esik. Ez a leggyakoribb típus, és általában küszöb alapú.
2. **Adathiány-riasztás** (data gap alert): az eszköz nem küld telemetriát a várt időn belül. A heartbeat (szívverés) hiánya arra utalhat, hogy az eszköz leállt, a rádiókapcsolat megszakadt, vagy a gateway puffer túlcsordult.
3. **Eszközhiba-riasztás** (device fault alert): az eszköz explicit módon hibát jelez, például a Last Will testamentum (LWT) által, vagy a `state` topicban megjelenő `local_alert` mezővel. Ez lehet szenzorhiba, alacsony akkumulátorszint, vagy a beágyazott szoftver watchdog újraindulása.

A folyamat- és adathiány-riasztásokat a backend (Grafana) és az edge is kiértékelheti. Az edge-riasztás alacsony késleltetést biztosít akkor is, ha a felhőkapcsolat megszakad, de az állapotot akkor is továbbítani kell a backend felé, amikor a kapcsolat helyreáll. A TerraSense-ben az edge aggregátor már előállít egy `local_alert` mezőt a `terrasense/<site>/<node>/state` retained topicra; ezt a backend felhasználhatja, de a végső döntést és az értesítést a központi riasztási motor hozza meg, hogy az értesítési csatornák egységesek legyenek.

A riasztási szabályoknak két fontos mechanizmust kell tartalmazniuk:

- **Időtartam** (for): a feltételnek egy meghatározott ideig folyamatosan igaznak kell lennie. Ez kiszűri a rövid, átmeneti kiugrásokat. Például egy ajtónyitás okozta 10 másodperces hőmérséklet-csökkenés ne váltson ki riasztást.
- **Hiszterézis** (hysteresis): a riasztás bekapcsolási és kikapcsolási küszöbe különbözik. Például a hőmérséklet-riasztás 35 °C felett indul, de csak 34 °C alatt szűnik meg. Ez megakadályozza, hogy az érték a küszöb körül ingadozva többször ki-be kapcsolja a riasztást.

A riasztás életciklusát a következő állapotgép írja le:



17.4. ábra: Riasztási állapotgép - A riasztás állapotai a feltétel kiértékelése során. A visszaállás után a szabály újra normál állapotba kerül.

Az ábra azt mutatja, hogy a riasztás akkor vált ki, ha a feltétel az időtartam alatt fennáll. Az operátor nyugtázhatja a riasztást, ami nem szünteti meg a mögöttes állapotot, csak jelzi, hogy tudomásul vették. Amikor a feltétel megszűnik, a riasztás visszaáll, és a szabály újra normál állapotba kerül.

A küszöb-riasztás hiszterézises kiértékelését a következő C++-szerű pszeudokód mutatja:

```
bool evaluate_threshold(double current, double high, double low, bool &alerting) {
    if (!alerting && current > high) return true;  // bekapcsolás
    if (alerting && current < low) return false;   // kikapcsolás
    return alerting;                               // nincs változás
}
```

A `high` és `low` közötti sáv a holtsáv (deadband), amelyben az állapot nem változik.

17.4.3 Döntési szempontok

Az alábbi táblázat összefoglalja, hogy a riasztási szabályt az edge-en vagy a backendben érdemes-e kiértékelni.

Szempont	Edge riasztás	Backend riasztás
Késleltetés	Alacsony, helyben dönt	Hálózati körülményektől függ
Offline működés	Működik, ha a felhő elérhetetlen	Nem működik
Globális állapot, korreláció	Korlátozott	Teljes, több eszköz adataiból
Konfiguráció frissítése	Lassabb, eszközre kell juttatni	Azonnal, központilag
Erőforrásigény az eszközön	Alacsony, de nem nulla	Nincs
Értesítés, eszkaláció	Helyi (pl. hangjelzés)	E-mail, webhook, SMS, push

A TerraSense-ben mindkét szintet használjuk: az edge az azonnali, biztonságkritikus döntéseket hozza meg (pl. öntözőszelep leállítása túl alacsony talajnedvesség esetén), a backend pedig az összes eszközre kiterjedő, értesítéssel járó riasztásokat kezeli. A `local_alert` állapotot a backend is figyeli, így az edge eseményei bekerülnek a központi riasztási folyamatba.

Az értesítési csatornák kiválasztásánál vegyük figyelembe a súlyosságot és az üzemeltetési modellt:

- **E-mail**: alacsony súlyosságú riasztásokhoz, vagy napi összesítőhöz. Nem valós idejű.
- **Webhook**: integráció más rendszerekbe (pl. Slack, Teams, vagy egy incidenskezelő). Alacsony késleltetés.
- **SMS / push**: kritikus riasztásokhoz, amikor az operátor nem a számítógép előtt van.
- **Hanghívás**: legmagasabb súlyosságú, éjszakai ügyelethez.

Az eszkaláció azt jelenti, hogy ha a riasztást nem nyugtázzák egy adott időn belül, akkor magasabb szintű értesítés indul. Például: először e-mail, 15 perc után webhook a csatornába, 30 perc után SMS a vezetőnek. Ezzel elkerülhető, hogy egyetlen elérhetetlen személy miatt a probléma észrevétlen maradjon.

17.4.4 Kidolgozott példa: folyamatriasztás hiszterézissel és időtartammal

Tegyük fel, hogy a TerraSense `gh1` üvegházában a hőmérsékletre vonatkozó szabály a következő:

- **Feltétel:** hőmérséklet > 35 °C.
- **Időtartam:** 5 perc folyamatosan.
- **Hiszterézis:** riasztás bekapcsol 35 °C felett, kikapcsol 34 °C alatt.

A következő idősor érkezik percenként (illusztratív adatok):

Idő (perc)	Hőmérséklet (°C)	Állapot	Megjegyzés
0	33,0	Normál	
1	34,5	Normál	
2	36,0	Figyelés	Feltétel igaz, időtartam indul
3	35,8	Figyelés	
4	35,5	Figyelés	
5	35,6	Figyelés	
6	35,2	**Riasztás**	5 perc eltelt, folyamatosan > 35 °C
7	36,0	Riasztás	
8	34,8	Riasztás	Még 34 °C felett, nincs visszaállás
9	33,9	**Visszaállás**	34 °C alá esett

A 6. percben a riasztás kivált, mert a feltétel a 2–6. percekben öt percen át fennállt. A 9. percben a hőmérséklet 33,9 °C, ami a 34 °C-os kikapcsolási küszöb alatt van, ezért a riasztás megszűnik. Ha a hiszterézis nem lenne, és a küszöb 35 °C lenne mindkét irányban, akkor a 8. és 9. perc közötti ingadozás esetleg többször is ki-be kapcsolná a riasztást.

Ugyanezt a logikát alkalmazhatjuk az adathiány-riasztásra: ha nem érkezik telemetria a `terrasense/gh1/n001/telemetry` topicról a 10 perces ciklushoz képest számított türelmi időn belül, akkor riasztás. A Grafanában ezt olyan lekérdezéssel valósítjuk meg, amely a friss fogadási idő (`server_ts`) alapján számol, nem a mérési idő (`ts`) alapján. A kettő szétválasztása azért fontos, mert egy késői backfill friss fogadásként jelenik meg, miközben a mért érték már elavult; erre külön, a `ts`-re épülő szabályt használunk.

17.4.5 Labor: Grafana-riasztás hiányzó adatra

Ebben a laborban a Grafana alerting rendszerét úgy konfiguráljuk, hogy riasztást küldjön, ha a `ts-node` eszköz nem küld telemetriát, illetve ha a fogadott adatok mérési időbélyege elavult. Feltételezzük, hogy a `~/iot-labor/stack/compose.yaml` már tartalmazza a Grafanát,

a PostgreSQL + TimescaleDB-t, és az adatbázisban a `telemetry` tábla a korábbi laborok szerint van feltöltve. Az értesítéshez egy webhook contact pointot használunk, amelyet egy saját konténerben futó, egyszerű HTTP végpont fogad.

A labor a Dashboardok Grafanával alfejezet stackjére épül: ugyanazt a `lab-grafana` szolgáltatást és ugyanazt a `~/iot-labor/i051/grafana-provisioning` mountot használjuk. **Nem adunk hozzá új Compose mountot**, ezért a meglévő bind mount alá írt riasztási fájlokat a Grafana restartja beolvassa. Ha valaki mégis új mountot adna a szolgáltatáshoz, a `docker compose restart` nem alkalmazza azt; ilyenkor `docker compose up -d --force-recreate lab-grafana` szükséges. A webhook külön szolgáltatás, ezért annak mountja a saját konténerének létrehozásakor érvényesül.

Munkakönyvtár: `~/iot-labor/i052`

```
mkdir -p ~/iot-labor/i052
mkdir -p ~/iot-labor/i051/grafana-provisioning/alerting
cd ~/iot-labor/i052
```

Előfeltétel: fut a stack (`docker compose -f ~/iot-labor/stack/compose.yaml up -d`), a Grafana elérhető a `http://localhost:3000` címen, és a Dashboardok Grafanával adatforrása (`terrasense-ds`, `grafana_ro`) működik.

17.4.5.1 1. A webhook-fogadó

Készítsük el a `~/iot-labor/i052/webhook_server.py` fájlt. Ez egy több soros, önálló Python-program, amely a `ThreadingHTTPServer`-rel fogadja a POST kéréseket, a JSON payloadot kiírja a konténer naplójába, és `200 OK` választ ad. A `Content-Length` hiányát és a túl nagy kérést is kezeli; a `/health` végpont a konténer állapotának ellenőrzésére szolgál.

```
#!/usr/bin/env python3
"""lab-webhook: a TerraSense riasztási labor webhook-fogadója."""
import json
import os
from http.server import BaseHTTPRequestHandler, ThreadingHTTPServer

HOST = "0.0.0.0"
PORT = int(os.environ.get("WEBHOOK_PORT", "8081"))
MAX_BODY_BYTES = 1_048_576


class AlertHandler(BaseHTTPRequestHandler):
    def _send_json(self, status: int, payload: dict) -> None:
        body = json.dumps(payload, ensure_ascii=False).encode("utf-8")
        self.send_response(status)
        self.send_header("Content-Type", "application/json; charset=utf-8")
        self.send_header("Content-Length", str(len(body)))
        self.end_headers()
        self.wfile.write(body)

    def do_GET(self) -> None:
        if self.path == "/health":
            self._send_json(200, {"status": "ok"})
        else:
            self._send_json(404, {"status": "not_found"})

    def do_POST(self) -> None:
        if self.path != "/alert":
            self._send_json(404, {"status": "not_found"})
            return
```

```
        try:
            length = int(self.headers.get("Content-Length", "0"))
        except ValueError:
            length = 0
        if length < 0 or length > MAX_BODY_BYTES:
            self._send_json(413, {"status": "too_large"})
            return
        raw = self.rfile.read(length) if length else b""
        try:
            payload = json.loads(raw.decode("utf-8"))
        except (UnicodeDecodeError, json.JSONDecodeError) as exc:
            print(f"[lab-webhook] érvénytelen JSON: {exc}", flush=True)
            self._send_json(400, {"status": "invalid_json"})
            return
        print(json.dumps(payload, ensure_ascii=False, indent=2), flush=True)
        self._send_json(200, {"status": "received"})

    def log_message(self, fmt: str, *args) -> None:
        print(f"[lab-webhook] {self.address_string()} {fmt % args}", flush=True)


def main() -> None:
    server = ThreadingHTTPServer((HOST, PORT), AlertHandler)
    print(f"[lab-webhook] figyelés: http://{HOST}:{PORT}/alert", flush=True)
    try:
        server.serve_forever()
    except KeyboardInterrupt:
        print("[lab-webhook] leállítás", flush=True)
    finally:
        server.server_close()


if __name__ == "__main__":
    main()
```

A fájlt először helyben ellenőrizzük szintaktikailag, majd egy rövid HTTP-próbával is:

```
cd ~/iot-labor/i052
python3 -m py_compile webhook_server.py
WEBHOOK_PORT=18081 python3 webhook_server.py &
sleep 1
curl -s -X POST -H 'Content-Type: application/json' \
  -d '{"status":"test","alerts":[{"labels":{"alertname":"proba"}}]}' \
  http://127.0.0.1:18081/alert
curl -s http://127.0.0.1:18081/health
kill %1
```

A POST válasza `{"status": "received"}`, a health válasza `{"status": "ok"}`, a naplóban pedig megjelenik a payload. Ez a helyi próba még nem a Grafana értesítése, hanem a fogadó működésének ellenőrzése.

17.4.5.2 2. A webhook szolgáltatás a Compose stackben

Egészítsük ki a közös `~/iot-labor/stack/compose.yaml` fájlt a `lab-webhook` szolgáltatással. A konténer a `python:3.12.3-slim` képet használja, a `~/iot-labor/i052` mappát csak olvashatóan mountolja, és a 8081-es portot publikálja. A 8080-as portot a `ts-backend` már foglalja, ezért a webhook külön portot kap. A szolgáltatás nem kap külön hálózatot: a Grafana és a webhook is az alapértelmezett compose-hálózaton van, így a Grafana a `lab-webhook` szolgáltatásnevet oldja fel, és nincs szükség a Linuxon nem garantált `host.docker.internal` címre.

```
  lab-webhook:
    image: python:3.12.3-slim
    container_name: lab-webhook
    command: ["python", "/app/webhook_server.py"]
    ports: ["8081:8081"]
    environment:
      WEBHOOK_PORT: "8081"
    volumes:
      - ../i052:/app:ro
```

Indítsuk el, és nézzük meg a naplóját:

```
cd ~/iot-labor/stack
docker compose up -d lab-webhook
docker logs -f lab-webhook
```

17.4.5.3 3. Contact point és értesítési útvonal

A Grafana alerting provisioning fájljai a meglévő Dashboardok Grafanával alfejezetbeli mount alá, a `~/iot-labor/i051/grafana-provisioning/alerting/` könyvtárba kerülnek. Először a contact pointot hozzuk létre a `contactpoints.yaml` fájlban. A webhook URL a konténer-hálózat szolgáltatásnevére mutat, nem `localhost`-ra és nem `host.docker.internal`-re.

```
apiVersion: 1

contactPoints:
  - orgId: 1
    name: terrasense-webhook
    receivers:
      - uid: terrasense-webhook-1
        type: webhook
        settings:
          url: http://lab-webhook:8081/alert
          httpMethod: POST
```

A contact point önmagában még nem küld értesítést: kell hozzá egy értesítési útvonal (notification policy). A `policies.yaml` a gyökér-policy receiverét a `terrasense-webhook` contact pointra állítja, és a riasztásokat `alertname`, `site` és `node` szerint csoportosítja. A `group_wait` és a `group_interval` csökkenti a riasztási fáradtságot; a `repeat_interval` négy óra, hogy egy fennálló riasztás ne generáljon percenként új értesítést.

```
apiVersion: 1

policies:
  - orgId: 1
    receiver: terrasense-webhook
    group_by:
      - alertname
      - site
      - node
    group_wait: 30s
    group_interval: 5m
    repeat_interval: 4h
```

17.4.5.4 4. A riasztási szabályok

A `terrasense.yaml` két szabályt definiál. Az első a friss fogadást vizsgálja: a `server_ts` a backend feldolgozási/fogadási ideje, ezért a `server_ts > now() - interval '15 minutes'` feltétel azt jelenti, hogy az elmúlt 15 percben érkezett-e érvényes minta. A 15 perc a 10 perces alapciklus plusz 5 perc türelmi idő; a `for: 2m` megakadályozza, hogy egyetlen kiértékelési hiba azonnal riasztást váltson ki. A második szabály a mérési időt (`ts`) vizsgálja: ha az elmúlt 15 percben érkezett friss fogadás mellett a legfrissebb mért érték is 30 percnél régebbi, akkor késői backfill vagy beragadt szenzoridő a valószínű ok. A címkékben a `site` és a `node` explicit módon szerepel, ezért az annotációkban a `{{ $labels.site }}` és `{{ $labels.node }}` mindig feloldható.

```yaml
apiVersion: 1

groups:
  - orgId: 1
    name: terrasense
    folder: TerraSense
    interval: 1m
    rules:
      - uid: ts-no-data
        title: "Nincs friss telemetria 15 perce"
        condition: C
        data:
          - refId: A
            relativeTimeRange:
              from: 900
              to: 0
            datasourceUid: terrasense-ds
            model:
              refId: A
              editorMode: code
              format: table
              rawQuery: true
              rawSql: |
                SELECT count(*) AS sample_count
                FROM telemetry
                WHERE server_ts > now() - interval '15 minutes'
                  AND site = 'gh1'
                  AND node_id = 'n001'
                  AND valid = true
          - refId: B
            datasourceUid: __expr__
            model:
              refId: B
              type: reduce
              expression: A
              reducer: last
              settings:
                mode: ""
          - refId: C
            datasourceUid: __expr__
            model:
              refId: C
              type: threshold
              expression: B
              conditions:
                - evaluator:
                    type: lt
                    params: [1]
        noDataState: Alerting
```

```
      execErrState: Alerting
      for: 2m
      labels:
        severity: critical
        site: gh1
        node: n001
      annotations:
        summary: "Nincs friss telemetria a {{ $labels.site }}/{{ $labels.node }} node-tól"
        description: "A 15 perces fogadási ablakban nem érkezett érvényes minta; a 10 perces
↪  ciklus plusz 5 perc türelmi idő telt el."

    - uid: ts-stale-sensor
      title: "Elavult szenzoradat (30 percnél régebbi mérés)"
      condition: C
      data:
        - refId: A
          relativeTimeRange:
            from: 900
            to: 0
          datasourceUid: terrasense-ds
          model:
            refId: A
            editorMode: code
            format: table
            rawQuery: true
            rawSql: |
              SELECT max(extract(epoch FROM (now() - ts))) AS staleness_seconds
              FROM telemetry
              WHERE server_ts > now() - interval '15 minutes'
                AND site = 'gh1'
                AND node_id = 'n001'
                AND valid = true
              HAVING count(*) > 0
        - refId: B
          datasourceUid: __expr__
          model:
            refId: B
            type: reduce
            expression: A
            reducer: last
            settings:
              mode: ""
        - refId: C
          datasourceUid: __expr__
          model:
            refId: C
            type: threshold
            expression: B
            conditions:
              - evaluator:
                  type: gt
                  params: [1800]
      noDataState: OK
      execErrState: Alerting
      for: 5m
      labels:
        severity: warning
        site: gh1
        node: n001
      annotations:
        summary: "A {{ $labels.site }}/{{ $labels.node }} mérési időbélyege 30 percnél
↪  régebbi"
        description: "A fogadás friss (server_ts), de a legfrissebb mért ts elavult; valószínű
↪  késői backfill vagy beragadt szenzoridő."
```

A `noDataState: Alerting` az első szabálynál azt jelenti, hogy ha a lekérdezés nem ad vissza adatsort, a riasztás akkor is kivált. A második szabálynál szándékosan `noDataState: OK`: ha az elmúlt 15 percben egyáltalán nincs friss fogadott sor, akkor az első szabály már riaszt, és nem akarunk duplikált értesítést. Az `execErrState: Alerting` mindkét szabálynál biztosítja, hogy egy hibás lekérdezés vagy elérhetetlen adatforrás ne maradjon csendben.

17.4.5.5 5. Betöltés és próba

Mivel a riasztási fájlok a meglévő Dashboardok Grafanával alfejezetbeli provisioning mount alatt vannak, nem kell új Compose mountot hozzáadni. Restartoljuk a Grafanát, majd ellenőrizzük a provisioning naplóját:

```
cd ~/iot-labor/stack
docker compose restart lab-grafana
sleep 5
docker compose logs lab-grafana | grep -i -E 'alerting|provisioning'
```

A Grafana UI `Alerting` → `Alert rules` oldalán a `ts-no-data` és a `ts-stale-sensor` szabály a `TerraSense` mappában jelenik meg. A `Contact points` oldalon a `terrasense-webhook`, a `Notification policies` oldalon pedig a `terrasense-webhook` receiver látszik.

A webhook működését a Grafana `Test` gombjával vagy egy kézi POST-tal is ellenőrizhetjük:

```
curl -s -X POST -H 'Content-Type: application/json' \
  -d '{"status":"test","alerts":[{"labels":{"alertname":"proba","site":"gh1","node":"n001"}}]}'
  ↪  \
  http://localhost:8081/alert
docker logs lab-webhook | tail -20
```

A webhook naplójában a Grafana által küldött JSON jelenik meg. A Grafana webhook payloadja egy `alerts` tömböt tartalmaz; az egyes riasztások `labels`, `annotations`, `status` és időbélyeg mezői ezen a tömbön belül vannak, nem külön top-level `alertname`/`summary` mezőként. A payload tartalmazhat top-level `state` és `title` mezőt is, de a `site`/`node` és a `summary` a riasztás `labels`, illetve `annotations` objektumában található.

Adathiány szimulálásához állítsuk le a `ts-node` szimulátort, vagy szüneteltessük a telemetria küldését. A 15 perces fogadási ablak és a 2 perces `for` miatt a riasztás a normál 10 perces ciklusnál jóval később, körülbelül 17 perc után vált ki; ez tudatos, hogy egyetlen kimaradt csomag ne okozzon riasztást. A késői backfill utáni elavult mérési időt a második szabály jelzi.

Takarítás: a webhook konténerét önmagában ne távolítsuk el, amíg a riasztási szabályok aktívak, mert a Grafana a hiányzó címre próbál értesíteni, és hibát naplóz. Ha végeztünk, előbb mozgassuk félre az alerting YAML-okat, restartoljuk a Grafanát, majd távolítsuk el a webhookot; a dashboard provisioning érintetlen marad:

```
mkdir -p ~/iot-labor/i052/alerting-backup
mv ~/iot-labor/i051/grafana-provisioning/alerting/*.yaml ~/iot-labor/i052/alerting-backup/
cd ~/iot-labor/stack
docker compose restart lab-grafana
docker compose rm -f lab-webhook
```

17.4.6 Ellenőrzés és hibakeresés

Tünet	Lehetséges ok	Javítás
A riasztás nem vált ki, pedig nincs adat	A lekérdezés rossz időablakot, rossz `node_id`-t vagy a nem létező `time` oszlopot használja	Ellenőrizze a query-t a Grafana Explore-ban; a fogadási időhöz `server_ts`, a mérési időhöz `ts` kell
A riasztás azonnal kivált, pedig van adat	A `count(*)` 0-t ad, mert a `valid` szűrő vagy a séma nem egyezik, vagy a `server_ts` helyett rossz oszlopra szűr	Ellenőrizze a `telemetry` oszlopait és a `valid = true` feltételt; nézze meg a Query Inspectort
A szabály `Error` állapotú	A `datasourceUid` nem `terrasense-ds`, a `grafana_ro` nem kapott `SELECT`-et, vagy a `rawSql` szintaktikailag hibás	Ellenőrizze a Dashboardok Grafanával adatforrását, a `\dp` jogosultságokat és a Grafana alerting naplóját
A riasztás `Alerting`, de a webhook nem kap értesítést	Hiányzik vagy rossz a notification policy, a contact point URL nem elérhető, vagy a `lab-webhook` nem fut	Ellenőrizze a `policies.yaml` receivert, a `http://lab-webhook:8081/alert` URL-t, a közös hálózatot és a `docker logs lab-webhook` kimenetét
A riasztás ki-be kapcsol (flapping)	Túl rövid a fogadási ablak vagy a `for`, illetve a 10 perces ciklus jittere nagyobb a türelmi időnél	Növelje az ablakot (10 perces ciklushoz 15 perc), a `for`-t és a `group_interval`-t
A provisioning nem töltődött be	A fájlok nincsenek a meglévő `~/iot-labor/i051/grafana-provisioning/alerting` alatt, vagy a Grafana nem indult újra	Ellenőrizze a bind mountot, a fájlneveket és a `docker compose logs lab-grafana` provisioning sorait

17.4.7 Tipikus hibák és félreértések

1. **Nem létező `time` oszlopra szűrés.** A `telemetry` táblában nincs `time` oszlop; a fogadási idő `server_ts`, a mérési idő `ts`. A dashboard `$__timeFilter` makrója dashboard-kontextusban a `ts`-re illeszkedik, a riasztási szabályban viszont nincs dashboard-időtartomány, ezért explicit `now() - interval` feltételt kell írni.
2. **A contact point önmagában nem értesít.** A contact point csak a címzett; a notification policy receiverének rá kell mutatnia. Contact point létrehozása értesítési útvonal nélkül csendben nem küld semmit.
3. **`host.docker.internal` használata Linuxon.** A Docker Engine alatt ez a név nem garantált. A webhookot saját konténerben, a közös compose-hálózaton kell futtatni, és a Grafana a `lab-webhook:8081` címet használja. A 8080-as portot a backend foglalja, ezért a webhook 8081-en figyel.
4. **A fogadási idő és a mérési idő összemosása.** Ha a riasztás csak a sorok számát nézi, egy késői backfill friss adatnak látszik, pedig a mért érték régi. Ezért külön szabály kell a `server_ts`-re (van-e friss fogadás) és a `ts`-re (friss-e a mért érték).
5. **A `noDataState` és az `execErrState` összekeverése.** A `noDataState` akkor érvényes, ha a lekérdezés nem ad adatsort; a lekérdezés hibáját az `execErrState` kezeli. A `count(*)` mindig ad egy sort, ezért a nulla darabszámot redukcióval és küszöbbel kell kezelni, nem `noData`-ként.

6. **Túl szigorú ablak a 10 perces ciklushoz.** Ha a fogadási ablak rövidebb, mint a ciklus plusz a hálózati késés, a normál jitter is riasztást okoz. A 15 perces ablak (10 perc ciklus + 5 perc türelmi idő) és a 2 perces `for` ezt kiszűri.

17.4.8 Feladatok

17.4.8.1 a) Fogalmi kérdés

Definiálja a három riasztási osztályt (folyamat, adathiány, eszközhiba), és adjon egy-egy konkrét TerraSense-példát mindegyikre. Magyarázza el, miért fontos megkülönböztetni őket az értesítési stratégia szempontjából.

17.4.8.2 b) Tervezési feladat

Egy fűtési rendszerben a hőmérséklet-riasztás bekapcsolási küszöbe 40 °C, kikapcsolási küszöbe 38 °C, a szükséges időtartam 5 perc. A hőmérséklet percenkénti értékei: 39, 41, 41, 40,5, 39,5, 38,5. Mikor és hányadik percben vált ki a riasztás? Lesz-e visszaállás a megadott idősor alatt? Indokolja a választ a hiszterézis és az időtartam figyelembevételével.

17.4.8.3 c) Implementációs feladat

Módosítsa a laborban létrehozott `ts-no-data` riasztási szabályt úgy, hogy 30 perces friss fogadási ablakot figyeljen, a `for` értéke 5 perc legyen, és a `summary` annotációban jelenjen meg a `site` és a `node` értéke. Adja meg a módosított YAML releváns részét, és magyarázza el, miért marad használható a `server_ts` alapú szemlélet a késői backfill esetén is.

17.4.8.4 d) Hibakeresési feladat

A Grafana riasztás nem vált ki, pedig a webhook szerver korábban működött, és a dashboardon látható, hogy nincs adat a `telemetry` táblában. Soroljon fel legalább három lehetséges okot, és mindegyikhez adjon javítási lépést.

17.4.9 Megoldások

17.4.9.1 a) Fogalmi kérdés

- **Folyamatriasztás:** a mért fizikai érték nem felel meg a követelménynek. Példa: a hőmérséklet meghaladja a 35 °C-ot a `gh1` üvegházban, ami a növények károsodásához vezethet. Az értesítés az agronómusnak szól, mert beavatkozásra van szükség.
- **Adathiány-riasztás:** az eszköz nem küld adatot a várt időn belül. Példa: a `n001` node nem publikál a `terrasense/gh1/n001/telemetry` topicra 15 perce. Ez lehet akkumulátorlemerülés, rádiós interferencia vagy gateway-probléma. Az értesítés a rendszergazdának szól.
- **Eszközhiba-riasztás:** az eszköz explicit hibát jelez. Példa: a `ts-node` belső diagnosztikája magas hőmérsékletet észlel a saját mikrokontrollerén, és ezt a `local_alert` mezőben beállítja a state topicban. Az értesítés a hardver-karbantartónak szól.

A megkülönböztetés azért fontos, mert a probléma jellege eltérő szakértelmet és válaszidőt igényel. Az adathiány-riasztás gyakran sürgősebb, mert a rendszer vakon repül, míg a folyamatriasztásnál az operátornak lehetősége van manuálisan ellenőrizni.

17.4.9.2 b) Tervezési feladat

A feltétel: hőmérséklet > 40 °C (bekapcsolás), hőmérséklet < 38 °C (kikapcsolás), időtartam 5 perc folyamatosan.

Az idősor:

Perc	Hőmérséklet	Állapot
1	39	Normál (feltétel hamis)
2	41	Figyelés indul (feltétel igaz, t=1)
3	41	Figyelés (t=2)
4	40,5	Figyelés (t=3)
5	39,5	A feltétel megszakad (39,5 $<$ 40), a számláló nullázódik
6	38,5	Normál (feltétel hamis)

Az 5. percben a hőmérséklet 39,5 °C, ami a 40 °C-os bekapcsolási küszöb alatt van, tehát a feltétel nem teljesül, és az időtartam számlálója nullázódik. Ezért a riasztás **nem vált ki**. A kikapcsolási küszöb (38 °C) soha nem lép érvénybe, mert a riasztás be sem kapcsolt. A válasz: nincs riasztás a megadott idősor alatt.

17.4.9.3 c) Implementációs feladat

A 30 perces ablakhoz a `relativeTimeRange.from` értéke 1800 másodperc, a `rawSql` pedig `interval '30 minutes'` feltételt használ. A `server_ts` továbbra is a fogadási idő, ezért a szabály a 30 percen belül érkezett érvényes mintákat számolja; a késői backfill fogadási szempontból friss, ezért erre a szabályra nem vált ki, hanem a külön `ts-stale-sensor` szabály jelzi a régi mérési időt. A `labels` alatt a `site` és a `node` explicit szerepel, így a `summary` sablon mindig feloldható.

```yaml
- uid: ts-no-data-30
  title: "Nincs friss telemetria 30 perce"
  condition: C
  data:
    - refId: A
      relativeTimeRange:
        from: 1800
        to: 0
      datasourceUid: terrasense-ds
      model:
        refId: A
        editorMode: code
        format: table
        rawQuery: true
        rawSql: |
          SELECT count(*) AS sample_count
          FROM telemetry
          WHERE server_ts > now() - interval '30 minutes'
            AND site = 'gh1'
            AND node_id = 'n001'
            AND valid = true
```

```
      - refId: B
        datasourceUid: __expr__
        model:
          refId: B
          type: reduce
          expression: A
          reducer: last
          settings:
            mode: ""
      - refId: C
        datasourceUid: __expr__
        model:
          refId: C
          type: threshold
          expression: B
          conditions:
            - evaluator:
                type: lt
                params: [1]
    noDataState: Alerting
    execErrState: Alerting
    for: 5m
    labels:
      severity: critical
      site: gh1
      node: n001
    annotations:
      summary: "Nincs friss telemetria a {{ $labels.site }}/{{ $labels.node }} node-tól 30
↪  perce"
      description: "A 30 perces fogadási ablakban nem érkezett érvényes minta."
```

A `site` és a `node` a statikus címkékből jelenik meg, nem a lekérdezés eredményéből; ez azért biztos, mert a szabály egyetlen ismert node-ra vonatkozik, és így a templát akkor is feloldható, ha a lekérdezés éppen nulla sort ad. Több node-os szabálynál a címkéket a lekérdezésnek kellene visszaadnia, és a szabályt node-onként vagy több adatsorral kellene megépíteni.

17.4.9.4 d) Hibakeresési feladat

Lehetséges okok:

1. **A query nem létező oszlopra szűr.** A `telemetry` táblában nincs `time` oszlop; a fogadási idő `server_ts`, a mérési idő `ts`. A Grafana Explore-ban futtassa le a `SELECT count(*) FROM telemetry WHERE server_ts > now() - interval '15 minutes' AND site = 'gh1' AND node_id = 'n001' AND valid = true` lekérdezést; ha ez hibát ad, javítsa az oszlopnevet. A dashboard `$__timeFilter` makrója nem használható közvetlenül riasztási szabályban, mert ott nincs dashboard-időtartomány.
2. **A `datasourceUid` nem egyezik a tényleges TimescaleDB datasource UID-jével.** A Dashboardok Grafanával alfejezetben létrehozott adatforrás UID-je `terrasense-ds`; ha a YAML-ban más szerepel, a szabály `Error` állapotú lesz. Ellenőrizze a Grafana adatforrások listájában az UID-t, és frissítse a YAML-t.
3. **A `grafana_ro` felhasználónak nincs joga a `telemetry` táblához.** Ilyenkor a lekérdezés `permission denied` hibát ad, és az `execErrState: Alerting` miatt a szabály hibás állapotba kerül. Futtassa újra a Dashboardok Grafanával `02_grafana_ro.sql` szkriptjét, és ellenőrizze a `\dp telemetry*` kimenetét.
4. **A notification policy nem a `terrasense-webhook` receivert használja, vagy hiányzik.** A szabály `Alerting` állapotba kerül, de a webhook nem kap értesítést. Ellenőrizze a `policies.yaml` `receiver` mezőjét és a Grafana `Notification policies` oldalát.

5. **A `lab-webhook` nem fut, vagy a Grafana nem éri el.** Ellenőrizze a `docker ps`-t, a `docker logs lab-webhook` kimenetét, a `http://lab-webhook:8081/alert` URL-t és azt, hogy mindkét konténer az alapértelmezett compose-hálózaton van. A 8080-as portot a backend foglalja, ezért a webhook 8081-et használ.
6. **A provisioning nem töltődött be.** Ha a riasztási YAML-ok nincsenek a meglévő `~/iot-labor/i051/grafana-provisioning/alerting` könyvtárban, vagy a Grafana nem indult újra, a szabály nem jelenik meg. Ellenőrizze a mountot és a `docker compose logs lab-grafana` provisioning sorait; új Compose mount hozzáadása esetén `docker compose up -d --force-recreate lab-grafana` kell.

17.4.10 Rövid összefoglaló

A riasztási szabályok feltételből, időtartamból és súlyosságból állnak. A három fő típus – folyamat-, adathiány- és eszközhiba-riasztás – eltérő válaszreakciót igényel. A hiszterézis és az időtartam megakadályozza a riasztási lebegést és a hamis pozitívokat. A TerraSense-ben az edge riasztás a `local_alert` mezőben jelenik meg a `state` topicban, a backend pedig a Grafana alerting segítségével végzi a központi kiértékelést és értesítést. A labor a `lab-grafana` szolgáltatást és a `~/iot-labor/i051/grafana-provisioning` meglévő mountját használja; a `~/iot-labor/i051/grafana-provisioning/alerting/contactpoints.yaml`, `policies.yaml` és `terrasense.yaml` fájlok definiálják a `terrasense-webhook` contact pointot, az értesítési útvonalat, valamint a `ts-no-data` és a `ts-stale-sensor` szabályt. A webhook a `lab-webhook` konténerben, a 8081-es porton fut, és a Grafana a `server_ts` (friss fogadás) és a `ts` (mérési idő) szétválasztásával különbözteti meg az élő adathiányt a késői backfilltől. **Továbbvitt eredmény.** A riasztási állapotok (OK, Alerting, Acknowledged, Resolved) és a súlyossági címke (`severity`) konvencióját használja a Megfigyelhetőség végponttól végpontig alfejezet a metrika- és naplókorellációhoz, valamint a Záróprojekt átvételi tesztjei során a riasztási lánc validálásához.

17.4.11 Források és ellenőrzési hiányok

- A Grafana alerting file provisioning YAML-formátuma, a `contactPoints`, a `policies` és a `groups` séma a Grafana 11.2.0 verzióra épül; a pontos mezőnevek és a `__expr__` redukciós/threshold adatforrás viselkedése verziófüggő, használat előtt a telepített verzió dokumentációjával ellenőrizendő.
- A Grafana webhook payloadja `alerts` tömböt, valamint az egyes riasztásokon belül `labels`, `annotations` és `status` mezőket tartalmaz; a fejezet nem top-level `alertname`/`summary` mezőket feltételez. A webhook-fogadó a teljes JSON-t naplózza, ezért a payload szerkezete a naplóban ellenőrizhető.
- A `lab-webhook` a `python:3.12.3-slim` képre és a Python standard `http.server` moduljára épül; a webhook-fogadó `py_compile` és helyi HTTP POST ellenőrzése megtörtént, a teljes Grafana–webhook láncot a szerző nem futtatta, mert a labor Docker/Grafana környezetet igényel. A riasztási szabályok és a provisioning YAML statikusan, a Grafana 11.2.0 dokumentációja alapján készültek.
- A `grafana_ro` / `grafana-lab-only` adatbázis-hitelesítő és a `lab-admin` / `lab-pass` Grafana-admin kizárólag a helyi laborrendszerhez tartozik; éles környezetben használni tilos.

VI. RÉSZ – Biztonság, eszközmenedzsment és üzemeltetés

18 IoT-biztonság alapjai

18.1 Fenyegetésmodellezés: támadási felület és STRIDE

18.1.1 Motiváció

A TerraSense rendszer üvegházakat monitoroz: a `ts-node` szenzorcsomópontok BLE vagy UDP csatornán küldenek telemetriát a `ts-gw` gatewaynek, amely MQTT-n továbbítja a brokerhez, onnan a `ts-backend` feldolgozza és tárolja, a Grafana pedig dashboardon megjeleníti. Egy kompromittált szenzor nemcsak hamis adatokat küldhet, hanem a gatewayen keresztül bejuthat a felhős hálózatba, vagy hamis parancsot juttathat el az öntözőszelephez. A biztonsági követelményeket az Nem funkcionális követelmények alfejezet rögzítette, de a követelményekből nem derül ki automatikusan, hogy pontosan mit és milyen sorrendben kell védeni. A A gateway szerepe alfejezetben meghatároztuk, hogy a `ts-gw` biztonsági határ, az Ingest: MQTT-ből a backendbe, idempotensen alfejezetben pedig láttuk a backend belépési pontját. Most ezeket a komponenseket egyetlen biztonsági modellben egyesítjük, hogy a fenyegetéseket szisztematikusan azonosítsuk, és kockázat alapján rangsoroljuk az ellenintézkedéseket.

18.1.2 Fogalmak és működés

Definíció. A **támadási felület** (attack surface) a rendszer azon pontjainak összessége, ahol egy támadó adatot vihet be, adatot olvashat ki, vagy a rendszer működését befolyásolhatja. Ide tartoznak a hálózati interfészek, a fizikai hozzáférést biztosító csatlakozók (pl. debug port), a szoftveres API-k, a rádiós csatornák és az emberi folyamatok is.

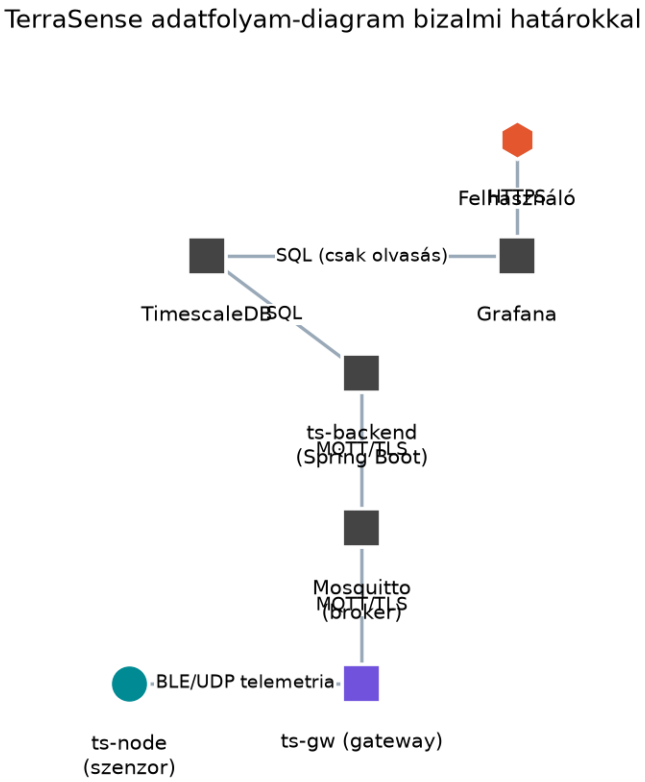
Definíció. A **bizalmi határ** (trust boundary) olyan vonal a rendszerben, amely két, eltérő megbízhatósági szintű zónát választ el. Az adatfolyamok keresztezhetik a határt, de ilyenkor a küldőt és a fogadót nem tekintjük egyformán megbízhatónak: a határ átlépésekor érvényesíteni kell az azonosítást, a sértetlenséget és a jogosultságot.

Definíció. A **STRIDE** a Microsoft által bevezetett fenyegetés-osztályozási modell, amely hat kategóriát különböztet meg:

- **S**poofing (megszemélyesítés): a támadó másik eszköznek vagy felhasználónak adja ki magát.
- **T**ampering (illetéktelen módosítás): adatok vagy firmware módosítása engedély nélkül.
- **R**epudiation (letagadhatóság): a támadó letagadhatja a műveletet, mert nincs megfelelő naplózás.
- **I**nformation Disclosure (információszivárgás): titkos adatok illetéktelen megismerése.
- **D**enial of Service (szolgáltatásmegtagadás): a szolgáltatás elérhetetlenné tétele.
- **E**levation of Privilege (jogosultság-kiterjesztés): a támadó magasabb jogosultságot szerez.

A módszer lépései: (1) adatfolyam-diagram készítése a rendszer komponenseivel, adattárolóival, adatfolyamaival és bizalmi határaival; (2) minden adatfolyamra, tárolóra és komponensre STRIDE-kérdések feltevése; (3) a fenyegetések kockázatának értékelése; (4) ellenintézkedések hozzárendelése.

A TerraSense adatfolyam-diagramja az alábbi elemekből áll:



18.1. ábra: TerraSense adatfolyam-diagram bizalmi határokkal - A bizalmi határok: eszköz-edge, edge-felhő, felhő belső, fizikai hozzáférés. A ts-node fizikailag az üvegházban található, a ts-gw az edge zónában, a broker, a backend, az adatbázis és a Grafana a felhőben vagy védett szerverteremben.

A diagramon látható, hogy a **ts-node** fizikailag az üvegházban van, így a támadó közvetlenül hozzáférhet. A **ts-gw** az edge zónában található, szintén fizikailag elérhető lehet, de általában zártabb környezetben. A **Mosquitto**, a **ts-backend**, a **TimescaleDB** és a **Grafana** a felhőben vagy a helyi szerverteremben futnak, és erősebb fizikai védelemmel rendelkeznek. A bizalmi határok tehát:

1. **Eszköz gateway (rádiós csatorna):** a BLE vagy UDP csatorna nem megbízható, a támadó lehallgathatja vagy hamis csomagokat küldhet.
2. **Gateway broker (MQTT/TLS):** a TLS-t megfelelően kell konfigurálni, de a gateway szoftverének kompromittálása esetén a TLS alagút nem véd.
3. **Broker backend (MQTT/TLS):** ugyanez a helyzet.
4. **Backend adatbázis:** a belső hálózat sem tekinthető automatikusan biztonságosnak.
5. **Grafana adatbázis (közvetlen, csak olvasás):** a dashboard a A TerraSense mintarendszer architektúrája és a Dashboardok Grafanával szerint nem a backend REST API-ján, hanem közvetlenül a TimescaleDB-n keresztül olvas. Ehhez külön adatbázis-fiókot és a szükséges nézetekre szűkített, csak olvasási jogot kell adni; a Grafana saját hitelesítése és a backend REST API-ját használó kliensek (mobil app, adminfelület) külön bizalmi utat jelentenek.
6. **Fizikai hozzáférés az eszközhöz:** a debug portok, a flash memória és a szenzorok fizikai manipulálása.

18.1.3 Döntési szempontok

A fenyegetésmodellezés mélységét a rendszer kritikussága és a fejlesztési fázis határozza meg. A TerraSense esetében a modellt a tervezés során készítjük el, és minden jelentős változásnál frissítjük.

Szempont	Lehetőség	Előny	Hátrány	Mikor válasszuk
Modellezési módszer	STRIDE	Szisztematikus, jól dokumentált, kategóriákra bont	Nem ad konkrét támadási forgatókönyveket	Kezdeti tervezéshez, fenyegetésleltárhoz
	Attack tree	Támadási célokból indul ki, mélységi elemzés	Időigényes, könnyen túlbonyolítható	Kritikus alrendszerek mély elemzéséhez
Kockázatértékelés	Kvalitatív (alacsony/közepes/magas)	Gyors, egyszerű	Pontatlan	Korai fázisban, gyors priorizáláshoz
	Félig számszerű (CVSS-súlyosság, DREAD-pontszám)	Számszerű, összehasonlítható	A CVSS a sérülékenység súlyosságát méri, nem a valószínűség × üzleti hatás szorzatát; a DREAD szubjektív	Érett rendszereknél, riportokhoz, a feltételezések dokumentálásával
Hatókör	Csak az eszköz	Kevesebb erőforrás	A rendszerszintű kockázatok kimaradnak	Egyetlen komponens fejlesztésekor
	Teljes adatút	Átfogó kép	Sok szereplő, bonyolult	Integrációs tervezéskor, biztonsági audit előtt

A TerraSense-nél a teljes adatútra kvalitatív STRIDE-modellt készítünk, mert a rendszer még fejlesztés alatt áll, és a cél az ellenintézkedések priorizálása.

A **CVSS** és a **DREAD** nem azonos a kockázatértékeléssel. A CVSS egy sérülékenység súlyosságát pontozza (az alap-pontszám a kihasználhatóságot és a hatást írja le, a temporális és környezeti módosítók ezt finomítják), de nem mondja meg, hogy a sérülékenységet milyen valószínűséggel használják ki az adott rendszerben, és nem fejez ki üzleti kárt. A DREAD (Damage, Reproducibility, Exploitability, Affected users, Discoverability) egy szubjektív, 1–10 közötti pontozási heurisztika, amelynek értékei nem mért gyakoriságok. A kockázatot továbbra is a valószínűség és a hatás szorzataként értelmezzük; ha CVSS- vagy DREAD-pontszámot használunk bemenetként, akkor rögzíteni kell a mögötte álló feltételezéseket, és a pontszámot nem szabad közvetlenül a kvalitatív kategóriákkal összemérni. A kvalitatív címke (alacsony, közepes, magas) tehát nem mért gyakoriságot és nem pénzügyi kárt jelent, hanem a dokumentált becslés eredménye.

18.1.4 Kidolgozott példa: TerraSense STRIDE-modell

Munkakönyvtár: `~/iot-labor/i053`

```
mkdir -p ~/iot-labor/i053 && cd ~/iot-labor/i053
```

A modellt egy Markdown-fájlban rögzítjük, amelyet a későbbi biztonsági alfejezetek ellenintézkedés-térképként használnak. A fájl tartalma az alábbi.

```
# TerraSense fenyegetésmodell (i053)

## Adatfolyam-diagram

A komponensek: ts-node (szenzor), ts-gw (gateway), Mosquitto (broker), ts-backend (Spring Boot),
TimescaleDB, Grafana. A bizalmi határok: eszköz-edge, edge-felhő, felhő belső, fizikai
↪  hozzáférés.

## STRIDE-táblázat

A Grafana közvetlenül, csak olvasási joggal éri el a TimescaleDB-t; a backend REST API-ját
a mobil app és az adminfelület használja. A kétféle kliens eltérő hitelesítési és
jogosultsági modellt igényel, ezért a táblázatban külön szerepelnek.

| Adatfolyam/elem | STRIDE | Fenyegetés | Kockázat | Ellenintézkedés |
|---|---|---|---|---|
| BLE/UDP telemetria (ts-node → ts-gw) | S | Hamis szenzor bejelentkezik ts-node-ként | Magas |
↪  Eszközazonosítás (i056), üzenet-hitelesítés (i055) |
| | T | A telemetria adatot módosítják (UDP-n vagy BLE GATT-értesítésben) | Közepes | UDP: DTLS
↪  1.3 (i057); BLE: LE Secure Connections párosítás és link-layer titkosítás, valamint
↪  alkalmazásszintű üzenet-hitelesítés (i055); séma-ellenőrzés (i032) |
| | I | Az adat lehallgatható | Közepes | UDP: DTLS 1.3 (i057); BLE: link-layer titkosítás
↪  párosítás után, az érzékeny mezők alkalmazásszintű védelme (i055) |
| | D | Rádiós zavarás, csomagelárasztás | Alacsony (valószínűség: alacsony, hatás: közepes) |
↪  BLE adaptív frekvenciaugrás, forgalomkorlátozás; kiesés alatt fail-safe szelepállapot (i015)
↪  és store-and-forward pufferelés (i037); maradványkockázat: széles sávú zavarás ellen nincs
↪  teljes védelem |
| ts-node firmware/flash | T | Illetéktelen firmware-módosítás | Magas | Secure boot (i058),
↪  aláírt frissítés (i060) |
| | E | Debug porton root jog | Magas | Debug letiltása, hozzáférés-védelem (i056) |
| | I | Flash kiolvasása (kulcsok) | Magas | Titkos kulcs védett tárolóban (i056) |
| MQTT (ts-gw → broker) | S | Hamis gateway csatlakozik | Magas | mTLS, tanúsítvány-alapú
↪  hitelesítés (i057) |
| | T | Üzenet módosítása | Közepes | TLS integritásvédelem (i057) |
| | I | Topic-tartalom lehallgatása | Közepes | TLS titkosítás (i057) |
| | D | Broker elárasztása | Közepes | Rate limiting, QoS 0/1 szabályok |
| MQTT (broker → backend) | S | Hamis backend feliratkozik | Magas | mTLS, ACL a brokeren (i057)
↪  |
| | T | Üzenet módosítása | Közepes | TLS integritásvédelem (i057) |
| | I | Topic-tartalom lehallgatása | Közepes | TLS titkosítás (i057) |
| SQL (Grafana → TimescaleDB, csak olvasás) | S | Hamis dashboard-ügyfél a Grafana nevében |
↪  Magas | Grafana-hitelesítés, hálózati hozzáférés-korlátozás, külön adatbázis-fiók |
| | I | A dashboard túl széles adatbázis-jogosultsággal adatot szivárogtat | Magas | Csak
↪  olvasó, a szükséges nézetekre szűkített adatbázis-szerepkör |
| | E | Dashboard-szerepkörből adminisztrátori jog | Közepes | Szerepkörök elkülönítése, audit
↪  napló (i062) |
| REST API (mobil app → backend) | S | Hamis felhasználó | Magas | OAuth2/OIDC, RBAC |
| REST API (adminfelület → backend) | S | Hamis adminisztrátor | Magas | OAuth2/OIDC,
↪  többfaktoros hitelesítés, RBAC |
| | E | Jogosultság-kiterjesztés | Közepes | Szerepkörök, audit napló (i062) |
| Adatbázis (backend → TimescaleDB) | I | SQL injection, adatszivárgás | Magas | Paraméterezett
↪  lekérdezések, hálózati szegmentáció |
| | T | Adatbázis-tartalom módosítása | Közepes | Hozzáférés-vezérlés, audit |
```

```
| Parancsok lefelé (backend → broker → ts-gw → ts-node) | S | Hamis parancs | Magas |
↪  Parancs-aláírás, eszközazonosítás (i055, i056) |
| | T | Parancs módosítása | Magas | TLS/mTLS (i057), séma-ellenőrzés |
| | D | Parancs-újraküldés, elárasztás | Közepes | Idempotencia, időkorlát (i049) |

**A helyi szakasz védelme.** A BLE GATT és az UDP nem TLS-felett fut, ezért a
[biztonságos kapcsolat](#i057) alfejezet mTLS-laborja önmagában nem védi a
ts-node-ts-gw szakaszt: az csak a ts-gw-broker és a broker-backend MQTT-láncot
biztosítja. UDP-n a DTLS 1.3 adhat lehallgatás és módosítás elleni védelmet;
valós BLE-n a LE Secure Connections párosítás és az ezt követő link-layer
titkosítás véd, de a GATT-értesítés tartalmát emellett alkalmazásszinten is
hitelesíteni kell (üzenet-MAC vagy aláírás, i055), mert a link-layer titkosítás
csak a két végpont közötti szakaszt védi, és a gateway kompromittálása esetén
nem nyújt védelmet. Amíg ez a helyi védelem nincs megvalósítva, a hamis helyi
telemetria és a hamis parancs elfogadása **nyitott kockázat**; a gateway-broker
mTLS működése ezt nem zárja ki.

## Kockázatmátrix

A kockázat = valószínűség × hatás, kvalitatív skálán.

| Valószínűség \ Hatás | Alacsony | Közepes | Magas |
|---|---|---|---|
| Alacsony | Alacsony | Alacsony | Közepes |
| Közepes | Alacsony | Közepes | Magas |
| Magas | Közepes | Magas | Kritikus |

A fenti táblázatban a kockázatokat ehhez a mátrixhoz igazítottuk.

**A rádiós zavarás értékelése.** A D sorban a valószínűség azért alacsony, mert
az üvegház nem célzott támadási cél, a környékbeli interferenciaforrások száma
korlátozott, és a zavaró eszköznek a helyszín közelében kell lennie. A hatás
közepes: a telemetria átmenetileg kiesik, de a ts-gw store-and-forward puffere
(i037) a kapcsolat helyreállása után visszatölti az adatokat, a szelep pedig
NC szelepként áramtalanítva, illetve a szoftveres fail-safe szerint zárt
alapállapotba kerül (i015). Az alacsony valószínűség és a közepes hatás a
mátrix szerint alacsony kockázatot ad. A BLE adaptív frekvenciaugrása a szűk
sávú zavarást kerüli el; az UDP-s szimulációban és széles sávú zavarás esetén
nincs ilyen védelem, ezért a zavarás **maradványkockázat**, amelyet a
rendszer üzemeltetési szinten vállal. Az értékelést a helyszín rádiós
felmérésével kell felülvizsgálni.

## Ellenintézkedés-térkép

Az alábbi táblázat a fenyegetéseket a későbbi alfejezetekben tárgyalt ellenintézkedésekhez
↪  rendeli.

| Fenyegetés | Ellenintézkedés | Alfejezet |
|---|---|---|
| Hamis eszköz vagy gateway | X.509 tanúsítvány, mTLS | [Eszközidentitás: X.509-tanúsítványok,
↪  PKI és kulcstárolás](#i056), [Biztonságos kapcsolat: TLS 1.3, mTLS és DTLS](#i057) |
| Firmware-módosítás | Secure boot, aláírt firmware | [Biztonságos boot és
↪  firmware-aláírás](#i058) |
| Kulcsok kiszivárgása | Kulcstárolás, hardveres védelem | [Eszközidentitás:
↪  X.509-tanúsítványok, PKI és kulcstárolás](#i056) |
| Parancs-hamisítás | Aláírás, hitelesítés | [Kriptográfiai alapok](#i055),
↪  [Eszközidentitás](#i056) |
| Adatlehallgatás | TLS 1.3 / DTLS 1.3 | [Biztonságos kapcsolat](#i057) |
| DoS a brokeren | Rate limiting, QoS-szabályok | [MQTT QoS, munkamenetek, retain, Last
↪  Will](#i027) (ellenőrizendő) |
| Jogosultság-kiterjesztés | RBAC, audit | [Megfigyelhetőség](#i062) |
| SQL injection | Paraméterezett lekérdezések | [Idősoros adatbázisok](#i045) |
```

Ez a fájl a **továbbvitt eredmény**. A későbbi alfejezetek ebből a térképből indulnak ki, amikor az egyes ellenintézkedéseket implementálják. A Tipikus IoT-sebezhetőségek és a szabályozás alfejezet a sebezhetőségek valós előfordulását és a jogi keretet tárgyalja, a Kriptográfiai alapok a kriptográfiai primitíveket, az Eszközidentitás a tanúsítványokat és a kulcstárolást, a Biztonságos boot és firmware-aláírás pedig a firmware-integritást.

A fenti STRIDE-táblázatban minden adatfolyamra és tárolóra megvizsgáltuk a hat kategóriát. Nem minden kombináció releváns, de a fontos fenyegetéseket rögzítettük. A kockázatértékelés kvalitatív: a valószínűséget és a hatást az iparági tapasztalatok és a TerraSense konkrét üzemeltetési környezete alapján becsültük. Az értékeket a rendszer változásakor felül kell vizsgálni.

18.1.5 Ellenőrzés és hibakeresés

A fenyegetésmodell akkor jó, ha:

- **Lefedettség:** minden adatfolyam, adattároló és külső entitás szerepel a STRIDE-táblázatban.
- **Nyomon követhetőség:** minden fenyegetéshez tartozik legalább egy ellenintézkedés, és minden ellenintézkedés visszavezethető egy fenyegetésre.
- **Prioritás:** a kockázati besorolás lehetővé teszi, hogy a magas kockázatú tételekkel kezdjük a munkát.
- **Frissesség:** a modell tükrözi az aktuális architektúrát.

Tipikus hibatünetek és javításuk:

Tünet	Ok	Javítás
A STRIDE-táblázatban sok üres cella van	Nem minden adatfolyamot vizsgáltunk meg	Menjünk végig minden adatfolyamon és tárolón, és tegyük fel a STRIDE-kérdéseket
A kockázati besorolás mindenütt „magas”	Nem választottuk szét a valószínűséget és a hatást	Használjunk külön skálát, és indokoljuk a becslést
Az ellenintézkedések nem kapcsolódnak konkrét technológiához	Csak általános javaslatokat írtunk	Hivatkozzunk a későbbi alfejezetekre, és adjunk konkrét megoldást
A modell nem tartalmazza a fizikai hozzáférést	Csak a hálózati rétegre koncentráltunk	Vegyük fel a debug portokat, a flash memóriát és a szenzorok manipulálását

A modell ellenőrzéséhez a TerraSense esetében egy egyszerű áttekintést végezhetünk: nyissuk meg a `terrasense-fenyegetesmodell.md` fájlt, és ellenőrizzük, hogy minden adatfolyam szerepel-e a STRIDE-táblázat első oszlopában. A diagramon szereplő adatfolyamok és adattárolók mindegyike megjelenik a táblázatban, és a Grafana közvetlen, csak olvasási adatbázis-hozzáférése külön sorban szerepel, tehát a lefedettség teljes.

18.1.6 Tipikus hibák és félreértések

1. **Csak a szoftveres támadási felületet nézzük.** Az IoT-eszközök fizikailag a támadó kezében lehetnek. A debug port, a flash kiolvasása, a szenzorok hamis jelekkel való befolyásolása ugyanolyan fontos, mint a hálózati támadások. A TerraSense ts-node az

üvegházban bárki által hozzáférhető, ezért a fizikai támadásokat a modellben explicit módon szerepeltetni kell.

2. **A bizalmi határt a hálózati határral azonosítjuk.** A gateway és a broker közötti TLS kapcsolat nem jelenti azt, hogy a gateway teljesen megbízható. Ha a gatewayt kompromittálják, a TLS alagút belsejében a támadó szabadon mozoghat. A bizalmi határ a komponensek közötti bizalmi szinteket jelöli, nem a titkosítás meglétét.
3. **Minden fenyegetést magas kockázatúnak minősítünk.** Ha minden magas, akkor nem tudunk priorizálni. A kockázatértékelés lényege, hogy a valószínűséget és a hatást külön becsüljük, és a magas kockázatú tételekre összpontosítsunk. A TerraSense esetében például a rádiós zavarás valószínűsége alacsony (nem célzott támadás, kevés helyi interferenciaforrás), a hatása közepes (átmeneti adatkiesés, amelyet a fail-safe szelepállapot és a store-and-forward puffer tompít), így a kockázat alacsony; a két értéket és az indokot a modell dokumentálja.
4. **A fenyegetésmodellt egyszer elkészítjük, és soha nem frissítjük.** Az architektúra változásával új adatfolyamok, új komponensek jelennek meg. A modellt minden jelentős változásnál felül kell vizsgálni, különben a későbbi biztonsági döntések téves alapokon nyugszanak.

18.1.7 Feladatok

18.1.7.1 1. Fogalmi kérdés

Magyarázza el, miért tekintjük bizalmi határnak a ts-gw és a Mosquitto broker közötti MQTT kapcsolatot, annak ellenére, hogy TLS-t használunk.

18.1.7.2 2. Tervezési feladat

Egészítse ki a STRIDE-táblázatot a parancsok lefelé irányuló adatfolyamára vonatkozóan: azonosítsa a Spoofing, Tampering, Repudiation és Denial of Service fenyegetéseket, és rendeljen hozzájuk ellenintézkedést a későbbi alfejezetekre hivatkozva.

18.1.7.3 3. Implementációs feladat

A `~/iot-labor/i053/terrasense-fenyegetesmodell.md` fájlban vegyen fel egy új fenyegetést: a ts-node szenzor hamis mérési adatokat küld (például egy támadó rádiós csatornán keresztül hamis üzenetet injektál). Határozza meg a STRIDE-kategóriát, becsülje meg a kockázatot, és rendeljen hozzá ellenintézkedést a Kriptográfiai alapok vagy az Eszközidentitás alfejezetre hivatkozva.

18.1.7.4 4. Hibakeresési feladat

Egy fejlesztő a következő STRIDE-táblázatot készítette a TerraSense-hez, de kihagyott egy fontos elemet. Azonosítsa a hiányzó adatfolyamot vagy tárolót, és indokolja, miért kritikus a biztonság szempontjából.

A fejlesztő táblázata:

Adatfolyam/elem	STRIDE	Fenyegetés
BLE/UDP telemetria	S, T, I, D	Hamis eszköz, módosítás, lehallgatás, zavarás
MQTT (ts-gw → broker)	S, T, I, D	Hamis gateway, módosítás, lehallgatás, elárasztás
MQTT (broker → backend)	S, T, I, D	Hamis backend, módosítás, lehallgatás, elárasztás
REST API	S, E	Hamis felhasználó, jogosultság-kiterjesztés

18.1.8 Megoldások

18.1.8.1 1. Fogalmi kérdés

A TLS biztosítja a kapcsolat titkosságát, integritását és a szerver hitelesítését, de nem szünteti meg a bizalmi határt. A bizalmi határ azt jelöli, hogy a kapcsolat két oldalán álló komponensek eltérő megbízhatósági szintűek. A ts-gw az edge zónában fut, és fizikailag hozzáférhető lehet, míg a Mosquitto a felhőben vagy védett szerverteremben van. Ha a ts-gw kompromittálódik, a támadó a TLS alagúton belül érvényes üzeneteket küldhet, mert a gateway rendelkezik a hitelesítő adatokkal. Ezért a bizalmi határ továbbra is fennáll: a brokernek nem szabad megbíznia a gatewayben, hanem minden üzenetet a saját szabályai szerint kell érvényesíteni, például topic-ACL-ekkel és tartalom-ellenőrzéssel. A TLS csak a szállítási réteg biztonságát adja, de a komponens kompromittálódása esetén nem véd.

18.1.8.2 2. Tervezési feladat

A parancsok lefelé (backend → broker → ts-gw → ts-node) adatfolyamra:

STRIDE	Fenyegetés	Ellenintézkedés	Alfejezet
Spoofing	Hamis backend parancsot ad ki	Parancs-aláírás, backend hitelesítés	Kriptográfiai alapok, Eszközidentitás
Tampering	A parancs tartalmát módosítják	TLS/mTLS, séma-ellenőrzés	Biztonságos kapcsolat
Repudiation	A parancs kiadója letagadja a műveletet	Audit napló, aláírt parancsok	Megfigyelhetőség
Denial of Service	Parancs-újraküldés, elárasztás	Idempotencia, időkorlát, rate limiting	Parancsok és konfiguráció lefelé

18.1.8.3 3. Implementációs feladat

A fájlban a STRIDE-táblázat BLE/UDP telemetria sorához hozzáadjuk a Spoofing kategóriához a hamis mérési adatok injektálását:

```
| BLE/UDP telemetria (ts-node → ts-gw) | S | Hamis szenzor bejelentkezik ts-node-ként, vagy
↪  hamis mérési adatokat injektál | Magas | Eszközazonosítás (i056), üzenet-hitelesítés (i055)
↪  |
```

Indoklás: a támadó a rádiós csatornán hamis üzenetet küldhet, amelyet a gateway nem tud megkülönböztetni a valóditól. Az ellenintézkedés az üzenet-hitelesítés (például MAC vagy aláírás) és az eszközazonosítás, amelyet a későbbi alfejezetek tárgyalnak.

18.1.8.4 4. Hibakeresési feladat

A táblázatból hiányzik a **ts-node firmware/flash** mint adattároló/elem, valamint az **adatbázis (backend → TimescaleDB)** adatfolyam. A legkritikusabb hiány a firmware/flash, mert az IoT-eszközök fizikailag hozzáférhetők, és a firmware illetéktelen módosítása a teljes eszköz kompromittálódásához vezethet. A STRIDE-modellben a Tampering és Elevation of Privilege kategóriákat itt kell vizsgálni: a támadó módosíthatja a firmware-t (T), vagy a debug porton keresztül root jogot szerezhet (E). Az ellenintézkedés a secure boot és az aláírt firmware-frissítés, amelyet a Biztonságos boot és firmware-aláírás alfejezet tárgyal.

18.1.9 Rövid összefoglaló

Az alfejezet elkészítette a TerraSense rendszer fenyegetésmodelljét. Elkészült az adatfolyam-diagram a bizalmi határokkal: eszköz-edge, edge-felhő, felhő belső és fizikai hozzáférés. A STRIDE-táblázat minden adatfolyamra, adattárolóra és komponensre azonosítja a megszemélyesítés, illetéktelen módosítás, letagadhatóság, információszivárgás, szolgáltatásmegtagadás és jogosultság-kiterjesztés fenyegetéseit. A kockázatértékelés kvalitatív mátrix alapján rangsorolja a tételeket. A **továbbvitt eredmény** a `~/iot-labor/i053/terrasense-fenyegetesmodell.md` fájl, amely az ellenintézkedés-térképet tartalmazza, és a későbbi alfejezetek – Tipikus IoT-sebezhetőségek és a szabályozás, Kriptográfiai alapok, Eszközidentitás, Biztonságos boot és firmware-aláírás – ebből indulnak ki. A modell a fizikai hozzáférést is explicit módon kezeli, mert a ts-node az üvegházban bárki által elérhető.

18.1.10 Források és ellenőrzési hiányok

- OWASP IoT Project – általános IoT-biztonsági irányelvek; a konkrét ajánlások és a projekt aktuális állapota ellenőrizendő.
- ETSI EN 303 645 – a fogyasztói IoT biztonsági alapkövetelményei; a szabvány pontos verziója és a megfelelőségi kritériumok ellenőrizendők.
- EU Cyber Resilience Act – a hatálybalépés, az alkalmazási kör és a kötelező határidők ellenőrizendők a hivatalos jogszabályban.
- NIST Cybersecurity for IoT Program – az ajánlások konkrét verziója ellenőrizendő.
- Microsoft STRIDE – a modell eredeti leírása; az itt használt kategóriák stabilak, de a módszertan részletei a Microsoft dokumentációjában ellenőrizendők.
- A TerraSense kockázati besorolások kvalitatív becslések, amelyeket az üzemeltetési környezet ismeretében felül kell vizsgálni. A laborban nem végeztünk valós támadási kísérleteket.

18.2 Tipikus IoT-sebezhetőségek és a szabályozás

18.2.1 Motiváció

Egy IoT-rendszer nem attól válik biztonságossá, hogy egyetlen rétegébe beépítünk egy titkosítást vagy egy tűzfalat. A támadó a leggyengébb láncszemet keresi, és az IoT-ban ez gyakran egy egyszerű, a fejlesztés során bagatellnek tűnő hiba: az alapértelmezett jelszó, a frissítés

nélkül hagyott firmware, a nyílt debug-port vagy a titkosítatlan telemetria. A TerraSense rendszerben az üvegházban elhelyezett ts-node egyszerű, olcsó eszköz, amely akár hálózati támadási felület is lehet, ha nem vesszük komolyan a sebezhetőségeket. Az előző alfejezetben felépítettük a fenyegetésmodellt; most a leggyakoribb hibákat és az azokra adott szabályozási válaszokat tekintjük át, hogy a későbbi fejezetekben (OTA-frissítés, flottakezelés, CI/CD) konkrét követelményeket tudjunk teljesíteni. A cél nem a jogi megfelelés önmagáért, hanem a gyakorlatban is működő védelmi intézkedések megalapozása.

18.2.2 Fogalmak és működés

Definíció (IoT-sebezhetőség). IoT-sebezhetőség alatt egy olyan hiányosságot értünk a rendszer hardver-, firmware-, szoftver-, konfigurációs vagy üzemeltetési rétegében, amely lehetővé teszi egy támadó számára, hogy a rendszer bizalmasságát, sértetlenségét vagy rendelkezésre állását megsértse, illetve az eszközt további támadásokhoz használja fel.

Definíció (SBOM). Az SBOM (Software Bill of Materials, szoftver-összetevőjegyzék) egy gépileg olvasható lista, amely egy szoftvertermékben található komponenseket, azok függőségeit, verzióit és kapcsolatait tartalmazza; az IoT-ban ez kiterjed a firmware-re, a beágyazott operációs rendszerre és a konténerekre is. Az SBOM önmagában **nem tartalmazza a sebezhetőségeket**: az ismert CVE-kat külön sebezhetőségi adatbázisból (például NVD, OSV) kell a komponensekhez rendelni, az érintettségről pedig külön nyilatkozat (például VEX) rendelkezhet. Az összetevőjegyzék és a változó sérülékenységi állapot tehát két külön adat, amelyet rendszeresen össze kell kapcsolni.

18.2.2.1 OWASP IoT Top 10

Az OWASP (Open Worldwide Application Security Project) rendszeresen közzéteszi az IoT-rendszerek leggyakoribb biztonsági hiányosságait. A lista pontos verziója és sorrendje ellenőrizendő, de a tipikus kategóriák a következők (a TerraSense-re vetítve):

1. **Gyenge, kitalálható vagy alapértelmezett jelszavak** – például a Mosquitto brokeren a `lab` / `lab` hitelesítés, vagy a fejlesztői konzol nyitva hagyása.
2. **Nem biztonságos hálózati szolgáltatások** – nyitott telnet/SSH port, debug-interfész, hitelesítés nélküli MQTT.
3. **Nem biztonságos ökoszisztéma-interfészek** – webes felület a backendhez vagy a gateway-hez, amely nem ellenőrzi a bemenetet vagy a jogosultságot.
4. **Frissítési mechanizmus hiánya** – a ts-node firmware-e nem tölthető be biztonságosan, nincs aláírás-ellenőrzés, nincs rollback.
5. **Nem biztonságos komponensek használata** – ismert sebezhetőséget tartalmazó Zephyr- vagy OpenSSL-verzió.
6. **Elégtelen adatvédelem** – a szenzoradatok titkosítatlanul mennek át a hálózaton, vagy a tárolt adatokhoz nem megfelelő a hozzáférés-szabályozás.
7. **Nem biztonságos adatátvitel és -tárolás** – például MQTT 1883-as porton TLS nélkül.
8. **Eszközmenedzsment hiánya** – nincs provisioning, nincs eszközidentitás, nem vonható vissza egy feltört csomópont.
9. **Nem biztonságos alapértelmezett beállítások** – debug-konzol engedélyezve, gyári kulcsok.
10. **Fizikai hozzáférés elleni védelem hiánya** – a debug-porton keresztül kiolvasható a firmware vagy a kulcs.

A lista nem hierarchikus, és egy adott rendszerre több elem is egyszerre teljesülhet; a TerraSense-ben például az első, a hetedik és a nyolcadik a legvalószínűbb hiányosság a kezdeti laborokban.

18.2.2.2 Esettanulmány: a Mirai botnet

A Mirai botnet 2016-ban vált hírhedtté, amikor nagy forgalmú DDoS-támadásokat indítottak vele, például a Dyn DNS-szolgáltató ellen, ami komoly internetes kieséseket okozott. A támadás alapja meglepően egyszerű volt: a Mirai kártevő előre definiált listából próbált ki gyári felhasználóneveket és jelszavakat (mint `admin/admin`, `root/12345`) a nyilvános internetre csatlakoztatott IoT-eszközökön – IP-kamerákon, otthoni routereken, DVR-eken. A fertőzés után az eszközök egy botnet részévé váltak, és parancsra forgalmat generáltak. A támadó nem bonyolult exploitot használt, hanem az eszközök gyenge jelszavait és a felhasználók közömbösségét. A tanulság: az IoT-ban a legegyszerűbb hiba is tömeges visszaélést tesz lehetővé, és a gyártó felelőssége, hogy az alapértelmezett beállítások biztonságosak legyenek, ne a felhasználóé.

18.2.2.3 ETSI EN 303 645

Az ETSI (European Telecommunications Standards Institute) 2020-ban tette közzé az EN 303 645 V2.1.1 szabványt, amely a fogyasztói IoT-eszközök kiberbiztonsági alapkövetelményeit határozza meg. A szabvány 13 követelménycsoportot (5.1–5.13) tartalmaz, amelyek közül a TerraSense szempontjából a legfontosabbak:

- **Nincs univerzális alapértelmezett jelszó** (5.1-1): a szabvány a minden eszközön azonos, gyári alapértelmezett jelszót tiltja; megengedett a felhasználónként vagy eszközönként egyedi hitelesítő, illetve az első bekapcsoláskor kötelezően megváltoztatandó jelszó.
- **Szoftverfrissítések** (5.3-1–5.3-12): az eszköznek biztonságos frissítési mechanizmust kell biztosítania, beleértve a frissítés hitelességének és integritásának ellenőrzését, a sikertelen frissítésből való helyreállást és a visszagörgetés (rollback) kezelését. A támogatási időszak közzététele külön rendelkezés (5.3-13), nem azonos a frissítési mechanizmus követelményével.
- **Biztonságos kommunikáció** (5.5-1, 5.5-2): az érzékeny adatok átvitelét titkosítani kell, és a hitelesítést úgy kell megvalósítani, hogy az megakadályozza a lehallgatást és a hamisítást.
- **Személyes adatok védelme** (5.8-1): a személyes adatokat – a TerraSense-ben például a telemetria időbélyegéből következtetett működési mintákat – a vonatkozó adatvédelmi szabályok szerint, megfelelő jogalapon kell gyűjteni és továbbítani; a hozzájárulás csak az egyik lehetséges jogalap (a GDPR 6. cikke több jogalapot ismer).
- **Támadási felület minimalizálása** (5.6-1): az eszközön csak a szükséges hálózati portok és szolgáltatások futhatnak; a debug-interfészeket le kell tiltani vagy védeni.
- **Kiesésekkel szembeni ellenálló képesség** (5.9-1): az eszköznek hálózati vagy áramkiesés esetén is biztonságosan kell működnie és helyre kell állnia, például watchdog, biztonságos újraindítás és helyi működés révén.
- **Telemetriai adatok vizsgálata** (5.10-1): a biztonsági szempontból releváns telemetriát rendszeresen vizsgálni kell, hogy a rendellenes viselkedés (például szokatlan csatlakozási minta) kiderüljön.

Egy telemetriai adat önmagában nem feltétlenül személyes adat: akkor minősül annak, ha egy azonosított vagy azonosítható személyhez kapcsolható (például az eszköz egy háztartáshoz vagy felhasználóhoz rendelhető). A gyűjtés célját, adatkörét és jogalapját ezért a kapcsolhatóság alapján kell meghatározni; a hozzájárulás csak az egyik lehetséges jogalap.

A szabvány nem egyenlő a jogszabállyal, de a CRA (lásd alább) hivatkozik rá, és számos gyártó ezt használja önkéntes megfelelőségi alapként. Az ETSI EN 303 645 alkalmazása **nem jelent**

automatikus CRA-megfelelést: a szabvány önkéntes műszaki alap, a CRA jogi kötelezettség, amelynek hatálya, termékbesorolása és megfelelőségértékelési útja termékfüggő. A TerraSense tanulási projekt nem forgalomba hozott termék, ezért a fejezet nem állít jogi megfelelést; a laborok a követelmények tudatos gyakorlását szolgálják.

18.2.2.4 EU Cyber Resilience Act (CRA)

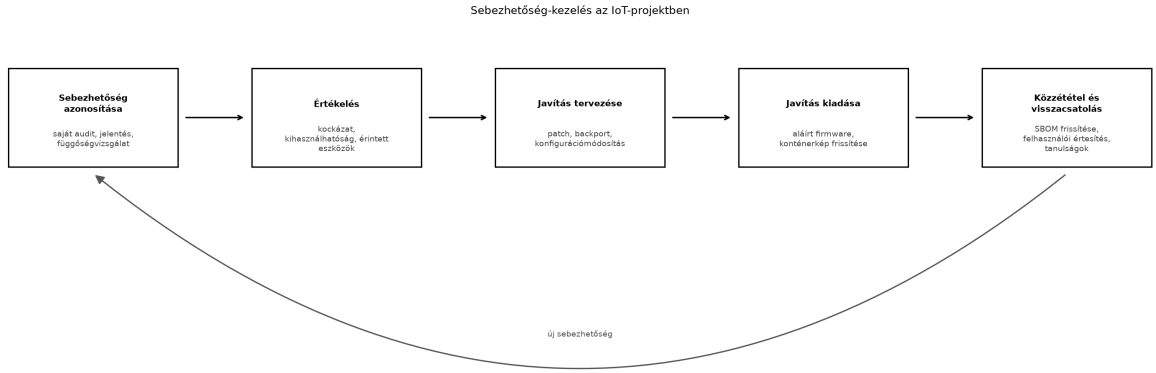
Az EU Cyber Resilience Act (CRA) egy rendelet, amely a digitális elemeket tartalmazó termékekre – így az IoT-eszközökre és szoftverekre – kötelező kiberbiztonsági követelményeket ír elő. A rendelet célja, hogy a termékek teljes életciklusa során (tervezés, fejlesztés, forgalomba hozatal, üzemeltetés) biztosítsa a biztonságot. A CRA főbb kötelezettségei:

- A gyártónak kockázatértékelést kell végeznie, és a terméket az alapvető kiberbiztonsági követelményeknek megfelelően kell terveznie és fejlesztenie.
- A szoftver-összetevőjegyzéket (SBOM) a műszaki dokumentáció részeként el kell készítenie, és a piacfelügyeleti hatóság kérésére rendelkezésre kell bocsátania; a **nyilvános közzététel nem általános kötelezettség**, a CRA 13. cikke és VII. melléklete nem ír elő mindenki számára elérhető publikálást.
- Nem minden ismert sebezhetőséget, hanem az **aktívan kihasznált sebezhetőségeket és a súlyos biztonsági incidenseket** kell jelentenie az érintett CSIRT-nek és a piacfelügyeleti hatóságnak; emellett a felhasználókat is értesítenie kell.
- A biztonsági frissítéseket a gyártó által meghatározott támogatási időszakban kell biztosítania; a CRA főszabályként **legalább öt évet** ír elő, kivéve, ha a termék várható élettartama ennél rövidebb.
- A súlyos incidenseket a CRA 14. cikke szerinti határidőkkel kell jelenteni.

Fontos: a CRA pontos határidői, átmeneti időszakai és a konkrét harmonizált szabványok folyamatosan alakulnak; a fejezet írásakor a rendelet hatálybalépését és alkalmazását mindig az aktuális jogi szöveg alapján kell ellenőrizni. A rendelet 2024. december 10-én lépett hatályba; a fő alkalmazási dátum 2027. december 11., a 14. cikk szerinti jelentési kötelezettség 2026. szeptember 11-től, a IV. fejezet 2026. június 11-től alkalmazandó. Ezek a dátumok jogszabály-módosítással változhatnak, ezért a hivatalos szövegből ellenőrizendők. A TerraSense tanulási projektje nem forgalomba hozott termék, de a gyakorlatban hasznos, ha a laborok során a CRA szellemében járunk el.

18.2.2.5 Sebezhetőség-kezelés, SBOM, közzétételi politika

A sebezhetőség-kezelés egy folyamatos ciklus, amelyet az alábbi ábra szemléltet:



18.2. ábra: Sebezhetőség-kezelés az IoT-projektben - A folyamat a CRA és az ETSI EN 303 645 követelményeit követi.

A folyamat kulcseleme az SBOM. A TerraSense esetében az SBOM tartalmazza a Zephyr verzióját, a Paho MQTT kliens verzióját, a Spring Boot és a Java függőségeit, a Mosquitto verzióját, valamint a konténeralapokat. Ha egy függőségben ismert sebezhetőség jelenik meg (például egy CVE az OpenSSL-ben), az SBOM és a sebezhetőségi adatbázis összekapcsolásával gyorsan meghatározható, melyik komponenst kell frissíteni; maga az SBOM csak az összetevőket és verziókat azonosítja. A közzétételi politika (vulnerability disclosure policy) meghatározza, hogy a fejlesztőcsapat hogyan fogadja és kezeli a külső bejelentéseket, milyen határidővel ad ki javítást, és hogyan tájékoztatja a felhasználókat anélkül, hogy a sebezhetőséget idő előtt nyilvánosságra hozná.

18.2.3 Döntési szempontok

A szabályozási követelmények és a gyakorlati megvalósítás közötti választás nem mindig egyértelmű. Az alábbi táblázat a TerraSense-ben felmerülő dilemmákat foglalja össze:

Kérdés	Lehetőség	Előny	Hátrány	TerraSense döntés
Hogyan hitelesítsük az eszközt?	Egyedi jelszó	Egyszerű	Gyenge, lopható	Nem tiltott (az ETSI 5.1-1 csak az univerzális alapértelmezett jelszót tiltja); a TerraSense mégis X.509 tanúsítványt választ
Milyen frissítési stratégia?	Nincs frissítés	Nulla fejlesztési költség	Kihasználható sebezhetőségek, CRA-megszegés	A/B kép frissítés rollbackkel (OTA-frissítés: A/B képek, rollback, delta, fokozatos kiadás)

Kérdés	Lehetőség	Előny	Hátrány	TerraSense döntés
Hol legyen a titkosítás?	Csak a felhőben	Kevesebb erőforrás az eszközön	Lehallgatható telemetria a LAN-on	TLS/mTLS az MQTT-hez (Biztonságos kapcsolat: TLS 1.3, mTLS és DTLS az MQTT/CoAP alatt)
SBOM készítése manuálisan vagy automatizáltan?	Manuális	Kevesebb bevezetési munka	Elavul, hiányos	Automatikus generálás a CI/CD-ben (CI/CD a firmware-től a backendig)
Mennyi ideig támogassuk a frissítéseket?	Csak a garancia ideje alatt	Olcsóbb	CRA szerint főszabályként legalább 5 év, a termék élettartamától függően	A TerraSense projektnél a kurzus ideje alatt, de ipari projektben legalább 5 év, a termék élettartamához igazítva

A lényeg: a szabályozás gyakran minimális, de egyértelmű elvárásokat fogalmaz meg, amelyeket nem lehet figyelmen kívül hagyni.

18.2.4 Kidolgozott példa: a TerraSense megfeleltetése az ETSI EN 303 645-nek

Továbbvitt eredmény. Az itt elkészített követelmény-megfeleltetési táblázatot használja a(z) Biztonságos boot és firmware-aláírás, a(z) OTA-frissítés: A/B képek, rollback, delta, fokozatos kiadás, a(z) Flottakezelés: nyilvántartás, konfiguráció, állapot és a(z) CI/CD a firmware-től a backendig alfejezet, amelyekben az itt jelzett hiányosságokra konkrét megoldásokat építünk.

Tekintsük a TerraSense jelenlegi, a korai laborokból származó állapotát, és végezzük el a megfeleltetést a legfontosabb ETSI-rendelkezésekre. A táblázatban a “Jelenlegi állapot” a fejlesztés során tipikus, célzottan be nem biztosított állapotot tükrözi; a “Teendő” oszlop a későbbi fejezetekben megvalósítandó feladatokat jelöli.

ETSI rendelkezés	TerraSense komponens	Jelenlegi állapot (tipikus hiba)	Teendő
5.1-1 (nincs univerzális alapértelmezett jelszó)	ts-node, ts-gw, broker	A laborokban `lab` / `lab` jelszó, eszközön nincs is hitelesítő	Egyedi kulcspár, X.509 tanúsítvány, a tanúsítvány identitásának ACL-hozzárendelése (Eszközidentitás: X.509-tanúsítványok, PKI és kulcstárolás, Biztonságos kapcsolat: TLS 1.3, mTLS és DTLS az MQTT/CoAP alatt)
5.2-1–5.2-3 (sebezhetőség-kezelés)	projekt	Nincs nyilvános bejelentési útvonal és válaszidő	Bejelentési politika, elérhető kapcsolat, javítási határidő (Tipikus IoT-sebezhetőségek és a szabályozás, CI/CD a firmware-től a backendig)
5.3-1–5.3-12 (frissítési mechanizmus)	ts-node firmware	Nincs OTA, a firmware csak kézzel, debuggerrel frissíthető	A/B partíció, aláírt és integritás-ellenőrzött kép, rollback, jelölt frissítési állapot (Biztonságos boot és firmware-aláírás, OTA-frissítés: A/B képek, rollback, delta, fokozatos kiadás)
5.3-13 (támogatási időszak közzététele)	termékdokumentáció	Nincs megadva támogatási időszak	A támogatási időszak rögzítése és közzététele (Flottakezelés: nyilvántartás, konfiguráció, állapot)
5.4-1 (érzékeny biztonsági paraméterek védelme)	ts-node kulcstárolás	A kulcs a firmware-ben vagy fájlban, védelem nélkül	Védett kulcstároló, hardveres védelem (Eszközidentitás: X.509-tanúsítványok, PKI és kulcstárolás)
5.5-1 (titkosított kommunikáció)	MQTT telemetria, parancsok	1883-as porton TLS nélkül, a gateway és a broker között is	mTLS a gateway és a broker között, TLS a backendhez (Biztonságos kapcsolat: TLS 1.3, mTLS és DTLS az MQTT/CoAP alatt)

ETSI rendelkezés	TerraSense komponens	Jelenlegi állapot (tipikus hiba)	Teendő
5.6-1, 5.6-4 (támadási felület, debug-interfész)	ts-node	Debug UART nyitva, felesleges shell parancsok	Debug-port letiltása éles buildben, minimális szolgáltatáskészlet (Eszközszoftver-architektúra: rétegek és állapotgépek, Biztonságos boot és firmware-aláírás)
5.7-1 (secure boot)	ts-node	Nincs ellenőrzött bootfolyamat	Aláírt firmware, secure boot, visszajelzés módosítás esetén (Biztonságos boot és firmware-aláírás)
5.8-1 (személyes adatok védelme)	Backend, telemetria	Minden adat tárolása korlátlanul, nincs hozzáférési szabályzat és dokumentált jogalap	Adatmegőrzési politika, szerepkör-alapú hozzáférés, minimális adatgyűjtés, jogalap rögzítése (Megőrzés, ritkítás és tárolási költség)
5.9-1 (kiesésekkel szembeni ellenálló képesség)	ts-node, ts-gw	Nincs watchdog, nincs offline puffer	Watchdog, állapotgép alapú újraindulás, store-and-forward (Hibatűrés: watchdog, újraindulás, hibanaplózás, Store-and-forward: offline működés és visszatöltés)
5.10-1 (telemetriai adatok vizsgálata)	Backend	Nincs biztonsági naplózás és anomáliafigyelés	Audit log, riasztás a rendellenes csatlakozásokra (Megfigyelhetőség végponttól végpontig: metrikák, naplók, nyomkövetés)
5.11-1 (felhasználói adatok törlése)	Backend, eszköz	Nincs törlési útvonal	Adattörlési funkció és dokumentált eljárás (Megőrzés, ritkítás és tárolási költség)
5.12-1 (telepítés és karbantartás egyszerűsége)	provisioning	Kézi, hibalehetőséget adó telepítés	Automatizált provisioning, ellenőrzőlista (Gyártás és üzembe helyezés (provisioning, onboarding))

ETSI rendelkezés	TerraSense komponens	Jelenlegi állapot (tipikus hiba)	Teendő
5.13-1 (bemenet-validáció)	ts-gw, Backend	A bemenet ellenőrzése esetleges	Séma-validáció, hossz- és tartományellenőrzés (Adatformátumok: JSON, CBOR, Protocol Buffers, SenML)

Ez a táblázat nem csak ellenőrzőlista, hanem a fejlesztési backlog alapja is. A következő fejezetekben ezeket a sorokat dolgozzuk ki.

18.2.5 Ellenőrzés és hibakeresés

A megfeleltetés után a gyakorlatban ellenőrizni kell, hogy a bevezetett intézkedések valóban működnek. A TerraSense-ben a következőket tesszük:

1. **Portellenőrzés**: `nmap`-mel vagy `ss -lntp`-vel megvizsgáljuk, hogy a broker csak a 8883-as (TLS) porton figyel-e, a 1883-as ne legyen nyitva.
2. **Hitelesítés ellenőrzése**: megpróbálunk csatlakozni a brokerhez `lab` / `lab` jelszóval; a kapcsolatnak el kell utasítódnia.
3. **Firmware-aláírás ellenőrzése**: a Zephyr build után ellenőrizzük, hogy a `signed.bin` fájl tartalmazza-e az aláírást, és a bootloader visszautasítja-e a módosított képet.
4. **SBOM teljessége**: a CI/CD futás után ellenőrizzük, hogy a generált `sbom.json` tartalmazza-e az összes függőséget, és hogy a függőségvizsgáló eszköz (például `trivy` vagy `grype`) nem talál-e kritikus CVE-t.

Tipikus hibatünetek:

Hibatünet	Lehetséges ok	Javítás
A Mosquitto 1883-as porton is figyel	A Compose-fájlban mindkét port nyitva	Csak a 8883-at tegyük közzé, a 1883-at csak belső hálózaton hagyjuk
Az eszköz nem tud csatlakozni mTLS-sel	Hiányzik a CA-tanúsítvány az eszközről, vagy a tanúsítvány lejárt	Ellenőrizzük a tanúsítványláncot és az óra szinkronizációját (Idő az IoT-ban: időbélyeg, időszinkron, sorrend)
A firmware-frissítés után az eszköz nem indul	Az A/B partíció beállítása hibás, a bootloader nem találja a képet	Ellenőrizzük a partíciós táblát és a bootloader beállításait (Biztonságos boot és firmware-aláírás)
Az SBOM hiányos	A CI/CD-ben a függőségvizsgáló nem fut a konténerképekre	A Docker build lépésnél is futtassunk SBOM-generálást

18.2.6 Tipikus hibák és félreértések

1. **Azt hisszük, hogy a „gyári alapértelmezett jelszavak tiltása” csak a felhasználói jelszavakra vonatkozik.** Valójában minden hitelesítő adatra vonatkozik, beleértve az adatbázis-, broker- és API-kulcsokat is. A TerraSense-ben a `ts-backend` által használt adatbázis-jelszót is egyedinek kell generálni, nem szabad a `docker-compose`-ban fix értékkel kiadni.
2. **A TLS bevezetésétől önmagában biztonságosnak tekintjük a rendszert.** A TLS védi az adatátvitelt, de nem véd a hitelesítés hiányától (ha a szerver nem ellenőrzi a klienst, vagy ha a tanúsítványt nem érvényesítik). Pontosan ezt a félreértést kezeli az mTLS követelménye.
3. **Az SBOM-ot egyenlőnek tekintjük a biztonsági audittal.** Az SBOM csak egy lista; önmagában nem javít semmit. A tényleges sebezhetőség-kezeléshez folyamatos monitorozás és frissítés kell.

18.2.7 Feladatok

18.2.7.1 a) Fogalmi kérdés

Melyik ETSI EN 303 645 rendelkezés tiltja az univerzális alapértelmezett jelszavakat, és hogyan teljesül ez a TerraSense-ben a tervezett megoldással?

18.2.7.2 b) Tervezési feladat

A TerraSense backendjének jelenlegi telepítése egyetlen `docker-compose.yml`-ben fix jelszavakat használ (`POSTGRES_PASSWORD: secret`). Tervezze meg, hogyan lehetne ezt úgy módosítani, hogy minden konténer egyedi, a hoszton titkosítottan tárolt jelszót kapjon, anélkül hogy a jelszó a Git-repozitóriumba kerülne. Milyen eszközt használna?

18.2.7.3 c) Implementációs feladat (előkészíti a továbbvitt eredményt)

Készítsen egy `~/iot-labor/i054/etsi-mapping.md` fájlt, amelyben a fenti táblázatot kiegészíti egy “Felelős fejezet” oszloppal, amely minden sorhoz hozzárendeli a megvalósítást tárgyaló későbbi alfejezetet (Eszközidentitás: X.509-tanúsítványok, PKI és kulcstárolás–CI/CD a firmware-től a backendig). A táblázatot használja a záróprojektben (Záróprojekt: a teljes TerraSense rendszer összeállítása és átvétele) ellenőrzőlistaként.

18.2.7.4 d) Hibakeresési feladat

Egy teszt során a `ts-node` nem tud csatlakozni a Mosquitto brokerhez mTLS-sel. A naplóban a hiba: `TLS error: certificate verify failed`. Sorolja fel a lehetséges okokat, és adja meg a hibakeresés lépéseit.

18.2.8 Megoldások

18.2.8.1 a) Megoldás

Az ETSI EN 303 645 5.1-1 rendelkezése tiltja az univerzális alapértelmezett jelszavakat. A TerraSense-ben az eszközök egyedi X.509 tanúsítványt kapnak a provisioning során, a broker pedig az eszköz tanúsítványa alapján azonosítja a klienst, így nincs közös jelszó. Ezt az Eszközidentitás: X.509-tanúsítványok, PKI és kulcstárolás és a Biztonságos kapcsolat: TLS 1.3, mTLS és DTLS alfejezetekben valósítjuk meg.

18.2.8.2 b) Megoldás

A `.env` fájl és a `.gitignore` önmagukban csak azt akadályozzák meg, hogy a titok a repóba kerüljön; nem titkosítanak, és a lemezen olvasható marad. A `docker compose` `secrets` szekciója helyi (nem Swarm) módban a hoszton lévő fájlt csatolja be a konténerbe, jellemzően `0600` jogosultsággal; ez sem titkosított tároló, csak elválasztja a titkot a képtől és a repótól. A Docker Swarm secret ezzel szemben a Swarm Raft-tárolójában titkosítva, tmpfs-en keresztül jut el a konténerhez, de a `docker compose` nem kezeli a Swarm secretjeit. A feladatban kért hosztoldali titkosított tároláshoz külső titokkezelő kell: például HashiCorp Vault, felhős KMS vagy SOPS + age. A laborban javasolt eljárás:

1. Konténerenként külön, véletlen jelszó generálása (`openssl rand -base64 32`).
2. A jelszavak titkosítása SOPS + age kulccsal a `~/iot-labor/i054/secrets/` könyvtárban; a nyilvános kulcs a repóban lehet, a privát kulcs a repón kívül, `0600` jogosultsággal.
3. Telepítéskor a titkosított fájl visszafejtése ideiglenes tmpfs könyvtárba (`sops -d` vagy `age -d`), `chmod 600`, majd a Compose `secrets` fájlmounttal a konténerbe injektálása; a sima szöveget nem írjuk a repóba, a képbe vagy tartós lemezre.
4. Éles környezetben Vault vagy KMS adja a titkot és a rotációt; a `.env` csak nem titkos konfigurációra való.

A `.gitignore` továbbra is véd a véletlen commit ellen, de nem helyettesíti a titkosítást.

18.2.8.3 c) Megoldás

A feladat egy konkrét fájl létrehozása; a tartalom a fenti táblázat kiegészítése, például:

```
| ETSI rendelkezés | Felelős fejezet |
|---|---|
| 5.1-1 | i056, i057 |
| 5.2-1-5.2-3 | i054, i064 |
| 5.3-1-5.3-12 | i058, i060 |
| 5.3-13 | i061 |
| 5.4-1 | i056 |
| 5.5-1 | i057 |
| 5.6-1, 5.6-4 | i018, i058 |
| 5.7-1 | i058 |
| 5.8-1 | i046 |
| 5.9-1 | i019, i037 |
| 5.10-1 | i062 |
| 5.11-1 | i046 |
| 5.12-1 | i059 |
| 5.13-1 | i032 |
```

Ezt a fájlt a későbbi fejezetekben referenciaként használhatjuk, és a záróprojektben ellenőrizzük, hogy minden hiányosság megoldódott-e.

18.2.8.4 d) Megoldás

Lehetséges okok:

- A ts-node-on nincs telepítve a broker CA-tanúsítványa, vagy a tanúsítványlánc nem megbízható.
- A **broker szervertanúsítványának** Subject Alternative Name mezője nem tartalmazza azt a DNS-nevet vagy IP-címet, amelyen a ts-node kapcsolódik. A kliens mindig a szerver nevét ellenőrzi, a sajátját nem; a szervertanúsítvány neve nem attól függ, hogy a broker `localhost`-on fut-e.
- A ts-node **kliens**tanúsítványának CN/SAN mezője nem egyezik azzal az identitással, amelyet a broker ACL-je vagy `use_identity_as_username` beállítása elvár. Ez nem TLS-kézfogási, hanem engedélyezési hiba.
- Valamelyik tanúsítvány lejárt, vagy az óra nincs szinkronizálva, és ezért a tanúsítványérvényesség ellenőrzése meghiúsul.
- A broker mTLS-t követel, de a ts-node nem mutatja be a saját tanúsítványát.
- A tanúsítványlánc hiányos, például a köztes CA-t nem küldi a kliens vagy a szerver.

Hibakeresési lépések: ellenőrizzük a `mosquitto` naplóját; a szerver nevének ellenőrzésével teszteljük a kézfogást: `openssl s_client -connect broker:8883 -CAfile ca.crt -cert client.crt -key client.key -verify_hostname broker`; nézzük meg a szerver SAN-mezőjét (`openssl x509 -in server.crt -noout -ext subjectAltName`) és a kliens identitását (`openssl x509 -in client.crt -noout -subject`); ellenőrizzük a `mosquitto` ACL és a `use_identity_as_username` beállítását; végül biztosítsuk az időszinkront (NTP).

18.2.9 Rövid összefoglaló

A leggyakoribb IoT-sebezhetőségek – alapértelmezett jelszó, frissítés hiánya, titkosítatlan kommunikáció – nem önálló hibák, hanem egymást erősítő hiányosságok. Az ETSI EN 303 645 és az EU Cyber Resilience Act minimális elvárásokat fogalmaz meg, amelyeket a TerraSense tervezése során iránymutatóként követünk: egyedi eszközidentitás, biztonságos frissítés, titkosított adatátvitel, minimális támadási felület és ellenálló képesség. A jogi megfelelés termék- és piacfüggő, és nem igazolható pusztán a technikai lista teljesítésével. Az SBOM és a közzétételi politika a sebezhetőség-kezelés eszközei, nem célok. A `~/iot-labor/i054/etsi-mapping.md` fájlban rögzítettük a TerraSense hiányosságait és a megfelelő fejezeteket; ezt a továbbvitt eredményt használja az OTA-frissítés, a Flottakezelés és a CI/CD alfejezet. A szabályozás pontos határidői és az OWASP-lista verziója ellenőrizendő.

18.2.10 Források és ellenőrzési hiányok

A következő források kiindulópontok, de a fejezet írásakor nem kerültek részletes ellenőrzésre:

- OWASP Internet of Things Project – az aktuális IoT Top 10 verziója és sorrendje ellenőrizendő.
- ETSI EN 303 645 – a rendelkezések szövege és az esetleges módosítások ellenőrizendők.
- EU Cyber Resilience Act – a hatálybalépés, az alkalmazás és a harmonizált szabványok aktuális állapota ellenőrizendő.
- NIST Cybersecurity for IoT Program – általános keretrendszer, a TerraSense esetében támpont.
- PSA Certified – a biztonsági tanúsítási szintek aktuális kritériumai ellenőrizendők.

Ez a fejezet nem nyújt jogi tanácsadást; a szabályozási megfeleléshez a mindenkori hivatalos szövegeket kell figyelembe venni.

19 Kriptográfia, identitás és biztonságos kommunikáció

19.1 Kriptográfiai alapok fejlesztőknek

19.1.1 Motiváció

A TerraSense rendszerben a ts-node eszközök telemetriát küldenek a ts-gw gateway-nek, amely aztán MQTT-n továbbítja a backend felé; a backend parancsokat küld lefelé. Ha a kommunikációs csatornát nem védjük, egy támadó hamis hőmérséklet-adatokat szúrhat be, módosíthatja a parancsokat (például kinyithatja az öntözőszelepet), vagy lehallgathatja az üzemeltetési adatokat. A(z) Fenyegetésmodellezés: támadási felület és STRIDE alfejezetben azonosítottuk ezeket a fenyegetéseket. A kriptográfia nem öncélú bonyolultság, hanem a biztonsági követelmények (bizalmasság, integritás, hitelesség) elérésének eszköze. Fejlesztőként a primitíveket a megfelelő helyen, a megfelelő módon kell használnunk, mert a rossz választás több kárt okozhat, mint a kriptográfia hiánya.

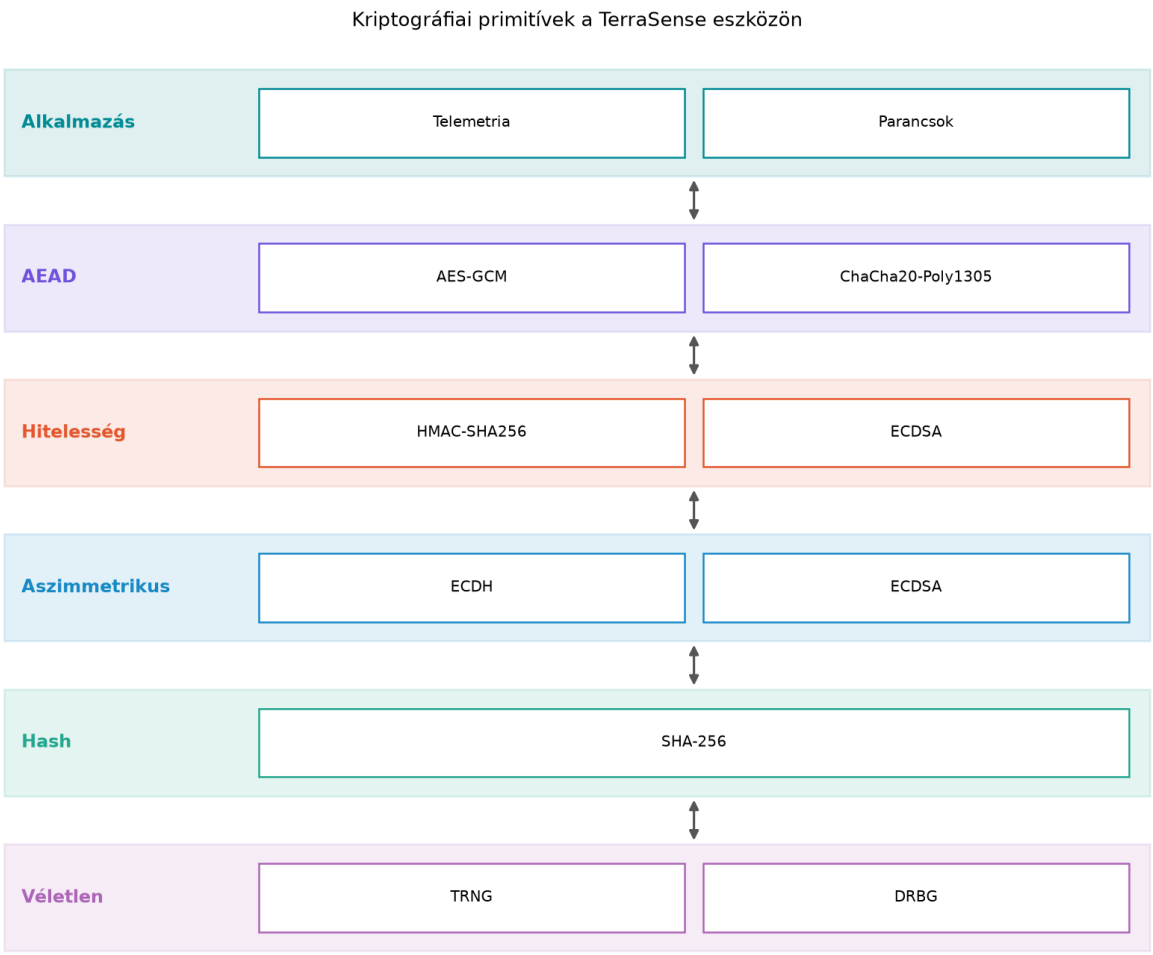
A következőkben a TerraSense eszközön futtatható, korlátozott erőforrású környezethez illeszkedő építőelemeket tekintjük át.

19.1.2 Fogalmak és működés

A kriptográfia négy alapvető védelmi célja a következő:

- **Bizalmasság (confidentiality):** az üzenetet csak a címzett ismerheti meg; illetéktelen fél a titkosított adatból nem tudja visszanyerni az eredeti tartalmat.
- **Integritás (integrity):** az adat nem változott meg jogosulatlanul; bármilyen módosítás észlelhető.
- **Hitelesség (authentication):** az üzenet valóban a megnevezett küldőtől származik, vagy a fél, akivel kommunikálunk, valóban az, akinek mondja magát.
- **Letagadhatatlanság (non-repudiation):** a küldő nem tagadhatja le, hogy ő küldte az adott üzenetet; ezt általában digitális aláírással érjük el.

Ezen célok elérésére különböző kriptográfiai primitíveket használunk. A primitívek **alapszintű algoritmusok vagy protokollok**, amelyek egy-egy biztonsági tulajdonságot nyújtanak. A megfelelő primitív kiválasztása a fenyegetésmodelltől, az erőforráskorlátoktól és a kulcsmenedzsment lehetőségeitől függ.



19.1. ábra: Kriptográfiai primitívek a TerraSense eszközön - Az alapelemek egymásra épülnek, és az alkalmazás szintjén kombinálódnak.

19.1.2.1 Szimmetrikus titkosítás: AES és AEAD

A szimmetrikus titkosítás ugyanazt a kulcsot használja a titkosításhoz és a visszafejtéshez. Az Advanced Encryption Standard (AES) a leggyakoribb blokkrejtjel; 128 bites blokkokat titkosít 128, 192 vagy 256 bites kulccsal. A blokkrejtjel önmagában csak fix méretű blokkot tud kezelni, ezért üzemmódra van szükség a hosszabb üzenetekhez. Az Electronic Codebook (ECB) üzemmód – amely minden blokkot azonos módon titkosít – **tilos**, mert az azonos plaintext blokkok azonos ciphertext blokkokat eredményeznek, ami mintázatot szivárogtat. A gyakorlatban a Galois/Counter Mode (GCM) az egyik leggyakrabban ajánlott AEAD-mód, de a választás a platformtól függ: AES hardvergyorsítás nélkül például a ChaCha20-Poly1305, kis memóriájú beágyazott rendszerekben az AES-CCM is jó választás lehet.

Definíció. AEAD (Authenticated Encryption with Associated Data): olyan titkosítási mód, amely egy műveletben biztosít bizalmasságot és integritást, valamint hitelesíti a titkosított adat mellett a kísérő (associated) adatokat is. Az AEAD bemenete a kulcs, a nonce (egyszer használatos szám), a plaintext és opcionálisan a kísérő adat (AAD, associated data); kimenete a ciphertext és az authentication tag. Az AAD-t nem titkosítjuk, de a hitelesítésbe bevonjuk: a címzett is ismeri, viszont a módosítása észlelhető. Az AES-GCM ennek egy konkrét, széles körben használt példája.

Az AEAD-nél ezért három, egymástól egyértelműen elválasztott részt kell kezelni: a titkosított plaintextet (amiből ciphertext lesz), a nem titkosított, de hitelesített AAD-t, valamint a kulcsot és a nonce-t. A fejléc bájtsorrendjét és a mezők határait rögzíteni kell (például hálózati bájtsorrendben), különben a támadó a mezőket áthelyezheti. Az AAD-ba érdemes belevenni a címzett identitását, az időbélyeget és a sorszámot, mert így a címzett cseréje és a visszajátszás is észlelhető.

Az AES-GCM biztonságának kritikus feltétele, hogy **ugyanazt a nonce-t soha ne használjuk fel kétszer ugyanazzal a kulccsal**. A nonce nem titkos, de egyedi kell legyen minden üzenethez. Ha újra felhasználjuk, a támadó visszafejtheti az üzeneteket, és hamis hitelesítést is produkálhat. A nonce-kezelést gyakran számlálóval vagy biztonságos véletlenszámmal oldjuk meg. A követelmény algoritmusfüggő: AES-GCM-nél az egyediség a döntő, ezért egy jól karbantartott számláló is megfelel; CBC-módnál viszont kiszámíthatatlan IV kell. Ha több eszköz közös kulcsot használ, a számlálót eszközazonosítóval kell kombinálni, és újraindulás után is perzisztensen kell folytatni, különben két eszköz vagy egy reboot után ismétlődhet a nonce.

19.1.2.2 Hash és HMAC

Definíció. A **hash függvény** tetszőleges hosszúságú bemenetből fix hosszúságú, gyakorlatilag egyedi kimenetet (digest) állít elő. Az SHA-256 256 bites kimenetet ad. A hash egyirányú (nem lehet visszafejteni) és gyakorlatilag ütközésálló (nem találunk két különböző bemenetet azonos kimenettel). A hash önmagában **nem** nyújt integritást, mert a támadó módosíthatja az adatot, és újra kiszámíthatja a hash-t.

Definíció. Az **HMAC** (Hash-based Message Authentication Code) egy titkos kulccsal kiegészített hash, amely hitelesített integritást nyújt. A számítás során a kulcsot a hash belsejébe és kívülre is belekeverjük, így aki nem ismeri a kulcsot, nem tud érvényes kódot generálni. Az HMAC-SHA256 az SHA-256-ot használja alapként, és az IoT-ban a leggyakoribb megosztott kulcsú hitelesítési primitív.

Az HMAC alkalmas arra, hogy a ts-node és a ts-gw között – ha megosztott kulccsal rendelkeznek – ellenőrizzük, hogy a telemetriai üzenet nem módosult, és valóban az adott eszköztől származik. A megosztott kulcsos hitelesítés hátránya, hogy a kulcsot mindkét oldalon tárolni kell, és kompromittálódása esetén nehéz a cseréje.

19.1.2.3 Aszimmetrikus kriptográfia: ECC

Az aszimmetrikus kriptográfia kulcspárt használ: a privát kulcs titkos, a publikus kulcs bárki számára hozzáférhető. A publikus kulccsal titkosíthatunk vagy ellenőrizhetünk, a privát kulccsal visszafejthetünk vagy aláírhatunk. Az RSA a legismertebb, de nagy kulcsot igényel (2048–4096 bit), ami korlátos eszközön lassú és memóriaigényes. Az elliptikus görbéken alapuló ECC (Elliptic Curve Cryptography) lényegesen kisebb kulcsokkal azonos biztonsági szintet nyújt.

Biztonsági szint (bit)	RSA kulcsméret (bit)	ECC kulcsméret (bit)
128	3072	256
192	7680	384
256	15360	521

Megjegyzés: a táblázat a NIST SP 800-57 ajánlásain alapuló illusztratív értékeket tartalmaz; a konkrét alkalmazás előtt ellenőrizze az aktuális ajánlást.

Az ECC két alapvető műveletét használjuk: az **ECDH** (Elliptic Curve Diffie-Hellman) kulcscserét, amellyel két fél közös titkot tud létrehozni a nyilvános csatornán keresztül, és az **ECDSA** (Elliptic Curve Digital Signature Algorithm) digitális aláírást, amely a privát kulccsal igazolja az üzenet eredetét. A kis kulcsméret miatt az ECC ideális a ts-node eszközön: kevesebb flash-, RAM- és CPU-igény, gyorsabb működés, kisebb tanúsítványméret.

Az ECDH önmagában nem hitelesített kulcscsere: egy közbeékelődő támadó két külön kulcscserét hozhat létre, és mindkét féllel külön beszélhet. Ezért az ECDH-t mindig hitelesített kézfogásban kell használni (tanúsítvány, digitális aláírás vagy PSK), és önmagában, hitelesítés nélkül nem tekinthető biztonságos protokollnak.

19.1.2.4 Véletlenszám-generálás

Minden kriptográfiai kulcs és aláírás, valamint a véletlenszerűen választott nonce vagy IV biztonsága azon múlik, hogy a véletlen számok valóban kiszámíthatatlanok-e. A számláló alapú nonce-nál nem a kiszámíthatatlanság, hanem az egyediség a követelmény: a számláló lehet előre jelezhető, de ugyanazzal a kulccsal nem ismétlődhet. **TRNG** (True Random Number Generator) fizikai zajforrásból (pl. hőzaj, oszcillátor jitter) származtat entrópiát. **DRBG** (Deterministic Random Bit Generator) egy determinisztikus algoritmus, amely a bemeneti entrópiából (seed) nagy mennyiségű, kriptográfiailag álvéletlen számot állít elő. A Zephyr operációs rendszer biztosít entrópia API-t és platformfüggő TRNG-t, de a pontos konfiguráció (pl. `CONFIG_TEST_RANDOM_GENERATOR` vagy `CONFIG_ENTROPY_DEVICE`) verziófüggő, ezért ellenőrizni kell. Az eszközön a boot után elegendő entrópiát kell gyűjteni, mielőtt kulcsot generálunk vagy nonce-ot használunk; egy determinisztikus seed-et ismerő támadó minden utána következő értéket megjósolhat.

19.1.2.5 Könyvtárak: mbedTLS és PSA Crypto API

A Zephyr ökoszisztémában az **mbedTLS** a leggyakrabban használt kriptográfiai könyvtár, amely C nyelven implementál primitíveket. Újabb fejlesztéseknél a **PSA Crypto API** az ajánlott interfész, amely egységes, hordozható réteget biztosít, és hardvergyorsítást is képes kihasználni. A pontos függvénynevek (pl. `psa_aead_encrypt`, `psa_generate_random`, `psa_sign_hash`) ellenőrzési igényt jelentenek, mert a Zephyr verziótól és a konfigurációtól függően eltérhetnek. A fejezet későbbi részei ezt az API-t fogják használni.

Továbbvitt eredmény. Az itt bemutatott AES-GCM-et, HMAC-ot, ECDSA-t, ECDH-t és a biztonságos véletlenszám-generálást használja az Eszközidentitás: X.509-tanúsítványok, PKI és kulcstárolás alfejezet tanúsítványokhoz, a Biztonságos kapcsolat: TLS 1.3, mTLS és DTLS az MQTT/CoAP alatt alfejezet a TLS/DTLS kézfogáshoz és rekordtitkosításhoz, valamint a Biztonságos boot és firmware-aláírás alfejezet a firmware-hitelesítéshez. A fogalmak és a választási szempontok megalapozzák ezeket a későbbi alkalmazásokat.

19.1.3 Döntési szempontok

A TerraSense-ben a primitívek kiválasztásakor a következő kérdéseket tesszük fel:

- Mit védünk? (bizalmasság, integritás, hitelesség)
- Milyen kulcsmenedzsment áll rendelkezésre? (megosztott kulcs, X.509, PKI)
- Mekkora az üzenet, milyen gyakori, mennyi energia áll rendelkezésre?
- Milyen könyvtárak érhetők el az eszközön?

A döntési táblázat:

Cél	Ajánlott primitív	Megjegyzés
Bizalmasság és integritás együtt	AEAD-mód: AES-GCM, ChaCha20-Poly1305 vagy AES-CCM	Az AEAD az elv; a konkrét algoritmust a platform, a hardvergyorsítás és a protokoll dönti el
Integritás, megosztott kulcsos hitelesség	HMAC-SHA256	Kulcsot mindkét oldalon tárolni kell
Integritás, publikus ellenőrzés	ECDSA	Digitális aláírás, privát kulcs az aláíró oldalán
Kulcscsere	ECDH hitelesített kézfogásban	Önmagában nem hitelesíti a feleket; tanúsítvány, aláírás vagy PSK szükséges
Hash (pl. ujjlenyomat, séma)	SHA-256	Nem hitelesítésre önmagában
Véletlen szám, nonce	TRNG + DRBG	A nonce egyediségét biztosítani kell; számláló is lehet, ha perzisztens

A döntési táblázat első sora nem azt jelenti, hogy minden titkosításhoz feltétlenül AES-GCM-et kell választani. Az AEAD követelmény; a konkrét algoritmus lehet AES-GCM, ChaCha20-Poly1305 vagy AES-CCM. Az AES-GCM hardveres AES-gyorsítással különösen hatékony, a ChaCha20-Poly1305 szoftveres környezetben lehet gyorsabb, az AES-CCM pedig kis memóriájú, korlátozott erőforrású eszközökön elterjedt. A választásnál a könyvtár támogatását, a nonce- és tagméretet, valamint az oldalcsatorna-kockázatot is figyelembe kell venni.

19.1.4 Kidolgozott példa: telemetriai üzenet védelme AES-GCM-mel

Tegyük fel, hogy a ts-node a következő, kissé egyszerűsített telemetriai JSON-t küldi (a valós sémában több mező van, de az elv ugyanaz):

```
{"n":"n001","t":22.5,"h":55.0,"s":30.0}
```

A karakterlánc hossza 39 bájt (illusztratív, nullterminátor nélkül; a bájthosszt érdemes programból, UTF-8 kódolással számítani). Ha ezt AES-128-GCM-mel titkosítjuk, a következő elemeket kapjuk:

- **Plaintext:** 39 bájt
- **Ciphertext:** 39 bájt (a GCM stream-szerű, a hossz megegyezik a plaintext hosszával)
- **Nonce:** 12 bájt (egyedi; biztonságos véletlen vagy perzisztens számláló)
- **Authentication tag:** 16 bájt (128 bites, a hitelesítéshez)
- **Associated data (fejléc):** 6 bájt, például a node_id és a seq röviden (nem titkos, de hitelesített)

A teljes hálózaton elküldött adat:

$$39 + 12 + 16 + 6 = 73 \text{ bájt}$$

Az overhead:

$$73 - 39 = 34 \text{ bájt}$$

Ez mintegy 87,2% méretnövekedés a titkosítás miatt. A példa jól mutatja, hogy kis üzeneteknél a nonce és a tag aránya jelentős; ezért IoT-ban gyakran érdemes kompaktabb kódolást (pl. CBOR, SenML) választani, és az AEAD overhead-jét a tervezéskor figyelembe venni. Ha az üzenet 200 bájt lenne, az overhead már csak 17%, így a hatás kisebb. Az AEAD-módok robusztussága és egyszerűsége miatt a tipikus IoT-forgalomnál még így is ajánlottak, de a rövid üzeneteknél a nonce-menedzsment és a tag mérete a mérnöki döntés része.

19.1.5 Labor: kulcsgenerálás, hash és HMAC az OpenSSL-lel

Előfeltétel: OpenSSL telepítve az Ubuntu 24.04 hoston (alaprendszerben elérhető). A labor célja a primitívek gyakorlati kézbevétele, nem éles kulcsok létrehozása.

Munkakönyvtár:

```
umask 077
mkdir -p ~/iot-labor/i055 && cd ~/iot-labor/i055
```

A `umask 077` azt eredményezi, hogy az újonnan létrehozott fájlok csak a tulajdonos számára olvashatók (a kulcsfájl jogosultsága 600), a könyvtárak pedig 700-as módot kapnak. A `umask` csak az adott shellre érvényes; új terminálban ismételje meg, vagy ellenőrizze a jogosultságokat a `stat -c '%a %n' lab-aes-key.hex` paranccsal.

Generáljunk egy labor-AES-kulcsot és számoljunk HMAC-ot:

```
# Labor-AES-kulcs generálása: 16 véletlen bájt, hex formában (32 karakter)
openssl rand -hex 16 > lab-aes-key.hex

# SHA-256 hash egy rövid üzenetre; a -r és a cut csak a hex digestet tartja meg
echo -n "hello TerraSense" | openssl dgst -sha256 -r | cut -d' ' -f1

# HMAC-SHA256 a TerraSense telemetria topic-ra, a 16 nyers bájtos kulccsal
echo -n "terrasense/gh1/n001 telemetry" | openssl dgst -sha256 -r \
  -mac HMAC -macopt hexkey:"$(cat lab-aes-key.hex)" | cut -d' ' -f1
```

Várható kimenet: mindkét parancs egy-egy 64 hex karakteres értéket ad. Az `openssl dgst` alapértelmezett, `-r` nélküli kimenete `SHA2-256(stdin)= <digest>` alakú, tehát a felirat is része lenne; a `-r` coreutils-formátumot ad (`<digest> *stdin`), a `cut` pedig csak a digestet tartja meg. A generált kulcs 16 bájt (128 bit) véletlen adat, amelyet a fájl 32 hex karakterrel ábrázol. Fontos: a `-hmac "..."` kapcsoló a megadott karakterláncot szó szerint, ASCII-bájtokként használná kulcsként, ezért a 32 hex karakterből 32 bájtos kulcs lenne; a `-mac HMAC -macopt hexkey:...` ezzel szemben a hex karaktereket 16 nyers bájtra alakítja. A hash fix hosszúságú, függetlenül a bemenet hosszától.

Ugyanez a HMAC független implementációval is ellenőrizhető:

```
python3 -c 'import hmac, hashlib; k = bytes.fromhex(open("lab-aes-key.hex").read().strip());
↪  print(hmac.new(k, b"terrasense/gh1/n001 telemetry", hashlib.sha256).hexdigest())'
```

A két értéknek meg kell egyeznie; ez igazolja, hogy a `hexkey:` valóban a 16 nyers bájtot használja kulcsként.

Ellenőrzés: ismételjük meg a HMAC parancsot ugyanazzal a kulccsal és üzenettel – azonos kimenetet kapunk. Módosítsuk az üzenetet (pl. egy karaktert), és a HMAC teljesen megváltozik. A kulcs nélkül a támadó nem tud érvényes HMAC-ot előállítani.

19.1.6 Ellenőrzés és hibakeresés

Tünet	Ok	Javítás
A HMAC nem egyezik a várt értékkel	A kulcs vagy az üzenet eltér (pl. szóköz, sortörés)	Ellenőrizze a kulcsfájlt, használja ugyanazt a `echo -n`-et
Az AES-GCM visszafejtés sikertelen (authentication failure)	A ciphertext, a tag vagy a nonce módosult; vagy rossz kulcs	Ellenőrizze a nonce egyediségét és a kulcsot
A Zephyr alkalmazásban `psa_aead_decrypt` hibát ad	A kulcs, a nonce, az AAD, a ciphertext vagy a tag nem egyezik a küldő oldalával; az AEAD-könyvtár nem tart nyilvántartást a korábbi nonce-okról, ezért a nonce-újrafelhasználást nem jelzi	Ellenőrizze a kulcsot, a nonce-ot és az AAD-t; a nonce egyediségét a protokollnak kell garantálnia (eszközazonosítóval kombinált, perzisztens számláló). A visszajátszás észlelése külön alkalmazásszintű sorszám vagy időbélyeg feladata
A generált kulcs gyenge (pl. determinisztikus)	Nem megfelelő entrópiaforrás, rossz seed	Kapcsolja be a TRNG-t a devicetree-ben, várjon elegendő entrópiát

19.1.7 Tipikus hibák és félreértések

1. **ECB mód használata.** Az AES-ECB azonos plaintext blokkhoz azonos ciphertextet ad, így a támadó mintázatot ismer fel. Mindig GCM-et (vagy más AEAD módot) használjunk.
2. **Nonce újrafelhasználása AES-GCM-nél.** Ugyanaz a kulcs+nonce pár katasztrofális: a támadó XOR-ral visszafejtheti az üzeneteket, és hamis üzenetet hitelesíthet. A nonce-ot számlálóként kell kezelni, eszközönként egyedi módon, és újraindulás után is perzisztensen kell folytatni; közös kulcsnál az eszközazonosítót is bele kell venni a nonce-ba.
3. **Saját kriptográfiai algoritmus tervezése.** A saját „csak néhány sort” kód mindig törhető. Csak auditált könyvtárat (mbedTLS, OpenSSL, PSA) szabad használni.
4. **Hash használata hitelesítésre.** A sima SHA-256 nem véd a hamisítás ellen, mert a támadó újraszámolhatja. MAC vagy digitális aláírás kell.

19.1.8 Feladatok

1. **Fogalmi:** Magyarázza el, miért nem elegendő az SHA-256 hash a telemetriai üzenet integritásának védelmére, és miért jobb az HMAC-SHA256.
2. **Számolási:** Egy TerraSense telemetriai üzenet 120 bájt hosszú. Számolja ki a teljes hálózati csomag hosszát, ha AES-256-GCM-mel titkosítjuk, a nonce 12 bájt, az authentication tag 16 bájt, az associated data 8 bájt. Mennyi az overhead százalékban?
3. **Implementációs:** Módosítsa a laborban a HMAC parancsot úgy, hogy a kulcsfájl neve `lab-aes-key.hex` legyen, a kulcsot 16 nyers bájtként használja (`-mac HMAC -macopt hexkey:...`), az üzenet a tényleges TerraSense telemetriai JSON (`{"node_id":"n001","seq":1,"ts":1699999999000,"temp":22.5,"rh":55.0,"soil":30.0}`), és a kimenetet mentse el egy `lab-telemetry.mac` fájlba. Ellenőrizze, hogy a fájl 64 hex karaktert tartalmaz, és hogy csak a `0-9a-f` karakterek szerepelnek benne.
4. **Hibakeresési:** Egy kolléga azt állítja, hogy az AES-CBC titkosítás önmagában megvédi az integritást. Miért téves ez, és milyen AEAD módot ajánlana neki? Magyarázza el a javítást.

19.1.9 Megoldások

1. A hash determinisztikus, kulcs nélküli. A támadó elkészítheti saját üzenetét, kiszámolhatja a hash-t, és elküldheti a hamis adatot. A HMAC titkos kulcsot kever a hash-be, így csak a kulcs ismerője tud érvényes kódot generálni.
2. A teljes hossz: $120 + 12 + 16 + 8 = 156$ bájt. Overhead: 36 bájt, ami $36/120 \cdot 100\% = 30\%$.
3. A parancs:

```
echo -n "{\"node_id\":\"n001\",\"seq\":1,\"ts\":1699999999000,\"temp\":22.5,\"rh\":55.0,\"soil
↪  \":30.0}"
↪  \
  | openssl dgst -sha256 -r -mac HMAC -macopt hexkey:"$(cat lab-aes-key.hex)" \
  | cut -d' ' -f1 > lab-telemetry.mac
wc -c lab-telemetry.mac  # 65: 64 hex karakter + újsor
grep -Eq '^[0-9a-f]{64}$' lab-telemetry.mac && echo "rendben: 64 hex karakter"
```

A `wc -c` 65-öt mutat a záró újsor miatt; a hex karakterek száma 64. A `grep` a karakterkészletet és a hosszt is ellenőrzi.

4. Az AES-CBC csak bizalmasságot ad, de nincs beépített integritás ellenőrzés. A támadó módosíthatja a ciphertextet, és a visszafejtett plaintext megváltozik (ha padding oracle támadás lehetséges, akkor az is veszély). Az integritás védelmére vagy külön HMAC-ot kell számolni a ciphertextre, vagy jobb, ha AEAD módot használunk (például AES-GCM, ChaCha20-Poly1305 vagy AES-CCM), amely egyben hitelesíti az adatot.

19.1.10 Rövid összefoglaló

Ebben az alfejezetben áttekintettük az IoT-fejlesztő számára elengedhetetlen kriptográfiai primitíveket. A négy védelmi cél: bizalmasság, integritás, hitelesség, letagadhatatlanság. A szimmetrikus titkosításra AEAD-mód ajánlott (például AES-GCM, ChaCha20-Poly1305 vagy AES-CCM), mert egy műveletben titkosít és hitelesít; a nonce egyedisége kritikus, de származhat perzisztens számlálóból is. Az SHA-256 hash önmagában nem elegendő, HMAC-SHA256-ot használunk megosztott kulcsú hitelességre. Az aszimmetrikus ECC kis kulccsal azonos biztonságot ad, az ECDH kulcscsere csak hitelesített kézfogásban biztonságos, az ECDSA aláírás. A véletlenszám forrás TRNG és DRBG. Zephyrben az mbedTLS/PSA Crypto API az ajánlott. **Továbbvitt**

eredmény: ezeket a primitíveket az Eszközidentitás: X.509-tanúsítványok, PKI és kulcstárolás, a Biztonságos kapcsolat: TLS 1.3, mTLS és DTLS az MQTT/CoAP alatt és a Biztonságos boot és firmware-aláírás alfejezetek alkalmazzák.

19.1.11 Források és ellenőrzési hiányok

Stabil szabványok:
- NIST FIPS 197 (AES), FIPS 180-4 (SHA-256), FIPS 198-1 (HMAC), SP 800-38D (GCM), SP 800-57 (kulcsméret-ajánlások).
- RFC 8446 (TLS 1.3), RFC 9147 (DTLS 1.3), RFC 5280 (X.509) – ezek a fejezet további részeiben kerülnek alkalmazásra.

Ellenőrzési hiányok:
- A Zephyr PSA Crypto API pontos függvénynevei, Kconfig opciók (pl. `CONFIG_PSA_CRYPTO`, `CONFIG_ENTROPY_DEVICE`) és az mbedTLS verziója a kiadástól függ; használat előtt ellenőrizze a Zephyr dokumentációját.
- Az OpenSSL parancssori kimenet pontos formátuma a verziótól függhet; a labor a gyakori elterjedt viselkedést mutatja.
- A táblázatban szereplő kulcsméretek a NIST ajánláson alapuló illusztratív értékek, az aktuális szabványoknak való megfeleléshez ellenőrizze a legfrissebb kiadást.

19.2 Eszközidentitás: X.509-tanúsítványok, PKI és kulcstárolás

19.2.1 Motiváció

A TerraSense rendszerben a `ts-node` eszközök és a `ts-gw` gateway hamarosan nem egyetlen fejlesztői környezetben futnak, hanem egy valódi üvegházban. A Mosquitto brokeren keresztül több tucat vagy több száz eszköz küld telemetriát, és a backend parancsokat ad lefelé. Ha minden eszköz ugyanazt a jelszót vagy előre megosztott kulcsot (Pre-Shared Key, PSK) használja, akkor egyetlen kompromittált eszköz kulcsának kiszivárgása az egész flottát veszélyezteti. Ráadásul nehéz megkülönböztetni, hogy egy MQTT-üzenet valóban a `gh1/n001` csomóponttól jött-e, vagy egy támadó csak lejátszotta a PSK-t.

Az eszközidentitás nem csak hitelesítést jelent, hanem azt is, hogy egy adott tanúsítvány vagy kulcs egy adott eszközhöz tartozik, és visszavonható anélkül, hogy a többi eszközt újra kellene kulcsolni. Ezt a problémát a nyilvános kulcsú infrastruktúra (Public Key Infrastructure, PKI) és az X.509 tanúsítványok oldják meg. Ebben az alfejezetben felépítjük a TerraSense labor-CA-ját, kiadunk szerver- és eszköztanúsítványokat, és összevetjük a PSK-alapú identitással. A létrehozott tanúsítványokat a Biztonságos kapcsolat: TLS 1.3, mTLS és DTLS az MQTT/CoAP alatt alfejezet használja a Mosquitto és a kliensek beállításához, a Gyártás és üzembe helyezés (provisioning, onboarding) alfejezet pedig a tanúsítványok eszközre juttatásához.

19.2.2 Fogalmak és működés

19.2.2.1 X.509 tanúsítvány

Definíció. Az X.509 tanúsítvány egy digitális dokumentum, amely egy nyilvános kulcsot egy azonosítóhoz (subject) köt, és amelyet egy hitelesítésszolgáltató (Certificate Authority, CA) ír alá. A tanúsítvány tartalmazza a tulajdonos nevét, a CA nevét, az érvényességi időt, a nyilvános kulcsot és a kiterjesztett mezőket.

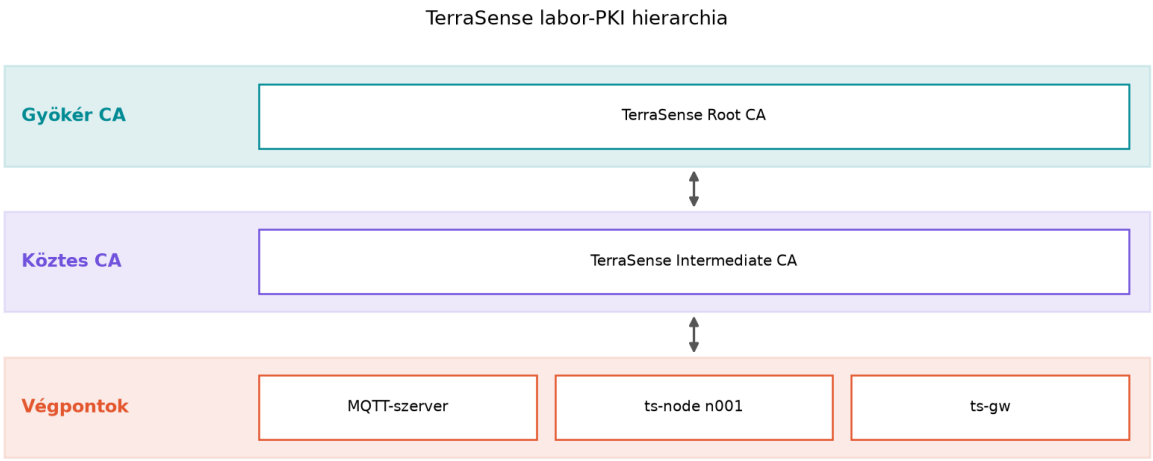
Az X.509 v3 tanúsítvány főbb mezői:

- `Subject`: a tanúsítvány alanya, például `CN=n001.gh1.terrasense`.
- `Issuer`: a kibocsátó CA neve, például `CN=TerraSense Intermediate CA`.
- `Serial Number`: a CA által egyedileg kiosztott sorozatszám; visszavonásnál erre hivatkozunk.
- `Validity`: `notBefore` és `notAfter` időpont; a tanúsítvány csak ebben az intervallumban érvényes.
- `Public Key`: a tanúsítványhoz tartozó nyilvános kulcs (például RSA-2048 vagy ECDSA P-256).
- `Signature`: a fenti mezők feletti aláírás, amelyet a CA privát kulcsával készítenek.
- `Extensions` (kiterjesztések): `BasicConstraints` (CA-e vagy végpont-e), `KeyUsage` (aláírásra, kulcscserére, tanúsítvány-aláírásra), `ExtendedKeyUsage` (serverAuth, clientAuth), `SubjectAltName` (SAN).

A gyakorlatban a `SubjectAltName` a döntő a hoszt azonosításánál. A `Common Name` (CN) ma már csak kompatibilitási okokból létezik; például egy MQTT-szerver tanúsítványnál a `DNS:localhost,IP:127.0.0.1` SAN-nak kell szerepelnie, és nem elég a `CN=mqtt.terrasense.local`.

Tanúsítványlánc. Az eszköz tanúsítványát ritkán írja alá közvetlenül a gyökér CA. A gyökér CA privát kulcsát offline, erősen védett helyen tartják; egy köztes CA végzi a napi kibocsátást. A lánc így néz ki:

- Gyökér CA (root CA): önaláírt, csak a köztes CA-t hitelesíti.
- Köztes CA (intermediate CA): a gyökér által aláírt, kiadja a végponti tanúsítványokat.
- Végponti tanúsítvány (leaf): szerver- vagy kliens-tanúsítvány.



19.2. ábra: TerraSense labor-PKI hierarchia - A gyökér CA csak a köztes CA-t hitelesíti, a köztes CA adja ki a szerver és eszköz tanúsítványokat.

A fogyasztó (például a `ts-node`) a tanúsítványt úgy ellenőrzi, hogy visszafejti a láncot a gyökérig: a végponti tanúsítvány `Issuer` mezője a köztes CA-ra mutat, a köztes `Issuer` mezője a gyökérre, és minden aláírást a kibocsátó nyilvános kulcsával ellenőriz. A gyökér tanúsítványt a fogyasztó megbízható horgonyként tárolja.

19.2.2.2 PKI és visszavonás

Definíció. A PKI (Public Key Infrastructure) a tanúsítványok kiadását, érvényesítését, megújítását és visszavonását szabályozó szervezeti és technikai keretrendszer. A CA-k mellett magában foglalja a regisztrációs hatóságot (RA), a tanúsítványtárakat és a visszavonási listákat.

A visszavonásra két fő mechanizmus létezik:

- **CRL (Certificate Revocation List):** a CA által időszakosan kiadott lista a visszavont tanúsítványok sorozatszámairól. A fogyasztónak le kell töltenie és gyorsítótáraznia a listát.
- **OCSP (Online Certificate Status Protocol):** a fogyasztó valós időben kérdezheti egy tanúsítvány állapotát. OCSP stapling esetén a szerver a saját tanúsítványa mellé csatolja a CA által aláírt, rövid élettartamú állapotválaszt, így a fogyasztónak nem kell közvetlenül a CA-t kérdeznie.

Az IoT-eszközök esetében mindkét módszernek komoly korlátai vannak: az eszköz gyakran nem rendelkezik megbízható idővel, a CRL túl nagy lehet a memóriához, és az OCSP-kérés késlelteti a kapcsolatfelvételt. A TerraSense esetében ezért rövid érvényességű eszköztanúsítványokat (például 2 év) és az éles rendszerben OCSP staplinget vagy rövid élettartamú, automatikusan megújuló tanúsítványokat érdemes használni.

19.2.2.3 PSK vs. tanúsítvány

Definíció. A PSK (Pre-Shared Key) egy előre megosztott szimmetrikus kulcs, amelyet mindkét fél ismer. A PSK-alapú azonosításnál a fél bizonyítja, hogy birtokában van a kulcsnak, például TLS-PSK kézfogásban.

Kétféle PSK-elrendezést kell megkülönböztetni. **Közös PSK** esetén minden eszköz ugyanazt a kulcsot használja: ez egyszerű és számításilag olcsó, de egyetlen eszköz kiszivárgása az egész flottát kompromittálja, és a kulcs cseréje minden eszközt érint. **Egyedi PSK** esetén minden eszköznek saját kulcsa van: egy kompromittált eszköz visszavonható anélkül, hogy a többit újra kellene kulcsolni, és az egyedi azonosítás is megvalósítható. Az egyedi PSK hátránya az üzemeltetési teher: a kulcsokat ki kell osztani, nyilván kell tartani és rotálni kell, a szerveroldali kulcsadatbázis pedig nagy értékű célpont; ezen kívül a szerver hitelesítése is a megosztott titkon alapul.

A tanúsítvány alapú identitás aszimmetrikus: minden eszköznek saját privát kulcsa és egyedi tanúsítványa van. Ha egy eszköz kompromittálódik, a tanúsítványát visszavonjuk, anélkül, hogy a többi eszköz kulcsát módosítani kellene. A tanúsítvány alapú TLS (mTLS) további előnye, hogy a szerver is azonosítja magát, így az eszköz nem csatlakozik hamis brokerhez. A tanúsítvány hátránya a PKI-üzemeltetés: a CA, a kiadás, a megújítás, az érvényességi idő és a visszavonás kezelése.

A három lehetőség összevetése ugyanazon szempontok mentén:

Szempont	Közös PSK	Egyedi PSK	Tanúsítvány
Kulcsméret / CPU-igény	kicsi, szimmetrikus	kicsi, szimmetrikus	nagyobb, aszimmetrikus
Egyedi azonosítás	nem	igen, kulcsonként	igen, tanúsítványonként
Egy eszköz kompromittálódása	az egész flotta	az adott eszköz	az adott eszköz
Visszavonás	minden eszköz újrakulcsolása	a kulcs törlése a szerverről	CRL/OCSP vagy rövid élettartam
Rotáció	minden eszközön	eszközönként, üzemeltetési teherrel	tanúsítvány megújítása
Skálázás	jó, de mindenki ugyanazt a titkot ismeri	közepes, kulcsnyilvántartás kell	jó, a CA skálázza
Infrastruktúra	nincs	kulcsnyilvántartás	CA, kiadás, megújítás
Szerverhitelesítés	a közös titok alapján	a megosztott titok alapján	tanúsítványlánc
Offline hitelesítés	nem lehetséges	nem lehetséges	tanúsítványlánc ellenőrizhető

Az egyedi PSK tehát nem azonos a közös PSK-val: megfelelő kulcskezeléssel az egyedi PSK is biztosíthat egyedi eszközidentitást és korlátozott kompromittálódást. A tanúsítvány nem önmagában „biztonságosabb” választás, hanem akkor előnyös, ha a flotta nagy, a szerverhitelesítés és a visszavonás követelmény, és a PKI üzemeltetése fenntartható. A TerraSense-ben azért választjuk a tanúsítványt, mert a flotta mérete és a visszavonási igény mellett a közös titok nélküli szerverhitelesítés és a szabványos tanúsítvány-életciklus nagyobb üzembiztonságot ad.

19.2.2.4 Kulcstárolás

Definíció. A kulcstárolás az eszköz privát kulcsának védelmét jelenti a kiolvasás, másolás és illetéktelen használat ellen. Az IoT-eszközökön a privát kulcs védelme kritikus, mert a támadó fizikailag hozzáférhet az eszközhöz.

- **Szoftveres tárolás (plaintext):** a kulcs a flash-en vagy a fájlrendszerben, titkosítatlanul van. Ez a legegyszerűbb, de fizikai hozzáféréssel kinyerhető. Laborban megfelelő, élesben kerülendő.
- **Secure Element (SE):** különálló, illetéktelen behatolás ellen védett chip, amely saját belső kulcsgenerálással és tárolással rendelkezik. A privát kulcs soha nem hagyja el a chipet; a műveletek (aláírás, TLS kézfogás) a chipen belül futnak.
- **TPM (Trusted Platform Module):** szabványosított biztonságos kriptoprocesszor, amely hasonló védelmet nyújt, és gyakran távoli igazolást (remote attestation) is támogat. Linux-alapú gatewayekben gyakori.
- **TrustZone / PSA:** ARM platformokon a TrustZone hardveresen elválaszt egy biztonságos világot a normál operációs rendszertől. A PSA (Platform Security Architecture) ezt a modellt szabványosítja, és tanúsítja a megvalósítást. A Zephyr is támogatja a PSA API-t.

A TerraSense `native_sim` környezetben természetesen szoftveres kulcstárolást használunk, de a valós hardver opcionális kiterjesztésénél legalább egy SE vagy TrustZone-alapú védelmet kell tervezni.

19.2.3 Döntési szempontok

A TerraSense-ben a PSK helyett tanúsítvány alapú azonosítást választunk, mert:

1. A flotta több tíz vagy száz eszközből áll. Az egyedi PSK is lehetséges, de a kulcsok kiadása, nyilvántartása és rotációja nagyobb operatív terhet jelent, és a szerveroldali kulcsadatbázis kompromittálódása az egész flottát érintené.
2. Az irányadó biztonsági elvárások (ETSI EN 303 645, Cyber Resilience Act) az egyedi eszközidentitást és a sebezhetőségek kezelését hangsúlyozzák; a tanúsítvány-életciklus ezt jobban támogatja.
3. A Mosquitto és a Paho kliens is jól támogatja az X.509-alapú mTLS-t.
4. A tanúsítványok visszavonása és megújítása szabványos (CRL/OCSP, rövid élettartam), a szerver hitelesítése pedig nem egy megosztott titkon alapul.

A tanúsítványok érvényességi idejét rövidre vesszük: a laborban 825 nap (2 év + puffer), éles rendszerben 6–12 hónap. A CA hierarchia legyen kétszintű: a gyökér offline, a köztes CA végzi a kiadást.

A privát kulcsok tárolására a `native_sim` környezetben szoftveres tárolást használunk, de a `ts-gw` valós hardverén TPM-et, a `ts-node`-on pedig SE-t vagy TrustZone-t érdemes.

19.2.4 Kidolgozott példa és labor

Munkakönyvtár: `~/iot-labor/i056`

```
umask 077
mkdir -p ~/iot-labor/i056 && cd ~/iot-labor/i056
mkdir -p ca certs
chmod 700 ca certs
printf '%s\n' '*.key' '*.srl' > .gitignore
```

A `umask 077` biztosítja, hogy az újonnan létrehozott kulcsfájlok csak a tulajdonos számára legyenek olvashatók (600), a könyvtárak pedig 700-as módot kapjanak. A `umask` csak az adott shellre érvényes; új terminálban ismételje meg, vagy ellenőrizze a jogosultságokat a `stat -c '%a %n' ca/*.key certs/*.key` paranccsal. A `.gitignore` megakadályozza, hogy a privát kulcsok véletlenül a laborrepóba kerüljenek. A CA privát kulcsát soha ne csatoljuk a Mosquitto vagy a backend konténerébe: a szolgáltatás csak a saját szerverkulcsát és a tanúsítványláncát kapja meg. Éles rendszerben a gyökér CA kulcsa offline, hardveres védelemmel (HSM vagy a hálózatról leválasztott gép) tárolandó; a labor kulcsai eldobhatók, és nem használhatók éles rendszerben.

A labor célja, hogy létrehozzuk a TerraSense labor-CA-ját, kiadjunk egy MQTT-szerver és egy `ts-node` eszköztanúsítványt, valamint ellenőrizzük a láncokat. A példában ECDSA P-256 kulcsokat használunk, mert a TLS 1.3-ban is támogatott, kisebb kulcs- és aláíráshosszal jár, mint az RSA, ami IoT-eszközöknél előnyös.

19.2.4.1 1. Gyökér CA

```
# Gyökér CA privát kulcs (ECDSA P-256)
openssl ecparam -genkey -name prime256v1 -out ca/root-ca.key

# Önaláírt gyökér tanúsítvány, 10 év
openssl req -x509 -new -key ca/root-ca.key -sha256 -days 3650 \
  -subj "/C=HU/O=TerraSense Lab/CN=TerraSense Root CA" \
  -addext "basicConstraints=critical,CA:TRUE" \
  -addext "keyUsage=critical,keyCertSign,cRLSign" \
  -out ca/root-ca.crt
```

A gyökér tanúsítványt a `req -x509` állítja ki, amely támogatja az `-addext` kapcsolót. A köztes és a végponti tanúsítványokat viszont az `x509 -req` írja alá, amelynél az `-addext` nem használható: a kiterjesztéseket külön konfigurációs fájlból, az `-extfile` és az `-extensions` kapcsolóval kell megadni.

19.2.4.2 2. Köztes CA

```
# Köztes CA privát kulcs
openssl ecparam -genkey -name prime256v1 -out ca/intermediate-ca.key

# Tanúsítványkérés (CSR)
openssl req -new -key ca/intermediate-ca.key \
  -subj "/C=HU/O=TerraSense Lab/CN=TerraSense Intermediate CA" \
  -out ca/intermediate-ca.csr

# A köztes CA kiterjesztései külön fájlban; az x509 -req nem ismeri az -addext-et
cat > ca/intermediate-ca.ext <<'EOF'
[v3_ca]
basicConstraints = critical,CA:TRUE,pathlen:0
keyUsage = critical,keyCertSign,cRLSign
EOF

# Aláírás a gyökérrel; pathlen:0 azt jelenti, hogy további CA-t nem adhat ki
openssl x509 -req -in ca/intermediate-ca.csr \
  -CA ca/root-ca.crt -CAkey ca/root-ca.key -CAcreateserial \
  -days 1825 -sha256 \
  -extfile ca/intermediate-ca.ext -extensions v3_ca \
  -out ca/intermediate-ca.crt
```

19.2.4.3 3. Szerver tanúsítvány (Mosquitto broker)

```
# Szerver privát kulcs
openssl ecparam -genkey -name prime256v1 -out certs/server.key

# CSR, SAN-nal (a CSR-ben ez még csak kérés; a kiadott tanúsítvány kiterjesztéseit az extfile
↪  adja)
openssl req -new -key certs/server.key \
  -subj "/C=HU/O=TerraSense Lab/CN=mqtt.terrasense.local" \
  -addext "subjectAltName=DNS:localhost,IP:127.0.0.1" \
  -out certs/server.csr

# A szerver kiterjesztései: SAN, serverAuth, CA:FALSE
cat > certs/server.ext <<'EOF'
[v3_server]
basicConstraints = critical,CA:FALSE
keyUsage = critical,digitalSignature
extendedKeyUsage = serverAuth
subjectAltName = DNS:localhost,IP:127.0.0.1
EOF

# Aláírás a köztes CA-val; a kiterjesztések az extfile-ból jönnek
openssl x509 -req -in certs/server.csr \
  -CA ca/intermediate-ca.crt -CAkey ca/intermediate-ca.key -CAcreateserial \
  -days 825 -sha256 \
  -extfile certs/server.ext -extensions v3_server \
  -out certs/server.crt
```

19.2.4.4 4. Eszköztanúsítvány a `ts-node` számára

```
# Eszköz privát kulcs
openssl ecparam -genkey -name prime256v1 -out certs/device-001.key

# CSR; a CN a TerraSense node azonosítója
openssl req -new -key certs/device-001.key \
  -subj "/C=HU/O=TerraSense Lab/CN=n001.gh1.terrasense" \
  -out certs/device-001.csr

# Az eszköz kiterjesztései: clientAuth, CA:FALSE
cat > certs/device-001.ext <<'EOF'
[v3_client]
basicConstraints = critical,CA:FALSE
keyUsage = critical,digitalSignature
extendedKeyUsage = clientAuth
EOF

# Aláírás a köztes CA-val; a kiterjesztések az extfile-ból jönnek
openssl x509 -req -in certs/device-001.csr \
  -CA ca/intermediate-ca.crt -CAkey ca/intermediate-ca.key -CAcreateserial \
  -days 825 -sha256 \
  -extfile certs/device-001.ext -extensions v3_client \
  -out certs/device-001.crt
```

19.2.4.5 5. Láncok összefűzése

A kliensek és a broker gyakran egyetlen fájlban várják a láncot: a végponti tanúsítványt, majd a köztes CA tanúsítványát.

```
cat certs/server.crt ca/intermediate-ca.crt > certs/server-chain.pem
cat certs/device-001.crt ca/intermediate-ca.crt > certs/device-001-chain.pem
```

19.2.4.6 6. Ellenőrzés

```
# Lánc ellenőrzése a gyökér megbízható horgonyával
openssl verify -CAfile ca/root-ca.crt -untrusted ca/intermediate-ca.crt \
  certs/device-001.crt
openssl verify -CAfile ca/root-ca.crt -untrusted ca/intermediate-ca.crt \
  certs/server.crt

# Alapinformációk
openssl x509 -in certs/device-001.crt -noout -subject -issuer -dates
openssl x509 -in certs/server.crt -noout -ext subjectAltName,extendedKeyUsage

# A privát kulcsok jogosultsága (elvárt: 600)
stat -c '%a %n' ca/*.key certs/*.key

# Ha a labor Git-repóban van, a kulcsok ne jelenjenek meg a státuszban
git status --short 2>/dev/null | grep '\.key$' || echo "nincs kulcs a git státuszban"
```

A várt kimenet az `openssl verify` parancsoknál:

```
certs/device-001.crt: OK
certs/server.crt: OK
```

A tanúsítványok adataiból látszik, hogy a kibocsátó a köztes CA, a tárgy pedig a megfelelő név. A szerver tanúsítvány SAN-ja `DNS:localhost` és `IP:127.0.0.1`, a kiterjesztett kulcshasználat `TLS Web Server Authentication`. A `stat` kimenetében minden kulcsnál `600` szerepel; a `git status` pedig nem listázza a `*.key` fájlokat. Ha mégis megjelennek, a `.gitignore` hiányos, vagy a fájl már verziókövetés alatt áll; utóbbi esetben a kulcsot cserélni kell, mert a Git-előzményből nem tüntethető el megnyugtatóan.

19.2.5 Ellenőrzés és hibakeresés

A labor eredményének ellenőrzéséhez a fenti `openssl verify` és `openssl x509` parancsok elegendők. A későbbi alfejezetekben a Mosquitto úgy lesz konfigurálva, hogy a `certs/server-chain.pem` és `certs/server.key` fájlokat, a kliensek pedig a `certs/device-001-chain.pem` és `certs/device-001.key` fájlokat használják.

Hibatünet	Ok	Javítás
`verify error: unable to get local issuer certificate`	A köztes CA tanúsítványa nincs megadva `-untrusted` opcióval, vagy a láncfájl hiányos	Adja meg a köztes CA-t `-untrusted`, vagy fűzze össze a láncfájlt
`verify error: certificate has expired`	A rendszer idő kívül esik a tanúsítvány érvényességi intervallumán	Ellenőrizze a `notBefore`/`notAfter` mezőket, állítsa be a rendszeridőt
`X509_V_ERR_INVALID_CA`	A láncban nem CA tanúsítvány szerepel CA-ként, vagy a `BasicConstraints` hiányzik	Ellenőrizze a `basicConstraints=critical,CA:TRUE` kiterjesztést

Hibatünet	Ok	Javítás
A Mosquitto nem fogadja el a tanúsítványt	A szerver tanúsítványban nincs `serverAuth` EKU, vagy a SAN nem egyezik a hosztnévvel	Adja hozzá a `extendedKeyUsage=serverAuth` és a megfelelő SAN bejegyzést

---

19.2.6 Tipikus hibák és félreértések

1. **A CN-t használják hosztnév-ellenőrzésre.** A modern TLS-kliensek a `SubjectAltName` mezőt ellenőrzik; ha a tanúsítványban nincs SAN, a kapcsolat hibás lesz, még akkor is, ha a CN egyezik.
2. **A teljes lánc elküldésének elmulasztása.** A kliens csak a végponti tanúsítványt küldi, a köztes CA-t nem. A szerver nem tudja felépíteni a láncot a gyökérig, és elutasítja a kapcsolatot. A láncfájlban a végponti tanúsítványt és a köztes CA tanúsítványát is szerepeltetni kell.
3. **A CA privát kulcs szoftveres tárolása éles rendszerben.** Ha a gyökér CA privát kulcsát egy hálózatba kapcsolt gépen tartják, a teljes PKI biztonsága megszűnik. A gyökeret offline, hardveres védelemmel kell tárolni, és a CA kulcsát soha nem szabad a szolgáltatáskonténerbe csatolni: a Mosquitto csak a saját szerverkulcsát és láncát kapja meg.
4. **Folyamatos OCSP-kapcsolat elvárása IoT-eszköztől.** Az eszközök gyakran offline működnek, és nem tudnak minden kézfogásnál OCSP-t kérdezni. Ilyenkor rövid élettartamú tanúsítvány vagy OCSP stapling a helyes megoldás, nem a folyamatos online ellenőrzés.
5. **A privát kulcsok jogosultságának és verziókövetésének elhanyagolása.** A laborban a kulcsok `umask 077` nélkül a többi felhasználó számára is olvashatók lehetnek, és a `.gitignore` hiányában véletlenül a repóba kerülhetnek. A kulcsfájlok jogosultsága 600, a kulcskönyvtáraké 700 legyen; a `*.key` fájlokat vegyük fel a `.gitignore`-ba, és commit előtt ellenőrizzük a `git status` kimenetét. A labor kulcsai nem azonosak az éles kulcsokkal, és nem használhatók éles rendszerben.

19.2.7 Feladatok

19.2.7.1 (a) Fogalmi kérdés

Magyarázza meg, miért jobb a TerraSense esetében a tanúsítvány alapú azonosítás a közös PSK-nál, ha a flotta 1000 eszközből áll? Hasonlítsa össze az egyedi PSK lehetőségével is: milyen üzemeltetési és biztonsági különbségek vannak?

19.2.7.2 (b) Tervezési feladat

Egy tanúsítvány érvényessége 825 nap. Ha a CA minden hónapban kiad egy CRL-t, és a CRL mérete 2000 visszavont tanúsítvány esetén 100 KiB, akkor mekkora CRL-t kell letöltenie egy eszköznek évente, ha havonta frissít? Hogyan csökkenthető ez az adatmennyiség?

19.2.7.3 (c) Implementációs feladat

Hozzon létre egy második eszköztanúsítványt a `ts-gw` gateway számára a laborban megismert módon. A CN legyen `gw.gh1.terrasense`, az EKU pedig `clientAuth`. Ellenőrizze a láncot a gyökér CA-val.

19.2.7.4 (d) Hibakeresési feladat

Távolítsa el a köztes CA tanúsítványt a `device-001` láncfájlból, majd futtassa az `openssl verify` parancsot a gyökérrel. Milyen hibát kap? Magyarázza meg, miért nem elég a gyökér CA a lánc felépítéséhez.

19.2.8 Megoldások

19.2.8.1 (a)

A közös PSK esetén egyetlen kiszivárgott kulcs az egész flottát érinti, és a kulccsere minden eszköz frissítését igényli. A tanúsítvány alapú identitásnál minden eszköznek saját privát kulcsa és tanúsítványa van; egy kompromittált eszközt a tanúsítvány visszavonásával lehet kizárni, a többi eszköz változatlanul működik. A tanúsítványokkal a szerver is hitelesíthető, és a mTLS mindkét irányú bizalmat nyújt. Az egyedi PSK szintén biztosíthat egyedi identitást, de a kulcsok nyilvántartása és rotációja nagyobb üzemeltetési terhet jelent, és a szerveroldali kulcsadatbázis kompromittálódása az egész flottát érintené; a tanúsítvány előnye a szabványos visszavonás és a szerverhitelesítés.

19.2.8.2 (b)

Az eszköz havonta letölt 100 KiB CRL-t, ami évente $\frac{12 \times 100\,\mathrm{KiB}}{1} = 1200\,\mathrm{KiB}$ adatforgalmat jelent. Ez csökkenthető delta-CRL-vel, OCSP staplinggel, vagy rövid élettartamú tanúsítványokkal, ahol nincs szükség nagy CRL-ekre.

19.2.8.3 (c)

```
cd ~/iot-labor/i056

cat > certs/gw.ext <<'EOF'
[v3_client]
basicConstraints = critical,CA:FALSE
keyUsage = critical,digitalSignature
extendedKeyUsage = clientAuth
EOF

openssl ecparam -genkey -name prime256v1 -out certs/gw.key
openssl req -new -key certs/gw.key \
  -subj "/C=HU/O=TerraSense Lab/CN=gw.gh1.terrasense" \
  -out certs/gw.csr
openssl x509 -req -in certs/gw.csr \
  -CA ca/intermediate-ca.crt -CAkey ca/intermediate-ca.key -CAcreateserial \
  -days 825 -sha256 \
  -extfile certs/gw.ext -extensions v3_client \
  -out certs/gw.crt

cat certs/gw.crt ca/intermediate-ca.crt > certs/gw-chain.pem
openssl verify -CAfile ca/root-ca.crt -untrusted ca/intermediate-ca.crt certs/gw.crt
```

A várt kimenet: `certs/gw.crt: OK`.

19.2.8.4 (d)

Ha a `gw-chain.pem`-ből elhagyjuk a köztes CA tanúsítványt, és csak az eszköztanúsítványt ellenőrizzük a gyökerrel, az `openssl verify` hibát ad: `unable to get local issuer certificate`. Ennek oka, hogy a gyökér CA közvetlenül nem írta alá az eszköz tanúsítványát; a lánc felépítéséhez szükség van a köztes CA tanúsítványára is, amelynek aláírását a gyökérrel ellenőrizhetjük.

19.2.9 Rövid összefoglaló

A TerraSense eszközidentitás alapját a kétszintű labor-PKI adja: a gyökér CA (`ca/root-ca.crt`, `ca/root-ca.key`) csak a köztes CA-t hitelesíti, a köztes CA (`ca/intermediate-ca.crt`, `ca/intermediate-ca.key`) pedig a végponti tanúsítványokat. Létrehoztuk a Mosquitto szerver tanúsítványát (`certs/server.crt`, SAN: `localhost`, `127.0.0.1`, EKU: `serverAuth`) és a `ts-node` eszköztanúsítványt (`certs/device-001.crt`, EKU: `clientAuth`), valamint a láncfájlokat (`certs/server-chain.pem`, `certs/device-001-chain.pem`). A tanúsítvány alapú identitás a közös PSK-nál skálázhatóbb és biztonságosabb; egyedi PSK-kal összevetve a fő előny a szabványos visszavonás és a szerverhitelesítés, nem a szimmetrikus kulcs önmagában. A privát kulcsokat élesben secure element, TPM vagy TrustZone/PSA védelemmel kell tárolni. A labor-CA és a tanúsítványok továbbvitt eredményként szolgálnak a Biztonságos kapcsolat: TLS 1.3, mTLS és DTLS az MQTT/CoAP alatt és a Gyártás és üzembe helyezés (provisioning, onboarding) alfejezetekhez.

19.2.10 Források és ellenőrzési hiányok

- X.509 PKI: RFC 5280.
- TLS 1.3: RFC 8446.
- DTLS 1.3: RFC 9147.
- PSA Certified: https://www.psacertified.org/ (a tanúsítási program részleteinek ellenőrzése szükséges).
- Az OpenSSL parancsok az OpenSSL 3.x verziójára vonatkoznak; az opciónevek és a pontos szintaxis ellenőrzése a telepített verzió dokumentációjában javasolt.
- A TPM és TrustZone/PSA API-k verzió- és implementációfüggő részletei ellenőrizendők.

19.3 Biztonságos kapcsolat: TLS 1.3, mTLS és DTLS az MQTT/CoAP alatt

19.3.1 Motiváció

A TerraSense jelenleg nyílt, hitelesítés nélküli MQTT-kapcsolatot használ: a `ts-node`, a `ts-gw` és a `ts-backend` az MQTT a gyakorlatban alfejezetben felépített módon a broker 1883-as portján, felhasználónév és titkosítás nélkül kommunikál. Ez egy valós telepítésen két azonnali problémát jelent:

1. **A telemetriaadatok lehallgathatók**, mert a TCP-kapcsolat tartalma olvasható marad a hálózati útvonalon. A hőmérséklet- és páratartalom-adatok elsőre nem tűnnek érzékenynek, de az üvegház működéséből következtetni lehet a termelési folyamatra, a jelenléti státuszból pedig arra, hogy mikor dolgozik valaki a létesítményben.
2. **Bárki csatlakozhat a brokerhez**, és hamis telemetriát publikálhat vagy érzékeny parancsokat küldhet a szelepnek. A hitelesítés hiánya nem csupán adatszivárgást, hanem aktív beavatkozást is lehetővé tesz.

A Kriptográfiai alapok fejlesztőknek és az Eszközidentitás: X.509-tanúsítványok, PKI és kulcstárolás alfejezetben felépítettük a TerraSense kétszintű, ECDSA P-256 kulcsokra épülő labor-PKI-jét: a gyökér CA-t (`ca/root-ca.crt`), a köztes CA-t (`ca/intermediate-ca.crt`), a broker szervertanúsítványát (`certs/server.crt`) és a `ts-node` eszköztanúsítványát (`certs/device-001.crt`). Ebben az alfejezetben erre a PKI-re építve állítjuk át a teljes MQTT-láncot kölcsönös TLS-hitelesítésre (mutual TLS, mTLS): a `ts-gw` és a `ts-backend` tanúsítvány alapján azonosítja magát, a Mosquitto pedig a tanúsítvány identitásához köti a topic-szintű jogosultságokat. Nem hozunk létre új CA-t: a tanúsítványokat továbbra is az Eszközidentitás: X.509-tanúsítványok, PKI és kulcstárolás alfejezetben létrehozott köztes CA írja alá, így a lánc, a fájlnevek és az identitások az egész könyvben egységesek maradnak.

A cél nem csupán a titkosítás, hanem az **eszközidentitás** kialakítása, amelyet a későbbi provisioning és onboarding, valamint a záróprojekt közvetlenül felhasznál.

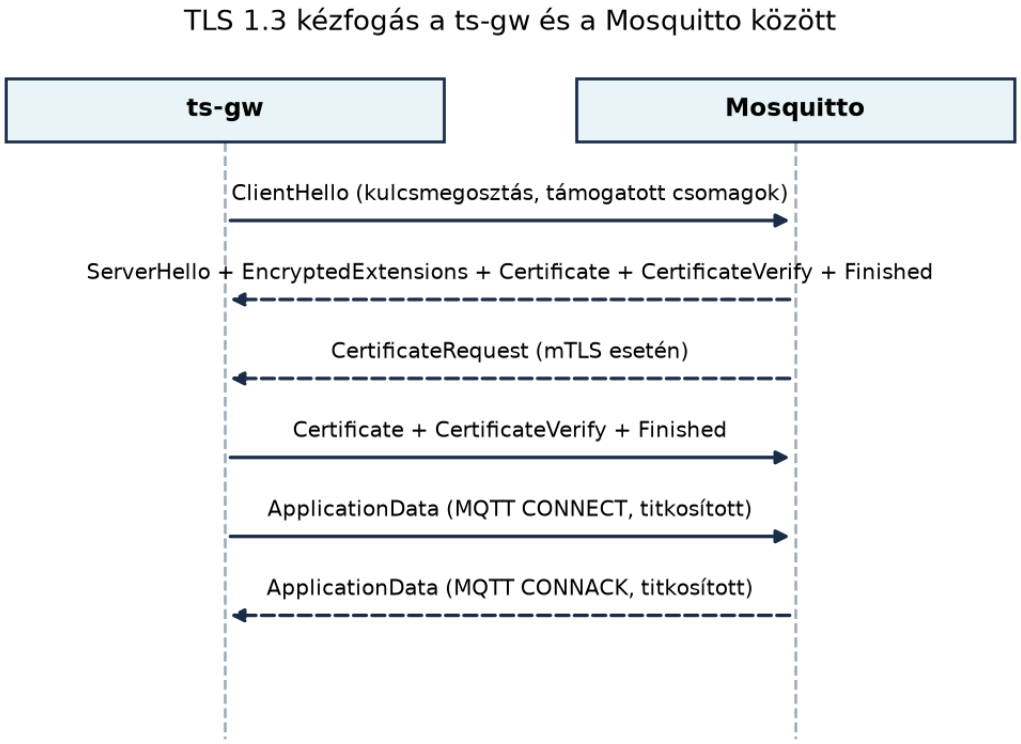
19.3.2 Fogalmak és működés

Definíció. A **Transport Layer Security (TLS)** egy olyan kriptográfiai protokoll, amely megbízható átviteli réteg (általában TCP) fölött biztosít titkosságot, integritást és hitelesítést. A TLS 1.3-at az RFC 8446 definiálja. A **kölcsönös TLS (mutual TLS, mTLS)** a TLS olyan változata, amelyben nemcsak a szerver, hanem a kliens is tanúsítvánnyal igazolja magát; a kliens

oldali tanúsítványt a szerver ugyanazzal a láncvalidálással ellenőrzi, mint a szerver tanúsítványát a kliens.

Definíció. A **tanúsítvány-ellenőrzés** során a fogadó fél a bemutatott X.509-tanúsítványt a következők szerint validálja: a tanúsítvány aláírása láncolatban visszavezethető egy megbízható gyökértanúsítványhoz (CA), a tanúsítvány érvényességi ideje aktuális, a tanúsítványban szereplő név megegyezik a kapcsolódási céllal (hosztnév vagy identitás), a tanúsítvány rendeltetése (a kiterjesztett kulcshasználat, EKU) illeszkedik a használathoz, és a tanúsítványt nem vonták vissza. Ha bármelyik ellenőrzés sikertelen, a kézfogás megszakad.

A TLS 1.3 kézfogás rövidebb, mint a korábbi 1.2-es változaté, mert a kliens az első üzenetben elküldi a kulcsmegosztási paramétereit, és a szerver egyetlen lépésben válaszol a saját kulcsmegosztásával és tanúsítványláncával. Üzenetsorrend-diagramon ez a következőképpen néz ki a `ts-gw` és a Mosquitto között:



19.3. ábra: TLS 1.3 kézfogás a ts-gw és a Mosquitto között - Az mTLS miatt a kliens is tanúsítványt mutat be; a Finished üzenetek után a payload titkosított.

A kézfogás során a szerver a `CertificateRequest` üzenettel kéri a kliens tanúsítványát. A `ts-gw` a saját tanúsítványát és a hozzá tartozó privát kulcs aláírását küldi el a `CertificateVerify` üzenetben, ezzel bizonyítja, hogy a privát kulcs birtokában van. A `Finished` üzenetek után minden adat, beleértve az MQTT `CONNECT` és `PUBLISH` csomagokat is, titkosított rekordokban halad.

19.3.2.1 A tanúsítvány mint engedélyezési alap

A mTLS nem csupán hitelesítésre alkalmas: a Mosquitto a bemutatott kliens tanúsítvány `Common Name` (CN) mezőjét felhasználhatja a **topic-szintű jogosultságok** (ACL) eldöntésére.

A `use_identity_as_username true` beállítás hatására a broker a kliens tanúsítványának CN-jét használja felhasználónévként, így az ACL-fájl `user` sorai a tanúsítvány identitására illeszkednek. A TerraSense-ben a CN site-hoz kötött identitás:

Szolgáltatás	Tanúsítvány CN-je	Szerep
`ts-gw` (gh1 gateway)	`gw.gh1.terrasense`	a gh1 node-jainak telemetriája, állapota és státusza; parancsok olvasása és eredményük írása
`ts-backend`	`backend.terrasense`	minden site adatának olvasása; parancsok kiadása
`ts-node` (n001)	`n001.gh1.terrasense`	eszköztanúsítvány a gateway felé; a broker felé a gateway közvetít

A **Subject Alternative Name (SAN)** ezzel szemben a hosztnév-ellenőrzéshez kell: a szervertanúsítvány SAN-ja `DNS:localhost` és `IP:127.0.0.1`, ezért a kliensek `localhost` néven ellenőrzik a brokert. Az ACL-hez nem a SAN-t, hanem a CN-t használjuk, mert a SAN kliens oldali tanúsítványnál nem kötelező, a CN viszont a TerraSense identitás hordozója. A `%u` helyettesítő a Mosquitto ACL-fájlban a felhasználónevet (tehát itt a CN-t) jelentené, a TerraSense ACL mégis explicit site-mintákat használ, mert így a jogosultságok site-onként olvashatók és auditálhatók.

19.3.2.2 DTLS korlátos eszközökhöz

A TLS feltételezi a megbízható TCP-átvitelt. Azokon a korlátos eszközökön, amelyek UDP-alapú CoAP-ot használnak, a **Datagram TLS (DTLS)** biztosít hasonló védelmet. A DTLS a TLS kézfogást datagram-alapú átvitelre adaptálja: újraküldési időzítőkkel, sorrendszámokkal és datagram-újraösszeállítással kezeli a csomagvesztést és a duplikációt. A DTLS 1.3 az RFC 9147-ben, a DTLS 1.2 az RFC 6347-ben van meghatározva. A `ts-node` opcionális CoAP-alapú konfigurációjában DTLS-t használhatnánk, de a jelen labor az MQTT mTLS-re összpontosít, mert a TerraSense elsődleges kommunikációs lánca MQTT-alapú.

Erőforrásigény. A TLS 1.3 kézfogás jellemzően néhány kilobájtnyi adatforgalmat jelent, és a modern mikrokontrollereken a kriptográfiai műveletek hardveres gyorsítással vagy szoftveres megvalósítással elvégezhetők. A `native_sim` környezetben nem mérünk valós fogyasztást; a tényleges eszközön a TLS-kézfogás energiaköltsége a kézfogások gyakoriságának csökkentésével (hosszú munkamenet, session resumption) optimalizálható.

19.3.3 Döntési szempontok

Szempont	Titkosítatlan MQTT	Szerveroldali TLS	mTLS
Kliens hitelesítése	nincs	felhasználónév/jelszó vagy nincs	tanúsítvány, erős identitás
Topic-szintű ACL alapja	IP vagy felhasználónév	felhasználónév	tanúsítvány CN/SAN
Kulcskezelés bonyolultsága	nincs	szerver oldali	minden eszköznek saját kulcs és tanúsítvány

Szempont	Titkosítatlan MQTT	Szerveroldali TLS	mTLS
Javasolt alkalmazás	fejlesztői teszt	kísérleti rendszer, belső hálózat	termelési TerraSense

A TerraSense termelési célja egyértelműen az mTLS. A nyílt portot a laborban lezárjuk, hogy véletlenül se maradjon hitelesítetlen belépési pont. A `ts-node` eszközökön opcionálisan szintén bevezethető az mTLS, de a nagyon korlátos hardveren a DTLS vagy a szimmetrikus kulcsos hitelesítés megfontolható; ezt a provisioning témakörében vizsgáljuk tovább.

19.3.4 Labor: mTLS a Mosquittón, Paho C++ kliens és Spring Boot beállítás

Munkakönyvtár: `~/iot-labor/i057`

A labor négy fő lépésből áll: az Eszközidentitás: X.509-tanúsítványok, PKI és kulcstárolás alfejezetben létrehozott PKI kiegészítése a gateway- és backend-tanúsítvánnyal, a visszavonási lista előkészítése, a Mosquitto konfigurációjának átállítása mTLS-re, majd a `ts-gw` (Paho C++ 1.3.2) és a `ts-backend` (Paho MQTT 5 Java kliens) kliensoldali beállítása. A tanúsítványokat nem hozzuk létre újra: a gyökér- és köztes CA, valamint a szervertanúsítvány az Eszközidentitás: X.509-tanúsítványok, PKI és kulcstárolás laborból származik.

19.3.4.1 1. A PKI kiegészítése a gateway- és backend-tanúsítvánnyal

Először ellenőrizzük, hogy az Eszközidentitás: X.509-tanúsítványok, PKI és kulcstárolás PKI-ja rendelkezésre áll. Ha a gyökér vagy a köztes CA hiányzik, előbb az Eszközidentitás: X.509-tanúsítványok, PKI és kulcstárolás 1–2. lépését kell újrafuttatni.

```
mkdir -p ~/iot-labor/i057 && cd ~/iot-labor/i057
umask 077
test -r "$HOME/iot-labor/i056/ca/root-ca.crt"
test -r "$HOME/iot-labor/i056/ca/intermediate-ca.crt"
test -r "$HOME/iot-labor/i056/certs/server-chain.pem"
test -r "$HOME/iot-labor/i056/certs/server.key"
```

A `ts-gw` tanúsítványa az Eszközidentitás: X.509-tanúsítványok, PKI és kulcstárolás (c) feladatából származik. Ha a gyakorlatot még nem végeztük el, a következő blokk létrehozza; ha már megvan, a fájlok érintetlenül maradnak.

```
cd "$HOME/iot-labor/i056"
if [ ! -r certs/gw-chain.pem ]; then
  cat > certs/gw.ext <<'EOF'
[v3_client]
basicConstraints = critical,CA:FALSE
keyUsage = critical,digitalSignature
extendedKeyUsage = clientAuth
EOF
  openssl ecparam -genkey -name prime256v1 -out certs/gw.key
  openssl req -new -key certs/gw.key \
    -subj "/C=HU/O=TerraSense Lab/CN=gw.gh1.terrasense" \
    -out certs/gw.csr
  openssl x509 -req -in certs/gw.csr \
    -CA ca/intermediate-ca.crt -CAkey ca/intermediate-ca.key -CAcreateserial \
    -days 825 -sha256 \
    -extfile certs/gw.ext -extensions v3_client \
    -out certs/gw.crt
```

```
  cat certs/gw.crt ca/intermediate-ca.crt > certs/gw-chain.pem
fi
```

A `ts-backend` tanúsítványa új: a CN `backend.terrasense`, az EKU `clientAuth`. A köztes CA-val írjuk alá, és a láncfájlba a végponti tanúsítvány mellé a köztes CA kerül.

```
cd "$HOME/iot-labor/i056"
cat > certs/backend.ext <<'EOF'
[v3_client]
basicConstraints = critical,CA:FALSE
keyUsage = critical,digitalSignature
extendedKeyUsage = clientAuth
EOF
openssl ecparam -genkey -name prime256v1 -out certs/backend.key
openssl req -new -key certs/backend.key \
  -subj "/C=HU/O=TerraSense Lab/CN=backend.terrasense" \
  -out certs/backend.csr
openssl x509 -req -in certs/backend.csr \
  -CA ca/intermediate-ca.crt -CAkey ca/intermediate-ca.key -CAcreateserial \
  -days 825 -sha256 \
  -extfile certs/backend.ext -extensions v3_client \
  -out certs/backend.crt
cat certs/backend.crt ca/intermediate-ca.crt > certs/backend-chain.pem
```

A láncokat a gyökér CA-val ellenőrizzük; a köztes CA-t `-untrusted` opcióval adjuk meg, mert nem trust anchor, csak a lánc része.

```
cd "$HOME/iot-labor/i056"
openssl verify -CAfile ca/root-ca.crt -untrusted ca/intermediate-ca.crt \
  certs/server.crt certs/gw.crt certs/backend.crt
openssl x509 -in certs/backend.crt -noout -subject -issuer -ext extendedKeyUsage
stat -c '%a %n' ca/*.key certs/server.key certs/gw.key certs/backend.key
```

A várt kimenet (illusztratív):

```
certs/server.crt: OK
certs/gw.crt: OK
certs/backend.crt: OK
subject=C = HU, O = TerraSense Lab, CN = backend.terrasense
issuer=C = HU, O = TerraSense Lab, CN = TerraSense Intermediate CA
X509v3 Extended Key Usage: TLS Web Client Authentication
600 ca/intermediate-ca.key
600 ca/root-ca.key
600 certs/backend.key
600 certs/gw.key
600 certs/server.key
```

A 600-as jogosultság és az `umask 077` együtt biztosítja, hogy a privát kulcsok csak a tulajdonos számára legyenek olvashatók. A CA privát kulcsait egyetlen szolgáltatáskonténerbe sem csatoljuk.

19.3.4.2 2. Visszavonási lista (CRL) előkészítése

A tanúsítvány visszavonása nem történik meg automatikusan attól, hogy a fájlt töröljük, vagy megvárjuk a lejáratát. A broker csak akkor utasítja el a visszavont klienst, ha **visszavonási listát (CRL)** tölt be és ellenőriz. A CRL-t a köztes CA írja alá. A Mosquitto 2.0.x `crlfile` direktívája `X509_load_crl_file` és `X509_V_FLAG_CRL_CHECK` beállítással dolgozik; ha a fájl nem olvasható, a broker el sem indul (fail-closed). A CRL frissítése nem automatikus: újragenerálás után a brokert újra kell indítani.

A CRL-generáláshoz az `openssl ca` parancsnak szüksége van egy adatbázisra és egy konfigurációs fájlra. Ezeket az Eszközidentitás: X.509-tanúsítványok, PKI és kulcstárolás CA-könyvtárában hozzuk létre.

```
cd "$HOME/iot-labor/i056"
cat > ca/intermediate-ca.cnf <<'EOF'
[ca]
default_ca = intermediate_ca

[intermediate_ca]
database = ca/index.txt
serial = ca/serial
crlnumber = ca/crlnumber
private_key = ca/intermediate-ca.key
certificate = ca/intermediate-ca.crt
default_md = sha256
default_crl_days = 30
policy = policy_anything

[policy_anything]
commonName = optional
countryName = optional
organizationName = optional
EOF
touch ca/index.txt
test -f ca/serial || printf '1000\n' > ca/serial
test -f ca/crlnumber || printf '1000\n' > ca/crlnumber
openssl ca -config ca/intermediate-ca.cnf -gencrl -out ca/intermediate-ca.crl
openssl crl -in ca/intermediate-ca.crl -noout -issuer -lastupdate -nextupdate
```

A várt kimenet (illusztratív; a dátumok a futtatás napjától függnek):

```
issuer=C = HU, O = TerraSense Lab, CN = TerraSense Intermediate CA
lastUpdate=Oct 31 10:00:00 2026 GMT
nextUpdate=Nov 30 10:00:00 2026 GMT
```

A `default_crl_days 30` azt jelenti, hogy a lista 30 napig érvényes. Ha lejár, az OpenSSL CRL-ellenőrzés `CRL has expired` hibát ad, és a broker a klienseket sem engedi be; ez szándékos fail-closed viselkedés. Éles rendszerben a CRL-t a `nextUpdate` előtt újra kell generálni, vagy rövid élettartamú tanúsítványokkal és OCSP staplinggel kell kiváltani. A labor nem használ OCSP-t, mert a `ts-node` és a gateway időnként offline lehet, és a folyamatos OCSP-kapcsolat nem garantálható.

19.3.4.3 3. Mosquitto konfiguráció mTLS-sel, ACL-lel és CRL-lel

A korábbi `stack/mosquitto/mosquitto.conf` fájlt módosítjuk. A titkosítatlan `listener 1883` sort eltávolítjuk vagy kommenteljük, és beállítjuk a TLS-listener-t. Az ACL-fájlt külön hozzuk létre.

Először az ACL-fájl. A `use_identity_as_username true` miatt a `user` sorokban a tanúsítványok CN-je szerepel. A gh1 gateway a saját site-ja node-jainak telemetriáját, állapotát és státuszát írja, a parancsokat olvassa, azok eredményét pedig írja; parancsot nem adhat ki. A backend minden site adatát olvassa, és egyedül ő írhat parancsot.

```
# ~/iot-labor/stack/mosquitto/acl.conf
# A use_identity_as_username miatt a "user" a kliens tanúsítványának CN-je.

# gh1 gateway: a saját site-ja node-jainak adatait írja, a parancsokat olvassa.
user gw.gh1.terrasense
```

```
topic write terrasense/gh1/+/telemetry
topic write terrasense/gh1/+/state
topic write terrasense/gh1/+/status
topic read terrasense/gh1/+/cmd
topic write terrasense/gh1/+/cmd/result

# backend: minden site adatát olvassa, parancsot csak ő írhat.
user backend.terrasense
topic read terrasense/+/+/telemetry
topic read terrasense/+/+/state
topic read terrasense/+/+/status
topic read terrasense/+/+/cmd/result
topic write terrasense/+/+/cmd
```

A Mosquitto konfigurációja a `stack/mosquitto/mosquitto.conf` fájlban:

```
# ~/iot-labor/stack/mosquitto/mosquitto.conf
per_listener_settings true

listener 8883
protocol mqtt
tls_version tlsv1.3
cafile /mosquitto/certs/root-ca.crt
certfile /mosquitto/certs/server-chain.pem
keyfile /mosquitto/certs/server.key
crlfile /mosquitto/certs/intermediate-ca.crl
require_certificate true
use_identity_as_username true
acl_file /mosquitto/acl.conf
```

A `per_listener_settings true` beállítás biztosítja, hogy az esetleges további listener-ek ne örököljék automatikusan a TLS-beállításokat. A `require_certificate true` kötelezővé teszi a kliens tanúsítványt. A `use_identity_as_username true` hatására a broker a kliens tanúsítvány CN-jét használja felhasználónévként, így az ACL-fájl `user gw.gh1.terrasense` és `user backend.terrasense` sorai érvényesülnek. A `tls_version tlsv1.3` a TLS 1.3-at írja elő; a TLS 1.2-es kézfogást a broker elutasítja. A `certfile` a szerverláncot (`server-chain.pem`) tartalmazza, hogy a broker a végponti tanúsítvány mellett a köztes CA-t is elküldje; a `cafile` csak a gyökér CA-t tartalmazza, mert a kliensek a saját láncukban küldik a köztes CA-t. A `crlfile` a köztes CA által aláírt visszavonási lista.

A `stack/compose.yaml` `lab-broker` szolgáltatásában a következő csatolásokat kell hozzáadni vagy módosítani. A `compose.yaml` a `~/iot-labor/stack/` könyvtárban van, ezért a `../i056/...` útvonal a `~/iot-labor/i056/...` fájlra mutat.

```
  lab-broker:
    image: eclipse-mosquitto:2
    ports:
      - "8883:8883"
    volumes:
      - ./mosquitto/mosquitto.conf:/mosquitto/config/mosquitto.conf:ro
      - ./mosquitto/acl.conf:/mosquitto/acl.conf:ro
      - ../i056/certs/server-chain.pem:/mosquitto/certs/server-chain.pem:ro
      - ../i056/certs/server.key:/mosquitto/certs/server.key:ro
      - ../i056/ca/root-ca.crt:/mosquitto/certs/root-ca.crt:ro
      - ../i056/ca/intermediate-ca.crl:/mosquitto/certs/intermediate-ca.crl:ro
```

A csatolások szándékosan fájlonkéntiek: a broker csak a saját szerverláncát, szerverkulcsát, a gyökér CA-t és a CRL-t kapja meg. A köztes CA privát kulcsa, a gateway és a backend privát kulcsa nem kerül a broker konténerébe. A 1883-as port nyilvánossá tételét töröljük vagy ne hozzuk létre.

A hivatalos `eclipse-mosquitto:2` image a `mosquitto` felhasználóval fut, ezért a 0600-as szerverkulcsot nem feltétlenül tudja olvasni. Ha a napló `Unable to load server key file` hibát jelez, a kulcs tulajdonosát/csoportját a konténer UID-jára kell állítani (például `sudo chown 1883:1883 ...` és `sudo chmod 640 ...`), vagy Docker secretet kell használni. A kulcsot soha ne tegyük világosan olvashatóvá (`chmod 644`).

19.3.4.4 4. A ts-gw Paho C++ kliensének beállítása

A A ts-gw szolgáltatás C++-ban laborban létrehozott `MqttPublisher` **Paho C++ 1.3.2** klienst használ, amely a Paho C `MQTTAsync` API-jára épül. A TLS-hez a Paho C++ könyvtárat SSL-támogatással kell buildelni, és a `connect_options` mellé `mqtt::ssl_options` objektumot kell adni.

Először a Paho C 1.3.13-at és a Paho C++ 1.3.2-t építjük fel SSL-támogatással egy közös prefixbe. A disztribúciós Paho C++ csomag az Ubuntu 24.04 alatt 1.2.0-s, ezért a könyvben rögzített 1.3.2 verzióhoz forrásból történő build szükséges.

```
mkdir -p "$HOME/iot-labor/third_party" && cd "$HOME/iot-labor/third_party"
git clone --depth 1 --branch v1.3.13 https://github.com/eclipse/paho.mqtt.c.git
cmake -S paho.mqtt.c -B paho-c-build \
  -DPAHO_WITH_SSL=ON -DPAHO_BUILD_SHARED=ON \
  -DCMAKE_INSTALL_PREFIX="$HOME/iot-labor/third_party/install"
cmake --build paho-c-build -j"$(nproc)"
cmake --install paho-c-build

git clone --depth 1 --branch v1.3.2 https://github.com/eclipse/paho.mqtt.cpp.git
cmake -S paho.mqtt.cpp -B paho-cpp-build \
  -DPAHO_WITH_SSL=ON \
  -DCMAKE_PREFIX_PATH="$HOME/iot-labor/third_party/install" \
  -DCMAKE_INSTALL_PREFIX="$HOME/iot-labor/third_party/install"
cmake --build paho-cpp-build -j"$(nproc)"
cmake --install paho-cpp-build
```

A Paho C++ SSL-támogatással a `paho-mqttpp3` pkg-config modul a `paho-mqtt3as` könyvtárra hivatkozik, mert a Paho C++ az aszinkron C API-ra épül. A `paho-mqtt3cs` a **szinkron C API** SSL-változata, amelyet a Paho C++ nem használ; a két könyvtárnév összekeverése tipikus hiba. A build után ellenőrizzük a verziót és a linkelést:

```
export PKG_CONFIG_PATH="$HOME/iot-labor/third_party/install/lib/pkgconfig:$PKG_CONFIG_PATH"
pkg-config --modversion paho-mqttpp3
cd "$HOME/iot-labor/i036/ts-gw"
cmake -S . -B build -DCMAKE_PREFIX_PATH="$HOME/iot-labor/third_party/install"
cmake --build build -j"$(nproc)"
ldd build/ts-gw | grep -E 'paho-mqtt3as|paho-mqttpp3'
```

A várt kimenet (illusztratív):

```
1.3.2
libpaho-mqttpp3.so.1 => /home/user/iot-labor/third_party/install/lib/libpaho-mqttpp3.so.1
libpaho-mqtt3as.so.1 => /home/user/iot-labor/third_party/install/lib/libpaho-mqtt3as.so.1
```

A `MqttPublisher::connect()` metódusát a következőképpen egészítjük ki. A függvény a teljes, fordítható változat; a `MqttPublisher` többi metódusa változatlan. A `client_` URI-je a konfigurációból érkezik, és ebben a laborban `ssl://localhost:8883`.

```
#include <cstdlib>
#include <mqtt/ssl_options.h>

bool MqttPublisher::connect(std::string& err) {
  if (client_.is_connected()) {
    return true;
  }
  const char* home_env = std::getenv("HOME");
  if (home_env == nullptr) {
    err = "HOME nincs beallitva";
    return false;
  }
  const std::string home(home_env);
  try {
    mqtt::ssl_options ssl;
    ssl.set_trust_store(home + "/iot-labor/i056/ca/root-ca.crt");
    ssl.set_key_store(home + "/iot-labor/i056/certs/gw-chain.pem");
    ssl.set_private_key(home + "/iot-labor/i056/certs/gw.key");
    ssl.set_verify(true);
    ssl.set_enable_server_cert_auth(true);

    mqtt::connect_options options;
    options.set_clean_session(true);
    options.set_keep_alive_interval(20);
    options.set_connect_timeout(5);
    options.set_ssl(ssl);

    auto token = client_.connect(options);
    if (!token->wait_for_completion(std::chrono::seconds(5))) {
      err = "MQTT connect timeout";
      return false;
    }
    if (token->get_return_code() != 0) {
      err = "MQTT connect refused, rc=" +
            std::to_string(token->get_return_code());
      return false;
    }
    return client_.is_connected();
  } catch (const mqtt::exception& e) {
    err = std::string("MQTT connect: ") + e.what();
    return false;
  }
}
```

A `set_trust_store` a gyökér CA-t adja meg trust anchorként, a `set_key_store` a gateway láncfájlját, a `set_private_key` a privát kulcsát. A `set_enable_server_cert_auth(true)` bekapcsolja a szervertanúsítvány láncának ellenőrzését, a `set_verify(true)` pedig a hosztnév-ellenőrzést: a Paho C ilyenkor `X509_check_host`-tal, IP-cím esetén `X509_check_ip_asc`-cal ellenőrzi a szervernevet. A `ts-gw` indításakor a `TS_GW_MQTT_BROKER` értéke `ssl://localhost:8883` legyen.

A Paho C/C++ `ssl_options` legfeljebb a TLS 1.2 szintű verziókonstanst ismeri (`MQTT_SSL_VERSION_TLS_1_2`); TLS 1.3-ra nincs külön konstans. Ezért a verziót nem kell és nem is lehet a kliensben 1.3-ra állítani: ha nem adunk meg verziót, az OpenSSL 3.0.13 a szerverrel egyeztetve a legmagasabb közös verziót, esetünkben a TLS 1.3-at használja. A tényleges verziót a 6. szakasz `openssl s_client -tls1_3` és `-tls1_2` tesztje igazolja.

19.3.4.5 5. A ts-backend SSL-konfigurációja (Paho MQTT 5 Java kliens)

A `ts-backend` MQTT-ingest komponense az Ingest: MQTT-ből a backendbe, idempotensen alfejezetben bemutatott **Paho MQTT v5 Java klienst** (`org.eclipse.paho.mqttv5.client`, 1.2.5) használja, nem Spring Integrationt. A Java kliens nem PEM-fájlt, hanem Java kulcstárolót vár, ezért a PEM-eket PKCS12 formátumba alakítjuk, és a Spring Boot YAML-kulcsokat `@ConfigurationProperties` osztályhoz kötjük. Önmagukban a `mqtt.ssl.trust-store` típusú kulcsok nem hatnak semmire, ha nincs őket feldolgozó kód.

Először létrehozzuk a backend kliens PKCS12 kulcstárolóját és a gyökér CA-t tartalmazó truststore-t:

```
cd "$HOME/iot-labor/i056"
openssl pkcs12 -export \
  -inkey certs/backend.key -in certs/backend-chain.pem \
  -name backend -out certs/backend.p12 \
  -passout pass:lab-p12-jelszo
keytool -importcert -noprompt -alias terrasense-root \
  -file ca/root-ca.crt \
  -keystore certs/truststore.p12 -storetype PKCS12 \
  -storepass lab-p12-jelszo
```

A várt kimenet (illusztratív): `Certificate was added to keystore`. A `backend.p12` a végponti tanúsítványt, a köztes CA-t és a privát kulcsot tartalmazza; a truststore csak a gyökér CA-t, mert a szerver a `server-chain.pem` révén a köztes CA-t is elküldi. A jelszó a laboré; éles rendszerben környezeti változóból vagy titokkezelőből kell jönnie, és a PKCS12 fájl jogosultsága 600 legyen.

A következő osztály a YAML `ingest.tls.*` kulcsait köti be, és felépíti a TLS 1.3 kézfogáshoz szükséges `SSLContext`-et. A `@Component` és a `@ConfigurationProperties` együtt gondoskodik arról, hogy a Spring Boot a tulajdonságokat a példányra töltse.

```
package hu.terrasense.backend.ingest;

import java.io.IOException;
import java.io.InputStream;
import java.nio.file.Files;
import java.nio.file.Path;
import java.security.GeneralSecurityException;
import java.security.KeyStore;
import javax.net.ssl.KeyManagerFactory;
import javax.net.ssl.SSLContext;
import javax.net.ssl.TrustManagerFactory;
import lombok.Data;
import org.springframework.boot.context.properties.ConfigurationProperties;
import org.springframework.stereotype.Component;

@Data
@Component
@ConfigurationProperties(prefix = "ingest.tls")
public class MqttTlsProperties {

    private Path keystore;
    private String keystorePassword;
    private Path truststore;
    private String truststorePassword;

    public SSLContext sslContext() throws GeneralSecurityException, IOException {
        if (keystore == null || truststore == null ||
                keystorePassword == null || truststorePassword == null) {
            throw new IllegalStateException(
```

```
                "Az ingest.tls.keystore, truststore es a jelszavak kotelezoek");
        }
        KeyStore keys = KeyStore.getInstance("PKCS12");
        try (InputStream in = Files.newInputStream(keystore)) {
            keys.load(in, keystorePassword.toCharArray());
        }
        KeyManagerFactory kmf =
            KeyManagerFactory.getInstance(KeyManagerFactory.getDefaultAlgorithm());
        kmf.init(keys, keystorePassword.toCharArray());

        KeyStore anchors = KeyStore.getInstance("PKCS12");
        try (InputStream in = Files.newInputStream(truststore)) {
            anchors.load(in, truststorePassword.toCharArray());
        }
        TrustManagerFactory tmf =
            TrustManagerFactory.getInstance(TrustManagerFactory.getDefaultAlgorithm());
        tmf.init(anchors);

        SSLContext context = SSLContext.getInstance("TLSv1.3");
        context.init(kmf.getKeyManagers(), tmf.getTrustManagers(), null);
        return context;
    }
}
```

A `TelemetryIngestService` az Ingest: MQTT-ből a backendbe, idempotensen alfejezetbeli változathoz képest egyetlen konstruktorparamétert és két kapcsolati beállítást kap. A konstruktor új változata:

```
private final MqttTlsProperties tls;

public TelemetryIngestService(
        TelemetryRepository repository,
        ObjectMapper objectMapper,
        IngestRetrySpool retrySpool,
        MqttTlsProperties tls,
        @Value("${ingest.broker-uri}") String brokerUri,
        @Value("${ingest.client-id}") String clientId,
        @Value("${ingest.topic-filter}") String topicFilter) {
    this.repository = repository;
    this.objectMapper = objectMapper;
    this.retrySpool = retrySpool;
    this.tls = tls;
    this.brokerUri = brokerUri;
    this.clientId = clientId;
    this.topicFilter = topicFilter;
}
```

A `start()` metódus első része a következőképpen módosul; a callback és a `subscribe()` hívás változatlan:

```
@PostConstruct
public void start() throws MqttException {
    client = new MqttClient(brokerUri, clientId);
    options = new MqttConnectionOptions();
    options.setAutomaticReconnect(true);
    options.setCleanStart(false);
    options.setSessionExpiryInterval(3600L);
    try {
        options.setSocketFactory(tls.sslContext().getSocketFactory());
    } catch (GeneralSecurityException | IOException e) {
        throw new IllegalStateException("A TLS kontextus inicializalasa sikertelen", e);
    }
    options.setHttpsHostnameVerificationEnabled(true);
```

```
    // ... a callback beallitasa es a client.connect(options) valtozatlan ...
}
```

A `setHttpsHostnameVerificationEnabled(true)` a Paho Java v5 kliensben a HTTPS-endpoint-azonosítást kapcsolja be, amely a tanúsítvány hosztneve és a kapcsolódási név egyezését ellenőrzi; ezért a `broker-uri` hosztneve `localhost` legyen, ne nyers IP-cím. Az `SSLContext.getInstance("TLSv1.3")` a Java 17-es runtime alatt a TLS 1.3-at használja.

Az `application.yaml` MQTT-szekciója a következőképpen néz ki. A jelszavak környezeti változókból jönnek, a fájlnevek pedig az Eszközidentitás: X.509-tanúsítványok, PKI és kulcstárolás PKI-ra mutatnak.

```
ingest:
  broker-uri: ${MQTT_BROKER_URI:ssl://localhost:8883}
  client-id: ${MQTT_CLIENT_ID:ts-backend-ingest}
  topic-filter: ${MQTT_TOPIC_FILTER:terrasense/+/+/telemetry}
  spool-path: ${INGEST_SPOOL_PATH:spool/ingest-retry.jsonl}
  tls:
    keystore: ${MQTT_TLS_KEYSTORE:${HOME}/iot-labor/i056/certs/backend.p12}
    keystore-password: ${MQTT_TLS_KEYSTORE_PASSWORD}
    truststore: ${MQTT_TLS_TRUSTSTORE:${HOME}/iot-labor/i056/certs/truststore.p12}
    truststore-password: ${MQTT_TLS_TRUSTSTORE_PASSWORD}
```

Ezzel a titkosítatlan `tcp://` URL helyett a backend `ssl://localhost:8883` címre csatlakozik, a PKCS12 kulcstárolóból bemutatja a `backend.terrasense` tanúsítványát, a truststore gyökér CA-jával pedig ellenőrzi a broker láncát. A korábbi, PEM-fájlokra mutató `mqtt.ssl.*` kulcsok hatástalanok voltak, mert a Paho Java nem PEM-et és nem Spring Integration property-neveket használ.

19.3.4.6 6. Tesztelés

A broker újraindítása után ellenőrizzük, hogy a titkosítatlan port tényleg nem elérhető:

```
nc -vz localhost 1883
```

A parancsnak kapcsolódási hibát kell jeleznie. A 8883-as porton először TLS-szinten ellenőrizzük a kézfogást és a protokollverziót. Az `openssl s_client` önmagában nem küld MQTT `CONNECT` csomagot, ezért a broker naplójában ilyenkor még nem jelenik meg `New client connected` sor; az csak a MQTT-szintű teszteknél várható.

```
openssl s_client -connect localhost:8883 -tls1_3 -servername localhost \
  -CAfile "$HOME/iot-labor/i056/ca/root-ca.crt" \
  -cert "$HOME/iot-labor/i056/certs/gw-chain.pem" \
  -key "$HOME/iot-labor/i056/certs/gw.key" \
  </dev/null 2>/dev/null | grep -E 'Protocol|Verify return code'
```

A várt kimenet (illusztratív):

```
Protocol  : TLSv1.3
Verify return code: 0 (ok)
```

A TLS 1.2-es kézfogásnak a broker `tls_version tlsv1.3` beállítása miatt meg kell hiúsulnia:

```
openssl s_client -connect localhost:8883 -tls1_2 -servername localhost \
  -CAfile "$HOME/iot-labor/i056/ca/root-ca.crt" \
  -cert "$HOME/iot-labor/i056/certs/gw-chain.pem" \
  -key "$HOME/iot-labor/i056/certs/gw.key" </dev/null
```

A várt eredmény: nem jön létre kapcsolat, a kliens `alert protocol version` vagy kézfogási hibát jelez, és a parancs nem nulla kilépési kóddal tér vissza. A hosztnév-ellenőrzés negatív tesztje:

```
openssl s_client -connect localhost:8883 -tls1_3 -verify_hostname wrong.example \
  -CAfile "$HOME/iot-labor/i056/ca/root-ca.crt" \
  -cert "$HOME/iot-labor/i056/certs/gw-chain.pem" \
  -key "$HOME/iot-labor/i056/certs/gw.key" \
  </dev/null 2>/dev/null | grep 'Verify return code'
```

A várt kimenet (illusztratív): `Verify return code: 62 (Hostname mismatch)`.

Ezután MQTT-szinten is tesztelünk. Először indítsuk el a backend tanúsítványával a feliratkozót, majd a gateway tanúsítványával publikáljunk. A `--tls-version tlsv1.3` a Mosquitto kliensében is kikényszeríti a protokollverziót.

```
mosquitto_sub -h localhost -p 8883 --tls-version tlsv1.3 \
  --cafile "$HOME/iot-labor/i056/ca/root-ca.crt" \
  --cert "$HOME/iot-labor/i056/certs/backend-chain.pem" \
  --key "$HOME/iot-labor/i056/certs/backend.key" \
  -t 'terrasense/gh1/+/telemetry' -v -C 1

mosquitto_pub -h localhost -p 8883 --tls-version tlsv1.3 \
  --cafile "$HOME/iot-labor/i056/ca/root-ca.crt" \
  --cert "$HOME/iot-labor/i056/certs/gw-chain.pem" \
  --key "$HOME/iot-labor/i056/certs/gw.key" \
  -t terrasense/gh1/n001/telemetry -q 1 \
  -m '{"schema_version":1,"site":"gh1","node":"n001"}'
```

A feliratkozó a várt módon kiírja az üzenetet (illusztratív):

```
terrasense/gh1/n001/telemetry {"schema_version":1,"site":"gh1","node":"n001"}
```

A broker naplója a MQTT-szintű kapcsolatról a következőhöz hasonló sorokat tartalmaz (illusztratív; a formátum a Mosquitto verziójától függ):

```
New connection from 127.0.0.1:5xxxx on port 8883.
New client connected from 127.0.0.1:5xxxx as gw.gh1.terrasense (p2, c1, k60).
```

Az ACL negatív tesztje: a gateway nem publikálhat a `cmd` topicra.

```
mosquitto_pub -h localhost -p 8883 --tls-version tlsv1.3 \
  --cafile "$HOME/iot-labor/i056/ca/root-ca.crt" \
  --cert "$HOME/iot-labor/i056/certs/gw-chain.pem" \
  --key "$HOME/iot-labor/i056/certs/gw.key" \
  -t terrasense/gh1/n001/cmd -q 1 -m '{"cmd":"open"}'
```

A parancs nem nulla kilépési kóddal tér vissza, a broker pedig a naplóban megtagadja a publikálást (illusztratív):

```
Denied PUBLISH from gw.gh1.terrasense (d0, q1, r0, m1, 'terrasense/gh1/n001/cmd', ...)
```

A visszavonási lista működését egy eldobható kliens-tanúsítvánnyal ellenőrizzük. A tanúsítványt a köztes CA-val írjuk alá, majd visszavonjuk, újrageneráljuk a CRL-t, és újraindítjuk a brokert.

```
cd "$HOME/iot-labor/i056"
cat > certs/revoked-client.ext <<'EOF'
[v3_client]
basicConstraints = critical,CA:FALSE
keyUsage = critical,digitalSignature
extendedKeyUsage = clientAuth
EOF
```

```
openssl ecparam -genkey -name prime256v1 -out certs/revoked-client.key
openssl req -new -key certs/revoked-client.key \
  -subj "/C=HU/O=TerraSense Lab/CN=test.revoked.terrasense" \
  -out certs/revoked-client.csr
openssl x509 -req -in certs/revoked-client.csr \
  -CA ca/intermediate-ca.crt -CAkey ca/intermediate-ca.key -CAcreateserial \
  -days 30 -sha256 \
  -extfile certs/revoked-client.ext -extensions v3_client \
  -out certs/revoked-client.crt
cat certs/revoked-client.crt ca/intermediate-ca.crt > certs/revoked-client-chain.pem

openssl ca -config ca/intermediate-ca.cnf -revoke certs/revoked-client.crt
openssl ca -config ca/intermediate-ca.cnf -gencrl -out ca/intermediate-ca.crl
openssl verify -crl_check -CRLfile ca/intermediate-ca.crl \
  -CAfile ca/root-ca.crt -untrusted ca/intermediate-ca.crt \
  certs/revoked-client.crt
```

Az `openssl verify` várt kimenete (illusztratív):

```
CN = test.revoked.terrasense
error 23 at 0 depth lookup: certificate revoked
error certs/revoked-client.crt: verification failed
```

Ezután a brokert újraindítjuk, hogy betöltse az új CRL-t, majd a visszavont tanúsítvánnyal próbálunk kapcsolódni:

```
docker compose -f "$HOME/iot-labor/stack/compose.yaml" restart lab-broker
mosquitto_pub -h localhost -p 8883 --tls-version tlsv1.3 \
  --cafile "$HOME/iot-labor/i056/ca/root-ca.crt" \
  --cert "$HOME/iot-labor/i056/certs/revoked-client-chain.pem" \
  --key "$HOME/iot-labor/i056/certs/revoked-client.key" \
  -t terrasense/gh1/n001/telemetry -m '{}'
```

A kapcsolat TLS-szinten megszakad, a broker naplójában a `certificate revoked` kifejezés szerepel (illusztratív), és a kliens nem tud publikálni. A gateway és a backend tanúsítványa nincs a CRL-en, ezért azok továbbra is működnek.

A lejárt tanúsítvány negatív tesztjéhez a köztes CA-val a múltban érvényes tanúsítványt írunk alá. Az `openssl ca -startdate/-enddate` kapcsolója explicit érvényességi ablakot ad; a fenti konfiguráció `policy_anything` szakasza teszi lehetővé a batch aláírást:

```
cd "$HOME/iot-labor/i056"
cat > certs/expired-client.ext <<'EOF'
[v3_client]
basicConstraints = critical,CA:FALSE
keyUsage = critical,digitalSignature
extendedKeyUsage = clientAuth
EOF
openssl ecparam -genkey -name prime256v1 -out certs/expired-client.key
openssl req -new -key certs/expired-client.key \
  -subj "/C=HU/O=TerraSense Lab/CN=test.expired.terrasense" \
  -out certs/expired-client.csr
openssl ca -config ca/intermediate-ca.cnf -batch \
  -in certs/expired-client.csr -out certs/expired-client.crt \
  -startdate 20200101000000Z -enddate 20200102000000Z \
  -extfile certs/expired-client.ext -extensions v3_client
cat certs/expired-client.crt ca/intermediate-ca.crt > certs/expired-client-chain.pem
openssl verify -CAfile ca/root-ca.crt -untrusted ca/intermediate-ca.crt \
  certs/expired-client.crt
```

A várt kimenet: `error 10 at 0 depth lookup: certificate has expired`. A lejárt lánccal a `mosquitto_pub` TLS-kézfogása meghiúsul, a broker naplójában a `certificate has expired` hiba jelenik meg; a kliens nem tud publikálni.

Az EKU-ellenőrzést a Mosquitto nem végzi el a kliens tanúsítványokon: a broker nem állít be X.509 célt (`X509_PURPOSE`), ezért a `clientAuth` EKU meglétét a TLS-kézfogás nem kényszeríti ki. Ezt a hiányt az ACL enyhíti: a szervertanúsítvány CN-je (`mqtt.terrasense.local`) nem szerepel egyetlen `user` sorban sem, így a szervertanúsítvány kliensként nem kap jogosultságot. A tanúsítványok rendeltetését hostoldalon is ellenőrizhetjük:

```
openssl verify -purpose sslclient \
  -CAfile "$HOME/iot-labor/i056/ca/root-ca.crt" \
  -untrusted "$HOME/iot-labor/i056/ca/intermediate-ca.crt" \
  "$HOME/iot-labor/i056/certs/server.crt"
```

A várt kimenet (illusztratív):

```
C = HU, O = TerraSense Lab, CN = mqtt.terrasense.local
error 26 at 0 depth lookup: unsuitable certificate purpose
error certs/server.crt: verification failed
```

A hibaüzenet szövege az OpenSSL verziójától függően `unsupported certificate purpose` is lehet; a 26-os hibakód (`X509_V_ERR_INVALID_PURPOSE`) a lényeg.

Végül a backendet indítjuk el a TLS-beállításokkal. A jelszavakat környezeti változóban adjuk át, hogy ne kerüljenek a shell historyba.

```
cd "$HOME/iot-labor/i044/ts-backend"
export MQTT_BROKER_URI=ssl://localhost:8883
export MQTT_TLS_KEYSTORE="$HOME/iot-labor/i056/certs/backend.p12"
export MQTT_TLS_KEYSTORE_PASSWORD=lab-p12-jelszo
export MQTT_TLS_TRUSTSTORE="$HOME/iot-labor/i056/certs/truststore.p12"
export MQTT_TLS_TRUSTSTORE_PASSWORD=lab-p12-jelszo
mvn spring-boot:run
```

A sikeres indulás naplója (illusztratív):

```
MQTT kapcsolat kész: reconnect=false, broker=ssl://localhost:8883
Feliratkozva: terrasense/+/+/telemetry
```

19.3.5 Ellenőrzés és hibakeresés

A labor sikerességének ellenőrzése a következő szinteken történik:

1. **Portellenőrzés:** a `nc -vz localhost 1883` parancs sikertelen, a 8883-as porton viszont létrejön a TLS 1.3 kézfogás.
2. **Protokollverzió:** az `openssl s_client -tls1_3` kimenetében `Protocol : TLSv1.3` szerepel, a `-tls1_2` kézfogás pedig meghiúsul.
3. **Tanúsítványlánc és hosztnév:** a `Verify return code: 0 (ok)` azt jelzi, hogy a lánc a gyökér CA-ig visszavezethető és a hosztnév egyezik; a `-verify_hostname wrong.example` esetén 62-es hibakódot kapunk.
4. **MQTT-szint:** a `mosquitto_pub`/`mosquitto_sub` pár a `terrasense/gh1/n001/telemetry` topicon átviszi az üzenetet, a broker naplója a CN-t használja felhasználónévként.
5. **ACL:** a gateway `cmd` topicra küldött PUBLISH-je megtagadásra kerül, a backend viszont publikálhat rá.
6. **Visszavonás:** a visszavont tanúsítványú kliens a CRL betöltése után nem tud kapcsolódni.

7. **Backend:** a Spring Boot ingest `ssl://` URL-lel indul, és a telemetriát a `terrasense/+/+/telemetry` topicra feliratkozva fogadja.

Tipikus hibák és javításuk:

Tünet	Ok	Javítás
`SSL error: certificate verify failed`	A trust store nem tartalmazza a gyökér CA-t, vagy a kliens nem a `localhost` nevet használja	Ellenőrizd a `root-ca.crt` útvonalát és a broker URI hosztnevét; a SAN `localhost` és `127.0.0.1`
A Mosquitto `unknown ca` üzenettel utasítja el a klienst	A kliens tanúsítványát nem a broker `cafile`-jában szereplő CA írta alá, vagy a kliens nem küldte el a köztes CA-t	Ellenőrizd a láncfájlt (`gw-chain.pem`, `backend-chain.pem`), és hogy a broker `cafile`-ja a gyökér CA
A kapcsolat létrejön, de a `PUBLISH` megtagadásra kerül	Az ACL `user` sora nem egyezik a tanúsítvány CN-jével, vagy a topic nem illeszkedik a mintára	Ellenőrizd a CN-t (`openssl x509 -in certs/gw.crt -noout -subject`) és az ACL-t; a `use_identity_as_username true` kötelező
A broker nem indul, `Error loading CRL file`	A CRL hiányzik, nem olvasható, lejárt, vagy nem a köztes CA írta alá	Generáld újra a CRL-t, ellenőrizd a mountot és a `nextUpdate` dátumot
`certificate revoked` a kézfogásban	A kliens tanúsítványa szerepel a CRL-en	Használj nem visszavont tanúsítványt, vagy a rotáció után generálj új CRL-t
A backend `unable to find valid certification path to requested target` hibát ad	A truststore nem tartalmazza a gyökér CA-t, vagy a szerver nem küldte el a köztes CA-t	Importáld a gyökér CA-t a PKCS12 truststore-ba; a broker `certfile`-ja `server-chain.pem` legyen
A backend `No subject alternative names present` vagy hosztnév-hibát ad	A `broker-uri` olyan nevet tartalmaz, amely nincs a szervertanúsítvány SAN-jában	Használj `localhost`-ot, vagy igényelj új szervertanúsítványt a megfelelő SAN-nal
A broker `Unable to load server key file` hibát jelez	A konténer `mosquitto` felhasználója nem olvassa a 0600-as kulcsot	Állítsd be a kulcs tulajdonosát/csoportját a konténer UID-jára, vagy használj secretet; ne lazítsd 644-re a jogosultságot
A Paho C++ build nem találja a `paho-mqttpp3` modult	A saját prefix nincs a `PKG_CONFIG_PATH`-ban, vagy a Paho C++ nem SSL-lel épült	Állítsd be a `PKG_CONFIG_PATH`-t, és ellenőrizd, hogy a Paho C++ `PAHO_WITH_SSL=ON`-nal készült

19.3.6 Tipikus hibák és félreértések

1. **A titkosított port és a nyílt port párhuzamos nyitva tartása.** Ha a 1883-as port nyitva marad, a támadó egyszerűen megkerüli a TLS-t. A laborban ezért a titkosítatlan listener-t töröljük, nem pedig egy második listener-ként meghagyjuk.

2. **A tanúsítvány CN-jét felhasználónévként kezelni, de a `use_identity_as_username`-t kihagyni.** Enélkül a Mosquitto nem tudja, hogy a CN-t kell az ACL-kiértékeléshez használni, és vagy anonim felhasználóként kezeli a klienst, vagy az ACL-fájl `user` sorait figyelmen kívül hagyja.
3. **A kliens privát kulcsának rossz védelme és a felesleges csatolások.** A laborban jelszó nélküli kulcsokat használunk, mert a `native_sim` környezet és a labor célja a működés demonstrálása. Éles rendszerben a privát kulcsot soha nem tároljuk jelszó nélkül a fájlrendszerben; a kriptográfiai alapok és a kulcstárolás alfejezetben tárgyalt hardveres védelmet kell alkalmazni. A broker konténerébe csak a szerver saját kulcsa és lánca kerüljön; a CA kulcsa és a kliensek kulcsa soha.
4. **A DTLS és a TLS összekeverése.** A DTLS nem egyszerűen „TLS UDP-n”, hanem külön protokoll, saját újraküldési és sorrendkezelési mechanizmussal. A korlátos eszközön futó CoAP-kliensnek DTLS-t kell használnia, míg az MQTT-alapú `ts-gw` TLS-t használ.
5. **A TLS 1.3 „beállítása” a Paho kliensben.** A Paho C és C++ `ssl_options` legfeljebb TLS 1.2-es verziókonstanst ismer; ha valaki a kliensben 1.3-at próbál beállítani, nem létező API-t keres. A helyes megoldás a verzió meg nem adása és a szerver `tls_version tlsv1.3` beállítása, majd a tényleges verzió ellenőrzése.
6. **A visszavonás „automatikusnak” feltételezése.** A tanúsítvány visszavonása önmagában semmit nem tesz: a CRL-t generálni, a brokerhez eljuttatni és a brokert újraindítani kell. A CRL lejárata és a fail-closed működés miatt a frissítés üzemeltetési feladat, nem egyszeri lépés.
7. **PEM-fájl átadása a Java kliensnek, vagy YAML-kulcsok feltételezése.** A Paho Java PKCS12 kulcstárolót és truststore-t vár, a Spring Boot pedig csak akkor tölti be a `ingest.tls.*` értékeket, ha van `@ConfigurationProperties` osztály, amelyhez kötni tudja őket.

19.3.7 Feladatok

19.3.7.1 (a) Fogalmi kérdés

Magyarázd el, miért nem elegendő a szerveroldali TLS a TerraSense-ben, és mit ad hozzá az mTLS a fenyegetésmodell szempontjából!

19.3.7.2 (b) Tervezési feladat

Tervezd meg az ACL-szabályokat egy második `ts-gw` gateway-hez a `gh2` site-on (CN=`gw.gh2.terrasense`). A gateway a saját site-ja node-jainak telemetriáját, állapotát és státuszát írja, a parancsokat olvassa és azok eredményét írja, de parancsot nem adhat ki, és nem érheti el a `gh1` site topicjait. Írd le a `user` és `topic` sorokat, és magyarázd meg, miért nem kap `write` jogot a `cmd` topicra!

19.3.7.3 (c) Implementációs feladat

Módosítsd a `~/iot-labor/i036/ts-gw` forráskódját a fenti Paho C++ SSL-beállításokkal, és buildeld újra a projektet úgy, hogy a `paho-mqtt3as` SSL-könyvtárhoz linkeljen. A `ts-gw` a `ssl://localhost:8883` URL-lel csatlakozzon a brokerhez. Ellenőrizd `openssl s_client -tls1_3`-mal, hogy a kézfogás TLS 1.3-at használ, és hogy a telemetria továbbra is megérkezik a `terrasense/gh1/n001/telemetry` topicra!

19.3.7.4 (d) Hibakeresési feladat

A Mosquitto hirtelen elkezdi elutasítani a `ts-backend` kapcsolódását `unknown ca` hibával. Sorold fel a lehetséges okokat, és add meg a diagnosztikai parancsokat, amelyekkel meghatározod a hibát! Térj ki a láncfájlra, a truststore-ra és a CRL-re is!

19.3.8 Megoldások

19.3.8.1 (a) Fogalmi kérdés

A szerveroldali TLS csak a broker identitását hitelesíti a kliens felé, és titkosítja az adatfolyamot. A szerver nem tudja, hogy ki a kliens; így bárki, aki eléri a hálózati portot, csatlakozhat, és hamis adatokat publikálhat vagy parancsokat küldhet. Az mTLS a kliens tanúsítványát is ellenőrzi, így a broker a tanúsítvány CN-jéből megbízhatóan azonosítja a klienst, és topic-szintű ACL-t alkalmazhat. Ez a TerraSense fenyegetésmodelljében a megszemélyesítés (spoofing) és az illetéktelen beavatkozás elleni védelmet erősíti. A tanúsítvány-alapú azonosítás emellett a visszavonást is lehetővé teszi: egy kompromittált gateway kulcsa a CRL-lel kizárható anélkül, hogy a többi eszközt újrakonfigurálnánk.

19.3.8.2 (b) Tervezési feladat

Az ACL-fájlba a következő sorok kerülnek:

```
user gw.gh2.terrasense
topic write terrasense/gh2/+/telemetry
topic write terrasense/gh2/+/state
topic write terrasense/gh2/+/status
topic read terrasense/gh2/+/cmd
topic write terrasense/gh2/+/cmd/result
```

A `gw.gh2.terrasense` tehát a saját site-jának telemetriáját, állapotát és státuszát írja, a parancsokat olvassa, az eredményüket írja, de a `cmd` topicra nem kap `write` jogot, mert parancsot csak a backend adhat ki. A `gh1` topicjai nem szerepelnek a szabályokban, ezért a Mosquitto alapértelmezett módon megtagadja őket. A `+` wildcard a site-on belüli bármely node-ot jelenti, így a szabály skálázható marad, a site-nevet viszont nem lehet wildcarddal helyettesíteni: az ACL site-onként rögzíti a jogosultságot. A `user` sor a tanúsítvány CN-je, nem a szolgáltatás neve, ezért a CN-nek pontosan `gw.gh2.terrasense`-nek kell lennie.

19.3.8.3 (c) Implementációs feladat

A `MqttPublisher.cpp`-ben a fenti `mqtt::ssl_options` beállításokat kell használni, a `TS_GW_MQTT_BROKER` értékét pedig `ssl://localhost:8883`-ra kell állítani. A Paho C++ 1.3.2-t `PAHO_WITH_SSL=ON` kapcsolóval kell buildelni, és a CMake-nek a saját prefixben lévő `paho-mqttpp3` modult kell megtalálnia. A linkelés helyessége `ldd build/ts-gw`-vel ellenőrizhető: a `libpaho-mqtt3as.so` jelenléte igazolja az SSL-támogatást. A `paho-mqtt3cs` nem a Paho C++ könyvtára, hanem a szinkron C API SSL-változata, ezért a `paho-mqtt3cs` linkelésére vonatkozó tanács hibás. A `ts-gw` indítása után az `openssl s_client -tls1_3` kimenetében `Protocol : TLSv1.3` látható, a broker naplójában pedig a `gw.gh1.terrasense` identitás jelenik meg; a `ts-backend` előfizető megkapja a telemetriát.

19.3.8.4 (d) Hibakeresési feladat

Az `unknown ca` TLS-riasztás azt jelenti, hogy a broker nem tudja a kliens tanúsítványát a `cafile`-ban megadott gyökér CA-ig visszavezetni. Lehetséges okok:

- A `ts-backend` tanúsítványát nem az Eszközidentitás: X.509-tanúsítványok, PKI és kulcstárolás köztes CA írta alá (például egy másik labor CA-val készült).
- A kliens nem küldte el a köztes CA-t, mert a `backend-chain.pem` hiányos, vagy a PKCS12 csak a végponti tanúsítványt tartalmazza.
- A broker `cafile`-ja nem a `root-ca.crt`, hanem egy másik CA vagy egy hiányos láncfájl.
- A backend a visszavont tanúsítványát használja, és a CRL betöltése miatt a kézfogás a vártnál korábban megszakad (ilyenkor a hiba `certificate revoked`).

Diagnosztikai parancsok:

```
openssl verify -CAfile "$HOME/iot-labor/i056/ca/root-ca.crt" \
  -untrusted "$HOME/iot-labor/i056/ca/intermediate-ca.crt" \
  "$HOME/iot-labor/i056/certs/backend.crt"
openssl x509 -in "$HOME/iot-labor/i056/certs/backend.crt" \
  -noout -issuer -subject -dates
openssl crl -in "$HOME/iot-labor/i056/ca/intermediate-ca.crl" -noout -issuer -nextupdate
openssl s_client -connect localhost:8883 -tls1_3 -servername localhost \
  -CAfile "$HOME/iot-labor/i056/ca/root-ca.crt" \
  -cert "$HOME/iot-labor/i056/certs/backend-chain.pem" \
  -key "$HOME/iot-labor/i056/certs/backend.key"
```

Az `openssl verify` megmutatja, hogy a tanúsítvány érvényes láncba tartozik-e; az `openssl x509` a kiállítót és az érvényességi időt; az `openssl crl` a visszavonási lista érvényességét; az `openssl s_client` pedig a kézfogás sikerességét és a hiba pontos okát. A Java oldalon a truststore tartalmát a `keytool -list -v -keystore certs/truststore.p12 -storepass ...` paranccsal ellenőrizhetjük.

19.3.9 Rövid összefoglaló

Ebben az alfejezetben a TerraSense MQTT-láncát mTLS-sel biztosítottuk, az Eszközidentitás: X.509-tanúsítványok, PKI és kulcstárolás alfejezetben létrehozott kétszintű ECDSA P-256 PKI-ra építve. Kiadtuk a `ts-gw` (`CN=gw.gh1.terrasense`) és a `ts-backend` (`CN=backend.terrasense`) kliens tanúsítványait a köztes CA-val, létrehoztuk a láncfájljaikat, és a gyökér CA-val ellenőriztük őket. A Mosquitto konfigurációját átállítottuk a 8883-as TLS 1.3 listener-re, `require_certificate true`, `use_identity_as_username true`, `crlfile` és `acl_file` beállításokkal, és bevezettük a `~/iot-labor/stack/mosquitto/acl.conf` ACL-fájlt, amely a tanúsítvány CN-je alapján site-szintű jogosultságot ad. A `ts-gw` Paho C++ 1.3.2 kliensét SSL-opciókkal és hosztnév-ellenőrzéssel, a `ts-backend` Paho MQTT 5 Java kliensét PKCS12 kulcstárolóval, truststore-ral és `TLSv1.3` SSLContext-tel állítottuk be. A visszavonást CRL-lel, a hosztnév- és EKU-ellenőrzést konkrét tesztekkel ellenőriztük. A titkosítatlan 1883-as portot lezártuk. A továbbvitt eredmény: a biztonságos broker-konfiguráció, a tanúsítványfájlok, a CRL és az ACL-szabályok, amelyeket az Eszközmenedzsment: provisioning, OTA, flotta és a Záróprojekt közvetlenül felhasznál.

19.3.10 Források és ellenőrzési hiányok

- RFC 8446 (TLS 1.3): https://www.rfc-editor.org/rfc/rfc8446 – a kézfogás üzenetneveinek és a tanúsítvány-ellenőrzési lépéseknek a forrása.

- RFC 9147 (DTLS 1.3): https://www.rfc-editor.org/rfc/rfc9147 – a DTLS datagram-adaptációs mechanizmusaihoz.
- RFC 5280 (X.509 és CRL): https://www.rfc-editor.org/rfc/rfc5280 – a tanúsítványlánc, az EKU és a visszavonási lista szabályaihoz.
- Mosquitto 2.0.x dokumentáció: https://mosquitto.org/man/mosquitto-conf-5.html – a `per_listener_settings`, `tls_version`, `require_certificate`, `use_identity_as_username`, `crlfile` és `acl_file` direktívák. A `crlfile` fail-closed viselkedése és az EKU-ellenőrzés hiánya a Mosquitto 2.0.x forráskódján (`net__load_crl_file`, `client_certificate_verify`) alapul; a laborban ellenőrizendő.
- Eclipse Paho C++ 1.3.2 `ssl_options`: https://github.com/eclipse/paho.mqtt.cpp/blob/v1.3.2/src/mqtt/ssions.h – a `set_trust_store`, `set_key_store`, `set_private_key`, `set_verify`, `set_enable_server_cert_auth` és `set_ssl_version` metódusok. A Paho C++ SSL-lel a `paho-mqtt3as` könyvtárra linkel, mert az `MQTTAsync` API-ra épül.
- Eclipse Paho MQTT C 1.3.13 `MQTTClient_SSLOptions`: https://github.com/eclipse/paho.mqtt.c/blob/v1.3.13/src/MQTTClient.h – a `trustStore`, `keyStore`, `privateKey`, `enableServerCertAuth` és `verify` mezők; a `verify` hosztnév-ellenőrzése `X509_check_host`/`X509_check_ip_asc`.
- Eclipse Paho MQTT 5 Java kliens 1.2.5: https://github.com/eclipse/paho.mqtt.java – a `MqttConnectionOptions.setSocketFactory`, `setSSLProperties` és `setHttpsHostnameVerificationEr` API-ja.
- OpenSSL 3.0.13 `ca`, `verify` és `crl` dokumentáció: https://docs.openssl.org/3.0/ – a CRL-generálás, a `-crl_check`, a `-purpose` és a `-verify_hostname` kapcsolókhoz. A fejezet OpenSSL-parancsai helyi, eldobható PKI-n statikusan ellenőrzöttek; a Mosquitto és a Paho futásidejű viselkedése a laborban ellenőrizendő.
- Spring Boot 3.3.4 `@ConfigurationProperties`: https://docs.spring.io/spring-boot/docs/3.3.4/reference/html/ – a YAML-kulcsok típusos kötéséhez.

20 Eszközmenedzsment: provisioning, OTA, flotta

20.1 Biztonságos boot és firmware-aláírás

20.1.1 Motiváció

A TerraSense üvegház-monitorozó rendszerben a `ts-node` eszközök a kihelyezés után fizikailag elérhetők lehetnek illetéktelen személyek számára. Ha a firmware-t védő mechanizmus nélkül bárki lecserélheti a flash tartalmát, a támadó saját kódot futtathat az eszközön: hamis telemetriát küldhet, öntözőszelepet nyithat, vagy további hálózati támadást indíthat. A biztonságos boot (secure boot) elsődleges célja, hogy csak olyan firmware induljon el, amelyet a gyártó vagy az üzemeltető aláírt, és amelyet azóta nem módosítottak. Ez a biztosíték az eszköz teljes szoftveres életciklusának alapja, a gyártástól az OTA-frissítésig.

20.1.2 Fogalmak és működés

Definíció. A **bizalmi lánc** (chain of trust) olyan réteges ellenőrzési sorozat, amelyben minden indítási lépés hitelesíti a következőt. A lánc kiindulópontja a **gyökér bizalmi pont** (root of trust, RoT): egy hardveres vagy gyártás során beégetett komponens, amelyet támadó nem tud módosítani. Az RoT általában egy boot ROM-ban tárolt elsőfokozatú betöltő, amely a második fokozatú betöltő (bootloader) aláírását ellenőrzi, majd a bootloader ellenőrzi az alkalmazás firmware-képét.

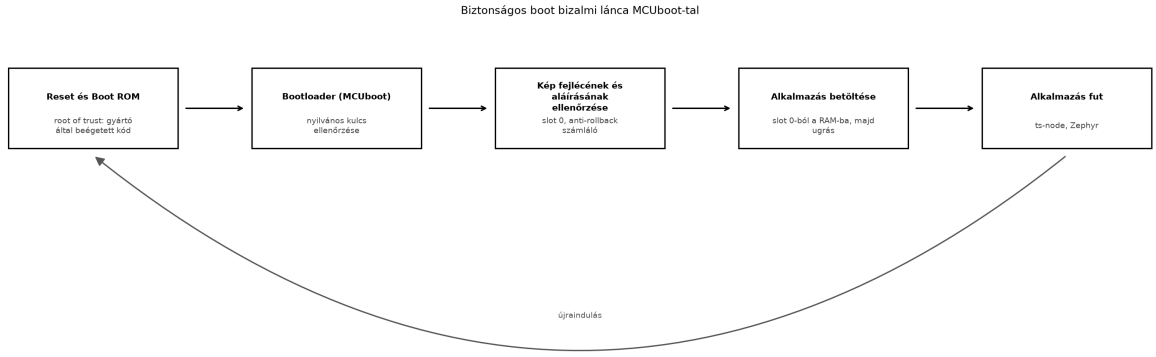
A Zephyr ökoszisztémában az ajánlott nyílt forrású bootloader az **MCUboot**. Az MCUboot két flash-területet, úgynevezett **slotot** használ: a `slot 0` az aktív alkalmazáskép, a `slot 1` pedig a csere- vagy frissítési kép helye. A bootloader induláskor ellenőrzi a `slot 0`-ban lévő kép aláírását, és ha érvényes, átadja a vezérlést az alkalmazásnak. Ha a `slot 1`-ben újabb, érvényes kép van, a bootloader kicserélheti a slotokat, majd elindítja az új verziót.

Definíció. Az **anti-rollback** (visszagörgetés elleni védelem) olyan mechanizmus, amely megakadályozza, hogy egy támadó egy régebbi, ismert hibákat tartalmazó, de érvényes aláírással rendelkező firmware-t töltsön vissza az eszközre. Az MCUboot a képen belül egy védett TLV-ben (`IMAGE_TLV_SEC_CNT`) hordozza a biztonsági számlálót (security counter), és ezt a bootloader egy megbízható, monoton számlálóhoz hasonlítja. A megbízható számláló platformfüggő: lehet OTP/fuse, védett flash, illetve secure element; a bootloader oldalán a `CONFIG_MCUBOOT_HW_DOWNGRADE_PREVENTION=y` kapcsolja be. Hardveres számláló nélkül a `CONFIG_MCUBOOT_DOWNGRADE_PREVENTION=y` csak a verziószámot kényszeríti monotonra, ami nem véd meg minden visszajátszástól. A számláló értékét az aláíró eszköz (`imgtool sign --security-counter ...`, illetve `west sign` esetén `--security-counter auto`) állítja, és a frissítési folyamatnak gondoskodnia kell annak növeléséről.

A firmware-kép formátuma MCUboot esetén a következő fő részekből áll (a slot elejétől a vége felé):

- **képfejléc** (image header, 32 B): magic, load address, `hdr_size`, `protect_tlv_size`, `img_size`, flag-ek és verzió. A slot elején áll; utána a `--header-size` értékéig (például 0x200) nullázott padding következik. Fontos: a `--header-size` nem a fejléc mérete, hanem az alkalmazás kezdő offsetje a sloton belül, hogy a vektorok a bootloader által várt helyre kerüljenek.
- **alkalmazáskép**: a linker által előállított bináris a `header_size` offset után.
- **védett TLV terület** (protected TLV): a kép után helyezkedik el, és a fejlécet is magában foglaló kép SHA-256 hash-ét, a nyilvános kulcs hash-ét, az aláírást (`IMAGE_TLV_SIGNATURE`; ECDSA P-256 esetén 64 B nyers aláírás plusz TLV-fejléc) és a biztonsági számlálót (`IMAGE_TLV_SEC_CNT`) tartalmazza. A bootloader ezt a területet a képfejléc `protect_tlv_size` mezője alapján találja meg, és az aláírással együtt ellenőrzi.
- **nem védett TLV terület** (opcionális): olyan metaadat, amelyet a bootloader nem ellenőriz aláírással.
- **swap trailer**: csak swap módban, a slot végén; a swap állapotot, a magic értéket és a jelzőket tartalmazza. Mérete a szektorok számától, a `BOOT_MAX_ALIGN` értékétől és a választott swap módtól függ (`boot_trailer_sz()`), ezért nem elég a fejléc és az aláírás méretével számolni.

A bootfolyamat a következő ábrán látható:



20.1. ábra: Biztonságos boot bizalmi lánca MCUboot-tal - A reset vektortól az alkalmazásig tartó folyamat. Az ellenőrzés sikertelensége leállítja a bootot, vagy hibakezelő módba lép.

A boot ROM a gyártó által zárt, nem módosítható kód; a bootloader a flash egy elkülönített területén tárolt, szintén aláírt kép. Az ellenőrzéshez a bootloader a flash-be égetett vagy hardveresen védett nyilvános kulcsot használ, így a támadó nem cserélheti a kulcsot a bootloader módosítása nélkül, ami viszont aláírás-ellenőrzéshez kötött.

A firmware-aláírás folyamata az `imgtool` parancssori eszközzel történik, amely az MCUboot referencia-implementációjának része. A gyártó generál egy privát/publikus kulcspárt (általában ECDSA P-256), a privát kulcsot biztonságos helyen tárolja, a nyilvános kulcsot pedig a bootloaderrel együtt beégeti a flash-be. Az alkalmazás lefordított binárisát az `imgtool sign` paranccsal írja alá, amely hozzáfűzi a fejlécet, a TLV-ket és az aláírást.

20.1.3 Döntési szempontok

A biztonságos boot bevezetésekor több tervezési döntést kell meghozni, amelyek befolyásolják a biztonsági szintet, a költséget és a bonyolultságot.

Szempont	Lehetséges megoldások	Előnyök	Hátrányok
Root of trust típusa	Szoftveres (flash-ben tárolt elsőfokozat)	Olcsó, könnyen frissíthető	Támadó által módosítható, ha nincs hardveres védelem
	Hardveres (ROM, Secure Element, TPM)	Magas biztonság, nem módosítható	Drágább, gyártói kötöttség
Aláírási algoritmus	ECDSA P-256 (MCUboot alapértelmezett)	Kis kulcs, gyors ellenőrzés, szabványos	Implementációt igényel a bootloaderben
	RSA (nagyobb kulcs)	Széles körben elterjedt	Nagyobb aláírás, lassabb ellenőrzés
Anti-rollback	Security counter + hardveres tárolás	Megakadályozza a régi sebezhető verziók visszatöltését	Hardveres támogatás kell, a számláló növelése visszafordíthatatlan
	Nincs anti-rollback	Egyszerűbb	Régi verzió visszatölthető, ha a támadó rendelkezik régi érvényes képpel
Kulcstárolás	Bootloaderbe égetett nyilvános kulcs	Egyszerű, nem igényel külön hardvert	Kulcscsere nehéz (firmware frissítést igényel)
	Secure Element (pl. ATECC608)	Kulcs fizikailag védett	Extra komponens, költség

A TerraSense `ts-node` eszközöknél a cél a jó ár-érték arány: kisméretű, olcsó MCU-kon a szoftveres RoT és a flash-be égetett nyilvános kulcs elegendő, ha az eszköz fizikai védelme korlátozott, de a támadási felületet elfogadják. Amennyiben az eszköz kritikus infrastruktúrához kapcsolódik, hardveres Secure Element javasolt.

20.1.4 Kidolgozott példa: aláírt firmware-kép készítése MCUboot-hoz

A labor célja egy valóban MCUboot-kompatibilis, aláírt kép előállítása a `ts-node` firmware-ből, a bootloader és az alkalmazás együttes, sysbuild-es buildjével. A `native_sim` boardon MCUboot nem fut, ezért a bootfolyamat ellenőrzése valós hardvert (nRF52840 DK) igényel; a build, a partíciók, a védett TLV-k és az aláírás hoston is előállíthatók és vizsgálhatók. A lépések a Zephyr 3.7.0 LTS west környezetében futnak; az `imgtool` 2.1.0 verziója a verziómátrix része.

Munkakönyvtár:

```
mkdir -p ~/iot-labor/i058 && cd ~/iot-labor/i058
```

1. lépés: Az imgtool telepítése

```
python3 -m pip install --user imgtool==2.1.0
```

2. lépés: Aláíró kulcspár generálása

```
imgtool keygen --key signing-key.pem --type ecdsa-p256
```

A `signing-key.pem` privát kulcsot a gyártás során védett környezetben kell tartani; a belőle származó nyilvános kulcs kerül a bootloaderbe. A sysbuild a `SB_CONFIG_BOOT_SIGNATURE_KEY_FILE` beállításon keresztül kapja meg ugyanezt a kulcsot, ezért a bootloader és az alkalmazás aláírása ugyanazzal a kulccsal történik.

3. lépés: Az alkalmazás és a sysbuild konfigurációja

A gyakorlathoz minimális Zephyr alkalmazást készítünk, amely a firmware verzióját naplózza.

`CMakeLists.txt`:

```
cmake_minimum_required(VERSION 3.20)
find_package(Zephyr REQUIRED HINTS $ENV{ZEPHYR_BASE})
project(ts_secure_boot_lab)
target_sources(app PRIVATE src/main.c)
```

`prj.conf`:

```
CONFIG_BOOTLOADER_MCUBOOT=y
CONFIG_LOG=y
CONFIG_MAIN_STACK_SIZE=2048
```

`src/main.c`:

```
#include <zephyr/kernel.h>
#include <zephyr/logging/log.h>

LOG_MODULE_REGISTER(i058, LOG_LEVEL_INF);

int main(void)
{
    LOG_INF("Signed image running, version 1.0.0");
    while (1) {
        k_sleep(K_SECONDS(1));
    }
}
```

A `CONFIG_BOOTLOADER_MCUBOOT=y` az alkalmazás oldalán a slot0-ra és a header offsetre vonatkozó linkerbeállítást választja; a bootloader maga külön image, amelyet a sysbuild épít. A `CMakeLists.txt` gyökerében hozzuk létre a `sysbuild.conf` és a `sysbuild/mcuboot.conf` fájlokat:

```
# sysbuild.conf - a bootloader és az alkalmazás közös sysbuild-beállításai
SB_CONFIG_BOOTLOADER_MCUBOOT=y
SB_CONFIG_BOOT_SIGNATURE_TYPE_ECDSA_P256=y

# sysbuild/mcuboot.conf - kizárólag az MCUboot image Kconfigja
CONFIG_MCUBOOT_DOWNGRADE_PREVENTION=y
CONFIG_MCUBOOT_HW_DOWNGRADE_PREVENTION=y
```

A `CONFIG_MCUBOOT_HW_DOWNGRADE_PREVENTION` a platform megbízható, monoton számlálóját használja; a `CONFIG_MCUBOOT_DOWNGRADE_PREVENTION` emellett a verziószám monotonitását is kikényszeríti. Hardveres számláló nélkül az utóbbi csak részleges védelmet ad, ezért a gyártásnak gondoskodnia kell a számláló inicializálásáról és növeléséről.

4. lépés: Sysbuild-es build

A kulcs abszolút útvonalát a parancssorban adjuk meg, mert a Kconfig string nem számol shell-változóval:

```
west build --sysbuild -b nrf52840dk/nrf52840 -d build \
  -DSB_CONFIG_BOOT_SIGNATURE_KEY_FILE="$PWD/signing-key.pem"
```

A sysbuild a következő kimeneteket állítja elő:

```
build/mcuboot/zephyr/zephyr.bin                  # a bootloader képe
build/ts_secure_boot_lab/zephyr/zephyr.bin       # az aláíratlan alkalmazás
build/ts_secure_boot_lab/zephyr/zephyr.signed.bin # a sysbuild által aláírt kép
build/ts_secure_boot_lab/partitions.yml          # slotok, címek, méretek
```

A `partitions.yml` az egyetlen hiteles forrás a `slot0_partition` méretére és a `header-size`/offset értékére; a kézi `imgtool sign` parancsba ezeket kell behelyettesíteni, nem beégetett konstansokat.

5. lépés: A kép és a TLV-k ellenőrzése

```
imgtool verify --key signing-key.pem \
  build/ts_secure_boot_lab/zephyr/zephyr.signed.bin
imgtool dumpinfo build/ts_secure_boot_lab/zephyr/zephyr.signed.bin
```

Várt kimenet (illusztratív):

```
Image version: 1.0.0
Signature valid.
```

Az `imgtool dumpinfo` mutatja a 32 bájtos képfejlécet, a védett TLV-ket (SHA-256, kulcs-hash, aláírás, security counter) és a swap trailer jelenlétét. A `zephyr.signed.bin` mérete nem `alkalmazás + 512 B + 64 B`: a védett TLV-k és a swap trailer is helyet foglalnak, ezért a slotméretet a `partitions.yml` és a tényleges signed kép mérete alapján kell ellenőrizni.

6. lépés: A sign.sh szkript elkészítése

A későbbi OTA-frissítéshez (OTA-frissítés: A/B képek, rollback, delta, fokozatos kiadás) automatizáljuk az aláírást. A szkript a `west sign` parancsot használja, amely a build konfigurációjából és a `partitions.yml`-ből olvassa a header- és slotméretet, ezért nincs beégetett érték:

```
#!/usr/bin/env bash
set -euo pipefail

BUILD_DIR="${1:?build dir}"
VERSION="${2:?version}"
SECURITY_COUNTER="${3:-auto}"

west sign -t imgtool -d "${BUILD_DIR}" -- \
    --key signing-key.pem \
    --version "${VERSION}" \
    --security-counter "${SECURITY_COUNTER}"

chmod +x sign.sh
./sign.sh build/ts_secure_boot_lab 1.0.0 auto
```

A `west sign` a `zephyr.bin`-ből `zephyr.signed.bin`-t készít, és a `--security-counter auto` az imgtool által a verzióból származtatott számlálót használja. Közvetlen `imgtool sign` csak akkor indokolt, ha a paramétereket a build fájljaiból vezetjük le, például:

```
SLOT_SIZE=$(awk '/slot0_partition:/{f=1} f && /size:/{print $2; exit}' \
  build/ts_secure_boot_lab/partitions.yml)
HEADER_SIZE=$(awk -F= '/^CONFIG_ROM_START_OFFSET=/{print $2}' \
  build/ts_secure_boot_lab/zephyr/.config)
imgtool sign --key signing-key.pem --header-size "${HEADER_SIZE}" \
  --align 8 --version 1.0.0 --slot-size "${SLOT_SIZE}" \
  --pad-header --security-counter auto \
  build/ts_secure_boot_lab/zephyr/zephyr.bin signed.bin
```

7. lépés: Flash-elés és bootpróba valós hardveren

A sysbuild-es buildből a `west flash -d build` a bootloadert és az alkalmazást együtt tölti fel az nRF52840 DK-ra. A bootloader a soros konzolon naplózza az ellenőrzés eredményét; sikeres ellenőrzés után az alkalmazás `Signed image running` üzenete jelenik meg. Ez a lépés valós hardvert igényel, ezért a labor környezetében külön végrehajtandó.

8. lépés: Negatív tesztek a valódi bootláncon

A hostoldali `imgtool verify` nem helyettesíti a bootloader döntését. A következő negatív teszteket a céleszközön kell elvégezni; a várt eredmény mindegyiknél az, hogy a bootloader nem indítja el a hibás képet, és a korábbi jó kép marad aktív:

- **Egybites módosítás:** a `zephyr.signed.bin` egy payload-bájtját megváltoztatjuk, majd a slot1-be töltjük (például mcumgr-ral) és frissítést kérünk. Az `imgtool verify` már hoston elbukik; a bootloader a védett TLV hash- vagy aláírás-ellenőrzésén bukik el, és nem vált.
- **Hibás aláírás:** a képet egy másik kulccsal (`other-key.pem`) írjuk alá. A bootloader a beégetett nyilvános kulccsal ellenőriz, ezért elutasítja; a slot0 továbbra is a régi képet indítja.
- **Régi security counter:** v1.1.0-t `--security-counter 2` értékkel, majd v1.0.0-t `--security-counter 1` értékkel töltünk fel. A `CONFIG_MCUBOOT_HW_DOWNGRADE_PREVENTION` miatt a régi számlálójú kép nem indul el.
- **Megszakított frissítés:** a swap közben áramszünetet szimulálunk (reset vagy a tápfeszültség megszakítása). Az MCUboot swap-állapot trailerének köszönhetően a következő indítás vagy befejezi a cserét, vagy visszaáll a korábbi jó slotra; a kép nem „téglásodik el”.

A tesztek pontos kimenete a board és az MCUboot verziójától függ; a labor a naplóban megjelenő elutasítási okot (`signature`, `security counter`, `swap`) rögzíti. A fejezet nem állítja, hogy ezeket a hardveres teszteket elvégezte: a build és az aláírás statikusan ellenőrzött, a bootlánc-próbák a labor feladata.

20.1.5 Ellenőrzés és hibakeresés

A labor során az alábbi parancsokkal győződhetünk meg a folyamat helyességéről:

- `imgtool verify --key signing-key.pem signed.bin` – ellenőrzi az aláírást és kiírja a verziót;
- `imgtool dumpinfo signed.bin` – megjeleníti a képfejléc mezőit (méret, verzió, security counter);
- `grep "version" signed.bin` – binárisban nem célszerű, helyette az `imgtool dumpinfo` használata javasolt.

Tipikus hibák és javításuk:

Tünet	Lehetséges ok	Javítás
`imgtool verify` hibát jelez: signature invalid	A nyilvános kulcs nem egyezik az aláíró privát kulccsal, vagy a kép sérült	Ellenőrizze, hogy a `--key` paraméter ugyanazt a kulcsot használja, mint amellyel aláírták; próbálja újra az aláírást

Tünet	Lehetséges ok	Javítás
`imgtool sign` hibát jelez: header size mismatch	A `--header-size` nem egyezik a build által használt alkalmazás-offsettel	A `--header-size` nem a 32 B-os képfejléc mérete, hanem az alkalmazás kezdő offsetje; a `west sign` ezt a build konfigurációjából olvassa. Kézi híváskor a `partitions.yml` és a `CONFIG_ROM_START_OFFSET` értékét kell használni, nem beégetett 0x200-at
A bootloader nem indul a valós hardveren	A bootloader nincs a flash elejére írva, vagy a `slot 0` kezdőcíme rossz	Ellenőrizze a linker szkriptet és a partíciós táblát; a `west flash` parancsnak a bootloadert és az alkalmazást együtt kell felírnia
Az alkalmazás nem indul a bootloader után	Az aláírt kép nem a `slot 0`-ba került, vagy a kép mérete meghaladja a slot méretét	Ellenőrizze a `--slot-size` értékét és a tényleges partíciós táblát
A fejlesztői debug port aktív	A felszabadított eszközön a debug interfész (SWD, JTAG) engedélyezett, ami a flash tartalmának módosítását teszi lehetővé	Gyártáskor zárja le a debug portot (például biztosítékokkal vagy flash-olvasásvédelemmel), ahogyan a következő szakaszban tárgyaljuk

20.1.6 Debug port lezárása és flash-olvasásvédelem

A biztonságos boot önmagában nem elegendő, ha a támadó közvetlenül a debug interfészen keresztül tudja a flash tartalmát olvasni vagy módosítani. A legtöbb MCU lehetőséget kínál a debug hozzáférés korlátozására:

- **Debug port lezárása:** a JTAG/SWD interfész letiltása, hogy a kihelyezett eszközön ne lehessen külső debuggerrel csatlakozni. Ezt gyakran biztosítékbit (fuse) beégetésével érik el, ami visszafordíthatatlan.
- **Flash-olvasásvédelem (read-out protection, RDP):** a flash tartalom kiolvasásának letiltása. Például az STM32 családban több RDP-szint létezik, amelyek fokozatosan korlátozzák a debug hozzáférést és a külső olvasást.
- **Write protection:** a bootloader és a biztonsági kulcsok tárolási területének írásvédelme hardveres regiszterekkel.

Ezek a beállítások a gyártás során alkalmazandók, mert utólag gyakran nem vagy csak korlátozottan módosíthatók. A fejlesztői példányokon a debug port nyitva marad a teszteléshez, de a gyártási folyamatba be kell építeni a lezárást. A TerraSense esetében a `ts-node` eszközök gyártási konfigurációjában dokumentálni kell ezeket a lépéseket.

20.1.7 Tipikus hibák és félreértések

1. **A képfejléc és a `--header-size` összekeverése.** A képfejléc 32 B; a `--header-size` (például 0x200) az alkalmazás offsetje a sloton belül, benne a fejléccel és a paddinggel.

A védett TLV-k az alkalmazáskép után helyezkednek el, az aláírás nem a fejlécben van, hanem a védett TLV területen.

2. **A biztonságos boot helyettesíti a titkosítást.** A boot ellenőrzi a firmware integritását és hitelességét, de nem titkosítja a flash tartalmát. Ha a firmware üzleti titkokat vagy titkos kulcsokat tartalmaz, azokat külön kell védeni (pl. Secure Element, titkosított flash).
3. **A debug port nyitva hagyása gyengíti a bizalmi láncot.** Hiába az aláírás, ha a támadó a debug interfészen keresztül módosíthatja a flash tartalmát vagy kiolvashatja a kulcsokat. A gyártásnak kötelező eleme a debug lezárása.
4. **Az anti-rollback hiánya.** Ha nincs security counter, egy támadó, aki birtokol egy régebbi, érvényes aláírású firmware-t, visszatöltheti azt, és kihasználhatja a már javított hibákat. Az anti-rollback a frissítési folyamat elengedhetetlen része.
5. **Ugyanaz a kulcs minden eszközhöz.** Ha a gyártásban minden eszközhöz ugyanazt a privát kulcsot használjuk, egyetlen kulcs kompromittálódása az összes eszközt sebezhetővé teszi. Kulcsonként célszerű per-device vagy per-batch kulcsokat alkalmazni, bár ez növeli az üzemeltetési bonyolultságot.

20.1.8 Feladatok

20.1.8.1 a) Fogalmi feladat

Magyarázza el, mi a különbség a root of trust és a bootloader között a biztonságos boot folyamatában. Miért nem elegendő a bootloader aláírása, ha a root of trust nem hardveres?

20.1.8.2 b) Számolási feladat

Egy Zephyr alkalmazás lefordított mérete 128 KiB. A slot elején a képfejléc 32 B, a `--header-size` (az alkalmazás offsetje) 0x200, a védett TLV terület SHA-256 hash-t, kulcs-hash-t, 64 B ECDSA P-256 aláírást és 4 B security countert tartalmaz TLV-fejlécekkel, a swap trailer pedig a szektorok számától függ. Mekkora legalább a `slot 0` mérete, ha a flash programozási egysége 8 B, a slot mérete 0x76000 B, a szektor mérete 4 KiB? Mutassa meg, miért nem elég a 64 B aláírást a fejlécméretbe beszámítani.

20.1.8.3 c) Implementációs feladat

Készítse el a `sign.sh` szkriptet a laborban leírtak szerint, majd futtassa le egy `v1.0.0` verziójú aláírt képre, és ellenőrizze az `imgtool verify` parancs kimenetét. Ez a szkript lesz a továbbvitt eredmény az OTA-frissítés: A/B képek, rollback, delta, fokozatos kiadás alfejezethez.

20.1.8.4 d) Hibakeresési feladat

Módosítsa a `sign.sh`-t úgy, hogy egy másik kulccsal próbálja ellenőrizni az aláírt képet. Mi történik, és milyen hibatünetet tapasztal? Írja le, hogyan lehetne ezt a hibát a gyártási folyamatban automatikusan észlelni.

20.1.9 Megoldások

20.1.9.1 a) Fogalmi feladat megoldása

A root of trust (RoT) a bizalmi lánc első láncszeme, amelynek hitelességét fizikai vagy gyártási garancia biztosítja; ez jellemzően egy gyártó által beégetett boot ROM kód. A bootloader egy második fokozatú betöltő, amelyet az RoT ellenőriz, majd a bootloader ellenőrzi az alkalmazást. Ha az RoT szoftveres (pl. flash-ben tárolt), a támadó módosíthatja azt, és így a teljes ellenőrzési lánc megkerülhető. Hardveres RoT esetén ez nem lehetséges, mert a kód fizikailag nem módosítható.

20.1.9.2 b) Számolási feladat megoldása

A szükséges hely nem `alkalmazás + 512 B + 64 B`. A helyes bontás:

- alkalmazáskép: 128 KiB = 131072 B;
- `--header-size` 0x200 = 512 B (a 32 B képfejléc és a hozzá tartozó padding);
- védett TLV-k: SHA-256 TLV 4 + 32 = 36 B, kulcs-hash TLV 4 + 32 = 36 B, aláírás TLV 4 + 64 = 68 B, security counter TLV 4 + 4 = 8 B, összesen 148 B, 8 B-hez igazítva 152 B;
- swap trailer (scratch/move mód): a 0x76000 B-os slot 4 KiB-es szektorokkal 118 szektort jelent; a swap-állapot tömb 118 × 4 × 3 = 1416 B, ehhez jön a swap size, a három állapotjelző 8 B-os paddingja és a 16 B magic, összesen körülbelül 1472 B. A pontos értéket a `boot_trailer_sz()` és a `partitions.yml` adja.

A minimális slotméret tehát 131072 + 512 + 152 + 1472 = 133208 B, 8 B-hez igazítva 133208 B. A 0x76000 B (483328 B) slot bőven elég; egy 128 KiB-os slot viszont nem férne el. A 64 B csak a nyers ECDSA-aláírás, a TLV-fejlécek, a többi TLV és a swap trailer miatt a tényleges többlet másfél KiB nagyságrendű. A végleges ellenőrzés mindig a `partitions.yml` és a `west sign` által előállított `zephyr.signed.bin` méretének összevetése.

20.1.9.3 c) Implementációs feladat megoldása

A `sign.sh` tartalmát a labor részben megadtuk. Futtatás:

```
./sign.sh build/ts_secure_boot_lab 1.0.0 auto
imgtool verify --key signing-key.pem \
  build/ts_secure_boot_lab/zephyr/zephyr.signed.bin
```

Várt kimenet:

```
Image version: 1.0.0
Signature valid.
```

A szkript a `signing-key.pem` privát kulcsot használja, a `west sign` pedig a header- és slotméretet a build konfigurációjából és a `partitions.yml`-ből olvassa, ezért nincs beégetett boardparaméter. A kimenet a `zephyr.signed.bin`; az OTA-frissítés: A/B képek, rollback, delta, fokozatos kiadás OTA-frissítés során ugyanezt a szkriptet hívjuk meg az új verzióval és növelt security counterrel.

20.1.9.4 d) Hibakeresési feladat megoldása

Generáljunk egy másik kulcsot:

```
imgtool keygen --key other-key.pem --type ecdsa-p256
imgtool verify --key other-key.pem signed.bin
```

Várt kimenet: hibaüzenet, például `Signature invalid` vagy hasonló, mert a nyilvános kulcs nem egyezik az aláíró privát kulccsal. A gyártási folyamatban ezt úgy lehet automatikusan észlelni, hogy a CI/CD pipeline az aláírás után azonnal lefuttatja az `imgtool verify` parancsot a hivatalos nyilvános kulccsal, és csak akkor engedi kiadni a firmware-t, ha a kilépési kód 0. A hostoldali verify azonban csak a fájl ellenőrzése: a valódi bootlánc-tesztet a 8. lépés írja le (hibás aláírású kép a slot1-ben, amit maga az MCUboot utasít el). A bootloader a beégetett nyilvános kulccsal ellenőriz, és hibás kulcs esetén nem indítja el az alkalmazást.

20.1.10 Rövid összefoglaló

A biztonságos boot a bizalmi lánc elvén alapul: a root of trust ellenőrzi a bootloadert, a bootloader pedig az alkalmazás aláírását. Az MCUboot a Zephyr ajánlott bootloadere, két slotot (`slot 0`, `slot 1`) használ; az aláírás és a security counter a kép utáni védett TLV területen van, nem a 32 bájtos képfejlécben. Az anti-rollback a platform megbízható, monoton számlálójával (`CONFIG_MCUBOOT_HW_DOWNGRADE_PREVENTION`) akadályozza meg a régebbi, sebezhető verziók visszatöltését. A debug portok lezárása és a flash-olvasásvédelem a fizikai támadások ellen véd. **Továbbvitt eredmények:** a `~/iot-labor/i058/` könyvtárban létrehozott `signing-key.pem` privát kulcs és a futtatható `sign.sh` szkript, amely a `west sign` parancsot használja, és a header-/slotméretet a build konfigurációjából és a `partitions.yml`-ből olvassa. A szkript ugyanazt a kulcsot és slot-beállításokat használja, és az OTA-frissítés: A/B képek, rollback, delta, fokozatos kiadás alfejezetben az új firmware verziók aláírására fog szolgálni.

20.1.11 Források és ellenőrzési hiányok

- MCUboot dokumentáció: https://docs.mcuboot.com/ (a képformátum, slotok, aláírás és anti-rollback leírása)
- Zephyr Project dokumentáció: https://docs.zephyrproject.org/latest/ (a `CONFIG_BOOTLOADER_MCUBOOT` és a bootloader integráció)
- IETF SUIT munkacsoport: https://datatracker.ietf.org/wg/suit/about/ (szabványos firmware-frissítési keretrendszer)
- OMA LwM2M specifikációk: https://www.openmobilealliance.org/specifications/lwm2m (eszközmenedzsment és firmware-frissítési objektumok)

Ellenőrzési igény: az `imgtool==2.1.0` és a hozzá tartozó `west sign` viselkedése, valamint a `--header-size`/`--slot-size` konkrét értékei boardról boardra és MCUboot-verziónként változnak, ezért a `partitions.yml` és a build config az irányadó. A `SB_CONFIG_BOOTLOADER_MCUBOOT`, `SB_CONFIG_BOOT_SIGNATURE_TYPE_ECDSA_P256`, `SB_CONFIG_BOOT_SIGNATURE_KEY_FILE`, `CONFIG_MCUBOOT_DOWNGRADE_PREVENTION` és `CONFIG_MCUBOOT_HW_DOWNGRADE_PREVENTION` nevek a Zephyr 3.7.0/MCUboot 2.1.0 párban érvényesek; a hardveres bootlánc-tesztek az nRF52840 DK-n végrehajtandók. A debug port lezárására és flash-olvasásvédelemre vonatkozó beállítások gyártó- és MCU-specifikusak.

20.2 Gyártás és üzembe helyezés (provisioning, onboarding)

20.2.1 Motiváció

A TerraSense üvegház-telepítés során több tucat vagy akár több száz `ts-node` kerül ki a helyszínekre. Mindegyiknek egyedi azonosítóval, saját tanúsítvánnyal és a helyi hálózatra érvényes konfigurációval kell rendelkeznie, mielőtt az első telemetriai üzenetet elküldheti. Ha a gyártósoron minden eszközbe ugyanazt a privát kulcsot és tanúsítványt írnánk, egyetlen eszköz kompromittálódása az egész flottát veszélyeztetné. Ha viszont minden eszközt kézzel regisztrálnánk a backendben, az üzembe helyezés lassú, hibás és drága lenne. A provisioning és az onboarding éppen ezt a két problémát oldja meg: automatizálja az identitás kiadását, és a helyszíni beállítást biztonságos, de kezelhető folyamattá teszi.

20.2.2 Fogalmak és működés

Definíció. A **provisioning** az a folyamat, amelynek során az eszköz megkapja az egyedi identitását (sorozatszám, kulcs, tanúsítvány), valamint a kezdeti biztonsági és konfigurációs anyagokat. Az **onboarding** ennek a helyszíni, jellemzően a hálózati csatlakozást és a felügyeleti rendszerbe való bevezetést magában foglaló része. A **zero-touch provisioning (ZTP)** olyan automatizált folyamat, amely minimális emberi beavatkozással juttatja el az eszközt a gyártótól a működő állapotig. A **just-in-time regisztráció (JITR)** pedig olyan backend-oldali mechanizmus, amely az eszköz első, hiteles kapcsolódásakor automatikusan hozza létre az eszköznyilvántartási rekordot.

20.2.2.1 Gyártási lépések: kulcsgenerálás, tanúsítvány, sorozatszám

A gyártósor a `ts-node` minden egyes példányához hozzárendel egy monoton növekvő sorozatszámot, például `TS-0001`. A kritikus lépés a kulcspár generálása: ezt lehetőség szerint közvetlenül az eszközön, egy biztonságos elemben (secure element) kell végrehajtani, hogy a privát kulcs soha ne hagyja el a hardvert. Ha a gyártósor egyszerűbb, szoftveres kulcsgenerálást használ, a privát kulcsot a gyártás során, védett környezetben írják a flash-be. A publikus kulcsból tanúsítvány-kérelmet (CSR) készítenek, amelyet a gyártó vagy a megrendelő CA-ja ír alá. Az így kapott X.509 tanúsítvány tartalmazza a sorozatszámot (például a CN mezőben), a szervezetet, és opcionálisan SAN-mezőben az eszköz URI-ját. A tanúsítvány láncot és a CA tanúsítványt is eltárolják az eszközön.

20.2.2.2 Helyszíni onboarding: BLE-alapú beállítás

A telepítő szakember a helyszínen bekapcsolja a `ts-node`-ot. Az eszköz a `BOOT` állapot után – amelyet az Eszközszoftver-architektúra: rétegek és állapotgépek alfejezetben definiáltunk – észleli, hogy a `provisioned` flag nem igaz, ezért a `PROVISIONING` állapotba lép. Itt a rádió BLE perifériaként hirdet, például `TS-PROV-<sorozatszám>` néven. A telepítő egy laptopon vagy mobilon futó `ts-provision` CLI-vel csatlakozik, és megadja a hálózati profilt, a site és node azonosítókat, valamint a gateway és a backend végpontját. A labor alapértelmezett `native_sim-udp` profiljában nincs Wi-Fi: a node a gateway UDP-címére küld, ahogy az IP az eszközön és a MQTT a gyakorlatban laborban szerepelt. Opcionális, valós Wi-Fi hardveren a `wifi` profil adja át az SSID-t és a jelszót; a BLE-kapcsolatot ilyenkor párosítással (pairing) és titkosítással kell védeni, hogy a hálózati hitelesítő adatok ne szivárogjanak. A `ts-node` a kapott adatokat a Zephyr settings/NVS rétegében tárolja, és csak a sikeres `settings_save_one` után állítja

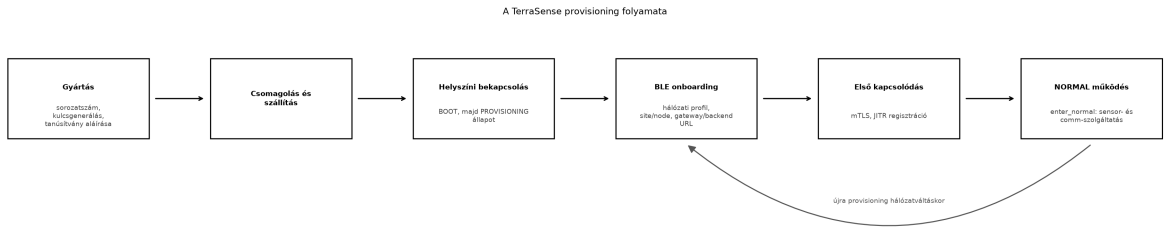
be a `provisioned` flaget, majd `APP_EVT_PROVISIONED` eseménnyel a `NORMAL` állapotba lép (a közös `enter_normal()` útvonalon, amely elindítja a sensor- és comm-szolgáltatást). A `NORMAL` állapotban a `native_sim-udp` profil a gatewaynek küld UDP-telemetriát, a `wifi` profil pedig csatlakozik a megadott SSID-hez, és onnan éri el a broker/gateway végpontot.

20.2.2.3 Eszköznyilvántartás és első kapcsolódás (JITR)

A backend az eszközöket egy nyilvántartásban tartja, amelyben minden rekord tartalmazza legalább a sorozatszámot, a site/node azonosítókat, a tanúsítvány ujjlenyomatát és a regisztráció állapotát. Kézi onboarding esetén a rendszergazda előre felveszi az eszközöket. JITR esetén a backend az első TLS-kézfogás során kapott tanúsítványt ellenőrzi a gyártói CA-val, majd ha érvényes, automatikusan létrehozza a rekordot. Ez gyors, de csak akkor biztonságos, ha a CA privát kulcsa megfelelően védett; különben bárki regisztrálhat hamis eszközt.

20.2.2.4 Matter onboarding röviden

A Matter okosotthon-szabvány (korábbi nevén Project CHIP) hasonló onboarding-jellegű mechanizmust definiál: a készülék egy beállítási kódot (setup code) és QR-kódot tartalmaz, BLE-n vagy hálózaton keresztül fogadja a hálózati hitelesítő adatokat, majd IP-alapú szolgáltatásokkal regisztrálja magát a Matter-ökoszisztémában. A TerraSense-ben saját, egyszerűbb folyamatot használunk, de a koncepció analóg. A Matter specifikus lépései és az aktuális verziók ellenőrzési igényűek.



20.2. ábra: A TerraSense provisioning folyamata - A lépések a ts-node életciklus-állapotgépéhez illeszkednek. A BLE onboarding és a JITR regisztráció csak a NORMAL állapot előtt történik.

A folyamat azért kapcsolódik szorosan az életciklus-állapotgéphez, mert a `PROVISIONING` állapot csak akkor aktív, ha a `provisioned` flag hamis, vagy ha a felhasználó explicit módon gyári visszaállítást kér. Új hálózatra költözéskor a technikus törli a beállított konfigurációt, így az eszköz újra a `PROVISIONING` állapotba kerül.

20.2.3 Döntési szempontok: zero-touch vs. kézi onboarding

A választás alapvetően a flotta méretétől, a telepítői szakértelemtől és a biztonsági követelményektől függ.

Szempont	Zero-touch (ZTP)	Kézi onboarding (pl. BLE CLI)
Emberi beavatkozás	minimális, gyakran csak áram alá helyezés	minden eszköznél szükséges
Hálózati hitelesítő adás	előre telepített, gyártósori vagy hálózati bootstrap	technikus adja meg
Biztonsági kockázat	a gyártósori konfiguráció és a bootstrap csatorna védelme kritikus	a BLE párosítás és a technikus képzése a kulcs
Költség gyártásban	magasabb (biztonságos elem, tesztelés)	alacsonyabb
Költség üzembe helyezésben	alacsony	magas, hibalehetőséggel
Alkalmazási példa	nagy távközlési vagy energiaszolgáltatói flotta	egyedi üvegházak, kis telephelyek

A TerraSense mintarendszerben a kézi BLE onboarding mellett döntöttünk, mert a célkörnyezet kis flotta, ahol a telepítő szakember jelen van, és a gyártósori automatikus konfiguráció aránytalanul drága lenne. A JITR-t viszont bekapcsoljuk, mert így nem kell minden eszközt előre kézzel regisztrálni a backendben.

20.2.4 Kidolgozott példa: provisioning labor és JITR szimuláció

A következő labor azt mutatja be, hogyan néz ki a TerraSense provisioning a gyakorlatban. Mivel a `native_sim` környezetben valódi BLE nincs, a BLE-s onboarding üzenetváltást egy Python CLI szimulálja: a technikus „csatlakoztatja” az eszközt, átadja a konfigurációt, a script pedig elvégzi a tanúsítványgenerálást és a backend JITR regisztrációját. Valós hardveren ugyanez a config a BLE GATT íráson keresztül érkezne meg a Zephyr settings-be.

Munkakönyvtár: `~/iot-labor/i059`

```
mkdir -p ~/iot-labor/i059/devices ~/iot-labor/i059/ca
cd ~/iot-labor/i059
# Labor-CA létrehozása, ha az i056-ból még nem áll rendelkezésre
openssl ecparam -genkey -name prime256v1 -out ca/ca.key
openssl req -x509 -new -key ca/ca.key -sha256 -days 3650 \
  -subj "/C=HU/O=TerraSense Lab/CN=TerraSense Lab CA" -out ca/ca.crt
```

Hozzuk létre a provisioning scriptet `provision_ts_node.py` néven:

```
#!/usr/bin/env python3
import argparse, fcntl, hashlib, json, os, pathlib, re, shutil, subprocess, sys, tempfile,
↪  datetime

SERIAL_RE = re.compile(r"^TS-[0-9]{4,8}$")
ID_RE = re.compile(r"^[a-z0-9][a-z0-9_-]{0,31}$")

def run(cmd):
    subprocess.run(cmd, check=True)

def utc_now():
    return datetime.datetime.now(datetime.timezone.utc).isoformat()

def cert_fingerprint(cert_path):
    # A PEM-fájl bájtjai helyett a DER-kódolt tanúsítvány SHA-256 lenyomata.
    der = subprocess.run(
```

```
        ["openssl", "x509", "-in", str(cert_path), "-outform", "DER"],
        check=True, capture_output=True).stdout
    return hashlib.sha256(der).hexdigest()

def atomic_write(path, text):
    path = pathlib.Path(path)
    fd, tmp = tempfile.mkstemp(dir=path.parent, prefix=path.name + ".", suffix=".tmp")
    try:
        with os.fdopen(fd, "w", encoding="utf-8") as f:
            f.write(text)
            f.flush()
            os.fsync(f.fileno())
        os.replace(tmp, path)
    finally:
        if os.path.exists(tmp):
            os.unlink(tmp)

def load_registry(path):
    return json.loads(path.read_text(encoding="utf-8")) if path.exists() else {}

def main():
    p = argparse.ArgumentParser(description="TerraSense ts-node provisioning")
    p.add_argument("--serial", required=True)
    p.add_argument("--site", required=True)
    p.add_argument("--node", required=True)
    p.add_argument("--network-profile", choices=("native_sim-udp", "wifi"),
                   default="native_sim-udp")
    p.add_argument("--gateway-host", default="127.0.0.1")
    p.add_argument("--gateway-port", type=int, default=4000)
    p.add_argument("--ssid")
    p.add_argument("--password")
    p.add_argument("--backend-url", default="mqtts://broker.local:8883")
    p.add_argument("--ca-dir", required=True)
    p.add_argument("--authorized-sites", default="authorized-sites.json")
    p.add_argument("--rotate", action="store_true",
                   help="meglévő sorozatszám kulcsának rotációja")
    p.add_argument("--reason", default="")
    args = p.parse_args()

    if not SERIAL_RE.fullmatch(args.serial):
        sys.exit("Érvénytelen sorozatszám (várt alak: TS-0001)")
    if not ID_RE.fullmatch(args.site) or not ID_RE.fullmatch(args.node):
        sys.exit("Érvénytelen site vagy node azonosító")
    if args.network_profile == "wifi" and (not args.ssid or not args.password):
        sys.exit("A wifi profilhoz --ssid és --password kell")
    if args.rotate and not args.reason:
        sys.exit("A rotációhoz --reason kötelező")

    auth_path = pathlib.Path(args.authorized_sites)
    if auth_path.exists():
        allowed_sites = json.loads(auth_path.read_text(encoding="utf-8")).get("sites", [])
        if args.site not in allowed_sites:
            sys.exit(f"A site ({args.site}) nincs engedélyezve a JITR számára")
    else:
        print("FIGYELEM: nincs authorized-sites.json; a labor minden site-ot elfogad")

    devices_root = pathlib.Path("devices").resolve()
    base = (devices_root / args.serial).resolve()
    if devices_root not in base.parents:
        sys.exit("A célútvonal a devices könyvtáron kívülre mutat")

    ca_crt = pathlib.Path(args.ca_dir) / "ca.crt"
    ca_key = pathlib.Path(args.ca_dir) / "ca.key"
```

```
    if not ca_crt.exists() or not ca_key.exists():
        sys.exit("CA fájlok nem találhatók")

    reg_path = pathlib.Path("devices.json")
    audit_path = pathlib.Path("devices-audit.jsonl")
    lock_path = pathlib.Path(".devices.lock")

    # A nyilvántartás olvasás-ellenőrzés-írás szakasza zárolt és atomikus.
    with open(lock_path, "w") as lock:
        fcntl.flock(lock, fcntl.LOCK_EX)
        devices = load_registry(reg_path)
        if args.serial in devices and not args.rotate:
            sys.exit("Sorozatszám már regisztrálva; rotációhoz használja a --rotate kapcsolót")
        if args.rotate and args.serial not in devices:
            sys.exit("A --rotate csak már regisztrált sorozatszámhoz használható")

        if base.exists() and any(base.iterdir()):
            if args.serial not in devices:
                sys.exit("Befejezetlen korábbi provisioning; előbb tisztázza a devices/<serial>
↪   könyvtárat")
            if not args.rotate:
                sys.exit("A devices/<serial> már létezik; rotációhoz --rotate")
            archive = base / "archive" / datetime.datetime.now().strftime("%Y%m%dT%H%M%SZ")
            archive.mkdir(parents=True, exist_ok=True)
            for name in ("device.key", "device.csr", "device.crt", "provisioning.json"):
                src = base / name
                if src.exists():
                    shutil.move(str(src), str(archive / name))

        base.mkdir(parents=True, exist_ok=True)

        # Eszközkulcs és CSR generálása, majd aláírás a labor-CA-val
        run(["openssl", "ecparam", "-genkey", "-name", "prime256v1",
             "-out", str(base / "device.key")])
        run(["openssl", "req", "-new", "-key", str(base / "device.key"),
             "-subj", f"/C=HU/O=TerraSense/CN=ts-node-{args.serial}",
             "-out", str(base / "device.csr")])
        run(["openssl", "x509", "-req", "-in", str(base / "device.csr"),
             "-CA", str(ca_crt), "-CAkey", str(ca_key),
             "-CAcreateserial", "-days", "3650",
             "-out", str(base / "device.crt"), "-sha256"])

        # JITR szimuláció: tanúsítvány ellenőrzése
        run(["openssl", "verify", "-CAfile", str(ca_crt), str(base / "device.crt")])

        fingerprint = cert_fingerprint(base / "device.crt")
        now = utc_now()
        config = {
            "serial": args.serial,
            "site": args.site,
            "node": args.node,
            "network_profile": args.network_profile,
            "gateway": {"host": args.gateway_host, "port": args.gateway_port},
            "backend_url": args.backend_url,
            "provisioning_ts": now,
        }
        if args.network_profile == "wifi":
            config["wifi"] = {"ssid": args.ssid, "password": args.password}
        (base / "provisioning.json").write_text(json.dumps(config, indent=2))

        record = {
            "serial": args.serial,
            "site": args.site,
```

```
            "node": args.node,
            "network_profile": args.network_profile,
            "cert_fingerprint_sha256": fingerprint,
            "status": "provisioned",
            "onboarded_at": now,
        }
        if args.serial in devices:
            record["previous_fingerprint_sha256"] = \
                devices[args.serial].get("cert_fingerprint_sha256")
            record["rotated_at"] = now
            record["rotation_reason"] = args.reason
            # A régi tanúsítvány visszavonása külön lépés (iO61 CRL-szolgáltatás).
            record["revocation_required"] = True
            action = "rotate"
        else:
            action = "provision"
        devices[args.serial] = record
        atomic_write(reg_path, json.dumps(devices, indent=2))
        with audit_path.open("a", encoding="utf-8") as audit:
            audit.write(json.dumps({
                "ts": now,
                "action": action,
                "serial": args.serial,
                "fingerprint_sha256": fingerprint,
                "reason": args.reason,
            }) + "\n")

    print(f"Provisioning kész: {args.serial} ({action})")

if __name__ == "__main__":
    main()
```

A script a sorozatszámot `^TS-[0-9]{4,8}$`, a site/node azonosítókat `^[a-z0-9][a-z0-9_-]{0,31}$` mintával ellenőrzi, a célútvonalat pedig `resolve()` után a `devices` könyvtáron belülre kényszeríti. Így a `../`-t vagy abszolút utat tartalmazó serial nem írhat a könyvtáron kívülre, és a strict minta miatt a `-subj` sem kaphat további RDN-t (például `/CN=` vagy új sor karaktert).

A `devices.json` frissítése `fcntl.flock`-kal zárolt olvasás-ellenőrzés-írás szakaszban, atomikusan történik: a script ideiglenes fájlba ír, `fsync`-el, majd `os.replace`-szel cseréli le a nyilvántartást. Így párhuzamos futás vagy megszakítás nem hagy félkész `devices.json`-t. Alapértelmezésben a már regisztrált sorozatszámot megtagadja; rotációhoz `--rotate --reason "..."` kell, ekkor a régi kulcs és tanúsítvány a `devices/<serial>/archive/<időbélyeg>/` alá kerül, a rekord pedig `previous_fingerprint_sha256`, `rotated_at`, `rotation_reason` mezőt kap, és a művelet bekerül a `devices-audit.jsonl` naplóba. A régi tanúsítvány visszavonása külön lépés (az Flottakezelés CRL-szolgáltatása); a rotáció csak megjelöli `revocation_required=true` értékkel.

A `cert_fingerprint_sha256` nem a PEM-fájl bájtjainak, hanem a DER-kódolt tanúsítvány SHA-256 lenyomata, ezért megegyezik az `openssl x509 -in device.crt -noout -fingerprint -sha256` kimenetével (a kettőspontok nélkül).

A labor a JITR **engedélyezési pontját** is modellezi: csak az `authorized-sites.json` fájlban felsorolt site-okra fogad el provisioningot. Hozzuk létre ezt a fájlt:

```
cat > authorized-sites.json <<'EOF'
{"sites": ["gh1"]}
EOF
```

Egy megbízható CA-tól származó, de nem engedélyezett site-ra (például `--site gh2`) kért eszköz így elutasításra kerül, nem kerül automatikusan a nyilvántartásba.

A JITR-szimuláció korlátai. A script nem végez valódi TLS-kézfogást, nem igazolja a privát kulcs birtoklását (proof of possession) a szerver felé, és az engedélyezést egy helyi JSON-lista modellezi. A valódi JITR a backend oldalán a TLS-kézfogáskor ellenőrzi a tanúsítványláncot, a CN/SAN és a kért site/node megfelelését, majd egy engedélyezési szabály (allowlist, jóváhagyás, esetleg attestation) alapján dönt a regisztrációról. A labor ezt a döntési pontot szimulálja, nem a teljes JITR-folyamatot; a proof of possession és a szerveroldali engedélyezés a záróprojekt feladata.

Futtassuk le egy `ts-node` példányra a labor alapértelmezett, `native_sim` UDP-profiljával:

```
python3 provision_ts_node.py --serial TS-0001 --site gh1 --node n001 \
  --network-profile native_sim-udp --gateway-host 127.0.0.1 --gateway-port 4000 \
  --backend-url mqtts://broker.local:8883 --ca-dir ca
```

Opcionális, valós Wi-Fi hardveren ugyanez a `wifi` profillal, ahol a hálózati hitelesítő adatok is átadódnak:

```
python3 provision_ts_node.py --serial TS-0002 --site gh1 --node n002 \
  --network-profile wifi --ssid TerraSenseAP --password labtitok \
  --backend-url mqtts://broker.local:8883 --ca-dir ca
```

Várt kimenet:

```
Provisioning kész: TS-0001 (provision)
```

A `devices/TS-0001/` könyvtárban megjelenik a `device.key`, `device.csr`, `device.crt` és `provisioning.json`. A `devices.json` nyilvántartásba bekerül a rekord a tanúsítvány DER-alapú SHA-256 ujjlenyomatával. A tanúsítványt a `openssl verify` parancs a labor-CA-val szemben ellenőrzi, ami a JITR szimulációja. A `provisioning.json` tartalma a következő (a pontos időbélyeg eltérhet):

```
{
  "serial": "TS-0001",
  "site": "gh1",
  "node": "n001",
  "network_profile": "native_sim-udp",
  "gateway": {"host": "127.0.0.1", "port": 4000},
  "backend_url": "mqtts://broker.local:8883",
  "provisioning_ts": "2025-01-01T00:00:00+00:00"
}
```

A `wifi` profilban a fenti objektum kiegészül egy `"wifi": {"ssid": "TerraSenseAP", "password": "labtitok"}` blokkal; a `native_sim-udp` profil szándékosan nem tartalmaz Wi-Fi-hitelesítő adatot, mert a szimulált node UDP-n küld a gatewaynek, és nem csatlakozik Wi-Fihez.

Takarítás csak a labor saját fájljaira terjedhet ki:

```
rm -rf ~/iot-labor/i059/devices ~/iot-labor/i059/devices.json ~/iot-labor/i059/ca
```

20.2.5 Ellenőrzés és hibakeresés

A sikeres provisioning után három dolgot kell ellenőrizni: a tanúsítvány érvényes-e (a `openssl verify` kimenetében `OK` szerepel), a `devices.json` tartalmazza-e az új rekordot, és a `provisioning.json` site/node azonosítói helyesek-e. A `native_sim-udp` profilban a `NORMAL` állapotba lépés után a gateway UDP-portján (4000) megjelenő telemetria, majd a gateway

MQTT-publikációja igazolja a működést; valós `wifi` hardveren a Wi-Fi-csatlakozás és az MQTT `terrasense/gh1/n001/status` retained üzenet megjelenése is ellenőrizendő.

Tünet	Lehetséges ok	Javítás
`openssl verify` hibát ad	a CA-tanúsítvány nem egyezik, vagy a lánc hiányos	ellenőrizze a `--ca-dir` útvonalat és a CA-t
A script nem találja a CA fájlokat	rossz munkakönyvtár vagy hiányzó CA	futtassa a labor CA-létrehozó parancsát, vagy másolja az Eszközidentitás: X.509-tanúsítványok, PKI és kulcstárolás alfejezetből
Az eszköz nem lép NORMAL állapotba	a `provisioned` flag nem íródott be a settings-be	ellenőrizze a Zephyr settings hívásokat és a flash-t
A JITR elutasítja az eszközt	a tanúsítvány lejárt, vagy a CN nem illeszkedik a sorozatszámhoz	frissítse a tanúsítványt, illetve javítsa a kiállítási paramétereket

20.2.6 Tipikus hibák és félreértések

1. **Közös privát kulcs használata több eszközön.** Ha egy kulcsot minden eszközbe beírnak, egy kompromittálódás az egész flottát leleplezi. Minden eszköznek saját kulcspárral és tanúsítvánnyal kell rendelkeznie.
2. **Jelszó plaintextben tárolása.** A `provisioning.json` csak a `wifi` profilban, a laborban egyszerűsítésből tartalmazza a Wi-Fi jelszót; a `native_sim-udp` profil nem tartalmaz Wi-Fi-hitelesítő adatot. Éles rendszerben a jelszót titkosítva, például a Zephyr settings titkosított részében vagy secure elementben kell tárolni.
3. **JITR túl megengedő CA-val.** Ha a JITR szabály túl széles CA-t fogad el, akkor bárki, aki hozzáfér egy érvényes tanúsítványhoz, hamis eszközt regisztrálhat. A JITR-t mindig a gyártói CA-ra kell korlátozni.
4. **BLE onboarding párosítás nélkül.** Ha a BLE-kapcsolatot nem védi párosítás és titkosítás, a hálózati hitelesítő adatok elfoghatók. A TerraSense-ben a BLE onboardingnak mindig párosított kapcsolaton kell futnia.

20.2.7 Feladatok

1. **Fogalmi/döntési feladat:** Magyarázza el, mi a különbség a provisioning és az onboarding között, és adja meg, hogy a TerraSense-ben melyik lépés tartozik melyik fogalomhoz.
2. **Tervezési feladat:** Tervezze meg a `devices.json` nyilvántartási sémáját úgy, hogy az támogassa a flotta későbbi kezelését (például parancsok küldése, OTA-csoportok). Milyen mezőket venne fel a mostani rekordba?
3. **Implementációs feladat:** Bővítse a `provision_ts_node.py` scriptet úgy, hogy ellenőrizze a sorozatszám egyediségét, és már regisztrált sorozatszám esetén hibával lépjen ki.
4. **Hibakeresési feladat:** Egy `ts-node` nem tud csatlakozni a backendhez, mert a JITR elutasítja. Soroljon fel legalább három lehetséges okot, és mindegyikhez adjon diagnosztikai lépést.

20.2.8 Megoldások

1. A provisioning az identitás és a konfigurációs anyagok gyártási kiadása (kulcs, tanúsítvány, sorozatszám). Az onboarding a helyszíni beállítás (hálózati csatlakozás, BLE-s konfiguráció) és a felügyeleti rendszerbe való bevezetés. A TerraSense-ben a kulcsgenerálás és a tanúsítvány aláírása provisioning, a BLE-s Wi-Fi beállítás és a JITR regisztráció onboarding.
2. A `devices.json` rekordba például érdemes felvenni: `ota_group`, `firmware_version`, `last_seen_ts`, `desired_state`, `reported_state`, `tags`. Ez lehetővé teszi a flottakezelést (Flottakezelés: nyilvántartás, konfiguráció, állapot) és az OTA-csoportos frissítést (OTA-frissítés: A/B képek, rollback, delta, fokozatos kiadás).
3. A script a `devices.json` zárolt betöltése után ellenőrzi, hogy a `args.serial` már kulcsként szerepel-e; ha igen és nincs `--rotate`, `sys.exit("Sorozatszám már regisztrálva...")`. A rotáció külön, indoklással (`--reason`) történik, a régi kulcsot és tanúsítványt archiválja, a registry `previous_fingerprint_sha256/rotated_at` mezővel és auditbejegyzéssel frissül; a régi tanúsítvány tényleges visszavonása a Flottakezelés: nyilvántartás, konfiguráció, állapot CRL-lépése. Ez a JITR megelőzése is: a backendnek sem szabad kétszer regisztrálnia.
4. Lehetséges okok: (a) a tanúsítvány aláírója nem a megbízható CA, (b) a tanúsítvány lejárt, (c) a tanúsítvány CN-je nem egyezik a sorozatszámmal, (d) a backend JITR szabálya túl szigorú, vagy a devices.json sérült. Diagnosztika: `openssl verify`, a tanúsítvány lejárati idejének ellenőrzése, a CN kiírása, a backend logok megtekintése.

20.2.9 Rövid összefoglaló

A TerraSense provisioning két szakaszból áll: a gyártás során minden `ts-node` egyedi sorozatszámot (pl. `TS-0001`), saját ECC-kulcspárt és a labor-CA által aláírt X.509 tanúsítványt kap. A helyszíni onboarding BLE-kapcsolaton keresztül történik: a technikus megadja a hálózati profilt (`native_sim-udp` a laborhoz, `wifi` opcionális hardverhez), a site/node azonosítókat (`gh1`, `n001`), a gateway UDP-címét és a backend MQTT/TLS URL-jét. A `ts-node` ezt a Zephyr settings/NVS-ben tárolja, a `provisioned` flaget csak sikeres mentés után állítja be, majd `APP_EVT_PROVISIONED` eseménnyel a `NORMAL` állapotba lép, ahol az `enter_normal()` elindítja a sensor- és comm-szolgáltatást; a `native_sim-udp` profil UDP-n küld a gatewaynek, a `wifi` profil pedig a megadott SSID-hez csatlakozik. A backend JITR-t használ: az első TLS-kézfogás tanúsítványát ellenőrzi, és automatikusan felveszi az eszközt a `devices.json` nyilvántartásba. A laborfájlok: `~/iot-labor/i059/provision_ts_node.py`, `~/iot-labor/i059/devices/TS-0001/` és `~/iot-labor/i059/devices.json`. **Továbbvitt eredmény:** ez a provisioning-folyamat, a `devices.json` rekordséma és a sorozatszám-konvenció szolgál alapul a Flottakezelés: nyilvántartás, konfiguráció, állapot és a Záróprojekt: a teljes TerraSense rendszer összeállítása és átvétele alfejezetekben.

20.2.10 Források és ellenőrzési hiányok

A tanúsítványkezelés alapelvei az X.509 szabványból (RFC 5280) és a TLS 1.3 specifikációból származnak. A BLE onboarding koncepciója a Bluetooth SIG specifikációin alapul. A Matter onboarding-rész kifejezetten ellenőrzési igényű: a konkrét setup code formátum, a BLE-szolgáltatások UUID-jei és a hálózati bootstrap üzenetek a Connectivity Standards Alliance aktuális dokumentációjától függenek. A Zephyr settings/NVS API-k és a biztonságos elem támogatás konkrét Kconfig- és devicetree-beállításai a Zephyr aktuális kiadásának dokumentációjában ellenőrizhetők.

20.3 OTA-frissítés: A/B képek, rollback, delta, fokozatos kiadás

20.3.1 Motiváció

A TerraSense üvegházban a `ts-node` eszközök folyamatosan mérik a hőmérsékletet, páratartalmat és talajnedvességet, valamint öntözőszelepet vezérelnek. Tegyük fel, hogy az öntözésvezérlő állapotgépében hibát találunk: fagyveszély esetén is nyitva hagyja a szelepet, ami a növények károsodásához vezethet. A javítás sürgős, de a 200 darab telepített csomópontot fizikailag felkeresni nem lehetséges. Az OTA-frissítés (Over-The-Air, azaz hálózaton keresztüli frissítés) az egyetlen járható út. A hibás frissítés azonban még rosszabb: ha az új kép nem indul, az eszköz „téglává” (brick) válik, és a teljes állomány elveszítheti a felügyeletet. Ezért olyan mechanizmus kell, amely áramszünet, félbeszakadt letöltés vagy rossz kép esetén is helyreállítja a rendszert – de ez csak megfelelően konfigurált bootloaderrel, watchdoggal és megerősítési logikával garantálható, nem pusztán attól, hogy két flash-slot létezik.

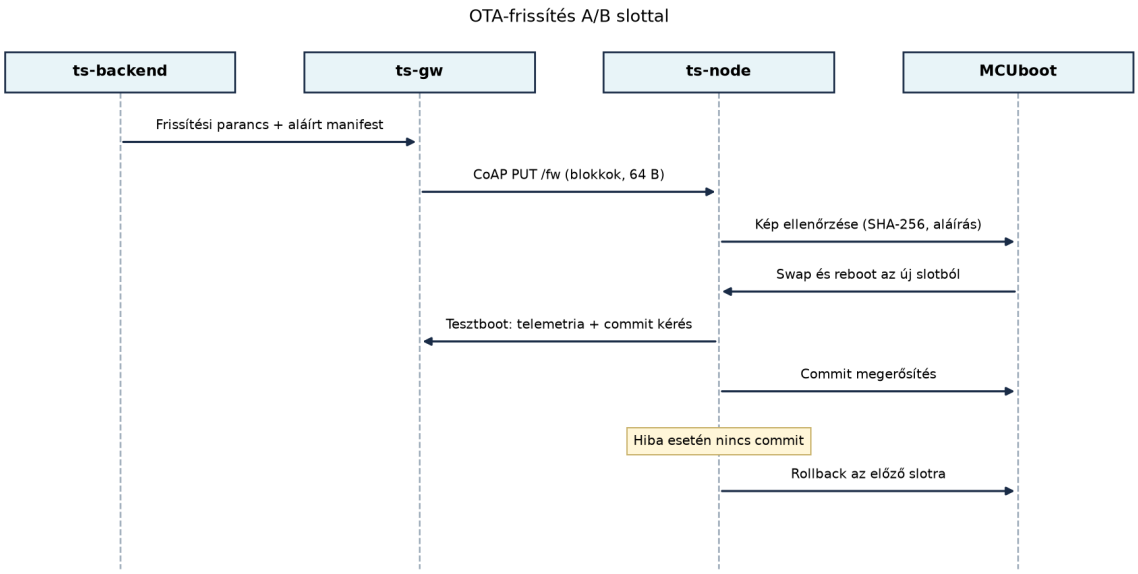
20.3.2 Fogalmak és működés

Definíció. - **OTA-frissítés (Over-The-Air update):** firmware- vagy alkalmazáskép cseréje hálózaton keresztül, fizikai hozzáférés nélkül. - **A/B slot:** két, logikailag elkülönített flash-terület. Az egyik az aktív (bootoló) kép, a másik fogadja az új verziót. A csere után a bootloader dönti el, melyikből indítson. Az MCUboot a swap állapotát flash-metaadatokban tartja, ezért egy swap közbeni újraindulás folytatható; ez azonban a swap módjától és a board flash-elrendezésétől függ. - **Tesztboot (test boot):** az új kép első, felügyelt indítása. Az alkalmazásnak a sikeres futásidejű ellenőrzés után meg kell erősítenie a képet (commit); ha ez elmarad, a bootloader a következő újrainduláskor visszaáll a korábbi slotra. - **Rollback:** visszatérés az előző, ismert jó slotra, ha az új kép nem indul, nem jut el a commitig, vagy a watchdog újraindítja. A visszaállás csak akkor automatikus, ha a bootloader tesztboot-módja, a watchdog és az alkalmazás commit-logikája együtt konfigurálva van. - **Delta-frissítés:** csak a két firmware-verzió közötti bináris különbség (delta) letöltése, nem a teljes kép. - **Canary kiadás:** a frissítés először egy kis, reprezentatív eszközcsoporton történik; a telemetriai adatok alapján döntenek a folytatásról.

A TerraSense eszközoldali frissítési folyamatának **tervezett szerződése** a CoAP: REST UDP fölött, Observe és blokkátvitel alfejezetben rögzített konvencióra épül: a frissítő kliens a `coap://ts-node/fw` erőforrásra PUT metódussal, 64 bájtos blokkokban küldi a firmware-t; az eszköz köztes válasza 2.31 Continue, az utolsó blokkra 2.04 Changed. A teljes kép megérkezése után az eszköz ellenőrzi az integritást (a Biztonságos boot és firmware-aláírás alfejezetben bemutatott SHA-256 és aláírás-ellenőrzés), majd a bootloader elvégzi a slotcserét és az új képből bootol.

> **A fejezet hatóköre.** Ez az alfejezet a frissítési folyamat szerződését, a manifest biztonsági követelményeit és az alkalmazásszintű állapotgépet definiálja. A `ts-node` oldali CoAP-vevő, a flash-író driver, az MCUboot-integráció és a gateway oldali orchestrátor implementációja nem része a laboroknak. A labor az állapotgépet modellezi hoston; a valódi, hálózaton keresztüli firmware-átvitel megvalósítása és céleszközön történő igazolása a záróprojekt feladata. Amíg ez el nem készül és nem bizonyított, a záróprojekt nem állíthatja, hogy működő OTA-frissítést tartalmaz.

Az alábbi szekvenciadiagram a normál sikeres frissítést és a rollback esetet vázolja:



20.3. ábra: OTA-frissítés A/B slottal - Tervezett folyamat; a rollback konfigurált tesztboot, watchdog és commit-logika mellett automatikus. A labor csak az állapotgépet modellezi.

A frissítési manifest (metaadat) a verzió, a kép mérete és SHA-256 lenyomata mellett tartalmazza a komponens- és hardverazonosítót, a delta-bázis verzióját, a minimális akkufeszültséget, az anti-rollback biztonsági számlálót és a manifest ECDSA-aláírását. A `ts-node` csak akkor kezdi el a letöltést, ha az akkufeszültség a küszöb felett van, a manifest aláírása érvényes, a `component` és a `hw_id` egyezik, és a `security_counter` nagyobb a készüléken tárolt értéknél. A gateway (`ts-gw`) szerepe a tervezett folyamatban a kép tárolása és a blokkátvitel felügyelete; ennek implementációja – a Gateway üzemeltetése alfejezet üzemeltetési mintái mellett – szintén a záróprojekt feladata.

20.3.3 Döntési szempontok

Az OTA-tervezésnél több kompromisszumot kell mérlegelni, ezeket az alábbi táblázat foglalja össze.

Döntés	Változatok	TerraSense választás	Indoklás
Letöltési protokoll	MQTT, HTTP, CoAP	CoAP blokkátvitel (CoAP: REST UDP fölött, Observe és blokkátvitel)	Kis memóriaigényű, UDP-n is működik, a blokkátvitel félbeszakadás-tűrő

Döntés	Változatok	TerraSense választás	Indoklás
Slotok száma	Egy slot, A/B	A/B (MCUboot)	A swap metaadatai alapján a megszakadt swap folytatható; a rollbackhez tesztboot és watchdog is kell
Képtípus	Teljes, delta	Teljes alapértelmezés, delta opcionális nagy flottánál	A delta csökkenti az átvitelt, de bonyolítja a verziókezelést
Megerősítés	Nincs, boot után azonnali, késleltetett commit	Tesztboot + commit alkalmazás által választott időkorláttal (a laborban 2 perc)	A hibás kép még elkapható a megerősítés előtt; a 2 perc nem bootloader-alapértelmezés
Kiadási stratégia	Egyszerre, canary, hullámok	Canary 5%, majd 20%, 50%, 100%	A hibák korai észlelése, a teljes állomány védelme

A frissítési folyamat állapotai a `ts-node` oldalán:

Állapot	Esemény	Következő állapot
IDLE	Manifest fogadása	DOWNLOADING
DOWNLOADING	Minden blokk megérkezett	VERIFYING
DOWNLOADING	Hálózati hiba, 3 újrapróbálkozás után	IDLE (hibajelentés)
VERIFYING	SHA-256 / aláírás OK	TEST_BOOT
VERIFYING	Ellenőrzés sikertelen	IDLE (hibajelentés)
TEST_BOOT	Commit parancs sikeres	COMMITTED
TEST_BOOT	Időtúllépés (pl. 2 perc)	ROLLBACK
COMMITTED	–	IDLE (új verzió aktív)
ROLLBACK	Bootloader visszacseréli a slotot	IDLE (régi verzió aktív)

A táblázat **alkalmazásszintű** modell. A gyakorlatban az MCUboot a swap állapotát a flash metaadataiban tartja, ezért a swap közbeni áramszünet után a bootloader a következő indításkor folytathatja vagy visszaállhat; ez azonban a swap módjától (scratch vagy move), a board flash-elrendezésétől és a bootloader verziójától függ. A „2 perces” visszaállás nem az MCUboot beépített tulajdonsága, hanem az alkalmazás által választott időkorlát: az alkalmazásnak a sikeres önteszt után explicit módon meg kell erősítenie a képet, a bootloader pedig a megerősítés

elmaradása esetén a következő resetkor visszaáll. Ha az új kép már a commit-kód elérése előtt beragad, csak egy engedélyezett hardver- vagy szoftver-watchdog tudja kikényszeríteni az újraindítást és ezzel a rollbacket. A watchdogot, a commit-időkorlátot és a swap módot ezért együtt kell beállítani és céleszközön tesztelni.

A `ts-gw` gateway frissítése eltérő: a Linux-alapú rendszeren konténer-image-et cserélünk. Az üzemeltetés alfejezetben leírt módon a Compose-ban a szolgáltatás image tagjét új verzióra állítjuk, a `docker compose pull && up -d` parancs futtatásával. Visszaállítás esetén a korábbi taget állítjuk vissza. Natív rendszeren szimbolikus linkkel mutathatunk az aktuális rootfs-re, A/B rootfs esetén a bootloader két partícióból választ. Az IETF SUIT manifest egy szabványos leíró formátum a frissítési metaadatok és függőségek leírására; a TerraSense-ben jelenleg saját JSON manifestet használunk, de a SUIT-ra való áttérés hosszú távú lehetőség.

20.3.4 Kidolgozott példa

Példa: a `gh1/n001` eszköz frissítése 1.6.2-ről 1.7.0-ra

Az új firmware mérete 96 KiB (98 304 B). A CoAP blokkméret 64 B, az átviteli hatékonyság UDP/6LoWPAN felett átlagosan 80% (illusztratív, nem mért érték). A szükséges blokkok száma:

$$\text{blokkok} = \left\lceil \frac{98\,304}{64} \right\rceil = 1536$$

A hatékony átviteli sebesség a szimulált UDP-kapcsolaton 1,2 kB/s (illusztratív). A letöltési idő:

$$t_{\text{letöltés}} = \frac{98\,304}{1,2 \cdot 1000} \approx 82 \text{ s}$$

Ha a delta-frissítés a két verzió között csak 18 KiB, akkor a letöltési idő körülbelül 15 s-ra csökken. Ez a különbség nagy flottánál jelentős: 200 eszköz esetén a teljes átvitel 16,4 MB helyett 3,0 MB, ami a korlátos LPWAN-kapacitást is kíméli.

A frissítés lépései:

1. A `ts-backend` a `terrasense/ota/gh1/n001/available` topicra közzéteszi az aláírt manifestet:

```
{
  "format": 1,
  "component": "ts-node",
  "hw_id": "terrasense-node-r2",
  "version": "1.7.0",
  "size": 98304,
  "sha256": "a1b2c3...",
  "delta_base": "1.6.2",
  "min_battery_mv": 3400,
  "security_counter": 42,
  "signature": {
    "alg": "ES256",
    "key_id": "ts-ota-2026",
    "value": "base64..."
  }
}
```

Az aláírás a manifest kanonikus JSON-jára (a `signature` mező nélkül) készül ECDSA P-256 kulccsal. A nyilvános kulcsot a firmware tartalmazza vagy a provisioning során kapja; a `key_id` a kulcsrotációt teszi lehetővé.

2. A `ts-gw` a manifestet fogadja, ellenőrzi az aláírását, majd a CoAP blokkátvitelt a `ts-node` felé elindítja. A `ts-node` is elvégzi ugyanezeket az ellenőrzéseket; a gateway ellenőrzése nem helyettesíti az eszközét.

3. A `ts-node` a blokkokat a flash másodlagos slotjába írja. A folyamat megszakadása esetén a már letöltött blokkok érvénytelenek, és a folytatás a manifest újbóli lekérésével indul.

4. Az utolsó blokk után a `ts-node` ellenőrzi a méretet, a SHA-256 lenyomatot, a kép aláírását, valamint azt, hogy a kép fejlécében lévő `security_counter` megegyezik a manifestben megadottal és nagyobb a készüléken tárolt értéknél. Ha bármelyik ellenőrzés sikertelen, a képet elutasítja, és a régi slotból indul. Ha minden érvényes, a bootloader (MCUboot) elvégzi a swapet, és az eszköz az új képből bootol.

5. A tesztboot során az alkalmazás a szokásos telemetriát küldi, elvégzi az öntesztet, és a beállított időkorláton (a laborban 2 percen) belül commitálja a képet a bootloader felé. Ha az alkalmazás nem jut el a commitig, vagy a watchdog az időkorlát előtt újraindítja, a bootloader a következő indításkor visszacseréli a slotokat, és a régi kép indul.

Fokozatos kiadás a TerraSense flottában.

A frissítést nem egyszerre kapja mind a 200 eszköz. A `ts-backend` a következő hullámokat hajtja végre:

1. **Canary (5% = 10 eszköz):** kiválasztás véletlenszerűen, de reprezentatív (különböző üvegházakból). A frissítés után 24 órán át figyeli a telemetriát: a rollback arány, a sikeres MQTT-üzenetek aránya és az öntözésvezérlő hibái.
2. **Első hullám (20% = 40 eszköz):** ha a canary csoportban a rollback arány $< 2\%$ és a telemetriai hibák nem nőttek, tovább lép.
3. **Második hullám (50% = 100 eszköz):** ugyanazok a feltételek, szélesebb körben.
4. **Teljes kiadás (100%):** ha az előző hullám is sikeres.

A leállítási feltételek a `ts-backend` riasztási motorjába vannak bekötve: ha a rollback arány eléri a 2%-ot, vagy az öntözésvezérlő hibás szelepnyitásainak száma meghaladja a napi 1-et, a kiadás leáll, és a flottakezelő (Flottakezelés) beavatkozhat.

Labor: OTA-állapotgép szimuláció C++-ban

A laborban nem valós hardvert, hanem egy C++ programot futtatunk, amely az A/B slotok és a tesztboot folyamatát modellezi. Ez segít megérteni az állapotátmeneteket és a rollback logikát anélkül, hogy valós flash-műveleteket kellene végezni.

Munkakönyvtár:

```
mkdir -p ~/iot-labor/i060 && cd ~/iot-labor/i060
```

Fájl: `ota_sim.cpp`

```
#include <iostream>
#include <string>
#include <utility>

enum class State { IDLE, DOWNLOADING, VERIFYING, TEST_BOOT, COMMITTED, ROLLBACK };

const char* to_string(State s) {
    switch (s) {
```

```
        case State::IDLE: return "IDLE";
        case State::DOWNLOADING: return "DOWNLOADING";
        case State::VERIFYING: return "VERIFYING";
        case State::TEST_BOOT: return "TEST_BOOT";
        case State::COMMITTED: return "COMMITTED";
        case State::ROLLBACK: return "ROLLBACK";
    }
    return "UNKNOWN";
}

class OtaDevice {
public:
    OtaDevice(std::string previous_version, std::string candidate_version)
        : previous_version_(std::move(previous_version)),
          running_version_(previous_version_),
          candidate_version_(std::move(candidate_version)) {}

    bool start_download() {
        if (state_ != State::IDLE) return reject("start_download");
        state_ = State::DOWNLOADING;
        return true;
    }

    bool download_complete() {
        if (state_ != State::DOWNLOADING) return reject("download_complete");
        state_ = State::VERIFYING;
        return true;
    }

    bool verify_complete(bool ok) {
        if (state_ != State::VERIFYING) return reject("verify_complete");
        if (!ok) {
            state_ = State::IDLE;   // a régi kép marad az aktív slotban
            return true;
        }
        // Az MCUboot swap után a jelölt kép fut, de még nem megerősített.
        running_version_ = candidate_version_;
        confirmed_ = false;
        state_ = State::TEST_BOOT;
        return true;
    }

    bool send_commit() {
        if (state_ != State::TEST_BOOT) return reject("send_commit");
        confirmed_ = true;
        state_ = State::COMMITTED;
        return true;
    }

    bool test_timeout() {
        if (state_ != State::TEST_BOOT) return reject("test_timeout");
        running_version_ = previous_version_;  // a bootloader visszacserél
        confirmed_ = true;
        state_ = State::ROLLBACK;
        return true;
    }

    bool rollback_done() {
        if (state_ != State::ROLLBACK) return reject("rollback_done");
        state_ = State::IDLE;
        return true;
    }
```

```
    State state() const { return state_; }
    const std::string& running_version() const { return running_version_; }
    bool confirmed() const { return confirmed_; }

    void report() const {
        std::cout << "state=" << to_string(state_)
                  << ", running=" << running_version_
                  << ", confirmed=" << (confirmed_ ? "yes" : "no") << "\n";
    }

private:
    bool reject(const char* event) {
        std::cout << "elutasitva: " << event << " (state="
                  << to_string(state_) << ")\n";
        return false;
    }

    std::string previous_version_;
    std::string running_version_;
    std::string candidate_version_;
    State state_{State::IDLE};
    bool confirmed_{true};
};

int main() {
    std::cout << "--- sikeres frissites ---\n";
    OtaDevice dev1("1.6.2", "1.7.0");
    dev1.start_download();
    dev1.download_complete();
    dev1.verify_complete(true);
    dev1.report();                  // TEST_BOOT, running=1.7.0, confirmed=no
    dev1.send_commit();
    dev1.report();                  // COMMITTED, running=1.7.0, confirmed=yes
    dev1.send_commit();             // ismetelt commit: elutasitva

    std::cout << "--- rollback ---\n";
    OtaDevice dev2("1.6.2", "1.7.0");
    dev2.start_download();
    dev2.download_complete();
    dev2.verify_complete(true);
    dev2.report();                  // TEST_BOOT, running=1.7.0, confirmed=no
    dev2.test_timeout();
    dev2.report();                  // ROLLBACK, running=1.6.2, confirmed=yes
    dev2.rollback_done();
    dev2.report();                  // IDLE, running=1.6.2, confirmed=yes

    std::cout << "--- ervenytelen atmenet ---\n";
    dev2.start_download();          // OK
    dev2.send_commit();             // elutasitva: nem TEST_BOOT

    return 0;
}
```

Fordítás és futtatás:

```
g++ -std=c++17 -Wall -o ota_sim ota_sim.cpp
./ota_sim
```

Várt kimenet (a futtatás eredménye ebben a laborban reprodukálható):

```
--- sikeres frissites ---
state=TEST_BOOT, running=1.7.0, confirmed=no
state=COMMITTED, running=1.7.0, confirmed=yes
```

```
elutasitva: send_commit (state=COMMITTED)
--- rollback ---
state=TEST_BOOT, running=1.7.0, confirmed=no
state=ROLLBACK, running=1.6.2, confirmed=yes
state=IDLE, running=1.6.2, confirmed=yes
--- ervenytelen atmenet ---
elutasitva: send_commit (state=DOWNLOADING)
```

A szimulátor megkülönbözteti a futó és a megerősített verziót: a swap után a `running` már az új verzió, de `confirmed=no`, amíg az alkalmazás nem commitál. Commit nélküli időtúllépésnél a bootloader visszacserél, és a `running` ismét a régi verzió. Az eseménymetódusok minden más állapotból `false` visszatérési értékkel utasítják el a hívást, így a valós vezérlőben nem lehet véletlenül commitálni vagy rollbackelni. A labor továbbra is csak az állapotgépet modellezi; a flash-írás és a valódi MCUboot swap nem része.

20.3.5 Ellenőrzés és hibakeresés

Az OTA-folyamat helyességét többszintű teszttel ellenőrizzük:

1. **Unit tesztek a manifest-ellenőrzésre:** a SHA-256 lenyomat és az aláírás érvénytelen értéke esetén a rendszernek hibát kell jelentenie, és nem szabad a TEST_BOOT állapotba lépnie.
2. **Áramszünet-szimuláció:** a CoAP-blokkátvitel közbeni megszakítást és újraindítást szimulálva az eszköznek a régi slotból kell tovább működnie, és a félbehagyott letöltést újra kell kezdenie.
3. **Rollback teszt:** a tesztboot időtúllépését mesterségesen előidézve (például a commit kiküldésének késleltetésével) a bootloadernek vissza kell térnie az előző verzióhoz. A tesztnek ki kell terjednie arra az esetre is, amikor az új kép már a commit-kód elérése előtt beragad: ilyenkor csak a watchdog által kiváltott reset után következik be a rollback, ezért a watchdogot külön is tesztelni kell.

Tipikus hibatünetek, okok és javítások:

Hibatünet	Ok	Javítás
A letöltés félbeszakad, és az eszköz nem indul	Nincs A/B slot, a félbeírt kép felülírta az aktív képet	A/B slot bevezetése MCUboot-tal
A rollback nem történik meg	A watchdog nincs engedélyezve, a kép a commit előtt beragad, vagy a tesztboot commit-időkorlátja túl hosszú	Watchdog engedélyezése és tesztelése; a commit-időkorlát és a swap mód együttes beállítása a céleszközön
Az eszköz más hardverre szánt képet fogad el	A manifestből hiányzik a `component`/`hw_id` ellenőrzése	A manifest aláírt `component` és `hw_id` mezőjének kötelező ellenőrzése a letöltés előtt

Hibatünet	Ok	Javítás
A készülék visszaáll egy sebezhető verzióra	Nincs anti-rollback számláló, vagy a manifest `security_counter` mezőjét nem ellenőrzik	A manifest és a képfejléc `security_counter` értékének összevetése a tárolt értékkel; a bootloader anti-rollback beállítása
A delta-frissítés hibás	A delta-bázis verziója nem egyezik a ténylegesen aktív verzióval	A manifestben kötelezően szerepeljen a `delta_base`, és a letöltés előtt ellenőrizze az eszköz
A canary kiadás nem reprezentatív	A kiválasztott eszközök mind egy üvegházból származnak	Rétegzett mintavétel: minden site-ból arányosan

20.3.6 Tipikus hibák és félreértések

1. **A commit elhagyása.** Sokan azt hiszik, hogy ha a boot sikeres, a frissítés kész. Azonban a boot sikeressége nem jelenti azt, hogy az alkalmazás valóban működik (pl. a hálózati stack nem csatlakozik). A commit parancs a futásidejű ellenőrzés után erősíti meg a frissítést.
2. **A letöltési folytatás hiánya.** Ha a CoAP blokkátvitel közepén megszakad a kapcsolat, a már letöltött blokkokat nem szabad automatikusan érvényesnek tekinteni. A folytatáshoz a manifest újbóli letöltése és a blokkok sorszám szerinti újrakérése szükséges, különben a kép hibás lehet.
3. **A delta-frissítés alkalmazása rossz bázisverzióra.** A delta csak akkor alkalmazható, ha a `delta_base` pontosan megegyezik az eszközön futó verzióval. Ha nem, az eszköznek teljes képet kell kérnie.

20.3.7 Feladatok

1. **Fogalmi/döntési kérdés.** Magyarázza el, miért szükséges a tesztboot commit, és mi történik, ha egy hibás kép mégis elindul.
2. **Számolási feladat.** Egy 128 KiB-os firmware-t CoAP blokkátvitellel töltünk le 64 B-os blokkokban. A hatékony átviteli sebesség 2,4 kB/s (illusztratív). Mennyi idő a letöltés? Hány blokk szükséges?
3. **Implementációs feladat.** Bővítse a labor `ota_sim.cpp` programját a fokozatos kiadás modellezésével: szimuláljon 100 eszközt, mindegyikhez rendeljen véletlenszerű rollback-valószínűséget, és írja ki, hány eszköz lép át a canary, az első és a második hullámon.
4. **Hibakeresési/kiterjesztési feladat.** Tegyük fel, hogy a delta-letöltés megszakad, és a `delta_base` verzió az eszközön már nem az 1.6.2. Hogyan kezelje ezt a rendszer? Vázolja a módosított állapotgépet.

20.3.8 Megoldások

1. A tesztboot commit azt igazolja, hogy az új firmware nemcsak elindul, hanem a kritikus funkciókat is ellátja (pl. csatlakozik a hálózathoz, telemetriát küld, parancsokat fogad). Ha a commit elmarad, a bootloader visszaállítja a régi képet. Ha a hibás kép mégis elindul, és a commit megtörténik, a hiba csak a következő frissítésnél javítható.
2. A blokkok száma:
$$\text{blokkok} = \left\lceil \frac{128 \cdot 1024}{64} \right\rceil = \lceil 2048 \rceil = 2048$$
A letöltési idő:
$$t = \frac{128 \cdot 1024}{2,4 \cdot 1000} \approx 54,6 \text{ s}$$
3. A kiterjesztett program váza: minden eszköz egy `OtaDevice` objektum, a `commit` sikeressége véletlen. A canary csoportba az első 5 eszköz kerül, ha ott a rollback arány $< 2\%$, indul a 20 eszközből álló első hullám, és így tovább. A logika megegyezik a szekvenciális kiadással, a véletlen csak a rollback bekövetkezésében játszik szerepet.
4. A rendszernek a manifestben megadott `delta_base` értéket minden esetben össze kell hasonlítania az eszköz aktuális verziójával. Ha az eltér, a `ts-node` teljes képet kér, és a delta-letöltést elutasítja. Az állapotgépben ez egy új átmenet: `VERIFYING` állapotba lépés előtt a `delta_base` ellenőrzése; ha nem egyezik, a `ts-node` a `terrasense/ota/gh1/n001/request_full` topicra üzenetet küld, és a backend elküldi a teljes képet.

20.3.9 Rövid összefoglaló

Az OTA-frissítés tervezett folyamata a TerraSense eszközökön A/B slottal, MCUboot swap mechanizmussal és CoAP blokkátvitellel történik. A letöltés a `coap://ts-node/fw` erőforrásra PUT 64 B-os blokkokban, 2.31 Continue / 2.04 Changed válaszokkal. A manifest aláírt, és tartalmazza a komponens- és hardverazonosítót, a méretet, a SHA-256 lenyomatot, a `security_counter` értéket és a delta bázisát. A kép ellenőrzése SHA-256 és aláírás-ellenőrzés, majd tesztboot és alkalmazás által vezérelt commit (a laborban 2 perces időkorláttal). A rollback csak engedélyezett watchdog és megfelelő bootloader-konfiguráció mellett automatikus. A delta-frissítés a `delta_base` verzió pontos egyezését követeli meg. A fokozatos kiadás canary 5%, majd 20%, 50%, 100% hullámok, leállási feltétel: rollback arány $> 2\%$ vagy öntözési hiba. **Továbbvitt eredmény:** az OTA-állapotgép (`ota_sim.cpp`), a manifest séma, a `terrasense/ota/...` topicok és a canary-hullámok a Flottakezelés, a CI/CD és a Záróprojekt alfejezetekben használandók. A CoAP-vevő, a flash-író és a gateway-orchestrátor implementációja a záróprojekt feladata; e fejezet nem állítja, hogy ezek elkészültek.

20.3.10 Források és ellenőrzési hiányok

- MCUboot dokumentáció: https://docs.mcuboot.com/ (a bootloader swap típusai és a `BOOT_UPGRADE_ONLY` opció pontos neve ellenőrizendő).
- IETF SUIT manifest: https://datatracker.ietf.org/wg/suit/about/ (a manifest formátum részletei ellenőrizendők).
- Zephyr OTA és MCUboot integráció: https://docs.zephyrproject.org/latest/ (az aktuális LTS verzióban a `CONFIG_BOOTLOADER_MCUBOOT` konfiguráció).
- A CoAP blokkátvitel szabványa: RFC 7959, https://www.rfc-editor.org/rfc/rfc7959 (stabil).
- A példában szereplő átviteli sebességek és letöltési idők illusztratívak, nem mért értékek.

20.4 Flottakezelés: nyilvántartás, konfiguráció, állapot

20.4.1 Motiváció

A TerraSense első pilotja három `gh1` üvegházban tíz `n001`–`n010` csomóponttal indult. A sikeres teszt után a rendszert száz üvegházra bővítik, és ezer eszköz küldi a telemetriát. Kézzel már nem lehet követni, melyik csomópont milyen firmware-t futtat, mikor jelentkezett utoljára, miért indult újra, és kapott-e már új mintavételezési konfigurációt. A flotta méretének növekedésével a működtetés minősége nem a rádióprotokollon vagy a brokeren múlik, hanem azon, hogy a backend rendelkezik-e **eszköznyilvántartással**, és abból ki tudja-e osztani a konfigurációt, figyelni tudja-e az állapotot, és le tudja-e vezényelni az eszköz életciklusának végét.

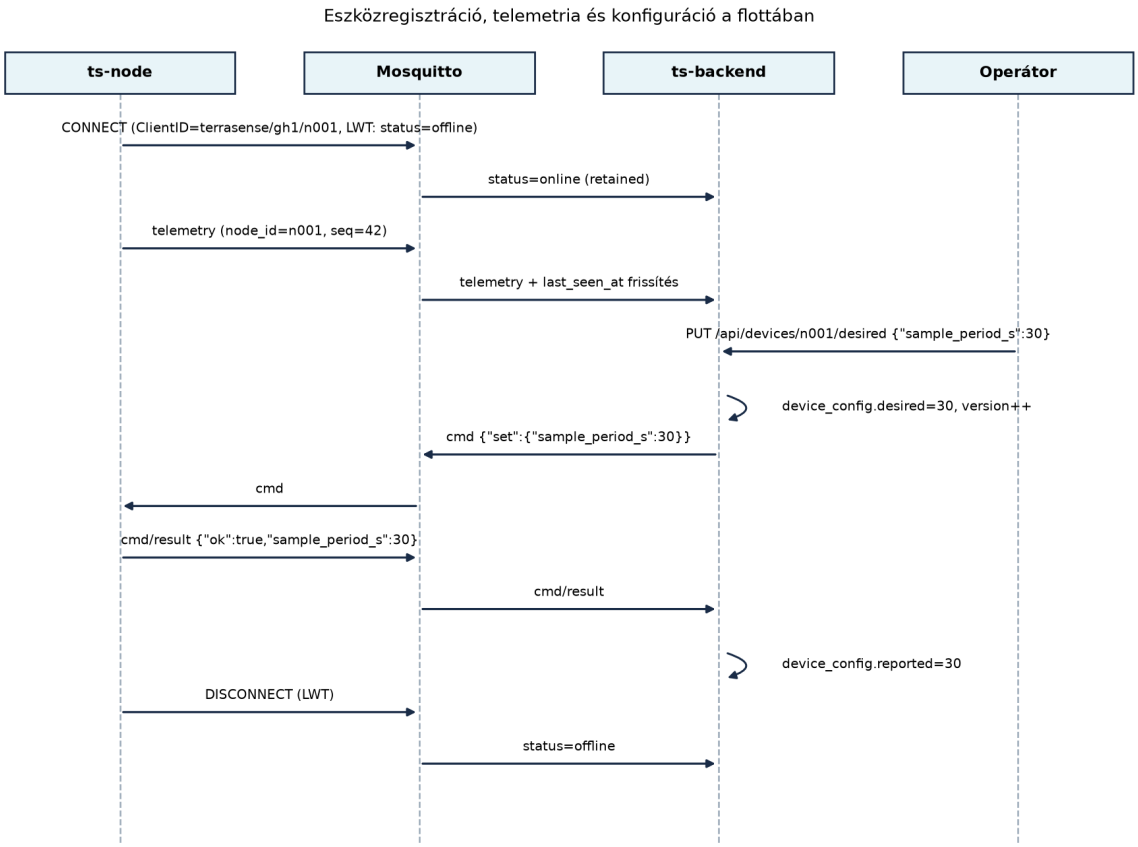
20.4.2 Fogalmak és működés

Definíció. Az **eszköznyilvántartás (device registry)** a flotta összes eszközének közös, lekérdezhető nyilvántartása. Minden eszközhöz egyértelmű azonosítót, hitelesítési metaadatokat, hardver- és szoftververziót, csoporttagságot, telepítési helyet, valamint az aktuális állapotra és a kiosztott konfigurációra vonatkozó hivatkozásokat rendel.

Definíció. A **flotta (fleet)** az egy rendszerhez tartozó, közösen menedzselt eszközök összessége. A flottakezelés magában foglalja az eszközök regisztrációját, csoportosítását, konfigurációját, állapotfigyelését, frissítését és kivezetését.

Definíció. Az **eszközárnyék (device shadow)** itt az Eszközárnyék és digitális iker: kívánt és jelentett állapot alfejezet digitális iker fogalmának flotta-szintű alkalmazása: minden eszközhöz egy **kívánt (desired)** és egy **jelentett (reported)** állapotot tárolunk. A kívánt állapotot az üzemeltető vagy a backend logika írja, a jelentett állapotot az eszköz ténylegesen visszajelzi. A kettő eltérése konfiguráció-kézbesítési problémát jelez.

A nyilvántartás és az árnyék kapcsolatát a következő szekvencia mutatja be:



20.4. ábra: Eszközregisztráció, telemetria és konfiguráció a flottában - A ts-node a LWT-vel jelzi a jelenlétét, a backend a desired állapotból parancsot generál, és a cmd/result visszaigazolásából frissíti a reported mezőt.

A diagram sorszáma: 1. ábra. A szöveg nélküli olvasó számára: az eszköz a `terrasense/gh1/n001/status` retained üzenettel jelzi az online állapotot, a broker a Last Will üzenetből automatikusan offline-ra vált. Az operátor egy REST hívással módosítja a kívánt beállítást, a backend parancsot küld az eszköznek a `terrasense/gh1/n001/cmd` topicra, majd az eszköz a `cmd/result` topicra válaszol, és a backend frissíti a jelentett állapotot.

A flotta-adatmodell a TerraSense-ben a `ts-backend` PostgreSQL adatbázisában a következő táblákkal valósul meg:

- `device_groups`: csoportok, például `gh1-öntöző`, `gh1-talaj`.
- `devices`: egyedi eszköz, a `node_id` azonosítóval (site+node összevontan `n001` a `gh1` alatt, hogy a topicból közvetlenül kereshető legyen).
- `device_config`: eszközönként egy sor, a `desired` és `reported` JSONB mezőkkel, valamint `version` egész számmal.
- `device_state`: az utolsó ismert futásállapot, beleértve a `reset_cause` értékét és az utolsó `seq` sorszámot.
- A `devices.last_seen_at` és a `device_state.status` mező pedig a jelenlétet tárolja.

20.4.3 Döntési szempontok

A nyilvántartás kialakításakor több tervezési döntést kell meghozni; a főbb kompromisszumokat a következő táblázat foglalja össze.

Kérdés	Lehetséges megoldás	Előny	Hátrány	TerraSense-választás
Hol tároljuk az eszközöket?	Külön flottaszolgáltatás (LwM2M)	szabványos, kész funkciók	plusz rendszer, zártabb adatmodell	saját táblák a meglévő backendben, mert a TerraSense-t magunk építjük
Csoportosítás módja	statikus csoportok	egyszerű, kiszámítható	nem követi a valós változásokat	címkék + csoportok vegyesen
Konfiguráció kiosztása	közvetlen parancs küldés	azonnali	nem tolerálja az eszköz offline állapotát	árnyék: a desired-et tároljuk, a kézbesítés aszinkron és újrapróbálható
Jelenlét követése	pollozás (REST)	megbízható	skálázhatatlan, késleltetett	MQTT LWT + retained status
Állapot perzisztencia	csak memória	gyors	újraindításkor elveszik	PostgreSQL + TimescaleDB, az állapotok historizálhatók

A **csoportok és címkék** szerepe az, hogy a műveleteket ne egyesével kelljen elvégezni. A TerraSense-ben minden eszközhöz legalább egy csoportot rendelünk, például `gh1-öntöző`, `gh1-talaj`. A csoporthoz tartozik egy **konfigurációs sablon**, amely az új eszközök alap desired állapotát adja. A címkék (például `kritikus`, `preview`) szabadon szűrhetővé teszik a flottát, anélkül hogy az adatmodellt mereven át kellene alakítani.

A **konfiguráció-kiosztás árnyékkal** elve: a backend soha nem közvetlenül írja az eszköz memóriáját, hanem a `device_config.desired` mezőben rögzíti a kívánt állapotot. Egy háttérfolyamat figyeli a desired és reported közti különbséget, és minden változásnál parancsot küld a `terrasense/<site>/<node>/cmd` topicra. Az eszköz a `cmd/result` topicra küldi a nyugtát, amelyben szerepel a ténylegesen alkalmazott érték. A backend ekkor frissíti a `reported` mezőt. Ha az eszköz offline, a parancs a brokeren megőrzött üzenetként vagy a backend újrapróbáló logikájával később is kézbesíthető; a desired mindvégig megmarad.

A **jelenlétfigyelés** az MQTT Last Will és Testament (LWT) mechanizmusra épül. A ts-node a `terrasense/<site>/<node>/status` retained topicra `online` üzenetet küld, és a CONNECT során LWT-t állít be `offline` tartalommal. Ha a kapcsolat törpe módban megszakad, a broker `offline` retained üzenetet publikál, amit a backend feliratkozással figyel, és frissíti a `device_state.status` mezőt. A telemetria érkezésekor a `devices.last_seen_at` is frissül, így a csendes, de élő kapcsolat is nyomon követhető.

20.4.4 Kidolgozott példa: flotta-adatmodell a gyakorlatban

Az alábbi példa a TerraSense flotta nyilvántartásának konkrét megvalósítását mutatja. Az adatmodell a `~/iot-labor/i061/schema.sql` fájlban kerül rögzítésre.

Munkakönyvtár:

```
mkdir -p ~/iot-labor/i061 && cd ~/iot-labor/i061
```

A PostgreSQL/TimescaleDB táblákat a következő SQL hozza létre:

```
-- schema.sql: TerraSense flotta-nyilvántartás
CREATE TABLE IF NOT EXISTS device_groups (
    group_id      TEXT PRIMARY KEY,
    name          TEXT NOT NULL,
    config_template JSONB NOT NULL DEFAULT '{}'::jsonb
);

CREATE TABLE IF NOT EXISTS devices (
    node_id       TEXT PRIMARY KEY,          -- pl. 'gh1/n001'
    site          TEXT NOT NULL,             -- 'gh1'
    node          TEXT NOT NULL,             -- 'n001'
    group_id      TEXT REFERENCES device_groups(group_id),
    fw_version    TEXT,
    cert_serial   TEXT,                      -- a provisionált X.509 tanúsítvány sorszámja
    cert_path     TEXT,                      -- a CA által kiadott tanúsítvány fájlútvonala
    cert_fingerprint_sha256 TEXT,            -- a tanúsítvány SHA-256 lenyomata
    active        BOOLEAN NOT NULL DEFAULT true,
    provisioned_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    last_seen_at  TIMESTAMPTZ,
    decommissioned_at TIMESTAMPTZ,
    decommission_reason TEXT
);

CREATE TABLE IF NOT EXISTS device_config (
    node_id       TEXT PRIMARY KEY REFERENCES devices(node_id),
    desired       JSONB NOT NULL DEFAULT '{}'::jsonb,
    reported      JSONB NOT NULL DEFAULT '{}'::jsonb,
    version       INT  NOT NULL DEFAULT 0,
    updated_at    TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE IF NOT EXISTS device_state (
    node_id       TEXT PRIMARY KEY REFERENCES devices(node_id),
    status        TEXT NOT NULL DEFAULT 'unknown',  -- online | offline | unknown
    reset_cause   TEXT,
    last_seq      BIGINT,
    last_uptime_s BIGINT
);

-- Meglévő adatbázis migrálása a kivezetéshez (új telepítésnél is lefut, idempotens):
ALTER TABLE devices ADD COLUMN IF NOT EXISTS cert_serial TEXT;
ALTER TABLE devices ADD COLUMN IF NOT EXISTS cert_path TEXT;
ALTER TABLE devices ADD COLUMN IF NOT EXISTS cert_fingerprint_sha256 TEXT;
ALTER TABLE devices ADD COLUMN IF NOT EXISTS active BOOLEAN NOT NULL DEFAULT true;
ALTER TABLE devices ADD COLUMN IF NOT EXISTS decommissioned_at TIMESTAMPTZ;
ALTER TABLE devices ADD COLUMN IF NOT EXISTS decommission_reason TEXT;

CREATE TABLE IF NOT EXISTS device_audit (
    audit_id      BIGSERIAL PRIMARY KEY,
    node_id       TEXT NOT NULL,
    action        TEXT NOT NULL,    -- decommission | revoke_certificate | purge_telemetry
    detail        JSONB NOT NULL DEFAULT '{}'::jsonb,
    performed_by  TEXT NOT NULL,
    performed_at  TIMESTAMPTZ NOT NULL DEFAULT now()
);
```

A `devices` tábla **node_id** értéke a topicból származó `<site>/<node>` kulcs, például `gh1/n001`. Ez garantálja, hogy a telemetria topicjából közvetlenül hivatkozhatunk a nyilvántartás rekordjára.

A `device_config.desired` mezőbe például a következő JSON kerül, amikor az operátor a mintavételezési időt 60-ról 30 másodpercre csökkenti:

```
{"sample_period_s": 30}
```

A backend ekkor a `device_config.version` értékét növeli, és egy MQTT parancsot küld a `terrasense/gh1/n001/cmd` topicra:

```
{"set":{"sample_period_s":30}}
```

Az eszköz válasza a `terrasense/gh1/n001/cmd/result` topicra:

```
{"ok":true,"sample_period_s":30}
```

A backend a válasz alapján frissíti a `device_config.reported` mezőt. A `reported` és `desired` közti eltérés akkor jelez hibát, ha a parancs nem kerül nyugtázásra.

20.4.5 Ellenőrzés és hibakeresés

A flotta-adatmodell teszteléséhez először töltsük be a sématáblákat a backend adatbázisába:

A közös stackben a PostgreSQL/TimescaleDB szolgáltatás neve `lab-timescaledb`, a felhasználó `terra`, az adatbázis `terrasense`; ezt a konvenciót az Idősoros adatbázisok és adatmodellezés laborban hoztuk létre. A séma betöltése:

```
docker compose -f ~/iot-labor/stack/compose.yaml exec -T lab-timescaledb \
    psql -U terra -d terrasense -f - < ~/iot-labor/i061/schema.sql
```

> **Megjegyzés:** a korábbi alfejezetekben előforduló `postgres/ts` és `ts-timescaledb` nevek a laborok közötti kísérletezés maradványai. Ettől kezdve a könyv a `lab-timescaledb` szolgáltatásnevet és a `terra` felhasználót használja; ha a helyi stack ettől eltér, a parancsot egyszer kell a tényleges névhez igazítani.

Regisztráljunk egy teszteszközt:

```
INSERT INTO device_groups (group_id, name, config_template)
VALUES ('gh1-öntöző', 'Üvegház 1 öntöző kör', '{"sample_period_s":60}');

INSERT INTO devices (node_id, site, node, group_id, fw_version)
VALUES ('gh1/n001', 'gh1', 'n001', 'gh1-öntöző', '1.2.0');

INSERT INTO device_config (node_id, desired, reported, version)
VALUES ('gh1/n001', '{"sample_period_s":60}', '{}', 0);

INSERT INTO device_state (node_id, status, reset_cause, last_seq, last_uptime_s)
VALUES ('gh1/n001', 'offline', 'power_on', NULL, NULL);
```

A lekérdezés az összes eszközre és az eltérésre:

```
SELECT d.node_id, d.group_id, d.fw_version, st.status,
       c.desired ->> 'sample_period_s' AS desired_period,
       c.reported ->> 'sample_period_s' AS reported_period, c.version
FROM devices d
JOIN device_state st USING (node_id)
JOIN device_config c USING (node_id);
```

Várt kimenet:

```
  node_id  |   group_id  | fw_version | status | desired_period | reported_period | version
-----------+-------------+------------+--------+----------------+-----------------+---------
 gh1/n001  | gh1-öntöző | 1.2.0      | offline| 60             | NULL            | 0
```

A leggyakoribb hibákat és javításukat az alábbi táblázat foglalja össze.

Tünet	Lehetséges ok	Javítás
A `last_seen_at` nem frissül	A telemetria ingest nem hívja a flotta szolgáltatást	Ellenőrizd az MQTT-feliratkozást a `terrasense/+/+/telemetry` topicra
Az eszköz forever offline marad	Az LWT retained üzenet elveszett, vagy a backend nem írja a `status` mezőt	Ellenőrizd a retained flaget a publishnál, és a status topic feliratkozást
A `reset_cause` NULL	Az eszköz nem küldi a boot üzenetben a reset okot	A Hibatűrés: watchdog, újraindulás, hibanaplózás labor boot üzenetét bővítsd ki a `reset_cause` mezővel, és a backend parserében kezeld
A parancs után a `reported` nem frissül	Az eszköz nem válaszol a `cmd/result` topicra	Ellenőrizd a parancs topicját és a válasz JSON sémáját; a backend timeout után hibát jelezzen

20.4.6 Tipikus hibák és félreértések

1. **A jelenlétet a telemetriából becsüljük.** Sokan azt gondolják, hogy ha nincs telemetria, az eszköz leállt. Pedig ha a mintavételezési idő 10 perc, akkor 9 percig csend van. A megbízható jelenlét az MQTT LWT + retained status, nem az utolsó üzenet ideje. A `last_seen_at` csak kiegészítő diagnosztikai adat.
2. **A kivezetés nem azonos az adatbázis-sor törlésével.** Ha egy eszközt leszerelünk, a hitelesítő kulcsát is vissza kell vonni, különben a megtalált eszközből a támadó a hálózathoz férhet. A kivezetés továbbá törölheti az adatokat, de a nyilvántartási rekordot megtarthatjuk audit célra (soft delete), ahelyett hogy fizikailag kitöröljük.
3. **A desired állapot a `device_config`-ban, nem az eszközön.** Ha a kívánt értéket csak közvetlen MQTT paranccsal küldjük el, az eszköz offline idején elvész. Az árnyék minta szerint a desired tárolva van, és a kézbesítés aszinkron, újrapróbálással.

20.4.7 Feladatok

20.4.7.1 a) Fogalmi/döntési kérdés

Mikor érdemes külön flottakezelő szolgáltatást (például egy LwM2M szervert) használni a saját nyilvántartás helyett? Fogalmazd meg a TerraSense esetére vonatkozó döntést.

20.4.7.2 b) Tervezési feladat

Tervezz SQL lekérdezést, amely megadja a flotta verzióeloszlását: hány eszköz fut az egyes `fw_version` értékeken csoportonként? Az eredményt csoport és verzió szerint rendezd.

20.4.7.3 c) Implementációs feladat

A TerraSense backendben valósíts meg egy `GET /api/devices/{node_id}/state` végpontot, amely visszaadja a `devices`, `device_config` és `device_state` táblákból az adott eszköz teljes állapotát egyetlen JSON objektumban.

20.4.7.4 d) Hibakeresési vagy kiterjesztési feladat

Egészítsd ki a TerraSense backendet egy **kivezetési** végponttal (`DELETE /api/devices/{site}/{node}`), amely: - ellenőrzi, hogy az eszköz már legalább 30 napja offline, - visszavonja az eszköz tanúsítványát a labor-CA CRL-jében (OpenSSL 3.0.13), - lezárja a `devices` rekordot (`active=false`, `decommissioned_at`, ok), és auditbejegyzést ír, - a telemetriai adatokat **nem** törli automatikusan: a megőrzés külön politika, a tömeges törlés csak külön, naplózott művelettel történhet. Írd le a lépéseket, az azonosítók egyeztetését (a telemetriában `site` + `node_id`, nem `site/node`) és a szükséges tranzakciós határokat.

20.4.8 Megoldások

20.4.8.1 a) Fogalmi/döntési kérdés

Külön LwM2M szervert akkor érdemes választani, ha a flotta nagy, heterogén, több gyártó eszközeit is kezelni kell, és a szabványos eszközmenedzsment (például OTA, szenzorértékek expozíciója) előnyt jelent. A TerraSense esetében a flotta homogén (`ts-node`), a backendünk már rendelkezik MQTT-ingesttel és eszközárnyékkal, ezért a saját nyilvántartás egyszerűbb és jobban integrálható. Döntés: saját táblák a `ts-backend`-ben.

20.4.8.2 b) Tervezési feladat

```
SELECT d.group_id, d.fw_version, count(*) AS darab
FROM devices d
GROUP BY d.group_id, d.fw_version
ORDER BY d.group_id, d.fw_version;
```

Ez csoportonként megadja, hány eszköz futtatja az egyes verziókat. A `devices` táblában a `fw_version` az OTA-frissítés: A/B képek, rollback, delta, fokozatos kiadás OTA labor logikájával frissül.

20.4.8.3 c) Implementációs feladat

A Spring Boot controller `DeviceStateController.java`:

```
package com.terrasense.backend.device;

import com.fasterxml.jackson.databind.JsonNode;
import com.fasterxml.jackson.databind.ObjectMapper;
import org.springframework.http.ResponseEntity;
import org.springframework.jdbc.core.JdbcTemplate;
import org.springframework.web.bind.annotation.*;

import java.time.OffsetDateTime;
import java.util.LinkedHashMap;
import java.util.List;
```

```
import java.util.Map;

@RestController
@RequestMapping("/api/devices")
public class DeviceStateController {

    private final JdbcTemplate jdbc;
    private final ObjectMapper objectMapper;

    public DeviceStateController(JdbcTemplate jdbc, ObjectMapper objectMapper) {
        this.jdbc = jdbc;
        this.objectMapper = objectMapper;
    }

    @GetMapping("/{site}/{node}/state")
    public ResponseEntity<Map<String, Object>> getDeviceState(
            @PathVariable String site, @PathVariable String node) {
        String nodeId = site + "/" + node;
        String sql = """
            SELECT d.node_id, d.site, d.node, d.group_id, d.fw_version,
                   d.provisioned_at, d.last_seen_at, d.active,
                   c.desired::text AS desired_json,
                   c.reported::text AS reported_json,
                   c.version AS config_version,
                   s.status, s.reset_cause, s.last_seq, s.last_uptime_s
            FROM devices d
            JOIN device_config c USING (node_id)
            JOIN device_state s USING (node_id)
            WHERE d.node_id = ?
            """;
        List<Map<String, Object>> rows = jdbc.query(sql, (rs, rowNum) -> {
            Map<String, Object> row = new LinkedHashMap<>();
            row.put("node_id", rs.getString("node_id"));
            row.put("site", rs.getString("site"));
            row.put("node", rs.getString("node"));
            row.put("group_id", rs.getString("group_id"));
            row.put("fw_version", rs.getString("fw_version"));
            row.put("provisioned_at", rs.getObject("provisioned_at", OffsetDateTime.class));
            row.put("last_seen_at", rs.getObject("last_seen_at", OffsetDateTime.class));
            row.put("active", rs.getBoolean("active"));
            row.put("desired", readJson(rs.getString("desired_json")));
            row.put("reported", readJson(rs.getString("reported_json")));
            row.put("config_version", rs.getInt("config_version"));
            row.put("status", rs.getString("status"));
            row.put("reset_cause", rs.getString("reset_cause"));
            row.put("last_seq", rs.getObject("last_seq", Long.class));
            row.put("last_uptime_s", rs.getObject("last_uptime_s", Long.class));
            return row;
        }, nodeId);
        if (rows.isEmpty()) {
            return ResponseEntity.notFound().build();
        }
        return ResponseEntity.ok(rows.get(0));
    }

    private JsonNode readJson(String json) {
        try {
            return json == null ? null : objectMapper.readTree(json);
        } catch (Exception e) {
            throw new IllegalStateException("Érvénytelen JSONB érték", e);
        }
    }
}
```

A `com.terrasense.backend.device` csomag a `com.terrasense.backend` főcsomag alatt van, ezért a Spring Boot alapértelmezett komponenskeresése megtalálja. A `node_id` a `site + "/" + node` összefűzésből áll elő, így a `GET /api/devices/gh1/n001/state` kérés a `gh1/n001` rekordot adja vissza. A `queryForList` helyett szándékosan `jdbc.query` és `LinkedHashMap` szerepel: a `queryForMap` üres eredményre kivételt dob, ehelyett itt hiányzó eszközre 404 a válasz. A `JSONB` mezőket `::text` alakban olvassuk, majd Jacksonnal `JsonNode`-ná alakítjuk; így a válasz valódi JSON-objektum lesz, nem `PGobject`-szerű burkoló. A `TIMESTAMPTZ` oszlopokat `OffsetDateTime`-ként olvassuk, nem `Timestamp` + `LocalDateTime` párossal.

20.4.8.4 d) Hibakeresési vagy kiterjesztési feladat

A kivezetés három, egymástól elkülönített felelősséget kapcsol össze: a nyilvántartási rekord lezárását, a hozzáférés-visszavonást és az adatmegőrzést. A végpont csak az első kettőt végzi el; a telemetria törlése külön, naplózott művelet.

```
package com.terrasense.backend.device;

import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.jdbc.core.JdbcTemplate;
import org.springframework.transaction.support.TransactionTemplate;
import org.springframework.web.bind.annotation.*;

import java.security.Principal;
import java.time.OffsetDateTime;
import java.util.Map;

@RestController
@RequestMapping("/api/devices")
public class DeviceLifecycleController {

    private final JdbcTemplate jdbc;
    private final TransactionTemplate tx;
    private final CertificateRevocationService revocationService;

    public DeviceLifecycleController(JdbcTemplate jdbc, TransactionTemplate tx,
                                     CertificateRevocationService revocationService) {
        this.jdbc = jdbc;
        this.tx = tx;
        this.revocationService = revocationService;
    }

    private record DeviceRow(String certSerial, String certPath, boolean active,
                             OffsetDateTime lastSeen, String status) {}

    @DeleteMapping("/{site}/{node}")
    public ResponseEntity<Void> decommission(@PathVariable String site,
                                             @PathVariable String node,
                                             Principal principal) {
        String nodeId = site + "/" + node;
        var rows = jdbc.query("""
            SELECT d.cert_serial, d.cert_path, d.active, d.last_seen_at, s.status
            FROM devices d
            JOIN device_state s USING (node_id)
            WHERE d.node_id = ?
            """, (rs, i) -> new DeviceRow(
                rs.getString("cert_serial"),
                rs.getString("cert_path"),
                rs.getBoolean("active"),
                rs.getObject("last_seen_at", OffsetDateTime.class),
```

```
            rs.getString("status")), nodeId);

    if (rows.isEmpty()) {
        return ResponseEntity.notFound().build();
    }
    DeviceRow row = rows.get(0);
    if (!row.active()) {
        return ResponseEntity.status(HttpStatus.CONFLICT).build();
    }
    if (!"offline".equals(row.status()) || row.lastSeen() == null
            || row.lastSeen().isAfter(OffsetDateTime.now().minusDays(30))) {
        return ResponseEntity.status(HttpStatus.CONFLICT).build();
    }
    if (row.certSerial() == null || row.certPath() == null) {
        return ResponseEntity.status(HttpStatus.CONFLICT).build();
    }

    // 1. Külső PKI-hívás a SQL tranzakción kívül. A visszavonás idempotens és
    //    fail-closed: ha a későbbi DB-művelet elhasal, a tanúsítvány akkor is
    //    visszavont marad, ami biztonságos irány.
    revocationService.revoke(row.certSerial(), row.certPath(), "cessationOfOperation");

    // 2. A nyilvántartás lezárása és az audit egy tranzakcióban.
    tx.executeWithoutResult(status -> {
        jdbc.update("""
            UPDATE devices
               SET active = false,
                   decommissioned_at = now(),
                   decommission_reason = ?
             WHERE node_id = ? AND active
            """, "decommissioned by operator", nodeId);
        audit(nodeId, "decommission",
              Map.of("cert_serial", row.certSerial()), principal);
    });
    return ResponseEntity.noContent().build();
}

@PostMapping("/{site}/{node}/purge-telemetry")
public ResponseEntity<Void> purgeTelemetry(@PathVariable String site,
                                           @PathVariable String node,
                                           @RequestParam String reason,
                                           Principal principal) {
    String nodeId = site + "/" + node;
    Long closed = jdbc.queryForObject(
        "SELECT count(*) FROM devices WHERE node_id = ? AND NOT active",
        Long.class, nodeId);
    if (closed == null || closed == 0) {
        return ResponseEntity.status(HttpStatus.CONFLICT).build();
    }
    int deleted = jdbc.update(
        "DELETE FROM telemetry WHERE site = ? AND node_id = ?", site, node);
    audit(nodeId, "purge_telemetry",
          Map.of("site", site, "node", node, "rows", deleted, "reason", reason),
          principal);
    return ResponseEntity.noContent().build();
}

private void audit(String nodeId, String action, Map<String, Object> detail,
                   Principal principal) {
    jdbc.update("""
        INSERT INTO device_audit (node_id, action, detail, performed_by)
        VALUES (?, ?, ?::jsonb, ?)
        """, nodeId, action, toJson(detail),
```

```
            principal == null ? "system" : principal.getName());
    }

    private static String toJson(Map<String, Object> detail) {
        try {
            return new com.fasterxml.jackson.databind.ObjectMapper()
                .writeValueAsString(detail);
        } catch (Exception e) {
            throw new IllegalStateException(e);
        }
    }
}
```

A tanúsítvány-visszavonó szolgáltatás interfésze és OpenSSL-alapú laborimplementációja:

```
package com.terrasense.backend.device;

public interface CertificateRevocationService {
    void revoke(String certSerial, String certPath, String reason);
}
```

```
package com.terrasense.backend.device;

import org.springframework.beans.factory.annotation.Value;
import org.springframework.stereotype.Service;

import java.io.IOException;
import java.nio.file.Path;
import java.util.List;

@Service
public class OpenSslCertificateRevocationService
        implements CertificateRevocationService {

    private final String caConfig;
    private final String crlFile;

    public OpenSslCertificateRevocationService(
            @Value("${terrasense.pki.ca-config}") String caConfig,
            @Value("${terrasense.pki.crl-file}") String crlFile) {
        this.caConfig = caConfig;
        this.crlFile = crlFile;
    }

    @Override
    public void revoke(String certSerial, String certPath, String reason) {
        // Az openssl ca -revoke ugyanarra a sorszámra ismételten hívható;
        // a labor-CA index.txt-jében ilyenkor nem keletkezik új bejegyzés.
        run(List.of("openssl", "ca", "-config", caConfig,
                    "-revoke", certPath, "-crl_reason", reason));
        run(List.of("openssl", "ca", "-config", caConfig,
                    "-gencrl", "-out", crlFile));
        run(List.of("openssl", "crl", "-in", crlFile, "-noout", "-text"));
    }

    private void run(List<String> command) {
        try {
            Process process = new ProcessBuilder(command)
                .redirectErrorStream(true)
                .start();
            String output = new String(process.getInputStream().readAllBytes());
            int exit = process.waitFor();
            if (exit != 0) {
                throw new IllegalStateException(
```

```
                    "PKI-parancs sikertelen: " + command + "\n" + output);
            }
        } catch (IOException | InterruptedException e) {
            Thread.currentThread().interrupt();
            throw new IllegalStateException("PKI-parancs hiba", e);
        }
    }
}
```

A `TIMESTAMPTZ` oszlopot itt is `rs.getObject(..., OffsetDateTime.class)` adja vissza; a `Timestamp.class` és a `LocalDateTime` kombinációja azért hibás, mert a PostgreSQL driver `Timestamp`-et ad, a `LocalDateTime` viszont zóna nélküli, és a két típus között nincs automatikus, időzóna-helyes átalakítás. A telemetria törlése a `site` és a `node_id` (`n001`) párost használja, nem a `gh1/n001` összevont kulcsot, mert az ingest a topicból külön tárolja a site-ot és a node-ot.

A tranzakciós határok: a PKI-hívás nem rollbackelhető, ezért a SQL tranzakción kívül, előbb fut; a rekord lezárása és az audit egy tranzakcióban történik. Ha a PKI-hívás sikerül, de a DB-commit meghiúsul, az eszköz tanúsítványa visszavont marad (fail-closed), és az operátor a műveletet idempotensen megismételheti. Nagyobb rendszerben ugyanezt outbox-mintával lehet pontosan egyszerűvé tenni: a tranzakció csak egy `revocation_outbox` sort ír, amelyet egy háttérfeldolgozó hajt végre.

20.4.9 Rövid összefoglaló

A Flottakezelés: nyilvántartás, konfiguráció, állapot alfejezet a TerraSense flottakezelését vezette be. Létrehoztuk a flotta-adatmodellt a `~/iot-labor/i061/schema.sql` fájlban: `device_groups` (csoportok és konfigurációs sablonok), `devices` (identitás, verzió, csoport, jelenléti idő, tanúsítvány-nyilvántartás és aktív státusz), `device_config` (desired/reported JSONB árnyék és verziószám), `device_state` (status, reset ok, utolsó szekvencia és uptime), valamint `device_audit` (kivezetési és adattörlési műveletek). A konfiguráció-kiosztásnál az árnyék mintát alkalmaztuk: a desired-et tároljuk, parancsot küldünk a `terrasense/<site>/<node>/cmd` topicra, és a `cmd/result` üzenetből frissítjük a reported mezőt. A jelenlétfigyelés az MQTT LWT + retained `status` üzenetekre épül. A kivezetés három külön felelősség: a tanúsítvány CRL-be kerül, a `devices` rekord lezárul (`active=false`, `decommissioned_at`, ok) és auditbejegyzés készül; a telemetria alapértelmezés szerint megmarad, törlése csak külön, indoklással naplózott művelettel lehetséges. Ezt az adatmodellt használja az Megfigyelhetőség végponttól végpontig: metrikák, naplók, nyomkövetés és a Záróprojekt: a teljes TerraSense rendszer összeállítása és átvétele alfejezet.

20.4.10 Források és ellenőrzési hiányok

- A Spring Boot 3.3.4 és a PostgreSQL JDBC driver `JSONB`-kezelését a laborban kell ellenőrizni; a `::text` + Jackson megoldás a tananyagban bemutatott, hordozható út, de produkciós környezetben egy explicit `TypeHandler` is indokolt lehet.
- A laborban használt `compose.yaml` szolgáltatás- és adatbázisnevei a korábbi laboroktól függnek; a parancsban feltüntetett `lab-timescaledb` szolgáltatás-, `terra` felhasználó- és `terrasense` adatbázisnév egyezőségét a helyi környezetben kell validálni.
- A tanúsítvány-visszavonás OpenSSL 3.0.13 parancsai (`openssl ca -revoke`, `-gencrl`, `openssl crl`) statikusan, a labor-CA konfigurációja alapján készültek; a tényleges CRL-fájl elérését és a CA `index.txt` állapotát a laborban kell ellenőrizni. A fejezet nem állítja, hogy a visszavonást végrehajtottuk.

- A LwM2M szerverek és felhős flottaszolgáltatások említése szándékosan általános; a konkrét termékösszehasonlítást a témaköri határok kizárják.

21 Megfigyelhetőség, tesztelés és szállítás

21.1 Megfigyelhetőség végponttól végpontig: metrikák, naplók, nyomkövetés

21.1.1 Motiváció

A TerraSense egyetlen hőmérséklet-mérése hosszú utat jár be, mire a Grafana-dashboardon grafikonként megjelenik: a `ts-node` szenzorcsomópont begyűjti, a `ts-gw` fogadja és MQTT-n továbbítja, a Mosquitto broker közvetíti, a `ts-backend` ingest-szolgáltatása feldolgozza, a TimescaleDB tárolja, végül a Grafana lekérdezi. Ha a felhasználó azt tapasztalja, hogy a legfrissebb adat több perce hiányzik, vagy a riasztás nem indult el, akkor az üzemeltetőnek azt kell eldöntenie, hogy a hibás mérés, az eldugult gateway, az MQTT-kapcsolat, a backend-kivétel vagy éppen a dashboard-konfiguráció okozza-e a problémát. A hagyományos, csak helyi naplófájlokra épülő hibakeresés ilyenkor lassú, mert minden réteget külön kell megvizsgálni, és a bejegyzések között nincs közös kapocs.

A megfigyelhetőség ezt a szétszabdalt képet rendezi össze: metrikákkal méri a rendszer viselkedését, strukturált naplókkal rögzíti az eseményeket, nyomokkal pedig egy-egy tranzakció útját követi végig a komponenseken. A TerraSense számára a legfontosabb, hogy egy `trace_id` korrelációs azonosító már az eszközön megszülessen, és ugyanaz az azonosító jelenjen meg a gateway, a backend és a dashboard naplóiban, valamint a nyomok attribútumaiban. Így egy elakadt mérés esetén nem kell találgatni: a `trace_id` alapján azonnal látható, melyik rétegben akadt el az adat.

21.1.2 Fogalmak és működés

Definíció. A megfigyelhetőség (observability) az a tulajdonság, amely lehetővé teszi, hogy a rendszer külső jelei – metrikák, naplók és nyomok – alapján megállapítsuk a belső állapotát, és ok-okozati összefüggéseket tárjunk fel anélkül, hogy új kódot kellene hozzáadnunk. A három pillér nem egymás helyettesítője, hanem egymást kiegészítő eszköz.

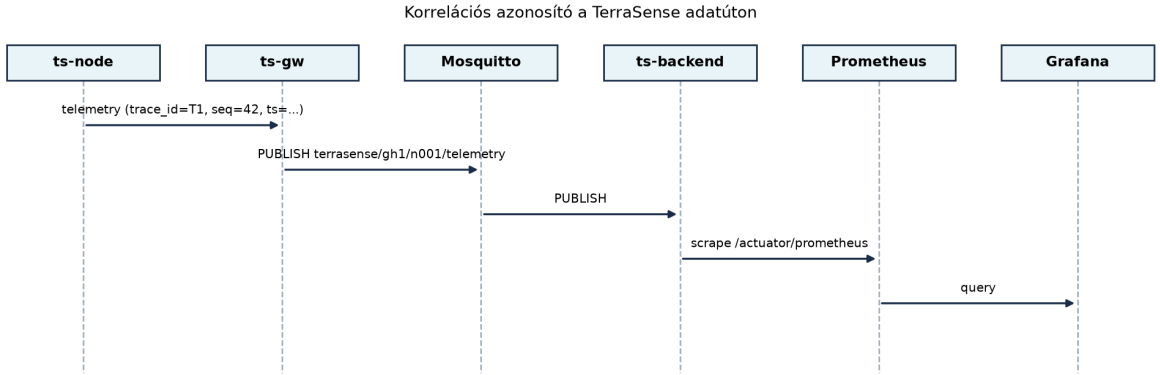
A metrika egy időben aggregált, numerikus mérőszám, amelyet periodikusan gyűjtünk. Az IoT-rendszerekben két nagy csoportra oszlik: az üzleti metrikákra, például az üzenetkésésre vagy a csomagvesztésre, és az erőforrás-metrikákra, például az akkumulátor-töltöttségre, a szabad heapre vagy a processzor-terhelésre. A metrikák tárolása olcsó, lekérdezésük gyors, ezért alkalmasak riasztási szabályok és dashboardok mögé.

A strukturált napló egy géppel könnyen feldolgozható, sémával rendelkező eseményrekord, általában JSON formátumban. A TerraSense-ben minden komponens a `timestamp`, `level`, `service`, `trace_id` és `message` mezőket kötelezően kiírja; a szabad szöveges üzenet mellett további mezők is megjelenhetnek, például `node_id`, `seq` vagy `mqtt_topic`. A napló az egyes események részletes kontextusát adja, de nagy forgalomnál a tárolása drága, ezért ritkítva, rotálva és csak a szükséges szinten gyűjtjük.

A nyom (trace) egy tranzakció teljes életútját leíró, hierarchikus műveletsor. Az OpenTelemetry fogalmi modelljében a trace spanekből áll: minden span egy-egy egységnyi munkát jelöl, és tartalmazza a művelet nevét, azonosítóját, időtartamát, az őt kiváltó szülő span azonosítóját, valamint attribútumokat. A korrelációs azonosító a TerraSense saját, az OpenTelemetrytől független `trace_id` mezője, amelyet a hordozható eszközoldalon is könnyű előállítani. Ez a 64 bites azonosító **nem** azonos az OpenTelemetry 128 bites trace ID-jével: a backendben a saját azonosítót span-attribútumként (`terrasense.trace_id`) rögzítjük, míg a szabványos trace contextet az OpenTelemetry SDK saját, 128 bites azonosítója adja.

Definíció. A korrelációs azonosító (correlation ID, trace ID) egy egyedi, a tranzakció vagy adatút elején generált azonosító, amelyet minden érintett komponens változtatás nélkül továbbvisz, és minden naplóbejegyzésben, nyomattribútumban vagy – ahol a backend támogatja – exemplarban szerepeltet. A TerraSense `trace_id` formátuma 16 karakter hosszú, hexadecimális, véletlenszerű érték, azaz 64 bit. Metrikacímkének soha nem használjuk, mert minden üzenethez új idősor tartozna.

A következő ábra egy telemetriaüzenet útját mutatja a `trace_id` végigkísérésével:



21.1. ábra: Korrelációs azonosító a TerraSense adatúton - A trace_id már az eszközön létrejön, és a teljes feldolgozási láncon azonos marad. Metrikacímkébe nem kerül; a naplókban, a span-attribútumban és opcionálisan exemplarban jelenik meg.

Az ábrán látható, hogy a `ts-node` a telemetriaüzenet `trace_id` mezőjében generálja az azonosítót, és a `ts-gw` valamint a `ts-backend` ezt nem írja felül. A backend a `trace_id`-t a naplóba, az OpenTelemetry span `terrasense.trace_id` attribútumába és – ha a registry támogatja – a hisztogram exemplárjába teszi; metrikacímkébe nem. A metrikákat a Prometheus rendszeres időközönként lekérdezi a backend `/actuator/prometheus` végpontjáról, a Grafana pedig a Prometheusból olvas.

21.1.3 Döntési szempontok

Nem minden számot érdemes metrikaként gyűjteni, és nem minden eseményt érdemes naplózni. A TerraSense esetében a következő szempontok alapján választunk:

Döntés	Választás	Indoklás
Eszközmetrikák közege	Telemetria topic `terrasense/<site>/<node>/telemetry`	Az eszköz nem nyit külön metrikavégpontot; az akkumulátor- és resetszámláló a meglévő mérési sémába kerül.
Gateway-metrikák közege	JSON-napló + lokális Prometheus exporter	A gateway erőforrásai korlátozottak, de a naplókat a journald gyűjti; a kritikus üzemi metrikákat a gatewayen futó exporter teszi elérhetővé.
Backend-metrikák közege	Micrometer + Prometheus	A Spring Boot Actuator szabványos, konténerben futtatható, és a Prometheus natívan scrape-eli.
Korrelációs azonosító helye	`trace_id` mező a telemetria JSON-ben	A CoAP és MQTT üzenet is hordozza; a gateway nem értelmezi, csak továbbítja.
Nyomkövetés a backendben	OpenTelemetry SDK, OTLP exporter	A backend belépési pontján span nyílik; a saját 64 bites `trace_id` a span `terrasense.trace_id` attribútuma, a szabványos 128 bites trace ID-t az SDK generálja.

A metrikák kiválasztásánál kerüljük a túl nagy kardinalitást: a `trace_id`-t például soha ne tegyük metrikacímkének, mert minden méréshez új idősor tartozna, és a Prometheus memóriahasználata robbanásszerűen nőne. Helyette a naplókban, a nyomokban és az exemplarokban keressük az egyedi azonosítót. A metrikacímkék csak alacsony kardinalitású értékeket tartalmazhatnak: `site`, `node`, `topic`, `status`. A `node` címke kardinalitása a flotta méretével nő, és a `topic` is legfeljebb öt értéket vehet fel node-onként, tehát szintén a flotta méretével skálázódik; több ezer eszköznél érdemes lehet csak `site` és `topic` szerint aggregálni, az egyedi eszközt pedig exemplarral vagy külön, ritkábban scrape-elt metrikával megőrizni.

21.1.4 Kidolgozott példa és labor

Munkakönyvtár: `~/iot-labor/i062`; előfeltétel a korábbi laborokból a `ts-node` Zephyr projekt, a `ts-gw` szolgáltatás, a `ts-backend` Maven projekt és a `~/iot-labor/stack/compose.yaml` futó környezet.

A labor célja, hogy a telemetriaüzenetbe `trace_id` kerüljön, a `ts-backend` késleltetés- és veszteségmetrikát számoljon, a Prometheus összegyűjtse, a Grafana pedig megjelenítse azokat.

21.1.4.1 1. lépés: A telemetria séma bővítése `trace_id` mezővel

A laborkönyvtárat a korábbi, Monoton óra és sorszám alfejezet Zephyr-projektjéből másoljuk, majd kiegészítjük a megfigyelhetőségi mezőkkel:

```
mkdir -p ~/iot-labor/i062
cp -r ~/iot-labor/i034 ~/iot-labor/i062/ts-node
cd ~/iot-labor/i062/ts-node
```

A korábbi, az Adatformátumok alfejezetben véglegesített séma kompatibilis módon bővül: az új mezők opcionálisak a régi kliensek számára, de az új `ts-node` mindig kitölti őket. A `src/main.c` mérési struktúrája és a hozzá tartozó időkezelés:

```
#include <zephyr/kernel.h>
#include <zephyr/random/random.h>
#include <stdio.h>
#include <stdint.h>
#include <stddef.h>

struct telemetry {
    char site[8];         /* "gh1" */
    char node_id[8];      /* "n001" */
    uint32_t boot_id;     /* bootonként új, pl. 0x3a7f2b1c */
    uint32_t seq;         /* bootonként 0-tól */
    int64_t uptime_ms;    /* monoton, boot óta */
    int64_t ts;           /* epoch ms, csak szinkronizált óra esetén; különben 0 */
    char trace_id[17];    /* 16 hex + '\0' */
    int16_t temp_mc;      /* °C * 100 */
    uint16_t rh_pct;      /* %RH * 10 */
    uint16_t soil_pct;    /* % * 10 */
    uint32_t reset_count;
    uint16_t battery_mv;
};
```

A `trace_id` generálásához a Zephyr `sys_rand_get` API-ját használjuk. A `native_sim` és a valós kártya entrópiaforrása eltérő; a buildhez a board által biztosított entrópia-driver kell (`CONFIG_ENTROPY_GENERATOR=y`), és a visszatérési értéket ellenőrizni kell:

```
static bool trace_id_generate(char out[17])
{
    uint8_t bytes[8];
    if (sys_rand_get(bytes, sizeof(bytes)) != 0) {
        return false;
    }
    for (int i = 0; i < 8; i++) {
        int b = bytes[i];
        out[2 * i]     = "0123456789abcdef"[b >> 4];
        out[2 * i + 1] = "0123456789abcdef"[b & 0x0f];
    }
    out[16] = '\0';
    return true;
}
```

A `sys_rand_get` Zephyr 3.7-ben `int` visszatérési értéket ad; a `0` a siker. Ha az entrópia még nem áll rendelkezésre, a függvény hibát jelez, és a hívó a mérést `trace_id` nélkül, de a saját `seq`-vel továbbíthatja; a correlation azonosító hiánya nem akadályozhatja a mérés útját.

Az időbélyegek három szintje. A `uptime_ms` a `k_uptime_get()` monoton, boot-relative értéke, amely intervalumok mérésére és a sorrend helyreállítására alkalmas. A `ts` csak akkor epoch ms, ha az eszköz órája már szinkronizált (SNTP, CoAP time vagy a gateway által visszaadott idő); szinkron nélkül `0`. A gateway a fogadáskor `gw_ts` néven saját abszolút epoch ms időbélyeget rendel az üzenethez, és ha a device `ts` értéke `0` vagy érvénytelen, a `ts` mezőt a `gw_ts`-ből normalizálja, `ts_source: "gateway"` jelöléssel. A `native_sim` alatt a host faliórája csak labor-helyettesítő: a `time()` használata nem garantált, és az így mért késleltetés illusztratív, nem hardveres valóság.

A telemetria JSON-je a kanonikus `measurements` objektumot használja, és a gateway a `trace_id`, `reset_count`, `battery_mv`, `boot_id` mezőket változtatás nélkül viszi tovább:

```
{"schema_version":1,"site":"gh1","node_id":"n001","boot_id":"3a7f2b1c",
 "seq":42,"ts":1717000000000,"ts_source":"gateway","gw_ts":1717000000123,
 "trace_id":"3a7f2b1c9e04d85a",
 "measurements":{
   "temperature":{"value":23.41,"unit":"Cel","valid":true},
   "humidity":{"value":61.2,"unit":"%RH","valid":true},
   "soil_moisture":{"value":34.0,"unit":"%","valid":true}},
 "reset_count":0,"battery_mv":3200}
```

A mezők útja a teljes láncon:

Firmware (`struct telemetry`)	MQTT JSON (`TelemetryMessage`)	Backend (`Telemetry`)	Felhasználás
`site`, `node_id`	`site`, `node_id`	`site`, `nodeId`	címkék és az eszköz kulcsa
`boot_id`	`boot_id` (hex string)	`bootId`	a `seq` számláló bootonkénti elkülönítése
`seq`	`seq`	`seq`	sorrend, duplikátum, hézag
`ts`	`ts`	`ts`	eseményidő, késleltetés
`uptime_ms`	`uptime_ms`	`uptimeMs`	eszközoldali intervallum
`trace_id`	`trace_id`	`traceId`	napló, span-attribútum, exemplar
`temp_mc`, `rh_pct`, `soil_pct`	`measurements.*`	`tempC`, `rhPct`, `soilPct`	üzleti metrikák
`reset_count`	`reset_count`	`resetCount`	reset counter
`battery_mv`	`battery_mv`	`batteryMv`	Gauge

A Zephyr projektet a szokásos paranccsal építjük és futtatjuk:

```
west build -b native_sim . -d build
./build/zephyr/zephyr.exe
```

Várt kimenet: a konzolon megjelenik a kanonikus JSON, benne a `trace_id` minden mérésnél új értékkel. A kimenet illusztratív; a `native_sim` időzítése nem hardverhű.

21.1.4.2 2. lépés: A ts-backend metrikák hozzáadása

A `ts-backend` `pom.xml`-jébe a Micrometer Prometheus registry és az Actuator függőségei kerülnek. A verziót a Spring Boot 3.3.4 parent POM kezeli:

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-actuator</artifactId>
</dependency>
<dependency>
    <groupId>io.micrometer</groupId>
```

```
    <artifactId>micrometer-registry-prometheus</artifactId>
</dependency>
```

Az `application.yml`-ben exponáljuk a Prometheus végpontot, és közös `application` címkét adunk minden metrikához:

```
management:
  endpoints:
    web:
      exposure:
        include: health,prometheus
  metrics:
    tags:
      application: ts-backend
```

A kanonikus DTO bővítése. A Ingest alfejezetben megismert `TelemetryMessage` rekordot egészítsük ki a megfigyelhetőségi mezőkkel. A teljes, használható változat (`src/main/java/com/terrasense/backend/TelemetryMessage.java`):

```
package com.terrasense.backend;

import com.fasterxml.jackson.annotation.JsonIgnoreProperties;

import java.util.LinkedHashMap;
import java.util.Map;

@JsonIgnoreProperties(ignoreUnknown = true)
public record TelemetryMessage(
        Integer schema_version,
        String node_id,
        Long seq,
        Long ts,
        String site,
        String boot_id,
        Map<String, Measurement> measurements,
        Double temperature_c,
        Double humidity_rh,
        Double soil_moisture_pct,
        String trace_id,
        Long reset_count,
        Integer battery_mv,
        Long uptime_ms,
        String ts_source,
        Long gw_ts) {

    public static final int CANONICAL_SCHEMA_VERSION = 1;

    public int effectiveSchemaVersion() {
        if (schema_version != null) {
            return schema_version;
        }
        return isLegacyFlat() ? 0 : -1;
    }

    public String effectiveBootId() {
        return (boot_id == null || boot_id.isBlank()) ? "boot-0" : boot_id;
    }

    public boolean isLegacyFlat() {
        return (measurements == null || measurements.isEmpty())
                && (temperature_c != null || humidity_rh != null
                    || soil_moisture_pct != null);
    }
```

```
    public Map<String, Measurement> normalizedMeasurements() {
        if (measurements != null && !measurements.isEmpty()) {
            return measurements;
        }
        Map<String, Measurement> legacy = new LinkedHashMap<>();
        if (temperature_c != null) {
            legacy.put("temperature", new Measurement(temperature_c, "Cel", true, null));
        }
        if (humidity_rh != null) {
            legacy.put("humidity", new Measurement(humidity_rh, "%RH", true, null));
        }
        if (soil_moisture_pct != null) {
            legacy.put("soil_moisture",
                new Measurement(soil_moisture_pct, "%", true, null));
        }
        return legacy;
    }
}
```

A rekord nevei snake_case-ben szerepelnek, mert a JSON is így hordozza őket; a Jackson a rekordkomponensek nevét közvetlenül leképezi. A `trace_id`, `reset_count`, `battery_mv`, `uptime_ms`, `ts_source` és `gw_ts` hiányozhat a régi klienseknél, ezért `String`/`Long`/`Integer` típusúak, nem primitivek. A `@JsonIgnoreProperties(ignoreUnknown = true)` miatt a jövőbeli mezők sem okoznak hibát.

A normalizált nézet. A metrikák és a naplózás ne a nyers MQTT-rekordot használja, hanem egy kis, típusos nézetet (`src/main/java/com/terrasense/backend/Telemetry.java`):

```
package com.terrasense.backend;

import java.util.Map;

public record Telemetry(
        String site,
        String nodeId,
        String bootId,
        long seq,
        long ts,
        String tsSource,
        Long gwTs,
        long uptimeMs,
        String traceId,
        double tempC,
        double rhPct,
        double soilPct,
        long resetCount,
        int batteryMv) {

    public static Telemetry from(String site, TelemetryMessage message) {
        Map<String, Measurement> measurements = message.normalizedMeasurements();
        return new Telemetry(
                site,
                message.node_id(),
                message.effectiveBootId(),
                message.seq() == null ? -1L : message.seq(),
                message.ts() == null ? 0L : message.ts(),
                message.ts_source() == null ? "unknown" : message.ts_source(),
                message.gw_ts(),
                message.uptime_ms() == null ? 0L : message.uptime_ms(),
                message.trace_id() == null ? "" : message.trace_id(),
                value(measurements, "temperature"),
```

```
                value(measurements, "humidity"),
                value(measurements, "soil_moisture"),
                message.reset_count() == null ? 0L : message.reset_count(),
                message.battery_mv() == null ? 0 : message.battery_mv());
    }

    private static double value(Map<String, Measurement> measurements, String key) {
        Measurement measurement = measurements.get(key);
        return measurement == null || measurement.value() == null
                ? Double.NaN
                : measurement.value();
    }
}
```

A `site` értéke a topicból jön (`terrasense/<site>/<node>/telemetry`), nem a payloadból; így a payload `site` mezőjének eltérését az Ingest: MQTT-ből a backendbe, idempotensen validációja külön kezeli. A `nodeId` a payload `node_id` értéke (`n001`), a per-device kulcs pedig a `site` és a `nodeId` párosa.

A sorrend- és hézagkövető. A `seq` hézagát nem szabad azonnal végleges veszteségként számolni, mert az MQTT és a gateway puffere átmenetileg felcserélheti az üzeneteket. A `SeqTracker` előzetes hézagot tart nyilván egy 30 másodperces reorder-ablakig, és csak utána számol végleges veszteséget (`src/main/java/com/terrasense/backend/SeqTracker.java`):

```
package com.terrasense.backend;

import java.util.HashMap;
import java.util.Map;
import java.util.TreeSet;

public final class SeqTracker {

    public enum Event { ACCEPTED, DUPLICATE, LATE }

    private final long reorderWindowMs;
    private final TreeSet<Long> seen = new TreeSet<>();
    private final Map<Long, Long> gapFirstSeenMs = new HashMap<>();
    private long finalizedThrough = -1;
    private long highestSeen = -1;
    private boolean initialized;

    public SeqTracker(long reorderWindowMs) {
        this.reorderWindowMs = reorderWindowMs;
    }

    public Event onMessage(long seq, long nowMs) {
        if (!initialized) {
            initialized = true;
            finalizedThrough = seq - 1;
        }
        if (seq <= finalizedThrough) {
            return Event.LATE;
        }
        if (!seen.add(seq)) {
            return Event.DUPLICATE;
        }
        highestSeen = Math.max(highestSeen, seq);
        for (long missing = finalizedThrough + 1; missing < highestSeen; missing++) {
            if (!seen.contains(missing)) {
                gapFirstSeenMs.putIfAbsent(missing, nowMs);
            }
        }
```

```
            gapFirstSeenMs.remove(seq);
            drainConsecutive();
            return Event.ACCEPTED;
        }

        public long sweep(long nowMs) {
            long lost = 0;
            while (true) {
                Long firstGap = null;
                for (Long gap : gapFirstSeenMs.keySet()) {
                    if (gap > finalizedThrough
                            && (firstGap == null || gap < firstGap)) {
                        firstGap = gap;
                    }
                }
                if (firstGap == null
                        || nowMs - gapFirstSeenMs.get(firstGap) < reorderWindowMs) {
                    return lost;
                }
                lost++;
                finalizedThrough = firstGap;
                gapFirstSeenMs.remove(firstGap);
                drainConsecutive();
            }
        }

        private void drainConsecutive() {
            while (seen.remove(finalizedThrough + 1)) {
                finalizedThrough++;
            }
            gapFirstSeenMs.keySet().removeIf(gap -> gap <= finalizedThrough);
        }
    }
```

Az 1, 3, 2 sorrendben a 3 megérkezésekor a 2 előzetes hézagként kerül a térképbe, a 2 megérkezésekor viszont kikerül, és a `finalizedThrough` 3-ig lép előre; végleges veszteség nem keletkezik. A `sweep()` csak a 30 másodperces ablak után számol végleges veszteséget, a `finalizedThrough` pedig soha nem lép vissza. A `boot_id` váltásakor új tracker indul, így az eszköz újraindulása nem okoz hamis hézagot.

Az ingest szolgáltatás. A `TelemetryIngestService` a trackert, a Timert, a Gauge-ot és a countereket eszközönként egyszer regisztrálja (`src/main/java/com/terrasense/backend/TelemetryIngestServic`

```
package com.terrasense.backend;

import io.micrometer.core.instrument.Counter;
import io.micrometer.core.instrument.Gauge;
import io.micrometer.core.instrument.MeterRegistry;
import io.micrometer.core.instrument.Timer;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.slf4j.MDC;
import org.springframework.scheduling.annotation.Scheduled;
import org.springframework.stereotype.Service;

import java.time.Duration;
import java.util.Map;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.atomic.AtomicInteger;
import java.util.concurrent.atomic.AtomicLong;

@Service
```

```
public class TelemetryIngestService {

    private static final Logger log =
            LoggerFactory.getLogger(TelemetryIngestService.class);
    private static final long REORDER_WINDOW_MS = 30_000;
    private static final long MAX_LATENCY_MS = Duration.ofHours(24).toMillis();

    private final MeterRegistry registry;
    private final Map<DeviceKey, Timer> latencyTimers = new ConcurrentHashMap<>();
    private final Map<DeviceKey, Counter> resetCounters = new ConcurrentHashMap<>();
    private final Map<DeviceKey, AtomicInteger> batteryByDevice =
            new ConcurrentHashMap<>();
    private final Map<DeviceKey, AtomicLong> resetHighWatermarks =
            new ConcurrentHashMap<>();
    private final Map<DeviceKey, BootState> seqTrackers = new ConcurrentHashMap<>();

    public TelemetryIngestService(MeterRegistry registry) {
        this.registry = registry;
    }

    public void process(Telemetry t) {
        DeviceKey key = new DeviceKey(t.site(), t.nodeId());
        long now = System.currentTimeMillis();

        if (t.ts() > 0 && now >= t.ts() && now - t.ts() <= MAX_LATENCY_MS) {
            latencyTimer(key).record(Duration.ofMillis(now - t.ts()));
        } else {
            counter("terrasense.telemetry.time_invalid", key).increment();
        }

        SeqTracker.Event event = seqTracker(key, t.bootId())
                .onMessage(t.seq(), now);
        if (event == SeqTracker.Event.LATE) {
            counter("terrasense.telemetry.late", key).increment();
        } else if (event == SeqTracker.Event.DUPLICATE) {
            counter("terrasense.telemetry.duplicate", key).increment();
        }

        updateResetCounter(key, t.resetCount());
        batteryGauge(key).set(t.batteryMv());

        MDC.put("trace_id", t.traceId());
        try {
            log.info("telemetry ingested: node_id={}, seq={}, trace_id={}, latency_ms={}",
                    t.nodeId(), t.seq(), t.traceId(),
                    t.ts() > 0 ? now - t.ts() : null);
        } finally {
            MDC.remove("trace_id");
        }
    }

    @Scheduled(fixedDelay = 5000)
    public void sweepSequenceGaps() {
        long now = System.currentTimeMillis();
        seqTrackers.forEach((key, state) -> {
            long lost = state.tracker().sweep(now);
            if (lost > 0) {
                missingCounter(key).increment(lost);
            }
        });
    }

    private Timer latencyTimer(DeviceKey key) {
```

```
        return latencyTimers.computeIfAbsent(key, k ->
            Timer.builder("terrasense.telemetry.latency")
                .description("Eseményidő és backend-fogadás közötti késleltetés")
                .tags("site", k.site(), "node", k.node())
                .serviceLevelObjectives(
                    Duration.ofMillis(25), Duration.ofMillis(50),
                    Duration.ofMillis(100), Duration.ofMillis(200),
                    Duration.ofMillis(500), Duration.ofSeconds(1),
                    Duration.ofSeconds(2), Duration.ofSeconds(5),
                    Duration.ofSeconds(10), Duration.ofSeconds(30))
                .register(registry));
    }

    private Counter missingCounter(DeviceKey key) {
        return counter("terrasense.telemetry.missing", key);
    }

    private Counter counter(String name, DeviceKey key) {
        return registry.counter(name, "site", key.site(), "node", key.node());
    }

    private void updateResetCounter(DeviceKey key, long resetCount) {
        AtomicLong high = resetHighWatermarks.computeIfAbsent(
                key, k -> new AtomicLong(resetCount));
        long previous = high.getAndUpdate(old -> Math.max(old, resetCount));
        if (resetCount > previous) {
            resetCounters.computeIfAbsent(key, k ->
                Counter.builder("terrasense.device.reset")
                    .description("Eszköz újraindulások száma")
                    .tags("site", k.site(), "node", k.node())
                    .register(registry))
                .increment(resetCount - previous);
        }
    }

    private AtomicInteger batteryGauge(DeviceKey key) {
        return batteryByDevice.computeIfAbsent(key, k -> {
            AtomicInteger value = new AtomicInteger();
            Gauge.builder("terrasense.device.battery_mv", value, AtomicInteger::get)
                .description("Legutóbbi mért akkumulátorfeszültség millivoltban")
                .tags("site", k.site(), "node", k.node())
                .register(registry);
            return value;
        });
    }

    private SeqTracker seqTracker(DeviceKey key, String bootId) {
        return seqTrackers.compute(key, (k, old) -> {
            if (old == null || !old.bootId().equals(bootId)) {
                return new BootState(bootId, new SeqTracker(REORDER_WINDOW_MS));
            }
            return old;
        }).tracker();
    }

    private record DeviceKey(String site, String node) {}
    private record BootState(String bootId, SeqTracker tracker) {}
}
```

A Timer beállításánál szándékosan a `serviceLevelObjectives` szerepel a `publishPercentiles` helyett. A `publishPercentiles(0.5, 0.95, 0.99)` kliensoldali, előre kiszámított percentiliseket exportál, amelyekből a Prometheus `histogram_quantile` függvénye nem tud dolgozni; a `_bucket` idősorokhoz histogram-konfiguráció kell. A `serviceLevelObjectives` a

megadott 10 határral pontosan azt a 10 véges bucketet adja, amelyet a számolási feladat használ; a Prometheus ezen felül a `+Inf` bucketet, valamint a `_sum` és `_count` sorokat is exportálja. A Timer neve ponttal tagolt (`terrasense.telemetry.latency`), a Prometheus registry ebből `terrasense_telemetry_latency_seconds_bucket` néven másodperc-alapú `_bucket` sorokat készít. A `terrasense:telemetry_latency_p95_ms` recording rule ezért a `histogram_quantile` eredményét 1000-rel szorozza, és a panel már milliszekundumban kapja az értéket; a `histogram_quantile` bemenete továbbra is a másodpercben exportált `_bucket` idősor.

A Gauge-nál a `batteryByDevice` térkép tartja erős referencián az `AtomicInteger`-t, ezért a Micrometer gyenge referenciája nem engedi a GC-nak eltüntetni az értéket. A Gauge-ot eszközönként csak egyszer regisztráljuk; a későbbi minták ugyanazt az `AtomicInteger` példányt frissítik `set()`-tel. A `reset_count` high watermarkja `AtomicLong`-ban él, így a counter csak a tényleges növekedést számolja, és a visszaálló vagy hiányzó érték nem csökkenti a számlálót.

A sorrendkövető állapota mémoire-ben van; a backend újraindulásakor az első üzenet inicializálja a `finalizedThrough` értéket, ezért az újraindulás nem okoz hamis veszteséget. A leállás alatt ténylegesen elveszett üzeneteket ez a memoria nem tudja visszamenőleg azonosítani; ha erre is szükség van, a `finalizedThrough` értékét a `telemetry` táblából kell inicializálni.

21.1.4.3 3. lépés: Prometheus hozzáadása a stack-hez

A `~/iot-labor/stack/compose.yaml`-ben a `ts-backend` és a Prometheus ugyanarra a `terrasense` hálózatra kerül. A meglévő `ts-backend` szolgáltatás alá vedd fel a `networks` sort, majd add hozzá a Prometheus szolgáltatást és a hálózat definícióját:

```
services:
  ts-backend:
    # ... a korábbi beállítások változatlanok ...
    networks: [terrasense]

  prometheus:
    image: prom/prometheus:v2.53.0
    container_name: lab-prometheus
    ports:
      - "9090:9090"
    volumes:
      - ./prometheus/prometheus.yml:/etc/prometheus/prometheus.yml:ro
      - ./prometheus/rules.yml:/etc/prometheus/rules.yml:ro
      - prometheus_data:/prometheus
    command:
      - --config.file=/etc/prometheus/prometheus.yml
      - --storage.tsdb.retention.time=15d
    depends_on:
      - ts-backend
    networks: [terrasense]

volumes:
  prometheus_data:

networks:
  terrasense:
    name: terrasense
```

A `~/iot-labor/stack/prometheus/prometheus.yml` fájl tartalma:

```
global:
  scrape_interval: 15s
```

```
  evaluation_interval: 15s
  external_labels:
    network: terrasense
    environment: lab

rule_files:
  - /etc/prometheus/rules.yml

scrape_configs:
  - job_name: 'ts-backend'
    metrics_path: '/actuator/prometheus'
    static_configs:
      - targets: ['ts-backend:8080']
        labels:
          component: 'ts-backend'
          network: 'terrasense'
```

A `ts-backend` konténer a Compose hálózaton `ts-backend` néven érhető el, a Prometheus ugyanazon a hálózaton fut, ezért a `ts-backend:8080` cél névfeloldása működik. Ha a stackben a backend szolgáltatás neve eltér, a `targets` értékét kell hozzá igazítani.

A dashboard két panelje nem a nyers, hosszú metrikanevet tartalmazó PromQL-t futtatja közvetlenül, hanem recording rule-t olvas. A szabályfájl a `~/iot-labor/stack/prometheus/rules.yml`:

```
groups:
  - name: terrasense-dashboard
    interval: 15s
    rules:
      - record: terrasense:telemetry_latency_p95_ms
        expr: >-
          histogram_quantile(0.95,
            sum by (le, site, node) (
              rate(terrasense_telemetry_latency_seconds_bucket[5m])))
          * 1000
      - record: terrasense:device_reset_increase_5m
        expr: >-
          sum by (site, node) (
            increase(terrasense_device_reset_total[5m]))
```

A `terrasense:telemetry_latency_p95_ms` szabály a másodpercben exportált `_bucket` sorokból számol, majd 1000-rel szoroz, ezért az eredmény már milliszekundum. A `terrasense:device_reset_increase_5m` az elmúlt 5 perc reset-növekményét adja. A Prometheus a szabályokat az `evaluation_interval` szerint, itt 15 másodpercenként értékeli; a dashboard a felvett idősorokat olvassa. Ha a `rules.yml` hiányzik vagy hibás, a nyers metrikák megvannak, de a két panel üres marad.

21.1.4.4 4. lépés: Grafana dashboard a metrikákhoz

A Grafana a Prometheus adatforrást és a dashboardot is provisioningból kapja, kézi importálás nélkül. A `~/iot-labor/stack/grafana/provisioning/datasources/prometheus.yml` fájl:

```
apiVersion: 1

datasources:
  - name: Prometheus
    uid: terrasense-prometheus
    type: prometheus
    access: proxy
    url: http://prometheus:9090
```

```
    isDefault: true
    editable: false
```

A `~/iot-labor/stack/grafana/provisioning/dashboards/terrasense.yml` provider:

```
apiVersion: 1

providers:
  - name: 'TerraSense Observability'
    orgId: 1
    folder: 'TerraSense'
    type: file
    disableDeletion: false
    updateIntervalSeconds: 30
    allowUiUpdates: false
    options:
      path: /var/lib/grafana/dashboards
      foldersFromFilesStructure: false
```

A Grafana szolgáltatásnak mindkét provisioning-könyvtárat és a dashboardfájlt tartalmazó könyvtárat mountolnia kell. A meglévő `lab-grafana` szolgáltatás `volumes` listájához:

```
    volumes:
      - ./grafana/provisioning:/etc/grafana/provisioning:ro
      - ./grafana/dashboards:/var/lib/grafana/dashboards:ro
      - grafana_data:/var/lib/grafana
    networks: [terrasense]
```

A `~/iot-labor/stack/grafana/dashboards/terrasense-observability.json` teljes dashboardja négy panelt tartalmaz. A `datasource` UID megegyezik a fenti `terrasense-prometheus` értékkel, a panelek `gridPos` mezője 2×2 elrendezést ad:

```
{
  "annotations": {"list": []},
  "editable": false,
  "fiscalYearStartMonth": 0,
  "graphTooltip": 1,
  "id": null,
  "links": [],
  "panels": [
    {
      "datasource": {"type": "prometheus", "uid": "terrasense-prometheus"},
      "fieldConfig": {
        "defaults": {
          "color": {"mode": "palette-classic"},
          "custom": {
            "axisCenteredZero": false,
            "axisColorMode": "text",
            "axisLabel": "",
            "axisPlacement": "auto",
            "barAlignment": 0,
            "drawStyle": "line",
            "fillOpacity": 10,
            "gradientMode": "none",
            "hideFrom": {"legend": false, "tooltip": false, "viz": false},
            "insertNulls": false,
            "lineInterpolation": "linear",
            "lineWidth": 1,
            "pointSize": 5,
            "scaleDistribution": {"type": "linear"},
            "showPoints": "never",
            "spanNulls": false,
            "stacking": {"group": "A", "mode": "none"},
```

```
        "thresholdsStyle": {"mode": "off"}
      },
      "mappings": [],
      "thresholds": {
        "mode": "absolute",
        "steps": [{"color": "green", "value": null}]
      },
      "unit": "ms"
    },
    "overrides": []
  },
  "gridPos": {"h": 8, "w": 12, "x": 0, "y": 0},
  "id": 1,
  "options": {
    "legend": {
      "calcs": [],
      "displayMode": "list",
      "placement": "bottom",
      "showLegend": true
    },
    "tooltip": {"mode": "multi", "sort": "desc"}
  },
  "targets": [
    {
      "datasource": {"type": "prometheus", "uid": "terrasense-prometheus"},
      "editorMode": "code",
      "expr": "terrasense:telemetry_latency_p95_ms",
      "legendFormat": "{{site}}/{{node}}",
      "range": true,
      "refId": "A"
    }
  ],
  "title": "Telemetry feldolgozási késleltetés (p95)",
  "type": "timeseries"
},
{
  "datasource": {"type": "prometheus", "uid": "terrasense-prometheus"},
  "fieldConfig": {
    "defaults": {
      "color": {"mode": "palette-classic"},
      "custom": {
        "axisCenteredZero": false,
        "axisColorMode": "text",
        "axisLabel": "",
        "axisPlacement": "auto",
        "barAlignment": 0,
        "drawStyle": "line",
        "fillOpacity": 10,
        "gradientMode": "none",
        "hideFrom": {"legend": false, "tooltip": false, "viz": false},
        "insertNulls": false,
        "lineInterpolation": "linear",
        "lineWidth": 1,
        "pointSize": 5,
        "scaleDistribution": {"type": "linear"},
        "showPoints": "never",
        "spanNulls": false,
        "stacking": {"group": "A", "mode": "none"},
        "thresholdsStyle": {"mode": "off"}
      },
      "mappings": [],
      "thresholds": {
        "mode": "absolute",
```

```
        "steps": [{"color": "green", "value": null}]
      },
      "unit": "short"
    },
    "overrides": []
  },
  "gridPos": {"h": 8, "w": 12, "x": 12, "y": 0},
  "id": 2,
  "options": {
    "legend": {
      "calcs": [],
      "displayMode": "list",
      "placement": "bottom",
      "showLegend": true
    },
    "tooltip": {"mode": "multi", "sort": "desc"}
  },
  "targets": [
    {
      "datasource": {"type": "prometheus", "uid": "terrasense-prometheus"},
      "editorMode": "code",
      "expr": "sum by (site, node) (rate(terrasense_telemetry_missing_total[5m]))",
      "legendFormat": "{{site}}/{{node}}",
      "range": true,
      "refId": "A"
    }
  ],
  "title": "Véglegesen hiányzó üzenetek (db/s)",
  "type": "timeseries"
},
{
  "datasource": {"type": "prometheus", "uid": "terrasense-prometheus"},
  "fieldConfig": {
    "defaults": {
      "color": {"mode": "palette-classic"},
      "custom": {
        "axisCenteredZero": false,
        "axisColorMode": "text",
        "axisLabel": "",
        "axisPlacement": "auto",
        "barAlignment": 0,
        "drawStyle": "bars",
        "fillOpacity": 20,
        "gradientMode": "none",
        "hideFrom": {"legend": false, "tooltip": false, "viz": false},
        "insertNulls": false,
        "lineInterpolation": "linear",
        "lineWidth": 1,
        "pointSize": 5,
        "scaleDistribution": {"type": "linear"},
        "showPoints": "never",
        "spanNulls": false,
        "stacking": {"group": "A", "mode": "none"},
        "thresholdsStyle": {"mode": "off"}
      },
      "mappings": [],
      "thresholds": {
        "mode": "absolute",
        "steps": [{"color": "green", "value": null}]
      },
      "unit": "short"
    },
    "overrides": []
```

```
    },
    "gridPos": {"h": 8, "w": 12, "x": 0, "y": 8},
    "id": 3,
    "options": {
      "legend": {
        "calcs": [],
        "displayMode": "list",
        "placement": "bottom",
        "showLegend": true
      },
      "tooltip": {"mode": "multi", "sort": "desc"}
    },
    "targets": [
      {
        "datasource": {"type": "prometheus", "uid": "terrasense-prometheus"},
        "editorMode": "code",
        "expr": "terrasense:device_reset_increase_5m",
        "legendFormat": "{{site}}/{{node}}",
        "range": true,
        "refId": "A"
      }
    ],
    "title": "Eszköz-resetek (5 perces növekmény)",
    "type": "timeseries"
  },
  {
    "datasource": {"type": "prometheus", "uid": "terrasense-prometheus"},
    "fieldConfig": {
      "defaults": {
        "color": {"mode": "palette-classic"},
        "custom": {
          "axisCenteredZero": false,
          "axisColorMode": "text",
          "axisLabel": "",
          "axisPlacement": "auto",
          "barAlignment": 0,
          "drawStyle": "line",
          "fillOpacity": 10,
          "gradientMode": "none",
          "hideFrom": {"legend": false, "tooltip": false, "viz": false},
          "insertNulls": false,
          "lineInterpolation": "linear",
          "lineWidth": 1,
          "pointSize": 5,
          "scaleDistribution": {"type": "linear"},
          "showPoints": "never",
          "spanNulls": false,
          "stacking": {"group": "A", "mode": "none"},
          "thresholdsStyle": {"mode": "off"}
        },
        "mappings": [],
        "thresholds": {
          "mode": "absolute",
          "steps": [{"color": "green", "value": null}]
        },
        "unit": "mvolt"
      },
      "overrides": []
    },
    "gridPos": {"h": 8, "w": 12, "x": 12, "y": 8},
    "id": 4,
    "options": {
      "legend": {
```

```
          "calcs": [],
          "displayMode": "list",
          "placement": "bottom",
          "showLegend": true
        },
        "tooltip": {"mode": "multi", "sort": "desc"}
      },
      "targets": [
        {
          "datasource": {"type": "prometheus", "uid": "terrasense-prometheus"},
          "editorMode": "code",
          "expr": "terrasense_device_battery_mv",
          "legendFormat": "{{site}}/{{node}}",
          "range": true,
          "refId": "A"
        }
      ],
      "title": "Akkumulátorfeszültség (mV)",
      "type": "timeseries"
    }
  ],
  "refresh": "30s",
  "schemaVersion": 39,
  "tags": ["terrasense", "observability"],
  "templating": {"list": []},
  "time": {"from": "now-1h", "to": "now"},
  "timepicker": {},
  "timezone": "utc",
  "title": "TerraSense Observability",
  "uid": "terrasense-observability",
  "version": 1
}
```

A dashboard JSON `uid` mezője `terrasense-observability`, a datasource UID `terrasense-prometheus`; a kettőnek a provisioning fájlokkal egyeznie kell. A dashboardfájlt a `~/iot-labor/stack/grafana/dashboards/` könyvtárba kell tenni, és a Grafana konténernek ugyanerre a könyvtárra kell mountolnia. A betöltéshez nem kell kézi import: a `docker compose restart lab-grafana` után a provider 30 másodpercen belül betölti a fájlt.

21.1.4.5 5. lépés: Futtatás és ellenőrzés

Indítsuk el a teljes stacket:

```
cd ~/iot-labor/stack
docker compose up -d
```

Indítsuk el a `ts-node` szimulátort és a `ts-gw`-t a korábbi laborok utasításai szerint. Ezután a Prometheus lekérdezésével ellenőrizzük, hogy a metrikák megérkeztek:

```
curl 'http://localhost:9090/api/v1/query?query=terrasense_telemetry_latency_seconds_count'
```

Várt kimenet: a válasz `status` mezője `success`, a `data.result` pedig legalább egy olyan sort tartalmaz, amelynek `value` mezője nullától nagyobb. A pontos szám a feldolgozott üzenetektől függ, ezért a könyv nem ad konkrét értéket. A `_bucket` sorok jelenlétét külön ellenőrizzük:

```
curl 'http://localhost:9090/api/v1/query?query=terrasense_telemetry_latency_seconds_bucket'
```

A válaszban a `le` címke értékei között meg kell jelennie a tíz beállított SLO-határnak és a `+Inf`-nek. A Grafana `http://localhost:3000` címen a `TerraSense Observability` dashboardon a késleltetés és a veszteség grafikonja jelenik meg; a dashboardot nem kell kézzel importálni.

21.1.5 Ellenőrzés és hibakeresés

A megfigyelhetőségi lánc ellenőrzése rétegenként történik:

1. **Eszköz:** a `ts-node` konzolján minden mérés JSON-je tartalmazza a `trace_id` mezőt, és az érték mérésenként változik.
2. **Gateway:** a `journalctl -u ts-gw -f` naplóban a `trace_id` mező megegyezik az eszköz által kiírttal.
3. **Mosquitto:** a `mosquitto_sub -t 'terrasense/#' -v` parancs kimenetén a JSON-ben látszik a `trace_id`.
4. **Backend:** a `/actuator/prometheus` végpont visszaadja a `terrasense_telemetry_latency_seconds_co` metrikát és a `terrasense_telemetry_latency_seconds_bucket` sorokat.
5. **Prometheus:** a `http://localhost:9090/targets` oldalon a `ts-backend` target állapota UP, és a legutóbbi scrape sikeres.
6. **Grafana:** a dashboard panel nem üres, és az idősor a friss adatokat mutatja.

Hibatünet	Lehetséges ok	Javítás
A Prometheus target DOWN	A `ts-backend` konténer nem fut, vagy a `metrics_path` hibás.	Ellenőrizzük a `docker compose ps` kimenetét és a `curl`-t a backend `/actuator/health` végpontjára.
A metrika értéke nulla	Nincs bejövő MQTT-üzenet, vagy az ingest nem hívódik meg.	Nézzük a backend naplóját; `mosquitto_sub`-bel ellenőrizzük a topicot.
A késleltetés értéke negatív vagy irreálisan nagy	Az eszköz `ts` mezője nem szinkronizált, vagy a gateway nem normalizálta a `gw_ts`-ből.	Ellenőrizzük a `ts_source` mezőt; szinkron nélkül a gatewaynek kell a `ts`-t a `gw_ts`-ből kitöltenie. A `time_invalid` counter növekedése erre utal.
A dashboard nem frissül	A Grafana provisioning path nem megfelelő, vagy a datasource UID hibás.	Ellenőrizzük a provisioning YAML-t és a dashboard JSON `datasource` mezőjét.

21.1.6 Tipikus hibák és félreértések

1. **Helyi idő, uptime és szinkronizált epoch összekeverése.** A `ts-node` `uptime_ms` értéke monoton, de nem epoch idő; a `native_sim` host faliórája labor-helyettesítő, nem az eszköz garantált órája. A `ts` csak szinkronizált eszközóra esetén epoch ms; egyébként a gatewaynek kell a `gw_ts`-ből normalizálnia. A backend a negatív vagy 24 óránál nagyobb késleltetést ne rögzítse, hanem `time_invalid` counterrel jelezze.
2. **Korrelációs azonosító hiánya a gateway naplójában.** Sok fejlesztő a `trace_id`-t csak a backendben generálja, így az eszköz és a gateway naplóit nem lehet összekötni. A `trace_id`-t már az eszközön elő kell állítani, és a gatewaynek átlátszóan tovább kell vinnie.
3. **Magas kardinalitású metrikacímke.** A `trace_id`-t vagy a `ts`-t metrikacímkének használva minden üzenet új idősor keletkezik, ami a Prometheus memóriáját percek alatt kimeríti. A korrelációs azonosító a naplóba, a nyomattribútumba és az exemplarba való, nem a metrikacímkébe.

4. **OpenTelemetry és saját `trace_id` kettőssége.** A saját `trace_id` 64 bites, az OpenTelemetry trace ID 128 bites. A saját azonosítót nem szabad a szabványos trace ID helyére tenni; a helyes megoldás a `terrasense.trace_id` span-attribútum, miközben az SDK saját 128 bites trace contextet generál. Így a napló és a nyom összekapcsolható anélkül, hogy a szabványos kontextus érvénytelenné válna.

5. **A `seq` hézag azonnali veszteségként kezelése.** Az MQTT és a gateway puffere átmenetileg felcserélheti az üzeneteket; a 3-as előtt érkező 2-es még nem veszett el. Csak egy reorder-ablak után, eszköz- és bootazonosító szerint elkülönített számlálóval szabad végleges veszteséget számolni. A `finalizedThrough` értéke soha nem léphet vissza.

6. **`publishPercentiles` használata `_bucket` helyett.** A `publishPercentiles` kliensoldali percentiliseket exportál, amelyekből a Prometheus `histogram_quantile` nem tud dolgozni. A p95-höz histogram-konfiguráció kell (`serviceLevelObjectives` vagy `publishPercentileHistogram`), és a panelen a másodpercben exportált értéket kell milliszekundumra váltani.

21.1.7 Feladatok

1. **Fogalmi/döntési kérdés.** Magyarázd el, miért nem alkalmas a naplózás önmagában a TerraSense késleltetésének folyamatos megfigyelésére, és miért jobb erre a metrika. Milyen esetben van szükség mégis a naplóra?

2. **Számolási/tervezési feladat.** A `ts-node` percenként 12 telemetriaüzenetet küld. A Prometheus 15 másodpercenként scrape-el. A `terrasense.telemetry.latency` Timerhez 10 véges SLO-bucket tartozik, site-onként 100 node-dal, összesen 5 site-on. (a) Hány aktív idősora van a hisztogramnak (bucketek, `+Inf`, `_sum`, `_count`)? (b) Mennyi a 30 napos scrape-minták száma? (c) Becsüld meg a lemezigenyet 1,5 B/minta tömörítéssel, és hasonlítsd össze a 6500 aktív sor 2 kB/soros head-memóriájával! (d) Miért nem érdemes a `trace_id`-t címkének tenni?

3. **Implementációs feladat.** Egészítsd ki a `ts-gw` naplózását úgy, hogy az MQTT-publikáláskor a napló JSON-jében szerepeljen a `trace_id` mező, amelyet a bejövő UDP-csomag JSON-jéből olvas ki. A gatewaynek nem kell értelmeznie a mezőt, csak továbbítania.

4. **Hibakeresési/kiterjesztési feladat.** A Grafana dashboardon a `terrasense_telemetry_missing_total` counter értéke folyamatosan nő, pedig a `ts-node` minden üzenetet elküld. Hol lehet a hiba? Javasolj legalább két lehetséges okot és a hozzájuk tartozó ellenőrzési lépést.

21.1.8 Megoldások

1. A naplózás minden egyes eseményt rögzít, de nincs időbeli aggregációja; a késleltetés folyamatos méréséhez hisztogram kell, amelyből percentilisek számíthatók. A napló akkor kell, ha egy konkrét mérés útját akarjuk visszafejteni a `trace_id` alapján, mert ott az egyedi kontextus (például a gateway sorhossza vagy a backend kivétel stack trace-e) megtalálható.

2. (a) A 10 véges SLO-bucket és a `+Inf` együtt 11 `le` idősor; ezekhez jön a `_sum` és a `_count`, tehát node-onként 13 idősor. 5 site $\times$ 100 node $=$ 500 node, így $500 \times 13 = 6500$ aktív idősor. (A Micrometer egyes verziói `_max` sort is exportálhatnak; a pontos darabszámot a scrape kimenetén kell ellenőrizni. A feladat a 13 sorral számol.)

(b) 15 másodperces scrape esetén percenként 4 minta készül. 30 nap = 30 × 24 × 60 = 43 200 perc, ezért egy idősor 43 200 × 4 = 172 800 mintát kap. A 6500 idősor összesen 6500 × 172 800 = 1 123 200 000 1,12 milliárd minta 30 nap alatt.

(c) A 6500 aktív sor head-memóriája 6500 × 2 kB = 13 MB; ez csak az aktív sorok fejében tárolt metaadat, nem a havi adat, és nem tartalmazza az indexet, a WAL-t és a blokkokat. A 30 napos lemezigeny 1,5 B/minta becsléssel 1,12 × 10^9 × 1,5 B 1,68 GB; 2 B/mintával 2,25 GB. A 15 napos retention ezt nagyjából felére csökkenti, az index és a WAL külön költség. A két számot tehát külön kell kezelni: a 13 MB a head-memória, a GB-os nagyságrend a lemez.

(d) A `trace_id` címkeként minden üzenethez új idősor keletkezne: 500 node × 12 üzenet/perc × 43 200 perc = 259 200 000 új idősor 30 nap alatt. Ezek a sorok nem tűnnek el a scrape után, hanem a Prometheus head blokkjában és indexében maradnak, így a kardinalitás néhány nap alatt kezelhetetlenné válik. A `trace_id` helye a napló, a span-attribútum és az exemplar.

3. A `ts-gw` C++ kódjában a JSON-feldolgozó könyvtár (például `nlohmann::json`) segítségével olvassuk ki a `trace_id` mezőt, normalizáljuk a `ts`-t, és a naplózóban is írjuk ki:

```
#include <chrono>
#include <string>
#include <nlohmann/json.hpp>

static long long now_epoch_ms() {
    using namespace std::chrono;
    return duration_cast<milliseconds>(
        system_clock::now().time_since_epoch()).count();
}

void publish_to_mqtt(const std::string& topic, const std::string& payload) {
    auto j = nlohmann::json::parse(payload);
    std::string trace_id = j.value("trace_id", "");
    long long ts = j.value("ts", 0LL);
    if (ts <= 0) {
        j["ts"] = now_epoch_ms();
        j["ts_source"] = "gateway";
    } else {
        j["ts_source"] = "device";
    }
    j["gw_ts"] = now_epoch_ms();
    logger.info("mqtt_publish", {{"topic", topic}, {"trace_id", trace_id}});
    mqtt_client.publish(topic, j.dump());
}
```

A gateway a `trace_id`-t nem értelmezi, csak továbbítja; a `ts` normalizálása és a `gw_ts` hozzáadása az időkezelési szerződés része. A `system_clock` a gateway NTP-szinkronizált óráját használja, nem a `ts-node` uptime-ját.

4. A `terrasense_telemetry_missing_total` counter növekedése azt jelenti, hogy a `SeqTracker` a reorder-ablak után véglegesen hiányzó üzenetet látott. Lehetséges okok és ellenőrzések:

   - **Tényleges csomagvesztés a `ts-node` és a `ts-gw` között.** A gateway `udp_rx_total` értéke elmarad a `ts-node` által küldött `seq`-ektől. Ellenőrzés: a gateway naplójában a `seq`-ek és az UDP-csomagok száma; a `terrasense_telemetry_late_total` és `duplicate_total` alakulása.

- **A `boot_id` nem változik újrainduláskor.** Ha az eszköz újraindul, de ugyanazt a `boot_id`-t küldi, a `seq` 0-tól indulása hamis hézagot okoz. Ellenőrzés: a payload `boot_id` mezője és a `reset_count` növekedése; a `boot_id`-nek bootonként újnak kell lennie.
- **A gateway store-and-forward puffere későn küldi újra a régi üzeneteket.** A `seq` a `finalizedThrough` alá esik, ezért `late_total` nő, nem `missing_total`. Ellenőrzés: a gateway pufferének hossza és a `gw_ts/ts` különbség.
- **A tracker kulcsa nem tartalmazza a site-ot vagy a node-ot.** Két site azonos `n001` node-ja összekeveredik. Ellenőrzés: a `DeviceKey(site, node)` használata és a Prometheus `site/node` címkéi.
- **A backend újraindult.** A tracker az első üzenettel inicializál, ezért önmagában nem okoz hamis veszteséget; a leállás alatt ténylegesen elveszett üzenetek viszont nem számlálódnak. Ellenőrzés: a backend indítási ideje és a `telemetry` tábla utolsó `seq` értéke.

21.1.9 Rövid összefoglaló

A TerraSense megfigyelhetősége három pilléren nyugszik: metrikákon, strukturált naplókon és nyomokon. A `trace_id` korrelációs azonosítót már a `ts-node` generálja, a `ts-gw` és a `ts-backend` változtatás nélkül továbbviszi, és a naplókban, valamint a `terrasense.trace_id` span-attribútumban szerepelteti; metrikacímkébe nem kerül. A backend Micrometerrel és Prometheus registryvel méri az eseményidő és a backend-fogadás közötti késleltetést (`terrasense.telemetry.latency`, másodpercben exportált `_bucket` sorokkal), a `seq` reorder-ablak utáni végleges veszteségét (`terrasense.telemetry.missing`), az eszköz-reseteket (`terrasense.device.reset`) és az akkumulátor-feszültséget (`terrasense.device.battery_mv`). Az időkezelés három szintje külön szerepel: a `ts-node` monoton uptime-ja, a gateway által normalizált abszolút `ts/gw_ts`, és a backend `server_ts`-e. A Prometheus a `/actuator/prometheus` végpontról scrape-el, a Grafana pedig a provisioningból betöltött `terrasense-observability.json` dashboardon jeleníti meg a p95 késleltetést és a hibamutatókat. **Továbbvitt eredmény.** A `trace_id` mező, a másodperc-alapú `terrasense_telemetry_latency_seconds` hisztogram, a `terrasense_telemetry_missing_total` counter, a `boot_id` szerinti seq-követés és a Grafana-dashboard konvenciója az Tesztelési szintek alfejezet integrációs tesztjeiben, valamint a Záróprojekt átvételi méréseiben használandó.

21.1.10 Források és ellenőrzési hiányok

- OpenTelemetry dokumentáció: https://opentelemetry.io/docs/ — a span- és trace-fogalmak stabilak, de a Java SDK és a Micrometer Tracing bridge pontos API-ja verziófüggő. A saját 64 bites `trace_id` és a szabványos 128 bites trace ID megkülönböztetése ebben a fejezetben tudatos tervezési döntés.
- Micrometer 1.13.x (Spring Boot 3.3.4): https://docs.micrometer.io/micrometer/reference/concepts/histogram-quantiles.html — a `Timer`, a `serviceLevelObjectives`, a `publishPercentileHistogram` és a `publishPercentiles` szemantikája itt dokumentált; a `_bucket` exportot a labor scrape-kimenetén kell ellenőrizni.
- Prometheus: https://prometheus.io/docs/practices/naming/ és https://prometheus.io/docs/prometheus/latest/querying/functions/ — a másodperc-alapú `_seconds` utótag, a `_bucket`/`_sum`/`_count` sorok és a `histogram_quantile` szintaxisa stabil, de a konkrét bucketek a Timer konfigurációjától függenek.

- Grafana 11.2 provisioning: https://grafana.com/docs/grafana/latest/administration/provisioning/ — a datasource UID, a dashboard provider és a dashboard JSON sémája verziófüggő; a fejezet a 11.2.0 image-et feltételezi.
- Zephyr 3.7 `sys_rand_get`: a https://docs.zephyrproject.org/3.7.0/ dokumentációban ellenőrizendő a `native_sim` és a választott kártya entrópiaforrása. A `CONFIG_ENTROPY_GENERATOR=y` önmagában nem garantál működő drivert; a build és a boot alatti entrópiaállapot ellenőrzése szükséges.
- **Ellenőrzési megjegyzés.** A fejezet Java-, C++- és Zephyr-kódpéldái statikusan ellenőrzöttek; a labort (Maven, Docker, Zephyr build, Prometheus scrape, Grafana) ebben a javítási körben nem futtattuk. A megadott kimenetek és a 100 ms-os ellenőrzés a labor elvégzése után igazolandók.

21.2 Tesztelési szintek: unit, integráció, HIL, terheléses teszt

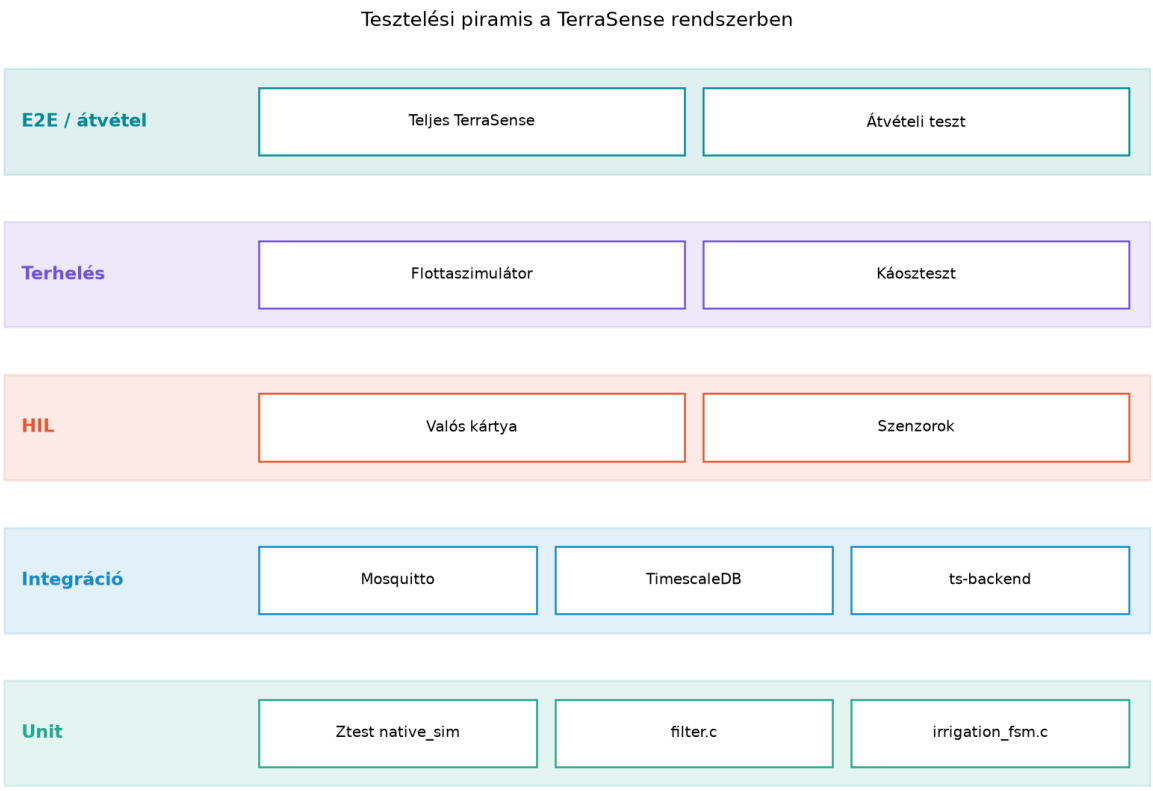
21.2.1 Motiváció

A TerraSense fejlesztése során a `ts-node` firmware-t és a `ts-backend` ingestjét külön-külön már teszteltük, de a valódi hibák gyakran a határokon jelentkeznek: amikor a broker újraindul, a hálózat megszakad, vagy száz virtuális node egyszerre küldi a telemetriát. Egy integrációs teszt például felfedi, hogy a `ts-backend` nem kezeli a duplikált MQTT-üzeneteket, vagy hogy a TimescaleDB séma nem felel meg a valós üzenetformátumnak. A terheléses teszt pedig megmutatja, hogy az ingest szűk keresztmetszete nem a broker, hanem a backend tranzakciókezelése. Ebben az alfejezetben felépítjük azt a tesztelési piramist, amely a hoston futó unit tesztektől a konténeres integrációs teszteken át a szimulált flottáig és a valódi hardvert használó HIL-tesztpadig vezet.

21.2.2 Fogalmak és működés

Definíció. A **tesztelési piramis** (test pyramid) a szoftvertesztelés klasszikus modellje, amely a teszteket végrehajtási költségük és számuk szerint rétegekbe rendezi: alul a gyors, sok apró **unit teszt**, középen a kisebb számú **integrációs teszt**, felül pedig a kevés, lassú **rendszer- vagy end-to-end (E2E) teszt** áll. IoT-rendszerben a piramis kiegészül a **HIL** (hardware-in-the-loop) és a **terheléses teszt** szintjével, mert a szoftver helyessége mellett a hardverrel való együttműködést és a skálázhatóságot is igazolni kell.

A TerraSense esetében a piramis rétegei a következőképpen alakulnak. A legalsó réteg a `ts-node` firmware hoston futtatott Ztest unit tesztjei, amelyeket az Eszközszoftver tesztelése a hoston alfejezetben készítettünk el. Fölötte az integrációs tesztek a `ts-backend` és a `ts-gw` egyes komponenseit valódi függőségekkel (Mosquitto, TimescaleDB) tesztelik konténerekben. A terheléses teszt sok virtuális node-ot szimulál, és azt vizsgálja, hogy a backend képes-e lépést tartani a telemetriaárammal. A HIL-tesztpad valódi fejlesztőkártyát és szenzorokat kapcsol a teszthurokba, hogy a hardverközeli hibákat is felfedje. Végül a záróprojekt átvételi tesztje a teljes rendszert ellenőrzi.



21.2. ábra: Tesztelési piramis a TerraSense rendszerben - A rétegenkénti tesztszám felfelé haladva csökken, a végrehajtási idő és a környezeti függőség nő.

Az ábra szöveges leírása: a piramis alján a legtöbb, leggyorsabb teszt található (`filter.c`, `irrigation_fsm.c` Ztest tesztjei); fölötte az integrációs tesztek a backend és a broker/adatbázis együttműködését ellenőrzik; a HIL-tesztpad valódi mikrovezérlőt használ; a terheléses és E2E tesztek a legritkábbak, de a legnagyobb lefedettséget adják.

Definíció. A **HIL-teszt** (hardware-in-the-loop) olyan tesztelési módszer, amelyben a valódi beágyazott hardver fut, de a környezetét (szenzorjelek, aktuátorok, rádiós hálózat) szimulátorok vagy laboratóriumi műszerek helyettesítik. Így a firmware a tényleges perifériákkal és időzítésekkel fut, miközben a tesztkörnyezet determinisztikusan előállítható és ismételhető.

Definíció. A **terheléses teszt** (load test) a rendszer viselkedését méri a várható vagy annál nagyobb terhelés alatt, például sok eszköz egyidejű üzenetküldése esetén. Célja az átviteli képesség, a késleltetés és az erőforrás-felhasználás számszerűsítése, valamint a szűk keresztmetszetek azonosítása.

21.2.3 Döntési szempontok

A tesztelési szintek közötti választás mindig a hiba költsége és a teszt végrehajtási ideje közötti kompromisszum. Az alábbi táblázat összefoglalja a fő szempontokat.

Szint	Milyen kérdésre válaszol?	Környezet	Gyakoriság	Költség
Unit	Helyes-e egy függvény vagy modul?	Host, mockokkal	Minden commitnál	Alacsony
Integráció	Együttműködnek-e a komponensek?	Konténerek, valódi broker/DB	Napi, merge-kor	Közepes
HIL	Működik-e a firmware a hardverrel?	Fejlesztőkártya, műszerek	Kiadás előtt	Magas
Terhelés	Bírja-e a várható flottaméretet?	Szimulált flotta	Terhelési kampányban	Közepes
E2E	Megfelel-e a teljes rendszer a követelményeknek?	Teljes stack	Átvételkor	Magas

A TerraSense fejlesztése során az a gyakorlati szabály, hogy minden új backend-funkcióhoz legalább egy integrációs teszt tartozzon, a terheléses tesztet pedig a flottaméret tervezésekor (például „100 node, 1 Hz telemetria”) futtassuk. A HIL-t a Zephyr BSP vagy az alacsony szintű periféria-illesztő módosításakor éri meg bevetni.

21.2.4 Labor: integrációs és terheléses teszt

Munkakönyvtár:

```
mkdir -p ~/iot-labor/i063 && cd ~/iot-labor/i063
python3 -m venv .venv
. .venv/bin/activate
pip install 'paho-mqtt==2.1.0'
```

A labor előfeltétele az Ingest: MQTT-ből a backendbe, idempotensen alfejezetben elkészített `ts-backend` projekt, amelyet a `~/iot-labor/i044/ts-backend` könyvtárban tárolunk. A labor során a projekt `src/test/` könyvtára alá tesszük az integrációs tesztet, a `~/iot-labor/i063` munkakönyvtárba pedig a flottaszimulátort. A teszt a Testcontainers segítségével külön, izolált Mosquitto brokert és TimescaleDB-t indít, ezért nem függ a `~/iot-labor/stack/compose.yaml` futó szolgáltatásaitól.

1. A tesztfüggőségek. A `ts-backend` `pom.xml` fájljában a `</properties>` zárótag után vegyük fel a Testcontainers BOM-ját, a `<dependencies>` szakaszba pedig a teszt-hatókörű függőségeket. A Paho MQTT 5 kliens már az Ingest: MQTT-ből a backendbe, idempotensen alfejezetbeli `pom.xml`-ben compile hatókörrel szerepel, ezért a teszt segédosztálya ugyanazt a klienst használja, mint az éles ingest:

```
<dependencyManagement>
    <dependencies>
        <dependency>
            <groupId>org.testcontainers</groupId>
            <artifactId>testcontainers-bom</artifactId>
            <version>1.20.3</version>
            <type>pom</type>
            <scope>import</scope>
```

```
        </dependency>
    </dependencies>
</dependencyManagement>

<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-test</artifactId>
    <scope>test</scope>
</dependency>
<dependency>
    <groupId>org.testcontainers</groupId>
    <artifactId>junit-jupiter</artifactId>
    <scope>test</scope>
</dependency>
<dependency>
    <groupId>org.testcontainers</groupId>
    <artifactId>postgresql</artifactId>
    <scope>test</scope>
</dependency>
<dependency>
    <groupId>org.awaitility</groupId>
    <artifactId>awaitility</artifactId>
    <version>4.2.2</version>
    <scope>test</scope>
</dependency>
```

A `spring-boot-starter-test` a JUnit 5-öt, az AssertJ-t és a Spring Boot tesztkontextusát hozza; a Testcontainers BOM a `junit-jupiter` és a `postgresql` modul verzióját egységesen 1.20.3-ra rögzíti. A Maven 3.9.9 és a Spring Boot 3.3.4 szülő a fordítás és a Surefire futtatás verzióját is meghatározza.

2. A teszt-broker konfigurációja. A Mosquitto 2 alapértelmezésben csak a loopback interfészen figyel, ezért a konténerben létrehozott `listener` bejegyzés nélkül a host felől nem lehet kapcsolódni. A `src/test/resources/mosquitto-test.conf` fájl csak a teszthez készül, éles környezetben nem használjuk:

```
listener 1883 0.0.0.0
allow_anonymous true
persistence false
```

Az `allow_anonymous true` kizárólag a tesztkonténer izolált hálózatán elfogadható; az éles TerraSense broker jelszófájlt és TLS-t használ, ahogyan az MQTT biztonság és a PKI alfejezetekben szerepel. A `persistence false` miatt a teszt végén nem marad vissza brokerállapot.

3. Az integrációs teszt. A `src/test/java/com/terrasense/backend/IngestIntegrationTest.java` fájl teljes tartalma:

```
package com.terrasense.backend;

import static org.assertj.core.api.Assertions.assertThat;
import static org.awaitility.Awaitility.await;

import java.nio.charset.StandardCharsets;
import java.nio.file.Path;
import java.time.Duration;
import java.util.UUID;
import org.eclipse.paho.mqttv5.client.IMqttToken;
import org.eclipse.paho.mqttv5.client.MqttClient;
import org.eclipse.paho.mqttv5.client.MqttConnectionOptions;
import org.eclipse.paho.mqttv5.common.MqttException;
import org.eclipse.paho.mqttv5.common.MqttMessage;
import org.junit.jupiter.api.Test;
```

```
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.context.SpringBootTest;
import org.springframework.jdbc.core.JdbcTemplate;
import org.springframework.test.context.DynamicPropertyRegistry;
import org.springframework.test.context.DynamicPropertySource;
import org.testcontainers.containers.GenericContainer;
import org.testcontainers.containers.PostgreSQLContainer;
import org.testcontainers.containers.wait.strategy.Wait;
import org.testcontainers.junit.jupiter.Container;
import org.testcontainers.junit.jupiter.Testcontainers;
import org.testcontainers.utility.DockerImageName;
import org.testcontainers.utility.MountableFile;

@Testcontainers
@SpringBootTest(
        webEnvironment = SpringBootTest.WebEnvironment.RANDOM_PORT,
        properties = {"spring.sql.init.mode=always"})
class IngestIntegrationTest {

    private static final String SITE = "gh1";
    private static final String NODE = "n001";
    private static final String BOOT_ID = "boot-0001";
    private static final long SEQ = 42L;
    private static final String TOPIC =
            "terrasense/" + SITE + "/" + NODE + "/telemetry";
    private static final String CLIENT_ID = "ts-backend-it-" + UUID.randomUUID();
    private static final String SPOOL_PATH = Path.of("target", "test-spool",
            UUID.randomUUID().toString(), "ingest-retry.jsonl")
            .toAbsolutePath().toString();

    @Container
    static final GenericContainer<?> mosquitto = new GenericContainer<>(
            DockerImageName.parse("eclipse-mosquitto:2.0.18"))
            .withExposedPorts(1883)
            .withCopyFileToContainer(
                    MountableFile.forClasspathResource("mosquitto-test.conf"),
                    "/mosquitto/config/mosquitto.conf")
            .withCommand("mosquitto", "-c", "/mosquitto/config/mosquitto.conf")
            .waitingFor(Wait.forLogMessage(".*mosquitto version .* running.*", 1)
                    .withStartupTimeout(Duration.ofSeconds(60)));

    @Container
    static final PostgreSQLContainer<?> timescale = new PostgreSQLContainer<>(
            DockerImageName.parse("timescale/timescaledb:2.17.2-pg16"))
            .withDatabaseName("terrasense_it")
            .withUsername("terra")
            .withPassword("lab-password");

    @DynamicPropertySource
    static void properties(DynamicPropertyRegistry registry) {
        registry.add("spring.datasource.url", timescale::getJdbcUrl);
        registry.add("spring.datasource.username", timescale::getUsername);
        registry.add("spring.datasource.password", timescale::getPassword);
        registry.add("ingest.broker-uri", () -> "tcp://" + mosquitto.getHost()
                + ":" + mosquitto.getMappedPort(1883));
        registry.add("ingest.client-id", () -> CLIENT_ID);
        registry.add("ingest.topic-filter", () -> "terrasense/+/+/telemetry");
        registry.add("ingest.spool-path", () -> SPOOL_PATH);
    }

    @Autowired
    private JdbcTemplate jdbc;
```

```
@Test
void telemetryIngestIsIdempotentAndConflictsGoToDeadLetter() throws Exception {
    String brokerUri = "tcp://" + mosquitto.getHost() + ":"
            + mosquitto.getMappedPort(1883);
    try (MqttTestPublisher publisher = new MqttTestPublisher(brokerUri)) {
        publisher.publish(TOPIC, payload(21.5));
        await().atMost(Duration.ofSeconds(20))
                .pollInterval(Duration.ofMillis(200))
                .alias("az első telemetria megérkezése")
                .untilAsserted(() ->
                        assertThat(countTelemetryRows()).isEqualTo(1));

        // Ugyanaz az esemény újra: nem keletkezhet második sor.
        publisher.publish(TOPIC, payload(21.5));
        // Ugyanaz a kulcs, eltérő tartalom: a DLQ-ba kell kerülnie.
        publisher.publish(TOPIC, payload(25.0));

        await().atMost(Duration.ofSeconds(20))
                .pollInterval(Duration.ofMillis(200))
                .alias("a tartalmi ütközés DLQ-ba kerülése")
                .untilAsserted(() ->
                        assertThat(countContentMismatches()).isEqualTo(1));
    }

    assertThat(countTelemetryRows()).isEqualTo(1);
    String storedTemperature = jdbc.queryForObject("""
            SELECT payload->'measurements'->'temperature'->>'value'
            FROM telemetry
            WHERE site = ? AND node_id = ? AND boot_id = ? AND seq = ?
            """, String.class, SITE, NODE, BOOT_ID, SEQ);
    assertThat(storedTemperature).isEqualTo("21.5");
    assertThat(countContentMismatches()).isEqualTo(1);
}

private int countTelemetryRows() {
    Integer count = jdbc.queryForObject("""
            SELECT count(*) FROM telemetry
            WHERE site = ? AND node_id = ? AND boot_id = ? AND seq = ?
            """, Integer.class, SITE, NODE, BOOT_ID, SEQ);
    return count == null ? 0 : count;
}

private int countContentMismatches() {
    Integer count = jdbc.queryForObject("""
            SELECT count(*) FROM dead_letter
            WHERE topic = ? AND reason LIKE ?
            """, Integer.class, TOPIC, "%eltérő tartalom%");
    return count == null ? 0 : count;
}

private static String payload(double temperatureC) {
    return """
            {"schema_version":1,"site":"%s","node_id":"%s","boot_id":"%s",
             "seq":%d,"ts":1700000000000,
             "measurements":{
               "temperature":{"value":%s,"unit":"Cel","valid":true},
               "humidity":{"value":55.0,"unit":"%%RH","valid":true},
               "soil_moisture":{"value":30.0,"unit":"%%","valid":true}}}
            """.formatted(SITE, NODE, BOOT_ID, SEQ, temperatureC);
}

private static final class MqttTestPublisher implements AutoCloseable {
```

```
        private static final long PUBLISH_TIMEOUT_MS = 10_000L;

        private final MqttClient client;

        private MqttTestPublisher(String brokerUri) throws MqttException {
            this.client = new MqttClient(
                    brokerUri, "it-publisher-" + UUID.randomUUID());
            MqttConnectionOptions options = new MqttConnectionOptions();
            options.setCleanStart(true);
            options.setAutomaticReconnect(false);
            options.setConnectionTimeout(10);
            client.connect(options);
        }

        private void publish(String topic, String json) throws MqttException {
            MqttMessage message =
                    new MqttMessage(json.getBytes(StandardCharsets.UTF_8));
            message.setQos(1);
            IMqttToken token = client.publish(topic, message);
            token.waitForCompletion(PUBLISH_TIMEOUT_MS);
        }

        @Override
        public void close() throws MqttException {
            if (client.isConnected()) {
                client.disconnect();
            }
            client.close();
        }
    }
}
```

A teszt felépítése a következő. A `PostgreSQLContainer` a `timescale/timescaledb:2.17.2-pg16` image-et indítja `terrasense_it` adatbázissal, és saját readiness-stratégiája addig vár, amíg a PostgreSQL el nem fogadja a kapcsolatokat. A Spring Boot `spring.sql.init.mode=always` beállítása miatt az Ingest: MQTT-ből a backendbe, idempotensen alfejezetbeli `schema.sql` ebben az izolált adatbázisban fut le, így a `telemetry` és a `dead_letter` tábla a teszt indulására létezik. A Mosquitto konténer a `mosquitto-test.conf` fájlt kapja, és a várakozás a „running” naplósorra épül, nem csupán a nyitott portra; így a broker valóban fogad kapcsolatot, mire a Spring kontextus elindul. A `@DynamicPropertySource` a tényleges `ingest.broker-uri`, `ingest.client-id`, `ingest.topic-filter` és `ingest.spool-path` kulcsokat írja felül, nem egy nem létező `mqtt.broker.uri` property-t. A spool útvonala `target/test-spool/` alá mutat, ezért a teszt nem ír a fejlesztői spoolba, és a Maven `clean` a maradékot eltakarítja.

Az idempotencia ellenőrzése szándékosan két üzenettel zárul. Az első publikálás után az Awaitility legfeljebb 20 másodpercig várja, hogy a `(site, node_id, boot_id, seq)` kulcson pontosan egy sor legyen. Ezután a teszt elküldi ugyanazt az üzenetet, majd egy azonos kulcsú, de eltérő hőmérsékletű változatot. Mivel ugyanaz a Paho kliens ugyanarra a topicra QoS 1-gyel, sorrendben publikál, a backend egyetlen callback-szálon dolgozza fel őket, ezért a DLQ-ba kerülő tartalmi ütközés egyben azt is bizonyítja, hogy a megelőző duplikátum már feldolgozódott. A teszt végén a `telemetry` táblában továbbra is egy sor van, és az eredeti `21.5` értéket tartalmazza; a `dead_letter` táblában pontosan egy, „eltérő tartalom” okú sor található. A `Thread.sleep(1000)` helyett minden várakozás feltételalapú és maximális határidős, ezért lassabb broker vagy adatbázis mellett is determinisztikus, hiba esetén pedig az Awaitility kiírja az utolsó állapotot.

A `MqttTestPublisher` `AutoCloseable`, ezért a `try`-with-resources a teszt kivétele esetén is lezárja a kapcsolatot. A `publish` a Paho MQTT 5 `IMqttToken.waitForCompletion` hívásával

megvárja a QoS 1-es PUBACK-ot, így a teszt nem attól függ, hogy a hálózat mikor szállítja az üzenetet.

4. A teszt futtatása. A `ts-backend` projekt gyökerében:

```
cd ~/iot-labor/i044/ts-backend
mvn -B clean verify
```

A kimenet alakja (illusztratív, a konkrét időzítések és a konténerindítási sorok gépfüggők):

```
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0
[INFO] BUILD SUCCESS
```

A `Tests run: 1` azt jelenti, hogy a teszt lefutott, nem azt, hogy a Docker nélküli környezetben is lefutna: a Testcontainers Docker daemont igényel. A példát a lektorálási környezetben nem futtattuk Maven, Docker vagy Testcontainers alatt; a Java-forrás statikus ellenőrzése történt, a tényleges `mvn verify` eredménye a labor elvégzésének része.

5. A flottaszimulátor. A `~/iot-labor/i063/flotta-szimulator.py` fájl teljes tartalma. A szimulátor a kanonikus, 1-es verziójú `measurements` payloadot küldi, node-onként 1-től induló `seq`-vel, QoS 1-gyel, és csak a broker által nyugtázott üzeneteket számolja elküldöttnek:

```
#!/usr/bin/env python3
import argparse
import json
import random
import threading
import time

import paho.mqtt.client as mqtt
from paho.mqtt.enums import CallbackAPIVersion


def gen_payload(site, node_id, boot_id, seq):
    return json.dumps({
        "schema_version": 1,
        "site": site,
        "node_id": node_id,
        "boot_id": boot_id,
        "seq": seq,
        "ts": int(time.time() * 1000),
        "measurements": {
            "temperature": {
                "value": round(20 + random.uniform(-2, 2), 2),
                "unit": "Cel", "valid": True,
            },
            "humidity": {
                "value": round(50 + random.uniform(-10, 10), 1),
                "unit": "%RH", "valid": True,
            },
            "soil_moisture": {
                "value": round(30 + random.uniform(-5, 5), 1),
                "unit": "%", "valid": True,
            },
        },
    }, separators=(",", ":"))


def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--broker", default="localhost")
    ap.add_argument("--port", type=int, default=1883)
```

```
ap.add_argument("--site", default="gh1")
ap.add_argument("--boot-id", default="sim-boot")
ap.add_argument("--nodes", type=int, default=50)
ap.add_argument("--interval", type=float, default=1.0)
ap.add_argument("--duration", type=float, default=30.0)
ap.add_argument("--publish-timeout", type=float, default=10.0)
args = ap.parse_args()

connected = threading.Event()
acked = 0
acked_lock = threading.Lock()

def on_connect(client, userdata, flags, reason_code, properties):
    if reason_code.is_failure:
        print(f"Kapcsolódási hiba: {reason_code}", flush=True)
        return
    connected.set()

def on_publish(client, userdata, mid, reason_code, properties):
    nonlocal acked
    with acked_lock:
        acked += 1

client = mqtt.Client(
    CallbackAPIVersion.VERSION2,
    client_id=f"flotta-szim-{int(time.time())}",
)
client.on_connect = on_connect
client.on_publish = on_publish
client.connect(args.broker, args.port, keepalive=60)
client.loop_start()
if not connected.wait(timeout=10.0):
    client.loop_stop()
    raise SystemExit("Nincs CONNACK 10 másodpercen belül.")

nodes = [f"n{n + 1:03d}" for n in range(args.nodes)]
round_no = 0
attempted = 0
queued = 0
rejected = 0
start = time.monotonic()
try:
    while time.monotonic() - start < args.duration:
        round_no += 1
        # Minden node minden körben publikál, ezért a közös körszámláló
        # node-onként 1-től induló, monoton seq-et ad.
        for node in nodes:
            topic = f"terrasense/{args.site}/{node}/telemetry"
            payload = gen_payload(args.site, node, args.boot_id, round_no)
            info = client.publish(topic, payload, qos=1)
            attempted += 1
            if info.rc != mqtt.MQTT_ERR_SUCCESS:
                rejected += 1
                continue
            queued += 1
            try:
                info.wait_for_publish(timeout=args.publish_timeout)
            except RuntimeError:
                pass
            if not info.is_published():
                print(f"PUBACK-időtúllépés: {topic} seq={round_no}",
                      flush=True)
        time.sleep(args.interval)
```

```
    finally:
        client.disconnect()
        client.loop_stop()

    elapsed = time.monotonic() - start
    with acked_lock:
        acked_snapshot = acked
    unacked = queued - acked_snapshot
    print(f"Kísérlet: {attempted}, sorba állítva: {queued}, "
          f"elutasítva: {rejected}, PUBACK: {acked_snapshot}, "
          f"nyugtázatlan: {unacked}")
    print(f"Átlagos nyugtázott sebesség: "
          f"{acked_snapshot / elapsed:.2f} üzenet/s")
    print(f"Node-onkénti seq tartomány: 1..{round_no}")


if __name__ == "__main__":
    main()
```

A számlálók jelentése szétválik. Az `attempted` a `publish` hívások száma, a `queued` a kliens által sikeresen sorba állított üzeneteké, a `rejected` a helyi hiba miatt elutasítottaké, az `acked` pedig az `on_publish` callback által látott PUBACK-oké. A QoS 1-es üzenet csak akkor számít átadottnak, ha a `wait_for_publish` visszatért és az `is_published()` igaz; a puszta `publish` visszatérése nem jelent brokeres nyugtázást. A ciklus végén a `disconnect` csak azután fut, hogy minden sorba állított üzenetre megérkezett a PUBACK vagy lejárt a publikálási időtúllépés, ezért a nyugtázatlanok száma valódi hiányt jelez, nem a leállított event loop miatt elveszett számlálást.

6. A terheléses teszt futtatása. Indítsuk el a stack brokert, a TimescaleDB-t és a backendet (a `ts-backend` az Ingest: MQTT-ből a backendbe, idempotensen alfejezetbeli vagy a későbbi, migrált sémával is futhat):

```
docker compose -f ~/iot-labor/stack/compose.yaml up -d lab-broker lab-timescaledb ts-backend
python3 flotta-szimulator.py --broker localhost --port 1883 \
    --nodes 100 --interval 0.5 --duration 30
```

A szimulátor kimenetének alakja (illusztratív):

```
Kísérlet: 6000, sorba állítva: 6000, elutasítva: 0, PUBACK: 6000, nyugtázatlan: 0
Átlagos nyugtázott sebesség: 199.87 üzenet/s
Node-onkénti seq tartomány: 1..60
```

A PUBACK-szám a broker által átvett üzeneteket mutatja, a tárolt egyedi események száma ettől eltérhet, mert a backend aszinkron dolgozza fel az üzeneteket, és a duplikátumokat az idempotens beszúrás szűri ki. A tárolt számot a nyers eseménytáblából kérdezhetjük le; az Ingest: MQTT-ből a backendbe, idempotensen alfejezetbeli sémában ez a `telemetry`, az Idősoros adatbázisok és adatmodellezés-ös migráció után a `telemetry_ingest` tábla:

```
docker compose -f ~/iot-labor/stack/compose.yaml exec -T lab-timescaledb \
    psql -U terra -d terrasense -c \
    "SELECT count(*) AS stored FROM telemetry_ingest \
     WHERE site = 'gh1' AND boot_id = 'sim-boot';"
```

A tényleges hiány a `PUBACK` és a `stored` különbsége, miután az ingest befejezte a feldolgozást. A kettő közötti eltérés nem feltétlenül hiba: a QoS 1 a brokerig garantál kézbesítést, a backend ingestje és a tranzakció viszont néhány másodperccel lemaradhat. A terheléses teszt során a ts-backend ingestjét a Megfigyelhetőség végponttól végpontig alfejezetben bevezetett metrikákkal is figyeljük (például a feldolgozási késleltetés percentilisei, a duplikátumok és a hiányzó seq-ek száma). A várt eredmény nem egy pontos szám, hanem az, hogy a backend hosszabb időn át

stabilan feldolgozza a beérkező üzeneteket, a nyugtázott és a tárolt egyedi események különbsége nem nő korlátlanul, és a monitorozott késleltetés elfogadható marad.

21.2.5 HIL-tesztpad

A `ts-node` firmware valódi hardveren történő teszteléséhez egy HIL-tesztpadot építünk. A tesztpad felépítése a következő: a fejlesztőkártya (például egy Zephyr által támogatott ARM kártya) a rajta futó `ts-node` firmware-rel; a szenzorok helyett egy programozható szenzor-emulátor (például egy másik mikrovezérlő vagy egy digitális-analóg konverter), amely a hőmérséklet, páratartalom és talajnedvesség jeleit adja; az aktuátor (öntözőszelep) helyett egy relé vagy LED, amelynek állapotát a host méri. A hoston futó tesztkliens az UART vagy ethernet kapcsolaton kapja a firmware logjait, és a tesztforgatókönyv vezérli az emulátort, valamint ellenőrzi a firmware reakcióit.



21.3. ábra: HIL-tesztpad felépítése - A tesztpad célja a hardverközeli hibák reprodukálása determinisztikus környezetben.

A HIL-tesztpad előnye, hogy a firmware a tényleges periféria-időzítésekkel fut, így a megszakítások, az ADC kalibráció és a watchdog viselkedése is ellenőrizhető. A `native_sim` ezt nem helyettesíti, de a HIL-tesztpad drága és lassú, ezért csak a kritikus hardverfüggő részekhez használjuk, például a száraz érintkező megszakításának vagy az alvásból ébredésnek a teszteléséhez.

21.2.6 Káoszteszt

A terheléses tesztet kiegészíthetjük káosztesszttel (chaos testing), amely során szándékosan hibákat idézünk elő a rendszerben, hogy megfigyeljük a helyreállást és az üzenetvesztés nélküli kézbesítést. Két tipikus jelenet a TerraSense esetében:

1. **Broker-újraindítás:** a flottaszimulátor futása közben a Mosquitto konténert leállítjuk és újraindítjuk. Az MQTT 5-ös kliensek (vagy a QoS 1/2 munkamenetek) újracsatlakoznak, és a ts-backend idempotens ingestje garantálja, hogy a duplikált üzenetek ne szennyezzék az adatbázist. A teszt sikeres, ha az újraindítás után a backend nem veszít el üzenetet, és nem rögzít duplikátumot.
2. **Hálózatkiesés a gateway és a backend között:** a `ts-gw` store-and-forward puffere tárolja a telemetriát, amíg a kapcsolat helyreáll, majd visszatölti az MQTT-n keresztül. A teszt során a gateway hálózati interfészét leszűrjük (például `iptables` DROP), majd visszaengedjük, és ellenőrizzük, hogy a visszatöltés sorrendje és az `seq` sorszámok monoton növekvőek maradnak.

21.2.7 Ellenőrzés és hibakeresés

Hibatünet	Ok	Javítás
Az `IngestIntegrationTest` `BUILD FAILURE` a TimescaleDB kapcsolatnál	A konténer nem állt készen, vagy a JDBC URL hibás	A `PostgreSQLContainer` saját readiness-stratégiája és a `getJdbcUrl()` használata; ellenőrizzük, hogy a Docker daemon fut, és hogy a `spring.datasource.url` nem mutat a fejlesztői `terrasense` adatbázisra
A teszt `Connection refused` hibát kap a Mosquitto felé	A broker a 2-es verzió alapértelmezése szerint csak a loopbacken figyel, vagy a konfigurációs fájl nem jutott el a konténerbe	Ellenőrizzük a `mosquitto-test.conf` `listener 1883 0.0.0.0` sorát és a `withCopyFileToContainer` útvonalát; a várakozás a „running” naplósorra épüljön
A terheléses teszt során a `ts-backend` CPU-ja 100%, és a késleltetés nő	A telemetria feldolgozása szinkron, egy szálon történik, vagy a tranzakció túl nagy	A kódban az ingest kötegelt beszúrással dolgozik; a `@Scheduled` feldolgozás és a tranzakcióméret ellenőrzése
A káoszteszt után a TimescaleDB-ben duplikált rekordok jelennek meg	Az idempotencia kulcs `(site, node_id, boot_id, seq)` nem egyedi a sémában, vagy az ingest nem ellenőrzi a duplikátumot	A repository-ban a `(site, node_id, boot_id, seq)` elsődleges kulcsot és az `INSERT ... ON CONFLICT DO NOTHING` műveletet kell ellenőrizni
A HIL-tesztpadon a firmware nem válaszol	A soros port nem megfelelő, vagy a firmware assert miatt leállt	A hoston a `dmesg`-ben ellenőrizzük az USB-TTL eszközt; a logokban keressük a watchdog vagy assert üzeneteket

21.2.8 Tipikus hibák és félreértések

1. **Azt hinni, hogy a unit tesztek elégségesek.** A hoston futó Ztest tesztek gyorsak, de nem fedik fel a brokerrel való MQTT-kézfogás, a hálózati késleltetés vagy a sémaeltérés hibáit. Az integrációs teszt nélkül a backend és a broker együttműködését csak élesben tesztelnénk.

2. **A terheléses tesztet a valós flotta méretével futtatni.** A szimulátor által generált üzenetek nem helyettesítik a valódi eszközök hálózati viselkedését (például a rádiós ütközéseket vagy a változó késleltetést), ezért a kapott átviteli sebesség felső becslés. A terheléses teszt célja nem a pontos szám, hanem a szűk keresztmetszet megtalálása.

3. **Az idempotencia teszt elhagyása.** Ha az ingest csak a `ts` időbélyeg alapján szúr be, akkor a QoS 1-es újraküldés duplikátumot okozhat. Az integrációs tesztben ezért mindig a `(site, node_id, boot_id, seq)` eseménykulcsot teszteljük.

4. **A HIL-tesztpadot a teljes firmware tesztelésére használni.** A HIL drága, ezért csak a hardverfüggő kódot tesztelje; a logikát a hoston, a `native_sim`-mel teszteljük.

21.2.9 Feladatok

1. **(a) Fogalmi kérdés** Ismertesse, miért nem elég a hoston futó unit tesztek a `ts-backend` ingestjének hibáinak felderítésére. Milyen hibatípusokat fed fel az integrációs teszt?

2. **(b) Tervezési feladat** Tegyük fel, hogy a TerraSense flottája 200 node-ot tartalmaz, és mindegyik másodpercenként egy telemetriaüzenetet küld. Az ingest feldolgozási ideje üzenetenként átlagosan 5 ms. Számítsa ki, hány üzenet érkezik másodpercenként, és mekkora a backend processzor kihasználtsága, ha egyetlen szálon dolgozik fel. Számoljon azzal, hogy a feldolgozás során a CPU 80%-ban a backend logikát, 20%-ban a rendszer fut. (A pontos műveleti idő illusztratív, a valós méréshez profilozás kell.)

3. **(c) Implementációs feladat** A laborban bemutatott szimulátor minden körben minden node-ra publikál, ezért a közös körszámláló node-onként 1-től induló `seq`-et ad. Egészítse ki a szimulátort explicit, node-onkénti `seq`-számlálóval, amely akkor sem ugrik, ha egy node-ot kihagyunk a körből, és a futás végén írja ki az összes nyugtázott (PUBACK-olt) üzenet számát és a mért átlagos sebességet (üzenet/s). A QoS maradjon 1. A kódot a `~/iot-labor/i063/flotta-szimulator.py` fájlba mentse, és futtassa `--nodes 20 --duration 10` paraméterekkel. Várható kimenet: a script a szimuláció végén kiírja a nyugtázott üzenetszámot és a sebességet.

4. **(d) Hibakeresési feladat** A terheléses teszt során a ts-backend ingestje időnként `ConnectionRefused` hibát dob a broker felé, amikor a broker újraindul. Milyen Paho MQTT 5 beállításokkal lehet a backend kliensét automatikus újracsatlakozásra bírni? Írja le a `setAutomaticReconnect`, a `setCleanStart` és a `setSessionExpiryInterval` opciók szerepét, és indokolja, miért szükséges a `connectComplete` callbackben újra feliratkozni a `terrasense/+/+/telemetry` topicra.

21.2.10 Megoldások

1. **Megoldás:** A unit tesztek kizárólag a függvény logikáját ellenőrzik mockolt függőségekkel. Nem fedik fel a valódi brokerrel való kapcsolat hibáit (például az MQTT 5 tulajdonságok félreértelmezését), a TimescaleDB sémájának eltérését, vagy a hálózati késleltetésből adódó sorrendiségi problémákat. Az integrációs teszt valódi konténerekben futtatja a komponenseket, így ezek a hibák reprodukálhatók.

2. **Megoldás:** Az üzenetek másodpercenkénti száma:

$$200 \text{ node} \times 1 \text{ üzenet/s} = 200 \text{ üzenet/s}.$$

A feldolgozási idő összesen:

$$200 \text{ üzenet/s} \times 5 \text{ ms} = 1000 \text{ ms/s} = 100\% \text{ CPU}.$$

A teljes CPU-kihasználtság tehát 100%, ezen belül a backend logika 800 ms/s-ot (80%), a rendszerfutás 200 ms/s-ot (20%) használ. Ezért párhuzamosítani kell a feldolgozást, vagy kötegelt beszúrást használni a TimescaleDB-be, különben a késleltetés növekszik és a backend túlterhelődik. A számítás illusztratív, a tényleges érték mérést igényel.

3. **Megoldás:** Az alapciklus közös `round_no` számlálója a normál futásban már node-onként 1-től induló, monoton `seq`-et ad, mert minden körben minden node pontosan egyszer publikál. Az alábbi módosítás ezt teszi explicitté, és lehetővé teszi egyes node-ok kihagyását anélkül, hogy a többi node `seq`-je ugrana. A QoS szándékosan 1 marad, tehát a mérés kézbesítési feltételei nem változnak; a valódi hiba, amit a labor javít, a PUBACK-várakozás és a külön nyugtázott számláló:

```
seq_per_node = {node: 0 for node in nodes}
skipped = set()
attempted = 0
queued = 0
acked_before = 0
start = time.monotonic()
while time.monotonic() - start < args.duration:
    for node in nodes:
        if node in skipped:
            continue
        seq_per_node[node] += 1
        topic = f"terrasense/{args.site}/{node}/telemetry"
        payload = gen_payload(args.site, node, args.boot_id,
                              seq_per_node[node])
        info = client.publish(topic, payload, qos=1)
        attempted += 1
        if info.rc != mqtt.MQTT_ERR_SUCCESS:
            continue
        queued += 1
        try:
            info.wait_for_publish(timeout=args.publish_timeout)
        except RuntimeError:
            pass
    time.sleep(args.interval)
```

A `seq_per_node` szótárat a szimulátor a `nodes` lista létrehozása után inicializálja, a `skipped` halmaz pedig a kihagyott node-okat tartalmazza; a kihagyás nem növeli a kimaradó node `seq`-jét. A futás végén a nyugtázott üzenetek száma az `on_publish` callback `acked` számlálója, a sebesség pedig `acked / elapsed`; a kísérlet, a sorba állított, az elutasított és a nyugtázatlan üzeneteket ugyanúgy kell kiírni, mint a laborbeli szimulátorban. Futtatás után például `PUBACK: 200` és `Átlagos nyugtázott sebesség: 20.00 üzenet/s` látható (20 node, 10 másodperc, 1 Hz); a pontos érték a futtatástól és a broker válaszidejétől függ.

4. **Megoldás:** A Paho MQTT 5 kliensnél a `setAutomaticReconnect(true)` engedélyezi, hogy a kliens a kapcsolat megszakadása után magától újracsatlakozzon. A `setCleanStart(false)` azt kéri, hogy a broker tartsa fenn a munkamenetet az újracsatlakozásig, a `setSessionExpiryInterval(3600)` pedig megadja, hány másodpercig őrizze meg a broker a munkamenetet és a QoS 1-es várakozó üzeneteket. A `cleanStart` és a `sessionExpiryInterval` nem ugyanaz: a `cleanStart` az új kapcsolat indulásakor dönt a korábbi munkamenet eldobásáról, a `sessionExpiryInterval` pedig a kapcsolat megszakadása utáni megőrzési idő. A TerraSense-nél a stabil `ingest.client-id`, a `cleanStart=false`, a `sessionExpiryInterval=3600` és a QoS 1 együtt biztosítja, hogy a rövid broker-újraindítás alatt várakozó üzeneteket a backend a visszacsatlakozás után megkapja; az idempotens ingest a duplikátumokat kiszűri. A `connectComplete` callbackben azért kell újra feliratkozni, mert az automatikus újracsatlakozás nem feltétlenül őrzi meg a feliratkozásokat minden broker- vagy munkamenet-állapot mellett; a kód minden sikeres csatlakozás után meghívja a `subscribe()` metódust.

21.2.11 Rövid összefoglaló

Az alfejezetben kiépítettük a TerraSense tesztelési piramisát, amely a hoston futó Ztest unit tesztektől a konténeres integrációs teszteken át a HIL-tesztpadig és a terheléses/káosztesztig terjed. Létrehoztuk az `IngestIntegrationTest.java` integrációs tesztet, amely Testcontainers-rel izolált Mosquitto és TimescaleDB konténereket indít, a tényleges `ingest.broker-uri` property-t használja, a kanonikus `measurements` payloadot küldi, feltételalapú várakozással ellenőrzi az idempotens beszúrást, és a tartalmi ütközést a `dead_letter` táblában igazolja. Elkészítettük a `flotta-szimulator.py` szimulátort, amely node-onkénti `seq`-et, QoS 1-et és PUBACK-alapú számlálást használ, így a kísérlet, a nyugtázott és a tárolt egyedi eseményszám összevethető. A HIL-tesztpad felépítését és a káoszteszt forgatókönyveit is ismertettük. **Továbbvitt eredmény:** az `IngestIntegrationTest.java` és a `flotta-szimulator.py` a CI/CD a firmware-től a backendig alfejezet pipeline-jába illeszkedik, és a Záróprojekt: a teljes TerraSense rendszer összeállítása és átvétele átvételi tesztjében is felhasználjuk.

21.2.12 Források és ellenőrzési hiányok

- Testcontainers Java API: a `@Container`, a `@Testcontainers` és a `DynamicPropertySource` működése a Spring Boot 3.3.4 és a Testcontainers 1.20.3 verziójához igazodik. A `PostgreSQLContainer` readiness-stratégiája és a `timescale/timescaledb:2.17.2-pg16` image kompatibilitása a labor elvégzésekor ellenőrizendő; a példát a lektorálási környezetben Docker és Maven nélkül nem futtattuk.
- Eclipse Paho MQTT 5 Java kliens: a `MqttClient.publish` `IMqttToken` visszatérési értéke és a `waitForCompletion` a Paho 1.2.5 API része, amelyet az Ingest: MQTT-ből a backendbe, idempotensen alfejezetbeli projekt használ. A Java-forrás statikus ellenőrzése megtörtént, a tényleges fordítás és tesztfuttatás a labor feladata.
- Paho MQTT Python kliens: a példa a `paho-mqtt==2.1.0` csomagra és a `CallbackAPIVersion.VERSION2` API-ra épül. A `wait_for_publish` időtúllépéskor `RuntimeError` kivételt dobhat, ezért a szimulátor ezt elkapja, és az `is_published()` értékből számol.
- Zephyr Ztest és Twister: a HIL-tesztpadhoz a Twister `--device-testing` kapcsolója a Zephyr 3.7.0 LTS dokumentációja szerint változhat; a HIL-tesztpadot a lektorálási környezetben nem futtattuk.
- A Mosquitto 2.0.18 és a TimescaleDB 2.17.2 image-ek a könyv rögzített verzióprofiljához igazodnak. A `mosquitto-test.conf` `allow_anonymous true` beállítása kizárólag a tesztkonténer izolált hálózatán érvényes; éles környezetben hitelesítés és TLS szükséges.

21.3 CI/CD a firmware-től a backendig

21.3.1 Motiváció

A TerraSense fejlesztése során a `ts-node` firmware, a `ts-gw` gateway és a `ts-backend` kódbázisa párhuzamosan változik. A kézi build és telepítés lassú, hibás és nem reprodukálható: egy fejlesztő más Zephyr SDK-val, más függőségverziókkal vagy más Docker-alaprendszerrel dolgozik, mint a kollégája, ezért a tesztelés eredménye nem megbízható. A biztonsági frissítéseknél (lásd OTA-frissítés: A/B képek, rollback, delta, fokozatos kiadás) és a teljes rendszer átvételénél (Záróprojekt: a teljes TerraSense rendszer összeállítása és átvétele) a kiadott műtermékeknek – firmware, konténerképek, konfigurációk – visszakövethetőnek, aláírtnak és ellenőrzöttnek kell lenniük. A CI/CD folyamat ezt automatizálja: minden commit után futtat buildet, teszteket, aláírást és SBOM-generálást, majd a kiadási szakaszban előkészíti a fokozatos OTA-telepítést.

21.3.2 Fogalmak és működés

Definíció. A **folytonos integráció (Continuous Integration, CI)** olyan gyakorlat, amelyben a fejlesztők kódváltoztatásai egy közös verziótárolóba kerülve automatikusan buildelésre és tesztelésre kerülnek. Célja a hibák korai felismerése és a mindig működő főág fenntartása.

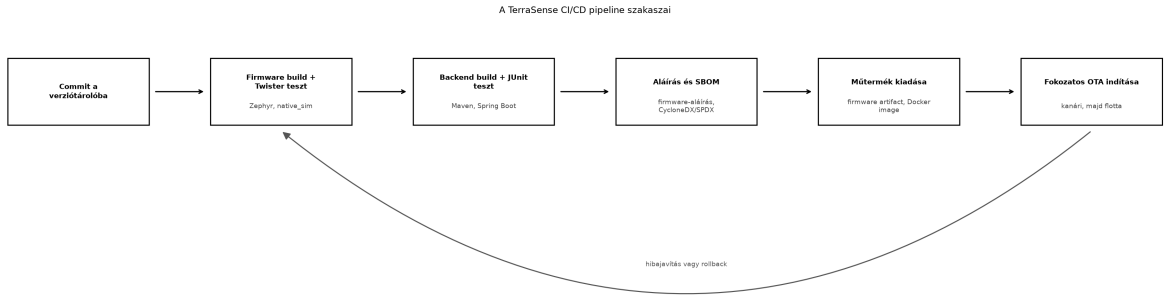
Definíció. A **folytonos szállítás (Continuous Delivery, CD)** a CI kiterjesztése: az ellenőrzött műtermékek automatikusan kiadható állapotba kerülnek, és a telepítés – esetünkben a fokozatos OTA – felügyelt, jóváhagyott lépésben történik. Ha a telepítés is teljesen automatikus, **folytonos telepítésről (Continuous Deployment)** beszélünk. Az IoT-ban a CD többnyire emberi vagy policy-alapú jóváhagyást igényel a flotta mérete és a visszafordíthatóság miatt.

Definíció. A **pipeline** szakaszokból (stage) álló automatizált folyamat, amelyben egy szakasz kimenete a következő bemenete. Egy tipikus IoT-pipeline szakaszai: előkészítés, build, teszt, aláírás, SBOM, műtermék-kiadás, OTA-kiadás.

Definíció. A **reprodukálható build** azt jelenti, hogy ugyanabból a forrásból, ugyanazokkal a függőségekkel és eszközökkel bárki, bármikor ugyanazt a bináris vagy konténerkép műterméket állítja elő. Ez a biztonsági audit és a visszakövethetőség alapja.

Definíció. Az **SBOM (Software Bill of Materials, szoftver-összetevőjegyzék)** géppel olvasható lista a szoftver összes alkotóeleméről, függőségéről, verziójáról és licencéről. Az SBOM a sebezhetőség-ellenőrzés bemenete, és a Cyber Resilience Act követelményei között is szerepel.

Az alábbi ábra a TerraSense pipeline fő szakaszait és azok sorrendjét mutatja:



21.4. ábra: A TerraSense CI/CD pipeline szakaszai - A lépések a commit után futnak; a loop a sikertelen tesztek vagy OTA-visszajelzés miatti javítást jelöli.

A fenti folyamatábra lépései szöveggel is leírva:

1. **Commit:** a fejlesztő a `ts-node`, a `ts-gw` vagy a `ts-backend` kódot a közös tárolóba tolja.
2. **Firmware build + Twister teszt:** konténerben futtatott Zephyr-build és a `west twister` tesztfuttató a host-tesztekkel (lásd Eszközszoftver tesztelése a hoston).
3. **Backend build + JUnit teszt:** Maven build, unit és integrációs tesztek.
4. **Aláírás és SBOM:** a firmware-kép aláírása, komponensenkénti SBOM-generálás.
5. **Műtermék kiadása:** a firmware-aláírt kép feltöltése az artifact tárolóba, a backend Docker image feltöltése a registrybe.
6. **Fokozatos OTA indítása:** a kiadott firmware verzió megnyitása a kanári csoportnak, majd a visszajelzés alapján szélesebb körben.

21.3.3 Döntési szempontok

Kérdés	Változatok	Kompromisszum
Hol futtassuk a buildet?	Konténerben	Reprodukálható, de nagyobb erőforrásigény; a konténer verzióját is rögzíteni kell.
	Natív CI-gépen	Gyorsabb lehet, de a toolchain eltérése rejtett hibákat hoz be.
Milyen tesztet futtassunk a pipeline-ban?	Csak unit + host-teszt	Gyors, de nem fedi a valós hardver hibáit.
	HIL vagy hardveres célra is	Lassabb, drágább, de közelebb a valósághoz; TerraSense alapértelmezésben `native_sim`-et használ.
Mikor generáljunk SBOM-ot?	Minden buildnél	Naprakész, de növeli a futási időt és a tárolási költséget.
	Csak release-nél	Kisebb költség, de a köztes build sebezhetősége észrevétlen maradhat.
Hogyan kezeljük az aláírási kulcsot?	HSM-ben vagy biztonságos kulcstárolóban	Erős védelem, de költségesebb; laborban szoftveres kulcs elég, jelölten csak tesztcélra.
	A CI-ben fájlként	Egyszerű, de a kulcsszivárgás súlyos kockázat.
Fokozatos OTA kiadásnál?	Kanári → béta → teljes	Kellő visszajelzés és rollback lehetőség, de lassabb terítés.
	Azonnali teljes terítés	Gyors, de egy hibás kép az egész flottát leállíthatja.

A TerraSense-ben konténeres buildet, minden buildnél komponensenkénti SBOM-generálást, kiadási szakaszban aláírást és kanári-alapú fokozatos OTA-t használunk.

21.3.4 Kidolgozott példa

A példa egy **konkrét GitLab CI-konfiguráció**, nem szolgáltató-semleges sablon. A `.gitlab-ci.yml` a GitLab CI 17.x szintaxisát használja (`stages`, `needs`, `artifacts`, `environment`, `rules`), mert ezek a szemantikai elemek csak egy konkrét CI-rendszerben validálhatók. A futtatáshoz a következő runnerkörnyezet kell:

- GitLab 17.x és GitLab Runner 17.4 vagy újabb, Docker executorral;
- legalább 4 CPU, 8 GB memória és 20 GB szabad lemez a firmware-buildhez;
- Docker 27.x a runneren; a `backend-test` és a `publish-backend` Docker-in-Docker szolgáltatást használ, ezért ezek a jobok privileged runnert igényelnek, a többi job nem kap Docker socketet;
- a `registry.lab.internal` belső registry és a `ci/Dockerfile.*` fájlokból egyszer buildelt, rögzített címkéjű buildimage-ek;
- védett `v*` tag és védett `firmware-signing`, illetve `production-ota` environment a GitLab-projekt beállításaiban.

A pipeline a saját, mellékelt Dockerfile-okból származó image-eket használja, így a Zephyr SDK, a CMake, a Paho és a Maven verziója nem a runner véletlen állapotától függ. A termelési ágon a image-hivatkozásokat a `ci/images.lock` fájlban rögzített digestekre kell cserélni (`crane digest <image>`); a könyvben a olvashatóbb tag-alak szerepel.

1. A west manifest. A `west init -l .` csak akkor reprodukálható, ha a repo gyökerében van `west.yml`, amely a Zephyr 3.7.0 LTS-t és a szükséges modulokat rögzíti:

```yaml
# west.yml - TerraSense west manifest
manifest:
  remotes:
    - name: zephyrproject-rtos
      url-base: https://github.com/zephyrproject-rtos
  projects:
    - name: zephyr
      remote: zephyrproject-rtos
      revision: v3.7.0
      import:
        name-allowlist:
          - cmsis
          - hal_nordic
          - mbedtls
          - mcuboot
          - segger
          - tinycrypt
  self:
    path: .
```

A `name-allowlist` szándékosan szűk: csak a `nrf52840dk/nrf52840` boardhoz és az MCUboothoz szükséges modulokat húzza be. A `west update` által létrehozott `zephyr/`, `modules/`, `bootloader/` és `tools/` könyvtár a `.gitignore`-ba kerül, mert a CI minden jobban frissen állítja elő őket.

2. A buildimage-ek. A firmware-build image a Zephyr SDK 0.16.8-at használja, amely a Zephyr 3.7.0 LTS-hez tartozó SDK. A `ci/Dockerfile.firmware`:

```
FROM ubuntu:24.04

ARG ZEPHYR_SDK_VERSION=0.16.8
ENV DEBIAN_FRONTEND=noninteractive

RUN apt-get update \
 && apt-get install -y --no-install-recommends \
      build-essential ca-certificates ccache cmake device-tree-compiler \
      git gperf ninja-build python3 python3-pip python3-venv \
      wget xz-utils \
 && rm -rf /var/lib/apt/lists/*

RUN python3 -m venv /opt/venv \
 && /opt/venv/bin/pip install --no-cache-dir west==1.2.0
ENV PATH="/opt/venv/bin:${PATH}"

RUN SDK_FILE="zephyr-sdk-${ZEPHYR_SDK_VERSION}_linux-x86_64.tar.xz" \
 && SDK_BASE="https://github.com/zephyrproject-rtos/sdk-ng/releases/download" \
 && wget -q "${SDK_BASE}/v${ZEPHYR_SDK_VERSION}/${SDK_FILE}" \
 && wget -q "${SDK_BASE}/v${ZEPHYR_SDK_VERSION}/${SDK_FILE}.sha256" \
 && EXPECTED=$(cut -d' ' -f1 "${SDK_FILE}.sha256") \
 && ACTUAL=$(sha256sum "${SDK_FILE}" | cut -d' ' -f1) \
 && test "$EXPECTED" = "$ACTUAL" \
 && tar -xJf "${SDK_FILE}" -C /opt \
 && /opt/zephyr-sdk-${ZEPHYR_SDK_VERSION}/setup.sh -t arm-zephyr-eabi -h -c \
```

```
 && rm "${SDK_FILE}"*
ENV ZEPHYR_SDK_INSTALL_DIR=/opt/zephyr-sdk-${ZEPHYR_SDK_VERSION}
```

A `setup.sh -t arm-zephyr-eabi -h -c` csak az ARM eszközláncot és a hoszteszközöket telepíti; a `native_sim` Twister-tesztekhez a hoszt `gcc`-je kell, amelyet a `build-essential` ad. A gateway buildimage a CMake és a Paho C++ 1.3.2 függőséget garantálja (`ci/Dockerfile.gateway`):

```
FROM ubuntu:24.04
ENV DEBIAN_FRONTEND=noninteractive

RUN apt-get update \
 && apt-get install -y --no-install-recommends \
      build-essential ca-certificates cmake pkg-config libpaho-mqttpp3-dev \
 && rm -rf /var/lib/apt/lists/*

RUN pkg-config --modversion paho-mqttpp3 | grep -qx '1.3.2'
```

A `grep -qx '1.3.2'` szándékosan megbuktatja a buildet, ha az Ubuntu 24.04 csomagja nem a rögzített Paho-verziót adja, így a pipeline nem linkel csendben más API-t. Az imgtool saját, kisméretű image-et kap (`ci/Dockerfile.imgtool`), hogy a sign jobban ne legyen jelen a teljes Zephyr SDK:

```
FROM python:3.12.7-slim
RUN pip install --no-cache-dir imgtool==2.1.0
```

3. A teljes pipeline. A `.gitlab-ci.yml` a következő. A `firmware-build` a valódi `nrf52840dk/nrf52840` boardra épít MCUboot-kompatibilis alkalmazásképet, a `firmware-test` `native_sim` célon futtatja a Twister-teszteket. A kétféle build szándékosan külön job: a `native_sim` `zephyr.exe` hosztfuttatható program, nem MCUboot-kép, ezért nem nevezhető át `.bin`-re és nem írható alá firmware-képként.

```
# .gitlab-ci.yml - TerraSense CI/CD (GitLab CI 17.x, Docker executor)
stages: [build, test, sbom, sign, publish, release]

variables:
  ZEPHYR_BUILD_IMAGE: registry.lab.internal/terrasense/ci/zephyr-build:3.7.0-0.16.8
  GATEWAY_BUILD_IMAGE: registry.lab.internal/terrasense/ci/gateway-build:24.04-1.3.2
  MAVEN_IMAGE: maven:3.9.9-eclipse-temurin-21
  IMGTOOL_IMAGE: registry.lab.internal/terrasense/ci/imgtool:2.1.0
  SYFT_IMAGE: registry.lab.internal/terrasense/ci/syft:1.14.0
  GRYPE_IMAGE: registry.lab.internal/terrasense/ci/grype:0.86.0
  IMAGE_REGISTRY: registry.lab.internal
  GIT_DEPTH: "20"
  GIT_STRATEGY: fetch

.ci-rules:
  rules:
    - if: '$CI_PIPELINE_SOURCE == "merge_request_event"'
    - if: '$CI_COMMIT_TAG =~ /^v\d+\.\d+\.\d+$/'
    - if: '$CI_COMMIT_BRANCH == $CI_DEFAULT_BRANCH'

.release-rules:
  rules:
    - if: '$CI_COMMIT_TAG =~ /^v\d+\.\d+\.\d+$/'

firmware-test:
  stage: test
  image: $ZEPHYR_BUILD_IMAGE
  extends: .ci-rules
  script:
    - west init -l .
```

```
    - west update --narrow -o=--depth=1
    - west twister -p native_sim -T tests --inline-logs --outdir twister-out
  artifacts:
    when: always
    paths:
      - twister-out/twister.json
      - twister-out/**/handler.log
    expire_in: 1 week

firmware-build:
  stage: build
  image: $ZEPHYR_BUILD_IMAGE
  extends: .ci-rules
  script:
    - west init -l .
    - west update --narrow -o=--depth=1
    - west spdx --init -d build-release/spdx
    - west build -b nrf52840dk/nrf52840 ts-node --build-dir build-release
        -- -DCONFIG_BOOTLOADER_MCUBOOT=y
    - west spdx -d build-release/spdx
    - mkdir -p artifacts/sbom
    - cp build-release/zephyr/zephyr.bin artifacts/ts-node_${CI_COMMIT_SHORT_SHA}.bin
    - cp build-release/zephyr/zephyr.elf artifacts/ts-node_${CI_COMMIT_SHORT_SHA}.elf
    - cp build-release/partitions.yml artifacts/partitions.yml
    - sha256sum artifacts/ts-node_${CI_COMMIT_SHORT_SHA}.bin
        > artifacts/ts-node_${CI_COMMIT_SHORT_SHA}.bin.sha256
    - SPDX_JSON=$(find build-release/spdx -name 'zephyr.spdx.json' | head -n 1)
    - test -s "$SPDX_JSON"
    - cp "$SPDX_JSON" artifacts/sbom/ts-node.spdx.json
    - mkdir -p artifacts/sbom/firmware
    - find build-release/spdx -name '*.spdx.json'
        -exec cp {} artifacts/sbom/firmware/ \;
  artifacts:
    paths:
      - artifacts/
    expire_in: 1 week

gateway-build:
  stage: build
  image: $GATEWAY_BUILD_IMAGE
  extends: .ci-rules
  script:
    - cmake -S ts-gw -B ts-gw/build -DCMAKE_BUILD_TYPE=Release
    - cmake --build ts-gw/build --parallel
    - mkdir -p artifacts
    - cp ts-gw/build/ts-gw artifacts/ts-gw_${CI_COMMIT_SHORT_SHA}
    - sha256sum artifacts/ts-gw_${CI_COMMIT_SHORT_SHA}
        > artifacts/ts-gw_${CI_COMMIT_SHORT_SHA}.sha256
  artifacts:
    paths:
      - artifacts/
    expire_in: 1 week

backend-build:
  stage: build
  image: $MAVEN_IMAGE
  extends: .ci-rules
  script:
    - mvn -B -q -f ts-backend/pom.xml -DskipTests package
    - mvn -B -q -f ts-backend/pom.xml
        org.cyclonedx:cyclonedx-maven-plugin:2.8.1:makeAggregateBom
    - mkdir -p artifacts/sbom
    - cp ts-backend/target/ts-backend-0.1.0.jar artifacts/ts-backend.jar
```

```
    - cp ts-backend/target/bom.json artifacts/sbom/ts-backend.cdx.json
    - test -s artifacts/sbom/ts-backend.cdx.json
    - sha256sum artifacts/ts-backend.jar > artifacts/ts-backend.jar.sha256
  artifacts:
    paths:
      - artifacts/
    expire_in: 1 week

backend-test:
  stage: test
  image: $MAVEN_IMAGE
  extends: .ci-rules
  services:
    - name: docker:27.3.1-dind
      alias: docker
  variables:
    DOCKER_HOST: tcp://docker:2376
    DOCKER_TLS_CERTDIR: "/certs"
    DOCKER_TLS_VERIFY: "1"
    DOCKER_CERT_PATH: "/certs/client"
    TESTCONTAINERS_RYUK_DISABLED: "true"
  script:
    - cd ts-backend
    - mvn -B verify
  artifacts:
    when: always
    reports:
      junit: ts-backend/target/surefire-reports/TEST-*.xml

sbom-gateway:
  stage: sbom
  image:
    name: $SYFT_IMAGE
    entrypoint: [""]
  extends: .ci-rules
  needs: ["gateway-build"]
  script:
    - mkdir -p artifacts/sbom
    - syft "$GATEWAY_BUILD_IMAGE" -o spdx-json=artifacts/sbom/ts-gw-toolchain.spdx.json
    - test -s artifacts/sbom/ts-gw-toolchain.spdx.json
  artifacts:
    paths:
      - artifacts/sbom/ts-gw-toolchain.spdx.json
    expire_in: 1 week

sbom-gate:
  stage: sbom
  image:
    name: $GRYPE_IMAGE
    entrypoint: [""]
  extends: .ci-rules
  needs: ["firmware-build", "sbom-gateway", "backend-build"]
  script:
    - mkdir -p artifacts/security
    - status=0
    - >-
      grype "sbom:artifacts/sbom/ts-node.spdx.json" --fail-on high
      -o json > artifacts/security/ts-node-grype.json || status=1
    - >-
      grype "sbom:artifacts/sbom/ts-gw-toolchain.spdx.json" --fail-on high
      -o json > artifacts/security/ts-gw-grype.json || status=1
    - >-
      grype "sbom:artifacts/sbom/ts-backend.cdx.json" --fail-on high
```

```
      -o json > artifacts/security/ts-backend-grype.json || status=1
    - exit "$status"
  artifacts:
    when: always
    paths:
      - artifacts/security/
    expire_in: 1 week

firmware-sign:
  stage: sign
  image:
    name: $IMGTOOL_IMAGE
    entrypoint: [""]
  extends: .release-rules
  needs:
    - job: firmware-build
      artifacts: true
    - job: firmware-test
      artifacts: false
  environment:
    name: firmware-signing
  resource_group: firmware-signing
  script:
    - VERSION="${CI_COMMIT_TAG#v}"
    - test -n "${TS_SIGN_KEY:-}" && test -f "$TS_SIGN_KEY"
    - mkdir -p artifacts
    - sha256sum -c artifacts/ts-node_${CI_COMMIT_SHORT_SHA}.bin.sha256
    - >-
      imgtool sign
      --key "$TS_SIGN_KEY"
      --header-size 0x200
      --align 4
      --version "$VERSION"
      --slot-size 0x76000
      --pad-header
      artifacts/ts-node_${CI_COMMIT_SHORT_SHA}.bin
      artifacts/ts-node_${VERSION}.signed.bin
    - >-
      imgtool verify --key keys/ts-node-signing-pub.pem
      artifacts/ts-node_${VERSION}.signed.bin
    - imgtool dumpinfo artifacts/ts-node_${VERSION}.signed.bin
    - sha256sum artifacts/ts-node_${VERSION}.signed.bin
        > artifacts/ts-node_${VERSION}.signed.bin.sha256
  artifacts:
    paths:
      - artifacts/ts-node_${VERSION}.signed.bin
      - artifacts/ts-node_${VERSION}.signed.bin.sha256
      - artifacts/partitions.yml
    expire_in: 1 week

publish-firmware:
  stage: publish
  image:
    name: curlimages/curl:8.10.1
    entrypoint: [""]
  extends: .release-rules
  needs: ["firmware-sign"]
  script:
    - VERSION="${CI_COMMIT_TAG#v}"
    - test -s artifacts/ts-node_${VERSION}.signed.bin
    - >-
      curl --fail --silent --show-error
      --header "Authorization: Bearer $ARTIFACT_UPLOAD_TOKEN"
```

```
      --upload-file artifacts/ts-node_${VERSION}.signed.bin
      "$ARTIFACT_UPLOAD_URL/ts-node/$VERSION/"
  environment:
    name: artifact-registry

publish-backend:
  stage: publish
  image: docker:27.3.1-cli
  extends: .release-rules
  needs: ["backend-build", "backend-test", "sbom-gate"]
  services:
    - name: docker:27.3.1-dind
      alias: docker
  variables:
    DOCKER_HOST: tcp://docker:2376
    DOCKER_TLS_CERTDIR: "/certs"
    DOCKER_TLS_VERIFY: "1"
    DOCKER_CERT_PATH: "/certs/client"
  environment:
    name: container-registry
  script:
    - echo "$CI_REGISTRY_PASSWORD" | docker login -u "$CI_REGISTRY_USER"
        --password-stdin "$CI_REGISTRY"
    - IMAGE="$CI_REGISTRY_IMAGE/ts-backend:$CI_COMMIT_TAG"
    - docker build -t "$IMAGE" ts-backend
    - docker push "$IMAGE"
    - mkdir -p artifacts
    - docker inspect --format='{{index .RepoDigests 0}}' "$IMAGE"
        | tee artifacts/ts-backend-image.txt
  artifacts:
    paths:
      - artifacts/ts-backend-image.txt
    expire_in: 1 week

sbom-backend-image:
  stage: publish
  image:
    name: $SYFT_IMAGE
    entrypoint: [""]
  extends: .release-rules
  needs: ["publish-backend"]
  variables:
    SYFT_REGISTRY_AUTH_AUTHORITY: $CI_REGISTRY
    SYFT_REGISTRY_AUTH_USERNAME: $CI_REGISTRY_USER
    SYFT_REGISTRY_AUTH_PASSWORD: $CI_REGISTRY_PASSWORD
  script:
    - IMAGE_DIGEST=$(cat artifacts/ts-backend-image.txt)
    - mkdir -p artifacts/sbom
    - syft "$IMAGE_DIGEST" -o spdx-json=artifacts/sbom/ts-backend-image.spdx.json
    - test -s artifacts/sbom/ts-backend-image.spdx.json
  artifacts:
    paths:
      - artifacts/sbom/ts-backend-image.spdx.json
    expire_in: 1 week

release-manifest:
  stage: release
  image: python:3.12.7-slim
  extends: .release-rules
  needs:
    - job: firmware-build
      artifacts: true
    - job: gateway-build
```

```
      artifacts: true
    - job: backend-build
      artifacts: true
    - job: sbom-gateway
      artifacts: true
    - job: sbom-gate
      artifacts: true
    - job: firmware-sign
      artifacts: true
    - job: publish-backend
      artifacts: true
    - job: sbom-backend-image
      artifacts: true
  script:
    - python3 ci/release_manifest.py --output artifacts/release-manifest.json
  artifacts:
    paths:
      - artifacts/release-manifest.json
      - artifacts/SHA256SUMS
    expire_in: 4 weeks

ota-release:
  stage: release
  image:
    name: curlimages/curl:8.10.1
    entrypoint: [""]
  extends: .release-rules
  needs: ["release-manifest", "publish-firmware"]
  environment:
    name: production-ota
    url: https://ota.lab.internal
  resource_group: production-ota
  when: manual
  allow_failure: false
  script:
    - VERSION="${CI_COMMIT_TAG#v}"
    - MANIFEST_SHA=$(sha256sum artifacts/release-manifest.json | cut -d' ' -f1)
    - BODY="{\"version\":\"$VERSION\",\"ring\":\"canary\","
    - BODY="$BODY\"manifest_sha256\":\"$MANIFEST_SHA\"}"
    - >-
      curl --fail --silent --show-error
      --header "Authorization: Bearer $OTA_CONTROL_TOKEN"
      --header "Content-Type: application/json"
      --data "$BODY"
      "$OTA_CONTROL_URL/release"
```

4. A kiadási manifest. A `ci/release_manifest.py` a `publish` és `release` szakaszokban összegyűlt műtermékeket hash-eli, és a release-azonosítóval együtt egyetlen JSON-fájlba írja. A fájl a kiadás visszakövethetőségének alapja; az `ota-release` ezt a fájlt is hitelesíti:

```
#!/usr/bin/env python3
import argparse
import hashlib
import json
import os
from datetime import datetime, timezone
from pathlib import Path


def sha256(path):
    digest = hashlib.sha256()
    with path.open("rb") as handle:
        for chunk in iter(lambda: handle.read(1024 * 1024), b""):
```

```
            digest.update(chunk)
    return digest.hexdigest()


def main():
    parser = argparse.ArgumentParser()
    parser.add_argument("--output", required=True)
    parser.add_argument("--artifacts", default="artifacts")
    args = parser.parse_args()

    root = Path(args.artifacts)
    output = Path(args.output)
    files = sorted(
        path for path in root.rglob("*")
        if path.is_file()
        and path.resolve() != output.resolve()
        and path.name != "SHA256SUMS"
    )
    artifacts = [
        {
            "path": str(path.relative_to(root)),
            "size": path.stat().st_size,
            "sha256": sha256(path),
        }
        for path in files
    ]
    manifest = {
        "release_id": f"{os.environ['CI_PIPELINE_ID']}-{os.environ['CI_COMMIT_SHORT_SHA']}",
        "version": os.environ["CI_COMMIT_TAG"].removeprefix("v"),
        "commit": os.environ["CI_COMMIT_SHA"],
        "pipeline_id": os.environ["CI_PIPELINE_ID"],
        "created_at": datetime.now(timezone.utc).isoformat(),
        "artifacts": artifacts,
    }
    output.parent.mkdir(parents=True, exist_ok=True)
    output.write_text(json.dumps(manifest, indent=2, ensure_ascii=False) + "\n",
                      encoding="utf-8")
    sums = root / "SHA256SUMS"
    sums.write_text(
        "".join(f"{item['sha256']}  {item['path']}\n" for item in artifacts),
        encoding="utf-8")
    missing = [item for item in artifacts if item["size"] == 0]
    if missing:
        raise SystemExit(f"Üres műtermék: {missing[0]['path']}")


if __name__ == "__main__":
    main()
```

A manifest minden fájlt felsorol, amely a `artifacts/` alatt található: a firmware binárist és ELF-et, a gateway binárist, a backend JAR-t, a három komponens SBOM-ját, a backend image digestjét, valamint a Grype-jelentéseket. A `SHA256SUMS` ugyanezeket a hash-eket adja shell-barát formában. Az `ota-release` a manifest SHA-256-ját küldi az OTA-vezérlőnek, így a kiadott verzió és a hozzá tartozó műtermékjegyzék együtt hitelesíthető.

5. A jobok magyarázata.

- A `firmware-test` a `native_sim` platformon futó Twister-teszteket futtatja. A `-p native_sim` a Twister platformkapcsolója; a korábbi `-b` a `west build` boardkapcsolója volt, és Twister alatt nem működik.
- A `firmware-build` a valódi `nrf52840dk/nrf52840` boardra épít. A `CONFIG_BOOTLOADER_MCUBOOT=y` az alkalmazásképet a MCUboot által várt elrendezésre igazítja; az MCUboot bootloader

maga külön image, amelyet a gyártás során égetnek a boardra, a CI csak az alkalmazás slot képét írja alá. A `build-release/zephyr/zephyr.bin` a nyers alkalmazáskép, nem a `native_sim` `zephyr.exe`.

- A `firmware-build` a `partitions.yml` fájlt is artifactként menti, hogy a sign jobban látható legyen a `slot0_partition` mérete. Az `imgtool sign --slot-size 0x76000` és `--header-size 0x200` értéke a `nrf52840dk/nrf52840` alapértelmezett MCUboot-partíciókiosztásából származik; a laborban a `partitions.yml` értékével kell összevetni, és eltérés esetén a pipeline-t kell javítani, nem a fájlt kézzel átnevezni.
- A `firmware-sign` közvetlenül `imgtool sign` parancsot futtat a letöltött `artifacts/ts-node_${CI_COMMIT_SHORT_SHA}.bin` fájlon, nem feltételezi, hogy a `west build` könyvtára a jobban elérhető. A `mkdir -p artifacts` és a pontos input/output útvonal biztosítja, hogy friss, izolált konténerben is létrejöjjön a deklarált `ts-node_${VERSION}.signed.bin`. A `sha256sum -c` a build job által rögzített hash-hez köti az aláírandó binárist, az `imgtool verify` pedig a nyilvános kulccsal ellenőrzi az eredményt.
- A `backend-build` a Maven `-f ts-backend/pom.xml` hívásával dolgozik, ezért nincs `cd` és nincs dupla `ts-backend/target` útvonal. A CycloneDX plugin verziója rögzített (`2.8.1`), a `bom.json` a `ts-backend/target/` alatt keletkezik, és onnan kerül az `artifacts/sbom/ts-backend.cdx.json` fájlba; a `test -s` ellenőrzi, hogy nem üres.
- A `backend-test` a Testcontainerses integrációs tesztet Docker-in-Docker mellett futtatja. A `TESTCONTAINERS_RYUK_DISABLED=true` a dind környezetben elkerüli a Ryuk konténer jogosultsági problémáit; a teszt maga a `IngestIntegrationTest`, amelyet a Tesztelési szintek alfejezetben készítettünk el.
- A `sbom-gateway` a gateway buildimage-et szkenneli Syft-tel, mert a C++ binárisból a linkelt rendszercsomagok nem derülnek ki megbízhatóan; a bináris hash-e a `gateway-build` artifactjában van, így a két információ együtt követhető. A `sbom-gate` Grype-vel mindhárom SBOM-on `high` súlyosságtól megbukik, és a jelentéseket akkor is megőrzi, ha a kapu hibára fut.
- A `firmware-sign` és az `ota-release` csak `v`-vel kezdődő, védett tagre fut. A `TS_SIGN_KEY`, az `OTA_CONTROL_TOKEN` és az `ARTIFACT_UPLOAD_TOKEN` védett, maszkolt CI-változó; a `TS_SIGN_KEY` típusa File, ezért a értéke egy útvonal. A `firmware-signing` és `production-ota` environment a GitLabban védett, csak a maintainer szerepkör érheti el. A merge request pipeline-jából a sign, publish és release jobok a `rules` miatt nem is jönnek létre, így nem megbízható PR-kód nem kap kulcsot.
- A `publish-backend` a staging image-et a tag alapján tölti fel, majd a digestet artifactként menti. A `sbom-backend-image` ezt a digestet szkenneli, így a release manifest a ténylegesen publikált image-hez kapcsolódik, nem egy újraépített, eltérő rétegekhez.
- A `release-manifest` minden szükséges artifactot `needs`-szel húz be; a GitLab a `needs` gráf szerint csak akkor indítja, ha a build, a teszt, az SBOM-kapu és az aláírás is sikeres. A `ota-release` manuális, de `allow_failure: false`, ezért a kiadás nem tekinthető sikeresnek, amíg valaki jóvá nem hagyja.

6. A pipeline statikus ellenőrzése. A példát a lektorálási környezetben nem futtattuk GitLabon, nem volt runner, registry és valódi `nrf52840dk` board. A YAML szintaktikailag elemezhető, a GitLab CI Lint a könyv példájának saját GitLab-példányon történő bemásolásakor validálja a `rules`, `needs` és `artifacts` szemantikáját. A záróprojekt csak a laborban ténylegesen lefutott pipeline-t tekinti futtathatónak; a könyv példája ettől még nem bizonyítottan működő CI-rendszer.

21.3.5 Ellenőrzés és hibakeresés

A pipeline futását a GitLab felületén, a jobok naplóiban és a kiadott műtermékek ellenőrzésével tudjuk követni. A következő táblázat a tipikus hibákat és azok okait foglalja össze.

Tünet	Lehetséges ok	Javítás
`west update` nem találja a manifestet vagy a függőségeket	Hiányzik a `west init -l .`, vagy a `west.yml` nem a repo gyökerében van	Ellenőrizzük a `west.yml` helyét és a `revision: v3.7.0` értéket; a `west init -l .` a manifest repositoryból indul
A Twister teszt lokálisan fut, de a CI-ben elhasal	Eltérő buildimage, hiányzó hoszt `gcc`, rossz platformkapcsoló	Ugyanazt a `ZEPHYR_BUILD_IMAGE` image-et használjuk, és `west twister -p native_sim` parancsot futtassunk
A firmware aláírása hiányzó kulcsra hibát ad	A `TS_SIGN_KEY` File-változó nincs beállítva, vagy a job nem védett tagre fut	A változót védett, maszkolt File-típusúként kell felvenni, és a sign jobot védett tagre korlátozni
Az `imgtool sign` „header size mismatch” hibát ad	A `--header-size` vagy a `--slot-size` nem egyezik a board partíciós kiosztásával	A `artifacts/partitions.yml` `slot0_partition` méretével kell összevetni; a `nrf52840dk/nrf52840` esetén a jelenlegi érték `0x76000`, de ezt a laborban ellenőrizni kell
Az SBOM nem keletkezik	A Maven-plugin verziója nem rögzített, vagy a `bom.json` útvonala rossz	A `cyclonedx-maven-plugin:2.8.1` hívást és a `ts-backend/target/bom.json` relatív útvonalat használjuk; a `test -s` bukjon meg, ha a fájl üres
A Grype-kapu nem találja az SBOM-ot	A `needs` nem húzta be a szükséges artifactot, vagy a fájlnév eltér	Ellenőrizzük a `needs` listát és a `sbom:` előtagot; a Grype a `sbom:<path>` formát várja
A Docker push meghiúsul	A registry nem elérhető a dind szolgáltatásból, vagy nincs jogosultság	Ellenőrizzük a `docker login` bemenetét, a `CI_REGISTRY_*` változókat és a runner dind beállítását
Az OTA-release nem indul el	A `when: manual` lépést nem hagyta jóvá senki, vagy a `production-ota` environment nem védett	A manuális jóváhagyás megerősítése, a védett environment és a `OTA_CONTROL_TOKEN` ellenőrzése

Tünet	Lehetséges ok	Javítás
A release-manifest hiányos	Egy `needs` job kimaradt, vagy a `sbom-backend-image` nem kapott digestet	A `needs` gráf és a `artifacts/ts-backend-image.txt` tartalmának ellenőrzése; a manifest a hiányzó fájlt üres mérettel jelzi

21.3.6 Tipikus hibák és félreértések

1. **Reprodukálhatatlan build:** ha a konténer image címkéjét `latest`-ként használjuk, a build időben eltérő alaprendszert kaphat. Mindig rögzített címkét vagy digestet, és a forrásban `west.yml`-t, illetve Maven `pom.xml`-t használjunk. A projekt saját Dockerfile-jai és a digestre váltott image-hivatkozások együtt adják a reprodukálhatóságot.
2. **Aláírási kulcs a repóban:** a privát kulcsot sokan a forráskód mellett tárolják, mert „úgyis csak labor". Ez a szokás élesben azonnali kompromittálódáshoz vezet. A kulcs csak a CI titkos tárolójában, HSM-ben vagy hardware security module-ban létezhet. A laborban is jelöljük külön a tesztkulcsot, és ne terjedjen ki a kiadási pipeline-ra.
3. **SBOM mint utólagos papírmunka:** ha az SBOM-ot csak a végén, kézzel készítjük, az elavult lesz, és a sebezhetőség-ellenőrzés téves eredményt ad. Az SBOM-generálást a build részeként, komponensenként kell futtatni, és a műtermékhez csatolni.
4. **Fokozatos OTA rossz kezelése:** az azonnali, 100%-os terítés nélkülözi a visszagörgetés lehetőségét, és egy rossz firmware az összes eszközt térdre kényszerítheti. A kanári gyűrűn mért állapot és a rollback-automatizmus (lásd OTA-frissítés: A/B képek, rollback, delta, fokozatos kiadás) nélkül ne adjunk ki új verziót.
5. **A `native_sim` kimenetét firmware-képnek tekinteni.** A `native_sim` `zephyr.exe` hosztfuttatható ELF, nem MCUboot alkalmazáskép. A `.bin` kiterjesztésre átnevezés nem változtat a formátumon, és a céleszköz bootloadere nem tudja elindítani. A kiadási firmware-t valódi boardra (`nrf52840dk/nrf52840`) kell buildelni, a `zephyr.bin` fájlt kell aláírni, és a szimulált flotta csak a tesztelést szolgálja.
6. **A „szolgáltató-semleges" YAML illúziója.** A `stages`, `needs`, `artifacts` és `environment` kulcsok nem általános pipeline-fogalmak, hanem egy konkrét CI-rendszer szemantikája. Ha a könyv szolgáltató-semleges példát ígér, akkor a leírás nem futtatható; ha futtatható példát ígér, akkor meg kell nevezni a CI-rendszert, a runnert és a szükséges beállításokat. Ez a fejezet az utóbbit választja.

21.3.7 Feladatok

1. **Fogalmi kérdés.** Miért fontos az SBOM az IoT-flotta biztonságában? Nevezzen meg legalább két fogyasztót az SBOM-nak, és magyarázza el, hogyan segíti a sebezhetőség-kezelést.
2. **Tervezési feladat.** Tervezzen CI/CD pipeline-szakaszokat a TerraSense `ts-node` firmware-re, a `ts-gw` gateway-re és a `ts-backend` szolgáltatásra. Határozza meg az egyes szakaszok bemeneteit, kimeneteit és a közöttük lévő függőségeket. Milyen műtermékek keletkeznek, és hol tárolják őket? Térjen ki arra, melyik műtermékhez milyen SBOM és ellenőrzési kapu tartozik.

3. **Implementációs feladat.** Egészítse ki a kidolgozott példában szereplő `.gitlab-ci.yml` állományt egy `gateway-build` szakasszal, amely a `ts-gw` C++ programot a projekt saját `ci/Dockerfile.gateway` image-ében buildeli (CMake, `make`), és az elkészült binárist, valamint a SHA-256 hash-ét artifactként tárolja. A szakasz a `build` szakaszfázisban fusson, és ne legyen függősége a firmware- vagy backend-feladatoktól.

4. **Hibakeresési feladat.** A pipeline-ban a `firmware-test` szakasz mindig elhasal, miközben a `firmware-build` sikerrel lefut. A naplóban a következő látható: `west twister: command not found`. Mi lehet az oka, és hogyan javítaná? A megoldás során ne csak a tünetet, hanem a valószínű okot is indokolja.

21.3.8 Megoldások

1. **Fogalmi kérdés.** Az SBOM fogyasztói közé tartozik a szoftverfejlesztő csapat (függőségkezelés), a biztonsági auditor (licencek és sebezhetőségek keresése), a vásárló vagy üzemeltető (transzparencia, bejelentési kötelezettség) és a sebezhetőség-szkenner (pl. OWASP Dependency-Check, Trivy, Grype). Az SBOM lehetővé teszi, hogy egy ismert sérülékenység (CVE) megjelenésekor azonnal meghatározzuk, mely komponensek és verziók érintettek, és így a javított verzióra való frissítést célzottan tervezzük meg.

2. **Tervezési feladat.** A pipeline szakaszai és függőségei: `firmware-build` (bemenet: forrás, `west.yml`; kimenet: `ts-node_<sha>.bin`, `.elf`, `partitions.yml`, firmware SPDX SBOM), `firmware-test` (bemenet: forrás és `west.yml`; kimenet: Twister-jelentés), `gateway-build` (bemenet: `ts-gw` forrás, `ci/Dockerfile.gateway`; kimenet: `ts-gw_<sha>` bináris és hash), `backend-build` (bemenet: `ts-backend` forrás, `pom.xml`; kimenet: JAR, CycloneDX SBOM), `backend-test` (bemenet: forrás; kimenet: JUnit-jelentés), `sbom-gateway` (bemenet: gateway buildimage; kimenet: SPDX SBOM), `sbom-gate` (bemenet: mindhárom SBOM; kimenet: Grype-jelentés és kapu), `firmware-sign` (bemenet: firmware bináris, védett kulcs; kimenet: aláírt `.bin` és hash), `publish-backend` (bemenet: JAR, SBOM; kimenet: image digest), `release-manifest` (bemenet: minden artifact; kimenet: `release-manifest.json`, `SHA256SUMS`), `ota-release` (bemenet: manifest és aláírt firmware; kimenet: OTA-kiadás). A műtermékeket dedikált artifact-tárolóban, a Docker image-eket registryben tárolják; minden artifacthoz hash és SBOM tartozik.

3. **Implementációs feladat.** A `ts-gw` buildjéhez a következő szakasz adható a pipeline-hoz. A szakasz a projekt saját, rögzített buildimage-ét használja, a CMake-et a `ts-gw` könyvtárra futtatja, és a binárist a tényleges `ts-gw/build/ts-gw` útvonalról másolja:

```
gateway-build:
  stage: build
  image: $GATEWAY_BUILD_IMAGE
  extends: .ci-rules
  script:
    - cmake -S ts-gw -B ts-gw/build -DCMAKE_BUILD_TYPE=Release
    - cmake --build ts-gw/build --parallel
    - mkdir -p artifacts
    - cp ts-gw/build/ts-gw artifacts/ts-gw_${CI_COMMIT_SHORT_SHA}
    - sha256sum artifacts/ts-gw_${CI_COMMIT_SHORT_SHA}
        > artifacts/ts-gw_${CI_COMMIT_SHORT_SHA}.sha256
  artifacts:
    paths:
      - artifacts/
    expire_in: 1 week
```

A szakasz a `build` fázisban fut, és nincs szüksége a firmware- vagy backend-feladatok kimenetére, ezért független. A konténer image a `ci/Dockerfile.gateway` fájlból készül, amely a CMake-en

kívül a Paho C++ 1.3.2 fejlesztői csomagját is tartalmazza, és a `pkg-config --modversion paho-mqttpp3` ellenőrzéssel megbukik, ha a verzió eltér.

4. **Hibakeresési feladat.** A hiba oka, hogy a `firmware-test` szakaszban olyan konténer image-et használunk, amely nem tartalmazza a `west` parancsot, vagy a `west` nem szerepel a `PATH`-ban. A `firmware-build` szakasz sikerül, mert ott a Zephyr SDK-t tartalmazó image fut, de a teszt-szakasz esetleg más, általános image-et használ. Javítás: a `firmware-test` szakaszban is ugyanazt a `ZEPHYR_BUILD_IMAGE` image-et használjuk, a `west init -l .` és `west update` lépésekkel inicializáljuk a workspace-t, és a Twistert a helyes `west twister -p native_sim` formában hívjuk. A `ZEPHYR_SDK_INSTALL_DIR` beállítása önmagában nem telepíti a `west` csomagot; a `west` a Python virtuális környezetből kerül a `PATH`-ra.

21.3.9 Rövid összefoglaló

Az alfejezetben a TerraSense rendszer CI/CD folyamatát terveztük meg egy konkrét GitLab CI-konfigurációban. A pipeline szakaszai: reprodukálható build a projekt saját Dockerfile-jaiból származó image-ekkel, firmware Twister-teszt `native_sim` platformon, valódi `nrf52840dk/nrf52840` build MCUboot-kompatibilis alkalmazásképpel, gateway CMake-build, backend Maven-teszt, komponensenkénti SBOM (SPDX a firmware-hez és a gateway toolchainhez, CycloneDX a backendhez), Grype-alapú sebezhetőségi kapu, `imgtool`-alapú firmware-aláírás védett tag és environment mellett, majd a műtermékek kiadása és a fokozatos OTA indítása kanári gyűrűvel. A `.gitlab-ci.yml`, a `west.yml`, a `ci/Dockerfile.firmware`, a `ci/Dockerfile.gateway`, a `ci/Dockerfile.imgtool` és a `ci/release_manifest.py` együtt adja a példát. A pipeline-t a lektorálási környezetben nem futtattuk; a záróprojekt Záróprojekt: a teljes TerraSense rendszer összeállítása és átvétele csak a laborban ténylegesen lefutott és ellenőrzött pipeline-t tekinti kiadásra alkalmasnak.

21.3.10 Források és ellenőrzési hiányok

- GitLab CI szintaxis és runnerkövetelmények: a `stages`, `needs`, `artifacts`, `environment`, `resource_group` és `rules` szemantikája a GitLab 17.x dokumentációjában ellenőrizendő. A példát GitLab CI Lint és valódi runner nélkül nem validáltuk; a laborban a saját GitLab-példányon kell lefuttatni.
- Zephyr 3.7.0 LTS, Zephyr SDK 0.16.8, `west` 1.2.0, `west twister`, `west spdx`, valamint a `nrf52840dk/nrf52840` MCUboot-partíciókiosztása (`slot0_partition` mérete, `MCUBOOT_HEADER_SIZE`) a Zephyr és az MCUboot dokumentációjában ellenőrizendő. A `0x76000` slotméret és a `0x200` header méret a board alapértelmezett kiosztásából származó érték; a laborban a `partitions.yml` az irányadó.
- Az `imgtool` 2.1.0 parancssori kapcsolói (`sign`, `verify`, `dumpinfo`, `--pad-header`, `--slot-size`, `--header-size`) az MCUboot dokumentációjában ellenőrizendők. A példát valódi `nrf52840dk` boardon nem futtattuk, ezért a céleszköz bootloadere általi elfogadás nem bizonyított.
- A CycloneDX Maven plugin 2.8.1, a Syft 1.14.0 és a Grype 0.86.0 verziója és parancssori kapcsolói a projekt dokumentációjában ellenőrizendők; a pipeline a digestre váltott image-eket feltételezi, a könyvben a tag-alak szerepel.
- A `west spdx` kimeneti fájlneve és a `--init`/`-d` kapcsolók Zephyr-verziófüggők; a példa a `find ... -name '*.spdx.json'` hívással teszi a fájlnevet nem kritikussá, de a formátumot (SPDX 2.3 JSON) a laborban ellenőrizni kell.
- A `CI_REGISTRY_*`, `TS_SIGN_KEY`, `OTA_CONTROL_TOKEN`, `ARTIFACT_UPLOAD_TOKEN` és a registry/artifact URL-ek a példában laborcélú helyettesítők, nem valós hitelesítési adatok.

A védett tag, a védett environment és a védett változók beállítása a GitLab-projekt beállításaiban történik, nem a YAML-ban; a YAML önmagában nem hozzáférési policy.

VII. RÉSZ – Platformok, záróprojekt és összefoglalás

22 IoT-platformok

22.1 Felhős és nyílt forrású IoT-platformok: mikor építs, mikor vásárolj

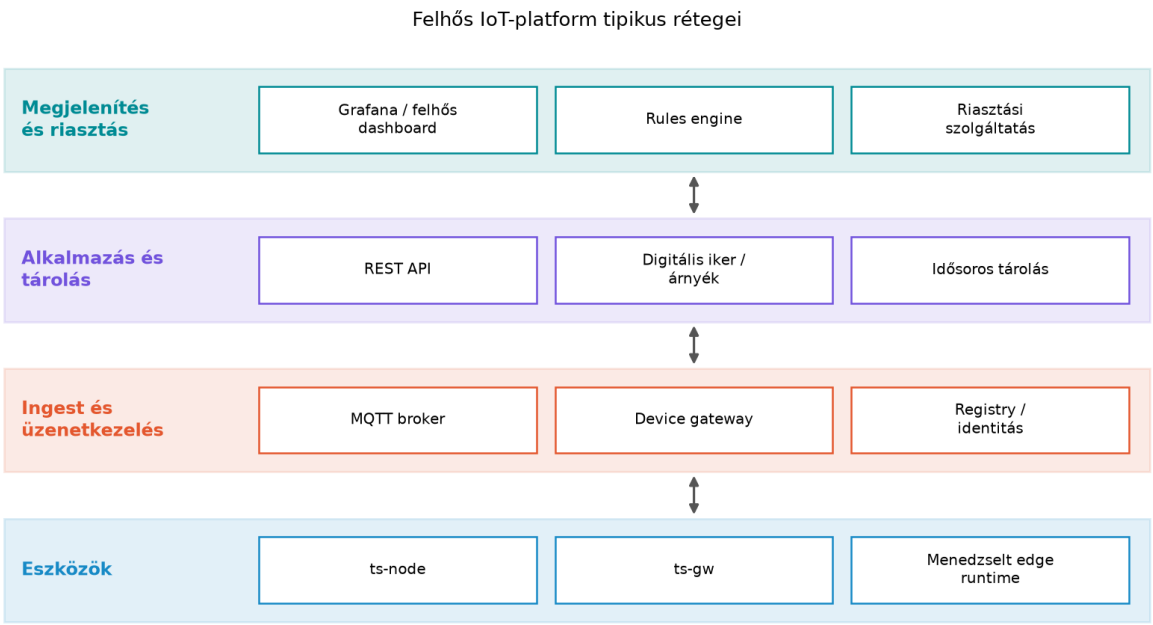
22.1.1 Motiváció

A könyv eddigi laborjaiban a TerraSense rendszert saját komponensekből építettük fel: `ts-node` a Zephyr-alapú szenzorcsomópont, `ts-gw` a C++-os gateway, `ts-backend` a Spring Boot szolgáltatás, Mosquitto a broker, TimescaleDB a tárolás, Grafana a megjelenítés. Ez a rétegzett architektúra jól működik akkor, ha a csapatnak van low-level, backend- és üzemeltetési kapacitása, és ha a cél a teljes kontroll, a testreszabhatóság vagy egy tanulási folyamat.

A valós ipari döntés azonban gyakran nem az, hogy *tudunk-e* ilyen rendszert építeni, hanem az, hogy *érdemes-e* fenntartani. Egy termék méretnövekedésével a flottakezelés, az OTA, a tanúsítvány-kezelés, a biztonsági monitorozás és a megfelelőség (például ETSI EN 303 645, EU Cyber Resilience Act) olyan költségeket generál, amelyeket egy platformszolgáltató méretgazdaságosan tud biztosítani. Ebben az alfejezetben a TerraSense saját komponenseit megfeleltetjük a felhős és nyílt forrású platformok tipikus szolgáltatásainak, majd egy döntési mátrixszal és költségbecsléssel segítjük a build vs. buy választást.

22.1.2 Felhős platformok tipikus komponensei

A nagy felhőszolgáltatók (például az AWS IoT Core, az Azure IoT Hub vagy a Google Cloud IoT) a TerraSense-hez hasonló architektúrákat kínálnak menedzselt szolgáltatásként. A tipikus komponensek az alábbiak; a konkrét szolgáltatásnevek gyorsan változhatnak, ezért minden platform esetén ellenőrizendők a dokumentációban.



22.1. ábra: Felhős IoT-platform tipikus rétegei - A TerraSense saját komponenseinek leképezése felhős szolgáltatásokra; a szolgáltatásnevek ellenőrzési igénnyel szerepelnek.

A fenti ábra azt mutatja, hogy a felhős platformok ugyanazokat a rétegeket fedik le, mint a TerraSense saját összeállítása:

- **Device gateway:** az MQTT-végpontot, a kapcsolatfelvételt, a QoS-kezelést és a protokollfordítást biztosítja. Ez megfelel a mi Mosquitto brokerünknek, gyakran beépített autentikációval, X.509-tanúsítvány-alapú azonosítással és mTLS-védelemmel.
- **Registry:** az eszközazonosítók, hitelesítő adatok, csoportok és metaadatok nyilvántartása. Ez a TerraSense `devices` táblájának és a flottakezelésnek a felhős megfelelője.
- **Shadow / digitális iker:** a kívánt (desired) és a jelentett (reported) állapotot tárolja, és a frissítést az eszközzel szinkronizálja. Ez megegyezik a `device_config` JSONB árnyék mezőivel, amelyet a Flottakezelés: nyilvántartás, konfiguráció, állapot alfejezetben vezettünk be.
- **Rules engine:** szabályok alapján továbbítja a bejövő telemetriát, riasztást vált ki, vagy más szolgáltatásba (például streamfeldolgozóba) továbbítja az adatokat. Ez a TerraSense riasztási szabályainak felhős megfelelője.
- **Idősoros tárolás és lekérdezés:** menedzselt adatbázis, gyakran TimescaleDB-alapú vagy dedikált idősoros motor. Ez a mi PostgreSQL + TimescaleDB kombinációnk.
- **OTA és eszközmenedzsment:** firmware-frissítés, A/B képek, rollback, csoportos kiadás. A TerraSense esetén ezt eddig saját kóddal és konvenciókkal valósítottuk meg, a Biztonságos boot és firmware-aláírás és az OTA-frissítés: A/B képek, rollback, delta, fokozatos kiadás alfejezetekben.
- **Edge runtime:** ha a feldolgozás egy része a helyszínen fut, a platform rendszerint saját gateway- vagy edge-futtatókörnyezetet kínál. Ez a `ts-gw` szerepét veheti át, sok esetben konténeralapú.

Definíció. *Menedzselt IoT-platform* alatt olyan felhőszolgáltatást értünk, amely az eszközök csatlakoztatását, hitelesítését, üzenetkezelését, állapotkezelését és gyakran a feldolgozást is

szolgáltatásként biztosítja. A fogyasztó általában az eszközök és az alkalmazáslogika szintjén dolgozik, az infrastruktúra üzemeltetését a szolgáltató vállalja.

A felhős platformok előnye a gyors indulás, a skálázhatóság és az, hogy a biztonsági alapszolgáltatásokat (például mTLS-végpont, tanúsítvány-visszavonás, hitelesítő adat-generálás) a szolgáltató karbantartja. A hátrány a szolgáltatói kötődés és a költség, amely nagy üzenetszámnál jelentős lehet.

22.1.3 Nyílt forrású platformok

A nyílt forrású alternatívák a felhős platformokhoz hasonló funkcionalitást kínálnak, de a kód és az üzemeltetés a saját infrastruktúrába kerül. A leggyakrabban említett rendszerek:

- **ThingsBoard:** webes felülettel, eszközregisztrációval, MQTT/CoAP/HTTP ingesttel, szabálymotorral és dashboardokkal rendelkező platform. Jól használható akkor, ha a TerraSense-hez hasonló rendszert szeretnénk, de a backend fejlesztését csökkenteni akarjuk.
- **Eclipse Hono / Eclipse Ditto:** a Hono az eszközök és a protokollok absztrakcióját biztosítja, a Ditto a digitális iker / árnyék képességet adja hozzá. A TerraSense `ts-backend` + TimescaleDB kombinációjának nyílt forrású felbontása lehet.
- **Mosquitto + saját backend:** ez gyakorlatilag a TerraSense jelenlegi felépítése. A Mosquitto biztosítja az MQTT-t, a saját `ts-backend` az ingestet és a REST API-t, a TimescaleDB a tárolást. A flottakezelést és az OTA-t pedig az Eszközmenedzsment: provisioning, OTA, flotta és a Flottakezelés: nyilvántartás, konfiguráció, állapot alfejezetekben leírt mintákra építjük.

A nyílt forrású platformok közös jellemzője, hogy a kód elérhető és auditálható, a rendszer a saját Kubernetes- vagy Docker-infrastruktúránkon futhat, és nincs külső felhőszolgáltatóhoz kötve. Ugyanakkor az üzemeltetés, a méretezés és a biztonsági frissítések a saját csapat terhét növelik, a választott platform API-ja, adatmodellje és a hozzá értő szakemberek pedig technológiai és szervezeti kötődést hoznak létre.

22.1.4 Döntési szempontok: build vs. buy

A következő táblázat a saját építésű rendszer (TerraSense), a nyílt forrású platform és a menedzselt felhős platform fő szempontjait foglalja össze.

Szempont	Saját építés (TerraSense)	Nyílt forrású (ThingsBoard, Hono/Ditto)	Menedzselt felhős
Induló költség	alacsony, de nagy mérnöki ráfordítás	közepes (telepítés, konfigurálás)	alacsony (konzol, SDK)
Működési költség	alacsony – saját infra	közepes – saját infra üzemeltetése	magas – használatarányos, de kiszámítható
Kontroll és testreszabás	teljes	jó – a kód módosítható	korlátozott – a szolgáltatás adottságai

Szempont	Saját építés (TerraSense)	Nyílt forrású (ThingsBoard, Hono/Ditto)	Menedzselt felhős
Szolgáltatói, technológiai és adatkötődés	külső platformszolgáltatóhoz nem köt, de hoszting-, domain- és hardverbeszállítók itt is vannak; a saját séma, a TimescaleDB-kiterjesztés, az eszköz-SDK és a csapattudás köt	nincs licenckötés, de a platform API-ja, adatmodellje, upgrade-útja és a szakértelem köt	erős: API, auth-flow, adatmodell; szabványos protokollal, exporttal és szerződéses kikötéssel csökkenthető
Skálázhatóság	saját felelősség	saját felelősség	a szolgáltató biztosítja
Biztonság	saját felelősség (mTLS, X.509, OTA)	saját felelősség, de auditálható kód	részben menedzselt, de a felelősségi határok tisztázandók
Fejlesztési idő	hosszú – minden komponenst meg kell írni	közepes – a keretrendszer gyorsít	rövid – SDK-val és konzollal
Migrációs költség és lock-in	közepes: a saját kód és séma önmagunkhoz köt; export és dokumentált séma nélkül nő	közepes: platformváltás adatmodell- és API-migrációval jár	magas, de adathordozhatósági kikötéssel, szabványos interfésszel és migrációs teszttel csökkenthető

A döntési mátrix alapján a TerraSense-edzőrendszerben a saját építés elsősorban akkor indokolt, ha a tanulási cél, a teljes kontroll vagy az egyedi követelmények (például speciális rádióprotokoll, zárt telephelyi működés) elsődlegesek. A menedzselt felhős platform akkor racionális, ha a flotta mérete nagy, a csapat nem akar infrastruktúrát üzemeltetni, és a termék gyors piacra vitele a cél. A nyílt forrású platform jó kompromisszum, ha a szolgáltatói kötődés kizárt, de a teljes backend fejlesztését el szeretnénk kerülni.

22.1.5 Szolgáltatói kötődés és hordozhatóság

A szolgáltatói kötődés (vendor lock-in) azt jelenti, hogy a rendszer a választott platform szolgáltatásaihoz, API-jaihoz és adatmodelljéhez kötődik, és a másik szolgáltatóra váltás jelentős fejlesztést igényel. A kötődést érdemes négy külön dimenzióban értékelni, mert a nyílt forrású vagy saját megoldás sem mentes mindegyiktől:

- **Szolgáltatói kötődés:** szerződés, árazás, támogatási szint, adatközpont helye és a szolgáltató megszűnésének kockázata. Menedzselt platformnál ez a legláthatóbb, de a saját üzemeltetésnél is megjelenik a hoszting-, hálózati- és domain-szolgáltatókban.
- **Technológiai kötődés:** framework, adatbázis-kiterjesztés, SDK, konténer-orchesztráció és verziófrissítési út. A TimescaleDB hipers táblái, a Spring Boot JdbcTemplate-je, a paho-mqtt hívásai és a Zephyr MCUboot-konfigurációja mind olyan döntés, amelyet egy váltásnál újra kell tervezni.

- **Adatkötődés:** séma, formátum, megőrzési idő, tulajdonjog, exportálhatóság és a törölt adatok visszaállíthatósága. Ha a telemetria csak egy platform saját formátumában exportálható, a kód szabadon cserélhető, az adat mégsem.
- **Szervezeti kötődés:** a csapat ismerete, a futó folyamatok, a tanúsítványok, az ügyeleti rend és a szabványosítási auditok. Egy platformváltás akkor is drága, ha a szoftver szabad, mert az embereknek újra kell tanulniuk a rendszert.

A migrációs költség ezért nem nulla akkor sem, ha a forráskód nyílt. A váltás tervezéséhez legalább a következő tételeket kell becsülni: adatexport és -import, a régi és az új rendszer párhuzamos üzemeltetése, az adapterek és a topic-/séma-leképezés átírása, az eszközök újraprovízionálása, a tanúsítványok újragenerálása, a csapat betanítása és a visszaállítási terv. Egy jó döntési dokumentum ezért nem azt állítja, hogy a lock-in nulla, hanem azt, hogy mekkora a kilépés költsége, mennyi ideig tart, és milyen szerződéses vagy technikai feltételek csökkentik.

A TerraSense rendszerben az MQTT, a topic-konvenció (`terrasense/<site>/<node>/telemetry` stb.), a JSON-alapú telemetria-séma és az X.509-tanúsítványok mind szabványos vagy nyílt elemek. Ezért a TerraSense saját backendjének cseréje egy másik MQTT-kompatibilis platformra az eszközoldalon viszonylag kis változtatással járhat. Ez a hordozhatóság egyik kulcsa, de nem jelenti azt, hogy a teljes váltás költségmentes: a backend ingest-adapterét, az adatbázis-sémát, a riasztási szabályokat és a flottakezelés API-ját külön kell migrálni, és az átállás alatt párhuzamos üzemeltetésre lehet szükség.

A menedzselt platformok gyakran kínálnak saját SDK-t, topic-struktúrát vagy auth-flow-t. Ha ezeket az eszközbe égetjük, a váltás költséges. A jó gyakorlat az, hogy az eszköz csak szabványos protokollokat (MQTT, CoAP, TLS, CBOR/JSON) használjon, a platform-specifikus SDK-t pedig csak az edge-en vagy a backend oldalán alkalmazzuk. Így a szolgáltatóváltás eszközoldali része a broker-végpont és a hitelesítési konfiguráció cseréjére korlátozódhat; a backend- és adatoldali migráció ettől még külön tervezendő, és a kilépési tervet a bevezetés előtt kell megírni.

22.1.6 Költségmodell-becslés számpéldával

A költségmodell illusztrálásához készítsünk egy feltételezett becslést egy 1000 eszközös TerraSense flottára. Az alapforgatókönyv a A TerraSense mintarendszer architektúrája és a Nem funkcionális követelmények: energia, költség, élettartam, megbízhatóság, biztonság alfejezetekben rögzített **10 perces normál telemetriai ciklust** használja, nem óránkénti küldést. Az eszközök tehát óránként 6, havonta $6 \times 24 \times 30 = 4{,}320$ üzenetet küldenek. A teljes flotta havi üzenetszáma:

$$1{,}000 \times 4{,}320 = 4{,}320{,}000 \,\text{üzenet/hónap}.$$

Tegyük fel, hogy minden üzenet mérete 200 B, tehát a havi forgalom:

$$4{,}320{,}000 \times 200\,\text{B} = 864{,}000{,}000\,\text{B} = 864\,\text{MB}.$$

A menedzselt felhős platformoknál a díj rendszerint az üzenetek száma, a feldolgozott byte-ok és az extra szolgáltatások (például OTA, tanúsítványtárolás) alapján képződik. Feltételezett árakkal (ezek **nem** ellenőrzött aktuális listaárak, csak a modell szemléltetésére szolgálnak):

- Üzenetdíj: 1 USD / millió üzenet
- Feldolgozási és tárolási díj: 0,002 USD / MB
- Eszközmenedzsment: 0,05 USD / eszköz / hónap

Ekkor a havi költség az alapforgatókönyvben:

$$\text{MQTT-díj: } \frac{4{,}320{,}000}{1{,}000{,}000} \times 1\,\text{USD} = 4{,}32\,\text{USD},$$

$$\text{Tárolás: } 864\,\text{MB} \times 0{,}002\,\text{USD/MB} = 1{,}728\,\text{USD},$$

$$\text{Eszközmenedzsment: } 1{,}000 \times 0{,}05\,\text{USD} = 50\,\text{USD}.$$

Havi összesen körülbelül 56 USD, évente körülbelül 672 USD. Ez a használatarányos díj azonban nem azonos a teljes birtoklási költséggel (TCO), mert a saját üzemeltetésben a mérnöki munka, az ügyelet, a mentés, a rendelkezésre állás és a frissítés költsége is jelentkezik, a menedzselt oldalon pedig a támogatási csomag és az integráció.

Érzékenységvizsgálat. Ha az eszközök óránként küldenek (ez nem az alapkövetelmény, csak egy ritkább változat), a havi üzenetszám 720 000, a forgalom 144 MB, a díj pedig 0,72+0,288+50 ≈ 51 USD. Az óránkénti változat tehát alig olcsóbb, mert az eszközmenedzsment fix díja dominál. Ha viszont 200 eszköz 5 másodpercenként küld, a havi üzenetszám 107 136 000 lesz, az MQTT-díj önmagában 107,14 USD, a tárolási díj pedig 42,85 USD, így a használatarányos díj körülbelül 200 USD/hóra nő. Ez mutatja, hogy a nagy adatgyakoriság a menedzselt költséget is megemeli, de 1000 eszköznél a mérnöki munka még mindig nagyobb tétel lehet.

Teljes birtoklási költség (TCO) azonos szolgáltatási szinten. Az összehasonlítás hároméves időtávra, 1000 eszközre és azonos, 99,9%-os rendelkezésre állási célra készül. Mindkét oldalon feltételezzük a napi mentést, a 30 napos visszatartást, a monitorozást, a biztonsági frissítéseket, a TLS-t és a rendelkezésre álló ügyeletet; a saját oldalon a 99,9% két csomópontos HA-val és ügyelettel érhető el, ami nem szerződéses SLA, hanem tervezési cél. A mérnöki óradíj 60 USD, az ügyeleti óradíj 75 USD; ezek szintén illusztratívak. A táblázat minden sora feltételezett, nem aktuális listaár:

Költségtétel	Saját üzemeltetés (3 év)	Menedzselt platform (3 év)	Megjegyzés
Infrastruktúra vagy használat	3 600 USD	2 018 USD	saját: 100 USD/hó (VM, tárhely, HA, mentés, monitorozás, domain); menedzselt: 56 USD/hó használat
Támogatás	0 USD	9 000 USD	menedzselt: 250 USD/hó támogatási csomag (üzleti órák + 24/7 eszkaláció); saját: a csapat látja el
Mérnöki munka	86 400 USD	17 280 USD	saját: 0,25 FTE (40 óra/hó); menedzselt: 0,05 FTE (8 óra/hó)

Költségtétel	Saját üzemeltetés (3 év)	Menedzselt platform (3 év)	Megjegyzés
Ügyelet	43 200 USD	0 USD	saját: 16 óra/hó; a menedzselt SLA része
Frissítés, biztonság, CI	17 280 USD	0 USD	saját: 8 óra/hó; a menedzseltnél a szolgáltató végzi
Bevezetés vagy migráció	7 200 USD	4 800 USD	saját: 120 óra zöldmezős építés; menedzselt: 80 óra integráció
Betanítás	2 400 USD	1 200 USD	saját: 40 óra; menedzselt: 20 óra
Kockázati tartalék (10%)	16 008 USD	3 430 USD	incidens, kimaradás, árváltozás
Összesen (3 év)	**176 088 USD**	**37 728 USD**	a feltételezett egységárakkal
Havi egyenérték	**4 891 USD**	**1 048 USD**	36 hónapra osztva

A feltételezett modellben a menedzselt platform hároméves TCO-ja a saját üzemeltetésének mintegy 21%-a, mert a mérnöki és ügyeleti munka dominál. Ez az eredmény azonban nem univerzális: ha a csapat már rendelkezik a saját stackkel, az üzemeltetési tudás és a kapacitás adott, a saját oldal marginális készpénzköltsége csak az infrastruktúra 100 USD/hója lehet, miközben a menedzselt platform használati díja a flotta növekedésével nő. A döntéshez ezért a következő érzékenységeket kell külön kezelni: a mérnöki óradíj és a rendelkezésre álló szabad kapacitás, a flotta mérete és adatgyakorisága, az adatvisszatartási követelmény, a kilépési/migrációs költség, valamint az, hogy a szolgáltató támogatása valóban lefedi-e a kívánt ügyeleti szintet. A TerraSense tanulórendszerében a saját építés didaktikai és kontrollszempontból indokolt, de a labor infrastruktúrájának költsége nem tekinthető kereskedelmi TCO-nak.

A fenti számpélda **illusztratív**, az árak feltételezettek. A tényleges döntéshez a szolgáltatók aktuális díjszabását, a pontos üzenetszámot, a támogatási szerződést és a saját csapat valós kapacitását kell használni. Az ár-összehasonlítást mindig azonos szolgáltatási szintre, azonos időtávra és a kilépési költséget is tartalmazó TCO-modellre kell alapozni.

22.1.7 Matter és okosotthon-ökoszisztémák

A Matter (korábban Project CHIP) az okosotthon-eszközök interoperabilitását célzó, a Connectivity Standards Alliance (CSA) által gondozott szabvány. Célja, hogy egy eszköz egyszer működjön a nagy okosotthon-ökoszisztémákkal (például Apple HomeKit, Google Home, Amazon Alexa), anélkül, hogy mindegyikhez külön kellene integrálni.

A Matter elsősorban **alkalmazási rétegbeli** szabvány: eszköztípusokat, klasztereket, attribútumokat, parancsokat és eseményeket, valamint egy interakciós modellt (olvasás, írás, feliratkozás, parancshívás) definiál. Emellett rögzíti a biztonsági és a beüzemelési (commissioning) folyamatot is. Nem egyetlen „szállítási réteg”, és nem lehet az MQTT egyszerű fordításaként leírni. A rétegek a következőképpen különíthetők el:

Réteg	Matter-megfelelés	Megjegyzés
Alkalmazási adatmodell	eszköztípusok, klaszterek, attribútumok, parancsok, események	ez a réteg felel meg az MQTT payload-sémának és a topic-szemantikának
Interakciós modell	read, write, subscribe, invoke	állapotközpontú, nem naplóközpontú
Biztonság	PASE a kezdeti beüzemeléshez, CASE az operatív kapcsolathoz, DAC/PAI/NOC tanúsítványok	a két fázis biztonsági modellje eltér
Szállítás	TCP vagy UDP, UDP esetén MRP megbízhatósággal	a TCP nagyobb üzenetekhez és bizonyos munkamenetekhez használható
Hálózat	IPv6	Thread, Wi-Fi vagy Ethernet hálózaton; border router szükséges a Thread esetén
Beüzemelés	BLE és/vagy mDNS felfedezés, majd PASE	a BLE csak a commissioning alatt játszik szerepet

A Matter tehát IPv6-os hálózaton működik; a szállítási réteg TCP vagy UDP, a BLE pedig nem az operatív forgalom hordozója, hanem a kezdeti beüzemelés egyik csatornája. Az operatív kapcsolat a beüzemelés után IP-n, CASE-munkamenetben fut.

A beüzemelés fázisai külön kezelendők:

1. **Felfedezés:** az eszköz BLE-hirdetéssel vagy mDNS-sel válik láthatóvá a commissioning alkalmazás számára.
2. **PASE (Passcode-Authenticated Session Establishment):** a felhasználó által megadott beüzemelési kód (passcode) alapján létrejön egy ideiglenes, hitelesített munkamenet. Ebben a fázisban még nincs operatív X.509-eszköztanúsítvány; a PASE a passcode-ból származtatott kulccsal védi a csatornát.
3. **Operatív hitelesítő adatok:** az eszköz a gyártói attestation-láncot (DAC – Device Attestation Certificate, PAI – Product Attestation Intermediate, PAA – Product Attestation Authority) használja a termék hitelesítéséhez, a commissioner pedig a saját fabricjéhez tartozó NOC-ot (Node Operational Certificate) állít ki. A bizalmi horgonyok tehát a gyártói attestation-lánc és a fabric hitelesítésszolgáltatója, nem egyetlen általános „X.509 PKI”.
4. **CASE (Certificate Authenticated Session Establishment):** az operatív forgalom már a NOC-hoz kötött, tanúsítványalapú munkamenetben fut, IP-n. A CASE nem azonos a PASE-szel, és a tanúsítványok nem a kezdeti passcode-os beüzemelést helyettesítik.

A TerraSense ipari-mezőgazdasági környezetében a Matter nem közvetlenül előírás, de a tanulsága fontos: a szabványos adatmodell és a közös provisioning-folyamat csökkenti az integrációs költséget, és növeli a hordozhatóságot. A Matter okosotthon-fókuszú, ezért nem váltja ki az ipari protokollokat (OPC UA, Modbus) vagy a nagy hatótávú LoRaWAN-t; a TerraSense belső kommunikációja továbbra is MQTT marad.

Ha a TerraSense-t később okosotthon-ökoszisztémába kellene integrálni, akkor a `ts-gw` vagy a `ts-backend` rétegben egy **alkalmazásszintű Matter bridge** kellene. A bridge nem csomagszintű protokollfordító: Matter-eszköznek vagy bridge-eszköznek látszik, bridged endpointokat hoz

létre a `ts-node` csomópontokhoz, és a `terrasense/...` telemetriát, illetve parancsokat Matter-attribútumokra, illetve -parancsokra képezi le. Ehhez a bridge-nek Matter stackre, fabric-beüzemelésre és a TerraSense adatmodelljének leképezésére van szüksége; a szűk erőforrású `ts-node`-ra nem telepítenénk Matter stacket. A fordítás a message translator EIP-mintára vezethető vissza, amelyet a Enterprise integrációs minták IoT-ban alfejezetben alkalmaztunk, de itt az üzenetek szemantikája is átalakul: a Matter állapotközpontú attribútumai és a telemetria idősortárolása nem azonos fogalmak, ezért a bridge-nek kell eldöntenie, mely mérések lesznek attribútumok, melyek események, és hogyan kezeli a késleltetett vagy hiányzó frissítést.

22.1.8 Ellenőrzés és hibakeresés

A build vs. buy döntés során a leggyakoribb objektív hibák az alábbiak:

Tünet	Ok	Javítás
A menedzselt platform első becslésben olcsó, de a számla váratlanul magas	Az árazás üzenetenként, byte-onként vagy szolgáltatásonként összetett; a tesztforgalom is számlázott	Készítsünk üzenetszám- és méretmodellt, használjunk alertet a havi keretre
A nyílt forrású platform telepítése után a frissítések elmaradnak	A csapat a telepítést üzemeltetésnek tekinti, de a frissítést nem ütemezi	A compose/Kubernetes verziókat foglaljuk be a CI/CD futamokba, a frissítést automatizáljuk
Az eszköz SDK-ját a platformra szabjuk, és a váltás lehetetlen	A platform-specifikus API-kat az eszköz firmware-jébe linkeljük	Az eszköz csak MQTT/CoAP és JSON/CBOR szinten kommunikáljon; a platform-SDK az edge-en vagy a backendben legyen
A saját rendszer biztonsági frissítései hónapokig csúsznak	A komponensek (Mosquitto, Zephyr, Spring Boot) verziókövetése nincs automatizálva	Használjunk konténer- és firmware-képek alapján SBOM-ot és automatizált CVE-ellenőrzést

22.1.9 Gyakori félreértések

- „A menedzselt platform kiváltja a biztonsági felelősséget.” Ez téves: a platform védi az infrastruktúráját, de az eszköz hitelesítési adatainak kezelése, az alkalmazásszintű jogosultságok, a telemetria épsége és a firmware-aláírás a mi felelősségünk.
- „A nyílt forrású platform automatikusan olcsóbb.” A licencdíj nincs, de az üzemeltetés, a telemetria-séma kialakítása, a biztonsági hardening és a skálázás mérnöki órákat igényel.
- „Az MQTT miatt hordozhatók vagyunk.” Az MQTT csak a szállítás. A topic-konvenció, a payload-séma, az auth-flow és az árnyék szemantika határozza meg a valódi hordozhatóságot.
- „A Matter minden IoT-problémát megold.” A Matter okosotthon-fókuszú, és nem fedi le az ipari protokollokat (OPC UA, Modbus) vagy a nagy hatótávú LoRaWAN-t. A TerraSense esetén a Matter legfeljebb a gateway-integráció egyik lehetséges iránya.
- „A Matter beüzemelése egyszerűen X.509-es csatlakozás.” A kezdeti PASE fázis passcode-alapú, a BLE csak a commissioning egyik csatornája, és az operatív CASE-munkamenet használ tanúsítványt. A DAC/PAI/NOC attestation- és fabric-lánc nem azonos az MQTT kliens-tanúsítványokkal, és a PASE-t nem lehet X.509-csatornaként leírni.

22.1.10 Feladatok

1. **Fogalmi/döntési kérdés:** Magyarázza el, miért nem elegendő az MQTT-támogatás ahhoz, hogy egy saját rendszert könnyen át lehessen vinni egy menedzselt felhős platformra. Milyen további elemekre kell figyelni?
2. **Számolási feladat:** Számítsa ki, hány százalékkal nő a havi MQTT-üzenetdíj a fenti feltételezett árazással, ha az 1000 eszközből 200 darab 5 másodpercenként küld telemetriát, a többi pedig a 10 perces alapciklusban.
3. **Implementációs feladat a TerraSense-ben:** Készítsen döntési dokumentumot a `~/iot-labor/i065/decision.md` fájlban, amelyben a TerraSense flottára (1000 eszköz, óránkénti telemetria) a saját, a nyílt forrású és a menedzselt opciót értékeli. A dokumentum tartalmazzon egy szemponttáblázatot, a költségbecslést és egy ajánlást.
4. **Hibakeresési/kiterjesztési feladat:** Tekintse át a TerraSense `ts-backend` kódját, és azonosítsa, mely komponenseket lehetne kiváltani menedzselt szolgáltatásokkal. Írja le, melyik cseréhez milyen interface-változtatás kellene ahhoz, hogy az eszköz és a gateway kódjához ne kelljen hozzányúlni.

22.1.11 Megoldások

1. Mert az MQTT csak a szállítási protokoll. A hordozhatósághoz szükséges a topic-konvenció, a payload-séma (JSON/CBOR), az eszközazonosítás (például node_id), a hitelesítési mechanizmus (X.509, JWT), a szekvencia- és időbélyegkezelés, valamint az árnyék/desired-reported szemantika is. Ha ezek közül bármelyik platform-specifikus, a váltás költséges lesz.

2. A 200 eszköz 5 másodpercenként küld, tehát percenként 12 üzenetet, óránként 720-at, naponta 17 280-at, havonta 518 400 üzenetet eszközönként. 200 eszközre ez 103 680 000 üzenet. A maradék 800 eszköz a 10 perces alapciklusban küld, ami eszközönként $6 \times 24 \times 30 = 4\,320$ üzenet, összesen 3 456 000 üzenet. A teljes havi mennyiség 107 136 000 üzenet. Az MQTT-díj: $\frac{107{,}136{,}000}{1{,}000{,}000} \times 1\,\text{USD} = 107{,}136\,\text{USD}$, szemben az alapforgatókönyv 4,32 USD-jével. A növekedés $\frac{107{,}136-4{,}32}{4{,}32} \approx 23{,}8$, azaz körülbelül 2380%. A tárolási díj is nő: $107\,136\,000 \times 200\ \text{B} = 21\,427{,}2$ MB, ennek díja 42,85 USD, így a használatarányos összeg az eszközmenedzsmenttel együtt körülbelül 200 USD/hó.

3. A decision.md például így épülhet fel:

```
# TerraSense build vs. buy - 1000 eszköz

## Szempontok
| Szempont | Saját | Nyílt forrású | Menedzselt |

## Költség (3 év, 10 perces alapciklus)
- Menedzselt használat: ~56 USD/hó + támogatás + integráció
- Saját infra: ~100 USD/hó (VM, HA, mentés, monitorozás, domain)
- Mérnöki munka és ügyelet: ... óra/hó × ... USD/óra
- Bevezetés és betanítás: ... óra
- Kockázati tartalék: a részösszeg 10%-a

## Lock-in és kilépés
- Szolgáltatói, technológiai, adat- és szervezeti kötődés
- Adatexport módja, formátuma és tesztje
- Kilépési költség, idő és párhuzamos üzem hossza

## Ajánlás
Kis flottánál a menedzselt platform gyors indulást ad. A tanulórendszer
esetén a saját építés marad a kontroll és a tanulási cél miatt. Nagy
```

adatmenetnél a saját infra versenyképes lehet, de a TCO-ban a mérnöki munka, az ügyelet és a migráció is szerepel.

4. A `ts-backend` MQTT-ingestje cserélhető menedzselt device gatewayre, ha a bejövő topicok és payload változatlanok. A TimescaleDB cserélhető menedzselt idősoros adatbázisra, ha a schema és a lekérdezések kompatibilisek. A flottakezelés (`devices`, `device_config`) cserélhető felhős registry-re, ha az eszközazonosító- és árnyék-modell megegyezik. A legfontosabb interface: az MQTT-topic-szerződés és a JSON-séma, ezért ezeket explicit, verziózott szerződésként kell kezelni.

22.1.12 Rövid összefoglaló

Az alfejezet a TerraSense saját komponenseit a felhős és nyílt forrású IoT-platformok tipikus szolgáltatásaival feleltette meg. A felhős platformok device gatewayt, registryt, shadower, rules enginet, idősoros tárolást és OTA-menedzsmentet kínálnak; ezek rendre a TerraSense Mosquitto, `devices` tábla, `device_config` árnyék, riasztási szabályok, TimescaleDB és saját OTA-folyamatainak felelnek meg. A nyílt forrású alternatívák (ThingsBoard, Eclipse Hono/Ditto) a saját építés és a menedzselt szolgáltatás közötti kompromisszumot kínálnak. A döntéshez nem elég a havi használati díjat összevetni: a 10 perces alapciklusra számolt, azonos szolgáltatási szintű, hároméves TCO-nak tartalmaznia kell a mérnöki munkát, az ügyeletet, a mentést, a rendelkezésre állást, a frissítést és a migrációt is. A lock-in értékelése négy dimenzióban (szolgáltatói, technológiai, adat- és szervezeti kötődés) történik, és a kilépés költségét akkor is meg kell becsülni, ha a forráskód nyílt. A Matter esetében külön kell választani az IPv6-os hálózati réteget, a TCP/UDP-s szállítást, a BLE-alapú commissioningot, a passcode-os PASE-t és a tanúsítványos CASE-t; a Matter bridge alkalmazásszintű leképezés, nem csomagszintű protokollfordító. **Továbbvitt eredmény:** a felhős komponensek megfeleltetési táblázata, a build vs. buy mátrix és a TCO-modell a Záróprojekt: a teljes TerraSense rendszer összeállítása és átvétele alfejezet döntési dokumentumában, valamint a Összefoglalás: döntési térkép, ellenőrzőlista, továbbtanulás összefoglalójában közvetlenül felhasználandó.

22.1.13 Források és ellenőrzési hiányok

- AWS IoT, Azure IoT, Google Cloud IoT dokumentációk: a pontos szolgáltatásnevek, árazási modellek és aktuális korlátok ellenőrizendők; a fenti árazási példa feltételezett, nem hivatkozott listaár.
- ThingsBoard, Eclipse Hono/Ditto dokumentációk: a telepítési és API-részletek változhatnak, ellenőrzési igénnyel.
- CSA Matter specifikáció és F28 (Matter projekt dokumentáció, PASE/CASE és commissioning): a Matter-rétegek, a PASE és CASE szétválasztása, a DAC/PAI/NOC attestation-lánc és a bridge-követelmények pontos leírása ellenőrzendő. A fejezet a PASE-t passcode-alapú, a CASE-t tanúsítványalapú munkamenetként írja le; ez az F28 és a Matter 1.x specifikáció szerinti értelmezés, amelyet a laborban a `chip-tool` commissioning naplójával lehet igazolni.
- ETSI EN 303 645, EU Cyber Resilience Act: a platformszolgáltatókra és a gyártókra vonatkozó kötelezettségek szövegszerű pontosítása ellenőrzendő.
- A TCO-számítás minden egységára (üzenetdíj, MB-díj, eszközdíj, mérnöki és ügyeleti óradíj, támogatási díj) illusztratív, nem aktuális listaár. A tényleges döntéshez a szolgáltatók aktuális díjszabását, a saját csapat valós kapacitását és a migrációs tervet kell használni; a modell nem állítja, hogy a saját vagy a menedzselt megoldás általában olcsóbb.

23 Zároprojekt és összefoglalás

23.1 Zároprojekt: a teljes TerraSense rendszer összeállítása és átvétele

A TerraSense egyes rétegei a korábbi laborokban külön-külön már működtek, de a rendszer akkor válik ténylegesen tesztelhetővé, ha minden komponens egyszerre, egyetlen hálózatban fut. Ebben a zároprojektben a cél nem új funkció fejlesztése, hanem az integráció: több szimulált `ts-node`, a `ts-gw`, a TLS-es Mosquitto, a `ts-backend`, a TimescaleDB és a Grafana összeállítása egyetlen Compose-környezetben, majd annak bizonyítása, hogy a korábban rögzített követelmények a teljes adatúton teljesülnek.

23.1.1 Motiváció

A laborok során a komponensek gyakran elszigetelten futottak: a firmware-t `native_sim` alatt, a backendet tesztprofillal, a brokert pedig ideiglenes konfigurációval indítottuk el. A valós IoT-projektekben a hibák jelentős része éppen az integrációs határokon jelentkezik: rossz tanúsítványútvonal, eltérő topic-konvenció, nem egyező séma, tűzfal által blokkolt kapcsolat vagy nem megfelelő konténer-függőségi sorrend. A zároprojekt ezeket a kockázatokat átvételi tesztekkel kezeli.

A TerraSense követelményeit a Nem funkcionális követelmények: energia, költség, élettartam, megbízhatóság, biztonság alfejezetben és a későbbi laborokban fokozatosan pontosítottuk. A zároprojektben ezekből egy minimális, de mérhető átvételi követelménykészletet használunk: a telemetria eljut az eszköztől az adatbázisig, a parancsok és az eszközárnyék működnek, a gateway offline pufferel és visszatölt, a biztonságos csatorna kötelező, a hiányzó adat pedig riasztást vált ki.

A követelmények nem egyformán igazolhatók a laborban. A telemetria adatút, a parancs-nyugta, az offline visszatöltés, az mTLS és a riasztási értesítés végponttól végpontig tesztelhető; a hétnapos pufferkapacitás, a 99,9%-os üzemi kézbesítés, a hardveres szelep-biztonsági zárás, az eszköz-X.509, az OTA-visszagördülés és a REST-jogosultság csak részben vagy egyáltalán nem. Ezeket a fejezet végén található **követelmény-nyomonkövetési mátrix** külön kezeli, és a zároprojekt nem tekinti automatikusan teljesítettnek őket.

23.1.2 Fogalmak és működés

Definíció. Az **átvételi teszt** (acceptance test) olyan fekete dobozos ellenőrzés, amely nem a fejlesztő feltételezéseit, hanem a megrendelői vagy rendszerkövetelményeket igazolja. Eredménye általában PASS vagy FAIL, és minden egyes átvételi teszthez egyértelműen hozzá kell rendelni legalább egy követelmény-azonosítót.

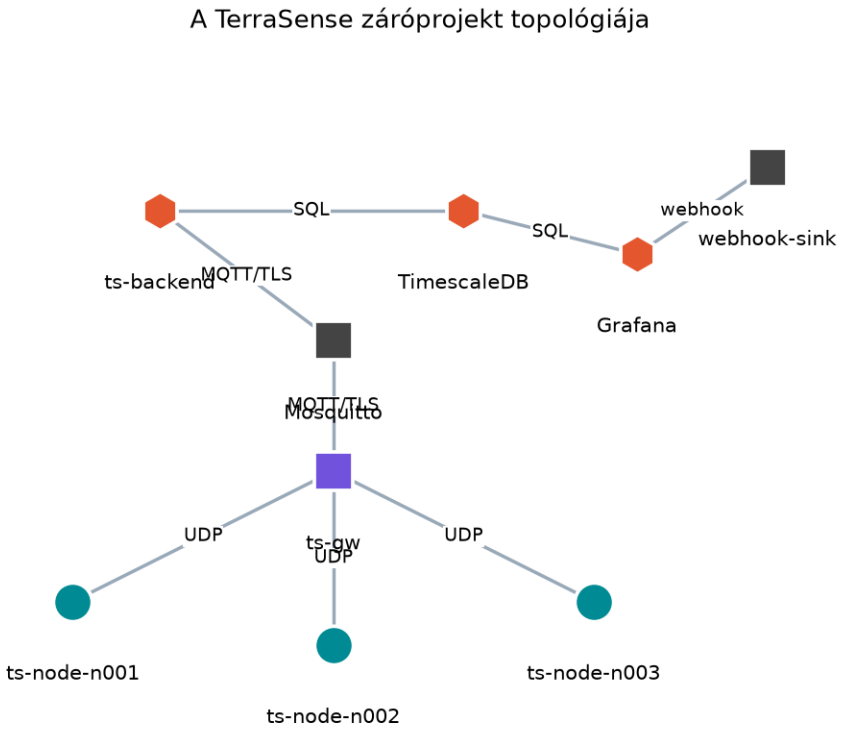
Definíció. A **követelmény-nyomonkövetés** (requirements traceability) az a táblázat vagy gráf, amely a követelményektől a tervdokumentumokon és a megvalósításon át a tesztekig mutatja a

kapcsolatot. IoT-rendszereknél ez azért fontos, mert egy biztonsági vagy adatkezelési követelmény könnyen „elvész” a rétegek között.

Definíció. A **Compose-profil** olyan jelölő, amellyel szolgáltatások egy csoportját csak akkor indítjuk el, ha a profilt explicit módon bekapcsoljuk. Ezzel ugyanaz a `compose.yaml` használható könnyű fejlesztői, teljes integrációs vagy tesztelői módban.

Definíció. A **tesztprofil** olyan explicit, dokumentált konfiguráció, amely a laborban lerövidíti az éles küszöbértékeket (például a hiányzóadat-riasztás 10 perces ablaka helyett 60 másodpercet használ), miközben a termelési alapérték a konfigurációban megmarad. A tesztprofil nem azonos a Compose-profillal: utóbbi szolgáltatásokat kapcsol be, előbbi paramétereket ír felül. A záróprojektben a tesztprofil minden eltérése névvel és indoklással szerepel, hogy a laboreredmény ne legyen összetéveszthető a termelési beállítással.

A záróprojekt topológiája:



23.1. ábra: A TerraSense záróprojekt topológiája - A szimulált node-ok UDP-n a ts-gw-hez kapcsolódnak, a ts-gw MQTT/TLS-en a brokerhez, a backend pedig a brokerhez és az adatbázishoz. A Grafana a webhook-sinknek küldi a riasztásokat.

A szimulált `ts-node` konténerek a záróprojekt **végleges, integrált üzenetszerződését** használják. Ez a szerződés a Gateway szerepe: híd, puffer, biztonsági határ alfejezetben bevezetett UDP/JSON átvitelre és az Adatformátumok: JSON, CBOR, Protocol Buffers, SenML alfejezetben rögzített végleges telemetriasémára épül, de a záróprojektben kiegészül a `site` és `boot_id` mezővel, valamint a szelepállapotot hordozó `actuators` objektummal:

```
{
  "kind": "telemetry",
  "schema_version": 1,
```

```
  "site": "gh1",
  "node_id": "n001",
  "boot_id": "9f3c1a7b2e40",
  "seq": 42,
  "ts": 1730000000000,
  "measurements": {
    "temperature": {"value": 23.4, "unit": "Cel", "valid": true, "reason": null},
    "humidity": {"value": 61.2, "unit": "%RH", "valid": true, "reason": null},
    "soil_moisture": {"value": 34.0, "unit": "%", "valid": true, "reason": null}
  },
  "actuators": {"valve_open": false}
}
```

A `seq` a `(site, node_id, boot_id)` hármason belül monoton növekvő; újraindulás után a `boot_id` megváltozik, ezért a `seq` újraindulhat. A `ts` az esemény ideje UTC-ben, epoch milliszekundumban; a `boot_id` a bootpillanatot azonosítja, és az idempotenciához is szükséges. A `measurements` objektumtérkép a hivatalos séma; a korábbi laborok `sensor_data` objektuma és a `T:23.4;RH:61.2;SM:34;seq:42` szöveges UDP-példája a záróprojektben nem érvényes. A `measurements` minden eleme `value/unit/valid/reason` négyest hordoz; a `valid=false` eset a hibás vagy hiányzó mérést jelöli, nem a mérés elhagyását. A `ts-gw` a `terrasense/<site>/<node>/telemetry` topicra publikál QoS 1 szinten, `retain=false` beállítással; a `state` és a `status` topic retained, a `cmd` és a `cmd/result` nem retained. A gateway a `terrasense/<site>/<node>/cmd` topicra érkező parancsot UDP-n továbbítja a node-nak, a node `terrasense/<site>/<node>/cmd/result` topicra küldött nyugtáját pedig a backendnek továbbítja. A `boot_id` változásakor a gateway a `(site, node_id)` párhoz tartozó `last_seq` ellenőrzést az új `boot_id`-hez igazítja, különben egy újraindult node első üzenetét tévesen rendellenes sorrendűnek minősítené.

A node a szelepállapot-változást külön `state` üzenetben is elküldi:

```
{
  "kind": "state",
  "site": "gh1",
  "node_id": "n001",
  "boot_id": "9f3c1a7b2e40",
  "seq": 42,
  "ts": 1730000000000,
  "valve_open": false
}
```

A gateway ezt a `terrasense/gh1/n001/state` topicra publikálja QoS 1 szinten, `retain=true` beállítással. A `state` üzenet a `seq`-et is hordozza, hogy a backend a telemetriával azonos módon tudja deduplikálni és sorrendezni. A `status` topic a jelenlétet hordozza (`online`/`offline`), a Last Will pedig ugyanerre a topicra kerül; a `cmd` és a `cmd/result` `retain` értéke hamis, mert egy elavult parancs újraindítás után nem hajtódhat végre.

A rendszerindítás a következő függőségi láncot követi:

1. A TimescaleDB a `schema/` init szkriptjeivel inicializálja az adatbázist, és a Mosquitto elindul.
2. A `ts-backend` csak a TimescaleDB `healthy` állapota után indul, a `ts-gw` pedig a Mosquitto után; mindkettő újrapróbálkozik, ha a függősége még nem fogad kapcsolatot.
3. A szimulált node-ok utolsóként indulnak, hogy a hálózati szolgáltatások már fogadókészek legyenek.
4. A Grafana a TimescaleDB-ből olvas, a dashboard és a riasztási szabály a provisioning fájlokból töltődik be; a riasztásokat a `webhook-sink` fogadja.

A `schema/` init szkriptjei csak **üres adatköteten** futnak le. Ezért az első indítás előtt, illetve a séma módosítása után `docker compose down -v` szükséges; a `-v` törli a `timescale-data`, a `gw-store` és az `alert-log` köteteket is. A laborban ez elfogadható, mert a cél a reprodukálható kiinduló állapot, nem az adatmegőrzés.

23.1.3 Döntési szempontok

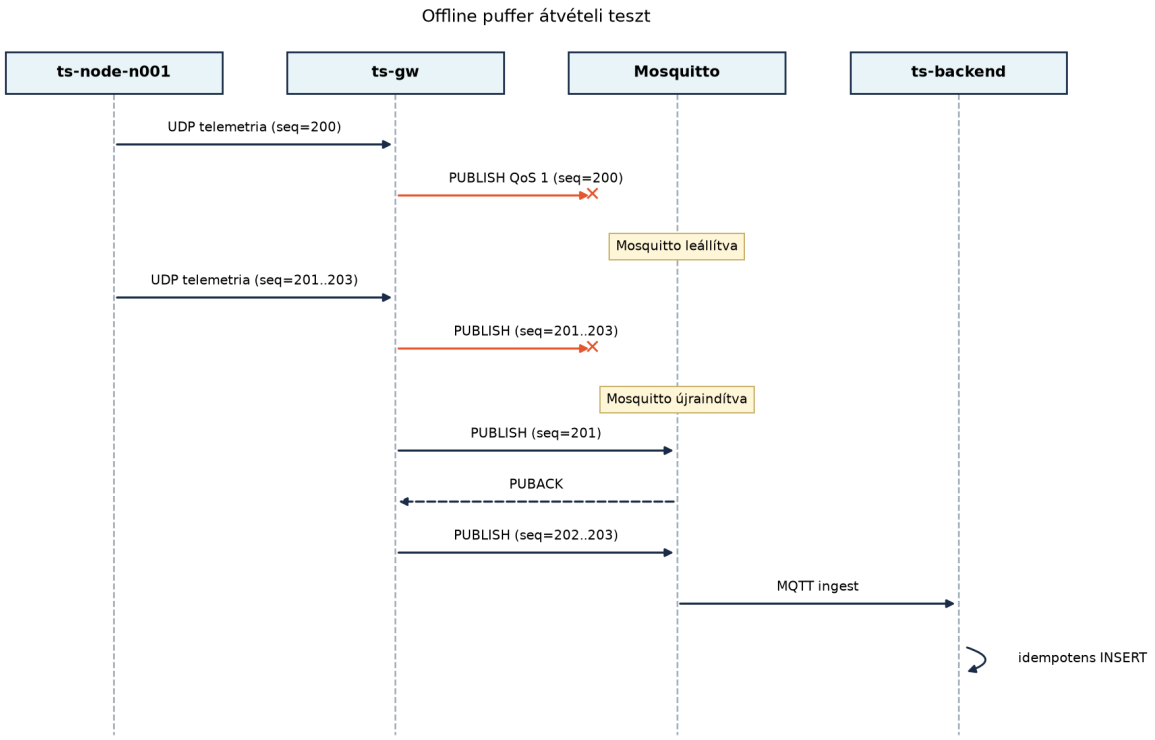
A záróprojektben három döntést érdemes tudatosan meghozni: mit mérjünk a teljes rendszeren, milyen tesztelési mélységet válasszunk, és hogyan kezeljük a szimulált node-ok számát.

Döntés	Lehetőség	Előny	Hátrány	Választás a záróprojektben
Node-ok indítása	egy folyamat több node-dal	egyszerűbb erőforrás-gazdálkodás	nehezebb hibát izolálni	három külön konténer
TLS-teszt	csak konfiguráció ellenőrzése	gyors	nem bizonyítja a lezárt portot és az ACL-t	pozitív és negatív kapcsolódási kísérlet
Offline teszt	broker leállítása hosszú időre	valószerű	lassú	35 másodperces leállás, `seq`-folytonosság ellenőrzésével
Mérés	pontos idők naplózása	reprodukálható	több műszerezést igényel	SQL-alapú esemény- és késleltetésmérés (`server_ts`, `seq`)
Hiányzóadat-teszt	a 10 perces termelési küszöb kivárása	valósághű	lassú	60 s-es tesztprofil, a termelési érték dokumentálva

A Compose-profilokat itt csak jelöljük, mert a záróprojektben az összes szolgáltatás egy profilba kerül. Hasznos lehet azonban külön profilt felvenni a node-szimulátorokra (`sim`) és a megjelenítő rétegre (`ui`), hogy a backend-fejlesztés során ne induljon el minden feleslegesen. A teljes záróprojekt az alapértelmezett profillal indul.

23.1.4 Kidolgozott példa: az offline puffer átvételi forgatókönyve

A legtanulságosabb átvételi teszt a `ts-gw` offline működését ellenőrzi. A Store-and-forward: offline működés és visszatöltés alfejezetben elkészített SQLite-puffernek a teljes rendszerben is meg kell őriznie az üzeneteket, ha a broker átmenetileg nem elérhető. A forgatókönyv:



23.2. ábra: Offline puffer átvételi teszt - A ts-gw a brokerleállás idején pufferel, majd a visszatérés után sorban visszatölt.

Az első `PUBLISH` azért vész el, mert a brokert a küldés előtt leállítjuk. A `ts-gw` a Paho kliens hibajelzése után nem törli a rekordot az SQLite-pufferből, hanem újrapróbálkozik. A node-ok eközben továbbra is 10 másodpercenként küldenek UDP-csomagot, így a 35 másodperces leállás alatt három új rekord kerül a pufferbe. Amikor a Mosquitto újraindul, a `ts-gw` a pufferből `seq` szerint visszatölt, és minden sikeres `PUBACK` után törli az adott rekordot. A backend idempotens ingestje a `(site, node_id, boot_id, seq, measurement_type, ts)` kulcsot használja, így az újraküldött üzenetek nem okoznak duplikátumot: egy eseményhez három mérési sor tartozik, és a visszatöltés után is három marad, nem hat. A `boot_id` azért része a kulcsnak, mert a node újraindulása után a `seq` nulláról indulhat; a `site` pedig azért, mert két telephely ugyanazt a rövid `n001` node-nevet használhatja.

23.1.5 Labor: a teljes TerraSense stack összeállítása és átvétele

Munkakönyvtár:

```
mkdir -p ~/iot-labor/i066 && cd ~/iot-labor/i066
```

A labor előfeltétele a korábbi laborkimenetek megléte. A záróprojekt nem közvetlenül a korábbi laborok részleges könyvtáraiból buildel, mert azok oktatási állapotot tartalmaznak; helyette egy külön, végleges integrációs projektet állít össze. A `~/iot-labor/i066/` fa:

```
~/iot-labor/i066/
  ts-backend/          # i043 váza + i044 ingest + i045 séma + i049 parancs + i050 árnyék +
↪  i061 flotta
  ts-gw/               # i038 Dockerfile és gateway + i037 store-and-forward + i057 TLS
  schema/              # 001_telemetry.sql, 002_fleet.sql, 003_command.sql, 004_alert.sql,
↪  005_seed.sql
```

```
grafana-provisioning/
    datasources/datasources.yml
    dashboards/dashboards.yml
    dashboards/dashboard.json
    alerting/terrasense-alerts.yml
mosquitto/           # mosquitto.conf, acl.conf
certs/               # lab-ca.crt, ts-gw.crt/key, ts-backend.crt/key, mosquitto.crt/key
ts_node_sim.py
webhook_sink.py
acceptance_test.sh
```

Az összeállítás forrásai:

Cél	Forrás	Megjegyzés
Spring Boot váz, REST, Dockerfile	`~/iot-labor/i043/`	a 14.3 projekt közvetlenül az `i043` gyökere, nem `i043/ts-backend`
MQTT ingest és idempotencia	`~/iot-labor/i044/ts-backend/`	a végleges kulcsot a záróprojekt egészíti ki `site` és `boot_id` mezővel
telemetria tábla, normalizálás	`~/iot-labor/i045/`	`server_ts` és hosszú tábla; a végleges egyedi kulcs a `site`/`boot_id` mezőt is tartalmazza
parancsmodell	`~/iot-labor/i049/`	`DeviceCommand`, `valve.open`, `duration_ms`, `status` nyugta; a végleges változat a `device_command` táblát használja
eszközárnyék	`~/iot-labor/i050/`	a laborban memóriabeli; a záróprojekt ezt a korlátot dokumentálja
flotta- és konfigurációs táblák	`~/iot-labor/i061/schema.sql`	`devices`, `device_config`, `device_state`, `device_groups`
gateway és Dockerfile	`~/iot-labor/i038/ts-gw/`	`TS_GW_*` környezeti változók
store-and-forward	`~/iot-labor/i037/ts-gw/`	`spool` tábla, `/var/lib/ts-gw/spool.db`; a `max_records` korlát a végleges változatban kötelező
TLS/mTLS	`~/iot-labor/i057/`	`lab-ca.crt`, `ts-gw.crt`, `ts-backend.crt`, `mosquitto.crt`
Grafana	`~/iot-labor/i051/`, `~/iot-labor/i052/`	az adatforrás és a riasztás a végleges compose hostneveihez igazítva

A végleges `ts-backend` nem memóriabeli megoldás: a parancsokat a `device_command` táblában tartja, a flotta- és konfigurációs adatokat a `devices`/`device_config`/`device_state` táblákból olvassa, a telemetriát pedig a végleges `telemetry` táblába írja. Az eszközárnyék a laborban továbbra is memóriabeli (`ShadowService`), ezért a záróprojekt átvételi mátrixa ezt „részben igazolt” állapotként rögzíti. A `ShadowService` a desired `OPEN`/`CLOSED` értéket a parancsszerződés

`valve.open`/`valve.close` típusára képezi le, a delta alapján a `cmd` topicra publikál, a `reported` értéket pedig a node `state` üzeneteiből frissíti. Maga az árnyék állapota memóriabeli, ezért a desired/reported pár a backend újraindításakor elveszik. A végleges gateway az `i038` Dockerfileját használja, kiegészítve a spool könyvtár létrehozásával, a `tsgw` felhasználó jogosultságával és a TLS környezeti változókkal.

A teljes `compose.yaml` a közös stack könyvtárba kerül:

```yaml
name: terrasense-lab

services:
  mosquitto:
    image: eclipse-mosquitto:2.0.18
    ports:
      - "8883:8883"
    volumes:
      - ../i066/mosquitto/mosquitto.conf:/mosquitto/config/mosquitto.conf:ro
      - ../i066/mosquitto/acl.conf:/mosquitto/acl.conf:ro
      - ../i066/certs:/mosquitto/certs:ro
    command: ["mosquitto", "-c", "/mosquitto/config/mosquitto.conf"]

  timescaledb:
    image: timescale/timescaledb:2.17.2-pg16
    environment:
      POSTGRES_DB: terrasense
      POSTGRES_USER: terrasense
      POSTGRES_PASSWORD: lab_password
    volumes:
      - timescale-data:/var/lib/postgresql/data
      - ../i066/schema:/docker-entrypoint-initdb.d:ro
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U terrasense -d terrasense"]
      interval: 5s
      timeout: 3s
      retries: 12

  ts-backend:
    build: ../i066/ts-backend
    depends_on:
      mosquitto:
        condition: service_started
      timescaledb:
        condition: service_healthy
    ports:
      - "127.0.0.1:8080:8080"
    environment:
      SPRING_DATASOURCE_URL: jdbc:postgresql://timescaledb:5432/terrasense
      SPRING_DATASOURCE_USERNAME: terrasense
      SPRING_DATASOURCE_PASSWORD: lab_password
      MQTT_URI: ssl://mosquitto:8883
      MQTT_CLIENT_ID: ts-backend
      MQTT_CA_CERT: /certs/lab-ca.crt
      MQTT_CLIENT_CERT: /certs/ts-backend.crt
      MQTT_CLIENT_KEY: /certs/ts-backend.key
    volumes:
      - ../i066/certs:/certs:ro

  ts-gw:
    build: ../i066/ts-gw
    depends_on:
      mosquitto:
        condition: service_started
    environment:
```

```
    TS_GW_MQTT_BROKER: mosquitto
    TS_GW_MQTT_PORT: "8883"
    TS_GW_MQTT_TLS: "true"
    TS_GW_TLS_CA: /certs/lab-ca.crt
    TS_GW_TLS_CERT: /certs/ts-gw.crt
    TS_GW_TLS_KEY: /certs/ts-gw.key
    TS_GW_CLIENT_ID: ts-gw
    TS_GW_UDP_PORT: "9000"
    TS_GW_SPOOL_MAX_RECORDS: "181440"
    TS_GW_LOG_LEVEL: info
  volumes:
    - ../i066/certs:/certs:ro
    - gw-store:/var/lib/ts-gw
  ports:
    - "9000:9000/udp"

ts-node-n001:
  image: python:3.12-slim
  depends_on:
    - ts-gw
  environment:
    SITE: gh1
    NODE_ID: n001
    GW_HOST: ts-gw
    GW_PORT: "9000"
    LISTEN_PORT: "9000"
    MAX_VALVE_OPEN_S: "60"
  volumes:
    - ../i066/ts_node_sim.py:/app/ts_node_sim.py:ro
  command: ["python", "/app/ts_node_sim.py"]
  restart: unless-stopped

ts-node-n002:
  image: python:3.12-slim
  depends_on:
    - ts-gw
  environment:
    SITE: gh1
    NODE_ID: n002
    GW_HOST: ts-gw
    GW_PORT: "9000"
    LISTEN_PORT: "9000"
    MAX_VALVE_OPEN_S: "60"
  volumes:
    - ../i066/ts_node_sim.py:/app/ts_node_sim.py:ro
  command: ["python", "/app/ts_node_sim.py"]
  restart: unless-stopped

ts-node-n003:
  image: python:3.12-slim
  depends_on:
    - ts-gw
  environment:
    SITE: gh1
    NODE_ID: n003
    GW_HOST: ts-gw
    GW_PORT: "9000"
    LISTEN_PORT: "9000"
    MAX_VALVE_OPEN_S: "60"
  volumes:
    - ../i066/ts_node_sim.py:/app/ts_node_sim.py:ro
  command: ["python", "/app/ts_node_sim.py"]
  restart: unless-stopped
```

```
  grafana:
    image: grafana/grafana:11.2.0
    depends_on:
      timescaledb:
        condition: service_healthy
    ports:
      - "3000:3000"
    environment:
      GF_SECURITY_ADMIN_PASSWORD: lab_admin
      GF_AUTH_ANONYMOUS_ENABLED: "false"
    volumes:
      - ../i066/grafana-provisioning/datasources:/etc/grafana/provisioning/datasources:ro
      - ../i066/grafana-provisioning/dashboards:/etc/grafana/provisioning/dashboards:ro
      - ../i066/grafana-provisioning/alerting:/etc/grafana/provisioning/alerting:ro

  webhook-sink:
    image: python:3.12-slim
    command: ["python", "/app/webhook_sink.py"]
    volumes:
      - ../i066/webhook_sink.py:/app/webhook_sink.py:ro
      - alert-log:/data
    expose:
      - "8081"

volumes:
  timescale-data:
  gw-store:
  alert-log:
```

A `ts-node` szolgáltatások Python-szimulátorok, amelyek a Zephyr `native_sim` node helyét veszik át az integrációs laborban; a valós firmware a Zephyr 3.7.0/SDK 0.16.8 verziómátrix szerint épül. A szimulátor a végleges UDP- és MQTT-szerződést gyakorolja, nem helyettesíti a hardveres időzítési és energiaviselkedést. A `ts-gw` és a `ts-backend` a `../i066/...` végleges integrációs könyvtárból buildel, nem a korábbi laborok részleges könyvtáraiból.

23.1.5.1 A végleges gateway Dockerfile-ja

A `~/iot-labor/i066/ts-gw/Dockerfile` a 12.3 labor Dockerfile-jára épül, de a store-and-forward réteghez szükséges SQLite-ot és az írható spool könyvtárat is tartalmazza:

```
# Build stage
FROM ubuntu:24.04 AS build
RUN apt-get update && apt-get install -y --no-install-recommends \
    g++ cmake pkg-config libpaho-mqtt-dev libsqlite3-dev libsystemd-dev \
    && rm -rf /var/lib/apt/lists/*
WORKDIR /src
COPY . .
RUN cmake -B build -DCMAKE_BUILD_TYPE=Release && cmake --build build -j"$(nproc)"

# Runtime stage
FROM ubuntu:24.04
RUN apt-get update && apt-get install -y --no-install-recommends \
    libpaho-mqtt1.0 libsqlite3-0 sqlite3 libsystemd0 ca-certificates \
    && rm -rf /var/lib/apt/lists/*
RUN useradd --system --no-create-home --shell /usr/sbin/nologin tsgw \
    && install -d -o tsgw -g tsgw -m 0750 /var/lib/ts-gw
COPY --from=build /src/build/ts-gw /usr/local/bin/ts-gw
USER tsgw
ENTRYPOINT ["/usr/local/bin/ts-gw"]
```

A `tsgw` felhasználó a `/var/lib/ts-gw` könyvtár tulajdonosa, ezért az SQLite spool a `gw-store` kötetben írható; a névvel ellátott kötet a konténer újraalkotása után is megőrzi a puffer tartalmát. A végleges `Config.cpp` a `TS_GW_MQTT_TLS`, `TS_GW_TLS_CA`, `TS_GW_TLS_CERT` és `TS_GW_TLS_KEY` változókat olvassa; ha a TLS be van kapcsolva, a kliens a három tanúsítvány nélkül nem indul el. A spool felső korlátja a `TS_GW_SPOOL_MAX_RECORDS`, túllépéskor a legrégebbi rekordok törlődnek. **Ellenőrzési igény.** A Paho MQTT C++ és az SQLite csomagok pontos neve Ubuntu 24.04-en ellenőrizendő; a laborban `apt-cache search paho` és `apt-cache search sqlite3` ad felvilágosítást.

A tanúsítványok neve és a Mosquitto hosztneve a végleges compose-ban eltér a 19.3 labor `localhost`-ra kiállított példányától. A `certs/` könyvtárban ezért a következő fájloknak kell lenniük: `lab-ca.crt`, `ts-gw.crt`, `ts-gw.key`, `ts-backend.crt`, `ts-backend.key`, `mosquitto.crt`, `mosquitto.key`. A Mosquitto szerver tanúsítványát a belső hálózati névre is ki kell állítani, legalább a következő SAN-okkal: `DNS:mosquitto`, `DNS:localhost`, `IP:127.0.0.1`. A 19.3 labor `CN=localhost`, `SAN=DNS:localhost,IP:127.0.0.1` tanúsítványa a `ts-backend` és a `ts-gw` `ssl://mosquitto:8883` kapcsolatához nem elegendő, mert a hosztnév-ellenőrzés a `mosquitto` nevet keresi. Az ACL a `/mosquitto/acl.conf` útvonalon van mountolva, a `require_certificate true` és a `use_identity_as_username true` beállítás mellett a `ts-gw` és a `ts-backend` CN-je az ACL-felhasználó.

A végleges `mosquitto.conf`:

```
per_listener_settings true

listener 8883
cafile /mosquitto/certs/lab-ca.crt
certfile /mosquitto/certs/mosquitto.crt
keyfile /mosquitto/certs/mosquitto.key
require_certificate true
use_identity_as_username true
acl_file /mosquitto/acl.conf
```

A végleges `acl.conf` a 19.3 labor ACL-jét egészíti ki a gateway parancs-feliratkozásával és a `cmd/result` publikálásával:

```
user ts-gw
topic write terrasense/+/+/telemetry
topic write terrasense/+/+/state
topic write terrasense/+/+/cmd/result
topic read terrasense/+/+/cmd

user ts-backend
topic read terrasense/+/+/telemetry
topic read terrasense/+/+/state
topic write terrasense/+/+/cmd
topic read terrasense/+/+/cmd/result
```

A `ts-gw` a `cmd` topicot csak olvashatja, ezért a T4 ACL-teszt jogosulatlan `PUBLISH`-t kísérel meg a `terrasense/gh1/n001/cmd` topicra; ennek a Mosquitto `Denied PUBLISH` naplóbejegyzéssel vagy `not authorised` klienshibával kell elutasítódnia. Az ACL nem tartalmaz `topic readwrite #` vagy névtelen hozzáférést, és a 1883-as listener nem létezik.

23.1.5.2 A végleges adatbázis-séma

A `~/iot-labor/i066/schema/` init szkriptjei a korábbi laborok tábláit egyetlen, a záróprojekthez igazított sémába rendezik. A `docker-entrypoint-initdb.d` a fájlokat betűrendben futtatja, ezért a számozott nevek sorrendje kötelező.

001_telemetry.sql:

```
CREATE EXTENSION IF NOT EXISTS timescaledb;

CREATE TABLE IF NOT EXISTS telemetry (
    server_ts        timestamptz NOT NULL DEFAULT now(),
    ts               timestamptz NOT NULL,
    site             text        NOT NULL,
    node_id          text        NOT NULL,
    seq              bigint      NOT NULL,
    boot_id          text        NOT NULL,
    schema_version   text        NOT NULL,
    measurement_type text        NOT NULL,
    value            double precision NOT NULL,
    unit             text,
    valid            boolean     NOT NULL DEFAULT true,
    UNIQUE (site, node_id, boot_id, seq, measurement_type, ts)
);

SELECT create_hypertable('telemetry', 'ts',
       chunk_time_interval => interval '7 days',
       if_not_exists => TRUE);
CREATE INDEX IF NOT EXISTS telemetry_node_ts ON telemetry (node_id, ts DESC);
```

A végleges egyedi kulcs a 15.2 labor `(node_id, seq, measurement_type, ts)` kulcsát egészíti ki `site` és `boot_id` mezővel. Erre azért van szükség, mert a `seq` újraindulás után nulláról indulhat, és két telephely azonos rövid node-nevet használhat. A `boot_id` ezért a végleges sémában `NOT NULL`. A `schema_version` az Idősoros adatbázisok és adatmodellezés táblájához hasonlóan szöveges; a wire formátum egész `1` értékét a backend `"1"` szövegként írja be.

`002_fleet.sql` a 17.3 labor tábláit tartalmazza változatlanul: `device_groups`, `devices`, `device_config`, `device_state`. A `devices.node_id` továbbra is a `site/node` alakú kulcs (`gh1/n001`), a telemetria `site` és `node_id` mezője pedig külön oszlop.

003_command.sql:

```
CREATE TABLE IF NOT EXISTS device_command (
    id          text PRIMARY KEY,
    site        text NOT NULL,
    node_id     text NOT NULL,
    type        text NOT NULL,
    params      jsonb NOT NULL DEFAULT '{}'::jsonb,
    expiry      bigint NOT NULL,
    state       text NOT NULL,
    created_at  timestamptz NOT NULL DEFAULT now(),
    updated_at  timestamptz NOT NULL DEFAULT now()
);
CREATE INDEX IF NOT EXISTS device_command_node_created
    ON device_command (node_id, created_at DESC);
CREATE INDEX IF NOT EXISTS device_command_open
    ON device_command (state) WHERE state IN ('PENDING', 'ACCEPTED');
```

A `device_command` a 17.1 labor memóriabeli `ConcurrentHashMap` tárolóját váltja fel. Az `id` szöveges UUID, a `params` JSONB, az `expiry` a DTO-hoz hasonlóan UTC epoch ms. A `CommandService` minden állapotátmenetnél ugyanabban a tranzakcióban frissíti a táblát; a `CommandController` a `POST /api/devices/{nodeId}/commands` válaszában a perzisztált rekordot adja vissza. A végleges `DeviceCommand` DTO a `site` mezővel bővül, hogy a több telephelyes üzemnapló egyértelmű legyen. A repository frissítése:

```
public void updateState(String id, CommandState state) {
    jdbc.update("""
```

```
        UPDATE device_command
           SET state = ?, updated_at = now()
         WHERE id = ?
        """, state.name(), id);
}
```

004_alert.sql:

```
CREATE TABLE IF NOT EXISTS alert_config (
    name      text PRIMARY KEY,
    threshold interval NOT NULL
);

INSERT INTO alert_config (name, threshold)
VALUES ('no_data', interval '10 minutes')
ON CONFLICT (name) DO NOTHING;

CREATE OR REPLACE VIEW missing_data_alert AS
SELECT d.site,
       d.node AS node_id,
       now() AS alert_ts,
       extract(epoch FROM now() -
           coalesce(max(t.server_ts), d.provisioned_at)) AS missing_seconds,
       (now() - coalesce(max(t.server_ts), d.provisioned_at)) >
           (SELECT threshold FROM alert_config WHERE name = 'no_data') AS alerting
FROM devices d
LEFT JOIN telemetry t
       ON t.site = d.site AND t.node_id = d.node
GROUP BY d.site, d.node, d.provisioned_at;
```

A nézet a `devices` táblából indul ki, ezért olyan regisztrált eszközt is jelez, amely soha nem küldött telemetriát; a `coalesce` ilyenkor a `provisioned_at` időpontot használja. A termelési küszöb 10 perc, a tesztprofil 60 másodpercre állítja. A `005_seed.sql` a `gh1/n001`–`gh1/n003` eszközöket regisztrálja, valamint a soha nem indított `gh1/n004` és `gh1/n005` eszközöket; a `device_config` és a `device_state` sorok is létrejönnek.

23.1.5.3 A végleges Grafana-provisioning

A `grafana-provisioning/datasources/datasources.yml` a végleges compose szolgáltatásnevére és a végleges adatbázis-felhasználóra mutat:

```
apiVersion: 1

datasources:
  - name: TerraSense TimescaleDB
    type: postgres
    access: proxy
    uid: terrasense-ds
    url: timescaledb:5432
    database: terrasense
    user: terrasense
    secureJsonData:
      password: lab_password
    jsonData:
      sslmode: disable
      postgresVersion: 1600
      timescaledb: true
```

A dashboard a korábbi `terrasense-lab` UID-t és a `telemetry` táblát használja, a `server_ts` és `ts` mezők megkülönböztetésével. A `grafana-provisioning/alerting/terrasense-alerts.yml` egyetlen, flottaszintű szabályt definiál:

```
apiVersion: 1

contactPoints:
  - orgId: 1
    name: lab-webhook
    receivers:
      - uid: lab-webhook
        type: webhook
        settings:
          url: http://webhook-sink:8081/alert
          httpMethod: POST

groups:
  - orgId: 1
    name: terrasense-no-data
    folder: TerraSense
    interval: 1m
    rules:
      - uid: ts-no-data
        title: Nincs telemetria adat
        condition: B
        data:
          - refId: A
            datasourceUid: terrasense-ds
            model:
              rawSql: |
                SELECT site, node_id, missing_seconds
                FROM missing_data_alert
                WHERE alerting
              format: table
          - refId: B
            datasourceUid: __expr__
            model:
              type: threshold
              expression: A
              conditions:
                - evaluator:
                    type: gt
                    params: [0]
        noDataState: OK
        execErrState: Error
        for: 0m
        labels:
          severity: critical
```

A szabály `noDataState: OK` beállítása szándékos: a nézet csak a hiányzó eszközökre ad sort, ezért az üres eredmény nem hiba, hanem egészséges flotta. A `webhook_sink.py` a `POST` törzset a `/data/alerts.log` fájlba írja, és `200 OK`-val válaszol; a T5 teszt ebből a naplóból igazolja, hogy a riasztás ténylegesen eljutott az értesítési pontig, nem csak az SQL-nézetben jelent meg. **Ellenőrzési igény.** A Grafana 11.2 alerting provisioning YAML-sémája verziófüggő; a labor előtt a `grafana/grafana:11.2.0` image dokumentációjában ellenőrizendő.

A `webhook_sink.py` a riasztási értesítés fogadója. A naplófájl a `alert-log` kötetben marad meg, ezért a teszt a konténer újraindítása után is ellenőrizheti:

```
import http.server
```

```
class Handler(http.server.BaseHTTPRequestHandler):
    def do_POST(self):
        length = int(self.headers.get("Content-Length", "0"))
        body = self.rfile.read(length)
        with open("/data/alerts.log", "ab") as log:
            log.write(body + b"\n")
        self.send_response(200)
        self.end_headers()
        self.wfile.write(b"ok")

    def log_message(self, fmt, *args):
        pass


http.server.HTTPServer(("0.0.0.0", 8081), Handler).serve_forever()
```

A `webhook-sink` nem része a termelési értesítési láncnak; a laborban a Grafana webhook-kimenetét helyettesíti, hogy a riasztás tényleges kézbesítését is igazolni lehessen. Termelésben ezt e-mail-, Slack- vagy incidenskezelő integráció váltja fel, amelynek elérhetőségét és hitelesítését külön kell üzemeltetni.

A `ts-gw` végleges SQLite spool táblája a 13.2 labor `(node_id, seq)` kulcsát `(site, node_id, boot_id, seq)` kulcsra bővíti, hogy egy újraindult node `seq=1` üzenete ne ütközzön a korábbi boot rekordjával:

```
CREATE TABLE IF NOT EXISTS spool (
    id      INTEGER PRIMARY KEY AUTOINCREMENT,
    site    TEXT NOT NULL,
    node_id TEXT NOT NULL,
    boot_id TEXT NOT NULL,
    seq     INTEGER NOT NULL,
    ts      INTEGER NOT NULL,
    topic   TEXT NOT NULL,
    payload TEXT NOT NULL,
    UNIQUE (site, node_id, boot_id, seq)
);
```

A spool `ts` mezője az esemény UTC epoch ms értéke, nem a feldolgozás ideje. A visszatöltés továbbra is `id` szerint, a beérkezés sorrendjében történik; a `seq` csak a deduplikációt és a diagnosztikát szolgálja.

A `~/iot-labor/i066/ts_node_sim.py` szimulátor a végleges JSON-szerződést küldi. A `boot_id` minden indításkor új, a `seq` nulláról indul, a `valve.open` parancsra `ACCEPTED`, majd a biztonsági idő letelte után `SUCCESS` nyugtát küld. A `duration_ms` értékét a `MAX_VALVE_OPEN_S` korlátozza; ez a szelep biztonsági zárása. Egyszerre legfeljebb egy aktív parancs lehet, ezért a backend a `PENDING` vagy `ACCEPTED` állapotú parancs idején újabbat `409 Conflict`-tal utasít el.

```
import json
import os
import random
import socket
import time
import uuid

SITE = os.environ.get("SITE", "gh1")
NODE_ID = os.environ.get("NODE_ID", "n001")
BOOT_ID = os.environ.get("BOOT_ID") or uuid.uuid4().hex[:12]
GW_HOST = os.environ.get("GW_HOST", "ts-gw")
GW_PORT = int(os.environ.get("GW_PORT", "9000"))
LISTEN_PORT = int(os.environ.get("LISTEN_PORT", "9000"))
MAX_VALVE_OPEN_S = int(os.environ.get("MAX_VALVE_OPEN_S", "60"))
```

```
TELEMETRY_PERIOD_S = float(os.environ.get("TELEMETRY_PERIOD_S", "10"))

sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
sock.bind(("0.0.0.0", LISTEN_PORT))
sock.settimeout(0.5)

seq = 0
valve_open = False
valve_deadline = 0.0
pending_command = None


def measurement(value, unit):
    return {"value": round(value, 2), "unit": unit, "valid": True, "reason": None}


def send_json(payload):
    sock.sendto(json.dumps(payload, separators=(",", ":")).encode("utf-8"),
                (GW_HOST, GW_PORT))


def telemetry_payload():
    return {
        "kind": "telemetry",
        "schema_version": 1,
        "site": SITE,
        "node_id": NODE_ID,
        "boot_id": BOOT_ID,
        "seq": seq,
        "ts": int(time.time() * 1000),
        "measurements": {
            "temperature": measurement(random.uniform(18.0, 28.0), "Cel"),
            "humidity": measurement(random.uniform(45.0, 75.0), "%RH"),
            "soil_moisture": measurement(random.uniform(20.0, 60.0), "%"),
        },
        "actuators": {"valve_open": valve_open},
    }


def send_state():
    send_json({
        "kind": "state",
        "site": SITE,
        "node_id": NODE_ID,
        "boot_id": BOOT_ID,
        "seq": seq,
        "ts": int(time.time() * 1000),
        "valve_open": valve_open,
    })


def send_result(command_id, status, result):
    send_json({
        "id": command_id,
        "site": SITE,
        "node_id": NODE_ID,
        "status": status,
        "result": result,
        "ts": int(time.time() * 1000),
    })


def handle_command(command):
```

```
    global valve_open, valve_deadline, pending_command
    command_id = command.get("id")
    if not command_id:
        return
    command_type = command.get("type")
    params = command.get("params") or {}
    if command_type == "valve.open":
        duration_ms = min(int(params.get("duration_ms", 5000)),
                          MAX_VALVE_OPEN_S * 1000)
        send_result(command_id, "ACCEPTED", {"duration_ms": duration_ms})
        valve_open = True
        valve_deadline = time.monotonic() + duration_ms / 1000.0
        pending_command = (command_id, int(time.time() * 1000), duration_ms)
        send_state()
    elif command_type == "valve.close":
        valve_open = False
        pending_command = None
        send_state()
        send_result(command_id, "SUCCESS", {"valve_open": False})
    else:
        send_result(command_id, "FAILED", {"reason": "unsupported command type"})


last_telemetry = 0.0
while True:
    now = time.monotonic()
    if now - last_telemetry >= TELEMETRY_PERIOD_S:
        seq += 1
        send_json(telemetry_payload())
        last_telemetry = now

    if valve_open and now >= valve_deadline:
        valve_open = False
        send_state()
        if pending_command is not None:
            command_id, opened_ms, duration_ms = pending_command
            send_result(command_id, "SUCCESS", {
                "valve_open": False,
                "valve_opened_ms": int(time.time() * 1000) - opened_ms,
            })
            pending_command = None

    try:
        data, _ = sock.recvfrom(4096)
        handle_command(json.loads(data))
    except socket.timeout:
        pass
```

A szimulátor a valós eszköz négy fontos tulajdonságát tartja meg: a `seq` a booton belül monoton, a `ts` UTC epoch ms, a parancsra adott válasz a parancs `id`-jére hivatkozik, a szelep pedig a megadott idő után magától zár. Ha a folyamat a szelep nyitva léte alatt leáll, a szimulált állapot elveszik; ezért a hardveres fail-safe zárást a mátrix külön, nem igazolt tételként kezeli. A `send_state()` a szelepállapot-változást azonnal a `state` topicra továbbítja, így az eszközárnyék reported mezője nem vár a következő telemetriára.

Az átvételi tesztek a `~/iot-labor/i066/acceptance_test.sh` fájlban:

```
#!/usr/bin/env bash
set -euo pipefail

STACK_DIR="${STACK_DIR:-$HOME/iot-labor/stack}"
I066_DIR="${I066_DIR:-$HOME/iot-labor/i066}"
```

```
CERT_DIR="$IOT66_DIR/certs"
TMP_DIR="$(mktemp -d)"
trap 'rm -rf "$TMP_DIR"' EXIT
cd "$STACK_DIR"

PSQL=(docker compose exec -T timescaledb psql -U terrasense -d terrasense -At -v
↪  ON_ERROR_STOP=1)

fail() { echo "FAIL: $*" >&2; exit 1; }
pass() { echo "PASS: $*"; }
scalar() { "${PSQL[@]}" -c "$1" | tr -d '[:space:]'; }
json_field() { python3 -c 'import json,sys; print(json.load(sys.stdin)[sys.argv[1]])' "$1"; }

wait_for_sql() {
  local query="$1" expected="$2" tries="${3:-30}" value=""
  for _ in $(seq 1 "$tries"); do
    value=$(scalar "$query" || true)
    [ "$value" = "$expected" ] && return 0
    sleep 2
  done
  return 1
}

need() {
  command -v "$1" >/dev/null 2>&1 || fail "hiányzó eszköz: $1"
}

need docker
need curl
need python3
need openssl
need mosquitto_pub
need awk

echo "== TerraSense átvételi tesztek (tesztprofil) =="

for svc in mosquitto timescaledb ts-backend ts-gw ts-node-n001 ts-node-n002 ts-node-n003 grafana
↪  webhook-sink; do
  docker compose ps --status running --services | grep -qx "$svc" || fail "T0: a(z) $svc
  ↪  szolgáltatás nem fut"
done
pass "T0 minden szolgáltatás fut"

wait_for_sql "SELECT count(DISTINCT node_id) FROM telemetry WHERE site='gh1';" "3" 30 \
  || fail "T1: nem érkezett mindhárom node telemetriája"
UNITS=$(scalar "SELECT count(*) FROM telemetry WHERE unit IS NULL OR unit = '';")
[ "$UNITS" = "0" ] || fail "T1: hiányzó mértékegység"
COMPLETE=$(scalar "SELECT count(DISTINCT node_id) FROM (
  SELECT site, node_id, boot_id, seq
  FROM telemetry WHERE site='gh1'
  GROUP BY 1,2,3,4
  HAVING count(DISTINCT measurement_type) = 3
) x;")
[ "$COMPLETE" = "3" ] || fail "T1: nincs három mérést hordozó esemény minden node-tól"
pass "T1 telemetria adatút (3 node, 3 mérés, mértékegységek)"

COMMAND_JSON=$(curl --fail-with-body -sS -X POST \
  "http://localhost:8080/api/devices/n001/commands" \
  -H 'Content-Type: application/json' \
  -d '{"type":"valve.open","params":{"duration_ms":15000},"ttl_ms":30000}')
COMMAND_ID=$(printf '%s' "$COMMAND_JSON" | json_field id)
sleep 5
SHADOW=$(curl --fail-with-body -sS "http://localhost:8080/api/shadows/gh1/n001")
```

```
REPORTED=$(printf '%s' "$SHADOW" | json_field reported)
[ "$REPORTED" = "OPEN" ] || fail "T2: a jelentett szelepállapot nem OPEN ($REPORTED)"
STATE=""
for _ in $(seq 1 15); do
  STATE=$(scalar "SELECT state FROM device_command WHERE id='$COMMAND_ID';")
  [ "$STATE" = "SUCCESS" ] && break
  sleep 2
done
[ "$STATE" = "SUCCESS" ] || fail "T2: a parancs nem került SUCCESS állapotba ($STATE)"
sleep 5
SHADOW=$(curl --fail-with-body -sS "http://localhost:8080/api/shadows/gh1/n001")
REPORTED=$(printf '%s' "$SHADOW" | json_field reported)
[ "$REPORTED" = "CLOSED" ] || fail "T2: a szelep nem záródott be a biztonsági idő után
↪  ($REPORTED)"
pass "T2 parancs, nyugta és szelep-biztonsági zárás"

VERSION=$(printf '%s' "$SHADOW" | json_field version)
curl --fail-with-body -sS -X PUT "http://localhost:8080/api/shadows/gh1/n001/desired" \
  -H 'Content-Type: application/json' \
  -d "{\"valveState\":\"CLOSED\",\"expectedVersion\":$VERSION}" >/dev/null
sleep 5
SHADOW=$(curl --fail-with-body -sS "http://localhost:8080/api/shadows/gh1/n001")
printf '%s' "$SHADOW" | python3 -c 'import json,sys; s=json.load(sys.stdin); assert s["desired"]
↪  == "CLOSED" and s["reported"] == "CLOSED", s'
pass "T2b eszközárnyék desired/reported"

BEFORE_SEQ=$(scalar "SELECT coalesce(max(seq), 0) FROM telemetry WHERE site='gh1' AND
↪  node_id='n001';")
BOOT_ID=$(scalar "SELECT boot_id FROM telemetry WHERE site='gh1' AND node_id='n001' ORDER BY
↪  server_ts DESC LIMIT 1;")
[ -n "$BOOT_ID" ] || fail "T3: nincs boot_id"
docker compose stop mosquitto >/dev/null
sleep 35
docker compose start mosquitto >/dev/null
sleep 25
AFTER_SEQ=$(scalar "SELECT coalesce(max(seq), 0) FROM telemetry WHERE site='gh1' AND
↪  node_id='n001' AND boot_id='$BOOT_ID';")
[ "$AFTER_SEQ" -gt "$BEFORE_SEQ" ] || fail "T3: a seq nem nőtt ($BEFORE_SEQ -> $AFTER_SEQ)"
MISSING=$(scalar "SELECT count(*) FROM generate_series($BEFORE_SEQ + 1, $AFTER_SEQ) AS s
  WHERE NOT EXISTS (
    SELECT 1 FROM telemetry
    WHERE site='gh1' AND node_id='n001' AND boot_id='$BOOT_ID' AND seq = s
  );")
[ "$MISSING" = "0" ] || fail "T3: $MISSING hiányzó seq a visszatöltés után"
BAD=$(scalar "SELECT count(*) FROM (
  SELECT seq FROM telemetry
  WHERE site='gh1' AND node_id='n001' AND boot_id='$BOOT_ID'
    AND seq BETWEEN $BEFORE_SEQ + 1 AND $AFTER_SEQ
  GROUP BY seq HAVING count(*) <> 3
) x;")
[ "$BAD" = "0" ] || fail "T3: duplikált vagy hiányos mérési sorok"
SPOOL_LEFT=$(docker compose exec -T ts-gw sqlite3 /var/lib/ts-gw/spool.db 'SELECT count(*) FROM
↪  spool;' | tr -d '[:space:]')
[ "$SPOOL_LEFT" = "0" ] || fail "T3: a puffer nem ürült ki ($SPOOL_LEFT)"
pass "T3 offline puffer és hiánytalan visszatöltés"

openssl s_client -connect localhost:8883 -servername mosquitto \
  -CAfile "$CERT_DIR/lab-ca.crt" -cert "$CERT_DIR/ts-gw.crt" -key "$CERT_DIR/ts-gw.key" \
  -verify_return_error -verify_hostname mosquitto </dev/null >"$TMP_DIR/tls-ok.out"
  ↪  2>"$TMP_DIR/tls-ok.err" \
  || fail "T4: a pozitív mTLS kapcsolat nem jött létre"
grep -q "Verify return code: 0 (ok)" "$TMP_DIR/tls-ok.out" || fail "T4: a tanúsítványlánc nem
↪  ellenőrizhető"
```

```
openssl s_client -connect localhost:8883 -servername mosquitto \
  -CAfile "$CERT_DIR/lab-ca.crt" -verify_return_error </dev/null >"$TMP_DIR/tls-nocert.out"
   ↪  2>"$TMP_DIR/tls-nocert.err" || true
grep -qiE 'certificate required|handshake failure|alert' "$TMP_DIR/tls-nocert.out"
 ↪  "$TMP_DIR/tls-nocert.err" \
  || fail "T4: a kliens-tanúsítvány nélküli kapcsolat nem a várt módon utasítódott el"

openssl s_client -connect localhost:8883 -servername mosquitto \
  -CAfile /etc/ssl/certs/ca-certificates.crt -cert "$CERT_DIR/ts-gw.crt" -key
   ↪  "$CERT_DIR/ts-gw.key" \
  -verify_return_error </dev/null >"$TMP_DIR/tls-wrongca.out" 2>"$TMP_DIR/tls-wrongca.err" ||
   ↪  true
grep -qiE 'unable to get local issuer|verify error|self-signed' "$TMP_DIR/tls-wrongca.out"
 ↪  "$TMP_DIR/tls-wrongca.err" \
  || fail "T4: a rossz CA-val végzett kapcsolat nem a várt módon utasítódott el"

openssl s_client -connect localhost:8883 -servername mosquitto \
  -CAfile "$CERT_DIR/lab-ca.crt" -cert "$CERT_DIR/ts-gw.crt" -key "$CERT_DIR/ts-gw.key" \
  -verify_return_error -verify_hostname wronghost </dev/null >"$TMP_DIR/tls-host.out"
   ↪  2>"$TMP_DIR/tls-host.err" || true
grep -qiE 'hostname mismatch|does not match|IP address mismatch' "$TMP_DIR/tls-host.out"
 ↪  "$TMP_DIR/tls-host.err" \
  || fail "T4: a rossz hosztnév nem a várt módon utasítódott el"

if timeout 2 bash -c '</dev/tcp/localhost/1883' 2>/dev/null; then
  fail "T4: a titkosítatlan 1883-as port elérhető"
fi

ACL_OUT=$(mosquitto_pub -h localhost -p 8883 \
  --cafile "$CERT_DIR/lab-ca.crt" --cert "$CERT_DIR/ts-gw.crt" --key "$CERT_DIR/ts-gw.key" \
  -t 'terrasense/gh1/n001/cmd' -m '{}' 2>&1) && fail "T4: a ts-gw jogosulatlan PUBLISH-t tudott
   ↪  küldeni"
printf '%s' "$ACL_OUT" | grep -qiE 'not authorised|not authorized' \
  || docker compose logs --since 30s mosquitto 2>&1 | grep -q 'Denied PUBLISH' \
  || fail "T4: az ACL-teszt nem a várt elutasítást kapta: $ACL_OUT"
pass "T4 mTLS, hosztnév, lezárt 1883 és ACL"

"${PSQL[@]}" -c "UPDATE alert_config SET threshold = interval '60 seconds' WHERE name =
 ↪  'no_data';" >/dev/null
docker compose stop ts-node-n003 >/dev/null
sleep 65
N004=$(scalar "SELECT count(*) FROM missing_data_alert WHERE alerting AND node_id = 'n004';")
N003=$(scalar "SELECT count(*) FROM missing_data_alert WHERE alerting AND node_id = 'n003';")
[ "$N004" = "1" ] || fail "T5: a soha nem jelentkező n004 nem szerepel a riasztásban"
[ "$N003" = "1" ] || fail "T5: a leállított n003 nem szerepel a riasztásban"
ALERT_SEEN=0
for _ in $(seq 1 12); do
  if docker compose exec -T webhook-sink grep -q 'n004' /data/alerts.log 2>/dev/null; then
    ALERT_SEEN=1
    break
  fi
  sleep 10
done
[ "$ALERT_SEEN" = "1" ] || fail "T5: a Grafana riasztás nem érkezett meg a webhook-sinkhez"
"${PSQL[@]}" -c "UPDATE alert_config SET threshold = interval '10 minutes' WHERE name =
 ↪  'no_data';" >/dev/null
docker compose start ts-node-n003 >/dev/null
pass "T5 hiányzó adat riasztás és webhook-értesítés"

SPOOL_COUNT=$(docker compose exec -T ts-gw sh <<'SH' | tr -d '[:space:]'
cp /var/lib/ts-gw/spool.db /tmp/spool-test.db
sqlite3 /tmp/spool-test.db <<'SQL'
```

```
DELETE FROM spool;
WITH RECURSIVE n(value) AS (
  SELECT 1
  UNION ALL
  SELECT value + 1 FROM n WHERE value < 25
)
INSERT INTO spool(site, node_id, boot_id, seq, ts, topic, payload)
SELECT 'gh1', 'n001', 'test-boot', value, value, 't', '{}' FROM n;
DELETE FROM spool WHERE id IN (
  SELECT id FROM spool ORDER BY id LIMIT (SELECT MAX(0, COUNT(*) - 20) FROM spool)
);
SELECT count(*) FROM spool;
SQL
SH
)
[ "$SPOOL_COUNT" = "20" ] || fail "T6: a spool korlát nem 20 rekordot hagyott ($SPOOL_COUNT)"
CAP=$(docker compose exec -T ts-gw printenv TS_GW_SPOOL_MAX_RECORDS | tr -d '[:space:]')
[ "$CAP" = "181440" ] || fail "T6: a termelési spool-korlát nem 181440 ($CAP)"
pass "T6 spool-korlát politika és termelési kapacitás"

"${PSQL[@]}" -c "SELECT node_id,
       round(avg(extract(epoch FROM (server_ts - ts)) * 1000)::numeric, 1) AS avg_latency_ms,
       count(DISTINCT seq) AS events
FROM telemetry
WHERE server_ts > now() - interval '10 minutes'
GROUP BY node_id
ORDER BY node_id;"
NEGATIVE=$(scalar "SELECT count(*) FROM telemetry
  WHERE server_ts > now() - interval '10 minutes'
    AND extract(epoch FROM (server_ts - ts)) * 1000 < 0;")
[ "$NEGATIVE" = "0" ] || fail "T7: negatív késleltetés, valószínű órahiba"
DELIVERY=$(scalar "SELECT coalesce(min(delivery_pct), 0) FROM (
  SELECT site, node_id, boot_id,
         round(100.0 * count(DISTINCT seq) / (max(seq) - min(seq) + 1), 3) AS delivery_pct
  FROM telemetry
  WHERE server_ts > now() - interval '10 minutes'
  GROUP BY 1,2,3
  HAVING max(seq) > min(seq)
) x;")
awk -v d="$DELIVERY" 'BEGIN { exit !(d >= 99.9) }' || fail "T7: a kézbesítés $DELIVERY%,
↪  kevesebb 99,9%-nál"
pass "T7 késleltetés és eseményalapú kézbesítés ($DELIVERY%)"

echo "== Minden átvételi teszt PASS =="
```

A `missing_data_alert` nézet nem része a teszt szkriptnek: a `schema/004_alert.sql` init szkript hozza létre, a `alert_config` táblával együtt. A teszt csak a `threshold` értékét állítja át 60 másodpercre, majd vissza 10 percre; a nézet definícióját nem módosítja.

Indítás:

```
cd ~/iot-labor/stack
docker compose up -d --build
cd ~/iot-labor/i066
chmod +x acceptance_test.sh
./acceptance_test.sh
```

A labor után a takarítás:

```
cd ~/iot-labor/stack
docker compose down -v
```

A `-v` kapcsoló a `timescale-data`, a `gw-store` és az `alert-log` köteteket is törli, ezért csak a záróprojekt laborja után használjuk.

23.1.6 Ellenőrzés és hibakeresés

A tesztek futtatása előtt érdemes a `docker compose ps` kimenetét ellenőrizni. Minden szolgáltatásnak `Up` vagy `healthy` állapotban kell lennie.

Tünet	Valószínű ok	Javítás
A `ts-gw` nem csatlakozik a brokerhez	rossz CA-útvonal, hiányzó kliens-tanúsítvány vagy ACL	`docker compose logs ts-gw`, ellenőrizze a `/certs/lab-ca.crt`, `/certs/ts-gw.crt` és `/certs/ts-gw.key` mountot, valamint a `ts-gw` ACL-sorait
A `ts-backend` SSL-hibát jelez a broker felé	a `mosquitto.crt` SAN-ja nem tartalmazza a `mosquitto` nevet	állítsa ki újra a szervertanúsítványt `DNS:mosquitto,DNS:localhost,IP:127.0.0` SAN-nal, majd indítsa újra a Mosquittót
A backend nem éri el a TimescaleDB-t	a healthcheck még nem futott le, vagy a séma init nem futott le	`docker compose logs timescaledb`, majd üres köteten `docker compose down -v` és újraindítás; az init csak üres adatkönyvtáron fut
A T2 parancs mindig `PENDING`	a node nem kapja meg a parancsot, vagy a gateway nem iratkozott fel a `cmd` topicra	`docker compose logs ts-gw` és `docker compose logs ts-node-n001`; ellenőrizze a `cmd` ACL `read` jogát és az UDP-portot
A T2 árnyék `reported` értéke nem változik	a gateway nem publikál a `state` topicra, vagy a backend nem iratkozott fel rá	ellenőrizze a `state` topicot, a `retain=true` beállítást és a backend MQTT-feliratkozását
A Grafana nem mutat adatot	rossz adatforrás UID, hosztnév vagy felhasználó	`docker compose logs grafana`; a datasource `uid: terrasense-ds`, `url: timescaledb:5432`, `user: terrasense`, `database: terrasense` legyen
A T3 nem növeli a `seq`-et	a ts-gw puffer nem perzisztens, a spool kötete nem írható, vagy a broker leállítása alatt a node nem küldött	ellenőrizze a `gw-store` kötetet, a `tsgw` jogosultságát és a script időzítését
A T5 nem küld webhookot	a Grafana szabály nem értékel, rossz a contact point URL, vagy a nézet küszöbe nem 60 s	ellenőrizze a `terrasense-alerts.yml` fájlt, a `http://webhook-sink:8081/alert` URL-t és az `alert_config` sorát

23.1.7 Mérés: végponttól végpontig késleltetés és veszteség

A méréshez a `ts_node_sim.py` szimulátor a `ts` mezőben rögzíti az esemény idejét, a backend ingestje pedig az adatbázisba írás pillanatában a `server_ts` mezőt tölti ki. A kettő szándékosan külön mező: a `ts` az eseményidő, a `server_ts` a feldolgozási idő. A késleltetés:

$$\Delta t = \text{server_ts} - \text{ts}$$

Mivel mindkét mező `timestamptz`, a különbség `interval`; ezért a milliszekundumos értékhez `extract(epoch ...)` kell, és a kapott másodpercet 1000-rel szorozzuk:

```
SELECT node_id,
       round(avg(extract(epoch FROM (server_ts - ts)) * 1000)::numeric, 1)
           AS avg_latency_ms,
       count(DISTINCT seq) AS events
FROM telemetry
WHERE server_ts > now() - interval '10 minutes'
GROUP BY node_id
ORDER BY node_id;
```

A `count(DISTINCT seq)` azért szerepel, mert egy eseményhez három mérési sor tartozik; a `count(*)` háromszor annyi „üzenetet” mutatna, mint amennyi valójában érkezett. A negatív késleltetés nem gyors kézbesítést, hanem órahibát jelez: az eszköz `ts` értéke a szerver `server_ts` értéke után van. A teszt ezért külön ellenőrzi, hogy nincs negatív érték.

A veszteség becsléséhez a `seq` értékeket `(site, node_id, boot_id)` csoportonként hasonlítjuk a tárolt eseményekhez:

```
SELECT site, node_id, boot_id,
       count(DISTINCT seq) AS stored_events,
       max(seq) - min(seq) + 1 AS expected_events,
       round(100.0 * count(DISTINCT seq)
             / (max(seq) - min(seq) + 1), 3) AS delivery_pct
FROM telemetry
WHERE server_ts > now() - interval '10 minutes'
GROUP BY site, node_id, boot_id;
```

A `boot_id` szerinti csoportosítás azért kell, mert egy újraindult node `seq` értéke nulláról indul; a bootok összevonása negatív vagy értelmezhetetlen `expected_events` értéket adhat. A rövid laborablakban a `delivery_pct` jellemzően 100%, de ez nem bizonyítja a hétnapos, 99,9%-os üzemi követelményt; a mátrix ezt külön, nyitott tételként kezeli.

Hétnapos pufferkapacitás. Három node 10 másodpercenként egy eseményt küld, ezért naponta $3 \cdot 8640 = 25\,920$ esemény keletkezik. Hét napra $7 \cdot 25\,920 = 181\,440$ esemény kell. A spool egy rekordja egy MQTT-üzenet (egy esemény), nem három adatbázissor; a 180 B-os JSON-nal a nyers pufferigény $181\,440 \cdot 180\,\text{B} \approx 32{,}7\,\text{MB}$. A `TS_GW_SPOOL_MAX_RECORDS=181440` ezt a darabszámot korlátozza; a kötetet ennél nagyobbra, például 128 MB-ra kell méretezni, hogy az SQLite overhead és a WAL is elférjen. A labor a korlát politikáját 20 rekordos helyi példányon ellenőrzi, a hétnapos üzemet nem futtatja.

23.1.8 Követelmény-nyomonkövetési mátrix

A záróprojekt átvétele nem azonos a Must követelmények automatikus teljesítésével. Az alábbi mátrix a követelményt a felelős komponenshez, az átvételi teszthez és a bizonyítékhoz rendeli, és a fennmaradó nyitott tételeket külön kezeli. A „részben igazolt” azt jelenti, hogy a mechanizmus vagy a laborváltozat működik, de a termelési követelmény (hét nap, hardver, valós flotta) nem igazolt. A táblázat állapotai az átvételi tesztek elfogadási feltételei; a lektoráló környezetben a stack nem futott, ezért a tesztblokkok tényleges lefutása a labor végrehajtásának eredménye.

ID	Must követelmény	Komponens	Teszt/bizonyíték	Állapot
M1	A telemetria három node-tól, három méréssel eljut az adatbázisig	ts-node, ts-gw, Mosquitto, ts-backend, TimescaleDB	T1	igazolt (labor)
M2	A parancs végigfut a REST → MQTT → UDP → node → `cmd/result` úton, és az állapotgép perzisztált	ts-backend, Mosquitto, ts-gw, ts-node, `device_command`	T2	igazolt (labor)
M3	Az eszközárnyék desired/reported állapota konzisztens	`ShadowService`, `state` topic	T2, T2b	részben igazolt: a reported a `state` topicból frissül, de az árnyék memóriabeli, újraindításkor elveszik
M4	A gateway offline pufferel és hiánytalanul visszatölt	ts-gw, SQLite spool, Mosquitto	T3	igazolt (35 s-os laborablak)
M5	A puffer hétnapos kapacitása és a korlát politikája	ts-gw spool, `TS_GW_SPOOL_MAX_RECORDS`	T6, számítás	részben igazolt: a drop-oldest politika és a 181 440-es korlát igazolt, a hétnapos üzem nem
M6	A kézbesítés legalább 99,9%-os	teljes adatút	T7	részben igazolt: a rövid laborablakban elvárt érték; hétnapos üzemi igazolás nyitott

ID	Must követelmény	Komponens	Teszt/bizonyíték	Állapot
M7	A szelep hiba esetén biztonságosan zár	ts-node szimulátor, `MAX_VALVE_OPEN_S`	T2	részben igazolt: a szoftveres időzítő zár, a hardveres fail-safe nem
M8	Az eszközök egyedi identitással és mTLS-sel kapcsolódnak	Mosquitto, ts-gw, ts-backend, `certs/`	T4	részben igazolt: a gateway és a backend mTLS-e igazolt; a node UDP-szimulátor, ezért node-X.509 nincs
M9	A hiányzó adat riasztást és értesítést vált ki	`missing_data_alert` nézet, Grafana, webhook-sink	T5	igazolt (nézet + webhook)
M10	Az OTA A/B frissítés és rollback működik	OTA-frissítés: A/B képek, rollback, delta, fokozatos kiadás modell (a stackben nincs OTA-komponens)	OTA-frissítés: A/B képek, rollback, delta, fokozatos kiadás modell	nem igazolt: a záróprojekt stack nem tartalmaz OTA-t, csak a modell szimulációja
M11	A REST végpont jogosultság-ellenőrzött	ts-backend REST	—	nem igazolt: a labor REST-je hitelesítés nélkül, csak `127.0.0.1:8080`-ra publikálva érhető el; termelésben kötelező
M12	A teljes stack reprodukálható	`compose.yaml`, `~/iot-labor/i066/` források	T0, `docker compose up -d --build`	igazolt (üres köteten)
M13	Az energiafogyasztás és az elemélettartam a követelményen belül van	nincs valós hardver	—	nem igazolt: a `native_sim`/UDP szimulátor nem modellez energia-fogyasztást; hardveres mérés szükséges

A mátrix a záróprojekt átvételi feltételeit rögzíti; a „nem igazolt” sorok nem hibaüzenetek, hanem a következő hardveres vagy üzemi fázis tételei. A záróprojekt akkor tekinthető lezártnak, ha minden „igazolt” sor zöld, a „részben igazolt” sorokhoz pedig dokumentált üzemeltetési terv tartozik. A REST-hitelesítés hiánya tudatos labor-kompromisszum: a port csak a hoszt loopback

interfészére van publikálva, de ez nem helyettesíti a termelési azonosítást és jogosultságkezelést.

23.1.9 Tipikus hibák és félreértések

1. **A titkosítatlan 1883-as port nyitva marad.** A Biztonságos kapcsolat alfejezetben már lezártuk, de ha a `compose.yaml` véletlenül publikálja, a T4 teszt ezt azonnal jelzi. A TLS-teszt csak akkor értékes, ha tényleges kapcsolódási kísérletet hajtunk végre, nem csak a konfigurációt olvassuk el.
2. **Az offline tesztben túl rövid várakozás.** Ha a brokert csak 5–10 másodpercre állítjuk le, előfordulhat, hogy a `ts-gw` ki sem érzékeli a kapcsolat megszakadását, mert a Paho kliens belső újracsatlakozási időzítője tovább próbálkozik. A 35 másodperc ebben a laborban tudatosan hosszabb, mint a 10 másodperces telemetria-intervallum.
3. **A parancs eredményét közvetlenül a REST válaszból várjuk.** A parancs feldolgozása aszinkron: a backend a `POST` kérésre legfeljebb a parancs elfogadását tudja visszaadni, a tényleges nyugta a `cmd/result` úton érkezik. Ezért kell a tesztben SQL-lel ellenőrizni az állapotgépet.
4. **Az idempotencia elhanyagolása.** A T3 visszatöltés során a ts-gw újraküldhet olyan üzeneteket, amelyeket a backend már látott. A végleges egyedi kulcs `(site, node_id, boot_id, seq, measurement_type, ts)`: a `node_id + seq` önmagában nem elég, mert egy újraindult node `seq`-je nulláról indul, és két telephelyen ugyanaz a rövid node-név is előfordulhat. A hosszú táblában egy eseményhez három mérési sor tartozik, ezért a kézbesítési statisztikában `count(DISTINCT seq)` szerepel, nem `count(*)`.

23.1.10 Feladatok

1. **Fogalmi, döntési kérdés.** Miért nem elég a T1 tesztben csak a `ts-gw` MQTT-üzeneteit figyelni? Milyen hiba maradhat rejtve, ha az adatbázist nem ellenőrizzük?
2. **Számolási feladat.** A záróprojektben három node 10 másodpercenként küld telemetriát. Hány telemetriaüzenet keletkezik naponta? Ha minden üzenet JSON-mérete 180 B, mekkora a napi nyers adatmennyiség?
3. **Implementációs feladat.** Bővítse az `acceptance_test.sh`-t egy további `ts-node-n006` szolgáltatással, és módosítsa a T1 tesztet úgy, hogy négy node-ot várjon. Miért nem alkalmas erre a `n004` azonosító?
4. **Hibakeresési feladat.** A T2 teszt során a parancs `TIMEOUT` állapotba kerül, pedig a node fut. Soroljon fel legalább három lehetséges okot, és mindegyikhez adjon ellenőrzési lépést.

23.1.11 Megoldások

1. A `ts-gw` MQTT-üzenetei csak azt bizonyítják, hogy a gateway publikált. Ha a backend nem iratkozott fel a telemetry topicra, vagy az idempotens ingest elutasítja az üzeneteket, az adatbázis üres maradhat. A T1 az adatbázisban a `node_id`-k számát ellenőrzi, így a teljes lánc működését igazolja.

2. A node-ok száma 3, az intervallum 10 s, így node-onként naponta $86400/10 = 8640$ üzenet keletkezik. Összesen $3 \cdot 8640 = 25\,920$ üzenet. A napi nyers adatmennyiség $25\,920 \cdot 180\,\text{B} \approx 4{,}67\,\text{MB}$.

3. A `compose.yaml`-be fel kell venni a `ts-node-n006` szolgáltatást a `ts-node-n003` mintájára, `SITE: gh1` és `NODE_ID: n006` értékkel; a `ts-gw` nem igényel node-onkénti konfigurációt, mert a `site` és `node_id` mezőt az UDP-üzenetből olvassa. A `devices` táblába is vegyük fel

`n006`-ot, hogy a hiányzó adat nézete ne adjon hamis riasztást. A T1 tesztben a küszöböt négyre kell emelni. A `n004` és a `n005` szándékosan soha nem jelentkező eszköz: a T5 riasztástesztjéhez kellenek, ezért nem használhatók fel a T1 bővítéséhez.

4. Lehetséges okok:

   - A `cmd` topicra a `ts-gw` nem iratkozott fel, ezért a parancs nem jut el a node-hoz. Ellenőrzés: `docker compose logs ts-gw`, keresse a feliratkozási hibát.
   - A node nem a megfelelő UDP porton figyel. Ellenőrzés: `docker compose exec ts-node-n001 printenv LISTEN_PORT` és `docker compose logs ts-node-n001`; a szimulátor a `LISTEN_PORT` (alapértelmezésben 9000) porton várja a `cmd` csomagot.
   - A parancs `expiry` mezője túl rövid, és a backend még a kézbesítés előtt lejártként kezeli. Ellenőrzés: a `device_command` tábla `expiry` és `state` mezője.
   - A `cmd/result` válasz nem megfelelő `id`-t használ, ezért a backend nem tudja korrelálni. Ellenőrzés: a gateway naplójában a publikált `cmd/result` üzenet `id` mezője.

23.1.12 Rövid összefoglaló

A záróprojektben a TerraSense rendszert egyetlen Compose-környezetben állítottuk össze. A `~/iot-labor/stack/compose.yaml` a rögzített verziómátrix szerinti képeket használja: `eclipse-mosquitto:2.0.18`, `timescale/timescaledb:2.17.2-pg16`, `grafana/grafana:11.2.0`; a `ts-backend` és a `ts-gw` a `~/iot-labor/i066/` végleges integrációs forrásaiból buildel. A stack tartalmazza a TLS-es Mosquittót, a TimescaleDB-t, a `ts-backend`-et, a `ts-gw`-t, három szimulált `ts-node`-ot, a Grafanát és a webhook-sinket. A `ts_node_sim.py` UDP-n küld telemetriát és parancsokat fogad; a `webhook_sink.py` a riasztások fogadását naplózza. Az `acceptance_test.sh` nyolc tesztblokkot futtat: T0 szolgáltatásállapot, T1 telemetria adatút, T2 parancs, nyugta és szelep-biztonsági zárás (T2b árnyék), T3 offline puffer és visszatöltés, T4 mTLS pozitív/negatív/ACL, T5 hiányzó adat riasztás és webhook, T6 spool-korlát, T7 késleltetés és eseményalapú kézbesítés. A `Követelmény-nyomonkövetési mátrix` a Must követelményeket igazolt, részben igazolt és nem igazolt állapotba sorolja; a hétnapos üzem, a 99,9%-os üzemi igazolás, a hardveres fail-safe, a node-X.509, az OTA rollback és a REST-hitelesítés továbbra is nyitott. **Továbbvitt eredmény.** A `compose.yaml`, a `ts_node_sim.py`, az `acceptance_test.sh` és az átvételi jegyzőkönyv eredményei az Összefoglalás: döntési térkép, ellenőrzőlista, továbbtanulás alfejezetben a döntési térkép és az ellenőrzőlista kiindulópontjai.

23.1.13 Források és ellenőrzési hiányok

- A Docker Compose specifikáció, a `depends_on`, a `condition` és a profilok pontos szintaxisa a Docker dokumentációjában ellenőrizendő.
- A `timescale/timescaledb:2.17.2-pg16`, az `eclipse-mosquitto:2.0.18` és a `grafana/grafana:11.2.0` konténerkép elérhetősége a Docker Hub-on ellenőrizendő; a verziók az `i000` verziómátrixához igazodnak.
- A `ts-backend` image buildje a `ts-backend` forráskód aktuális állapotától függ; a Spring Boot és a Paho Java kliens pontos konfigurációja a korábbi laborokban rögzített.
- A `ts-gw` Paho C++ kliens `PUBLISH`/`PUBACK` viselkedését a Paho dokumentációja és a MQTT QoS, munkamenetek, retain, Last Will és MQTT 5 alfejezet alapján használtuk; a pontos socket- és TLS-API a Paho verziójától függ.
- Az átvételi scriptet és a Compose-fájlt a lektoráló környezet statikusan ellenőrizte (`bash -n`, illetve YAML-értelmezés); a teljes stack futtatása és a tesztblokkok tényleges lefutása

ebben a környezetben nem történt meg. A fejezetben szereplő elvárt kimenetek ezért követelmények, nem mért eredmények.
- A mérési eredmények gépfüggők; a laborban közölt késleltetés- és veszteségértékek csak illusztratív becslések, nem ténylegesen mért adatok.

23.2 Összefoglalás: döntési térkép, ellenőrzőlista, továbbtanulás

23.2.1 Motiváció

A záróprojektben a TerraSense-rendszer már együtt fut: az eszköz, a gateway, a broker, a backend, a TimescaleDB, a Grafana és a labor-PKI mind a `~/iot-labor/stack/compose.yaml` környezetből reprodukálható. A mérnök számára a legnagyobb kockázat ekkor már nem egy-egy API vagy protokoll ismerete, hanem az, hogy a sok laborban meghozott döntésből melyik miért született, és egy új telephelyen, egy megváltozott követelmény vagy egy biztonsági észrevétel esetén hogyan kell újra dönteni. Az Záróprojekt: a teljes TerraSense rendszer összeállítása és átvétele alfejezet után ez az összefoglaló éppen azt a célt szolgálja, hogy a konkrét TerraSense-döntéseket újrahasznosítható tervezési tudássá alakítsa.

23.2.2 Fogalmak és működés

Definíció. A **döntési térkép** olyan dokumentált navigációs eszköz, amely egy IoT-rendszer fő architekturális kérdéseit döntési pontokra bontja, mindegyik ponthoz hozzárendeli az értékelhető opciókat, a döntést befolyásoló kritériumokat és az adott projektben választott megoldást. A döntési térkép nem egyszeri recept, hanem újrafelhasználható gondolkodási váz, amelyet egy új telephely, új szenzortípus vagy új üzemeltetési modell esetén újra kell járni.

Az alábbi folyamatábra a TerraSense fő döntési pontjait és a könyvben meghozott választásokat mutatja.



23.3. ábra: A TerraSense döntési térképe - A döntési pontok és a záróprojektben rögzített választások. A zárójelben jelölt képességek csak tervezettek vagy szimuláltak.

A lépések sorrendje nem merev, de kifejezi a legfontosabb összefüggéseket: a rádió erősen befolyásolja a használható protokollokat, a protokoll a kézbesítési garanciákat, a formátum a sémaverziózást, a biztonsági modell pedig az OTA és a flottakezelés kialakítását.

A döntési térkép fő sorait az alábbi táblázat foglalja össze.

Döntési pont	Fő opciók	TerraSense-választás	Döntő szempont
Rádió	BLE, LoRaWAN, Thread/IP, Wi-Fi, mobil	BLE; laborban UDP	hatótáv, energia, adatsebesség, gateway jelenléte

Döntési pont	Fő opciók	TerraSense-választás	Döntő szempont
Protokoll	MQTT, CoAP, LwM2M, Modbus/OPC UA	MQTT, QoS 1 (a broker 5-képes; a kliensek 3.1.1/5 szinten is csatlakoznak)	pub/sub broker, QoS, retained, Last Will, integráció
Formátum	JSON, CBOR, Protocol Buffers, SenML	JSON, sémaverzió	olvashatóság, tooling, eszközkorlátok
Edge/felhő munkamegosztás	eszköz, gateway, felhő	gateway szűrés, felhő tárolás	késleltetés, offline működés, sávszélesség
Tárolás	relációs, idősoros, stream	PostgreSQL + TimescaleDB	idősorlekérdezések, megőrzés, SQL
Biztonság	TLS/mTLS, DTLS, secure boot	mTLS MQTT, labor-PKI	eszközméret, PKI üzemeltetés, fenyegetésmodell
Frissítés	delta, A/B, fokozatos kiadás	A/B OTA rollback (terv; a laborban nem implementált)	megbízhatóság, flash méret, visszagördülés

A döntési térkép a választásokat mutatja, nem a megvalósítás állapotát. A záróprojekt végleges üzenetszerződése ezért külön táblázatban szerepel, mert a korábbi laborok részleges konvenciói (például a retained telemetria vagy a `node_id+seq` kulcs) itt nem érvényesek.

Topic	Irány	QoS	Retain	Tartalom
`terrasense/<site>/<node>/telemetry`	node → gw → broker → backend	1	false	`kind=telemetry` esemény, `measurements` objektummal
`terrasense/<site>/<node>/state`	node → gw → broker → backend	1	true	`kind=state`, `valve_open`
`terrasense/<site>/<node>/status`	node/gw → broker	1	true	jelenlét (`online`/`offline`); a Last Will retained üzenete
`terrasense/<site>/<node>/cmd`	backend → broker → gw → node	1	false	parancs (`valve.open`/`valve.close`, `duration_ms`)
`terrasense/<site>/<node>/cmd/result`	node → gw → broker → backend	1	false	nyugta (`ACCEPTED`/`SUCCESS`/`FAILED`, `id`)

A telemetria `retain=false` szabálya tudatos: a retained telemetria a brokerben maradó utolsó mintát jelentené, amelyet egy későn csatlakozó fogyasztó tévesen friss mérésként dolgozhatna fel. A retained állapot a `state` és a `status` topicra való, ahol az utolsó ismert állapot éppen a kívánt viselkedés. A labor UDP-alapú node-szimulátora nem állít be MQTT Last Willt; a `status`/Last Will sor a konvenció része, a hardveres node feladata, ezért az átvételi tesztek ezt nem igazolják.

Eseményazonosság. Egy telemetriaesemény azonosítója a `(site, node_id, boot_id, seq)` négyes; az adatbázisban ehhez a mérési típus és az eseményidő is hozzátartozik, ezért a végleges egyedi kulcs `(site, node_id, boot_id, seq, measurement_type, ts)`. A `site` a rövid node-nevek ütközését, a `boot_id` a `seq` újraindulás utáni újrakezdését kezeli. Egy esemény a hosszú táblában három mérési sort hoz létre, ezért az üzenetszámot `count(DISTINCT seq)` adja, nem a sorok száma.

Megvalósított, szimulált és tervezett képességek. A döntési térkép ne legyen félreérthető: a záróprojekt nem minden döntési pontot valósít meg teljes mélységében.

Képesség	Állapot a záróprojektben
BLE-rádió	szimulált: a laborban UDP helyettesíti
MQTT broker, QoS 1, retain, ACL	megvalósított (Mosquitto 2.0.18)
MQTT 5	a broker támogatja; a labor kliensei 3.1.1-en is csatlakoznak; a szerződés nem használ MQTT 5-specifikus funkciót
Telemetria, parancs, nyugta és szelepállapot	megvalósított
Idempotens ingest, store-and-forward	megvalósított
Eszközárnyék	megvalósított, de memóriabeli
Grafana riasztás és webhook	megvalósított
Node-X.509 és node-szintű mTLS	tervezett: a node UDP-szimulátor
A/B OTA és rollback	csak modell és terv (OTA-frissítés: A/B képek, rollback, delta, fokozatos kiadás); a záróprojekt stackben nincs OTA
Hardveres fail-safe	tervezett
REST-hitelesítés	nincs a laborban; tervezett
Hétnapos üzem, 99,9%-os rendelkezésre állás	nem igazolt

A táblázat állapotai összhangban vannak a záróprojekt követelmény-nyomonkövetési mátrixával; ahol a labor csak részbizonyítékot ad, ott a mátrix „részben igazolt” vagy „nem igazolt” állapotot használ.

23.2.3 Döntési szempontok

Az IoT-döntések ritkán hozhatók meg egymástól függetlenül. A rádió kijelöli a megengedhető energia- és adatkeretet, ezáltal szűkíti a protokollválasztékot; a protokoll meghatározza a lehetséges kézbesítési szemantikát; a formátum és a sémaverziózás a későbbi evolúció költségét; az edge/felhő munkamegosztás pedig a rendszer offline tűrőképességét. A biztonsági modell nem utólagos réteg, hanem a provisioning, a kommunikáció és az OTA közös alapja.

A TerraSense döntéseinek indoklása:

- **BLE + UDP szimuláció:** valós hardveren a BLE GATT a helyi, alacsony fogyasztású szenzorcsomópont felől a Linux-gatewayhez csatlakozik; laborban az UDP ugyanezt az adatutat csomagolja konténerek közé, mert a `native_sim` nem modellez valós rádiót.
- **MQTT, QoS 1:** a brokeroldali pub/sub jól illik a több előfizetőhöz (ingest, állapotfigyelő, riasztás), a QoS 1 legalább egyszeri kézbesítést ad. A Mosquitto 2.0.18 MQTT 5-képes, de a labor kliensei (a Paho C++ gateway, a Paho Python szimulátor és a Paho Java backend) 3.1.1 szinten is csatlakozhatnak; a végleges szerződés nem támaszkodik MQTT 5-specifikus funkcióra, ezért a viselkedés mindkét protokollszinten azonos. Az MQTT 5 reason code-jai és munkamenet-kezelése opcionális többlet, nem a működés feltétele.

- **JSON, sémaverzió:** a TerraSense kis üzeneteinél a JSON olvashatósága és a backend-tooling előnye felülmúlja a CBOR vagy Protobuf tömörségét; az `seq`/`ts`/`node_id` mezők és az explicit sémaverzió teszik evolválhatóvá a telemetriát.
- **Gateway előfeldolgozás, felhő tárolás:** a ts-gw szűri és előagregálja az adatokat, a hosszú távú idősorok és a dashboardok a felhőben maradnak, mert a hálózati kiesések miatt a gateway a pufferhatár, nem a végleges elemzési réteg.
- **TimescaleDB:** az idősoros adatok SQL-alapú szűrése, folyamatos aggregation és partíciókezelése egyszerűbbé teszi a telemetria-elemzést, mint egy általános relációs tábla vagy egy külön stream-adatbázis.

A döntési térkép használata során minden döntési pontnál fel kell tenni a következő kérdést: *mi változott a TerraSense-hez képest?* Ha például egy új telephelyen nincs helyi gateway, a BLE helyett LoRaWAN vagy mobil IP lehet szükséges; ha az energia évtizedes elemélettartamot követel, a protokoll- és formátumválasztás a CoAP/CBOR irányába tolódhat el. A térkép célja tehát nem az, hogy a TerraSense-választást erőltesse, hanem hogy a kompromisszumokat láthatóvá tegye.

23.2.4 Tervezési ellenőrzőlista a továbbvitt eredmények alapján

A döntési térkép a *mit válasszunk*, az ellenőrzőlista a *teljes-e a megvalósítás* kérdésére válaszol. A lista a könyv korábbi „Továbbvitt eredmény” jelölésű döntéseire épül, ezért a záróprojekt átvétele előtt minden sorhoz bizonyítékot kell rendelni.

Azonosítás és idő. Minden telemetriaüzenet tartalmazza a `kind`, `schema_version`, `site`, `node_id`, `boot_id`, `seq` és `ts` mezőt; a `ts` UTC, epoch ms, a `boot_id` a bootpillanatot azonosítja. A topic a `terrasense/<site>/<node>/telemetry` konvenciót követi, a `seq` pedig a `(site, node_id, boot_id)` hármon belül monoton. Ellenőrzés: a Adatformátumok: JSON, CBOR, Protocol Buffers, SenML és az Idő az IoT-ban: időbélyeg, időszinkron, sorrend alfejezetek szerint.

Protokollgaranciák. A nem veszíthető adatok MQTT QoS 1 szinten közlekednek. A végleges konvenció: `telemetry` QoS 1, retain=false; `state` QoS 1, retain=true; `status` QoS 1, a Last Will üzenet retained; `cmd` és `cmd/result` QoS 1, retain=false. A labor brokerje perzisztencia nélkül fut, ezért a munkamenet-állapot és a retained üzenetek a broker újraindítását nem élik túl; ezt a node állapotüzenete és a gateway store-and-forward puffere kompenzálja. Ellenőrzés: MQTT QoS, munkamenetek, retain, Last Will és MQTT 5.

Formátum és séma. Az üzenet explicit sémaverziót hordoz; a fogyasztó a régebbi sémaüzeneteket is értelmezi, vagy a verzió alapján elutasítja. Ellenőrzés: Sémák és verziózás: kompatibilis változtatások.

Idempotens ingest. A backend ugyanazt az eseményt at-least-once kézbesítés mellett sem tárolja kétszer; a deduplikáció a `(site, node_id, boot_id, seq, measurement_type, ts)` kulcson alapul. A `site` és a `boot_id` azért kell, mert a rövid node-nevek telephelyek között ütközhetnek, és a `seq` újraindulás után nulláról indul. Ellenőrzés: Ingest: MQTT-ből a backendbe, idempotensen.

Store-and-forward. A ts-gw offline időszakban puffereli a telemetriát, majd visszatölt; a puffer felső korlátja és a korlát túllépésének kezelése dokumentált. Ellenőrzés: Store-and-forward: offline működés és visszatöltés.

Biztonság. Nincs nyílt, hitelesítés nélküli brokerport; a kapcsolat mTLS-t használ; az eszköz rendelkezik egyedi identitással; nincs gyártói alapértelmezett jelszó. A laborban a gateway

és a backend mTLS-e igazolt, a node UDP-szimulátor azonban nem rendelkezik saját X.509-tanúsítvánnyal, ezért a node-szintű identitás a hardveres fázis követelménye. Ellenőrzés: Eszközidentitás: X.509-tanúsítványok, PKI és kulcstárolás és Biztonságos kapcsolat: TLS 1.3, mTLS és DTLS az MQTT/CoAP alatt.

Frissítés. A tervezett firmware A/B vagy azzal egyenértékű frissítést és rollbacket támogat; a kép aláírt; a fokozatos kiadás lehetősége adott. A záróprojekt stack nem tartalmaz OTA-komponenst, ezért ez a sor a laborban nem igazolható, csak az OTA-frissítés: A/B képek, rollback, delta, fokozatos kiadás modellje és a követelmény rögzített.

Megfigyelhetőség. A naplók UTC-ben készülnek; a metrikák elérik az üzemeltetőt; az eszközök állapota a dashboardon látható. A Last Will üzenetek megjelenítése a hardveres node feladata, mert a labor UDP-szimulátora nem állít be MQTT Last Willt. Ellenőrzés: Megfigyelhetőség végponttól végpontig: metrikák, naplók, nyomkövetés.

Reprodukálhatóság. A teljes TerraSense a `~/iot-labor/stack/compose.yaml` és a záróprojekt laborfájljaiból újraépíthető; az átvételi tesztek futtathatók, és nem szorulnak manuális varázsváltozókra. Ellenőrzés: Záróprojekt: a teljes TerraSense rendszer összeállítása és átvétele.

A lista nem a laborok könyvelése, hanem kockázatcsökkentő eszköz: ha egy sorra nincs bizonyíték, a hiányzó komponenst még a záróprojekt lezárása előtt vissza kell vezetni a megfelelő alfejezetbe.

23.2.5 Ellenőrzés és hibakeresés

Az ellenőrzőlista használata akkor hatékony, ha minden sorhoz konkrét futási bizonyítékot rendelünk: az azonosítás és idő sor a `telemetry` táblából kinyert JSON-üzenet, a protokollgaranciák a Mosquitto logja és a T4 mTLS-teszt, az idempotencia a backend-teszt adatbázisállapota, a frissítés sora pedig az OTA-frissítés: A/B képek, rollback, delta, fokozatos kiadás modelljére hivatkozik, mert a záróprojekt stack nem tartalmaz OTA-t. Ha az átvételi tesztek egy részlete „zöld”, az még nem jelenti azt, hogy a lista teljes: a záróprojektben gyakran a pozitív főút van tesztelve, a hiba- és offline utak nincsenek.

Tipikus ellenőrzési tünetek:

Tünet	Valószínű ok	Javítás
A Grafana-diagram hőmérsékletgörbéje kettőződik	QoS 1 újbóli kézbesítés, nincs deduplikáció	idempotens ingest a `(site, node_id, boot_id, seq, measurement_type, ts)` kulcson; a hosszú táblában eseményenként három sor várható
Az eszköz telemetriája a brokeren megjelenik, de a backend nem látja	topic-névtérés, rossz site/node tag	topic-konvenció ellenőrzése
Az offline gateway visszatöltése eltolt időbélyegeket ír	az eszköz a helyi időt szövegként konvertálja, az óra nincs szinkronizálva, vagy a gateway a feldolgozás idejét írja a `ts` helyére	`ts` UTC epoch ms a szinkronizált falórából; a feldolgozási idő külön `server_ts` mezőbe kerüljön
A MOSQUITTO külső hálózatról elérhető, nincs TLS	alapértelmezett konfiguráció maradt nyitva	csak mTLS-listener, labor-CA

A hibakeresés során mindig az adatút mentén haladjunk: eszköz → gateway → broker → ingest → tároló → dashboard. Ezzel a sorrenddel a hiba gyorsan beszűkíthető egy rétegre.

23.2.6 Tipikus hibák és félreértések

Falióra, monoton óra és időzóna összekeverése. A `ts` mezőhöz a falióra (wall clock) UTC epoch ms értéke kell; a fejlesztői laptop vagy a node rendszerórája önmagában nem hiba, ha megfelelően szinkronizált (például NTP-vel) és UTC-ben adja az epochértéket. A hiba a hibás konverzió, az elállított vagy nem szinkronizált óra, illetve a helyi idő szövegként való beírása. A monoton óra (például `CLOCK_MONOTONIC`) az intervallummméréshez és az időkorlátokhoz való, nem eseményidőbélyeghez, mert nincs rögzített epochja. A helyi időzóna csak megjelenítési kérdés: ugyanaz az epochérték Budapesten és UTC-ben ugyanazt az eseményidőt jelenti. A helyes szabály: eseményidő = UTC epoch ms a szinkronizált falórából; megjelenítés = zónázott konverzió; a feldolgozási idő külön mező (`server_ts`). A Idő az IoT-ban: időbélyeg, időszinkron, sorrend alfejezetben tárgyalt szabályok itt is kötelezőek.

QoS 1 idempotencia nélkül. Az MQTT QoS 1 garantálja a legalább egyszeri kézbesítést, de nem garantálja a pontosan egyszeri tárolást. Ha a backend minden PUBLISH-t feltétel nélkül beszúr, a hálózati nyugtázás elvesztéséből vagy a gateway store-and-forward visszatöltéséből duplikátumok keletkeznek. Az idempotens ingest és a `seq` alapú deduplikáció nem kényelmi funkció, hanem az elemzés helyességének feltétele.

A frissíthetőség hiánya. Egy IoT-rendszer akkor is sebezhető marad, ha ma minden teszt zöld, de a firmware vagy a gateway-szoftver nem frissíthető hibajavításra egy későbbi CVE után. Az A/B OTA, a rollback és a fokozatos kiadás az OTA-frissítés alfejezetben bemutatott tervezési modell része; a záróprojekt laborban ezt nem implementáltuk, ezért a frissíthetőség a hardveres és produkciós fázis követelménye marad, nem a labor igazolt képessége.

Alapértelmezett jelszavak. A fejlesztői kényelemből nyitva hagyott névtelen MQTT-hozzáférés vagy gyári jelszó a legkönnyebb támadási felület. A TerraSense laborban ezért a belső broker is csak mTLS-listenerrel fut, és a labor-CA egyértelműen laborcélú. A Tipikus IoT-sebezhetőségek és a szabályozás alfejezetben bemutatott esetek jól mutatják, hogy a tömeges fertőzések ritkán igényelnek kifinomult kriptográfiai támadást.

23.2.7 Feladatok

Az alábbi feladatok önellenőrző kérdéssorként is szolgálnak.

1. **Fogalmi kérdés.** Mi a különbség a döntési térkép és a tervezési ellenőrzőlista között? Adj példát arra, amikor a döntési térkép változik, de az ellenőrzőlista nem.
2. **Tervezési feladat.** Tételezd fel, hogy egy új `gh2` telephelyen nincs BLE-gateway, csak egy LoRaWAN-átjáró érhető el. Melyik TerraSense-döntési pontok változnak, és mely ellenőrzőlista-sorok maradnak érvényben?
3. **Implementációs feladat.** A záróprojekt `~/iot-labor/i066/` átvételi tesztjeihez adj hozzá egy új esetet, amely ugyanazt az MQTT-telemetriaüzenetet kétszer küldi be. Milyen adatbázis-kulcsot és milyen asszertet kell használni az idempotens ingest igazolásához?
4. **Hibakeresési feladat.** Egy ügyfél azt jelzi, hogy a Grafana-panel esténként egyórás lyukakat mutat, de a gateway naplói szerint az eszköz egész éjjel mért. Hol kezdenéd a hibakeresést, és melyik ellenőrzőlista-sor valószínűleg sérül?

23.2.8 Megoldások

1. A döntési térkép a *választási lehetőségeket és azok kompromisszumait* rendszerezi; az ellenőrzőlista a *megvalósítás teljességi kritériumait*. A döntési térkép változik, ha egy új rádió vagy protokoll lép be; az ellenőrzőlista viszont attól még érvényes marad, hogy az adott protokollhoz is kötelező idempotens ingest, UTC-idő, sémaverzió és frissíthetőség tartozzon.
2. Változik a rádió döntési pont: BLE helyett LoRaWAN. Ebből következően a protokoll döntés is újraértékelendő: az MQTT helyett CoAP vagy LwM2M lehet indokolt, és a formátum is tömörebb kódolásba fordulhat. Az ellenőrzőlista azonosítás/idő, sémaverzió, idempotens ingest, store-and-forward, biztonság, frissítés és megfigyelhetőség sora anyagilag érvényben marad, csak az egyes bizonyítékok protokollja változik.
3. A teszt ugyanazt az eseményt `site`, `node_id`, `boot_id` és `seq` megegyező értékkel kétszer publikálja. Az asszert azt ellenőrzi, hogy a `telemetry` táblában az eseményhez pontosan három mérési sor tartozik (nem hat), és az ingest naplója a második esetben deduplikációt jelez. A mögöttes egyedi kulcs `(site, node_id, boot_id, seq, measurement_type, ts)`; a `node_id` és `seq` pár nem elég, mert a `seq` újraindulás után ismétlődhet, és a rövid node-nevek telephelyek között ütközhetnek.
4. A hibakeresést a gateway store-and-forward puffer és a broker kapcsolatánál érdemes kezdeni, mert az esti óra lyuk azt jelzi, hogy az eszköz mér, de az adat nem jut el a backendig. Leggyakrabban a gateway offline puffere töltődik, majd a visszatöltés késleltetve érkezik; a store-and-forward sor sérül. Ha a `ts` mező nem az eseményidőt, hanem a feldolgozási időt hordozta, az idősor időeltolódást is mutathat.

23.2.9 Rövid összefoglaló

A döntési térkép hét fő pont köré rendezi a TerraSense-t: rádió (BLE, laborban UDP), protokoll (MQTT, QoS 1; a broker 5-képes, a kliensek 3.1.1/5 szinten is csatlakoznak), formátum (JSON, sémaverzió), edge/felhő munkamegosztás (ts-gw előfeldolgozás, felhő tárolás), tárolás (TimescaleDB), biztonság (mTLS, labor-PKI) és frissítés (A/B OTA rollback, csak tervezett). A tervezési ellenőrzőlista a korábbi „Továbbvitt eredményekre” épül: azonosítás és UTC-idő, protokollgaranciák, sémaverzió, idempotens ingest, store-and-forward, mTLS, A/B rollback, megfigyelhetőség és reprodukálhatóság. A végleges üzenetszerződés szerint a telemetria QoS 1, retain=false; a `state` és a `status` retained; az esemény azonossága `(site, node_id, boot_id, seq)`, a tárolt egyedi kulcs `(site, node_id, boot_id, seq, measurement_type, ts)`. A megvalósított, szimulált és csak tervezett képességek külön táblázata megmutatja, hogy a node-X.509, az OTA, a hardveres fail-safe, a REST-hitelesítés és a hétnapos üzemi igazolás nem része a labor bizonyított eredményeinek. A leggyakoribb hibák: a falióra, a monoton óra és az időzóna összekeverése, QoS 1 idempotencia nélkül, frissíthetőség hiánya és alapértelmezett jelszavak. Az önellenőrző kérdéssor azt igazolja, hogy az olvasó nemcsak a TerraSense-laborokat tudja lefuttatni, hanem a döntéseket új követelményekhez is hozzá tudja igazítani.

23.2.10 Források és ellenőrzési hiányok

Ez az alfejezet nem vezet be új külső forrást vagy API-t. A döntési térkép és az ellenőrzőlista a könyv korábbi alfejezeteinek laboreredményeit és döntéseit összegzi; a szoftververziók a `~/iot-labor/stack/compose.yaml` aktuális állapotából olvashatók ki. Az iparági statisztikákat és szabályozási határidőket itt nem ismételjük meg; azok a Tipikus IoT-sebezhetőségek és a szabályozás, a Biztonságos kapcsolat: TLS 1.3, mTLS és DTLS az MQTT/CoAP alatt és az

OTA-frissítés: A/B képek, rollback, delta, fokozatos kiadás alfejezetekben jelölt ellenőrzési igények szerint kezelendők.